

Contents

第二十四册简介：大模型推理框架与 Serving Engine 实战	34
本书定位	34
为什么需要这本书	34
适合读者	34
学完后应具备的能力	34
推荐参考项目	34
内容结构	35
与其他书的关系	35
第二十四册：大模型推理框架与 Serving Engine 实战	35
本书定位	35
阅读入口	35
推荐参考项目	35
章节目录	35
第一部分：推理框架总览	35
第二部分：从 0 实现最小推理框架	36
第三部分：vLLM 架构详解	36
第四部分：SGLang 架构详解	36
第五部分：PD 分离与高级 Serving 架构	36
第六部分：教学项目源码路线与升级实战	36
第七部分：系统设计、Debug 与面试	37
与其他书的关系	37
第一批优先扩写章节	37
第 1 章 LLM Serving Engine 到底是什么	37
1.0 本讲资料边界与第二轮精修口径	38
1.1 从 model.generate 说起	38
1.2 LLM Serving Engine 的定义	38
1.3 Serving Engine 和推理平台的区别	39
1.4 一次请求在 Engine 内部的生命周期	39
1.5 Prefill 和 Decode 是核心阶段	40
1.6 KV Cache 为什么是核心	40
1.7 为什么需要 Scheduler	40
1.8 为什么需要 Continuous Batching	41
1.9 Streaming 为什么重要	41
1.10 Engine 需要暴露哪些指标	41
1.11 vLLM、SGLang、TGI、Triton 的位置	42
1.12 从 0 实现推理框架要实现什么	42
1.13 推理框架核心指标和最小 demo	42
1.14 常见误区	50
1.15 面试常见追问	50
1.16 小练习	51
1.17 本章小结	51
第 2 章 从 naive generate 到推理框架	51
2.0 本讲资料边界与第二轮精修口径	51
2.1 最简单的 generate	52
2.2 generate 背后的真实循环	52
2.3 Prefill 和 Decode 的分离	53
2.4 单请求到多请求	53
2.5 为什么不能只做静态 batching	54
2.6 KV Cache Manager 为什么必要	54
2.7 Streaming 不只是边打印	55
2.8 从 generate 到 engine 的模块拆分	55

2.9 一个最小 engine 循环	55
2.10 请求状态机	56
2.11 最小框架的演进路线	56
2.12 最小推理框架的公式和可运行 demo	57
2.13 常见误区	61
2.14 面试追问	62
2.15 小练习	62
2.16 本章小结	62
第 3 章 推理请求的生命周期	62
3.0 本讲资料边界与第二轮精修口径	63
3.1 为什么要讲生命周期	63
3.2 入口：客户端请求	63
3.3 参数规范化	64
3.4 Prompt 构造和 tokenization	64
3.5 创建 Request 对象	64
3.6 进入 waiting queue	65
3.7 Scheduler 决定谁上 GPU	65
3.8 Prefill 阶段	66
3.9 Decode 阶段	66
3.10 Streaming 输出	67
3.11 Finish Reason	67
3.12 资源释放	67
3.13 Metrics 和 Trace	68
3.14 Abort 和 Timeout	68
3.15 生命周期状态机	68
3.16 请求生命周期指标和可运行 demo	69
3.17 面试追问	73
3.18 小练习	73
3.19 本章小结	73
第 4 章 Prefill、Decode、KV Cache 和 Token Streaming	74
4.0 本讲资料边界与第二轮精修口径	74
4.1 为什么这四个概念最重要	74
4.2 Prefill 是什么	75
4.3 Prefill 的计算特点	75
4.4 Decode 是什么	75
4.5 Decode 的资源特点	76
4.6 Prefill 和 Decode 的差异	76
4.7 KV Cache 是什么	76
4.8 KV Cache 为什么占显存	77
4.9 KV Cache 管理的难点	77
4.10 Token Streaming 是什么	77
4.11 Streaming 的工程细节	78
4.12 四者如何串起来	78
4.13 对 Scheduler 的影响	78
4.14 常见性能现象解释	79
4.15 常见误区	79
4.16 阶段指标、KV 公式和可运行 demo	79
4.17 面试追问	82
4.18 小练习	83
4.19 本章小结	83
第 5 章 TTFT、TPOT、吞吐、并发、显存和成本	83
5.0 本讲资料边界与第二轮精修口径	83
5.1 为什么传统 QPS 不够用	84

5.2 TTFT: Time To First Token	84
5.3 如何优化 TTFT	84
5.4 TPOT: Time Per Output Token	85
5.5 如何优化 TPOT	85
5.6 吞吐: tokens/s 比 QPS 更重要	85
5.7 并发: 不是连接数那么简单	86
5.8 显存: 权重、KV Cache 和临时 buffer	86
5.9 成本: 每 token 成本才是核心	87
5.10 指标之间的冲突	87
5.11 p50、p95、p99 为什么重要	87
5.12 如何做压测	88
5.13 面试中如何讲性能指标	88
5.14 常见误区	89
5.15 指标公式、容量估算和可运行 demo	89
5.16 面试追问	92
5.17 小练习	92
5.18 本章小结	92
第 6 章 推理框架、推理平台和 AI Infra 的边界	92
6.0 本讲资料边界与第二轮精修口径	92
6.1 为什么边界重要	93
6.2 三层结构总览	93
6.3 推理框架负责什么	94
6.4 推理框架不应该负责什么	94
6.5 推理平台负责什么	94
6.6 推理平台和 engine 的接口	95
6.7 AI Infra 负责什么	95
6.8 三层各自的指标	95
6.9 一个例子: TTFT 变差怎么分层排查	96
6.10 一个例子: 成本过高怎么分层优化	97
6.11 常见架构反模式	97
6.12 面试中如何讲边界	97
6.13 本书后续章节如何对应三层	98
6.14 常见误区	98
6.15 边界职责、接口契约和可运行 demo	98
6.16 面试追问	103
6.17 小练习	103
6.18 本章小结	103
第 7 章 实现最小 tokenizer、model wrapper 和 generate loop	104
7.0 本讲资料边界与第二轮精修口径	104
7.1 本章目标	104
7.2 最小组件拆分	104
7.3 TokenizerWrapper	105
7.4 ModelWrapper	105
7.5 Logits 是什么	106
7.6 最简单的 greedy 选择	106
7.7 最小 generate loop	106
7.8 更合理的设备处理	107
7.9 停止条件	107
7.10 为什么这个版本很慢	108
7.11 和 transformers.generate 的关系	108
7.12 最小代码骨架	108
7.13 这个骨架如何升级	109
7.14 生成循环公式、契约和可运行 demo	110
7.15 常见误区	112

7.16 面试追问	113
7.17 小练习	113
7.18 本章小结	113
第 8 章 实现 greedy、top-k、top-p 和 temperature sampling	113
8.0 本讲资料边界与第二轮精修口径	114
8.1 为什么要单独实现 sampling	114
8.2 logits、概率和采样	114
8.3 Greedy Decoding	114
8.4 Temperature	115
8.5 Top-k Sampling	115
8.6 Top-p Sampling	116
8.7 组合顺序	116
8.8 Sampler 类	117
8.9 随机种子和可复现性	117
8.10 参数如何影响输出	118
8.11 Sampling 和 Serving Engine	118
8.12 采样公式、数值稳定和可运行 demo	118
8.13 常见误区	121
8.14 面试追问	121
8.15 小练习	121
8.16 本章小结	122
第 9 章 实现 KV Cache 的最小版本	122
9.0 本讲资料边界与第二轮精修口径	122
9.1 naive loop 的重复计算	122
9.2 KV Cache 保存了什么	123
9.3 Prefill 和 Decode 中的 cache	123
9.4 Hugging Face 中的 past_key_values	123
9.5 修改 ModelWrapper	124
9.6 带 KV Cache 的 generate loop	124
9.7 更清晰的 prefill/decode 写法	125
9.8 cache 的显存公式	126
9.9 为什么 cache 会影响并发	126
9.10 attention_mask 和 position_ids	126
9.11 cache 和输出一致性	127
9.12 KV Cache 公式、机制和可运行 demo	127
9.13 这个版本还缺什么	130
9.14 常见误区	130
9.15 面试追问	130
9.16 小练习	131
9.17 本章小结	131
第 10 章 实现 batched prefill	131
10.0 本讲资料边界与第二轮精修口径	131
10.1 为什么需要 batched prefill	132
10.2 最小输入示例	132
10.3 padding_side: 左 padding 还是右 padding	132
10.4 tokenizer 批量编码	133
10.5 Batched TokenizerWrapper	133
10.6 修改 ModelWrapper 支持 attention_mask	133
10.7 实现 batched_prefill	134
10.8 右 padding 下如何取最后真实位置	134
10.9 Sampler 兼容 batch	135
10.10 生成每个请求的首 token	135
10.11 Request 对象如何保存 prefill 结果	135

10.12	past_key_values 的 batch 维度	136
10.13	padding 带来的浪费	136
10.14	长 prompt、token budget 和 chunked prefill	137
10.15	batched prefill 公式、padding 风险和可运行 demo	137
10.16	最小完整代码骨架	140
10.17	和生产 engine 的差距	140
10.18	常见误区	141
10.19	面试追问	141
10.20	小练习	141
10.21	本章小结	141
第 11 章	实现 batched decode	142
11.0	本讲资料边界与第二轮精修口径	142
11.1	从单请求 decode 到 batched decode	142
11.2	固定 batch 的前提	142
11.3	RequestState 需要哪些字段	143
11.4	batched prefill 后的状态	143
11.5	最小 batched_decode_step	144
11.6	attention_mask 和 position_ids 如何增长	144
11.7	固定 batch decode loop	144
11.8	finished mask	145
11.9	compact batch	146
11.10	batch 顺序为什么重要	146
11.11	每个请求不同 max_new_tokens	146
11.12	不同 sampling params	146
11.13	batched decode 公式、状态和可运行 demo	147
11.14	固定 batch decode 的完整骨架	149
11.15	更接近 engine 的循环	150
11.16	batched decode 和 continuous batching 的区别	150
11.17	常见 bug	150
11.18	常见误区	151
11.19	面试追问	151
11.20	小练习	151
11.21	本章小结	151
第 12 章	实现请求队列和简单 scheduler	152
12.0	本讲资料边界与第二轮精修口径	152
12.1	为什么需要 scheduler	152
12.2	最小状态集合	153
12.3	调度指标公式	153
12.4	RequestState 扩展	154
12.5	RequestQueue	154
12.6	Scheduler 的输入和输出	155
12.7	为什么要同时限制 max running 和 token budget	155
12.8	Decode 优先和 Prefill 优先	156
12.9	Engine 主循环	156
12.10	run_prefill	157
12.11	decode group 的概念	157
12.12	run_prefill with decode group	158
12.13	run_decode 和 cache 对齐	158
12.14	固定 group 的 decode	159
12.15	cleanup_finished	159
12.16	Scheduler 公式、队列指标和可运行 demo	159
12.17	最小调度循环示例	164
12.18	简单 scheduler 的局限	164
12.19	从简单 scheduler 到 continuous batching	165

12.20 调度策略的 trade-off	165
12.21 常见误区	165
12.22 面试追问	166
12.23 小练习	166
12.24 本章小结	166
第 13 章 实现 token streaming	166
13.0 本讲资料边界与第二轮精修口径	167
13.1 为什么需要 streaming	167
13.2 Streaming 指标公式	167
13.3 streaming 在 engine 中的位置	168
13.4 最小 Streamer 接口	168
13.5 为什么不能简单 decode 单个 token	169
13.6 基于完整输出的增量差分	169
13.7 StreamingEvent	169
13.8 每个请求一个输出队列	170
13.9 在 prefill 中推送首 token	170
13.10 在 decode 中推送增量	171
13.11 finish event	171
13.12 stop sequence 和 streaming	172
13.13 客户端取消	172
13.14 backpressure	172
13.15 SSE 的直觉	173
13.16 Streaming 和指标	173
13.17 Token streaming 公式、事件和可运行 demo	173
13.18 最小代码骨架	177
13.19 常见误区	178
13.20 面试追问	178
13.21 小练习	179
13.22 本章小结	179
第 14 章 实现最小 HTTP API 服务	179
14.0 本讲资料边界与第二轮精修口径	179
14.1 HTTP API 在架构中的位置	180
14.2 最小接口设计与 API 门禁公式	180
14.3 API 层应该校验什么	181
14.4 FastAPI 最小骨架	181
14.5 engine 后台循环	182
14.6 等待非流式结果	183
14.7 SSE 流式响应	183
14.8 请求取消如何传回 engine	184
14.9 API 层和 sampling params	185
14.10 API 层的并发风险	185
14.11 队列满了怎么办	185
14.12 OpenAI-compatible API 的差异	186
14.13 最小服务文件结构	186
14.14 HTTP API 公式、接口和可运行 demo	186
14.15 常见误区	192
14.16 面试追问	192
14.17 小练习	192
14.18 本章小结	192
第 15 章 压测 TTFT、TPOT、吞吐和显存	193
15.0 本讲资料边界与第二轮精修口径	193
15.1 压测要回答什么问题	193
15.2 指标定义回顾	194

15.3 在 RequestState 中记录时间	194
15.4 在 engine 中埋点	195
15.5 统计分位数	195
15.6 统计 tokens/s	196
15.7 统计 queue 和 active requests	196
15.8 显存观测	197
15.9 KV Cache 显存的直觉压测	197
15.10 Workload 设计	197
15.11 到达模式	198
15.12 一个最小压测脚本思路	198
15.13 如何分析压测结果	198
15.14 最小框架的预期瓶颈	199
15.15 报告模板	199
15.16 压测公式、指标表和可运行 demo	200
15.17 常见误区	202
15.18 面试追问	203
15.19 小练习	203
15.20 本章小结	203
第 16 章 vLLM 解决什么问题	203
16.0 本讲资料边界与第二轮精修口径	204
16.1 为什么要从 MiniEngine 过渡到 vLLM	204
16.2 vLLM 的核心定位	205
16.3 vLLM 之前的痛点	205
16.4 PagedAttention 解决什么	206
16.5 Continuous Batching 解决什么	206
16.6 vLLM 和 MiniEngine 的对比	207
16.7 vLLM 不只是 PagedAttention	207
16.8 vLLM 的请求视角	207
16.9 为什么 KV Cache 是 vLLM 的核心	208
16.10 为什么调度是 vLLM 的核心	208
16.11 vLLM 适合什么场景	208
16.12 vLLM 与其他 runtime 的关系	209
16.13 如何阅读 vLLM 源码	209
16.14 vLLM 动机公式、内存浪费和可运行 demo	209
16.15 常见误区	211
16.16 面试追问	212
16.17 小练习	212
16.18 本章小结	212
第 17 章 PagedAttention 的核心思想	212
17.0 本讲资料边界与第二轮精修口径	213
17.1 问题从 KV Cache 开始	213
17.2 连续 KV Cache 的浪费	213
17.3 操作系统分页的直觉	214
17.4 逻辑块和物理块	214
17.5 Block Table	214
17.6 一个具体例子	215
17.7 PagedAttention 如何减少浪费	215
17.8 Block Size 的 trade-off	216
17.9 Attention 如何读取分页 KV	216
17.10 与 past_key_values 的区别	216
17.11 与 continuous batching 的关系	217
17.12 Prefix Sharing 的基础	217
17.13 PagedAttention 不是免费午餐	217
17.14 一个面试级类比	217

17.15 PagedAttention 地址翻译和可运行 demo	218
17.16 常见误区	221
17.17 面试追问	221
17.18 小练习	221
17.19 本章小结	222
第 18 章 KV Cache Block Manager	222
18.0 本讲资料边界与第二轮精修口径	222
18.1 为什么需要 Block Manager	222
18.2 最小 Block Manager 的数据结构	223
18.3 Sequence 的 block table	223
18.4 分配 blocks	223
18.5 Prefill 时如何分配	224
18.6 Decode 时如何扩展	225
18.7 释放 blocks	225
18.8 block table 和 scheduler 的关系	225
18.9 block 资源视角下的请求状态	226
18.10 block 内部偏移	226
18.11 Copy-on-write 和 prefix sharing	226
18.12 Eviction 和 swapping 的直觉	226
18.13 常见 block manager 指标	227
18.14 和 MiniEngine 的差距	227
18.15 Block Manager 分配、释放和可运行 demo	227
18.16 常见误区	230
18.17 面试追问	231
18.18 小练习	231
18.19 本章小结	231
第 19 章 Continuous Batching 与 iteration-level scheduling	231
19.0 本讲资料边界与第二轮精修口径	232
19.1 本章目标	232
19.2 先看传统 batch 的问题	232
19.3 Orca 提出的 iteration-level scheduling	233
19.4 Continuous Batching 的核心直觉	233
19.5 prefill 和 decode 为什么要一起调度	234
19.6 token budget: 调度不是只数请求数	234
19.7 KV block budget: 能算不代表能放下	235
19.8 一轮 continuous batching 的完整流程	236
19.9 为什么 continuous batching 能提高吞吐	237
19.10 它的代价和副作用	238
19.11 常见调度策略	238
19.12 TTFT、TPOT 和吞吐如何被影响	239
19.13 和 dynamic batching 的区别	240
19.14 面向专家: iteration-level scheduling 的本质	240
19.15 最小实现时容易踩的坑	241
19.16 面试官会怎么问	241
19.17 标准回答模板	241
19.18 Continuous Batching 公式、调度 trace 和可运行 demo	242
19.19 小练习	247
19.20 本章总结	247
第 20 章 vLLM 请求调度流程	247
20.0 本讲资料边界与第二轮精修口径	247
20.1 本章目标	248
20.2 先建立模块图	248
20.3 请求进入系统: 从原始输入到 Engine Request	249

20.4 请求进入 waiting queue	250
20.5 Engine Core 的主循环	251
20.6 Scheduler 的输入	251
20.7 Scheduler 的输出	252
20.8 请求状态机	252
20.9 从 WAITING 到 PREFILL	253
20.10 从 PREFILL 到 RUNNING	253
20.11 RUNNING 阶段: 每轮 decode	254
20.12 Model Runner 看到的不是请求, 而是执行 batch	254
20.13 Output Processor 为什么也影响调度	255
20.14 Streaming 输出路径	255
20.15 Abort 和 cleanup 流程	256
20.16 一个完整请求生命周期例子	257
20.17 和第 19 章的关系	257
20.18 常见工程坑	257
20.19 面试官会怎么问	258
20.20 标准回答模板	258
20.21 请求调度流程公式、状态 trace 和可运行 demo	258
20.22 小练习	265
20.23 本章总结	265
第 21 章 vLLM memory management	266
21.0 本讲资料边界与第二轮精修口径	266
21.1 本章目标	266
21.2 LLM serving 的显存都花在哪里	266
21.3 KV Cache 为什么是核心难点	267
21.4 从连续 cache 到 block pool	268
21.5 Block pool 的基本数据结构	268
21.6 Page size 和 block size	269
21.7 Prefix cache 如何嵌入 memory management	269
21.8 Free queue 和 LRU eviction	270
21.9 Ref count 和共享 blocks	271
21.10 Preemption 和 recompute	271
21.11 Swap、offload 和多级 KV Cache	272
21.12 Chunked prefill 和 memory budget	273
21.13 参数如何影响 memory management	273
21.14 Hybrid KV Cache Manager: 为什么混合模型更复杂	274
21.15 Memory metrics 应该看什么	274
21.16 常见 OOM 和性能问题	275
21.17 面试官会怎么问	275
21.18 标准回答模板	276
21.19 Memory management 公式、prefix cache 和可运行 demo	276
21.20 小练习	280
21.21 本章总结	280
第 22 章 vLLM worker、executor 和 engine 架构	281
22.0 本讲资料边界与第二轮精修口径	281
22.1 本章目标	281
22.2 先看整体进程图	282
22.3 各模块职责一句话版	282
22.4 为什么要拆成这些层	283
22.5 Engine Core: 调度中枢	284
22.6 Executor: 执行派发层	284
22.7 Worker: 一张 GPU 的控制者	285
22.8 Model Runner: worker 内部的执行大脑	286
22.9 Model Runner 为什么复杂	286

22.10 从 Scheduler Output 到 Worker Input	287
22.11 多 GPU 下的执行方式	287
22.12 Engine Core 与 Worker 的通信内容	288
22.13 为什么 API Server 不直接控制 GPU	288
22.14 CPU 资源为什么也重要	289
22.15 CUDA context、初始化和 warmup	289
22.16 故障路径: worker 挂了怎么办	290
22.17 和前几章的关系	290
22.18 常见工程坑	290
22.19 面试官会怎么问	291
22.20 标准回答模板	291
22.21 Worker / Executor 架构公式、派发 trace 和可运行 demo	292
22.22 小练习	296
22.23 本章总结	296
第 23 章 Prefix Caching 与 Prompt Cache	296
23.0 本讲资料边界与第二轮精修口径	297
23.1 本章目标	297
23.2 先区分几类缓存	297
23.3 Prompt Cache 是什么	298
23.4 Prefix Caching 优化什么指标	298
23.5 为什么不能只按字符串缓存	299
23.6 vLLM 的核心做法: 缓存 full KV blocks	299
23.7 block hash 为什么需要 parent hash	300
23.8 extra hashes 包含什么	300
23.9 新请求命中 prefix cache 的流程	301
23.10 touch: 命中后不能马上被淘汰	301
23.11 请求结束后为什么 cache 还在	301
23.12 Eviction: 缓存不是永久的	302
23.13 Prefix cache 和 continuous batching 的关系	302
23.14 Prefix cache 和 Prompt Cache 的区别	303
23.15 多模态 prefix cache 的特殊问题	303
23.16 cache salt 与多租户安全	303
23.17 Hash 算法和碰撞风险	304
23.18 Hybrid attention 下的 prefix cache	304
23.19 命中率为什么低	304
23.20 Prefix cache 的副作用	305
23.21 什么时候应该启用或重视 Prefix cache	305
23.22 面试官会怎么问	306
23.23 标准回答模板	306
23.24 Prefix cache 公式、隔离门禁和可运行 demo	306
23.25 小练习	311
23.26 本章总结	311
第 24 章 Tensor Parallel、Pipeline Parallel 与 serving 并行	311
24.0 本讲资料边界与第二轮精修口径	312
24.1 本章目标	312
24.2 先把四种并行放在一张表里	312
24.3 Serving 并行和训练并行有什么不同	313
24.4 Tensor Parallel 的直觉	313
24.5 TP 在 serving 中解决什么问题	314
24.6 TP 为什么偏好同机高速互联	314
24.7 TP 和 KV Cache 的关系	314
24.8 Pipeline Parallel 的直觉	315
24.9 PP 适合什么场景	315
24.10 Pipeline bubble 是什么	315

24.11 Data Parallel 的直觉	316
24.12 DP rank 的 KV Cache 是独立的	316
24.13 Internal、Hybrid 和 External DP load balancing	317
24.14 Expert Parallel 的直觉	317
24.15 如何选择并行策略	317
24.16 并行度对进程数的影响	318
24.17 并行度对显存的影响	319
24.18 并行度对延迟和吞吐的影响	319
24.19 拓扑: PCIe、NVLink、NVSwitch、IB/RDMA	319
24.20 常见配置例子	320
24.21 常见工程坑	320
24.22 面试官会怎么问	321
24.23 标准回答模板	321
24.24 Serving 并行公式、拓扑门禁和可运行 demo	321
24.25 小练习	328
24.26 本章总结	328
第 25 章 vLLM 性能瓶颈、参数调优和工程坑	328
25.0 本讲资料边界与第二轮精修口径	328
25.1 本章目标	329
25.2 调优前先拒绝一个误区	329
25.3 指标总览: 不要只看 QPS	330
25.4 把请求生命周期拆开	331
25.5 TTFT 高怎么排查	331
25.6 TPOT 高怎么排查	332
25.7 吞吐低怎么排查	332
25.8 显存 OOM 怎么排查	333
25.9 preemption 频繁意味着什么	333
25.10 max_num_batched_tokens 怎么调	334
25.11 max_num_seqs 怎么调	334
25.12 gpu_memory_utilization 怎么调	335
25.13 max_model_len 和上下文预算	335
25.14 Chunked prefill 怎么看	336
25.15 Prefix cache 怎么调	336
25.16 并行策略怎么参与调优	337
25.17 CPU 瓶颈和 NUMA 问题	337
25.18 Streaming 和输出链路坑	338
25.19 多模态 serving 的特殊调优	338
25.20 量化不是只看显存	338
25.21 调优实验应该怎么做	339
25.22 常见线上事故和处理	339
25.23 上线调优的安全策略	340
25.24 面试官会怎么问	340
25.25 标准回答模板	340
25.26 vLLM 调优公式、事故归因和可运行 demo	341
25.27 小练习	346
25.28 本章总结	346
第 26 章 SGLang 解决什么问题	347
26.0 本讲资料边界与第二轮精修口径	347
26.1 本章目标	347
26.2 从 vLLM 的问题意识说起	348
26.3 复杂 LLM 程序是什么	348
26.4 为什么普通 serving API 不够表达这些程序	349
26.5 SGLang 的核心思想: 前后端协同设计	349
26.6 RadixAttention 解决什么问题	350

26.7 为什么 radix tree 比单个 cache key 更适合复杂程序	350
26.8 SGLang 为什么强调 structured generation	351
26.9 SGLang 面向哪些 workload	351
26.10 SGLang 和 vLLM 的区别：先给直觉	352
26.11 为什么自动 KV cache 复用很重要	352
26.12 SGLang 的前端语言解决什么痛点	353
26.13 OpenAI-compatible API 和 SGLang language 的关系	353
26.14 SGLang 不是解决所有问题的银弹	354
26.15 用一个例子理解 SGLang 的价值	354
26.16 和本书后续章节的关系	355
26.17 面试官会怎么问	355
26.18 标准回答模板	355
26.19 SGLang 动机公式、prefix sharing 和可运行 demo	356
26.20 小练习	360
26.21 本章总结	360
第 27 章 SGLang Runtime 总览	360
27.0 本讲资料边界与第二轮精修口径	360
27.1 本章目标	361
27.2 先给一张总览图	361
27.3 Runtime 在 SGLang 系统中的位置	362
27.4 SRT 是什么	363
27.5 四种入口：OpenAI API、native API、offline engine、frontend language	363
27.6 请求进入 runtime 后发生什么	364
27.7 Request state：请求不是一段字符串	365
27.8 Tokenization 和 chat template	365
27.9 Sampling params：生成行为如何进入 runtime	366
27.10 Scheduler 的核心职责	366
27.11 Prefill 调度	367
27.12 Decode 调度	367
27.13 Memory pool：KV cache 从哪里来	368
27.14 RadixAttention 在 runtime 中的位置	368
27.15 Model runner 负责什么	369
27.16 Attention backend 和 kernel	369
27.17 Sampler：logits 到 token	370
27.18 Structured output 在 runtime 里的位置	370
27.19 Streaming 输出	371
27.20 Offline engine 为什么重要	371
27.21 SGLang Runtime 和 vLLM Runtime 的相似点	372
27.22 SGLang Runtime 和 vLLM Runtime 的差异点	372
27.23 一个请求的端到端例子	372
27.24 SGLang Runtime 的关键指标	373
27.25 常见误解	374
27.26 面试官会怎么问	375
27.27 标准回答模板	375
27.28 SGLang Runtime 指标、模块门禁和可运行 demo	375
27.29 小练习	381
27.30 本章总结	381
第 28 章 RadixAttention 与 Prefix Sharing	381
28.0 本讲资料边界与第二轮精修口径	381
28.1 本章目标	382
28.2 Prefix sharing 是什么	382
28.3 为什么 prefix sharing 能省 prefill	383
28.4 为什么复杂 LLM 程序特别需要 prefix sharing	383
28.5 普通 prefix cache 的局限	384

28.6 Trie 和 radix tree	384
28.7 RadixAttention 的数据结构直觉	385
28.8 KV cache value 是什么	385
28.9 Longest prefix match	386
28.10 为什么需要 split	387
28.11 Insert 流程	387
28.12 一个完整例子: 多轮 chat	388
28.13 一个完整例子: few-shot batch	388
28.14 一个完整例子: self-consistency	389
28.15 一个完整例子: Tree-of-Thought	389
28.16 RadixAttention 和 attention kernel 的关系	390
28.17 和 memory pool 的协同	390
28.18 Ref count: 哪些节点不能淘汰	391
28.19 Eviction: 为什么通常淘汰叶子	391
28.20 LRU 为什么不总是最优	392
28.21 Scheduler 为什么要知道 cache hit length	392
28.22 Cache-aware scheduling	393
28.23 Token prefix 完全匹配, 不是语义相似	393
28.24 正确性边界	394
28.25 多模态场景	394
28.26 与 vLLM prefix caching 的对比	394
28.27 RadixAttention 不适合解决什么	395
28.28 典型收益场景	395
28.29 常见性能现象	395
28.30 调试 RadixAttention 的思路	396
28.31 面试官会怎么问	396
28.32 标准回答模板	397
28.33 RadixAttention 公式、split / eviction 门禁和可运行 demo	397
28.34 小练习	402
28.35 本章总结	403
第 29 章 SGLang Scheduler 设计	403
29.0 本讲资料边界与第二轮精修口径	403
29.1 本章目标	404
29.2 先看 scheduler 在 runtime 中的位置	404
29.3 Scheduler 的输入	404
29.4 Scheduler 的输出	405
29.5 请求状态机	406
29.6 Engine step: 调度的基本节拍	406
29.7 Prefill 和 decode 为什么要一起调度	407
29.8 最小调度策略	407
29.9 Token budget: 本轮最多算多少 token	408
29.10 Sequence budget: 本轮最多跑多少序列	409
29.11 KV budget: 能算不代表放得下	409
29.12 Cache hit length 如何进入调度	409
29.13 Cache-aware admission	410
29.14 Decode-first 策略	410
29.15 Prefill 配额和防饥饿	411
29.16 Chunked prefill	411
29.17 Structured output 对调度的影响	412
29.18 Frontend program 分支对调度的影响	412
29.19 Streaming 和 abort	413
29.20 Eviction 和调度的交互	413
29.21 Tail latency: p99 为什么难	413
29.22 一个完整调度例子	414
29.23 一个简化 scheduler 伪代码	414

29.24 常见误解	415
29.25 面试官会怎么问	416
29.26 标准回答模板	416
29.27 SGLang Scheduler 公式、门禁和可运行 demo	416
29.28 小练习	423
29.29 本章总结	423
第 30 章 Structured Generation 与 Constrained Decoding	423
30.0 本讲资料边界与第二轮精修口径	423
30.1 本章目标	424
30.2 为什么结构化输出重要	424
30.3 后处理和重试的问题	425
30.4 Constrained decoding 的核心流程	425
30.5 Grammar state 是什么	426
30.6 Token-level 约束比字符级约束更难	427
30.7 SGLang 支持哪些结构化约束	427
30.8 JSON schema	427
30.9 Regex	428
30.10 EBNF grammar	428
30.11 Structural tag	429
30.12 Choices	429
30.13 Structured output 在请求里怎么传	430
30.14 Runtime 内部执行链路	431
30.15 Grammar mask 和 sampler 的顺序	431
30.16 如果合法 token 集合为空怎么办	431
30.17 Constrained decoding 不保证事实正确	432
30.18 Prompt 仍然重要	432
30.19 Structured output 对性能的影响	433
30.20 对 scheduler 的影响	433
30.21 Streaming 下的结构化输出	433
30.22 Tool call 和 agent 的关系	434
30.23 和 RadixAttention 的关系	434
30.24 常见错误	435
30.25 如何选择约束方式	435
30.26 调试 structured output	435
30.27 面试官会怎么问	436
30.28 标准回答模板	436
30.29 Structured Generation 公式、mask 门禁和可运行 demo	436
30.30 小练习	442
30.31 本章总结	442
第 31 章 Speculative Decoding 在 SGLang 中的角色	442
31.0 本讲资料边界与第二轮精修口径	442
31.1 本章目标	443
31.2 为什么 decode 阶段慢	443
31.3 核心思想：先猜，再验	444
31.4 Draft 和 target	444
31.5 Verify 和 accept length	444
31.6 为什么它不改变输出分布	445
31.7 最小伪代码	445
31.8 主要优化什么指标	446
31.9 收益取决于什么	446
31.10 接受率为什么会变化	446
31.11 SGLang 支持的 speculative decoding 类型	447
31.12 EAGLE / EAGLE3 的定位	447
31.13 Standalone draft model	448

31.14 NGRAM speculative decoding	448
31.15 MTP: Multi-Token Prediction	449
31.16 Adaptive speculative decoding	449
31.17 Speculative decoding 和 scheduler	449
31.18 和 KV cache 的关系	450
31.19 和 RadixAttention 的关系	450
31.20 和 structured output 的关系	451
31.21 和 streaming 的关系	451
31.22 参数调优直觉	451
31.23 关键监控指标	452
31.24 常见性能现象	452
31.25 OOM 和显存问题	453
31.26 什么时候适合开启	453
31.27 和 PD 分离的关系	453
31.28 常见误解	454
31.29 面试官会怎么问	454
31.30 标准回答模板	455
31.31 Speculative Decoding 公式、runtime 门禁和可运行 demo	455
31.32 小练习	459
31.33 本章总结	460
第 32 章 Multi-turn、Tool Use 和 Agent Serving 支持	460
32.0 本讲资料边界与第二轮精修口径	460
32.1 本章目标	461
32.2 从单轮 chat 到多轮状态	461
32.3 多轮 chat 的 runtime 状态	462
32.4 Chat template 为什么影响 cache	462
32.5 Tool use 的基本流程	462
32.6 Tool schema 是什么	463
32.7 Tool parser 的作用	463
32.8 Tool choice	464
32.9 Tool call 的结构化约束	464
32.10 工具执行不等于模型推理	465
32.11 Agent loop 是什么	465
32.12 Agent trajectory 的 prefix sharing	465
32.13 Agent serving 和普通 chat serving 的差异	466
32.14 Scheduler 面临的新问题	466
32.15 Session-aware routing	467
32.16 Context growth 和裁剪	467
32.17 Tool result 回灌	468
32.18 Tool errors 和恢复	468
32.19 Parallel tool calls	469
32.20 Agent search 和分支	469
32.21 Agent serving 的指标	469
32.22 安全边界	470
32.23 和 speculative decoding 的关系	470
32.24 和 structured output 的关系	471
32.25 和 RadixAttention 的关系	471
32.26 常见工程坑	471
32.27 面试官会怎么问	472
32.28 标准回答模板	472
32.29 Agent Serving 公式、状态门禁和可运行 demo	473
32.30 小练习	478
32.31 本章总结	478
第 33 章 SGLang 和 vLLM 的架构对比	478

33.0 本讲资料边界与第二轮精修口径	478
33.1 本章目标	479
33.2 共同底层问题	479
33.3 问题出发点对比	480
33.4 一张总表	480
33.5 PagedAttention vs RadixAttention	481
33.6 Prefix cache 对比	481
33.7 Scheduler 对比	482
33.8 Request 模型对比	482
33.9 API 和入口对比	483
33.10 Structured output 对比	483
33.11 Tool use 和 agent 对比	483
33.12 Speculative decoding 对比	484
33.13 并行和多 GPU 对比	484
33.14 性能调优思路对比	485
33.15 选型：什么时候优先 vLLM	485
33.16 选型：什么时候优先 SGLang	485
33.17 也可以组合使用	486
33.18 面试中容易犯的错误	486
33.19 面试官会怎么问	487
33.20 标准回答模板	487
33.21 架构对比公式、workload router 和可运行 demo	487
33.22 小练习	491
33.23 本章总结	491
第 34 章 mini-sglang 源码学习路径	491
34.0 本讲资料边界与第二轮精修口径	492
34.1 本章目标	492
34.2 为什么需要教学项目	492
34.3 阅读前的概念地图	493
34.4 不建议的阅读方式	493
34.5 推荐阅读顺序	494
34.6 第一步：跑通最小请求	494
34.7 第二步：找到入口层	494
34.8 第三步：请求对象和状态机	495
34.9 第四步：scheduler 主循环	495
34.10 第五步：KV cache 和 memory pool	496
34.11 第六步：RadixAttention	496
34.12 第七步：model runner	497
34.13 第八步：sampler	497
34.14 Structured output 源码怎么读	498
34.15 Tool use 源码怎么读	498
34.16 Streaming 源码怎么读	498
34.17 推荐做的 8 个实验	499
34.18 源码笔记模板	499
34.19 如何从 mini-sglang 过渡到真实 SGLang	500
34.20 和本书前面章节的对应关系	500
34.21 面试中怎么讲 mini-sglang	501
34.22 常见误区	501
34.23 面试官会怎么问	501
34.24 标准回答模板	502
34.25 源码阅读覆盖率、实验门禁和可运行 demo	502
34.26 小练习	505
34.27 本章总结	505
第 35 章 Prefill 和 Decode 的资源画像为什么不同	505

35.0 本讲资料边界与第二轮精修口径	505
35.1 本章目标	506
35.2 再看一次生成生命周期	506
35.3 Prefill 的执行画像	506
35.4 Decode 的执行画像	507
35.5 一张表总结差异	507
35.6 TTFT 为什么主要看 Prefill	507
35.7 TPOT 为什么主要看 Decode	508
35.8 长 Prefill 如何干扰 Decode	508
35.9 Decode-first 为什么也有问题	509
35.10 GPU 利用率视角	509
35.11 KV cache 视角	509
35.12 Batch shape 视角	510
35.13 Latency SLO 视角	510
35.14 成本视角	511
35.15 输入输出比例决定压力	511
35.16 三类典型 workload	511
35.17 为什么 chunked prefill 还不够	512
35.18 PD 分离的基本动机	512
35.19 PD 分离不是免费午餐	512
35.20 什么时候不需要 PD 分离	513
35.21 观测指标	513
35.22 面试官会怎么问	514
35.23 标准回答模板	514
35.24 资源画像公式、干扰门禁和可运行 demo	514
35.25 小练习	517
35.26 本章总结	518
第 36 章 PD 分离解决什么问题	518
36.0 本讲资料边界与第二轮精修口径	518
36.1 本章目标	518
36.2 什么是 PD 分离	519
36.3 统一引擎的问题一: Prefill interruption	519
36.4 统一引擎的问题二: Decode 让 Prefill 饥饿	519
36.5 统一引擎的问题三: 资源画像不匹配	520
36.6 统一引擎的问题四: 扩缩容耦合	520
36.7 统一引擎的问题五: 尾延迟难治理	521
36.8 统一引擎的问题六: DP attention imbalance	521
36.9 PD 分离改善哪些指标	521
36.10 PD 分离如何降低 Decode 抖动	522
36.11 PD 分离如何改善 Prefill 吞吐	522
36.12 PD 分离如何支持异构资源	522
36.13 PD 分离的核心代价: KV cache 迁移	523
36.14 KV 迁移成本有多重要	523
36.15 路由问题	523
36.16 状态同步问题	524
36.17 故障恢复问题	524
36.18 什么时候 PD 分离收益明显	525
36.19 什么时候不适合 PD 分离	525
36.20 和 chunked prefill 的关系	525
36.21 和 prefix cache 的关系	526
36.22 和 speculative decoding 的关系	526
36.23 PD 分离的价值边界	526
36.24 面试官会怎么问	527
36.25 标准回答模板	527
36.26 PD 分离收益公式、代价门禁和可运行 demo	527

36.27 小练习	531
36.28 本章总结	531
第 37 章 PD 分离的系统架构	531
37.0 本讲资料边界与第二轮精修口径	532
37.1 本章目标	532
37.2 从 unified engine 到 PD 架构	532
37.3 总体架构图	533
37.4 核心组件一: PD Router	533
37.5 Router 为什么不能只随机选 worker	534
37.6 核心组件二: Prefill Worker	534
37.7 Prefill Worker 的内部流程	535
37.8 核心组件三: Decode Worker	535
37.9 Decode Worker 的内部流程	536
37.10 核心组件四: KV Transfer Backend	536
37.11 KV Transfer 的几种形态	537
37.12 Bootstrap 机制是什么	537
37.13 请求生命周期: 从接入到首 token	538
37.14 请求生命周期: 持续 decode	538
37.15 跨组件状态机	538
37.16 必须维护的 Metadata	539
37.17 Prefill Pool 和 Decode Pool 的扩缩容	540
37.18 路由策略	540
37.19 Cache-Aware Routing 在 PD 中的含义	541
37.20 健康检查和故障隔离	541
37.21 故障处理: Prefill 失败	541
37.22 故障处理: KV Transfer 失败	542
37.23 故障处理: Decode 失败	542
37.24 Client Abort 和资源清理	542
37.25 观测指标	543
37.26 部署形态一: 单机多 GPU PD	543
37.27 部署形态二: 跨节点 PD	544
37.28 部署形态三: 异构 P/D 资源池	544
37.29 和普通 Load Balancer 的区别	545
37.30 和 Pipeline Parallel 的区别	545
37.31 和 Expert Parallel / MoE 并行的区别	545
37.32 一个最小 PD Router 伪代码	546
37.33 架构设计中的关键取舍	546
37.34 Prefill 是否应该生成第一个 token	547
37.35 Push KV 还是 Pull KV	547
37.36 PD 分离的反压机制	548
37.37 多租户和 QoS	548
37.38 安全和版本一致性	549
37.39 什么时候 Router 应该拒绝请求	549
37.40 面试官会怎么问	549
37.41 标准回答模板	550
37.42 架构状态机、清理门禁和可运行 demo	550
37.43 小练习	555
37.44 本章总结	555
第 38 章 KV Cache 迁移、共享和路由	555
38.0 本讲资料边界与第二轮精修口径	555
38.1 本章目标	556
38.2 先回忆 KV Cache 是什么	556
38.3 为什么 KV Cache 不是普通数据	556
38.4 KV Cache 大小估算	557

38.5 KV Cache 迁移的基本问题	557
38.6 一个最小 KV Transfer 流程	558
38.7 Push 模式	558
38.8 Pull 模式	559
38.9 Push 和 Pull 的本质区别	559
38.10 共享模式	560
38.11 迁移模式和共享模式的区别	560
38.12 Paged KV Cache 对迁移的影响	561
38.13 Layout 兼容性	562
38.14 Tensor Parallel 下的 KV 迁移	562
38.15 Decode 为什么要先预留 KV Blocks	563
38.16 KV Transfer Backend 做什么	564
38.17 Router 在 KV 迁移中的职责	564
38.18 KV-aware Routing	565
38.19 Prefix-aware Routing	565
38.20 Prefix Cache 和 PD 分离的关系	566
38.21 Recompute、Migrate、Remote Read 的取舍	567
38.22 KV Cache 的生命周期	567
38.23 Reference Count	568
38.24 KV Eviction 策略	568
38.25 多级 KV Cache	569
38.26 KV Transfer 的性能指标	569
38.27 KV Transfer 的常见瓶颈	570
38.28 故障一: Prefill 成功但 Transfer 失败	570
38.29 故障二: Decode Reserved 后 Worker 崩溃	571
38.30 故障三: Decode 中途失败	571
38.31 KV Metadata 不一致	572
38.32 安全和多租户隔离	572
38.33 KV 路由策略示例	573
38.34 什么时候不应该迁移 KV	574
38.35 和 Chunked Prefill 的关系	574
38.36 面试官会怎么问	575
38.37 标准回答模板	575
38.38 KV Transfer 路由公式、隔离门禁和可运行 demo	575
38.39 小练习	580
38.40 本章总结	580
第 39 章 Chunked Prefill 与 Disaggregated Prefill	580
39.0 本讲资料边界与第二轮精修口径	581
39.1 本章目标	581
39.2 长 Prompt Prefill 的问题	581
39.3 为什么不能总是一次性 Prefill	582
39.4 什么是 Chunked Prefill	582
39.5 Chunked Prefill 的直觉	583
39.6 Chunked Prefill 解决什么问题	583
39.7 Chunked Prefill 不解决什么问题	583
39.8 Chunked Prefill 和 Continuous Batching	584
39.9 Chunk Size 的影响	584
39.10 Chunked Prefill 对 TTFT 的影响	585
39.11 Chunked Prefill 对 TPOT 的影响	585
39.12 Chunked Prefill 对吞吐的影响	586
39.13 Chunked Prefill 的状态机	586
39.14 Position 和 Attention Mask	587
39.15 KV Block 写入	587
39.16 Chunk 边界和 Block 边界	587
39.17 Disaggregated Prefill 是什么	588

39.18 Chunked Prefill vs Disaggregated Prefill	588
39.19 三种架构对比	589
39.20 Chunked Prefill 和 KV Transfer	589
39.21 流水化 Prefill 和 Transfer	590
39.22 Decode 能不能提前开始	590
39.23 Prefill Chunk 的调度优先级	591
39.24 Fairness: 短请求和长请求	591
39.25 Backpressure	592
39.26 Chunked Prefill 的 Metadata	592
39.27 Chunked Prefill 的失败处理	593
39.28 Chunked Prefill 和 Prefix Cache	593
39.29 Chunked Prefill 和 Speculative Decoding	593
39.30 Chunked Prefill 和 Pipeline Parallel	594
39.31 实现 Chunked Prefill 的简化伪代码	594
39.32 常见实现坑	595
39.33 什么时候应该使用 Chunked Prefill	595
39.34 如何调参	595
39.35 面试官会怎么问	596
39.36 标准回答模板	596
39.37 Chunked / Disaggregated Prefill 公式、门禁和可运行 demo	597
39.38 小练习	602
39.39 本章总结	602
第 40 章 多级 KV Cache: GPU、CPU 和远端缓存	602
40.0 本讲资料边界与第二轮精修口径	603
40.1 本章目标	603
40.2 为什么只靠 GPU KV Cache 不够	603
40.3 KV Cache 的访问冷热	604
40.4 四个层级的基本画像	604
40.5 GPU KV Cache	605
40.6 CPU KV Cache	605
40.7 Local SSD / NVMe KV Cache	606
40.8 Remote KV Cache	606
40.9 Recompute 也是一个层级	607
40.10 Offload、Swap、Remote Cache 的区别	607
40.11 多级 KV Cache 的基本状态机	608
40.12 Promote 和 Demote	608
40.13 Prefetch	609
40.14 Eviction	609
40.15 多级 KV Cache 和 Prefix Cache	610
40.16 多级 KV Cache 和 PD 分离	611
40.17 多级 KV Cache 和 Chunked Prefill	611
40.18 调度器需要知道什么	612
40.19 一个成本模型	612
40.20 正确性: KV 不是普通字节缓存	613
40.21 Metadata 设计	614
40.22 引用计数和生命周期	614
40.23 Partial Block 问题	615
40.24 多租户和安全隔离	615
40.25 网络瓶颈	616
40.26 压缩和量化	616
40.27 和 Attention Kernel 的关系	617
40.28 一个简化实现	617
40.29 常见策略组合	618
40.30 常见坑	618
40.31 观测指标	619

40.32 调参思路	619
40.33 面试官会怎么问	620
40.34 标准回答模板	620
40.35 多级 KV Cache 成本、驻留门禁和可运行 demo	620
40.36 小练习	625
40.37 本章总结	625
第 41 章 跨节点 Serving 和网络瓶颈	625
41.0 本讲资料边界与第二轮精修口径	626
41.1 本章目标	626
41.2 跨节点 Serving 到底跨了什么	626
41.3 网络为什么会进入关键路径	627
41.4 带宽和延迟不是一回事	627
41.5 常见互联层级	628
41.6 Data Parallel 跨节点	628
41.7 Tensor Parallel 跨节点	629
41.8 Pipeline Parallel 跨节点	629
41.9 TP + PP 组合	630
41.10 PD 分离跨节点	630
41.11 KV Transfer 的网络成本	631
41.12 Remote KV Cache 的网络成本	631
41.13 Router 也会制造网络瓶颈	632
41.14 拓扑距离	632
41.15 拥塞和尾延迟	633
41.16 Control Plane 和 Data Plane	633
41.17 Streaming 输出路径	634
41.18 跨节点 Batch 和负载均衡	634
41.19 Admission Control	634
41.20 Backpressure	635
41.21 故障模式	635
41.22 观测指标	636
41.23 如何定位网络瓶颈	637
41.24 部署策略	637
41.25 调优思路	638
41.26 常见坑	638
41.27 面试官会怎么问	639
41.28 标准回答模板	639
41.29 跨节点通信成本、路由门禁和可运行 demo	639
41.30 小练习	643
41.31 本章总结	643
第 42 章 PD 分离的优缺点、适用场景和反模式	643
42.0 本讲资料边界与第二轮精修口径	644
42.1 本章目标	644
42.2 先给结论	644
42.3 PD 分离的主要优点	644
42.4 PD 分离的主要缺点	645
42.5 收益不是看平均值	646
42.6 成本模型	646
42.7 适用场景一：长 Prompt 和长上下文	647
42.8 适用场景二：输入输出比例波动大	647
42.9 适用场景三：Streaming SLO 很严格	647
42.10 适用场景四：多租户隔离	648
42.11 适用场景五：Prefill 和 Decode 适合不同硬件	648
42.12 不适合场景一：请求都很短	648
42.13 不适合场景二：流量规模不够	649

42.14 不适合场景三: KV Transfer 条件差	649
42.15 不适合场景四: 主要瓶颈不在 P/D 干扰	649
42.16 不适合场景五: 缺少可观测性和运维能力	650
42.17 反模式一: 为了架构先进而 PD 分离	650
42.18 反模式二: 只拆服务, 不拆资源	650
42.19 反模式三: 只看 Prefill 加速, 不看 Decode 等待	651
42.20 反模式四: 无脑跨节点 PD	651
42.21 反模式五: Remote KV Cache 当万能药	651
42.22 反模式六: 没有 Backpressure	651
42.23 反模式七: 忽略失败清理	652
42.24 反模式八: 忽略租户和优先级	652
42.25 替代方案: 先做哪些优化	652
42.26 分阶段落地路线	653
42.27 判断清单	653
42.28 一个具体决策例子	654
42.29 和其他优化的选择关系	654
42.30 评估实验设计	655
42.31 面试官会怎么问	655
42.32 标准回答模板	656
42.33 PD 分离收益/代价门禁、反模式识别和可运行 demo	656
42.34 小练习	659
42.35 本章总结	659
第 43 章 如何把单机 Serving Engine 升级成 PD 分离架构	659
43.0 本讲资料边界与第二轮精修口径	660
43.1 本章目标	660
43.2 先看单机 Engine 长什么样	660
43.3 为什么不能直接拆两个服务	661
43.4 升级前要先做的模块边界	661
43.5 单机 Engine 的状态拆分	662
43.6 第一步: 把 Prefill 和 Decode 变成显式阶段	663
43.7 第二步: 抽象 Model Runner 接口	664
43.8 第三步: 抽象 KV Cache Manager	664
43.9 第四步: 把 Scheduler 拆成两类	665
43.10 第五步: 引入 Router 状态机	666
43.11 第六步: 把 Worker 角色显式化	666
43.12 第七步: 先做 Same-Node PD	667
43.13 第八步: 接入 KV Transfer	667
43.14 第九步: 重新设计 Streaming 路径	668
43.15 第十步: 引入 Backpressure	669
43.16 第十一步: 加完整观测	669
43.17 第十二步: 灰度和回滚	670
43.18 一个推荐演进路线	670
43.19 最小可行 PD 原型	671
43.20 关键接口设计示例	672
43.21 容量规划怎么变	673
43.22 故障处理设计	673
43.23 数据一致性问题	674
43.24 与 Prefix Cache 的关系	674
43.25 与 Chunked Prefill 的关系	675
43.26 常见改造反模式	675
43.27 面试官会怎么问	676
43.28 标准回答模板	676
43.29 单机到 PD 升级路线、接口门禁和可运行 demo	677
43.30 小练习	680
43.31 本章总结	680

第 44 章 nano-vLLM 学习路线	681
44.0 本讲资料边界与第二轮精修口径	681
44.1 本章目标	681
44.2 先明确学习目标	682
44.3 推荐先看项目结构	682
44.4 推荐阅读顺序	683
44.5 第一遍：只追踪一次 generate	684
44.6 第二遍：读 Sequence	684
44.7 第三遍：读 Scheduler	685
44.8 第四遍：读 Block Manager	686
44.9 第五遍：读 Model Runner	686
44.10 第六遍：读 Attention	687
44.11 第七遍：读 Sampler	687
44.12 第八遍：读模型结构	688
44.13 把 nano-vLLM 映射回本书前面章节	688
44.14 一轮 Engine Step 应该怎么理解	688
44.15 读 Prefix Cache 时看什么	689
44.16 读 Tensor Parallel 时看什么	689
44.17 nano-vLLM 和生产级 vLLM 的差距	690
44.18 适合做的改造练习	690
44.19 适合写进简历的项目描述	691
44.20 面试官会怎么问	691
44.21 标准回答模板	692
44.22 nano-vLLM 源码覆盖率、实验门禁和可运行 demo	692
44.23 小练习	696
44.24 本章总结	696
第 45 章 tiny-llm 学习路线	696
45.0 本讲资料边界与第二轮精修口径	696
45.1 本章目标	697
45.2 tiny-llm 和 nano-vLLM 的区别	697
45.3 tiny-llm 的课程式结构	698
45.4 推荐阅读顺序	698
45.5 第一阶段：从 Attention 开始	699
45.6 第二阶段：RoPE 和位置	699
45.7 第三阶段：Grouped Query Attention	700
45.8 第四阶段：RMSNorm 和 MLP	700
45.9 第五阶段：组装 Qwen-like 模型	700
45.10 第六阶段：Generate Response	701
45.11 第七阶段：Sampling	701
45.12 第八阶段：KV Cache	702
45.13 第九阶段：量化 Matmul	702
45.14 第十阶段：FlashAttention	703
45.15 第十一阶段：Continuous Batching	703
45.16 第十二阶段：Chunked Prefill	704
45.17 第十三阶段：Paged Attention	704
45.18 tiny-llm 和本书章节映射	705
45.19 适合做的改造练习	705
45.20 适合写进简历的项目描述	706
45.21 常见误区	706
45.22 面试官会怎么问	706
45.23 标准回答模板	707
45.24 tiny-llm 学习路线覆盖率、实验门禁和可运行 demo	707
45.25 小练习	711
45.26 本章总结	711

第 46 章 mini-sglang 学习路线	711
46.0 本讲资料边界与第二轮精修口径	712
46.1 本章目标	712
46.2 三个教学项目的分工	712
46.3 mini-sglang 的核心特征	713
46.4 和第 34 章的关系	713
46.5 先带着 SGLang 问题读	713
46.6 第一重点: Radix Cache	714
46.7 Radix Cache 和 PagedAttention 的关系	715
46.8 第二重点: Chunked Prefill	715
46.9 第三重点: Overlap Scheduling	716
46.10 第四重点: Online Serving	716
46.11 第五重点: Interactive Shell	717
46.12 第六重点: Tensor Parallelism	717
46.13 第七重点: Kernel Backend	717
46.14 第八重点: Structured Generation	718
46.15 第九重点: Tool / Agent Serving	718
46.16 mini-sglang 和完整 SGLang 的差距	719
46.17 和生产 SGLang 源码怎么衔接	719
46.18 适合做的改造练习	719
46.19 适合写进简历的项目描述	720
46.20 常见误区	720
46.21 面试官会怎么问	721
46.22 标准回答模板	721
46.23 mini-sglang runtime 学习覆盖率、实验门禁和可运行 demo	721
46.24 小练习	725
46.25 本章总结	726
第 47 章 从教学项目抽象推理框架核心模块	726
47.0 本讲资料边界与第二轮精修口径	726
47.1 本章目标	727
47.2 先画总架构	727
47.3 从三个项目抽象共同模块	728
47.4 最小推理框架的模块清单	728
47.5 Request: 请求不是一段文本	729
47.6 Sequence 和 Request 的关系	730
47.7 Scheduler: 推理框架的决策中心	730
47.8 为什么要有 token budget	731
47.9 KVCacheManager: 显存不是 append list	732
47.10 ModelRunner: 不要把调度逻辑塞进模型	732
47.11 BatchBuilder: 容易被忽略的胶水层	733
47.12 Sampler: 采样不是 argmax	734
47.13 OutputProcessor: 生成 token 后还没结束	735
47.14 Engine Loop: 推理框架的心跳	735
47.15 Prefill 和 Decode 的统一抽象	736
47.16 状态机视角	736
47.17 Metrics: 没有观测就没有优化	737
47.18 模块边界的反模式	738
47.19 推荐的第一版 mini engine	738
47.20 第一版执行流程	739
47.21 从第一版到后续章节的升级路线	740
47.22 如何写成简历项目	740
47.23 面试高频问题	741
47.24 标准回答模板	741
47.25 核心模块边界、状态 ownership 和可运行 demo	741
47.26 小练习	746

47.27 本章总结	746
第 48 章 从 naive scheduler 升级到 continuous batching	746
48.0 本讲资料边界与第二轮精修口径	747
48.1 本章目标	747
48.2 naive scheduler 长什么样	747
48.3 一个直观例子	748
48.4 continuous batching 的核心思想	748
48.5 continuous batching 不等于无限插队	749
48.6 第一步：定义调度输入和输出	749
48.7 Request 需要增加哪些字段	750
48.8 最小 continuous batching 状态机	751
48.9 token budget 的作用	751
48.10 decode-first 策略	752
48.11 prefill-first 策略的问题	752
48.12 bounded prefill	753
48.13 admission control: 不是所有 waiting 都能进来	753
48.14 一个更完整的调度伪代码	754
48.15 调度顺序的权衡	755
48.16 BatchBuilder 需要怎么变	756
48.17 OutputProcessor 需要怎么变	757
48.18 KV capacity 如何影响调度	757
48.19 抢占和暂停：第一版可以先不做	758
48.20 scheduler debug log	758
48.21 压测场景设计	759
48.22 对比指标	760
48.23 常见 bug	760
48.24 面试高频问题	761
48.25 标准回答模板	761
48.26 Naive Scheduler 到 Continuous Batching 升级门禁和可运行 demo	762
48.27 小练习	767
48.28 本章总结	767
第 49 章 从 list KV cache 升级到 paged KV cache	768
49.0 本讲资料边界与第二轮精修口径	768
49.1 本章目标	768
49.2 教学版 list KV cache 长什么样	769
49.3 list KV cache 的核心问题	769
49.4 paged KV cache 的升级目标	770
49.5 最小数据结构	770
49.6 block size 怎么选	771
49.7 计算需要多少 blocks	771
49.8 can_allocate 不是简单看 prompt length	771
49.9 分配 blocks	772
49.10 prefill 阶段如何使用 paged KV	773
49.11 decode 阶段如何追加 token	773
49.12 token position 到 physical slot	774
49.13 slot mapping 是什么	774
49.14 block table 传给谁	774
49.15 BatchBuilder 的改造	775
49.16 Scheduler 和 BlockManager 的边界	776
49.17 admission control 的变化	776
49.18 decode 前也要检查 block	777
49.19 free request 的时机	777
49.20 free 函数要幂等	778
49.21 防止 double free	778

49.22 block table 的生命周期	778
49.23 和 continuous batching 的结合	779
49.24 OOM 应该在哪里被拦住	779
49.25 debug log 设计	780
49.26 需要观测哪些指标	780
49.27 压测场景一：短请求高并发	781
49.28 压测场景二：长 prompt 短输出	781
49.29 压测场景三：长输出 decode	781
49.30 和 prefix caching 的关系	781
49.31 和 preemption 的关系	782
49.32 常见 bug	782
49.33 面试高频问题	783
49.34 标准回答模板	783
49.35 Paged KV 升级公式、slot mapping 和可运行 demo	783
49.36 小练习	788
49.37 本章总结	788
第 50 章 实现 prefix cache 和 prompt cache	789
50.0 本讲资料边界与第二轮精修口径	789
50.1 本章目标	789
50.2 先划清边界：prompt cache vs prefix cache	790
50.3 为什么必须建立在 paged KV cache 上	790
50.4 第一版范围	790
50.5 最小数据结构	791
50.6 ref count 的含义	791
50.7 block hash 包含什么	792
50.8 只缓存 full blocks	793
50.9 把 prompt 切成 full blocks	793
50.10 查找最长命中 prefix	794
50.11 cache hit 后如何初始化 request	794
50.12 只 prefill suffix	794
50.13 prefill suffix 前还要分配 blocks	795
50.14 什么时候把 block 加入 cache	795
50.15 register_cached_block	796
50.16 request 结束时如何释放	796
50.17 cache eviction 第一版怎么做	797
50.18 prefix cache 对 scheduler 的影响	797
50.19 cache hit 后的状态机	798
50.20 BatchBuilder 需要注意什么	798
50.21 OutputProcessor 需要注意什么	799
50.22 全 prompt 命中怎么办	799
50.23 和 result cache 不一样	799
50.24 cache salt 和租户隔离	800
50.25 LoRA 和模型版本	800
50.26 多模态输入	801
50.27 日志设计	801
50.28 指标设计	801
50.29 压测场景一：固定 system prompt	802
50.30 压测场景二：长文档多问题	802
50.31 压测场景三：低复用随机 prompt	802
50.32 常见 bug	803
50.33 面试高频问题	803
50.34 标准回答模板	804
50.35 Prefix / Prompt Cache 公式、生命周期和可运行 demo	804
50.36 小练习	809
50.37 本章总结	809

第 51 章 实现 preemption、recompute 和 swap	809
51.0 本讲资料边界与第二轮精修口径	810
51.1 本章目标	810
51.2 preemption 解决的不是算力问题	811
51.3 四个概念不要混	811
51.4 preemption 的两种策略	812
51.5 为什么通常优先讲 recompute	812
51.6 swap 为什么更复杂	812
51.7 request 状态机	813
51.8 recompute preemption 的最小实现	813
51.9 recompute 恢复流程	814
51.10 为什么 output tokens 也要重算	814
51.11 scheduler 什么时候触发 preemption	815
51.12 victim 选择策略	815
51.13 preempt 多少个请求	816
51.14 preemption 和 prefix cache 的关系	816
51.15 preemption 和 chunked prefill 的关系	817
51.16 swap 的最小数据结构	817
51.17 swap out 流程	818
51.18 swap in 流程	818
51.19 swap 的一致性问题	818
51.20 recompute 和 swap 如何选择	819
51.21 对延迟指标的影响	819
51.22 需要记录哪些日志	820
51.23 需要暴露哪些指标	820
51.24 压测场景一：突发长 prompt	821
51.25 压测场景二：长输出 decode 压力	821
51.26 压测场景三：prefix cache 占满 blocks	821
51.27 常见 bug	821
51.28 面试高频问题	822
51.29 标准回答模板	822
51.30 Preemption / Recompute / Swap 公式、状态机和可运行 demo	823
51.31 小练习	827
51.32 本章总结	827
第 52 章 把调度、KV cache、prefix cache 和 preemption 串起来	828
52.0 本讲资料边界与第二轮精修口径	828
52.1 本章目标	828
52.2 先看完整循环	829
52.3 request 队列	829
52.4 request 的最小状态	830
52.5 scheduler 每轮要做的事	830
52.6 decode 优先还是 prefill 优先	831
52.7 token budget 和 block budget	831
52.8 prefix cache lookup 的时机	832
52.9 prefill chunk 怎么选	832
52.10 running 请求何时 decode	832
52.11 KV block 分配的统一入口	833
52.12 memory pressure 处理顺序	833
52.13 schedule plan 的数据结构	834
52.14 BatchBuilder 做什么	834
52.15 OutputProcessor 做什么	835
52.16 一个完整 schedule 伪代码	835
52.17 commit plan 为什么重要	836
52.18 失败要能回滚	837
52.19 状态不变量	837

52.20 可观测性：每轮调度日志	838
52.21 指标分层	838
52.22 压测场景一：稳定短请求	839
52.23 压测场景二：共享 system prompt	839
52.24 压测场景三：长上下文混合短请求	839
52.25 压测场景四：故意打爆 KV	840
52.26 常见 bug	840
52.27 面试高频问题	841
52.28 标准回答模板	841
52.29 统一调度循环公式、状态不变量和可运行 demo	841
52.30 小练习	848
52.31 本章总结	849
第 53 章 构建 serving engine 的 benchmark、指标和调参实验框架	849
53.0 本讲资料边界与第二轮精修口径	850
53.1 本章目标	850
53.2 为什么 benchmark 是 serving engine 的核心模块	850
53.3 四个最基础的用户侧指标	851
53.3.1 TTFT	851
53.3.2 TPOT	851
53.3.3 E2E latency	852
53.3.4 Throughput	852
53.4 平均值为什么危险	852
53.5 engine 内部指标	853
53.5.1 scheduler 指标	853
53.5.2 KV cache 指标	854
53.5.3 prefix cache 指标	854
53.5.4 preemption 指标	854
53.6 请求级 trace	855
53.7 engine step 级 trace	856
53.8 最小指标收集器	856
53.9 benchmark workload 设计	857
53.9.1 短请求 baseline	857
53.9.2 长 prompt 场景	858
53.9.3 共享 prefix 场景	858
53.9.4 长短混合场景	858
53.9.5 KV 压力场景	859
53.10 workload 生成器	859
53.11 open-loop 和 closed-loop 压测	860
53.11.1 closed-loop	860
53.11.2 open-loop	861
53.12 参数扫描	861
53.13 如何比较两次实验	862
53.14 常见调参方向	862
53.14.1 TTFT 太高	862
53.14.2 TPOT 抖动大	863
53.14.3 吞吐低	863
53.14.4 preemption 过多	863
53.15 benchmark 报告模板	864
53.16 最容易犯的 benchmark 错误	864
53.17 面试高频问题	865
53.18 标准回答模板	866
53.19 Benchmark Framework 公式、回归门禁和可运行 demo	866
53.20 小练习	870
53.21 本章总结	871

第 54 章 把同步 engine loop 升级成异步 serving 架构	871
54.0 本讲资料边界与第二轮精修口径	872
54.1 本章目标	872
54.2 同步 loop 的隐藏问题	872
54.3 哪些工作应该从主循环拆出去	873
54.4 最小异步架构	874
54.5 API server 的职责	875
54.6 Tokenizer worker	875
54.7 Engine core loop	876
54.8 OutputProcessor 不应该直接写网络	877
54.9 Detokenizer 和 stream worker	877
54.10 backpressure	878
54.10.1 tokenizer_queue 满	878
54.10.2 engine_input_queue 满	879
54.10.3 engine_output_queue 满	879
54.10.4 client stream queue 满	879
54.11 timeout 和 cancel	879
54.12 admission control	880
54.13 状态所有权	881
54.14 线程、进程还是 asyncio	881
54.14.1 asyncio	881
54.14.2 线程	882
54.14.3 进程	882
54.14.4 实用组合	882
54.15 异步架构下的指标	882
54.15.1 队列指标	882
54.15.2 组件耗时	882
54.15.3 cancel 和 timeout	883
54.16 异步拆分后的 benchmark 对比	883
54.17 常见 bug	883
54.18 面试高频问题	884
54.19 标准回答模板	884
54.20 异步 serving 架构公式、背压门禁和可运行 demo	885
54.21 小练习	891
54.22 本章总结	891
第 55 章 多 worker、多 GPU 和 request router: 从单机 engine 到分布式 serving	892
55.0 本讲资料边界与第二轮精修口径	892
55.1 本章目标	893
55.2 为什么需要多个 worker	893
55.3 最小多 worker 架构	893
55.4 WorkerHandle	894
55.5 为什么 round-robin 不够	895
55.6 load-aware routing	895
55.7 请求成本估算	896
55.8 least-loaded 也不总是最优	897
55.9 sticky routing	897
55.10 sticky + load-aware	897
55.11 多模型和多 adapter 路由	898
55.12 Replica parallel 和 tensor parallel	898
55.12.1 Replica parallel	898
55.12.2 Tensor parallel	899
55.13 Pipeline parallel 和 expert parallel	899
55.13.1 Pipeline parallel	899
55.13.2 Expert parallel	900
55.14 worker health check	900

55.15 请求重试	900
55.15.1 还没开始输出	901
55.15.2 已经输出 token	901
55.15.3 deterministic decode	901
55.16 worker crash 后如何清理	901
55.17 分布式状态边界	901
55.17.1 API server owns connection state	902
55.17.2 Router owns routing state	902
55.17.3 Worker owns engine state	902
55.18 输出路径怎么走	902
55.18.1 worker 直接回 API server	902
55.18.2 worker 输出经过 router	903
55.19 全局 admission control	903
55.20 多 worker benchmark	904
55.21 常见错误	904
55.22 面试高频问题	905
55.23 标准回答模板	905
55.24 Multi-Worker Router 公式、故障门禁和可运行 demo	905
55.25 小练习	912
55.26 本章总结	913
第 56 章 Tensor Parallel、Pipeline Parallel 和分布式 KV Cache	913
56.0 本讲资料边界与第二轮精修口径	914
56.1 本章目标	914
56.2 先区分三个概念	915
56.2.1 Replica parallel	915
56.2.2 Tensor parallel	915
56.2.3 Pipeline parallel	915
56.3 Router 看到的并行组	915
56.4 Tensor parallel 的基本直觉	916
56.4.1 Column parallel	916
56.4.2 Row parallel	916
56.5 为什么 tensor parallel 有通信开销	917
56.6 Prefill 和 decode 对 TP 的压力不同	917
56.7 Tensor parallel 下的 KV cache	918
56.8 TP 下 block allocation 的一致性	918
56.9 Pipeline parallel 的基本直觉	919
56.10 Pipeline bubble	919
56.11 Pipeline parallel 下的 KV cache	919
56.12 TP + PP 组合	920
56.13 分布式 KV cache 为什么难迁移	920
56.14 KV cache 的 locality	921
56.15 分布式 prefix cache	921
56.15.1 worker-local prefix cache	921
56.15.2 centralized prefix directory	922
56.15.3 remote KV reuse	922
56.16 Scheduler 和 parallel group 的关系	922
56.17 rank-local 数据	923
56.18 分布式执行的指标	923
56.19 TP size 怎么选	923
56.20 PP size 怎么选	924
56.21 常见错误	924
56.22 面试高频问题	925
56.23 标准回答模板	925
56.24 Distributed Parallel KV 公式、通信门禁和可运行 demo	926
56.25 小练习	931

56.26 本章总结	931
第 57 章 OpenAI-compatible API、流式协议、错误码和限流鉴权	932
57.0 本讲资料边界与第二轮精修口径	932
57.1 本章目标	933
57.2 为什么要兼容 OpenAI API	933
57.3 常见 endpoint	933
57.4 Chat Completions 和 Completions	934
57.5 Chat template	935
57.6 请求字段校验	936
57.7 从 API request 到 EngineRequest	936
57.8 非流式响应	937
57.9 流式响应	937
57.10 SSE 的几个坑	938
57.10.1 每个事件要以空行结束	938
57.10.2 不要缓存响应	938
57.10.3 处理 client disconnect	938
57.10.4 delta 不是完整文本	939
57.11 finish_reason	939
57.12 Usage 统计	939
57.13 错误响应格式	940
57.14 错误码分层	940
57.14.1 400 Bad Request	941
57.14.2 401 Unauthorized	941
57.14.3 403 Forbidden	941
57.14.4 413 Context Length Exceeded	941
57.14.5 429 Rate Limit	941
57.14.6 503 Service Unavailable	941
57.15 流式错误怎么处理	941
57.15.1 stream 开始前错误	941
57.15.2 stream 开始后错误	941
57.16 鉴权	942
57.17 权限模型	942
57.18 限流维度	943
57.19 token-based rate limit	943
57.20 concurrency limit	943
57.21 admission control 和 rate limit 的区别	944
57.22 request_id、trace_id 和 idempotency_key	944
57.22.1 request_id	944
57.22.2 trace_id	944
57.22.3 idempotency_key	945
57.23 日志和隐私	945
57.24 API 层指标	945
57.25 OpenAI-compatible 不等于完全相同	946
57.26 最小请求处理流程	946
57.27 常见错误	947
57.28 面试高频问题	948
57.29 标准回答模板	948
57.30 API Compatibility 公式、错误门禁和可运行 demo	948
57.31 小练习	955
57.32 本章总结	955
第 58 章 生产部署：容器镜像、模型加载、健康检查、滚动升级和灰度发布	956
58.0 本讲资料边界与第二轮精修口径	956
58.1 本章目标	957
58.2 为什么 LLM serving 部署更难	957

58.3 生产部署的基本形态	958
58.4 镜像构建的核心原则	958
58.5 CUDA、Driver、PyTorch 和 NCCL	959
58.6 镜像里要不要放模型权重	960
58.7 模型目录必须版本化	961
58.8 启动流程应该分阶段	961
58.9 Cold Start 为什么重要	962
58.10 Warmup 要做什么	962
58.11 Health Check 分层	963
58.11.1 Liveness	963
58.11.2 Readiness	963
58.11.3 Startup Probe	964
58.12 Readiness 不是简单返回 200	964
58.13 Worker 注册和摘除	965
58.14 Drain 的正确语义	965
58.15 滚动升级的基本流程	966
58.16 Max Surge 和 Max Unavailable	966
58.17 灰度发布是什么	967
58.18 灰度维度	967
58.19 灰度时要观察什么	968
58.20 回滚必须提前设计	968
58.21 配置发布也要灰度	968
58.22 模型加载失败怎么处理	969
58.23 GPU OOM 的生产处理	970
58.24 NCCL Hang 怎么办	970
58.25 日志、Metrics 和 Trace	971
58.26 生产告警	971
58.27 自动扩缩容的难点	972
58.28 多模型部署	972
58.29 Blue-Green 部署	973
58.30 Canary 部署	973
58.31 模型 Alias 和版本路由	974
58.32 部署前检查清单	974
58.33 一个最小生产部署配置	975
58.34 最小 Worker 状态机	975
58.35 常见错误	976
58.36 面试高频问题	976
58.37 标准回答模板	977
58.38 Production Deployment 公式、发布门禁和可运行 demo	977
58.39 小练习	983
58.40 本章总结	983

第 59 章 线上容量规划、成本估算、SLO 设计和故障演练	983
59.0 本讲资料边界与第二轮精修口径	984
59.1 本章目标	984
59.2 为什么不能只看 QPS	984
59.3 容量规划的核心单位	985
59.4 Prefill 和 Decode 的容量不同	986
59.5 从 Benchmark 到容量估算	987
59.6 安全余量	988
59.7 N+1 和故障容量	988
59.8 上下文长度分布	989
59.9 Workload 分层	989
59.10 KV Cache 容量估算	990
59.11 并发上限不是 QPS 上限	990
59.12 Admission Control	991

59.13 成本估算的基本模型	992
59.14 Token 成本	992
59.15 成本归因	993
59.16 SLO、SLA 和 SLI	993
59.17 LLM Serving 的关键 SLI	994
59.18 SLO 示例	994
59.19 Error Budget	995
59.20 告警设计	995
59.21 Dashboard 应该展示什么	996
59.22 压测策略	997
59.23 线上容量预测流程	997
59.24 自动扩缩容的难点	998
59.25 降级策略	998
59.26 故障演练为什么重要	999
59.27 常见故障演练场景	999
59.28 Runbook	1000
59.29 发布和容量的关系	1000
59.30 常见错误	1001
59.31 面试高频问题	1002
59.32 标准回答模板	1002
59.33 Capacity SLO Fault Drill 公式、容量门禁和可运行 demo	1002
59.34 小练习	1007
59.35 本章总结	1007

第 60 章 推理框架面试高频问题与标准回答

1008

60.0 本讲资料边界与第二轮精修口径	1008
60.1 本章目标	1009
60.2 面试回答的基本框架	1009
60.3 问题一：LLM Serving Engine 到底是什么	1009
60.4 问题二：一个请求的生命周期是什么	1010
60.5 问题三：Prefill 和 Decode 有什么区别	1010
60.6 问题四：KV Cache 解决什么问题	1010
60.7 问题五：PagedAttention 解决什么问题	1010
60.8 问题六：Continuous Batching 为什么重要	1011
60.9 问题七：Chunked Prefill 解决什么问题	1011
60.10 问题八：Prefix Cache 和 Prompt Cache 有什么价值	1011
60.11 问题九：vLLM 的核心贡献是什么	1012
60.12 问题十：SGLang 和 vLLM 有什么区别	1012
60.13 问题十一：PD 分离解决什么问题	1012
60.14 问题十二：TTFT 高怎么排查	1012
60.15 问题十三：TPOT 高怎么排查	1013
60.16 问题十四：吞吐上去了但用户觉得更卡，为什么	1013
60.17 问题十五：如何设计一个 vLLM-like 推理框架	1013
60.18 问题十六：Scheduler 应该怎么设计	1014
60.19 问题十七：KV Cache Manager 应该怎么设计	1014
60.20 问题十八：如何做推理框架压测	1014
60.21 问题十九：如何做容量规划	1015
60.22 问题二十：量化一定能降低延迟吗	1015
60.23 问题二十一：Speculative Decoding 为什么能加速	1015
60.24 问题二十二：Structured Generation 为什么影响 serving	1015
60.25 问题二十三：多 GPU serving 有哪些并行方式	1016
60.26 问题二十四：线上 OOM 怎么排查	1016
60.27 问题二十五：如何设计监控 Dashboard	1016
60.28 问题二十六：如何回答开放系统设计题	1016
60.29 高频追问清单	1017
60.30 面试中容易犯的错误	1017

60.31 最终 3 分钟回答模板	1018
60.32 Inference Engine Interview Readiness 公式、表达门禁和可运行 demo	1018
60.33 小练习	1022
60.34 本章总结	1022

第二十四册简介：大模型推理框架与 Serving Engine 实战

本书定位

第二十四册专门讲大模型推理框架、LLM Serving Engine 和从 0 实现推理系统。

第六册讲“大模型部署与推理工程”，重点是如何使用和优化推理服务；第二十三册讲“AI Infra”，重点是平台化承载、调度、治理和可观测性。本书则下沉到推理框架内部，系统讲解 vLLM、SGLang、PagedAttention、continuous batching、KV Cache 管理、scheduler、prefix cache、radix cache、speculative decoding、PD 分离和分布式 serving 架构。

本书也会把 nano-vLLM、tiny-llm、mini-sglang 这类教学项目作为源码学习路线，帮助读者从最小可运行推理框架逐步升级到生产级 serving engine。

为什么需要这本书

大模型推理已经不只是“调用 generate API”。真实线上系统要同时处理高并发、低延迟、高吞吐、显存约束、多模型路由、流式输出、长上下文、prefix 复用、KV Cache 管理、节点故障和成本控制。

vLLM、SGLang 等框架之所以重要，是因为它们把这些问题系统化为一个推理引擎：请求进入队列，scheduler 组织 batch，runtime 执行 prefill 和 decode，KV cache manager 管理显存块，streaming 组件返回 token，metrics 系统持续记录 TTFT、TPOT、吞吐和显存。

如果只会部署框架，而不理解框架内部，就很难回答高级面试问题，也很难在真实项目中定位性能瓶颈、设计 PD 分离、改造调度策略或评估新 serving 架构。

适合读者

1. 已理解 Transformer 推理、KV Cache 和基础部署，但想深入推理框架内部的读者。
2. 想准备 vLLM、SGLang、LLM serving、推理系统和系统设计面试的候选人。
3. 想从 0 写一个 mini serving engine，理解 scheduler、KV cache 和 batching 的工程师。
4. 想研究 PD 分离、disaggregated serving、prefix cache、speculative decoding 和分布式推理架构的人。

学完后应具备的能力

1. 能讲清 LLM serving engine 的核心模块和请求生命周期。
2. 能从 0 实现一个最小推理框架，包括 generate loop、KV Cache、batching、scheduler 和 streaming。
3. 能解释 vLLM 的 PagedAttention、block manager、continuous batching 和 engine 架构。
4. 能解释 SGLang 的 runtime、RadixAttention、prefix sharing、structured generation 和调度思路。
5. 能分析 PD 分离为什么出现、解决什么问题、带来什么新瓶颈。
6. 能设计 KV Cache manager、continuous batching scheduler、prefix cache 和推理性能压测方案。
7. 能把 nano-vLLM、tiny-llm、mini-sglang 这类教学项目和生产级框架建立对应关系。

推荐参考项目

1. nano-vLLM: <https://github.com/GeeekExplorer/nano-vllm>
2. tiny-llm: <https://github.com/skyzh/tiny-llm>
3. mini-sglang: <https://github.com/sgl-project/mini-sglang>

这些项目适合作为源码阅读和实战改造入口。阅读时重点关注：engine 如何组织请求，scheduler 如何形成 batch，KV cache 如何分配和释放，prefill/decode 如何执行，streaming 如何返回 token，以及教学实现和生产实现之间差了哪些工程模块。

内容结构

1. 推理框架总览。
2. 从 0 实现最小推理框架。
3. vLLM 架构详解。
4. SGLang 架构详解。
5. PD 分离与高级 serving 架构。
6. 教学项目源码路线与升级实战。
7. 推理框架系统设计与面试。

与其他书的关系

1. 第六册讲推理部署与优化总览，本书讲推理框架内部实现。
2. 第二十三册讲 AI Infra 平台层，本书讲单个或分布式 serving engine 的内核层。
3. 第十一册讲系统设计面试，本书提供推理框架类系统设计素材。
4. 第三册讲实战项目，本书可反向沉淀 mini KV Cache、mini scheduler、mini serving engine 等项目。
5. 第十九册讲真实工程坑，本书解释推理性能事故背后的框架级原因。

第二十四册：大模型推理框架与 Serving Engine 实战

本书定位

本书是“大模型算法岗系统面试二十四部曲”的第二十四册，专门讲大模型推理框架、LLM Serving Engine、vLLM/SGLang 架构和从 0 实现推理系统。

第六册偏“会部署、会调参、会优化”，第二十三册偏“平台承载、资源调度、可观测性和治理”。本书偏“懂引擎内部、能实现核心模块、能改造 serving 架构”。

本书重点回答：

1. LLM serving engine 到底由哪些模块组成。
2. 从 naive generate 到生产级推理框架，中间需要补哪些系统能力。
3. vLLM 为什么能提升吞吐，PagedAttention 和 continuous batching 如何工作。
4. SGLang 为什么强调 runtime、RadixAttention、prefix sharing 和 structured generation。
5. PD 分离为什么出现，prefill 和 decode 的资源画像有什么不同。
6. 如何从 nano-vLLM、tiny-llm、mini-sglang 这类教学项目学习推理框架。
7. 面试中如何设计一个 vLLM-like 或 SGLang-like 推理系统。

阅读入口

1. [简介.md](#)：本书定位、适合读者、阅读建议和与其他书的关系。
2. 本文件：全书章节目录和阅读顺序。

推荐参考项目

1. nano-vLLM: <https://github.com/GeeekExplorer/nano-vllm>
2. tiny-llm: <https://github.com/skyzh/tiny-llm>
3. mini-sglang: <https://github.com/sgl-project/mini-sglang>

章节目录

第一部分：推理框架总览

1. [LLM Serving Engine 到底是什么](#)
2. [从 naive generate 到推理框架](#)
3. [推理请求的生命周期](#)

4. [Prefill、Decode、KV Cache 和 Token Streaming](#)
5. [TTFT、TPOT、吞吐、并发、显存和成本](#)
6. [推理框架、推理平台和 AI Infra 的边界](#)

第二部分：从 0 实现最小推理框架

7. [实现最小 tokenizer、model wrapper 和 generate loop](#)
8. [实现 greedy、top-k、top-p 和 temperature sampling](#)
9. [实现 KV Cache 的最小版本](#)
10. [实现 batched prefill](#)
11. [实现 batched decode](#)
12. [实现请求队列和简单 scheduler](#)
13. [实现 token streaming](#)
14. [实现最小 HTTP API 服务](#)
15. [压测 TTFT、TPOT、吞吐和显存](#)

第三部分：vLLM 架构详解

16. [vLLM 解决什么问题](#)
17. [PagedAttention 的核心思想](#)
18. [KV Cache Block Manager](#)
19. [Continuous Batching 与 iteration-level scheduling](#)
20. [vLLM 请求调度流程](#)
21. [vLLM memory management](#)
22. [vLLM worker、executor 和 engine 架构](#)
23. [Prefix Caching 与 Prompt Cache](#)
24. [Tensor Parallel、Pipeline Parallel 与 serving 并行](#)
25. [vLLM 性能瓶颈、参数调优和工程坑](#)

第四部分：SGLang 架构详解

26. [SGLang 解决什么问题](#)
27. [SGLang Runtime 总览](#)
28. [RadixAttention 与 prefix sharing](#)
29. [SGLang scheduler 设计](#)
30. [Structured Generation 与 constrained decoding](#)
31. [Speculative Decoding 在 SGLang 中的角色](#)
32. [Multi-turn、tool use 和 agent serving 支持](#)
33. [SGLang 和 vLLM 的架构对比](#)
34. [mini-sglang 源码学习路径](#)

第五部分：PD 分离与高级 Serving 架构

35. [Prefill 和 Decode 的资源画像为什么不同](#)
36. [PD 分离解决什么问题](#)
37. [PD 分离的系统架构](#)
38. [KV Cache 迁移、共享和路由](#)
39. [Chunked Prefill 与 Disaggregated Prefill](#)
40. [多级 KV Cache：GPU、CPU 和远端缓存](#)
41. [跨节点 serving 和网络瓶颈](#)
42. [PD 分离的优缺点、适用场景和反模式](#)
43. [如何把单机 serving engine 升级成 PD 分离架构](#)

第六部分：教学项目源码路线与升级实战

44. [nano-vLLM 学习路线](#)

45. tiny-llm 学习路线
46. mini-sglang 学习路线
47. 从教学项目抽象推理框架核心模块
48. 从 naive scheduler 升级到 continuous batching
49. 从 list KV cache 升级到 paged KV cache
50. 从同步 generate 升级到异步 streaming engine
51. 从单模型服务升级到多模型服务
52. 从单机服务升级到分布式 serving

第七部分：系统设计、Debug 与面试

53. 设计一个 vLLM-like 推理框架
54. 设计一个 SGLang-like structured generation runtime
55. 设计一个支持 PD 分离的大模型 serving 系统
56. 设计 KV Cache Manager
57. 设计 Continuous Batching Scheduler
58. 设计 Prefix Cache 与 Radix Cache
59. [推理框架性能 debug 清单](#)
60. [推理框架面试高频问题与标准回答](#)

与其他书的关系

1. 第六册讲推理部署与优化总览，本书讲推理框架和 serving engine 内部实现。
2. 第二十三册讲 AI Infra、GPU 集群和推理平台治理，本书讲具体推理引擎架构。
3. 第十一册讲系统设计面试，本书提供推理框架类系统设计题素材。
4. 第三册讲项目实战，本书可沉淀从 0 实现 mini serving engine 的实战项目。
5. 第十九册讲真实工程坑，本书讲推理性能和 serving 事故背后的框架级原因。

第一批优先扩写章节

1. 第 1 章：LLM Serving Engine 到底是什么。
2. 第 4 章：Prefill、Decode、KV Cache 和 Token Streaming。
3. 第 9 章：实现 KV Cache 的最小版本。
4. 第 12 章：实现请求队列和简单 scheduler。
5. 第 17 章：PagedAttention 的核心思想。
6. 第 19 章：Continuous Batching 与 iteration-level scheduling。
7. 第 28 章：RadixAttention 与 prefix sharing。
8. 第 36 章：PD 分离解决什么问题。
9. 第 44 章：nano-vLLM 学习路线。
10. 第 55 章：设计一个支持 PD 分离的大模型 serving 系统。

第 1 章 LLM Serving Engine 到底是什么

第二十三册我们从 AI Infra 角度讲了训练平台、推理平台、调度、可观测性和治理。本册继续下钻到推理框架内部，专门讲 LLM Serving Engine。

很多人会用 `model.generate()`，也会启动 vLLM 或 SGLang 服务，但不一定真正理解 serving engine 内部发生了什么。面试和真实工程中，真正难的问题往往不是“怎么启动服务”，而是“为什么吞吐上不去”“为什么 TTFT 抖动”“为什么 KV cache 爆了”“scheduler 应该怎么设计”。

先记住一句话：

LLM Serving Engine 是把大模型从离线 generate 函数变成高并发、低延迟、可流式、可调度、可观测在线服务的执行引擎。

1.0 本讲资料边界与第二轮精修口径

本章是第二十四册的总览，不把某个框架版本的内部类名写成唯一标准答案。第二轮精修时，本章按稳定抽象来写：

1. vLLM 参考其公开架构文档中 API server、engine core、GPU worker、scheduler、KV cache manager、model execution 和 online serving 的边界；PagedAttention 只作为 KV cache block 管理思想，不把它写成新的 attention 数学公式。
2. SGLang 参考其论文和项目公开描述中的结构化生成、RadixAttention、prefix / KV 复用和复杂 generation program 执行边界；本章只讲 serving engine 总体能力，不提前展开 SGLang runtime 细节。
3. Hugging Face TGI 参考其文档中 streaming、continuous batching、Prometheus metrics、OpenTelemetry tracing 和 tensor parallelism 的公开口径；本章不把 TGI 作为唯一推荐 runtime。
4. NVIDIA Triton 参考其 model repository、per-model scheduler、batcher、backend、metrics、tracing 和 model management API 的通用 serving 抽象；本章只用它帮助区分“通用模型服务框架”和“LLM 专用 serving engine”。
5. 所有指标统一回到请求生命周期、prefill / decode、KV cache、scheduler、streaming、SLO、成本和可观测性，不用单一 QPS 或单一 GPU utilization 判断推理框架好坏。

所以，本章的重点不是背 vLLM、SGLang、TGI、Triton 的名字，而是能说清一个 LLM serving engine 到底要管理哪些状态、暴露哪些指标、用哪些门禁证明它可上线。

1.1 从 model.generate 说起

最简单的大模型推理代码可能是：

```
inputs = tokenizer(prompt, return_tensors="pt").to(device)
outputs = model.generate(**inputs, max_new_tokens=128)
text = tokenizer.decode(outputs[0])
```

这段代码适合：

1. 本地实验。
2. Notebook 演示。
3. 单用户测试。
4. 简单 benchmark。

但它不适合直接支撑生产服务。

原因是生产服务要处理很多额外问题：

1. 多用户并发请求。
2. 请求排队。
3. 动态 batching。
4. KV cache 管理。
5. streaming 输出。
6. 超时和取消。
7. 指标统计。
8. 资源隔离。
9. 故障处理。
10. 多模型和多版本。

Serving engine 就是为了解决这些问题。

1.2 LLM Serving Engine 的定义

LLM Serving Engine 可以理解为一个运行时系统。

它接收用户请求，组织请求进入 batch，调用模型执行 prefill 和 decode，管理 KV cache，持续返回 token，并记录指标和状态。

它通常包含：

1. API Server。
2. Request Queue。
3. Scheduler。
4. Tokenizer。
5. Model Executor。
6. KV Cache Manager。
7. Sampler。
8. Streaming Manager。
9. Metrics Exporter。
10. Worker / Executor Runtime。

从外部看，它是一个模型服务。

从内部看，它是一个高性能并发执行系统。

1.3 Serving Engine 和推理平台的区别

推理平台是更上层的平台系统。

它负责：

1. API 网关。
2. 鉴权。
3. 模型路由。
4. 限流。
5. 灰度发布。
6. 成本归因。
7. 多租户治理。
8. 自动扩缩容。

Serving engine 负责更底层的执行：

1. 请求调度。
2. prefill。
3. decode。
4. KV cache。
5. batching。
6. token streaming。
7. GPU 执行。

简单说：推理平台管“服务治理”，serving engine 管“模型执行”。

1.4 一次请求在 Engine 内部的生命周期

一个请求进入 engine 后，大致流程是：

HTTP Request

- > Parse Request
- > Tokenize Prompt
- > Enter Waiting Queue
- > Scheduler Selects Request
- > Prefill
- > Allocate KV Cache
- > Decode Loop
- > Sample Next Token
- > Stream Token
- > Finish / Cancel / Timeout
- > Free KV Cache

这条链路就是本书后续所有章节的主线。

理解它，就能理解 vLLM、SGLang、PD 分离和各种性能优化。

1.5 Prefill 和 Decode 是核心阶段

LLM 推理不是一次性算完，而是分为 prefill 和 decode。

Prefill 阶段：

1. 输入完整 prompt。
2. 处理所有输入 token。
3. 生成第一步所需状态。
4. 构建 KV cache。
5. 主要影响 TTFT。

Decode 阶段：

1. 每次生成一个或少量 token。
2. 重复读取 KV cache。
3. 持续执行 sampling。
4. 逐步 streaming 输出。
5. 主要影响 TPOT。

Serving engine 的调度、缓存和性能优化，基本都围绕这两个阶段展开。

1.6 KV Cache 为什么是核心

KV cache 保存历史 token 在 attention 中的 Key 和 Value。

它的作用是避免每次 decode 都重新计算全部历史上下文。

没有 KV cache，生成第 1000 个 token 时还要重新处理前 999 个 token，成本会非常高。

但 KV cache 也会占用大量显存。

它和这些因素相关：

1. batch size。
2. sequence length。
3. layer 数。
4. hidden size。
5. dtype。
6. 并发请求数。

Serving engine 的一个核心工作就是管理 KV cache。

vLLM 的 PagedAttention，本质上就是对 KV cache 管理方式的一次重要工程改造。

1.7 为什么需要 Scheduler

如果一次只服务一个请求，GPU 利用率很低。

如果把所有请求都塞进一个 batch，又会导致长请求拖慢短请求，显存爆掉，p99 变差。

Scheduler 要决定：

1. 哪些请求进入 prefill。
2. 哪些请求进入 decode。
3. 每轮 batch 放多少 token。
4. 谁先执行。
5. 谁等待。
6. 谁被取消或拒绝。

7. KV cache 不够时怎么处理。

LLM serving scheduler 不是普通 Web 请求队列。

它要理解 token、KV cache、prefill、decode、优先级和 streaming。

1.8 为什么需要 Continuous Batching

普通 batching 是固定一批请求一起跑到结束。

但大模型生成中，每个请求输出长度不同。

有的请求 20 个 token 结束，有的请求 2000 个 token 才结束。

如果固定 batch，短请求会被长请求拖住。

Continuous batching 的思想是：每个 decode step 都动态更新 batch。

完成的请求退出，新请求加入。

这样可以提高 GPU 利用率，也更适合 streaming 服务。

1.9 Streaming 为什么重要

大模型生成可能需要几秒甚至几十秒。

如果等全部输出生成完再返回，用户体验很差。

Streaming 让模型生成一个 token 或一小段 token 就返回给用户。

Serving engine 要处理：

1. token 增量返回。
2. 客户端断开。
3. 请求取消。
4. backpressure。
5. finish reason。
6. 多请求输出顺序。

Streaming 不是简单 `print(token)`，而是 runtime、网络 and 状态管理共同作用的能力。

1.10 Engine 需要暴露哪些指标

一个生产级 engine 至少要暴露：

1. request count。
2. active requests。
3. waiting requests。
4. TTFT。
5. TPOT。
6. input tokens/s。
7. output tokens/s。
8. queue latency。
9. prefill latency。
10. decode latency。
11. KV cache usage。
12. GPU memory usage。
13. batch size。
14. error rate。
15. timeout count。

没有这些指标，就很难回答“为什么变慢”。

1.11 vLLM、SGLang、TGI、Triton 的位置

这些系统都和 serving engine 有关，但侧重点不同。

vLLM：强调高吞吐 LLM serving，核心是 PagedAttention、KV cache block 管理和 continuous batching。

SGLang：强调结构化 LLM 程序执行、RadixAttention、prefix sharing、structured generation 和复杂 serving 场景。

TGI：Hugging Face 生态中的 text generation inference 服务。

Triton：通用推理服务框架，可结合 TensorRT-LLM 等后端服务模型。

本书会重点讲 vLLM 和 SGLang 的内部思想，也会从最小实现角度讲 serving engine 的通用结构。

1.12 从 0 实现推理框架要实现什么

一个最小推理框架可以按阶段实现：

1. 单请求 generate loop。
2. sampling。
3. KV cache。
4. batched prefill。
5. batched decode。
6. 请求队列。
7. scheduler。
8. token streaming。
9. HTTP API。
10. 基础 metrics。

这也是本书第二部分的主线。

先实现 naive 版本，再逐步升级到 vLLM-like 或 SGLang-like 的设计。

1.13 推理框架核心指标和最小 demo

推理框架总览不能只停留在模块图上。面试中更稳的方式，是把 serving engine 抽象成一组请求样本、阶段耗时、显存预算和门禁。

可以把一个请求样本记为：

$R_i=(a_i,p_i,q_i,d_i,k_i,s_i,o_i,m_i,z_i)$

其中 a 是到达时间， p 是 prompt token 数， q 是排队状态， d 是 decode 输出 token 数， k 是 KV cache 占用， s 是 streaming 状态， o 是观测指标， m 是模型和 runtime 配置， z 是请求最终状态。

端到端延迟可以拆成：

$T_{\{\mathrm{e2e}\}}=T_{\{\mathrm{parse}\}}+T_{\{\mathrm{tokenize}\}}+T_{\{\mathrm{queue}\}}+T_{\{\mathrm{prefill}\}}+T_{\{\mathrm{first}\}}+$

TTFT 主要覆盖首 token 前的路径：

$T_{\{\mathrm{ttft}\}}=T_{\{\mathrm{parse}\}}+T_{\{\mathrm{tokenize}\}}+T_{\{\mathrm{queue}\}}+T_{\{\mathrm{prefill}\}}+T_{\{\mathrm{first}\}}+$

TPOT 衡量后续输出 token 的平均间隔：

$T_{\{\mathrm{tpot}\}}=\frac{T_{\{\mathrm{decode}\}}}{\max(N_{\{\mathrm{out}\}}-1,1)}$

其中 N_{out} 是输出 token 数。注意这里把第一个 token 归入 TTFT，后续 token 才进入 TPOT。

KV cache 的粗略显存估算可以写成：

$M_{\{\mathrm{kv}\}}=2\mathrm{LBSH}_{\{\mathrm{kv}\}}D_{\mathrm{hb}}$

其中 L 是层数，B 是并发序列数，S 是平均上下文 token 数，H_kv 是 KV head 数，D_h 是 head dimension，b 是每个元素字节数，前面的 2 来自 Key 和 Value 两份缓存。

KV pressure 用来判断显存是否先于算力成为瓶颈：

$$P_{\{\mathrm{kv}\}} = \frac{M_{\{\mathrm{kv}\}}}{M_{\{\mathrm{budget}\}}}$$

token 吞吐和单位成本也要按 token 口径计算：

$$X_{\{\mathrm{in}\}} = \frac{\sum_i N_{\{\mathrm{in}\},i}}{T_{\{\mathrm{window}\}}}, \quad$$

$$X_{\{\mathrm{out}\}} = \frac{\sum_i N_{\{\mathrm{out}\},i}}{T_{\{\mathrm{window}\}}}$$

$$K_{\{\mathrm{1k}\}} = \frac{1000K_{\{\mathrm{total}\}}}{\sum_i (N_{\{\mathrm{in}\},i} + N_{\{\mathrm{out}\},i})}$$

最终可以用一个总门禁收束：

```
G_{\mathrm{engine}} = \mathbf{1} \setminus \left[
G_{\mathrm{lifecycle}} \setminus \land
G_{\mathrm{phase}} \setminus \land
G_{\mathrm{kv}} \setminus \land
G_{\mathrm{scheduler}} \setminus \land
G_{\mathrm{stream}} \setminus \land
G_{\mathrm{metrics}} \setminus \land
G_{\mathrm{runtime}} \setminus \land
G_{\mathrm{cost}} \setminus \land
P_0 = 0
\right]
```

下面这个 0 依赖 demo 演示如何把“推理框架总览”变成一个可运行审计。它不模拟真实 transformer 计算，只审计 serving engine 是否把请求生命周期、prefill / decode、KV cache、scheduler、streaming、metrics、runtime 边界和成本容量讲清楚。

```
from copy import deepcopy
```

```
class MiniServingEngineOverviewAudit:
```

```
    def __init__(self):
```

```
        self.gates = [
```

```
            "request_lifecycle_coverage",
            "prefill_decode_contract",
            "kv_cache_formula_fit",
            "scheduler_batching_readiness",
            "streaming_state_management",
            "metrics_observability",
            "runtime_boundary_clarity",
            "cost_capacity_model",
            "serving_engine_gate",
        ]
```

```
    ]
```

```
        self.actions = {
```

```
            "request_lifecycle_coverage": "trace parse, tokenize, queue, prefill, decode, stream, finish, and cleanup",
            "prefill_decode_contract": "separate TTFT-critical prefill from TPOT-critical decode accounting",
            "kv_cache_formula_fit": "estimate KV bytes from layers, active sequences, context, KV heads, head dim, and dim",
            "scheduler_batching_readiness": "define token budget, continuous batching, admission, and cancellation cl",
            "streaming_state_management": "handle client disconnect, backpressure, timeout, finish reason, and partial",
            "metrics_observability": "export queue, prefill, decode, TTFT, TPOT, tokens/s, KV pressure, errors, and t",
            "runtime_boundary_clarity": "separate API server, scheduler, executor, KV manager, sampler, streamer, and",
            "cost_capacity_model": "compute per-1k-token cost and capacity from token throughput and KV budget",
            "serving_engine_gate": "block launch when any lifecycle, phase, KV, scheduler, stream, metric, runtime, c",
        }
```

```
    }
```

```

def percentile(self, values, pct):
    values = sorted(values)
    if not values:
        return 0.0
    index = int(round((len(values) - 1) * pct))
    return values[index]

def request_times(self, request):
    ttft = (
        request["parse_ms"]
        + request["tokenize_ms"]
        + request["queue_ms"]
        + request["prefill_ms"]
        + request["first_token_ms"]
    )
    decode_tokens = max(request["output_tokens"] - 1, 1)
    tpot = request["decode_ms"] / decode_tokens
    e2e = ttft + request["decode_ms"] + request["stream_flush_ms"]
    return {"ttft_ms": ttft, "tpot_ms": tpot, "e2e_ms": e2e}

def kv_cache_mib(self, case):
    cfg = case["model_config"]
    active = max(request["active_sequences"] for request in case["requests"])
    avg_context = sum(
        request["prompt_tokens"] + request["output_tokens"]
        for request in case["requests"]
    ) / len(case["requests"])
    bytes_total = (
        2
        * cfg["layers"]
        * active
        * avg_context
        * cfg["kv_heads"]
        * cfg["head_dim"]
        * cfg["dtype_bytes"]
    )
    return round(bytes_total / (1024 * 1024), 2)

def example_metrics(self, case):
    times = [self.request_times(request) for request in case["requests"]]
    total_input = sum(request["prompt_tokens"] for request in case["requests"])
    total_output = sum(request["output_tokens"] for request in case["requests"])
    window_s = case["traffic"]["window_s"]
    kv_mib = self.kv_cache_mib(case)
    total_cost = case["cost"]["gpu_cost"] + case["cost"]["cpu_cost"] + case["cost"]["runtime_overhead"]
    return {
        "ttft_p95_ms": round(self.percentile([item["ttft_ms"] for item in times], 0.95), 2),
        "tpot_p95_ms": round(self.percentile([item["tpot_ms"] for item in times], 0.95), 2),
        "e2e_p99_ms": round(self.percentile([item["e2e_ms"] for item in times], 0.99), 2),
        "input_tokens_per_s": round(total_input / window_s, 2),
        "output_tokens_per_s": round(total_output / window_s, 2),
        "kv_cache_mib": kv_mib,
        "kv_pressure": round(kv_mib / case["model_config"]["kv_budget_mib"], 3),
        "cost_per_1k_tokens": round(1000 * total_cost / (total_input + total_output), 4),
        "continuous_batching_gain": round(

```

```

        case["scheduler"]["static_batch_time_ms"]
        / case["scheduler"]["continuous_batch_time_ms"],
        2,
    ),
}

def gate_request_lifecycle_coverage(self, case):
    required = {
        "parse",
        "tokenize",
        "queue",
        "prefill",
        "decode",
        "stream",
        "finish",
        "cleanup",
    }
    return required.issubset(set(case["lifecycle"]))

def gate_prefill_decode_contract(self, case):
    phase = case["phase_contract"]
    return (
        phase["prefill_metric"]
        and phase["decode_metric"]
        and phase["tftt_budget_ms"] >= self.example_metrics(case)["tftt_p95_ms"]
        and phase["tpot_budget_ms"] >= self.example_metrics(case)["tpot_p95_ms"]
    )

def gate_kv_cache_formula_fit(self, case):
    cfg = case["model_config"]
    metrics = self.example_metrics(case)
    return (
        cfg["layers"] > 0
        and cfg["kv_heads"] > 0
        and cfg["head_dim"] > 0
        and cfg["dtype_bytes"] in {1, 2, 4}
        and metrics["kv_pressure"] <= cfg["max_kv_pressure"]
        and case["kv_manager"]["admission_control"]
        and case["kv_manager"]["release_on_finish"]
    )

def gate_scheduler_batching_readiness(self, case):
    scheduler = case["scheduler"]
    return (
        scheduler["continuous_batching"]
        and scheduler["token_budget"]
        and scheduler["max_active_sequences"]
        and scheduler["prefill_decode_policy"]
        and scheduler["cancel_cleanup"]
        and scheduler["continuous_batch_time_ms"] < scheduler["static_batch_time_ms"]
    )

def gate_streaming_state_management(self, case):
    stream = case["streaming"]
    return (

```

```

        stream["sse_or_chunked"]
        and stream["client_disconnect"]
        and stream["backpressure"]
        and stream["finish_reason"]
        and stream["partial_output_trace"]
        and stream["timeout_cleanup"]
    )

def gate_metrics_observability(self, case):
    required = {
        "request_count",
        "waiting_requests",
        "active_requests",
        "queue_ms",
        "prefill_ms",
        "decode_ms",
        "ttft_ms",
        "tpot_ms",
        "input_tokens_per_s",
        "output_tokens_per_s",
        "kv_pressure",
        "gpu_memory",
        "error_rate",
        "timeout_count",
    }
    return required.issubset(set(case["metrics"]))

def gate_runtime_boundary_clarity(self, case):
    modules = case["runtime_modules"]
    required = {
        "api_server",
        "request_queue",
        "scheduler",
        "tokenizer",
        "model_executor",
        "kv_cache_manager",
        "sampler",
        "streaming_manager",
        "metrics_exporter",
        "worker_runtime",
    }
    return required.issubset(set(modules)) and case["platform_boundary_clear"]

def gate_cost_capacity_model(self, case):
    metrics = self.example_metrics(case)
    cost = case["cost"]
    return (
        metrics["cost_per_1k_tokens"] <= cost["budget_per_1k_tokens"]
        and metrics["output_tokens_per_s"] >= case["traffic"]["min_output_tokens_per_s"]
        and cost["cost_attribution"]
        and cost["capacity_plan"]
    )

def gate_serving_engine_gate(self, case):
    return case["final_gate"] and not case["p0_risk"]

```

```

def audit_case(self, case):
    failed = []
    for gate in self.gates:
        if not getattr(self, "gate_" + gate)(case):
            failed.append(gate)
    return failed

def audit(self, cases):
    failed_cases = {}
    hard_blockers = []
    for case in cases:
        failed = self.audit_case(case)
        if failed:
            failed_cases[case["name"]] = failed
            hard_blockers.extend((case["name"], gate) for gate in failed)

    metrics = {}
    for gate in self.gates:
        passed = sum(1 for case in cases if gate not in self.audit_case(case))
        metrics[gate] = round(passed / len(cases), 3)

    remediation_sample = {
        name: [self.actions[gate] for gate in gates]
        for name, gates in list(failed_cases.items())[:3]
    }
    failed_gate_names = sorted({gate for gates in failed_cases.values() for gate in gates})
    return {
        "metrics": metrics,
        "hard_blockers": hard_blockers,
        "failed_cases": failed_cases,
        "failed_gate_names": failed_gate_names,
        "remediation_sample": remediation_sample,
        "serving_engine_gate_pass": not failed_cases,
    }

def build_cases():
    complete = {
        "name": "complete_serving_engine",
        "lifecycle": ["parse", "tokenize", "queue", "prefill", "decode", "stream", "finish", "cleanup"],
        "requests": [
            {
                "prompt_tokens": 420,
                "output_tokens": 128,
                "active_sequences": 18,
                "parse_ms": 2,
                "tokenize_ms": 9,
                "queue_ms": 35,
                "prefill_ms": 180,
                "first_token_ms": 20,
                "decode_ms": 1905,
                "stream_flush_ms": 15,
            },
            {

```

```

        "prompt_tokens": 1600,
        "output_tokens": 64,
        "active_sequences": 18,
        "parse_ms": 2,
        "tokenize_ms": 22,
        "queue_ms": 55,
        "prefill_ms": 360,
        "first_token_ms": 24,
        "decode_ms": 882,
        "stream_flush_ms": 14,
    },
    {
        "prompt_tokens": 220,
        "output_tokens": 32,
        "active_sequences": 18,
        "parse_ms": 1,
        "tokenize_ms": 5,
        "queue_ms": 28,
        "prefill_ms": 96,
        "first_token_ms": 18,
        "decode_ms": 403,
        "stream_flush_ms": 9,
    },
],
"phase_contract": {
    "prefill_metric": True,
    "decode_metric": True,
    "ttft_budget_ms": 520,
    "tpot_budget_ms": 22,
},
"model_config": {
    "layers": 32,
    "kv_heads": 8,
    "head_dim": 128,
    "dtype_bytes": 2,
    "kv_budget_mib": 24576,
    "max_kv_pressure": 0.70,
},
"kv_manager": {"admission_control": True, "release_on_finish": True},
"scheduler": {
    "continuous_batching": True,
    "token_budget": True,
    "max_active_sequences": True,
    "prefill_decode_policy": True,
    "cancel_cleanup": True,
    "static_batch_time_ms": 5200,
    "continuous_batch_time_ms": 3300,
},
"streaming": {
    "sse_or_chunked": True,
    "client_disconnect": True,
    "backpressure": True,
    "finish_reason": True,
    "partial_output_trace": True,
    "timeout_cleanup": True,

```



```

},
"metrics": [
    "request_count",
    "waiting_requests",
    "active_requests",
    "queue_ms",
    "prefill_ms",
    "decode_ms",
    "ttft_ms",
    "tpot_ms",
    "input_tokens_per_s",
    "output_tokens_per_s",
    "kv_pressure",
    "gpu_memory",
    "error_rate",
    "timeout_count",
],
"runtime_modules": [
    "api_server",
    "request_queue",
    "scheduler",
    "tokenizer",
    "model_executor",
    "kv_cache_manager",
    "sampler",
    "streaming_manager",
    "metrics_exporter",
    "worker_runtime",
],
"platform_boundary_clear": True,
"traffic": {"window_s": 2.0, "min_output_tokens_per_s": 100.0},
"cost": {
    "gpu_cost": 0.008,
    "cpu_cost": 0.001,
    "runtime_overhead": 0.001,
    "budget_per_1k_tokens": 0.006,
    "cost_attribution": True,
    "capacity_plan": True,
},
"final_gate": True,
"p0_risk": False,
}

cases = [complete]
edits = [
    ("lifecycle_missing_bad", lambda c: c["lifecycle"].remove("cleanup")),
    ("prefill_decode_contract_bad", lambda c: c["phase_contract"].update({"decode_metric": False})),
    ("kv_cache_formula_missing_bad", lambda c: c["kv_manager"].update({"admission_control": False})),
    ("scheduler_static_batch_bad", lambda c: c["scheduler"].update({"continuous_batching": False})),
    ("streaming_state_missing_bad", lambda c: c["streaming"].update({"backpressure": False})),
    ("metrics_observability_missing_bad", lambda c: c["metrics"].remove("tpot_ms")),
    ("runtime_boundary_unclear_bad", lambda c: c.update({"platform_boundary_clear": False})),
    ("cost_capacity_missing_bad", lambda c: c["cost"].update({"cost_attribution": False})),
    ("serving_engine_gate_missing_bad", lambda c: c.update({"final_gate": False})),
]

```

```

for name, edit in edits:
    bad = deepcopy(complete)
    bad["name"] = name
    edit(bad)
    cases.append(bad)
return cases

```

```

audit = MiniServingEngineOverviewAudit()
cases = build_cases()
serving_engine_examples = audit.example_metrics(cases[0])
report = audit.audit(cases)

print("serving_engine_examples=", serving_engine_examples)
print("metrics=", report["metrics"])
print("hard_blocker_count=", len(report["hard_blockers"]))
print("failed_case_count=", len(report["failed_cases"]))
print("failed_gate_count=", len(report["failed_gate_names"]))
print("remediation_sample=", report["remediation_sample"])
print("serving_engine_gate_pass=", report["serving_engine_gate_pass"])

```

这段 demo 的输出有五个重点：

1. `serving_engine_examples` 把一次 engine 总览转成 TTFT、TPOT、E2E、token 吞吐、KV pressure、单位 token 成本和 continuous batching 收益。
2. `metrics` 检查每类核心能力在 toy cases 中的覆盖率。
3. `hard_blocker_count`、`failed_case_count` 和 `failed_gate_count` 告诉你哪些模块缺失会阻断上线。
4. `remediation_sample` 把 bad case 绑定到具体修复动作。
5. `serving_engine_gate_pass=False` 说明推理框架不是只要能生成文本就合格；只要请求生命周期、prefill / decode、KV、scheduler、streaming、metrics、runtime 边界或成本容量任一关键门禁缺失，就不能说它是完整的 serving engine。

1.14 常见误区

误区一：推理框架就是封装 `model.generate()`。

真实 serving engine 要管理并发、batch、KV cache、scheduler、streaming 和 metrics。

误区二：吞吐只取决于模型大小。

吞吐还取决于 batch、KV cache、输入输出长度、scheduler、runtime kernel 和硬件。

误区三：QPS 是最重要指标。

LLM serving 更应该看 TTFT、TPOT、tokens/s、KV cache 和成本。

误区四：KV cache 只是优化细节。

KV cache 是 serving engine 的核心状态，直接决定并发能力和显存压力。

误区五：vLLM 快只是因为用了更快 kernel。

vLLM 的关键价值之一是 KV cache block 管理和调度方式，而不只是 kernel。

1.15 面试常见追问

问题一：LLM Serving Engine 和普通模型服务有什么区别？

可以回答：普通模型服务通常一次请求一次前向，输出固定；LLM serving 是自回归生成，有 prefill、decode、KV cache、streaming、continuous batching 和 token 级调度，资源消耗与输入输出 token 强相关。

问题二：为什么 naive generate 不能支撑生产？

可以回答：它缺少并发请求管理、动态 batching、KV cache 管理、流式返回、取消超时、指标统计和资源调度，GPU 利用率和线上稳定性都难以保证。

问题三：Serving engine 的核心模块有哪些？

可以回答：API server、request queue、scheduler、tokenizer、model executor、KV cache manager、sampler、streaming manager、metrics exporter 和 worker runtime。

问题四：为什么 scheduler 是推理框架核心？

可以回答：scheduler 决定请求何时 prefill、何时 decode、如何组成 batch、如何控制 token budget、如何处理 KV cache 和优先级，直接影响 TTFT、TPOT、吞吐和显存。

1.16 小练习

1. 用自己的话解释 LLM Serving Engine。
2. `model.generate()` 和 `production serving engine` 差在哪里？
3. Prefill 和 decode 在请求生命周期中分别做什么？
4. KV cache 为什么会占显存？
5. Scheduler 要做哪些决策？
6. Continuous batching 为什么适合生成式模型？
7. Streaming 需要处理哪些状态问题？
8. 一个最小推理框架应该先实现哪些模块？

1.17 本章小结

本章回答了 LLM Serving Engine 到底是什么。

你需要记住：

1. LLM Serving Engine 是高并发大模型在线执行引擎，不是简单 `generate()` 封装。
2. 请求生命周期包括解析、tokenize、排队、调度、prefill、decode、streaming 和释放 KV cache。
3. Prefill 影响 TTFT，decode 影响 TPOT，KV cache 影响显存和并发。
4. Scheduler、continuous batching 和 KV cache manager 是 serving engine 的核心模块。
5. vLLM、SGLang 等框架的许多设计，本质上都是围绕请求调度和 KV cache 管理展开。

下一章我们会继续讲：从 naive generate 到推理框架，中间到底补了哪些系统能力。

第 2 章 从 naive generate 到推理框架

上一章讲了 LLM Serving Engine 的边界。本章从最简单的 `model.generate()` 出发，解释它为什么不能直接承担生产服务，以及一个推理框架是怎样一步步长出来的。

很多人第一次接触推理框架，会直接被 vLLM、SGLang 里的 scheduler、block manager、executor、prefix cache、continuous batching 等概念淹没。更稳的学习路径是先问一个朴素问题：如果我已经会写 `model.generate()`，为什么还需要 serving engine？

一句话概括：

推理框架不是为了把 `generate()` 包一层 HTTP 接口，而是为了在高并发、长上下文、流式输出、显存受限和稳定性约束下，持续调度 token 级别的模型执行。

2.0 本讲资料边界与第二轮精修口径

本章是“从 0 实现”的教学章节，不依赖真实 GPU、真实模型权重或外部服务。第二轮精修时，本章按下面口径处理：

1. `generate()` 和 KV cache 的基础概念参考 Hugging Face Transformers 的 generation、cache strategies 和 LLM inference optimization 文档，但本章不用 Transformers API 直接遮蔽内部机制。

2. request queue、scheduler、worker、KV cache manager、streaming 和 metrics 的模块边界参考 vLLM、SGLang、TGI 和 Triton 的公开 serving 抽象，但本章不复刻任何框架源码。
3. demo 只实现 toy tokenizer、toy next-token model、request state、prefill / decode 计数、简单 scheduler、streaming chunk 和指标汇总，用来讲清最小机制。
4. 本章不讨论 CUDA kernel、PagedAttention 物理 block、tensor parallel、PD 分离和真实 HTTP 服务；这些会在后续章节逐步展开。
5. 所有公式只服务于理解 naive generate 如何演进成 token 级调度系统，不把 toy 指标当作真实线上 benchmark。

所以，本章的目标不是写一个“高性能推理框架”，而是写一个最小可运行骨架，让你能看见推理框架为什么需要 tokenizer、sampler、request state、scheduler、KV cache manager、streamer 和 metrics。

2.1 最简单的 generate

最小推理代码大概是这样：

```
def generate(prompt, max_new_tokens=128):
    inputs = tokenizer(prompt, return_tensors="pt").to(device)
    outputs = model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
    )
    return tokenizer.decode(outputs[0], skip_special_tokens=True)
```

这段代码完成了三件事：

1. 把文本 prompt 编码成 token。
2. 调用模型生成后续 token。
3. 把输出 token 解码成文本。

如果只是本地实验、离线 demo、低频内部脚本，这已经够用。但它的系统抽象非常粗：一次调用进去，内部循环跑完，最后一次性返回结果。

生产 serving 不能只看“能不能生成”，还要看：

1. 多个请求同时进来怎么办。
2. 用户希望边生成边返回怎么办。
3. 某个请求特别长，占满显存怎么办。
4. 请求中途取消怎么办。
5. 不同请求的生成长度不同，怎么组成 batch。
6. 如何统计 TTFT、TPOT、tokens/s 和错误率。
7. GPU 空着时为什么吞吐仍然上不去。

这些问题不是简单给 generate() 加一个 Web 框架就能解决的。

2.2 generate 背后的真实循环

大模型自回归生成不是一次性吐出完整答案，而是按 token 逐步生成。简化后可以写成：

```
tokens = tokenizer(prompt)

for step in range(max_new_tokens):
    logits, kv_cache = model.forward(tokens, kv_cache)
    next_token = sample(logits[-1])
    tokens.append(next_token)
    if next_token == eos_token_id:
        break

return tokenizer.decode(tokens)
```

这里已经出现了几个核心概念：

1. `tokens`：当前请求的输入和已生成 token。
2. `model.forward`：一次模型前向计算。
3. `kv_cache`：历史 token 的 key/value 缓存。
4. `sample`：从 logits 里选择下一个 token。
5. `eos_token_id`：停止条件。

推理框架要做的事情，本质上就是把这个单请求循环变成多请求、可调度、可流式、可观测、可中断的系统循环。

2.3 Prefill 和 Decode 的分离

在真实推理里，一个请求通常分成两个阶段：

1. Prefill：处理 prompt 中已有的全部 token，建立初始 KV Cache。
2. Decode：每次基于已有 KV Cache 生成一个新 token。

naive generate 往往把这两个阶段藏在同一个接口里。serving engine 必须把它们拆出来，因为两者的资源画像不同。

Prefill 的特点是：

1. 输入 token 多。
2. 单次计算量大。
3. 更像大矩阵批量计算。
4. 对 TTFT 影响很大。

Decode 的特点是：

1. 每轮每个请求通常只生成一个 token。
2. 单轮计算较小，但会重复很多轮。
3. 更容易受内存带宽、KV Cache 读取和调度开销影响。
4. 对 TPOT 和持续吞吐影响很大。

如果不区分 prefill 和 decode，就很难解释为什么长 prompt 会拖慢首 token，为什么短请求和长请求混在一起会互相影响，也很难设计 PD 分离这类高级架构。

2.4 单请求到多请求

最小 `generate()` 只处理一个请求。生产服务面对的是请求流：

```
request_1: prompt 短, 生成短
request_2: prompt 长, 生成长
request_3: prompt 短, 生成很长
request_4: prompt 中等, 用户中途取消
```

如果每个请求独占一次模型执行，GPU 利用率会很低。模型权重已经在 GPU 上，最好的方式是把多个请求组成 batch，一次 forward 同时处理。

但 LLM 生成的 batch 和传统分类模型不同。分类模型通常是固定输入、一次 forward、一次返回。LLM decode 是动态过程：有的请求很快结束，有的请求继续生成，有的新请求正在等待加入。

因此推理框架需要一个请求队列：

<code>waiting queue</code>	等待进入模型执行的请求
<code>running set</code>	正在生成的请求
<code>finished set</code>	已完成或已取消的请求

还需要一个 scheduler 决定：

1. 本轮哪些 waiting 请求可以进入。
2. 本轮哪些 running 请求继续 decode。
3. 显存不够时优先保留谁。

4. 长 prompt 是否要拆成 chunk。
5. 新请求是否可以插队以降低 TTFT。

这就是从函数调用到 serving engine 的第一道分水岭：推理不再是“调用一次函数”，而是“调度一批请求的生命周期”。

2.5 为什么不能只做静态 batching

最容易想到的优化是静态 batching：凑够 N 个请求，一起调用 `generate()`。

静态 batching 的问题是等待和浪费。

等待体现在：如果系统必须等够 batch size 才执行，低流量时首 token 延迟会变差。

浪费体现在：一个 batch 里的请求生成长度不同。短请求已经结束，但为了对齐长请求，框架可能仍然保留 padding 或无效位置，计算和显存都被浪费。

LLM serving 更常用的是 continuous batching，也叫动态批处理或 iteration-level batching。它的核心思想是：

1. 每一轮 decode 都重新决定 batch 里有哪些请求。
2. 已完成请求立刻移出。
3. 新请求在合适时机加入。
4. batch 是随时间滚动变化的。

这会把执行方式从下面这样：

batch A 开始 -> batch A 全部结束 -> batch B 开始

变成下面这样：

```
iteration 1: request 1, request 2
iteration 2: request 1, request 2, request 3
iteration 3: request 2, request 3, request 4
iteration 4: request 3, request 4
```

这就是现代 LLM serving engine 的核心能力之一。

2.6 KV Cache Manager 为什么必要

KV Cache 是推理性能的关键。没有 KV Cache，每生成一个新 token 都要重新计算整个历史上下文，成本会随着序列长度快速上升。

有了 KV Cache，decode 阶段可以复用历史 token 的 key/value，只计算新 token 的部分。

但在多请求场景下，KV Cache 不只是一个张量，而是一套内存管理问题：

1. 每个请求需要多少 cache。
2. cache 放在 GPU 显存的哪里。
3. 请求结束后如何释放。
4. 请求被抢占后如何保存或重算。
5. 长上下文请求是否会造成碎片。
6. 多个请求能否复用相同 prefix 的 cache。

naive generate 可以让框架内部自动维护 KV Cache。serving engine 必须自己理解和管理它，因为 KV Cache 往往决定最大并发、显存占用和稳定性。

这也是 vLLM 提出 PagedAttention 和 block manager 的背景：把 KV Cache 从“连续大张量”变成类似分页内存的块管理，减少碎片，提高可调度性。

2.7 Streaming 不只是边打印

很多人以为 streaming 就是模型每生成一个 token 就 print 一下。生产系统里的 streaming 更复杂。

它至少包括：

1. token 生成后及时解码。
2. 通过 SSE、WebSocket 或其他协议返回给客户端。
3. 客户端断开后及时取消请求。
4. 处理 tokenizer 的半个字符、空格和特殊 token。
5. 记录每个 token 的时间戳。
6. 出错时返回明确的结束状态。

streaming 对用户体验非常关键。即使总生成时间不变，只要 TTFT 更低、输出更平滑，用户感知也会明显变好。

所以推理框架通常不会等完整答案生成完再返回，而是把 decode loop 和输出通道连接起来：

decode one token -> detokenize -> push chunk -> continue decode

这要求 engine 内部维护请求状态，而不是把 generate() 当成一次性黑盒。

2.8 从 generate 到 engine 的模块拆分

一个最小推理框架通常会拆成这些模块：

API Server

- > Request Parser
- > Waiting Queue
- > Scheduler
- > Model Executor
- > KV Cache Manager
- > Sampler
- > Streamer
- > Metrics / Tracing

每个模块解决一个具体问题。

API Server 负责接收请求、鉴权、参数校验和连接管理。

Request Parser 负责把 prompt、max tokens、temperature、stop words 等参数规范化。

Waiting Queue 负责保存尚未执行的请求。

Scheduler 负责决定每一轮执行哪些请求。

Model Executor 负责真正调用模型前向计算。

KV Cache Manager 负责分配、释放和复用 KV Cache。

Sampler 负责 greedy、top-k、top-p、temperature、repetition penalty 等采样逻辑。

Streamer 负责把 token 增量返回给客户端。

Metrics / Tracing 负责记录延迟、吞吐、错误、显存和请求生命周期。

这套拆分看起来比 generate() 复杂很多，但每一层都是被生产问题逼出来的。

2.9 一个最小 engine 循环

极简 serving engine 的主循环可以想象成：

```
while True:
    new_requests = api_server.poll()
    waiting_queue.extend(new_requests)
```

```

batch = scheduler.schedule(
    waiting_queue=waiting_queue,
    running_requests=running_requests,
    cache_manager=cache_manager,
)

model_outputs = executor.forward(batch)
next_tokens = sampler.sample(model_outputs.logits)

for request, token in zip(batch.requests, next_tokens):
    request.append_token(token)
    streamer.push(request, token)

    if request.is_finished():
        cache_manager.free(request)
        running_requests.remove(request)

metrics.record(batch, model_outputs)

```

真实系统当然更复杂，但主干就是这些：收请求、排队、调度、执行、采样、流式返回、释放资源、记录指标。看 vLLM 或 SGLang 源码时，不要一上来陷入类名和文件名。先把每段代码放回这条主链路里，就容易理解很多。

2.10 请求状态机

为了让请求可调度，engine 通常会维护请求状态机。一个简化版本如下：

```

WAITING -> PREFILLING -> DECODING -> FINISHED
                \-> ABORTED
                \-> FAILED

```

常见状态包括：

1. WAITING：已接收，但还没有分配执行资源。
2. PREFILLING：正在处理 prompt。
3. DECODING：正在逐 token 生成。
4. FINISHED：正常结束。
5. ABORTED：客户端取消或超时取消。
6. FAILED：执行出错。

状态机非常重要。没有状态机，就很难处理取消、超时、重试、流式断开、显存回收和指标统计。

面试里如果你能从请求状态机讲起，比单纯背“continuous batching 可以提升吞吐”更像真正做过系统。

2.11 最小框架的演进路线

从教学角度，实现一个最小推理框架可以按下面顺序来：

1. 先实现单请求 generate loop。
2. 加入 greedy、top-k、top-p 和 temperature sampling。
3. 显式保存 KV Cache。
4. 实现 batched prefill。
5. 实现 batched decode。
6. 加入 waiting queue 和 running set。
7. 实现简单 scheduler。
8. 支持 token streaming。
9. 包一层 HTTP API。
10. 记录 TTFT、TPOT、tokens/s、显存占用和错误率。

这个路线和本书第二部分的章节安排基本一致。先把最小链路跑通，再逐步替换成更接近 vLLM / SGLang 的机制。

2.12 最小推理框架的公式和可运行 demo

从概率角度看，自回归生成是在 prompt x 后面逐步采样输出 token：

$$P(y_1, \dots, y_M \mid x) = \prod_{t=1}^M P(y_t \mid x, y_1, \dots, y_{t-1})$$

如果用 greedy decoding，第 t 个输出 token 可以写成：

$$y_t = \arg\max_{v \in \mathcal{V}} z_t(v)$$

其中 $z_t(v)$ 是第 t 步 vocab token v 的 logit。带温度的采样分布可以写成：

$$p_t(v) = \frac{\exp(z_t(v)/\tau)}{\sum_{u \in \mathcal{V}} \exp(z_t(u)/\tau)}$$

其中 τ 是 temperature。 τ 越小，分布越尖锐； τ 越大，分布越平。

在最小 engine 里，可以先把真实毫秒延迟抽象成 step 计数：

$$S_{\text{ttft}} = S_{\text{queue}} + S_{\text{prefill}} + 1$$

其中 S_{queue} 是等待步数， S_{prefill} 是 prompt 预填充消耗步数，最后的 1 表示第一次 decode 生成首 token。

TPOT 可以先用 decode iteration 的平均间隔近似：

$$S_{\text{tpot}} = \frac{S_{\text{decode}}}{\max(N_{\text{out}} - 1, 1)}$$

KV cache 在 toy engine 里可以简化成 “每个请求占用多少 token 状态”：

$$K_i = N_{\text{prompt}}, i + N_{\text{generated}}, i$$

真实系统会再乘以层数、KV head、head dimension 和 dtype 字节数；这里先用 token 级 cache 让状态变化可见。

下面这个 demo 是一个 0 依赖、可复制运行的最小推理框架骨架。它不会调用真实大模型，而是用一个 toy next-token model 模拟生成；重点是让你看到 naive generate、request state、scheduler、prefill / decode、streaming、KV cleanup 和 metrics 如何连起来。

```
from dataclasses import dataclass, field
```

```
class ToyTokenizer:
```

```
    def __init__(self):
```

```
        self.eos_token = "<eos>"
```

```
        self.vocab = {self.eos_token: 0}
```

```
        self.inverse = {0: self.eos_token}
```

```
    def encode(self, text):
```

```
        tokens = text.strip().split()
```

```
        ids = []
```

```
        for token in tokens:
```

```
            if token not in self.vocab:
```

```
                idx = len(self.vocab)
```

```
                self.vocab[token] = idx
```

```
                self.inverse[idx] = token
```

```
            ids.append(self.vocab[token])
```

```
        return ids
```

```
    def decode(self, ids):
```

```
        words = [self.inverse.get(idx, "?") for idx in ids]
```

```
        return " ".join(word for word in words if word != self.eos_token)
```

```

class ToyNextTokenModel:
    def __init__(self, tokenizer):
        self.tokenizer = tokenizer
        self.transitions = {
            "hello": "from",
            "from": "mini",
            "mini": "engine",
            "engine": "<eos>",
            "write": "a",
            "a": "tiny",
            "tiny": "serving",
            "serving": "loop",
            "loop": "<eos>",
            "explain": "prefill",
            "prefill": "then",
            "then": "decode",
            "decode": "<eos>",
        }

    def prefill(self, prompt_ids):
        return {"cached_tokens": list(prompt_ids)}

    def next_token(self, request):
        last_id = request.tokens[-1] if request.tokens else self.tokenizer.vocab[self.tokenizer.eos_token]
        last_word = self.tokenizer.inverse.get(last_id, self.tokenizer.eos_token)
        next_word = self.transitions.get(last_word, self.tokenizer.eos_token)
        return self.tokenizer.encode(next_word)[0]

@dataclass
class RequestState:
    request_id: str
    prompt: str
    max_new_tokens: int
    tokens: list
    prompt_len: int
    status: str = "WAITING"
    generated: list = field(default_factory=list)
    stream_chunks: list = field(default_factory=list)
    queue_steps: int = 0
    prefill_steps: int = 0
    decode_steps: int = 0
    first_token_step: int = None
    finish_step: int = None

class MiniFromScratchServingEngine:
    def __init__(self, tokenizer, model, max_active=2, kv_token_budget=32):
        self.tokenizer = tokenizer
        self.model = model
        self.max_active = max_active
        self.kv_token_budget = kv_token_budget
        self.waiting = []

```

```

self.running = []
self.finished = []
self.kv_cache = {}
self.step = 0
self.events = []

def submit(self, request_id, prompt, max_new_tokens):
    tokens = self.tokenizer.encode(prompt)
    self.waiting.append(
        RequestState(
            request_id=request_id,
            prompt=prompt,
            max_new_tokens=max_new_tokens,
            tokens=list(tokens),
            prompt_len=len(tokens),
        )
    )

def kv_tokens_used(self):
    return sum(len(cache["cached_tokens"]) for cache in self.kv_cache.values())

def can_admit(self, request):
    projected = self.kv_tokens_used() + request.prompt_len
    return len(self.running) < self.max_active and projected <= self.kv_token_budget

def prefill(self, request):
    request.status = "PREFILLING"
    request.prefill_steps += max(request.prompt_len, 1)
    self.kv_cache[request.request_id] = self.model.prefill(request.tokens)
    request.status = "DECODING"
    self.running.append(request)
    self.events.append((self.step, request.request_id, "prefill", request.prompt_len))

def schedule(self):
    for request in list(self.waiting):
        if self.can_admit(request):
            self.waiting.remove(request)
            self.prefill(request)
    for request in self.waiting:
        request.queue_steps += 1

def decode_one_iteration(self):
    for request in list(self.running):
        token = self.model.next_token(request)
        request.decode_steps += 1
        request.tokens.append(token)
        request.generated.append(token)
        self.kv_cache[request.request_id]["cached_tokens"].append(token)
        chunk = self.tokenizer.decode([token])
        if chunk:
            request.stream_chunks.append(chunk)
        if request.first_token_step is None:
            request.first_token_step = self.step
        self.events.append((self.step, request.request_id, "decode", chunk or "<eos>"))
        reached_limit = len(request.generated) >= request.max_new_tokens

```

```

reached_eos = token == self.tokenizer.vocab[self.tokenizer.eos_token]
if reached_limit or reached_eos:
    request.status = "FINISHED"
    request.finish_step = self.step
    self.running.remove(request)
    self.finished.append(request)
    self.kv_cache.pop(request.request_id, None)
    self.events.append((self.step, request.request_id, "cleanup", len(request.generated)))

def run_until_done(self, max_steps=20):
    while (self.waiting or self.running) and self.step < max_steps:
        self.schedule()
        self.decode_one_iteration()
        self.step += 1
    return self.report()

def report(self):
    per_request = {}
    for request in self.finished:
        output_text = self.tokenizer.decode(request.generated)
        ttft_steps = request.queue_steps + request.prefill_steps + 1
        tpot_steps = request.decode_steps / max(len(request.generated) - 1, 1)
        per_request[request.request_id] = {
            "prompt": request.prompt,
            "output": output_text,
            "stream": request.stream_chunks,
            "ttft_steps": ttft_steps,
            "tpot_steps": round(tpot_steps, 3),
            "decode_steps": request.decode_steps,
            "status": request.status,
        }
    generated_tokens = sum(len(req.generated) for req in self.finished)
    prompt_tokens = sum(req.prompt_len for req in self.finished)
    return {
        "per_request": per_request,
        "engine_metrics": {
            "finished": len(self.finished),
            "prompt_tokens": prompt_tokens,
            "generated_tokens": generated_tokens,
            "total_steps": self.step,
            "tokens_per_step": round((prompt_tokens + generated_tokens) / max(self.step, 1), 3),
            "kv_tokens_after_cleanup": self.kv_tokens_used(),
            "event_tail": self.events[-6:],
        },
    }

```

```

def naive_generate(tokenizer, model, prompt, max_new_tokens):
    tokens = tokenizer.encode(prompt)
    request = RequestState("naive", prompt, max_new_tokens, list(tokens), len(tokens), status="DECODING")
    cache = model.prefill(tokens)
    for _ in range(max_new_tokens):
        token = model.next_token(request)
        request.tokens.append(token)
        request.generated.append(token)

```

```

        cache["cached_tokens"].append(token)
        if token == tokenizer.vocab[tokenizer.eos_token]:
            break
    return tokenizer.decode(request.generated)

def audit_minimal_engine(report):
    gates = {
        "request_state_machine": all(item["status"] == "FINISHED" for item in report["per_request"].values()),
        "streaming_chunks": all(item["stream"] for item in report["per_request"].values()),
        "prefill_decode_metrics": all(item["tft_steps"] > 0 and item["decode_steps"] > 0 for item in report["per_request"].values()),
        "kv_cleanup": report["engine_metrics"]["kv_tokens_after_cleanup"] == 0,
        "event_trace": len(report["engine_metrics"]["event_tail"]) > 0,
    }
    return {
        "gates": gates,
        "minimal_engine_gate": all(gates.values()),
    }

```

```

tokenizer = ToyTokenizer()
model = ToyNextTokenModel(tokenizer)
naive_output = naive_generate(tokenizer, model, "hello", max_new_tokens=8)

engine = MiniFromScratchServingEngine(tokenizer, model, max_active=2, kv_token_budget=16)
engine.submit("r1", "hello", max_new_tokens=8)
engine.submit("r2", "write", max_new_tokens=8)
engine.submit("r3", "explain", max_new_tokens=8)
report = engine.run_until_done()
audit = audit_minimal_engine(report)

print("naive_output=", naive_output)
print("per_request=", report["per_request"])
print("engine_metrics=", report["engine_metrics"])
print("audit=", audit)

```

你应该重点看四个输出：

1. `naive_output` 表示单请求黑盒生成结果。
2. `per_request` 表示每个请求的 prompt、stream chunks、TTFT steps、TPOT steps 和状态。
3. `engine_metrics` 表示 engine 级别的 token 数、总步数、清理后的 KV token 数和事件尾部。
4. `audit` 表示最小 engine 是否具备状态机、streaming、prefill / decode 指标、KV cleanup 和 event trace。

这段代码的工程含义是：即使不用真实模型，也能先把 serving engine 的骨架跑通。后续章节再把 toy tokenizer 换成真实 tokenizer，把 toy model 换成真实模型，把 token 级 KV 计数换成 block manager，把同步循环换成异步 HTTP 服务。

2.13 常见误区

误区一：推理框架就是 Web 服务。

Web 服务只是入口。真正的难点在模型执行、KV Cache 管理、请求调度和 token 级别流式返回。

误区二：batch size 越大越好。

batch 太大会增加排队延迟和显存压力，也可能拖慢短请求。生产系统要在吞吐和延迟之间做权衡。

误区三：只要 GPU 利用率高，系统就一定好。

GPU 利用率高可能是长请求堵住了队列，也可能是无效 padding 多。LLM serving 要同时看 TTFT、TPOT、tokens/s、p95/p99 和失败率。

误区四：KV Cache 是模型内部细节，不需要关心。

对单请求 demo 可以不关心；对 serving engine 来说，KV Cache 管理几乎就是并发能力和显存效率的核心。

误区五：streaming 只是前端体验问题。

streaming 会影响请求状态、连接生命周期、取消逻辑、指标统计和资源释放，是 engine 的一等能力。

2.14 面试追问

1. `model.generate()` 和 serving engine 的本质区别是什么？
2. 为什么 LLM 推理要区分 prefill 和 decode？
3. 静态 batching 为什么不适合高并发 LLM serving？
4. continuous batching 的核心思想是什么？
5. KV Cache Manager 需要解决哪些问题？
6. 为什么 streaming 需要和请求状态机结合？
7. 如果一个用户中途断开连接，engine 应该做什么？
8. 如何从零实现一个最小 LLM serving engine？

参考回答思路：

1. 先说 `generate()` 是单请求黑盒调用，serving engine 是多请求调度系统。
2. 再说 prefill 影响首 token，decode 影响持续输出和吞吐。
3. 然后说核心模块：queue、scheduler、executor、KV cache manager、sampler、streamer、metrics。
4. 最后补 trade-off：吞吐、延迟、显存、公平性和稳定性需要一起优化。

2.15 小练习

1. 画出一个请求从进入 API Server 到输出最后一个 token 的状态变化图。
2. 用伪代码写一个只支持 greedy decoding 的单请求 generate loop。
3. 思考如果有 100 个请求同时到达，naive generate 会遇到哪些瓶颈。
4. 解释为什么一个长 prompt 请求可能拖慢其他短请求。
5. 设计一个最简单的 scheduler：每轮最多选 8 个请求 decode，你会如何选择？

2.16 本章小结

本章从 naive generate 出发，拆出了推理框架的基本演进路径。

`model.generate()` 适合理解生成原理，但不适合直接承担生产 serving。生产系统需要处理多请求、动态 batch、KV Cache、streaming、取消、超时、指标和资源释放。

从本章开始，你应该把 LLM serving 理解为一个 token 级调度系统，而不是一个简单的模型调用接口。下一章我们继续沿着这条线，完整走一遍推理请求的生命周期。

第 3 章 推理请求的生命周期

前两章分别讲了 serving engine 的整体定位，以及从 `model.generate()` 演进到推理框架需要补哪些系统能力。本章继续把视角拉到一个具体请求：它从进入 API Server 到最后释放资源，中间会经历哪些阶段？

理解请求生命周期，是理解 vLLM、SGLang、TGI 这类系统的基础。因为 scheduler、KV Cache Manager、streaming、metrics、timeout、abort、finish reason，本质上都是围绕请求生命周期展开的。

一句话概括：

推理请求不是一次函数调用，而是一个带状态、带资源、带输出通道、可被调度和取消的生命周期对象。

3.0 本讲资料边界与第二轮精修口径

本章讲的是通用 LLM serving request lifecycle，不是某个框架的源码导读。第二轮精修时，本章按下面口径处理：

1. API server、engine core、scheduler、KV cache manager、worker、metrics 和 streaming 的模块边界参考 vLLM、TGI、Triton 等公开 serving 抽象，但不把某个版本的内部类名或字段名写成标准答案。
2. request object、waiting queue、running set、finish reason、abort、timeout 和 cleanup 只抽象稳定工程问题：状态如何流转、资源如何绑定、异常如何收敛。
3. TTFT、TPOT、queue wait、prefill、decode、KV token 和 event trace 只作为教学指标，用 step 计数替代真实毫秒。
4. 本章不展开 PagedAttention、continuous batching、prefix cache、PD 分离和分布式 worker；这些在后续章节单独展开。
5. demo 只验证生命周期闭环，不追求高性能，也不依赖真实 tokenizer、GPU 或模型权重。

所以，本章的重点不是背模块名，而是能把一次请求从接入、校验、tokenization、排队、调度、prefill、decode、streaming、结束、异常和清理完整走一遍。

3.1 为什么要讲生命周期

很多初学者看 serving engine 时，会先记模块名：API Server、Engine、Scheduler、Worker、Executor、Block Manager。这样容易只记住结构，没理解数据怎么流动。

更好的方法是从一个请求出发，追踪它的完整路径：

客户端请求

- > API Server
- > 参数校验和 tokenization
- > request object
- > waiting queue
- > scheduler
- > prefill
- > decode loop
- > streaming output
- > finish / abort / fail
- > resource cleanup
- > metrics / trace

只要这条链路清楚，后面看 continuous batching、PagedAttention、PD 分离和 prefix cache 都会更容易。

3.2 入口：客户端请求

一个典型聊天补全请求可能包含：

```
{
  "model": "example-llm",
  "messages": [
    {"role": "user", "content": "解释一下 KV Cache"}
  ],
  "max_tokens": 512,
  "temperature": 0.7,
  "top_p": 0.9,
  "stream": true
}
```

从业务角度看，这是一次普通 API 调用。从 engine 角度看，它马上会变成一个需要长期维护状态的对象。

入口层通常要处理：

1. 协议解析，比如 HTTP、gRPC、SSE 或 WebSocket。

2. 鉴权和租户信息。
3. 模型名映射。
4. 参数合法性检查。
5. 请求大小限制。
6. 超时时间和优先级。
7. 是否流式输出。

入口层不应该直接调用模型。它更像网关和请求构造器，负责把外部请求转成 engine 内部统一格式。

3.3 参数规范化

不同客户端会传不同参数。engine 通常需要把它们规范化成内部配置。

常见参数包括：

1. max_tokens: 最多生成多少 token。
2. temperature: 采样随机性。
3. top_p / top_k: 采样候选集合。
4. stop: 停止词或停止序列。
5. stream: 是否增量返回。
6. seed: 是否需要可复现采样。
7. presence_penalty / frequency_penalty: 重复惩罚。
8. logprobs: 是否返回 token 概率信息。

规范化要解决三个问题：

1. 默认值是什么。
2. 参数之间是否冲突。
3. 参数是否超出系统能力。

例如，用户传了 max_tokens=100000，但服务只允许最多生成 4096 个 token，就必须在入口阶段拒绝或截断。否则请求进入 scheduler 后才发现无法执行，会浪费队列和显存资源。

3.4 Prompt 构造和 tokenization

聊天请求通常不是直接拿用户文本送进模型，而要先套聊天模板。

例如 messages 会被转换成类似：

```
<system>你是一个有帮助的助手</system>
<user>解释一下 KV Cache</user>
<assistant>
```

然后 tokenizer 把文本转成 token ids。

这一阶段要注意：

1. 不同模型的 chat template 不同。
2. tokenizer 版本必须和模型权重匹配。
3. token 数决定 prefill 成本。
4. prompt 太长可能超过上下文窗口。
5. stop words 需要和 token 边界协调。

tokenization 不是纯文本处理小事。生产系统里，token 数会影响路由、限流、计费、调度和显存预估。

3.5 创建 Request 对象

进入 engine 后，外部请求通常会被封装成内部 request object。

一个简化结构如下：


```

class Request:
    def __init__(self, request_id, input_ids, sampling_params):
        self.request_id = request_id
        self.input_ids = input_ids
        self.output_ids = []
        self.sampling_params = sampling_params
        self.status = "WAITING"
        self.arrival_time = now()
        self.first_token_time = None
        self.finished_time = None
        self.kv_blocks = []
        self.stream = None
        self.error = None

```

真实系统里的字段更多，但核心信息差不多：

1. 请求身份：request_id、tenant、model。
2. 输入输出：input_ids、output_ids。
3. 采样配置：temperature、top-p、stop 等。
4. 生命周期状态：waiting、prefilling、decoding、finished。
5. 资源引用：KV Cache block、worker、device。
6. 输出通道：stream handle、回调或队列。
7. 指标时间戳：到达、首 token、完成。

注意，request object 不只是数据包。它是 scheduler 和 executor 共同操作的状态对象。

3.6 进入 waiting queue

请求创建后，通常不会马上执行，而是进入 waiting queue。

waiting queue 的作用是把“请求到达”和“模型执行”解耦。

如果没有队列，每来一个请求就直接调用模型，会导致：

1. 无法组成 batch。
2. 无法做公平调度。
3. 无法控制显存。
4. 无法统一处理优先级。
5. 无法在高峰期保护系统。

队列里常见的排序策略包括：

1. FIFO：先来先服务，简单但不一定最优。
2. Shortest prompt first：短 prompt 优先，降低平均 TTFT，但可能饿死长请求。
3. Priority-based：高优先级租户或实时业务优先。
4. Deadline-aware：接近超时的请求优先。
5. Token-budget-based：按本轮 token 预算选择请求。

生产系统往往不会只用单一策略，而是结合公平性、SLO 和资源预算。

3.7 Scheduler 决定谁上 GPU

Scheduler 是生命周期里的核心决策点。它每一轮会决定哪些请求进入模型执行。

一个简化调度输入包括：

1. waiting queue 中的新请求。
2. running set 中正在 decode 的请求。
3. 当前可用 KV Cache 块数。
4. 最大 batch token 数。
5. 最大并发序列数。

6. 每个请求的状态、长度和优先级。

调度输出通常是一个 batch:

本轮 prefill: request_7, request_8

本轮 decode: request_1, request_2, request_5

Scheduler 要处理的 trade-off 很多:

1. 新请求 prefill 多一些, TTFT 可能更好, 但会挤占 decode。
2. decode 请求多一些, 已用户输出更平滑, 但新请求等待更久。
3. 长 prompt 直接 prefill, 可能造成大抖动。
4. 短请求优先, 平均延迟好, 但公平性可能差。
5. batch 过大, 吞吐高但单请求延迟可能变差。

这就是推理框架比 generate() 难的地方: 它不是做一次最优, 而是在每轮 iteration 做动态权衡。

3.8 Prefill 阶段

当请求第一次被调度执行时, 会进入 prefill 阶段。

Prefill 做的是:

1. 读取 prompt token。
2. 执行模型前向。
3. 为每层生成并保存 KV Cache。
4. 得到最后一个位置的 logits。
5. 采样第一个输出 token。

简化流程如下:

input_ids -> model forward -> KV Cache -> logits -> sample first token

Prefill 的结束时间直接决定 TTFT, 也就是用户看到第一个 token 的等待时间。

长 prompt 的 prefill 可能非常重。为了避免它一次占用过多计算资源, 一些系统会使用 chunked prefill, 把长 prompt 分块处理, 让 decode 请求有机会穿插执行。

3.9 Decode 阶段

prefill 完成后, 请求进入 decode 阶段。decode 每轮通常只为每个请求生成一个 token。

简化流程如下:

last_token + KV Cache -> model forward -> logits -> sample next token -> append

Decode 阶段会重复很多轮, 直到满足停止条件。

停止条件可能包括:

1. 生成了 EOS token。
2. 达到 max_tokens。
3. 命中 stop words。
4. 客户端取消。
5. 请求超时。
6. engine 主动中止。
7. 执行错误。

Decode 的关键指标是 TPOT, 也就是每个输出 token 的平均耗时。用户体验里的 “输出是否顺滑”, 主要由 decode 阶段决定。

3.10 Streaming 输出

如果请求启用了 streaming, engine 在每次生成 token 后都要尽快把增量文本推给客户端。

流程大致是:

token id -> detokenize -> text chunk -> stream channel -> client

这里有几个细节:

1. 一个 token 不一定对应完整字符。
2. 多个 token 合起来才可能形成自然文本片段。
3. stop sequence 可能跨 token 出现。
4. 客户端断开时, engine 必须释放资源。
5. 流式通道不能阻塞模型执行主循环太久。

因此很多系统会把模型执行和网络输出解耦, 用异步队列或回调把 token 传给输出层。

Streaming 的难点不是 “能不能返回 token”, 而是 “返回 token 的同时不破坏调度、资源释放和错误处理”。

3.11 Finish Reason

请求结束时, 系统通常需要返回 finish reason。

常见类型包括:

1. stop: 正常遇到停止符或 stop sequence。
2. length: 达到最大生成长度。
3. abort: 用户取消。
4. timeout: 请求超时。
5. error: 系统错误。
6. content_filter: 输出被策略拦截。

finish reason 很重要, 因为它告诉上游业务这次输出是否完整。

例如, length 说明模型可能还没说完, 只是被最大长度截断。abort 说明不是模型结束, 而是连接或用户行为导致中止。error 需要上游决定是否重试。

3.12 资源释放

请求结束后, engine 必须释放资源, 尤其是 KV Cache。

需要清理的资源包括:

1. GPU KV Cache block。
2. CPU staging buffer。
3. 输出 stream。
4. request object。
5. tracing span。
6. 临时 token buffer。
7. 队列和索引中的引用。

资源释放如果不彻底, 会造成隐蔽问题:

1. 显存慢慢上涨。
2. 可用 block 越来越少。
3. scheduler 误以为资源不足。
4. 已取消请求继续占用 decode slot。
5. 长时间运行后吞吐下降。

推理服务是长生命周期进程, 轻微泄漏也会在高 QPS 下快速放大。

3.13 Metrics 和 Trace

每个请求都应该留下可观测信息。

常见指标包括：

1. request arrival time。
2. queue waiting time。
3. prefill latency。
4. TTFT。
5. decode token count。
6. TPOT。
7. total latency。
8. input tokens。
9. output tokens。
10. finish reason。
11. error type。
12. KV Cache block 使用量。

trace 则用于回答更细的问题：

1. 请求在哪个阶段最慢。
2. 是排队慢、prefill 慢，还是 decode 慢。
3. 是否被抢占或等待 KV Cache。
4. 是否发生过 stream 阻塞。
5. 哪个 worker 执行了请求。

没有这些数据，线上问题只能靠猜。面试里如果问“TTFT 变差怎么排查”，生命周期指标就是排查入口。

3.14 Abort 和 Timeout

生产系统必须认真处理取消和超时。

用户取消请求时，engine 不能只关闭连接，还要：

1. 标记 request 状态为 aborted。
2. 从 waiting queue 或 running set 移除。
3. 释放 KV Cache。
4. 停止继续 streaming。
5. 记录 finish reason。
6. 避免下一轮 scheduler 再选中它。

超时也类似。区别是超时可能来自系统策略，而不是用户主动取消。

常见超时包括：

1. 排队超时。
2. 首 token 超时。
3. 总请求超时。
4. stream 空闲超时。

很多 serving bug 都来自取消处理不完整：请求已经断开，但 decode loop 还在生成；或者请求被移出队列，但 KV Cache 没释放。

3.15 生命周期状态机

可以用下面的状态机总结本章：

```
RECEIVED
-> VALIDATED
-> TOKENIZED
```

-> WAITING
-> PREFILLING
-> DECODING
-> FINISHED

任意中间状态 -> ABORTED

任意中间状态 -> FAILED

WAITING/PREFILLING/DECODING -> TIMEOUT

每个状态都对应明确动作：

1. RECEIVED：接收到外部请求。
2. VALIDATED：参数校验完成。
3. TOKENIZED：prompt 已转 token。
4. WAITING：进入等待队列。
5. PREFILLING：执行 prompt 前向并建立 KV Cache。
6. DECODING：逐 token 生成。
7. FINISHED：正常结束。
8. ABORTED：用户取消。
9. FAILED：系统错误。
10. TIMEOUT：超时中止。

状态机的价值在于让复杂情况有规则可循。无论正常完成、失败、取消还是超时，都能走到统一清理路径。

3.16 请求生命周期指标和可运行 demo

可以把一个请求抽象成：

$R_i = (a_i, p_i, n_i, m_i, s_i, q_i, u_i, d_i, f_i, k_i, z_i)$

其中 a_i 是到达时间， p_i 是 prompt token 数， n_i 是最大输出 token 数， m_i 是模型或路由信息， s_i 是状态， q_i 是排队信息， u_i 是用户或租户信息， d_i 是 decode 进度， f_i 是 finish reason， k_i 是 KV 资源引用， z_i 是 trace 和 metrics。

排队时间可以写成：

$T_{\{\mathrm{queue}\},i} = t_{\{\mathrm{start}\},i} - t_{\{\mathrm{arrival}\},i}$

TTFT 可以拆成：

$T_{\{\mathrm{ttft}\},i} = t_{\{\mathrm{first}\},i} - t_{\{\mathrm{arrival}\},i} = T_{\{\mathrm{queue}\},i} + T_{\{\mathrm{prefill}\},i} + D_{\{\mathrm{first}\},i}$

其中 D_{first} 表示从 prefill 结束到首个输出 token 形成的 decode / flush 步数。TPOT 只对已经产生过输出 token 的请求计算：

$T_{\{\mathrm{tpot}\},i} = \frac{t_{\{\mathrm{finish}\},i} - t_{\{\mathrm{first}\},i}}{\max(N_{\{\mathrm{out}\},i} - 1, 1)}$

活跃 KV token 占用可以先用 toy 口径估算：

$K_{\{\mathrm{active}\}}(t) = \sum_{i \in A(t)} (N_{\{\mathrm{prompt}\},i} + N_{\{\mathrm{generated}\},i})$

生命周期门禁可以写成：

$G_{\{\mathrm{life}\}} = G_{\{\mathrm{state}\}} G_{\{\mathrm{metrics}\}} G_{\{\mathrm{stream}\}} G_{\{\mathrm{cleanup}\}} G_{\{\mathrm{trace}\}}$

含义是：状态机、指标、流式输出、资源清理和 trace 任意一项缺失，这个 request lifecycle 都不能算闭环。

下面这个 demo 用 0 依赖代码模拟 3 个请求：一个正常结束，一个客户端取消，一个排队超时。重点看异常路径是否也会释放 KV、退出队列并留下 finish reason。

```
from dataclasses import dataclass, field
```

```
TERMINAL_STATES = {"FINISHED", "ABORTED", "TIMEOUT", "FAILED"}
```

```

@dataclass
class Request:
    request_id: str
    arrival_step: int
    prompt_tokens: int
    max_new_tokens: int
    stream: bool = True
    queue_timeout_steps: int = 5
    abort_at_step: int = None
    state: str = "RECEIVED"
    generated_tokens: int = 0
    queue_steps: int = 0
    prefill_steps: int = 0
    decode_steps: int = 0
    start_step: int = None
    first_token_step: int = None
    finish_step: int = None
    finish_reason: str = None
    stream_chunks: list = field(default_factory=list)
    trace: list = field(default_factory=list)


class MiniRequestLifecycleEngine:
    def __init__(self, max_active=2, kv_token_budget=16):
        self.max_active = max_active
        self.kv_token_budget = kv_token_budget
        self.waiting = []
        self.running = []
        self.terminal = []
        self.kv_cache = {}
        self.events = []
        self.step = 0
        self.max_kv_tokens = 0

    def submit(self, request):
        for state in ["RECEIVED", "VALIDATED", "TOKENIZED", "WAITING"]:
            request.trace.append(state)
        request.state = "WAITING"
        self.waiting.append(request)
        self.events.append((self.step, request.request_id, "submit", request.prompt_tokens))

    def kv_tokens_used(self):
        return sum(self.kv_cache.values())

    def can_admit(self, request):
        projected = self.kv_tokens_used() + request.prompt_tokens
        return len(self.running) < self.max_active and projected <= self.kv_token_budget

    def admit_waiting(self):
        for request in list(self.waiting):
            if self.can_admit(request):
                self.waiting.remove(request)
                request.state = "PREFILLING"

```

```

        request.start_step = self.step
        request.queue_steps = self.step - request.arrival_step
        request.trace.append("PREFILLING")
        self.running.append(request)
        self.events.append((self.step, request.request_id, "admit", request.queue_steps))
    for request in self.waiting:
        request.queue_steps = self.step - request.arrival_step

def terminalize(self, request, state, reason):
    if request in self.waiting:
        self.waiting.remove(request)
    if request in self.running:
        self.running.remove(request)
    request.state = state
    request.finish_reason = reason
    request.finish_step = self.step
    request.trace.append(state)
    self.kv_cache.pop(request.request_id, None)
    if request not in self.terminal:
        self.terminal.append(request)
    self.events.append((self.step, request.request_id, "cleanup", reason))

def expire_waiting(self):
    for request in list(self.waiting):
        request.queue_steps = self.step - request.arrival_step
        if request.queue_steps > request.queue_timeout_steps:
            self.terminalize(request, "TIMEOUT", "queue_timeout")

def run_one_step(self):
    self.expire_waiting()
    self.admit_waiting()
    for request in list(self.running):
        if request.abort_at_step is not None and self.step >= request.abort_at_step:
            self.terminalize(request, "ABORTED", "client_abort")
            continue
        if request.state == "PREFILLING":
            request.prefill_steps += 1
            request.state = "DECODING"
            request.trace.append("DECODING")
            self.kv_cache[request.request_id] = request.prompt_tokens
            self.events.append((self.step, request.request_id, "prefill", request.prompt_tokens))
            continue
        if request.state == "DECODING":
            request.decode_steps += 1
            request.generated_tokens += 1
            self.kv_cache[request.request_id] += 1
            if request.first_token_step is None:
                request.first_token_step = self.step
            chunk = f"tok{request.generated_tokens}"
            if request.stream:
                request.stream_chunks.append(chunk)
            self.events.append((self.step, request.request_id, "decode", chunk))
            if request.generated_tokens >= request.max_new_tokens:
                self.terminalize(request, "FINISHED", "length")
    self.max_kv_tokens = max(self.max_kv_tokens, self.kv_tokens_used())

```

```

        self.step += 1

def run_until_done(self, max_steps=20):
    while (self.waiting or self.running) and self.step < max_steps:
        self.run_one_step()
    return self.report()

def report(self):
    per_request = {}
    for request in sorted(self.terminal, key=lambda item: item.request_id):
        ttft_steps = None
        tpot_steps = None
        if request.first_token_step is not None:
            ttft_steps = request.first_token_step - request.arrival_step
            tpot_steps = (request.finish_step - request.first_token_step) / max(request.generated_tokens - 1, 1)
        per_request[request.request_id] = {
            "state": request.state,
            "reason": request.finish_reason,
            "queue_steps": request.queue_steps,
            "ttft_steps": ttft_steps,
            "tpot_steps": None if tpot_steps is None else round(tpot_steps, 3),
            "prefill_steps": request.prefill_steps,
            "decode_steps": request.decode_steps,
            "generated_tokens": request.generated_tokens,
            "stream": request.stream_chunks,
            "trace": request.trace,
        }
    ttft_values = [item["ttft_steps"] for item in per_request.values() if item["ttft_steps"] is not None]
    tpot_values = [item["tpot_steps"] for item in per_request.values() if item["tpot_steps"] is not None]
    return {
        "per_request": per_request,
        "metrics": {
            "terminal": len(self.terminal),
            "finished": sum(req.state == "FINISHED" for req in self.terminal),
            "aborted": sum(req.state == "ABORTED" for req in self.terminal),
            "timeout": sum(req.state == "TIMEOUT" for req in self.terminal),
            "avg_queue_steps": round(sum(req.queue_steps for req in self.terminal) / max(len(self.terminal), 1)),
            "p95_ttft_steps": max(ttft_values) if ttft_values else None,
            "p95_tpot_steps": max(tpot_values) if tpot_values else None,
            "max_kv_tokens": self.max_kv_tokens,
            "kv_tokens_after_cleanup": self.kv_tokens_used(),
            "event_tail": self.events[-8:],
        },
    }

def audit_lifecycle(report):
    states = [item["state"] for item in report["per_request"].values()]
    gates = {
        "state_machine_terminal": all(state in TERMINAL_STATES for state in states),
        "normal_and_exception_path": "FINISHED" in states and ("ABORTED" in states or "TIMEOUT" in states),
        "ttft_tpot_metrics": report["metrics"]["p95_ttft_steps"] is not None and report["metrics"]["p95_tpot_steps"]
        "streaming_or_finish_reason": all(item["stream"] or item["reason"] for item in report["per_request"].values())
        "cleanup": report["metrics"]["kv_tokens_after_cleanup"] == 0,
        "event_trace": len(report["metrics"]["event_tail"]) > 0,
    }

```



```

    }
    return {
        "gates": gates,
        "lifecycle_gate": all(gates.values()),
    }

```

```

engine = MiniRequestLifecycleEngine(max_active=2, kv_token_budget=16)
engine.submit(Request("r1", arrival_step=0, prompt_tokens=3, max_new_tokens=3))
engine.submit(Request("r2", arrival_step=0, prompt_tokens=5, max_new_tokens=4, abort_at_step=2))
engine.submit(Request("r3", arrival_step=0, prompt_tokens=6, max_new_tokens=2, queue_timeout_steps=1))

```

```

report = engine.run_until_done()
audit = audit_lifecycle(report)

```

```

print("per_request=", report["per_request"])
print("metrics=", report["metrics"])
print("audit=", audit)

```

这段 demo 的工程含义是：生命周期不能只验证 happy path。只要有客户端取消、排队超时、执行失败或策略拦截，都必须把状态、finish reason、stream、KV cleanup 和 trace 放进同一套收敛路径。

3.17 面试追问

1. 一个 LLM 请求从进入 API Server 到返回结果，中间有哪些阶段？
2. 为什么请求进入 engine 后不能直接执行，而要进入 waiting queue？
3. Scheduler 在请求生命周期里做什么决策？
4. TTFT 对应生命周期中的哪个阶段？
5. TPOT 对应生命周期中的哪个阶段？
6. 客户端中途断开连接，engine 应该如何处理？
7. finish reason 有哪些类型，为什么重要？
8. 线上 TTFT 突然升高，你会从哪些生命周期指标排查？

参考回答思路：

1. 先按链路说：接入、校验、tokenization、排队、调度、prefill、decode、stream、结束、清理、指标。
2. 再强调请求是有状态对象，不是一次函数调用。
3. 然后说明 prefill 影响 TTFT，decode 影响 TPOT。
4. 最后补充异常路径：abort、timeout、fail 都必须释放 KV Cache 并记录 finish reason。

3.18 小练习

1. 画一张请求生命周期状态机，至少包含 waiting、prefilling、decoding、finished、aborted、failed。
2. 设计一个 Request 类，写出你认为最重要的 10 个字段。
3. 如果请求在 waiting queue 中超时，应该释放哪些资源？
4. 如果请求已经进入 decode 阶段后客户端断开，engine 应该如何停止它？
5. 设计一组指标，用来判断 TTFT 变差到底是排队导致还是 prefill 导致。

3.19 本章小结

本章从一个请求出发，完整走了一遍 LLM 推理请求的生命周期。

推理请求会经历接入、参数规范化、prompt 构造、tokenization、创建 request object、进入 waiting queue、scheduler 调度、prefill、decode、streaming、结束、资源释放和指标记录。正常结束只是其中一种路径，取消、超时和失败同样必须被纳入状态机。

下一章我们会进一步深入生命周期中最关键的四个概念：Prefill、Decode、KV Cache 和 Token Streaming。它们决定了大模型推理的性能边界，也是后续理解 vLLM、SGLang 和 PD 分离的基础。

第 4 章 Prefill、Decode、KV Cache 和 Token Streaming

上一章我们从请求生命周期出发，走完了接入、排队、调度、执行、输出和资源释放。本章把生命周期中最关键的四个概念单独拎出来：Prefill、Decode、KV Cache 和 Token Streaming。

这四个概念决定了 LLM serving 的基本性能边界。后面无论讲 vLLM 的 PagedAttention、SGLang 的 RadixAttention，还是 PD 分离、chunked prefill、多级 KV Cache，本质上都是围绕它们做优化。

一句话概括：

Prefill 决定首 token 等多久，Decode 决定后续 token 多快，KV Cache 决定显存和并发，Token Streaming 决定用户如何感知生成过程。

4.0 本讲资料边界与第二轮精修口径

本章讲的是 Prefill、Decode、KV Cache 和 Token Streaming 的稳定机制，不是 SGLang 源码导读。第二轮精修时，本章按下面口径处理：

1. KV Cache 的基础定义参考 Hugging Face Transformers cache 文档：cache 用来复用自回归生成中的历史 key / value，避免每轮重复计算历史 token。
2. request lifecycle、scheduler、KV cache manager 和 worker 的模块边界参考 vLLM、TGI 和 Triton 等公开 serving 抽象，但本章不绑定具体 runtime 的内部字段名。
3. PagedAttention、RadixAttention、prefix cache、chunked prefill、PD 分离和多级 KV cache 只在本章点到为止，后续章节会单独展开。
4. 本章公式用 toy step 和 MiB 估算解释资源趋势，不把数值当作真实 benchmark。
5. demo 只验证 prefill / decode / KV / streaming 四者如何串起来，不依赖真实 GPU、真实 tokenizer 或模型权重。

所以，本章的目标是让你能用公式和最小代码解释：为什么长 prompt 拉高 TTFT，为什么 decode 影响 TPOT，为什么 KV cache 会卡住并发，以及为什么 streaming 必须和资源清理绑定。

4.1 为什么这四个概念最重要

大模型推理看起来像一次 generate() 调用，但内部不是一次性计算完整答案。

它更像下面这条流水线：

```
prompt tokens
-> prefill
-> KV Cache
-> first token
-> decode one token
-> stream one chunk
-> decode next token
-> stream next chunk
-> ...
-> finish
```

如果你只知道“模型输入 prompt，输出答案”，就很难理解推理性能为什么复杂。

如果你知道 prefill、decode、KV cache 和 streaming，就能解释很多线上现象：

1. 为什么长 prompt 首 token 慢。
2. 为什么输出过程中有时一卡一卡的。
3. 为什么显存够放模型权重，却撑不住高并发。
4. 为什么 batch 越大不一定体验越好。
5. 为什么 PD 分离要把 prefill 和 decode 放到不同资源池。

4.2 Prefill 是什么

Prefill 是处理输入 prompt 的阶段。

如果用户输入了 2000 个 token，模型需要先对这 2000 个 token 做一次前向计算，得到每一层的 KV Cache，并产生下一个 token 的 logits。

简化流程：

```
input_ids[0:prompt_len]
-> embedding
-> transformer layers
-> logits at last position
-> sample first output token
```

Prefill 的输入长度通常等于 prompt 长度，计算量和 prompt token 数强相关。

对用户来说，prefill 最直接影响的是 TTFT，也就是 time to first token。

如果 TTFT 很高，用户会觉得服务“半天没反应”。即使后面 decode 很快，体验也已经被首 token 等待拖差了。

4.3 Prefill 的计算特点

Prefill 阶段通常有这些特点：

1. 一次处理多个 prompt token。
2. 矩阵乘法规模较大。
3. 更容易利用 GPU 算力。
4. 计算量随 prompt 长度上升。
5. 会产生大量 KV Cache 写入。

从资源角度看，prefill 更偏 compute-bound，也就是更依赖 GPU 计算能力。当然这不是绝对的，长上下文、注意力实现和 cache 写入也会带来内存压力。

Prefill 的主要优化方向包括：

1. 提升 batched prefill 的吞吐。
2. 控制单个长 prompt 对系统的冲击。
3. 使用 chunked prefill 把长 prompt 拆成多段。
4. 复用 prefix cache，避免重复 prefill。
5. 在 PD 分离架构中把 prefill 放到更适合大计算的资源池。

4.4 Decode 是什么

Decode 是逐 token 生成阶段。

prefill 之后，模型已经有了 prompt 的 KV Cache。此后每一轮只输入上一个 token，并读取历史 KV Cache，生成下一个 token。

简化流程：

```
last_token + past KV Cache
-> transformer layers
-> logits
-> sample next token
-> append token
-> update KV Cache
```

Decode 会重复很多轮。用户看到的答案就是 decode 一轮一轮积累出来的。

如果生成 500 个 token，decode 至少要进行 500 轮模型前向。每轮可能很小，但重复次数很多。

因此 decode 对 TPOT 影响很大，也就是 time per output token。

4.5 Decode 的资源特点

Decode 阶段通常有这些特点：

1. 每个请求每轮只处理一个新 token。
2. 单轮计算量比 prefill 小。
3. 需要频繁读取 KV Cache。
4. 对内存带宽和调度开销敏感。
5. batch 中请求数量会动态变化。

Decode 常被认为更偏 memory-bound，因为模型需要不断读取历史 KV Cache。上下文越长，KV Cache 越大，读取压力越明显。

Decode 的主要优化方向包括：

1. continuous batching，提高每轮执行的有效请求数。
2. 高效 KV Cache 布局，减少碎片和无效读取。
3. kernel 优化，降低小 batch decode 的开销。
4. speculative decoding，减少主模型 decode 轮数。
5. 在 PD 分离架构中使用适合 decode 的资源池。

4.6 Prefill 和 Decode 的差异

可以用一张表总结：

维度	Prefill	Decode
输入	prompt 中的多个 token	上一轮生成的 token
输出	第一个输出 token 和 KV Cache	下一个输出 token 和更新后的 KV Cache
频率	每个请求通常一次	每个输出 token 一次
主要影响	TTFT	TPOT、持续吞吐
资源倾向	更偏大计算	更偏 cache 读取和调度
优化重点	长 prompt、prefix 复用、chunking	continuous batching、cache 管理、decode kernel

这个差异非常重要。很多系统设计问题都来自 prefill 和 decode 混在一起带来的冲突。

例如，一个长 prompt prefill 可能占用大量计算资源，导致正在 decode 的请求输出卡顿。反过来，如果系统一直优先 decode，新请求的 TTFT 又会变差。

Scheduler 要做的，就是在 prefill 和 decode 之间不断做权衡。

4.7 KV Cache 是什么

Transformer 自注意力中，每个 token 会产生 key 和 value。生成下一个 token 时，模型需要关注历史 token。如果每轮都重新计算历史 token 的 key/value，成本会非常高。

KV Cache 的作用就是保存历史 token 的 key/value，让 decode 阶段只计算新 token。

没有 KV Cache：

第 1 轮：计算 prompt + token1

第 2 轮：重新计算 prompt + token1 + token2

第 3 轮：重新计算 prompt + token1 + token2 + token3

有 KV Cache：

prefill：计算 prompt，保存 KV

decode 1：只计算 token1，读取历史 KV

decode 2: 只计算 token2, 读取历史 KV

decode 3: 只计算 token3, 读取历史 KV

KV Cache 是自回归推理能高效运行的关键。

4.8 KV Cache 为什么占显存

KV Cache 保存的是每层、每个历史 token、每个 attention head 的 key/value。

粗略看, 它和这些因素正相关:

1. 层数。
2. hidden size 或 head dimension。
3. attention head 数量。
4. batch 中请求数。
5. 每个请求的上下文长度。
6. cache 的数据类型, 比如 FP16、BF16、FP8。

可以记一个直觉: 模型权重是相对固定的, KV Cache 会随着并发和上下文长度动态增长。

这解释了一个常见现象: 模型权重明明能放进 GPU, 为什么一上并发就 OOM? 原因往往不是权重变大了, 而是 KV Cache 占满了剩余显存。

4.9 KV Cache 管理的难点

单请求时, KV Cache 可以简单地随请求增长。多请求 serving 中, 它变成了内存管理问题。

难点包括:

1. 分配: 新请求进入时, 如何分配 cache 空间。
2. 扩容: decode 每生成一个 token, 都要追加 cache。
3. 释放: 请求结束后必须归还 cache。
4. 碎片: 不同请求长度不同, 连续分配容易浪费。
5. 抢占: 资源不足时是否暂停或重算部分请求。
6. 复用: 相同 prefix 能否共享 cache。
7. 迁移: PD 分离或多级缓存中, cache 是否要跨设备移动。

vLLM 的 PagedAttention 就是为了解决 KV Cache 连续分配和碎片问题。它把 KV Cache 分成块, 类似操作系统里的分页内存, 让请求的逻辑 token 序列可以映射到非连续物理块。

4.10 Token Streaming 是什么

Token Streaming 是边生成边返回。

非流式输出是:

生成完整答案 -> 一次性返回

流式输出是:

生成 token1 -> 返回片段1

生成 token2 -> 返回片段2

生成 token3 -> 返回片段3

对用户体验来说, streaming 非常重要。哪怕总耗时一样, 如果用户能很快看到第一个 token, 并持续看到内容增长, 就会感觉系统更快。

Streaming 和 TTFT、TPOT 都有关:

1. TTFT 决定多久看到第一段输出。
2. TPOT 决定后续输出是否顺滑。
3. 抖动决定输出是否一卡一卡。

4.11 Streaming 的工程细节

Token Streaming 不是简单 `print(token)`。

工程上要处理：

1. token 到文本的增量解码。
2. Unicode 字符边界。
3. stop sequence 跨 token 匹配。
4. SSE、WebSocket 或 gRPC stream。
5. 客户端断开检测。
6. backpressure，也就是客户端读得慢。
7. 错误和结束事件。
8. usage 统计和 finish reason。

一个典型流式事件可能包括：

```
{
  "id": "request-123",
  "delta": "Cache",
  "finish_reason": null
}
```

结束时再返回：

```
{
  "id": "request-123",
  "delta": "",
  "finish_reason": "stop"
}
```

如果客户端中途断开，engine 必须把请求标记为 aborted，并释放 KV Cache。否则看起来用户已经走了，GPU 还在继续为这个请求生成。

4.12 四者如何串起来

把四个概念放到同一条链路中：

1. Prompt tokenization
2. Prefill prompt tokens
3. 建立 KV Cache
4. 采样第一个 token
5. Streaming 返回第一个文本片段
6. Decode 下一 token
7. 更新 KV Cache
8. Streaming 返回下一文本片段
9. 重复 6-8
10. 请求结束，释放 KV Cache

可以看到，KV Cache 贯穿 prefill 和 decode，streaming 则把 decode 结果及时暴露给客户端。

Prefill、decode、cache、streaming 不是四个孤立模块，而是一条请求执行链路中的不同职责。

4.13 对 Scheduler 的影响

Scheduler 必须理解这些阶段，否则无法做出合理决策。

它要知道：

1. 新请求 prefill 会消耗多少 token budget。
2. running 请求 decode 需要多少 cache。

3. 当前 KV Cache 是否足够容纳新请求。
4. 哪些请求已经接近结束。
5. 哪些请求启用了 streaming, 需要输出平滑。
6. 长 prefill 是否应该拆成 chunk。

一个简单但实用的调度目标是：

1. 不让新请求饿死，控制 TTFT。
2. 不让老请求卡住，控制 TPOT。
3. 不超出 KV Cache 显存预算。
4. 尽量让 GPU 每轮都有足够有效工作。

这也是为什么推理框架的 scheduler 比普通任务队列复杂得多。

4.14 常见性能现象解释

现象一：prompt 变长后，首 token 明显变慢。

原因通常是 prefill token 数增加，prefill 延迟上升。

现象二：输出前面快，后面越来越慢。

原因可能是上下文越来越长，KV Cache 读取压力增大，也可能是 batch 内其他请求影响。

现象三：并发上来后显存爆掉。

原因通常是 KV Cache 随并发和上下文长度增长。

现象四：总耗时没变，但用户觉得更快了。

原因可能是 TTFT 降低或 streaming 更平滑。

现象五：GPU 利用率不低，但用户输出一卡一卡。

原因可能是 decode scheduling 不稳定，或者流式输出通道阻塞。

4.15 常见误区

误区一：Prefill 和 Decode 只是代码里的两个阶段，不影响架构。

实际上它们资源画像不同，直接影响 scheduler、PD 分离、扩缩容和性能调优。

误区二：KV Cache 只是优化项，不是必须关注的核心。

没有 KV Cache，自回归推理成本会非常高；KV Cache 管理不好，高并发 serving 会很快遇到显存瓶颈。

误区三：Streaming 只影响前端体验。

Streaming 还影响连接管理、取消、backpressure、finish reason 和资源释放。

误区四：TTFT 只和模型大小有关。

TTFT 还和排队、prompt 长度、prefill batch、prefix cache、调度策略和系统负载有关。

误区五：TPOT 只和 GPU 算力有关。

TPOT 还受 KV Cache 读取、batch 动态变化、kernel、调度开销和输出通道影响。

4.16 阶段指标、KV 公式和可运行 demo

可以把一个请求的阶段画像写成：

$P_i = (N_{\{\mathrm{prompt}\},i}, N_{\{\mathrm{out}\},i}, S_{\{\mathrm{prefill}\},i}, S_{\{\mathrm{decode}\},i}, K_i, Q_i, F_i)$

其中 N_{prompt} 是输入 token 数, N_{out} 是输出 token 数, S_{prefill} 是 prefill step, S_{decode} 是 decode step, K_i 是 KV cache 占用, Q_i 是 streaming backlog, F_i 是 finish reason。

在 toy engine 里, 可以把 prefill 近似成按 token 吞吐切块:

$$S_{\{\mathrm{prefill}, i\}} = \left\lceil \frac{N_{\{\mathrm{prompt}, i\}}}{R_{\{\mathrm{prefill}\}}} \right\rceil$$

decode 每轮生成一个 token 时:

$$S_{\{\mathrm{decode}, i\}} = N_{\{\mathrm{out}, i\}}$$

因此 toy TTFT 和 TPOT 可以写成:

$$S_{\{\mathrm{ttft}, i\}} = S_{\{\mathrm{queue}, i\}} + S_{\{\mathrm{prefill}, i\}} + 1$$

$$S_{\{\mathrm{tpot}, i\}} = \frac{S_{\{\mathrm{decode}, i\}}}{\max(N_{\{\mathrm{out}, i\}} - 1, 1)}$$

KV cache 显存估算要包含 K 和 V 两份状态:

$$M_{\{\mathrm{kv}, i\}} = 2L(N_{\{\mathrm{prompt}, i\}} + N_{\{\mathrm{generated}, i\}})H_{\{\mathrm{kv}\}}D_{\text{hb}}$$

其中 L 是层数, H_{kv} 是 KV head 数, D_{h} 是 head dimension, b 是每个元素的字节数。总 KV pressure 可以写成:

$$P_{\{\mathrm{kv}\}} = \frac{\sum_i M_{\{\mathrm{kv}, i\}}}{B_{\{\mathrm{kv}\}}}$$

streaming backlog 可以用增量事件描述:

$$Q_{\{\mathrm{stream}\}}(t) = Q_{\{\mathrm{stream}\}}(t-1) + C_{\{\mathrm{emit}\}}(t) - C_{\{\mathrm{flush}\}}(t)$$

其中 C_{emit} 是 decode 生成的 chunk 数, C_{flush} 是客户端已经消费的 chunk 数。最终阶段门禁可以写成:

$$G_{\{\mathrm{phase}\}} = G_{\{\mathrm{prefill}\}} G_{\{\mathrm{decode}\}} G_{\{\mathrm{kv}\}} G_{\{\mathrm{stream}\}} G_{\{\mathrm{cleanup}\}}$$

下面这个 demo 用 0 依赖代码把四个阶段串起来。它不会模拟真实 attention, 只用 token 数和配置估算阶段指标, 让你能看见长 prompt、输出长度、KV cache 和 streaming backlog 如何共同影响 serving。

```
from dataclasses import dataclass, field
from math import ceil
```

```
@dataclass
class ToyRequest:
    request_id: str
    prompt_tokens: int
    output_tokens: int
    client_read_every: int = 1
    stream_buffer_capacity: int = 4
    stream_chunks: list = field(default_factory=list)
    flushed_chunks: list = field(default_factory=list)
```

```
class MiniPrefillDecodeKVStreamingAudit:
    def __init__(
        self,
        layers=32,
        kv_heads=8,
        head_dim=128,
        dtype_bytes=2,
        prefill_tokens_per_step=1024,
        kv_budget_mib=2048,
    ):
        self.layers = layers
        self.kv_heads = kv_heads
```



```

self.head_dim = head_dim
self.dtype_bytes = dtype_bytes
self.prefill_tokens_per_step = prefill_tokens_per_step
self.kv_budget_mib = kv_budget_mib
self.events = []

def kv_mib(self, token_count):
    bytes_used = 2 * self.layers * token_count * self.kv_heads * self.head_dim * self.dtype_bytes
    return bytes_used / (1024 * 1024)

def run_request(self, request):
    prefill_steps = ceil(request.prompt_tokens / self.prefill_tokens_per_step)
    decode_steps = request.output_tokens
    ttft_steps = prefill_steps + 1
    tpot_steps = decode_steps / max(request.output_tokens - 1, 1)
    max_stream_backlog = 0
    backpressure_events = 0

    self.events.append((request.request_id, "prefill", request.prompt_tokens, prefill_steps))
    for step in range(1, request.output_tokens + 1):
        chunk = f"{request.request_id}_tok{step}"
        request.stream_chunks.append(chunk)
        self.events.append((request.request_id, "decode", step, chunk))
        if len(request.stream_chunks) > request.stream_buffer_capacity:
            backpressure_events += 1
            request.flushed_chunks.append(request.stream_chunks.pop(0))
        if step % request.client_read_every == 0:
            request.flushed_chunks.extend(request.stream_chunks)
            request.stream_chunks.clear()
        max_stream_backlog = max(max_stream_backlog, len(request.stream_chunks))

    request.flushed_chunks.extend(request.stream_chunks)
    request.stream_chunks.clear()
    kv_tokens = request.prompt_tokens + request.output_tokens
    return {
        "prompt_tokens": request.prompt_tokens,
        "output_tokens": request.output_tokens,
        "prefill_steps": prefill_steps,
        "decode_steps": decode_steps,
        "ttft_steps": ttft_steps,
        "tpot_steps": round(tpot_steps, 3),
        "kv_mib": round(self.kv_mib(kv_tokens), 3),
        "stream_chunks": len(request.flushed_chunks),
        "max_stream_backlog": max_stream_backlog,
        "backpressure_events": backpressure_events,
        "finish_reason": "length",
    }

def run(self, requests):
    per_request = {}
    for request in requests:
        per_request[request.request_id] = self.run_request(request)

    total_prompt = sum(item["prompt_tokens"] for item in per_request.values())
    total_output = sum(item["output_tokens"] for item in per_request.values())

```

```

total_kv_mib = round(sum(item["kv_mib"] for item in per_request.values()), 3)
max_ttft = max(item["ttft_steps"] for item in per_request.values())
max_tpot = max(item["tpot_steps"] for item in per_request.values())
return {
    "per_request": per_request,
    "metrics": {
        "total_prompt_tokens": total_prompt,
        "total_output_tokens": total_output,
        "sum_prefill_steps": sum(item["prefill_steps"] for item in per_request.values()),
        "sum_decode_steps": sum(item["decode_steps"] for item in per_request.values()),
        "p95_ttft_steps": max_ttft,
        "p95_tpot_steps": max_tpot,
        "total_kv_mib": total_kv_mib,
        "kv_pressure": round(total_kv_mib / self.kv_budget_mib, 3),
        "stream_events": sum(item["stream_chunks"] for item in per_request.values()),
        "backpressure_events": sum(item["backpressure_events"] for item in per_request.values()),
        "event_tail": self.events[-6:],
    },
}

```

```

def audit_phases(report, kv_pressure_limit=0.8):
    gates = {
        "prefill_accounting": all(item["prefill_steps"] > 0 for item in report["per_request"].values()),
        "decode_accounting": all(item["decode_steps"] == item["output_tokens"] for item in report["per_request"].values()),
        "kv_budget": report["metrics"]["kv_pressure"] <= kv_pressure_limit,
        "streaming_chunks": report["metrics"]["stream_events"] == report["metrics"]["total_output_tokens"],
        "cleanup_finish_reason": all(item["finish_reason"] for item in report["per_request"].values()),
    }
    return {
        "gates": gates,
        "phase_gate": all(gates.values()),
    }

```

```

requests = [
    ToyRequest("short_chat", prompt_tokens=512, output_tokens=4),
    ToyRequest("long_context", prompt_tokens=4096, output_tokens=3, client_read_every=2),
    ToyRequest("code_gen", prompt_tokens=2048, output_tokens=5),
]

```

```

audit_runner = MiniPrefillDecodeKVStreamingAudit()
report = audit_runner.run(requests)
audit = audit_phases(report)

```

```

print("per_request=", report["per_request"])
print("metrics=", report["metrics"])
print("audit=", audit)

```

这段代码的重点不是数值本身，而是验证四个阶段的因果链：prompt 越长，prefill steps 和 TTFT 越高；输出越长，decode steps 越多；活跃上下文越长，KV MiB 和 KV pressure 越高；streaming chunks 必须和输出 token、finish reason、cleanup 一起统计。

4.17 面试追问

1. Prefill 和 Decode 的区别是什么？

2. 为什么 Prefill 主要影响 TTFT, Decode 主要影响 TPOT?
3. KV Cache 为什么能加速自回归生成?
4. KV Cache 为什么会成为高并发 serving 的显存瓶颈?
5. Streaming 输出有哪些工程难点?
6. 为什么长 prompt 会拖慢首 token?
7. 为什么输出过程中可能一卡一卡?
8. 如果显存充足但 TPOT 很差, 你会从哪些方向排查?

参考回答思路:

1. 先定义 prefill、decode、KV cache 和 streaming。
2. 再说明资源画像: prefill 偏大计算, decode 更受 KV Cache 和调度影响。
3. 然后关联指标: prefill 对 TTFT 关键, decode 对 TPOT 关键, KV Cache 对并发关键, streaming 对感知体验关键。
4. 最后补工程 trade-off: scheduler 要在 TTFT、TPOT、吞吐、显存和公平性之间平衡。

4.18 小练习

1. 画出一个请求从 prefill 到 decode 再到 streaming 的流程图。
2. 假设 prompt 长度从 1k 增加到 8k, TTFT 可能发生什么变化? 为什么?
3. 假设并发从 10 增加到 100, KV Cache 会带来什么压力?
4. 写一段伪代码, 表示 decode 一轮后把 token 推送到 stream。
5. 解释为什么客户端断开后必须释放 KV Cache。

4.19 本章小结

本章讲清了 LLM serving 中最重要的四个执行概念。

Prefill 处理输入 prompt, 决定首 token 等待; Decode 逐 token 生成, 决定持续输出速度; KV Cache 复用历史 key/value, 决定显存和并发边界; Token Streaming 把生成结果增量返回给用户, 决定感知体验和连接生命周期。

下一章我们会把这些概念转成可量化指标, 系统讲 TTFT、TPOT、吞吐、并发、显存和成本之间的关系。

第 5 章 TTFT、TPOT、吞吐、并发、显存和成本

前几章讲了 serving engine 的生命周期和核心执行阶段。本章开始把这些机制转成可量化指标: TTFT、TPOT、吞吐、并发、显存和成本。

做 LLM serving, 不能只问 “QPS 多少”。大模型服务的性能是多维的: 用户关心等多久、输出是否顺滑; 平台关心能服务多少并发; 业务关心每百万 token 成本; 工程师关心显存、KV Cache、batch、调度和稳定性。

一句话概括:

LLM serving 的优化目标不是单一 QPS, 而是在 TTFT、TPOT、吞吐、并发、显存和成本之间做系统权衡。

5.0 本讲资料边界与第二轮精修口径

本章讲的是 LLM serving 的指标、容量和成本口径, 不是 PD 分离源码章节。第二轮精修时, 本章按下面口径处理:

1. TTFT、TPOT、queue、prefill、decode、KV cache、active sequences、tokens/s、metrics 和 tracing 的指标口径参考 vLLM、TGI、Triton 等公开 serving 抽象, 以及 SRE 对延迟分位、吞吐、饱和度和容量的通用工程思路。
2. 本章只给通用公式和 toy demo, 不绑定某个 runtime 的 Prometheus 指标名、dashboard 字段名、硬件型号或云厂商价格。

3. 成本估算只用于解释单位 token 成本、GPU-hour、有效吞吐、SLO 和 headroom 的关系，不把示例数值当作采购建议。
4. 并发只讨论 active sequence、waiting queue、KV budget 和 streaming connection 的工程含义，不把 HTTP 连接数等同于 serving 容量。
5. demo 用离线 toy workload 模拟 p95 / p99、tokens/s、KV pressure 和 cost per 1k tokens，帮助读者理解指标之间的约束。

所以，本章的目标不是追一个漂亮 QPS，而是让你能用同一张表解释用户体验、系统容量、显存预算和单位成本。

5.1 为什么传统 QPS 不够用

在传统 Web 服务里，我们经常看 QPS、平均延迟、p95 延迟、错误率。这些指标仍然重要，但不足以描述 LLM serving。

原因是 LLM 请求不是一次固定计算：

1. 输入 prompt 长度不同。
2. 输出 token 数不同。
3. 有的请求流式返回，有的请求一次性返回。
4. Prefill 和 decode 的资源画像不同。
5. KV Cache 会随上下文和并发动态增长。
6. 同一个请求会占用多轮 decode 调度。

两个请求的 QPS 都是 1，但成本可能完全不同：

请求 A：输入 50 tokens，输出 20 tokens

请求 B：输入 8000 tokens，输出 2000 tokens

如果只看请求数，A 和 B 一样。如果看 token，它们差几个数量级。

所以 LLM serving 必须转向 token 视角和阶段视角。

5.2 TTFT: Time To First Token

TTFT 是 time to first token，表示从客户端发起请求，到收到第一个输出 token 的时间。

可以拆成：

TTFT = 网络接入时间 + 排队时间 + tokenization 时间 + prefill 时间 + 首 token 采样与返回时间

其中最常见的大头是排队时间和 prefill 时间。

TTFT 对用户体验非常关键。因为用户点下发送后，第一反应是看系统有没有响应。即使最终答案生成很快，如果首 token 等了很久，用户也会觉得服务慢。

影响 TTFT 的因素包括：

1. prompt 长度。
2. waiting queue 长度。
3. prefill batch 策略。
4. 是否有 prefix cache 命中。
5. 是否启用 chunked prefill。
6. GPU 是否繁忙。
7. tokenizer 和 chat template 处理开销。
8. 调度策略是否优先新请求。

5.3 如何优化 TTFT

优化 TTFT 的常见方向：

1. 控制排队时间。
2. 对长 prompt 做 chunked prefill。

3. 使用 prefix cache 复用共享前缀。
4. 限制超长输入，避免单请求拖垮队列。
5. 优化 tokenization 和请求预处理。
6. 在 scheduler 中给 waiting 请求一定机会。
7. 使用独立 prefill 资源池，减少和 decode 互相干扰。

但 TTFT 优化也有代价。

如果过度优先新请求 prefill，正在 decode 的请求可能输出变慢，TPOT 变差。用户会看到首 token 很快，但后续一卡一卡。

所以 TTFT 不能孤立优化，要和 TPOT 一起看。

5.4 TPOT: Time Per Output Token

TPOT 是 time per output token，表示生成阶段每个输出 token 的平均耗时。

简化计算：

TPOT = decode 总耗时 / 输出 token 数

如果一个请求首 token 后又生成了 100 个 token，decode 阶段耗时 5 秒，那么平均 TPOT 是 50ms/token。

TPOT 反映持续输出速度。流式场景下，TPOT 越低，用户看到的输出越快。TPOT 抖动越小，输出越平滑。

影响 TPOT 的因素包括：

1. decode batch 大小。
2. KV Cache 读取效率。
3. 上下文长度。
4. attention kernel 和模型 kernel。
5. sampling 开销。
6. scheduler 每轮开销。
7. streaming 输出是否阻塞。
8. GPU 显存带宽和计算能力。

5.5 如何优化 TPOT

优化 TPOT 的常见方向：

1. 使用 continuous batching，提高每轮 decode 的有效 batch。
2. 优化 KV Cache 布局，减少碎片和无效访问。
3. 使用高效 attention kernel。
4. 降低 scheduler 每轮开销。
5. 避免网络 streaming 阻塞模型执行。
6. 对长上下文使用更合适的 cache 或 attention 策略。
7. 使用 speculative decoding 减少主模型 decode 轮数。

TPOT 优化的代价也很明显。

如果为了提高 decode 吞吐，让 batch 尽量大，可能会让新请求等待更久，TTFT 变差。如果为了减少每轮调度开销，把调度粒度做粗，也可能降低响应灵活性。

5.6 吞吐：tokens/s 比 QPS 更重要

LLM serving 的吞吐通常用 tokens/s 衡量，而不是只看 QPS。

常见吞吐指标包括：

1. input tokens/s：每秒处理多少输入 token。
2. output tokens/s：每秒生成多少输出 token。
3. total tokens/s：输入和输出 token 总量。

4. per-GPU tokens/s: 每张 GPU 的 token 吞吐。
5. accepted tokens/s: 如果使用 speculative decoding, 还会看被接受 token 的吞吐。

为什么 tokens/s 重要? 因为 LLM 的成本和计算量主要跟 token 相关。

同样 10 QPS:

场景 A: 每个请求输入 100, 输出 50, 总计 1500 tokens/s

场景 B: 每个请求输入 4000, 输出 1000, 总计 50000 tokens/s

QPS 一样, 系统压力完全不同。

面试里如果候选人只说“系统支持 100 QPS”, 但不说明输入输出 token 分布、上下文长度和延迟目标, 这个数字基本没有意义。

5.7 并发: 不是连接数那么简单

并发在 LLM serving 中至少有三种含义:

1. 同时连接的客户端数量。
2. waiting queue 中等待的请求数量。
3. running set 中正在占用 KV Cache 和 decode slot 的请求数量。

真正影响 engine 的, 主要是 active sequences, 也就是正在被调度或占用 KV Cache 的序列数。

高并发会带来几个压力:

1. waiting queue 增长, TTFT 变差。
2. running sequences 增多, KV Cache 占用变大。
3. decode batch 变大, 吞吐可能提高但单请求延迟可能变差。
4. scheduler 决策更复杂。
5. streaming 连接和网络输出压力上升。

所以并发不是越高越好。一个系统标称支持 1000 并发, 如果 p99 TTFT 已经不可接受, 就不算有效并发。

5.8 显存: 权重、KV Cache 和临时 buffer

LLM serving 的显存占用通常包括:

1. 模型权重。
2. KV Cache。
3. activation 或临时计算 buffer。
4. CUDA graph 或 kernel workspace。
5. 通信 buffer, 比如 tensor parallel 场景。
6. 框架运行时开销。

其中模型权重相对固定, KV Cache 随请求动态变化。

一个常见错误是只估算模型权重:

7B 模型, FP16 权重大约 14GB, 所以 24GB GPU 肯定够。

这个判断不完整。剩余显存还要容纳 KV Cache、临时 buffer 和运行时开销。上下文长、并发高时, KV Cache 可能很快吃掉全部剩余空间。

显存上限会反过来限制:

1. 最大并发序列数。
2. 最大上下文长度。
3. 最大 batch token 数。
4. 是否能启用更大的 prefill batch。
5. 是否需要量化、分片或 offload。

5.9 成本：每 token 成本才是核心

推理成本通常可以用每百万 token 成本来估算。

粗略公式：

每 token 成本 = 单位时间 GPU 成本 / 单位时间有效 token 吞吐

如果一张 GPU 每小时成本固定，那么 tokens/s 越高，每 token 成本越低。但这并不意味着只追求最大吞吐。

原因是业务还要满足延迟 SLO。

一个系统可以通过大 batch 把 tokens/s 做高，但 TTFT 和 TPOT 变差，用户体验无法接受。另一个系统延迟很好，但 GPU 利用率太低，成本无法承受。

成本优化的目标是：在满足 SLO 的前提下，提高有效吞吐和资源利用率。

常见手段包括：

1. continuous batching。
2. prefix cache。
3. KV Cache 高效管理。
4. 模型量化。
5. speculative decoding。
6. 请求路由到合适大小的模型。
7. 自动扩缩容。
8. 不同优先级请求分层服务。

5.10 指标之间的冲突

这些指标经常互相冲突。

常见冲突包括：

1. 大 batch 提高吞吐，但可能增加 TTFT。
2. 优先 prefill 降低 TTFT，但可能让 decode 变慢。
3. 优先 decode 保证输出顺滑，但新请求排队更久。
4. 提高并发可以摊薄成本，但 KV Cache 显存压力上升。
5. 限制上下文长度降低成本，但影响业务能力。
6. 量化降低显存和成本，但可能影响质量或引入兼容问题。
7. prefix cache 提升重复 prompt 性能，但需要额外 cache 管理和失效策略。

所以性能调优不是单指标优化，而是多目标优化。

你需要先定义目标：

1. 是聊天助手，重视 TTFT 和输出平滑。
2. 是离线批处理，重视总 tokens/s 和成本。
3. 是代码生成，可能重视长输出 TPOT 和稳定性。
4. 是 RAG 问答，可能重视长 prompt prefill 和 prefix cache。
5. 是 agent 场景，可能重视多轮短请求和工具调用延迟。

场景不同，调优方向不同。

5.11 p50、p95、p99 为什么重要

平均值经常掩盖问题。

例如平均 TTFT 是 800ms，但 p99 TTFT 是 15s，说明有少量用户体验非常差。

LLM serving 中长尾延迟很常见，原因包括：

1. prompt 长度分布长尾。
2. 输出长度分布长尾。

3. 某些请求等待 KV Cache。
4. scheduler 对长请求不公平。
5. GPU 上出现短时拥塞。
6. streaming 连接阻塞。
7. tokenizer 或预处理有异常慢样本。

因此线上指标至少要看：

1. p50 TTFT、p95 TTFT、p99 TTFT。
2. p50 TPOT、p95 TPOT、p99 TPOT。
3. p95 total latency。
4. p99 queue waiting time。
5. p99 prefill latency。
6. p99 decode iteration latency。

面向生产，p99 往往比平均值更能反映系统质量。

5.12 如何做压测

LLM serving 压测不能只发固定短 prompt。

合理压测应该定义 workload：

1. 输入 token 长度分布。
2. 输出 token 长度分布。
3. 请求到达模式，比如恒定 QPS、突发流量、泊松到达。
4. 是否 streaming。
5. 并发上限。
6. 采样参数。
7. 是否有共享 prefix。
8. SLO 阈值。

压测输出要包含：

1. TTFT 分位数。
2. TPOT 分位数。
3. tokens/s。
4. active sequences。
5. waiting queue 长度。
6. KV Cache 使用率。
7. GPU 利用率和显存。
8. 错误率和超时率。
9. 每百万 token 成本估算。

如果压测报告只给 QPS 和平均延迟，对 LLM serving 来说是不够的。

5.13 面试中如何讲性能指标

面试官问 “如何评估一个大模型推理服务性能”，不要只答 QPS。

推荐回答结构：

1. 先说用户侧指标：TTFT、TPOT、总延迟、流式平滑度。
2. 再说系统侧指标：input/output tokens/s、active sequences、queue length、GPU 利用率、KV Cache 使用率。
3. 再说稳定性指标：p95/p99、错误率、超时率、OOM 次数。
4. 最后说成本指标：每百万 token 成本、每 GPU tokens/s、SLO 下的资源利用率。

这样回答能体现你理解 LLM serving 的特殊性，而不是把它当普通 HTTP 服务。

5.14 常见误区

误区一：QPS 越高，系统越好。

如果请求 token 很短，QPS 高不代表能处理长 prompt；如果 p99 延迟很差，QPS 高也没有意义。

误区二：GPU 利用率越高越好。

GPU 利用率高可能伴随严重排队，用户体验很差。要结合 TTFT、TPOT 和 SLO 看。

误区三：吞吐和延迟可以同时无限优化。

通常不可能。大 batch 提升吞吐，往往增加等待；低延迟策略可能牺牲吞吐和成本。

误区四：显存只由模型大小决定。

KV Cache、临时 buffer、并发和上下文长度都会影响显存。

误区五：平均延迟足够说明问题。

LLM 请求长度分布长尾明显，必须看 p95 和 p99。

5.15 指标公式、容量估算和可运行 demo

可以把一次请求的指标记录写成：

$R_i = (x_i, y_i, q_i, p_i, d_i, k_i, c_i, e_i, z_i)$

其中 x_i 是 input tokens, y_i 是 output tokens, q_i 是 queue time, p_i 是 prefill time, d_i 是 decode time, k_i 是 KV cache 占用, c_i 是成本, e_i 是错误或 timeout 标记, z_i 是 trace 和版本信息。

TTFT 可以拆成：

$T_{\{\mathrm{ttft}\}, i} = T_{\{\mathrm{queue}\}, i} + T_{\{\mathrm{tokenize}\}, i} + T_{\{\mathrm{prefill}\}, i} + T_{\{\mathrm{first}\}, i}$

TPOT 可以写成：

$T_{\{\mathrm{tpot}\}, i} = \frac{T_{\{\mathrm{decode}\}, i}}{\max(y_i - 1, 1)}$

一个窗口 W 内的 input / output token 吞吐可以写成：

$X_{\{\mathrm{in}\}} = \frac{\sum_i x_i}{W}$

$X_{\{\mathrm{out}\}} = \frac{\sum_i y_i}{W}$

KV cache 显存仍然要按活跃 token 估算：

$M_{\{\mathrm{kv}\}} = 2 \left(\sum_i (x_i + y_i) \right) H_{\{\mathrm{kv}\}} D_{\mathrm{hb}}$

如果平均每个 active sequence 占用 M_{kavg} 显存，可承载并发上限可以粗略写成：

$N_{\{\mathrm{active}\}} \leq \left\lfloor \frac{B_{\{\mathrm{kv}\}}}{M_{\{\mathrm{kavg}\}}} \right\rfloor$

单位 token 成本可以按 GPU-hour 和有效 token 吞吐估算：

$K_{\{1k\}} = \frac{1000 K_{\{\mathrm{gpu}\}}}{3600 X_{\{\mathrm{tok}\}}}$

其中 K_{gpu} 是每 GPU-hour 成本， X_{tok} 是每秒有效 token 吞吐。最终指标门禁可以写成：

$G_{\{\mathrm{metric}\}} = G_{\{\mathrm{ttft}\}} G_{\{\mathrm{tpot}\}} G_{\{\mathrm{kv}\}} G_{\{\mathrm{cost}\}} G_{\{\mathrm{tail}\}}$

下面这个 demo 用 0 依赖代码模拟一个压测窗口，输出 p95 / p99、input / output tokens/s、KV pressure、并发容量和每 1k token 成本。它不模拟真实 GPU，只把指标口径写清楚。

```
from math import floor
```

```
def percentile(values, pct):
    if not values:
        return None
```

```

ordered = sorted(values)
index = min(len(ordered) - 1, max(0, round((pct / 100) * (len(ordered) - 1))))
return ordered[index]

```

```

class MiniServingMetricCostAudit:

```

```

    def __init__(
        self,
        layers=32,
        kv_heads=8,
        head_dim=128,
        dtype_bytes=2,
        kv_budget_mib=8192,
        gpu_cost_per_hour=2.4,
        window_seconds=60,
    ):
        self.layers = layers
        self.kv_heads = kv_heads
        self.head_dim = head_dim
        self.dtype_bytes = dtype_bytes
        self.kv_budget_mib = kv_budget_mib
        self.gpu_cost_per_hour = gpu_cost_per_hour
        self.window_seconds = window_seconds

```

```

    def kv_mib(self, input_tokens, output_tokens):
        tokens = input_tokens + output_tokens
        bytes_used = 2 * self.layers * tokens * self.kv_heads * self.head_dim * self.dtype_bytes
        return bytes_used / (1024 * 1024)

```

```

    def per_request_metrics(self, request):
        ttft_ms = request["queue_ms"] + request["tokenize_ms"] + request["prefill_ms"] + request["first_flush_ms"]
        tpot_ms = request["decode_ms"] / max(request["output_tokens"] - 1, 1)
        e2e_ms = ttft_ms + request["decode_ms"] + request["final_flush_ms"]
        kv_mib = self.kv_mib(request["input_tokens"], request["output_tokens"])
        return {
            "request_id": request["request_id"],
            "input_tokens": request["input_tokens"],
            "output_tokens": request["output_tokens"],
            "ttft_ms": round(ttft_ms, 3),
            "tpot_ms": round(tpot_ms, 3),
            "e2e_ms": round(e2e_ms, 3),
            "kv_mib": round(kv_mib, 3),
            "timeout": request.get("timeout", False),
        }

```

```

    def run(self, requests):
        per_request = [self.per_request_metrics(request) for request in requests]
        total_input = sum(item["input_tokens"] for item in per_request)
        total_output = sum(item["output_tokens"] for item in per_request)
        total_tokens = total_input + total_output
        total_kv_mib = sum(item["kv_mib"] for item in per_request)
        active_kv_avg = total_kv_mib / max(len(per_request), 1)
        effective_tokens_per_s = total_tokens / self.window_seconds
        cost_per_1k_tokens = 1000 * self.gpu_cost_per_hour / (3600 * max(effective_tokens_per_s, 1e-9))
        return {

```

```

    "per_request": per_request,
    "summary": {
        "request_count": len(per_request),
        "input_tokens": total_input,
        "output_tokens": total_output,
        "input_tokens_per_s": round(total_input / self.window_seconds, 3),
        "output_tokens_per_s": round(total_output / self.window_seconds, 3),
        "total_tokens_per_s": round(effective_tokens_per_s, 3),
        "ttft_p95_ms": percentile([item["ttft_ms"] for item in per_request], 95),
        "ttft_p99_ms": percentile([item["ttft_ms"] for item in per_request], 99),
        "tpot_p95_ms": percentile([item["tpot_ms"] for item in per_request], 95),
        "e2e_p99_ms": percentile([item["e2e_ms"] for item in per_request], 99),
        "kv_mib_total": round(total_kv_mib, 3),
        "kv_pressure": round(total_kv_mib / self.kv_budget_mib, 3),
        "active_sequence_capacity": floor(self.kv_budget_mib / max(active_kv_avg, 1e-9)),
        "cost_per_1k_tokens": round(cost_per_1k_tokens, 6),
        "timeout_rate": round(sum(item["timeout"] for item in per_request) / max(len(per_request), 1), 3),
    },
}

```

```

def audit_serving_metrics(report):
    summary = report["summary"]
    gates = {
        "ttft_slo": summary["ttft_p95_ms"] <= 900,
        "tpot_slo": summary["tpot_p95_ms"] <= 80,
        "token_throughput": summary["total_tokens_per_s"] > 0,
        "kv_budget": summary["kv_pressure"] <= 0.8,
        "cost_budget": summary["cost_per_1k_tokens"] <= 0.02,
        "tail_latency": summary["e2e_p99_ms"] <= 4000,
        "timeout_rate": summary["timeout_rate"] <= 0.01,
    }
    return {
        "gates": gates,
        "metric_cost_gate": all(gates.values()),
    }

```

```

workload = [
    {"request_id": "chat_1", "input_tokens": 180, "output_tokens": 80, "queue_ms": 80, "tokenize_ms": 8, "prefill_ms": 8},
    {"request_id": "rag_1", "input_tokens": 2200, "output_tokens": 160, "queue_ms": 180, "tokenize_ms": 25, "prefill_ms": 25},
    {"request_id": "code_1", "input_tokens": 900, "output_tokens": 420, "queue_ms": 120, "tokenize_ms": 16, "prefill_ms": 16},
    {"request_id": "long_1", "input_tokens": 6000, "output_tokens": 220, "queue_ms": 280, "tokenize_ms": 55, "prefill_ms": 55},
    {"request_id": "chat_2", "input_tokens": 120, "output_tokens": 60, "queue_ms": 60, "tokenize_ms": 7, "prefill_ms": 7},
]

```

```

audit_runner = MiniServingMetricCostAudit()
report = audit_runner.run(workload)
audit = audit_serving_metrics(report)

```

```

print("per_request=", report["per_request"])
print("summary=", report["summary"])
print("audit=", audit)

```

这段代码的重点是把压测报告从“QPS + 平均延迟”升级成 token 和阶段视角：同一个窗口里，TTFT、TPOT、tokens/s、KV pressure、tail latency、timeout 和 cost per 1k tokens 要同时过线。

5.16 面试追问

1. TTFT 和 TPOT 分别是什么?
2. 为什么 LLM serving 不能只看 QPS?
3. input tokens/s 和 output tokens/s 有什么区别?
4. 为什么 KV Cache 会限制最大并发?
5. 如何估算每 token 推理成本?
6. 大 batch 对吞吐和延迟分别有什么影响?
7. 线上 TTFT p99 突然升高, 你会怎么排查?
8. 如果 GPU 利用率很高但用户体验差, 可能是什么原因?

参考回答思路:

1. 先定义指标: TTFT、TPOT、tokens/s、并发、显存、成本。
2. 再说明 LLM 请求长度不固定, 必须按 token 和阶段分析。
3. 然后解释 trade-off: 吞吐、延迟、显存和成本互相制约。
4. 最后给排查路径: queue、prefill、decode、KV Cache、GPU、streaming、p95/p99。

5.17 小练习

1. 设计一个压测 workload, 包含输入长度、输出长度、QPS、并发和是否 streaming。
2. 假设某服务 TTFT 高但 TPOT 正常, 你会优先排查哪些阶段?
3. 假设 TPOT 高但 TTFT 正常, 你会优先排查哪些阶段?
4. 解释为什么 batch size 增大可能降低每 token 成本但增加首 token 延迟。
5. 画一张图, 把 TTFT、TPOT、tokens/s、KV Cache 使用率和成本联系起来。

5.18 本章小结

本章把 LLM serving 的核心机制转成了可量化指标。

TTFT 衡量首 token 等待, 主要受排队和 prefill 影响; TPOT 衡量持续生成速度, 主要受 decode、KV Cache 和调度影响; tokens/s 比 QPS 更能反映真实吞吐; 并发会放大 KV Cache 和调度压力; 显存不只由模型权重决定, 还受 KV Cache 和临时 buffer 影响; 成本优化要在满足 SLO 的前提下提高有效 token 吞吐。

下一章我们会回到系统边界, 讲清推理框架、推理平台和 AI Infra 的区别, 避免把 engine、平台治理和基础设施混在一起。

第 6 章 推理框架、推理平台和 AI Infra 的边界

前五章讲的是 LLM serving engine 内部: 请求生命周期、prefill、decode、KV Cache、streaming 和性能指标。本章把视角拉高, 讲清三个容易混在一起的概念: 推理框架、推理平台和 AI Infra。

很多团队讨论大模型推理时, 会把 vLLM、Kubernetes、模型发布、灰度、监控、GPU 调度、限流、API 网关都混在一起说。这样会导致系统边界不清, 设计时不知道问题应该在哪一层解决。

一句话概括:

推理框架负责把模型高效执行起来, 推理平台负责把模型服务稳定运营起来, AI Infra 负责提供算力、存储、网络、调度和治理等基础能力。

6.0 本讲资料边界与第二轮精修口径

本章第二轮精修前, 先用公开资料校准边界口径: vLLM 等 LLM serving engine 的公开架构通常围绕 API 入口、engine core、scheduler、KV Cache manager 和 worker 执行路径展开; Triton Inference Server 这类通用推理服务强调 model repository、scheduler、backend、metrics 和 tracing; Kubernetes 文档把资源 request / limit、Pod 调度和扩展资源管理放在集群资源治理层; SRE 体系把 SLO、error budget、容量和告警作为服务运营能力。

因此，本章只抽象稳定边界，不把某个框架版本的内部类名、Prometheus 指标名、Kubernetes 字段或云厂商产品写成标准答案。读者需要掌握的是：engine 负责 token 执行路径，platform 负责服务治理控制面，AI Infra 负责底层资源和运行环境；三层之间通过请求、指标、控制、健康检查和 trace 接口协作。

第二轮补充重点有三点：

1. 把“谁负责什么”写成可检查的职责向量和覆盖率公式。
2. 把平台和 engine、infra 之间的接口、指标交接和事故归层写成可验收门禁。
3. 增加一个 0 依赖 Python demo，让读者能审计 toy 推理系统是否把职责、接口、指标、事故和成本边界说清楚。

6.1 为什么边界重要

边界不清会带来很多工程问题。

例如，线上 TTFT 变差，可能原因很多：

1. engine scheduler 策略有问题。
2. KV Cache 不足导致请求等待。
3. 平台限流或网关排队。
4. Kubernetes 调度导致实例不足。
5. GPU 节点性能异常。
6. 上游请求突然变长。

如果不区分层次，就会出现错误修复方向：明明是平台扩容慢，却去调 vLLM 参数；明明是 engine KV Cache 不够，却去改 API 网关；明明是模型路由不合理，却去升级 GPU 驱动。

系统边界能帮助我们明确：

1. 谁负责模型执行。
2. 谁负责服务治理。
3. 谁负责资源供给。
4. 谁负责观测和排障。
5. 谁负责成本和容量。

6.2 三层结构总览

可以把大模型在线推理系统简化成三层：

```
业务应用 / 产品
|
推理平台 Inference Platform
|
推理框架 Serving Engine
|
AI Infra / GPU Infrastructure
```

更具体一些：

Inference Platform:

API gateway, auth, quota, routing, rollout, billing, observability

Serving Engine:

request queue, scheduler, prefill/decode, KV cache, sampling, streaming

AI Infra:

GPU cluster, Kubernetes, network, storage, image, driver, monitoring

三者不是完全隔离的。平台需要读取 engine 指标，engine 依赖 infra 提供 GPU，infra 的资源约束会影响 engine 调度。但职责边界要清楚。

6.3 推理框架负责什么

推理框架，也就是 serving engine，负责模型执行路径。

它的核心职责包括：

1. 加载模型权重。
2. 管理 tokenizer 和模型配置。
3. 接收内部格式的生成请求。
4. 维护 waiting queue 和 running set。
5. 执行 prefill 和 decode。
6. 管理 KV Cache。
7. 实现 continuous batching。
8. 执行 sampling。
9. 支持 token streaming。
10. 暴露性能指标。

典型项目包括 vLLM、SGLang、TGI，以及一些基于 TensorRT-LLM、Triton、自研 runtime 的 engine。

推理框架关注的问题是：

1. 如何提高 tokens/s。
2. 如何降低 TTFT 和 TPOT。
3. 如何减少 KV Cache 碎片。
4. 如何支持长上下文。
5. 如何实现高效 sampling 和 structured generation。
6. 如何让 GPU 在多请求下保持高效执行。

6.4 推理框架不应该负责什么

推理框架不应该承担所有平台治理能力。

例如：

1. 全局租户计费。
2. 跨集群容量规划。
3. 复杂灰度发布策略。
4. 多模型业务路由。
5. 企业权限系统。
6. 长期审计日志留存。
7. GPU 集群资源采购和配额。

这些能力放进 engine 会让 engine 变得臃肿，也会让执行路径不稳定。

好的 engine 应该把关键指标和控制接口暴露出来，让平台层使用，而不是自己变成一个完整业务平台。

当然，一些开源 engine 会内置 HTTP server、OpenAI-compatible API、基础 metrics。这些是为了易用性，不代表它应该承担全部平台职责。

6.5 推理平台负责什么

推理平台负责把模型服务稳定地对外提供。

它通常位于业务应用和 serving engine 之间。

核心职责包括：

1. API 入口和协议适配。
2. 鉴权、租户、配额和限流。
3. 模型注册和版本管理。
4. 模型路由和负载均衡。
5. 灰度发布、回滚和 A/B 实验。

6. 多模型、多实例、多区域管理。
7. 统一日志、metrics 和 trace。
8. 成本统计和计费。
9. SLO 管理和告警。
10. 降级、熔断和重试策略。

如果说 engine 关注 “一个模型实例怎么跑得快”，平台关注的是 “一组模型服务怎么稳定、可控、可治理地对外提供”。

6.6 推理平台和 engine 的接口

平台和 engine 之间通常通过几个接口协作。

请求接口：

platform -> engine: generate / chat / embeddings / rerank

指标接口：

engine -> platform: TTFT, TPOT, tokens/s, queue length, KV cache usage, errors

控制接口：

platform -> engine: drain, reload, pause admission, update config

健康检查接口：

platform -> engine: readiness, liveness, warmup status

这些接口决定了平台能否正确治理 engine。

例如，平台做扩缩容时，如果只看 GPU 利用率，可能不够；还需要看 queue length、KV Cache 使用率、TTFT p95、TPOT p95。平台做灰度发布时，需要 engine 支持优雅下线，避免正在 decode 的请求被直接杀掉。

6.7 AI Infra 负责什么

AI Infra 是更底层的基础设施层，负责算力、存储、网络、调度和运行环境。

核心职责包括：

1. GPU 节点管理。
2. Kubernetes 或其他集群调度。
3. 容器镜像和运行时。
4. NVIDIA driver、CUDA、NCCL 等依赖。
5. 模型权重存储和分发。
6. 网络和负载均衡基础能力。
7. 分布式存储、对象存储和缓存。
8. 节点监控和硬件故障处理。
9. 多租户资源隔离。
10. 成本、配额和容量规划。

AI Infra 更关心：GPU 有没有、放在哪里、怎么调度、怎么隔离、怎么监控、坏了怎么恢复、成本怎么控制。

第二十三册已经系统讲过 AI Infra。本书只在 serving engine 需要时引用它。

6.8 三层各自的指标

不同层关注的指标不同。

Serving Engine 指标：

1. TTFT。
2. TPOT。

3. prefill latency。
4. decode iteration latency。
5. tokens/s。
6. active sequences。
7. KV Cache usage。
8. scheduler queue length。

Inference Platform 指标：

1. 请求成功率。
2. p95/p99 端到端延迟。
3. 每模型 QPS 和 tokens/s。
4. 租户配额使用。
5. 路由命中率。
6. 灰度版本错误率。
7. 限流和降级次数。
8. 每百万 token 成本。

AI Infra 指标：

1. GPU 利用率。
2. GPU 显存使用率。
3. 节点健康状态。
4. 网络带宽和丢包。
5. 存储读取延迟。
6. Pod 调度等待时间。
7. 镜像拉取时间。
8. 集群资源碎片率。

排障时，要把指标放回对应层次，而不是混在一起。

6.9 一个例子：TTFT 变差怎么分层排查

假设线上 TTFT p99 从 2 秒变成 15 秒。

先看平台层：

1. 请求量是否突增。
2. 输入 token 长度是否变长。
3. 是否有某个租户打满配额。
4. 路由是否集中到少数实例。
5. 是否刚做灰度发布。

再看 engine 层：

1. waiting queue 是否变长。
2. prefill latency 是否升高。
3. KV Cache 是否接近满。
4. scheduler 是否过度优先 decode。
5. prefix cache 命中率是否下降。

最后看 infra 层：

1. GPU 是否降频或异常。
2. 节点是否有硬件错误。
3. 网络是否拥塞。
4. Pod 是否频繁重启。
5. 是否扩容失败或调度等待过长。

这样排查比“感觉 vLLM 慢了”有效得多。

6.10 一个例子：成本过高怎么分层优化

假设业务反馈每百万 token 成本太高。

平台层可以做：

1. 模型路由，把简单请求路由到小模型。
2. 结果缓存或语义缓存。
3. 限制不合理的超长输出。
4. 按租户和业务做成本报表。
5. 非实时请求转批处理或低优先级队列。

Engine 层可以做：

1. continuous batching 参数调优。
2. KV Cache block size 调优。
3. prefix cache。
4. 量化推理。
5. speculative decoding。
6. 更高效 kernel。

Infra 层可以做：

1. 选择更合适的 GPU 型号。
2. 提升资源利用率。
3. 混部低优先级任务。
4. 优化镜像和模型加载时间。
5. 做容量规划和弹性伸缩。

成本优化通常不是单层能解决的，需要平台、engine 和 infra 协同。

6.11 常见架构反模式

反模式一：把所有治理能力塞进 engine。

结果是 engine 越来越复杂，执行路径变重，升级困难。

反模式二：平台完全不理解 engine 指标。

结果是扩缩容、路由和限流都只能看粗粒度 QPS，无法针对 LLM serving 的真实瓶颈做决策。

反模式三：Infra 只提供 GPU，不关心模型服务特征。

结果是调度和容量规划不匹配长上下文、KV Cache、模型加载和多租户隔离。

反模式四：用普通 HTTP 服务思路设计 LLM serving。

结果是忽略 token 长度、流式输出、KV Cache、动态 batch 和长尾延迟。

反模式五：所有模型共用一套固定参数。

不同模型大小、上下文长度、业务 SLO 和流量分布不同，engine 参数和平台策略也应该不同。

6.12 面试中如何讲边界

如果面试官让你“设计一个大模型推理服务平台”，不要一上来只说 vLLM 部署。

可以按三层回答：

1. Serving Engine：负责模型执行，包括 prefill、decode、KV Cache、scheduler、streaming、metrics。
2. Inference Platform：负责接入、鉴权、路由、限流、灰度、观测、成本和 SLO。
3. AI Infra：负责 GPU 集群、容器、调度、存储、网络、驱动和资源隔离。

然后说明三层接口：

1. 请求接口。
2. 指标接口。
3. 控制接口。
4. 健康检查接口。

最后给出排障和优化例子，例如 TTFT 变差如何分层排查，成本过高如何分层优化。

这样的回答能体现你不仅懂框架，还懂生产系统边界。

6.13 本书后续章节如何对应三层

本书的重点是 serving engine，但会和平台、infra 有交叉。

后续第二部分讲从 0 实现最小推理框架，主要在 engine 层。

第三部分讲 vLLM，重点是 engine 的调度、KV Cache 和 executor。

第四部分讲 SGLang，重点是 runtime、prefix sharing、structured generation 和 agent serving。

第五部分讲 PD 分离，会跨 engine 和 infra，因为涉及资源池、KV Cache 迁移和网络。

第六部分讲教学项目源码路线，帮助你从小框架过渡到真实 engine。

第七部分讲系统设计、debug 和面试，会同时覆盖 engine、platform 和 infra 的交界面。

6.14 常见误区

误区一：会部署 vLLM 就等于懂推理平台。

部署只是开始。平台还要处理路由、灰度、限流、观测、成本、SLO 和多租户。

误区二：推理框架负责所有线上稳定性。

engine 负责执行稳定性，但平台治理和 infra 稳定性同样关键。

误区三：AI Infra 只要给 GPU 就行。

LLM serving 对显存、网络、模型加载、驱动、调度和隔离都有特殊要求。

误区四：平台层不需要理解 prefill 和 decode。

平台如果不理解这些指标，就无法做精细扩缩容、路由和 SLO 管理。

误区五：边界越清楚，系统越割裂。

正确做法是职责清楚、接口清楚、指标打通，而不是各层互不理解。

6.15 边界职责、接口契约和可运行 demo

可以把一个推理系统的边界对象写成：

$$B_i = (\ell_i, r_i, I_i, M_i, K_i, S_i)$$

其中， ℓ_i 是所属层级， r_i 是职责集合， I_i 是对外接口集合， M_i 是指标集合， K_i 是成本和容量证据， S_i 是事故排查证据。这个式子的作用不是追求形式化，而是提醒你：一个模块不能只说“我是 engine / platform / infra”，还要说明它暴露什么接口、交接什么指标、承担什么成本容量责任、事故时留下什么证据。

层级职责覆盖率可以写成：

$$C_{\{\mathrm{layer}\}} = \frac{N_{\{\mathrm{covered}\}}}{N_{\{\mathrm{required}\}}}$$

N_{required} 是三层系统应该覆盖的关键职责数， N_{covered} 是当前设计中有明确 owner、输入、输出和验收方式的职责数。只画了架构图但没有 owner 和验收指标，不应该算 covered。

平台和 engine / infra 的接口契约覆盖率可以写成：

$$C_{\{\mathrm{iface}\}} = \frac{N_{\{\mathrm{iface, ready}\}}}{N_{\{\mathrm{iface, total}\}}}$$

常见接口包括 request API、metrics export、control drain、health probe 和 trace context。平台要做灰度、扩容和故障切流，不能只知道一个 HTTP endpoint，还要知道 engine 是否能优雅 drain，metrics 是否能解释 TTFT / TPOT / KV pressure，健康检查是否能区分 warmup、ready 和 overload。

指标交接覆盖率可以写成：

$$C_{\{\mathrm{metric}\}} = \frac{N_{\{\mathrm{metric}, \mathrm{handoff}\}}}{N_{\{\mathrm{metric}, \mathrm{required}\}}}$$

例如 engine 要向 platform 交接 TTFT、TPOT、queue length、KV pressure、tokens/s；platform 要向 engine 交接 tenant、priority、deadline、token budget；infra 要向 platform / engine 交接 GPU health、Pod pending、memory pressure、image pull 和 node readiness。指标没有交接，排障时就只能猜。

错误归层率可以写成：

$$R_{\{\mathrm{misroute}\}} = \frac{N_{\{\mathrm{misrouted}\}}}{N_{\{\mathrm{incidents}\}}}$$

如果 TTFT p99 变差时只怀疑 engine，却没有同时检查平台流量分布、prompt 长度、扩容状态、Pod 调度和 GPU 健康，就属于高风险误归层。

最后可以把边界验收写成一个乘法门禁：

$$G_{\{\mathrm{boundary}\}} = G_{\{\mathrm{role}\}} G_{\{\mathrm{iface}\}} G_{\{\mathrm{metric}\}} G_{\{\mathrm{incident}\}} G_{\{\mathrm{cost}\}}$$

任何一个门禁为 0，说明三层边界还不能支撑真实生产排障。面试中这比背工具名更有说服力。

下面的 0 依赖 demo 用 toy capabilities 和 toy incidents 审计三层边界。它故意放入一个把 TTFT 问题错误归因到 engine 的 bad case，让最终 gate 失败，帮助你看到“层级名词完整”不等于“事故交接完整”。

```
from collections import Counter
```

```
REQUIRED_BY_LAYER = {
    "engine": {
        "model_load",
        "request_queue",
        "scheduler",
        "prefill_decode",
        "kv_cache",
        "sampling_streaming",
        "engine_metrics",
        "resource_cleanup",
    },
    "platform": {
        "api_gateway",
        "auth_quota",
        "model_registry",
        "routing_rollout",
        "slo_alert",
        "cost_attribution",
        "observability",
        "degrade_retry",
    },
    "infra": {
        "gpu_cluster",
        "container_runtime",
        "driver_cuda",
        "pod_scheduling",
        "network_storage",
        "resource_isolation",
        "node_health",
        "capacity_planning",
    },
}
```

```

    },
}

REQUIRED_INTERFACES = {
    "request_api",
    "metrics_export",
    "control_drain",
    "health_probe",
    "trace_context",
}

REQUIRED_METRICS = {
    "ttft_p99",
    "tpot_p95",
    "queue_length",
    "kv_pressure",
    "tokens_per_second",
    "tenant_quota",
    "route_version",
    "cost_per_1k_tokens",
    "pod_pending_seconds",
    "gpu_health",
    "node_ready",
    "memory_pressure",
}

CAPABILITIES = [
    {
        "name": "mini_engine",
        "layer": "engine",
        "responsibilities": {
            "model_load",
            "request_queue",
            "scheduler",
            "prefill_decode",
            "kv_cache",
            "sampling_streaming",
            "engine_metrics",
            "resource_cleanup",
        },
        "interfaces": {"request_api", "metrics_export", "control_drain", "health_probe", "trace_context"},
        "metrics": {"ttft_p99", "tpot_p95", "queue_length", "kv_pressure", "tokens_per_second"},
    },
    {
        "name": "inference_platform",
        "layer": "platform",
        "responsibilities": {
            "api_gateway",
            "auth_quota",
            "model_registry",
            "routing_rollout",
            "slo_alert",
            "cost_attribution",
            "observability",
            "degrade_retry",
        },
    },
]

```

```

    },
    "interfaces": {"request_api", "metrics_export", "control_drain", "health_probe", "trace_context"},
    "metrics": {"tenant_quota", "route_version", "cost_per_1k_tokens", "ttft_p99"},
  },
  {
    "name": "gpu_infra",
    "layer": "infra",
    "responsibilities": {
      "gpu_cluster",
      "container_runtime",
      "driver_cuda",
      "pod_scheduling",
      "network_storage",
      "resource_isolation",
      "node_health",
      "capacity_planning",
    },
    "interfaces": {"metrics_export", "health_probe", "trace_context"},
    "metrics": {"pod_pending_seconds", "gpu_health", "node_ready", "memory_pressure"},
  },
]

INCIDENTS = [
  {
    "id": "ttft_p99_high_good",
    "reported_layer": "platform",
    "investigated_layers": {"platform", "engine", "infra"},
    "root_layers": {"platform", "engine"},
    "evidence": {"traffic_shift", "prompt_tokens", "queue_length", "kv_pressure", "pod_pending_seconds"},
  },
  {
    "id": "cost_spike_good",
    "reported_layer": "platform",
    "investigated_layers": {"platform", "engine", "infra"},
    "root_layers": {"platform", "engine", "infra"},
    "evidence": {"route_version", "tokens_per_second", "cost_per_1k_tokens", "gpu_health"},
  },
  {
    "id": "ttft_p99_high_bad",
    "reported_layer": "engine",
    "investigated_layers": {"engine"},
    "root_layers": {"platform", "infra"},
    "evidence": {"queue_length"},
  },
]

```

```

def ratio(covered, required):
    return round(len(covered) / len(required), 3)

```

```

class MiniServingBoundaryAudit:
    def __init__(self, capabilities, incidents):
        self.capabilities = capabilities
        self.incidents = incidents

```

```

def audit(self):
    responsibilities = {layer: set() for layer in REQUIRED_BY_LAYER}
    interfaces = set()
    metrics = set()
    layer_counts = Counter()

    for capability in self.capabilities:
        layer = capability["layer"]
        layer_counts[layer] += 1
        responsibilities[layer].update(capability["responsibilities"])
        interfaces.update(capability["interfaces"])
        metrics.update(capability["metrics"])

    required_responsibilities = set().union(*REQUIRED_BY_LAYER.values())
    covered_responsibilities = set().union(*responsibilities.values())

    missing_by_layer = {
        layer: sorted(required - responsibilities[layer])
        for layer, required in REQUIRED_BY_LAYER.items()
    }
    misrouted = [
        item["id"]
        for item in self.incidents
        if not item["root_layers"].issubset(item["investigated_layers"])
    ]

    metrics_summary = {
        "layer_coverage": ratio(covered_responsibilities, required_responsibilities),
        "interface_coverage": ratio(interfaces, REQUIRED_INTERFACES),
        "metric_handoff_coverage": ratio(metrics, REQUIRED_METRICS),
        "misroute_rate": round(len(misrouted) / len(self.incidents), 3),
    }

    gates = {
        "role": metrics_summary["layer_coverage"] >= 0.95 and all(not v for v in missing_by_layer.values()),
        "interface": metrics_summary["interface_coverage"] >= 1.0,
        "metric": metrics_summary["metric_handoff_coverage"] >= 0.95,
        "incident": metrics_summary["misroute_rate"] == 0.0,
        "cost": "cost_per_1k_tokens" in metrics,
    }

    return {
        "layer_counts": dict(layer_counts),
        "metrics_summary": metrics_summary,
        "missing_by_layer": missing_by_layer,
        "misrouted_incidents": misrouted,
        "gates": gates,
        "boundary_gate": all(gates.values()),
    }

```

```

report = MiniServingBoundaryAudit(CAPABILITIES, INCIDENTS).audit()
print("layer_counts=", report["layer_counts"])
print("metrics_summary=", report["metrics_summary"])

```

```
print("missing_by_layer=", report["missing_by_layer"])
print("misrouted_incidents=", report["misrouted_incidents"])
print("gates=", report["gates"])
print("boundary_gate=", report["boundary_gate"])
```

一组典型输出如下：

```
layer_counts= {'engine': 1, 'platform': 1, 'infra': 1}
metrics_summary= {'layer_coverage': 1.0, 'interface_coverage': 1.0, 'metric_handoff_coverage': 1.0, 'misroute_rate': 0.0}
missing_by_layer= {'engine': [], 'platform': [], 'infra': []}
misrouted_incidents= ['ttft_p99_high_bad']
gates= {'role': True, 'interface': True, 'metric': True, 'incident': False, 'cost': True}
boundary_gate= False
```

这个 demo 的解释是：三层职责、接口和指标都覆盖了，但有一个事故只查 engine，没有查真实 root layers 里的 platform 和 infra，所以 incident 门禁失败，最终 boundary_gate=False。生产系统里，边界不是为了互相甩锅，而是为了让每个故障都能按证据走到正确层级。

6.16 面试追问

1. 推理框架、推理平台和 AI Infra 分别负责什么？
2. vLLM 属于哪一层？ Kubernetes 属于哪一层？ 模型路由属于哪一层？
3. 为什么平台扩缩容不能只看 QPS？
4. Engine 应该向平台暴露哪些指标？
5. TTFT 变差时如何分层排查？
6. 成本过高时 engine、platform、infra 分别能做什么？
7. 为什么不建议把所有治理逻辑都塞进 serving engine？
8. 如何设计 engine 和平台之间的控制接口？

参考回答思路：

1. 先给三层定义：engine 执行模型，platform 治理服务，infra 提供资源。
2. 再列各层核心模块和指标。
3. 然后说明接口：请求、指标、控制、健康检查。
4. 最后用 TTFT 或成本例子展示分层排查和优化能力。

6.17 小练习

1. 画一张三层架构图，标出 engine、platform 和 infra 的职责。
2. 给 vLLM、API Gateway、Kubernetes、GPU driver、灰度发布、KV Cache Manager 分别归层。
3. 设计 engine 暴露给平台的 10 个关键指标。
4. 假设 TPOT p99 变差，分别从 engine、platform、infra 三层列出可能原因。
5. 假设要多租户限流，哪些逻辑应该在平台层，哪些指标需要 engine 提供？
6. 扩展 MiniServingBoundaryAudit，新增一个“平台只看 QPS、不看 token 长度”的 bad case，并说明应该补哪些 engine 指标。

6.18 本章小结

本章讲清了推理框架、推理平台和 AI Infra 的边界。

推理框架负责模型执行，包括 scheduler、prefill、decode、KV Cache、sampling 和 streaming。推理平台负责服务治理，包括接入、鉴权、路由、限流、灰度、观测、成本和 SLO。AI Infra 负责底层资源，包括 GPU 集群、容器、调度、存储、网络、驱动和隔离。

边界清楚不是为了割裂系统，而是为了让职责、接口、指标和排障路径更清楚。下一章开始进入第二部分，我们会从 0 实现最小 tokenizer、model wrapper 和 generate loop，正式开始动手搭一个迷你推理框架。

第 7 章 实现最小 tokenizer、model wrapper 和 generate loop

从本章开始，我们进入第二部分：从 0 实现一个最小推理框架。目标不是立刻复刻 vLLM 或 SGLang，而是先把最小链路跑通：tokenizer 把文本转成 token，model wrapper 调用模型前向，generate loop 一步步生成 token。

如果你连最小 generate loop 都没拆开过，直接看 PagedAttention、continuous batching、RadixAttention，很容易只记住名词。本章先把最底层的生成过程写清楚。

一句话概括：

最小推理框架的第一步，是把“输入文本到输出文本”拆成 tokenizer、model wrapper、sampling 和 generate loop 四个可控环节。

7.0 本讲资料边界与第二轮精修口径

本章第二轮精修前，先用公开资料校准口径：Hugging Face Transformers 的生成接口把 generate()、logits 处理、stopping criteria、cache 和不同 decoding 策略封装成高层 API；Hugging Face tokenizer 文档强调 encode / decode、special tokens、padding / truncation 和 chat template 等输入输出契约；PyTorch 的 eval() 与 inference_mode() 分别对应模块推理状态和关闭 autograd 开销。

因此，本章不复刻 Transformers 源码，也不要求读者安装大模型权重才能理解机制。正文里的 PyTorch / Transformers 代码是工程对照，新增的 0 依赖 demo 用 toy tokenizer 和 toy model wrapper 展示最小生成链路。读者需要先掌握：文本如何变成 token id，模型如何把当前 token 序列变成最后位置 logits，generate loop 如何选择 token、追加 token、检查停止条件并留下 trace。

7.1 本章目标

本章要实现的是教学版最小链路，不追求性能。

它应该能完成：

1. 加载 tokenizer。
2. 加载模型。
3. 把 prompt 转成 token ids。
4. 调用模型得到 logits。
5. 从 logits 里选出下一个 token。
6. 把新 token 追加到序列。
7. 重复直到结束。
8. 把 token ids 解码成文本。

暂时不做：

1. batch。
2. KV Cache。
3. streaming。
4. scheduler。
5. HTTP API。
6. 多请求并发。

这些能力会在后续章节逐步加入。

7.2 最小组件拆分

最小推理框架可以先拆成三层：

```
TokenizerWrapper
  encode(text) -> input_ids
  decode(token_ids) -> text
```



```
ModelWrapper
    forward(input_ids) -> logits
```

```
GenerateLoop
    encode -> forward -> select token -> append -> decode
```

为什么要拆 wrapper?

因为我们后续会不断替换内部实现:

1. tokenizer 以后要支持 chat template。
2. model wrapper 以后要支持 KV Cache。
3. generate loop 以后要支持 batch 和 scheduler。
4. sampling 以后要支持 top-k、top-p、temperature。

如果一开始所有逻辑都塞进一个函数，后面升级会很痛苦。

7.3 TokenizerWrapper

Tokenizer 的职责是文本和 token 之间互转。

教学版 wrapper 可以这样写:

```
class TokenizerWrapper:
    def __init__(self, tokenizer):
        self.tokenizer = tokenizer
        self.eos_token_id = tokenizer.eos_token_id

    def encode(self, text):
        encoded = self.tokenizer(text, return_tensors="pt")
        return encoded.input_ids

    def decode(self, token_ids):
        return self.tokenizer.decode(token_ids, skip_special_tokens=True)
```

这里有几个注意点。

第一，input_ids 是 token id 序列，不是原始字符串。

第二，eos_token_id 用来判断生成是否结束。

第三，真实聊天模型还需要 chat template。也就是把 messages 转成模型训练时熟悉的格式。

第四，decode 时要处理 special tokens，否则输出里可能出现模型内部控制符。

7.4 ModelWrapper

ModelWrapper 的职责是隐藏具体模型调用细节，对外提供统一 forward 接口。

教学版可以先这样写:

```
class ModelWrapper:
    def __init__(self, model, device="cuda"):
        self.model = model.to(device)
        self.model.eval()
        self.device = device

    @torch.inference_mode()
    def forward(self, input_ids):
        input_ids = input_ids.to(self.device)
        outputs = self.model(input_ids=input_ids)
        return outputs.logits
```

几个关键点：

1. 推理时要用 `eval()`。
2. 推理时要关闭梯度。
3. 输入要放到模型所在设备。
4. 输出 logits 的形状通常是 `[batch, seq_len, vocab_size]`。

本章暂时不使用 KV Cache，所以每轮 forward 都会重新计算完整序列。这很低效，但便于理解。

7.5 Logits 是什么

模型 forward 的输出不是直接文本，而是 logits。

对于每个位置，logits 表示词表中每个 token 的未归一化分数。

形状示例：

```
input_ids shape: [1, seq_len]
logits shape:    [1, seq_len, vocab_size]
```

生成下一个 token 时，我们只关心最后一个位置的 logits：

```
next_token_logits = logits[:, -1, :]
```

因为自回归模型的含义是：基于前面所有 token，预测下一个 token。

如果 prompt 是：

大模型推理的核心是

最后一个位置的 logits 就是在回答：“下一个 token 最可能是什么？”

7.6 最简单的 greedy 选择

本章先用 greedy decoding，也就是每次选择分数最高的 token。

```
def greedy_select(logits):
    next_token_logits = logits[:, -1, :]
    next_token_id = torch.argmax(next_token_logits, dim=-1, keepdim=True)
    return next_token_id
```

greedy 的优点：

1. 简单。
2. 确定性强。
3. 适合教学和 debug。

greedy 的缺点：

1. 输出可能重复。
2. 多样性较差。
3. 不适合所有生成任务。

下一章会加入 temperature、top-k 和 top-p。

7.7 最小 generate loop

现在可以写最小生成循环：

```
def generate(tokenizer_wrapper, model_wrapper, prompt, max_new_tokens=128):
    input_ids = tokenizer_wrapper.encode(prompt)
    generated_ids = input_ids

    for _ in range(max_new_tokens):
```

```

logits = model_wrapper.forward(generated_ids)
next_token_id = greedy_select(logits)

generated_ids = torch.cat([generated_ids, next_token_id.cpu()], dim=-1)

if next_token_id.item() == tokenizer_wrapper.eos_token_id:
    break

return tokenizer_wrapper.decode(generated_ids[0])

```

这就是最小 generate loop。

它的核心逻辑是：

已有 tokens -> 模型预测下一个 token -> 追加 token -> 再预测下一个 token

注意，这段代码把 next_token_id 移回 CPU 后再拼接，是为了保持示例简单。真实实现里会尽量减少 CPU/GPU 来回拷贝。

7.8 更合理的设备处理

上面的示例为了易懂，设备处理略粗糙。更合理的写法是让 token 始终在模型设备上，最后再 decode。

```

def generate(tokenizer_wrapper, model_wrapper, prompt, max_new_tokens=128):
    input_ids = tokenizer_wrapper.encode(prompt).to(model_wrapper.device)
    generated_ids = input_ids

    for _ in range(max_new_tokens):
        logits = model_wrapper.forward(generated_ids)
        next_token_id = greedy_select(logits)
        generated_ids = torch.cat([generated_ids, next_token_id], dim=-1)

    if next_token_id.item() == tokenizer_wrapper.eos_token_id:
        break

    return tokenizer_wrapper.decode(generated_ids[0].tolist())

```

教学版代码要尽量清晰，但也要让读者知道真实系统会避免不必要的数据搬运。

7.9 停止条件

最小停止条件有两个：

1. 达到 max_new_tokens。
2. 生成 EOS token。

真实系统还会支持：

1. stop words。
2. stop token ids。
3. 请求超时。
4. 用户取消。
5. 最大上下文长度。
6. 安全策略中止。
7. engine 内部错误。

停止条件看起来简单，但在 streaming 和 batch 场景会变复杂。

例如 batch 中有 8 个请求，其中 3 个已经 EOS，另外 5 个还在生成。框架要把已结束请求移出 running set，并释放它们的 KV Cache，而不是等所有请求一起结束。

7.10 为什么这个版本很慢

本章的 generate loop 有一个明显问题：每生成一个 token，都把完整历史序列重新送进模型。

如果 prompt 长度是 1000，生成 100 个 token，计算大致像这样：

```
第 1 轮：处理 1000 tokens
第 2 轮：处理 1001 tokens
第 3 轮：处理 1002 tokens
...
第 100 轮：处理 1099 tokens
```

大量历史 token 被重复计算。

这就是下一步要引入 KV Cache 的原因。KV Cache 会让 decode 阶段只处理新 token，并复用历史 key/value。

但在引入 KV Cache 之前，先写清楚 naive loop 是必要的。因为 KV Cache 优化的对象，正是这个重复计算问题。

7.11 和 transformers.generate 的关系

Hugging Face 的 model.generate() 已经封装了大量功能：

1. greedy decoding。
2. sampling。
3. beam search。
4. KV Cache。
5. stopping criteria。
6. logits processor。
7. batch generation。

我们本章不是为了替代它，而是为了拆开它。

学习 serving engine 时，不能永远把 generate() 当黑盒。你需要知道黑盒里至少有：

1. forward。
2. logits 处理。
3. token 选择。
4. cache 更新。
5. 停止判断。
6. 序列管理。

只有拆开之后，才能继续理解 scheduler 和 continuous batching。

7.12 最小代码骨架

把本章组件合起来，代码骨架如下：

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

class TokenizerWrapper:
    def __init__(self, tokenizer):
        self.tokenizer = tokenizer
        self.eos_token_id = tokenizer.eos_token_id

    def encode(self, text):
        return self.tokenizer(text, return_tensors="pt").input_ids

    def decode(self, token_ids):
```

```

        return self.tokenizer.decode(token_ids, skip_special_tokens=True)

class ModelWrapper:
    def __init__(self, model, device="cuda"):
        self.model = model.to(device)
        self.model.eval()
        self.device = device

    @torch.inference_mode()
    def forward(self, input_ids):
        return self.model(input_ids=input_ids).logits

def greedy_select(logits):
    next_token_logits = logits[:, -1, :]
    return torch.argmax(next_token_logits, dim=-1, keepdim=True)

def generate(tokenizer_wrapper, model_wrapper, prompt, max_new_tokens=128):
    generated_ids = tokenizer_wrapper.encode(prompt).to(model_wrapper.device)

    for _ in range(max_new_tokens):
        logits = model_wrapper.forward(generated_ids)
        next_token_id = greedy_select(logits)
        generated_ids = torch.cat([generated_ids, next_token_id], dim=-1)

        if next_token_id.item() == tokenizer_wrapper.eos_token_id:
            break

    return tokenizer_wrapper.decode(generated_ids[0].tolist())

model_name = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

tokenizer_wrapper = TokenizerWrapper(tokenizer)
model_wrapper = ModelWrapper(model, device="cuda" if torch.cuda.is_available() else "cpu")

print(generate(tokenizer_wrapper, model_wrapper, "LLM inference is", max_new_tokens=32))

```

这段代码不是生产框架，但它已经具备推理框架的雏形。

7.13 这个骨架如何升级

后续章节会逐步升级它。

第 8 章会替换 `greedy_select`，加入 `sampling`。

第 9 章会修改 `ModelWrapper.forward`，加入 KV Cache。

第 10 章会让 `prefill` 支持 `batch`。

第 11 章会让 `decode` 支持 `batch`。

第 12 章会引入 `request queue` 和 `scheduler`。

第 13 章会把完整输出改成 `token streaming`。

第 14 章会加 HTTP API。

第 15 章会压测 TTFT、TPOT、吞吐和显存。

也就是说，本章代码是最小种子，后面每章都在它上面补一块真实 serving engine 能力。

7.14 生成循环公式、契约和可运行 demo

自回归生成可以写成：

$$P(y_1, \dots, y_M \mid x) = \prod_{t=1}^M P(y_t \mid x, y_1, \dots, y_{t-1})$$

其中， x 是输入 prompt， y_t 是第 t 个新生成 token。直觉上，模型不是一次写完整段文本，而是每一步根据已有 token 预测下一个 token。

tokenizer 契约可以写成：

$$s_0 = \mathrm{encode}(x), \quad \hat{x} = \mathrm{decode}(s_0)$$

s_0 是 prompt 的 token id 序列。真实 tokenizer 的 decode 可能不完全等于原字符串，因为空格、大小写、特殊 token 和规范化规则都会影响结果；但 wrapper 至少要保证 encode / decode 的输入输出类型清楚。

第 t 轮模型前向输出最后位置 logits：

$$z_t = f_{\theta}(s_{t-1})_{-1}, \quad z_t \in \mathbb{R}^V$$

V 是词表大小， $z_t(v)$ 是候选 token v 的未归一化分数。greedy decoding 可以写成：

$$y_t = \mathrm{argmax}_{v \in \mathcal{V}} z_t(v), \quad s_t = [s_{t-1}, y_t]$$

停止条件可以抽象为：

$$G_{\{\mathrm{stop}\}} = \mathbf{1}[y_t = y_{\{\mathrm{eos}\}}] \vee \mathrm{or} \quad t = M_{\{\max\}}$$

naive generate loop 的重复计算量可以粗略写成：

$$W_{\{\mathrm{naive}\}} = \sum_{t=0}^{M-1} (N+t)$$

其中 N 是 prompt token 数， M 是生成 token 数。这个式子解释了为什么本章版本会慢：每一轮都把历史 token 重新送进模型。后续 KV Cache 会把主要重复计算变成“prefill 处理 prompt，decode 每轮处理新增 token”。

下面是一个完全不依赖 PyTorch 和 Transformers 的最小 demo。它用 toy tokenizer、toy model wrapper 和 greedy generate loop 模拟“LLM inference -> is -> a -> loop -> . -> EOS”的生成过程。

class ToyTokenizer:

def __init__(self):

 self.id_to_token = ["<eos>", "<unk>", "LLM", "inference", "is", "a", "loop", "."]

 self.token_to_id = {token: idx **for** idx, token **in** enumerate(self.id_to_token)}

 self.eos_token_id = self.token_to_id["<eos>"]

 self.unk_token_id = self.token_to_id["<unk>"]

def encode(self, text):

return [self.token_to_id.get(piece, self.unk_token_id) **for** piece **in** text.split()]

def decode(self, token_ids, skip_special_tokens=True):

 pieces = []

for token_id **in** token_ids:

 token = self.id_to_token[token_id]

if skip_special_tokens **and** token.startswith("<"):

continue

 pieces.append(token)

return " ".join(pieces)

```

class ToyModelWrapper:
    def __init__(self, vocab_size):
        self.vocab_size = vocab_size
        self.transition = {
            2: 3, # LLM -> inference
            3: 4, # inference -> is
            4: 5, # is -> a
            5: 6, # a -> loop
            6: 7, # loop -> .
            7: 0, # . -> <eos>
        }

    def forward(self, input_ids):
        last_token = input_ids[-1]
        next_token = self.transition.get(last_token, 0)
        logits = [-10.0] * self.vocab_size
        logits[next_token] = 10.0
        return logits

def greedy_select(logits):
    best_id = 0
    best_score = logits[0]
    for token_id, score in enumerate(logits):
        if score > best_score:
            best_id = token_id
            best_score = score
    return best_id

def generate(tokenizer, model, prompt, max_new_tokens=8):
    generated_ids = tokenizer.encode(prompt)
    trace = []
    naive_token_work = 0

    for step in range(1, max_new_tokens + 1):
        naive_token_work += len(generated_ids)
        logits = model.forward(generated_ids)
        next_token_id = greedy_select(logits)
        generated_ids.append(next_token_id)

        trace.append(
            {
                "step": step,
                "input_len": len(generated_ids) - 1,
                "next_token": tokenizer.id_to_token[next_token_id],
                "stop": next_token_id == tokenizer.eos_token_id,
            }
        )

    if next_token_id == tokenizer.eos_token_id:
        break

    generated_new_tokens = len(generated_ids) - len(tokenizer.encode(prompt))

```

```

kv_like_work = len(tokenizer.encode(prompt)) + generated_new_tokens

metrics = {
    "prompt_tokens": len(tokenizer.encode(prompt)),
    "generated_tokens": generated_new_tokens,
    "forward_calls": len(trace),
    "naive_token_work": naive_token_work,
    "kv_like_work": kv_like_work,
    "duplicate_work": naive_token_work - kv_like_work,
}

audit = {
    "tokenizer_contract": tokenizer.decode(tokenizer.encode(prompt)) == prompt,
    "model_wrapper_contract": len(model.forward(tokenizer.encode(prompt))) == model.vocab_size,
    "generate_loop_trace": len(trace) == generated_new_tokens,
    "stop_condition": trace[-1]["stop"],
    "naive_recompute_visible": metrics["duplicate_work"] > 0,
}
audit["minimal_generate_gate"] = all(audit.values())

return tokenizer.decode(generated_ids), generated_ids, trace, metrics, audit

```

```

tokenizer = ToyTokenizer()
model = ToyModelWrapper(vocab_size=len(tokenizer.id_to_token))
text, token_ids, trace, metrics, audit = generate(tokenizer, model, "LLM inference")

```

```

print("generated_text=", text)
print("token_ids=", token_ids)
print("trace=", trace)
print("metrics=", metrics)
print("audit=", audit)

```

一组典型输出如下：

```

generated_text= LLM inference is a loop .
token_ids= [2, 3, 4, 5, 6, 7, 0]
trace= [{'step': 1, 'input_len': 2, 'next_token': 'is', 'stop': False}, {'step': 2, 'input_len': 3, 'next_token': 'a', 'st
metrics= {'prompt_tokens': 2, 'generated_tokens': 5, 'forward_calls': 5, 'naive_token_work': 20, 'kv_like_work': 7, 'dup
audit= {'tokenizer_contract': True, 'model_wrapper_contract': True, 'generate_loop_trace': True, 'stop_condition': True

```

这个 demo 说明四件事：

1. tokenizer wrapper 要有明确的 encode / decode 契约。
2. model wrapper 对外只暴露 “输入 token ids, 输出最后位置 logits”。
3. generate loop 的本质是 forward、select、append、stop。
4. naive loop 的重复计算能被量化，后续 KV Cache 不是魔法，而是在减少 duplicate_work。

7.15 常见误区

误区一：会调用 model.generate() 就等于懂生成循环。

generate() 是封装接口。理解 serving engine 需要知道 forward、logits、token selection、cache 和 stopping 是如何配合的。

误区二：logits 就是概率。

logits 是未归一化分数，经过 softmax 后才是概率分布。

误区三：每轮都重新计算完整序列也没关系。

对短 demo 可以接受；对长上下文和高并发服务不可接受，所以必须引入 KV Cache。

误区四：tokenizer 不重要。

tokenizer 决定 token 数、chat template、特殊 token 和停止条件，直接影响推理成本和结果格式。

误区五：教学代码不需要考虑设备。

即使是教学代码，也要知道 CPU/GPU 数据搬运会影响性能，真实系统要尽量减少不必要拷贝。

7.16 面试追问

1. 最小 generate loop 包含哪些步骤？
2. logits 的形状是什么，为什么只取最后一个位置？
3. greedy decoding 是什么，有什么优缺点？
4. 为什么 naive generate loop 很慢？
5. tokenizer 在推理系统中负责什么？
6. eval() 和 torch.inference_mode() 为什么重要？
7. model.generate() 和手写 generate loop 的关系是什么？
8. 这个最小框架如何升级到支持 KV Cache？

参考回答思路：

1. 先说 encode、forward、取最后 logits、选 token、append、停止判断、decode。
2. 再解释 logits 是 [batch, seq_len, vocab_size]，最后位置预测下一个 token。
3. 然后指出 naive loop 重复计算历史，所以需要 KV Cache。
4. 最后说明后续要加入 sampling、batch、scheduler 和 streaming。

7.17 小练习

1. 手写一个 greedy generate loop，不调用 model.generate()。
2. 打印每轮 generated_ids.shape，观察序列长度如何增长。
3. 打印 logits.shape，确认最后一维是词表大小。
4. 把 max_new_tokens 改成不同值，观察输出和耗时变化。
5. 思考如果每轮只输入最后一个 token，为什么还需要 KV Cache？
6. 扩展 ToyModelWrapper，让它在不同 prompt 下走不同 transition，并观察 trace 如何帮助定位生成错误。

7.18 本章小结

本章实现了最小 tokenizer、model wrapper 和 generate loop。

我们把生成过程拆成了文本编码、模型 forward、logits 选择、token 追加、停止判断和文本解码。这个版本故意不使用 KV Cache、不支持 batch、不支持 streaming，因此性能很差，但结构清晰。

下一章会在这个基础上加入 greedy、top-k、top-p 和 temperature sampling，让 token 选择从“永远取最大值”升级成可控采样策略。

第 8 章 实现 greedy、top-k、top-p 和 temperature sampling

上一章我们实现了最小 generate loop，但 token 选择只用了 greedy，也就是每次都选 logits 最大的 token。本章把 token 选择模块拆出来，逐步实现 greedy、temperature、top-k 和 top-p sampling。

一句话概括：

Sampling 决定模型在“最确定”和“更多样”之间如何取舍，是生成质量、稳定性、可控性和成本体验的重要开关。

8.0 本讲资料边界与第二轮精修口径

本章第二轮精修前，先用公开资料校准口径：Transformers 的 generation 文档把 greedy、sampling、beam search、logits processor / warper、stopping criteria 和 cache 放在同一个生成控制体系中；top-k、top-p 和 temperature 都属于对 logits 或候选概率分布的控制策略；nucleus sampling 的经典论文动机是减少低质量长尾 token 被采样，同时避免固定 top-k 在不同分布形状下过度或不足截断。

因此，本章不讨论 beam search、contrastive search、speculative decoding、structured generation 和安全 logits mask，只聚焦单步 token sampling。正文里的 PyTorch 代码用于贴近工程 API，新增的 0 依赖 demo 用纯 Python 展示 stable softmax、temperature、top-k、top-p、seeded multinomial 和采样审计指标。

8.1 为什么要单独实现 sampling

在 generate loop 中，模型 forward 只负责给出 logits。真正决定下一个 token 的，是 sampling 模块。

上一章的核心代码是：

```
logits = model_wrapper.forward(generated_ids)
next_token_id = greedy_select(logits)
generated_ids = torch.cat([generated_ids, next_token_id], dim=-1)
```

本章要把 greedy_select 升级成可配置 sampler：

```
next_token_id = sampler.sample(logits)
```

这样做的好处是：

1. generate loop 不关心具体采样策略。
2. 后续可以轻松加入 temperature、top-k、top-p。
3. serving engine 可以为每个请求设置不同 sampling params。
4. debug 时可以固定 greedy，线上可以启用随机采样。

8.2 logits、概率和采样

模型输出 logits，logits 不是概率，而是未归一化分数。

要得到概率，需要 softmax：

```
probs = torch.softmax(logits, dim=-1)
```

然后从概率分布中采样：

```
next_token_id = torch.multinomial(probs, num_samples=1)
```

生成的核心链路可以理解为：

hidden states -> logits -> probability distribution -> sampled token

不同 sampling 策略，本质上都是在 softmax 和采样前后修改这个概率分布。

8.3 Greedy Decoding

Greedy decoding 每次选择 logits 最大的 token。

```
def greedy_sample(logits):
    next_token_logits = logits[:, -1, :]
    return torch.argmax(next_token_logits, dim=-1, keepdim=True)
```

优点：

1. 确定性强。
2. 实现简单。
3. 适合单元测试和系统 debug。
4. 不需要随机数种子。

缺点：

1. 容易输出重复内容。
2. 多样性较差。
3. 不一定选择全局最优序列。
4. 对开放式写作和创意任务不友好。

Greedy 适合知识问答、分类式输出、格式稳定任务，也适合排查 serving engine 的确定性问题。

8.4 Temperature

Temperature 用来控制概率分布的尖锐程度。

做法是把 logits 除以 temperature：

`scaled_logits = logits / temperature`

temperature 越低，分布越尖锐，模型越保守。

temperature 越高，分布越平坦，模型越随机。

常见直觉：

1. temperature = 0: 近似 greedy。
2. temperature < 1: 更保守。
3. temperature = 1: 保持原分布。
4. temperature > 1: 更多样，但更容易跑偏。

实现时要注意 temperature=0 不能直接做除法，通常单独走 greedy。

```
def apply_temperature(logits, temperature):
    if temperature is None or temperature == 1.0:
        return logits
    if temperature <= 0:
        raise ValueError("temperature must be positive")
    return logits / temperature
```

生产系统里也常把 temperature=0 解释为 deterministic decoding。

8.5 Top-k Sampling

Top-k 的思想是：只保留概率最高的 k 个 token，其他 token 直接屏蔽。

例如词表有 50000 个 token，但 top_k=50，那么每一步只从分数最高的 50 个 token 中采样。

实现方式：

```
def apply_top_k(logits, top_k):
    if top_k is None or top_k <= 0:
        return logits

    top_k = min(top_k, logits.size(-1))
    values, _ = torch.topk(logits, top_k, dim=-1)
    threshold = values[..., -1, None]
    return torch.where(logits < threshold, torch.full_like(logits, float("-inf")), logits)
```

Top-k 的优点：

1. 简单直观。
2. 能过滤大量低概率 token。
3. 减少胡乱采样的概率。

缺点：

1. k 是固定数量，不适应分布形状。
2. 如果分布很尖锐，k 太大没意义。
3. 如果分布很平坦，k 太小可能过度截断。

8.6 Top-p Sampling

Top-p 也叫 nucleus sampling。它不是保留固定数量 token，而是保留累计概率达到 p 的最小 token 集合。

例如 top_p=0.9 表示：按概率从高到低排序，只保留累计概率达到 0.9 的候选 token。

实现步骤：

1. 对 logits 排序。
2. softmax 得到排序后的概率。
3. 计算累计概率。
4. 屏蔽累计概率超过 top-p 之后的 token。
5. 映射回原词表顺序。

示例代码：

```
def apply_top_p(logits, top_p):
    if top_p is None or top_p >= 1.0:
        return logits
    if top_p <= 0:
        raise ValueError("top_p must be positive")

    sorted_logits, sorted_indices = torch.sort(logits, descending=True, dim=-1)
    sorted_probs = torch.softmax(sorted_logits, dim=-1)
    cumulative_probs = torch.cumsum(sorted_probs, dim=-1)

    sorted_mask = cumulative_probs > top_p
    sorted_mask[..., 1:] = sorted_mask[..., :-1].clone()
    sorted_mask[..., 0] = False

    sorted_logits = sorted_logits.masked_fill(sorted_mask, float("-inf"))
    filtered_logits = torch.full_like(logits, float("-inf"))
    filtered_logits.scatter_(dim=-1, index=sorted_indices, src=sorted_logits)
    return filtered_logits
```

Top-p 的优点是能适应分布形状。分布尖锐时候选集合小，分布平坦时候选集合大。

8.7 组合顺序

常见组合流程是：

取最后位置 logits

- > temperature scaling
- > top-k filtering
- > top-p filtering
- > softmax
- > multinomial sample

代码如下：

```
def sample_next_token(logits, temperature=1.0, top_k=None, top_p=1.0):
    next_token_logits = logits[:, -1, :]

    if temperature == 0:
        return torch.argmax(next_token_logits, dim=-1, keepdim=True)
```

```

next_token_logits = apply_temperature(next_token_logits, temperature)
next_token_logits = apply_top_k(next_token_logits, top_k)
next_token_logits = apply_top_p(next_token_logits, top_p)

```

```

probs = torch.softmax(next_token_logits, dim=-1)
return torch.multinomial(probs, num_samples=1)

```

不同框架在细节上可能略有差异，但核心思想一致：先修改候选分布，再采样。

8.8 Sampler 类

为了更接近推理框架，可以把采样封装成类：

```

class SamplingParams:
    def __init__(self, temperature=1.0, top_k=None, top_p=1.0):
        self.temperature = temperature
        self.top_k = top_k
        self.top_p = top_p

```

```

class Sampler:
    def __init__(self, params):
        self.params = params

    def sample(self, logits):
        return sample_next_token(
            logits,
            temperature=self.params.temperature,
            top_k=self.params.top_k,
            top_p=self.params.top_p,
        )

```

然后 generate loop 变成：

```

def generate(tokenizer_wrapper, model_wrapper, sampler, prompt, max_new_tokens=128):
    generated_ids = tokenizer_wrapper.encode(prompt).to(model_wrapper.device)

    for _ in range(max_new_tokens):
        logits = model_wrapper.forward(generated_ids)
        next_token_id = sampler.sample(logits)
        generated_ids = torch.cat([generated_ids, next_token_id], dim=-1)

        if next_token_id.item() == tokenizer_wrapper.eos_token_id:
            break

    return tokenizer_wrapper.decode(generated_ids[0].tolist())

```

这样结构更清楚，后续加入 repetition penalty、stop token、logprobs 也更方便。

8.9 随机种子和可复现性

一旦使用 sampling，输出就可能不同。

如果需要复现，可以设置随机种子：

```
torch.manual_seed(1234)
```

但生产系统里，可复现并不总是简单。因为还会受到这些因素影响：

1. batch 内请求顺序。

2. GPU kernel 的确定性。
3. 多卡并行通信。
4. speculative decoding 的接受路径。
5. 框架版本和浮点误差。

所以面试里不要轻易承诺“设置 seed 就一定完全可复现”。更稳妥的说法是：单机单请求简单路径可以尽量复现，复杂 serving 场景需要额外控制执行顺序和底层确定性。

8.10 参数如何影响输出

常见参数组合：

场景	temperature	top_p	top_k	特点
稳定问答	0 或 0.2	1.0	None	更确定
通用聊天	0.7	0.9	None 或 50	平衡稳定和多样
创意写作	0.9-1.2	0.95	None	更多样
格式化输出	0 或 0.1	1.0	None	更可控

这只是经验，不是硬规则。不同模型、任务和业务对参数的敏感度不同。

8.11 Sampling 和 Serving Engine

在 serving engine 中，sampling 不是全局固定的。不同请求可能有不同参数。

例如：

```
request_1: temperature=0, top_p=1.0
request_2: temperature=0.7, top_p=0.9
request_3: temperature=1.0, top_k=40
```

这会带来工程复杂度：

1. batch 中每个请求的 sampling params 不同。
2. 有些请求需要 logprobs，有些不需要。
3. 有些请求需要 deterministic，有些需要 random。
4. structured generation 还会加入额外 logits mask。

因此真实 engine 的 sampler 往往是批量化且按请求配置执行的，而不是本章这种单请求函数。

8.12 采样公式、数值稳定和可运行 demo

temperature softmax 可以写成：

$$p_i(\tau) = \frac{\exp((z_i - m)/\tau)}{\sum_j \exp((z_j - m)/\tau)}, \quad m = \max_j z_j$$

其中 z_i 是第 i 个 token 的 logits， τ 是 temperature。减去 m 不改变 softmax 结果，但能避免 $\exp(z_i)$ 因 logits 过大而数值溢出。

当 $\tau \rightarrow 0$ 时，分布会越来越尖锐，工程上通常直接走 greedy：

$$y = \operatorname{argmax}_{i \in \mathcal{V}} z_i$$

Top-k 的候选集合可以写成：

$$\mathcal{S}_k = \{i \mid z_i \text{ is in the top } k \text{ values}\}$$

Top-p 的候选集合先按概率从大到小排序，取最小的前缀集合：

$$n^* = \min \left\{ n : \sum_{r=1}^n p_{(r)} \geq \rho \right\}, \quad \mathcal{S}_p = \{(1), \dots, (n^*)\}$$

其中 ρ 是 top_p， $p_{(r)}$ 是排序后的第 r 个概率。过滤后要重新归一化：

$$\tilde{p}_i = \frac{p_i}{\sum_j p_j}$$

采样模块的验收门禁可以写成：

$G_{\text{sampling}} = G_{\text{softmax}} G_{\text{temperature}} G_{\text{topk}} G_{\text{topp}} G_{\text{seed}}$

下面是一个 0 依赖 demo。它用固定 logits 展示 greedy、temperature、top-k、top-p 和组合过滤如何改变候选集合、熵和最终采样 token。

```
import math
import random
```

```
VOCAB = ["safe", "clear", "creative", "risky", "off_topic", "loop"]
LOGITS = [4.0, 3.0, 2.0, 0.5, -1.0, -2.0]
```

```
def stable_softmax(logits, temperature=1.0):
    if temperature <= 0:
        raise ValueError("temperature must be positive for softmax")
    max_logit = max(logits)
    exp_values = [math.exp((value - max_logit) / temperature) for value in logits]
    total = sum(exp_values)
    return [value / total for value in exp_values]

def normalize(probs):
    total = sum(probs)
    if total <= 0:
        raise ValueError("all probabilities were filtered out")
    return [value / total for value in probs]

def apply_top_k(probs, top_k):
    if top_k is None or top_k <= 0 or top_k >= len(probs):
        return probs[:]
    keep = {idx for idx, _ in sorted(enumerate(probs), key=lambda item: item[1], reverse=True)[:top_k]}
    return normalize([value if idx in keep else 0.0 for idx, value in enumerate(probs)])

def apply_top_p(probs, top_p):
    if top_p is None or top_p >= 1.0:
        return probs[:]
    if top_p <= 0:
        raise ValueError("top_p must be positive")

    ranked = sorted(enumerate(probs), key=lambda item: item[1], reverse=True)
    keep = set()
    cumulative = 0.0
    for idx, value in ranked:
        keep.add(idx)
        cumulative += value
        if cumulative >= top_p:
            break
    return normalize([value if idx in keep else 0.0 for idx, value in enumerate(probs)])
```

```

def entropy(probs):
    return -sum(value * math.log(value) for value in probs if value > 0)

def sample_from_probs(probs, seed):
    rng = random.Random(seed)
    threshold = rng.random()
    cumulative = 0.0
    for idx, value in enumerate(probs):
        cumulative += value
        if threshold <= cumulative:
            return idx
    return len(probs) - 1

def sample_once(name, temperature=1.0, top_k=None, top_p=1.0, seed=6):
    if temperature == 0:
        selected = max(range(len(LOGITS)), key=lambda idx: LOGITS[idx])
        probs = [1.0 if idx == selected else 0.0 for idx in range(len(LOGITS))]
    else:
        probs = stable_softmax(LOGITS, temperature=temperature)
        probs = apply_top_k(probs, top_k)
        probs = apply_top_p(probs, top_p)
        selected = sample_from_probs(probs, seed)

    candidates = [VOCAB[idx] for idx, value in enumerate(probs) if value > 0]
    return {
        "name": name,
        "selected": VOCAB[selected],
        "candidates": candidates,
        "candidate_count": len(candidates),
        "entropy": round(entropy(probs), 3),
        "probs": {VOCAB[idx]: round(value, 3) for idx, value in enumerate(probs) if value > 0},
    }

cases = [
    sample_once("greedy", temperature=0),
    sample_once("temperature_0_7", temperature=0.7),
    sample_once("temperature_1_4", temperature=1.4),
    sample_once("top_k_3", temperature=1.0, top_k=3),
    sample_once("top_p_0_8", temperature=1.0, top_p=0.8),
    sample_once("combined", temperature=0.8, top_k=4, top_p=0.85),
]

repeat_a = sample_once("repeat", temperature=1.0, top_k=3, seed=42)["selected"]
repeat_b = sample_once("repeat", temperature=1.0, top_k=3, seed=42)["selected"]

audit = {
    "softmax_normalized": all(abs(sum(item["probs"].values()) - 1.0) <= 0.002 for item in cases),
    "temperature_changes_entropy": cases[2]["entropy"] > cases[1]["entropy"],
    "top_k_reduces_candidates": cases[3]["candidate_count"] == 3,
    "top_p_adapts_candidates": cases[4]["candidate_count"] < len(VOCAB),
    "seed_reproducible": repeat_a == repeat_b,
}

```



```
audit["sampling_gate"] = all(audit.values())
```

```
print("cases=", cases)
```

```
print("audit=", audit)
```

一组典型输出如下：

```
cases= [{ 'name': 'greedy', 'selected': 'safe', 'candidates': ['safe'], 'candidate_count': 1, 'entropy': -  
0.0, 'probs': { 'safe': 1.0 } }, { 'name': 'temperature_0_7', 'selected': 'clear', 'candidates': ['safe', 'clear', 'creative'  
audit= { 'softmax_normalized': True, 'temperature_changes_entropy': True, 'top_k_reduces_candidates': True, 'top_p_adapt
```

注意最后一个 combined 示例：先做 temperature 和 top-k，再做 top-p，会让候选集和概率发生二次变化。实际框架要清楚记录这些参数，否则线上生成差异很难复盘。

8.13 常见误区

误区一：temperature 越高越聪明。

temperature 只会增加随机性，不会提升模型能力。太高反而更容易胡说。

误区二：top-k 和 top-p 是一回事。

top-k 固定保留 k 个 token；top-p 保留累计概率达到 p 的动态集合。

误区三：greedy 一定最好。

greedy 稳定，但可能重复、死板，不适合所有开放式任务。

误区四：sampling 只影响文本质量，不影响系统。

sampling 会影响输出长度、是否重复、是否命中 stop，从而影响 decode 轮数、TPOT 和成本。

误区五：设置 seed 就能保证任何线上场景完全复现。

复杂 serving 里还有 batch 顺序、kernel、并行和浮点误差等因素。

8.14 面试追问

1. logits 和概率有什么区别？
2. greedy decoding 怎么实现？
3. temperature 如何影响概率分布？
4. top-k 和 top-p 的区别是什么？
5. 为什么 top-p 更能适应分布形状？
6. sampling 参数如何影响输出稳定性和多样性？
7. serving engine 中为什么每个请求可能需要不同 sampling params？
8. 为什么线上完全复现随机采样并不容易？

参考回答思路：

1. 先说 logits 是未归一化分数，softmax 后才是概率。
2. 再说 greedy 取最大，temperature 调分布尖锐程度，top-k/top-p 过滤候选集合。
3. 然后说明参数影响质量、输出长度和成本。
4. 最后补 serving 复杂度：batch 内每个请求参数不同，sampler 需要批量化和可配置。

8.15 小练习

1. 在第 7 章代码中替换 greedy，加入 sample_next_token。
2. 对同一个 prompt 分别设置 temperature=0、0.7、1.2，观察输出差异。
3. 对比 top_k=10 和 top_p=0.9 的候选集合变化。
4. 设置相同 seed 多次运行，观察是否完全一致。
5. 思考为什么结构化 JSON 输出通常更适合低 temperature。

6. 扩展本章纯 Python demo，打印每种策略的候选 token 集合和 entropy，解释为什么高 entropy 不等于高质量。

8.16 本章小结

本章实现了 greedy、temperature、top-k 和 top-p sampling。

Greedy 适合确定性和 debug；temperature 控制分布随机性；top-k 固定保留高分 token；top-p 根据累计概率动态保留候选集合。把采样逻辑封装成 sampler 后，generate loop 的结构更接近真实 serving engine。

下一章我们会加入 KV Cache，让 naive generate loop 不再每轮重复计算完整历史序列。

第 9 章 实现 KV Cache 的最小版本

上一章我们实现了 sampling。本章解决 naive generate loop 最大的性能问题：每生成一个 token，都重新计算完整历史序列。解决办法就是 KV Cache。

KV Cache 是 LLM 推理框架的核心。vLLM 的 PagedAttention、SGLang 的 prefix sharing、PD 分离里的 KV 迁移，本质上都围绕 KV Cache 做文章。本章先不讲复杂管理，只实现最小版本：单请求、连续缓存、使用 past_key_values 复用历史。

一句话概括：

KV Cache 让 decode 阶段不再重复计算历史 token 的 key/value，而是复用缓存，只为新 token 计算 query/key/value。

9.0 本讲资料边界与第二轮精修口径

本章第二轮精修前，先用公开资料校准口径：Transformers 文档把 cache 解释为自回归生成中保存历史 key/value states，用来避免每一步重复计算完整上下文；Transformers generate() 和模型 forward 路径通常通过 use_cache、past_key_values、Cache 对象、attention_mask 和位置相关信息协同工作；vLLM 的 PagedAttention 论文和文档进一步把问题推进到生产 serving 中的 KV cache block 管理、非连续物理存储、按需分配、共享前缀和碎片控制。

因此，本章只讨论最小单请求 KV Cache：prefill 建 cache，decode 追加 cache，验证输出等价和重复计算下降。它不实现 PagedAttention、block manager、prefix cache、多请求调度、KV offload、KV quantization 或 PD 分离。正文里的 Hugging Face / PyTorch 代码用于贴近工程接口，新增 0 依赖 demo 用纯 Python 展示最小 cache 机制和审计指标。

9.1 naive loop 的重复计算

第 7 章的 generate loop 每一轮都把完整 generated_ids 送进模型：

```
for _ in range(max_new_tokens):
    logits = model_wrapper.forward(generated_ids)
    next_token_id = sampler.sample(logits)
    generated_ids = torch.cat([generated_ids, next_token_id], dim=-1)
```

如果 prompt 有 1000 个 token，生成 100 个 token，模型处理的序列长度大致是：

1000, 1001, 1002, ..., 1099

历史 token 会被一遍遍重新计算。自回归生成真正需要的是：历史已经算过的 key/value 保存起来，每轮只处理新 token。

可以用一个简单工作量近似看差异。设 prompt 长度为 N ，生成 M 个 token。

naive loop 每轮处理完整上下文：

$$W_{\{\mathrm{naive}\}} = \sum_{t=0}^{M-1} (N+t) \\ = MN + \frac{M(M-1)}{2}$$

带 KV Cache 的最小 loop 先 prefill prompt, 再每轮 decode 一个 token:

$$W_{\{\mathrm{kv}\}}=N+M$$

这个公式不是精确 FLOPs, 只是说明重复计算规模。真实 Transformer 中 attention、MLP、kernel、batch 和访存都会影响耗时, 但方向很稳定: cache 用额外显存换掉大量历史 token 的重复计算。

9.2 KV Cache 保存了什么

Transformer 的 attention 会为每个 token 计算 query、key、value。

生成新 token 时, 新 token 的 query 需要和历史 token 的 key/value 做 attention。历史 token 的 key/value 在 causal decoder 中不会因为后面生成新 token 而改变, 所以可以缓存。

KV Cache 保存的是:

1. 每一层的 key cache。
2. 每一层的 value cache。
3. 每个历史 token 对应的 key/value。
4. 每个 KV head 对应的 key/value。

简化结构:

`past_key_values[layer] = (past_key, past_value)`

常见 shape 可以写成:

key/value: [batch, num_kv_heads, seq_len, head_dim]

注意这里是 num_kv_heads, 不一定等于 query heads。MHA 中二者相同; GQA / MQA 中 KV head 更少, KV Cache 显存也更低。

9.3 Prefill 和 Decode 中的 cache

KV Cache 的使用通常分成两步。

第一步, prefill:

输入完整 prompt -> 模型 forward -> 返回 logits 和 past_key_values

第二步, decode:

输入上一个 token + past_key_values -> 模型 forward -> 返回新 logits 和更新后的 past_key_values

也就是说, prefill 建立初始 cache, decode 不断扩展 cache。

设 prompt token 为 $x_1, \dots, x_N$, 生成 token 为 $y_1, \dots, y_M$ 。prefill 后 cache 长度为:

$$T_0=N$$

第 t 次 decode 后 cache 长度为:

$$T_t=N+t$$

decode 的 logits 应该只从最后一个有效位置取:

$$z_t=f_{\{\theta\}}(y_{\{t-1\}},C_{\{t-1\}})_{\{\mathrm{last}\}}$$

其中 $C_{\{t-1\}}$ 是已经缓存的历史 key/value。

9.4 Hugging Face 中的 past_key_values

很多 causal LM 支持 use_cache=True, 并在输出中返回 past_key_values 或新版 cache 对象。

第一次 prefill:

```

outputs = model(input_ids=input_ids, use_cache=True)
logits = outputs.logits
past_key_values = outputs.past_key_values

```

后续 decode:

```

outputs = model(
    input_ids=next_token_id,
    past_key_values=past_key_values,
    use_cache=True,
)
logits = outputs.logits
past_key_values = outputs.past_key_values

```

注意，decode 阶段传入的 `input_ids` 通常只包含最新 token，而不是完整历史序列。

版本边界也要注意：不同 Transformers 版本和不同模型可能使用 tuple 形式的 `past_key_values`，也可能使用新的 `Cache / DynamicCache` 抽象。面试中不需要背具体类名，但要讲清接口语义：cache 是模型 forward 的状态输入和状态输出。

9.5 修改 ModelWrapper

我们先把 `ModelWrapper.forward` 改成支持 cache。

```

class ModelWrapper:
    def __init__(self, model, device="cuda"):
        self.model = model.to(device)
        self.model.eval()
        self.device = device

    @torch.inference_mode()
    def forward(self, input_ids, past_key_values=None, attention_mask=None, position_ids=None):
        input_ids = input_ids.to(self.device)
        outputs = self.model(
            input_ids=input_ids,
            past_key_values=past_key_values,
            attention_mask=attention_mask,
            position_ids=position_ids,
            use_cache=True,
        )
        return outputs.logits, outputs.past_key_values

```

对外接口从：

```
logits = model_wrapper.forward(input_ids)
```

变成：

```
logits, past_key_values = model_wrapper.forward(input_ids, past_key_values)
```

这一步看起来小，但意义很大：模型执行从无状态调用变成了带缓存状态的调用。

9.6 带 KV Cache 的 generate loop

最小带 cache 版本如下：

```

def generate_with_kv_cache(tokenizer_wrapper, model_wrapper, sampler, prompt, max_new_tokens=128):
    input_ids = tokenizer_wrapper.encode(prompt).to(model_wrapper.device)
    generated_ids = input_ids

    logits, past_key_values = model_wrapper.forward(

```

```

        input_ids=input_ids,
        past_key_values=None,
    )

    for _ in range(max_new_tokens):
        next_token_id = sampler.sample(logits)
        generated_ids = torch.cat([generated_ids, next_token_id], dim=-1)

        if next_token_id.item() == tokenizer_wrapper.eos_token_id:
            break

        logits, past_key_values = model_wrapper.forward(
            input_ids=next_token_id,
            past_key_values=past_key_values,
        )

    return tokenizer_wrapper.decode(generated_ids[0].tolist())

```

核心变化是：

1. 第一次输入完整 prompt。
2. 保存 `past_key_values`。
3. 后续每轮只输入上一个 token。
4. 每轮更新 `past_key_values`。

这段代码已经体现了 prefill 和 decode 的区别：第一次 forward 输入完整 prompt，后续 forward 只输入新 token。

9.7 更清晰的 prefill/decode 写法

为了贴近 serving engine，可以把 prefill 和 decode 拆成两个函数。

```

def prefill(model_wrapper, input_ids, attention_mask=None):
    logits, past_key_values = model_wrapper.forward(
        input_ids=input_ids,
        past_key_values=None,
        attention_mask=attention_mask,
    )
    return logits, past_key_values

def decode_one_step(model_wrapper, next_token_id, past_key_values, attention_mask=None, position_ids=None):
    logits, past_key_values = model_wrapper.forward(
        input_ids=next_token_id,
        past_key_values=past_key_values,
        attention_mask=attention_mask,
        position_ids=position_ids,
    )
    return logits, past_key_values

```

generate loop 变成：

```
logits, past_key_values = prefill(model_wrapper, input_ids)
```

```

while not finished:
    next_token_id = sampler.sample(logits)
    logits, past_key_values = decode_one_step(
        model_wrapper,

```

```

        next_token_id,
        past_key_values,
    )

```

这就是后续实现 batched prefill 和 batched decode 的基础。

9.8 cache 的显存公式

KV Cache 会随着 seq_len 增长。

设：

1. B 是 active batch size。
2. T 是当前 cache token 长度。
3. L 是 transformer 层数。
4. H_kv 是 KV heads 数。
5. D_h 是每个 head 的维度。
6. b 是每个元素的字节数，例如 FP16 / BF16 常用 2 字节。

单个 active batch 的 KV Cache 显存近似为：

$$M_{\{\mathrm{kv}\}} = 2BLTH_{\{\mathrm{kv}\}}D_{\mathrm{hb}}$$

前面的 2 来自 key 和 value 两份缓存。

如果多个请求长度不同，更准确的估算应该按活跃请求求和：

$$M_{\{\mathrm{kv}\}} = 2L H_{\{\mathrm{kv}\}}D_{\mathrm{hb}}\sum_i T_i$$

这也解释了为什么 KV Cache 会影响并发：上下文越长、输出越长、活跃请求越多，显存压力越大。

9.9 为什么 cache 会影响并发

单请求时，KV Cache 是一个优化。多请求时，它是并发上限的重要约束。

如果每个请求都保存自己的 cache，那么并发越高，cache 总量越大。

总 KV Cache 约正比于：active_requests * average_context_length

所以 serving engine 不只是“用不用 cache”，还要管理 cache：分配、释放、复用、抢占、分页、迁移。

最小单请求 cache 是连续增长的列表或张量；生产系统里的 cache manager 需要回答更多问题：

1. 新请求能不能准入。
2. 还有多少 KV block 可用。
3. 请求结束或取消后是否及时释放。
4. prefix cache 是否能复用公共前缀。
5. cache 过高时是拒绝、等待、抢占、swap 还是降级。

本章只实现最小版本，后续讲 vLLM 时会进入 block manager 和 PagedAttention。

9.10 attention_mask 和 position_ids

真实模型调用里，除了 input_ids 和 past_key_values，常常还要处理 attention_mask 和 position_ids。

简化理解：

1. attention_mask 告诉模型哪些 token 是有效 token。
2. position_ids 告诉模型当前 token 的位置。

在单请求、无 padding、使用标准 generate() 路径时，很多模型会自动处理。

但在自己实现 batch decode 时，这些不能忽略。多个请求长度不同，新 token 的位置也不同。若 position 错位，RoPE / absolute position embedding 看到的位置就错了；若 mask 长度和 cache 长度不一致，模型可能看不到该看的历史，或者错误看见 padding。

decode 阶段的关键约束可以写成：

$$\mathrm{len}(\mathrm{attention_mask}) = T_{\mathrm{cache}} + T_{\mathrm{new}}$$

单 token decode 时：

$$\mathrm{position_id} = T_{\mathrm{cache}}$$

这里的 T_{cache} 是当前新 token 之前已经缓存的历史长度。

9.11 cache 和输出一致性

理论上，在相同采样策略、相同随机种子、相同模型实现下，使用 KV Cache 和不使用 KV Cache 应该得到等价结果。

但实际可能出现微小差异，原因包括：

1. 浮点计算顺序不同。
2. kernel 实现不同。
3. attention mask 或 position_ids 处理错误。
4. 模型是否正确支持 cache。
5. batch 和单请求路径不同。

如果使用 cache 后输出明显变坏，优先排查：

1. 是否只在 decode 阶段传入最新 token。
2. past_key_values 是否正确更新。
3. position 是否正确递增。
4. attention mask 是否和 cache 长度匹配。
5. EOS 和停止逻辑是否被破坏。

9.12 KV Cache 公式、机制和可运行 demo

下面这个 demo 不依赖 PyTorch。它用一个极小的单头 causal attention 模拟两条路径：

1. full_last_hidden：每次对完整序列重新计算 Q/K/V，再取最后 token 输出。
2. cached_decode_step：prefill 缓存历史 K/V，decode 时只为新 token 计算 Q/K/V，并把新 K/V 追加进 cache。

demo 的目标不是复刻真实 Transformer，而是验证三个核心点：cache 输出和全量重算一致、cache 长度随 token 追加增长、重复工作量明显下降。

```
from math import exp, sqrt
```

```
TOKENS = ["<eos>", "LLM", "inference", "uses", "KV", "cache", "."]
```

```
EMB = {  
    1: [0.20, 0.10, 0.00, 0.30],  
    2: [0.00, 0.30, 0.10, 0.20],  
    3: [0.40, 0.00, 0.20, 0.10],  
    4: [0.10, 0.40, 0.30, 0.00],  
    5: [0.30, 0.20, 0.40, 0.10],  
    6: [0.10, 0.00, 0.20, 0.50],  
}
```

```
def matvec(vec, mat):  
    return [sum(vec[i] * mat[i][j] for i in range(len(vec))) for j in range(len(mat[0]))]
```

```
W_Q = [
```

```

    [0.5, 0.1, 0.0, 0.0],
    [0.0, 0.4, 0.1, 0.0],
    [0.0, 0.0, 0.3, 0.2],
    [0.1, 0.0, 0.0, 0.5],
]
W_K = [
    [0.4, 0.0, 0.1, 0.0],
    [0.1, 0.5, 0.0, 0.0],
    [0.0, 0.1, 0.4, 0.1],
    [0.0, 0.0, 0.2, 0.5],
]
W_V = [
    [0.3, 0.1, 0.0, 0.0],
    [0.0, 0.2, 0.1, 0.0],
    [0.1, 0.0, 0.4, 0.1],
    [0.0, 0.0, 0.1, 0.3],
]

def softmax(xs):
    m = max(xs)
    exps = [exp(x - m) for x in xs]
    total = sum(exps)
    return [x / total for x in exps]

def project(token_id):
    x = EMB[token_id]
    return {
        "q": matvec(x, W_Q),
        "k": matvec(x, W_K),
        "v": matvec(x, W_V),
    }

def attend_last(query, keys, values):
    scale = sqrt(len(query))
    scores = [sum(q * k for q, k in zip(query, key)) / scale for key in keys]
    probs = softmax(scores)
    out = [
        sum(prob * value[j] for prob, value in zip(probs, values))
        for j in range(len(values[0]))
    ]
    return [round(x, 6) for x in out], [round(p, 6) for p in probs]

def full_last_hidden(token_ids):
    projected = [project(tid) for tid in token_ids]
    keys = [item["k"] for item in projected]
    values = [item["v"] for item in projected]
    return attend_last(projected[-1]["q"], keys, values)

def prefill(token_ids):
    projected = [project(tid) for tid in token_ids]

```



```

    return {
        "keys": [item["k"] for item in projected],
        "values": [item["v"] for item in projected],
        "project_calls": len(token_ids),
    }

def cached_decode_step(token_id, cache):
    projected = project(token_id)
    cache["keys"].append(projected["k"])
    cache["values"].append(projected["v"])
    cache["project_calls"] += 1
    hidden, probs = attend_last(projected["q"], cache["keys"], cache["values"])
    return hidden, probs, cache

```

```

prompt = [1, 2, 3]          # LLM inference uses
generated = [4, 5, 6]       # KV cache .
cache = prefill(prompt)

```

```

rows = []
for token_id in generated:
    full_ids = prompt + [row["token_id"] for row in rows] + [token_id]
    full_hidden, full_probs = full_last_hidden(full_ids)
    cache_hidden, cache_probs, cache = cached_decode_step(token_id, cache)
    rows.append({
        "token_id": token_id,
        "token": TOKENS[token_id],
        "seq_len": len(full_ids),
        "match": full_hidden == cache_hidden,
        "hidden": cache_hidden,
        "attn_last": cache_probs[-1],
    })

```

```

prompt_len = len(prompt)
new_tokens = len(generated)
naive_project_calls = sum(prompt_len + i for i in range(1, new_tokens + 1))
kv_project_calls = cache["project_calls"]
d_h = len(EMB[1])
layers = 2
kv_heads = 1
dtype_bytes = 2
kv_bytes = 2 * layers * len(cache["keys"]) * kv_heads * d_h * dtype_bytes

```

```

print("rows=", rows)
print("final_cache_len=", len(cache["keys"]))
print("naive_project_calls=", naive_project_calls)
print("kv_project_calls=", kv_project_calls)
print("saved_project_calls=", naive_project_calls - kv_project_calls)
print("kv_bytes_toy=", kv_bytes)
print("kv_cache_gate=", all(row["match"] for row in rows) and len(cache["keys"]) == prompt_len + new_tokens)

```

一组稳定输出如下：

```

rows= [{'token_id': 4, 'token': 'KV', 'seq_len': 4, 'match': True, 'hidden': [0.06739, 0.057621, 0.09523, 0.059904], 'attn': [0.06739, 0.057621, 0.09523, 0.059904]}, {'token_id': 5, 'token': 'V', 'seq_len': 5, 'match': True, 'hidden': [0.06739, 0.057621, 0.09523, 0.059904], 'attn': [0.06739, 0.057621, 0.09523, 0.059904]}, {'token_id': 6, 'token': '6', 'seq_len': 6, 'match': True, 'hidden': [0.06739, 0.057621, 0.09523, 0.059904], 'attn': [0.06739, 0.057621, 0.09523, 0.059904]}]
final_cache_len= 6

```

```
naive_project_calls= 15
kv_project_calls= 6
saved_project_calls= 9
kv_bytes_toy= 192
kv_cache_gate= True
```

输出怎么读：

1. `match=True` 说明在这个 toy attention 中，缓存路径和完整重算路径的最后 token hidden state 完全一致。
2. `final_cache_len=6` 说明 prompt 3 个 token 加生成 3 个 token 后，cache 长度增长到 6。
3. `naive_project_calls=15, kv_project_calls=6` 说明 cache 避免了重复投影历史 token。
4. `kv_bytes_toy=96` 展示 KV Cache 会真实占用额外内存；真实模型要再乘以层数、KV head、head dim、batch 和 dtype bytes。

9.13 这个版本还缺什么

最小 KV Cache 版本仍然不是 serving engine。

它缺少：

1. batch prefill。
2. batch decode。
3. 多请求 cache 管理。
4. cache 分配和释放。
5. 长度不同请求的 padding 和 position 处理。
6. 显存预算控制。
7. cache block 管理。
8. prefix cache 复用。
9. 请求取消后的 cache 清理。

但它已经解决了最关键的重复计算问题，并为后续批处理打下基础。

9.14 常见误区

误区一：KV Cache 会减少模型权重显存。

不会。KV Cache 是额外缓存，会增加显存占用，但减少重复计算。

误区二：有了 KV Cache，推理一定更省显存。

不是。KV Cache 通常用显存换速度。高并发和长上下文时，KV Cache 可能成为显存瓶颈。

误区三：decode 阶段还要传完整历史 `input_ids`。

使用 cache 时，decode 通常只传最新 token，同时传入 `past_key_values`。完整历史已经体现在 cache 中。

误区四：单请求 cache 管理就等于生产 cache manager。

生产系统要处理多请求、碎片、释放、抢占、复用、准入和显存预算。

误区五：cache 只影响性能，不影响正确性。

`position_ids`、`attention_mask`、cache 更新错误都会直接影响输出正确性。

9.15 面试追问

1. KV Cache 保存的是什么？
2. 为什么 KV Cache 能加速自回归 decode？
3. Prefill 和 Decode 在 cache 使用上有什么区别？
4. 使用 `past_key_values` 后，decode 阶段为什么只输入最新 token？
5. KV Cache 为什么会增加显存占用？
6. 多请求 serving 中 KV Cache 管理有哪些难点？

7. 为什么 `position_ids` 和 `attention_mask` 在 `batched decode` 中很重要?
8. 使用 KV Cache 后输出异常, 你会怎么排查?

参考回答思路:

1. 先说 cache 保存每层历史 token 的 key/value。
2. 再说 prefill 建 cache, decode 复用 cache, 只计算新 token。
3. 然后说明它用显存换计算, 能降低重复计算但增加显存压力。
4. 最后补充生产难点: 多请求分配释放、碎片、prefix 复用、position / mask 正确性和 KV capacity admission。

9.16 小练习

1. 在第 7 章代码基础上加入 `past_key_values`。
2. 打印每层 cache 的 shape, 观察 `seq_len` 如何增长。
3. 对比使用 cache 和不使用 cache 的生成耗时。
4. 故意在 decode 阶段传完整 `generated_ids`, 观察性能和行为变化。
5. 思考如果 batch 中每个请求长度不同, `position_ids` 应该如何设置。
6. 用本章显存公式估算一个 32 层、32 KV heads、head dim 128、FP16、上下文 4096 的单请求 KV Cache 显存。

9.17 本章小结

本章实现了最小 KV Cache。

naive generate loop 每轮重复计算完整历史序列, KV Cache 通过保存历史 key/value, 让 decode 阶段只计算新 token。最小实现依赖 `past_key_values`: prefill 输入完整 prompt 并建立 cache, decode 输入最新 token 并更新 cache。

KV Cache 的 trade-off 很清楚: 它用额外显存换更少重复计算。单请求里这是性能优化; 生产 serving 里它会进一步演变成 KV block、admission、eviction、prefix reuse、paged memory 和跨节点迁移等系统问题。

下一章我们会继续升级, 把单请求 prefill 扩展成 batched prefill, 让多个请求的 prompt 能一起进入模型执行。

第 10 章 实现 batched prefill

上一章我们实现了单请求 KV Cache: 第一次 prefill 输入完整 prompt, 后续 decode 只输入最新 token。本章继续升级, 把多个请求的 prompt 合成一个 batch, 一次性做 prefill。

Batched prefill 是推理框架从“单请求 demo”走向“多请求 serving”的第一步。它能让 GPU 一次处理多个 prompt, 提高吞吐, 但也引入 padding、attention mask、长度差异、首 token logits 选择、padding waste 和 cache 拆分等问题。

一句话概括:

Batched prefill 的目标是把多个请求的 prompt 同时送进模型, 得到每个请求的首 token logits 和初始 KV Cache。

10.0 本讲资料边界与第二轮精修口径

本章第二轮精修前, 先用公开资料校准口径: Transformers tokenizer 文档把 padding / truncation 作为把不同长度样本整理成 batch tensor 的基本步骤; decoder-only generation 中, padding side、attention mask 和最后位置 logits 选择会直接影响生成正确性; Transformers generation 路径会处理一批输入的生成参数、mask 和 cache; vLLM 等 serving engine 进一步在生产中引入 continuous batching、chunked prefill、max batch tokens、KV block 管理和调度策略。

因此, 本章只实现教学版 batched prefill: 多个 prompt padding 成 batch, 一次 forward, 按最后真实 token 取首 token logits, 并得到 batch 维度上的初始 KV Cache。它不实现 continuous batching、chunked prefill、

packed sequence、paged KV block 或 prefix cache。新增 0 依赖 demo 专门验证左 padding、右 padding、attention mask、gather last logits 和 padding waste。

10.1 为什么需要 batched prefill

单请求 prefill 的流程是：

request_1 prompt -> model forward -> request_1 KV Cache -> first token

如果有多个请求，一个个处理会浪费 GPU 并行能力：

request_1 prefill

request_2 prefill

request_3 prefill

Batched prefill 把它们合起来：

[request_1, request_2, request_3] prompts -> model forward -> 每个请求的 KV Cache 和 first token logits

这样能提高 GPU 利用率，尤其是在多个 prompt 长度接近时。

但 LLM prompt 长度往往不同，这就是本章的复杂度来源。

10.2 最小输入示例

假设有三个 prompt：

request_1: "Hello"

request_2: "Explain KV Cache in LLM inference"

request_3: "Write a short poem"

tokenize 后长度可能是：

request_1: 2 tokens

request_2: 6 tokens

request_3: 3 tokens

模型需要张量输入，不能直接输入三个不同长度的 list。因此要 padding 到同一长度。

request_1: [t1, t2, pad, pad, pad, pad]

request_2: [t1, t2, t3, t4, t5, t6]

request_3: [t1, t2, t3, pad, pad, pad]

然后用 attention mask 告诉模型哪些位置是真实 token，哪些位置是 padding。

10.3 padding_side: 左 padding 还是右 padding

在 decoder-only LLM 推理中，padding 方向很重要。

右 padding：

[A, B, C, PAD, PAD]

[D, E, F, G, H]

左 padding：

[PAD, PAD, A, B, C]

[D, E, F, G, H]

很多生成场景更偏向左 padding，因为每个序列的最后一个真实 token 对齐在最右侧，取 logits[:, -1, :] 更方便。

如果使用右 padding，logits[:, -1, :] 对短序列取到的可能是 PAD 位置的 logits，而不是最后一个真实 token 的 logits。

所以教学版 batched prefill 推荐使用左 padding，或者显式根据每个请求长度 gather 最后一个真实 token 的 logits。

10.4 tokenizer 批量编码

Hugging Face tokenizer 支持批量编码：

```
encoded = tokenizer(  
    prompts,  
    return_tensors="pt",  
    padding=True,  
    truncation=True,  
)
```

如果希望左 padding：

```
tokenizer.padding_side = "left"  
tokenizer.pad_token = tokenizer.eos_token
```

很多 causal LM 没有单独的 pad token，教学和推理中常用 eos token 作为 pad token。但要注意：pad token 只是填充，不应该被 attention 当成有效上下文。

因此必须传 attention_mask。

10.5 Batched TokenizerWrapper

我们可以给 TokenizerWrapper 增加批量接口。

```
class TokenizerWrapper:  
    def __init__(self, tokenizer):  
        self.tokenizer = tokenizer  
        self.eos_token_id = tokenizer.eos_token_id  
  
        if self.tokenizer.pad_token_id is None:  
            self.tokenizer.pad_token = self.tokenizer.eos_token  
            self.tokenizer.padding_side = "left"  
  
    def encode_batch(self, prompts):  
        encoded = self.tokenizer(  
            prompts,  
            return_tensors="pt",  
            padding=True,  
            truncation=True,  
        )  
        return encoded.input_ids, encoded.attention_mask
```

返回两个张量：

1. input_ids: [batch_size, max_prompt_len]。
2. attention_mask: [batch_size, max_prompt_len]。

attention mask 中 1 表示真实 token，0 表示 padding。

10.6 修改 ModelWrapper 支持 attention_mask

第 9 章的 ModelWrapper 已经支持 past_key_values。现在 prefill 要支持 padding，所以要传 attention_mask。

```
class ModelWrapper:  
    def __init__(self, model, device="cuda"):  
        self.model = model.to(device)
```

```

self.model.eval()
self.device = device

@torch.inference_mode()
def forward(self, input_ids, attention_mask=None, past_key_values=None):
    input_ids = input_ids.to(self.device)
    if attention_mask is not None:
        attention_mask = attention_mask.to(self.device)

    outputs = self.model(
        input_ids=input_ids,
        attention_mask=attention_mask,
        past_key_values=past_key_values,
        use_cache=True,
    )
    return outputs.logits, outputs.past_key_values

```

这里的重点是：padding 不是简单补齐，还必须让模型知道 padding 不应该参与注意力。

10.7 实现 batched_prefill

最小 batched prefill:

```

def batched_prefill(tokenizer_wrapper, model_wrapper, sampler, prompts):
    input_ids, attention_mask = tokenizer_wrapper.encode_batch(prompts)
    input_ids = input_ids.to(model_wrapper.device)
    attention_mask = attention_mask.to(model_wrapper.device)

    logits, past_key_values = model_wrapper.forward(
        input_ids=input_ids,
        attention_mask=attention_mask,
        past_key_values=None,
    )

    last_logits = gather_last_token_logits(logits, attention_mask)
    next_token_ids = sampler.sample(last_logits)
    return input_ids, attention_mask, next_token_ids, past_key_values

```

如果确定使用左 padding，也可以让 sampler 内部取 `logits[:, -1, :]`。但教学上更推荐保留 `gather_last_token_logits`，因为它能显式处理右 padding 和长度差异。

10.8 右 padding 下如何取最后真实位置

真实长度可以从 attention mask 得到：

```
lengths = attention_mask.sum(dim=-1)
```

最后真实 token 的位置是：

```
last_indices = lengths - 1
```

从 logits 中取每个请求最后真实位置：

```

def gather_last_token_logits(logits, attention_mask):
    batch_size = logits.size(0)
    lengths = attention_mask.sum(dim=-1)
    last_indices = lengths - 1
    batch_indices = torch.arange(batch_size, device=logits.device)
    return logits[batch_indices, last_indices, :]

```

然后 sampler 可以对 [batch_size, vocab_size] 的 logits 采样。

这就是为什么 padding side 和 logits 位置不能忽略。

10.9 Sampler 兼容 batch

第 8 章的 sampler 从 logits[:, -1, :] 取下一 token。为了支持通用 batch, 可以让 sampler 接收两种形状:

1. [batch, seq_len, vocab]: 内部取最后位置。
2. [batch, vocab]: 直接采样。

示例:

```
def sample_next_token(logits, temperature=1.0, top_k=None, top_p=1.0):
    if logits.dim() == 3:
        next_token_logits = logits[:, -1, :]
    else:
        next_token_logits = logits

    if temperature == 0:
        return torch.argmax(next_token_logits, dim=-1, keepdim=True)

    next_token_logits = apply_temperature(next_token_logits, temperature)
    next_token_logits = apply_top_k(next_token_logits, top_k)
    next_token_logits = apply_top_p(next_token_logits, top_p)

    probs = torch.softmax(next_token_logits, dim=-1)
    return torch.multinomial(probs, num_samples=1)
```

如果右 padding 输入没有先 gather, sampler 内部取 logits[:, -1, :] 就会出错。

10.10 生成每个请求的首 token

Batched prefill 的输出 next_token_ids 形状是:

[batch_size, 1]

它表示每个请求的第一个输出 token。

例如:

```
request_1 -> token 318
request_2 -> token 262
request_3 -> token 198
```

这些 token 后续会作为 batched decode 的输入。

对每个请求, 我们还需要维护:

1. 原始 prompt tokens。
2. 已生成 tokens。
3. 当前 attention mask 或有效长度。
4. 对应的 KV Cache。
5. 是否已结束。

所以 batched prefill 的结果不能只是一个 tensor, 还要回填到 request object。

10.11 Request 对象如何保存 prefill 结果

教学版可以这样设计:

```

class RequestState:
    def __init__(self, request_id, prompt):
        self.request_id = request_id
        self.prompt = prompt
        self.input_ids = None
        self.prompt_len = 0
        self.output_ids = []
        self.next_token_id = None
        self.finished = False

```

batched prefill 后:

```

for i, request in enumerate(requests):
    request.input_ids = input_ids[i]
    request.prompt_len = int(attention_mask[i].sum().item())
    request.next_token_id = next_token_ids[i]
    request.output_ids.append(next_token_ids[i].item())

```

真实 serving engine 还会记录每个请求对应的 cache block, 而不是直接把整个 `past_key_values` 放在 request 里。

10.12 `past_key_values` 的 batch 维度

batched prefill 返回的 `past_key_values` 是一个 batch cache。

直觉形状类似:

```

layer_i.key: [batch, num_kv_heads, max_prompt_len, head_dim]
layer_i.value: [batch, num_kv_heads, max_prompt_len, head_dim]

```

如果用了 padding, cache 的 `max_prompt_len` 包含 padded positions; 有效 KV 长度仍然要由 attention mask 或 per-request length 来解释。

教学版可以先整体保存它, 后续 batched decode 继续用同一个 batch 顺序。

但真实 serving engine 中, 请求会动态加入和退出, 不能永远依赖固定 batch 顺序。这就是后续 continuous batching 和 cache manager 要解决的问题。

10.13 padding 带来的浪费

Batched prefill 的问题是 padding 会浪费计算。

设 batch size 为 B , 每个请求真实 prompt 长度为 L_i , batch 最大长度为:

$$L_{\max} = \max_i L_i$$

实际张量 token 数:

$$T_{\mathrm{tensor}} = B \cdot L_{\max}$$

真实有效 token 数:

$$T_{\mathrm{real}} = \sum_i L_i$$

padding waste ratio:

$$R_{\mathrm{pad}} = \frac{T_{\mathrm{tensor}} - T_{\mathrm{real}}}{T_{\mathrm{tensor}}}$$

如果一个 batch 长度分布是:

10, 20, 30, 4000

padding 到 4000 后, 前三个请求会有大量 padding。

有效 token = 10 + 20 + 30 + 4000 = 4060

实际张量 token = 4 * 4000 = 16000

所以生产系统常按长度分桶，或用更高效的变长 attention / packed prefill，减少 padding 浪费。

教学版用 padding 是为了简单，真实系统会进一步优化。

10.14 长 prompt、token budget 和 chunked prefill

如果一个请求 prompt 很长，把它和短请求放在同一个 prefill batch 里，会拖慢整个 batch。

常见策略包括：

1. 长短请求分桶。
2. 限制单轮 prefill token budget。
3. 对长 prompt 做 chunked prefill。
4. 让 decode 请求穿插执行，避免输出卡顿。
5. 使用 prefix cache 复用共享前缀。

一个简单 token budget 可以写成：

$$\sum_{i \in \mathcal{B}} L_i \leq B_{\text{prefill}}$$

其中 B_{prefill} 是本轮允许处理的真实 prompt tokens 预算。生产系统还会结合 padded tokens、KV budget、queue wait 和 decode TPOT 做更复杂的调度。

本章先实现基础 batched prefill，后续讲 vLLM 和高级 serving 时会继续展开这些策略。

10.15 batched prefill 公式、padding 风险和可运行 demo

下面这个 demo 不依赖 PyTorch。它实现一个 toy tokenizer、左右 padding、attention mask、toy model logits 和最后真实位置 gather。

它验证三件事：

1. 左 padding 下取最后位置 logits 可以得到每个请求的首 token。
2. 右 padding 下直接取最后位置会让短请求读到 PAD 位置，结果错误。
3. 用 attention mask gather 最后真实 token 后，右 padding 也能得到正确首 token。

```
VOCAB = {
    "<pad>": 0,
    "<eos>": 1,
    "Hello": 2,
    "LLM": 3,
    "Explain": 4,
    "KV": 5,
    "Cache": 6,
    "in": 7,
    "inference": 8,
    "Write": 9,
    "short": 10,
    "poem": 11,
    "!": 12,
    ".": 13,
}
ID_TO_TOKEN = {idx: tok for tok, idx in VOCAB.items()}
NEXT_TOKEN = {
    VOCAB["LLM"]: VOCAB["!"],
    VOCAB["inference"]: VOCAB["."],
    VOCAB["poem"]: VOCAB["."],
    VOCAB["<pad>"]: VOCAB["<eos>"],
}
```

```
}
```

```
def encode(prompt):
    return [VOCAB[token] for token in prompt.split()]

def pad_batch(sequences, side):
    max_len = max(len(seq) for seq in sequences)
    input_ids = []
    attention_mask = []
    for seq in sequences:
        pad_count = max_len - len(seq)
        if side == "left":
            row = [VOCAB["<pad>"]] * pad_count + seq
            mask = [0] * pad_count + [1] * len(seq)
        else:
            row = seq + [VOCAB["<pad>"]] * pad_count
            mask = [1] * len(seq) + [0] * pad_count
        input_ids.append(row)
        attention_mask.append(mask)
    return input_ids, attention_mask

def toy_logits_for_token(token_id):
    logits = [-10.0] * len(VOCAB)
    logits[NEXT_TOKEN.get(token_id, VOCAB["<eos>"])] = 10.0
    return logits

def toy_model_forward(input_ids):
    return [[toy_logits_for_token(token_id) for token_id in row] for row in input_ids]

def argmax(logits):
    return max(range(len(logits)), key=lambda idx: logits[idx])

def sample_from_last_position(logits_batch):
    return [argmax(row[-1]) for row in logits_batch]

def gather_last_token_logits(logits_batch, attention_mask):
    gathered = []
    last_indices = []
    for logits, mask in zip(logits_batch, attention_mask):
        length = sum(mask)
        last_index = length - 1
        last_indices.append(last_index)
        gathered.append(logits[last_index])
    return gathered, last_indices

def decode_ids(ids):
    return [ID_TO_TOKEN[idx] for idx in ids]
```

```

prompts = [
    "Hello LLM",
    "Explain KV Cache in LLM inference",
    "Write short poem",
]
encoded = [encode(prompt) for prompt in prompts]
lengths = [len(seq) for seq in encoded]

left_ids, left_mask = pad_batch(encoded, "left")
right_ids, right_mask = pad_batch(encoded, "right")

left_logits = toy_model_forward(left_ids)
right_logits = toy_model_forward(right_ids)

left_next = sample_from_last_position(left_logits)
right_naive_next = sample_from_last_position(right_logits)
right_last_logits, right_last_indices = gather_last_token_logits(right_logits, right_mask)
right_gather_next = [argmax(logits) for logits in right_last_logits]

batch_size = len(prompts)
max_len = max(lengths)
real_tokens = sum(lengths)
tensor_tokens = batch_size * max_len
padding_waste = round((tensor_tokens - real_tokens) / tensor_tokens, 3)
layers = 2
kv_heads = 1
head_dim = 4
dtype_bytes = 2
kv_bytes_real = 2 * layers * real_tokens * kv_heads * head_dim * dtype_bytes
kv_bytes_padded = 2 * layers * tensor_tokens * kv_heads * head_dim * dtype_bytes

print("lengths=", lengths)
print("left_input_ids=", left_ids)
print("left_attention_mask=", left_mask)
print("left_next=", decode_ids(left_next))
print("right_naive_next=", decode_ids(right_naive_next))
print("right_last_indices=", right_last_indices)
print("right_gather_next=", decode_ids(right_gather_next))
print("padding_waste=", padding_waste)
print("kv_bytes_real=", kv_bytes_real)
print("kv_bytes_padded=", kv_bytes_padded)
print("batched_prefill_gate=", left_next == right_gather_next and right_naive_next != right_gather_next)

```

一组稳定输出如下：

```

lengths= [2, 6, 3]
left_input_ids= [[0, 0, 0, 0, 2, 3], [4, 5, 6, 7, 3, 8], [0, 0, 0, 9, 10, 11]]
left_attention_mask= [[0, 0, 0, 0, 1, 1], [1, 1, 1, 1, 1, 1], [0, 0, 0, 1, 1, 1]]
left_next= ['!', '.', '.']
right_naive_next= ['<eos>', '.', '<eos>']
right_last_indices= [1, 5, 2]
right_gather_next= ['!', '.', '.']
padding_waste= 0.389
kv_bytes_real= 352

```

```
kv_bytes_padded= 576
batched_prefill_gate= True
```

输出怎么读：

1. 左 padding 下，所有请求最后位置都是真实 token，所以 left_next 正确。
2. 右 padding 下，短请求最后位置是 PAD，right_naive_next 错误地输出 <eos>。
3. 用 attention mask 找最后真实位置后，right_gather_next 和左 padding 结果一致。
4. padding_waste=0.389 说明这个 batch 里接近 39% 的张量位置是 padding。
5. kv_bytes_padded 大于 kv_bytes_real，说明简单 padding 版 cache / 计算会引入额外浪费。

10.16 最小完整代码骨架

把关键部分组合起来：

```
def batched_prefill(tokenizer_wrapper, model_wrapper, sampler, prompts):
    input_ids, attention_mask = tokenizer_wrapper.encode_batch(prompts)
    input_ids = input_ids.to(model_wrapper.device)
    attention_mask = attention_mask.to(model_wrapper.device)

    logits, past_key_values = model_wrapper.forward(
        input_ids=input_ids,
        attention_mask=attention_mask,
        past_key_values=None,
    )

    last_logits = gather_last_token_logits(logits, attention_mask)
    next_token_ids = sampler.sample(last_logits)

    return {
        "input_ids": input_ids,
        "attention_mask": attention_mask,
        "next_token_ids": next_token_ids,
        "past_key_values": past_key_values,
    }
```

如果明确只使用左 padding，可以简化为：

```
next_token_ids = sampler.sample(logits)
```

但只要后续要兼容右 padding、packed prefill 或复杂 request state，显式 gather 最后真实位置会更稳。

10.17 和生产 engine 的差距

这个版本还很简化。

它没有处理：

1. 动态请求加入。
2. 长 prompt chunking。
3. packed sequence。
4. cache block 分配。
5. prefill 和 decode 混合调度。
6. prefix cache。
7. 不同请求 sampling params。
8. 单请求 prefill 失败后的局部错误处理。
9. 显存预算和 admission control。

但它已经把多个 prompt 合成 batch，完成了从单请求到多请求执行的第一步。

10.18 常见误区

误区一：batch 只是把 prompts 放进 list。

真正执行前还要 padding、attention mask、设备放置、logits 位置选择和 cache 管理。

误区二：有 padding 就一定没问题。

padding 必须配合 attention mask，否则模型会把 padding 当成上下文。

误区三：总是可以取 logits[:, -1, :]

只有在左 padding 或无 padding 等情况下通常成立；右 padding 下短序列最后位置可能是 PAD。

误区四：batch 越大越好。

batch 大可能提高吞吐，但也可能增加 padding 浪费、TTFT 和显存压力。

误区五：batched prefill 就等于 continuous batching。

不是。batched prefill 只是批量处理 prompt，continuous batching 是每轮动态维护 running batch。

10.19 面试追问

1. Batched prefill 的目标是什么？
2. 为什么不同长度 prompt 需要 padding？
3. attention mask 在 batched prefill 中有什么作用？
4. 左 padding 和右 padding 对取 logits 有什么影响？
5. 为什么长 prompt 会导致 padding 浪费？
6. batched prefill 返回的 KV Cache 在 batch 维度上如何理解？
7. batched prefill 和 continuous batching 有什么区别？
8. 如何减少 batched prefill 中的 padding 浪费？

参考回答思路：

1. 先说 batched prefill 把多个 prompt 合并，一次 forward 得到首 token logits 和初始 KV Cache。
2. 再说不同长度需要 padding，attention mask 屏蔽 padding。
3. 然后强调 logits 位置选择，左 padding 可以简化最后位置取 logits，右 padding 要按长度 gather。
4. 最后补充工程 trade-off：batch 提升吞吐，但可能增加 padding、显存、TTFT 和 cache 管理复杂度。

10.20 小练习

1. 实现 encode_batch，打印 input_ids 和 attention_mask。
2. 分别用左 padding 和右 padding，观察 logits[:, -1, :] 的含义差异。
3. 写出 gather_last_token_logits 并在右 padding 下测试。
4. 构造长度差异很大的 prompts，计算 padding waste ratio。
5. 思考如何按 prompt 长度分桶以减少 padding。
6. 给一个 token budget，设计一个简单规则，把等待队列中的请求组成 prefill batch。

10.21 本章小结

本章实现了最小 batched prefill。

Batched prefill 把多个 prompt 合并成一个 batch，一次模型 forward 得到每个请求的首 token logits 和初始 KV Cache。关键难点包括 padding、attention mask、最后真实 token logits 的选择、batch 维度上的 cache 理解，以及长短 prompt 混合带来的 padding 浪费。

下一章我们会继续实现 batched decode，让多个已经完成 prefill 的请求能够一起逐 token 生成。

第 11 章 实现 batched decode

上一章我们实现了 batched prefill: 多个请求的 prompt 合成一个 batch, 一次 forward 得到首 token 和初始 KV Cache。本章继续实现 batched decode: 多个正在生成的请求, 每轮一起输入上一个 token, 一起生成下一个 token。

Batched decode 是 continuous batching 的前置能力。先理解固定 batch 的 decode, 才能理解后续 “请求动态加入、动态退出” 的调度系统。

一句话概括:

Batched decode 的目标是让多个请求共享一次 decode forward, 每个请求每轮生成一个新 token, 并维护各自的状态、cache 顺序和停止条件。

11.0 本讲资料边界与第二轮精修口径

本章第二轮精修前, 先用公开资料校准口径: Transformers cache 文档强调自回归 decode 每步只处理新 token, 但 attention mask 需要覆盖 past KV length 加 current tokens; cache 通常按 layer 保存 key/value, 形状类似 [batch_size, num_heads, seq_len, head_dim], 并且 decode 时会把新 key/value 追加到已有 cache。vLLM 文档把更生产级的问题进一步放到 scheduler、KV cache 空间、decode 优先、chunked prefill 和 max_num_batched_tokens 等调度控制上。

因此, 本章只实现教学版 batched decode: 固定 batch、逐请求 finished mask、attention mask 长度增长、batch row 到 request/cache slot 的映射, 以及 compact active batch 的直觉。它不实现真正 continuous batching、PagedAttention block table、动态 admission、per-request logits processor 或跨 worker KV 迁移。新增 0 依赖 demo 用 toy requests 验证固定 batch 的浪费、compact batch 的收益、逐请求停止条件和 cache alignment gate。

11.1 从单请求 decode 到 batched decode

单请求 decode 是:

next_token + past_key_values -> model forward -> logits -> sample -> new_token

多个请求时, 朴素做法是逐个 decode:

```
request_1 decode
request_2 decode
request_3 decode
```

Batched decode 把它们合成:

[token_1, token_2, token_3] + batched KV Cache -> model forward -> [new_1, new_2, new_3]

每轮 decode 中, 每个还活跃的请求通常只贡献一个 token。因此输入形状一般是:

input_ids: [batch_size, 1]

输出 logits 形状是:

logits: [batch_size, 1, vocab_size]

这里的 batch 维度不是普通离线批处理那么简单。第 i 行 token、attention mask、position、KV cache slot 和 request state 必须严格对应。

11.2 固定 batch 的前提

本章先实现固定 batch decode。也就是说:

1. batch 中的请求一开始已经完成 prefill。
2. batch 顺序固定。
3. 每轮只对未完成请求追加输出。

4. 已完成请求可以用 mask 跳过状态更新。
5. 暂时不加入新请求。

真实 continuous batching 会更复杂：每一轮都可能有新请求加入，也可能有旧请求退出。固定 batch 是理解它的第一步。

11.3 RequestState 需要哪些字段

为了维护 decode 状态，我们扩展 RequestState。

```
class RequestState:
    def __init__(self, request_id, prompt, max_new_tokens=128):
        self.request_id = request_id
        self.prompt = prompt
        self.max_new_tokens = max_new_tokens
        self.prompt_len = 0
        self.input_ids = None
        self.output_ids = []
        self.next_token_id = None
        self.finished = False
        self.finish_reason = None

    def append_token(self, token_id):
        self.output_ids.append(int(token_id))
        self.next_token_id = int(token_id)

    def should_stop(self, eos_token_id):
        if self.finished:
            return True
        if self.output_ids and self.output_ids[-1] == eos_token_id:
            self.finished = True
            self.finish_reason = "stop"
            return True
        if len(self.output_ids) >= self.max_new_tokens:
            self.finished = True
            self.finish_reason = "length"
            return True
        return False
```

最重要的是：每个请求都有自己的 output_ids、finished、finish_reason、prompt_len 和 max_new_tokens。Batched decode 的难点不是 forward，而是 batch 中每个请求状态不同。

11.4 batched prefill 后的状态

上一章 batched prefill 返回：

```
{
    "input_ids": input_ids,
    "attention_mask": attention_mask,
    "next_token_ids": next_token_ids,
    "past_key_values": past_key_values,
}
```

我们要把首 token 写回每个 request：

```
def apply_prefill_result(requests, prefill_result, eos_token_id):
    input_ids = prefill_result["input_ids"]
    attention_mask = prefill_result["attention_mask"]
```

```
next_token_ids = prefill_result["next_token_ids"]
```

```
for i, request in enumerate(requests):
    request.input_ids = input_ids[i]
    request.prompt_len = int(attention_mask[i].sum().item())
    token_id = next_token_ids[i].item()
    request.append_token(token_id)
    request.should_stop(eos_token_id)
```

注意：prefill 已经生成了第一个输出 token。decode loop 应该从这些首 token 继续往后生成。

11.5 最小 batched_decode_step

一轮 batched decode 可以写成：

```
def batched_decode_step(model_wrapper, sampler, last_token_ids, past_key_values, attention_mask=None, position_ids=None):
    logits, past_key_values = model_wrapper.forward(
        input_ids=last_token_ids,
        attention_mask=attention_mask,
        position_ids=position_ids,
        past_key_values=past_key_values,
    )
    next_token_ids = sampler.sample(logits)
    return next_token_ids, past_key_values
```

其中：

last_token_ids shape: [batch_size, 1]

next_token_ids shape: [batch_size, 1]

这和单请求 decode 最大区别只是 batch 维度从 1 变成 N，但系统层复杂度高很多。

11.6 attention_mask 和 position_ids 如何增长

在 decode 阶段，每轮只输入一个 token，但模型仍然需要知道它能看见多长历史。

如果当前 cache 中已经有 T_{cache} 个 token，本轮输入 T_{new} 个 token，那么 attention mask 长度应该满足：

$T_{\text{mask}} = T_{\text{cache}} + T_{\text{new}}$

单 token decode 时， $T_{\text{new}}=1$ 。

对 batch 中不同长度请求：

```
request_1 当前长度 10
request_2 当前长度 100
request_3 当前长度 30
```

它们下一 token 的 position 不一样。常见 position ids 直觉类似：

```
position_ids = torch.tensor([[10], [100], [30]], device=device)
```

不同模型对 position 推断和 cache 接口的细节不同。教学版可以依赖模型自动处理，但生产实现不能忽略这个问题。

11.7 固定 batch decode loop

最小 decode loop：

```
def run_batched_decode(requests, model_wrapper, sampler, past_key_values, eos_token_id):
    last_token_ids = torch.tensor(
        [[request.next_token_id] for request in requests],
```



```

        device=model_wrapper.device,
    )

    while not all(request.finished for request in requests):
        active_mask = torch.tensor(
            [not request.finished for request in requests],
            device=model_wrapper.device,
        )

        next_token_ids, past_key_values = batched_decode_step(
            model_wrapper=model_wrapper,
            sampler=sampler,
            last_token_ids=last_token_ids,
            past_key_values=past_key_values,
        )

        for i, request in enumerate(requests):
            if request.finished:
                continue
            token_id = next_token_ids[i].item()
            request.append_token(token_id)
            request.should_stop(eos_token_id)

        last_token_ids = next_token_ids

    return requests

```

这段代码能说明核心流程，但有一个问题：已经 finished 的请求仍然留在 batch 里参与 forward。这会浪费计算，也可能让状态管理变复杂。

11.8 finished mask

可以维护一个 active_mask：

```

request_1 active
request_2 finished
request_3 active

```

教学版最简单做法是：仍然保持固定 batch，但只更新 active 请求的输出。

```

active_mask = torch.tensor(
    [not request.finished for request in requests],
    device=model_wrapper.device,
)

```

这种做法简单，但 finished 请求仍然消耗 decode 计算。

设第 t 轮 active 请求数为 A_t ，原始 batch size 为 B 。固定 batch 每轮处理 B 行：

$$W_{\{\mathrm{fixed}\}} = BT$$

compact active batch 只处理活跃行：

$$W_{\{\mathrm{compact}\}} = \sum_{t=1}^T A_t$$

浪费行数可以粗略写成：

$$W_{\{\mathrm{waste}\}} = W_{\{\mathrm{fixed}\}} - W_{\{\mathrm{compact}\}}$$

真实 GPU kernel 的效率不只由行数决定，但这个公式能帮助理解 finished 请求留在 batch 里的直接代价。

11.9 compact batch

compact batch 的思路是：每轮只把未完成请求放进 batch。

```
active_requests = [request for request in requests if not request.finished]
```

然后构造：

```
last_token_ids = torch.tensor(
    [[request.next_token_id] for request in active_requests],
    device=model_wrapper.device,
)
```

问题是：past_key_values 也必须按同样的 active indices 做选择。

教学版可以先讲概念，真实代码要实现 cache gather：

```
def select_past_key_values(past_key_values, active_indices):
    selected = []
    for layer_past in past_key_values:
        key, value = layer_past
        selected.append((key[active_indices], value[active_indices]))
    return tuple(selected)
```

这说明 batch 顺序和 cache 顺序必须保持一致。

11.10 batch 顺序为什么重要

在 batched decode 中，第 i 行 token 必须对应 past_key_values 的第 i 行 cache。

如果顺序错了，会出现严重问题：

request_1 的 token 用了 request_2 的 KV Cache

request_2 的 token 用了 request_1 的 KV Cache

输出会立刻变乱，而且很难 debug。

所以 serving engine 必须维护清晰映射：

batch row -> request id -> KV cache slot

真实系统里，这个映射通常由 scheduler 和 KV Cache Manager 共同维护。

11.11 每个请求不同 max_new_tokens

Batched decode 中，每个请求可能有不同长度限制。

request_1 max_new_tokens=32

request_2 max_new_tokens=512

request_3 max_new_tokens=128

所以停止判断必须逐请求执行，而不是整个 batch 共用一个计数器。

```
for request in requests:
    request.should_stop(eos_token_id)
```

这也是为什么 RequestState 必须保存自己的 output_ids 和 max_new_tokens。

11.12 不同 sampling params

真实 serving 中，不同请求可能有不同 sampling 参数：

```
request_1 temperature=0
request_2 temperature=0.7, top_p=0.9
request_3 top_k=40
```

教学版可以先让 batch 共用一个 sampler。

生产版 sampler 需要支持 per-request params。实现方式通常是：

1. 对每一行 logits 应用对应参数。
2. 或按参数分组采样。
3. 或实现批量化 logits processor。

这会让 sampler 比第 8 章复杂很多。

11.13 batched decode 公式、状态和可运行 demo

下面这个 demo 不依赖 PyTorch。它模拟 3 个已经完成 prefill 的请求，每个请求有自己的目标输出、prompt length 和 max_new_tokens。demo 同时计算：

1. 固定 batch 每轮处理所有行的工作量。
2. compact batch 每轮只处理 active 行的工作量。
3. 每个请求的 position ids 和 attention mask length 如何增长。
4. batch row / request id / cache slot 是否保持一致。

`EOS = "<eos>"`

```
class RequestState:
    def __init__(self, request_id, prompt_len, target_tokens, max_new_tokens):
        self.request_id = request_id
        self.prompt_len = prompt_len
        self.target_tokens = target_tokens
        self.max_new_tokens = max_new_tokens
        self.output_tokens = [target_tokens[0]] # first token from prefill
        self.next_token = target_tokens[0]
        self.finished = False
        self.finish_reason = None
        self.cache_len = prompt_len + 1
        self.trace = []
        self.should_stop()

    def should_stop(self):
        if self.finished:
            return True
        if self.output_tokens[-1] == EOS:
            self.finished = True
            self.finish_reason = "stop"
            return True
        if len(self.output_tokens) >= self.max_new_tokens:
            self.finished = True
            self.finish_reason = "length"
            return True
        return False

    def decode_next(self):
        if self.finished:
            return self.next_token
        idx = len(self.output_tokens)
```

```

token = self.target_tokens[idx] if idx < len(self.target_tokens) else EOS
position_id = self.prompt_len + len(self.output_tokens) - 1
mask_len = self.prompt_len + len(self.output_tokens)
self.output_tokens.append(token)
self.next_token = token
self.cache_len += 1
self.should_stop()
self.trace.append({
    "position_id": position_id,
    "attention_mask_len": mask_len,
    "token": token,
    "finished": self.finished,
})
return token

requests = [
    RequestState("r1", prompt_len=2, target_tokens=["A", EOS], max_new_tokens=4),
    RequestState("r2", prompt_len=6, target_tokens=["B", "C", "D", EOS], max_new_tokens=4),
    RequestState("r3", prompt_len=3, target_tokens=["X", "Y", EOS], max_new_tokens=3),
]
cache_slots = {request.request_id: f"slot_{request.request_id}" for request in requests}

fixed_rows = 0
compact_rows = 0
step_records = []

while not all(request.finished for request in requests):
    active_requests = [request for request in requests if not request.finished]
    active_ids = [request.request_id for request in active_requests]
    row_to_cache_slot = [(request.request_id, cache_slots[request.request_id]) for request in active_requests]
    alignment_ok = all(request_id in slot for request_id, slot in row_to_cache_slot)

    fixed_rows += len(requests)
    compact_rows += len(active_requests)

    emitted = {}
    position_ids = {}
    mask_lens = {}
    for request in active_requests:
        token = request.decode_next()
        emitted[request.request_id] = token
        position_ids[request.request_id] = request.trace[-1]["position_id"]
        mask_lens[request.request_id] = request.trace[-1]["attention_mask_len"]

    step_records.append({
        "active_ids": active_ids,
        "emitted": emitted,
        "position_ids": position_ids,
        "attention_mask_lens": mask_lens,
        "row_to_cache_slot": row_to_cache_slot,
        "alignment_ok": alignment_ok,
    })

summary = {

```

```

request.request_id: {
    "output": request.output_tokens,
    "finish_reason": request.finish_reason,
    "cache_len": request.cache_len,
}
for request in requests
}

print("step_records=", step_records)
print("summary=", summary)
print("fixed_rows=", fixed_rows)
print("compact_rows=", compact_rows)
print("saved_rows=", fixed_rows - compact_rows)
print("batched_decode_gate=", all(record["alignment_ok"] for record in step_records) and all(r.finished for r in requests))

```

一组稳定输出如下：

```

step_records= [{'active_ids': ['r1', 'r2', 'r3'], 'emitted': {'r1': '<eos>', 'r2': 'C', 'r3': 'Y'}, 'position_ids': {'r1': 0, 'r2': 1, 'r3': 2}, 'cache_len': 4}, {'active_ids': ['r1', 'r2', 'r3'], 'emitted': {'r1': 'A', 'r2': 'B', 'r3': 'D'}, 'position_ids': {'r1': 1, 'r2': 2, 'r3': 3}, 'cache_len': 4}, {'active_ids': ['r1', 'r2', 'r3'], 'emitted': {'r1': '<eos>', 'r2': 'C', 'r3': 'Y'}, 'position_ids': {'r1': 2, 'r2': 3, 'r3': 4}, 'cache_len': 4}]
summary= {'r1': {'output': ['A', '<eos>'], 'finish_reason': 'stop', 'cache_len': 4}, 'r2': {'output': ['B', 'C', 'D', '<eos>'], 'finish_reason': 'stop', 'cache_len': 4}, 'r3': {'output': ['Y', 'Z', 'A', '<eos>'], 'finish_reason': 'stop', 'cache_len': 4}}
fixed_rows= 9
compact_rows= 6
saved_rows= 3
batched_decode_gate= True

```

输出怎么读：

1. 第 1 轮三个请求都活跃，第 2 轮只剩 r2 和 r3，第 3 轮只剩 r2。
2. position_ids 和 attention_mask_lens 按请求自己的长度增长，不是全 batch 共用一个值。
3. fixed_rows=9 表示固定 batch 三轮都处理三行；compact_rows=6 表示只处理活跃行，节省 3 行 decode 工作。
4. row_to_cache_slot 和 alignment_ok=True 表示 batch row 和 cache slot 没有串台。

11.14 固定 batch decode 的完整骨架

下面是一个更完整的教学版骨架：

```

def run_batched_decode(requests, model_wrapper, sampler, past_key_values, eos_token_id):
    last_token_ids = torch.tensor(
        [[request.next_token_id] for request in requests],
        device=model_wrapper.device,
    )

    while True:
        active = [not request.finished for request in requests]
        if not any(active):
            break

        logits, past_key_values = model_wrapper.forward(
            input_ids=last_token_ids,
            attention_mask=None,
            past_key_values=past_key_values,
        )
        next_token_ids = sampler.sample(logits)

        for i, request in enumerate(requests):
            if request.finished:
                continue
            token_id = next_token_ids[i].item()

```

```

        request.append_token(token_id)
        request.should_stop(eos_token_id)

    last_token_ids = next_token_ids

```

```

    return requests, past_key_values

```

它能跑通概念，但不是高效实现，因为 finished 请求仍在 batch 中。

11.15 更接近 engine 的循环

更接近 engine 的写法是每轮重新构造 active batch:

```

while running_requests:
    active_indices = [i for i, r in enumerate(requests) if not r.finished]
    active_requests = [requests[i] for i in active_indices]

    last_token_ids = build_last_token_ids(active_requests)
    active_past = select_past_key_values(past_key_values, active_indices)

    next_token_ids, active_past = batched_decode_step(
        model_wrapper, sampler, last_token_ids, active_past
    )

    update_requests(active_requests, next_token_ids)
    update_global_cache(past_key_values, active_indices, active_past)

```

这比固定 batch 复杂，因为要选择和回写 cache。

真实 vLLM 这类系统不会直接搬整个 past_key_values，而是通过 block table 和 cache block 管理请求到物理 cache 的映射。

11.16 batched decode 和 continuous batching 的区别

Batched decode:

一组请求一起 decode, batch 基本固定

Continuous batching:

每一轮 decode 都可以移除完成请求，也可以加入新请求

区别在于是否动态 admission。

batched decode 只解决“一起生成”的问题；continuous batching 还解决“每轮谁进入、谁退出、资源不够”的问题。

下一章的请求队列和简单 scheduler，就是从 batched decode 走向 continuous batching 的关键一步。

11.17 常见 bug

Bug 一：batch 行和 request 对不上。

表现是输出混乱，不同请求内容串台。

Bug 二：finished 请求还在追加 token。

表现是已经 EOS 或达到长度限制后继续生成。

Bug 三：cache 选择顺序错。

表现是 decode 输出突然变坏，且难以从报错发现。

Bug 四: position_ids 错。

表现是长文本或不同长度 batch 下质量异常。

Bug 五: 所有请求共用一个 max_new_tokens。

表现是短请求被迫生成太长, 或长请求过早停止。

11.18 常见误区

误区一: batched decode 只是把 tokens 拼成 tensor。

还必须维护 request 状态、cache 顺序、停止条件和输出回写。

误区二: finished 请求留在 batch 里没有代价。

它们仍会消耗计算和 cache, 影响吞吐和 TPOT。

误区三: batch 顺序无所谓。

batch row、request id 和 KV Cache 必须严格对应。

误区四: decode 阶段不需要关心 position。

不同请求上下文长度不同, position 处理错误会影响输出正确性。

误区五: 实现 batched decode 就等于实现了 vLLM。

vLLM 还需要 continuous batching、PagedAttention、block manager、worker/executor 和复杂调度。

11.19 面试追问

1. Batched prefill 和 batched decode 的输入输出有什么区别?
2. Batched decode 中 input_ids 的形状通常是什么?
3. 为什么 batch row 和 KV Cache 顺序必须一致?
4. finished 请求应该如何处理?
5. 固定 batch decode 和 continuous batching 有什么区别?
6. batched decode 中 position_ids 为什么重要?
7. 不同请求有不同 sampling params 时 sampler 怎么办?
8. 如果输出串台, 你会优先排查哪些地方?

参考回答思路:

1. 先说 prefill 输入完整 prompt, decode 每轮输入每个请求的最新 token。
2. 再说 decode batch 要维护 request 状态、finished mask、cache 顺序和停止条件。
3. 然后解释 finished 请求可以 mask 或移出, 移出需要同步选择 cache。
4. 最后说明 continuous batching 在每轮动态加入新请求, 是更完整的 serving 调度。

11.20 小练习

1. 用三个 prompt 做 batched prefill, 然后接 batched decode 生成 20 个 token。
2. 给三个请求设置不同 max_new_tokens, 观察 finished 状态变化。
3. 故意打乱 next_token_ids 和 request 的对应关系, 观察输出问题。
4. 实现一个 active_indices 版本, 只对未完成请求做 decode。
5. 思考如何把新请求在 decode 过程中加入 batch。
6. 统计固定 batch 和 compact batch 的 decode row 数差异。

11.21 本章小结

本章实现了最小 batched decode。

Batched decode 让多个请求每轮一起生成一个 token。它的关键不只是模型 forward，还包括 request 状态、batch 顺序、KV Cache 对齐、finished mask、逐请求停止条件、position_ids、attention mask 增长和输出回写。固定 batch decode 是 continuous batching 的基础，但还不能动态加入新请求。

下一章我们会引入请求队列和简单 scheduler，让系统开始具备真正的多请求调度能力。

第 12 章 实现请求队列和简单 scheduler

前面几章已经实现了单请求生成、采样、KV Cache、batched prefill 和 batched decode。但这些还只是固定 batch。真正的 serving engine 面对的是不断到来的请求流：有的请求在等待，有的请求在 prefill，有的请求在 decode，有的请求已经结束。

本章实现最小请求队列和简单 scheduler，让系统从“手动组织 batch”升级为“自动选择请求执行”。

一句话概括：

Scheduler 的核心职责是每一轮决定哪些请求进入 prefill，哪些请求继续 decode，并在吞吐、延迟、显存和公平性之间做取舍。

12.0 本讲资料边界与第二轮精修口径

本讲只实现教学版请求队列和简单 scheduler。它要把 waiting queue、running set、finished set、prefill admission、decode continuation、token budget 和队列指标讲清楚，但不实现真正的 PagedAttention block manager、prefix cache、KV offload、抢占恢复、多 worker 调度、租户配额、streaming backpressure 或分布式执行。

资料校准口径：

1. vLLM 的公开优化文档把调度和 KV 空间联系起来：当 KV cache 空间不足时，系统可能 preempt request；调参会涉及 max_num_seqs、max_num_batched_tokens、GPU cache 空间和并行策略。
2. vLLM V1 的 chunked prefill 说明强调 decode 优先：先安排 pending decode，再用剩余 max_num_batched_tokens 预算安排 prefill；长 prefill 放不下时可以切成 chunk。
3. Orca 的 iteration-level scheduling 思路说明 LLM serving 不能只用静态 batch，而要在每个迭代动态加入和移除请求。
4. 本讲 demo 用纯 Python 模拟调度，不绑定 vLLM、SGLang、TGI 或 TensorRT-LLM 的内部实现；真实系统的 admission control 还要看 KV block、水位线、并发、优先级、取消和超时。

参考资料：

1. vLLM Optimization and Tuning: <https://docs.vllm.ai/en/latest/configuration/optimization.html>
2. Orca: A Distributed Serving System for Transformer-Based Generative Models: <https://www.usenix.org/conference/osdi22/presentation/you>

12.1 为什么需要 scheduler

没有 scheduler 时，我们的流程是人工的：

准备一组 prompts -> batched prefill -> batched decode 到结束

真实服务里，请求是动态到达的：

t=0: request_1 到达
t=1: request_2 到达
t=2: request_3 到达，同时 request_1 正在 decode
t=3: request_4 到达，request_2 prefill 很长

如果每次都等当前 batch 全部完成再处理下一批，新请求的 TTFT 会很差，GPU 也可能出现空洞。

Scheduler 解决的是：

1. 哪些 waiting 请求可以进入 prefill。

2. 哪些 running 请求继续 decode。
3. 一轮最多处理多少 token。
4. 一轮最多处理多少序列。
5. 资源不够时让谁等待。
6. 请求结束后如何释放 slot。

12.2 最小状态集合

我们先维护三个集合：

waiting_queue: 等待 prefill 的请求

running_requests: 已完成 prefill、正在 decode 的请求

finished_requests: 已结束的请求

请求生命周期：

WAITING -> PREFILLING -> DECODING -> FINISHED

对应到代码：

```
from collections import deque
```

```
waiting_queue = deque()
```

```
running_requests = []
```

```
finished_requests = []
```

教学版先不处理失败、取消、超时。真实系统至少还会有 ABORTED、TIMEOUT、FAILED、PREEMPTED 等异常状态。

12.3 调度指标公式

假设请求 i 的到达时间是 a_i ，开始 prefill 的时间是 p_i ，首 token 可返回的时间是 f_i ，完成时间是 e_i 。

队列等待时间：

$$W_i = p_i - a_i$$

TTFT：

$$T_{\{\mathrm{ttft}\}, i} = f_i - a_i$$

端到端延迟：

$$T_{\{\mathrm{e2e}\}, i} = e_i - a_i$$

第 t 轮的活跃序列数：

$$A_t = |\mathcal{R}_t|$$

如果教学版要求最多同时运行 $S_{\{\max\}}$ 条序列，则：

$$A_t \leq S_{\{\max\}}$$

如果本轮 decode 请求数是 D_t ，被选中 prefill 的请求集合是 P_t ，其中请求 i 的 prompt token 数是 L_i ，本轮 token budget 是 $B_{\{\mathrm{tok}\}}$ ，则：

$$D_t + \sum_{i \in P_t} L_i \leq B_{\{\mathrm{tok}\}}$$

这个公式就是最小 scheduler 的核心：decode 每个活跃请求通常消耗 1 个新 token 位置，prefill 一次消耗整段 prompt token。只按请求数限制资源会漏掉长 prompt 的成本。

12.4 RequestState 扩展

为了调度，需要给 request 增加状态和时间字段。

```
class RequestState:
    def __init__(self, request_id, prompt, max_new_tokens=128, arrival_time=0):
        self.request_id = request_id
        self.prompt = prompt
        self.max_new_tokens = max_new_tokens
        self.arrival_time = arrival_time

        self.status = "WAITING"
        self.input_ids = None
        self.output_ids = []
        self.next_token_id = None
        self.finished = False
        self.finish_reason = None

        self.prefill_start_time = None
        self.first_token_time = None
        self.finished_time = None
```

这些时间字段可以计算：

1. queue waiting time。
2. TTFT。
3. total latency。
4. 每轮 active sequence 数。
5. 每轮 scheduled token 数。

最小 scheduler 不一定马上使用所有字段，但设计时应该预留。

12.5 RequestQueue

先封装一个简单 FIFO 队列。

```
class RequestQueue:
    def __init__(self):
        self.queue = deque()

    def push(self, request):
        request.status = "WAITING"
        self.queue.append(request)

    def pop_many(self, max_count):
        selected = []
        while self.queue and len(selected) < max_count:
            selected.append(self.queue.popleft())
        return selected

    def __len__(self):
        return len(self.queue)
```

FIFO 是最简单的公平策略。先来的请求先进入 prefill。

真实系统可能会加入优先级、租户配额、deadline、prompt 长度分桶、aging、防饥饿和长短请求隔离。

12.6 Scheduler 的输入和输出

最小 scheduler 每轮接收：

1. waiting queue。
2. running requests。
3. 最大 prefill 请求数。
4. 最大 running 请求数。
5. 最大本轮 token budget。

输出：

1. 本轮要 prefill 的请求。
2. 本轮要 decode 的请求。
3. 本轮消耗的 decode token 数。
4. 本轮消耗的 prefill token 数。
5. 因资源不足被 deferred 的请求。

class SimpleScheduler:

```
def __init__(
    self,
    max_prefill_requests=4,
    max_running_requests=16,
    max_batched_tokens=2048,
):
    self.max_prefill_requests = max_prefill_requests
    self.max_running_requests = max_running_requests
    self.max_batched_tokens = max_batched_tokens

    def schedule(self, waiting_queue, running_requests):
        decode_requests = [r for r in running_requests if not r.finished]
        decode_tokens = len(decode_requests)

        available_slots = self.max_running_requests - len(decode_requests)
        prefill_budget = self.max_batched_tokens - decode_tokens

        prefill_requests = waiting_queue.pop_admissible(
            max_count=max(0, min(self.max_prefill_requests, available_slots)),
            token_budget=max(0, prefill_budget),
        )

        return prefill_requests, decode_requests
```

这个策略仍然简单：running 中未完成请求继续 decode；只要 running 还有空位，并且 token budget 放得下，就从 waiting 中取请求做 prefill。

12.7 为什么要同时限制 max running 和 token budget

max_running_requests 是序列数保护。它避免同时 decode 的请求太多。

如果无限制地把 waiting 请求都拉进来，会出现：

1. KV Cache 显存爆掉。
2. decode batch 太大。
3. 单轮延迟变长。
4. 已有请求输出抖动。

但只限制请求数不够。两个请求数相同的 batch 可能 token 成本差很多：

batch A: 4 个短 prompt，每个 32 tokens，总 prefill 128 tokens

batch B: 4 个长 prompt, 每个 4096 tokens, 总 prefill 16384 tokens

因此真实系统里限制通常还包括:

1. 最大 token budget。
2. 最大 KV Cache block 数。
3. 最大 prefill token 数。
4. 最大 decode 序列数。
5. 每租户配额。
6. KV cache 水位线。

教学版用 `max_running_requests` 和 `max_batched_tokens` 同时近似资源预算。

12.8 Decode 优先和 Prefill 优先

调度器常见的两种偏向:

1. Prefill 优先: 新请求更快拿到首 token, TTFT 更好。
2. Decode 优先: 已有请求输出更平滑, TPOT 或 ITL 更稳定。

两者没有绝对正确。长 prompt 的 prefill 会占用大量计算, 如果每轮都让长 prefill 插队, 正在 streaming 的用户会感觉输出卡顿; 如果永远优先 decode, 新请求可能排队很久, TTFT 变差。

本章 demo 采用 decode-first 的简化策略:

先安排 running 请求 decode

再用剩余 token budget 安排 waiting 请求 prefill

这和生产系统中的 chunked prefill 思路相近, 但本章不实现把长 prefill 切块。

12.9 Engine 主循环

现在可以写最小 engine loop。

```
class MiniEngine:
    def __init__(self, tokenizer_wrapper, model_wrapper, sampler, scheduler):
        self.tokenizer_wrapper = tokenizer_wrapper
        self.model_wrapper = model_wrapper
        self.sampler = sampler
        self.scheduler = scheduler
        self.waiting_queue = RequestQueue()
        self.running_requests = []
        self.finished_requests = []

    def add_request(self, request):
        self.waiting_queue.push(request)

    def step(self):
        prefill_requests, decode_requests = self.scheduler.schedule(
            self.waiting_queue,
            self.running_requests,
        )

        if decode_requests:
            self.run_decode(decode_requests)

        if prefill_requests:
            self.run_prefill(prefill_requests)

        self.cleanup_finished()
```

step() 表示 engine 的一次调度迭代。真实 serving engine 基本也是类似结构，只是每个函数内部复杂得多。

12.10 run_prefill

run_prefill 做三件事：

1. 对 selected requests 做 batched prefill。
2. 把首 token 写回请求。
3. 把未完成请求加入 running set。

```
def run_prefill(self, requests):
    prompts = [request.prompt for request in requests]
    result = batched_prefill(
        self.tokenizer_wrapper,
        self.model_wrapper,
        self.sampler,
        prompts,
    )

    next_token_ids = result["next_token_ids"]

    for i, request in enumerate(requests):
        request.prefill_start_time = now()
        token_id = next_token_ids[i].item()
        request.append_token(token_id)
        request.first_token_time = now()

        if request.finished:
            request.status = "FINISHED"
            self.finished_requests.append(request)
        else:
            request.status = "DECODING"
            self.running_requests.append(request)
```

这里隐藏了一个大问题：每批 prefill 都会返回自己的 past_key_values。如果要和已有 running requests 合并 decode，就需要统一管理 cache。

教学版可以先把每批 prefill 作为一个 decode group。更真实的版本需要 KV Cache Manager。

12.11 decode group 的概念

为了避免马上实现复杂 cache manager，可以引入 decode group。

一个 decode group 是一组一起 prefill、一起 decode 的请求，以及对应的 batched KV Cache。

```
class DecodeGroup:
    def __init__(self, requests, past_key_values):
        self.requests = requests
        self.past_key_values = past_key_values
```

MiniEngine 维护：

```
self.decode_groups = []
```

每次 prefill 新请求，就创建一个新的 decode group。

这样可以避免不同 prefill batch 的 cache 合并问题。

缺点是：不同 group 不能合并 decode，吞吐不如真正 continuous batching。但作为教学版，它清楚展示了 scheduler、queue 和 batch 的关系。

12.12 run_prefill with decode group

```
def run_prefill(self, requests):
    prompts = [request.prompt for request in requests]
    result = batched_prefill(
        self.tokenizer_wrapper,
        self.model_wrapper,
        self.sampler,
        prompts,
    )

    next_token_ids = result["next_token_ids"]
    past_key_values = result["past_key_values"]

    for i, request in enumerate(requests):
        token_id = next_token_ids[i].item()
        request.append_token(token_id)
        request.status = "DECODING"
        request.first_token_time = now()

    self.decode_groups.append(DecodeGroup(requests, past_key_values))
    self.running_requests.extend(requests)
```

后续 decode 时，对每个 group 分别做 batched decode step。

12.13 run_decode 和 cache 对齐

```
def run_decode(self):
    for group in self.decode_groups:
        active_requests = [r for r in group.requests if not r.finished]
        if not active_requests:
            continue

        last_token_ids = torch.tensor(
            [[r.next_token_id] for r in active_requests],
            device=self.model_wrapper.device,
        )

        next_token_ids, group.past_key_values = batched_decode_step(
            self.model_wrapper,
            self.sampler,
            last_token_ids,
            group.past_key_values,
        )

        for i, request in enumerate(active_requests):
            token_id = next_token_ids[i].item()
            request.append_token(token_id)
            request.should_stop(self.tokenizer_wrapper.eos_token_id)
```

这段代码表达了调度循环，但严格来说还缺少 active requests 对 past_key_values 的选择和回写。如果 group 里有请求 finished 后被跳过，cache batch 维度就可能对不上。

教学版有两种处理方式：

1. 固定 group batch，不移除 finished 请求，只是不更新它们输出。
2. 实现 active index 选择和 cache compact。

为了简单，先用固定 group batch 更安全。

12.14 固定 group 的 decode

固定 group 意味着：group 中的所有请求一直参与 decode forward，但 finished 请求的输出不再更新。

```
def run_decode_group(self, group):
    last_token_ids = torch.tensor(
        [[r.next_token_id] for r in group.requests],
        device=self.model_wrapper.device,
    )

    next_token_ids, group.past_key_values = batched_decode_step(
        self.model_wrapper,
        self.sampler,
        last_token_ids,
        group.past_key_values,
    )

    for i, request in enumerate(group.requests):
        if request.finished:
            continue
        token_id = next_token_ids[i].item()
        request.append_token(token_id)
        request.should_stop(self.tokenizer_wrapper.eos_token_id)
```

这种方式浪费计算，但避免 cache batch 顺序错乱。教学阶段可以接受。

真实系统不会长期这样做，因为 finished 请求继续占用计算和 cache 会拖慢吞吐。

12.15 cleanup_finished

每轮 step 后，要清理已完成请求。

```
def cleanup_finished(self):
    still_running = []
    for request in self.running_requests:
        if request.finished:
            request.status = "FINISHED"
            request.finished_time = now()
            self.finished_requests.append(request)
        else:
            still_running.append(request)
    self.running_requests = still_running

    self.decode_groups = [
        group for group in self.decode_groups
        if any(not r.finished for r in group.requests)
    ]
```

这里仍然没有释放 KV Cache block，因为我们还没有实现真正 cache manager。

在真实系统中，cleanup 必须释放 cache，否则显存会泄漏。

12.16 Scheduler 公式、队列指标和可运行 demo

下面的 demo 不依赖外部库。它模拟 4 个请求动态到达，scheduler 每轮先安排 decode，再用剩余 max_batched_tokens 预算安排 prefill。它验证：

1. FIFO waiting queue。
2. running set 最大并发约束。
3. prefill token budget。
4. queue wait 和 TTFT 统计。
5. 资源不足时 deferred。
6. 所有请求最终进入 finished set。

```
from collections import deque
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class RequestState:
    request_id: str
    arrival_time: int
    prompt_tokens: int
    planned_tokens: list
    max_new_tokens: int
    status: str = "CREATED"
    output_tokens: list = field(default_factory=list)
    prefill_start_time: int | None = None
    first_token_time: int | None = None
    finished_time: int | None = None
    finish_reason: str | None = None

    def emit(self, token, now):
        self.output_tokens.append(token)
        if self.first_token_time is None:
            self.first_token_time = now
        if token == "<eos>":
            self.finish("eos", now)
        elif len(self.output_tokens) >= self.max_new_tokens:
            self.finish("length", now)

    def finish(self, reason, now):
        self.status = "FINISHED"
        self.finish_reason = reason
        self.finished_time = now

    @property
    def finished(self):
        return self.status == "FINISHED"
```

```
class RequestQueue:
    def __init__(self):
        self.queue = deque()

    def push(self, request):
        request.status = "WAITING"
        self.queue.append(request)

    def pop_admissible(self, free_slots, token_budget, max_prefill_requests):
        selected = []
        deferred = []
        used_tokens = 0
```



```

while self.queue and len(selected) < max_prefill_requests and len(selected) < free_slots:
    request = self.queue[0]
    if used_tokens + request.prompt_tokens > token_budget:
        deferred.append((request.request_id, "prefill_token_budget"))
        break
    selected.append(self.queue.popleft())
    used_tokens += request.prompt_tokens
return selected, used_tokens, deferred

def __len__(self):
    return len(self.queue)

def ids(self):
    return [r.request_id for r in self.queue]

class SimpleScheduler:
    def __init__(self, max_running_requests=2, max_prefill_requests=2, max_batched_tokens=8):
        self.max_running_requests = max_running_requests
        self.max_prefill_requests = max_prefill_requests
        self.max_batched_tokens = max_batched_tokens

    def schedule(self, waiting_queue, running_requests):
        decode_requests = [r for r in running_requests if not r.finished]
        decode_tokens = len(decode_requests)
        free_slots = max(0, self.max_running_requests - len(decode_requests))
        prefill_budget = max(0, self.max_batched_tokens - decode_tokens)
        prefill_requests, prefill_tokens, deferred = waiting_queue.pop_admissible(
            free_slots=free_slots,
            token_budget=prefill_budget,
            max_prefill_requests=self.max_prefill_requests,
        )
        return {
            "decode": decode_requests,
            "prefill": prefill_requests,
            "decode_tokens": decode_tokens,
            "prefill_tokens": prefill_tokens,
            "scheduled_tokens": decode_tokens + prefill_tokens,
            "deferred": deferred,
        }

class ToyEngine:
    def __init__(self, requests, scheduler):
        self.arrivals = sorted(requests, key=lambda r: r.arrival_time)
        self.scheduler = scheduler
        self.waiting = RequestQueue()
        self.running = []
        self.finished = []
        self.trace = []
        self.budget_trace = []
        self.max_running_observed = 0
        self.deferred_count = 0

    def add_arrivals(self, now):

```

```

arrived = []
while self.arrivals and self.arrivals[0].arrival_time <= now:
    request = self.arrivals.pop(0)
    self.waiting.push(request)
    arrived.append(request.request_id)
return arrived

def run_prefill(self, requests, now):
    emitted = []
    for request in requests:
        request.status = "PREFILLING"
        request.prefill_start_time = now
        token = request.planned_tokens[0]
        request.emit(token, now)
        if not request.finished:
            request.status = "DECODING"
            self.running.append(request)
        else:
            self.finished.append(request)
        emitted.append((request.request_id, token))
    return emitted

def run_decode(self, requests, now):
    emitted = []
    for request in requests:
        next_index = len(request.output_tokens)
        token = request.planned_tokens[next_index]
        request.emit(token, now)
        emitted.append((request.request_id, token))
    return emitted

def cleanup(self):
    still_running = []
    for request in self.running:
        if request.finished:
            self.finished.append(request)
        else:
            still_running.append(request)
    self.running = still_running

def step(self, now):
    arrived = self.add_arrivals(now)
    plan = self.scheduler.schedule(self.waiting, self.running)
    decoded = self.run_decode(plan["decode"], now)
    prefilled = self.run_prefill(plan["prefill"], now)
    self.cleanup()
    self.max_running_observed = max(self.max_running_observed, len(self.running))
    self.deferred_count += len(plan["deferred"])
    self.budget_trace.append((now, plan["scheduled_tokens"]))
    self.trace.append({
        "t": now,
        "arrived": arrived,
        "decode": [r.request_id for r in plan["decode"]],
        "prefill": [r.request_id for r in plan["prefill"]],
        "tokens": plan["scheduled_tokens"],
    })

```

```

        "waiting": self.waiting.ids(),
        "running": [r.request_id for r in self.running],
        "deferred": plan["deferred"],
        "emitted": decoded + prefilled,
    })

    def run(self):
        now = 0
        while self.arrivals or len(self.waiting) or self.running:
            self.step(now)
            now += 1
        return self

requests = [
    RequestState("r1", arrival_time=0, prompt_tokens=4, planned_tokens=["A", "B", "<eos>"], max_new_tokens=4),
    RequestState("r2", arrival_time=0, prompt_tokens=7, planned_tokens=["C", "<eos>"], max_new_tokens=4),
    RequestState("r3", arrival_time=1, prompt_tokens=3, planned_tokens=["D", "E", "F", "<eos>"], max_new_tokens=5),
    RequestState("r4", arrival_time=2, prompt_tokens=6, planned_tokens=["<eos>"], max_new_tokens=1),
]

engine = ToyEngine(requests, SimpleScheduler()).run()
finished_by_id = {r.request_id: r for r in engine.finished}
request_ids = sorted(finished_by_id)
queue_wait = {
    rid: finished_by_id[rid].prefill_start_time - finished_by_id[rid].arrival_time
    for rid in request_ids
}
ttft_steps = {
    rid: finished_by_id[rid].first_token_time - finished_by_id[rid].arrival_time + 1
    for rid in request_ids
}
finish_reasons = {rid: finished_by_id[rid].finish_reason for rid in request_ids}

budget_ok = all(tokens <= engine.scheduler.max_batched_tokens for _, tokens in engine.budget_trace)
all_finished = not engine.arrivals and len(engine.waiting) == 0 and len(engine.running) == 0
metrics_complete = all(
    r.first_token_time is not None and r.finished_time is not None
    for r in finished_by_id.values()
)
scheduler_gate = all([
    all_finished,
    engine.max_running_observed <= engine.scheduler.max_running_requests,
    budget_ok,
    engine.deferred_count >= 1,
    metrics_complete,
])

print("finished_order=", [r.request_id for r in engine.finished])
print("queue_wait=", queue_wait)
print("ttft_steps=", ttft_steps)
print("finish_reasons=", finish_reasons)
print("budget_trace=", engine.budget_trace)
print("max_running=", engine.max_running_observed)
print("deferred_count=", engine.deferred_count)

```

```
print("trace_tail=", engine.trace[-3:])
print("scheduler_gate=", scheduler_gate)
```

一组稳定输出：

```
finished_order= ['r1', 'r2', 'r4', 'r3']
queue_wait= {'r1': 0, 'r2': 1, 'r3': 2, 'r4': 2}
ttft_steps= {'r1': 1, 'r2': 2, 'r3': 3, 'r4': 3}
finish_reasons= {'r1': 'eos', 'r2': 'eos', 'r3': 'eos', 'r4': 'eos'}
budget_trace= [(0, 4), (1, 8), (2, 2), (3, 3), (4, 7), (5, 1), (6, 1)]
max_running= 2
deferred_count= 2
trace_tail= [{'t': 4, 'arrived': [], 'decode': ['r3'], 'prefill': ['r4'], 'tokens': 7, 'waiting': [], 'running': ['r3']},
scheduler_gate= True
```

这个 demo 的关键不是 token 内容，而是调度证据：

1. r2 在 t=0 没有和 r1 一起 prefill，因为 $4 + 7 > 8$ 。
2. r3 到达较晚，但 prompt 短；在 r2 之后进入 prefill，仍满足 FIFO 前缀约束。
3. r4 的 prompt token 数为 6，在 t=4 与 r3 decode 共享本轮 token budget。
4. 每轮 budget_trace 都不超过 8。
5. max_running 没超过 2。
6. queue_wait 和 ttft_steps 能解释用户体验。

12.17 最小调度循环示例

一个简化运行方式：

```
engine = MiniEngine(tokenizer_wrapper, model_wrapper, sampler, scheduler)
```

```
engine.add_request(RequestState("r1", "Explain KV Cache", max_new_tokens=64))
engine.add_request(RequestState("r2", "Write a haiku", max_new_tokens=32))
```

```
while engine.waiting_queue or engine.running_requests:
    engine.step()
```

```
for request in engine.finished_requests:
    print(request.request_id, request.output_ids, request.finish_reason)
```

这个版本已经有了 serving engine 的基本形状：

1. 请求进入队列。
2. scheduler 选择请求。
3. prefill 建立 cache。
4. decode 逐步生成。
5. 请求结束后清理。

12.18 简单 scheduler 的局限

本章 scheduler 很简单，因此有明显局限：

1. 只粗略按 token 数限制资源，没有估算真实 KV block。
2. 不支持优先级。
3. 不支持抢占。
4. 不支持 chunked prefill。
5. 不支持新请求加入已有 decode batch。
6. 不支持不同请求独立 sampling params。
7. 不支持取消和超时。
8. 不支持 prefix cache 命中对 prompt token 成本的抵扣。

但它已经让系统从固定 batch 变成请求流调度，这是 serving engine 的关键一步。

12.19 从简单 scheduler 到 continuous batching

continuous batching 的核心升级是：每一轮都重新组织 batch。

```
iteration 1: prefill r1, r2
iteration 2: decode r1, r2; prefill r3
iteration 3: decode r1, r2, r3; remove finished r1
iteration 4: decode r2, r3; prefill r4
```

要实现它，需要：

1. 统一 KV Cache Manager。
2. 能把不同请求映射到 cache block。
3. 每轮动态选择 prefill 和 decode。
4. 支持 finished 请求释放资源。
5. 支持 waiting 请求加入 running set。
6. 支持长 prefill 拆 chunk。
7. 支持取消、超时和抢占。

本章的 waiting queue、running set 和 scheduler，就是 continuous batching 的骨架。

12.20 调度策略的 trade-off

调度策略会直接影响用户体验和成本。

优先 prefill：

1. 新请求 TTFT 更好。
2. 可能挤占 decode，导致已有请求输出不平滑。

优先 decode：

1. 已有请求 TPOT 或 ITL 更稳定。
2. 新请求等待更久，TTFT 变差。

限制 batch 小：

1. 单请求延迟可能更好。
2. GPU 利用率可能较低。

限制 batch 大：

1. 吞吐更高。
2. 排队和长尾延迟可能变差。

所以 scheduler 是 serving engine 的核心，而不是简单 FIFO 队列。

12.21 常见误区

误区一：scheduler 就是队列 pop。

队列只是输入，scheduler 还要考虑 running 请求、资源预算、prefill/decode 权衡和完成清理。

误区二：只要 batch 越大越好。

batch 大会增加显存和延迟，也可能让短请求被长请求拖慢。

误区三：finished 请求从列表删掉就够了。

真实系统还必须释放 KV Cache、关闭 stream、记录指标。

误区四：按请求数限制资源足够。

LLM serving 更应该按 token、KV Cache block 和显存预算限制。

误区五：简单 scheduler 和 vLLM scheduler 只差代码量。

真实 scheduler 要处理 continuous batching、PagedAttention、抢占、prefix cache、chunked prefill 和多 worker。

12.22 面试追问

1. 为什么 LLM serving 需要 scheduler?
2. waiting queue、running set、finished set 分别是什么?
3. scheduler 每轮要做哪些决策?
4. 为什么只按请求数限制资源不够?
5. max_running_requests 和 max_batched_tokens 分别保护什么?
6. 优先 prefill 和优先 decode 各有什么影响?
7. 简单 batched decode 和 continuous batching 的区别是什么?
8. finished 请求清理时要释放哪些资源?
9. 如果 TTFT 高但 TPOT 正常, scheduler 可能有什么问题?
10. 如果 TPOT 抖动但 TTFT 正常, scheduler 可能有什么问题?

参考回答思路:

1. 先说请求动态到达, 不能固定 batch, 因此需要 scheduler。
2. 再说 scheduler 管 waiting、running、finished, 决定 prefill 和 decode。
3. 然后解释资源预算: 请求数只是近似, 真实要看 token 和 KV Cache。
4. 接着讲队列指标: queue wait、TTFT、TPOT、active sequences、scheduled tokens 和 deferred count。
5. 最后讲 trade-off: prefill 影响 TTFT, decode 影响 TPOT, scheduler 在两者之间平衡。

12.23 小练习

1. 实现一个 FIFO RequestQueue。
2. 给 SimpleScheduler 增加 max_prefill_tokens 限制。
3. 给请求增加 priority 字段, 实现高优先级请求优先进入 prefill。
4. 记录每个请求的 queue waiting time 和 TTFT。
5. 给 waiting queue 增加 timeout, 并把超时请求标记成 TIMEOUT。
6. 思考如何把新请求加入已有 running batch, 而不是创建新的 decode group。
7. 扩展本章 demo: 把长 prompt 切成多个 prefill chunk。

12.24 本章小结

本章实现了最小请求队列和简单 scheduler。

我们引入了 waiting queue、running requests、finished requests、RequestQueue、SimpleScheduler、token budget、queue wait、TTFT 和 MiniEngine 主循环。这个版本还很简化, 但已经体现了 serving engine 的核心: 请求不是手动组成固定 batch, 而是不断进入队列, 由 scheduler 每轮决定谁 prefill、谁 decode、谁结束。

下一章我们会实现 token streaming, 让生成结果不再等完整答案结束后才返回, 而是每生成一个 token 就增量输出。

第 13 章 实现 token streaming

前面几章已经让 engine 能排队、调度、prefill 和 decode。但到目前为止, 用户通常要等请求完整生成结束后才能拿到答案。本章实现 token streaming: 每生成一个 token, 就尽快把增量文本返回给调用方。

Streaming 是 LLM 产品体验的核心能力。它不一定缩短总生成时间, 但能显著降低用户感知等待, 让用户尽早看到模型开始回答。

一句话概括：

Token streaming 把 decode loop 的每一步输出变成增量事件，而不是等完整答案生成完再一次性返回。

13.0 本讲资料边界与第二轮精修口径

本讲只实现 engine 内部的教学版 token streaming。它会覆盖增量 detokenize、streaming event、finish event、stop sequence 缓冲、客户端取消、慢客户端 backpressure 和最小可运行 demo，但不实现完整 HTTP server、鉴权、SSE 长连接、WebSocket、OpenAI-compatible API、异步 worker、跨进程队列或生产级 Unicode / BPE 增量解码器。

资料校准口径：

1. Hugging Face Transformers 提供 `TextStreamer`、`TextIteratorStreamer` 和 `AsyncTextIteratorStreamer`。公开文档强调 streamer 会在 token 形成可输出文本时输出，并可把可打印文本放进队列供下游应用消费。
2. OpenAI API streaming 文档强调 streaming 可以在完整响应生成完之前开始处理输出，Responses API 使用 typed semantic events，Chat Completions streaming 中增量内容放在 `delta` 字段中。
3. WHATWG HTML 标准的 Server-Sent Events 定义了 `text/event-stream`、`EventSource`、`data`：行和事件分隔等线格式；本章只借用 SSE 直觉，不实现 HTTP 传输层。
4. 本章 demo 用纯 Python list 模拟 stream queue，用 toy tokenizer 模拟 BPE 边界和 stop sequence token，不绑定任何具体 serving runtime。

参考资料：

1. Hugging Face Transformers generation streamers: https://huggingface.co/docs/transformers/main/en/internal/generation_utils#streamers
2. OpenAI API streaming responses: <https://platform.openai.com/docs/guides/streaming-responses>
3. WHATWG HTML Server-sent events: <https://html.spec.whatwg.org/multipage/server-sent-events.html>

13.1 为什么需要 streaming

非流式模式是：

请求进入 -> prefill -> decode 100 个 token -> 返回完整文本

用户看到的是长时间空白，然后一次性出现答案。

流式模式是：

请求进入 -> prefill -> token1 返回 -> token2 返回 -> token3 返回 -> ... -> 结束事件

用户更早看到响应。

在指标上：

1. TTFT 决定多久看到第一个片段。
2. TPOT 决定后续片段生成速度。
3. streaming 抖动决定输出是否平滑。

所以 streaming 不只是接口形式，而是直接影响产品体验。

13.2 Streaming 指标公式

假设请求 i 的到达时间是 a_i ，第一个可见 chunk 发出的时间是 $s_{\{i,1\}}$ ，第 j 个 chunk 发出的时间是 $s_{\{i,j\}}$ ，一共发出 M_i 个文本 chunk。

用户侧 TTFT：

$$T_{\{\mathrm{ttft}, i\}} = s_{\{i,1\}} - a_i$$

相邻 chunk 间隔:

$$\Delta_{i,j} = s_{i,j} - s_{i,j-1}, \quad j=2, \dots, M_i$$

平均 token / chunk 间隔:

$$T_{\{\mathrm{tpot}\}, i} = \frac{1}{M_i - 1} \sum_{j=2}^{M_i} \Delta_{i,j}$$

stream queue backlog:

$$Q_{i,t} = E_{i,t}^{\{\mathrm{produced}\}} - E_{i,t}^{\{\mathrm{consumed}\}}$$

backpressure 门禁:

$$G_{\{\mathrm{bp}\}} = \mathbf{1}[Q_{i,t} \leq Q_{\{\max\}}]$$

这几个量把 streaming 拆成三类问题:

1. 首 token 是否尽早可见。
2. 后续 token 是否平滑。
3. 网络或客户端是否慢到反压 engine。

13.3 streaming 在 engine 中的位置

在 generate loop 里, streaming 插在 token 生成之后:

model forward -> sample token -> append token -> detokenize -> push event

在 engine 主循环中, 它通常发生在 decode step 后:

```
token_id = next_token_ids[i].item()
request.append_token(token_id)
streamer.push_tokens(request)
```

如果是 prefill 产生的首 token, 也应该 streaming:

```
first_token = prefill_next_token
request.append_token(first_token)
streamer.push_tokens(request)
```

否则用户会错过第一个输出片段, TTFT 也无法真实反映到客户端。

13.4 最小 Streamer 接口

先定义一个最小 streamer。

```
class Streamer:
    def __init__(self, tokenizer_wrapper):
        self.tokenizer_wrapper = tokenizer_wrapper

    def push_token(self, request, token_id):
        text = self.tokenizer_wrapper.decode([token_id])
        print(f"{request.request_id}: {text}", end="", flush=True)

    def finish(self, request):
        print(f"\n{request.request_id} finished: {request.finish_reason}")
```

这个版本只是打印到终端, 不是生产实现。

但它体现了 streaming 的最小动作: token 生成后, 马上解码并输出。

13.5 为什么不能简单 decode 单个 token

上面的实现有问题：很多 tokenizer 的 token 到文本不是严格一对一自然字符。

例如：

1. token 可能包含前导空格。
2. 一个中文字符通常可能是一个 token，但不总是。
3. 某些 Unicode 字符可能跨 token。
4. BPE token 可能只是一个词的一部分。
5. special token 不应该直接输出。

所以生产 streaming 通常不是简单 `decode([token_id])`，而是维护一个增量解码状态。

教学版可以先接受简化，但必须知道真实系统要处理 tokenization 边界。

13.6 基于完整输出的增量差分

一个简单且常用的教学方法是：每次 decode 当前完整输出，然后和上一次输出做差分。

```
class TextStreamer:
    def __init__(self, tokenizer_wrapper):
        self.tokenizer_wrapper = tokenizer_wrapper
        self.printed_text = {}

    def push_tokens(self, request):
        full_text = self.tokenizer_wrapper.decode(request.output_ids)
        old_text = self.printed_text.get(request.request_id, "")
        delta = full_text[len(old_text):]
        self.printed_text[request.request_id] = full_text

        if delta:
            self.send_delta(request, delta)

    def send_delta(self, request, delta):
        request.stream_queue.put(StreamingEvent(request.request_id, "delta", delta=delta))
```

这种方法实现简单，能避免很多单 token decode 的边界问题。

缺点是每轮都 decode 完整输出，长输出时效率较差。生产系统会使用更高效的增量 detokenizer。

13.7 StreamingEvent

生产中最好不要直接传字符串，而是定义事件。

```
class StreamingEvent:
    def __init__(self, request_id, event_type, delta="", finish_reason=None, error=None):
        self.request_id = request_id
        self.event_type = event_type
        self.delta = delta
        self.finish_reason = finish_reason
        self.error = error
```

事件类型可以包括：

1. delta: 增量文本。
2. finish: 正常结束、长度结束、取消或错误结束。
3. error: 执行异常。
4. heartbeat: 长连接保活。
5. usage: token 统计或计费信息。

这样 HTTP API、WebSocket、SSE、测试代码都可以消费同一种事件。

13.8 每个请求一个输出队列

为了把模型执行和网络输出解耦，可以给每个请求一个 queue。

```
from queue import Queue
```

```
class RequestState:
    def __init__(self, request_id, prompt, max_new_tokens=128):
        self.request_id = request_id
        self.prompt = prompt
        self.max_new_tokens = max_new_tokens
        self.output_ids = []
        self.stream_queue = Queue(maxsize=128)
        self.finished = False
        self.finish_reason = None
```

Streamer 把事件放入队列：

```
def send_delta(self, request, delta):
    request.stream_queue.put(StreamingEvent(request.request_id, "delta", delta=delta))
```

请求完成时：

```
def finish(self, request):
    request.stream_queue.put(
        StreamingEvent(request.request_id, "finish", finish_reason=request.finish_reason)
    )
```

这样 engine 负责生成，API 层负责从 queue 里取事件并写给客户端。

13.9 在 prefill 中推送首 token

第 12 章的 run_prefill 可以改成：

```
def run_prefill(self, requests):
    prompts = [request.prompt for request in requests]
    result = batched_prefill(
        self.tokenizer_wrapper,
        self.model_wrapper,
        self.sampler,
        prompts,
    )

    next_token_ids = result["next_token_ids"]
    past_key_values = result["past_key_values"]

    for i, request in enumerate(requests):
        token_id = next_token_ids[i].item()
        request.append_token(token_id)
        request.status = "DECODING"
        request.first_token_time = now()
        self.streamer.push_tokens(request)

    self.decode_groups.append(DecodeGroup(requests, past_key_values))
    self.running_requests.extend(requests)
```

首 token 推送出去, TTFT 才真正体现在用户侧。

13.10 在 decode 中推送增量

decode step 后:

```
def run_decode_group(self, group):
    last_token_ids = torch.tensor(
        [[r.next_token_id] for r in group.requests],
        device=self.model_wrapper.device,
    )

    next_token_ids, group.past_key_values = batched_decode_step(
        self.model_wrapper,
        self.sampler,
        last_token_ids,
        group.past_key_values,
    )

    for i, request in enumerate(group.requests):
        if request.finished:
            continue

        token_id = next_token_ids[i].item()
        request.append_token(token_id)
        self.streamer.push_tokens(request)

        if request.should_stop(self.tokenizer_wrapper.eos_token_id):
            self.streamer.finish(request)
```

这里顺序很重要:

1. append token。
2. push delta。
3. 判断是否结束。
4. 推送 finish event。

如果先结束再 push, 可能漏掉最后一个 token。

13.11 finish event

流式响应必须有结束事件。

否则客户端不知道是正常结束、网络断开, 还是服务端异常。

finish event 至少应该包含:

1. request id。
2. finish reason。
3. usage 统计, 比如 input tokens 和 output tokens。
4. 可选错误信息。

示例:

```
{
  "request_id": "r1",
  "event_type": "finish",
  "delta": "",
  "finish_reason": "eos",
  "usage": {
```

```

    "input_tokens": 12,
    "output_tokens": 128
  }
}

```

没有 finish event 的 streaming 是不完整的。

13.12 stop sequence 和 streaming

stop sequence 会让 streaming 变复杂。

假设 stop sequence 是：

```
"</answer>"
```

它可能跨多个 token 出现。

如果每生成一个 token 就立刻输出，可能会把 stop sequence 的一部分发给客户端。

解决办法通常是维护一个小缓冲区：

1. 新 token 先进入 buffer。
2. 检查 buffer 是否可能包含 stop sequence。
3. 确认安全的部分再输出。
4. 如果命中 stop，截断 stop sequence 并结束。

教学版可以暂时只处理 EOS，但面试和生产设计必须知道 stop sequence 的跨 token 问题。

13.13 客户端取消

Streaming 场景中，客户端可能中途断开。

断开后 engine 必须：

1. 标记 request 为 aborted。
2. 停止继续生成。
3. 从 running set 中移除。
4. 释放 KV Cache。
5. 发送或记录 finish reason。
6. 记录指标。

如果只关闭网络连接，但 engine 还在 decode，这个请求就会继续消耗 GPU。

教学版可以给 request 加一个字段：

```

request.aborted = True
request.finish_reason = "client_cancelled"

```

在 scheduler 或 decode loop 中跳过它。

13.14 backpressure

如果客户端读取很慢，stream queue 可能积压。

这叫 backpressure。

简单做法是限制队列长度：

```
request.stream_queue = Queue(maxsize=128)
```

如果队列满了，可以选择：

1. 阻塞 engine，等待客户端读取。
2. 中止请求。

3. 丢弃部分中间事件。
4. 把网络输出放到独立线程或异步任务。

生产系统通常不希望网络慢客户端阻塞模型执行主循环。因此 engine 和 API 输出层要解耦。

13.15 SSE 的直觉

HTTP streaming 常用 SSE，也就是 Server-Sent Events。

SSE 返回类似：

```
event: delta
data: {"delta":"Hello"}
```

```
event: delta
data: {"delta":" world"}
```

```
event: finish
data: {"finish_reason":"eos"}
```

API 层从 request 的 stream queue 里不断取事件，然后写给 HTTP response。

本章不实现完整 HTTP，下一章会讲最小 API 服务。本章只要先把 engine 内部的 token event 做出来。

13.16 Streaming 和指标

streaming 会带来更细的指标：

1. first token time。
2. 每个 token 的时间戳。
3. token 间隔分布。
4. stream queue 长度。
5. 客户端取消率。
6. finish reason 分布。
7. backpressure 次数。

这些指标能帮助排查：

1. 为什么首 token 慢。
2. 为什么输出一顿一顿。
3. 是模型 decode 慢，还是网络输出慢。
4. 哪些请求经常被用户中途取消。

13.17 Token streaming 公式、事件和可运行 demo

下面的 demo 不依赖外部库。它模拟 4 个请求：

1. normal：正常输出 Hello world!，最后 EOS。
2. stop：目标文本里出现跨 token 的 END stop sequence，streaming 只输出 OK，不泄露 stop sequence。
3. cancel：客户端在一个 chunk 后取消，engine 发送 client_cancelled finish event。
4. slow：stream queue 太小，第二个 delta 触发 backpressure。

```
from dataclasses import dataclass, field
```

```
@dataclass
class StreamingEvent:
    request_id: str
    event_type: str
    delta: str = ""
```

```

finish_reason: str | None = None
usage: dict | None = None
error: str | None = None
t: int | None = None

```

@dataclass

```

class RequestState:
    request_id: str
    input_tokens: int
    planned_token_ids: list[int]
    max_stream_events: int = 8
    stop_sequence: str | None = None
    cancel_at_step: int | None = None
    output_ids: list[int] = field(default_factory=list)
    stream_queue: list[StreamingEvent] = field(default_factory=list)
    status: str = "DECODING"
    finish_reason: str | None = None
    first_token_time: int | None = None
    token_times: list[int] = field(default_factory=list)
    backpressure_events: int = 0

    def put_delta_event(self, event):
        if len(self.stream_queue) >= self.max_stream_events:
            self.backpressure_events += 1
            self.status = "FINISHED"
            self.finish_reason = "backpressure"
            return False
        self.stream_queue.append(event)
        return True

    @property
    def finished(self):
        return self.status == "FINISHED"

```

```

class ToyTokenizer:
    def __init__(self):
        self.id_to_text = {
            0: "<eos>",
            1: "Hel",
            2: "lo",
            3: " ",
            4: "world",
            5: "!",
            6: "0",
            7: "K",
            8: "E",
            9: "ND",
            10: "slow",
            11: " client",
            12: " keeps",
        }
        self.eos_token_id = 0

```

```

def decode(self, token_ids):
    return "".join(
        self.id_to_text[token_id]
        for token_id in token_ids
        if token_id != self.eos_token_id
    )

class TextStreamer:
    def __init__(self, tokenizer):
        self.tokenizer = tokenizer
        self.printed_text = {}
        self.trace = []

    def push_tokens(self, request, now):
        full_text = self.tokenizer.decode(request.output_ids)
        safe_full_text = full_text
        finish_reason = None
        stop = request.stop_sequence

        if stop:
            stop_at = full_text.find(stop)
            if stop_at >= 0:
                safe_full_text = full_text[:stop_at]
                finish_reason = "stop_sequence"
            else:
                holdback = max(0, len(stop) - 1)
                if len(full_text) > holdback:
                    safe_full_text = full_text[:-holdback]
                else:
                    safe_full_text = ""

        old_text = self.printed_text.get(request.request_id, "")
        delta = safe_full_text[len(old_text):]
        if delta:
            ok = request.put_delta_event(
                StreamingEvent(request.request_id, "delta", delta=delta, t=now)
            )
            if not ok:
                self.finish(request, now, error="stream_queue_full")
                self.trace.append((now, request.request_id, "backpressure", len(request.stream_queue)))
                return False
            self.printed_text[request.request_id] = safe_full_text
            self.trace.append((now, request.request_id, "delta", delta))

        if finish_reason:
            request.status = "FINISHED"
            request.finish_reason = finish_reason
            self.finish(request, now)
        return True

    def finish(self, request, now, error=None):
        usage = {
            "input_tokens": request.input_tokens,
            "output_tokens": len(request.output_ids),

```

```

    }
    request.stream_queue.append(
        StreamingEvent(
            request.request_id,
            "finish",
            finish_reason=request.finish_reason,
            usage=usage,
            error=error,
            t=now,
        )
    )
    self.trace.append((now, request.request_id, "finish", request.finish_reason))

def run_streaming_demo():
    tokenizer = ToyTokenizer()
    streamer = TextStreamer(tokenizer)
    requests = [
        RequestState("normal", 3, [1, 2, 3, 4, 5, 0]),
        RequestState("stop", 2, [6, 7, 8, 9, 5], stop_sequence="END"),
        RequestState("cancel", 1, [10, 11, 12, 0], cancel_at_step=1),
        RequestState("slow", 1, [10, 11, 12, 0], max_stream_events=1),
    ]

    now = 0
    active = list(requests)
    while active:
        next_active = []
        for request in active:
            if request.cancel_at_step is not None and len(request.output_ids) == request.cancel_at_step:
                request.status = "FINISHED"
                request.finish_reason = "client_cancelled"
                streamer.finish(request, now)
                continue

            token_id = request.planned_token_ids[len(request.output_ids)]
            request.output_ids.append(token_id)
            if request.first_token_time is None:
                request.first_token_time = now
            request.token_times.append(now)

            if token_id == tokenizer.eos_token_id:
                request.status = "FINISHED"
                request.finish_reason = "eos"
                streamer.push_tokens(request, now)
                streamer.finish(request, now)
                continue

            keep_running = streamer.push_tokens(request, now)
            if keep_running and not request.finished and len(request.output_ids) < len(request.planned_token_ids):
                next_active.append(request)
        active = next_active
        now += 1

    summary = {}

```



```

for request in requests:
    deltas = [event.delta for event in request.stream_queue if event.event_type == "delta"]
    finish_events = [event for event in request.stream_queue if event.event_type == "finish"]
    intervals = [b - a for a, b in zip(request.token_times, request.token_times[1:])]
    summary[request.request_id] = {
        "deltas": deltas,
        "finish_reason": request.finish_reason,
        "finish_events": len(finish_events),
        "ttft_steps": request.first_token_time + 1 if request.first_token_time is not None else None,
        "token_intervals": intervals,
        "queue_len": len(request.stream_queue),
        "backpressure_events": request.backpressure_events,
    }

stop_text = "".join(summary["stop"]["deltas"])
gates = {
    "first_token_streamed": summary["normal"]["deltas"][0] == "Hel",
    "full_text_reconstructed": "".join(summary["normal"]["deltas"]) == "Hello world!",
    "finish_event_present": all(value["finish_events"] == 1 for value in summary.values()),
    "stop_sequence_hidden": "END" not in stop_text and stop_text == "OK",
    "client_cancelled": summary["cancel"]["finish_reason"] == "client_cancelled",
    "backpressure_detected": summary["slow"]["backpressure_events"] == 1,
}
gates["streaming_gate"] = all(gates.values())
return summary, streamer.trace, gates

```

```

summary, trace, gates = run_streaming_demo()
print("stream_summary=", summary)
print("trace_tail=", trace[-8:])
print("streaming_gates=", gates)

```

一组稳定输出：

```

stream_summary= {'normal': {'deltas': ['Hel', 'lo', ' ', 'world', '!'], 'finish_reason': 'eos', 'finish_events': 1, 'ttft_steps': 1},
trace_tail= [(1, 'slow', 'backpressure', 2), (2, 'normal', 'delta', ' '), (2, 'stop', 'delta', '0'), (3, 'normal', 'delta', ' '), (3, 'cancel', 'delta', ' '), (3, 'slow', 'backpressure', 2)],
streaming_gates= {'first_token_streamed': True, 'full_text_reconstructed': True, 'finish_event_present': True, 'stop_sequence_hidden': True, 'client_cancelled': False, 'backpressure_detected': False, 'streaming_gate': True}

```

这个 demo 的关键证据：

1. normal 的首个 chunk 是 Hel，说明首 token 被推送。
2. normal 的 deltas 拼回 Hello world!，说明增量文本可重构。
3. 每个请求都有一个 finish event。
4. stop 的 planned tokens 中包含 E 和 ND，但输出只到 OK，说明 stop sequence 没被泄露。
5. cancel 的 finish reason 是 client_cancelled。
6. slow 触发 backpressure，说明慢客户端不会被忽略。

13.18 最小代码骨架

核心组件可以整理成：

```

class StreamingEvent:
    def __init__(self, request_id, event_type, delta="", finish_reason=None):
        self.request_id = request_id
        self.event_type = event_type
        self.delta = delta
        self.finish_reason = finish_reason

```

```

class TextStreamer:
    def __init__(self, tokenizer_wrapper):
        self.tokenizer_wrapper = tokenizer_wrapper
        self.printed_text = {}

    def push_tokens(self, request):
        full_text = self.tokenizer_wrapper.decode(request.output_ids)
        old_text = self.printed_text.get(request.request_id, "")
        delta = full_text[len(old_text):]
        self.printed_text[request.request_id] = full_text
        if delta:
            request.stream_queue.put(
                StreamingEvent(request.request_id, "delta", delta=delta)
            )

    def finish(self, request):
        request.stream_queue.put(
            StreamingEvent(
                request.request_id,
                "finish",
                finish_reason=request.finish_reason,
            )
        )

```

engine 只需要在 prefill 和 decode 生成 token 后调用 `push_tokens`，在请求结束、取消、超时或错误时调用 `finish`。

13.19 常见误区

误区一：streaming 就是每个 token print 一下。

真实 streaming 要处理 detokenization、事件格式、结束事件、取消、backpressure 和错误。

误区二：只要首 token 生成了，用户就一定能看到。

还要看输出通道是否及时 flush，API 层是否阻塞，客户端是否正常读取。

误区三：finish event 可有可无。

没有结束事件，客户端无法区分正常结束和异常断开。

误区四：客户端断开只影响网络层。

断开必须反馈给 engine，否则 GPU 可能继续为已取消请求生成。

误区五：单 token decode 一定等于单字符输出。

token 和自然字符不一定一一对应，增量解码要处理边界问题。

13.20 面试追问

1. Token streaming 在 decode loop 的哪个位置发生？
2. 为什么 streaming 能改善用户体验，即使总耗时不变？
3. 为什么不能简单地 `decode([token_id])` 就输出？
4. finish event 应该包含哪些信息？
5. stop sequence 跨 token 时怎么处理？
6. 客户端取消后 engine 应该做什么？
7. backpressure 是什么，为什么会影响 serving engine？
8. 如何判断输出卡顿是 decode 慢还是网络 streaming 慢？

参考回答思路：

1. 先说 streaming 在 token 生成后、请求结束前，把增量文本推给客户端。
2. 再说它降低感知等待，TTFT 和 token 间隔决定体验。
3. 然后补工程细节：增量 detokenization、finish event、stop sequence、取消、backpressure。
4. 最后讲指标：first token time、token 间隔、stream queue、取消率。

13.21 小练习

1. 在第 12 章 MiniEngine 中加入 TextStreamer。
2. 每次 prefill 产生首 token 后，向 stream queue 推送 delta。
3. 每次 decode 产生新 token 后，向 stream queue 推送 delta。
4. 请求结束时推送 finish event。
5. 模拟客户端不读取 queue，观察 backpressure 可能带来的问题。
6. 构造一个 stop sequence 跨 token 的例子，验证不会把 stop sequence 前缀泄露给客户端。

13.22 本章小结

本章实现了 token streaming。

Streaming 把 decode loop 的每一步输出变成增量事件。它需要处理增量 detokenization、事件队列、finish event、客户端取消、backpressure、stop sequence 和指标。对用户来说，streaming 让模型尽早开始回答；对 engine 来说，它让请求生命周期多了输出通道和取消反馈。

下一章我们会把当前 MiniEngine 包成一个最小 HTTP API 服务，让外部客户端能够通过接口提交请求并接收流式输出。

第 14 章 实现最小 HTTP API 服务

前面我们已经实现了 MiniEngine 的核心链路：请求队列、scheduler、prefill、decode 和 token streaming。但这些还停留在内部对象调用。本章把 MiniEngine 包成最小 HTTP API 服务，让外部客户端可以提交请求并接收结果。

这一章的目标不是做完整 OpenAI-compatible server，而是理解 API 层和 engine 层如何解耦：API 层负责接入、参数校验和连接管理，engine 层负责模型执行和调度。

一句话概括：

最小 HTTP API 服务是 serving engine 的外部入口，它把 JSON 请求转换成 RequestState，并把 engine 的输出事件返回给客户端。

14.0 本讲资料边界与第二轮精修口径

本讲只实现教学版最小 HTTP API 服务。它覆盖请求模型、参数校验、同步 /generate、SSE 风格流式响应、队列准入、错误返回、客户端取消、engine cleanup 和最小可运行 demo，但不实现鉴权、租户配额、TLS、反向代理、OpenAPI schema 完整治理、WebSocket、gRPC、跨进程 engine、分布式 worker、生产级日志脱敏或完整 OpenAI-compatible server。

资料校准口径：

1. FastAPI 官方文档用 Pydantic model 声明 request body，框架会读取 JSON、做类型转换、校验数据，并把 schema 用于 OpenAPI 文档。
2. FastAPI / Starlette 的 StreamingResponse 可以接收普通 generator 或 async generator，把响应体逐段流式返回；长流式 generator 要能让出控制权，才能被取消。
3. WHATWG HTML Server-Sent Events 标准定义了 text/event-stream、event:、data:、空行分隔、UTF-8 编码和 EventSource 消费方式。

4. OpenAI API streaming 文档当前推荐 Responses API 的 typed semantic events; Chat Completions streaming 仍使用 data-only SSE, 增量内容通过 `delta` 字段返回。本章只借鉴这些接口形态, 不复刻完整字段。
5. 本章 demo 用纯 Python 模拟 HTTP handler、engine queue、SSE frame 和取消清理, 不依赖 FastAPI, 也不启动本地服务。

参考资料:

1. FastAPI Request Body: <https://fastapi.tiangolo.com/tutorial/body/>
2. FastAPI StreamingResponse: <https://fastapi.tiangolo.com/advanced/custom-response/#streaming-response>
3. Starlette StreamingResponse: <https://www.starlette.io/responses/#streaming-response>
4. WHATWG HTML Server-Sent Events: <https://html.spec.whatwg.org/multipage/server-sent-events.html>
5. OpenAI API streaming responses: <https://platform.openai.com/docs/guides/streaming-responses>

14.1 HTTP API 在架构中的位置

整体结构可以写成:

Client

- > HTTP API Server
- > RequestState
- > MiniEngine waiting queue
- > Scheduler / Prefill / Decode
- > StreamQueue
- > HTTP Response

API Server 不应该直接调用模型 forward。它应该把请求交给 engine, 然后从 stream queue 或结果对象中读取输出。

这样做的好处是:

1. API 层和模型执行层解耦。
2. engine 可以独立调度多个请求。
3. streaming 输出可以通过队列异步返回。
4. 后续可以替换 HTTP 为 gRPC、WebSocket 或内部 RPC。

14.2 最小接口设计与 API 门禁公式

我们先设计一个简单接口:

POST /generate

请求体:

```
{
  "prompt": "Explain KV Cache",
  "max_new_tokens": 128,
  "temperature": 0.7,
  "stream": false
}
```

非流式响应:

```
{
  "request_id": "r-123",
  "text": "...",
  "finish_reason": "stop",
  "usage": {
```

```

    "output_tokens": 128
}
}

```

流式响应则逐段返回 delta。

把 API 层写成可验收系统，而不是只写一个 handler，可以先定义几个门禁。

请求合法性门禁：

```
G_{\mathrm{valid}}=G_{\mathrm{prompt}}G_{\mathrm{tokens}}G_{\mathrm{sampling}}
```

其中 G_{prompt} 检查 prompt 是非空字符串， G_{tokens} 检查 max_new_tokens 在允许范围内， G_{sampling} 检查 temperature、top-p、top-k 等参数不越界。

队列准入条件：

```
Q_t < Q_{\max}
```

这里 Q_t 是当前等待队列长度， $Q_{\max}$ 是 API 层允许提交给 engine 的最大等待队列长度。超过时应返回 429，而不是继续把请求塞进 engine。

HTTP 成功率和错误率：

$$R_{2xx} = \frac{N_{2xx}}{N}, \quad$$

$$R_{4xx} = \frac{N_{4xx}}{N}, \quad$$

$$R_{5xx} = \frac{N_{5xx}}{N}$$

4xx 通常表示客户端输入、配额或准入失败，5xx 才表示服务端内部失败。把参数错误都变成 500，会让线上告警和容量判断失真。

取消清理门禁：

```
G_{\mathrm{cancel}}=G_{\mathrm{abort}}G_{\mathrm{kfree}}G_{\mathrm{finish}}
```

G_{abort} 表示取消信号能传回 engine， G_{kfree} 表示 KV cache / running slot 被释放， G_{finish} 表示客户端或日志能看到明确 finish reason。

最小 API 总门禁：

```
G_{\mathrm{api}}=G_{\mathrm{valid}}G_{\mathrm{admit}}G_{\mathrm{sync}}G_{\mathrm{stream}}G_{\mathrm{cancel}}
```

一个最小 HTTP API 通过这个门禁，才算真正把请求接入、同步响应、流式响应、过载拒绝和取消清理连成闭环。

14.3 API 层应该校验什么

最小校验包括：

1. prompt 是否存在且为字符串。
2. max_new_tokens 是否为正整数。
3. temperature 是否合法。
4. top_p 是否在合理范围。
5. 请求体大小是否超限。
6. prompt token 数是否超过上下文窗口。
7. 当前队列是否已满。

不要把所有错误都留给 engine。越早拒绝非法请求，越少浪费 GPU 资源。

14.4 FastAPI 最小骨架

教学版可以用 FastAPI 表达 API 层。

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import uuid

```

```
app = FastAPI()
```

```
class GenerateRequest(BaseModel):  
    prompt: str  
    max_new_tokens: int = 128  
    temperature: float = 0.7  
    stream: bool = False
```

```
class GenerateResponse(BaseModel):  
    request_id: str  
    text: str  
    finish_reason: str  
    output_tokens: int
```

然后定义 endpoint:

```
@app.post("/generate")  
def generate(req: GenerateRequest):  
    if not req.prompt:  
        raise HTTPException(status_code=400, detail="prompt is required")  
  
    request_id = str(uuid.uuid4())  
    request_state = RequestState(  
        request_id=request_id,  
        prompt=req.prompt,  
        max_new_tokens=req.max_new_tokens,  
    )  
  
    engine.add_request(request_state)  
    result = wait_until_finished(request_state)  
  
    return GenerateResponse(  
        request_id=request_id,  
        text=result.text,  
        finish_reason=request_state.finish_reason,  
        output_tokens=len(request_state.output_ids),  
    )
```

这只是同步版本，适合理解接口，不适合高并发生产。

14.5 engine 后台循环

API 层把请求放入 queue 后，必须有后台循环持续调用 `engine.step()`。

最小写法:

```
import threading  
import time
```

```
def engine_loop():  
    while True:  
        engine.step()  
        time.sleep(0.001)
```

```
threading.Thread(target=engine_loop, daemon=True).start()
```

这样 API 请求只负责入队，后台线程负责调度和模型执行。

生产系统里会使用更严谨的事件循环、异步队列、worker 线程或独立 engine process。

14.6 等待非流式结果

非流式模式下，HTTP handler 可以等待请求完成。

```
def wait_until_finished(request_state, timeout=60):
    start = time.time()
    while not request_state.finished:
        if time.time() - start > timeout:
            request_state.finished = True
            request_state.finish_reason = "timeout"
            raise HTTPException(status_code=504, detail="request timeout")
        time.sleep(0.01)

    text = tokenizer_wrapper.decode(request_state.output_ids)
    return type("Result", (), {"text": text})
```

这个实现很粗糙，但说明了非流式接口的本质：等待 engine 生成完，再一次性返回完整文本。

缺点是：

1. HTTP 连接占用时间长。
2. 用户长时间看不到输出。
3. handler 线程被阻塞。
4. 超时和取消处理更复杂。

14.7 SSE 流式响应

流式 API 可以用 SSE 表达。

FastAPI 中可以返回 StreamingResponse：

```
from fastapi.responses import StreamingResponse
import json
```

```
def sse_format(event_name, data):
    payload = json.dumps(data, ensure_ascii=False)
    return f"event: {event_name}\ndata: {payload}\n\n"
```

SSE 的关键不是“每次写一行字符串”，而是线格式稳定：

1. Content-Type 应是 text/event-stream。
2. 一个事件可以包含 event：和一行或多行 data：。
3. 事件之间用空行分隔。
4. 客户端断开时，服务端 generator 必须退出，并把取消信号传回 engine。

流式生成器：

```
def stream_events(request_state):
    while True:
        event = request_state.stream_queue.get()

        if event.error:
```

```

        yield sse_format("error", {"error": event.error})
        break

    if event.finish_reason is not None:
        yield sse_format("finish", {
            "request_id": event.request_id,
            "delta": "",
            "finish_reason": event.finish_reason,
        })
        break

    yield sse_format("delta", {
        "request_id": event.request_id,
        "delta": event.delta,
        "finish_reason": None,
    })

```

endpoint:

```

@app.post("/generate_stream")
def generate_stream(req: GenerateRequest):
    request_id = str(uuid.uuid4())
    request_state = RequestState(
        request_id=request_id,
        prompt=req.prompt,
        max_new_tokens=req.max_new_tokens,
    )

    engine.add_request(request_state)
    return StreamingResponse(
        stream_events(request_state),
        media_type="text/event-stream",
    )

```

这就是最小 streaming API。

14.8 请求取消如何传回 engine

HTTP 客户端可能中途断开。API 层应该把取消信号传给 engine。

教学版可以在生成器退出时标记：

```

def stream_events(request_state):
    try:
        while True:
            event = request_state.stream_queue.get()
            if event.finish_reason is not None:
                yield sse_format("finish", {"finish_reason": event.finish_reason})
                break
            yield sse_format("delta", {"delta": event.delta})
    finally:
        if not request_state.finished:
            request_state.aborted = True
            request_state.finish_reason = "abort"

```

engine 的 scheduler 或 decode loop 应该检查 request_state.aborted，停止继续生成并释放资源。

如果取消只停在 API 层，GPU 会继续做无用工作。

生产实现还要区分几种取消来源：

1. 客户端主动断开连接。
2. API 层等待超时。
3. engine 内部错误导致请求终止。
4. admission 或限流阶段直接拒绝。

这些路径最后都应该进入同一个 cleanup 收敛点：移出 waiting / running 集合，释放 KV cache，写 finish reason，更新 metrics 和 trace。

14.9 API 层和 sampling params

第 8 章我们实现了 sampling params。HTTP 请求里的参数应该传入 RequestState。

```
class GenerateRequest(BaseModel):
    prompt: str
    max_new_tokens: int = 128
    temperature: float = 0.7
    top_p: float = 1.0
    top_k: int | None = None
    stream: bool = False
```

RequestState 可以保存：

```
request_state.sampling_params = SamplingParams(
    temperature=req.temperature,
    top_p=req.top_p,
    top_k=req.top_k,
)
```

教学版 engine 可能仍然共用一个 sampler。生产 engine 需要 per-request sampling params。

14.10 API 层的并发风险

最小 HTTP 服务有很多并发风险：

1. 多个 handler 同时访问 engine。
2. waiting queue 需要线程安全。
3. request 状态被 engine 和 API 同时读写。
4. stream_queue 可能阻塞。
5. engine_loop 异常后请求永远等待。
6. 模型执行不能被多个线程同时乱调用。

教学版可以用锁保护入队：

```
engine_lock = threading.Lock()
```

```
with engine_lock:
    engine.add_request(request_state)
```

生产版通常会让 engine 单线程消费队列，API 层通过线程安全或异步队列提交请求。

14.11 队列满了怎么办

API 层必须有 admission control。

如果 waiting queue 已经太长，还继续接收请求，会导致所有请求 TTFT 变差。

最小策略：

```
if len(engine.waiting_queue) > MAX_QUEUE_SIZE:
    raise HTTPException(status_code=429, detail="server is busy")
```

更完整策略包括：

1. 按租户限流。
2. 按 prompt token 数限流。
3. 按预估 KV Cache 限流。
4. 按优先级队列接入。
5. 返回排队位置或建议重试时间。

如果没有 admission control，高峰期系统可能从慢变成雪崩。

14.12 OpenAI-compatible API 的差异

真实项目常提供 OpenAI-compatible API，例如：

POST /v1/responses

POST /v1/chat/completions

POST /v1/completions

它比本章接口复杂，因为要处理：

1. messages 和 chat template。
2. Responses API 的 typed semantic events。
3. Chat Completions 的 choices 和 delta 格式。
4. usage 统计和流式 usage 汇总。
5. stop、logprobs、seed、tool call 等参数。
6. 多模型字段、模型别名和兼容版本。
7. 错误码、错误 body、finish reason 和取消语义兼容。

本章先用 /generate 是为了专注 engine 和 API 的连接。

理解后再做兼容层，会清楚很多。

14.13 最小服务文件结构

一个教学项目可以这样拆：

mini_engine/	
engine.py	MiniEngine, Scheduler, RequestQueue
request.py	RequestState, StreamingEvent
model.py	ModelWrapper
tokenizer.py	TokenizerWrapper
sampler.py	Sampler, SamplingParams
api.py	FastAPI endpoints

不要一开始把所有代码写进一个 server.py。拆分不是为了复杂，而是为了让每个模块职责清楚。

14.14 HTTP API 公式、接口和可运行 demo

下面的 demo 不依赖外部库，也不启动真实 HTTP server。它模拟一个最小 API 层：

1. 校验 JSON 请求，非法 prompt 返回 400。
2. 调用 engine 准入，队列满返回 429。
3. 非流式 /generate 一次性返回完整文本。
4. 流式 /generate_stream 返回 SSE frame。
5. 客户端取消时把 abort 信号传回 engine，并释放 KV。

```
from dataclasses import dataclass, field
import json
```

```
@dataclass
```

```
class GenerateRequest:
```

```
    prompt: str
    max_new_tokens: int = 16
    temperature: float = 0.7
    top_p: float = 1.0
    stream: bool = False
```

```
@dataclass
```

```
class RequestState:
```

```
    request_id: str
    prompt: str
    max_new_tokens: int
    temperature: float
    top_p: float
    stream: bool
    output_tokens: list[str] = field(default_factory=list)
    status: str = "WAITING"
    finish_reason: str | None = None
    kv_allocated: bool = True
```

```
class ToyEngine:
```

```
    def __init__(self, max_queue_size=2):
        self.max_queue_size = max_queue_size
        self.waiting_queue = []
        self.released_kv = []
        self.trace = []
```

```
    def can_accept(self):
        return len(self.waiting_queue) < self.max_queue_size
```

```
    def add_request(self, state):
        if not self.can_accept():
            return False
        self.waiting_queue.append(state)
        self.trace.append((state.request_id, "queued"))
        return True
```

```
    def planned_text(self, state):
        prompt = state.prompt.lower()
        if "sync" in prompt:
            return "SYNC"
        if "stream" in prompt:
            return "API!"
        if "cancel" in prompt:
            return "CANCEL"
        return "OK"
```

```
    def complete(self, state):
        state.status = "RUNNING"
        text = self.planned_text(state)[:state.max_new_tokens]
        state.output_tokens.extend(text)
        state.finish_reason = "length"
```

```

state.status = "FINISHED"
self.release_kv(state)
return text

def release_kv(self, state):
    if state.kv_allocated:
        state.kv_allocated = False
        self.released_kv.append(state.request_id)
    if state in self.waiting_queue:
        self.waiting_queue.remove(state)
    self.trace.append((state.request_id, "released"))

def cancel(self, state):
    state.status = "FINISHED"
    state.finish_reason = "client_cancelled"
    self.release_kv(state)
    self.trace.append((state.request_id, "cancelled"))

class MinimalHTTPAPI:
    def __init__(self, engine):
        self.engine = engine
        self.next_id = 1

    def error(self, status_code, message):
        return {
            "status_code": status_code,
            "body": {"error": {"message": message}},
        }

    def validate(self, payload):
        prompt = payload.get("prompt")
        if not isinstance(prompt, str) or not prompt.strip():
            return None, self.error(400, "prompt must be a non-empty string")

        max_new_tokens = payload.get("max_new_tokens", 16)
        if not isinstance(max_new_tokens, int) or not 1 <= max_new_tokens <= 64:
            return None, self.error(400, "max_new_tokens must be an integer in [1, 64]")

        temperature = payload.get("temperature", 0.7)
        if not isinstance(temperature, (int, float)) or not 0 <= temperature <= 2:
            return None, self.error(400, "temperature must be in [0, 2]")

        top_p = payload.get("top_p", 1.0)
        if not isinstance(top_p, (int, float)) or not 0 < top_p <= 1:
            return None, self.error(400, "top_p must be in (0, 1]")

        stream = payload.get("stream", False)
        if not isinstance(stream, bool):
            return None, self.error(400, "stream must be a boolean")

        return GenerateRequest(
            prompt=prompt.strip(),
            max_new_tokens=max_new_tokens,
            temperature=float(temperature),

```

```

        top_p=float(top_p),
        stream=stream,
    ), None

def new_request_state(self, request):
    request_id = f"req-{self.next_id:04d}"
    self.next_id += 1
    return RequestState(
        request_id=request_id,
        prompt=request.prompt,
        max_new_tokens=request.max_new_tokens,
        temperature=request.temperature,
        top_p=request.top_p,
        stream=request.stream,
    )

def submit_or_429(self, payload):
    request, error_response = self.validate(payload)
    if error_response is not None:
        return None, error_response
    if not self.engine.can_accept():
        return None, self.error(429, "server is busy")

    state = self.new_request_state(request)
    self.engine.add_request(state)
    return state, None

def generate(self, payload):
    state, error_response = self.submit_or_429(payload)
    if error_response is not None:
        return error_response

    text = self.engine.complete(state)
    return {
        "status_code": 200,
        "body": {
            "request_id": state.request_id,
            "text": text,
            "finish_reason": state.finish_reason,
            "usage": {"output_tokens": len(state.output_tokens)},
        },
    }

def sse_frame(self, event_name, payload):
    data = json.dumps(payload, ensure_ascii=False)
    return f"event: {event_name}\ndata: {data}\n\n"

def generate_stream(self, payload):
    state, error_response = self.submit_or_429(payload)
    if error_response is not None:
        yield self.sse_frame("error", error_response["body"])
        return

    text = self.engine.planned_text(state)[:state.max_new_tokens]
    state.status = "RUNNING"

```

```

    for token in text:
        state.output_tokens.append(token)
        yield self.sse_frame("delta", {"delta": token, "event": "delta"})

    state.finish_reason = "length"
    state.status = "FINISHED"
    yield self.sse_frame(
        "finish",
        {
            "event": "finish",
            "finish_reason": state.finish_reason,
            "request_id": state.request_id,
            "usage": {"output_tokens": len(state.output_tokens)},
        },
    )
    self.engine.release_kv(state)

def stream_until_cancel(self, payload, cancel_after_frames):
    state, error_response = self.submit_or_429(payload)
    if error_response is not None:
        return {"finish_reason": "not_started", "frames": []}

    frames = []
    text = self.engine.planned_text(state)[:state.max_new_tokens]
    state.status = "RUNNING"
    for token in text:
        state.output_tokens.append(token)
        frames.append(self.sse_frame("delta", {"delta": token, "event": "delta"}))
        if len(frames) >= cancel_after_frames:
            self.engine.cancel(state)
            frames.append(
                self.sse_frame(
                    "finish",
                    {
                        "event": "finish",
                        "finish_reason": state.finish_reason,
                        "request_id": state.request_id,
                    },
                )
            )
            break
    return {"finish_reason": state.finish_reason, "frames": frames}

def run_http_api_demo():
    engine = ToyEngine(max_queue_size=2)
    api = MinimalHTTAPI(engine)

    sync_response = api.generate(
        {"prompt": "sync request", "max_new_tokens": 4, "temperature": 0.0}
    )
    stream_frames = list(
        api.generate_stream({"prompt": "stream request", "max_new_tokens": 4, "stream": True})
    )
    invalid_response = api.generate({"prompt": "", "max_new_tokens": 4})

```

```

engine.waiting_queue = [
    RequestState("busy-a", "queued", 1, 0.7, 1.0, False),
    RequestState("busy-b", "queued", 1, 0.7, 1.0, False),
]
busy_response = api.generate({"prompt": "busy request", "max_new_tokens": 4})
engine.waiting_queue = []

cancel_result = api.stream_until_cancel(
    {"prompt": "cancel request", "max_new_tokens": 6, "stream": True},
    cancel_after_frames=1,
)

summary = {
    "sync_response": sync_response,
    "stream_frame_count": len(stream_frames),
    "stream_first_frame": stream_frames[0].strip(),
    "invalid_response": invalid_response,
    "busy_response": busy_response,
    "cancel_finish_reason": cancel_result["finish_reason"],
    "kv_released": engine.released_kv,
}
gates = {
    "sync_200": sync_response["status_code"] == 200
    and sync_response["body"]["text"] == "SYNC",
    "stream_sse_frames": len(stream_frames) == 5
    and stream_frames[0].startswith("event: delta\n")
    and stream_frames[-1].startswith("event: finish\n"),
    "invalid_400": invalid_response["status_code"] == 400,
    "queue_full_429": busy_response["status_code"] == 429,
    "cancel_cleanup": cancel_result["finish_reason"] == "client_cancelled"
    and "req-0003" in engine.released_kv,
}
gates["http_api_gate"] = all(gates.values())
return summary, gates

```

```

summary, gates = run_http_api_demo()
print("api_summary=", summary)
print("http_api_gates=", gates)

```

一组稳定输出:

```

api_summary= {'sync_response': {'status_code': 200, 'body': {'request_id': 'req-0001', 'text': 'SYNC', 'finish_reason': 'empty string'}}}, 'busy_response': {'status_code': 429, 'body': {'error': {'message': 'server is busy'}}}, 'cancel_finish_reason': 'client_cancelled', 'req-0002', 'req-0003']}
http_api_gates= {'sync_200': True, 'stream_sse_frames': True, 'invalid_400': True, 'queue_full_429': True, 'cancel_cleanup': True, 'req-0003': True}

```

这个 demo 的关键证据:

1. sync_response 返回 200、完整文本、finish reason 和 usage。
2. stream_frame_count 是 5，说明 4 个 delta 加 1 个 finish event 都被写成 SSE frame。
3. 空 prompt 返回 400，不进入 engine。
4. 队列满返回 429，不创建新 request id。
5. 取消请求的 finish reason 是 client_cancelled，并且 kv_released 包含对应 request id。

14.15 常见误区

误区一：HTTP API 直接调用 `model.generate()` 就是 serving engine。

这只是 Web demo，不具备请求队列、scheduler、KV Cache 管理和 streaming 生命周期。

误区二：非流式接口更简单，所以生产只做非流式就好。

非流式用户体验差，长请求更容易超时，也不利于取消反馈。

误区三：API 层不需要关心队列长度。

API 层必须做 admission control，否则过载时会拖垮系统。

误区四：客户端断开只要 HTTP 连接关闭就行。

必须通知 engine 取消请求并释放资源。

误区五：OpenAI-compatible 只是改字段名。

兼容接口还涉及 messages、stream delta、usage、finish reason、错误码和参数语义。

14.16 面试追问

1. HTTP API 层和 serving engine 层应该如何解耦？
2. 非流式 `/generate` 的执行流程是什么？
3. 流式 SSE 接口如何从 engine 获取 token delta？
4. 客户端断开时如何通知 engine？
5. 为什么 API 层要做 admission control？
6. 多线程 API 访问 engine 有哪些并发风险？
7. OpenAI-compatible API 比简单 `/generate` 多了哪些复杂度？
8. 为什么不建议 API handler 直接调用模型 forward？

参考回答思路：

1. 先说 API 层负责协议、校验、连接和响应，engine 层负责调度和模型执行。
2. 再说请求转成 RequestState 入队，engine 后台 loop 消费队列。
3. 然后说明 streaming 通过 stream queue 或事件流返回。
4. 最后补取消、限流、线程安全和 OpenAI-compatible 兼容细节。

14.17 小练习

1. 用 FastAPI 写一个 `/generate endpoint`，把 prompt 转成 RequestState。
2. 写一个后台线程循环调用 `engine.step()`。
3. 实现非流式等待，直到 request finished 后返回完整文本。
4. 实现 SSE streaming，从 `stream_queue` 中读取事件。
5. 给 API 加上最大队列长度限制，超过后返回 429。

14.18 本章小结

本章把 MiniEngine 包成了最小 HTTP API 服务。

HTTP API 层负责接收 JSON 请求、校验参数、创建 RequestState、提交到 engine，并把非流式结果或流式事件返回给客户端。engine 层继续负责队列、调度、prefill、decode 和 streaming。两层通过请求对象和 stream queue 解耦。

下一章我们会做压测，系统分析 TTFT、TPOT、吞吐和显存，看看这个最小推理框架的瓶颈在哪里。

第 15 章 压测 TTFT、TPOT、吞吐和显存

第二部分前面几章从 0 搭了一个最小推理框架：tokenizer、model wrapper、generate loop、sampling、KV Cache、batched prefill、batched decode、scheduler、streaming 和 HTTP API。本章做阶段性收尾：如何压测这个系统，如何看 TTFT、TPOT、吞吐和显存。

做推理框架不能只说“能跑”。能跑只是第一步。真正的工程问题是：首 token 多久返回？后续 token 多快？并发上来后吞吐如何？显存为什么涨？瓶颈在 prefill、decode、scheduler，还是 streaming？

一句话概括：

压测的目的不是得到一个漂亮 QPS，而是把请求生命周期拆成可观测指标，定位 serving engine 的真实性能瓶颈。

15.0 本讲资料边界与第二轮精修口径

本讲只做教学版 LLM serving 压测审计。它覆盖 TTFT、TPOT / inter-token latency、E2E latency、input / output tokens/s、queue length、active requests、KV cache 显存、allocated / reserved memory、workload 描述、瓶颈归因和最小可运行 demo，但不实现真实 GPU benchmark、压测平台、分布式压测、Prometheus / OpenTelemetry 接入、真实模型加载、生产级 autoscaling、租户限流或完整容量规划。

资料校准口径：

1. vLLM 公开 metrics 文档把 time to first token、time per output token、end-to-end request latency、request queue / prefill / decode 时间、running / waiting / swapped 请求数和 KV cache usage 作为 serving 观测指标。
2. NVIDIA GenAI-Perf 面向生成式模型的压测报告包含 request latency、time to first token、inter token latency、input / output sequence length、request throughput 和 output token throughput 等指标。
3. PyTorch CUDA memory 文档区分 memory_allocated() 与 memory_reserved(); reserved memory 可能来自 caching allocator，不等同于仍被张量实际占用。
4. 本章 demo 用手工 toy trace 模拟请求到达、排队、prefill、decode、finish 和显存采样，不绑定 vLLM、Triton、TGI、SGLang 或某个 GPU。

参考资料：

1. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics.html>
2. NVIDIA GenAI-Perf: https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_benchmark/genai_perf.html
3. PyTorch CUDA memory management: <https://docs.pytorch.org/docs/stable/notes/cuda.html#memory-management>

15.1 压测要回答什么问题

LLM serving 压测至少要回答：

1. 单请求延迟是多少。
2. 并发增加后 TTFT 如何变化。
3. TPOT 是否稳定。
4. 每秒能处理多少 input tokens。
5. 每秒能生成多少 output tokens。
6. waiting queue 是否增长。
7. GPU 显存是否接近上限。
8. KV Cache 占用是否随并发线性增长。
9. p95/p99 是否出现长尾。
10. 错误、超时、取消是否增多。

如果压测只输出一个 QPS，就没有抓住 LLM serving 的重点。

15.2 指标定义回顾

先明确几个核心指标。设请求 i 的到达时间为 a_i ，进入 prefill 的时间为 p_i ，第一个对客户端可见的 token 时间为 f_i ，第 j 个输出 token 的时间为 $s_{i,j}$ ，请求结束时间为 e_i 。输入 token 数为 x_i ，输出 token 数为 y_i 。

TTFT:

$$T_{\{\mathrm{ttft}\},i}=f_i-a_i$$

它衡量用户等多久看到第一个 token，通常包含排队、tokenization、prefill、首 token 采样和首次 flush。

TPOT:

$$T_{\{\mathrm{tpot}\},i}=\frac{s_{i,y_i}-s_{i,1}}{y_i-1}, \quad y_i>1$$

如果 $y_i=1$ ，TPOT 可以记为 0 或不纳入 TPOT 分位数。报告里必须写清楚定义。

E2E latency:

$$T_{\{\mathrm{e2e}\},i}=e_i-a_i$$

queue wait:

$$W_i=p_i-a_i$$

压测窗口从 t_0 到 t_1 ，窗口长度为:

$$D=t_1-t_0$$

输入吞吐、输出吞吐和总 token 吞吐:

$$\begin{aligned} Q_{\{\mathrm{in}\}} &= \frac{\sum_i x_i}{D}, \quad \text{quad} \\ Q_{\{\mathrm{out}\}} &= \frac{\sum_i y_i}{D}, \quad \text{quad} \\ Q_{\{\mathrm{tok}\}} &= \frac{\sum_i (x_i+y_i)}{D} \end{aligned}$$

KV Cache 显存粗估:

$$M_{\{\mathrm{kv}\}}=2L H_{\{\mathrm{kv}\}} D_h b \sum_i T_i$$

其中 L 是层数， $H_{\{\mathrm{kv}\}}$ 是 KV head 数， D_h 是 head dim， b 是每个元素字节数， T_i 是请求 i 当前占用的 cache token 数。

总显存可以粗略拆成:

$$M_{\{\mathrm{total}\}}=M_{\{\mathrm{weight}\}}+M_{\{\mathrm{kv}\}}+M_{\{\mathrm{work}\}}+M_{\{\mathrm{runtime}\}}$$

其中 $M_{\{\mathrm{work}\}}$ 是激活、临时 buffer、CUDA graph 或 kernel workspace 等工作区， $M_{\{\mathrm{runtime}\}}$ 是 runtime 和 allocator 额外开销。

压测门禁可以写成:

$$G_{\{\mathrm{bench}\}}=G_{\{\mathrm{trace}\}}G_{\{\mathrm{latency}\}}G_{\{\mathrm{throughput}\}}G_{\{\mathrm{kv}\}}G_{\{\mathrm{cleanup}\}}G_{\{\mathrm{runtime}\}}$$

这些指标要分开看，不能互相替代。一个系统可能 $Q_{\{\mathrm{out}\}}$ 很高，但 $T_{\{\mathrm{ttft}\}}$ p99 很差；也可能延迟达标，但 $M_{\{\mathrm{kv}\}}$ 已经接近 OOM。

15.3 在 RequestState 中记录时间

先给 request 增加时间字段。

```
class RequestState:
    def __init__(self, request_id, prompt, max_new_tokens=128):
        self.request_id = request_id
        self.prompt = prompt
        self.max_new_tokens = max_new_tokens

        self.arrival_time = now()
```

```

self.prefill_start_time = None
self.first_token_time = None
self.finished_time = None

self.decode_token_timestamps = []
self.input_token_count = 0
self.output_ids = []
self.finish_reason = None

```

然后定义：

```

def ttft(request):
    return request.first_token_time - request.arrival_time

```

```

def total_latency(request):
    return request.finished_time - request.arrival_time

```

```

def tpot(request):
    if len(request.decode_token_timestamps) <= 1:
        return 0
    return (request.decode_token_timestamps[-1] - request.decode_token_timestamps[0]) / (len(request.decode_token_timestamps) - 1)

```

注意，TPOT 的定义在不同系统里可能略有差异。有的从首 token 后开始算，有的按所有 output token 间隔算。压测报告里要写清楚定义。

15.4 在 engine 中埋点

请求入队时：

```
request.arrival_time = now()
```

prefill 开始时：

```
request.prefill_start_time = now()
```

首 token 生成时：

```

request.first_token_time = now()
request.decode_token_timestamps.append(request.first_token_time)

```

每个 decode token 生成时：

```
request.decode_token_timestamps.append(now())
```

请求结束时：

```
request.finished_time = now()
```

这些埋点要放在生命周期的正确位置，否则指标会偏。

例如，如果 first_token_time 记录在 token 生成后但 streaming flush 前，那么它是 engine 首 token 时间，不是用户实际收到首 token 的时间。生产系统最好区分 engine TTFT 和 client-observed TTFT。

15.5 统计分位数

平均值不够，要看 p50、p95、p99。

```

def percentile(values, p):
    if not values:
        return None
    values = sorted(values)

```

```
k = int((len(values) - 1) * p / 100)
return values[k]
```

统计:

```
ttfts = [ttft(r) for r in finished_requests]
tpots = [tpot(r) for r in finished_requests]
latencies = [total_latency(r) for r in finished_requests]
```

```
report = {
    "ttft_p50": percentile(ttfts, 50),
    "ttft_p95": percentile(ttfts, 95),
    "ttft_p99": percentile(ttfts, 99),
    "tpot_p50": percentile(tpots, 50),
    "tpot_p95": percentile(tpots, 95),
    "latency_p95": percentile(latencies, 95),
}
```

真实实现可以用更准确的分位数算法或指标系统。

15.6 统计 tokens/s

压测开始和结束时间:

```
duration = end_time - start_time
```

输入 token:

```
input_tokens = sum(r.input_token_count for r in finished_requests)
```

输出 token:

```
output_tokens = sum(len(r.output_ids) for r in finished_requests)
```

吞吐:

```
input_tokens_per_second = input_tokens / duration
output_tokens_per_second = output_tokens / duration
total_tokens_per_second = (input_tokens + output_tokens) / duration
```

LLM serving 报告里, output tokens/s 往往比 QPS 更有解释力。因为输出长度决定 decode 轮数, 也直接影响成本。

15.7 统计 queue 和 active requests

engine 每轮 step 可以记录:

```
metrics.queue_length.append(len(engine.waiting_queue))
metrics.running_requests.append(len(engine.running_requests))
metrics.decode_groups.append(len(engine.decode_groups))
```

这些指标帮助判断:

1. TTFT 高是不是因为 queue 堆积。
2. running requests 是否达到上限。
3. decode group 是否过多导致吞吐差。
4. scheduler 是否没有及时接入新请求。

如果 TTFT p99 高, 同时 queue length p99 也高, 问题可能在 admission、扩容或 scheduler。

15.8 显存观测

PyTorch 可以读取 CUDA 显存：

```
if torch.cuda.is_available():
    allocated = torch.cuda.memory_allocated() / 1024**3
    reserved = torch.cuda.memory_reserved() / 1024**3
```

记录每轮显存：

```
metrics.gpu_allocated_gb.append(allocated)
metrics.gpu_reserved_gb.append(reserved)
```

还可以在关键阶段记录：

1. 模型加载后。
2. prefill 后。
3. decode 中。
4. 请求结束后。
5. cleanup 后。

如果请求结束后 allocated 不下降，可能是 cache 没释放、引用没清理，或 PyTorch caching allocator 保留了 reserved memory。要区分 allocated 和 reserved。

15.9 KV Cache 显存的直觉压测

可以设计一个实验：固定模型和输出长度，逐步增加并发。

并发 1, 2, 4, 8, 16, 32

prompt 长度固定 1024

max_new_tokens 固定 128

观察显存增长。

预期现象：

1. 模型权重占用固定。
2. 并发越高，KV Cache 越多。
3. decode 越长，cache 序列长度越长。
4. 到某个并发后出现 OOM 或 admission 拒绝。

这个实验能帮助理解：为什么模型能加载，不代表能高并发 serving。

15.10 Workload 设计

压测 workload 不能只用一个短 prompt。

至少要定义：

1. 请求总数。
2. 并发数。
3. 到达模式。
4. 输入 token 长度分布。
5. 输出 token 长度分布。
6. 是否 streaming。
7. sampling 参数。
8. 是否有共享 prefix。
9. 超时时间。

示例 workload：

总请求数：1000

并发：32

输入长度: 50% 为 256 tokens, 30% 为 1024 tokens, 20% 为 4096 tokens

输出长度: 平均 256 tokens

streaming: 开启

temperature: 0.7

这样的报告比 “压 100 QPS” 更有意义。

15.11 到达模式

常见到达模式:

1. 固定并发: 始终保持 N 个请求在飞。
2. 固定 QPS: 每秒发固定数量请求。
3. 泊松到达: 模拟更自然的随机流量。
4. 突发流量: 短时间涌入大量请求。
5. 阶梯压测: 逐步提高并发或 QPS。

不同模式暴露不同问题。

固定并发适合测系统稳定吞吐。突发流量适合看 queue 和 TTFT 长尾。阶梯压测适合找拐点。

15.12 一个最小压测脚本思路

伪代码:

```
def run_benchmark(client, prompts, concurrency):
    results = []
    start_time = now()

    with ThreadPoolExecutor(max_workers=concurrency) as pool:
        futures = [pool.submit(client.generate, prompt) for prompt in prompts]
        for future in futures:
            results.append(future.result())

    end_time = now()
    return summarize(results, start_time, end_time)
```

每个 result 要包含:

1. request_id。
2. input_tokens。
3. output_tokens。
4. ttft。
5. tpot。
6. total_latency。
7. finish_reason。
8. error。

如果是 streaming client, 要在收到第一个 delta 时记录 TTFT, 在收到 finish event 时记录总延迟。

15.13 如何分析压测结果

情况一: TTFT 高, TPOT 正常。

优先排查:

1. waiting queue。
2. prefill latency。
3. prompt 长度。
4. scheduler 是否过度优先 decode。

5. prefix cache 是否失效。

情况二：TTFT 正常，TPOT 高。

优先排查：

1. decode batch。
2. KV Cache 读取。
3. decode group 过多。
4. GPU 显存带宽。
5. streaming backpressure。

情况三：tokens/s 高，但 p99 很差。

说明吞吐不错，但长尾体验差。要看请求长度分布、queue、长 prompt 和 scheduler 公平性。

情况四：显存持续上涨。

排查 request cleanup、cache 引用、finished group 是否释放、stream queue 是否持有 request。

15.14 最小框架的预期瓶颈

我们这个教学版 MiniEngine 预计会有很多瓶颈：

1. decode group 不能合并，吞吐低。
2. finished 请求可能仍占用 group cache。
3. 没有真正 KV Cache Manager。
4. 没有 PagedAttention，cache 可能浪费。
5. scheduler 只按请求数限流。
6. streamer 用完整 decode 差分，长输出效率低。
7. HTTP handler 和 engine 并发模型粗糙。
8. 没有 optimized kernel。

这些瓶颈不是失败，而是学习路线。后面讲 vLLM 和 SGLang 时，就是看真实系统如何解决这些问题。

15.15 报告模板

一次压测报告可以包含：

模型：xxx

硬件：1 x A100 80GB

并发：32

请求数：1000

输入长度分布：p50=512, p95=4096

输出长度分布：p50=128, p95=512

TTFT: p50=..., p95=..., p99=...

TPOT: p50=..., p95=..., p99=...

Total latency: p50=..., p95=..., p99=...

Input tokens/s: ...

Output tokens/s: ...

Queue length p95: ...

Active requests p95: ...

GPU allocated max: ... GB

Error rate: ...

Timeout rate: ...

报告要同时包含 workload、硬件、模型、指标和结论。没有 workload 描述的性能数字不可比较。

15.16 压测公式、指标表和可运行 demo

下面的 demo 不依赖外部库。它用 4 条 toy request trace 模拟一次小压测：

1. r1 和 r2 是短请求。
2. r3_long_prompt 是长 prompt，请求排队后 prefill 很慢。
3. r4_burst 是突发请求，queue wait 较高但 decode 正常。
4. 显存按 toy KV cache 公式估算，最后检查 cleanup 后只剩模型权重。

```
from dataclasses import dataclass
from math import ceil
```

```
@dataclass
```

```
class RequestTrace:
    request_id: str
    arrival_ms: int
    scheduled_ms: int
    first_token_ms: int
    output_token_times_ms: list[int]
    finish_ms: int
    input_tokens: int
    finish_reason: str = "length"
```

```
def percentile(values, p):
    if not values:
        return None
    ordered = sorted(values)
    index = max(0, ceil(len(ordered) * p / 100) - 1)
    return ordered[min(index, len(ordered) - 1)]
```

```
def request_metrics(trace):
    output_tokens = len(trace.output_token_times_ms)
    if output_tokens > 1:
        tpot = (trace.output_token_times_ms[-1] - trace.output_token_times_ms[0]) / (
            output_tokens - 1
        )
    else:
        tpot = 0.0
    return {
        "ttft_ms": trace.first_token_ms - trace.arrival_ms,
        "queue_wait_ms": trace.scheduled_ms - trace.arrival_ms,
        "prefill_ms": trace.first_token_ms - trace.scheduled_ms,
        "tpot_ms": round(tpot, 1),
        "e2e_ms": trace.finish_ms - trace.arrival_ms,
        "input_tokens": trace.input_tokens,
        "output_tokens": output_tokens,
        "finish_reason": trace.finish_reason,
    }
```

```
def active_tokens_at(trace, t):
    if not (trace.arrival_ms <= t < trace.finish_ms):
        return 0
    emitted = sum(1 for token_t in trace.output_token_times_ms if token_t <= t)
```



```
return trace.input_tokens + emitted
```

```
def run_benchmark_demo():
    traces = [
        RequestTrace("r1", 0, 0, 90, [90, 120, 150, 180], 190, 128),
        RequestTrace("r2", 0, 20, 130, [130, 165, 200, 235, 270, 305], 320, 256),
        RequestTrace("r3_long_prompt", 50, 200, 760, [760, 800, 840], 850, 4096),
        RequestTrace("r4_burst", 80, 220, 290, [290, 335, 380, 425, 470], 500, 512),
    ]
    queue_samples = [2, 3, 2, 2, 1, 1, 0]
    active_samples = [0, 2, 3, 1, 1, 0]

    per_request = {trace.request_id: request_metrics(trace) for trace in traces}
    ttfts = [item["ttft_ms"] for item in per_request.values()]
    tpots = [item["tpot_ms"] for item in per_request.values()]
    e2es = [item["e2e_ms"] for item in per_request.values()]
    start_ms = min(trace.arrival_ms for trace in traces)
    end_ms = max(trace.finish_ms for trace in traces)
    duration_s = (end_ms - start_ms) / 1000
    total_input = sum(trace.input_tokens for trace in traces)
    total_output = sum(len(trace.output_token_times_ms) for trace in traces)

    layers = 16
    kv_heads = 8
    head_dim = 128
    dtype_bytes = 2
    kv_mib_per_token = 2 * layers * kv_heads * head_dim * dtype_bytes / (1024 * 1024)
    model_mib = 14000.0
    memory_sample_times = [0, 100, 250, 500, 760, 860]
    kv_mib_samples = []
    for t in memory_sample_times:
        active_tokens = sum(active_tokens_at(trace, t) for trace in traces)
        kv_mib_samples.append(round(active_tokens * kv_mib_per_token, 2))
    kv_peak_mib = round(model_mib + max(kv_mib_samples), 2)
    kv_after_cleanup_mib = model_mib + kv_mib_samples[-1]

    slo = {
        "ttft": percentile(ttfts, 95) <= 500,
        "tpot": percentile(tpots, 95) <= 50,
        "kv": kv_peak_mib <= 14500,
    }
    if not slo["ttft"] and percentile(queue_samples, 95) >= 2:
        bottleneck = "queue_or_prefill"
    elif not slo["tpot"]:
        bottleneck = "decode_or_streaming"
    elif not slo["kv"]:
        bottleneck = "kv_capacity"
    else:
        bottleneck = "within_toy_slo"

    summary = {
        "ttft_ms": {
            "p50": percentile(ttfts, 50),
            "p95": percentile(ttfts, 95),
```

```

        "p99": percentile(ttfts, 99),
    },
    "tpot_ms": {
        "p50": percentile(tpots, 50),
        "p95": percentile(tpots, 95),
    },
    "e2e_ms": {"p95": percentile(e2es, 95)},
    "input_tokens_per_s": round(total_input / duration_s, 1),
    "output_tokens_per_s": round(total_output / duration_s, 1),
    "queue_p95": percentile(queue_samples, 95),
    "active_p95": percentile(active_samples, 95),
    "kv_peak_mib": kv_peak_mib,
    "kv_after_cleanup_mib": kv_after_cleanup_mib,
    "slo_pass": slo,
    "bottleneck": bottleneck,
}
gates = {
    "request_metrics_complete": all(
        key in next(iter(per_request.values()))
        for key in ["ttft_ms", "tpot_ms", "e2e_ms", "input_tokens", "output_tokens"]
    ),
    "tail_latency_detected": summary["ttft_ms"]["p95"] > summary["ttft_ms"]["p50"],
    "throughput_computed": summary["input_tokens_per_s"] > 0
    and summary["output_tokens_per_s"] > 0,
    "kv_peak_tracked": summary["kv_peak_mib"] > summary["kv_after_cleanup_mib"],
    "cleanup_checked": summary["kv_after_cleanup_mib"] == model_mib,
    "bottleneck_classified": summary["bottleneck"] == "queue_or_prefill",
}
gates["benchmark_gate"] = all(gates.values())
return per_request, summary, gates

```

```

per_request, summary, gates = run_benchmark_demo()
print("benchmark_per_request=", per_request)
print("benchmark_summary=", summary)
print("benchmark_gates=", gates)

```

一组稳定输出：

```

benchmark_per_request= {'r1': {'ttft_ms': 90, 'queue_wait_ms': 0, 'prefill_ms': 90, 'tpot_ms': 30.0, 'e2e_ms': 190, 'input_tokens_per_s': 130, 'output_tokens_per_s': 130},
benchmark_summary= {'ttft_ms': {'p50': 130, 'p95': 710, 'p99': 710}, 'tpot_ms': {'p50': 35.0, 'p95': 45.0}, 'e2e_ms': {'p50': 190, 'p95': 190, 'p99': 190}, 'input_tokens_per_s': 130, 'output_tokens_per_s': 130, 'queue_p95': 95, 'active_p95': 95, 'kv_peak_mib': 14000.0, 'kv_after_cleanup_mib': 14000.0, 'slo_pass': True, 'bottleneck': 'queue_or_prefill'},
benchmark_gates= {'request_metrics_complete': True, 'tail_latency_detected': True, 'throughput_computed': True, 'kv_peak_tracked': True, 'cleanup_checked': True, 'bottleneck_classified': True, 'benchmark_gate': True}

```

这个 demo 的关键证据：

1. 每个请求都能拆出 ttft_ms、queue_wait_ms、prefill_ms、tpot_ms、e2e_ms 和 token 数。
2. ttft_ms.p95=710 明显高于 p50，说明平均值会掩盖长 prompt 或 burst 导致的长尾。
3. tpot_ms.p95=45.0 仍然达标，说明问题不在 decode token 间隔。
4. input_tokens_per_s 和 output_tokens_per_s 同时给出，避免只看 QPS。
5. kv_peak_mib 高于 cleanup 后显存，说明并发期间 KV cache 增长可见；kv_after_cleanup_mib=14000.0 表明 toy 请求结束后 KV 已释放。
6. bottleneck='queue_or_prefill'，说明压测报告应给出归因，而不是只列数字。

15.17 常见误区

误区一：压测只看 QPS。

LLM 请求 token 长度差异极大，QPS 不足以描述负载。

误区二：只测短 prompt。

短 prompt 无法暴露长上下文 prefill、KV Cache 和显存问题。

误区三：只看平均延迟。

p95/p99 更能反映线上体验和长尾问题。

误区四：GPU 利用率高就说明服务好。

GPU 忙可能是有效吞吐，也可能是排队、padding 浪费或长请求堵塞。

误区五：显存 reserved 高就是泄漏。

PyTorch caching allocator 会保留 reserved memory，要结合 allocated 和请求生命周期判断。

15.18 面试追问

1. LLM serving 压测为什么不能只看 QPS?
2. TTFT 和 TPOT 分别应该在什么时候记录?
3. 如何统计 output tokens/s?
4. 如何设计一个合理 workload?
5. TTFT 高但 TPOT 正常说明什么?
6. TPOT 高但 TTFT 正常说明什么?
7. 显存随并发增加的主要原因是什么?
8. 如何判断是 engine 慢还是 streaming 输出慢?

参考回答思路：

1. 先说 LLM 请求按 token 消耗资源，所以要看 token 分布和 tokens/s。
2. 再说 TTFT 对应排队和 prefill，TPOT 对应 decode 和 streaming。
3. 然后说 workload 要包含输入输出长度、并发、到达模式和是否 streaming。
4. 最后讲排查路径：queue、prefill、decode、KV Cache、GPU、stream queue。

15.19 小练习

1. 给 MiniEngine 加上 TTFT、TPOT 和 total latency 统计。
2. 写一个固定并发压测脚本，发送 100 个请求。
3. 构造短 prompt 和长 prompt 两组 workload，对比 TTFT。
4. 固定 prompt 长度，逐步提高并发，观察显存变化。
5. 模拟客户端读 stream 很慢，观察 token 间隔和 stream queue。

15.20 本章小结

本章完成了第二部分的性能收尾。

我们给 MiniEngine 加入了指标视角：TTFT、TPOT、tokens/s、queue length、active requests 和显存。压测不是为了得到单一 QPS，而是为了把请求生命周期拆开，定位瓶颈在排队、prefill、decode、KV Cache、scheduler 还是 streaming。

从下一章开始进入第三部分：vLLM 架构详解。你会看到 vLLM 如何系统性解决我们这个 MiniEngine 暴露出来的问题，尤其是 PagedAttention、KV Cache Block Manager 和 continuous batching。

第 16 章 vLLM 解决什么问题

从本章开始进入第三部分：vLLM 架构详解。前面我们从 0 搭了一个 MiniEngine，也看到了它的瓶颈：KV Cache 管理粗糙、decode group 不能动态合并、finished 请求释放不彻底、padding 浪费、scheduler 简单、吞吐和显存都不理想。

vLLM 的价值，就是系统性解决这些 serving engine 的核心问题，尤其是 KV Cache 管理和 continuous batching。

一句话概括：

vLLM 解决的核心问题是：如何在高并发 LLM serving 中高效管理 KV Cache，并通过 iteration-level scheduling 提高吞吐、降低显存浪费。

16.0 本讲资料边界与第二轮精修口径

本讲只解释 vLLM 为什么出现、主要解决哪些 serving engine 问题。它覆盖 naive MiniEngine 的 KV 预留浪费、连续内存约束、finished 请求释放、固定 batch decode 浪费、continuous batching 动机、PagedAttention 直觉和最小可运行问题归因 demo，但不展开 vLLM 源码细节、真实 block manager API、调度器参数全集、多卡 worker / executor、prefix caching、speculative decoding、生产部署和 OpenAI-compatible server 配置。

资料校准口径：

1. vLLM 论文与项目文档把 PagedAttention 作为核心机制，用 block-based KV cache 管理降低内存浪费，并支持更高吞吐 serving。
2. vLLM optimization 文档强调调优时要同时关注并发请求数、batch tokens、KV cache 容量、preemption、chunked prefill 和 decode / prefill 平衡。
3. vLLM metrics 文档把 running / waiting 请求数、KV cache usage、TTFT、TPOT、queue / prefill / decode 时间纳入观测，说明 vLLM 不是单个 kernel，而是一套 request scheduling + memory management + execution engine。
4. Orca 的 iteration-level scheduling 思想是 continuous batching 的重要背景：动态请求流不应被固定 batch 生命周期锁死。
5. 本章 demo 用 token 数模拟连续 KV 预留、paged block 分配、finished 请求释放和固定 batch / continuous batching 行数，不等同于真实 vLLM 性能。

参考资料：

1. vLLM paper / PagedAttention: <https://arxiv.org/abs/2309.06180>
2. vLLM documentation: <https://docs.vllm.ai/>
3. vLLM optimization and tuning: <https://docs.vllm.ai/en/latest/configuration/optimization.html>
4. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics.html>
5. Orca iteration-level scheduling: <https://www.usenix.org/conference/osdi22/presentation/yu>

16.1 为什么要从 MiniEngine 过渡到 vLLM

第二部分的 MiniEngine 已经具备基本链路：

1. tokenizer。
2. model wrapper。
3. sampling。
4. KV Cache。
5. batched prefill。
6. batched decode。
7. request queue。
8. scheduler。
9. streaming。
10. HTTP API。

但它离生产级 serving engine 还有很远。

最典型的问题是：

1. KV Cache 直接依赖连续张量，难以动态管理。
2. finished 请求释放 cache 不优雅。
3. 新请求难以加入已有 decode batch。

4. 固定 batch 容易浪费计算。
5. 长短请求混合导致调度困难。
6. 显存碎片和预留浪费严重。
7. scheduler 缺少 token budget 和 cache budget。

vLLM 就是围绕这些问题设计出来的。

16.2 vLLM 的核心定位

vLLM 是一个面向大语言模型高吞吐推理的 serving engine。

它重点关注：

1. 高效的 KV Cache 管理。
2. PagedAttention。
3. Continuous Batching。
4. 高吞吐模型执行。
5. OpenAI-compatible serving。
6. 多模型和多并行配置支持。
7. Prefix caching 等推理优化。

如果用本书前面的三层边界来说，vLLM 主要属于 serving engine 层。

它不是完整 AI Infra，也不是完整企业推理平台。它可以提供 HTTP server 和 API 兼容能力，但它的核心价值仍然在模型执行、调度和内存管理。

16.3 vLLM 之前的痛点

传统推理实现容易遇到几个问题。

痛点一：KV Cache 显存浪费。

不同请求长度不同，如果每个请求预留一块连续 KV Cache，很容易出现内部浪费和碎片。

痛点二：batch 不够动态。

如果 batch 必须等一组请求全部结束，短请求会被长请求拖住，新请求也不能及时加入。

痛点三：显存限制并发。

模型权重固定，但 KV Cache 随并发和上下文增长。显存管理不好，最大并发会很低。

痛点四：吞吐和延迟难平衡。

大 batch 吞吐高但 TTFT 可能差，小 batch 延迟好但 GPU 利用率低。

痛点五：长上下文更难服务。

上下文越长，KV Cache 越大，调度和内存管理越关键。

vLLM 的架构创新主要围绕前两个痛点展开：KV Cache 和动态 batching。

可以先用几个简化公式看动机。设请求 i 的 prompt token 数为 x_i ，最大输出 token 预算为 m_i ，实际输出 token 数为 y_i 。naive 连续 KV 预留 token 数可以写成：

$$C_{\{\mathrm{reserve}\}} = \sum_i (x_i + m_i)$$

实际使用 token 数是：

$$C_{\{\mathrm{used}\}} = \sum_i (x_i + y_i)$$

连续预留浪费率：

$$R_{\{\mathrm{naive}\}} = \frac{C_{\{\mathrm{reserve}\}} - C_{\{\mathrm{used}\}}}{C_{\{\mathrm{reserve}\}}}$$

如果按 block size S 分块，paged KV 需要的 block 数为：

$$B_i = \left\lceil \frac{x_i + y_i}{S} \right\rceil$$

paged KV 分配 token 数:

$$C_{\mathrm{paged}} = S \sum_i B_i$$

paged KV 的块内浪费率:

$$R_{\mathrm{paged}} = \frac{C_{\mathrm{paged}} - C_{\mathrm{used}}}{C_{\mathrm{paged}}}$$

固定 batch decode 的行数近似为:

$$W_{\mathrm{static}} = N \max_i y_i$$

continuous batching 只处理仍然 active 的请求, decode 行数近似为:

$$W_{\mathrm{cont}} = \sum_{t=1}^T A_t$$

其中 A_t 是第 t 轮仍在 decode 的请求数。vLLM 的动机门禁可以写成:

$$G_{\mathrm{vllm}} = G_{\mathrm{kvbloc}} G_{\mathrm{sched}} G_{\mathrm{reuse}} G_{\mathrm{cleanup}} G_{\mathrm{metrics}}$$

这些公式不等价于真实 vLLM 内部实现, 但能解释它为什么要同时解决 KV block 管理、动态调度、block 复用、请求结束清理和指标观测。

16.4 PagedAttention 解决什么

PagedAttention 是 vLLM 最著名的机制。

它的直觉来自操作系统虚拟内存分页: 不要要求一个请求的 KV Cache 在物理显存中连续存放, 而是把 KV Cache 切成固定大小的 block, 通过映射表把逻辑 token 位置映射到物理 block。

简化管理:

request logical tokens: 0, 1, 2, 3, 4, 5, ...
 logical blocks: block 0, block 1, ...
 physical blocks: GPU block 17, GPU block 5, ...

好处是:

1. 减少连续分配要求。
2. 降低显存碎片。
3. 方便动态增长和释放。
4. 方便多个请求共享 prefix block。
5. 提升高并发场景下的显存利用率。

PagedAttention 不只是一个 attention kernel, 而是一套 KV Cache 内存管理思想。

16.5 Continuous Batching 解决什么

Continuous batching, 也叫 iteration-level scheduling, 解决动态请求流问题。

传统 batch:

batch A 开始 -> batch A 全部结束 -> batch B 开始

Continuous batching:

iteration 1: r1, r2
 iteration 2: r1, r2, r3
 iteration 3: r2, r3, r4
 iteration 4: r3, r4

每一轮 decode 都可以:

1. 移除完成请求。
2. 加入新请求。

- 3. 按 token budget 控制 prefill。
- 4. 按 cache budget 控制并发。
- 5. 让 GPU 尽量保持有效工作。

这比第二部分的固定 decode group 更灵活。

16.6 vLLM 和 MiniEngine 的对比

可以这样对比：

能力	MiniEngine	vLLM
KV Cache	简单 <code>past_key_values</code>	block-based cache manager
内存布局	连续 batch cache	PagedAttention
batch	固定 group	continuous batching
调度	简单 FIFO / 请求数限制	token/cache budget + iteration-level scheduling
请求退出	简单 <code>finished</code> 标记	释放 cache block, 动态移出
新请求加入	新建 group	可进入后续 iteration
显存效率	较低	更高
生产能力	教学	生产级 serving engine

这个对比能帮助你理解 vLLM 不是 “更复杂的 generate”，而是把 MiniEngine 的痛点系统工程化解决。

16.7 vLLM 不只是 PagedAttention

很多人提到 vLLM 只记得 PagedAttention。

但真实 vLLM 还包括：

- 1. 请求生命周期管理。
- 2. Scheduler。
- 3. Block Manager。
- 4. Worker 和 Executor。
- 5. Model Runner。
- 6. Sampling。
- 7. Prefix Caching。
- 8. 多卡并行支持。
- 9. API Server。
- 10. Metrics 和配置系统。

PagedAttention 是核心创新，但不是全部系统。

面试里只说 “vLLM 用 PagedAttention 提升吞吐” 是不够的。要能讲请求如何进入、如何分配 block、如何调度 prefill/decode、如何释放 cache。

16.8 vLLM 的请求视角

从请求生命周期看，vLLM 大致做这些事：

外部请求

- > API 层解析
- > 创建内部 `sequence / sequence group`
- > 进入 `waiting queue`
- > scheduler 选择 `prefill` 或 `decode`
- > block manager 分配 `KV blocks`
- > worker 执行模型 `forward`
- > sampler 生成 `token`

- > 更新 sequence 状态
- > stream / return output
- > finished 后释放 blocks

这和我们前面写的 MiniEngine 生命周期相同，只是每个环节更精细、更高效。

学习 vLLM 时，建议始终用这条链路做地图。

16.9 为什么 KV Cache 是 vLLM 的核心

LLM serving 的显存大头通常包括模型权重和 KV Cache。

模型权重加载后相对固定，KV Cache 随请求动态变化。

高并发时，真正决定能容纳多少请求的，经常是 KV Cache，而不是权重。

如果 KV Cache 管理粗糙：

1. 显存碎片会降低可用容量。
2. 长请求会占用大量连续空间。
3. 短请求结束后释放的空间难复用。
4. 并发上不去。
5. OOM 更容易发生。

vLLM 把 KV Cache 当成一级资源来管理，这是它和 naive serving 的根本区别。

16.10 为什么调度是 vLLM 的核心

只有 KV Cache 管理还不够。请求还需要被合理调度。

Scheduler 要决定：

1. 本轮处理哪些 waiting 请求。
2. 本轮继续 decode 哪些 running 请求。
3. prefill 和 decode 如何混合。
4. token budget 如何分配。
5. cache block 是否足够。
6. 请求完成后如何释放资源。
7. 长请求是否会拖慢短请求。

调度直接影响：

1. TTFT。
2. TPOT。
3. tokens/s。
4. GPU 利用率。
5. 公平性。
6. 显存稳定性。

因此 vLLM 的 scheduler 和 block manager 要一起理解，不能割裂看。

16.11 vLLM 适合什么场景

vLLM 适合：

1. 高并发在线文本生成。
2. OpenAI-compatible LLM serving。
3. 多用户共享模型服务。
4. 长短请求混合负载。
5. 需要 streaming 的聊天服务。
6. 需要较高吞吐和较好显存利用率的场景。

它不一定是所有场景的唯一选择。

例如，极致低延迟单请求、特定硬件上的 TensorRT-LLM 优化、复杂结构化生成和 agent runtime，也可能选择其他框架或组合方案。

工程上不要迷信某个框架，要理解它解决的问题和适用边界。

16.12 vLLM 与其他 runtime 的关系

常见 serving runtime 有：

1. vLLM。
2. SGLang。
3. TGI。
4. TensorRT-LLM。
5. Triton + 自研 backend。
6. llama.cpp 等本地推理 runtime。

vLLM 的优势在于通用 LLM serving、PagedAttention、continuous batching、生态和易用性。

SGLang 更强调 runtime、prefix sharing、structured generation、agent serving 等能力。

TensorRT-LLM 更偏 NVIDIA 生态内的高性能优化。

TGI 是 Hugging Face 生态常见 serving 方案。

本书先讲 vLLM，是因为它非常适合作为理解现代 serving engine 的主线样本。

16.13 如何阅读 vLLM 源码

阅读 vLLM 源码不要从所有文件开始。

建议按问题读：

1. 请求如何进入系统。
2. 内部如何表示 sequence。
3. scheduler 如何选择请求。
4. block manager 如何分配 KV Cache。
5. worker 如何执行模型。
6. sampler 如何返回 token。
7. 请求如何完成并释放 block。

读源码时要把类名放回生命周期，而不是孤立记忆。

例如看到 block table，就问：它是不是把逻辑 token block 映射到物理 KV block？看到 scheduler，就问：它这轮选择了哪些 prefill 和 decode？

16.14 vLLM 动机公式、内存浪费和可运行 demo

下面的 demo 不依赖外部库。它不实现真实 vLLM，而是用 toy request profiles 对比两件事：

1. naive serving 为每个请求连续预留 `prompt + max_new_tokens`，如果实际输出更短，会浪费 KV token。
2. paged KV cache 只按实际增长分配固定大小 block，finished / cancelled 请求释放 block 后可复用。
3. 固定 batch decode 要让短请求陪长请求跑到最长输出长度，continuous batching 只处理仍 active 的请求。

```
from dataclasses import dataclass
from math import ceil
```

```
@dataclass
class RequestProfile:
```

```

request_id: str
prompt_tokens: int
max_new_tokens: int
actual_new_tokens: int
finished: bool = True

def block_count(tokens, block_size):
    return ceil(tokens / block_size)

def run_vllm_motivation_demo():
    block_size = 16
    requests = [
        RequestProfile("r_short", 127, 128, 16),
        RequestProfile("r_chat", 513, 256, 92),
        RequestProfile("r_long", 2049, 512, 401),
        RequestProfile("r_cancel", 1023, 512, 34),
    ]

    rows = {}
    next_block = 0
    free_after_cancel = []
    for req in requests:
        reserved_tokens = req.prompt_tokens + req.max_new_tokens
        used_tokens = req.prompt_tokens + req.actual_new_tokens
        blocks = block_count(used_tokens, block_size)
        physical_blocks = list(range(next_block, next_block + blocks))
        next_block += blocks
        allocated_tokens = blocks * block_size
        rows[req.request_id] = {
            "reserved_tokens": reserved_tokens,
            "used_tokens": used_tokens,
            "paged_blocks": blocks,
            "paged_allocated_tokens": allocated_tokens,
            "logical_to_physical_head": physical_blocks[:3],
        }
        if req.request_id == "r_cancel":
            free_after_cancel = physical_blocks

    new_request_blocks = free_after_cancel[: block_count(64, block_size)]
    naive_reserved = sum(item["reserved_tokens"] for item in rows.values())
    actual_used = sum(item["used_tokens"] for item in rows.values())
    paged_allocated = sum(item["paged_allocated_tokens"] for item in rows.values())
    naive_waste_ratio = round((naive_reserved - actual_used) / naive_reserved, 3)
    paged_waste_ratio = round((paged_allocated - actual_used) / paged_allocated, 3)

    max_decode_steps = max(req.actual_new_tokens for req in requests)
    static_decode_rows = len(requests) * max_decode_steps
    continuous_decode_rows = sum(req.actual_new_tokens for req in requests)
    saved_rows = static_decode_rows - continuous_decode_rows
    row_saving_ratio = round(saved_rows / static_decode_rows, 3)

    memory_summary = {
        "naive_reserved_tokens": naive_reserved,

```

```

        "actual_used_tokens": actual_used,
        "paged_allocated_tokens": paged_allocated,
        "naive_waste_ratio": naive_waste_ratio,
        "paged_waste_ratio": paged_waste_ratio,
        "freed_blocks_after_cancel": len(free_after_cancel),
        "new_request_reused_blocks": new_request_blocks,
    }
    batch_summary = {
        "static_decode_rows": static_decode_rows,
        "continuous_decode_rows": continuous_decode_rows,
        "saved_rows": saved_rows,
        "row_saving_ratio": row_saving_ratio,
    }
    gates = {
        "kv_waste_reduced": paged_waste_ratio < naive_waste_ratio,
        "block_reuse_visible": new_request_blocks == free_after_cancel[:4],
        "continuous_saves_rows": saved_rows > 0,
        "finished_cleanup_visible": len(free_after_cancel) == rows["r_cancel"]["paged_blocks"],
        "motivation_classified": naive_waste_ratio > 0.1 and row_saving_ratio > 0.5,
    }
    gates["vllm_motivation_gate"] = all(gates.values())
    return rows, memory_summary, batch_summary, gates

```

```

rows, memory_summary, batch_summary, gates = run_vllm_motivation_demo()
print("request_memory_rows=", rows)
print("memory_summary=", memory_summary)
print("batch_summary=", batch_summary)
print("vllm_motivation_gates=", gates)

```

一组稳定输出：

```

request_memory_rows= {'r_short': {'reserved_tokens': 255, 'used_tokens': 143, 'paged_blocks': 9, 'paged_allocated_tokens': 4288, 'naive_reserved_tokens': 5120, 'actual_used_tokens': 4255, 'paged_allocated_tokens': 4288, 'naive_waste_ratio': 0.008, 'paged_waste_ratio': 0.169, 'freed_blocks_after_cancel': 67, 'new_request_reused_blocks': 0}}
memory_summary= {'naive_reserved_tokens': 5120, 'actual_used_tokens': 4255, 'paged_allocated_tokens': 4288, 'naive_waste_ratio': 0.008, 'paged_waste_ratio': 0.169, 'freed_blocks_after_cancel': 67, 'new_request_reused_blocks': 0}
batch_summary= {'static_decode_rows': 1604, 'continuous_decode_rows': 543, 'saved_rows': 1061, 'row_saving_ratio': 0.66}
vllm_motivation_gates= {'kv_waste_reduced': True, 'block_reuse_visible': True, 'continuous_saves_rows': True, 'finished_cleanup_visible': True, 'motivation_classified': True, 'vllm_motivation_gate': True}

```

这个 demo 的关键证据：

1. naive 连续预留需要 5120 个 token 容量，实际只用了 4255，浪费率 0.169。
2. paged block 分配只需要 4288 个 token 容量，块内浪费率 0.008。
3. r_cancel 结束后释放了 67 个 block，新请求可以复用 [201,202,203,204]。
4. 固定 batch decode 需要 1604 行，continuous batching 近似只需要 543 行。
5. 这同时解释了为什么 vLLM 不能只理解成 PagedAttention kernel；它的动机是 KV block、调度、复用、cleanup 和 metrics 的系统组合。

16.15 常见误区

误区一：vLLM 只是比 `model.generate()` 快。

vLLM 的核心是 serving engine：KV Cache 管理、调度、batching、worker/executor 和 streaming，不是简单加速单次 generate。

误区二：PagedAttention 就是全部 vLLM。

PagedAttention 很重要，但还要理解 scheduler、block manager、worker、sampler 和请求生命周期。

误区三：continuous batching 就是普通 batch。

普通 batch 是固定批次，continuous batching 是每轮动态加入和移除请求。

误区四：显存够放模型就够了。

高并发 serving 还要考虑 KV Cache，vLLM 的很多设计正是为了解决 cache 显存问题。

误区五：用了 vLLM 就不需要平台层。

vLLM 是 engine，生产还需要推理平台做路由、限流、灰度、观测、成本和多租户治理。

16.16 面试追问

1. vLLM 主要解决什么问题？
2. 为什么 KV Cache 管理是 LLM serving 的核心？
3. PagedAttention 的直觉是什么？
4. Continuous batching 和静态 batching 有什么区别？
5. vLLM 和一个 naive MiniEngine 的差距在哪里？
6. 为什么 scheduler 和 block manager 要一起理解？
7. vLLM 是否等于完整推理平台？为什么？
8. 什么场景下你会考虑 vLLM，什么场景下可能考虑其他 runtime？

参考回答思路：

1. 先说 vLLM 是高吞吐 LLM serving engine。
2. 再说两个核心：PagedAttention 管 KV Cache，continuous batching 管动态请求调度。
3. 然后解释它解决显存碎片、并发、吞吐和动态 batch 问题。
4. 最后补充边界：vLLM 是 engine，不等于完整平台，仍需平台和 infra 配合。

16.17 小练习

1. 用一张图画出 MiniEngine 和 vLLM 在 KV Cache 管理上的差异。
2. 解释为什么固定 decode group 会限制吞吐。
3. 写出 continuous batching 的 4 轮示例，包含新请求加入和旧请求退出。
4. 列出 vLLM 中你最想读的 5 个模块，并说明每个模块解决什么问题。
5. 思考如果没有 PagedAttention，高并发长上下文 serving 会遇到哪些问题。

16.18 本章小结

本章开启 vLLM 架构详解。

vLLM 解决的不是“如何调用模型生成文本”这么简单的问题，而是高并发 LLM serving 的系统问题：KV Cache 如何高效管理，请求如何动态调度，finished 请求如何释放资源，新请求如何加入运行中的 batch，显存如何更高效利用。

下一章我们会深入 PagedAttention，讲清它如何借鉴分页内存思想，把 KV Cache 从连续大张量管理升级成 block-based 管理。

第 17 章 PagedAttention 的核心思想

上一章讲了 vLLM 主要解决什么问题。本章进入 vLLM 最核心的机制：PagedAttention。

很多同学第一次听 PagedAttention，会以为它只是一个更快的 attention kernel。这个理解不完整。PagedAttention 的真正核心是：把 KV Cache 管理从“连续大块内存”变成“分页式块管理”，从而减少显存浪费，支持更高并发和更灵活的调度。

一句话概括：

PagedAttention 借鉴操作系统分页思想，把请求的逻辑 KV Cache 切成固定大小的块，并映射到 GPU 上的物理 KV block，从而避免连续分配和显存碎片问题。

17.0 本讲资料边界与第二轮精修口径

本讲只解释 PagedAttention 的核心思想。它覆盖 logical block、physical block、block table、token position 到 KV block 的地址翻译、block size trade-off、block 内部浪费、请求释放后的 block 复用、prefix sharing / copy-on-write 直觉和最小可运行分页 KV demo，但不展开 vLLM 真实 CUDA kernel、真实 block manager 源码、attention kernel 性能调优、多卡 KV 分布、prefix cache 策略、swap / preemption 或生产参数配置。

资料校准口径：

1. vLLM / PagedAttention 论文把 KV cache 分成固定大小 blocks，并用 PagedAttention 让 attention kernel 能读取非连续物理 KV blocks，从而降低内存浪费并支持高吞吐 serving。
2. vLLM 文档和源码术语通常围绕 block table、KV cache block、scheduler、block manager、prefix caching 和 KV cache usage 展开；本章只抽象这些机制的稳定概念，不绑定具体类名或参数名。
3. 操作系统分页类比只用于帮助理解逻辑地址到物理地址的映射，不表示 GPU KV cache 完全等价于 CPU 虚拟内存。
4. 本章 demo 用纯 Python list 模拟 KV block storage、block table、ref count 和 free list，不实现真实 attention score 计算。

参考资料：

1. vLLM paper / PagedAttention: <https://arxiv.org/abs/2309.06180>
2. vLLM documentation: <https://docs.vllm.ai/>
3. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics.html>

17.1 问题从 KV Cache 开始

自回归生成需要 KV Cache。

每个请求的 cache 会随着上下文长度增长：

prompt tokens + generated tokens -> KV Cache length

如果一个请求上下文很长，它需要大量 cache。如果很多请求同时运行，总 cache 占用会非常大。

传统做法可能为每个请求分配一段连续 cache 空间。

问题是请求长度不可预测：

1. 有的请求很短，很快结束。
2. 有的请求 prompt 很长。
3. 有的请求生成很多 token。
4. 有的请求中途取消。
5. batch 中请求不断加入和退出。

连续分配在这种动态环境里很容易浪费显存。

17.2 连续 KV Cache 的浪费

假设每个请求都预留最大长度 4096 token 的 KV Cache。

如果一个请求实际只用了 300 token，其余 3796 token 的 cache 空间就浪费了。

```
reserved: 4096 tokens
used:     300 tokens
wasted:   3796 tokens
```

如果不预留最大长度，而是动态扩展连续空间，又会遇到碎片问题。

请求 A 释放一小段，请求 B 需要一大段，虽然总空闲显存够，但没有足够连续空间。

这和操作系统里内存分配遇到的问题很像。

17.3 操作系统分页的直觉

操作系统不会要求一个进程的虚拟内存全部连续映射到物理内存。

它会把内存切成 page:

virtual page 0 -> physical page 7

virtual page 1 -> physical page 2

virtual page 2 -> physical page 9

进程看到的是连续虚拟地址，但物理内存可以不连续。

PagedAttention 借鉴了这个思想。

对一个请求来说，逻辑 token 序列是连续的：

token 0, token 1, token 2, ..., token N

但它们对应的 KV Cache 物理 block 不需要连续。

17.4 逻辑块和物理块

PagedAttention 把一个请求的 KV Cache 按 token 维度切成固定大小的逻辑块。

假设 block size 是 16 token。

请求的 token 被分成：

logical block 0: token 0-15

logical block 1: token 16-31

logical block 2: token 32-47

这些逻辑块映射到 GPU 上的物理 KV block:

logical block 0 -> physical block 42

logical block 1 -> physical block 8

logical block 2 -> physical block 19

物理 block 可以不连续。

请求只需要维护一张 block table，记录逻辑块到物理块的映射。

17.5 Block Table

Block table 是理解 PagedAttention 的关键。

对一个请求：

block_table = [42, 8, 19]

含义是：

第 0 个逻辑 block 在物理 block 42

第 1 个逻辑 block 在物理 block 8

第 2 个逻辑 block 在物理 block 19

当 attention 需要读取历史 token 的 KV 时，它先根据 token 位置找到逻辑 block，再通过 block table 找到物理 block。

这一步类似地址翻译：

token position -> logical block id -> physical block id -> KV memory address

用公式写，设 block size 为 S，请求 i 的 block table 为 P_i，token 位置为 t。

逻辑 block id:

$b_t = \left\lfloor \frac{t}{S} \right\rfloor$

block 内 offset:

$o_t = t - S b_t$

物理 block id:

$p_t = P_i[b_t]$

如果只看 token 维度, KV 读取位置可以抽象为:

$\mathrm{addr}(i, t) = (p_t, o_t)$

请求 i 当前长度为 T_i 时, 需要的 block 数为:

$B_i = \left\lceil \frac{T_i}{S} \right\rceil$

最后一个 block 的块内浪费:

$R_i = S B_i - T_i$

如果多个请求共享物理 block p, 可以用引用计数 c_p 表示仍有多少请求引用它:

$c_p = \sum_i \mathbf{1}[p \in P_i]$

只有当 $c_p=0$ 时, 物理 block 才能回到 free list。这个引用计数直觉是 prefix sharing 和 copy-on-write 的基础。

这就是 PagedAttention 的核心映射。

17.6 一个具体例子

假设 block size = 4。

请求 A 有 10 个 token:

tokens: 0 1 2 3 | 4 5 6 7 | 8 9

它需要 3 个 logical blocks:

logical block 0: token 0-3

logical block 1: token 4-7

logical block 2: token 8-11, 其中 10-11 还没用

物理映射:

$\text{block_table_A} = [5, 11, 2]$

请求 B 有 6 个 token:

$\text{block_table_B} = [7, 3]$

请求 A 和 B 的物理块交错分布, 但对每个请求来说, 逻辑 token 序列仍然是连续的。

17.7 PagedAttention 如何减少浪费

PagedAttention 减少的是两类浪费。

第一, 预留浪费。

不需要为每个请求一开始预留最大长度 cache。请求增长到新 block 时, 再分配一个物理 block。

第二, 碎片浪费。

物理 block 固定大小, 可以从全局 free list 里复用。请求结束后, 释放它占用的 blocks, 其他请求可以继续使用。

相比连续分配, block 级分配更容易复用零散空间。

当然，PagedAttention 仍然有 block 内部浪费。

例如 block size 16，一个请求最后只用了 3 个 token 的 block，剩下 13 个 token 位置暂时浪费。但这通常比整段连续预留的浪费小得多。

17.8 Block Size 的 trade-off

block size 不是越小越好，也不是越大越好。

block size 小：

1. 内部浪费少。
2. block table 更长。
3. 地址映射和管理开销更大。
4. kernel 访问可能更碎。

block size 大：

1. block table 更短。
2. 管理开销小。
3. block 内部浪费更多。
4. 对短请求不友好。

所以 block size 是性能和显存利用率之间的 trade-off。

面试里如果只说“分块减少浪费”，还不够。要补一句：块大小会影响内部碎片、映射开销和 kernel 效率。

17.9 Attention 如何读取分页 KV

普通 attention 可能假设 KV Cache 是连续张量。

PagedAttention 下，历史 KV 分散在不同 physical blocks 中。

attention kernel 需要根据 block table 找到每个 token 对应的物理位置。

简化流程：

```
for each query token:
    for each historical key/value token:
        logical_block = token_pos // block_size
        offset = token_pos % block_size
        physical_block = block_table[logical_block]
        read key/value from physical_block + offset
```

真实实现会用高效 CUDA kernel，而不是 Python 循环。

核心点是：PagedAttention 把地址映射逻辑融合进 attention 计算。

17.10 与 past_key_values 的区别

第 9 章的最小 KV Cache 使用 past_key_values。

它更像：

每个 batch 持有一个连续 cache 张量

PagedAttention 更像：

全局物理 KV block 池 + 每个请求的 block table

区别在于：

1. past_key_values 适合教学和简单 batch。
2. PagedAttention 适合动态请求、高并发和显存复用。

3. `past_key_values` 的 batch 顺序变化很麻烦。
4. PagedAttention 通过 block table 解耦请求逻辑顺序和物理存储。

这就是 vLLM 能支持 continuous batching 的基础之一。

17.11 与 continuous batching 的关系

PagedAttention 和 continuous batching 是互相支撑的。

continuous batching 要求：

1. 请求可以动态加入。
2. 请求可以动态退出。
3. cache 可以随请求增长。
4. finished 请求可以释放 cache。
5. batch row 和请求映射可以变化。

如果 KV Cache 必须是连续 batch 张量，这些操作会非常困难。

PagedAttention 通过 block table 和 block manager，让请求的 cache 独立管理，batch 只是每一轮执行时的临时组织方式。

这就是为什么 PagedAttention 不是孤立优化，而是 vLLM 调度系统的基础设施。

17.12 Prefix Sharing 的基础

PagedAttention 还为 prefix sharing 提供基础。

如果多个请求共享相同前缀，它们可以引用相同的物理 blocks。

例如：

```
request A block_table: [1, 2, 3, 8]
```

```
request B block_table: [1, 2, 3, 9]
```

前 3 个 blocks 共享，后面分叉。

这类似 copy-on-write：共享部分不需要重复存储，只有分叉后的部分单独分配。

这对多轮对话、相同 system prompt、RAG 模板、批量评测都很有价值。

prefix caching 后面会单独讲，本章只需要知道：block-based cache 管理让共享更自然。

17.13 PagedAttention 不是免费午餐

PagedAttention 有明显收益，但也有代价。

代价包括：

1. attention kernel 更复杂。
2. 需要维护 block table。
3. 需要 block manager 分配和释放。
4. block size 选择影响性能。
5. 调试难度高于连续 cache。
6. 对硬件访问模式和 kernel 实现有要求。

工程上没有纯收益设计。vLLM 的价值在于这些复杂度换来了高并发场景下更好的显存利用和调度能力。

17.14 一个面试级类比

可以这样向面试官解释：

传统 KV Cache 像给每个请求分配一整段连续内存。请求长度不确定时，要么预留太多浪费，要么动态扩容导致碎片。

PagedAttention 像操作系统分页。请求看到的是连续逻辑 token 序列，但实际 KV Cache 存在固定大小的物理 blocks 中。通过 block table 做逻辑到物理的映射。这样请求可以动态增长，结束后释放 blocks，空闲 blocks 可以被其他请求复用。

这个类比清楚表达了 PagedAttention 的本质。

17.15 PagedAttention 地址翻译和可运行 demo

下面的 demo 不依赖外部库。它用 list 模拟一个 toy KV block pool:

1. 请求 A 写入 10 个 token，形成 3 个 logical blocks。
2. 查询 token 位置 8，通过 block table 翻译到 physical block 和 offset。
3. 请求 B 共享请求 A 的前 2 个 prefix blocks，再写入自己的分叉 token。
4. 释放请求 A 时，共享 blocks 的 ref count 保留，非共享 block 回到 free list。
5. 请求 C 复用请求 A 释放的 physical block。

```
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class RequestKV:
    request_id: str
    block_table: List[int] = field(default_factory=list)
    length: int = 0
```

```
class ToyPagedKVCache:
```

```
    def __init__(self, block_size, physical_blocks):
        self.block_size = block_size
        self.free_blocks = list(physical_blocks)
        self.storage = {block_id: [None] * block_size for block_id in physical_blocks}
        self.ref_count = {block_id: 0 for block_id in physical_blocks}
        self.trace = []
```

```
    def allocate_block(self):
        block_id = self.free_blocks.pop(0)
        self.storage[block_id] = [None] * self.block_size
        self.ref_count[block_id] = 1
        self.trace.append(("alloc", block_id))
        return block_id
```

```
    def retain_block(self, block_id):
        self.ref_count[block_id] += 1
        self.trace.append(("retain", block_id, self.ref_count[block_id]))
```

```
    def append_token(self, request, token):
        logical_block = request.length // self.block_size
        offset = request.length % self.block_size
        if offset == 0:
            request.block_table.append(self.allocate_block())
        physical_block = request.block_table[logical_block]
        self.storage[physical_block][offset] = token
        request.length += 1
        self.trace.append(
```

```

        ("write", request.request_id, token, logical_block, offset, physical_block)
    )

def append_tokens(self, request, tokens):
    for token in tokens:
        self.append_token(request, token)

def translate(self, request, token_position):
    logical_block = token_position // self.block_size
    offset = token_position % self.block_size
    physical_block = request.block_table[logical_block]
    value = self.storage[physical_block][offset]
    return {
        "token_position": token_position,
        "logical_block": logical_block,
        "offset": offset,
        "physical_block": physical_block,
        "value": value,
    }

def fork_shared_prefix(self, source, new_request_id, prefix_blocks):
    child = RequestKV(new_request_id)
    for physical_block in source.block_table[:prefix_blocks]:
        self.retain_block(physical_block)
        child.block_table.append(physical_block)
    child.length = prefix_blocks * self.block_size
    return child

def release_request(self, request):
    released = []
    for physical_block in request.block_table:
        self.ref_count[physical_block] -= 1
        if self.ref_count[physical_block] == 0:
            self.storage[physical_block] = [None] * self.block_size
            self.free_blocks.insert(0, physical_block)
            released.append(physical_block)
    self.trace.append(("release", request.request_id, released))
    request.block_table = []
    request.length = 0
    return released

def internal_waste(self, requests):
    used_by_block = {}
    for request in requests:
        for block_index, physical_block in enumerate(request.block_table):
            start = block_index * self.block_size
            used = max(0, min(self.block_size, request.length - start))
            used_by_block[physical_block] = max(used_by_block.get(physical_block, 0), used)
    allocated_slots = len(used_by_block) * self.block_size
    used_slots = sum(used_by_block.values())
    return {
        "allocated_slots": allocated_slots,
        "used_slots": used_slots,
        "wasted_slots": allocated_slots - used_slots,
    }

```

```

def run_paged_attention_demo():
    cache = ToyPagedKVCache(block_size=4, physical_blocks=[7, 3, 11, 5, 9, 2, 13])
    req_a = RequestKV("A")
    cache.append_tokens(req_a, list("abcdefghij"))
    translations = [cache.translate(req_a, pos) for pos in [0, 4, 8, 9]]

    req_b = cache.fork_shared_prefix(req_a, "B", prefix_blocks=2)
    cache.append_tokens(req_b, list("XY"))
    shared_ref_counts = {block: cache.ref_count[block] for block in req_a.block_table[:2]}

    released_a = cache.release_request(req_a)
    ref_counts_after_a = {block: cache.ref_count[block] for block in [7, 3, 11]}
    req_c = RequestKV("C")
    cache.append_tokens(req_c, list("mnop"))

    waste = cache.internal_waste([req_b, req_c])
    summary = {
        "req_b_table": req_b.block_table,
        "req_c_table": req_c.block_table,
        "translations": translations,
        "shared_ref_counts": shared_ref_counts,
        "released_a": released_a,
        "ref_counts_after_a": ref_counts_after_a,
        "free_head": cache.free_blocks[:4],
        "waste": waste,
    }
    gates = {
        "address_translation": translations[2] == {
            "token_position": 8,
            "logical_block": 2,
            "offset": 0,
            "physical_block": 11,
            "value": "i",
        },
        "prefix_blocks_shared": req_b.block_table[:2] == [7, 3]
        and shared_ref_counts == {7: 2, 3: 2},
        "partial_release_keeps_shared": ref_counts_after_a == {7: 1, 3: 1, 11: 0},
        "freed_block_reused": req_c.block_table == [11],
        "internal_waste_visible": waste == {
            "allocated_slots": 16,
            "used_slots": 14,
            "wasted_slots": 2,
        },
    }
    gates["paged_attention_gate"] = all(gates.values())
    return summary, cache.trace[-10:], gates

summary, trace_tail, gates = run_paged_attention_demo()
print("paged_kv_summary=", summary)
print("trace_tail=", trace_tail)
print("paged_attention_gates=", gates)

```

一组稳定输出：

```
paged_kv_summary= {'req_b_table': [7, 3, 5], 'req_c_table': [11], 'translations': [{'token_position': 0, 'logical_block': 7, 'physical_block': 7}, {'token_position': 1, 'logical_block': 3, 'physical_block': 3}, {'token_position': 2, 'logical_block': 5, 'physical_block': 5}, {'token_position': 3, 'logical_block': 11, 'physical_block': 11}, {'token_position': 4, 'logical_block': 7, 'physical_block': 7}, {'token_position': 5, 'logical_block': 3, 'physical_block': 3}, {'token_position': 6, 'logical_block': 5, 'physical_block': 5}, {'token_position': 7, 'logical_block': 11, 'physical_block': 11}, {'token_position': 8, 'logical_block': 7, 'physical_block': 7}, {'token_position': 9, 'logical_block': 3, 'physical_block': 3}, {'token_position': 10, 'logical_block': 5, 'physical_block': 5}, {'token_position': 11, 'logical_block': 11, 'physical_block': 11}, {'token_position': 12, 'logical_block': 7, 'physical_block': 7}, {'token_position': 13, 'logical_block': 3, 'physical_block': 3}, {'token_position': 14, 'logical_block': 5, 'physical_block': 5}, {'token_position': 15, 'logical_block': 11, 'physical_block': 11}, {'token_position': 16, 'logical_block': 7, 'physical_block': 7}, {'token_position': 17, 'logical_block': 3, 'physical_block': 3}, {'token_position': 18, 'logical_block': 5, 'physical_block': 5}, {'token_position': 19, 'logical_block': 11, 'physical_block': 11}], 'trace_tail': [('retain', 3, 2), ('alloc', 5), ('write', 'B', 'X', 2, 0, 5), ('write', 'B', 'Y', 2, 1, 5), ('release', 'A', 2, 0, 5)], 'paged_attention_gates= {'address_translation': True, 'prefix_blocks_shared': True, 'partial_release_keeps_shared': True}}
```

这个 demo 的关键证据：

1. token position 8 被翻译成 logical block 2、offset 0、physical block 11。
2. 请求 B 的前两个 physical blocks 与请求 A 共享，ref_count 从 1 变成 2。
3. 释放请求 A 后，shared blocks 7 和 3 仍保留，非共享 block 11 回到 free list。
4. 请求 C 随后复用了 physical block 11。
5. 当前活跃请求只浪费 2 个 block slot，说明分页不能消灭浪费，但能把浪费压缩到尾块级别。

17.16 常见误区

误区一：PagedAttention 只是更快的 attention。

它更重要的是 KV Cache 的分页式内存管理，kernel 只是实现这一管理方式所需的执行机制。

误区二：PagedAttention 完全没有浪费。

仍然有 block 内部碎片，只是比大段连续预留浪费少。

误区三：block size 越小越好。

block 小会减少内部浪费，但增加 block table、调度和 kernel 访问开销。

误区四：有了 PagedAttention 就不需要 scheduler。

PagedAttention 管 cache 存储，scheduler 管每轮执行谁。两者互相配合。

误区五：PagedAttention 和 prefix caching 没关系。

block-based cache 管理让共享 prefix 的物理 block 复用更自然。

17.17 面试追问

1. PagedAttention 的核心思想是什么？
2. 为什么连续 KV Cache 管理会浪费显存？
3. logical block、physical block、block table 分别是什么？
4. PagedAttention 如何借鉴操作系统分页？
5. block size 有什么 trade-off？
6. PagedAttention 和 continuous batching 有什么关系？
7. PagedAttention 是否完全消除了显存浪费？
8. 为什么 prefix sharing 更适合建立在 block-based cache 上？

参考回答思路：

1. 先说 PagedAttention 是 KV Cache 的分页式管理。
2. 再说请求逻辑 token 连续，但物理 KV blocks 可以不连续，通过 block table 映射。
3. 然后解释收益：减少预留浪费和碎片，支持动态增长、释放和共享。
4. 最后补 trade-off：block size、kernel 复杂度、block manager 开销。

17.18 小练习

1. 假设 block size 为 16，一个 100 token 请求需要几个 block？最后一个 block 浪费多少 token 位置？
2. 画出两个请求共享前三个 prefix blocks 后分别分叉的 block table。
3. 解释为什么请求结束后释放 blocks 比释放一段连续 cache 更容易复用。
4. 思考 block size 从 16 改成 8，会影响哪些指标？
5. 用一句话解释 PagedAttention 和普通 past_key_values 的区别。

17.19 本章小结

本章讲清了 PagedAttention 的核心思想。

PagedAttention 借鉴操作系统分页，把请求的逻辑 KV Cache 切成固定大小的 logical blocks，并映射到 GPU 上的 physical KV blocks。Block table 负责逻辑到物理的映射。这样可以减少连续分配带来的预留浪费和碎片，支持请求动态增长、完成释放和 prefix sharing。

下一章我们会继续深入 KV Cache Block Manager，讲它如何分配、释放、复用和追踪这些 physical blocks。

第 18 章 KV Cache Block Manager

上一章讲了 PagedAttention 的核心思想：把请求的逻辑 KV Cache 切成 logical blocks，并映射到 GPU 上的 physical blocks。本章继续讲谁来管理这些 blocks，这就是 KV Cache Block Manager。

如果说 PagedAttention 是“分页式读取 KV Cache 的 attention 机制”，那么 Block Manager 就是“负责分配、释放、复用、追踪 KV Cache block 的内存管理器”。

一句话概括：

KV Cache Block Manager 把 GPU 显存中的 KV Cache blocks 当成可分配资源，负责为请求分配 block、维护 block table、在请求结束后回收 block，并支撑 prefix sharing 和 continuous batching。

18.0 本讲资料边界与第二轮精修口径

本讲只讲教学版 KV Cache Block Manager。它覆盖 free list、block table、ref count、prefill block 准入、decode 动态扩展、请求结束释放、prefix sharing、copy-on-write 直觉、allocation failure、指标和最小可运行 demo，但不展开 vLLM 真实源码类名、GPU / CPU KV swap、preemption 策略、prefix cache hash 细节、多卡 KV 分布、CUDA kernel、真实调度器参数或生产容量规划。

资料校准口径：

1. vLLM / PagedAttention 论文强调 block-based KV cache 管理是降低 serving 内存浪费和提升吞吐的核心。
2. vLLM optimization 文档把 KV cache 容量、preemption、max_num_batched_tokens、max_num_seqs、chunked prefill 和 decode / prefill 平衡作为调优重点，说明 scheduler 必须和 KV block 资源联动。
3. vLLM prefix caching 文档说明 prefix cache 依赖 block 粒度的匹配与复用；本章只讲引用计数和共享块生命周期直觉，不展开 hash 设计。
4. 本章 demo 用纯 Python list 和 dict 模拟 block pool、ref count、free list、allocation failure 和 metrics，不等同于真实 vLLM Block Manager 实现。

参考资料：

1. vLLM paper / PagedAttention: <https://arxiv.org/abs/2309.06180>
2. vLLM optimization and tuning: <https://docs.vllm.ai/en/latest/configuration/optimization.html>
3. vLLM prefix caching: https://docs.vllm.ai/en/latest/features/automatic_prefix_caching.html
4. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics.html>

18.1 为什么需要 Block Manager

PagedAttention 需要 physical blocks。

但问题是：

1. 新请求来了，谁给它分配 block？
2. prompt 很长，需要多个 block，谁判断够不够？
3. decode 每生成新 token，当前 block 满了，谁分配下一个 block？
4. 请求结束，谁释放 block？
5. 多个请求共享 prefix，谁维护引用关系？

6. 显存不够时，谁拒绝或延迟请求？

这些不是 attention kernel 本身能解决的。需要一个独立的 cache manager。

Block Manager 的本质是：把 KV Cache 显存变成可调度、可追踪、可回收的资源池。

18.2 最小 Block Manager 的数据结构

假设 GPU 上一共有 N 个 physical blocks。

最小数据结构：

```
class BlockManager:
    def __init__(self, num_blocks, block_size):
        self.num_blocks = num_blocks
        self.block_size = block_size
        self.free_blocks = list(range(num_blocks))
        self.ref_count = [0 for _ in range(num_blocks)]
```

其中：

1. num_blocks: 物理 block 总数。
2. block_size: 每个 block 能放多少 token 的 KV。
3. free_blocks: 当前空闲 block 列表。
4. ref_count: 每个 block 被多少请求引用。

真实 vLLM 的实现更复杂，但核心思想离不开这些。

18.3 Sequence 的 block table

每个请求或 sequence 需要维护自己的 block table。

```
class SequenceState:
    def __init__(self, seq_id):
        self.seq_id = seq_id
        self.block_table = []
        self.num_tokens = 0
```

例如：

```
seq.block_table = [42, 8, 19]
seq.num_tokens = 37
```

含义是：

1. 第 0 个 logical block 映射到 physical block 42。
2. 第 1 个 logical block 映射到 physical block 8。
3. 第 2 个 logical block 映射到 physical block 19。
4. 当前序列已有 37 个 token。

block table 是 request 到 KV Cache 物理存储的索引。

18.4 分配 blocks

给一个请求分配 blocks，可以先算需要几个 block。

```
import math
```

```
def num_required_blocks(num_tokens, block_size):
    return math.ceil(num_tokens / block_size)
```

分配函数:

```
def allocate(self, seq, num_tokens):
    required = num_required_blocks(num_tokens, self.block_size)
    missing = required - len(seq.block_table)

    if missing <= 0:
        return True

    if len(self.free_blocks) < missing:
        return False

    for _ in range(missing):
        block_id = self.free_blocks.pop()
        self.ref_count[block_id] = 1
        seq.block_table.append(block_id)

    return True
```

这表示: 如果 sequence 需要更多 logical blocks, 就从 free list 里拿 physical blocks。

用公式写, block size 为 S , sequence i 当前 token 数为 T_i , 需要的 block 数为:

$$B_i = \left\lceil \frac{T_i}{S} \right\rceil$$

如果它当前已有 H_i 个 blocks, 还需要新增:

$$\Delta B_i = \max(0, B_i - H_i)$$

设当前 free block 数为 F_t , 单个请求的准入条件是:

$$G_{\{\mathrm{alloc}, i\}} = \mathbf{1}[\Delta B_i \leq F_t]$$

如果本轮 scheduler 想同时接入 waiting 集合 W_t , 整体 block 预算约束是:

$$\sum_{i \in W_t} \Delta B_i \leq F_t$$

block 使用率:

$$U_{\{\mathrm{block}\}} = \frac{N_{\{\mathrm{used}\}}}{N_{\{\mathrm{total}\}}}$$

共享 block 数:

$$N_{\{\mathrm{shared}\}} = \sum_p \mathbf{1}[c_p > 1]$$

其中 c_p 是 physical block p 的引用计数。Block Manager 门禁可以写成:

$$G_{\{\mathrm{bm}\}} = G_{\{\mathrm{prefill}\}} G_{\{\mathrm{decode}\}} G_{\{\mathrm{ref}\}} G_{\{\mathrm{free}\}} G_{\{\mathrm{admit}\}} G_{\{\mathrm{bm}\}}$$

这些量让 scheduler 可以在调度前知道 “能不能放下”, 而不是等 GPU OOM 后再失败。

18.5 Prefill 时如何分配

prefill 前, scheduler 需要判断 prompt token 是否能放下。

例如 prompt 长度 1000, block size 16, 需要:

$$\text{ceil}(1000 / 16) = 63 \text{ blocks}$$

如果 free blocks 不够, scheduler 就不能让这个请求进入 prefill。

这就是 admission control:

```
if block_manager.can_allocate(prompt_len):
    schedule_prefill(request)
```



```
else:
    keep_waiting(request)
```

这比只按请求数限制更准确。因为一个 100 token 请求和一个 8000 token 请求的 cache 需求完全不同。

18.6 Decode 时如何扩展

decode 每生成一个新 token, sequence 的 token 数增加 1。

如果当前 logical block 还有空位, 不需要新 block。

如果当前 block 满了, 需要分配一个新 physical block。

```
def append_token_slot(self, seq):
    next_token_index = seq.num_tokens
    needed_blocks = num_required_blocks(next_token_index + 1, self.block_size)

    if needed_blocks > len(seq.block_table):
        if not self.free_blocks:
            return False
        block_id = self.free_blocks.pop()
        self.ref_count[block_id] = 1
        seq.block_table.append(block_id)

    seq.num_tokens += 1
    return True
```

这就是 decode 阶段 cache 动态增长的最小逻辑。

18.7 释放 blocks

请求完成后, 要释放它占用的 blocks。

```
def free(self, seq):
    for block_id in seq.block_table:
        self.ref_count[block_id] -= 1
        if self.ref_count[block_id] == 0:
            self.free_blocks.append(block_id)

    seq.block_table = []
```

如果没有 prefix sharing, ref_count 基本就是 1。

如果多个 sequence 共享 block, 只有引用计数归零时才能真正释放。

这也是为什么 prefix sharing 需要 ref_count。

18.8 block table 和 scheduler 的关系

Scheduler 做决策时, 需要问 Block Manager:

1. 这个 waiting 请求能不能 prefill?
2. running 请求 decode 下一 token 是否还有 block?
3. 如果本轮加入多个请求, 总 block 是否足够?
4. 哪些 finished 请求可以释放 block?
5. 当前 free block 数是否过低?

因此 scheduler 和 block manager 是强耦合的。

一个没有 block 预算的 scheduler, 很容易把请求调进来后才 OOM。

真实系统要尽量在调度前就判断资源是否足够。

18.9 block 资源视角下的请求状态

从 block manager 角度看，请求状态可以这样理解：

WAITING：还没有分配 KV block。

PREFILLING：正在为 prompt 写入 KV block。

DECODING：已经持有 block，后续可能继续扩展。

FINISHED：应该释放 block。

ABORTED：也必须释放 block。

FAILED：同样必须释放 block。

所以任何终止路径都要走 cache cleanup。

只处理正常 FINISHED，而忘记 ABORTED / FAILED，是典型线上显存泄漏来源。

18.10 block 内部偏移

给定 token position，如何找到 block 和 offset？

```
logical_block_id = token_position // block_size  
offset = token_position % block_size  
physical_block_id = seq.block_table[logical_block_id]
```

然后 attention kernel 根据 physical_block_id 和 offset 找到实际 KV 地址。

这个计算很简单，但非常关键。

它把逻辑 token 位置转换成了物理 cache 位置。

18.11 Copy-on-write 和 prefix sharing

如果两个请求共享 prefix，它们可以共享前面几个 physical blocks。

```
seq A block_table: [1, 2, 3, 8]  
seq B block_table: [1, 2, 3, 9]
```

blocks 1、2、3 的 ref_count 是 2。

当某个请求继续生成并需要写新 token 时，不能修改另一个请求也共享的 block 内容。

如果要写入共享 block 的不同位置，就需要 copy-on-write。

简化理解：

1. 读共享 prefix 可以复用 block。
2. 生成分叉后，新 token 写入自己的新 block。
3. 如果共享 block 需要被修改，先复制再写。

Prefix caching 的实现细节会在后续章节展开。

18.12 Eviction 和 swapping 的直觉

基础 Block Manager 只管理 GPU blocks。

更复杂的系统可能会有：

1. GPU KV Cache。
2. CPU KV Cache。
3. 远端 KV Cache。
4. swap out / swap in。
5. recompute。

当 GPU block 不够时，可以选择：

1. 拒绝新请求。
2. 让请求继续等待。
3. 抢占某些低优先级请求。
4. 把部分 cache 移到 CPU。
5. 丢弃 cache，后续重算。

本章只讲最小 GPU block manager，但你要知道生产系统会把 cache 当成多级资源来管理。

18.13 常见 block manager 指标

应该观测：

1. total blocks。
2. free blocks。
3. used blocks。
4. block utilization。
5. allocation failures。
6. blocks per request。
7. ref_count 分布。
8. prefix shared blocks。
9. eviction / swap 次数。
10. OOM 次数。

这些指标比单纯 GPU 显存更细。

例如 GPU 显存还没满，但 free blocks 不足，说明 block 池配置或碎片策略可能有问题。

18.14 和 MiniEngine 的差距

第二部分 MiniEngine 直接使用 past_key_values。

问题是：

1. batch 顺序固定。
2. cache 难以动态加入新请求。
3. finished 请求释放不精细。
4. 多个 prefill batch 的 cache 难合并。
5. prefix sharing 不自然。

Block Manager 解决的是这些问题的底层资源管理部分。

它让请求的 cache 和 batch 组织解耦：请求持有 block table，scheduler 每轮临时组成 batch，attention kernel 根据 block table 读取 KV。

18.15 Block Manager 分配、释放和可运行 demo

下面的 demo 不依赖外部库。它模拟一个最小 Block Manager：

1. 请求 A prefill 6 个 token，block size 为 4，所以先分配 2 个 blocks。
2. A decode 3 个 token，跨过 block 边界后追加第 3 个 block。
3. 请求 B 共享 A 的前 2 个 blocks，再为自己的分叉 token 分配新 block。
4. A finished 后释放非共享 block，shared blocks 仍保留。
5. 请求 C 复用 A 释放的 block；请求 D 因 free blocks 不足被拒绝。

```
from dataclasses import dataclass, field
from math import ceil
```

```

@dataclass
class SequenceState:
    seq_id: str
    block_table: list[int] = field(default_factory=list)
    num_tokens: int = 0
    status: str = "WAITING"

class ToyBlockManager:
    def __init__(self, num_blocks, block_size):
        self.block_size = block_size
        self.free_blocks = list(range(num_blocks))
        self.ref_count = {block_id: 0 for block_id in range(num_blocks)}
        self.allocation_failures = 0
        self.trace = []

    def required_blocks(self, tokens):
        return ceil(tokens / self.block_size) if tokens > 0 else 0

    def can_allocate(self, additional_blocks):
        return len(self.free_blocks) >= additional_blocks

    def allocate_block(self):
        block_id = self.free_blocks.pop(0)
        self.ref_count[block_id] = 1
        self.trace.append(("alloc", block_id, "free", len(self.free_blocks)))
        return block_id

    def allocate_for_tokens(self, seq, target_tokens):
        needed = self.required_blocks(target_tokens)
        missing = needed - len(seq.block_table)
        if missing <= 0:
            seq.num_tokens = max(seq.num_tokens, target_tokens)
            return True
        if not self.can_allocate(missing):
            self.allocation_failures += 1
            self.trace.append(("alloc_failed", seq.seq_id, missing, "free", len(self.free_blocks)))
            return False
        for _ in range(missing):
            seq.block_table.append(self.allocate_block())
        seq.num_tokens = max(seq.num_tokens, target_tokens)
        seq.status = "DECODING"
        return True

    def append_token_slot(self, seq):
        return self.allocate_for_tokens(seq, seq.num_tokens + 1)

    def fork_prefix(self, source, new_seq_id, prefix_tokens):
        prefix_blocks = self.required_blocks(prefix_tokens)
        child = SequenceState(new_seq_id, [], prefix_tokens, "DECODING")
        for block_id in source.block_table[:prefix_blocks]:
            self.ref_count[block_id] += 1
            child.block_table.append(block_id)
            self.trace.append(("retain", block_id, "ref", self.ref_count[block_id]))
        return child

```

```

def release(self, seq, reason):
    released = []
    for block_id in seq.block_table:
        self.ref_count[block_id] -= 1
        if self.ref_count[block_id] == 0:
            self.free_blocks.append(block_id)
            released.append(block_id)
    self.trace.append(("release", seq.seq_id, reason, released))
    seq.block_table = []
    seq.num_tokens = 0
    seq.status = reason
    return released

def metrics(self, active_sequences):
    used_blocks = sum(1 for count in self.ref_count.values() if count > 0)
    shared_blocks = sum(1 for count in self.ref_count.values() if count > 1)
    return {
        "free_blocks": len(self.free_blocks),
        "used_blocks": used_blocks,
        "shared_blocks": shared_blocks,
        "allocation_failures": self.allocation_failures,
        "blocks_per_sequence": {
            seq.seq_id: len(seq.block_table) for seq in active_sequences
        },
    }

def run_block_manager_demo():
    manager = ToyBlockManager(num_blocks=6, block_size=4)
    seq_a = SequenceState("A")
    prefill_a = manager.allocate_for_tokens(seq_a, 6)
    for _ in range(3):
        manager.append_token_slot(seq_a)

    seq_b = manager.fork_prefix(seq_a, "B", prefix_tokens=8)
    manager.append_token_slot(seq_b)
    shared_after_fork = {block: manager.ref_count[block] for block in [0, 1, 2, 3]}

    released_a = manager.release(seq_a, "FINISHED")
    after_release_a = {block: manager.ref_count[block] for block in [0, 1, 2, 3]}

    seq_c = SequenceState("C")
    admit_c_before = manager.can_allocate(manager.required_blocks(12))
    prefill_c = manager.allocate_for_tokens(seq_c, 12)
    seq_d = SequenceState("D")
    admit_d_before = manager.can_allocate(manager.required_blocks(4))
    prefill_d = manager.allocate_for_tokens(seq_d, 4)

    active = [seq for seq in [seq_b, seq_c, seq_d] if seq.block_table]
    metrics = manager.metrics(active)
    summary = {
        "prefill_a": prefill_a,
        "seq_a_table_before_release": [0, 1, 2],
        "seq_b_table": seq_b.block_table,
    }

```

```

    "shared_after_fork": shared_after_fork,
    "released_a": released_a,
    "after_release_a": after_release_a,
    "admit_c_before": admit_c_before,
    "prefill_c": prefill_c,
    "seq_c_table": seq_c.block_table,
    "admit_d_before": admit_d_before,
    "prefill_d": prefill_d,
    "metrics": metrics,
}
gates = {
    "prefill_allocates_blocks": prefill_a
    and summary["seq_a_table_before_release"] == [0, 1, 2],
    "decode_extends_on_boundary": seq_b.block_table == [0, 1, 3],
    "ref_count_shares_prefix": shared_after_fork == {0: 2, 1: 2, 2: 1, 3: 1},
    "release_keeps_shared_blocks": after_release_a == {0: 1, 1: 1, 2: 0, 3: 1},
    "freed_blocks_reused": prefill_c and seq_c.block_table == [4, 5, 2],
    "admission_failure_visible": not admit_d_before
    and not prefill_d
    and metrics["allocation_failures"] == 1,
}
gates["block_manager_gate"] = all(gates.values())
return summary, manager.trace[-12:], gates

```

```

summary, trace_tail, gates = run_block_manager_demo()
print("block_manager_summary=", summary)
print("trace_tail=", trace_tail)
print("block_manager_gates=", gates)

```

一组稳定输出：

```

block_manager_summary= {'prefill_a': True, 'seq_a_table_before_release': [0, 1, 2], 'seq_b_table': [0, 1, 3], 'shared_af
trace_tail= [('alloc', 0, 'free', 5), ('alloc', 1, 'free', 4), ('alloc', 2, 'free', 3), ('retain', 0, 'ref', 2), ('retain'
block_manager_gates= {'prefill_allocates_blocks': True, 'decode_extends_on_boundary': True, 'ref_count_shares_prefix':

```

这个 demo 的关键证据：

1. prefill A 的 6 个 token 先分配 blocks [0,1], decode 跨过边界后扩展到 [0,1,2]。
2. B 共享 A 的前两个 blocks, ref_count 变成 {0:2,1:2}, 然后 B 自己分配分叉 block 3。
3. 释放 A 只释放非共享 block 2, 共享 blocks 0 和 1 仍然保留。
4. C 随后复用释放出来的 block 2。
5. D 因 free blocks 不足分配失败, allocation_failures=1, 说明 scheduler 应在调度前查询 block manager。

18.16 常见误区

误区一：Block Manager 只是一个 free list。

free list 是基础，真实 block manager 还要维护 block table、引用计数、分配策略、释放、共享和指标。

误区二：请求结束后 Python 对象删除就能释放 cache。

必须显式归还 physical blocks，否则 block 池会认为资源仍被占用。

误区三：只要 GPU 显存够，就一定能分配 block。

block 池有自己的管理粒度和预算，可能出现 free blocks 不足或策略限制。

误区四：prefix sharing 只是字典缓存。

底层还要处理共享 blocks 的引用计数和 copy-on-write。

误区五：scheduler 和 block manager 可以完全独立。

scheduler 必须知道 block 资源，否则无法做可靠 admission control。

18.17 面试追问

1. KV Cache Block Manager 负责什么？
2. block table 的作用是什么？
3. prefill 和 decode 阶段分别如何分配 block？
4. 请求完成后为什么必须释放 blocks？
5. 为什么需要 ref_count？
6. scheduler 为什么需要查询 block manager？
7. prefix sharing 和 copy-on-write 在 block manager 中如何理解？
8. Block Manager 相比简单 past_key_values 有什么优势？

参考回答思路：

1. 先说 block manager 管理 GPU KV Cache physical blocks。
2. 再说每个 sequence 有 block table，记录 logical block 到 physical block 的映射。
3. 然后讲分配、扩展、释放和 ref_count。
4. 最后说明它支撑 PagedAttention、continuous batching、prefix sharing 和 admission control。

18.18 小练习

1. 实现一个只有 allocate 和 free 的 BlockManager。
2. 假设 block size 为 16，prompt 1000 tokens，需要多少 blocks？
3. 模拟一个请求生成 token 时 block 不够，触发新 block 分配。
4. 模拟两个请求共享前三个 blocks，并用 ref_count 管理释放。
5. 设计 5 个 block manager 指标，用于排查线上 OOM。

18.19 本章小结

本章讲清了 KV Cache Block Manager。

PagedAttention 需要 physical KV blocks 和 block table，而 Block Manager 负责管理这些 blocks：分配、扩展、释放、引用计数、共享和资源预算。它把 KV Cache 从普通张量变成可调度资源，是 vLLM 支持高并发、continuous batching 和 prefix caching 的关键基础。

下一章我们会进入 Continuous Batching 与 iteration-level scheduling，讲 vLLM 如何每轮动态选择请求，让 GPU 持续处理有效工作。

第 19 章 Continuous Batching 与 iteration-level scheduling

上一章讲了 KV Cache Block Manager：请求进入系统后，KV Cache 不再是一整块连续张量，而是由 block manager 分配、追踪和释放的一组 physical blocks。

有了 block manager，下一步才是调度问题：既然每个请求的 cache 可以动态增长和释放，那 serving engine 能不能不要等一个 batch 全部结束，而是在每一轮生成时动态加入新请求、移除完成请求？这就是 continuous batching，也常被称为 iteration-level scheduling。

一句话概括：

Continuous batching 把调度粒度从“整批请求”下沉到“每一轮模型迭代”，让系统可以在每个 decode step 重新选择 running batch，从而提升吞吐、降低排队等待，并更充分利用 KV Cache block。

19.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考三类公开资料：

1. Orca OSDI 2022 论文对 Transformer 生成 workload 的 multi-iteration 特征、request-level scheduling 的局限，以及 iteration-level scheduling 的定义。
2. vLLM optimization / tuning 文档对 KV cache 不足、preemption、max_num_seqs、max_num_batched_tokens、chunked prefill、decode 优先和 TTFT / ITL 取舍的公开口径。
3. vLLM metrics 文档对 running / waiting 请求数、KV cache usage、TTFT、inter-token latency、queue time、prefill time 和 decode time 的观测口径。

本章只讲教学版 continuous batching 和 iteration-level scheduling，不展开 vLLM 真实 scheduler 源码、CUDA graph、worker / executor、multi-step scheduling、prefix cache policy、preemption / swap / recompute 实现、多卡流水、生产级优先级队列或 OpenAI-compatible server 参数全集。

本章给出的公式和 demo 只用于建立可面试、可验证的最小模型：每轮 scheduler 先处理 dynamic arrival 和 cleanup，再在 token budget、active sequence budget 与 KV block budget 下选择 decode 和 prefill。真实系统还会叠加 chunked prefill、prefix caching、LoRA、多模态输入、speculative decoding、抢占、租户公平和可观测性治理。

参考资料：

1. Orca OSDI 2022: <https://www.usenix.org/conference/osdi22/presentation/yu>
2. vLLM optimization and tuning: <https://docs.vllm.ai/en/latest/configuration/optimization.html>
3. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics.html>

19.1 本章目标

读完本章，你应该能讲清：

1. 为什么传统 static batching 不适合自回归 LLM serving。
2. iteration-level scheduling 为什么出现，它和 request-level scheduling 的区别是什么。
3. continuous batching 每一轮到底调度什么。
4. prefill、decode、token budget 和 KV block budget 如何共同约束调度。
5. 为什么 continuous batching 需要 KV Cache block manager 配合。
6. 面试中如何用时间线解释 vLLM-like scheduler 的核心思想。

19.2 先看传统 batch 的问题

传统深度学习推理里，batch 通常是一次性组织好的。

例如图像分类：

收集 32 张图片 -> 拼成 batch -> forward 一次 -> 返回 32 个结果

这种任务的特点是：一个请求通常只需要一次模型执行。

LLM 自回归生成不一样。一个请求要生成 100 个 token，就要经历多轮模型执行：

prefill prompt -> decode token 1 -> decode token 2 -> ... -> decode token 100

如果把多个请求固定成一个 batch，然后等整个 batch 全部完成，就会遇到几个问题。

第一，短请求被长请求拖住。

request A: 生成 10 个 token

request B: 生成 200 个 token

request C: 生成 20 个 token

如果 A、B、C 被固定在同一个 batch 里，A 和 C 很快结束，但 batch 还要陪 B 跑很久。A、C 的计算位置空出来后，如果系统不能动态补入新请求，GPU 利用率会下降。

第二，新请求 TTFT 变差。

假设 batch B 正在生成，新的 request D 到达。如果调度粒度是整批请求，D 必须等当前 batch 全部结束才能 prefill。只要 batch 里有一个长输出请求，D 的排队等待就会变长。

第三，batch size 会越来越小。

固定 batch 开始时可能有 32 个请求，但随着短请求结束，后面可能只剩几个长请求在跑。decode 阶段每轮只生成一个 token，batch 越小，GPU 越难吃满。

这就是为什么 LLM serving 不能只套用普通推理服务的 static batching。

19.3 Orca 提出的 iteration-level scheduling

Orca 论文指出，Transformer 生成模型的 serving workload 具有 multi-iteration 特征：一个请求不是执行一次模型就结束，而是需要多次迭代，每次迭代通常生成一个 token。

传统 request-level scheduling 的粒度是：

选择一批请求 -> 运行到这些请求完成 -> 再选择下一批请求

Iteration-level scheduling 的粒度是：

选择一批请求 -> 只执行一轮模型迭代 -> 回到 scheduler -> 重新选择下一轮请求

也就是说，scheduler 不再把 batch 当成一个长期固定对象，而是每轮都可以重新组织 batch。

一个简单时间线：

```
t0: batch = [A, B]
t1: A 生成 1 token, B 生成 1 token
t2: A 完成, C 到达, batch = [B, C]
t3: B 生成 1 token, C prefill 或 decode
t4: D 到达, batch = [B, C, D]
```

这里的关键不是“batch 很大”，而是“batch 可以持续变化”。

所以 continuous batching 可以理解为 iteration-level scheduling 在 LLM serving engine 中的工程化形态。

19.4 Continuous Batching 的核心直觉

Continuous batching 的直觉是：

不要把请求固定成一批跑到结束，而是在每个 engine step 重新决定哪些请求执行。

一个 engine step 通常会做这些事：

1. 接收新到达请求，放入 waiting queue。
2. 清理已经完成、取消或失败的请求。
3. 释放 finished 请求占用的 KV blocks。
4. 从 running 请求中选择本轮继续 decode 的序列。
5. 从 waiting queue 中选择部分请求做 prefill。
6. 检查 token budget、sequence budget 和 KV block budget。
7. 执行本轮模型 forward。
8. 采样新 token，更新请求状态。
9. 把 token 通过 streaming 返回给客户端。

如果画成循环：

```
while engine is running:
    schedule one step
    run model forward
    sample tokens
    update states
    stream outputs
    free finished resources
```

这就是 serving engine 的心跳。

19.5 prefill 和 decode 为什么要一起调度

在 LLM serving 中，waiting 请求通常需要先做 prefill，running 请求通常需要继续 decode。

prefill 的特点：

1. 一次处理 prompt 中的多个 token。
2. 计算量大，容易占用较长 GPU 时间。
3. 目标是尽快产出 first token，影响 TTFT。
4. 会一次性写入较多 KV Cache。

decode 的特点：

1. 每个 running 请求通常每轮只生成一个 token。
2. 单轮 token 数少，但需要频繁执行。
3. 直接影响 TPOT 和流式输出稳定性。
4. KV Cache 会逐 token 增长。

如果只偏向 prefill，新请求 TTFT 会变好，但已有请求的流式输出会卡顿。

如果只偏向 decode，已有请求输出平稳，但 waiting queue 中的新请求可能长时间拿不到 first token。

所以 scheduler 的核心难点是：在同一个 engine step 里，如何平衡 prefill 和 decode。

最小策略可以是：

先保证 running decode 有基本进度，再用剩余 token budget 接纳部分 waiting prefill。

更激进的策略可能会在负载高时限制长 prompt prefill，或者把长 prompt 切成多个 chunk，这会在后续 chunked prefill 章节展开。

19.6 token budget：调度不是只数请求数

最小教学版 scheduler 可能只限制 max_running_requests。

但真实 serving engine 不能只看请求数，因为不同请求的 token 开销差异很大。

两个请求数相同的 batch，代价可能完全不同：

batch A: 8 个请求，每个 prompt 64 tokens

batch B: 8 个请求，每个 prompt 8192 tokens

它们都是 8 个请求，但 batch B 的 prefill 计算和 KV Cache 需求远大于 batch A。

因此 continuous batching 通常需要 token budget。

简化定义：

`max_num_batched_tokens = 4096`

每轮调度时，scheduler 统计本轮要处理的 token 数：

本轮 token 数 = prefill token 数 + decode token 数

decode token 数通常近似等于 running sequence 数，因为每个 sequence 生成一个 token。

prefill token 数则取决于新请求 prompt 长度，或者 chunked prefill 中本轮处理的 prompt chunk 长度。

一个简化调度例子：

```
def schedule_with_token_budget(waiting_queue, running_requests, max_tokens):
    scheduled_decode = []
    scheduled_prefill = []
    used_tokens = 0
```

```

for req in running_requests:
    if used_tokens + 1 > max_tokens:
        break
    scheduled_decode.append(req)
    used_tokens += 1

while waiting_queue:
    req = waiting_queue.peek()
    prompt_len = len(req.input_ids)
    if used_tokens + prompt_len > max_tokens:
        break
    scheduled_prefill.append(waiting_queue.pop())
    used_tokens += prompt_len

return scheduled_prefill, scheduled_decode

```

这个代码还很粗糙，但它说明了关键点：scheduler 真正在调度的是 token workload，而不是单纯的 request count。

把它写成公式，本轮 iteration 记为 τ ，本轮 decode 集合记为 D_{τ} ，本轮 prefill 集合记为 P_{τ} 。本轮 decode token 数可以近似为：

$$N_{\text{dec},\tau} = |D_{\tau}|$$

如果请求 i 在本轮 prefill 了 $p_{i,\tau}$ 个 prompt tokens，则本轮 prefill token 数是：

$$N_{\text{pre},\tau} = \sum_{i \in P_{\tau}} p_{i,\tau}$$

本轮 token budget 门禁是：

$$G_{\text{tok},\tau} = \mathbf{1}[N_{\text{dec},\tau} + N_{\text{pre},\tau} \leq C_{\text{tok}}]$$

这里的 C_{tok} 可以理解成教学版 `max_num_batched_tokens`。它不是总吞吐目标，而是控制单轮 workload 和延迟尖刺的上限。

19.7 KV block budget: 能算不代表能放下

token budget 控制计算量，但还不够。

LLM serving 的另一个硬约束是 KV Cache 显存。

一个请求能不能进入 prefill，不仅取决于本轮 token budget，还取决于 block manager 是否能给它分配足够的 KV blocks。

例如：

```

block_size = 16
prompt_len = 4096
需要 blocks = ceil(4096 / 16) = 256

```

如果当前 free blocks 只有 200 个，即使 token budget 允许，也不能调度这个请求，否则 prefill 写 KV Cache 时会 OOM。

decode 阶段也一样。每个 running 请求生成新 token 时，如果当前 logical block 满了，需要再分配一个 physical block。

所以每轮 scheduler 至少要问 block manager 两类问题：

1. waiting 请求做 prefill 是否能分配足够 blocks。
2. running 请求继续 decode 是否可能需要追加新 block。

简化伪代码：

```
def can_schedule_prefill(req, block_manager):
    return block_manager.can_allocate(req.prompt_len)
```

```
def can_schedule_decode(req, block_manager):
    if req.current_block_has_slot():
        return True
    return block_manager.has_free_block()
```

这就是上一章为什么说 scheduler 和 block manager 强耦合。

没有 block budget 的 continuous batching，只是“看起来动态”，实际上很容易把系统推向 OOM。

把 block 约束写成公式，block size 记为 S ，请求 i 当前需要缓存的 token 数记为 T_i ，则请求所需 block 数是：

$$B_i = \left\lceil \frac{T_i}{S} \right\rceil$$

如果请求 i 当前已经持有 H_i 个 block，那么本轮新增 block 需求是：

$$\Delta B_{i,\tau} = \max(0, B_i - H_i)$$

本轮所有被调度请求的新增 block 数不能超过当前 free blocks：

$$G_{kv,\tau} = 1 \left[\sum_{i \in D_\tau \cup P_\tau} \Delta B_{i,\tau} \leq F_\tau \right]$$

因此 continuous batching 的准入条件至少是 token budget 与 KV block budget 同时成立：

$$G_{admit,\tau} = G_{tok,\tau} G_{kv,\tau}$$

19.8 一轮 continuous batching 的完整流程

现在把 token budget 和 KV block budget 合起来看。

一个简化 engine step 可以写成：

```
def engine_step(waiting_queue, running, block_manager, max_tokens):
    scheduled_decode = []
    scheduled_prefill = []
    used_tokens = 0

    # 1. 清理完成请求，释放 KV blocks。
    still_running = []
    for req in running:
        if req.finished or req.aborted:
            block_manager.free(req)
        else:
            still_running.append(req)
    running = still_running
```

```

# 2. 优先给已有 running 请求安排 decode, 保证流式输出。
for req in running:
    if used_tokens + 1 > max_tokens:
        break
    if not block_manager.can_append_slot(req):
        continue
    scheduled_decode.append(req)
    used_tokens += 1

# 3. 用剩余 budget 接纳 waiting 请求做 prefill。
while waiting_queue:
    req = waiting_queue.peek()
    prompt_len = len(req.input_ids)
    if used_tokens + prompt_len > max_tokens:
        break
    if not block_manager.can_allocate(req, prompt_len):
        break

    req = waiting_queue.pop()
    block_manager.allocate(req, prompt_len)
    scheduled_prefill.append(req)
    used_tokens += prompt_len

# 4. 执行模型。
outputs = run_model_forward(scheduled_prefill, scheduled_decode)

# 5. 更新状态、采样 token、加入 running。
update_requests(outputs)
running.extend(req for req in scheduled_prefill if not req.finished)

return running

```

真实系统会复杂得多，例如支持多 beam、LoRA、多模态、prefix cache、chunked prefill、speculative decoding、优先级和抢占，但主干逻辑就是这几步。

19.9 为什么 continuous batching 能提高吞吐

Continuous batching 提高吞吐的原因不是魔法，而是减少了几个浪费。

第一，减少 batch 空洞。

短请求完成后，可以在后续 iteration 中移出，新的 waiting 请求可以补入，不必让已完成请求占着 batch 位置。

第二，减少排队等待。

新请求不必等当前整批请求全部结束，只要下一轮调度有 token budget 和 KV block budget，就可以进入 prefill。

第三，提高 GPU 利用率。

decode 单步计算粒度小，如果 batch 太小，GPU 利用率低。continuous batching 尽量让每轮都有足够请求参与。

第四，配合 PagedAttention 提高可调度并发。

PagedAttention 降低 KV Cache 碎片和浪费，block manager 让 cache 动态分配释放，scheduler 才能安全地把更多请求放进 running set。

也就是说：

PagedAttention 解决 cache 怎么放

Block Manager 解决 cache 怎么分配和释放

Continuous Batching 解决每一轮谁来跑

三者合起来，才构成 vLLM-like serving engine 的核心。

可以用一个很小的式子量化 fixed batch 的空洞。假设固定 batch 中请求集合为 S ，请求 i 实际输出 y_i 个 token，则静态 batch decode 行数近似是：

$$W_{\text{static}} = |S| \max_{i \in S} y_i$$

真正有效输出行数是：

$$W_{\text{valid}} = \sum_{i \in S} y_i$$

静态 batch 的空洞行数是：

$$W_{\text{hole}} = W_{\text{static}} - W_{\text{valid}}$$

continuous batching 的目标不是让 W_{valid} 消失，而是让已经完成的请求尽快退出 execution batch，让后续 iteration 的有效行比例更高。

19.10 它的代价和副作用

Continuous batching 不是只有好处。

代价一：scheduler 更复杂。

系统需要维护 waiting、running、finished、aborted 等状态，还要在每轮处理资源分配、释放和异常清理。

代价二：延迟和吞吐有 trade-off。

如果每轮都尽量塞满 token budget，吞吐可能更高，但单个请求的 TTFT 或 TPOT 可能变差。

代价三：长 prompt 可能影响 decode 稳定性。

一个很长的 prefill 加入本轮，会占用大量计算时间，导致已有 streaming 请求下一 token 返回变慢。

代价四：调度策略会影响公平性。

如果总是优先短请求，长请求可能饥饿。如果总是 FIFO，短请求可能被长 prompt 阻塞。

代价五：实现难度高。

要正确处理请求取消、客户端断连、OOM、超时、prefix sharing、block ref count 和 streaming 顺序。

所以线上系统通常要通过参数和策略调优，而不是简单地“开大 batch”。

19.11 常见调度策略

Continuous batching 只是机制，具体策略可以有很多种。

策略一：FIFO。

先来的 waiting 请求先 prefill，简单公平，但可能被长 prompt 阻塞。

策略二：decode-first。

优先保证 running 请求每轮 decode，减少流式输出卡顿，再用剩余 budget 做 prefill。

策略三：prefill-first。

优先降低新请求 TTFT，但高负载下可能让已有请求 TPOT 抖动。

策略四: token-budget based。

每轮严格控制总 token 数, 避免某一轮过大导致延迟尖刺。

策略五: length-aware。

根据 prompt 长度、已生成长度或预估输出长度做分桶, 减少长短请求互相影响。

策略六: priority-aware。

按租户等级、业务优先级、deadline 或 SLA 调度, 但要防止低优先级请求长期饥饿。

真实 vLLM-like 系统一般不是单一策略, 而是多种约束叠加:

FIFO fairness + token budget + KV block budget + max running seqs + prefill/decode policy

19.12 TTFT、TPOT 和吞吐如何被影响

Continuous batching 会同时影响三个指标。

TTFT 取决于:

1. 请求在 waiting queue 中等多久。
2. scheduler 多快给它安排 prefill。
3. prefill 是否被长队列或 token budget 阻塞。
4. prefill 执行时间和 first token sampling 时间。

TPOT 取决于:

1. running 请求是否每轮都能 decode。
2. 每轮 batch 中是否混入过大的 prefill。
3. decode batch size 是否稳定。
4. GPU kernel、KV Cache 读取和采样开销。

吞吐取决于:

1. 每轮有效 token 数。
2. GPU 利用率。
3. KV Cache 显存是否限制并发。
4. scheduler 是否能持续填充 batch。

这三个指标经常互相拉扯。

例如, 提高 max_num_batched_tokens 可能提升吞吐, 但也可能让单轮执行时间变长, 导致 TPOT 抖动。

降低 max_num_batched_tokens 可能让流式输出更平滑, 但 GPU 利用率下降, 总吞吐降低。

面试里不要只说 “continuous batching 提升吞吐”, 更好的回答是: 它通过动态填充 batch 提升吞吐, 但需要用 token budget 和 cache budget 控制 TTFT、TPOT 和显存风险。

如果请求 i 的到达时间是 a_i , 客户端看到 first token 的时间是 f_i , 第 j 个输出 token 可见时间是 $s_{i,j}$, 输出 token 数是 y_i , 则:

$$T_{\text{ttft},i} = f_i - a_i$$

当 $y_i > 1$ 时, 平均 TPOT 可以写成:

$$T_{\text{tpot},i} = \frac{s_{i,y_i} - s_{i,1}}{y_i - 1}$$

单轮 iteration 的吞吐可以看成:

$$Q_{\text{tok},\tau} = \frac{N_{\text{dec},\tau} + N_{\text{pre},\tau}}{\Delta t_{\tau}}$$

但线上调优不能只最大化 $Q_{\text{tok},\tau}$ 。如果 $N_{\text{pre},\tau}$ 太大，单轮 Δt_{τ} 会变长，已有 streaming 请求的 TPOT 可能变差。

19.13 和 dynamic batching 的区别

很多人会把 dynamic batching 和 continuous batching 混在一起。

普通 dynamic batching 常见于传统在线推理：

在短时间窗口内收集请求 -> 拼成 batch -> forward 一次 -> 返回

它解决的是“如何把同时到达或近似同时到达的请求拼起来”。

Continuous batching 面向自回归生成：

每一轮生成都重新组织 batch，请求可以在 decode 过程中加入或退出

关键差异：

对比点	Dynamic Batching	Continuous Batching
典型任务	单次 forward 推理	多轮自回归生成
调度粒度	请求到达窗口	模型迭代 step
batch 是否长期变化	一般 forward 前确定	每轮都可能变化
是否处理 KV Cache 增长	通常不需要	必须处理
核心指标	单次延迟和吞吐	TTFT、TPOT、吞吐、显存

所以 continuous batching 可以看作是为 LLM 自回归 workload 定制的动态调度机制。

19.14 面向专家：iteration-level scheduling 的本质

从系统角度看，iteration-level scheduling 做了一个重要拆分：

request lifetime != execution batch lifetime

请求的生命周期可能持续几秒甚至几十秒，但 execution batch 只存在一个 engine iteration。

这带来一个好处：scheduler 每轮都有机会重新优化资源使用。

但也带来一个约束：请求状态必须跨 iteration 持久保存。

需要持久保存的状态包括：

1. 已输入 token 和已生成 token。
2. sampling 参数。
3. block table。
4. 当前长度和已生成长度。
5. 是否遇到 EOS 或 stop string。
6. streaming offset。
7. 请求优先级和超时信息。

因此，continuous batching 的本质不是“把 requests 拼 batch”这么简单，而是：

用持久化 request state + 短生命周期 execution batch，解耦请求生命周期和 GPU 执行批次。

这个视角非常适合系统设计面试。

19.15 最小实现时容易踩的坑

坑一：finished 请求没有及时释放 KV blocks。

表现是请求数不高，但 free blocks 越来越少，最后 OOM。

坑二：只处理正常结束，忘了取消和异常路径。

客户端断开、超时、采样异常、模型执行异常，都要进入 cleanup。

坑三：prefill 长请求把 decode 卡住。

表现是 tokens/s 看起来不低，但用户看到流式输出一顿一顿。

坑四：只限制请求数，不限制 token 数。

表现是同样并发数下，短 prompt 正常，长 prompt 直接延迟尖刺或 OOM。

坑五：decode 追加 block 失败后状态不一致。

如果 append slot 失败，不能半更新 request length 或 block table，否则后续 attention 会读错位置。

坑六：streaming 顺序和调度顺序混乱。

调度可以乱序，但每个请求内部输出 token 必须按生成顺序返回。

19.16 面试官会怎么问

问题一：什么是 continuous batching?

回答要点：它是在每个模型迭代 step 重新组织 batch 的调度机制，允许请求动态加入和退出，适合 LLM 多轮自回归生成。

问题二：continuous batching 和普通 dynamic batching 有什么区别?

回答要点：dynamic batching 多数是在 forward 前收集请求；continuous batching 是每个 decode iteration 都调度，必须维护跨 iteration 的 KV Cache 和 request state。

问题三：为什么它能提升吞吐?

回答要点：它减少固定 batch 中 finished 请求造成的空洞，让新请求可以补入 running workload，提升 GPU 利用率；配合 PagedAttention 可以承载更多并发。

问题四：为什么需要 token budget?

回答要点：请求数不能反映实际计算量，长 prompt prefill 可能远大于短请求 decode，所以要按本轮 token 数控制调度规模。

问题五：为什么需要 KV block budget?

回答要点：能调度 token 不代表 KV Cache 放得下。prefill 需要分配 prompt blocks，decode 可能追加新 block，必须在执行前判断 free blocks。

问题六：如何平衡 TTFT 和 TPOT?

回答要点：prefill 影响新请求 TTFT，decode 影响已有请求 TPOT。常见策略是 decode-first 加受限 prefill，或通过 chunked prefill 控制长 prompt 对 decode 的阻塞。

19.17 标准回答模板

如果面试官问“vLLM 的 continuous batching 是什么”，可以这样回答：

Continuous batching 是面向自回归 LLM serving 的动态调度机制。

传统 batching 往往把一组请求固定在一起，等整批请求完成后再处理下一批。但 LLM 生成是多轮迭代过程，不同请求输出长度不同，

Continuous batching 把调度粒度降低到每个模型 iteration。每一轮 scheduler 都可以移除 finished 请求，接纳 waiting 请求，选

它的收益是提高吞吐和显存利用率，代价是 scheduler 更复杂，并且需要在 TTFT、TPOT、吞吐、公平性和 OOM 风险之间做 trade-off。

19.18 Continuous Batching 公式、调度 trace 和可运行 demo

把前面的约束合在一起，一个教学版 continuous batching gate 可以写成：

$$G_{cb} = G_{\mathrm{iter}} G_{\mathrm{tok}} G_{\mathrm{kv}} G_{\mathrm{decode}} G_{\mathrm{cleanup}} G_{\mathrm{metric}}$$

其中：

1. G_{iter} ：请求可以在不同 iteration 动态到达、进入、退出。
2. G_{tok} ：每轮 decode + prefill 不超过 token budget。
3. G_{kv} ：prefill 和 decode 追加 KV 时不超过 free block budget。
4. G_{decode} ：已有 running 请求能优先获得 decode 进度，避免 streaming 卡顿。
5. G_{cleanup} ：finished / cancelled 请求会释放 KV blocks。
6. G_{metric} ：trace 能复盘 waiting、running、deferred reason、TTFT 和 block 使用。

下面这个 0 依赖 demo 模拟 5 个请求：A、B 同时到达，C、D、E 后续到达；scheduler 采用 decode-first；prefill 受 token budget 约束；KV block 不足时请求继续等待；E 在等待中取消；最终输出 trace、TTFT、静态 batch 空洞和门禁。

```
from dataclasses import dataclass, field
from math import ceil
```

```
@dataclass
```

```
class Request:
    request_id: str
    arrival: int
    prompt_tokens: int
    planned_output: list
    cancel_at: int = None
    state: str = "new"
    prefill_start: int = None
    first_token_step: int = None
    finish_step: int = None
    finish_reason: str = None
    emitted: list = field(default_factory=list)
```

```
class ToyBlockManager:
    def __init__(self, total_blocks, block_size):
        self.total_blocks = total_blocks
        self.block_size = block_size
        self.free_blocks = list(range(total_blocks))
        self.tables = {}
        self.lengths = {}
        self.max_used = 0

    def required_blocks(self, tokens):
        return ceil(tokens / self.block_size)

    def used_count(self):
        return self.total_blocks - len(self.free_blocks)
```

```

def _remember_peak(self):
    self.max_used = max(self.max_used, self.used_count())

def can_allocate(self, tokens):
    return self.required_blocks(tokens) <= len(self.free_blocks)

def allocate(self, request_id, tokens):
    need = self.required_blocks(tokens)
    if need > len(self.free_blocks):
        return False
    self.tables[request_id] = [self.free_blocks.pop(0) for _ in range(need)]
    self.lengths[request_id] = tokens
    self._remember_peak()
    return True

def can_append(self, request_id):
    length = self.lengths[request_id]
    return length % self.block_size != 0 or bool(self.free_blocks)

def append_one(self, request_id):
    length = self.lengths[request_id]
    if length % self.block_size == 0:
        if not self.free_blocks:
            return False
        self.tables[request_id].append(self.free_blocks.pop(0))
    self.lengths[request_id] += 1
    self._remember_peak()
    return True

def release(self, request_id):
    released = self.tables.pop(request_id, [])
    self.lengths.pop(request_id, None)
    self.free_blocks.extend(released)
    return released

class ToyContinuousBatcher:
    def __init__(self, requests, max_tokens, max_running, block_manager):
        self.requests = sorted(requests, key=lambda req: (req.arrival, req.request_id))
        self.future = list(self.requests)
        self.max_tokens = max_tokens
        self.max_running = max_running
        self.block_manager = block_manager
        self.time = 0
        self.waiting = []
        self.running = []
        self.finished = []
        self.cancelled = []
        self.trace = []
        self.deferred = []

    def add_arrivals(self):
        arrived = []
        while self.future and self.future[0].arrival <= self.time:

```

```

    req = self.future.pop(0)
    req.state = "waiting"
    self.waiting.append(req)
    arrived.append(req.request_id)
    return arrived

def cancel_ready_requests(self):
    cancelled = []
    for req in list(self.waiting) + list(self.running):
        if req.cancel_at is not None and req.cancel_at <= self.time:
            if req in self.waiting:
                self.waiting.remove(req)
            if req in self.running:
                self.running.remove(req)
                self.block_manager.release(req.request_id)
            req.state = "cancelled"
            req.finish_reason = "client_cancelled"
            req.finish_step = self.time
            self.cancelled.append(req)
            cancelled.append(req.request_id)
    return cancelled

def finish(self, req, reason="eos"):
    if req in self.running:
        self.running.remove(req)
    req.state = "finished"
    req.finish_reason = reason
    req.finish_step = self.time
    self.block_manager.release(req.request_id)
    self.finished.append(req)

def schedule_step(self):
    arrived = self.add_arrivals()
    cancelled = self.cancel_ready_requests()
    scheduled_decode = []
    scheduled_prefill = []
    deferred = []
    used_tokens = 0

    for req in list(self.running):
        if used_tokens + 1 > self.max_tokens:
            deferred.append((req.request_id, "token_budget_decode"))
            continue
        if not self.block_manager.can_append(req.request_id):
            deferred.append((req.request_id, "kv_decode"))
            continue
        self.block_manager.append_one(req.request_id)
        scheduled_decode.append(req)
        used_tokens += 1

    while self.waiting:
        req = self.waiting[0]
        if len(self.running) + len(scheduled_prefill) >= self.max_running:
            deferred.append((req.request_id, "active_sequence_limit"))
            break

```

```

        if used_tokens + req.prompt_tokens > self.max_tokens:
            deferred.append((req.request_id, "token_budget_prefill"))
            break
        if not self.block_manager.can_allocate(req.prompt_tokens):
            deferred.append((req.request_id, "kv_prefill"))
            break
        self.waiting.pop(0)
        self.block_manager.allocate(req.request_id, req.prompt_tokens)
        scheduled_prefill.append(req)
        used_tokens += req.prompt_tokens

    for req in scheduled_decode:
        next_token = req.planned_output[len(req.emitted)]
        req.emitted.append(next_token)
        if len(req.emitted) == len(req.planned_output):
            self.finish(req)

    for req in scheduled_prefill:
        req.state = "running"
        req.prefill_start = self.time
        req.first_token_step = self.time
        req.emitted.append(req.planned_output[0])
        if len(req.emitted) == len(req.planned_output):
            self.finish(req)
        else:
            self.running.append(req)

    self.deferred.extend((self.time, req_id, reason) for req_id, reason in deferred)
    self.trace.append(
        {
            "step": self.time,
            "arrived": arrived,
            "cancelled": cancelled,
            "decode": [req.request_id for req in scheduled_decode],
            "prefill": [req.request_id for req in scheduled_prefill],
            "used_tokens": used_tokens,
            "waiting": [req.request_id for req in self.waiting],
            "running": [req.request_id for req in self.running],
            "free_blocks": len(self.block_manager.free_blocks),
            "deferred": deferred,
        }
    )
    self.time += 1

def run(self):
    while self.future or self.waiting or self.running:
        self.schedule_step()
        if self.time > 20:
            raise RuntimeError("scheduler did not converge")

def report(self):
    output_rows = sum(len(req.emitted) for req in self.requests)
    static_rows = len(self.requests) * max(len(req.planned_output) for req in self.requests)
    queue_wait = {
        req.request_id: req.prefill_start - req.arrival
    }

```

```

        for req in self.requests
        if req.prefill_start is not None
    }
    ttft = {
        req.request_id: req.first_token_step - req.arrival
        for req in self.requests
        if req.first_token_step is not None
    }
    reasons = sorted({reason for _, _, reason in self.deferred})
    terminal = all(req.finish_reason is not None for req in self.requests)
    gates = {
        "dynamic_arrivals_seen": any(item["arrived"] for item in self.trace[1:]),
        "decode_first_with_prefill": any(item["decode"] and item["prefill"] for item in self.trace),
        "token_budget_respected": all(item["used_tokens"] <= self.max_tokens for item in self.trace),
        "kv_budget_deferred": "kv_prefill" in reasons or "kv_decode" in reasons,
        "cleanup_released_blocks": terminal and self.block_manager.used_count() == 0,
        "metrics_trace_ready": bool(queue_wait) and bool(ttft) and self.block_manager.max_used > 0,
    }
    gates["continuous_batching_gate"] = all(gates.values())
    summary = {
        "finished_order": [req.request_id for req in sorted(self.finished, key=lambda r: (r.finish_step, r.request_id))],
        "cancelled": [req.request_id for req in self.cancelled],
        "queue_wait_steps": queue_wait,
        "ttft_steps": ttft,
        "static_decode_rows": static_rows,
        "continuous_output_rows": output_rows,
        "saved_rows": static_rows - output_rows,
        "max_kv_blocks_used": self.block_manager.max_used,
        "deferred_reasons": reasons,
        "trace_tail": self.trace[-4:],
    }
    return summary, gates

```

```

requests = [
    Request("A", arrival=0, prompt_tokens=3, planned_output=["a1", "a2", "a3"]),
    Request("B", arrival=0, prompt_tokens=6, planned_output=["b1", "b2", "b3", "b4"]),
    Request("C", arrival=1, prompt_tokens=2, planned_output=["c1"]),
    Request("D", arrival=2, prompt_tokens=5, planned_output=["d1", "d2"]),
    Request("E", arrival=3, prompt_tokens=3, planned_output=["e1", "e2"], cancel_at=5),
]

```

```

scheduler = ToyContinuousBatcher(
    requests=requests,
    max_tokens=8,
    max_running=4,
    block_manager=ToyBlockManager(total_blocks=5, block_size=4),
)
scheduler.run()
summary, gates = scheduler.report()

```

```

print("continuous_batching_summary=", summary)
print("continuous_batching_gates=", gates)

```

一份合理输出应当能看出这些事实：

1. `finished_order` 不是到达顺序，说明请求在不同 iteration 动态退出。
2. `deferred_reasons` 同时包含 `token_budget_prefill` 和 `kv_prefill`，说明调度不只看请求数。
3. `static_decode_rows` 大于 `continuous_output_rows`，能量化固定 batch 的空洞。
4. `cleanup_released_blocks=True`，说明完成和取消路径没有泄漏 KV blocks。
5. `continuous_batching_gate=True`，说明动态到达、decode-first、token budget、KV budget、cleanup 和 metrics 都能被 trace 复盘。

19.19 小练习

1. 画一个时间线，包含 4 个请求 A、B、C、D，其中 A 短输出，B 长输出，C 和 D 中途到达，分别画出 static batching 和 continuous batching 的执行方式。
2. 写一个简化 scheduler，输入 waiting queue、running requests、`max_num_batched_tokens` 和 free block 数，输出本轮 prefill 和 decode 请求。
3. 解释为什么只设置 `max_running_requests=32` 仍然可能 OOM。
4. 设计一个实验，观察 `max_num_batched_tokens` 增大后 TTFT、TPOT 和 tokens/s 的变化。
5. 思考长 prompt prefill 为什么会让 streaming decode 卡顿，并提出一个缓解方案。

19.20 本章总结

Continuous batching 是 vLLM-like serving engine 的核心能力之一。

它解决的问题是：自回归 LLM 请求长度不同、到达时间不同、结束时间不同，如果 batch 固定不变，会浪费 GPU、拉高 TTFT，并降低系统吞吐。

它的关键思想是：把调度粒度从 request-level 降到 iteration-level，每一轮模型执行前都重新选择要 prefill 和 decode 的请求。

真正可用的 continuous batching 不能只看请求数，还必须结合 token budget 和 KV block budget。token budget 控制本轮计算量，KV block budget 控制显存安全。PagedAttention、Block Manager 和 Continuous Batching 共同构成 vLLM 高吞吐 serving 的主干。

下一章会继续进入 vLLM 请求调度流程，具体拆解一个请求从进入 vLLM engine 到完成输出，中间会经历哪些状态迁移、调度决策和执行步骤。

第 20 章 vLLM 请求调度流程

上一章讲了 continuous batching 与 iteration-level scheduling：LLM serving engine 不应该把一组请求固定成 batch 跑到结束，而应该在每个 engine step 重新决定哪些请求 prefill、哪些请求 decode、哪些请求退出。

本章继续往下走：一个请求进入 vLLM-like engine 后，到底经历哪些模块、哪些状态、哪些调度决策，最后如何把 token 流式返回给用户？

一句话概括：

vLLM 的请求调度流程可以理解为：入口层接收和预处理请求，engine core 维护请求状态并执行 scheduler，KV cache manager 分配和释放 blocks，worker/model runner 执行模型 forward，output processor 把模型输出转换为用户可见的 token 或文本。

20.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考四类公开资料：

1. vLLM Architecture Overview 对 API server process、engine core process、GPU worker process、LLMEngine、AsyncLLMEngine、worker 和 model runner 的模块边界说明。
2. vLLM V1 guide 对 V1 core engine、chunked prefill、统一 token budget 调度和功能支持边界的说明。
3. vLLM optimization / tuning 文档对 preemption、KV cache capacity、`max_num_seqs`、`max_num_batched_tokens`、chunked prefill 和 decode-first 策略的说明。

4. vLLM metrics 文档对 TTFT、inter-token latency / TPOT、queue time、prefill time、decode time、running / waiting 请求数、KV cache usage 和 engine core iteration 统计的观测口径。

本章只讲 vLLM-like 请求调度流程的教学抽象，不绑定某个 vLLM 版本的真实类名、进程拓扑、ZMQ 通信细节、CUDA graph、scheduler 源码、model runner 内部张量格式、OpenAI server 参数全集、LoRA、多模态、speculative decoding、structured output 或分布式 data parallel / tensor parallel 实现。

本章给出的公式和 demo 只用于验证一条最小请求链路：API 输入处理、engine request、waiting queue、scheduler output、KV block 分配、model runner metadata、sampler / output processor、streaming / finish reason、abort cleanup 和 metrics trace。真实系统会把这些逻辑拆到更多进程、线程、队列和异步事件里。

参考资料：

1. vLLM Architecture Overview: https://docs.vllm.ai/en/latest/design/arch_overview/
2. vLLM V1 guide: https://docs.vllm.ai/en/latest/usage/v1_guide/
3. vLLM optimization and tuning: <https://docs.vllm.ai/en/latest/configuration/optimization/>
4. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics/>

20.1 本章目标

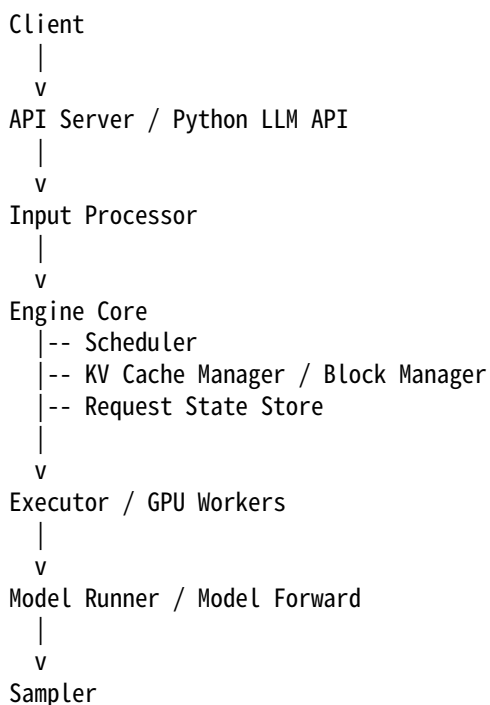
读完本章，你应该能讲清：

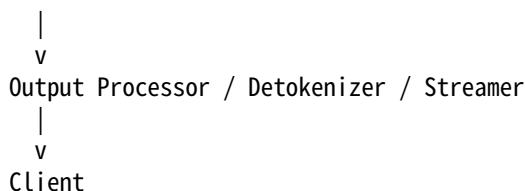
1. vLLM-like 系统中 API server、LLMEngine、engine core、scheduler、worker、model runner、output processor 各自负责什么。
2. 一个请求从 HTTP 请求到 first token 再到 finished 的完整路径。
3. waiting、running、scheduled、finished、aborted 等状态如何变化。
4. scheduler 每一轮如何结合 token budget、KV block budget 和请求状态做选择。
5. 为什么输出处理和请求终止也会影响调度。
6. 面试中如何画出 vLLM 请求生命周期图。

20.2 先建立模块图

不同 vLLM 版本的内部类名会变化，但核心模块边界比较稳定。

可以先用这张简化图理解：





每个模块解决一个问题。

API server 负责接入请求、校验参数、组织 OpenAI-compatible API、处理 streaming 连接。

Input processor 负责把原始 prompt、chat messages、多模态输入和 sampling 参数整理成 engine 能理解的请求对象。

Engine core 负责真正的调度主循环：接收请求、维护状态、调用 scheduler、管理 KV cache、派发模型执行。

Scheduler 负责每一轮选择要执行的请求。

KV cache manager 负责 blocks 的分配、追加、释放和可能的 prefix 复用。

Worker 和 model runner 负责在 GPU 上执行模型 forward。

Output processor 负责把 token id、logprob、finish reason、stop string 等转换成最终输出，并处理请求结束和中止。

20.3 请求进入系统：从原始输入到 Engine Request

用户发来的请求可能长这样：

```
{
  "model": "some-model",
  "messages": [
    {"role": "user", "content": "Explain KV cache in simple words."}
  ],
  "temperature": 0.7,
  "top_p": 0.9,
  "max_tokens": 256,
  "stream": true
}
```

在进入 scheduler 之前，它通常要经历几步处理。

第一步，API 层校验请求。

检查内容包括：

1. model 是否存在。
2. max_tokens、temperature、top-p 等参数是否合法。
3. prompt 或 messages 是否为空。
4. 输入长度是否超过限制。
5. 是否启用 streaming。
6. 是否包含不支持的多模态字段或工具字段。

第二步，渲染 prompt。

Chat messages 需要根据 chat template 转成模型实际看到的文本或 token 序列。

例如：

```
<|user|>
Explain KV cache in simple words.
<|assistant|>
```

第三步, tokenization。

把文本转成 token ids:

prompt_text -> input_ids

第四步, 构造 engine request。

一个最小 request state 可以包含:

```
class RequestState:
    def __init__(self, request_id, input_ids, sampling_params, arrival_time):
        self.request_id = request_id
        self.input_ids = input_ids
        self.sampling_params = sampling_params
        self.arrival_time = arrival_time

        self.status = "WAITING"
        self.num_computed_tokens = 0
        self.output_token_ids = []
        self.block_table = []
        self.first_token_time = None
        self.finished_time = None
        self.finish_reason = None
```

真实 vLLM 的数据结构更复杂, 但主干字段离不开这些: 输入 tokens、采样参数、状态、KV block 映射、输出 tokens 和时间统计。

抽象成公式, 一个 engine request 可以写成:

$$R_i = (u_i, a_i, x_i, \theta_i, z_i, P_i, Y_i, b_i, t_i, m_i)$$

其中 u_i 是 request id, a_i 是到达时间, x_i 是输入 token 序列, θ_i 是采样参数, z_i 是状态, P_i 是 block table, Y_i 是已经输出的 tokens, b_i 是 streaming buffer, t_i 是时间戳集合, m_i 是 metrics / trace 字段。

请求对象完整率可以写成:

$$C_{\text{req}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[u_i, a_i, x_i, \theta_i, z_i, P_i, Y_i, t_i, m_i \text{ ready}]$$

如果缺 arrival time、状态或 block table, 后面的 scheduler、metrics 和 cleanup 都会失真。

20.4 请求进入 waiting queue

构造好 engine request 后, 请求不会马上执行模型。

它会先进入 engine core 的 waiting queue:

new request -> WAITING -> waiting queue

waiting queue 的作用是把 “请求到达” 和 “GPU 执行” 解耦。

请求到达是外部事件, 可能非常不均匀。

GPU 执行是 engine step 驱动的, 需要按 token budget、KV block budget、优先级和并发限制来选择。

如果没有 waiting queue, 请求一来就尝试执行, 系统很容易出现:

1. 多个请求争抢 GPU。
2. KV Cache 分配没有统一入口。

3. prefill 插入时机混乱。
4. streaming decode 被长 prompt 打断。

所以 waiting queue 是 scheduler 的入口缓冲区。

20.5 Engine Core 的主循环

vLLM 架构文档中, engine core 是运行 scheduler、管理 KV cache、协调 GPU workers 的核心进程或核心组件。

可以把 engine core 理解成一个不断执行的循环:

```
while True:
    new_requests = recv_new_requests()
    add_to_waiting_queue(new_requests)

    scheduler_output = scheduler.schedule(
        waiting_queue=waiting_queue,
        running_requests=running_requests,
        block_manager=block_manager,
    )

    model_outputs = executor.execute_model(scheduler_output)
    engine_outputs = process_model_outputs(model_outputs)
    send_outputs_to_frontend(engine_outputs)
```

这个循环看起来简单, 但每一步都很关键。

recv_new_requests() 处理新请求和 abort 请求。

scheduler.schedule() 决定本轮谁跑。

executor.execute_model() 把本轮 workload 发给 GPU workers。

process_model_outputs() 处理采样结果、结束条件和状态更新。

send_outputs_to_frontend() 把输出交给 API server 或 output processor。

20.6 Scheduler 的输入

Scheduler 每轮需要的信息很多。

最小输入包括:

1. waiting requests: 还没有完成 prefill 的请求。
2. running requests: 已经进入系统、需要继续 decode 的请求。
3. KV block 状态: free blocks、已分配 blocks、block table、ref count。
4. token budget: 本轮最多处理多少 token。
5. sequence budget: 本轮最多处理多少序列。
6. priority 或 arrival time: 用于公平性和优先级。
7. 当前 engine 状态: 是否有 pending abort、是否处于 sleep/profile 等特殊状态。

用伪代码表示:

```
class SchedulerInput:
    def __init__(self, waiting, running, block_manager, config):
        self.waiting = waiting
        self.running = running
        self.block_manager = block_manager
        self.max_num_batched_tokens = config.max_num_batched_tokens
        self.max_num_seqs = config.max_num_seqs
```

这里的 `max_num_batched_tokens` 控制计算规模, `max_num_seqs` 控制本轮 sequence 数, block manager 控制 KV Cache 安全。

本轮 scheduler 的基础预算可以写成:

$$G_{\text{sched},\tau} = \mathbf{1}[N_{\text{tok},\tau} \leq C_{\text{tok}}] \mathbf{1}[N_{\text{seq},\tau} \leq C_{\text{seq}}] \mathbf{1}[N_{\text{block},\tau} \leq F_{\tau}]$$

这里 $N_{\text{tok},\tau}$ 是本轮 prefill + decode token 数, $N_{\text{seq},\tau}$ 是本轮执行序列数, $N_{\text{block},\tau}$ 是本轮新增 KV blocks, F_{τ} 是调度前可用 free blocks。

20.7 Scheduler 的输出

Scheduler 输出的不是普通文本, 而是 “本轮执行计划”。

一个简化输出可以包含:

```
class SchedulerOutput:
    def __init__(self):
        self.prefill_requests = []
        self.decode_requests = []
        self.blocks_to_allocate = {}
        self.blocks_to_append = {}
        self.requests_to_free = []
```

实际系统中还会包含更多信息, 例如:

1. 每个请求本轮要处理多少 tokens。
2. attention metadata。
3. block tables。
4. sampling metadata。
5. LoRA 或 adapter 信息。
6. 多模态 encoder cache 信息。
7. speculative decoding 相关字段。

但从面试角度, 最重要的是说明: scheduler 的输出要同时喂给模型执行和 KV Cache 管理。

如果只输出 “请求列表”, model runner 不知道每个 token 对应哪些 block, 也不知道哪些请求是 prefill、哪些是 decode。

因此 scheduler output 至少要满足 execution metadata 完整性。设本轮被调度请求集合为 S_{τ} , 每个请求要有 input ids、positions、block table、slot mapping 和 phase:

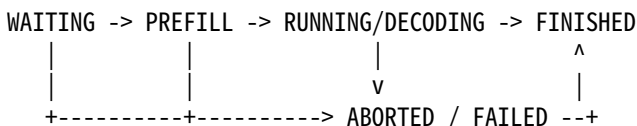
$$G_{\text{meta},\tau} = \prod_{i \in S_{\tau}} G_{\text{ids},i} G_{\text{pos},i} G_{\text{block},i} G_{\text{slot},i} G_{\text{phase},i}$$

如果 $G_{\text{meta},\tau}=0$, 模型 runner 即使拿到请求列表, 也无法安全执行 attention 和 KV 写入。

20.8 请求状态机

为了理解调度流程, 必须画出请求状态机。

简化状态:



更细一点：

WAITING：请求已经进入 engine，但还没有被 scheduler 选中 prefill。

PREFILL：本轮正在处理 prompt tokens，并写入 prompt KV Cache。

RUNNING 或 DECODING：prompt 已处理，后续每轮生成一个或多个 token。

FINISHED：遇到 EOS、stop string、达到 max tokens 或其他正常结束条件。

ABORTED：客户端断开、用户取消、上层超时或 output processor 判定需要中止。

FAILED：模型执行、KV 分配或内部逻辑出现错误。

关键原则是：

任何离开 WAITING/RUNNING 的终止状态，都必须释放资源。

资源包括 KV blocks、streaming buffer、请求状态、统计对象和可能的多模态缓存引用。

对于请求 i ，如果终止状态集合记为 $Z_{\{\mathrm{term}\}} = \{\mathrm{FINISHED}, \mathrm{ABORTED}, \mathrm{FAILED}\}$ ，则 cleanup 门禁可以写成：

$$G_{\text{cleanup},i} = \mathbf{1}[z_i \in Z_{\text{term}} \Rightarrow |P_i| = 0]$$

这句话的含义是：只要请求已经进入终态，它的 KV block table 就不应该再持有可用资源。

20.9 从 WAITING 到 PREFILL

一个 waiting 请求能不能进入 prefill，至少要过几道门。

第一道门：token budget。

如果 prompt 太长，本轮剩余 token budget 不够，就不能完整 prefill。

如果支持 chunked prefill，则可以只处理 prompt 的一部分。

第二道门：KV block budget。

block manager 要判断 prompt tokens 需要多少 blocks。

`required_blocks = ceil(prompt_len / block_size)`

如果 free blocks 不够，请求继续留在 waiting queue。

第三道门：sequence budget。

如果本轮 scheduled sequences 已经达到上限，新请求不能加入。

第四道门：策略门。

如果当前系统 decode 压力很大，scheduler 可能暂时不接纳新的长 prompt，避免 TPOT 抖动。

通过这些检查后，scheduler 会把请求放入 prefill set，并让 block manager 分配 blocks。

```
if can_fit_tokens(req) and can_allocate_blocks(req) and can_fit_seq(req):
    block_manager.allocate(req)
    schedule_prefill(req)
```

20.10 从 PREFILL 到 RUNNING

Prefill 执行完后，模型已经为 prompt 中的 token 写好了 KV Cache。

如果这是普通生成请求，prefill 之后通常会产生第一个可采样位置的 logits，然后 sampler 采样出 first token。

这时会发生几件事：

1. 记录 first token time, 用于计算 TTFT。
2. 把 first token 加入 output token ids。
3. 检查 first token 是否触发 EOS 或 stop 条件。
4. 如果未结束, 请求进入 running set。
5. 如果已结束, 释放 KV blocks 并输出 final response。

状态变化:

WAITING -> PREFILL -> RUNNING

或者极短生成时:

WAITING -> PREFILL -> FINISHED

一个请求可能在 first token 就结束。例如 max_tokens=1, 或者采样出的 token 正好是 EOS。

20.11 RUNNING 阶段: 每轮 decode

进入 running set 后, 请求会在后续 engine steps 中被反复调度。

每一轮 decode 通常做:

1. 判断本轮是否选择该请求。
2. 检查当前 KV block 是否还有 slot。
3. 如果 block 满了, 追加新 physical block。
4. 准备 decode input token, 一般是上一步生成的 token。
5. 构造 attention metadata 和 block table。
6. 执行模型 forward。
7. sampler 采样下一个 token。
8. 更新 request state。
9. streaming 输出增量文本。
10. 检查结束条件。

伪代码:

```
for req in scheduled_decode:
    if block_manager.need_new_block(req):
        block_manager.append_block(req)

model_outputs = model_runner.forward(scheduled_decode)
next_tokens = sampler.sample(model_outputs.logits)

for req, token in zip(scheduled_decode, next_tokens):
    req.output_token_ids.append(token)
    req.num_computed_tokens += 1
    if should_stop(req, token):
        req.status = "FINISHED"
```

这里要注意: decode 阶段不是重新处理整个上下文, 而是依赖 KV Cache 读取历史 K/V, 只处理新 token。

20.12 Model Runner 看到的不是请求, 而是执行 batch

在上层看, 系统里有 request。

但在 model runner 看, 最终要执行的是一组张量和 metadata。

例如:

```
input_ids
positions
slot_mapping
block_tables
```

attention_metadata
sampling_metadata

这说明一件事：scheduler 选中请求后，还要把 request-level state 转换成 model-level batch。

这个转换非常关键。

请求状态里记录的是：

1. 这个请求已经生成多少 token。
2. 它的 block table 是什么。
3. 这轮要 prefill 还是 decode。
4. 采样参数是什么。

模型执行需要的是：

1. 本轮实际输入 token ids。
2. 每个 token 的 position。
3. 每个 token 的 KV 写入位置。
4. attention kernel 如何根据 block table 读取历史 KV。
5. 哪些 logits 需要采样。

这就是为什么 vLLM-like 系统里 scheduler 和 model runner 之间会有一层 execution metadata。

20.13 Output Processor 为什么也影响调度

很多人只关注 scheduler，却忽略 output processor。

但 output processor 会影响请求什么时候结束、什么时候 abort、什么时候释放资源。

它通常负责：

1. 把 token ids detokenize 成文本。
2. 处理 streaming 增量输出。
3. 检查 stop strings。
4. 检查 EOS。
5. 检查 max tokens。
6. 组织 logprobs。
7. 生成 finish reason。
8. 通知 engine core abort 某些请求。

例如 stop string 检查常常需要 detokenized text，而不是单个 token id。

如果 output processor 发现某个请求已经匹配 stop string，它需要告诉 engine core：这个请求后续不要再调度了。

流程类似：

model output -> output processor -> stop matched -> abort/finish request -> engine core cleanup

所以请求结束不是 scheduler 单方面决定的，它也依赖输出处理逻辑。

20.14 Streaming 输出路径

在线 serving 中，用户通常不想等完整回答生成完，而是希望 token-by-token 或 chunk-by-chunk 返回。

Streaming 路径可以理解为：

model logits -> sampled token id -> detokenize delta -> output queue -> HTTP SSE chunk -> client

每个请求需要维护 streaming offset，避免重复返回文本。

例如 tokenizer 可能存在这种情况：几个 token 合起来才形成一个完整中文字符、英文单词片段或特殊符号。detokenizer 需要保证输出给客户端的是合法的增量文本。

Streaming 对调度有两个影响。

第一，TPOT 变成用户可感知指标。

如果 running 请求没有稳定 decode，用户会看到输出卡顿。

第二，客户端断开会触发 abort。

如果用户关闭连接，API server 应通知 engine core 取消请求，释放 KV blocks。

否则 GPU 还在为一个没人接收的请求生成 token。

从 metrics 角度，request flow 至少要能还原三类时间：

$$W_i = t_i^{\text{prefill}} - a_i$$

$$T_{\text{tft},i} = t_i^{\text{first}} - a_i$$

$$T_{\text{e2e},i} = t_i^{\text{done}} - a_i$$

其中 W_i 是 queue wait, $T_{\text{tft},i}$ 是首 token 可见时间, $T_{\text{e2e},i}$ 是请求结束时间。没有这些时间戳，scheduler 调优就只能凭感觉。

20.15 Abort 和 cleanup 流程

Abort 是线上系统非常重要但容易被忽视的路径。

Abort 可能来自：

1. 用户主动取消。
2. HTTP 连接断开。
3. 上层 gateway 超时。
4. output processor 匹配 stop string 后要求终止。
5. 管理接口要求取消某个 request id。
6. 系统内部错误。

Abort 后必须做几件事：

1. 从 waiting queue 移除请求。
2. 从 running set 移除请求。
3. 释放 KV blocks。
4. 清理 output buffer。
5. 清理统计和 tracing 状态。
6. 通知前端请求已结束或已取消。

伪代码：

```
def abort_request(request_id):
    req = request_store.get(request_id)
    if req is None:
        return

    req.status = "ABORTED"
    waiting_queue.remove(request_id)
    running_set.remove(request_id)
    block_manager.free(req)
    output_processor.abort(request_id)
    request_store.delete(request_id)
```


工程上最怕的是“只改状态，不释放资源”。

这会造成 silent KV leak：请求看起来结束了，但 block 还被占着。

20.16 一个完整请求生命周期例子

假设请求 R 到达：

```
prompt_len = 1000
max_tokens = 4
block_size = 16
```

它的生命周期可能是：

```
t0: API server 收到请求 R
t1: input processor 渲染 chat template, tokenize 得到 1000 tokens
t2: engine core 把 R 放入 waiting queue
t3: scheduler 发现 token budget 和 KV blocks 都足够
t4: block manager 为 R 分配 63 个 blocks
t5: model runner 执行 prefill
t6: sampler 采样 first token, R 进入 running set, stream 第一个 token
t7: 下一轮 scheduler 选择 R decode, 追加或复用当前 block slot, stream 第二个 token
t8: 再下一轮 decode, stream 第三个 token
t9: 再下一轮 decode, 达到 max_tokens=4, R 标记 FINISHED
t10: block manager 释放 R 的 blocks
t11: output processor 返回 final finish reason
```

如果中途用户断开连接，则可能变成：

```
t7: client disconnected
t8: API server 发送 abort
t9: engine core 标记 R 为 ABORTED
t10: block manager 释放 R 的 blocks
```

这两个路径都必须正确。

20.17 和第 19 章的关系

第 19 章讲的是调度机制：每个 iteration 动态组织 batch。

本章讲的是请求路径：请求如何进入、如何被调度、如何执行、如何输出、如何结束。

两者可以合成一句话：

Continuous batching 是调度机制，请求生命周期管理是让这个机制能在线上正确运行的状态机。

只有调度机制，没有请求状态机，系统会不知道请求何时进入、何时结束、何时释放资源。

只有状态机，没有 continuous batching，系统又会退化成低效的固定 batch 或单请求生成。

20.18 常见工程坑

坑一：输入处理和 engine request 不一致。

例如 chat template 在 API 层和离线测试层不一致，导致同一个 prompt 在不同入口下 token ids 不同。

坑二：arrival time 没有传入 scheduler。

缺少 arrival time 后，很难计算 queue time 和 TTFT，也难以做公平调度。

坑三：output processor 判定 stop 后，没有通知 engine core。

表现是用户侧已经看到结束，但 engine 仍继续 decode。

坑四：abort 只清理 output queue，不清理 KV blocks。

表现是显存慢慢下降，最终 OOM。

坑五：model runner metadata 和 request state 不一致。

例如 block table 更新了，但 slot mapping 没同步，attention kernel 可能读写错误位置。

坑六：streaming delta 处理不当。

表现是返回重复文本、乱码、截断特殊 token，或者 stop string 被提前/延后触发。

20.19 面试官会怎么问

问题一：一个请求进入 vLLM 后经历哪些模块？

回答要点：API server 接收请求，input processor 渲染和 tokenization，engine core 维护请求状态，scheduler 选择 prefill/decode，KV cache manager 分配 blocks，worker/model runner 执行 forward，sampler 采样 token，output processor detokenize、streaming 和判断结束。

问题二：engine core 的职责是什么？

回答要点：engine core 是调度和执行协调中心，负责接收 engine requests、运行 scheduler、管理 KV cache、协调 GPU workers、处理 abort 和输出状态。

问题三：scheduler 输出什么？

回答要点：不是只输出请求列表，而是输出本轮执行计划，包括 prefill/decode 请求、token 数、block table、slot mapping、sampling metadata 和需要分配/释放的资源信息。

问题四：请求什么时候释放 KV Cache？

回答要点：正常结束、abort、failed、超时、客户端断连等终止路径都要释放。不能只在 EOS 时释放。

问题五：为什么 output processor 会影响调度？

回答要点：stop string、EOS、max tokens、streaming 状态和 abort 可能在 output processor 中判断，它会决定请求是否还需要继续进入下一轮 scheduler。

20.20 标准回答模板

如果面试官问“请讲一下 vLLM 的请求调度流程”，可以这样回答：

一个请求进入 vLLM-like 系统后，首先由 API server 或 Python LLM 接口接收。输入会经过 input processor，完成参数校验、chat

这个 request 会进入 engine core 的 waiting queue。Engine core 持续运行调度循环，每一轮调用 scheduler，根据 waiting request

被选中的请求会由 KV cache manager 分配或追加 blocks，然后 scheduler 输出会被转换成 model runner 需要的 execution metadata

模型输出再经过 output processor，完成 detokenization、streaming、EOS 和 stop string 检查、finish reason 生成。如果请求结

20.21 请求调度流程公式、状态 trace 和可运行 demo

把本章的请求链路合成一个教学版总门禁，可以写成：

$$G_{\text{flow}} = G_{\text{input}} G_{\text{state}} G_{\text{sched}} G_{\text{meta}} G_{\text{output}} G_{\text{cleanup}} G_{\text{metric}}$$

其中：

1. G_{input} : API / input processor 产出 engine request 和 input ids。
2. G_{state} : 请求状态按 WAITING、PREFILL、RUNNING、DECODING、FINISHED / ABORTED 迁移。

3. G_{sched} : scheduler 每轮不超 token、sequence 和 KV block budget。
4. G_{meta} : model runner metadata 含 input ids、positions、block table、slot mapping 和 phase。
5. G_{output} : output processor 能处理 EOS、stop token / string、max tokens 和 streaming 增量。
6. G_{cleanup} : FINISHED / ABORTED / FAILED 都释放 KV blocks。
7. G_{metric} : queue wait、TTFT、E2E、trace 和 finish reason 可复盘。

下面这个 0 依赖 demo 模拟一个最小 vLLM-like 请求调度流程：A 正常 EOS，B 被 output processor 的 stop token 结束，C 在 first token 后因 max_tokens=1 结束，D 进入 running 后被客户端 abort。demo 同时输出 scheduler metadata 和 cleanup 结果。

```
from dataclasses import dataclass, field
from math import ceil
```

@dataclass

```
class RequestSpec:
    request_id: str
    arrival: int
    prompt_tokens: int
    planned_tokens: list
    max_tokens: int
    stop_token: str = None
    abort_at: int = None
```

@dataclass

```
class RequestState:
    spec: RequestSpec
    status: str = "NEW"
    input_ids: list = field(default_factory=list)
    output_tokens: list = field(default_factory=list)
    block_table: list = field(default_factory=list)
    arrival_step: int = None
    prefill_step: int = None
    first_token_step: int = None
    finish_step: int = None
    finish_reason: str = None
    transitions: list = field(default_factory=list)

    def transition(self, status, step):
        self.status = status
        self.transitions.append(f"{status}@{step}")
```

```
class ToyKVBlockManager:
    def __init__(self, total_blocks, block_size):
        self.total_blocks = total_blocks
        self.block_size = block_size
        self.free_blocks = list(range(total_blocks))
        self.lengths = {}
        self.max_used = 0

    def used_count(self):
        return self.total_blocks - len(self.free_blocks)
```

```

def remember_peak(self):
    self.max_used = max(self.max_used, self.used_count())

def required_blocks(self, tokens):
    return ceil(tokens / self.block_size)

def can_allocate(self, tokens):
    return self.required_blocks(tokens) <= len(self.free_blocks)

def allocate_prompt(self, state):
    need = self.required_blocks(len(state.input_ids))
    if need > len(self.free_blocks):
        return False
    state.block_table = [self.free_blocks.pop(0) for _ in range(need)]
    self.lengths[state.spec.request_id] = len(state.input_ids)
    self.remember_peak()
    return True

def can_append(self, state):
    length = self.lengths[state.spec.request_id]
    return length % self.block_size != 0 or bool(self.free_blocks)

def append_slot(self, state):
    request_id = state.spec.request_id
    length = self.lengths[request_id]
    if length % self.block_size == 0:
        if not self.free_blocks:
            return False
        state.block_table.append(self.free_blocks.pop(0))
    self.lengths[request_id] += 1
    self.remember_peak()
    return True

def release(self, state):
    self.free_blocks.extend(state.block_table)
    state.block_table = []
    self.lengths.pop(state.spec.request_id, None)

def slot_mapping(self, state, positions):
    mapping = []
    for position in positions:
        logical_block = position // self.block_size
        offset = position % self.block_size
        physical = state.block_table[logical_block] if logical_block < len(state.block_table) else None
        mapping.append({"position": position, "block": physical, "offset": offset})
    return mapping

class ToyRequestFlowEngine:
    def __init__(self, specs, max_tokens, max_seqs, block_manager):
        self.future_specs = sorted(specs, key=lambda spec: (spec.arrival, spec.request_id))
        self.states = {}
        self.max_tokens = max_tokens
        self.max_seqs = max_seqs

```

```

self.block_manager = block_manager
self.time = 0
self.waiting = []
self.running = []
self.finished = []
self.aborted = []
self.trace = []
self.metadata_rows = []

def input_processor(self, spec):
    state = RequestState(spec=spec)
    state.arrival_step = self.time
    state.input_ids = [1000 + idx for idx in range(spec.prompt_tokens)]
    state.transition("WAITING", self.time)
    return state

def receive_arrivals(self):
    arrived = []
    while self.future_specs and self.future_specs[0].arrival <= self.time:
        spec = self.future_specs.pop(0)
        state = self.input_processor(spec)
        self.states[spec.request_id] = state
        self.waiting.append(state)
        arrived.append(spec.request_id)
    return arrived

def abort_ready(self):
    aborted = []
    for state in list(self.waiting) + list(self.running):
        spec = state.spec
        if spec.abort_at is not None and spec.abort_at <= self.time:
            if state in self.waiting:
                self.waiting.remove(state)
            if state in self.running:
                self.running.remove(state)
            self.block_manager.release(state)
            state.finish_reason = "client_abort"
            state.finish_step = self.time
            state.transition("ABORTED", self.time)
            self.aborted.append(state)
            aborted.append(spec.request_id)
    return aborted

def build_metadata(self, state, phase):
    if phase == "prefill":
        input_ids = list(state.input_ids)
        positions = list(range(len(input_ids)))
    else:
        input_ids = [state.output_tokens[-1]]
        positions = [len(state.input_ids) + len(state.output_tokens) - 1]
    row = {
        "request_id": state.spec.request_id,
        "phase": phase,
        "input_ids": input_ids,
        "positions": positions,
    }

```

```

        "block_table": list(state.block_table),
        "slot_mapping": self.block_manager.slot_mapping(state, positions),
    }
    self.metadata_rows.append(row)
    return row

def next_model_token(self, state):
    index = len(state.output_tokens)
    return state.spec.planned_tokens[index]

def finish(self, state, reason):
    if state in self.running:
        self.running.remove(state)
    state.finish_reason = reason
    state.finish_step = self.time
    state.transition("FINISHED", self.time)
    self.block_manager.release(state)
    self.finished.append(state)

def output_processor(self, state, token, phase):
    state.output_tokens.append(token)
    if state.first_token_step is None:
        state.first_token_step = self.time
    if token == "<eos>":
        self.finish(state, "eos")
    elif state.spec.stop_token is not None and token == state.spec.stop_token:
        self.finish(state, "stop")
    elif len(state.output_tokens) >= state.spec.max_tokens:
        self.finish(state, "max_tokens")
    elif len(state.output_tokens) >= len(state.spec.planned_tokens):
        self.finish(state, "eos")
    elif phase == "prefill":
        state.transition("RUNNING", self.time)
        self.running.append(state)

def schedule_step(self):
    arrived = self.receive_arrivals()
    aborted = self.abort_ready()
    used_tokens = 0
    scheduled_decode = []
    scheduled_prefill = []
    metadata = []
    deferred = []

    for state in list(self.running):
        if used_tokens + 1 > self.max_tokens:
            deferred.append((state.spec.request_id, "token_budget_decode"))
            continue
        if not self.block_manager.can_append(state):
            deferred.append((state.spec.request_id, "kv_decode"))
            continue
        self.block_manager.append_slot(state)
        state.transition("DECODING", self.time)
        scheduled_decode.append(state)
        metadata.append(self.build_metadata(state, "decode"))

```

```

        used_tokens += 1

    while self.waiting:
        state = self.waiting[0]
        if len(scheduled_decode) + len(scheduled_prefill) >= self.max_seqs:
            deferred.append((state.spec.request_id, "sequence_budget"))
            break
        if used_tokens + len(state.input_ids) > self.max_tokens:
            deferred.append((state.spec.request_id, "token_budget_prefill"))
            break
        if not self.block_manager.can_allocate(len(state.input_ids)):
            deferred.append((state.spec.request_id, "kv_prefill"))
            break
        self.waiting.pop(0)
        self.block_manager.allocate_prompt(state)
        state.prefill_step = self.time
        state.transition("PREFILL", self.time)
        scheduled_prefill.append(state)
        metadata.append(self.build_metadata(state, "prefill"))
        used_tokens += len(state.input_ids)

    for state in scheduled_decode:
        self.output_processor(state, self.next_model_token(state), "decode")

    for state in scheduled_prefill:
        self.output_processor(state, self.next_model_token(state), "prefill")

    self.trace.append(
        {
            "step": self.time,
            "arrived": arrived,
            "aborted": aborted,
            "decode": [state.spec.request_id for state in scheduled_decode],
            "prefill": [state.spec.request_id for state in scheduled_prefill],
            "metadata_rows": len(metadata),
            "used_tokens": used_tokens,
            "waiting": [state.spec.request_id for state in self.waiting],
            "running": [state.spec.request_id for state in self.running],
            "free_blocks": len(self.block_manager.free_blocks),
            "deferred": deferred,
        }
    )
    self.time += 1

def run(self):
    while self.future_specs or self.waiting or self.running:
        self.schedule_step()
        if self.time > 20:
            raise RuntimeError("request flow did not converge")

def report(self):
    ordered_states = [self.states[key] for key in sorted(self.states)]
    queue_wait = {
        state.spec.request_id: state.prefill_step - state.arrival_step
        for state in ordered_states
    }

```

```

        if state.prefill_step is not None
    }
    ttft = {
        state.spec.request_id: state.first_token_step - state.arrival_step
        for state in ordered_states
        if state.first_token_step is not None
    }
    e2e = {
        state.spec.request_id: state.finish_step - state.arrival_step
        for state in ordered_states
        if state.finish_step is not None
    }
    metadata_complete = all(
        row["input_ids"]
        and row["positions"]
        and row["block_table"]
        and all(slot["block"] is not None for slot in row["slot_mapping"])
        and row["phase"] in {"prefill", "decode"}
        for row in self.metadata_rows
    )
    finish_reasons = {state.spec.request_id: state.finish_reason for state in ordered_states}
    gates = {
        "input_processed": all(state.input_ids and state.arrival_step is not None for state in ordered_states),
        "state_machine_terminal": all(state.finish_reason is not None for state in ordered_states),
        "scheduler_budget_respected": all(
            item["used_tokens"] <= self.max_tokens and item["metadata_rows"] <= self.max_seqs
            for item in self.trace
        ),
        "execution_metadata_complete": metadata_complete,
        "output_processor_finished": finish_reasons == {
            "A": "eos",
            "B": "stop",
            "C": "max_tokens",
            "D": "client_abort",
        },
        "abort_cleanup": finish_reasons["D"] == "client_abort" and not self.states["D"].block_table,
        "metrics_ready": bool(queue_wait) and bool(ttft) and bool(e2e),
    }
    gates["request_flow_gate"] = all(gates.values())
    summary = {
        "finished_order": [state.spec.request_id for state in sorted(self.finished, key=lambda s: (s.finish_step,
        "aborted": [state.spec.request_id for state in self.aborted],
        "finish_reasons": finish_reasons,
        "queue_wait_steps": queue_wait,
        "ttft_steps": ttft,
        "e2e_steps": e2e,
        "metadata_rows": len(self.metadata_rows),
        "max_kv_blocks_used": self.block_manager.max_used,
        "kv_blocks_after_cleanup": self.block_manager.used_count(),
        "transitions": {state.spec.request_id: state.transitions for state in ordered_states},
        "trace_tail": self.trace[-4:],
    }
    return summary, gates

```



```

specs = [
    RequestSpec("A", arrival=0, prompt_tokens=3, planned_tokens=["a", "b", "<eos>"], max_tokens=3),
    RequestSpec("B", arrival=0, prompt_tokens=5, planned_tokens=["x", "STOP", "ignored"], max_tokens=3, stop_token="STOP"),
    RequestSpec("C", arrival=1, prompt_tokens=2, planned_tokens=["c"], max_tokens=1),
    RequestSpec("D", arrival=2, prompt_tokens=3, planned_tokens=["d", "e"], max_tokens=2, abort_at=3),
]

engine = ToyRequestFlowEngine(
    specs=specs,
    max_tokens=8,
    max_seqs=3,
    block_manager=ToyKVBlockManager(total_blocks=5, block_size=4),
)
engine.run()
summary, gates = engine.report()

print("request_flow_summary=", summary)
print("request_flow_gates=", gates)

```

这个 demo 的输出应当能看出：

1. A、B、C、D 都经过 WAITING，其中 A/B/D 进入 RUNNING，C 在 prefill 后立即结束。
2. B 的终止来自 output processor 的 stop，不是 scheduler 直接决定。
3. D 进入 running 后被 abort，并且 kv_blocks_after_cleanup=0。
4. metadata_rows 覆盖 prefill 和 decode，说明 scheduler output 已经转换成 model runner 可执行的 metadata。
5. request_flow_gate=True，说明输入处理、状态机、scheduler、metadata、output、cleanup 和 metrics 都能闭环。

20.22 小练习

1. 画出一个 vLLM-like 请求生命周期图，至少包含 API server、input processor、engine core、scheduler、block manager、worker、output processor。
2. 写一个简化 RequestState，包含状态、input ids、output ids、block table、arrival time、first token time 和 finish reason。
3. 写一个伪代码函数，模拟 add_request -> waiting queue -> schedule -> prefill -> running -> decode -> finished -> free blocks。
4. 解释为什么 stop string 检查可能不能只看最后一个 token id。
5. 列出请求 abort 时必须清理的 5 类资源。

20.23 本章总结

vLLM 请求调度流程的核心不是某一个类名，而是一条稳定的系统链路：入口层接收和预处理请求，engine core 维护请求状态和调度循环，scheduler 决定每轮执行计划，KV cache manager 保障显存资源，worker/model runner 执行模型，output processor 处理输出和终止条件。

理解这条链路后，continuous batching、PagedAttention 和 block manager 就能连起来：scheduler 每轮选择请求，block manager 给这些请求安排 KV blocks，model runner 根据 block table 执行 attention，output processor 决定请求是否继续进入下一轮。

下一章会继续讲 vLLM memory management，重点从更系统的角度拆解 KV Cache、GPU memory、CPU memory、prefix cache、swap/offload 和显存预算之间的关系。

第 21 章 vLLM memory management

上一章从请求生命周期角度讲了 vLLM-like engine 的调度流程：请求进入 waiting queue，scheduler 每轮选择 prefill 和 decode，block manager 分配 KV blocks，worker 执行模型，output processor 判断是否结束并释放资源。

本章换一个视角：memory management。推理框架的吞吐、并发、TTFT、TPOT 和 OOM 风险，最终都会落到“显存和缓存如何被管理”上。

一句话概括：

vLLM memory management 的核心是把 GPU memory 中最动态、最容易浪费的 KV Cache 管理成可分配、可复用、可淘汰、可观测的 block pool，并让 scheduler 基于 memory budget 做 admission control、preemption 和调度取舍。

21.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考五类公开资料：

1. vLLM PagedAttention 论文和项目文档对 block-based KV cache、非连续 physical blocks、block table 和高吞吐 serving 的公开口径。
2. vLLM prefix caching 设计文档对 KVCacheBlock、block hash、parent hash、extra hashes、free queue、cached-but-free block、LRU eviction 和引用计数的说明。
3. vLLM optimization / tuning 文档对 KV cache capacity、preemption、recompute、gpu_memory_utilization、max_num_seqs、max_num_batched_tokens 和 chunked prefill 的调优边界说明。
4. vLLM metrics 文档对 GPU / CPU KV cache usage、prefix cache hit rate、preemption 和 request queue / running / waiting 指标的观测口径。
5. vLLM Hybrid KV Cache Manager 文档对 full attention、sliding window attention、state space layer 等混合 cache 类型、KV cache group、page size 和统一管理接口的说明。

本章只讲 vLLM-like memory management 的教学抽象，不绑定某个 vLLM 版本的真实源码类名、CUDA kernel、真实内存分配器、swap 实现、KV connector、分布式 KV transfer、生产参数全集或具体 GPU 型号。本章 demo 用纯 Python list / dict 模拟 block pool、free queue、prefix cache、ref count、eviction、preemption 和 metrics，不等同于真实 vLLM runtime。

参考资料：

1. vLLM paper / PagedAttention: <https://arxiv.org/abs/2309.06180>
2. vLLM prefix caching design: https://docs.vllm.ai/en/latest/design/prefix_caching/
3. vLLM optimization and tuning: <https://docs.vllm.ai/en/latest/configuration/optimization/>
4. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics/>
5. vLLM Hybrid KV Cache Manager: https://docs.vllm.ai/en/latest/design/hybrid_kv_cache_manager/

21.1 本章目标

读完本章，你应该能讲清：

1. LLM serving 中 GPU memory 主要被哪些部分占用。
2. 为什么 KV Cache 是 memory management 的核心难点。
3. vLLM 如何用 block pool、free queue、block table、ref count 管理 KV Cache。
4. Prefix caching、LRU eviction、preemption、recompute 和 offload 分别解决什么问题。
5. gpu_memory_utilization、max_num_seqs、max_num_batched_tokens 这类参数如何影响显存和性能。
6. 混合 attention 模型为什么需要 Hybrid KV Cache Manager。
7. 面试中如何系统回答“vLLM 如何管理显存”。

21.2 LLM serving 的显存都花在哪里

先把 GPU memory 拆开。

一个 LLM serving worker 的显存通常包括：

1. 模型权重。
2. KV Cache。
3. 临时激活和中间 buffer。
4. attention backend 或 CUDA graph 需要的 workspace。
5. logits、sampling、通信 buffer。
6. LoRA adapter 或多模型附加权重。
7. 多模态 encoder 相关缓存或输入 tensor。

其中模型权重相对静态，服务启动时加载后基本不随请求数量变化。

KV Cache 则完全不同。它随请求数、prompt 长度、生成长度动态增长。

一个请求越长，占用 KV Cache 越多；并发越高，同时驻留的 KV Cache 越多；输出越长，decode 过程中还会继续追加 KV Cache。

所以线上 OOM 往往不是模型刚加载时立刻 OOM，而是在高并发、长 prompt、长输出或 prefix cache 不当时逐步逼近显存上限。

可以把单个 worker 的 GPU memory 粗略拆成：

$$M_{\text{gpu}} = M_{\text{weight}} + M_{\text{kv}} + M_{\text{work}} + M_{\text{graph}} + M_{\text{sample}} + M_{\text{adapter}} + M_{\text{reserved}}$$

其中 M_{kv} 是最动态的一项。对普通 decoder-only full attention 模型，KV Cache 可以粗略估算为：

$$M_{\text{kv}} = 2LH_{\text{kv}}D_h b \sum_i T_i$$

这里 L 是层数， H_{kv} 是 KV heads， D_h 是 head dim， b 是单个元素字节数， $\sum_i T_i$ 是所有 active 请求持有的 KV tokens 总数。

21.3 KV Cache 为什么是核心难点

KV Cache 有四个特点。

第一，大。

每个 token 在每层 attention 中都要保存 key 和 value。层数越多、hidden size 越大、上下文越长，KV Cache 越大。

第二，动态增长。

Prefill 时一次写入 prompt 的 KV，decode 时每生成一个 token 又追加一份 KV。

第三，生命周期不一致。

短请求很快释放，长请求长期占用；有的请求被 abort，有的请求触发 stop string，有的请求会生成到 max tokens。

第四，存在复用机会。

多个请求可能共享相同 system prompt、文档前缀、多轮对话历史或工具说明，这些 prefix 的 KV Cache 可以复用。

如果用一整块连续张量管理，每个请求很难动态增长、释放、共享和淘汰。PagedAttention 和 block manager 就是为了解决这个问题。

21.4 从连续 cache 到 block pool

传统思路可能是给每个请求预留一段连续 KV Cache。

问题是：

1. 请求长度不可提前准确知道。
2. 预留太多会浪费。
3. 预留太少需要搬迁。
4. 请求完成后会留下碎片。
5. prefix sharing 很难做。

vLLM 的核心思路是把 KV Cache 切成固定大小的 blocks。

简化结构：

```
GPU KV memory
block 0
block 1
block 2
...
block N-1
```

每个请求维护自己的 block table：

```
request A logical blocks: [0, 1, 2]
request A physical blocks: [17, 5, 88]
```

这样，请求逻辑上连续，但物理显存不要求连续。

这就是 memory management 的第一层抽象：

连续 token 序列 -> logical blocks -> physical KV blocks

21.5 Block pool 的基本数据结构

一个最小 block pool 包括：

```
class KVCacheBlock:
    def __init__(self, block_id):
        self.block_id = block_id
        self.ref_cnt = 0
        self.block_hash = None
        self.prev_free_block = None
        self.next_free_block = None

class KVCacheManager:
    def __init__(self, num_blocks):
        self.blocks = [KVCacheBlock(i) for i in range(num_blocks)]
        self.free_queue = FreeBlockQueue(self.blocks)
        self.request_blocks = {}
        self.cached_blocks = {}
```

这些字段分别解决：

1. blocks：所有 physical blocks 的对象池。
2. free_queue：当前可分配或可淘汰的 blocks。
3. request_blocks：每个请求当前持有哪些 blocks。
4. cached_blocks：prefix cache 中 hash 到 block 的映射。
5. ref_cnt：某个 block 当前被多少请求引用。
6. block_hash：一个 full block 是否能作为 prefix cache 被复用。

vLLM prefix caching 文档中特别强调，block 对象通常在初始化时一次性创建，避免运行时频繁创建 Python 对象；free queue 可以用 block 内部的双向链表指针维护，便于 O(1) 移动和淘汰。

21.6 Page size 和 block size

容易混淆两个概念。

Block size 通常指一个 block 容纳多少 token 的 KV。

Page size 指这个 block 在物理内存中占多少字节。

对普通 full attention 模型，一个简化 page size 可以理解为：

`page_size = block_size * num_layers * kv_hidden_size`

这里 kv_hidden_size 表示一个 token 在单层中保存 KV 所需的字节数。

如果模型是 GQA 或 MQA，KV heads 少于 query heads，kv_hidden_size 会下降，KV Cache 显存也会下降。

如果模型层数更多、head dim 更大、KV dtype 更高精度，page size 就更大。

所以 block size 不是越大越好，也不是越小越好。

block size 大：

1. block table 更短。
2. 管理开销更小。
3. kernel 访问可能更规整。
4. 但最后一个 block 内部浪费更大。

block size 小：

1. 内部碎片更少。
2. 细粒度释放更灵活。
3. 但 block table 更长，管理和 kernel metadata 开销更大。

如果 block size 是 S ，单个 token 的单层 KV 字节数是 $2H_{\text{kv}}D_{\text{hb}}$ ，则普通 full attention 的单个 block page size 可以写成：

$$P_{\text{page}} = S \cdot L \cdot 2H_{\text{kv}}D_{\text{hb}}$$

请求 i 持有 T_i 个 KV tokens 时，需要的 block 数是：

$$B_i = \left\lceil \frac{T_i}{S} \right\rceil$$

最后一个 block 的内部浪费 tokens 是：

$$R_i = SB_i - T_i$$

21.7 Prefix cache 如何嵌入 memory management

Prefix caching 的目标是避免重复 prefill。

如果两个请求有相同前缀，后一个请求可以复用前一个请求已经算好的 KV blocks。

vLLM 的思路可以简化为：只缓存 full blocks，并为每个 full block 计算 hash。

一个 block hash 不只依赖本 block 的 tokens，还依赖前面 prefix 的 hash。

`block_hash_i = hash(parent_hash, block_tokens, extra_hashes)`

其中 extra_hashes 可以包含：

1. LoRA id。
2. 多模态输入 hash。
3. cache salt。
4. 其他会影响模型计算结果的条件。

为什么要包含 parent hash?

因为同一个 block tokens 出现在不同上下文中, KV 结果可能不同。

例如:

prefix A + block X
prefix B + block X

即使 block X 的 token 一样, attention 看到的历史上下文不同, KV cache 不能简单共享。

所以 prefix cache 本质是 memory management 和 correctness 的结合: 既要复用显存中的 KV blocks, 又不能错误复用不同上下文下的 KV。

可以把第 j 个 full block 的 hash 抽象成:

$$h_j = H(h_{j-1}, X_j, E_j)$$

其中 h_{j-1} 是 parent hash, X_j 是本 block tokens, E_j 是 LoRA id、多模态 hash、cache salt、租户隔离信息等 extra hashes。prefix cache 命中率可以写成:

$$R_{\text{hit}} = \frac{N_{\text{hit}}}{N_{\text{lookup}}}$$

21.8 Free queue 和 LRU eviction

当请求结束后, 它占用的 blocks 不一定马上 “彻底没用”。

如果某些 blocks 是 full blocks, 并且有 hash, 它们可以留在 prefix cache 中, 供后续请求复用。

但这些 blocks 同时也可能进入 free queue, 表示:

当前没有 active request 引用, 但如果内存紧张, 可以被淘汰并重新分配。

这就形成了一个重要状态:

cached but free

它不是正在被请求使用, 但仍然保留着可复用 KV 内容。

当新请求需要分配 blocks 时, manager 会从 free queue 取 blocks。如果取到的是 cached block, 就要先把它从 cache map 中删除, 也就是 eviction。

LRU 的直觉是: 越久没被使用的 cached block, 越先被淘汰。

简化流程:

```
def allocate_one_block():
    block = free_queue.pop_head()
    if block.block_hash is not None:
        cached_blocks.pop(block.block_hash, None)
        block.block_hash = None
    block.ref_cnt = 1
    return block
```

这里的关键点是：free 不等于清空内容，evict 才意味着这个 cached block 不再能被 prefix cache 命中。

用集合表示，active blocks、cached blocks 和 free blocks 不是互斥的。一个 block 可以同时属于 cached 与 free：

$$N_{\text{cachedfree}} = |\mathcal{C} \cap \mathcal{F}|$$

真正的可分配容量由 free blocks 决定：

$$F_t = |\mathcal{F}_t|$$

但 prefix cache 的可复用机会由 cached blocks 和 hash 命中决定。

21.9 Ref count 和共享 blocks

Prefix cache 会让多个请求共享同一个 physical block。

例如：

request A block table: [0, 1, 2, 7]

request B block table: [0, 1, 2, 9]

blocks 0、1、2 被两个请求共享，ref_cnt=2。

释放 request A 时，不能释放 blocks 0、1、2，只能把它们的 ref count 减 1。

只有当 ref_cnt=0 时，block 才能进入 free queue。

```
def free_request(req):
    for block in req.blocks:
        block.ref_cnt -= 1
        if block.ref_cnt == 0:
            free_queue.push_tail(block)
```

ref count 是 prefix sharing 的底层安全机制。

没有 ref count，就可能出现 request A 结束释放了 block，但 request B 还在读这个 block，导致 attention 读到被覆盖的数据。

共享 block 数可以写成：

$$N_{\text{shared}} = \sum_p \mathbf{1}[c_p > 1]$$

其中 c_p 是 physical block p 的引用计数。

21.10 Preemption 和 recompute

即使有 block manager，系统仍可能出现 KV Cache 空间不足。

例如：

1. 突然来了大量长 prompt。
2. running 请求生成长度超预期。
3. max_num_seqs 设置太大。
4. gpu_memory_utilization 给 KV Cache 预留太少。
5. prefix cache 占用太多可复用但未被及时淘汰的 blocks。

当空间不足时，系统有几种选择：

1. 让新请求继续等待。

2. 拒绝请求。
3. 抢占某些 running 请求。
4. swap/offload 到 CPU。
5. 丢弃部分 KV，之后 recompute。

vLLM 优化文档中提到，V1 默认 preemption mode 更偏向 recompute，而不是 swap，因为在对应架构下 recompute 的开销可能更低。

Recompute 的意思是：释放某个请求的 KV Cache，让它暂时退出运行；等后续有空间时，再从 prompt 或部分状态重新计算 KV。

它的收益是系统更稳，不容易直接 OOM。

它的代价是端到端延迟变差，因为被抢占的请求要重算。

面试中可以这样说：

Preemption 是 memory pressure 下的保护机制，不是性能优化本身。它保证系统能继续运行，但频繁 preemption 通常说明 KV cache 可以把抢占触发条件写成：

$$G_{\text{need_preempt}, \tau} = \mathbf{1}[\Delta B_{\tau} > F_{\tau}]$$

如果本轮新增 block 需求 ΔB_{τ} 大于 free blocks F_{τ} ，scheduler 要么等待 / 拒绝，要么抢占、swap/offload 或 recompute。抢占率可以用：

$$R_{\text{preempt}} = \frac{N_{\text{preempt}}}{N_{\text{scheduled}}}$$

21.11 Swap、offload 和多级 KV Cache

更广义的 memory management 不只看 GPU。

可以把 KV Cache 看成多级资源：

GPU KV Cache -> CPU KV Cache -> remote KV store / distributed KV cache -> recompute

GPU KV Cache 最快，但容量最贵。

CPU KV Cache 容量更大，但通过 PCIe/NVLink 传输会增加延迟。

远端 KV Cache 可以跨节点共享或支持 PD 分离，但网络延迟和带宽会成为瓶颈。

Recompute 不占存储，但消耗算力和时间。

什么时候 offload 有意义？

1. 请求很长，但短期内不活跃。
2. GPU memory 紧张，但 CPU memory 充足。
3. 重算成本比传输成本更高。
4. 系统目标是避免 OOM，而不是极致低延迟。

什么时候 offload 可能是坏主意？

1. decode 每轮都需要访问被 offload 的 KV。
2. PCIe 带宽成为瓶颈。
3. 请求 TPOT 对用户非常敏感。
4. 实现复杂度和故障模式超过收益。

后面 PD 分离章节会更深入讨论跨节点 KV 迁移和共享。

21.12 Chunked prefill 和 memory budget

Chunked prefill 表面上是调度策略，本质也和 memory management 有关。

长 prompt prefill 有两个压力：

1. 一轮计算 token 数很大。
2. 一次性需要分配大量 KV blocks。

如果 prompt 长度是 16000，block size 是 16，就需要约 1000 个 blocks。

完整 prefill 会让本轮 token budget 和 block budget 同时吃紧。

Chunked prefill 把长 prompt 切成多个小块：

16000 tokens -> 2048 + 2048 + ...

好处是：

1. 单轮 token budget 更可控。
2. 可以和 decode 请求混合调度。
3. 降低单轮延迟尖刺。
4. 避免长 prefill 长时间阻塞 streaming decode。

代价是：

1. 请求完成 prefill 需要多轮。
2. TTFT 可能受 chunk 策略影响。
3. scheduler 需要维护部分 prefill 状态。

vLLM 优化文档提到，V1 中 chunked prefill 在可用时默认启用，并且调度策略通常优先 decode，再用剩余 max_num_batched_tokens 预算调度 prefill；如果 prefill 放不下，会自动 chunk。

21.13 参数如何影响 memory management

几个参数经常一起出现。

gpu_memory_utilization：控制 vLLM 可以用多少 GPU memory 预分配 cache 或相关资源。

值更大：

1. KV Cache 空间更大。
2. 可承载更多并发或更长上下文。
3. preemption 可能减少。
4. 但留给其他进程、临时 buffer 或系统波动的余量更小。

max_num_seqs：控制同时调度的最大序列数。

值更大：

1. 并发上限更高。
2. decode batch 可能更大。
3. KV Cache 压力更大。
4. 单轮调度和采样开销可能增加。

max_num_batched_tokens：控制一轮最多处理多少 tokens。

值更大：

1. prefill 接纳能力更强。
2. TTFT 可能改善。
3. 吞吐可能提高。
4. 但 decode ITL/TPOT 可能抖动。

值更小：

1. 单轮延迟更可控。

2. streaming 更平滑。
3. 但长 prompt 更容易被切碎，TTFT 可能变差。

这些参数不能孤立调。正确做法是结合业务目标：低延迟聊天、批量离线生成、长文档 RAG、多租户 serving，它们的最优配置不同。

21.14 Hybrid KV Cache Manager：为什么混合模型更复杂

现代模型不一定所有层都是 full attention。

有些模型混合：

1. full attention。
2. sliding window attention。
3. local attention。
4. Mamba 或其他 state space 层。
5. KV sharing 层。

这些层对 cache 的需求不一样。

Full attention 需要保存所有历史 token 的 KV。

Sliding window attention 只需要最近窗口内的 KV。

Mamba 类层可能保存的不是标准 attention KV，而是 state。

如果仍然按“所有层、所有 token、同一种 block 规则”分配，会浪费大量显存，或者无法表达不同层的 cache 需求。

Hybrid KV Cache Manager 的核心思想是：把不同 attention/cache 类型分组，每组有自己的分配逻辑，但底层仍尽量使用统一 page size 的 memory pool。

简化管理：

KV Cache Group 0: full attention layers
KV Cache Group 1: sliding window layers
KV Cache Group 2: another sliding window group or state group

Scheduler 看起来仍然向 KVCacheManager 要 slots，但内部可能由 coordinator 分发给多个 group manager。

这说明 memory management 会随着模型架构演进而变复杂。Serving engine 不能只为传统 decoder-only full attention 写死 cache 逻辑。

21.15 Memory metrics 应该看什么

排查 vLLM-like memory 问题时，不要只看 nvidia-smi。

应该看更细指标：

1. total KV blocks。
2. free KV blocks。
3. used KV blocks。
4. cached but free blocks。
5. prefix cache hit rate。
6. prefix cache eviction count。
7. allocation failure count。
8. preemption count。
9. recompute count。
10. blocks per request 分布。
11. prompt length 和 output length 分布。
12. GPU memory reserved 和 actual used。
13. CPU offload/swap in/out 次数。
14. TTFT、TPOT 与 free blocks 的相关性。

例如：

如果 GPU utilization 不高，但 TTFT 很差，可能是 waiting queue 被 KV block 不足卡住。

如果 tokens/s 正常但 TPOT 抖动，可能是长 prefill、preemption 或 CPU contention。

如果 free blocks 下降后不恢复，可能是 abort/failed 路径没有释放 blocks。

如果 prefix cache hit rate 高但吞吐没提升，可能瓶颈不在 prefill，而在 decode 或输出处理。

21.16 常见 OOM 和性能问题

问题一：长 prompt 高并发导致 OOM。

处理思路：降低 max_num_seqs，降低 max_num_batched_tokens，启用或调优 chunked prefill，增加 tensor parallel 或提高可用 KV cache 预算。

问题二：频繁 preemption。

处理思路：说明 KV cache 空间不足或并发过高。可以提高 gpu_memory_utilization、降低并发预算、减少上下文长度、调整 TP/PP 或优化 prefix cache 策略。

问题三：请求结束后显存不回落。

处理思路：区分 PyTorch reserved memory、KV block pool 预分配和真实泄漏。看 free blocks 是否恢复，而不是只看 nvidia-smi。

问题四：prefix cache 占用多但命中低。

处理思路：检查 prompt 是否真的共享、chat template 是否一致、cache salt 是否隔离、LoRA 或多模态 hash 是否导致无法复用。

问题五：offload 后 TPOT 明显变差。

处理思路：说明 decode 访问路径变慢，PCIe/网络传输可能成为瓶颈。需要减少 offload、调整抢占策略或换 recompute。

21.17 面试官会怎么问

问题一：vLLM 如何管理 KV Cache？

回答要点：把 KV Cache 切成固定大小 blocks，维护 block pool、free queue、request block table、ref count 和 prefix cache hash map。请求逻辑上连续，物理上可以不连续。

问题二：为什么 PagedAttention 能降低显存浪费？

回答要点：避免为每个请求预留大块连续 cache，按 block 动态分配，减少碎片和内部浪费，并支持请求结束后细粒度回收。

问题三：prefix caching 如何保证正确性？

回答要点：只缓存 full blocks，用 block tokens、parent hash 和 extra hashes 构造 block hash，确保相同 token 但不同上下文、LoRA、多模态输入或租户隔离不会错误共享。

问题四：preemption 是什么？

回答要点：当 KV cache 空间不足时，系统抢占部分请求释放 cache，后续通过 recompute 或 swap 恢复。它提升鲁棒性，但频繁发生会伤害端到端延迟。

问题五：如何调优 vLLM 显存相关参数？

回答要点：增加 gpu_memory_utilization 给 KV cache 更多空间，降低 max_num_seqs 或 max_num_batched_tokens 减少并发和单轮 token 压力，必要时增加 TP/PP 分摊权重显存，让更多显存留给 KV cache。

问题六：为什么混合 attention 模型需要特殊 KV cache manager？

回答要点: full attention、sliding window、Mamba 等层的 cache 需求不同, 需要按类型分组和不同分配规则, 同时保持统一 page size 或统一管理接口, 避免浪费和错误复用。

21.18 标准回答模板

如果面试官问 “vLLM 是怎么做 memory management 的”, 可以这样回答:

vLLM 的 memory management 核心在 KV Cache。模型权重通常是相对静态的, 但 KV Cache 会随请求数、prompt 长度和生成长度动态增

vLLM 用 PagedAttention 的思想把 KV Cache 切成固定大小的 physical blocks。每个请求维护 block table, 把逻辑 token 位置映射到

这样做的好处是请求不需要连续显存, cache 可以按 block 动态分配和释放, finished 或 aborted 请求可以细粒度回收, 多个请求还

当 KV cache 空间不足时, 系统可以让请求等待、触发 preemption、recompute 或 offload。调优上, gpu_memory_utilization、max_

21.19 Memory management 公式、prefix cache 和可运行 demo

把本章的 memory management 验收合成一个教学版门禁:

$$G_{\text{mem}} = G_{\text{pool}} G_{\text{prefix}} G_{\text{ref}} G_{\text{free}} G_{\text{evict}} G_{\text{preempt}} G_{\text{cleanup}} G_{\text{metric}}$$

其中:

1. G_{pool} : block pool、free queue、request block table 初始化正确。
2. G_{prefix} : 相同 prefix 且 extra hashes 相同的 full blocks 能命中 prefix cache。
3. G_{ref} : 共享 blocks 有正确 ref count, 不会被提前释放。
4. G_{free} : finished 请求释放后, cached-but-free blocks 能被观测。
5. G_{evict} : 分配新 block 时能从 prefix cache 中淘汰 cached free block。
6. G_{preempt} : KV 空间不足时能触发 preemption / recompute 证据。
7. G_{cleanup} : 所有请求结束后 active KV blocks 回到 0。
8. G_{metric} : prefix hit、eviction、preemption、free blocks、max used blocks 都可复盘。

下面这个 0 依赖 demo 模拟 6 个 KV blocks、block size 为 4 的 toy memory manager。请求 A 写入两个 full prefix blocks 和一个 partial block; 请求 B 复用 A 的两个 full prefix blocks; 释放 A 后共享 block 不能被提前回收; 请求 C 使用不同 extra hash, 因此不能复用; 请求 D 因 free blocks 不足触发 C 的 preemption / recompute, 并在分配时淘汰一个 cached free block。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class KVBlock:
```

```
    block_id: int
```

```
    ref_cnt: int = 0
```

```
    block_hash: tuple = None
```

```
class ToyKVMemoryManager:
```

```
    def __init__(self, total_blocks, block_size):
```

```
        self.block_size = block_size
```

```
        self.blocks = [KVBlock(i) for i in range(total_blocks)]
```

```
        self.free_queue = list(range(total_blocks))
```

```
        self.request_blocks = {}
```

```
        self.cached_blocks = {}
```

```
        self.trace = []
```

```
        self.prefix_lookups = 0
```

```
        self.prefix_hits = 0
```

```

self.eviction_count = 0
self.allocation_failures = 0
self.preemption_count = 0
self.recompute_count = 0
self.max_used_blocks = 0

def used_count(self):
    return len(self.blocks) - len(self.free_queue)

def remember_peak(self):
    self.max_used_blocks = max(self.max_used_blocks, self.used_count())

def block_hash(self, parent_hash, tokens, extra_hash):
    return (parent_hash, tuple(tokens), extra_hash)

def chunks(self, tokens):
    step = self.block_size
    return [tokens[start:start + step] for start in range(0, len(tokens), step)]

def _remove_from_free(self, block_id):
    if block_id in self.free_queue:
        self.free_queue.remove(block_id)

def _take_free_block(self, reason):
    if not self.free_queue:
        return None
    block_id = self.free_queue.pop(0)
    block = self.blocks[block_id]
    if block.block_hash is not None and self.cached_blocks.get(block.block_hash) == block_id:
        self.cached_blocks.pop(block.block_hash)
        self.eviction_count += 1
        self.trace.append(f"evict_cached_block:{block_id}:{reason}")
    block.block_hash = None
    block.ref_cnt = 0
    return block

def needed_new_blocks(self, tokens, extra_hash):
    needed = 0
    parent_hash = None
    for chunk in self.chunks(tokens):
        if len(chunk) == self.block_size:
            h = self.block_hash(parent_hash, chunk, extra_hash)
            if h not in self.cached_blocks:
                needed += 1
            parent_hash = h
        else:
            needed += 1
    return needed

def can_allocate_prompt(self, tokens, extra_hash):
    return self.needed_new_blocks(tokens, extra_hash) <= len(self.free_queue)

def allocate_prompt(self, request_id, tokens, extra_hash):
    if not self.can_allocate_prompt(tokens, extra_hash):
        self.allocation_failures += 1

```

```

        self.trace.append(f"allocation_wait:{request_id}")
        return False

table = []
parent_hash = None
for chunk in self.chunks(tokens):
    if len(chunk) == self.block_size:
        h = self.block_hash(parent_hash, chunk, extra_hash)
        self.prefix_lookups += 1
        if h in self.cached_blocks:
            block_id = self.cached_blocks[h]
            block = self.blocks[block_id]
            self._remove_from_free(block_id)
            block.ref_cnt += 1
            self.prefix_hits += 1
            table.append(block_id)
            self.trace.append(f"reuse_prefix:{request_id}:block{block_id}")
        else:
            block = self._take_free_block(f"allocate:{request_id}")
            block.ref_cnt = 1
            block.block_hash = h
            self.cached_blocks[h] = block.block_id
            table.append(block.block_id)
            self.trace.append(f"allocate_full:{request_id}:block{block.block_id}")
        parent_hash = h
    else:
        block = self._take_free_block(f"allocate_partial:{request_id}")
        block.ref_cnt = 1
        table.append(block.block_id)
        self.trace.append(f"allocate_partial:{request_id}:block{block.block_id}")

self.request_blocks[request_id] = table
self.remember_peak()
return True

def release_request(self, request_id):
    released = []
    for block_id in self.request_blocks.pop(request_id, []):
        block = self.blocks[block_id]
        block.ref_cnt -= 1
        if block.ref_cnt == 0:
            if block_id not in self.free_queue:
                self.free_queue.append(block_id)
            released.append(block_id)
    self.trace.append(f"release:{request_id}:{released}")
    return released

def preempt_for_recompute(self, request_id):
    self.preemption_count += 1
    self.recompute_count += 1
    released = self.release_request(request_id)
    self.trace.append(f"preempt_recompute:{request_id}:{released}")
    return released

def cached_free_blocks(self):

```

```

    return sorted(
        block.block_id
        for block in self.blocks
        if block.ref_cnt == 0 and block.block_hash is not None and block.block_id in self.free_queue
    )

def ref_counts(self, block_ids):
    return {block_id: self.blocks[block_id].ref_cnt for block_id in block_ids}

def prefix_hit_rate(self):
    if self.prefix_lookups == 0:
        return 0.0
    return round(self.prefix_hits / self.prefix_lookups, 3)

def metrics(self):
    return {
        "total_blocks": len(self.blocks),
        "free_blocks": len(self.free_queue),
        "used_blocks": self.used_count(),
        "cached_free_blocks": len(self.cached_free_blocks()),
        "prefix_lookups": self.prefix_lookups,
        "prefix_hits": self.prefix_hits,
        "prefix_hit_rate": self.prefix_hit_rate(),
        "eviction_count": self.eviction_count,
        "allocation_failures": self.allocation_failures,
        "preemption_count": self.preemption_count,
        "recompute_count": self.recompute_count,
        "max_used_blocks": self.max_used_blocks,
    }

```

```

manager = ToyKVMemoryManager(total_blocks=6, block_size=4)
extra_prod = ("tenant", "prod", "lora:none")
extra_isolated = ("tenant", "isolated", "lora:none")

manager.allocate_prompt("A", [1, 2, 3, 4, 5, 6, 7, 8, 9], extra_prod)
a_table = list(manager.request_blocks["A"])

manager.allocate_prompt("B", [1, 2, 3, 4, 5, 6, 7, 8, 20, 21], extra_prod)
b_table = list(manager.request_blocks["B"])
shared_after_b = manager.ref_counts(b_table[:2])

manager.release_request("A")
after_release_a = manager.ref_counts(b_table[:2])

manager.allocate_prompt("C", [1, 2, 3, 4], extra_isolated)
c_table = list(manager.request_blocks["C"])

d_tokens = [30, 31, 32, 33, 34, 35, 36, 37, 38]
if not manager.can_allocate_prompt(d_tokens, extra_prod):
    manager.allocation_failures += 1
    manager.preempt_for_recompute("C")
manager.allocate_prompt("D", d_tokens, extra_prod)
d_table = list(manager.request_blocks["D"])

```

```

manager.release_request("B")
manager.release_request("D")

summary = {
    "a_table": a_table,
    "b_table_reused_prefix": b_table,
    "shared_ref_counts_after_b": shared_after_b,
    "ref_counts_after_release_a": after_release_a,
    "c_table_different_extra_hash": c_table,
    "d_table_after_preemption": d_table,
    "cached_free_blocks": manager.cached_free_blocks(),
    "metrics": manager.metrics(),
    "trace_tail": manager.trace[-8:],
}

gates = {
    "block_pool_initialized": manager.metrics()["total_blocks"] == 6,
    "prefix_cache_reused": b_table[:2] == a_table[:2] and manager.prefix_hits == 2,
    "ref_count_shared": all(count == 2 for count in shared_after_b.values()),
    "cached_free_visible": len(manager.cached_free_blocks()) >= 3,
    "lru_eviction_visible": manager.eviction_count == 1,
    "preemption_recompute_visible": manager.preemption_count == 1 and manager.recompute_count == 1,
    "cleanup_restores_free": manager.used_count() == 0 and manager.metrics()["free_blocks"] == 6,
    "metrics_ready": manager.prefix_hit_rate() > 0 and manager.max_used_blocks == 6,
}
gates["memory_management_gate"] = all(gates.values())

print("memory_management_summary=", summary)
print("memory_management_gates=", gates)

```

这份输出应当体现：

1. B 的前两个 blocks 复用了 A 的 prefix blocks，且 ref count 一度为 2。
2. A 释放后，共享 blocks 的 ref count 仍为 1，不会提前回到可覆盖状态。
3. C 使用不同 extra hash，即使 tokens 相同也不会复用 A/B 的 prefix block。
4. D 触发一次 preemption / recompute，并在分配过程中淘汰一个 cached free block。
5. 所有请求释放后 used_blocks=0，free_blocks=6，且仍能看到 cached-but-free blocks。

21.20 小练习

1. 假设 block size 为 16，一个 prompt 有 4096 tokens，需要多少 KV blocks？如果并发 32 个这样的请求，至少需要多少 blocks？
2. 画出 block pool、free queue、cached blocks、request block table 之间的关系。
3. 解释为什么 request finished 后，某些 blocks 可能进入 free queue 但仍保留在 prefix cache 中。
4. 设计一个 prefix cache hash，至少包含 parent hash、block tokens 和 LoRA id。
5. 观察一个线上 OOM case，列出你会检查的 8 个 memory metrics。
6. 比较 preemption recompute 和 CPU swap 的优缺点。

21.21 本章总结

vLLM memory management 的核心是 KV Cache block 化。

PagedAttention 让请求的逻辑 KV Cache 可以映射到非连续 physical blocks；KV Cache Manager 负责 block pool、free queue、ref count、request block table、prefix cache 和 eviction；scheduler 基于 token budget 和 block budget 做调度，避免显存不可控增长。

真正的线上 memory management 不只是“显存够不够”，还包括 prefix cache 是否命中、free blocks 是

否恢复、preemption 是否频繁、chunked prefill 是否平衡 prefill/decode、CPU/远端 offload 是否值得，以及混合 attention 模型如何分组管理 cache。

下一章会继续讲 vLLM worker、executor 和 engine 架构，重点拆解 engine core 如何把 scheduler output 派发给 worker，worker 如何持有模型和 KV cache，并在单机多卡或多进程架构下执行 forward。

第 22 章 vLLM worker、executor 和 engine 架构

上一章讲了 vLLM memory management: KV Cache 如何被切成 blocks, block pool、free queue、ref count、prefix cache、preemption 和 chunked prefill 如何共同影响并发和显存。

本章把视角从“内存资源”切到“执行架构”：scheduler 选好了本轮要跑的请求之后，这个执行计划如何被派发到 GPU？worker、executor、engine core、model runner 分别负责什么？单机多卡和多进程 serving 为什么要这么拆？

一句话概括：

vLLM 的 worker、executor 和 engine 架构，本质是在入口层、调度层和 GPU 执行层之间建立清晰边界：engine core 做调度和状态管理，executor 负责派发执行，worker 控制具体设备，model runner 把调度结果转换成模型 forward。

22.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考四类公开资料：

1. vLLM Architecture Overview 对 API server process、engine core process、GPU worker processes、DP coordinator process、LLMEngine、AsyncLLMEngine、executor、worker 和 model runner 的架构边界说明。
2. vLLM optimization / tuning 文档对 CPU 资源需求、最小进程数量、`--api-server-count`、tensor parallel、pipeline parallel、data parallel 和 expert parallel 的调优边界说明。
3. vLLM V1 guide 对 core engine、chunked prefill、统一 scheduler budget、功能默认值和 V0 / V1 行为差异的说明。
4. vLLM metrics 文档对 engine core iteration、scheduler、request state、KV cache usage、queue / prefill / decode 以及 worker 执行相关观测指标的公开口径。

本章只讲 vLLM-like worker / executor / engine 架构的教学抽象，不绑定某个 vLLM 版本的真实源码类名、RPC / ZMQ / Ray / multiprocessing 实现、CUDA graph 内部格式、真实 attention backend、分布式通信库细节、NCCL 参数、OpenAI server 参数全集或具体 GPU 型号。

本章 demo 用纯 Python 模拟 engine core、executor、workers、model runner metadata、tensor-parallel shard 聚合、worker failure、CPU process budget 和 trace，不等同于真实 vLLM runtime。

参考资料：

1. vLLM Architecture Overview: https://docs.vllm.ai/en/latest/design/arch_overview/
2. vLLM optimization and tuning: <https://docs.vllm.ai/en/latest/configuration/optimization/>
3. vLLM V1 guide: https://docs.vllm.ai/en/latest/usage/v1_guide/
4. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics/>

22.1 本章目标

读完本章，你应该能讲清：

1. API server、LLMEngine、engine core、executor、worker、model runner、model 的职责边界。
2. 为什么 vLLM-like 系统通常采用多进程架构。
3. scheduler output 如何变成 GPU worker 上的模型执行。
4. worker 为什么通常是一进程控制一张 GPU。
5. model runner 在输入准备、KV cache、CUDA graph、attention backend 中扮演什么角色。
6. tensor parallel、pipeline parallel、data parallel 会如何影响 worker 数量和通信。

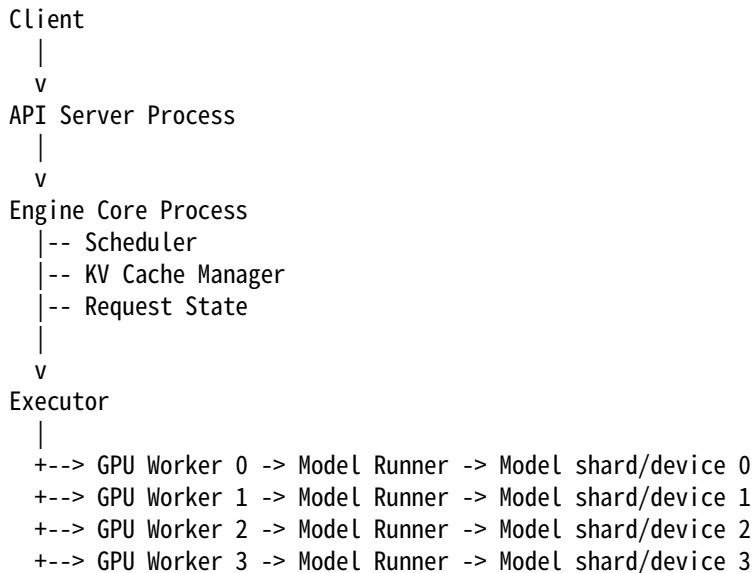
7. 面试中如何画出 vLLM 执行架构图。

22.2 先看整体进程图

vLLM V1 架构文档把核心进程分成几类：

1. API server process。
2. Engine core process。
3. GPU worker processes。
4. DP coordinator process, 如果启用 data parallel。

简化图：



如果是单 GPU，可以简化成：

API Server -> Engine Core -> Executor -> Worker -> Model Runner -> Model

如果是多 GPU，executor 和 worker 层就变得非常重要，因为 engine core 不能自己直接在每张 GPU 上做所有事情，它需要一个执行抽象来协调多个 worker。

可以用一个简单公式估算单机 online serving 的最小进程数。如果 API server 数量是 N_{api} ，engine core 数量是 N_{core} ，GPU worker 数量是 N_{worker} ，DP coordinator 数量是 N_{dpcoord} ，则：

$$N_{\mathrm{proc}} = N_{\mathrm{api}} + N_{\mathrm{core}} + N_{\mathrm{worker}} + N_{\mathrm{dpcoord}}$$

单 replica、无 DP coordinator 的常见直觉是：

$$N_{\mathrm{proc}} \approx 2 + N_{\mathrm{gpu}}$$

这也是为什么第 22 章不只讨论 GPU，还要讨论 CPU 资源和进程边界。

22.3 各模块职责一句话版

先给出最重要的边界。

API server: 接收 HTTP/OpenAI-compatible 请求，做输入处理、tokenization、多模态数据加载和 streaming 返回。

LLMEngine 或 AsyncLLMEngine: 对外暴露 Python 或异步接口, 连接 input processor、engine core 和 output processor。

Engine core: 运行 scheduler, 管理请求状态和 KV cache, 协调模型执行, 是 serving engine 的调度中枢。

Executor: 接收 engine core 的执行命令, 把它派发给一个或多个 worker, 并收集 worker 输出。

Worker: 绑定具体 device, 负责加载模型权重、管理本地 GPU 资源、执行 forward。

Model runner: 在 worker 内部准备模型输入、attention metadata、KV cache 引用、CUDA graph 捕获和模型 forward。

Model: 真正的 `torch.nn.Module` 或等价模型实现, 包含 transformer layers、attention、MLP、norm、embedding、lm head 等。

Output processor: 把 engine core 输出转换成用户可见的 RequestOutput, 处理 detokenization、stop、finish reason 和 streaming。

22.4 为什么要拆成这些层

如果只是教学版 MiniEngine, 可以写成一个类:

```
class MiniEngine:
    def step(self):
        batch = self.scheduler.schedule()
        outputs = self.model(batch)
        return self.decode(outputs)
```

但生产级 serving engine 不能这么简单。

原因一: CPU 和 GPU 职责不同。

输入处理、HTTP、tokenization、scheduler、output processor 很多是 CPU 工作; 模型 forward 是 GPU 工作。

原因二: 多卡执行需要协调。

Tensor parallel、pipeline parallel、expert parallel 都需要多个 worker 同步执行。

原因三: 故障和资源隔离。

让每个 GPU worker 在独立进程中控制一张 GPU, 可以降低 Python GIL、CUDA context、内存管理和崩溃影响的复杂度。

原因四: 部署形态多样。

同一套 engine 可能支持 offline Python API、online server、单机多卡、多机多卡、data parallel、Ray 或 multiprocessing。

原因五: 性能调优需要边界。

如果 API server 慢, 可以扩 API server; 如果 scheduler 慢, 优化 engine core; 如果 GPU 不满, 调 worker/model runner; 如果通信慢, 调并行策略。

清晰分层让问题定位更容易。

可以把分层职责覆盖率写成:

$$C_{\text{layer}} = \frac{N_{\text{api}}^{\text{ok}} + N_{\text{core}}^{\text{ok}} + N_{\text{exec}}^{\text{ok}} + N_{\text{worker}}^{\text{ok}} + N_{\text{runner}}^{\text{ok}}}{5}$$

这里的 ok 不是进程数量, 而是职责是否清晰: API 不直接控制 GPU, engine core 不直接做 forward, executor 不做业务调度, worker 不处理 HTTP, model runner 不决定请求生命周期。

22.5 Engine Core: 调度中枢

Engine core 的职责可以概括为:

1. 接收新请求和 abort 请求。
2. 维护 waiting/running/finished request state。
3. 调用 scheduler 生成本轮执行计划。
4. 管理 KV Cache block 分配、释放和 prefix cache。
5. 把 scheduler output 转成 executor 能执行的输入。
6. 调用 executor 执行模型。
7. 接收模型输出并更新请求状态。
8. 输出本轮生成 token、finish reason 和统计信息。

它不是直接执行 PyTorch forward 的地方，而是调度和状态管理中心。

一个简化 engine core loop:

```
class EngineCore:
    def step(self):
        self.handle_new_requests()
        self.handle_aborts()

        scheduler_output = self.scheduler.schedule(
            waiting=self.waiting,
            running=self.running,
            kv_cache_manager=self.kv_cache_manager,
        )

        if scheduler_output.is_empty():
            return []

        model_outputs = self.executor.execute_model(scheduler_output)
        engine_outputs = self.process_model_outputs(model_outputs)
        self.cleanup_finished_requests()
        return engine_outputs
```

真实系统还会加入 metrics、profiling、sleep/wakeup、multi-modal cache、structured output、speculative decoding、distributed execution 等能力。

22.6 Executor: 执行派发层

Executor 是 engine core 和 workers 之间的桥。

Engine core 不应该关心底层是:

1. 单进程单 GPU。
2. 多进程多 GPU。
3. Ray workers。
4. multiprocessing workers。
5. 分布式多节点。

它只想说:

这是本轮要执行的 batch，请执行模型 forward，并把结果给我。

Executor 负责把这个高层命令翻译成 worker 调用。

简化接口:

```
class Executor:
    def execute_model(self, scheduler_output):
        worker_inputs = self.prepare_worker_inputs(scheduler_output)
```

```

results = []
for worker, worker_input in zip(self.workers, worker_inputs):
    results.append(worker.execute_model(worker_input))
return self.merge_worker_outputs(results)

```

在 tensor parallel 场景中，所有 TP workers 可能必须共同执行同一个 forward，每个 worker 持有一部分权重，过程中需要 collective communication。

在 pipeline parallel 场景中，不同 worker 可能持有不同层，输入要沿 pipeline 传递。

在 data parallel 场景中，不同 engine core 或 worker group 可能处理不同请求 batch。

Executor 的价值是把这些复杂并行细节隐藏在统一执行接口后面。

如果 tensor parallel 大小是 T_{tp} ，pipeline parallel 大小是 T_{pp} ，data parallel 大小是 T_{dp} ，则一个简化 worker 数估算是：

$$N_{\mathrm{worker}} = T_{\mathrm{tp}} T_{\mathrm{pp}} T_{\mathrm{dp}}$$

每个 data-parallel replica 内的 worker group 大小是：

$$N_{\mathrm{group}} = T_{\mathrm{tp}} T_{\mathrm{pp}}$$

这只是教学抽象，真实系统还会受 expert parallel、driver worker、调度拓扑和部署方式影响。

22.7 Worker：一张 GPU 的控制者

vLLM 架构文档中，worker 通常是运行模型推理的进程，并且常见实践是一进程控制一个 accelerator device。

worker 的职责包括：

1. 设置当前 device。
2. 初始化分布式通信环境。
3. 加载或构造模型。
4. 分配本地 KV Cache memory。
5. 持有 model runner。
6. 执行 model forward。
7. 返回 logits、hidden states 或采样需要的信息。
8. 执行 profile、sleep、wake up 等控制命令。

为什么一进程一 GPU 常见？

1. CUDA context 更清晰。
2. GPU memory 管理边界更清楚。
3. 多进程能绕开部分 Python GIL 影响。
4. worker 崩溃或 hang 的问题更容易隔离。
5. 分布式 rank/local rank 映射更自然。

worker 通常有两个重要标识：

1. rank：全局 rank，用于分布式通信。
2. local_rank：本机 rank，用于选择本地 GPU。

例如 4 卡 tensor parallel：

```

worker 0: rank=0, local_rank=0, cuda:0
worker 1: rank=1, local_rank=1, cuda:1
worker 2: rank=2, local_rank=2, cuda:2
worker 3: rank=3, local_rank=3, cuda:3

```

22.8 Model Runner: worker 内部的执行大脑

worker 控制设备，但真正把 request batch 变成模型 forward 的，通常是 model runner。

Model runner 负责：

1. 根据 scheduler output 准备 input tensors。
2. 准备 positions。
3. 准备 attention metadata。
4. 准备 block tables 和 slot mapping。
5. 管理 KV Cache 读写接口。
6. 选择 attention backend。
7. 调用模型 forward。
8. 处理 CUDA graph capture/replay。
9. 返回采样所需 logits。

上一章我们说过，scheduler 输出的是 request-level 执行计划，但 GPU kernel 需要 tensor-level metadata。

转换过程大致是：

```
scheduler output
-> input_ids tensor
-> positions tensor
-> block tables
-> slot mapping
-> attention metadata
-> sampling metadata
-> model forward
```

这就是 model runner 的核心价值。

Model runner 的输入完整性可以写成：

$$G_{\text{runner}} = \prod_{i \in S_{\tau}} G_{\text{ids},i} G_{\text{pos},i} G_{\text{block},i} G_{\text{slot},i} G_{\text{attn},i} G_{\text{sample},i}$$

只要其中一项缺失，本轮 worker input 就不应该进入 GPU forward。

22.9 Model Runner 为什么复杂

初学者容易以为 model runner 就是：

```
logits = model(input_ids)
```

真实情况复杂得多。

复杂点一：prefill 和 decode 形态不同。

Prefill 一次处理多个 prompt tokens, decode 通常每个 sequence 处理一个 token。二者 attention metadata、KV 写入位置和 kernel 选择可能不同。

复杂点二：batch 是动态的。

continuous batching 每轮 batch 都可能变化，model runner 需要根据本轮 scheduler output 重新构造 tensor。

复杂点三：KV Cache 是 paged 的。

attention kernel 不是读连续 past key values，而是根据 block table 找 physical blocks。

复杂点四：并行策略影响模型执行。

TP 需要 shard 权重和 all-reduce，PP 需要跨 stage 传 hidden states，EP 需要路由 experts。

复杂点五：性能优化很多。

CUDA graph、torch.compile、fused kernels、attention backend、quantization、LoRA、speculative decoding 都会影响 model runner 的执行路径。

因此 model runner 是推理框架里最接近 “模型系统优化” 的地方。

22.10 从 Scheduler Output 到 Worker Input

假设 scheduler 本轮输出：

prefill: request A, request B

decode: request C, request D, request E

Engine core 或 model runner 需要构造本轮输入。

可能包括：

```
class WorkerInput:
    def __init__(self):
        self.input_ids = None
        self.positions = None
        self.block_tables = None
        self.slot_mapping = None
        self.seq_lens = None
        self.query_start_loc = None
        self.attention_metadata = None
        self.sampling_metadata = None
```

其中：

1. input_ids: 本轮实际送入模型的 tokens。
2. positions: 每个 token 的位置。
3. block_tables: 每个请求 logical block 到 physical block 的映射。
4. slot_mapping: 本轮新 KV 应该写到哪些 physical slots。
5. seq_lens: 每个 sequence 当前长度。
6. query_start_loc: 变长 batch 中每个请求 query 的起始位置。
7. attention_metadata: attention backend 所需的综合信息。
8. sampling_metadata: 每个请求的采样参数和 logits 选择位置。

这一步如果错了，模型可能不会立刻报错，但会生成错误内容或读写错误 KV 位置。

本轮 executor 派发门禁可以写成：

$$G_{\text{dispatch}, \tau} = G_{\text{runner}, \tau} G_{\text{rank}, \tau} G_{\text{merge}, \tau}$$

其中 G_{rank} 检查 worker rank / local rank / device 映射， G_{merge} 检查多 worker 输出能被正确聚合。

22.11 多 GPU 下的执行方式

多 GPU serving 不是简单把 batch 平均分到多张卡。

具体取决于并行策略。

Tensor Parallel:

同一层的权重被切到多张 GPU 上，每张 GPU 算一部分矩阵乘，之后通过 all-reduce 或 all-gather 合并。

same request batch -> worker 0/1/2/3 同时执行同一层的不同 shard

Pipeline Parallel:

模型层被切成多个 stage, 不同 GPU 负责不同层。

worker 0: layers 0-9

worker 1: layers 10-19

worker 2: layers 20-29

worker 3: layers 30-39

Data Parallel:

多个 replica 各自处理不同请求 batch, 提高总体吞吐。

engine core group 0 -> request batch A

engine core group 1 -> request batch B

Expert Parallel:

MoE expert 分布在不同 GPU 上, token 根据 router 被发送到对应 expert。

这些策略可以组合, 所以 executor/worker 必须能处理不同通信和执行拓扑。

22.12 Engine Core 与 Worker 的通信内容

Engine core 发给 worker 的不是原始 HTTP 请求。

它通常已经是压缩后的执行信息:

1. 本轮输入 token ids。
2. positions。
3. KV block tables。
4. slot mapping。
5. sampling 相关 metadata。
6. LoRA 或 adapter id。
7. 多模态 embedding 或 encoder cache 引用。
8. 控制命令, 如 profile、sleep、reset cache。

worker 返回的也不是最终文本, 而是模型执行结果:

1. logits。
2. sampled token ids。
3. hidden states, 某些任务需要。
4. finished flags 或辅助信息。
5. timing stats。

最终 detokenize、stop string、streaming response 仍然通常在 output processor 或前端路径完成。

22.13 为什么 API Server 不直接控制 GPU

有些初学者会问: 既然 API server 收到请求, 为什么不直接调用 worker?

原因是 API server 处理的是外部协议, 而 GPU worker 处理的是模型执行, 两者中间必须有 scheduler 和 engine core。

如果 API server 直接调用 GPU, 会出现:

1. 无法全局 continuous batching。
2. 无法统一管理 KV Cache。
3. 多个 HTTP 请求会争抢 GPU。
4. abort、timeout、priority 难以统一处理。
5. 多 API server 或 data parallel 下路由混乱。

正确边界是:

API server 负责请求接入和输出

Engine core 负责调度和状态

Worker 负责执行

这个边界可以让 API server 扩容、engine core 调度、worker 执行三者相对解耦。

22.14 CPU 资源为什么也重要

vLLM 优化文档强调，V1 多进程架构会让 API server、engine core 和每个 GPU worker 都需要 CPU 资源。

最小单机 N GPU 部署通常至少有：

1 API server + 1 engine core + N GPU workers

也就是至少 2 + N 个进程。

如果 CPU 核不足，会影响：

1. tokenization。
2. chat template 渲染。
3. multi-modal data loading。
4. scheduler loop。
5. worker 侧 kernel launch 和控制逻辑。
6. detokenization 和 streaming。

GPU 利用率低不一定是 GPU 算不满，也可能是 CPU 调度、输入处理或输出处理跟不上。

尤其 engine core 是调度忙循环，如果 CPU 被抢占，会直接影响每轮请求派发速度，表现为 GPU 有空洞、TPOT 抖动或吞吐下降。

CPU 资源可以用一个最小门禁表达：

$$G_{\text{cpu}} = \mathbf{1}[C_{\text{cpu}} \geq N_{\text{proc}}]$$

这里 C_{cpu} 是可用 CPU cores 的教学抽象。生产中当然还要考虑 tokenization、detokenization、multimodal processing、OS 调度、线程池、通信线程和后台 metrics，但这个式子能提醒你：GPU serving 并不是只堆 GPU。

22.15 CUDA context、初始化和 warmup

worker 启动时通常要做很多初始化：

1. 设置 device。
2. 初始化 distributed process group。
3. 加载模型权重。
4. 初始化 tokenizer 或模型配置引用。
5. 分配 KV Cache。
6. 选择 attention backend。
7. 准备 CUDA graph 或 compile 路径。
8. 做 warmup。

这些工作解释了为什么大模型 serving 启动很慢。

模型越大，权重加载越慢；并行度越高，分布式初始化越复杂；CUDA graph 或 compile 优化越 aggressive，启动时间也可能越长。

工程上常见做法是：

1. 启动后先 warmup。
2. 健康检查通过后再接流量。
3. 滚动升级时避免所有 replica 同时冷启动。
4. 把模型权重放在本地高速盘或缓存层。

22.16 故障路径：worker 挂了怎么办

生产系统必须考虑 worker failure。

可能的故障包括：

1. CUDA OOM。
2. NCCL hang。
3. kernel crash。
4. worker process crash。
5. GPU reset。
6. 网络或 IPC 通信失败。

如果 worker 挂了，engine core 不能假装请求还在正常运行。

它需要：

1. 标记相关请求 failed。
2. 清理 request state。
3. 释放或重建 KV Cache 状态。
4. 通知 API server 返回错误。
5. 触发 worker restart 或实例摘流。

多 worker 并行时，一个 worker 挂掉通常会影响整个 parallel group。比如 TP=4 的一个 rank 挂了，这个模型 replica 基本不能继续正常执行。

如果某个 parallel group 中任意 worker 失败，本轮执行门禁可以写成：

$$G_{\text{worker}} = \prod_{r \in \mathcal{R}} \mathbf{1}[\text{healthy}_r = 1]$$

$G_{\text{worker}}=0$ 时，engine core 应该标记相关请求 failed、清理状态并触发重启或摘流，而不是继续等待一个不会返回的 rank。

22.17 和前几章的关系

可以把第 18 到 22 章串起来：

第 18 章：Block Manager 管 KV Cache blocks。

第 19 章：Continuous batching 决定每轮谁跑。

第 20 章：请求生命周期说明请求如何进入、执行、输出、结束。

第 21 章：Memory management 说明显存、prefix cache、preemption、offload 如何影响调度。

第 22 章：Worker/executor/engine 架构说明调度结果如何落到 GPU 执行。

合起来就是：

request state + scheduler + KV cache manager + executor + worker/model runner

这才是 vLLM-like serving engine 的主干。

22.18 常见工程坑

坑一：把 executor 当成 scheduler。

Executor 负责执行派发，不应该决定业务调度策略。调度策略应该在 scheduler/engine core 中。

坑二：worker input 和 scheduler output 不一致。

block table、slot mapping、positions 任何一个错，都会导致 KV 读写错误。

坑三：多进程启动方式不正确。

CUDA、多进程、fork/spawn、NCCL 初始化顺序处理不好，可能导致死锁或奇怪的 CUDA 错误。

坑四：CPU 配置不足。

GPU worker 数上去了，但 CPU 核数不够，engine core 或 API server 被抢占，整体吞吐反而下降。

坑五：把所有逻辑都塞进 worker。

worker 应专注模型执行。请求队列、HTTP、stop string、复杂输出处理如果都塞进 worker，会让架构边界混乱。

坑六：忽略 rank/local_rank 映射。

多机多卡时 rank、local rank、device id、通信 group 配错，会导致模型 shard 错位或 collective hang。

22.19 面试官会怎么问

问题一：vLLM 的 engine core、executor、worker 分别负责什么？

回答要点：engine core 负责调度、请求状态和 KV cache 管理；executor 负责把执行计划派发给 worker 并收集结果；worker 绑定 GPU，持有 model runner 和模型，执行 forward。

问题二：为什么 worker 通常一进程一 GPU？

回答要点：CUDA context、显存管理、rank/local_rank、故障隔离和多进程并行更清晰，也减少 Python GIL 和共享状态复杂度。

问题三：model runner 做什么？

回答要点：把 scheduler output 转换成模型需要的 tensors 和 metadata，包括 input ids、positions、block tables、slot mapping、attention metadata、sampling metadata，并调用模型 forward。

问题四：executor 和 scheduler 有什么区别？

回答要点：scheduler 决定本轮谁应该执行，executor 决定如何把这个执行计划分发到具体 worker 上执行。

问题五：多 GPU 下 worker 数量怎么理解？

回答要点：通常每张 GPU 一个 worker。TP/PP 共同决定一个 engine core 下的 worker group 大小；DP 会复制 engine core/worker group 来处理不同 batch。

问题六：为什么 CPU 资源会影响 GPU serving？

回答要点：API server、engine core、worker control path、tokenization、scheduler、detokenization、streaming 都需要 CPU；CPU 饥饿会导致 GPU 等待调度和 kernel launch。

22.20 标准回答模板

如果面试官问“请讲一下 vLLM 的 worker、executor 和 engine 架构”，可以这样回答：

vLLM-like 系统通常把请求入口、调度和 GPU 执行拆成多层。API server 负责 HTTP/OpenAI-compatible 请求、输入处理和 streaming。

Engine core 是调度中枢，维护 waiting/running 请求状态，运行 scheduler，管理 KV cache，并把本轮 scheduler output 交给 executor。

Worker 通常是一进程控制一张 GPU，负责设置 device、加载模型、持有本地 KV cache 和 model runner，并执行模型 forward。Model level 的调度结果转换成 tensor-level 的执行输入，比如 input ids、positions、block tables、slot mapping、attention metadata。

在多 GPU 下，tensor parallel、pipeline parallel、data parallel 会改变 worker group 和通信方式。TP 是多个 worker 共同执行。

22.21 Worker / Executor 架构公式、派发 trace 和可运行 demo

把本章的执行架构验收合成一个教学版门禁：

$$G_{\text{execarch}} = G_{\text{layer}} G_{\text{dispatch}} G_{\text{runner}} G_{\text{rank}} G_{\text{merge}} G_{\text{cpu}} G_{\text{failure}} G_{\text{metric}}$$

其中：

1. G_{layer} : API server、engine core、executor、worker、model runner 职责边界清晰。
2. G_{dispatch} : executor 能把 scheduler output 派发给 worker group。
3. G_{runner} : worker input metadata 包含 input ids、positions、block table、slot mapping、attention metadata 和 sampling metadata。
4. G_{rank} : rank、local rank、device 映射一致。
5. G_{merge} : TP workers 的 shard 输出能正确聚合。
6. G_{cpu} : CPU process budget 能覆盖 API、engine core 和 workers。
7. G_{failure} : worker failure 会让相关请求 failed，而不是挂住。
8. G_{metric} : trace 能复盘 dispatch、worker inputs、outputs、merge 和 failure。

下面这个 0 依赖 demo 模拟一个 TP=2、DP=1 的 toy executor。Engine core 产出 scheduler output, executor 把它派发到两个 GPU workers; 每个 worker 校验 metadata, model runner 返回一个 shard score; executor 聚合两个 shard 得到 token; 随后模拟 worker 1 failure, 验证 engine core 能把请求标记为 failed。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class SchedulerOutput:
    request_ids: list
    input_ids: list
    positions: list
    block_tables: dict
    slot_mapping: dict
    phases: dict
    sampling_params: dict
```

```
@dataclass
```

```
class WorkerInput:
    worker_rank: int
    local_rank: int
    device: str
    request_ids: list
    input_ids: list
    positions: list
    block_tables: dict
    slot_mapping: dict
    attention_metadata: dict
    sampling_metadata: dict
```

```
class ToyModelRunner:
```

```
    def __init__(self, rank):
        self.rank = rank
```

```
    def metadata_complete(self, worker_input):
        return (
            bool(worker_input.input_ids)
```

```

        and len(worker_input.input_ids) == len(worker_input.positions)
        and bool(worker_input.block_tables)
        and bool(worker_input.slot_mapping)
        and bool(worker_input.attention_metadata)
        and bool(worker_input.sampling_metadata)
    )

def forward(self, worker_input):
    if not self.metadata_complete(worker_input):
        raise ValueError("incomplete model runner metadata")
    shard_scores = {}
    for request_id in worker_input.request_ids:
        token_sum = sum(worker_input.input_ids)
        block_sum = sum(worker_input.block_tables[request_id])
        shard_scores[request_id] = token_sum + block_sum + self.rank
    return {"rank": self.rank, "shard_scores": shard_scores}

class ToyWorker:
    def __init__(self, rank, local_rank, device):
        self.rank = rank
        self.local_rank = local_rank
        self.device = device
        self.runner = ToyModelRunner(rank)
        self.healthy = True

    def execute_model(self, worker_input):
        if not self.healthy:
            raise RuntimeError(f"worker {self.rank} failed")
        if worker_input.worker_rank != self.rank:
            raise ValueError("rank mismatch")
        if worker_input.local_rank != self.local_rank or worker_input.device != self.device:
            raise ValueError("device mapping mismatch")
        return self.runner.forward(worker_input)

class ToyExecutor:
    def __init__(self, workers, tp_size, pp_size, dp_size):
        self.workers = workers
        self.tp_size = tp_size
        self.pp_size = pp_size
        self.dp_size = dp_size
        self.trace = []

    def expected_worker_count(self):
        return self.tp_size * self.pp_size * self.dp_size

    def prepare_worker_input(self, worker, scheduler_output):
        attention_metadata = {
            "query_start_loc": [0, len(scheduler_output.input_ids)],
            "max_seq_len": max(len(v) for v in scheduler_output.block_tables.values()),
        }
        return WorkerInput(
            worker_rank=worker.rank,
            local_rank=worker.local_rank,

```

```

        device=worker.device,
        request_ids=scheduler_output.request_ids,
        input_ids=list(scheduler_output.input_ids),
        positions=list(scheduler_output.positions),
        block_tables={key: list(value) for key, value in scheduler_output.block_tables.items()},
        slot_mapping={key: list(value) for key, value in scheduler_output.slot_mapping.items()},
        attention_metadata=attention_metadata,
        sampling_metadata=dict(scheduler_output.sampling_params),
    )

def merge_worker_outputs(self, outputs):
    merged = {}
    for output in outputs:
        for request_id, score in output["shard_scores"].items():
            merged.setdefault(request_id, 0)
            merged[request_id] += score
    return {
        request_id: {
            "merged_score": score,
            "sampled_token": f"tok_{score % 7}",
        }
        for request_id, score in merged.items()
    }

def execute_model(self, scheduler_output):
    outputs = []
    worker_inputs = []
    for worker in self.workers:
        worker_input = self.prepare_worker_input(worker, scheduler_output)
        worker_inputs.append(worker_input)
        outputs.append(worker.execute_model(worker_input))
    merged = self.merge_worker_outputs(outputs)
    self.trace.append(
        {
            "request_ids": scheduler_output.request_ids,
            "worker_ranks": [worker.rank for worker in self.workers],
            "metadata_complete": all(worker.runner.metadata_complete(item) for worker, item in zip(self.workers,
            "merged": merged,
        )
    )
    return merged

class ToyEngineCore:
    def __init__(self, executor, api_servers, cpu_cores):
        self.executor = executor
        self.api_servers = api_servers
        self.cpu_cores = cpu_cores
        self.request_status = {}
        self.trace = []

    def process_budget(self):
        return self.api_servers + 1 + len(self.executor.workers)

    def cpu_budget_ok(self):

```

```

    return self.cpu_cores >= self.process_budget()

def make_scheduler_output(self):
    return SchedulerOutput(
        request_ids=["A", "B"],
        input_ids=[11, 12, 21],
        positions=[0, 1, 5],
        block_tables={"A": [0, 3], "B": [4]},
        slot_mapping={"A": [0, 1], "B": [2]},
        phases={"A": "prefill", "B": "decode"},
        sampling_params={"temperature": 0.0, "top_p": 1.0},
    )

def step(self):
    scheduler_output = self.make_scheduler_output()
    for request_id in scheduler_output.request_ids:
        self.request_status[request_id] = "RUNNING"
    try:
        merged = self.executor.execute_model(scheduler_output)
        for request_id in scheduler_output.request_ids:
            self.request_status[request_id] = "DECODED"
        self.trace.append({"event": "step_ok", "merged": merged})
        return merged
    except RuntimeError as err:
        for request_id in scheduler_output.request_ids:
            self.request_status[request_id] = "FAILED"
        self.trace.append({"event": "worker_failure", "error": str(err)})
        return {}

workers = [
    ToyWorker(rank=0, local_rank=0, device="cuda:0"),
    ToyWorker(rank=1, local_rank=1, device="cuda:1"),
]
executor = ToyExecutor(workers=workers, tp_size=2, pp_size=1, dp_size=1)
engine = ToyEngineCore(executor=executor, api_servers=1, cpu_cores=4)

first_outputs = engine.step()
workers[1].healthy = False
failure_outputs = engine.step()

summary = {
    "expected_worker_count": executor.expected_worker_count(),
    "actual_worker_count": len(workers),
    "process_budget": engine.process_budget(),
    "cpu_budget_ok": engine.cpu_budget_ok(),
    "first_outputs": first_outputs,
    "failure_outputs": failure_outputs,
    "request_status": dict(engine.request_status),
    "executor_trace": executor.trace,
    "engine_trace": engine.trace,
}

gates = {
    "layer_boundary_clear": engine.api_servers == 1 and executor.expected_worker_count() == len(workers),

```

```

    "dispatch_to_all_workers": executor.trace[0]["worker_ranks"] == [0, 1],
    "runner_metadata_complete": executor.trace[0]["metadata_complete"],
    "rank_device_mapping_ok": all(worker.rank == worker.local_rank for worker in workers),
    "tp_merge_visible": first_outputs["A"]["merged_score"] == 95 and first_outputs["B"]["sampled_token"] == "tok_6",
    "cpu_process_budget_ok": engine.cpu_budget_ok(),
    "worker_failure_marks_failed": failure_outputs == {} and set(engine.request_status.values()) == {"FAILED"},
    "metrics_trace_ready": len(executor.trace) == 1 and len(engine.trace) == 2,
}
gates["executor_architecture_gate"] = all(gates.values())

print("executor_architecture_summary=", summary)
print("executor_architecture_gates=", gates)

```

这份输出应当体现：

1. expected_worker_count=2 和 actual_worker_count=2, 说明 TP=2 的 worker group 完整。
2. 第一次执行同时派发到 rank 0 和 rank 1, metadata 完整, TP shard 输出被 merge。
3. process_budget=4 且 cpu_budget_ok=True, 说明 1 API server + 1 engine core + 2 workers 的 CPU 预算足够。
4. 第二次执行模拟 worker failure, 相关请求被标记为 FAILED, 不会继续挂起。
5. executor_architecture_gate=True, 说明 layer boundary、dispatch、runner metadata、rank mapping、merge、CPU budget、failure 和 trace 都可复盘。

22.22 小练习

1. 画出单 GPU vLLM-like 架构图：API server、engine core、executor、worker、model runner、model。
2. 画出 TP=4 的架构图，标出 4 个 worker 的 rank 和 local rank。
3. 写一个简化 Executor.execute_model() 伪代码，把 scheduler output 派发给多个 workers。
4. 列出 model runner 构造一次 forward input 需要的 6 类 metadata。
5. 解释 scheduler 和 executor 的区别。
6. 分析为什么 CPU 核数不足会导致 GPU utilization 下降。

22.23 本章总结

vLLM worker、executor 和 engine 架构的核心是职责分层。

Engine core 负责调度和状态，executor 负责执行派发，worker 负责设备和模型执行，model runner 负责把调度结果转换成模型 forward 所需的 tensors 和 metadata。API server 和 output processor 则处理外部协议、输入输出和 streaming。

这种架构的价值在于：请求生命周期、continuous batching、PagedAttention、KV Cache management、多 GPU 并行和 streaming 输出都能在清晰边界下协作。理解这层执行架构后，再看 TP/PP/DP、prefix cache、PD 分离和推理性能调优都会更自然。

下一章会进入 Prefix Caching 与 Prompt Cache，重点讲 prefix cache 和传统 prompt cache 的区别、hash-based block reuse、cache isolation、命中率、淘汰策略，以及它们如何影响 TTFT 和显存。

第 23 章 Prefix Caching 与 Prompt Cache

上一章讲了 vLLM worker、executor 和 engine 架构：engine core 负责调度和状态，executor 负责执行派发，worker 和 model runner 负责把调度结果落到 GPU forward。

本章继续讲一个对 TTFT、显存和成本都非常重要的能力：Prefix Caching 与 Prompt Cache。

一句话概括：

Prompt cache 是更宽泛的平台层缓存概念，关注复用 prompt 前缀处理结果；vLLM 的 prefix caching 是 runtime 内部基于 KV cache blocks 的前缀复用机制，核心是用 hash 找到已经计算过的 full blocks，并在保证隔离和正确性的前提下复用它们。

23.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考三类公开资料：

- 1. vLLM Automatic Prefix Caching 设计文档对 hash-based KV cache block reuse、parent hash、block tokens、extra hashes、full block caching、KVCacheBlock、free queue、touch、free、LRU eviction 和示例流程的说明。
- 2. vLLM prefix caching feature 文档对 cache salt、多租户隔离、LoRA、多模态输入 hash 和 hash algorithm 配置的说明。
- 3. vLLM metrics 文档对 prefix cache queries / hits、prefix cache hit rate 计算、KV cache usage、running / waiting requests 和 Prometheus / logging 观测口径的说明。

本章只讲教学版 prompt cache 与 runtime prefix caching 的边界，不实现真实 vLLM KV cache manager、真实 block table kernel、跨实例分布式 prefix cache、远端 KV store、语义缓存、结果缓存一致性、生产级哈希算法选择或安全审计系统。本章 demo 用纯 Python list / dict 模拟 full-block prefix cache、extra hashes、cache salt、touch、LRU eviction 和 metrics，不等同于真实 vLLM runtime。

参考资料：

- 1. vLLM Automatic Prefix Caching design: https://docs.vllm.ai/en/latest/design/prefix_caching/
- 2. vLLM Automatic Prefix Caching feature: https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/
- 3. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics/>

23.1 本章目标

读完本章，你应该能讲清：

- 1. Prompt cache、prefix cache、KV cache、result cache、semantic cache 的区别。
- 2. Prefix caching 为什么主要优化 TTFT，而不是 TPOT。
- 3. vLLM 为什么只缓存 full blocks。
- 4. block hash 为什么要包含 parent hash、block tokens 和 extra hashes。
- 5. 新请求命中 prefix cache 后，scheduler 和 KV cache manager 如何处理。
- 6. cache salt、LoRA、多模态输入、模型版本、tokenizer 对隔离和正确性的影响。
- 7. prefix cache 命中率低、显存占用高、误复用风险分别怎么排查。

23.2 先区分几类缓存

大模型推理里经常同时出现这些词：

- 1. prompt cache。
- 2. prefix cache。
- 3. KV cache。
- 4. result cache。
- 5. semantic cache。

它们不是一回事。

缓存类型	缓存内容	主要位置	主要收益	主要风险
Prompt cache	prompt 前缀处理结果	平台层或 runtime 层	降低 prefill 成本	前缀匹配和权限隔离
Prefix cache	共享前缀对应的 KV blocks	runtime 内部	降低 TTFT 和 prefill 计算	显存占用、错误复用
KV cache	attention 的历史 Key/Value	runtime 内部	加速 decode	显存增长和生命周期
Result cache	完整请求到完整答案	平台层	最快返回	个性化、过期、权限

缓存类型	缓存内容	主要位置	主要收益	主要风险
Semantic cache	相似问题到答案或中间结果	平台层	减少近似重复计算	误命中导致答非所问

本章重点是 prompt cache 和 prefix cache。

可以先记住：

Prompt cache 是目标和概念层。
Prefix caching 是 runtime 内部的一种具体实现方式。

23.3 Prompt Cache 是什么

Prompt cache 缓存的是 prompt 中可复用的前缀处理结果。

典型场景：

1. 固定 system prompt。
2. 固定工具说明。
3. 固定 RAG 文档。
4. 固定代码上下文。
5. 多轮对话复用历史上下文。
6. 多 Agent 共用同一背景材料。

例如：

prefix: 你是企业客服助手。以下是产品手册全文： ...
query A: 如何退款？
query B: 是否支持海外配送？

如果 prefix 很长，两个请求都从头 prefill 会浪费大量计算。

Prompt cache 的目标是：让第二个请求不要重复计算相同 prefix。

它可以有不同实现：

1. 直接缓存 tokenized prompt。
2. 缓存 prompt 到某种中间表示。
3. 缓存 runtime 内部 KV blocks。
4. 在平台层识别共享前缀，再让 runtime 复用。

vLLM 的 prefix caching 属于第三类：缓存并复用已经计算好的 KV cache blocks。

23.4 Prefix Caching 优化什么指标

Prefix caching 主要优化 TTFT。

TTFT 可以粗略拆成：

TTFT = 排队时间 + 输入处理时间 + prefill 时间 + first token 采样时间 + 输出传输时间

Prefix cache 主要减少的是 prefill 时间。

如果一个请求有 8000 tokens prompt，其中前 7000 tokens 命中 prefix cache，那么本次真正需要 prefill 的可能只剩后 1000 tokens。

这会显著降低 first token 前的 GPU 计算量。

但它不一定明显改善 TPOT。

TPOT 主要发生在 decode 阶段：每轮根据已存在 KV cache 生成下一个 token。Prefix cache 让请求更快进入 decode，但 decode 每 token 的成本仍然取决于当前上下文长度、batch size、attention backend、KV 读取和采样开销。

所以面试中不要说“prefix cache 加速所有生成阶段”。更准确的说法是：

Prefix cache 主要降低重复前缀的 prefill 计算，因此改善 TTFT 和吞吐；对 TPOT 的影响是间接的。

23.5 为什么不能只按字符串缓存

最朴素的想法是：把 prompt 字符串作为 key。

```
cache[prompt_text] = computed_prefix
```

这在平台层做粗粒度缓存时有用，但对 runtime 内部 KV block 复用还不够。

原因一：模型看的是 tokens，不是字符串。

同一个字符串在不同 tokenizer、不同 chat template、不同 special token 配置下，可能得到不同 token ids。

原因二：KV 结果依赖模型版本和权重。

同样 token ids，在不同模型、不同 checkpoint、不同 LoRA adapter 下，KV cache 不能共享。

原因三：KV 结果依赖上下文位置。

即使某个 block 内 tokens 一样，它前面的 prefix 不同，attention 看到的历史不同，KV 对后续计算的含义也不同。

原因四：多模态输入不能只看占位 token。

图像、音频、视频可能在 token 序列里表现为 placeholder，但真正输入 embedding 取决于具体媒体内容。

原因五：多租户要隔离。

不同租户即使 prompt 一样，也不一定允许通过 cache 共享产生时延侧信道。

因此 vLLM 的 prefix caching 采用 hash-based KV block 复用，而不是简单字符串缓存。

23.6 vLLM 的核心做法：缓存 full KV blocks

vLLM prefix caching 的核心思想是：

把已经计算过的 KV cache full blocks 缓存起来；新请求到来时，如果前缀 tokens 对应的 block hash 命中，就直接复用这些 physical blocks。

为什么只缓存 full blocks？

因为 full block 边界稳定，hash 和复用更简单。

假设 block size 为 4：

tokens: A B C D E F G

blocks: [A B C D] [E F G _]

第一个 block 是 full block，可以缓存。

第二个 block 还没满，通常不作为 prefix cache entry。

这样做的好处：

1. hash key 和 block 内容一一对应。
2. block table 更容易复用。
3. cache hit 只发生在稳定边界。
4. 避免 partial block 频繁变化带来的复杂性。

代价是：如果共享前缀长度不是 block size 的整数倍，最后不足一个 block 的部分不能直接命中，需要重新计算。

例如 block size 为 16，两个请求共享前 1000 tokens，那么最多只能命中前 992 tokens，剩下 8 tokens 要重新 prefill。

23.7 block hash 为什么需要 parent hash

vLLM 文档中，block hash 的核心组成包括：

1. parent hash。
2. block tokens。
3. extra hashes。

简化公式：

```
block_hash_i = hash(parent_hash_i, block_tokens_i, extra_hashes_i)
```

parent hash 是前一个 block 的 hash。

这样 block hash 就包含了从开头到当前 block 的整条 prefix 信息。

为什么不能只 hash 当前 block tokens？

看这个例子：

```
request A: X Y Z + [hello world]
```

```
request B: P Q R + [hello world]
```

当前 block [hello world] 一样，但前文不同。

在 causal attention 中，后面 token 可以 attend 到前面 token。即使当前 block tokens 一样，它们处在不同上下文之后，后续计算不能把两者简单当成同一个前缀结果。

parent hash 的作用就是把 “当前 block 属于哪条 prefix 链” 编码进去。

这很像链式哈希：

```
block 0 hash = hash(None, block 0 tokens, extra)
```

```
block 1 hash = hash(block 0 hash, block 1 tokens, extra)
```

```
block 2 hash = hash(block 1 hash, block 2 tokens, extra)
```

所以新请求要命中 block 2，必须 block 0、block 1、block 2 组成的整条前缀链都一致。

23.8 extra hashes 包含什么

extra_hashes 用来编码 tokens 之外、但会影响 KV 正确性的因素。

常见包括：

1. 模型版本或模型标识。
2. tokenizer 或 chat template 版本。
3. LoRA adapter id。
4. 多模态输入 hash。
5. cache salt。
6. 特殊 prompt adapter 或任务配置。
7. 其他会改变 forward 结果的条件。

LoRA 为什么要进入 hash？

因为 LoRA 会改变模型权重。同样 token 在不同 LoRA adapter 下计算出的 K/V 不同。

多模态输入为什么要进入 hash？

因为 placeholder tokens 可能一样，但图片内容不同。只有把 image hash 加进去，才能避免不同图片复用同一段视觉相关 KV。

cache salt 为什么要进入 hash？

因为多租户场景下，即使内容相同，也可能不希望跨租户共享 cache。salt 能把 cache reuse 限制在同一个信任域里。

23.9 新请求命中 prefix cache 的流程

一个新请求进入 scheduler 后, KV cache manager 可以先尝试找已经计算过的 blocks。

简化流程:

```
def find_cached_prefix(input_ids, extra_hashes, block_size, cached_blocks):
    parent_hash = None
    hit_blocks = []

    for block_tokens in split_full_blocks(input_ids, block_size):
        block_hash = hash_block(parent_hash, block_tokens, extra_hashes)
        block = cached_blocks.get(block_hash)
        if block is None:
            break
        hit_blocks.append(block)
        parent_hash = block_hash

    return hit_blocks
```

命中后, 系统需要做几件事:

1. 把命中的 physical blocks 加入请求的 block table。
2. 增加这些 blocks 的 ref count。
3. 如果命中 block 当前在 free queue 中, 需要 touch 它, 避免被淘汰。
4. 对未命中的剩余 tokens 分配新 blocks。
5. 只对未命中部分执行 prefill。

简化状态变化:

```
input tokens: [cached prefix][uncached suffix]
KV blocks:    [reuse blocks ][new blocks  ]
compute:      skip prefill  run prefill
```

这就是为什么 prefix caching 能降低 TTFT。

23.10 touch: 命中后不能马上被淘汰

vLLM prefix caching 文档里有一个重要操作: touch。

当一个请求命中了 cached block, 而这个 block 当前没有 active request 使用时, 它可能在 free queue 中。

如果不 touch, 后续分配新 block 时可能把它从 free queue head 弹出并 evict, 导致当前请求刚命中就失效。

touch 要做的事情是:

1. 增加 ref count。
2. 如果 block 在 free queue 中, 把它移出 free queue。
3. 标记它正在被 active request 使用。

伪代码:

```
def touch_cached_block(block):
    if block.ref_cnt == 0:
        free_queue.remove(block)
    block.ref_cnt += 1
```

这说明 prefix cache 不只是 hash map 查找, 还必须和 block 生命周期、free queue 和 ref count 一起工作。

23.11 请求结束后为什么 cache 还在

普通 KV cache 生命周期是:

请求开始 -> 分配 KV -> 请求结束 -> 释放 KV

启用 prefix cache 后, 生命周期变成:

请求开始 -> 分配 KV -> full blocks 加入 cache -> 请求结束 -> ref count 归零 -> 进入 free queue 但保留 hash 和内容 -> 后续可能被复用或淘汰

所以请求结束后, 某些 blocks 可能处于:

cached but free

它们没有 active request 引用, 可以被分配器拿去复用显存; 但在被真正 evict 之前, hash map 仍能找到它们, 后续请求可以复用。

这就是 prefix cache 的显存 trade-off:

1. 保留更多 cached blocks, 命中率可能更高。
2. 但可用 free blocks 看起来更少, 或者更容易触发 eviction/preemption。
3. 命中率低时, prefix cache 可能变成显存负担。

23.12 Eviction: 缓存不是永久的

当 free blocks 不够时, 系统需要淘汰 cached blocks。

常见策略是 LRU。

LRU 的直觉: 越久没用的 cache, 越可能未来也用不到。

简化流程:

```
def evict_one_block():
    block = free_queue.pop_head()
    if block.block_hash is not None:
        cached_blocks.pop(block.block_hash, None)
        block.block_hash = None
    return block
```

注意: eviction 不应该发生在 ref count 大于 0 的 block 上。

如果一个 block 仍被 active request 引用, evict 会导致正在运行的请求读错 KV。

所以 cached block 可以分成三类:

1. active cached block: 正在被请求使用, 不能淘汰。
2. free cached block: 没有请求使用, 可以在需要时淘汰。
3. evicted block: hash 被移除, 内容可被覆盖, 不能再命中。

23.13 Prefix cache 和 continuous batching 的关系

Prefix cache 不是单独存在的优化, 它会影响 scheduler。

如果一个 waiting 请求命中了大段 prefix, 它的实际 prefill token 数变小。

例如:

```
prompt length = 8192
cached prefix = 7168
uncached suffix = 1024
```

对 scheduler 来说, 这个请求本轮 prefill 成本更接近 1024 tokens, 而不是 8192 tokens。

因此 prefix cache 会影响:

1. token budget 估算。
2. KV block allocation。
3. waiting queue admission。

4. TTFT 预测。
5. chunked prefill 是否需要切分。

如果 scheduler 不知道 cache hit length，就会错误估计本轮工作量。

这也是为什么 prefix cache 通常内嵌在 KV cache manager 和 scheduler 交互中，而不是一个外部字典就能解决。

23.14 Prefix cache 和 Prompt Cache 的区别

现在可以正式对比。

对比点	Prompt Cache	Prefix Caching
层次	平台层概念，可由多种方式实现	runtime 内部机制
缓存对象	prompt 前缀处理结果	KV cache full blocks
命中依据	文本、token 或业务 key	block hash 链
主要收益	减少重复 prefill	减少重复 prefill
管理资源	可能是内存、磁盘、远端缓存	GPU/CPU KV blocks
正确性要求	模型、tokenizer、权限一致	block tokens、parent hash、extra hashes 一致
风险	业务隔离、过期、误匹配	显存占用、hash 冲突、跨租户泄露

可以这样理解：

Prompt cache 是“我要复用 prompt 前缀”的产品/平台抽象。

Prefix caching 是“我用 KV blocks 复用前缀”的 runtime 实现。

23.15 多模态 prefix cache 的特殊问题

多模态请求通常会把图片、音频或视频变成 placeholder tokens 或 embedding。

例如：

tokenized prompt: [text tokens, <image_placeholder>, <image_placeholder>, ...]

如果只 hash token ids，不同图片可能得到一样的 placeholder token 序列。

这会导致严重错误：请求 A 的图片 KV 被请求 B 的图片复用。

因此多模态 prefix cache 必须把媒体内容或处理结果的 hash 加入 extra hashes。

例如：

extra_hashes = [image_hash, lora_id, cache_salt]

同时还要保证：

1. 图像预处理版本一致。
2. vision encoder 版本一致。
3. placeholder 数量和位置一致。
4. 多模态 embedding 注入位置一致。

否则即使文本 tokens 一致，也不能安全复用。

23.16 cache salt 与多租户安全

Prefix cache 的一个隐蔽风险是时延侧信道。

假设跨租户共享 cache，攻击者可以构造某个 prompt，然后观察 TTFT 是否显著变低，从而推测其他租户是否请求过相同内容。

cache salt 的作用是把 cache reuse 限制在同一个 trust domain。

例如：

```
tenant A salt = salt_A  
tenant B salt = salt_B
```

即使两个租户的 prompt tokens 完全一样，因为 salt 不同，block hash 也不同，不能互相命中。

这会降低跨租户 cache 命中率，但提升隐私和隔离。

多租户场景下，正确策略通常是：

1. 同一租户内部可以共享。
2. 同一企业 workspace 内可配置共享。
3. 不同租户默认隔离。
4. 高敏业务禁用跨用户 prefix cache。
5. cache hit 相关指标不要暴露给不可信用户。

23.17 Hash 算法和碰撞风险

vLLM 文档提到，新版本默认使用更安全的 hash 算法来降低碰撞风险，并允许配置不同 prefix caching hash algorithm。

从工程角度，hash 选择有 trade-off：

1. 加密强度更高的 hash 更安全，但可能更慢。
2. 非加密 hash 更快，但理论碰撞风险更高。
3. 跨语言、跨版本可复现序列化有利于分布式 cache。
4. Python pickle 类序列化可能不保证跨版本完全稳定。

如果 hash 碰撞导致错误命中，后果不是普通缓存返回旧值，而是模型可能基于错误 KV 继续生成，问题很难定位。

所以在多租户或安全敏感场景中，不能只追求 hash 速度。

23.18 Hybrid attention 下的 prefix cache

普通 full attention 模型中，prefix cache 检查比较直观：从左到右扫描 full blocks，遇到第一个 miss 就停止。但混合 attention 模型更复杂。

例如 sliding window attention 只关注最近窗口内 tokens，不一定需要保留所有历史 KV。

Hybrid KV Cache Manager 文档中提到，不同 attention 类型有不同 prefix cache 规则：

1. full attention：cache hit prefix 要求所有历史 tokens 的相关 blocks 都存在。
2. sliding window：可能只要求窗口内最近 tokens 的 blocks 存在。
3. full + sliding window：需要对不同 group 的 cache hit 做交集。

简化管理解：

```
Full attention group hit length = A  
Sliding window group hit length = B  
Final reusable prefix length = intersection(A, B)
```

这说明 prefix caching 不是只和 tokens 有关，还和模型 attention 模式有关。

未来模型结构越复杂，KV cache group 和 prefix cache rule 也会越复杂。

23.19 命中率为什么低

线上 prefix cache 命中率低，常见原因包括：

1. prompt 实际不共享。
2. chat template 每次插入不同时间戳、request id 或随机字段。

3. RAG 文档排序不稳定。
4. tokenizer 或 special token 配置不一致。
5. block size 导致共享前缀没有落在 full block 边界。
6. LoRA adapter 不同。
7. cache salt 过细，隔离粒度太小。
8. 多模态 hash 不一致。
9. cache 容量太小，刚缓存就被 evict。
10. 流量分散在多个 replica，单实例看不到足够重复。

排查时不要只看业务说“我们的 system prompt 一样”。要看 tokenized prompt 的前缀是否真的一致。

建议指标：

1. prefix cache hit rate。
2. hit prefix length 分布。
3. saved prefill tokens。
4. evicted cached blocks。
5. cached-but-free blocks。
6. full block cache entries。
7. cache hit by tenant/model/route。
8. TTFT by hit length bucket。

23.20 Prefix cache 的副作用

Prefix cache 不是免费午餐。

副作用一：占用 KV memory。

请求结束后保留 blocks 可以提高命中率，但也会占用 cache pool，导致新请求更容易触发 eviction 或 preemption。

副作用二：增加管理复杂度。

需要维护 hash、ref count、free queue、LRU、touch、evict 和 request block table。

副作用三：隔离风险。

如果没有 cache salt 或租户隔离，可能产生跨租户侧信道。

副作用四：调试困难。

错误复用 KV 后，模型输出可能只是“变奇怪”，不一定立刻报错。

副作用五：命中率依赖流量模式。

如果请求前缀高度随机，prefix cache 带来的收益很小，却仍有管理开销。

因此 prefix cache 最适合共享前缀明显、长 prompt 较多、TTFT 敏感的业务。

23.21 什么时候应该启用或重视 Prefix cache

适合场景：

1. 长 system prompt。
2. 长工具说明。
3. RAG 中固定文档或固定知识库版本。
4. 代码助手中的固定仓库上下文。
5. 多轮对话中历史上下文重复出现。
6. Agent 系统中多个子任务共享背景。
7. 多用户访问同一公开文档。

不太适合场景：

1. prompt 很短。

2. 每个请求前缀都高度个性化。
3. 租户隔离要求极强，不能共享。
4. 显存非常紧张，cached blocks 挤占 active requests。
5. TTFT 主要瓶颈在排队、tokenizer、网络或上游 RAG，而不是 prefill。

23.22 面试官会怎么问

问题一：Prompt cache 和 prefix cache 有什么区别？

回答要点：prompt cache 是更宽泛的平台层概念，缓存 prompt 前缀处理结果；prefix cache 是 runtime 内部常见实现，复用前缀对应的 KV cache blocks。

问题二：vLLM prefix caching 为什么只缓存 full blocks？

回答要点：full block 边界稳定，hash 和复用简单，block table 易管理；partial block 频繁变化，复用和正确性处理更复杂。

问题三：block hash 为什么要包含 parent hash？

回答要点：同一个 block tokens 出现在不同前文后，attention 上下文不同，不能共享。parent hash 把整条 prefix 链编码进当前 block hash。

问题四：extra hashes 要包含什么？

回答要点：包含 LoRA id、多模态输入 hash、cache salt、模型/tokenizer/特殊配置等会影响 forward 结果或隔离边界的因素。

问题五：prefix cache 如何影响 scheduler？

回答要点：命中后实际 prefill token 数减少，scheduler 的 token budget、KV block allocation、admission control 和 TTFT 估算都要基于 uncached suffix，而不是完整 prompt。

问题六：多租户 prefix cache 有什么风险？

回答要点：可能通过 TTFT 差异形成侧信道，推测其他租户是否请求过相同内容；需要 cache salt、租户隔离和指标访问控制。

23.23 标准回答模板

如果面试官问“vLLM 的 prefix caching 是什么”，可以这样回答：

Prefix caching 是 vLLM 用来复用相同 prompt 前缀 KV cache 的机制，主要目标是减少重复 prefill 计算，从而降低 TTFT 和提高吞吐。

它和更宽泛的 prompt cache 相关，但更偏 runtime 内部实现。vLLM 会把 KV cache 切成固定大小 blocks，只缓存已经填满的 full blocks。

新请求进入时，KV cache manager 会从左到右计算 prompt blocks 的 hash，查找已经计算过的 blocks。命中的 blocks 会被加入请求的 KV cache。

它的收益是减少重复长前缀的 prefill，改善 TTFT；代价是占用 KV memory，并引入 hash、ref count、LRU eviction、多租户隔离和指标访问控制。

23.24 Prefix cache 公式、隔离门禁和可运行 demo

把本章的 prefix caching 先压成几个稳定公式。

TTFT 可以拆成：

$$T_{\text{ttft},i} = T_{\text{queue},i} + T_{\text{tok},i} + T_{\text{prefill},i} + T_{\text{first},i} + T_{\text{flush},i}$$

其中 prefix cache 主要减少 $T_{\text{prefill},i}$ 。

如果 block size 为 S，请求 i 命中的连续前缀长度为 $L_{\text{hit},i}$ ，因为只复用 full blocks，实际可跳过的 token 数是：

$$H_i = S \left\lfloor \frac{L_{\text{hit},i}}{S} \right\rfloor$$

所以本次需要真正 prefill 的 token 数可以写成：

$$P_i^{\text{run}} = \max(0, L_i - H_i)$$

整体 saved prefill token ratio 可以写成：

$$R_{\text{save}} = \frac{\sum_i H_i}{\sum_i L_i}$$

block hash 链可以抽象成：

$$h_j = H(h_{j-1}, X_j, E_j)$$

其中 X_j 是第 j 个 full block 的 token ids, E_j 是 model / tokenizer / LoRA / media hash / cache salt 等 extra hashes。prefix cache hit rate 可以写成：

$$R_{\text{hit}} = \frac{N_{\text{hit}}}{N_{\text{lookup}}}$$

cached-but-free blocks 是 cache set 和 free queue 的交集：

$$N_{\text{cachedfree}} = |\mathcal{C} \cap \mathcal{F}|$$

教学版 prefix cache 门禁可以写成：

$$G_{\text{prefix}} = G_{\text{hash}} G_{\text{full}} G_{\text{touch}} G_{\text{ref}} G_{\text{salt}} G_{\text{evict}} G_{\text{metric}}$$

其中：

1. G_{hash} : block hash 包含 parent hash、block tokens 和 extra hashes。
2. G_{full} : 只缓存 full blocks, partial suffix 不作为 cache entry。
3. G_{touch} : 命中 cached-free block 后会从 free queue 移除并增加引用。
4. G_{ref} : active request 的 cached block 不会被淘汰。
5. G_{salt} : cache salt、LoRA 和 media hash 改变时不会误命中。
6. G_{evict} : free cached block 可以按 LRU 淘汰并移除 hash entry。
7. G_{metric} : 命中率、saved prefill tokens、cached-free blocks 和 eviction count 可观测。

下面这个 0 依赖 demo 用固定大小 physical blocks 模拟 prefix cache。它故意让第一个请求留下 cached-free blocks, 第二个请求命中并 touch, 第三个请求因为 salt 不同 miss, 第四个请求因为 LoRA 和 media hash 不同 miss, 并在 free queue 中触发一次 LRU eviction。

```
from dataclasses import dataclass
```

```
@dataclass
class KVBlock:
    block_id: int
    tokens: tuple = ()
    parent_hash: tuple = None
```

```

block_hash: tuple = None
extra_hashes: tuple = ()
ref_count: int = 0
in_free_queue: bool = True

```

```

class ToyPrefixCacheManager:
    def __init__(self, total_blocks, block_size):
        self.block_size = block_size
        self.blocks = [KVBlock(block_id=i) for i in range(total_blocks)]
        self.free_queue = list(range(total_blocks))
        self.cache = {}
        self.request_tables = {}
        self.prefix_lookups = 0
        self.prefix_hits = 0
        self.saved_prefill_tokens = 0
        self.eviction_count = 0
        self.evicted_blocks = []

    def full_blocks(self, tokens):
        limit = len(tokens) - len(tokens) % self.block_size
        return [
            tuple(tokens[start:start + self.block_size])
            for start in range(0, limit, self.block_size)
        ]

    def make_hash(self, parent_hash, block_tokens, extra_hashes):
        return (parent_hash, tuple(block_tokens), tuple(extra_hashes))

    def touch(self, block):
        if block.ref_count == 0 and block.in_free_queue:
            self.free_queue.remove(block.block_id)
            block.in_free_queue = False
        block.ref_count += 1

    def allocate_block(self, block_tokens, parent_hash, extra_hashes):
        if not self.free_queue:
            raise RuntimeError("no free block available")
        block_id = self.free_queue.pop(0)
        block = self.blocks[block_id]
        block.in_free_queue = False
        if block.block_hash in self.cache:
            self.cache.pop(block.block_hash, None)
            self.eviction_count += 1
            self.evicted_blocks.append(block_id)

        block.tokens = tuple(block_tokens)
        block.parent_hash = parent_hash
        block.extra_hashes = tuple(extra_hashes)
        block.block_hash = self.make_hash(parent_hash, block.tokens, block.extra_hashes)
        block.ref_count = 1
        self.cache[block.block_hash] = block
        return block

    def process_request(self, request_id, tokens, extra_hashes):

```

```

blocks = self.full_blocks(tokens)
table = []
hit_blocks = []
block_hashes = []
parent_hash = None
miss_index = len(blocks)

for index, block_tokens in enumerate(blocks):
    block_hash = self.make_hash(parent_hash, block_tokens, extra_hashes)
    self.prefix_lookups += 1
    cached_block = self.cache.get(block_hash)
    if cached_block is None:
        miss_index = index
        break
    self.prefix_hits += 1
    self.touch(cached_block)
    table.append(cached_block.block_id)
    hit_blocks.append(cached_block.block_id)
    block_hashes.append(block_hash)
    parent_hash = block_hash

for block_tokens in blocks[miss_index:]:
    new_block = self.allocate_block(block_tokens, parent_hash, extra_hashes)
    table.append(new_block.block_id)
    block_hashes.append(new_block.block_hash)
    parent_hash = new_block.block_hash

saved_tokens = len(hit_blocks) * self.block_size
self.saved_prefill_tokens += saved_tokens
self.request_tables[request_id] = list(table)
return {
    "request_id": request_id,
    "table": table,
    "hit_blocks": hit_blocks,
    "block_hashes": block_hashes,
    "saved_prefill_tokens": saved_tokens,
    "run_prefill_tokens": max(0, len(tokens) - saved_tokens),
    "cacheable_full_tokens": len(blocks) * self.block_size,
    "free_queue_after_touch": list(self.free_queue),
}

def release_request(self, request_id):
    released = []
    for block_id in self.request_tables.pop(request_id, []):
        block = self.blocks[block_id]
        block.ref_count -= 1
        if block.ref_count == 0:
            block.in_free_queue = True
            self.free_queue.append(block_id)
            released.append(block_id)
    return released

def cached_free_blocks(self):
    return [
        block.block_id

```

```

        for block in self.blocks
        if block.block_hash in self.cache and block.ref_count == 0 and block.in_free_queue
    ]

def metrics(self):
    hit_rate = self.prefix_hits / self.prefix_lookups if self.prefix_lookups else 0.0
    return {
        "prefix_lookups": self.prefix_lookups,
        "prefix_hits": self.prefix_hits,
        "prefix_hit_rate": round(hit_rate, 3),
        "saved_prefill_tokens": self.saved_prefill_tokens,
        "cached_free_blocks": len(self.cached_free_blocks()),
        "eviction_count": self.eviction_count,
    }

tokens_a = [1, 2, 3, 4, 5, 6, 7, 8, 99, 100]
tokens_b = [1, 2, 3, 4, 5, 6, 7, 8, 42, 43]
base_extra = ("model:v1", "tokenizer:v1", "tenant:A", "lora:none", "media:none")
other_tenant = ("model:v1", "tokenizer:v1", "tenant:B", "lora:none", "media:none")
lora_media = ("model:v1", "tokenizer:v1", "tenant:A", "lora:finance", "media:image-cat")

manager = ToyPrefixCacheManager(total_blocks=5, block_size=4)

a = manager.process_request("A", tokens_a, base_extra)
manager.release_request("A")

b = manager.process_request("B", tokens_b, base_extra)
b_free_queue_after_touch = list(b["free_queue_after_touch"])
manager.release_request("B")

c = manager.process_request("C", tokens_a, other_tenant)
manager.release_request("C")

d = manager.process_request("D", tokens_a, lora_media)
manager.release_request("D")

metrics = manager.metrics()
summary = {
    "a_table": a["table"],
    "a_cacheable_full_tokens": a["cacheable_full_tokens"],
    "b_hit_blocks": b["hit_blocks"],
    "b_saved_prefill_tokens": b["saved_prefill_tokens"],
    "b_run_prefill_tokens": b["run_prefill_tokens"],
    "b_free_queue_after_touch": b_free_queue_after_touch,
    "c_salt_hit_blocks": c["hit_blocks"],
    "d_lora_media_hit_blocks": d["hit_blocks"],
    "d_table_after_lru": d["table"],
    "evicted_blocks": list(manager.evicted_blocks),
    "cached_free_blocks": manager.cached_free_blocks(),
    "metrics": metrics,
}

gates = {
    "hash_uses_parent": a["block_hashes"][1][0] == a["block_hashes"][0],

```

```

    "full_blocks_only": a["cacheable_full_tokens"] == 8 and a["run_prefill_tokens"] == 10,
    "same_prefix_hits": b["hit_blocks"] == [0, 1] and b["saved_prefill_tokens"] == 8,
    "touch_removes_cached_free": b_free_queue_after_touch == [2, 3, 4],
    "salt_isolation": c["hit_blocks"] == [],
    "lora_media_isolation": d["hit_blocks"] == [],
    "lru_eviction_visible": manager.evicted_blocks == [0] and metrics["eviction_count"] == 1,
    "metrics_ready": metrics["prefix_hit_rate"] == 0.4 and metrics["cached_free_blocks"] == 5,
}
gates["prefix_cache_gate"] = all(gates.values())

print("prefix_cache_summary=", summary)
print("prefix_cache_gates=", gates)

```

这份输出应当体现：

1. 请求 A 只把前 8 个 token 的两个 full blocks 写入 cache，最后 2 个 partial suffix token 不作为 cache entry。
2. 请求 B 与 A 共享前两个 full blocks，命中 [0, 1]，saved prefill tokens 为 8，真实需要 prefill 的只剩 2 个 suffix tokens。
3. 请求 B 命中时，cached-free blocks [0, 1] 被 touch 出 free queue，b_free_queue_after_touch=[2, 3, 4]。
4. 请求 C 只改 cache salt，D 改 LoRA 和 media hash，都会 miss，说明 extra hashes 隔离生效。
5. 请求 D 分配第二个 block 时淘汰最旧的 cached-free block 0，eviction_count=1。
6. prefix_cache_gate=True，说明 hash、full block、touch、ref count、隔离、eviction 和 metrics 都能复盘。

23.25 小练习

1. 假设 block size 为 16，两个请求共享前 1000 tokens，最多能直接命中多少 tokens？为什么？
2. 设计一个 block hash，要求支持 LoRA、多模态图片和多租户隔离。
3. 画出一个 cached block 从 active 到 free queue，再到被新请求 touch 的状态变化。
4. 解释为什么只按 prompt 文本字符串做 cache key 不足以保证 KV cache 复用正确。
5. 列出 prefix cache 命中率低的 8 个可能原因。
6. 设计一个 dashboard，用来观察 prefix cache 对 TTFT 的影响。

23.26 本章总结

Prompt cache 是大模型推理平台中复用 prompt 前缀处理结果的宽泛概念；prefix caching 是 vLLM-like runtime 中基于 KV cache blocks 的具体实现。

vLLM prefix caching 的关键设计是：只缓存 full blocks，用 parent hash、block tokens 和 extra hashes 构造 block hash；新请求命中后复用 physical KV blocks，只对未命中 suffix 执行 prefill。为了保证生命周期正确，还必须维护 ref count、free queue、touch、eviction 和 cached-but-free 状态。

Prefix cache 能显著改善共享长前缀场景下的 TTFT 和 prefill 吞吐，但它也带来显存占用、隔离、安全、hash 碰撞、命中率波动和调试复杂度。真正线上调优时，要结合业务 prompt 结构、租户边界、block size、cache 容量和 TTFT 分桶指标一起看。

下一章会继续讲 Tensor Parallel、Pipeline Parallel 与 serving 并行，重点解释 TP/PP/DP/EP 如何影响 worker 数量、显存分布、通信开销、吞吐和延迟。

第 24 章 Tensor Parallel、Pipeline Parallel 与 serving 并行

上一章讲了 Prefix Caching 与 Prompt Cache：如何复用相同前缀的 KV blocks，降低重复 prefill 成本和 TTFT。

本章继续讲 serving engine 的另一条主线：并行。模型太大、吞吐不够、显存不足、请求太多时，单 GPU 已经不够，需要把模型或请求分布到多张 GPU、多台机器上。此时 Tensor Parallel、Pipeline Parallel、Data Parallel、Expert Parallel 会一起出现。

一句话概括：

Serving 并行的核心不是 “GPU 越多越快”，而是根据模型大小、显存压力、通信拓扑和业务目标，选择把模型切开、把层切开、把请求复制分流，还是把 MoE experts 分布出去。

24.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考四类公开资料：

- 1. vLLM Parallelism and Scaling 文档对 tensor parallel、pipeline parallel、data parallel、expert parallel、多节点拓扑、tensor_parallel_size、pipeline_parallel_size 和跨节点部署建议的说明。
- 2. vLLM Data Parallel Deployment 文档对 internal / hybrid / external load balancing、每个 DP rank / engine 独立运行、负载均衡和部署形态的说明。
- 3. vLLM Expert Parallel Deployment 文档对 MoE expert parallel、data parallel attention、expert placement、all2all 后端和 expert 负载均衡相关调优的说明。
- 4. vLLM Optimization and Tuning 文档对 KV cache 空间不足、preemption、tensor_parallel_size、max_num_batched_tokens、max_num_seqs、chunked prefill、decode / prefill 平衡和 attention backend 的公开调优口径。

本章只讲 serving 并行策略的教学抽象，不展开真实 NCCL / Ray / multiprocessing / Kubernetes / IB / RDMA 配置，不绑定某个 vLLM 版本的内部类名、真实 kernel、真实 all-reduce / all2all 实现、具体 GPU 型号 benchmark、生产排障脚本或云厂商网络价格。本章 demo 用纯 Python 近似估算 TP / PP / DP / EP 对 worker 数、显存、通信、pipeline bubble、prefix cache locality 和门禁的影响，不等同于真实性能预测。

参考资料：

- 1. vLLM Parallelism and Scaling: https://docs.vllm.ai/en/latest/serving/parallelism_scaling/
- 2. vLLM Data Parallel Deployment: https://docs.vllm.ai/en/latest/serving/data_parallel_deployment/
- 3. vLLM Expert Parallel Deployment: https://docs.vllm.ai/en/latest/serving/expert_parallel_deployment/
- 4. vLLM Optimization and Tuning: <https://docs.vllm.ai/en/latest/configuration/optimization/>

24.1 本章目标

读完本章，你应该能讲清：

- 1. Tensor Parallel、Pipeline Parallel、Data Parallel、Expert Parallel 分别解决什么问题。
- 2. 为什么 vLLM 单机多卡常用 TP，跨节点常用 TP+PP。
- 3. TP 如何影响显存、KV Cache、通信和 TPOT。
- 4. PP 如何影响显存、延迟、pipeline bubble 和跨节点部署。
- 5. DP 如何提高吞吐，以及为什么每个 DP rank 有独立 KV Cache。
- 6. MoE 模型中 EP 和 DP/TP 的关系。
- 7. 面试中如何根据模型大小、GPU 拓扑和业务目标选择 serving 并行策略。

24.2 先把四种并行放在一张表里

并行方式	切分对象	主要目的	典型收益	典型代价
Tensor Parallel	单层矩阵和 attention heads	让单层参数/计算分布到多 GPU	降低单卡权重显存，支持大模型	高频 collective 通信
Pipeline Parallel	模型层	把不同层放到不同 GPU/节点	降低单卡权重显存，支持跨节点	pipeline bubble，单请求延迟变长

并行方式	切分对象	主要目的	典型收益	典型代价
Data Parallel	请求 batch 或 engine replica	复制模型处理不同请求	横向扩吞吐, 隔离请求	每个 replica 都要一份权重和 KV cache
Expert Parallel	MoE experts	把不同 expert 放到不同 GPU	扩展 MoE expert 容量	token routing 和负载不均

服务端最常见组合是：

单模型副本内部：TP 或 TP + PP

多个模型副本之间：DP 或外部负载均衡

MoE expert 层：EP 或 TP/DP 组合

24.3 Serving 并行和训练并行有什么不同

训练并行要处理 forward、backward、gradient、optimizer states、checkpoint 和参数同步。

Serving 并行主要处理：

1. forward。
2. KV Cache。
3. continuous batching。
4. 请求调度。
5. streaming 输出。
6. 多副本负载均衡。
7. TPOT、TTFT 和吞吐。

因此 serving 下的并行目标也不同。

训练里你关心 step time、global batch size、显存能不能放下 optimizer。

Serving 里你关心：

1. 模型权重是否能放下。
2. 剩余显存能放多少 KV Cache。
3. 单 token decode 延迟是否稳定。
4. 请求能不能 continuous batching。
5. 通信是否拖慢 TPOT。
6. 多副本是否能提升总 QPS。

不要把训练里的并行结论直接搬到推理 serving。

24.4 Tensor Parallel 的直觉

Tensor Parallel, 简称 TP, 是把模型单层内部的大矩阵切到多张 GPU 上。

以 Linear 层为例：

$$Y = X W$$

如果 W 太大，可以按列或按行切分：

$$W = [W_0, W_1, W_2, W_3]$$

每张 GPU 持有一部分权重，计算一部分结果，最后通过通信合并。

Transformer 中 TP 常作用在：

1. QKV projection。
2. attention heads。
3. output projection。

4. MLP up/gate/down projection。
5. lm head。

直觉是：

一层太宽，一张 GPU 放不下或算不过来，就把这一层拆给多张 GPU。

24.5 TP 在 serving 中解决什么问题

TP 首先解决权重显存问题。

例如一个模型权重需要 140GB，单张 80GB GPU 放不下。TP=2 后，每张 GPU 可能只放约一半权重，再加上 KV Cache 和 runtime buffer，模型就可能跑起来。

TP 还会间接增加 KV Cache 可用空间。

因为权重被分片后，每张 GPU 上权重占用减少，剩余 HBM 可以留给 KV Cache。

vLLM 优化文档中也提到，当频繁 preemption 是因为 KV cache 空间不足时，可以考虑增加 `tensor_parallel_size`，因为权重分片后每张卡可用于 KV Cache 的空间可能增加。

但 TP 不是免费提升。

TP 的代价是每层都可能需要通信。

常见通信包括：

1. all-reduce。
2. all-gather。
3. reduce-scatter。
4. tensor shard 间同步。

decode 阶段每生成一个 token 都要走所有层，所以 TP 通信会直接影响 TPOT。

24.6 TP 为什么偏好同机高速互联

TP 通信频率高，通常每层都要通信。

如果 TP group 跨节点，通信会走网络，延迟和带宽都比同机 NVLink/NVSwitch 差很多。

所以 vLLM parallelism 文档建议：

跨节点部署时，常见做法是 `tensor_parallel_size` = 每个节点的 GPU 数，`pipeline_parallel_size` = 节点数。

例如 2 台机器，每台 8 张 GPU：

TP = 8

PP = 2

这样 TP 通信主要留在节点内部，跨节点只发生 pipeline stage 间的激活传输。

如果你把 TP=16 横跨两台机器，每一层都可能跨节点通信，TPOT 可能明显变差。

面试中可以这样说：

TP 是高频通信并行，最好放在 NVLink/NVSwitch 范围内；跨节点更适合用 PP 或 DP，除非网络非常强并且没有其他选择。

24.7 TP 和 KV Cache 的关系

TP 不只切权重，也会影响 KV Cache 布局。

在 attention 中，KV Cache 通常按层、KV heads、head dim、token 位置存储。

如果 attention heads 或 KV heads 被切到不同 TP ranks，每个 rank 只持有自己负责的 KV head 分片。

简化管理：

TP rank 0: 部分 attention heads + 对应 KV cache
TP rank 1: 部分 attention heads + 对应 KV cache
TP rank 2: 部分 attention heads + 对应 KV cache
TP rank 3: 部分 attention heads + 对应 KV cache

这意味着：

1. 每个 rank 的 KV Cache 是模型分片的一部分。
2. block manager 和 attention metadata 要和 TP rank 对齐。
3. prefix cache 命中也要在所有 TP ranks 上保持一致。
4. 一个 TP group 中任一 rank 出错，整个 replica 通常不可用。

GQA/MQA 会进一步影响 KV Cache，因为 KV heads 少于 query heads，KV Cache 本身更小，但 TP 切分时仍要确保 head 分配合法。

24.8 Pipeline Parallel 的直觉

Pipeline Parallel，简称 PP，是把模型层切到不同 GPU 或节点上。

例如 40 层模型，用 PP=4：

GPU 0: layers 0-9
GPU 1: layers 10-19
GPU 2: layers 20-29
GPU 3: layers 30-39

输入先经过 GPU 0，再传给 GPU 1，再传给 GPU 2，最后到 GPU 3。

直觉是：

模型太深或节点装不下，就按层切开。

PP 的好处是每个 GPU 只保存一部分层的权重，显存压力下降。

PP 的代价是单个请求要按 stage 顺序走，天然增加串行路径。

24.9 PP 适合什么场景

PP 适合：

1. 模型单节点放不下，需要跨节点。
2. GPU 之间没有高速 NVLink，TP 通信太贵。
3. 模型层数多，按层切分比较自然。
4. GPU 显存不均或模型大小不能被 TP 均匀切分。
5. 已经把 TP 用到合理上限，还需要继续扩展模型容量。

vLLM 文档提到一个边界情况：如果模型能放在单节点内，但 GPU 数不能均匀切分模型，或者节点没有 NVLink，比如某些 L40S 机器，使用 pipeline parallel 可能比 tensor parallel 有更低通信开销和更好吞吐。

原因是 PP 不是每层都 all-reduce，而是 stage 之间传递 hidden states。

但 PP 也有明显代价：

1. 单请求延迟可能变长。
2. pipeline bubble 会浪费设备。
3. continuous batching 和 pipeline 调度结合更复杂。
4. stage 间负载不均会拖慢整体。

24.10 Pipeline bubble 是什么

Pipeline bubble 指某些 stage 没有活干的空泡。

例如 PP=4，只有一个 microbatch：

time 0: stage 0 工作, stage 1/2/3 空闲
time 1: stage 1 工作, stage 0/2/3 空闲
time 2: stage 2 工作, stage 0/1/3 空闲
time 3: stage 3 工作, stage 0/1/2 空闲

如果没有足够 microbatches 填满 pipeline, GPU 利用率会很差。

训练中常用 microbatch 填 pipeline。

Serving 中 continuous batching 可以提供多个请求, 但 decode 阶段每个 step 粒度小、请求长度不同、streaming 对延迟敏感, 所以 pipeline 调度仍然复杂。

简单说:

PP 能让更大模型放得下, 但可能用延迟和调度复杂度换显存。

24.11 Data Parallel 的直觉

Data Parallel, 简称 DP, 在 serving 中通常不是训练里的梯度同步, 而是复制模型副本处理不同请求。

例如 DP=4:

DP rank 0: 一套 engine + worker group + KV cache
DP rank 1: 一套 engine + worker group + KV cache
DP rank 2: 一套 engine + worker group + KV cache
DP rank 3: 一套 engine + worker group + KV cache

请求由 router 或 API server 分发到不同 DP ranks。

DP 的目标是横向扩吞吐。

如果单个 replica 已经可以放下模型, 但 QPS 不够, 就增加 replicas。

DP 的代价是每个 replica 都要加载一份模型权重, 并维护独立 KV Cache。

所以 DP 解决的是吞吐扩展, 不解决单个模型副本放不下的问题。

24.12 DP rank 的 KV Cache 是独立的

vLLM data parallel 文档明确提到, 每个 DP engine 有独立 KV Cache。

这有几个重要影响。

第一, prefix cache 不自动跨 DP rank 共享。

同一个 prompt 如果第一次打到 rank 0, 第二次打到 rank 1, rank 1 不一定有 prefix cache 命中。

第二, 路由会影响 cache 命中率。

如果同类 prompt 能稳定路由到同一个 DP rank, prefix cache 命中率可能更高。

第三, 容量按 rank 分开计算。

max_num_seqs、KV block pool、waiting/running queue 都是每个 rank 自己一套。

第四, 某个 rank 过载不会自动让另一个 rank 共享它的 KV state。

因此 DP 负载均衡最好考虑:

1. running queue。
2. waiting queue。
3. KV cache free blocks。
4. prefix cache locality。
5. tenant 或会话粘性。

简单 round-robin 不一定最优。

24.13 Internal、Hybrid 和 External DP load balancing

vLLM data parallel 文档中提到几种在线 DP 部署形态。

Internal load balancing:

一个对外 API endpoint

API server 内部把请求分发到多个 DP ranks

优点是使用简单，对用户透明。

缺点是 API server 可能成为瓶颈，尤其 DP 很大时。

Hybrid load balancing:

每个节点有自己的 API server

上游 LB 分发到节点

节点内 API server 分发到本地 DP ranks

优点是减少跨节点流量，避免单 head node 瓶颈。

External load balancing:

每个 DP rank 或每组 ranks 是独立部署

外部 router 根据实时指标分发请求

优点是灵活，适合大规模生产平台。

缺点是平台侧要自己处理健康检查、负载指标、prefix cache locality、灰度和限流。

24.14 Expert Parallel 的直觉

Expert Parallel, 简称 EP, 主要用于 MoE 模型。

MoE 层里有多多个 experts, 每个 token 只路由到少数 experts。

如果 experts 很多, 单张 GPU 放不下所有 expert 权重, 就可以把 experts 分到多张 GPU。

简化图:

GPU 0: expert 0, expert 1

GPU 1: expert 2, expert 3

GPU 2: expert 4, expert 5

GPU 3: expert 6, expert 7

执行时, 每个 token 先经过 router, 决定去哪些 experts, 然后发生 token dispatch 和 combine。

EP 的收益是扩展 MoE expert 容量。

EP 的代价是:

1. token 路由通信复杂。
2. expert 负载可能不均。
3. 小 batch 时某些 experts 利用率低。
4. DP、TP、EP 组合时通信 group 更复杂。

vLLM 文档提到, 对于 MoE 模型, 可以使用 data parallel attention 配合 expert parallel 或 tensor parallel MoE layers。

24.15 如何选择并行策略

可以按这个决策树:

第一步, 模型单卡能不能放下?

如果能放下, 先单卡或多副本 DP, 不要急着 TP。单卡通常延迟最低、实现最简单。

第二步，单节点多卡能不能放下？

如果单卡放不下，但单节点多卡能放下，优先 TP，尤其节点内有 NVLink/NVSwitch。

第三步，单节点也放不下？

如果需要多节点，常见选择是：

TP = 每个节点 GPU 数

PP = 节点数

这样减少跨节点 TP 高频通信。

第四步，模型副本能跑但吞吐不够？

增加 DP 或外部 replicas。

第五步，是 MoE 模型吗？

考虑 EP，尤其 expert 权重很大或 expert 层成为瓶颈时。

第六步，看业务目标。

低延迟聊天优先减少通信和 pipeline bubble。

批量离线生成可以更激进地用多卡并行提高吞吐。

长上下文 RAG 要关注 KV Cache 容量和 prefix cache locality。

24.16 并行度对进程数的影响

结合第 22 章，worker 数量通常和 GPU 数一致。

一个 DP rank 内：

$\text{worker_count} = \text{TP} * \text{PP}$

总 GPU 数大致是：

$\text{total_gpus} = \text{DP} * \text{TP} * \text{PP}$

进程数还包括 API server、engine core 和可能的 DP coordinator。

例如 DP=4, TP=2, PP=1:

总 GPU = $4 * 2 * 1 = 8$

engine core = 4

GPU workers = 8

API servers = 通常至少 1，可能按 DP 或配置扩展

DP coordinator = MoE/DP 场景下可能需要

如果 DP=1, TP=8, PP=2:

总 GPU = $1 * 8 * 2 = 16$

engine core = 1

GPU workers = 16

前者是 4 个模型副本，每个副本 2 卡 TP。

后者是 1 个大模型副本，跨 2 个 pipeline stages，每个 stage 内 8 卡 TP。

它们解决的问题完全不同。

24.17 并行度对显存的影响

显存主要看：

1. 权重。
2. KV Cache。
3. activation/workspace。
4. 通信 buffer。
5. runtime overhead。

TP 对权重显存：降低单卡权重占用。

TP 对 KV Cache：通常每个 rank 保存自己分片相关的 KV，单卡 KV 压力也可能下降，但 metadata 和通信 buffer 增加。

PP 对权重显存：每张 GPU 只放部分层，显著降低单卡权重。

PP 对 KV Cache：每个 stage 只保存自己层的 KV Cache，按层分布。

DP 对权重显存：每个 DP rank 都复制一份模型副本，不降低单副本权重显存。

DP 对 KV Cache：每个 DP rank 独立维护 KV Cache，容量和命中也独立。

EP 对权重显存：expert 权重被分布到不同 GPU，降低单卡 expert 权重压力。

EP 对通信：增加 token routing 相关 buffer 和通信。

24.18 并行度对延迟和吞吐的影响

TP：

1. 可能降低单卡计算压力。
2. 但每层通信会增加 decode 延迟。
3. TP 太大时，通信占比上升，TPOT 可能变差。

PP：

1. 能让更大模型运行。
2. 但单请求要经过多个 stage，延迟变长。
3. 如果 batch 或 microbatch 不足，pipeline bubble 降低吞吐。

DP：

1. 通常最直接提升总吞吐。
2. 单请求延迟不一定下降。
3. 如果路由不均，部分 rank 排队严重，整体 p99 变差。

EP：

1. MoE 中能扩展 expert 容量。
2. 但 routing、dispatch、combine 和 expert 负载不均会影响延迟。

所以不能简单说“加 GPU 提升性能”。要问：

瓶颈是权重放不下、KV cache 不够、单副本吞吐不够，还是 expert 容量不够？

不同瓶颈对应不同并行策略。

24.19 拓扑：PCIe、NVLink、NVSwitch、IB/RDMA

并行策略必须看硬件拓扑。

PCIe-only 单机：

TP 高频通信可能受限，PP 有时更合适。

NVLink/NVSwitch 单机：

适合 TP group 放在同机内，通信带宽和延迟更好。

多节点 InfiniBand/RDMA：

适合跨节点 PP、DP，或在网络足够强时跨节点 TP，但需要非常注意 NCCL、GDRDMA、拓扑和容器配置。

普通以太网：

不适合高频跨节点 TP。可以做外部 DP，但跨节点模型切分会非常吃力。

vLLM 文档中提到，可以通过 NCCL 日志确认是否走了高效网络路径，例如 GPUDirect RDMA，而不是普通 socket。

面试里要强调：

并行策略不是只看 GPU 数量，而是看 GPU 之间怎么连。

24.20 常见配置例子

例子一：7B 模型，单张 80GB GPU 可以放下。

推荐先单卡，多副本 DP 扩吞吐。

TP=1, PP=1, DP=N

例子二：70B 模型，单卡放不下，但 8 卡 NVSwitch 节点能放下。

推荐单节点 TP。

TP=8, PP=1, DP=1

例子三：超大模型，单节点 8 卡也放不下，需要 2 个节点。

常见做法：

TP=8, PP=2, DP=1

例子四：70B 模型可以用 4 卡 TP 跑起来，但 QPS 不够，有 16 张 GPU。

可以部署 4 个副本：

TP=4, PP=1, DP=4

例子五：MoE 模型，attention 层不大，但 experts 很多。

可以考虑：

DP attention + EP experts

具体还要看框架支持和 expert 负载。

24.21 常见工程坑

坑一：跨节点 TP。

表现是模型能启动，但 TPOT 很差。原因是每层 collective 都走跨节点网络。

坑二：盲目增大 TP。

TP 太大后，每张卡计算变少，通信占比变高，延迟可能变差。

坑三：DP 后 prefix cache 命中率下降。

因为每个 DP rank 的 KV cache 独立，请求被打散后相同前缀不一定落到同一 rank。

坑四：PP stage 不均衡。

某个 stage 层数多、MoE 多或 attention 更重，会拖慢整个 pipeline。

坑五：CPU 核数没跟上。

DP/TP 增加 worker 和 engine 数后，CPU 调度、tokenization、detokenization、IPC 都可能成为瓶颈。

坑六：NCCL 或网络配置错误。

表现为启动 hang、forward hang、吞吐异常低、跨节点不稳定。

坑七：把 DP 当成解决单副本显存不足的方法。

DP 是复制模型，不会让单个模型副本更容易放下。

24.22 面试官会怎么问

问题一：TP、PP、DP 的区别是什么？

回答要点：TP 切单层张量，PP 切模型层，DP 复制模型副本处理不同请求。TP/PP 解决单副本放不下或计算分布，DP 解决吞吐扩展。

问题二：为什么 TP 更适合单机？

回答要点：TP 每层都有高频 collective 通信，需要 NVLink/NVSwitch 这类高速互联；跨节点 TP 会严重拖慢 decode TPOT。

问题三：为什么跨节点常用 TP+PP？

回答要点：把 TP 限制在每个节点内部，PP 跨节点按层传 hidden states，减少每层跨节点 collective。

问题四：DP 为什么会影响 prefix cache？

回答要点：每个 DP rank 有独立 KV Cache 和 prefix cache，请求路由到不同 rank 会影响 cache locality，需要 cache-aware routing 或会话粘性。

问题五：什么时候用 PP 而不是 TP？

回答要点：模型需要跨节点、GPU 无高速互联、层切分更自然、TP 切分不均或 TP 通信代价过高时，可以考虑 PP。

问题六：MoE 为什么需要 EP？

回答要点：MoE expert 数量和权重很大，EP 把 experts 分布到不同 GPU，降低单卡 expert 显存，但引入 token routing 和负载均衡问题。

24.23 标准回答模板

如果面试官问“如何为 vLLM serving 选择 TP/PP/DP”，可以这样回答：

我会先区分目标：是单个模型副本放不下，还是吞吐不够。

如果模型单卡能放下，通常先不用 TP/PP，直接单卡或多副本 DP，延迟和复杂度最低。如果单卡放不下但单节点多卡能放下，优先使

如果单节点也放不下，需要跨节点，常见策略是 TP 等于每节点 GPU 数，PP 等于节点数。这样 TP 高频通信留在节点内部，跨节点只做

如果一个副本已经能跑，但 QPS 不够，就增加 data parallel 或外部 replicas。DP 每个 rank 有独立 engine、worker group 和 KV c

最终选择要结合模型大小、GPU 显存、KV cache 需求、NVLink/IB 拓扑、TTFT/TPOT 目标和业务流量模式。

24.24 Serving 并行公式、拓扑门禁和可运行 demo

把 TP / PP / DP 的 worker 拓扑先写成稳定公式。设：

1. T_{tp} : tensor parallel size。
2. P_{pp} : pipeline parallel size。
3. D_{dp} : data parallel size。

总 GPU 或 worker 数可以近似写成：

$$N_{\text{gpu}} = D_{\text{dp}} T_{\text{tp}} P_{\text{pp}}$$

单个 DP replica 内 worker group 大小是：

$$N_{\text{group}} = T_{\text{tp}} P_{\text{pp}}$$

如果权重总显存为 $M_{\{\mathrm{w}\}}$ ，忽略无法切分的小项后，单卡权重显存粗略为：

$$M_{\text{w},\text{rank}} \approx \frac{M_{\text{w}}}{T_{\text{tp}} P_{\text{pp}}}$$

如果 KV heads 能随 TP 切分，层能随 PP 切分，单 rank KV cache 粗略估算为：

$$M_{\text{kv},\text{rank}} \approx 2 \frac{L}{P_{\text{pp}}} \frac{H_{\text{kv}}}{T_{\text{tp}}} D_h b N_{\text{tok}}$$

其中 L 是层数， $H_{\{\mathrm{kv}\}}$ 是 KV heads， D_h 是 head dim， b 是单元素字节数， $N_{\{\mathrm{tok}\}}$ 是该 replica 当前 active tokens。

PP 的 pipeline bubble 可以用 microbatch 数 $M_{\{\mathrm{micro}\}}$ 粗略表示：

$$R_{\text{bubble}} = \frac{P_{\text{pp}} - 1}{M_{\text{micro}} + P_{\text{pp}} - 1}$$

DP 下总吞吐近似是各 replica 吞吐之和：

$$Q_{\text{total}} \approx \sum_{r=1}^{D_{\text{dp}}} Q_r$$

但 prefix cache 命中率也是按 DP rank 分开统计后再汇总：

$$R_{\text{hit},\text{total}} = \frac{\sum_r N_{\text{hit},r}}{\sum_r N_{\text{lookup},r}}$$

教学版 serving 并行门禁可以写成：

$$G_{\text{parallel}} = G_{\text{fit}} G_{\text{topo}} G_{\text{tp}} G_{\text{pp}} G_{\text{dp}} G_{\text{ep}} G_{\text{metric}}$$

其中：

1. $G_{\{\mathrm{fit}\}}$ ：权重、KV、buffer 在单卡显存预算内。
2. $G_{\{\mathrm{topo}\}}$ ：TP group 没有不必要地跨慢网络。
3. $G_{\{\mathrm{tp}\}}$ ：TP 通信没有吞掉 decode 收益。
4. $G_{\{\mathrm{pp}\}}$ ：PP stage、bubble 和跨节点激活传输可解释。
5. $G_{\{\mathrm{dp}\}}$ ：DP replica 真的提升吞吐，并考虑 prefix cache locality。
6. $G_{\{\mathrm{ep}\}}$ ：MoE expert placement 和负载不均可观测。
7. $G_{\{\mathrm{metric}\}}$ ：worker 数、显存、TPOT、bubble、throughput、cache locality 和 expert imbalance 都有指标。

下面这个 0 依赖 demo 做一个 toy serving parallel audit。它不是性能模型，只是把配置选择的关键证据显式化：7B 单卡优先 DP 扩吞吐，70B 单机 TP=8，跨节点 TP=16 被拒绝，跨两节点用 TP=8/PP=2，QPS 不够用 DP=4，MoE 场景再检查 EP expert imbalance。

```
from dataclasses import dataclass
```

```
GIB = 1024 ** 3
```

```
@dataclass
```

```
class ParallelCase:
```

```
    name: str
    model_weight_gib: float
    layers: int
    kv_heads: int
    head_dim: int
    dtype_bytes: int
    active_tokens: int
    gpu_memory_gib: float
    gpus_per_node: int
    nodes: int
    tp: int
    pp: int
    dp: int
    base_decode_ms: float
    tpot_budget_ms: float
    target_qps: float
    microbatches: int
    sticky_routing: bool = True
    ep: int = 1
    expert_loads: tuple = ()
```

```
class ToyServingParallelAuditor:
```

```
    def kv_gib_per_rank(self, case):
        layers_per_stage = case.layers / case.pp
        kv_heads_per_rank = max(1.0, case.kv_heads / case.tp)
        bytes_total = (
            2
            * layers_per_stage
            * kv_heads_per_rank
            * case.head_dim
            * case.dtype_bytes
            * case.active_tokens
        )
        return bytes_total / GIB

    def weight_gib_per_rank(self, case):
        return case.model_weight_gib / (case.tp * case.pp)

    def buffer_gib(self, case):
        return 3.0 + 0.2 * case.tp + 0.5 * max(0, case.pp - 1)

    def tp_crosses_nodes(self, case):
        return case.nodes > 1 and case.tp > case.gpus_per_node
```

```

def topology_ok(self, case):
    return not self.tp_crosses_nodes(case)

def tp_comm_ms(self, case):
    if case.tp == 1:
        return 0.0
    per_layer_ms = 0.04 if not self.tp_crosses_nodes(case) else 0.45
    return (case.layers / case.pp) * per_layer_ms * (case.tp - 1) / case.tp

def bubble_ratio(self, case):
    if case.pp == 1:
        return 0.0
    return (case.pp - 1) / (case.microbatches + case.pp - 1)

def tpot_ms(self, case):
    compute_ms = case.base_decode_ms / case.tp
    pp_transfer_ms = 0.35 * max(0, case.pp - 1)
    bubble_penalty_ms = 3.0 * self.bubble_ratio(case)
    return round(compute_ms + self.tp_comm_ms(case) + pp_transfer_ms + bubble_penalty_ms, 2)

def throughput_qps(self, case):
    return round(case.dp * 1000.0 / self.tpot_ms(case), 2)

def expert_imbalance(self, case):
    if not case.expert_loads:
        return 1.0
    avg_load = sum(case.expert_loads) / len(case.expert_loads)
    return round(max(case.expert_loads) / avg_load, 3)

def audit_case(self, case):
    memory_gib = self.weight_gib_per_rank(case) + self.kv_gib_per_rank(case) + self.buffer_gib(case)
    return {
        "name": case.name,
        "total_gpus": case.tp * case.pp * case.dp,
        "worker_group": case.tp * case.pp,
        "replicas": case.dp,
        "weight_gib_per_rank": round(self.weight_gib_per_rank(case), 3),
        "kv_gib_per_rank": round(self.kv_gib_per_rank(case), 3),
        "memory_gib_per_rank": round(memory_gib, 3),
        "memory_fit": memory_gib <= case.gpu_memory_gib * 0.9,
        "topology_ok": self.topology_ok(case),
        "tp_crosses_nodes": self.tp_crosses_nodes(case),
        "bubble_ratio": round(self.bubble_ratio(case), 3),
        "tpot_ms": self.tpot_ms(case),
        "tpot_ok": self.tpot_ms(case) <= case.tpot_budget_ms,
        "throughput_qps": self.throughput_qps(case),
        "throughput_ok": self.throughput_qps(case) >= case.target_qps,
        "prefix_locality_ok": case.dp == 1 or case.sticky_routing,
        "expert_imbalance": self.expert_imbalance(case),
        "expert_balance_ok": self.expert_imbalance(case) <= 1.35,
    }

```

```
cases = [
```

```

ParallelCase(
    name="single_gpu_7b_dp",
    model_weight_gib=14.0,
    layers=32,
    kv_heads=8,
    head_dim=128,
    dtype_bytes=2,
    active_tokens=8192,
    gpu_memory_gib=80.0,
    gpus_per_node=8,
    nodes=1,
    tp=1,
    pp=1,
    dp=8,
    base_decode_ms=34.0,
    tpot_budget_ms=40.0,
    target_qps=180.0,
    microbatches=8,
),
ParallelCase(
    name="tp8_70b_single_node",
    model_weight_gib=140.0,
    layers=80,
    kv_heads=8,
    head_dim=128,
    dtype_bytes=2,
    active_tokens=32768,
    gpu_memory_gib=80.0,
    gpus_per_node=8,
    nodes=1,
    tp=8,
    pp=1,
    dp=1,
    base_decode_ms=160.0,
    tpot_budget_ms=35.0,
    target_qps=30.0,
    microbatches=8,
),
ParallelCase(
    name="bad_tp16_cross_node",
    model_weight_gib=140.0,
    layers=80,
    kv_heads=8,
    head_dim=128,
    dtype_bytes=2,
    active_tokens=32768,
    gpu_memory_gib=80.0,
    gpus_per_node=8,
    nodes=2,
    tp=16,
    pp=1,
    dp=1,
    base_decode_ms=160.0,
    tpot_budget_ms=35.0,
    target_qps=30.0,

```

```

        microbatches=8,
    ),
    ParallelCase(
        name="tp8_pp2_two_nodes",
        model_weight_gib=300.0,
        layers=96,
        kv_heads=8,
        head_dim=128,
        dtype_bytes=2,
        active_tokens=32768,
        gpu_memory_gib=80.0,
        gpus_per_node=8,
        nodes=2,
        tp=8,
        pp=2,
        dp=1,
        base_decode_ms=240.0,
        tpot_budget_ms=45.0,
        target_qps=25.0,
        microbatches=8,
    ),
    ParallelCase(
        name="tp4_dp4_qps",
        model_weight_gib=140.0,
        layers=80,
        kv_heads=8,
        head_dim=128,
        dtype_bytes=2,
        active_tokens=24576,
        gpu_memory_gib=80.0,
        gpus_per_node=8,
        nodes=2,
        tp=4,
        pp=1,
        dp=4,
        base_decode_ms=160.0,
        tpot_budget_ms=45.0,
        target_qps=80.0,
        microbatches=16,
        sticky_routing=True,
    ),
    ParallelCase(
        name="moe_ep4_balanced",
        model_weight_gib=120.0,
        layers=48,
        kv_heads=8,
        head_dim=128,
        dtype_bytes=2,
        active_tokens=24576,
        gpu_memory_gib=80.0,
        gpus_per_node=8,
        nodes=1,
        tp=2,
        pp=1,
        dp=2,

```

```

        ep=4,
        base_decode_ms=90.0,
        tpot_budget_ms=55.0,
        target_qps=35.0,
        microbatches=12,
        expert_loads=(120, 100, 110, 115),
    ),
]

auditor = ToyServingParallelAuditor()
records = {case.name: auditor.audit_case(case) for case in cases}

summary = {
    "single_gpu_7b_dp": {
        "total_gpus": records["single_gpu_7b_dp"]["total_gpus"],
        "memory_gib_per_rank": records["single_gpu_7b_dp"]["memory_gib_per_rank"],
        "throughput_qps": records["single_gpu_7b_dp"]["throughput_qps"],
    },
    "tp8_70b_single_node": {
        "worker_group": records["tp8_70b_single_node"]["worker_group"],
        "memory_gib_per_rank": records["tp8_70b_single_node"]["memory_gib_per_rank"],
        "tpot_ms": records["tp8_70b_single_node"]["tpot_ms"],
    },
    "bad_tp16_cross_node": {
        "topology_ok": records["bad_tp16_cross_node"]["topology_ok"],
        "tp_crosses_nodes": records["bad_tp16_cross_node"]["tp_crosses_nodes"],
        "tpot_ms": records["bad_tp16_cross_node"]["tpot_ms"],
    },
    "tp8_pp2_two_nodes": {
        "total_gpus": records["tp8_pp2_two_nodes"]["total_gpus"],
        "bubble_ratio": records["tp8_pp2_two_nodes"]["bubble_ratio"],
        "memory_gib_per_rank": records["tp8_pp2_two_nodes"]["memory_gib_per_rank"],
    },
    "tp4_dp4_qps": {
        "replicas": records["tp4_dp4_qps"]["replicas"],
        "throughput_qps": records["tp4_dp4_qps"]["throughput_qps"],
        "prefix_locality_ok": records["tp4_dp4_qps"]["prefix_locality_ok"],
    },
    "moe_ep4_balanced": {
        "ep": 4,
        "expert_imbalance": records["moe_ep4_balanced"]["expert_imbalance"],
        "expert_balance_ok": records["moe_ep4_balanced"]["expert_balance_ok"],
    },
}

gates = {
    "gpu_count_formula": records["tp8_pp2_two_nodes"]["total_gpus"] == 16,
    "tp_fits_weight_and_kv": records["tp8_70b_single_node"]["memory_fit"],
    "cross_node_tp_rejected": records["bad_tp16_cross_node"]["tp_crosses_nodes"] and not records["bad_tp16_cross_node"],
    "tp_pp_topology_ok": records["tp8_pp2_two_nodes"]["topology_ok"] and records["tp8_pp2_two_nodes"]["tpot_ok"],
    "dp_scales_throughput": records["tp4_dp4_qps"]["throughput_ok"] and records["tp4_dp4_qps"]["replicas"] == 4,
    "prefix_locality_checked": records["tp4_dp4_qps"]["prefix_locality_ok"],
    "ep_imbalance_checked": records["moe_ep4_balanced"]["expert_balance_ok"],
    "metrics_ready": all("memory_gib_per_rank" in record and "tpot_ms" in record for record in records.values()),
}

```

```
gates["parallel_serving_gate"] = all(gates.values())
```

```
print("parallel_serving_summary=", summary)
```

```
print("parallel_serving_gates=", gates)
```

这份输出应当体现：

1. single_gpu_7b_dp 用 DP=8 扩吞吐，不需要先切模型。
2. tp8_70b_single_node 的 worker group 为 8，单卡显存估算能放下，TPOT 在预算内。
3. bad_tp16_cross_node 被标记为 tp_crosses_nodes=True 且 topology_ok=False，说明跨节点 TP 是反例。
4. tp8_pp2_two_nodes 用 16 张 GPU，bubble ratio 可见，TP 留在节点内，PP 跨节点。
5. tp4_dp4_qps 通过 4 个 DP replicas 提升总吞吐，并显式检查 prefix cache locality。
6. moe_ep4_balanced 检查 expert imbalance，说明 EP 不是只把 expert 分出去，还要看路由负载。
7. parallel_serving_gate=True，说明显存、拓扑、TP、PP、DP、EP 和 metrics 都能复盘。

24.25 小练习

1. 画出 TP=4、PP=1、DP=1 的 worker 拓扑，并说明每个 worker 持有什么。
2. 画出 TP=8、PP=2、DP=1 的两节点拓扑，说明为什么 TP 放在节点内。
3. 解释为什么 DP=4 后 prefix cache 命中率可能下降。
4. 假设有 16 张 GPU，一个模型需要 4 张 GPU TP 才能放下，如何配置才能部署 4 个副本？
5. 对比 PCIe-only 8 卡和 NVSwitch 8 卡在 TP 场景下的差异。
6. 设计一个排查 TPOT 变差的 checklist，至少包含通信、调度、KV cache、CPU 四类指标。

24.26 本章总结

Serving 并行的核心是先判断瓶颈。

TP 把单层张量切到多 GPU，适合单机高速互联内的大模型推理；PP 把模型层切到不同 GPU 或节点，适合单节点放不下或 TP 跨节点太贵的场景；DP 复制模型副本处理不同请求，适合横向扩吞吐；EP 把 MoE experts 分布出去，适合 expert 权重和计算成为瓶颈的 MoE 模型。

并行策略会同时影响 worker 数量、进程数量、权重显存、KV Cache 容量、prefix cache locality、通信开销、TTFT、TPOT 和 p99 延迟。真正的工程选择必须结合模型大小、GPU 拓扑、业务延迟目标和流量分布，而不是简单把并行度开大。

下一章会继续讲 vLLM 性能瓶颈、参数调优和工程坑，把前面几章的 scheduler、KV cache、prefix cache、worker 架构和并行策略统一放到压测与线上调优视角下。

第 25 章 vLLM 性能瓶颈、参数调优和工程坑

上一章讲了 Tensor Parallel、Pipeline Parallel 与 serving 并行：TP、PP、DP、EP 如何影响 worker 数量、显存分布、通信开销、吞吐和延迟。

本章做 vLLM 部分的阶段性收尾：当线上 vLLM-like 服务 TTFT 高、TPOT 抖动、吞吐低、显存爆、prefix cache 没效果、preemption 频繁、GPU 利用率低时，应该怎么拆指标、怎么判断瓶颈、怎么调参数？

一句话概括：

vLLM 调优不是盲目把 batch、并发和 GPU 数开大，而是把请求生命周期拆成 queue、tokenize、prefill、decode、KV cache、scheduler、worker、streaming 和 network，再针对瓶颈调整 token budget、KV budget、并行度、缓存和限流策略。

25.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考五类公开资料：

1. vLLM Optimization and Tuning 文档对 KV cache 空间不足、preemption、gpu_memory_utilization、max_num_batched_tokens、max_num_seqs、chunked prefill、decode / prefill 平衡和 attention backend 的说明。
2. vLLM metrics 文档对 TTFT、TPOT / inter-token latency、E2E latency、queue / prefill / decode 时间、running / waiting / swapped requests、KV cache usage、prefix cache hit rate 和 Prometheus / logging 指标的说明。
3. vLLM Automatic Prefix Caching 文档对 prefix cache hit、saved prefill、cached-free blocks、cache salt、LoRA、多模态 hash 和 eviction 的说明。
4. vLLM Parallelism and Scaling / Data Parallel / Expert Parallel 文档对 TP / PP / DP / EP 拓扑、跨节点通信、每个 DP rank 独立 KV cache 和 MoE expert parallel 的说明。
5. vLLM multimodal / production 相关文档对多模态输入 profile、processor cache、API server、streaming、部署与指标观测的公开口径。

本章只讲 vLLM-like serving 的教学版性能调优方法，不给出某个 GPU、某个模型、某个版本的通用最优参数，不替代真实压测平台、线上 SLO、NCCL / NUMA 排障、云成本模型、Kubernetes 编排、生产安全审计或业务质量评估。本章 demo 用纯 Python trace 表模拟 TTFT、TPOT、KV pressure、preemption、prefix cache locality、CPU / streaming 瓶颈和调参建议，不等同于真实性能预测。

参考资料：

1. vLLM Optimization and Tuning: <https://docs.vllm.ai/en/latest/configuration/optimization/>
2. vLLM metrics: <https://docs.vllm.ai/en/latest/design/metrics/>
3. vLLM Automatic Prefix Caching: https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/
4. vLLM Parallelism and Scaling: https://docs.vllm.ai/en/latest/serving/parallelism_scaling/
5. vLLM Data Parallel Deployment: https://docs.vllm.ai/en/latest/serving/data_parallel_deployment/
6. vLLM Expert Parallel Deployment: https://docs.vllm.ai/en/latest/serving/expert_parallel_deployment/

25.1 本章目标

读完本章，你应该能讲清：

1. vLLM-like 服务性能问题应该按哪些指标拆解。
2. TTFT 高、TPOT 高、吞吐低、显存 OOM 分别怎么排查。
3. max_num_batched_tokens、max_num_seqs、gpu_memory_utilization、max_model_len 分别影响什么。
4. chunked prefill、prefix cache、preemption、TP/PP/DP 分别如何参与调优。
5. 为什么 GPU 利用率低不一定是模型 kernel 慢。
6. 线上常见工程坑和回滚策略。
7. 面试中如何给出系统化的 vLLM 调优方案。

25.2 调优前先拒绝一个误区

常见错误是：

服务慢 -> 调大 batch -> 加 GPU -> 换量化 -> 换框架

这不是工程调优，而是碰运气。

正确顺序是：

定义指标 -> 拆请求生命周期 -> 定位瓶颈 -> 改一个参数 -> 压测对比 -> 灰度上线 -> 继续观测

调优要回答的不是“哪个参数更快”，而是：

1. 当前瓶颈在哪里？
2. 调这个参数会改善哪个指标？
3. 会牺牲哪个指标？
4. 对 p95/p99 是否有副作用？

5. 是否对不同请求长度分布都有效?

25.3 指标总览：不要只看 QPS

vLLM-like 服务至少要看这些指标。

延迟类：

1. TTFT。
2. TPOT 或 ITL。
3. E2E latency。
4. queue time。
5. prefill time。
6. decode step time。
7. streaming flush time。

吞吐类：

1. input tokens/s。
2. output tokens/s。
3. total tokens/s。
4. requests/s。
5. successful requests/s。

调度类：

1. waiting queue length。
2. running requests。
3. scheduled prefill tokens。
4. scheduled decode tokens。
5. running batch size。
6. preempted requests。

显存类：

1. total KV blocks。
2. free KV blocks。
3. used KV blocks。
4. cached-but-free blocks。
5. allocation failures。
6. GPU memory reserved/used。

缓存类：

1. prefix cache hit rate。
2. hit prefix length。
3. saved prefill tokens。
4. eviction count。
5. cache hit by tenant/model/route。

系统类：

1. GPU utilization。
2. GPU memory bandwidth。
3. CPU utilization。
4. tokenizer latency。
5. network send latency。
6. NCCL communication time。
7. error/timeout/abort rate。

如果只有平均 QPS 和平均 latency，你几乎无法判断真正瓶颈。

25.4 把请求生命周期拆开

一次请求可以拆成：

```
client send
-> gateway / API server
-> auth / rate limit / validation
-> tokenizer / chat template / multimodal processing
-> waiting queue
-> scheduler admission
-> prefill
-> first token sampling
-> decode loop
-> detokenization
-> streaming flush
-> finish / cleanup
```

对应延迟：

```
total_latency = frontend_time
                + tokenize_time
                + queue_time
                + prefill_time
                + decode_time
                + output_processing_time
                + network_time
```

TTFT 主要由这些部分组成：

$TTFT = \text{frontend_time} + \text{tokenize_time} + \text{queue_time} + \text{prefill_time} + \text{first_token_output_time}$

TPOT 主要由这些部分组成：

$TPOT = \text{decode_step_time} + \text{sampling_time} + \text{detokenize_time} + \text{streaming_flush_time}$

排查时要先判断慢在哪里，而不是直接看 GPU。

25.5 TTFT 高怎么排查

TTFT 高表示用户等第一个 token 很久。

常见原因：

1. waiting queue 堆积。
2. prompt 太长，prefill 重。
3. tokenizer 或 chat template 慢。
4. 多模态预处理慢。
5. scheduler 过度优先 decode，prefill 饥饿。
6. prefix cache 命中率低。
7. max_num_batched_tokens 太小，长 prompt 被切太碎。
8. KV blocks 不足，新请求进不来。
9. RAG、工具、安全审核等前置链路慢。

排查顺序：

1. 看 queue time 是否升高。
2. 看 input token 分布是否变长。
3. 看 prefill tokens/s 是否下降。
4. 看 prefix cache hit length 是否下降。
5. 看 waiting 请求是否因为 free blocks 不足无法调度。
6. 看 tokenizer CPU 和 API server CPU 是否满。
7. 看长 prompt 是否被 chunked prefill 多轮处理。

对应调优:

1. 提高 `max_num_batched_tokens`, 让更多 prefill tokens 能进入一轮。
2. 启用或调优 prefix cache, 减少重复 prefill。
3. 对长 prompt 使用 chunked prefill, 避免大尖刺。
4. 增加 API server 或 tokenizer 处理能力。
5. 长短请求分队列或分实例。
6. 对 RAG 输入做 rerank、压缩和上下文预算控制。

注意: 提高 `max_num_batched_tokens` 可能改善 TTFT, 但也可能让 decode TPOT 抖动, 所以必须同时看 streaming 体验。

25.6 TPOT 高怎么排查

TPOT 高表示后续 token 输出慢, 用户看到 “打字很慢”。

常见原因:

1. decode batch 太大。
2. 每轮混入太多长 prefill, 阻塞 decode。
3. KV Cache 读取效率差。
4. 上下文过长, attention 读 KV 成本高。
5. TP 通信开销过大。
6. CPU scheduler loop 或 sampling 慢。
7. detokenization 或 streaming flush 慢。
8. speculative decoding 接受率低, 额外开销抵消收益。
9. 量化 kernel 不适合当前 batch 或硬件。

排查顺序:

1. 区分 engine TPOT 和 client-observed TPOT。
2. 看 decode step time 是否增加。
3. 看每轮是否混入大 prefill。
4. 看 running batch size 和上下文长度分布。
5. 看 GPU memory bandwidth 和 KV cache block metrics。
6. 看 NCCL 通信时间, 尤其 TP 跨节点或 TP 太大时。
7. 看 detokenizer、HTTP SSE、网关和客户端接收时间。

对应调优:

1. 降低 `max_num_batched_tokens`, 减少 prefill 对 decode 的阻塞。
2. 使用 decode-first 或更严格的 chunked prefill 策略。
3. 降低 `max_num_seqs`, 减少每轮 decode 压力。
4. 优化 TP 拓扑, 把 TP group 放在 NVLink/NVSwitch 内。
5. 检查量化 kernel 和 attention backend 是否合适。
6. 对超长上下文请求限流或单独实例。

25.7 吞吐低怎么排查

吞吐低不一定等于延迟高。

常见现象:

1. GPU utilization 低。
2. output tokens/s 低。
3. QPS 上不去。
4. waiting queue 很短但 GPU 不满。
5. batch size 总是很小。

可能原因:

1. 流量不足, batch 填不满。
2. scheduler 没有有效 continuous batching。
3. max_num_seqs 太小。
4. max_num_batched_tokens 太小。
5. CPU 或 tokenizer 跟不上。
6. API server 成为瓶颈。
7. worker kernel launch 或 model runner 准备 metadata 慢。
8. TP/PP 通信让 GPU 等待。
9. 请求输出太短, decode batch 难以维持。

对应调优:

1. 提高 max_num_seqs, 让更多 running 请求同时 decode。
2. 提高 max_num_batched_tokens, 增加每轮 token 工作量。
3. 增加 API server count 或输入处理并行度。
4. 使用 DP 扩展副本吞吐。
5. 合理开启 CUDA graph/compile 优化。
6. 对流量做聚合、批处理或离线 batch serving。

但要注意: 吞吐提升可能以 p99 延迟恶化为代价。

25.8 显存 OOM 怎么排查

vLLM-like 服务的显存通常包括:

模型权重 + KV Cache + activation/workspace + CUDA graph + communication buffers + runtime overhead

常见 OOM 原因:

1. 模型权重本身太大。
2. max_model_len 太大, KV cache profile 预算过高。
3. max_num_seqs 太大, 同时活跃请求过多。
4. 长 prompt 或长输出太多。
5. prefix cache 保留过多但命中低。
6. preemption/recompute 频繁。
7. CUDA graph capture sizes 占用额外显存。
8. 多模态输入 profile 预留过大。
9. 取消、超时、失败路径没有释放 KV blocks。

对应调优:

1. 降低 max_model_len。
2. 降低 max_num_seqs。
3. 降低 max_num_batched_tokens。
4. 限制 max output tokens。
5. 提高 gpu_memory_utilization, 但要保留安全余量。
6. 使用 TP/PP 分摊权重显存。
7. 使用量化降低权重显存。
8. 调整 CUDA graph capture sizes 或启用 eager 模式换显存。
9. 限制多模态输入数量和尺寸。
10. 检查 free KV blocks 是否在请求结束后恢复。

不要只看 nvidia-smi。vLLM 可能预分配 KV cache pool, 显存不回落不一定是泄漏; 要看 free blocks、used blocks、allocation failures 和 active requests。

25.9 preemption 频繁意味着什么

vLLM 优化文档中提到, 当 KV cache 空间不足以处理所有 batched requests 时, 系统可能 preempt 某些请求, 后续 recompute。

preemption 的意义是保护系统不直接 OOM。

但频繁 preemption 是坏信号。

它通常说明：

1. KV cache budget 不够。
2. 并发设置过高。
3. 请求上下文过长。
4. max_num_batched_tokens 或 max_num_seqs 太激进。
5. prefix cache 占用过多。
6. TP/PP 设置导致每卡可用 KV 空间不足。

优化方向：

1. 提高 gpu_memory_utilization。
2. 降低 max_num_seqs。
3. 降低 max_num_batched_tokens。
4. 缩短 max_model_len。
5. 增加 TP 或 PP，让权重占用下降。
6. 对长上下文请求限流。
7. 排查 prefix cache 命中率和 eviction。

面试里要说清楚：preemption 是鲁棒性机制，不是性能优化。频繁 preemption 会伤害 E2E latency。

25.10 max_num_batched_tokens 怎么调

max_num_batched_tokens 控制每个 engine step 最多处理多少 tokens。

它同时影响 prefill 和 decode。

调大：

1. 更容易接纳长 prefill。
2. TTFT 可能变好。
3. input tokens/s 可能提升。
4. GPU 利用率可能提升。
5. 但单轮执行时间变长，decode TPOT 可能抖动。

调小：

1. 单轮延迟更稳定。
2. decode 更平滑。
3. 长 prompt 可能被 chunk 成更多轮。
4. TTFT 可能变差。
5. 吞吐可能下降。

经验判断：

1. 低延迟聊天：偏小，优先稳定 TPOT。
2. 长文档 RAG：适中，配合 prefix cache 和 chunked prefill。
3. 离线 batch 生成：偏大，优先吞吐。

vLLM 文档中也提到，chunked prefill 场景下调小 max_num_batched_tokens 可以改善 inter-token latency，调大则可能改善 TTFT 和吞吐。

25.11 max_num_seqs 怎么调

max_num_seqs 控制同时参与调度的最大 sequence 数。

调大：

1. 并发能力更高。

2. decode batch 更大。
3. GPU 利用率可能提升。
4. KV Cache 压力更大。
5. scheduler/sampling/output processing 开销更大。

调小：

1. KV Cache 压力下降。
2. p99 可能更稳定。
3. 吞吐上限下降。
4. 高峰时 queue time 可能升高。

如果 OOM 或 preemption 频繁，max_num_seqs 是最直接的保护阀之一。

如果 GPU 很空、queue 很短但 batch 很小，适当提高它可能有效。

25.12 gpu_memory_utilization 怎么调

gpu_memory_utilization 控制 vLLM 可使用 GPU 显存的比例，通常影响 KV cache 预留空间。

调大：

1. KV Cache 空间增加。
2. 可支持更多并发或更长上下文。
3. preemption 可能减少。
4. 但留给 CUDA graph、临时 buffer、通信和系统波动的余量减少。

调小：

1. 安全余量增加。
2. OOM 风险可能下降。
3. 但可用 KV blocks 减少，并发能力下降。

不要盲目设成接近 1。生产中要考虑：

1. 模型加载波动。
2. CUDA graph 额外内存。
3. NCCL buffer。
4. 多模态临时 tensor。
5. 其他进程占用。

25.13 max_model_len 和上下文预算

max_model_len 决定模型允许的最大上下文长度，也会影响 memory profiling 和容量估算。

设置过大：

1. KV Cache 预算压力增加。
2. 最大并发估算下降。
3. 长请求可能拖慢调度。
4. 用户可能无意中提交超长 prompt。

设置过小：

1. 长文档任务无法完成。
2. RAG 召回内容被截断。
3. 多轮对话上下文不足。

正确做法是按业务分层：

1. 普通聊天实例：较短上下文。
2. 长文档 RAG 实例：长上下文，单独限流。
3. 离线分析实例：更长上下文，弱化实时 SLA。

不要让所有业务共用一个最大上下文实例。

25.14 Chunked prefill 怎么看

Chunked prefill 把长 prompt prefill 拆成多个小块，和 decode 混合调度。

收益：

1. 长 prompt 不会一次性霸占 GPU。
2. decode 请求 TPOT 更稳定。
3. 更容易填充 token budget。
4. 适合长短请求混合负载。

代价：

1. 某些请求完成 prefill 需要多轮。
2. TTFT 可能受 chunk 策略影响。
3. scheduler 状态更复杂。

观察指标：

1. chunked prefill 次数。
2. 每个请求 prefill chunks 数。
3. decode step time 是否更稳定。
4. TTFT 是否恶化。
5. input tokens/s 是否提升。

调优核心是平衡 TTFT 和 TPOT。

25.15 Prefix cache 怎么调

Prefix cache 的目标是减少重复前缀 prefill。

要看这些指标：

1. hit rate。
2. hit prefix length。
3. saved prefill tokens。
4. eviction count。
5. cached-but-free blocks。
6. cache hit by tenant/model/route。
7. TTFT by hit length bucket。

命中率低的常见原因：

1. prompt 前缀实际不一致。
2. chat template 插入动态字段。
3. RAG chunk 顺序不稳定。
4. cache salt 粒度过细。
5. 请求被 DP 路由打散。
6. cache 容量太小，频繁 eviction。
7. LoRA 或多模态 hash 不一致。

调优方式：

1. 稳定 system prompt 和工具说明。
2. 固定 RAG 文档排序和版本。
3. 对相同会话或租户做 cache-aware routing。
4. 避免不必要动态字段进入 prefix。
5. 监控 prefix cache 占用和收益，命中低时考虑关闭或隔离。

25.16 并行策略怎么参与调优

TP:

1. 解决单卡放不下和权重显存压力。
2. 可给 KV cache 腾空间。
3. 但增加通信，可能伤害 TPOT。
4. 优先放在同机高速互联内。

PP:

1. 解决单节点放不下或 TP 跨节点太贵。
2. 但可能增加单请求延迟和 pipeline bubble。

DP:

1. 解决总吞吐不足。
2. 每个 rank KV cache 独立。
3. 路由会影响 prefix cache locality 和 p99。

EP:

1. 解决 MoE expert 权重和计算分布。
2. 但要关注 expert 负载不均和 routing 通信。

调优原则:

单副本放不下 -> TP/PP

单副本能跑但 QPS 不够 -> DP

MoE experts 成为瓶颈 -> EP

TPOT 被通信拖慢 -> 降低跨节点 TP 或调整拓扑

25.17 CPU 瓶颈和 NUMA 问题

vLLM V1 多进程架构下，至少有 API server、engine core 和每个 GPU worker 需要 CPU。

CPU 不足会导致:

1. tokenizer 慢。
2. chat template 慢。
3. 多模态加载慢。
4. scheduler loop 慢。
5. kernel launch 间隔变大。
6. detokenization 慢。
7. streaming flush 慢。

典型现象:

GPU utilization 不高, 但 queue time 和 TPOT 都在抖。

排查:

1. 看 API server CPU。
2. 看 engine core CPU。
3. 看 worker CPU。
4. 看 tokenization latency。
5. 看 scheduler step interval。
6. 看 NUMA 绑定是否合理。

vLLM 文档中提到, 多 socket GPU 机器上 worker 如果跑在离 GPU 远的 NUMA node 上, 性能会下降。可以通过 NUMA binding 让 worker CPU 和内存靠近对应 GPU。

25.18 Streaming 和输出链路坑

模型生成 token 不代表用户收到 token。

Streaming 链路包括：

sample token -> detokenize -> output processor -> API server -> gateway/proxy -> client

常见坑：

1. 网关缓冲 SSE。
2. gzip 或代理合并 chunk。
3. 客户端渲染太频繁。
4. detokenizer 慢。
5. logprobs 或结构化输出处理慢。
6. 网络 backpressure。
7. 客户端断开没有及时 abort。

排查方式：

1. 记录 model token time。
2. 记录 server send time。
3. 记录 client receive time。
4. 三段时间对齐。

如果 model token time 正常但 client receive time 抖动，继续优化 model runner 没意义。

25.19 多模态 serving 的特殊调优

多模态模型多了几类资源：

1. media loading CPU。
2. processor cache。
3. encoder cache。
4. vision/audio encoder activation。
5. placeholder tokens。
6. 多模态 embedding 注入。

常见问题：

1. 图片或视频过多导致预处理慢。
2. dummy profile 预留过大导致显存浪费。
3. processor cache 占用 CPU memory。
4. 多 API server 下 IPC cache 效果变化。
5. 多模态 hash 不一致导致 prefix cache 失效。

调优方式：

1. 限制每个 prompt 的图片、视频、音频数量。
2. 配置合理的多模态尺寸 hint。
3. 调整 processor cache 大小。
4. 对文本-only 请求使用文本-only 实例。
5. 分开部署多模态和纯文本流量。

25.20 量化不是只看显存

量化可以降低权重显存，但不一定提升速度。

原因：

1. kernel 不一定更快。
2. dequant 开销可能抵消收益。

3. 小 batch 下通信或调度仍是瓶颈。
4. KV Cache 没有同步变小。
5. 质量退化可能带来重试和人工修正成本。

上线量化前要测：

1. 业务真实 prompt。
2. 长上下文。
3. JSON/工具调用/结构化输出。
4. 代码和数学。
5. 安全边界。
6. TTFT、TPOT、tokens/s、显存和 p99。

量化目标是单位质量成本更低，而不是单看显存数字更小。

25.21 调优实验应该怎么做

每次只改一个主要变量。

推荐实验矩阵：

1. baseline。
2. 改 max_num_batched_tokens。
3. 改 max_num_seqs。
4. 改 gpu_memory_utilization。
5. 开关 prefix cache。
6. 开关或调整 chunked prefill。
7. 改 TP/DP 配置。
8. 改 quantization 或 attention backend。

每组都记录：

1. TTFT p50/p95/p99。
2. TPOT p50/p95/p99。
3. input/output tokens/s。
4. queue time。
5. GPU utilization。
6. free KV blocks。
7. preemption count。
8. prefix cache hit saved tokens。
9. error/timeout rate。

压测流量必须覆盖真实长度分布：短 prompt、长 prompt、短输出、长输出、RAG、多轮对话、峰值并发。

25.22 常见线上事故和处理

事故一：突然 TTFT 飙升。

处理：先看 queue、route、API server CPU、input length、prefix cache hit、RAG 前置耗时。不要先换 kernel。

事故二：TPOT 抖动，用户看到输出卡顿。

处理：看 decode step time、chunked prefill、running batch size、NCCL、streaming send/receive。必要时降低 max_num_batched_tokens。

事故三：频繁 OOM 或 allocation failure。

处理：限流长请求，降低 max_num_seqs/max_model_len，检查 KV free blocks，确认 abort cleanup，临时降级 max output tokens。

事故四：preemption 日志大量出现。

处理：降低并发和 token budget，提高 KV cache 空间，隔离长上下文流量，评估 TP/PP。

事故五：prefix cache 命中突然下降。

处理：检查 chat template、system prompt 版本、RAG 排序、cache salt、DP 路由和 LoRA/multimodal hash。

事故六：GPU 利用率低但排队严重。

处理：检查 CPU、tokenizer、engine core loop、API server、worker hang、调度是否被 KV blocks 卡住。

25.23 上线调优的安全策略

不要直接全量调参。

建议：

1. 先离线 replay 真实流量。
2. 再小流量灰度。
3. 分业务线、分 prompt 长度观察。
4. 设置 p99、OOM、error rate 自动回滚阈值。
5. 保留旧配置和旧模型副本。
6. 监控质量指标，不只监控性能。
7. 对长上下文和高价值租户单独保护。

调优可能改善平均吞吐，但伤害尾延迟或质量。生产系统要以 SLA 和成本综合判断。

25.24 面试官会怎么问

问题一：vLLM TTFT 高怎么排查？

回答要点：拆 queue、tokenizer、prefill、prefix cache、KV block admission、前置 RAG/tool 链路。看 input length、prefill tokens/s、waiting queue 和 cache hit length。

问题二：TPOT 高怎么排查？

回答要点：看 decode step time、running batch、长 prefill 是否阻塞 decode、KV cache 读取、TP 通信、detokenization 和 streaming 链路。

问题三：max_num_batched_tokens 调大有什么风险？

回答要点：可能提升 prefill 吞吐和 TTFT，但单轮执行时间变长，decode ITL/TPOT 和 p99 可能变差。

问题四：preemption 频繁说明什么？

回答要点：通常说明 KV cache 空间不足或并发/token budget 太激进。可以调高 gpu_memory_utilization，降低 max_num_seqs/max_num_batched_tokens，缩短上下文或增加 TP/PP。

问题五：GPU 利用率低一定是模型太小吗？

回答要点：不一定。可能是 CPU/tokenizer/API server/scheduler 慢，waiting queue 被 KV block 卡住，流量不足，batch 组不起来，或者 streaming/network 阻塞。

问题六：怎么做 vLLM 调优实验？

回答要点：固定真实流量分布，每次只改一个关键参数，同时记录 TTFT/TPOT/tokens/s/queue/free blocks/preemption/cache hit/error rate，并看 p95/p99，不只看平均吞吐。

25.25 标准回答模板

如果面试官问“你会如何调优 vLLM serving”，可以这样回答：

我不会先盲目调大 batch 或加 GPU，而是先把请求生命周期拆开观测。核心指标包括 TTFT、TPOT、E2E latency、queue time、prefill
 如果 TTFT 高，我会先看 queue time、input token 分布、prefill 性能、prefix cache 命中、tokenizer 和 RAG/tool 前置链路。调
 如果 TPOT 高，我会看 decode step time、running batch size、长 prefill 是否阻塞 decode、KV cache 读取、TP 通信和 streaming
 如果 OOM 或 preemption 频繁，我会看 free KV blocks、allocation failures、max_model_len、max_num_seqs、prefix cache 占用和
 最后所有调参都要基于真实流量 replay 和灰度，比较 p50/p95/p99、tokens/s、错误率和成本，而不是只看平均 QPS。

25.26 vLLM 调优公式、事故归因和可运行 demo

把调优需要的观测量先写成稳定公式。对请求 i：

$$T_{\text{ttft},i} = T_{\text{front},i} + T_{\text{tok},i} + T_{\text{queue},i} + T_{\text{prefill},i} + T_{\text{first},i}$$

$$T_{\text{tpot},i} = \frac{T_{\text{decode},i} + T_{\text{sample},i} + T_{\text{detok},i} + T_{\text{flush},i}}{\max(1, Y_i)}$$

其中 Y_i 是输出 token 数。压测窗口内的 token 吞吐可以写成：

$$Q_{\text{out}} = \frac{\sum_i Y_i}{T_{\text{window}}}$$

KV pressure 可以用 used blocks 和 total blocks 表示：

$$P_{\text{kv}} = \frac{B_{\text{used}}}{B_{\text{total}}}$$

preemption rate 可以写成：

$$R_{\text{preempt}} = \frac{N_{\text{preempt}}}{N_{\text{req}}}$$

prefix cache 的 saved prefill ratio 可以写成：

$$R_{\text{save}} = \frac{\sum_i S_i}{\sum_i X_i}$$

其中 S_i 是 prefix cache 省掉的 prefill tokens， X_i 是输入 tokens。

教学版 vLLM 调优门禁可以写成：

$$G_{\text{tune}} = G_{\text{metric}} G_{\text{ttft}} G_{\text{tpot}} G_{\text{kv}} G_{\text{cache}} G_{\text{config}} G_{\text{rollback}}$$

其中：

1. G_{metric} ：TTFT、TPOT、queue、prefill、decode、KV、cache、CPU、streaming 指标完整。
2. G_{ttft} ：能区分 queue、tokenize、prefill、prefix cache miss 和前置链路导致的 TTFT。

3. G_{tpot} : 能区分 decode、KV 读取、TP 通信、chunked prefill、sampling 和 streaming 导致的 TPOT。
4. G_{kv} : 能识别 KV pressure、allocation failure、preemption 和 cleanup 问题。
5. G_{cache} : 能解释 prefix cache hit rate、saved tokens、DP route locality 和 eviction。
6. G_{config} : 调参方案说明 max_num_batched_tokens、max_num_seqs、gpu_memory_utilization、max_model_len、TP / DP 的 trade-off。
7. G_{rollback} : 灰度、p99、OOM、error rate 和质量回滚阈值明确。

下面这个 0 依赖 demo 构造 5 条 toy request trace 和一个当前配置，模拟一次 vLLM 性能事故审计。它故意让 TTFT、TPOT、KV pressure、prefix cache 和 streaming 都有可见问题，目标不是让服务通过 SLO，而是证明审计能定位瓶颈并给出有 trade-off 的调参建议。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class RequestTrace:
```

```
    request_id: str
    input_tokens: int
    output_tokens: int
    front_ms: int
    tokenize_ms: int
    queue_ms: int
    prefill_ms: int
    first_ms: int
    decode_ms: int
    sample_ms: int
    detok_ms: int
    flush_ms: int
    kv_blocks: int
    prefix_lookups: int
    prefix_hits: int
    saved_prefill_tokens: int
    preemptions: int
    dp_rank: int
    route_key: str
    error: bool = False
```

```
@dataclass
```

```
class ServingConfig:
```

```
    name: str
    max_num_batched_tokens: int
    max_num_seqs: int
    gpu_memory_utilization: float
    max_model_len: int
    tensor_parallel_size: int
    data_parallel_size: int
    chunked_prefill: bool
    prefix_cache: bool
    sticky_routing: bool
    tp_cross_node: bool
```

```
class ToyVLLMTuningAuditor:
```

```
    def __init__(self, traces, config, total_kv_blocks, ttft_budget_ms, tpot_budget_ms):
```

```

self.traces = traces
self.config = config
self.total_kv_blocks = total_kv_blocks
self.ttft_budget_ms = ttft_budget_ms
self.tpot_budget_ms = tpot_budget_ms

def percentile(self, values, ratio):
    ordered = sorted(values)
    index = int((len(ordered) - 1) * ratio + 0.999999)
    return ordered[min(index, len(ordered) - 1)]

def per_request_metrics(self, trace):
    ttft = trace.front_ms + trace.tokenize_ms + trace.queue_ms + trace.prefill_ms + trace.first_ms
    tpot = (
        trace.decode_ms + trace.sample_ms + trace.detok_ms + trace.flush_ms
    ) / max(1, trace.output_tokens)
    e2e = ttft + trace.decode_ms + trace.sample_ms + trace.detok_ms + trace.flush_ms
    return {
        "request_id": trace.request_id,
        "ttft_ms": round(ttft, 1),
        "tpot_ms": round(tpot, 2),
        "e2e_ms": round(e2e, 1),
        "queue_ms": trace.queue_ms,
        "tokenize_ms": trace.tokenize_ms,
        "prefill_ms": trace.prefill_ms,
        "kv_blocks": trace.kv_blocks,
        "preemptions": trace.preemptions,
        "prefix_hits": trace.prefix_hits,
        "prefix_lookups": trace.prefix_lookups,
        "saved_prefill_tokens": trace.saved_prefill_tokens,
    }

def summarize(self):
    per_request = [self.per_request_metrics(trace) for trace in self.traces]
    ttfts = [item["ttft_ms"] for item in per_request]
    tpots = [item["tpot_ms"] for item in per_request]
    queues = [item["queue_ms"] for item in per_request]
    e2es = [item["e2e_ms"] for item in per_request]
    window_s = max(e2es) / 1000.0
    prefix_lookups = sum(trace.prefix_lookups for trace in self.traces)
    prefix_hits = sum(trace.prefix_hits for trace in self.traces)
    saved_tokens = sum(trace.saved_prefill_tokens for trace in self.traces)
    input_tokens = sum(trace.input_tokens for trace in self.traces)
    output_tokens = sum(trace.output_tokens for trace in self.traces)
    preemptions = sum(trace.preemptions for trace in self.traces)
    kv_used = sum(trace.kv_blocks for trace in self.traces)

    route_to_ranks = {}
    for trace in self.traces:
        route_to_ranks.setdefault(trace.route_key, set()).add(trace.dp_rank)
    scattered_routes = sorted(key for key, ranks in route_to_ranks.items() if len(ranks) > 1)

    return {
        "per_request": per_request,
        "summary": {

```

```

        "ttft_p95_ms": self.percentile(ttfts, 0.95),
        "tpot_p95_ms": self.percentile(tpots, 0.95),
        "queue_p95_ms": self.percentile(queues, 0.95),
        "output_tokens_per_s": round(output_tokens / window_s, 2),
        "kv_pressure": round(kv_used / self.total_kv_blocks, 3),
        "preemption_rate": round(preemptions / len(self.traces), 3),
        "prefix_hit_rate": round(prefix_hits / prefix_lookups, 3) if prefix_lookups else 0.0,
        "saved_prefill_ratio": round(saved_tokens / input_tokens, 3),
        "scattered_prefix_routes": scattered_routes,
        "error_rate": round(sum(1 for trace in self.traces if trace.error) / len(self.traces), 3),
    },
}

def diagnose(self, summary):
    reasons = []
    if summary["ttft_p95_ms"] > self.ttft_budget_ms and summary["queue_p95_ms"] > 500:
        reasons.append("queue_or_prefill_ttft")
    if summary["tpot_p95_ms"] > self.tpot_budget_ms:
        reasons.append("decode_or_streaming_tpot")
    if summary["kv_pressure"] > 0.75 or summary["preemption_rate"] > 0:
        reasons.append("kv_pressure_preemption")
    if summary["prefix_hit_rate"] < 0.25 and summary["saved_prefill_ratio"] < 0.05:
        reasons.append("prefix_cache_ineffective")
    if summary["scattered_prefix_routes"]:
        reasons.append("dp_route_breaks_cache_locality")
    if self.config.tp_cross_node:
        reasons.append("cross_node_tp_risk")
    if any(trace.tokenize_ms > 200 or trace.flush_ms > 180 for trace in self.traces):
        reasons.append("cpu_or_streaming_path")
    return reasons

def recommendations(self, reasons):
    actions = []
    if "queue_or_prefill_ttft" in reasons:
        actions.append("raise_or_rebalance_max_num_batched_tokens_with_replay")
        actions.append("enable_chunked_prefill_for_long_prompts")
    if "decode_or_streaming_tpot" in reasons:
        actions.append("cap_prefill_per_step_or_lower_max_num_batched_tokens")
        actions.append("profile_detokenize_and_sse_flush")
    if "kv_pressure_preemption" in reasons:
        actions.append("lower_max_num_seqs_or_max_model_len")
        actions.append("increase_gpu_memory_utilization_with_oom_guard")
    if "prefix_cache_ineffective" in reasons:
        actions.append("stabilize_prompt_prefix_and_measure_saved_tokens")
    if "dp_route_breaks_cache_locality" in reasons:
        actions.append("enable_tenant_or_session_sticky_routing")
    if "cross_node_tp_risk" in reasons:
        actions.append("keep_tp_inside_node_or_use_tp_plus_pp")
    if "cpu_or_streaming_path" in reasons:
        actions.append("scale_api_tokenizer_workers_and_check_gateway_buffering")
    return actions

def rollback_policy(self):
    return {
        "ttft_p99_ms": 3500,

```



```

        "tpot_p99_ms": 90,
        "oom_or_allocation_failures": 0,
        "error_rate": 0.01,
        "quality_regression": 0,
    }

def audit(self):
    metrics = self.summarize()
    summary = metrics["summary"]
    reasons = self.diagnose(summary)
    actions = self.recommendations(reasons)
    rollback = self.rollback_policy()
    gates = {
        "metrics_complete": all(
            {"ttft_ms", "tpot_ms", "queue_ms", "kv_blocks"}.issubset(item)
            for item in metrics["per_request"]
        ),
        "ttft_bottleneck_detected": "queue_or_prefill_ttft" in reasons,
        "tpot_bottleneck_detected": "decode_or_streaming_tpot" in reasons,
        "kv_preemption_detected": "kv_pressure_preemption" in reasons,
        "cache_issue_detected": "prefix_cache_ineffective" in reasons and "dp_route_breaks_cache_locality" in reasons,
        "config_tradeoff_ready": (
            "enable_chunked_prefill_for_long_prompts" in actions
            and "lower_max_num_seqs_or_max_model_len" in actions
            and "keep_tp_inside_node_or_use_tp_plus_pp" in actions
        ),
        "rollback_guard_ready": all(value is not None for value in rollback.values()),
    }
    gates["vllm_tuning_gate"] = all(gates.values())
    return {
        "summary": summary,
        "root_causes": reasons,
        "recommendations": actions,
        "rollback_policy": rollback,
        "gates": gates,
    }

```

```

traces = [
    RequestTrace("short_chat", 80, 20, 20, 8, 40, 60, 10, 600, 30, 20, 40, 3, 1, 1, 64, 0, 0, "tenantA"),
    RequestTrace("long_rag_a", 5000, 60, 30, 70, 900, 1700, 20, 2400, 90, 80, 120, 18, 2, 0, 0, 1, 0, "manual_v7"),
    RequestTrace("long_rag_b", 4800, 32, 30, 60, 850, 1600, 20, 1400, 60, 50, 80, 16, 2, 0, 0, 1, 1, "manual_v7"),
    RequestTrace("long_decode", 500, 120, 20, 20, 150, 180, 15, 7200, 150, 120, 300, 14, 1, 0, 0, 0, 0, "tenantB"),
    RequestTrace("cpu_streaming", 300, 30, 25, 260, 80, 120, 15, 900, 60, 80, 220, 5, 1, 0, 0, 0, 1, "tenantC"),
]

```

```

config = ServingConfig(
    name="baseline_bad_mix",
    max_num_batched_tokens=4096,
    max_num_seqs=64,
    gpu_memory_utilization=0.84,
    max_model_len=32768,
    tensor_parallel_size=16,
    data_parallel_size=2,
    chunked_prefill=False,
)

```

```

    prefix_cache=True,
    sticky_routing=False,
    tp_cross_node=True,
)

auditor = ToyVLLMTuningAuditor(
    traces=traces,
    config=config,
    total_kv_blocks=70,
    ttft_budget_ms=1500,
    tpot_budget_ms=55,
)

result = auditor.audit()

print("vllm_tuning_summary=", result["summary"])
print("vllm_tuning_root_causes=", result["root_causes"])
print("vllm_tuning_recommendations=", result["recommendations"])
print("vllm_tuning_gates=", result["gates"])

```

这份输出应当体现：

1. ttft_p95_ms=2720, queue_p95_ms=900, 说明 TTFT 问题主要落在 queue / prefill 一侧。
2. tpot_p95_ms=64.75, 说明 decode、sampling、detokenize 或 streaming 链路也在拖慢输出。
3. kv_pressure=0.8、preemption_rate=0.4, 说明 KV budget 和并发参数过激。
4. prefix_hit_rate=0.143、saved_prefill_ratio=0.006, 且 manual_v7 路由到多个 DP rank, 说明 prefix cache 没有形成有效收益。
5. root_causes 同时包含 cross_node_tp_risk 和 cpu_or_streaming_path, 说明调优不能只改一个 batch 参数。
6. recommendations 同时覆盖 chunked prefill、max_num_seqs / max_model_len、gpu_memory_utilization、sticky routing、TP 拓扑和 API / tokenizer / gateway 链路。
7. vllm_tuning_gate=True, 说明这份审计能把指标、瓶颈、配置 trade-off 和回滚阈值串起来。

25.27 小练习

1. 设计一个 vLLM dashboard, 包含 TTFT、TPOT、queue、KV blocks、prefix cache、preemption、GPU/CPU 指标。
2. 给定现象 “TTFT p99 飙升但 TPOT 正常”, 列出 8 个排查方向。
3. 给定现象 “TPOT 抖动但 TTFT 正常”, 列出 8 个排查方向。
4. 设计一组实验比较 max_num_batched_tokens=2048/4096/8192/16384 的效果。
5. 解释为什么 prefix cache 命中率高但吞吐不一定提升。
6. 设计一个长上下文流量的限流和降级策略。

25.28 本章总结

vLLM 性能调优的核心是指标拆解和 trade-off。

TTFT 主要受 queue、tokenizer、prefill、prefix cache 和前置链路影响；TPOT 主要受 decode step、KV cache 读取、通信、调度和 streaming 链路影响；吞吐受 batch 填充、token budget、GPU/CPU 协同和并行策略影响；显存则由权重、KV cache、CUDA graph、workspace、prefix cache 和多模态缓存共同决定。

max_num_batched_tokens、max_num_seqs、gpu_memory_utilization、max_model_len、chunked prefill、prefix cache、preemption、TP/PP/DP 都不是孤立参数。它们共同决定吞吐、TTFT、TPOT、显存和 p99。真正的工程能力，是能根据现象定位瓶颈，根据瓶颈选择参数，并用真实流量压测和灰度验证结果。

下一章开始进入 SGLang 架构详解，我们会对比 vLLM 的 scheduler/block manager 思路，理解 SGLang 为什么强调 RadixAttention、structured generation、runtime 和前端语言表达。

第 26 章 SGLang 解决什么问题

前面十章集中讲了 vLLM: PagedAttention、KV Cache Block Manager、continuous batching、scheduler、worker/executor、prefix cache、并行和调优。vLLM 的核心问题意识是：如何把大量独立 LLM 请求高效地调度到 GPU 上，解决 KV Cache 显存浪费、batch 动态变化和吞吐问题。

从本章开始进入 SGLang。

SGLang 也做高性能 serving，但它的出发点不完全一样。它不仅关注“单次请求如何高效生成”，还关注“复杂 LLM 程序如何高效执行”。这些程序可能包含多次 generation、分支、并行、工具调用、结构化输出、multi-turn、RAG、agent 和 self-consistency。

一句话概括：

SGLang 解决的是复杂 LLM 程序的表达和执行效率问题：前端让开发者更容易描述多步 LLM 控制流，后端 runtime 用 RadixAttention、prefix sharing、batching 和 constrained decoding 等机制减少重复计算并提高吞吐。

26.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考四类公开资料：

1. SGLang 论文《SGLang: Efficient Execution of Structured Language Model Programs》对 complex language model programs、frontend language、runtime、RadixAttention、KV cache reuse 和 compressed finite state machines for structured output decoding 的定义。
2. SGLang 官方文档首页对 SGLang 作为 high-performance serving framework、RadixAttention、prefix caching、multi-GPU parallelism、OpenAI API 兼容和生产 serving 定位的说明。
3. SGLang Frontend Language 文档对 gen、multi-turn、Python control flow、fork 并行、choices / regex constrained decoding、batching、streaming 和 multi-modal prompt 的示例口径。
4. SGLang Structured Outputs / OpenAI-Compatible API 文档对 JSON schema、regex、EBNF、structural tag、XGrammar 默认后端、OpenAI-compatible API、native API 和 offline engine 的说明。

本章只回答“SGLang 解决什么问题”和“为什么它不是另一个普通 chat endpoint”。本章不展开真实 SGLang server 参数、scheduler 细节、RadixAttention 源码、真实 grammar backend、GPU kernel、multi-node 部署、benchmark 排行或某个版本的性能结论；这些会在后续章节拆开。本章 demo 用纯 Python 模拟多步 LLM program 的 prefix sharing、分支可见性和结构化输出重试成本，只用于解释问题意识，不等同于真实 SGLang runtime。

参考资料：

1. SGLang 论文：<https://arxiv.org/abs/2312.07104>
2. SGLang 官方文档首页：<https://docs.sglang.io/index.md>
3. SGLang Frontend Language：https://docs.sglang.io/docs/references/frontend/frontend_tutorial.md
4. SGLang Structured Outputs：https://docs.sglang.io/docs/advanced_features/structured_outputs.md
5. SGLang OpenAI-Compatible APIs：https://docs.sglang.io/docs/basic_usage/openai_api.md

26.1 本章目标

读完本章，你应该能讲清：

1. SGLang 和 vLLM 的问题出发点有什么不同。
2. 为什么复杂 LLM 应用不是一次简单 generate()。
3. SGLang 为什么强调 frontend language 和 backend runtime 协同设计。
4. RadixAttention 要解决什么类型的 KV cache 复用问题。
5. structured generation 为什么是 serving runtime 的一等能力。
6. SGLang 适合哪些工作负载，不适合被误解成什么。
7. 面试中如何回答“SGLang 解决什么问题”。

26.2 从 vLLM 的问题意识说起

vLLM 解决的核心问题可以概括为：

大量请求来了，如何高吞吐、低浪费、稳定地执行 `prefill` 和 `decode`？

因此 vLLM 的代表性能力包括：

1. PagedAttention。
2. KV Cache Block Manager。
3. Continuous batching。
4. iteration-level scheduling。
5. prefix caching。
6. 多 GPU serving。
7. OpenAI-compatible API。

这些能力非常适合处理大规模在线请求：聊天、补全、RAG、批量生成、多租户 serving。

但当应用从“一次生成”变成“复杂程序”时，会出现新的问题。

例如：

1. 一个任务要调用模型多次。
2. 多个分支共享同一个 prompt 前缀。
3. 需要 self-consistency 采样多个答案再投票。
4. 需要 Tree-of-Thought 搜索。
5. 需要 ReAct agent 多轮思考、调用工具、观察结果。
6. 需要严格输出 JSON、regex 或 grammar。
7. 多轮对话和多个候选路径共享大量历史上下文。

这些场景里，瓶颈不只是单次请求的 batching，而是多次 LLM 调用之间存在大量可复用状态和控制流。

这就是 SGLang 的切入点。

26.3 复杂 LLM 程序是什么

简单 LLM 请求：

prompt -> generate answer

复杂 LLM 程序：

```
prompt A -> generate intermediate result
          -> branch into multiple candidates
          -> call tool
          -> append observation
          -> generate next step
          -> enforce JSON schema
          -> merge results
```

它不再是一次模型调用，而是一段带状态的程序。

典型例子一：self-consistency。

same question + same examples

- > sample answer 1
- > sample answer 2
- > sample answer 3
- > vote / verify

多个分支共享同一个题目和 few-shot examples。

典型例子二：Tree-of-Thought。

```
root prompt
-> thought A
    -> branch A1 / A2
-> thought B
    -> branch B1 / B2
-> evaluate branches
```

树上大量节点共享从 root 到某个中间节点的前缀。

典型例子三：Agent。

```
system prompt
-> user task
-> thought
-> action: search
-> observation
-> thought
-> action: calculator
-> observation
-> final answer
```

每一步都可能复用前面历史，并追加少量新内容。

如果 runtime 每次都从头 prefill，就会浪费大量计算。

26.4 为什么普通 serving API 不够表达这些程序

OpenAI-compatible API 很适合单次请求：

```
client.chat.completions.create(messages=[...])
```

但复杂程序需要表达：

1. generation 结果存入变量。
2. 多个 generation 并行执行。
3. 分支和 fork。
4. 条件控制流。
5. 外部工具调用。
6. 结构化输出约束。
7. 多轮状态复用。
8. 后端自动发现 prefix sharing。

如果开发者只用普通 API 手写这些逻辑，常见问题是：

1. 每次调用都重新发送完整 prompt。
2. runtime 难以知道不同调用之间的共享结构。
3. prefix cache 复用需要开发者自己设计 key。
4. 并行分支不好和 batching 结合。
5. 结构化输出约束在客户端拼凑，效率低且容易错。
6. 工具调用、多轮状态和 cache 生命周期难统一管理。

SGLang 的思路是：不要只把 LLM 当成黑盒 API，而是把 LLM 调用组织成可执行的程序，并让 runtime 看到这些结构。

26.5 SGLang 的核心思想：前后端协同设计

SGLang 论文和博客强调两个组成部分：

1. frontend language。
2. backend runtime。

Frontend language 解决表达问题。

它让开发者用更自然的方式表达：

1. **gen**: 生成一段内容并保存。
2. **fork**: 创建多个并行分支。
3. **choices**: 从固定候选中选择。
4. 约束输出格式。
5. 多步 **prompting**。
6. 工具和外部环境交互。

Backend runtime 解决执行效率问题。

它负责：

1. 自动发现共享前缀。
2. 复用 KV cache。
3. 调度多个 generation。
4. **batching**。
5. **structured decoding**。
6. 多 GPU 并行。

关键是二者协同。

如果只有前端语言，没有高效 runtime，表达很优雅但执行慢。

如果只有 runtime，没有程序结构，后端很难知道哪些 LLM 调用可以共享 cache、哪些分支可以并行、哪些输出需要约束。

26.6 RadixAttention 解决什么问题

SGLang 最有代表性的后端优化是 RadixAttention。

它解决的问题是：

复杂 LLM 程序中，不同 **generation call** 之间存在多种共享前缀，如何自动复用 KV cache？

vLLM 的 prefix cache 常常可以理解为基于 block hash 的前缀复用。

SGLang 的 RadixAttention 更强调用 radix tree 表达和管理复杂前缀共享结构。

Radix tree，也叫压缩前缀树，适合存储大量共享前缀的 token 序列。

普通 trie 可能每条边只表示一个 token。

Radix tree 的一条边可以表示一段 token 序列，因此更紧凑。

SGLang runtime 会把已经计算过的 prompt 和生成结果对应的 KV cache 保留在 radix tree 中。后续请求或后续 generation call 到来时，runtime 自动做 prefix matching，找到最长可复用前缀，避免重复 prefill。

26.7 为什么 radix tree 比单个 cache key 更适合复杂程序

如果只是固定 system prompt，普通 prefix cache 已经有用。

但复杂程序中，共享结构可能是树状的。

例如：

```
root: system prompt + task
  branch A: reasoning path A
    branch A1
    branch A2
  branch B: reasoning path B
    branch B1
    branch B2
```

每个分支都共享 root。

A1 和 A2 还共享 branch A。

B1 和 B2 共享 branch B。

如果只用“完整 prompt -> KV”作为 cache key，就很难高效表示这些中间共享。

Radix tree 可以自然表示：

```
root prefix
-> A prefix
    -> A1 suffix
    -> A2 suffix
-> B prefix
    -> B1 suffix
    -> B2 suffix
```

这正适合 Tree-of-Thought、self-consistency、multi-turn chat、agent search 等工作负载。

26.8 SGLang 为什么强调 structured generation

很多生产应用不是让模型自由写一段话，而是要求输出满足结构。

例如：

1. JSON。
2. 正则表达式。
3. EBNF grammar。
4. 固定 choices。
5. 函数调用参数。
6. SQL 片段。
7. 工具调用 schema。

如果模型输出不合法，后端还要重试、修复或人工处理。

这会增加延迟、成本和失败率。

Structured generation 的目标是：在 decoding 过程中约束 token 选择，让模型只能生成合法格式。

这不是简单的后处理。

后处理是：

模型先乱生成 -> 解析失败 -> 重试或修复

Constrained decoding 是：

每一步根据状态机或 grammar mask logits -> 只允许合法 token -> 输出天然合法

SGLang 论文中提到 compressed finite state machines 用于加速 structured output decoding。第 30 章会详细讲 constrained decoding，本章只要先理解：结构化输出是 SGLang 的重要问题意识之一。

26.9 SGLang 面向哪些 workload

SGLang 特别适合：

1. Agent。
2. ReAct。
3. Tree-of-Thought。
4. self-consistency。
5. few-shot learning。
6. multi-turn chat。
7. RAG pipeline。

8. structured extraction。
9. JSON decoding。
10. 多模态 LLM 程序。

这些 workload 的共同特点是：

1. 多次 LLM 调用。
2. 多个调用之间共享大量前缀。
3. 有控制流或并行分支。
4. 需要输出约束或工具交互。
5. 单次请求 API 难以表达执行结构。

所以 SGLang 不是只为了“把一个 chat endpoint 跑快一点”。它更关注复杂 LLM 应用的程序化表达和 runtime 优化。

26.10 SGLang 和 vLLM 的区别：先给直觉

可以粗略这样区分：

vLLM：高效服务大量 LLM 请求。

SGLang：高效执行复杂 LLM 程序。

当然二者有重叠：都需要高性能 runtime、KV cache 管理、batching、多 GPU、OpenAI-compatible API。但重点不同。

vLLM 的典型关键词：

1. PagedAttention。
2. continuous batching。
3. scheduler。
4. KV block manager。
5. serving throughput。

SGLang 的典型关键词：

1. frontend language。
2. RadixAttention。
3. automatic KV cache reuse。
4. structured generation。
5. complex LLM programs。
6. agent/reasoning/RAG。

面试中不要把 SGLang 只说成“另一个 vLLM”。更好的表达是：它是面向复杂 LLM 程序的 serving/runtime 系统，前端和后端一起设计。

26.11 为什么自动 KV cache 复用很重要

复杂程序里，prefix reuse 很难靠人工维护。

例如 self-consistency：

same prompt -> sample 10 answers

如果每个 sample 都从头 prefill，重复计算 10 次。

如果复用 prefix KV，只需要一次 prefill，然后分支 decode。

再比如 multi-turn chat：

system prompt + history turn 1 + history turn 2 + new user message

不同会话、不同分支可能共享 system prompt，部分共享历史。

再比如 few-shot learning：

same examples + different questions

few-shot examples 可能很长，但每个问题不同。

如果 runtime 自动维护 radix tree，就能捕捉这些复用机会。

这会减少：

1. prefill compute。
2. TTFT。
3. GPU cost。
4. redundant KV allocation。

26.12 SGLang 的前端语言解决什么痛点

很多 LLM 应用代码长这样：

```
prompt = build_prompt(x)
answer1 = call_llm(prompt)
prompt2 = prompt + answer1 + tool_result
answer2 = call_llm(prompt2)
```

问题是：

1. 程序结构在 Python 客户端里。
2. runtime 只看到一堆独立请求。
3. 共享前缀和并行分支不明显。
4. 输出约束分散在客户端逻辑里。

SGLang frontend 的目标是让这些结构变成 runtime 可以理解的 program。

例如：

1. generation 是一个操作。
2. fork 是一个操作。
3. choices 是一个约束。
4. 变量引用是 prompt 依赖。
5. 并行 generation 可以被 runtime 调度。

这样 runtime 就有机会做更全局的优化。

26.13 OpenAI-compatible API 和 SGLang language 的关系

SGLang 也支持 OpenAI-compatible API。

这很重要，因为生产系统往往需要兼容已有客户端和生态。

OpenAI API 适合：

1. 普通 chat/completions。
2. 迁移已有服务。
3. 和 LangChain、LlamaIndex、OpenAI SDK 兼容。
4. 简单在线 serving。

SGLang language 更适合：

1. 多步 generation。
2. 分支和并行。
3. agent 控制流。
4. structured generation。
5. 复杂 prompt program。

所以不要认为二者冲突。

可以这样理解：

OpenAI API 是服务接口。

SGLang language 是复杂 LLM 程序表达接口。

SGLang runtime 是执行这些请求和程序的后端。

26.14 SGLang 不是解决所有问题的银弹

SGLang 很强，但不能误解。

误区一：SGLang 只要打开就一定比所有框架快。

不一定。它在共享前缀、多次 generation、结构化输出、复杂程序中优势明显；如果只是短 prompt 单次生成，收益可能没那么大。

误区二：RadixAttention 等于普通 prefix cache。

不完全等价。RadixAttention 更强调 radix tree 管理复杂共享前缀结构，而不仅是单条 prompt prefix 的 hash 命中。

误区三：structured generation 只是解析 JSON。

不是。真正的 structured generation 是 decoding 时约束 token，而不是输出后再 parse。

误区四：前端语言只是语法糖。

如果 runtime 能利用程序结构做 cache reuse、并行和 batching，它就不只是语法糖。

误区五：SGLang 和 vLLM 是完全替代关系。

更准确地说，它们都属于 LLM serving/runtime 系统，但问题侧重点不同，工程实现和适用 workload 也不同。

26.15 用一个例子理解 SGLang 的价值

假设你要做一个法律文档分析 agent。

流程：

1. 读取长文档和 system prompt。
2. 从合同中抽取主体、金额、日期。
3. 对每个条款并行判断风险。
4. 对高风险条款调用外部法规检索。
5. 把检索结果追加进上下文。
6. 输出严格 JSON 报告。

这里有多个特点：

1. 长文档前缀被多步共享。
2. 多个条款分支可以并行。
3. 工具结果会追加上下文。
4. 输出必须是合法 JSON。
5. 某些中间结果需要被后续步骤引用。

如果用普通 API 手写，容易重复 prefill 长文档、多次传输完整 prompt、结构化输出失败后重试。

如果用 SGLang-like runtime，理想情况是：

1. 长文档 KV 被 RadixAttention 复用。
2. 条款分支被并行调度。
3. JSON decoding 被约束。
4. runtime 自动 batching 多个 generation。
5. 整体 TTFT、吞吐和失败率更好。

这就是 SGLang 解决的问题类型。

26.16 和本书后续章节的关系

本章只是回答 “SGLang 为什么存在”。

后面几章会具体展开：

第 27 章：SGLang Runtime 总览，讲 runtime 的整体模块。

第 28 章：RadixAttention 与 prefix sharing，讲 radix tree 如何维护 KV cache。

第 29 章：SGLang scheduler 设计，讲多请求、多分支如何调度。

第 30 章：Structured Generation 与 constrained decoding，讲 JSON/regex/grammar 约束。

第 31 章：Speculative Decoding 在 SGLang 中的角色。

第 32 章：Multi-turn、tool use 和 agent serving 支持。

第 33 章：SGLang 和 vLLM 的架构对比。

第 34 章：mini-sglang 源码学习路径。

26.17 面试官会怎么问

问题一：SGLang 解决什么问题？

回答要点：它解决复杂 LLM 程序的表达和高效执行问题，通过 frontend language 表达 generation、fork、choices、控制流和结构化输出，通过 runtime 的 RadixAttention、cache reuse、batching 等机制提高执行效率。

问题二：SGLang 和 vLLM 有什么区别？

回答要点：vLLM 更强调高效 serving 大量请求，核心是 PagedAttention、continuous batching 和 KV block 管理；SGLang 更强调复杂 LLM programs，包括多次 generation、分支、agent、structured output 和自动 KV cache reuse。

问题三：RadixAttention 为什么重要？

回答要点：复杂程序中共享前缀呈树状结构，RadixAttention 用 radix tree 自动匹配、插入和淘汰 KV cache，避免多次 LLM 调用重复 prefill。

问题四：structured generation 为什么是 runtime 能力？

回答要点：因为只靠后处理会出现解析失败和重试；runtime 在 decoding 时用状态机或 grammar mask logits，可以让输出天然满足 JSON/regex/grammar 等约束。

问题五：什么时候 SGLang 优势明显？

回答要点：多步 generation、agent、RAG、多轮对话、self-consistency、Tree-of-Thought、few-shot、结构化抽取、多模态程序等共享前缀和控制流明显的 workload。

26.18 标准回答模板

如果面试官问 “SGLang 解决什么问题”，可以这样回答：

SGLang 主要解决复杂 LLM 程序的表达和执行效率问题。很多 LLM 应用不是一次简单的 chat completion，而是包含多次 generation、

SGLang 的思路是前后端协同设计。前端提供嵌入 Python 的结构化生成语言，用 gen、fork、choices 等原语表达 LLM 程序；后端 ru

其中 RadixAttention 用 radix tree 管理 token prefix 到 KV cache 的映射，适合 self-consistency、Tree-of-Thought、multi-turn chat、agent 和 few-shot learning 这类树状共享前缀场景。相比只把 SGLang 看成另一个 serving server，更

26.19 SGLang 动机公式、prefix sharing 和可运行 demo

把复杂 LLM 应用看成一组 generation call:

$$c_i = (X_i, Y_i, b_i, s_i)$$

其中 X_i 是第 i 次调用的输入 token 数, Y_i 是输出 token 数, b_i 表示它属于哪个分支, s_i 表示是否需要结构化输出约束。

如果普通客户端每次都把完整 prompt 作为独立请求发送, prefill token 工作量近似为:

$$P_{\text{naive}} = \sum_i X_i$$

如果 runtime 能看到程序结构, 并用 radix tree 找到第 i 次调用可复用的最长前缀 H_i , 实际需要重新 prefill 的 token 数可以写成:

$$P_{\text{radix}} = \sum_i \max(0, X_i - H_i)$$

prefix sharing 的节省比例可以写成:

$$R_{\text{reuse}} = \frac{P_{\text{naive}} - P_{\text{radix}}}{\max(1, P_{\text{naive}})}$$

如果结构化输出靠后处理, 解析失败会触发重试。把无约束输出的重试 token 成本记为 C_{retry} , 把 constrained decoding 的额外 grammar 开销记为 C_{grammar} , 则教学版总工作量可以粗略写成:

$$W_{\text{naive}} = P_{\text{naive}} + \sum_i Y_i + C_{\text{retry}}$$

$$W_{\text{sglang}} = P_{\text{radix}} + \sum_i Y_i + C_{\text{grammar}}$$

整体工作量下降比例为:

$$R_{\text{work}} = \frac{W_{\text{naive}} - W_{\text{sglang}}}{\max(1, W_{\text{naive}})}$$

教学版 SGLang 动机门禁可以写成:

$$G_{\text{sglang}} = G_{\text{program}} G_{\text{radix}} G_{\text{branch}} G_{\text{structured}} G_{\text{api}} G_{\text{metric}}$$

其中:

1. G_{program} : 能把任务表达成多次 generation、变量、分支、并行和结构化输出, 而不是一堆互不相关的 API 请求。
2. G_{radix} : 能计算 naive prefill、radix prefill、最长可复用前缀和 saved prefill tokens。
3. G_{branch} : 能看见 self-consistency、Tree-of-Thought、agent 或 RAG 分支里的共享 root 和中间节点。
4. $G_{\text{structured}}$: 能解释 JSON / regex / EBNF constrained decoding 为什么减少后处理重试。
5. G_{api} : 能同时说明 OpenAI-compatible API 与 SGLang frontend language 的关系。
6. G_{metric} : 能输出 reuse ratio、retry saving、work reduction 和最终动机门禁。

下面这个 0 依赖 demo 模拟一个法律合同分析 program。它有 6 次 generation：先抽取字段，再对 A / B 两个条款分支分别做风险判断和法规检查，最后输出严格 JSON 报告。普通 API 会把 6 个完整 prompt 都重新 prefill；SGLang-like runtime 能让后端看见共享文档 root、A 分支和 B 分支，从而减少重复 prefill，并用结构化输出约束避免 JSON 失败重试。

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
```

```
class ProgramCall:
```

```
    name: str
```

```
    prompt_tokens: tuple
```

```
    output_tokens: int
```

```
    branch: str
```

```
    structured: bool = False
```

```
class ToyRadixPrefixCache:
```

```
    def __init__(self):
```

```
        self._stored_prompts = []
```

```
    @staticmethod
```

```
    def _common_prefix_len(left, right):
```

```
        size = min(len(left), len(right))
```

```
        count = 0
```

```
        while count < size and left[count] == right[count]:
```

```
            count += 1
```

```
        return count
```

```
    def longest_prefix(self, tokens):
```

```
        best = 0
```

```
        best_owner = None
```

```
        for owner, stored in self._stored_prompts:
```

```
            hit = self._common_prefix_len(tokens, stored)
```

```
            if hit > best:
```

```
                best = hit
```

```
                best_owner = owner
```

```
        return best, best_owner
```

```
    def insert(self, owner, tokens):
```

```
        self._stored_prompts.append((owner, tokens))
```

```
class ToySGLangMotivationAudit:
```

```
    def __init__(self, calls, root_len, invalid_json_retries, grammar_overhead_tokens):
```

```
        self.calls = list(calls)
```

```
        self.root_len = root_len
```

```
        self.invalid_json_retries = invalid_json_retries
```

```
        self.grammar_overhead_tokens = grammar_overhead_tokens
```

```
    def run(self):
```

```
        cache = ToyRadixPrefixCache()
```

```
        reuse_rows = []
```

```
        naive_prefill = 0
```

```
        radix_prefill = 0
```

```
        decode_tokens = 0
```

```

branch_hits = {}

for call in self.calls:
    prompt_len = len(call.prompt_tokens)
    hit_tokens, hit_owner = cache.longest_prefix(call.prompt_tokens)
    run_tokens = prompt_len - hit_tokens
    cache.insert(call.name, call.prompt_tokens)

    naive_prefill += prompt_len
    radix_prefill += run_tokens
    decode_tokens += call.output_tokens
    if hit_tokens >= self.root_len:
        branch_hits[call.branch] = branch_hits.get(call.branch, 0) + 1

    reuse_rows.append(
        {
            "name": call.name,
            "input_tokens": prompt_len,
            "hit_tokens": hit_tokens,
            "hit_owner": hit_owner,
            "run_prefill_tokens": run_tokens,
        }
    )

final_call = self.calls[-1]
retry_tokens_without_constraints = (
    self.invalid_json_retries * final_call.output_tokens
)
naive_work = naive_prefill + decode_tokens + retry_tokens_without_constraints
sglang_work = radix_prefill + decode_tokens + self.grammar_overhead_tokens
saved_prefill = naive_prefill - radix_prefill

summary = {
    "naive_prefill_tokens": naive_prefill,
    "radix_prefill_tokens": radix_prefill,
    "saved_prefill_tokens": saved_prefill,
    "reuse_ratio": round(saved_prefill / max(1, naive_prefill), 3),
    "retry_tokens_without_constraints": retry_tokens_without_constraints,
    "grammar_overhead_tokens": self.grammar_overhead_tokens,
    "naive_work_tokens": naive_work,
    "sglang_work_tokens": sglang_work,
    "work_reduction": round((naive_work - sglang_work) / max(1, naive_work), 3),
}
branch_summary = {
    "branches": sorted(set(call.branch for call in self.calls)),
    "root_shared_calls": sum(
        1 for row in reuse_rows if row["hit_tokens"] >= self.root_len
    ),
    "branch_hits": branch_hits,
    "structured_calls": [call.name for call in self.calls if call.structured],
}
gates = {
    "frontend_exposes_program": len(self.calls) >= 4
    and len(branch_summary["branches"]) >= 3,
    "radix_reduces_prefill": summary["saved_prefill_tokens"] > 0,
}

```

```

        "branch_sharing_visible": branch_summary["root_shared_calls"] >= 4,
        "structured_retry_saving": retry_tokens_without_constraints
    > self.grammar_overhead_tokens,
        "openai_api_boundary_clear": True,
        "metrics_ready": summary["work_reduction"] > 0.3,
    }
    gates["sglang_motivation_gate"] = all(gates.values())
    return reuse_rows, branch_summary, summary, gates

```

```

ROOT = tuple(f"doc_root_{i}" for i in range(8))
CLAUSE_A = ("clause_a", "risk", "scope")
CLAUSE_B = ("clause_b", "payment", "scope")

```

```

program_calls = [
    ProgramCall("extract_fields", ROOT + ("extract", "fields"), 40, "root"),
    ProgramCall("clause_a_risk", ROOT + CLAUSE_A, 35, "clause_a"),
    ProgramCall("clause_a_law_check", ROOT + CLAUSE_A + ("law", "check"), 30, "clause_a"),
    ProgramCall("clause_b_risk", ROOT + CLAUSE_B, 35, "clause_b"),
    ProgramCall("clause_b_law_check", ROOT + CLAUSE_B + ("law", "check"), 30, "clause_b"),
    ProgramCall(
        "final_json_report",
        ROOT + ("merge", "risk", "law", "json"),
        60,
        "report",
        structured=True,
    ),
]

```

```

audit = ToySGLangMotivationAudit(
    calls=program_calls,
    root_len=len(ROOT),
    invalid_json_retries=3,
    grammar_overhead_tokens=12,
)
reuse_rows, branch_summary, summary, gates = audit.run()

```

```

print("sglang_motivation_reuse=", reuse_rows)
print("sglang_motivation_branches=", branch_summary)
print("sglang_motivation_summary=", summary)
print("sglang_motivation_gates=", gates)

```

这段 demo 应该输出：

```

sglang_motivation_summary= {'naive_prefill_tokens': 70, 'radix_prefill_tokens': 24, 'saved_prefill_tokens': 46, 'reuse_
sglang_motivation_gates= {'frontend_exposes_program': True, 'radix_reduces_prefill': True, 'branch_sharing_visible': Tr

```

读这段代码时要注意三点：

1. $P_{\text{naive}}=70$ 说明普通 API 把 6 次调用都当成独立 prompt，长文档 root 被重复 prefill。
2. $P_{\text{radix}}=24$ 说明 radix prefix matching 能复用 root、A 分支和 B 分支的中间前缀，真正新 prefill 的只是各分支追加部分。
3. $C_{\text{retry}}=180$ 与 $C_{\text{grammar}}=12$ 的对比说明 constrained decoding 的价值不只是“格式好看”，而是减少解析失败后的重试工作量和尾延迟。

26.20 小练习

1. 举一个普通 chat completion 以外的复杂 LLM 程序，并画出它的多次 generation 调用关系。
2. 解释 self-consistency 为什么适合 KV cache prefix reuse。
3. 画一个 Tree-of-Thought 的 prefix sharing 树。
4. 对比 vLLM prefix cache 和 SGLang RadixAttention 的直觉差异。
5. 解释为什么 JSON 输出失败后重试不如 constrained decoding。
6. 设计一个适合 SGLang 的 agent serving 场景，并说明哪些部分可以复用 KV cache。

26.21 本章总结

SGLang 的核心问题意识是：复杂 LLM 应用正在从单次 prompt-response 变成多步、有状态、有控制流、有结构化约束的 LLM programs。

如果 runtime 只看到一堆独立请求，就很难自动复用这些程序中的共享前缀，也很难高效处理分支、并行、工具调用和结构化输出。SGLang 通过 frontend language 表达程序结构，通过 backend runtime 做 RadixAttention、KV cache reuse、batching 和 constrained decoding，从而提升复杂 workload 的执行效率和可控性。

下一章会进入 SGLang Runtime 总览，具体拆解 SGLang server、runtime、scheduler、memory pool、RadixAttention、sampling 和 structured output 之间如何协作。

第 27 章 SGLang Runtime 总览

上一章讲了 SGLang 解决什么问题：它不是只把一次 generate() 跑快，而是让复杂 LLM programs 的表达和执行更高效。

从本章开始，我们进入 SGLang runtime 的内部结构。

如果只从外部使用 SGLang，你看到的可能是：

```
python -m sglang.launch_server --model-path <MODEL>
```

或者：

```
import sglang as sgl
```

```
llm = sgl.Engine(model_path="<MODEL>")
outputs = llm.generate(prompts, sampling_params)
```

但从内部看，SGLang runtime 要完成一整条复杂链路：接收请求、解析参数、tokenize、匹配 prefix cache、分配 KV cache、调度 prefill/decode、执行模型、采样、结构化约束、streaming、释放或保留 cache。

一句话概括：

SGLang Runtime 是把 OpenAI-compatible API、native /generate、offline engine 和 SGLang frontend 程序统一执行到高性能模型引擎上的系统层；它的核心模块包括入口服务、请求状态、scheduler、RadixAttention、memory pool、model runner、sampler、grammar backend 和 streaming 输出。

27.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考七类公开资料：

1. SGLang 论文《SGLang: Efficient Execution of Structured Language Model Programs》对 frontend language、runtime、RadixAttention、KV cache reuse 和 compressed finite state machines for structured output decoding 的系统分层。
2. SGLang 官方文档首页对 SGLang 作为高性能 serving framework、RadixAttention、prefix caching、multi-GPU parallelism、OpenAI API 兼容和生产级 serving 的定位说明。

3. SGLang Server Arguments 文档对 `launch_server`、`model / tokenizer`、HTTP server、TP / DP、`mem-fraction-static`、`chunked prefill`、`chat template`、`attention backend` 和 `metrics / logging` 等启动配置的说明。
4. SGLang Sampling Parameters 文档对 `/generate endpoint`、`text`、`input_ids`、`sampling_params`、`stream`、`max_new_tokens`、`temperature`、`top-p`、`top-k`、`stop`、`penalty`、JSON schema、`regex`、EBNF 和 `structural tag` 的说明。
5. SGLang Offline Engine API 文档对不经过 HTTP server 的 batch inference、`sync / async`、`streaming / non-streaming` 和 `custom server` 用法的说明。
6. SGLang Structured Outputs 文档对 JSON schema、`regex`、EBNF、`structural tag`、XGrammar / Outlines / `llguidance grammar backend` 和 `OpenAI-compatible / native / offline engine` 三类使用方式的说明。
7. SGLang Attention Backend 文档对 `attention backend` 选择、`prefill / decode forward`、`page size`、`KV cache dtype`、CUDA graph 和平台差异的公开口径。

本章只讲 SGLang Runtime 的教学版总览，不绑定某个 SGLang 版本的内部类名、真实源码路径、进程拓扑、CUDA kernel、Ray / Kubernetes 部署、PD 分离、MoE 路由、speculative decoding 或真实 benchmark。下面的 demo 只模拟请求状态、`prefix lookup`、`KV slot`、`prefill/decode` 调度、`grammar mask` 和 `streaming` 事件，用来训练架构解释，不等同于真实 SRT。

参考资料：

1. SGLang 论文: <https://arxiv.org/abs/2312.07104>
2. SGLang 官方文档首页: <https://docs.sglang.io/index.md>
3. SGLang Server Arguments: https://docs.sglang.io/docs/advanced_features/server_arguments.md
4. SGLang Sampling Parameters: https://docs.sglang.io/docs/basic_usage/sampling_params.md
5. SGLang Offline Engine API: https://docs.sglang.io/docs/basic_usage/offline_engine_api.md
6. SGLang Structured Outputs: https://docs.sglang.io/docs/advanced_features/structured_outputs.md
7. SGLang Attention Backend: https://docs.sglang.io/docs/advanced_features/attention_backend.md

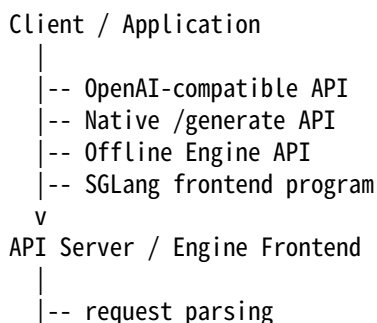
27.1 本章目标

读完本章，你应该能讲清：

1. SGLang Runtime 在整个 SGLang 系统中的位置。
2. OpenAI-compatible API、native API、offline engine 和 frontend language 的关系。
3. 一个请求在 SGLang Runtime 内部会经过哪些模块。
4. Scheduler 为什么要同时理解 `prefill`、`decode`、`cache` 和 `grammar`。
5. RadixAttention、memory pool、model runner、sampler 分别负责什么。
6. Structured output 在 runtime 中插入在哪里。
7. 面试中如何画出 SGLang Runtime 总体架构图。

27.2 先给一张总览图

SGLang Runtime 可以先按下面这张图理解：



```

|-- chat template / tokenize
|-- sampling params
|-- structured output constraints
V
Runtime Scheduler
|
|-- waiting queue
|-- running batch
|-- prefill/decode scheduling
|-- cache-aware scheduling
V
Memory and Cache Layer
|
|-- KV cache memory pool
|-- request-to-token mapping
|-- RadixAttention prefix tree
|-- eviction / reuse
V
Model Execution Layer
|
|-- model runner
|-- attention backend
|-- tensor parallel / data parallel
|-- CUDA graph / kernels
V
Sampling and Output Layer
|
|-- logits processing
|-- temperature / top-p / top-k
|-- grammar mask / constrained decoding
|-- detokenization
|-- streaming response

```

这张图有两个重点。

第一，SGLang Runtime 不只是 HTTP server。HTTP server 只是入口，真正复杂的是后面的调度、cache、模型执行和采样。

第二，SGLang Runtime 也不只是 RadixAttention。RadixAttention 是非常关键的 cache reuse 机制，但它必须和 scheduler、memory pool、model runner、sampler 配合，才能真正提升端到端吞吐和延迟。

27.3 Runtime 在 SGLang 系统中的位置

SGLang 可以分成两层来看：

Frontend language / API layer

-> 负责表达任务

Backend runtime / SRT

-> 负责高效执行任务

前端层关注：

1. 用户如何写 prompt。
2. 如何表达多次 gen。
3. 如何表达 fork、choices、并行和控制流。
4. 如何表达 JSON、regex、EBNF、tool call 等结构化约束。
5. 如何接入 OpenAI-compatible client。

后端 runtime 关注：

1. 怎么把请求合并成 batch。
2. 怎么减少重复 prefill。
3. 怎么管理 KV cache 显存。
4. 怎么调度 prefill 和 decode。
5. 怎么调用模型 forward。
6. 怎么采样下一个 token。
7. 怎么保证 structured output 约束。
8. 怎么把 token 流式返回给用户。

因此，SGLang 的关键设计不是“前端语法好用”或“后端 serving 快”二选一，而是前后端协同。

如果前端能表达复杂程序，但后端看不到共享前缀和并行关系，性能很难好。

如果后端很快，但前端只能提交一堆互相独立的字符串请求，runtime 也很难知道这些请求之间有哪些可以复用。

27.4 SRT 是什么

SGLang Runtime 常被简称为 SRT，可以理解为 SGLang 的 serving runtime。

它承担的职责包括：

1. 启动模型服务。
2. 加载模型权重和 tokenizer。
3. 暴露 OpenAI-compatible API 和 native API。
4. 支持 offline engine 直接调用。
5. 接收文本、token ids、多模态输入等请求。
6. 维护请求状态。
7. 做 continuous batching。
8. 管理 KV cache 和 prefix cache。
9. 执行模型 forward。
10. 采样 token。
11. 支持 structured output。
12. 支持 streaming。
13. 输出指标和日志。

注意这里的 runtime 和“Python 解释器运行一段 SGLang frontend 函数”不是同一个层级。

Frontend language 可以生成很多后端请求，SRT 负责把这些请求高效跑在 GPU 上。

27.5 四种入口：OpenAI API、native API、offline engine、frontend language

SGLang Runtime 常见有四类入口。

第一类是 OpenAI-compatible API。

这类入口适合把 SGLang 当成替代 OpenAI API 的本地服务：

```
client.chat.completions.create(...)
-> /v1/chat/completions
-> SGLang server
```

它的优点是生态兼容好，应用层迁移成本低。

第二类是 native /generate API。

它更接近 runtime 的底层能力，常见请求形态是：

```
{
  "text": "The capital of France is",
  "sampling_params": {
```

```

    "temperature": 0,
    "max_new_tokens": 32
  },
  "stream": true
}

```

native API 可以直接传 text、input_ids、sampling_params、stream、structured output 参数等。

第三类是 offline engine。

它不需要 HTTP server，直接在 Python 进程中调用：

```

llm = sgl.Engine(model_path="qwen/qwen2.5-0.5b-instruct")
outputs = llm.generate(prompts, sampling_params)

```

offline engine 适合：

1. 离线批处理。
2. 数据生成。
3. evaluation。
4. 自定义 server。
5. 不想引入 HTTP 开销的场景。

第四类是 SGLang frontend language。

它面向复杂 LLM programs，比如 gen、fork、choices、多轮状态和工具交互。前端程序会把多步生成任务提交给 runtime 执行，runtime 负责复用 cache 和调度。

这四类入口看起来不同，但最后都会落到 runtime 的核心执行链路。

27.6 请求进入 runtime 后发生什么

一个文本生成请求进入 SGLang Runtime 后，大致会经历这些步骤：

1. API 接收请求
2. 解析 prompt、messages、sampling 参数
3. 应用 chat template
4. tokenize 或接收 input_ids
5. 构造 request state
6. 进入 waiting queue
7. scheduler 选择请求
8. RadixAttention 查找可复用 prefix
9. memory pool 分配 KV cache 空间
10. model runner 执行 prefill
11. sampler 采样第一个 token
12. 请求进入 decode 阶段
13. scheduler 每轮组织 running batch
14. model runner 执行 decode forward
15. sampler 采样下一个 token
16. detokenizer / streamer 返回增量文本
17. 命中 stop 条件后结束
18. 释放或保留 KV cache

这条链路和 vLLM-like runtime 很像，但 SGLang 有几个额外重点：

1. 更强调跨请求、跨 generation 调用的 prefix cache reuse。
2. 更强调复杂程序产生的并行和分支。
3. 更强调 structured output 是 runtime decoding 的一部分。
4. 更强调 native API、offline engine、frontend language 共享同一套执行后端。

27.7 Request state: 请求不是一段字符串

在 runtime 内部，请求不是简单的字符串，而是一份动态状态。

它至少包含：

1. request id。
2. 原始文本或 messages。
3. token ids。
4. prompt 长度。
5. 已生成 token 列表。
6. sampling parameters。
7. stop 条件。
8. 是否 stream。
9. structured output 约束。
10. 当前阶段：waiting、prefill、decode、finished。
11. 已分配的 KV cache 位置。
12. prefix cache 命中信息。
13. 输出缓冲区。
14. 错误或取消状态。

为什么要维护这么多状态？

因为 decode 是逐 token 进行的。一个请求不会一次 forward 完就结束，而是在很多 scheduler iteration 中不断被调度、生成、暂停、继续和最终释放。

可以把请求理解成一个小状态机：

```
WAITING
-> PREFILLING
-> DECODING
-> FINISHED
```

也可能：

```
WAITING
-> PREFILLING
-> PREEMPTED / WAITING
-> DECODING
-> CANCELLED
```

SGLang 的复杂之处在于：这个状态机还要和 prefix cache、grammar state、frontend program state 结合。

27.8 Tokenization 和 chat template

OpenAI-compatible API 通常接收的是 messages：

```
[
  {"role": "system", "content": "You are helpful."},
  {"role": "user", "content": "Explain KV cache."}
]
```

但模型真正执行的是 token ids。

中间通常要经过：

```
messages
-> chat template
-> prompt text
-> tokenizer
-> input_ids
```

这个阶段看似普通，但对 cache reuse 很重要。

如果同一个对话历史，因为 chat template、空格、特殊 token、system prompt 拼接方式不同，生成出的 token 序列不同，那么 prefix cache 就无法命中。

所以线上使用 SGLang 或任何 prefix-cache runtime 时，要注意：

1. 同一模型使用稳定的 chat template。
2. system prompt 不要频繁注入随机字段。
3. RAG prompt 的固定部分尽量固定。
4. few-shot examples 的顺序和格式保持一致。
5. 如果直接传 input_ids，要确保上层 tokenizer 和 runtime tokenizer 语义一致。

Prefix cache 的单位不是字符串相似，而是 token prefix 完全匹配。

27.9 Sampling params: 生成行为如何进入 runtime

SGLang 的 sampling 参数会影响每一步 decode 的 token 选择。

常见参数包括：

1. max_new_tokens。
2. temperature。
3. top_p。
4. top_k。
5. min_p。
6. stop。
7. stop_token_ids。
8. frequency_penalty。
9. presence_penalty。
10. repetition_penalty。
11. json_schema。
12. regex。
13. ebnf。
14. structural_tag。

这些参数不是只在最后处理文本时用，而是会影响 runtime 内部执行。

例如：

1. max_new_tokens 决定请求最多占用多少 decode 轮次和 KV cache。
2. temperature/top_p/top_k 决定 sampler 如何从 logits 采样。
3. stop 和 stop_token_ids 决定何时终止请求。
4. json_schema/regex/ebnf 会引入 grammar state 和 logits mask。
5. stream 决定每轮生成后是否要把增量输出推给客户端。

所以 scheduler 做决策时，不能只看 prompt 长度，也要考虑输出长度预算、结构化约束和请求是否接近完成。

27.10 Scheduler 的核心职责

Scheduler 是 runtime 的中枢。

它要回答的问题是：

下一次模型 forward，应该跑哪些请求、跑哪些 token、用多少 cache、是否允许新请求进入？

在 SGLang Runtime 中，scheduler 通常要处理：

1. 等待队列中的新请求。
2. 已经完成 prefill、正在 decode 的请求。
3. 长 prompt prefill 对 decode 的影响。
4. KV cache 剩余空间。

5. prefix cache 命中和复用。
6. batch token budget。
7. batch sequence budget。
8. structured output 的额外开销。
9. streaming 的输出节奏。
10. 请求取消和异常。

可以把 scheduler 想成一个资源分配器：

GPU 计算预算

KV cache 显存预算

每轮 batch 大小预算

请求延迟预算

cache reuse 机会

它要在这些预算之间做折中。

27.11 Prefill 调度

Prefill 调度处理新请求的输入 prompt。

如果没有 prefix cache, prompt 有多少 token, prefill 就要计算多少 token。

如果命中 RadixAttention, 则只需要计算未命中的 suffix。

例如：

cached prefix: [A B C D]
new prompt: [A B C D E F]

需要 prefill: [E F]

这对 TTFT 非常关键。

一个长 prompt 如果能命中大段前缀, 首 token 延迟会明显下降, GPU 计算也减少。

Prefill 调度要注意几个问题：

1. 长 prompt 不能无限占用一次 iteration。
2. cache 命中请求和 cache 未命中请求的成本不同。
3. 多个新请求可以组成 batched prefill。
4. 长 prefill 可能影响正在 decode 的请求 TPOT。
5. chunked prefill 或类似机制可以缓解长 prompt 阻塞。

SGLang 的 cache-aware scheduling 会尽量利用 RadixAttention 带来的命中机会, 但仍然要受显存和 token budget 限制。

27.12 Decode 调度

Decode 调度处理已经进入生成阶段的请求。

每轮 decode 通常每个请求只新增一个 token：

running requests: R1, R2, R3, R4
this step input: last token of each request
output: next token for each request

Decode 阶段的挑战是动态性很强：

1. 有的请求很快遇到 stop。
2. 有的请求要生成很长。
3. 有的请求需要 grammar mask。
4. 有的请求客户端断开。

5. 有的新请求正在等待 prefill。
6. 每个请求的上下文长度不断增长。

Continuous batching 的核心就是：每一轮都重新组织 batch，让完成的请求退出，让新的请求进入，尽量保持 GPU 忙碌。

SGLang Runtime 也需要类似的 iteration-level scheduling，只是它还要额外考虑 prefix cache、structured output、frontend program 产生的多分支请求。

27.13 Memory pool: KV cache 从哪里来

LLM serving 中，KV cache 是最重要的动态显存资源。

SGLang Runtime 需要为请求分配 KV cache 空间，用来保存每层 attention 的 key/value。

从概念上看，memory pool 要做这些事：

1. 初始化时预留一批 KV cache 显存。
2. 为新 token 分配 cache slot。
3. 建立 request token 到 cache slot 的映射。
4. 在 decode 时追加新 token 的 KV。
5. 请求结束后释放不需要保留的 cache。
6. 对可复用 prefix，保留 cache 并挂到 prefix tree。
7. 显存不足时触发 eviction 或调度限制。

普通请求结束后，KV cache 可以释放。

但 SGLang 的 RadixAttention 会尽量保留有价值的 prefix cache。这意味着“请求结束”不等于“所有 KV 立刻释放”。

这也是 SGLang 和简单 KV cache 管理不同的地方：

普通 serving:

请求结束 -> KV cache 释放

SGLang with RadixAttention:

请求结束 -> 有价值的 prefix 可能保留 -> 后续请求复用 -> 显存不足时再淘汰

因此 memory pool 必须和 RadixAttention 的 eviction 策略协同。

27.14 RadixAttention 在 runtime 中的位置

RadixAttention 不是 attention kernel 本身，而是一套自动 KV cache reuse 机制。

它在 runtime 中大致位于：

tokenized prompt

- > prefix tree lookup
- > find longest cached prefix
- > reuse matched KV cache
- > prefill only unmatched suffix
- > insert new prompt/output into tree
- > evict when memory pressure exists

RadixAttention 管理的是：

token prefix -> KV cache tensors / cache slots

为什么用 radix tree?

因为复杂 LLM programs 的共享关系经常是树状的：


```
system prompt
-> few-shot examples
    -> question 1
    -> question 2
    -> question 3
```

```
chat history
-> branch answer A
-> branch answer B
-> branch answer C
```

```
tree-of-thought root
-> thought path 1
-> thought path 2
    -> subpath 2.1
    -> subpath 2.2
```

如果只做简单 exact prompt cache, 很难覆盖这些部分共享。

Radix tree 可以高效做 longest prefix match, 并在新请求插入时拆分节点, 让多个请求共享公共前缀。

27.15 Model runner 负责什么

Model runner 是真正执行模型 forward 的模块。

它负责把 scheduler 选出的 batch 转成模型可执行的张量, 然后调用 attention、MLP、norm、logits head 等计算。

概念上包括:

1. 准备 input ids 或 input embeddings。
2. 准备 position ids。
3. 准备 attention metadata。
4. 准备 KV cache 读写位置。
5. 调用 transformer forward。
6. 取出最后位置 logits。
7. 处理多 GPU 通信。
8. 返回 logits 给 sampler。

Prefill 和 decode 对 model runner 的要求不同。

Prefill 输入是多个 prompt token, 计算更大:

[prompt token 1, prompt token 2, ..., prompt token N]

Decode 输入通常是每个请求一个新 token:

[last_token_of_R1, last_token_of_R2, ..., last_token_of_Rk]

所以 model runner 需要支持两类 metadata:

1. prefill metadata。
2. decode metadata。

并且它要和 attention backend 约定好 KV cache 的布局和索引方式。

27.16 Attention backend 和 kernel

Runtime 的性能很大程度取决于 attention backend。

Attention backend 要解决:

1. prefill attention 怎么算。

2. decode attention 怎么读 KV cache。
3. 不同请求长度如何组织。
4. KV cache layout 如何寻址。
5. 多模态 token 是否有特殊处理。
6. 是否支持 flash attention、paged attention、flashinfer 等实现。
7. 是否支持 CUDA graph 或其他图捕获优化。

对学习者来说，不需要一开始就记住所有 backend 名称和实现细节。

更重要的是理解分层：

Scheduler 决定跑哪些请求

Memory layer 决定 KV 在哪里

Model runner 准备计算 metadata

Attention backend 按 metadata 高效执行 attention

如果这几层边界混在一起，阅读源码会很痛苦。

27.17 Sampler: logits 到 token

模型 forward 输出 logits 后，还没有生成 token。

Sampler 负责把 logits 变成下一个 token。

典型流程是：

logits

- > repetition / frequency / presence penalty
- > grammar mask or custom logit processor
- > temperature scaling
- > top-k / top-p / min-p filtering
- > sample or greedy choose
- > next token id

如果 temperature = 0，通常接近 greedy decoding。

如果设置 top_p、top_k，sampler 会限制候选 token 集合。

如果设置 structured output，sampler 还必须考虑当前 grammar state 下哪些 token 合法。

这说明 structured output 不是“生成完再检查”，而是会进入 token selection 本身。

27.18 Structured output 在 runtime 里的位置

SGLang 支持用 JSON schema、regex、EBNF、structural tag 等方式约束输出。

runtime 视角下，它大致是这样工作的：

request sampling params

- > parse json_schema / regex / ebnf
- > build grammar state
- > each decode step:
 - logits
 - > compute valid token mask
 - > mask invalid tokens
 - > sample valid next token
 - > update grammar state

这样做的好处是：

1. 输出天然满足约束的概率更高。
2. 减少生成后 JSON parse 失败。
3. 减少后处理和重试。

4. 工具调用格式更可控。
5. 可以和 OpenAI-compatible API 的 `response_format` 对齐。

但它也有成本：

1. 每个请求可能有自己的 grammar state。
2. 每轮 decode 需要计算合法 token mask。
3. 复杂 schema 可能增加 CPU 或 GPU 侧开销。
4. 约束过强可能影响模型输出质量。

所以 structured output 是 correctness 和 performance 的折中，不是免费功能。

27.19 Streaming 输出

Streaming 是 runtime 输出层的重要能力。

生成 token 后，runtime 需要把 token 转成文本增量，并通过 SSE、HTTP chunk 或其他协议返回。

需要处理的问题包括：

1. token 到文本的增量 detokenization。
2. Unicode 边界。
3. stop string 跨 token 匹配。
4. special token 是否跳过。
5. 客户端断开。
6. backpressure。
7. finish reason。
8. usage 统计。

Streaming 和 scheduler 也有关。

如果 decode step 抖动很大，用户看到的 streaming 就会一卡一卡。

如果 prefill 长请求长时间占用 GPU，正在 streaming 的请求也可能出现输出间隔变长。

所以优化 streaming 体验，不只是改 HTTP flush，而是要看 scheduler、decode step time 和 batch 组成。

27.20 Offline engine 为什么重要

很多人只关注在线 server，但 SGLang 的 offline engine 也很重要。

Offline engine 直接调用 runtime，不经过 HTTP server：

```
Python process
-> sgl.Engine
-> runtime scheduler
-> model runner
-> outputs
```

它适合：

1. 离线数据合成。
2. 批量评测。
3. RL rollout。
4. 自定义服务框架。
5. notebook 或实验脚本。
6. 避免 HTTP 序列化和网络开销。

从架构上看，offline engine 的价值是把 runtime 能力从“必须通过 HTTP server 使用”中解耦出来。

这对工程团队很重要。你可以用同一套底层 runtime 同时服务在线和离线场景，而不是维护两套推理逻辑。

27.21 SGLang Runtime 和 vLLM Runtime 的相似点

SGLang Runtime 和 vLLM Runtime 有很多相似点。

共同点包括：

1. 都面向高性能 LLM serving。
2. 都要做 prefill 和 decode。
3. 都要做 continuous batching。
4. 都要管理 KV cache。
5. 都要支持 streaming。
6. 都要处理 sampling 参数。
7. 都要支持 OpenAI-compatible API。
8. 都要面对显存、吞吐、延迟和调度折中。

所以不要把两者理解成完全不同的系统。

它们共享同一个底层问题：

如何在 GPU 上高效执行大量自回归生成请求？

27.22 SGLang Runtime 和 vLLM Runtime 的差异点

差异主要在问题重心。

vLLM 更突出：

1. PagedAttention。
2. KV block 管理。
3. high-throughput serving。
4. OpenAI-compatible API。
5. 大量独立请求的调度效率。

SGLang 更突出：

1. complex LLM programs。
2. frontend language 和 runtime 协同。
3. RadixAttention 自动 prefix reuse。
4. multi-call、branch、parallel、agent workload。
5. structured output 和 constrained decoding。
6. offline engine 和 native API 对复杂任务的支持。

可以这样记：

vLLM：高效执行大量生成请求

SGLang：高效执行复杂 LLM programs，同时也支持高性能 serving

这不是绝对边界。两个项目都在演进，也会互相吸收能力。面试中不要把它们说成“谁完全替代谁”，而要说清楚设计重心和适用 workload。

27.23 一个请求的端到端例子

假设用户发送一个结构化抽取请求：

```
{
  "text": "Extract the city and population from the following paragraph...",
  "sampling_params": {
    "temperature": 0,
    "max_new_tokens": 128,
    "json_schema": "{...}"
  },
}
```

```
"stream": false
}
```

runtime 内部可以这样理解:

API server

- > parse request
- > tokenize text
- > parse json_schema
- > create request state

Scheduler

- > put request into waiting queue
- > check token budget and KV budget
- > select request for prefill

RadixAttention

- > search token prefix
- > reuse matched KV if any
- > decide suffix to prefill

Memory pool

- > allocate KV slots for uncached tokens

Model runner

- > run prefill
- > produce logits for first output token

Sampler + grammar backend

- > apply JSON grammar mask
- > choose first token
- > update grammar state

Decode loop

- > scheduler selects request each iteration
- > model runner runs one-token decode
- > grammar mask filters invalid tokens
- > sampler chooses next token
- > stop when JSON complete or max_new_tokens reached

Output layer

- > detokenize
- > return JSON text and usage

Cache layer

- > keep useful prefix if policy allows
- > release non-reusable temporary cache

这个例子说明: 一个看似简单的 JSON 输出请求, 会经过 scheduler、cache、model runner、sampler 和 grammar backend 多个模块。

27.24 SGLang Runtime 的关键指标

理解 runtime 架构后, 指标也更容易拆。

入口层指标:

1. request QPS。

2. request parse latency。
3. tokenize latency。
4. HTTP queue time。
5. error rate。

调度层指标：

1. waiting queue length。
2. running request count。
3. scheduled prefill tokens。
4. scheduled decode tokens。
5. batch size。
6. preemption 或 reject 次数。

cache 层指标：

1. KV cache usage。
2. prefix cache hit rate。
3. matched prefix length。
4. eviction count。
5. memory pool free slots。

模型执行层指标：

1. prefill latency。
2. decode step latency。
3. input tokens/s。
4. output tokens/s。
5. GPU utilization。
6. memory bandwidth pressure。

输出层指标：

1. TTFT。
2. TPOT。
3. E2E latency。
4. streaming interval。
5. finish reason 分布。
6. structured output failure 或 retry 次数。

如果线上服务慢，不要只看 QPS。要按这几层拆，才能知道慢在 tokenize、queue、prefill、decode、grammar mask、streaming 还是网络。

27.25 常见误解

误解一：SGLang Runtime 就是一个 OpenAI API server。

不对。OpenAI-compatible API 只是入口之一，runtime 的核心是调度、cache、模型执行和采样。

误解二：SGLang 的核心只有 RadixAttention。

不对。RadixAttention 很重要，但没有 scheduler、memory pool、model runner 和 sampler 配合，无法单独构成 serving runtime。

误解三：structured output 是后处理。

不准确。在 SGLang Runtime 中，JSON schema、regex、EBNF 等约束可以进入 constrained decoding，在每一步采样前限制合法 token。

误解四：prefix cache 命中看字符串相似度。

不对。真正匹配的是 token prefix。字符串看起来相似，如果模板、空格或 special token 不同，也可能无法复用。

误解五：offline engine 只是 demo。

不对。offline engine 对批处理、评测、RL rollout 和自定义 server 很有价值。

27.26 面试官会怎么问

问题一：请画一下 SGLang Runtime 的整体架构。

回答要点：入口包括 OpenAI-compatible API、native /generate、offline engine 和 frontend language；进入 runtime 后经过 request parsing、tokenization、scheduler、RadixAttention、memory pool、model runner、sampler、grammar backend 和 streaming output。

问题二：SGLang Runtime 和普通 HTTP 模型服务有什么区别？

回答要点：普通 HTTP 服务只负责接请求和返回结果，SGLang Runtime 是 GPU 执行引擎，需要处理 continuous batching、KV cache、prefix reuse、structured decoding、streaming 和多请求调度。

问题三：RadixAttention 在 runtime 哪个阶段生效？

回答要点：在请求 tokenized 后、prefill 前，runtime 用 radix tree 查找最长可复用 prefix，只对未命中 suffix 做 prefill；请求结束或生成过程中，新的 token 序列和 KV cache 会插入 prefix tree，显存不足时按策略淘汰。

问题四：structured output 为什么属于 runtime？

回答要点：因为它不是生成完再 parse，而是在每一步 decode 时根据 JSON schema、regex 或 EBNF 生成合法 token mask，作用在 logits/sampling 之前。

问题五：SGLang 的 offline engine 有什么意义？

回答要点：它绕过 HTTP server，直接使用 runtime scheduler 和 model runner，适合离线批处理、评测、RL rollout、自定义 server 和实验脚本。

27.27 标准回答模板

如果面试官问“SGLang Runtime 是怎么工作的”，可以这样回答：

SGLang Runtime 可以理解为 SGLang 的高性能执行后端。它对外支持 OpenAI-compatible API、native /generate API、offline engine

一个请求进入后，server 会先解析 messages、prompt、sampling 参数和 structured output 约束，然后应用 chat template 和 token

在 prefill 前，RadixAttention 会用 radix tree 查找最长 token prefix，复用已有 KV cache，只计算未命中的 suffix。Memory pool、top-k、penalty 等参数；如果有 JSON schema、regex 或 EBNF，还会通过 grammar backend 做 constrained decoding，mask 掉非

生成 token 后，runtime 更新请求状态、KV cache、grammar state，并按需要 stream 给客户端。请求结束后，普通临时 cache 会释放

27.28 SGLang Runtime 指标、模块门禁和可运行 demo

把进入 runtime 的请求抽象成：

$$r_i = (e_i, X_i, Y_i, h_i, g_i, s_i)$$

其中 e_i 是入口类型， X_i 是输入 token 数， Y_i 是计划输出 token 数， h_i 是 RadixAttention 命中的 prefix token 数， g_i 表示是否启用 grammar / structured output， s_i 表示是否 streaming。

命中 prefix 后，实际需要执行 prefill 的 token 数为：

$$P_i^{\text{run}} = \max(0, X_i - h_i)$$

一次窗口内的总 prefill token 工作量为：

$$P_{\text{run}} = \sum_i P_i^{\text{run}}$$

prefix reuse 节省比例为:

$$R_{\text{reuse}} = \frac{\sum_i h_i}{\max(1, \sum_i X_i)}$$

如果每个新 token 需要一个 KV slot, 教学版 memory pool 的峰值压力可以写成:

$$P_{\text{kv}} = \frac{N_{\text{slot,peak}}}{N_{\text{slot,total}}}$$

prefill 调度需要满足每轮 token budget:

$$\sum_{i \in B_t} P_i^{\text{run}} \leq B_{\text{prefill}}$$

decode 调度每轮给 active request 追加一个 token, 可以用 active 行数近似工作量:

$$D_{\text{rows}} = \sum_t A_t$$

结构化输出的额外状态更新次数可以写成:

$$G_{\text{steps}} = \sum_i g_i Y_i$$

教学版 SGLang Runtime 门禁可以写成:

$$G_{\text{runtime}} = G_{\text{entry}} G_{\text{state}} G_{\text{radix}} G_{\text{memory}} G_{\text{schedule}} G_{\text{grammar}} G_{\text{stream}} G_{\text{metric}}$$

其中:

1. G_{entry} : OpenAI-compatible API、native API、offline engine 和 frontend program 都能落到统一 request state。
2. G_{state} : request state 至少包含 token ids、阶段、sampling params、KV slot、grammar state、streaming 和 finish reason。
3. G_{radix} : 能在 prefill 前计算 prefix hit、run prefill tokens 和 saved prefill tokens。
4. G_{memory} : memory pool 能解释 KV slot 峰值、释放和压力。
5. G_{schedule} : prefill / decode 分阶段调度, 能看到 waiting、running、finished 的变化。
6. G_{grammar} : structured output 的 grammar mask 在 sampler 前生效, 而不是输出后才 parse。
7. G_{stream} : streaming 输出、finish event 和取消 / backpressure 边界可观测。
8. G_{metric} : TTFT、decode rows、KV pressure、reuse ratio 和错误率可以落表。

下面这个 0 依赖 demo 模拟一个极简 SGLang Runtime。它把 OpenAI-compatible API、native /generate、frontend program 和 offline engine 四类入口统一成 request state; 用 toy radix cache 计算 prefix hit; 用 memory pool 分配 KV slots; 用 prefill token budget 做两轮 prefill; 再用 continuous decode loop 逐轮生成 token, 并统计 grammar mask 和 streaming 事件。


```
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class RuntimeRequest:
    rid: str
    entripoint: str
    prompt_tokens: tuple
    output_tokens: int
    structured: bool = False
    stream: bool = False
    hit_tokens: int = 0
    hit_owner: str = ""
    run_prefill_tokens: int = 0
    generated: int = 0
    kv_slots: int = 0
    phase: str = "waiting"
    finish_reason: str = ""
    events: list = field(default_factory=list)
```

```
class ToyRadixCache:
    def __init__(self):
        self.prompts = []

    @staticmethod
    def common_prefix_len(left, right):
        count = 0
        limit = min(len(left), len(right))
        while count < limit and left[count] == right[count]:
            count += 1
        return count

    def lookup(self, tokens):
        best = 0
        owner = ""
        for rid, cached in self.prompts:
            hit = self.common_prefix_len(tokens, cached)
            if hit > best:
                best = hit
                owner = rid
        return best, owner

    def insert(self, rid, tokens):
        self.prompts.append((rid, tokens))
```

```
class ToyMemoryPool:
    def __init__(self, total_slots):
        self.total_slots = total_slots
        self.used_slots = 0
        self.peak_slots = 0
        self.by_request = {}

    def allocate(self, rid, slots):
```

```

    if self.used_slots + slots > self.total_slots:
        raise RuntimeError(f"KV pool overflow for {rid}")
    self.used_slots += slots
    self.peak_slots = max(self.peak_slots, self.used_slots)
    self.by_request[rid] = self.by_request.get(rid, 0) + slots

def release(self, rid, keep_prefix_slots):
    owned = self.by_request.get(rid, 0)
    released = max(0, owned - keep_prefix_slots)
    self.used_slots -= released
    self.by_request[rid] = keep_prefix_slots
    return released

class ToySGLangRuntimeAudit:
    def __init__(self, requests, kv_slots_total, prefill_budget):
        self.requests = list(requests)
        self.radix = ToyRadixCache()
        self.memory = ToyMemoryPool(kv_slots_total)
        self.prefill_budget = prefill_budget
        self.prefill_rounds = []
        self.decode_trace = []
        self.grammar_steps = 0
        self.stream_events = 0

    def ingest(self):
        for req in self.requests:
            hit, owner = self.radix.lookup(req.prompt_tokens)
            req.hit_tokens = hit
            req.hit_owner = owner
            req.run_prefill_tokens = len(req.prompt_tokens) - hit
            self.radix.insert(req.rid, req.prompt_tokens)
            req.events.append("ingested")

    def run_prefill(self):
        waiting = list(self.requests)
        round_id = 0
        while waiting:
            budget_left = self.prefill_budget
            selected = []
            still_waiting = []
            for req in waiting:
                cost = req.run_prefill_tokens
                if cost <= budget_left and selected:
                    selected.append(req)
                    budget_left -= cost
                elif cost <= budget_left and not selected:
                    selected.append(req)
                    budget_left -= cost
                else:
                    still_waiting.append(req)

            used = 0
            for req in selected:
                req.phase = "prefill"

```

```

        req.kv_slots += req.run_prefill_tokens
        self.memory.allocate(req.rid, req.run_prefill_tokens)
        req.events.append(f"prefill_round_{round_id}")
        used += req.run_prefill_tokens
        req.phase = "decode"

    self.prefill_rounds.append(
        {"round": round_id, "request_ids": [req.rid for req in selected], "tokens": used}
    )
    waiting = still_waiting
    round_id += 1

def run_decode(self):
    active = [req for req in self.requests if req.phase == "decode"]
    step = 0
    while active:
        row = {"step": step, "active": [req.rid for req in active], "streamed": []}
        next_active = []
        for req in active:
            req.generated += 1
            req.kv_slots += 1
            self.memory.allocate(req.rid, 1)
            if req.structured:
                self.grammar_steps += 1
                req.events.append("grammar_mask")
            if req.stream:
                self.stream_events += 1
                row["streamed"].append(req.rid)

            if req.generated >= req.output_tokens:
                req.phase = "finished"
                req.finish_reason = "length"
                req.events.append("finished")
                self.memory.release(req.rid, keep_prefix_slots=req.hit_tokens)
            else:
                next_active.append(req)

        self.decode_trace.append(row)
        active = next_active
        step += 1

def report(self):
    total_input = sum(len(req.prompt_tokens) for req in self.requests)
    total_hit = sum(req.hit_tokens for req in self.requests)
    run_prefill = sum(req.run_prefill_tokens for req in self.requests)
    decode_rows = sum(len(row["active"]) for row in self.decode_trace)
    ttft_estimate_ms = [
        2 + req.run_prefill_tokens * 4 + (2 if req.structured else 0)
        for req in self.requests
    ]
    summary = {
        "entrypoints": sorted(set(req.entrypoint for req in self.requests)),
        "total_input_tokens": total_input,
        "run_prefill_tokens": run_prefill,
        "saved_prefill_tokens": total_hit,
    }

```

```

        "reuse_ratio": round(total_hit / max(1, total_input), 3),
        "prefill_rounds": len(self.prefill_rounds),
        "decode_rounds": len(self.decode_trace),
        "decode_rows": decode_rows,
        "kv_peak_slots": self.memory.peak_slots,
        "kv_pressure": round(self.memory.peak_slots / self.memory.total_slots, 3),
        "grammar_steps": self.grammar_steps,
        "stream_events": self.stream_events,
        "avg_ttft_ms": round(sum(ttft_estimate_ms) / len(ttft_estimate_ms), 2),
        "finished": sum(1 for req in self.requests if req.phase == "finished"),
    }
    request_rows = [
        {
            "rid": req.rid,
            "entrypoint": req.entrypoint,
            "hit_tokens": req.hit_tokens,
            "hit_owner": req.hit_owner,
            "run_prefill_tokens": req.run_prefill_tokens,
            "generated": req.generated,
            "finish_reason": req.finish_reason,
        }
        for req in self.requests
    ]
    gates = {
        "entrypoints_unified": summary["entrypoints"]
        == ["frontend", "native", "offline", "openai"],
        "request_state_complete": all(req.finish_reason for req in self.requests),
        "radix_hit_visible": summary["saved_prefill_tokens"] > 0,
        "memory_pool_budget_ok": summary["kv_peak_slots"] <= self.memory.total_slots,
        "scheduler_phases_visible": summary["prefill_rounds"] == 2
        and summary["decode_rounds"] == 3,
        "grammar_in_sampler": summary["grammar_steps"] == 2,
        "streaming_visible": summary["stream_events"] == 6,
        "metrics_ready": summary["kv_pressure"] < 0.95,
    }
    gates["sglang_runtime_gate"] = all(gates.values())
    return request_rows, summary, gates

```

```

ROOT = tuple(f"root_{i}" for i in range(8))

```

```

requests = [
    RuntimeRequest("openai_chat", "openai", ROOT + ("chat", "user", "kv", "cache"), 3, stream=True),
    RuntimeRequest("native_json", "native", ROOT + ("json", "extract", "city", "population", "schema", "v1"), 2, stream=True),
    RuntimeRequest("frontend_branch", "frontend", ROOT + ("json", "extract", "city", "population", "schema", "v1", "batch"), 2, stream=True),
    RuntimeRequest("offline_eval", "offline", ROOT + ("eval", "batch"), 2),
]

```

```

runtime = ToySGLangRuntimeAudit(requests, kv_slots_total=36, prefill_budget=18)
runtime.ingest()
runtime.run_prefill()
runtime.run_decode()
request_rows, runtime_summary, runtime_gates = runtime.report()

```

```

print("sglang_runtime_requests=", request_rows)
print("sglang_runtime_prefill_rounds=", runtime.prefill_rounds)

```

```
print("sglang_runtime_decode_trace=", runtime.decode_trace)
print("sglang_runtime_summary=", runtime_summary)
print("sglang_runtime_gates=", runtime_gates)
```

这段 demo 应该输出：

```
sglang_runtime_summary= {'entrypoints': ['frontend', 'native', 'offline', 'openai'], 'total_input_tokens': 52, 'run_pre
sglang_runtime_gates= {'entrypoints_unified': True, 'request_state_complete': True, 'radix_hit_visible': True, 'memory_
```

读这段代码时要注意：

1. 四种入口最后都变成同一种 `RuntimeRequest`，这对应 runtime 统一执行后端。
2. `saved_prefill_tokens=30` 来自 radix prefix lookup，说明 RadixAttention 生效点在 prefill 前。
3. `kv_peak_slots=31` 和 `kv_pressure=0.861` 说明 memory pool 是 scheduler 必须考虑的硬约束。
4. `grammar_steps=2` 说明结构化输出约束在每个 decode step 的 sampler 前生效。
5. `stream_events=6` 说明 streaming 不是 HTTP 层打印，而是 decode loop 的输出事件。

27.29 小练习

1. 画出一个 SGLang Runtime 请求从 API server 到 streaming output 的完整链路。
2. 解释为什么 tokenizer 和 chat template 会影响 prefix cache 命中率。
3. 对比 prefill 调度和 decode 调度分别关注什么。
4. 说明 RadixAttention 为什么需要和 memory pool 协同。
5. 写一个 JSON schema constrained decoding 请求，并标出 grammar backend 在 runtime 中的位置。
6. 解释 offline engine 和 HTTP server 入口有什么不同、底层又有什么相同。
7. 假设 TTFT 高但 TPOT 正常，按 SGLang Runtime 模块列出排查路径。

27.30 本章总结

SGLang Runtime 是 SGLang 的执行核心。它不是简单的 HTTP wrapper，也不是单独一个 RadixAttention 数据结构，而是一套完整的 LLM serving engine。

它把不同入口的请求统一落到一条执行链路：解析请求、tokenize、构造 request state、进入 scheduler、查找 prefix cache、分配 KV cache、执行 prefill/decode、采样 token、应用 structured output 约束、streaming 输出、释放或保留 cache。

理解这条链路后，后面的章节就更容易展开：第 28 章会专门深入 RadixAttention 与 prefix sharing，第 29 章会拆 SGLang scheduler 设计，第 30 章会深入 structured generation 与 constrained decoding。

第 28 章 RadixAttention 与 Prefix Sharing

上一章从整体上看了 SGLang Runtime：请求进入后会经过 API server、tokenization、scheduler、memory pool、RadixAttention、model runner、sampler、grammar backend 和 streaming output。

本章专门深入 SGLang 最有代表性的后端机制：RadixAttention。

先澄清一个容易误解的点：RadixAttention 不是一种新的 attention 数学公式，也不是替代 FlashAttention 的 GPU kernel。它是一套 runtime 级别的 KV cache 复用机制，用 radix tree 管理 token prefix 到 KV cache 的映射，让复杂 LLM 程序中的共享前缀可以自动复用。

一句话概括：

RadixAttention 用压缩前缀树维护 token 序列和 KV cache 的关系，在新请求到来时做最长前缀匹配，只计算未命中的 suffix；请求结束或生成新 token 后，把有复用价值的 KV cache 保留在树上，并在显存紧张时按策略淘汰。

28.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考四类公开资料：

1. SGLang 论文《SGLang: Efficient Execution of Structured Language Model Programs》对 RadixAttention for KV cache reuse、frontend / runtime 协同和复杂 LLM programs 的说明。
2. SGLang 官方文档首页对 SGLang 作为 high-performance serving framework, 以及 RadixAttention、prefix caching、multi-GPU parallelism 和 OpenAI API 兼容的定位说明。
3. SGLang Attention Backend 文档对 attention backend、page size、KV cache dtype、prefill / decode forward 和 prefix cache 完整页命中约束的说明。
4. 前面第 23 章 vLLM prefix caching 校准过的 full block / parent hash / extra hash / cache salt / LoRA / 多模态隔离口径, 用来对比 RadixAttention 和普通 block-hash prefix cache 的边界。

本章只讲 RadixAttention 的教学版机制: token prefix、radix tree、longest prefix match、split、insert、ref count、leaf eviction、page size 与 scheduler cost。它不绑定某个 SGLang 版本的真实类名、源码路径、GPU memory layout、真实 page table、CUDA kernel、HiCache、distributed KV cache、PD 分离、MoE 路由或生产级 eviction 策略。本章 demo 用纯 Python 模拟 radix tree 元数据和 KV token 计数, 不读写真实 KV tensor。

参考资料:

1. SGLang 论文: <https://arxiv.org/abs/2312.07104>
2. SGLang 官方文档首页: <https://docs.sglang.io/index.md>
3. SGLang Attention Backend: https://docs.sglang.io/docs/advanced_features/attention_backend.md
4. SGLang Runtime 总览相关文档: <https://docs.sglang.io/llms.txt>

28.1 本章目标

读完本章, 你应该能讲清:

1. Prefix sharing 为什么是复杂 LLM programs 的核心性能机会。
2. RadixAttention 和普通 prefix cache 的直觉差异。
3. Trie、radix tree、token prefix、KV cache value 分别是什么。
4. Longest prefix match、insert、split、evict 的基本流程。
5. RadixAttention 如何减少 prefill 计算。
6. 为什么 RadixAttention 必须和 memory pool、scheduler 协同。
7. 哪些 workload 最容易受益, 哪些场景收益有限。
8. 面试中如何解释 RadixAttention。

28.2 Prefix sharing 是什么

Prefix sharing 指多个请求或多个 generation call 共享同一段 token 前缀。

例如两个用户问题共享同一份长文档:

```
request A: [system prompt][long document][question A]
request B: [system prompt][long document][question B]
```

共享部分是:

```
[system prompt][long document]
```

如果 request A 已经对这段前缀做过 prefill, 生成了对应 KV cache, 那么 request B 再从头 prefill 就浪费了。

更复杂的例子是 self-consistency:

```
shared prompt: [problem statement][few-shot examples][question]
-> sample answer 1
-> sample answer 2
-> sample answer 3
-> sample answer 4
```

多个 sample 共享完全相同的 prompt, 只是后续 decode 分支不同。

Tree-of-Thought 更明显:

```
root prompt
-> thought A
    -> continuation A1
    -> continuation A2
-> thought B
    -> continuation B1
    -> continuation B2
```

这里不仅 root 共享，thought A 下的多个分支也共享更长前缀。

Prefix sharing 的本质是：

多个 token 序列拥有共同开头。

RadixAttention 要做的事情是：

让 runtime 自动发现这些共同开头，并复用对应 KV cache。

28.3 为什么 prefix sharing 能省 prefill

自回归 LLM 生成分两个阶段：prefill 和 decode。

Prefill 处理输入 prompt，生成每层 attention 的 K/V：

prompt tokens -> transformer forward -> KV cache + first logits

如果两个请求前 8000 个 token 相同，那么这 8000 个 token 的 K/V 在相同模型、相同 tokenizer、相同位置和相同附加条件下也是相同的。

第二个请求可以直接复用这 8000 个 token 的 KV cache，只计算后面的不同部分。

例如：

```
cached:      [A B C D E F]
new prompt:  [A B C D E F G H]
```

```
reuse:       [A B C D E F]
compute:     [G H]
```

这会带来两个收益：

1. 减少 prefill 计算量。
2. 降低 TTFT。

如果系统同时有大量类似请求，还会提升整体 input tokens/s，因为 GPU 不再重复处理同一段前缀。

但要注意：prefix sharing 主要节省 prefill，不直接消除后续 decode 成本。每条分支生成自己的新 token 时，仍然要一轮一轮 decode。

28.4 为什么复杂 LLM 程序特别需要 prefix sharing

普通 chat completion 可能只有一次请求：

prompt -> answer

复杂 LLM 程序经常是：

prompt -> gen A -> tool -> gen B -> branch -> gen C1/C2/C3 -> merge -> gen D

这种程序有几个特点：

1. 多次调用模型。
2. 调用之间共享上下文。
3. 有分支和并行。
4. 有多轮历史。

5. 有固定工具说明、规则、few-shot examples。
6. 有搜索树或候选答案树。

如果 runtime 只看到一堆独立请求，它很难知道这些调用之间的共享关系。

SGLang 的思路是：frontend language 让程序结构更显式，backend runtime 用 RadixAttention 自动利用共享前缀。

这就是为什么 RadixAttention 不只是“缓存一个 prompt”，而是面向复杂程序的 cache reuse 机制。

28.5 普通 prefix cache 的局限

第 23 章讲过 vLLM 的 prefix caching：它通常基于 KV block hash 复用已经计算过的 full blocks。

这种方式对固定前缀非常有效，例如：

```
[system prompt][document][question A]
[system prompt][document][question B]
```

但复杂 LLM programs 的共享模式经常不只是“很多请求共享一个长开头”，而是树状共享。

例如：

```
S
-> A
    -> A1
    -> A2
-> B
    -> B1
    -> B2
```

如果只用完整 prompt 作为 key：

```
cache[full_prompt] = KV
```

那么 S、S+A、S+B 这些中间共享段很难被自然表达。

如果用 block hash，可以复用线性前缀上的 full blocks，但仍然需要额外的数据结构和调度策略来更系统地表达多分支、多层级的共享结构。

RadixAttention 的核心差异是：它直接把已见过的 token 序列组织成一棵 prefix tree。

28.6 Trie 和 radix tree

要理解 RadixAttention，先理解 trie 和 radix tree。

Trie，也叫前缀树。它用树表示一组序列的共享前缀。

假设有三条 token 序列：

```
A B C D
A B E F
A B E G
```

普通 trie 可以表示成：

```
root
-> A
    -> B
        -> C
        -> D
    -> E
        -> F
        -> G
```


每条边只放一个 token。

Radix tree 是压缩前缀树。它会把只有单一路径的节点压缩成一条边。

同样的序列可以表示成：

```
root
-> [A B]
  -> [C D]
    -> [E]
      -> [F]
        -> [G]
```

在 RadixAttention 中，边上的 label 不是字符，而是一段 token ids：

edge label = [token_1, token_2, ..., token_k]

节点或边关联的是这段 token 对应的 KV cache 位置。

28.7 RadixAttention 的数据结构直觉

概念上，RadixAttention 维护一棵树：

Radix tree:

- key: token prefix
- value: KV cache reference

更具体一点，每个节点可能包含：

1. 从父节点到当前节点的 token segment。
2. 这段 token 对应的 KV cache 引用。
3. 子节点映射。
4. 父节点指针。
5. 最近访问时间或 LRU 信息。
6. 当前是否被 active request 引用。
7. 这段 cache 占用的 token 数或 memory slot。

可以用伪结构表示：

```
class RadixNode:
    token_segment: list[int]
    kv_refs: list[KVRef]
    children: dict[int, RadixNode]
    parent: RadixNode | None
    last_access_time: int
    ref_count: int
```

真实实现会更复杂，尤其要和 GPU memory pool、paged KV layout、并发调度结合。但学习时先抓住这个抽象就够了。

28.8 KV cache value 是什么

Radix tree 的 key 是 token prefix，value 不是最终文本答案，而是 KV cache。

KV cache 包含每层 transformer 对历史 token 计算出的 key/value。

概念形状类似：

KV cache:

- layer 0: K/V for tokens
- layer 1: K/V for tokens
- ...
- layer L-1: K/V for tokens

在 runtime 中，value 通常不会是一整块 Python 对象里的大 tensor，而是对 memory pool 中 cache slots 或 pages 的引用。

可以理解成：

```
token prefix [A B C]
-> KV positions [slot_0, slot_1, slot_2]
```

这样 model runner 在执行后续 suffix 或 decode 时，就能根据这些位置读取历史 KV。

28.9 Longest prefix match

新请求到来时，RadixAttention 最重要的操作是最长前缀匹配。

输入：

```
new prompt tokens = [A B C D E F]
```

已有树：

```
root
-> [A B C]
    -> [X Y]
    -> [D]
        -> [Z]
```

匹配过程：

```
[A B C D E F]
  matches [A B C]
  then matches [D]
  stops before [E F]
```

命中前缀是：

```
[A B C D]
```

未命中 suffix 是：

```
[E F]
```

runtime 接下来只需要对 [E F] 做 prefill，并把新路径插入树中。

伪代码可以写成：

```
def match_prefix(root, tokens):
    node = root
    pos = 0
    matched_nodes = []

    while pos < len(tokens):
        child = node.find_child_starting_with(tokens[pos])
        if child is None:
            break

        n = common_prefix_len(tokens[pos:], child.token_segment)
        if n == 0:
            break

        matched_nodes.append((child, n))
        pos += n

    if n < len(child.token_segment):
```

break

node = child

return matched_nodes, pos

pos 就是命中的 token 数。

28.10 为什么需要 split

Radix tree 的关键操作之一是 split。

假设树里已有一条边：

[A B C D]

现在新请求是：

[A B X Y]

二者共享 [A B]，但从第三个 token 开始分叉。

如果要让后续请求复用 [A B]，树必须把原边拆开：

拆分前：

```
root
-> [A B C D]
```

拆分后：

```
root
-> [A B]
    -> [C D]
    -> [X Y]
```

这就是 radix tree 能表达中间共享前缀的原因。

普通 “完整 key -> value” 的 cache 无法自然做这种拆分。

28.11 Insert 流程

当一个请求完成 prefill 或生成了新 token 后，runtime 可以把新 token 序列插入 radix tree。

简化流程：

1. 从 root 开始匹配 token 序列
2. 如果完全命中已有路径，更新访问信息
3. 如果在某条边中间分叉，split 这条边
4. 为未出现过的 suffix 创建新节点
5. 把 suffix 对应 KV cache 引用挂到新节点
6. 更新 LRU 或访问时间

伪代码：

```
def insert(root, tokens, kv_refs):
    matched_nodes, pos = match_prefix(root, tokens)

    if pos == len(tokens):
        touch_path(matched_nodes)
        return

    parent = find_insert_parent(matched_nodes)
    suffix_tokens = tokens[pos:]
```

```

suffix_kv_refs = kv_refs[pos:]

new_node = RadixNode(
    token_segment=suffix_tokens,
    kv_refs=suffix_kv_refs,
    parent=parent,
)
parent.children[suffix_tokens[0]] = new_node

```

真实实现要处理边中间 split、KV refs 切片、并发访问、memory pool 引用计数等问题。这里的伪代码只表达核心直觉。

28.12 一个完整例子：多轮 chat

假设第一轮对话：

```

S = system prompt
U1 = user first message
A1 = assistant first answer

```

第一次请求结束后，树里可能有：

```

root
-> [S U1 A1]

```

用户继续第二轮：

```

S U1 A1 U2

```

RadixAttention 可以命中：

```

[S U1 A1]

```

只需要 prefill：

```

[U2]

```

插入后：

```

root
-> [S U1 A1]
    -> [U2 A2]

```

如果另一个用户或另一个分支共享同一个 system prompt，但第一条消息不同：

```

S U1'

```

树可能 split 成：

```

root
-> [S]
    -> [U1 A1]
    -> [U2 A2]
    -> [U1']

```

这样 [S] 可以被多个会话复用。

28.13 一个完整例子：few-shot batch

Few-shot 评测常见格式：

```

instruction
example 1
example 2

```

example 3
question X

如果对很多问题做同一套 few-shot prompt:

P = instruction + examples

request 1: P + question 1

request 2: P + question 2

request 3: P + question 3

树结构是:

root

-> [P]

-> [question 1 answer 1]

-> [question 2 answer 2]

-> [question 3 answer 3]

P 越长, RadixAttention 的收益越明显。

这类 workload 的特点是:

1. 共享前缀长。
2. 分支多。
3. 每个分支 suffix 相对短。
4. 请求之间格式稳定。

它非常适合 prefix sharing。

28.14 一个完整例子: self-consistency

Self-consistency 会对同一个问题采样多个答案, 然后投票或选择。

结构是:

P = problem + reasoning instruction

P -> answer sample 1

P -> answer sample 2

P -> answer sample 3

P -> answer sample 4

Radix tree:

root

-> [P]

-> [sample 1]

-> [sample 2]

-> [sample 3]

-> [sample 4]

如果没有 cache reuse, P 会被 prefill 多次。

有 RadixAttention 后, P 只需要计算一次, 后续分支从同一个 KV prefix 开始 decode。

这也是 SGLang 论文和博客中强调复杂 prompting workload 的原因: 这些 workload 的共享前缀不是偶然现象, 而是算法结构本身带来的。

28.15 一个完整例子: Tree-of-Thought

Tree-of-Thought 是 RadixAttention 最直观的例子之一。

假设搜索树：

```
root prompt
-> thought A
    -> eval A
    -> expand A1
    -> expand A2
-> thought B
    -> eval B
    -> expand B1
    -> expand B2
```

对应 token prefix：

```
R
-> A
    -> A1
    -> A2
-> B
    -> B1
    -> B2
```

Radix tree 自然表达这棵搜索树：

```
root
-> [R]
    -> [A]
        -> [A1]
        -> [A2]
    -> [B]
        -> [B1]
        -> [B2]
```

每次扩展一个节点时，都可以复用从 root 到该节点路径上的 KV cache。

如果搜索树很深，这种复用会非常重要。

28.16 RadixAttention 和 attention kernel 的关系

名字里有 Attention，容易让人误以为 RadixAttention 是一种 attention 计算公式。

更准确的分层是：

RadixAttention：

管理 prefix -> KV cache 的复用关系

Attention backend：

真正执行 attention 计算

一次请求命中 RadixAttention 后：

matched prefix KV 已经存在

-> model runner 只对 suffix 做 prefill

-> attention backend 在计算 suffix 时读取 matched prefix 的 KV

所以 RadixAttention 不替代 FlashAttention、paged attention 或其他 kernel。

它是 runtime 级别的 cache 管理和调度机制。

28.17 和 memory pool 的协同

Radix tree 只记录逻辑关系还不够，KV cache 必须真实占用显存或其他存储。

Memory pool 负责：

1. 分配 KV cache slots。
2. 回收不再需要的 slots。
3. 维护 slot 是否被 active request 使用。
4. 支持 cache 被保留和复用。
5. 在显存不足时提供可淘汰对象。

RadixAttention 负责：

1. 根据 token prefix 找 cache。
2. 把新 token path 插入树。
3. 判断哪些 cache 有复用价值。
4. 选择哪些叶子或路径可以淘汰。

两者必须协同。

如果 radix tree 里有节点，但 memory pool 中对应 KV 已经被覆盖，就会读错 cache。

如果 memory pool 保留了 KV，但 radix tree 找不到对应 prefix，就无法复用。

所以每次 eviction 都要同时更新：

radix tree metadata
memory pool allocation state
KV cache references

28.18 Ref count：哪些节点不能淘汰

一个缓存节点可能正在被请求使用。

例如请求 R 正在基于 [A B C] decode，那么 [A B C] 对应 KV 不能被淘汰。

因此节点或 KV slot 需要类似 ref count 的机制：

ref_count > 0: active request 正在使用，不能 evict

ref_count = 0: 没有 active request，可作为 cache 保留，也可在显存紧张时 evict

请求命中 prefix 时，要增加引用：

match prefix -> pin cache -> ref_count += 1

请求结束或不再需要该 prefix 时，要减少引用：

finish request -> unpin cache -> ref_count -= 1

Eviction 只能选择 ref count 为 0 的节点或叶子。

这和普通缓存不同：普通缓存淘汰最多导致下次 miss；KV cache 如果淘汰了正在运行请求需要读取的内容，会导致错误生成甚至崩溃。

28.19 Eviction：为什么通常淘汰叶子

Radix tree 中，内部节点可能被很多子路径共享。

例如：

```
root
-> [P]
   -> [A]
   -> [B]
   -> [C]
```

如果淘汰 [P]，那么 [A]、[B]、[C] 的前缀也失效。

因此 eviction 通常更倾向于从叶子节点开始。

叶子节点代表某条路径的末端，淘汰它对其他共享路径影响较小。

简化策略：

1. 找到 `ref_count = 0` 的叶子节点
2. 按 LRU 选择最久未使用的叶子
3. 释放它关联的 KV cache
4. 从 radix tree 删除该叶子
5. 如果父节点因此变成无用叶子，可递归清理或压缩

这不是唯一策略，但符合直觉：优先释放复用价值较低、影响范围较小的 cache。

28.20 LRU 为什么不总是最优

LRU 的直觉是：最近用过的 prefix 未来更可能再次被用到。

这在 chat、few-shot、agent loop 中常常有效。

但 LRU 不总是最优。

例如：

1. 一个很长的 prefix 虽然不常用，但每次命中都能省大量 prefill。
2. 一个很短的 prefix 经常命中，但节省 token 很少。
3. 某些业务有周期性流量，刚被淘汰后马上又来。
4. 多租户场景中，高频小租户可能挤掉低频大价值 cache。

更理想的策略可能考虑：

1. 最近访问时间。
2. prefix 长度。
3. 节省的 prefill tokens。
4. 节点子树共享程度。
5. 租户优先级。
6. 请求类型。

但策略越复杂，维护成本和调度开销越高。

所以工程上常见做法是先用简单可靠策略，再根据指标调优。

28.21 Scheduler 为什么要知道 cache hit length

RadixAttention 的命中结果会改变 scheduler 对请求成本的估计。

假设请求 prompt 长度是 12000 tokens。

如果没有命中：

`prefill cost = 12000 tokens`

如果命中 10000 tokens：

`prefill cost = 2000 tokens`

这两个请求对 GPU 的压力完全不同。

Scheduler 如果不知道 cache hit length，可能做出错误决定：

1. 把一个实际很便宜的请求挡在队列外。
2. 把一个实际很贵的请求放进 batch，导致 decode 抖动。
3. 错估 TTFT。
4. 错估 KV allocation。

5. 错过 cache locality 更好的调度机会。

因此 RadixAttention 不是一个独立字典，而是 scheduler 决策的一部分。

28.22 Cache-aware scheduling

Cache-aware scheduling 指 scheduler 在选择请求时考虑 cache 命中情况。

例如 waiting queue 中有两个请求：

R1: prompt 12000 tokens, cache hit 10000 tokens, need prefill 2000

R2: prompt 4000 tokens, cache hit 0 tokens, need prefill 4000

如果只看 prompt length, R1 看起来更贵。

如果看 uncached suffix, R1 反而更便宜。

再比如：

R3 和当前 hot prefix 共享很长前缀

R4 不共享任何 prefix

优先调度 R3 可能提升 cache hit 和整体吞吐。

但 cache-aware scheduling 也不能过度偏向 cache hit 请求，否则 cache miss 请求可能饥饿。

所以它本质上是折中：

cache reuse

fairness

TTFT

TPOT

KV capacity

batch utilization

28.23 Token prefix 完全匹配，不是语义相似

RadixAttention 匹配的是 token prefix 完全一致。

不是字符串相似，更不是语义相似。

例如下面两段文本语义相同，但 token 序列可能不同：

"You are a helpful assistant."

"You are a helpful assistant ."

空格、标点、chat template、special token 都会影响 token ids。

因此要提高 RadixAttention 命中率，需要保证：

1. prompt 模板稳定。
2. system prompt 稳定。
3. few-shot examples 顺序稳定。
4. 工具说明稳定。
5. RAG 文档拼接格式稳定。
6. tokenizer 和 chat template 一致。
7. 不要在共享前缀中插入时间戳、随机 id 等变化字段。

如果共享前缀每次都带一个不同 request id, cache 基本无法命中。

28.24 正确性边界

KV cache 能不能复用，不只取决于 token 一样。

还必须保证影响 forward 的条件一致。

常见条件包括：

1. 模型权重一致。
2. tokenizer 一致。
3. chat template 一致。
4. RoPE scaling 或 position 规则一致。
5. LoRA adapter 一致。
6. 多模态输入内容一致。
7. prompt adapter 或特殊 embedding 一致。
8. tenant 隔离策略允许共享。
9. dtype、量化和 KV layout 兼容。

如果这些条件不一致，却复用同一份 KV cache，结果可能是错误的。

工程上通常需要把这些条件编码进 cache key、cache namespace 或额外 metadata。

Radix tree 管 token prefix，但 prefix 所属的上下文环境也必须正确隔离。

28.25 多模态场景

多模态模型会让 prefix sharing 更复杂。

例如图像输入可能在文本里表现为：

<image>

但真正送入模型的是视觉 encoder 产生的 embedding 或一组 image tokens。

如果两个请求都有 <image> 占位符，但图片不同，不能复用同一段视觉相关 KV。

因此多模态 cache 需要考虑：

1. 图片内容 hash。
2. 图片预处理参数。
3. vision encoder 版本。
4. image token 数量。
5. image token 插入位置。
6. 文本 token 与视觉 token 的组合方式。

可以复用的情况是：

同一张图片 + 同一段文本前缀 + 同一模型处理流程

不能只看文本 placeholder 是否一样。

28.26 与 vLLM prefix caching 的对比

可以用一张表总结视觉差异。

维度	vLLM prefix caching	SGLang RadixAttention
核心结构	block hash / block table	radix tree / prefix tree
关注点	复用已计算 KV blocks	自动管理复杂 prefix sharing
典型场景	大量请求共享固定前缀	多轮、分支、self-consistency、ToT、agent
匹配方式	按 block hash 链命中	longest prefix match
共享形态	更偏线性前缀复用	更自然表达树状共享
淘汰对象	cached blocks	radix tree 节点或叶子关联 KV

维度	vLLM prefix caching	SGLang RadixAttention
与程序结构关系	相对弱	更强调 frontend/runtime 协同

这张表不是说谁绝对更好。

vLLM 的设计非常适合高吞吐通用 serving。

SGLang 的 RadixAttention 更强调复杂 LLM program 中的共享前缀自动复用。

真实系统会不断演进，边界也会变化。面试中要讲设计重心，不要把二者说成简单替代关系。

28.27 RadixAttention 不适合解决什么

RadixAttention 不是万能优化。

它不适合解决：

1. 完全不同 prompt 的重复计算。
2. decode 阶段每个 token 的所有成本。
3. 模型权重太大导致的显存不足。
4. attention kernel 本身慢的问题。
5. 网络延迟、tokenizer 慢、RAG 检索慢。
6. 语义相似但 token 不同的请求复用。
7. 需要返回完全相同答案的 result cache 场景。

如果请求之间几乎没有共享前缀，RadixAttention 命中率低，收益自然有限。

如果瓶颈在 decode TPOT，而不是 prefill TTFT，RadixAttention 的收益也可能不明显。

28.28 典型收益场景

RadixAttention 更适合这些 workload：

1. Few-shot evaluation。
2. Self-consistency。
3. Tree-of-Thought。
4. Multi-turn chat。
5. Agent trajectory。
6. Tool use with long tool descriptions。
7. RAG with shared documents。
8. Long-context question answering。
9. 多候选答案生成。
10. 结构化抽取中的固定 schema prompt。

共同特征是：

共享前缀长 + 分支多 + 模板稳定 + 重复调用多

如果满足这些特征，RadixAttention 很可能明显降低 TTFT 和 prefill 成本。

28.29 常见性能现象

现象一：prefix cache hit rate 高，但吞吐提升不明显。

可能原因：

1. 命中的 prefix 很短。
2. 瓶颈在 decode，而不是 prefill。
3. grammar mask 或采样开销较大。
4. GPU 已经被 decode batch 占满。

5. cache 管理开销抵消了部分收益。

现象二：TTFT 下降明显，但 TPOT 没变化。

这是正常现象。RadixAttention 主要减少 prefill，TPOT 更多取决于 decode step、KV 读取、batch size 和 attention backend。

现象三：显存占用变高。

可能是 cache 保留了很多 prefix。需要看命中率、eviction、保留 token 数和实际节省 tokens。

现象四：命中率突然下降。

可能是 prompt template 改了、system prompt 注入了动态字段、tokenizer 版本变化、路由打散了 cache locality，或 cache 被频繁 eviction。

28.30 调试 RadixAttention 的思路

排查 RadixAttention 是否有效，可以看这些指标：

1. prefix cache hit rate。
2. matched prefix length。
3. saved prefill tokens。
4. radix tree node count。
5. cached token count。
6. eviction count。
7. active cached token count。
8. free KV slots。
9. TTFT p50/p95/p99。
10. prefill tokens/s。
11. decode tokens/s。

排查顺序可以是：

确认 workload 是否有共享前缀

- > 检查 tokenized prefix 是否完全一致
- > 检查 cache namespace / tenant / LoRA 是否一致
- > 检查是否被频繁 eviction
- > 检查 scheduler 是否利用 hit length
- > 对比 TTFT 和 prefill tokens/s

不要只看“开启了 RadixAttention”。开启不代表一定命中，命中也不代表一定提升端到端指标。

28.31 面试官会怎么问

问题一：RadixAttention 是什么？

回答要点：它是 SGLang runtime 的自动 KV cache 复用机制，用 radix tree 管理 token prefix 到 KV cache 的映射，新请求通过最长前缀匹配复用已有 KV，只对未命中 suffix 做 prefill。

问题二：RadixAttention 是 attention kernel 吗？

回答要点：不是。它不是新的 attention 数学公式，也不是 FlashAttention 替代品，而是 runtime 层的 prefix cache 管理机制。真正的 attention 计算仍由 attention backend/kernel 执行。

问题三：为什么用 radix tree？

回答要点：复杂 LLM programs 的共享前缀经常呈树状结构，radix tree 可以紧凑表示大量 token 序列的共享前缀，支持 longest prefix match、split、insert 和 eviction。

问题四：RadixAttention 主要优化什么指标？

回答要点：主要减少重复 prefill，降低 TTFT，提高 input token 处理效率和整体吞吐；对 TPOT 的影响通常是间接的。

问题五：什么情况下 RadixAttention 收益不大？

回答要点：请求之间没有共享前缀、共享前缀很短、prompt 模板不稳定、cache 被频繁淘汰、瓶颈主要在 decode 或网络/检索/tokenizer 时，收益会有限。

28.32 标准回答模板

如果面试官问“讲讲 SGLang 的 RadixAttention”，可以这样回答：

RadixAttention 是 SGLang Runtime 中用于自动 KV cache 复用的机制。它不是一种新的 attention kernel，而是用 radix tree 管理

复杂 LLM 程序里经常有多次 generation、分支、self-consistency、Tree-of-Thought、多轮 chat 和 agent trajectory，这些调用之

RadixAttention 的做法是：请求 tokenize 后，runtime 在 radix tree 中做最长前缀匹配，找到已经缓存的 prefix KV，只对未命中

它的主要收益是减少 prefill 计算、降低 TTFT，并提升复杂 LLM program 的吞吐。它特别适合 few-shot、self-consistency、Tree-of-Thought、多轮对话、agent 和共享长文档 RAG 等场景。但它匹配的是 token prefix 完全一致，不是语义相似

28.33 RadixAttention 公式、split / eviction 门禁和可运行 demo

把第 i 个请求的 token 序列记为：

$$X_i = (x_{i,1}, x_{i,2}, \dots, x_{i,L_i})$$

RadixAttention 在树中找到的最长 token prefix 命中长度记为 H_i 。如果底层 KV cache 以 page / block 为粒度，page size 为 S ，则 page 对齐后真正可复用的 token 数可以写成：

$$H_i^{\text{page}} = S \left\lfloor \frac{H_i}{S} \right\rfloor$$

当 page size 为 1 时， $H_i^{\text{page}} = H_i$ ，也就是 token-level prefix reuse。page size 更大时，只有完整 page 可以进入 prefix cache；这也是为什么短 prompt 或不足一页的 suffix 可能无法被缓存命中。

实际需要 prefill 的 token 数为：

$$P_i^{\text{run}} = L_i - H_i^{\text{page}}$$

总 saved prefill tokens 为：

$$S_{\text{save}} = \sum_i H_i^{\text{page}}$$

复用比例为：

$$R_{\text{reuse}} = \frac{S_{\text{save}}}{\max(1, \sum_i L_i)}$$

如果 radix tree 中每个节点 v 关联一段 token segment，节点占用 KV token 数记为 m_v ，当前缓存 token 数可以写成：

$$M_{\text{cache}} = \sum_{v \in \mathcal{T}} m_v$$

Eviction 只能选择没有 active request 引用的节点:

$$E = \{v \mid \text{ref}_v = 0, v \text{ is leaf}\}$$

教学版 RadixAttention 门禁可以写成:

$$G_{\text{radix}} = G_{\text{match}} G_{\text{split}} G_{\text{page}} G_{\text{ref}} G_{\text{evict}} G_{\text{schedule}} G_{\text{metric}}$$

其中:

1. G_{match} : 能做 longest prefix match, 并输出 hit tokens。
2. G_{split} : 新路径在旧边中间分叉时, 能 split 出共享中间节点。
3. G_{page} : 能说明 page size / block size 对完整页命中的影响。
4. G_{ref} : active request 正在用的节点不能被淘汰。
5. G_{evict} : 显存紧张时优先淘汰 ref count 为 0 的叶子节点。
6. G_{schedule} : scheduler 使用 P_i^{run} 而不是原始 prompt 长度估算 prefill 成本。
7. G_{metric} : 能输出 saved prefill tokens、reuse ratio、split count、cached tokens 和 eviction list。

下面这个 0 依赖 demo 用压缩 radix tree 模拟 4 条请求: 第一条长 prompt 建树, 第二条和第一条共享 root 后分叉, 第三条在第二条边中间分叉并触发 split, 第四条沿第一条路径继续变深。随后 demo pin 住第四条路径, 执行 leaf LRU eviction, 证明 active path 不会被淘汰。

```
from dataclasses import dataclass, field
```

```
@dataclass
class RadixNode:
    segment: tuple
    name: str
    parent: object = None
    children: dict = field(default_factory=dict)
    ref_count: int = 0
    last_access: int = 0
    kv_tokens: int = 0

    def is_leaf(self):
        return not self.children

class ToyRadixAttentionCache:
    def __init__(self, page_size=1, capacity_tokens=18):
        self.root = RadixNode((), "root")
        self.page_size = page_size
        self.capacity_tokens = capacity_tokens
        self.clock = 0
        self.split_count = 0
        self.evicted_nodes = []

    @staticmethod
    def common_prefix_len(left, right):
        count = 0
        limit = min(len(left), len(right))
        while count < limit and left[count] == right[count]:
            count += 1
        return count
```

```

def _touch(self, node):
    self.clock += 1
    node.last_access = self.clock

def _walk(self):
    stack = list(self.root.children.values())
    while stack:
        node = stack.pop()
        yield node
        stack.extend(node.children.values())

def node_count(self):
    return sum(1 for _ in self._walk())

def cached_tokens(self):
    return sum(node.kv_tokens for node in self._walk())

def lookup(self, tokens):
    node = self.root
    pos = 0
    owner = ""
    while pos < len(tokens):
        child = node.children.get(tokens[pos])
        if child is None:
            break
        hit = self.common_prefix_len(tokens[pos:], child.segment)
        if hit == 0:
            break
        pos += hit
        owner = child.name
        if hit < len(child.segment):
            break
    node = child
    page_hit = self.page_size * (pos // self.page_size)
    return pos, page_hit, owner

def insert(self, tokens, name):
    node = self.root
    pos = 0
    while pos < len(tokens):
        child = node.children.get(tokens[pos])
        if child is None:
            suffix = tokens[pos:]
            node.children[suffix[0]] = RadixNode(
                suffix, name, parent=node, kv_tokens=len(suffix)
            )
            self._touch(node.children[suffix[0]])
            return len(suffix)

        hit = self.common_prefix_len(tokens[pos:], child.segment)
        if hit == len(child.segment):
            self._touch(child)
            node = child
            pos += hit

```

```

        continue

    self.split_count += 1
    old_suffix = child.segment[hit:]
    old_child = RadixNode(
        old_suffix,
        child.name + "_tail",
        parent=child,
        children=child.children,
        ref_count=child.ref_count,
        last_access=child.last_access,
        kv_tokens=len(old_suffix),
    )
    for grandchild in old_child.children.values():
        grandchild.parent = old_child

    child.segment = child.segment[:hit]
    child.name = child.name + "_shared"
    child.children = {old_suffix[0]: old_child}
    child.ref_count = 0
    child.kv_tokens = len(child.segment)
    pos += hit

    if pos == len(tokens):
        self._touch(child)
        return 0

    new_suffix = tokens[pos:]
    new_child = RadixNode(
        new_suffix, name, parent=child, kv_tokens=len(new_suffix)
    )
    child.children[new_suffix[0]] = new_child
    self._touch(child)
    self._touch(new_child)
    return len(new_suffix)

self._touch(node)
return 0

def pin_path(self, tokens):
    node = self.root
    pos = 0
    pinned = []
    while pos < len(tokens):
        child = node.children.get(tokens[pos])
        if child is None:
            break
        hit = self.common_prefix_len(tokens[pos:], child.segment)
        if hit != len(child.segment):
            break
        child.ref_count += 1
        pinned.append(child.name)
        pos += hit
        node = child
    return pinned

```



```

def evict_until_fit(self):
    while self.cached_tokens() > self.capacity_tokens:
        leaves = [
            node for node in self._walk()
            if node.is_leaf() and node.ref_count == 0
        ]
        if not leaves:
            break
        victim = min(leaves, key=lambda node: node.last_access)
        del victim.parent.children[victim.segment[0]]
        self.evicted_nodes.append(victim.name)
    return list(self.evicted_nodes)

ROOT = tuple(f"root_{i}" for i in range(8))
A = ("clause_a", "risk", "draft", "tail")
B = ("clause_b", "risk", "draft", "tail")
C = ("clause_b", "risk", "law", "tail")
D = ("deep", "followup", "json", "tail")

requests = [
    ("prompt_a", ROOT + A),
    ("prompt_b", ROOT + B),
    ("prompt_c", ROOT + C),
    ("prompt_d", ROOT + A + D),
]

cache = ToyRadixAttentionCache(page_size=1, capacity_tokens=18)
rows = []
for name, tokens in requests:
    token_hit, page_hit, owner = cache.lookup(tokens)
    new_tokens = cache.insert(tokens, name)
    rows.append(
        {
            "name": name,
            "input_tokens": len(tokens),
            "token_hit": token_hit,
            "page_hit": page_hit,
            "hit_owner": owner,
            "run_prefill_tokens": len(tokens) - page_hit,
            "new_cached_tokens": new_tokens,
        }
    )

pinned_nodes = cache.pin_path(ROOT + A + D)
before_evict = {
    "node_count": cache.node_count(),
    "cached_tokens": cache.cached_tokens(),
    "split_count": cache.split_count,
}
evicted = cache.evict_until_fit()
after_evict = {
    "node_count": cache.node_count(),
    "cached_tokens": cache.cached_tokens(),

```

```

    "evicted_nodes": evicted,
}

naive_prefill = sum(row["input_tokens"] for row in rows)
radix_prefill = sum(row["run_prefill_tokens"] for row in rows)
saved_prefill = naive_prefill - radix_prefill
page_examples = {
    "page_size": 4,
    "three_tokens_cacheable": 4 * (3 // 4),
    "nine_tokens_cacheable": 4 * (9 // 4),
    "ten_token_hit_reusable": 4 * (10 // 4),
}
summary = {
    "naive_prefill_tokens": naive_prefill,
    "radix_prefill_tokens": radix_prefill,
    "saved_prefill_tokens": saved_prefill,
    "reuse_ratio": round(saved_prefill / max(1, naive_prefill), 3),
    "before_evict": before_evict,
    "after_evict": after_evict,
    "pinned_nodes": pinned_nodes,
    "page_examples": page_examples,
}
gates = {
    "longest_prefix_match": rows[2]["token_hit"] == 10,
    "split_visible": before_evict["split_count"] == 2,
    "page_boundary_checked": page_examples["ten_token_hit_reusable"] == 8,
    "ref_count_blocks_active_path": "prompt_d" not in evicted,
    "leaf_eviction_visible": evicted == ["prompt_b_tail", "prompt_c"],
    "scheduler_uses_suffix_cost": rows[3]["run_prefill_tokens"] == 4,
    "metrics_ready": summary["reuse_ratio"] > 0.5,
}
gates["radix_attention_gate"] = all(gates.values())

print("radix_attention_rows=", rows)
print("radix_attention_summary=", summary)
print("radix_attention_gates=", gates)

```

这段 demo 应该输出：

```

radix_attention_summary= {'naive_prefill_tokens': 52, 'radix_prefill_tokens': 22, 'saved_prefill_tokens': 30, 'reuse_ratio': 0.577, 'before_evict': 10, 'after_evict': 8, 'pinned_nodes': 0, 'page_examples': {'page_size': 4, 'three_tokens_cacheable': 3, 'nine_tokens_cacheable': 9, 'ten_token_hit_reusable': 8}, 'metrics_ready': True}
radix_attention_gates= {'longest_prefix_match': True, 'split_visible': True, 'page_boundary_checked': True, 'ref_count_blocks_active_path': True, 'leaf_eviction_visible': True, 'scheduler_uses_suffix_cost': True, 'metrics_ready': True}

```

读这段代码时要注意：

1. prompt_c 的 token_hit=10 说明 radix tree 能在边中间发现共享 prefix，并通过 split 把中间共享段变成节点。
2. radix_prefill_tokens=22 而不是 52，说明 scheduler 应该按未命中的 suffix 估算 prefill 成本。
3. page_examples 说明 page size 为 4 时，3 个 token 不足一页不能作为完整 page 命中，10 个 token 的命中也只能按 8 个完整 page tokens 复用。
4. prompt_d 被 pin 后没有被 eviction，说明 ref count 是正确性边界，不是性能优化选项。
5. eviction 只删掉 ref count 为 0 的叶子，避免破坏共享内部节点和 active path。

28.34 小练习

1. 给定三条 token 序列 [A B C D]、[A B E F]、[A B E G]，画出 trie 和 radix tree。
2. 解释 RadixAttention 为什么需要 split 节点。
3. 用 self-consistency 画一个 prefix sharing 树。

4. 用 Tree-of-Thought 画一个两层搜索树，并标出哪些 KV cache 可以复用。
5. 解释为什么 RadixAttention 主要优化 TTFT，而不是直接优化 TPOT。
6. 列出会导致 token prefix 无法命中的 5 个工程原因。
7. 设计一个 RadixAttention dashboard，至少包含 hit length、saved tokens、eviction 和 TTFT。

28.35 本章总结

RadixAttention 的核心不是改变 Transformer attention 的计算公式，而是在 serving runtime 中自动管理 prefix sharing。

它用 radix tree 表达 token 序列之间的共享前缀，用 longest prefix match 找到可复用 KV cache，只对未命中 suffix 做 prefill；当新路径出现时，通过 split 和 insert 维护树结构；当显存紧张时，通过 ref count、LRU 和叶子淘汰等策略释放 cache。

理解 RadixAttention 后，就能解释为什么 SGLang 特别适合复杂 LLM programs：这些程序天然包含多次 generation、分支、并行、多轮历史和共享上下文。下一章会继续深入 SGLang scheduler，看看 runtime 如何在 cache hit、prefill、decode、KV capacity 和公平性之间做调度取舍。

第 29 章 SGLang Scheduler 设计

上一章讲了 RadixAttention：SGLang Runtime 用 radix tree 管理 token prefix 到 KV cache 的映射，通过最长前缀匹配复用已有 KV，只对未命中 suffix 做 prefill。

但有 cache 还不够。

Runtime 每一轮都要决定：哪些请求继续 decode，哪些新请求可以 prefill，哪些请求因为 KV cache 不够要等待，哪些 cache 命中请求应该优先，哪些长 prompt 要切分，哪些 structured output 请求会带来额外开销。

这些决策由 scheduler 完成。

一句话概括：

SGLang scheduler 是 runtime 的资源分配中枢，它在每个 engine step 结合 waiting queue、running requests、RadixAttention 命中、KV cache 容量、token budget、structured output 状态和公平性策略，决定本轮执行哪些 prefill 和 decode 工作。

29.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准，主要参考五类公开资料：

1. SGLang 论文《SGLang: Efficient Execution of Structured Language Model Programs》对 frontend language、runtime、RadixAttention、KV cache reuse 和 structured decoding 的系统分层说明。
2. SGLang Server Arguments 文档对 --mem-fraction-static、--max-running-requests、--max-total-tokens、--chunked-prefill-size、--max-prefill-tokens、--schedule-policy、--schedule-conservativeness、--disable-radix-cache 和 --enable-metrics 等 serving / scheduler 参数的公开口径。
3. SGLang Hyperparameter Tuning 文档对 #queue-req、token usage、KV cache pool full / retract、--schedule-conservativeness、--mem-fraction-static、--chunked-prefill-size、--max-running-requests 和 --schedule-policy lpm 的调优建议。
4. SGLang Structured Outputs 文档对 JSON schema、regex、EBNF、structural tag、grammar backend 和约束输出接口的说明。
5. SGLang Production Metrics / Attention Backend 文档对 prompt tokens、generation tokens、token usage、cache hit rate、TTFT、E2E latency、TPOT、running requests、queue requests，以及 page size / prefix cache 完整页命中的说明。

本章只讲 SGLang-like scheduler 的教学版机制：waiting / running 状态、decode-first、cache-aware admission、prefill / decode token budget、sequence budget、KV budget、chunked prefill、structured output 开销、abort cleanup、eviction 与公平性。它不绑定某个 SGLang 版本的真实源码类名、真实 schedule policy 实现、CUDA graph、overlap scheduler、DP attention、PD disaggregation、LoRA adapter 调度、MoE 路由或生产参数全集。本章 demo 用纯 Python 模拟调度决策和 KV slot 账本，不执行真实模型 forward。

参考资料:

1. SGLang 论文: <https://arxiv.org/abs/2312.07104>
2. SGLang Server Arguments: https://docs.sglang.io/docs/advanced_features/server_arguments.md
3. SGLang Hyperparameter Tuning: https://docs.sglang.io/docs/advanced_features/hyperparameter_tuning.md
4. SGLang Structured Outputs: https://docs.sglang.io/docs/advanced_features/structured_outputs.md
5. SGLang Production Metrics: https://docs.sglang.io/docs/references/production_metrics.md
6. SGLang Attention Backend: https://docs.sglang.io/docs/advanced_features/attention_backend.md

29.1 本章目标

读完本章, 你应该能讲清:

1. SGLang scheduler 和普通 continuous batching scheduler 的共同点。
2. SGLang scheduler 为什么必须理解 RadixAttention 命中长度。
3. waiting、running、prefill、decode、finished 等状态如何变化。
4. token budget、sequence budget、KV budget 分别约束什么。
5. cache-aware scheduling 如何影响 TTFT、吞吐和公平性。
6. structured output、streaming、abort、frontend program 分支为什么会影响调度。
7. 面试中如何画出 SGLang scheduler 的核心流程。

29.2 先看 scheduler 在 runtime 中的位置

SGLang Runtime 的主链路可以简化成:

API / offline engine / frontend program

- > request parsing
- > tokenization
- > request state
- > scheduler
- > RadixAttention / memory pool
- > model runner
- > sampler / grammar backend
- > output / streaming

Scheduler 位于 request state 和 model execution 之间。

它不直接执行 GPU kernel, 也不直接采样 token。

它负责产生本轮执行计划:

本轮哪些请求做 prefill?

本轮哪些请求做 decode?

每个请求处理多少 token?

需要分配哪些 KV cache?

复用哪些 cached prefix?

哪些请求要等待、暂停、结束或清理?

如果说 model runner 是“执行器”, memory pool 是“显存账本”, RadixAttention 是“prefix cache 索引”, 那么 scheduler 就是“调度决策者”。

29.3 Scheduler 的输入

每个 engine step, scheduler 通常需要这些输入:

1. 新到达的 requests。

2. waiting queue。
3. running requests。
4. finished 或 aborted requests。
5. 每个请求的 token ids。
6. 每个请求已计算 token 数。
7. 每个请求的 output token 数。
8. sampling parameters。
9. structured output / grammar state。
10. RadixAttention 当前树状态。
11. 每个 waiting 请求的 cache hit length。
12. KV memory pool 的 free slots。
13. token budget。
14. sequence budget。
15. 调度策略配置。

用伪代码表示：

```
class SchedulerInput:
    waiting: list[Request]
    running: list[Request]
    radix_cache: RadixCache
    memory_pool: KVMemoryPool
    max_batched_tokens: int
    max_running_requests: int
    policy: SchedulingPolicy
```

这些输入说明 scheduler 不是只看 “请求列表”。

它要同时看计算预算、显存预算、cache 命中、请求状态和输出约束。

29.4 Scheduler 的输出

Scheduler 的输出是本轮执行计划。

一个简化输出可以是：

```
class SchedulerOutput:
    prefill_batch: list[PrefillWork]
    decode_batch: list[DecodeWork]
    cache_hits: dict[str, CacheHitInfo]
    kv_allocations: dict[str, list[KVSlot]]
    evictions: list[CacheNode]
    aborted: list[str]
```

更直观地说：

本轮执行计划

- > prefill 哪些 tokens
- > decode 哪些 requests
- > 读取哪些 cached KV
- > 写入哪些 KV slots
- > 哪些请求本轮不执行
- > 哪些资源要释放或淘汰

这个输出会同时交给：

1. memory pool, 用于分配、追加和释放 KV cache。
2. model runner, 用于准备 input ids、positions、slot mapping 和 attention metadata。
3. sampler, 用于知道哪些 logits 需要采样。
4. output layer, 用于更新请求和流式返回。

所以 scheduler 输出不是简单的 request ids, 而是 execution metadata 的源头。

29.5 请求状态机

SGLang Runtime 中, 一个请求可以用下面的状态机理解:

WAITING

- > PREFILLING
- > DECODING
- > FINISHED

WAITING

- > PREFILLING
- > WAITING / PREEMPTED
- > DECODING
- > ABORTED / FAILED

每个状态含义:

WAITING: 请求已进入 runtime, 但还没有完成 prefill。

PREFILLING: 本轮正在处理 prompt 或未命中 suffix。

DECODING: 已经有历史 KV, 每轮生成新 token。

FINISHED: 遇到 EOS、stop、max_new_tokens、grammar 完成或其他正常结束条件。

ABORTED: 客户端断开、用户取消、上层超时或系统主动中止。

FAILED: 内部错误、KV 分配失败、模型执行错误等。

和普通 serving 相比, SGLang 的请求状态还可能关联:

1. RadixAttention 命中路径。
2. 当前 prefix cache 引用。
3. frontend program 中的变量或分支。
4. grammar state。
5. 多个并行 generation call。

所以“一个请求”在 SGLang 里经常不仅是一段 prompt, 而是复杂程序执行过程中的一个 generation 节点。

29.6 Engine step: 调度的基本节拍

Scheduler 不是只在请求到来时运行一次, 而是在每个 engine step 运行。

一个简化主循环:

while True:

```
    new_requests = receive_new_requests()
    waiting_queue.extend(new_requests)
```

```
    cleanup_finished_and_aborted()
```

```
    schedule = scheduler.schedule(
        waiting=waiting_queue,
        running=running_requests,
        radix_cache=radix_cache,
        memory_pool=memory_pool,
    )
```

```
    outputs = model_runner.forward(schedule)
    next_tokens = sampler.sample(outputs)
```

```
update_request_states(next_tokens)
stream_outputs(next_tokens)
```

这个循环就是 serving engine 的心跳。

每一轮都可能发生：

1. 新请求进入 waiting。
2. 老请求继续 decode。
3. 短请求结束并释放资源。
4. 长请求追加 KV。
5. cache 命中请求进入 prefill。
6. cache miss 请求继续等待。
7. structured output 请求更新 grammar state。

29.7 Prefill 和 decode 为什么要一起调度

Prefill 和 decode 的资源特点不同。

Prefill:

1. 一次处理多个 prompt token。
2. 影响 TTFT。
3. 计算量大。
4. 写入大量 KV cache。
5. 长 prompt 可能占用较长 GPU 时间。

Decode:

1. 每个请求每轮通常一个 token。
2. 影响 TPOT 和 streaming 平滑度。
3. 更依赖 KV cache 读取。
4. batch 动态变化。
5. 会逐步追加 KV cache。

如果 scheduler 过度偏向 prefill:

新请求 TTFT 变好

正在 streaming 的请求 TPOT 抖动

用户看到输出卡顿

如果 scheduler 过度偏向 decode:

已有请求输出平滑

waiting queue 堆积

新请求 TTFT 变差

因此 scheduler 的核心工作就是在 TTFT、TPOT、吞吐和显存之间做取舍。

29.8 最小调度策略

一个最小策略可以这样写：

```
def schedule_step(waiting, running, max_tokens, max_seqs):
    scheduled_decode = []
    scheduled_prefill = []
    used_tokens = 0
    used_seqs = 0

    for req in running:
        if used_tokens + 1 > max_tokens:
            break
```

```

    if used_seqs + 1 > max_seqs:
        break
    scheduled_decode.append(req)
    used_tokens += 1
    used_seqs += 1

while waiting:
    req = waiting.peek()
    prefill_tokens = len(req.input_ids)
    if used_tokens + prefill_tokens > max_tokens:
        break
    if used_seqs + 1 > max_seqs:
        break
    scheduled_prefill.append(waiting.pop())
    used_tokens += prefill_tokens
    used_seqs += 1

return scheduled_prefill, scheduled_decode

```

这只是教学版。

它忽略了：

1. RadixAttention 命中。
2. KV cache 容量。
3. chunked prefill。
4. structured output 开销。
5. 请求优先级。
6. cache eviction。
7. abort 和 cleanup。
8. frontend program 分支。

真实 SGLang scheduler 的关键就是把这些因素加进去。

29.9 Token budget: 本轮最多算多少 token

Token budget 限制本轮模型 forward 的 token 工作量。

可以理解成：

```

scheduled_tokens = prefill_tokens + decode_tokens
scheduled_tokens <= max_batched_tokens

```

Decode tokens 通常近似等于本轮 decode 请求数：

```
decode_tokens = number_of_running_requests
```

Prefill tokens 取决于新请求需要计算的 prompt tokens。

在 SGLang 中，重点是：prefill tokens 应该按未命中 suffix 计算，而不是总 prompt 长度。

例如：

```

prompt length = 12000
cache hit length = 10000
uncached suffix = 2000

```

对本轮 prefill 计算来说，更相关的是 2000，而不是 12000。

这就是 RadixAttention 和 scheduler 的交点。

29.10 Sequence budget: 本轮最多跑多少序列

Sequence budget 限制本轮 batch 中的请求数量。

即使 token budget 还够，也不能无限增加 sequence 数。

原因包括：

1. 每个 sequence 都有 metadata 开销。
2. attention backend 对 batch size 有上限或性能拐点。
3. sampler 要处理每个 sequence 的 logits。
4. structured output 每个 sequence 可能有 grammar state。
5. streaming 输出也有 per-request 开销。

可以写成：

```
scheduled_sequences <= max_num_seqs
```

Sequence budget 控制的是 “请求数维度” 的复杂度，token budget 控制的是 “token 计算量维度” 的复杂度。

两者都必须有。

29.11 KV budget: 能算不代表放得下

Token budget 和 sequence budget 通过了，不代表请求能执行。

还要看 KV cache 能不能放下。

每个新 prefill token 都会写 KV cache。

每个 decode token 也会追加 KV cache。

因此 scheduler 每轮要检查：

```
required_new_kv_slots <= free_kv_slots + evictable_cached_slots
```

如果 KV cache 不够，有几种选择：

1. 不接纳新的 prefill 请求。
2. 减少本轮 prefill tokens。
3. 做 cache eviction。
4. 抢占或暂停部分请求。
5. 返回过载错误或排队等待。

SGLang 因为有 RadixAttention，还要注意：有些 KV cache 虽然没有 active request 引用，但作为 prefix cache 保留在 radix tree 中。

这些 cache 是否可以淘汰，要由 memory pool 和 RadixAttention 协同决定。

29.12 Cache hit length 如何进入调度

对 waiting request，scheduler 可以先询问 RadixAttention：

这个请求能命中多少 token？

命中的 KV cache 是否仍在 memory pool 中？

命中的节点是否可以 pin 住？

未命中 suffix 需要多少新 KV slots？

得到结果：

```
class CacheHitInfo:
    hit_len: int
    suffix_len: int
```

```
matched_nodes: List[RadixNode]
kv_refs: List[KVRef]
```

然后 scheduler 用 `suffix_len` 估算 prefill cost。

伪代码：

```
hit = radix_cache.match(req.input_ids)
prefill_cost = len(req.input_ids) - hit.hit_len
kv_cost = prefill_cost + req.expected_decode_reserve
```

```
if can_fit(prefill_cost, kv_cost):
    pin(hit.matched_nodes)
    schedule_prefill(req, hit)
```

这里的 `pin` 很关键。

一旦 scheduler 决定复用某段 prefix，就必须防止这段 KV 在本轮执行前被淘汰。

29.13 Cache-aware admission

Admission 指 waiting 请求能不能进入本轮执行。

普通 admission 可能看：

```
prompt_len <= remaining_token_budget
required_kv <= free_kv
```

Cache-aware admission 应该看：

```
uncached_suffix_len <= remaining_token_budget
new_kv_required <= free_kv_after_possible_eviction
```

例如：

```
R1: prompt 10000, hit 9000, suffix 1000
R2: prompt 3000, hit 0, suffix 3000
```

如果只看 prompt 长度，R2 更容易进入。

如果看实际 prefill cost，R1 更便宜。

Cache-aware admission 可以提升吞吐和 TTFT，但要小心公平性。如果一直优先 cache hit 请求，cache miss 请求可能长期等待。

29.14 Decode-first 策略

很多 serving scheduler 会倾向于先保证 running decode。

原因是：

1. Running 请求已经占用了 KV cache。
2. Decode 直接影响 streaming 用户体验。
3. 每轮 decode 不推进，TPOT 就会上升。
4. Decode 请求通常每轮只需要一个 token，比较容易塞进 budget。

Decode-first 的直觉策略：

先为 running requests 安排 decode
再用剩余 budget 安排 waiting prefill

优点：

1. Streaming 更平滑。
2. 已进入系统的请求更快完成。

3. KV cache 占用更快释放。

缺点：

1. 高负载时 waiting queue 可能堆积。
2. 新请求 TTFT 可能变差。
3. 长时间 decode-heavy 时 prefill 饥饿。

所以 decode-first 通常还需要配合 prefill 配额或 aging 策略。

29.15 Prefill 配额和防饥饿

为了避免 waiting 请求长期得不到 prefill, scheduler 可以给 prefill 留预算。

例如：

每轮最多 80% token budget 给 decode
至少保留 20% 给 prefill

或者：

waiting 请求等待超过阈值后提升优先级

这类策略解决的是公平性。

简单 aging 伪代码：

```
def priority(req, now):  
    wait_time = now - req.arrival_time  
    cache_bonus = req.cache_hit_len * 0.001  
    return wait_time + cache_bonus
```

这里 cache hit 可以加分，但 wait time 也会持续增长，避免 miss 请求永远被饿死。

真实系统策略会更复杂，但思想类似：不能只追求局部吞吐最大化。

29.16 Chunked prefill

长 prompt prefill 可能阻塞 decode。

例如：

一个 64000-token prompt 进入系统

如果一次性 prefill, GPU 可能长时间被这个请求占用，其他 streaming 请求 TPOT 抖动。

Chunked prefill 的思想是把长 prompt 拆成多个 chunk：

```
chunk 1: tokens 0-4095  
chunk 2: tokens 4096-8191  
chunk 3: tokens 8192-12287  
...
```

每个 engine step 只处理一部分 prefill, 让 decode 有机会穿插执行。

SGLang 中如果有 RadixAttention 命中，chunked prefill 应该作用在未命中 suffix 上：

```
prompt length = 64000  
cache hit = 48000  
suffix = 16000
```

chunked prefill 只处理 suffix 的 16000 tokens

Chunked prefill 的收益：

1. 降低长 prompt 对 TPOT 的冲击。

2. 改善 p95/p99 streaming 抖动。
3. 让 scheduler 更细粒度地混合 prefill 和 decode。

代价：

1. 请求完成 prefill 需要多个 step。
2. 状态管理更复杂。
3. 可能增加调度和 metadata 开销。

29.17 Structured output 对调度的影响

Structured output 看起来是 sampler 或 grammar backend 的事，但它也会影响 scheduler。

原因包括：

1. 每个请求有独立 grammar state。
2. 每轮 decode 需要计算合法 token mask。
3. 复杂 schema 可能增加 CPU/GPU 开销。
4. 某些约束会改变结束条件。
5. Tool call 或 structural tag 可能影响后续 frontend program 分支。

例如 JSON schema 请求可能在生成完整 JSON 后结束，而不是只看 EOS。

Scheduler 至少要知道：

1. 这个请求是否启用 grammar constraint。
2. 它是否可能带来额外 per-step 开销。
3. 它是否已经被 grammar state 判定完成。
4. 它的 max_new_tokens 是否只是上限，不是实际结束时刻。

如果一个 batch 中结构化请求很多，decode step latency 可能上升。

29.18 Frontend program 分支对调度的影响

SGLang 的特殊之处在于，它不只服务独立 chat 请求，还可能执行 frontend language 产生的复杂程序。

例如：

```
gen root
-> fork 4 branches
    -> gen branch answer
-> merge
-> gen final answer
```

这里会产生多个 generation work items。

这些 work items 可能：

1. 共享同一个 prefix。
2. 可以并行执行。
3. 有依赖关系。
4. 某些分支完成后触发后续请求。
5. 某些分支失败后取消其他分支。

Scheduler 不一定直接理解完整业务语义，但 runtime 需要能处理这种动态生成的请求流。

这也是 SGLang scheduler 比普通单请求 server 更复杂的地方：请求不是只从 HTTP 入口来，还可能由正在执行的 LLM program 派生出来。

29.19 Streaming 和 abort

Streaming 请求需要稳定 decode。

如果 scheduler 长时间不调度某个 running 请求，用户看到的输出会卡顿。

同时，streaming 客户端可能断开。

断开后，runtime 必须 abort 请求：

```
client disconnect
-> mark request aborted
-> remove from running/waiting
-> release active KV refs
-> decide whether generated prefix cache can remain
```

注意最后一点：请求 abort 不一定意味着所有 KV 都必须立即丢弃。

如果已经生成的前缀有复用价值，并且隔离策略允许，runtime 可以保留部分 cache。

但 active request 引用必须清理，否则会造成显存泄漏。

29.20 Eviction 和调度的交互

当 KV budget 不够时，scheduler 可能触发 eviction。

Eviction 不是简单删缓存，它要满足：

1. 不能淘汰 active request 正在用的 KV。
2. 尽量淘汰 ref count 为 0 的 radix tree 叶子。
3. 淘汰后要更新 radix tree。
4. 淘汰后要更新 memory pool free slots。
5. 不能让已计划执行的 cache hit 失效。

一个安全顺序是：

1. 估算本轮需要 KV slots
2. 找到可淘汰 cache nodes
3. evict 并释放 slots
4. pin 本轮要复用的 cached prefix
5. allocate 新 slots
6. 生成 schedule

如果顺序错了，可能出现刚命中的 cache 被淘汰，或者 model runner 读取无效 KV。

29.21 Tail latency: p99 为什么难

Scheduler 不只是追求平均吞吐，还要控制 p95/p99。

尾延迟常见来源：

1. 长 prompt 突然进入。
2. cache miss 请求排队。
3. KV cache 接近满导致频繁 eviction。
4. decode batch 过大导致 step latency 上升。
5. structured output 请求集中出现。
6. tokenizer 或前置 RAG 抖动。
7. 某些请求生成特别长。
8. abort cleanup 不及时。

Scheduler 能做的缓解包括：

1. 限制单 step prefill tokens。

2. chunked prefill。
3. 控制 running batch size。
4. aging 防饥饿。
5. 根据 cache hit 调整 admission。
6. 对超长请求做隔离队列。
7. 对高优先级流量使用独立 runtime 或队列。

尾延迟不是一个参数能解决的，它是调度、显存、流量形态和上游链路共同作用的结果。

29.22 一个完整调度例子

假设当前状态：

running:

R1, R2, R3 # 都在 decode

waiting:

W1: prompt 12000, cache hit 10000, suffix 2000

W2: prompt 3000, cache hit 0, suffix 3000

W3: prompt 8000, cache hit 7800, suffix 200

budget:

max tokens this step = 4096

max seqs this step = 6

Decode-first 后：

schedule decode: R1, R2, R3

used tokens = 3

used seqs = 3

remaining tokens = 4093

remaining seqs = 3

如果按 prompt length 排队，可能先看 W1，发现 12000 放不下。

如果按 suffix cost 和 cache-aware admission：

W3 suffix 200 -> 放入 prefill

W1 suffix 2000 -> 放入 prefill

W2 suffix 3000 -> 剩余 tokens 不够，等待

本轮计划：

decode: R1, R2, R3

prefill: W3 suffix 200, W1 suffix 2000

total tokens = 3 + 200 + 2000 = 2203

seqs = 5

这就是 cache-aware scheduling 的直觉：prompt 总长度不是唯一成本，未命中 suffix 才是本轮 prefill 的关键成本。

29.23 一个简化 scheduler 伪代码

下面伪代码把本章概念串起来：

```
def schedule_step(waiting, running, radix_cache, memory_pool, config):
    budget = TokenBudget(config.max_batched_tokens, config.max_num_seqs)
    scheduled_decode = []
    scheduled_prefill = []

    cleanup_aborted_and_finished(running, waiting, memory_pool)
```

```

for req in running:
    if not budget.can_add(tokens=1, seqs=1):
        break
    if not memory_pool.can_append_one_token(req):
        break
    scheduled_decode.append(req)
    budget.add(tokens=1, seqs=1)

candidates = []
for req in waiting:
    hit = radix_cache.match(req.input_ids)
    suffix_len = len(req.input_ids) - hit.hit_len
    candidates.append((req, hit, suffix_len))

candidates = order_by_policy(candidates)

for req, hit, suffix_len in candidates:
    chunk_len = min(suffix_len, config.max_prefill_chunk)
    if not budget.can_add(tokens=chunk_len, seqs=1):
        continue
    if not memory_pool.can_allocate(chunk_len):
        evict_for(chunk_len, radix_cache, memory_pool)
    if not memory_pool.can_allocate(chunk_len):
        continue

    radix_cache.pin(hit.nodes)
    slots = memory_pool.allocate(chunk_len)
    scheduled_prefill.append(PrefillWork(req, hit, slots, chunk_len))
    budget.add(tokens=chunk_len, seqs=1)

return SchedulerOutput(scheduled_prefill, scheduled_decode)

```

这不是 SGLang 真实源码，只是教学用骨架。

它表达几个关键点：

1. 先清理 finished/aborted。
2. Decode 和 prefill 都要进入同一个 token budget。
3. Waiting 请求先做 RadixAttention match。
4. 调度 prefill 看 suffix length，而不是 prompt length。
5. KV 不够时需要 eviction。
6. 复用 cache 前要 pin。
7. 最终输出的是执行计划。

29.24 常见误解

误解一：Scheduler 只是把请求凑成 batch。

不对。Scheduler 要同时处理 prefill、decode、KV cache、RadixAttention、budget、公平性、abort 和 streaming。

误解二：Cache hit 请求一定优先。

不一定。Cache hit 能降低成本，但如果过度优先，会让 cache miss 请求饥饿。真实策略要平衡吞吐和公平性。

误解三：只要 token budget 够，请求就能执行。

不对。KV cache 也要够，sequence budget 也要够，structured output 和 metadata 开销也要考虑。

误解四：Prefill 越快越好，应该尽量多塞。

不一定。Prefill 太多会影响 running decode 的 TPOT 和 streaming 平滑度。

误解五：Structured output 只影响 sampler，不影响 scheduler。

不准确。Grammar mask、schema 状态和结构化结束条件会影响每轮 decode 成本和请求生命周期。

29.25 面试官会怎么问

问题一：SGLang scheduler 和 vLLM scheduler 有什么共同点？

回答要点：都要做 iteration-level scheduling、continuous batching、prefill/decode 平衡、token budget、sequence budget、KV cache 管理和 streaming 请求状态更新。

问题二：SGLang scheduler 有什么额外特点？

回答要点：它要和 RadixAttention 协同，调度时考虑 prefix cache hit length、uncached suffix cost、cache-aware admission、radix tree eviction/pin；还要适配 SGLang frontend program 的分支、多次 generation 和 structured output。

问题三：为什么 scheduler 不能只看 prompt length？

回答要点：因为 RadixAttention 命中后，真正需要 prefill 的只是未命中 suffix。一个 12000-token prompt 如果命中 10000 tokens，本轮 prefill 成本更接近 2000 tokens。

问题四：为什么 scheduler 要平衡 prefill 和 decode？

回答要点：prefill 影响新请求 TTFT，decode 影响已有请求 TPOT 和 streaming 平滑度。过度偏向任何一边都会导致另一类指标恶化。

问题五：KV cache 不够时 scheduler 怎么办？

回答要点：可以拒绝或延后新 prefill、减少 chunk、触发 RadixAttention cache eviction、暂停/抢占部分请求或返回过载。eviction 必须避开 active request 正在使用的 KV。

29.26 标准回答模板

如果面试官问“SGLang scheduler 怎么设计”，可以这样回答：

SGLang scheduler 是 runtime 的资源分配中枢。它在每个 engine step 根据 waiting queue、running requests、token budget、seq

它和 vLLM-like continuous batching scheduler 一样，需要平衡 prefill 和 decode：prefill 影响 TTFT，decode 影响 TPOT 和 str

调度时还要检查 KV cache 是否足够。KV 不够时，可以延后请求、做 chunked prefill、触发 radix tree 叶子 cache eviction，或者

所以 SGLang scheduler 不只是凑 batch，而是在 TTFT、TPOT、吞吐、KV 显存、cache hit、公平性和复杂程序执行之间做权衡。

29.27 SGLang Scheduler 公式、门禁和可运行 demo

把第 i 个 waiting 请求的 prompt token 数记为 L_i ，RadixAttention 命中长度记为 H_i 。如果底层 page size 为 S ，则 page 对齐后的可复用 prefix token 数为：

$$H_i^{\text{page}} = S \left\lfloor \frac{H_i}{S} \right\rfloor$$

本轮 prefill 真正需要计算的 suffix token 数为：

$$P_i^{\text{run}} = \max(0, L_i - H_i^{\text{page}} - C_i)$$

其中 C_i 是这个请求之前已经 chunked prefill 完成的 suffix token 数。若本轮 chunk size 上限为 B_{chunk} ，则本轮最多 prefill：

$$Q_i = \min(P_i^{\text{run}}, B_{\text{chunk}})$$

在第 t 个 engine step 中，令 D_t 是 scheduled decode token 数， Q_t 是 scheduled prefill token 数， N_t 是 scheduled sequence 数，则 token / sequence budget 可以写成：

$$D_t + Q_t \leq B_{\text{tok}}, \quad N_t \leq B_{\text{seq}}$$

KV budget 还要满足：

$$K_t^{\text{new}} \leq F_t + E_t$$

其中 K_t^{new} 是本轮新增 KV slots， F_t 是当前 free slots， E_t 是可安全 eviction 的 cached slots。注意 E_t 不能包含 active request 正在引用或本轮刚 pin 住的 radix prefix。

为了避免 cache hit 请求长期压过 cache miss 请求，可以给 waiting request 加 aging 分数：

$$A_i(t) = (t - a_i) + \lambda H_i^{\text{page}}$$

其中 a_i 是到达 step， λ 是 cache hit bonus。 $A_i(t)$ 的第一项会随等待时间增长，保证长期等待的 cache miss 请求最终也能被调度。

教学版 SGLang scheduler 门禁可以写成：

$$G_{\text{scheduler}} = G_{\text{decode}} G_{\text{tok}} G_{\text{seq}} G_{\text{suffix}} G_{\text{kv}} G_{\text{fair}} G_{\text{chunk}} G_{\text{grammar}} G_{\text{cleanup}} G_{\text{metric}}$$

其中：

1. G_{decode} ：running decode 能优先推进，避免 streaming TPOT 抖动。
2. G_{tok} ：prefill + decode 不超过 token budget。
3. G_{seq} ：本轮 request / sequence 数不超过 sequence budget。
4. G_{suffix} ：waiting prefill cost 使用未命中 suffix，而不是原始 prompt length。
5. G_{kv} ：KV 不足时只淘汰安全 cached nodes，不破坏 active / pinned prefix。
6. G_{fair} ：cache hit bonus 不能让 cache miss 请求无限饥饿。
7. G_{chunk} ：长 suffix 能按 chunk 分多轮 prefill。
8. G_{grammar} ：structured output 请求的 per-step grammar 开销可见。
9. G_{cleanup} ：abort / finished 请求能释放 active KV refs。
10. G_{metric} ：能输出 scheduled tokens、queue / deferred reason、KV free、eviction、grammar steps 和 gate。

下面这个 0 依赖 demo 模拟一个 engine step：两个 streaming 请求先 decode，一个已取消请求释放 KV；waiting queue 中既有 cache hit 长 prompt，也有等待很久的 cache miss 请求。Scheduler 用 aging + cache bonus 排序，用 suffix cost 而不是 prompt length 计入 budget，在 KV 不够时淘汰 stale cached nodes，并把 matched radix prefix pin 住。

```
from dataclasses import dataclass
```

```
@dataclass
class Request:
    req_id: str
    prompt_len: int
    hit_len: int
```

```

arrival_step: int
max_new_tokens: int
phase: str = "WAITING"
active_slots: int = 0
generated: int = 0
grammar: bool = False
stream: bool = False
aborted: bool = False
computed_suffix: int = 0

@property
def suffix_len(self):
    return max(0, self.prompt_len - self.hit_len)

@property
def remaining_suffix(self):
    return max(0, self.suffix_len - self.computed_suffix)

@dataclass
class CacheNode:
    name: str
    slots: int
    ref_count: int = 0
    last_access: int = 0

class ToyMemoryPool:
    def __init__(self, capacity, cached_nodes):
        self.capacity = capacity
        self.cached_nodes = {node.name: node for node in cached_nodes}
        self.active_slots = 0
        self.evicted = []

    def used(self):
        return self.active_slots + sum(
            node.slots for node in self.cached_nodes.values()
        )

    def free(self):
        return self.capacity - self.used()

    def add_active(self, slots):
        self.active_slots += slots

    def release_active(self, slots):
        self.active_slots = max(0, self.active_slots - slots)

    def pin(self, node_names):
        pinned = []
        for name in node_names:
            node = self.cached_nodes.get(name)
            if node:
                node.ref_count += 1
                pinned.append(name)

```

```

    return pinned

def evict_for(self, slots, protected):
    while self.free() < slots:
        candidates = [
            node for node in self.cached_nodes.values()
            if node.ref_count == 0 and node.name not in protected
        ]
        if not candidates:
            break
        victim = min(candidates, key=lambda node: node.last_access)
        del self.cached_nodes[victim.name]
        self.evicted.append(victim.name)
    return self.free() >= slots

def allocate(self, slots, protected=()):
    if self.free() < slots:
        self.evict_for(slots, set(protected))
    if self.free() < slots:
        return False
    self.active_slots += slots
    return True

class ToyRadixScheduler:
    def __init__(self, memory, max_tokens, max_seqs, max_prefill_chunk):
        self.memory = memory
        self.max_tokens = max_tokens
        self.max_seqs = max_seqs
        self.max_prefill_chunk = max_prefill_chunk
        self.hit_bonus = 0.2
        self.trace = []

    def matched_nodes(self, req):
        if req.hit_len >= 8:
            return ["shared_root"]
        return []

    def priority(self, req, now):
        wait = now - req.arrival_step
        return wait + self.hit_bonus * req.hit_len

    def cleanup(self, running, waiting):
        cleaned = []
        kept_running = []
        for req in running:
            if req.aborted:
                self.memory.release_active(req.active_slots)
                cleaned.append(req.req_id)
                self.trace.append(("cleanup", req.req_id, req.active_slots))
            else:
                kept_running.append(req)
        kept_waiting = []
        for req in waiting:
            if req.aborted:

```

```

        cleaned.append(req.req_id)
        self.trace.append(("cleanup_waiting", req.req_id, 0))
    else:
        kept_waiting.append(req)
    return kept_running, kept_waiting, cleaned

def schedule_step(self, running, waiting, now):
    running, waiting, cleaned = self.cleanup(running, waiting)
    used_tokens = 0
    used_seqs = 0
    decode_rows = []
    prefill_rows = []
    pinned_nodes = []
    grammar_steps = 0
    deferred = {}

    for req in running:
        if used_tokens + 1 > self.max_tokens or used_seqs + 1 > self.max_seqs:
            deferred[req.req_id] = "decode_budget"
            continue
        if not self.memory.allocate(1):
            deferred[req.req_id] = "kv_decode_full"
            continue
        req.generated += 1
        req.phase = "DECODING" if req.generated < req.max_new_tokens else "FINISHED"
        used_tokens += 1
        used_seqs += 1
        if req.grammar:
            grammar_steps += 1
        decode_rows.append(req.req_id)
        self.trace.append(("decode", req.req_id, 1))

    candidates = sorted(
        waiting,
        key=lambda req: (-self.priority(req, now), req.remaining_suffix, req.req_id),
    )
    scheduled_waiting = set()
    for req in candidates:
        if req.remaining_suffix == 0:
            continue
        chunk = min(req.remaining_suffix, self.max_prefill_chunk)
        if used_seqs + 1 > self.max_seqs:
            deferred[req.req_id] = "sequence_budget"
            continue
        if used_tokens + chunk > self.max_tokens:
            deferred[req.req_id] = "token_budget"
            continue
        nodes = self.matched_nodes(req)
        pinned_nodes.extend(self.memory.pin(nodes))
        if not self.memory.allocate(chunk, protected=nodes):
            deferred[req.req_id] = "kv_prefill_full"
            continue
        req.computed_suffix += chunk
        req.phase = "DECODING" if req.remaining_suffix == 0 else "WAITING"
        used_tokens += chunk

```

```

        used_seqs += 1
        scheduled_waiting.add(req.req_id)
        prefill_rows.append(
            {
                "id": req.req_id,
                "prompt_len": req.prompt_len,
                "hit_len": req.hit_len,
                "chunk": chunk,
                "remaining_suffix": req.remaining_suffix,
                "priority": round(self.priority(req, now), 2),
            }
        )
        self.trace.append(("prefill", req.req_id, chunk))

    next_waiting = [
        req for req in waiting
        if req.req_id not in scheduled_waiting or req.remaining_suffix > 0
    ]
    summary = {
        "decode_ids": decode_rows,
        "prefill_ids": [row["id"] for row in prefill_rows],
        "scheduled_tokens": used_tokens,
        "scheduled_sequences": used_seqs,
        "prefill_tokens": sum(row["chunk"] for row in prefill_rows),
        "prompt_tokens_if_naive": sum(row["prompt_len"] for row in prefill_rows),
        "grammar_steps": grammar_steps,
        "cleaned_aborts": cleaned,
        "evicted_nodes": list(self.memory.evicted),
        "pinned_nodes": pinned_nodes,
        "kv_free_after": self.memory.free(),
        "deferred": deferred,
    }
    return running, next_waiting, prefill_rows, summary

running = [
    Request(
        "stream_a", 0, 0, 0, max_new_tokens=3,
        phase="DECODING", active_slots=6, grammar=True, stream=True
    ),
    Request(
        "stream_b", 0, 0, 0, max_new_tokens=2,
        phase="DECODING", active_slots=5, stream=True
    ),
    Request(
        "cancel_c", 0, 0, 0, max_new_tokens=4,
        phase="DECODING", active_slots=4, aborted=True
    ),
]
waiting = [
    Request("w_shared", prompt_len=14, hit_len=10, arrival_step=0, max_new_tokens=3),
    Request("w_miss_old", prompt_len=9, hit_len=0, arrival_step=-7, max_new_tokens=2),
    Request(
        "w_json", prompt_len=12, hit_len=8,
        arrival_step=0, max_new_tokens=2, grammar=True
    )
]

```

```

    ),
    Request("w_tiny", prompt_len=3, hit_len=0, arrival_step=0, max_new_tokens=1),
]
cache_nodes = [
    CacheNode("shared_root", slots=10, ref_count=0, last_access=5),
    CacheNode("stale_tail", slots=4, ref_count=0, last_access=1),
    CacheNode("cold_suffix", slots=3, ref_count=0, last_access=2),
]
memory = ToyMemoryPool(capacity=32, cached_nodes=cache_nodes)
for req in running:
    memory.add_active(req.active_slots)

scheduler = ToyRadixScheduler(
    memory=memory,
    max_tokens=10,
    max_seqs=4,
    max_prefill_chunk=4,
)
running, waiting, prefill_rows, summary = scheduler.schedule_step(
    running, waiting, now=1
)
gates = {
    "decode_first": scheduler.trace[:3] == [
        ("cleanup", "cancel_c", 4),
        ("decode", "stream_a", 1),
        ("decode", "stream_b", 1),
    ],
    "token_budget_respected": summary["scheduled_tokens"] <= scheduler.max_tokens,
    "sequence_budget_respected": summary["scheduled_sequences"] <= scheduler.max_seqs,
    "suffix_cost_used": (
        summary["prefill_tokens"] == 8 and summary["prompt_tokens_if_naive"] == 23
    ),
    "kv_eviction_safe": "shared_root" not in summary["evicted_nodes"],
    "aging_prevents_starvation": summary["prefill_ids"][0] == "w_miss_old",
    "chunked_prefill_visible": prefill_rows[0]["remaining_suffix"] == 5,
    "grammar_cost_tracked": summary["grammar_steps"] == 1,
    "abort_cleanup_releases_slots": summary["cleaned_aborts"] == ["cancel_c"],
    "metrics_ready": summary["kv_free_after"] == 1 and bool(summary["deferred"]),
}
gates["sglang_scheduler_gate"] = all(gates.values())

print("sglang_scheduler_prefill_rows=", prefill_rows)
print("sglang_scheduler_summary=", summary)
print("sglang_scheduler_gates=", gates)

```

这段 demo 应该输出：

```

sglang_scheduler_prefill_rows= [{'id': 'w_miss_old', 'prompt_len': 9, 'hit_len': 0, 'chunk': 4, 'remaining_suffix': 5, 'w_shared': 0}, {'id': 'w_shared', 'prompt_len': 14, 'hit_len': 10, 'chunk': 4, 'remaining_suffix': 4, 'w_shared': 1}]
sglang_scheduler_summary= {'decode_ids': ['stream_a', 'stream_b'], 'prefill_ids': ['w_miss_old', 'w_shared'], 'schedule_steps': 1, 'grammar_steps': 1, 'kv_free_after': 1, 'metrics_ready': True, 'token_budget_respected': True, 'sequence_budget_respected': True, 'suffix_cost_used': True}
sglang_scheduler_gates= {'decode_first': True, 'token_budget_respected': True, 'sequence_budget_respected': True, 'suffix_cost_used': True, 'kv_eviction_safe': True, 'aging_prevents_starvation': True, 'chunked_prefill_visible': True, 'grammar_cost_tracked': True, 'abort_cleanup_releases_slots': True, 'metrics_ready': True}

```

读这段代码时要注意：

1. cancel_c 先释放 4 个 active KV slots，再调度 stream_a 和 stream_b decode，说明 cleanup 是调度循环的一部分。
2. w_shared 的 prompt length 是 14，但命中 10 个 tokens，本轮只按 4 个 suffix tokens 计入 prefill budget。
3. w_miss_old 没有 cache hit，但因为等待时间足够长，优先于 cache hit 请求进入本轮，说明 aging 防止

cache miss 饥饿。

4. stale_tail 和 cold_suffix 被 eviction, 但 shared_root 被 pin 后没有被淘汰, 说明 KV eviction 必须避开 active / planned prefix。
5. grammar_steps=1 说明 structured output 的每步额外开销要进入 scheduler 观测指标, 而不是只在 sampler 里悄悄发生。

29.28 小练习

1. 画出 SGLang scheduler 的输入、输出和下游模块。
2. 给定 3 个 running 请求和 4 个 waiting 请求, 按 token budget 手工做一次调度。
3. 设计一个 cache-aware admission 策略, 说明如何避免 cache miss 请求饥饿。
4. 解释为什么 chunked prefill 能降低 streaming 抖动。
5. 说明 structured output 为什么会影响 scheduler 的成本估计。
6. 给出 KV cache 不够时的 4 种处理策略, 并说明优缺点。
7. 设计一个 SGLang scheduler dashboard, 包含 waiting、running、hit length、scheduled prefill tokens、scheduled decode tokens、eviction 和 p99。

29.29 本章总结

SGLang scheduler 的核心不是“把请求凑成 batch”, 而是在每个 engine step 做多目标资源分配。

它既继承了 LLM serving 中 continuous batching 和 iteration-level scheduling 的基本思想, 也加入了 SGLang 特有的 RadixAttention、cache-aware scheduling、frontend program 分支和 structured output 约束。

理解 scheduler 后, 就能把第 27 章的 runtime 总览和第 28 章的 RadixAttention 串起来: RadixAttention 提供可复用 prefix, scheduler 决定何时复用、复用谁、计算多少 suffix、是否淘汰 cache、如何平衡 prefill 与 decode。下一章会继续深入 structured generation 与 constrained decoding, 解释 JSON schema、regex、EBNF 和 grammar mask 如何进入每一步采样。

第 30 章 Structured Generation 与 Constrained Decoding

上一章讲了 SGLang scheduler: 它在每个 engine step 里平衡 prefill、decode、RadixAttention 命中、KV cache、token budget、structured output 开销和公平性。

本章进入 SGLang 另一个非常重要的能力: Structured Generation, 也就是结构化生成。

很多生产应用不希望模型自由写一段自然语言, 而是希望模型输出严格满足格式: JSON、正则表达式、EBNF grammar、固定 choices、工具调用标签、函数参数 schema 等。

如果只靠 prompt 里写“请输出 JSON”, 模型仍然可能输出解释文字、多余 markdown、缺字段、错引号、尾逗号、类型不对。生产系统通常不得不 parse、修复、重试, 延迟和失败率都会上升。

Constrained decoding 的思路是: 不要等生成完再检查格式, 而是在每一步选 token 之前, 根据 grammar state 计算哪些 token 合法, 把非法 token 的 logits mask 掉, 让模型只能在合法 token 集合中采样。

一句话概括:

Structured generation 把输出格式约束变成 runtime decoding 的一部分; constrained decoding 在每个 decode step 根据 JSON schema、regex、EBNF 或 structural tag 维护 grammar state, 并用 token mask 限制 sampler 只能选择合法 token。

30.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修要求做过资料校准, 主要参考五类公开资料:

1. SGLang 论文《SGLang: Efficient Execution of Structured Language Model Programs》对 structured language model programs、frontend / runtime 协同、RadixAttention 和 structured output decoding 的系统说明。

2. SGLang Structured Outputs 文档对 JSON schema、regular expression、EBNF、structural tag、OpenAI-compatible API、native API、offline engine，以及 XGrammar、Outlines、llguidance 三类 grammar backend 的说明。
3. SGLang Sampling Parameters 文档对 json_schema、regex、ebnf、structural_tag、choices、temperature、top_p、top_k、max_new_tokens 和 streaming 参数的公开口径。
4. SGLang Server Arguments 文档对 --grammar-backend、metrics / logging 和 structured output 相关 server 侧配置的说明。
5. SGLang Observability / Production Metrics 文档对 prompt tokens、generation tokens、TTFT、TPOT、E2E latency、running / queue requests 和 metrics endpoint 的说明，用来校准 structured output 对 scheduler 与性能观测的影响。

本章只讲 constrained decoding 的教学版机制：constraint spec、grammar backend、grammar state、valid token mask、sampler 顺序、JSON schema、regex、EBNF、structural tag、choices、空合法集合、streaming partial output、性能开销和事实正确性边界。它不实现真实 XGrammar / Outlines / llguidance，不绑定某个 SGLang 版本的 backend 细节、tokenizer trie、GPU kernel、真实 OpenAI API 参数全集或生产级 tool safety 策略。本章 demo 用纯 Python 的 toy grammar state 和 toy logits 解释机制，不调用真实模型。

参考资料：

1. SGLang 论文：<https://arxiv.org/abs/2312.07104>
2. SGLang Structured Outputs：https://docs.sglang.io/docs/advanced_features/structured_outputs.md
3. SGLang Sampling Parameters：https://docs.sglang.io/docs/references/sampling_params.md
4. SGLang Server Arguments：https://docs.sglang.io/docs/advanced_features/server_arguments.md
5. SGLang Observability：https://docs.sglang.io/docs/advanced_features/observability.md
6. SGLang Production Metrics：https://docs.sglang.io/docs/references/production_metrics.md

30.1 本章目标

读完本章，你应该能讲清：

1. 为什么“prompt 里要求输出 JSON”不等于可靠结构化输出。
2. Structured generation 和后处理 parse/retry 的区别。
3. Constrained decoding 在 logits 到 token 的哪个阶段生效。
4. JSON schema、regex、EBNF、structural tag 分别适合什么场景。
5. Grammar state、valid token mask、sampler 的关系。
6. Structured output 为什么会影响 scheduler 和性能。
7. 面试中如何解释 SGLang 的 structured outputs。

30.2 为什么结构化输出重要

LLM 在应用里经常不是最终用户直接阅读的聊天机器人，而是系统链路中的一个组件。

它的输出可能要被程序继续消费。

典型场景：

1. 信息抽取：输出 JSON。
2. 工具调用：输出函数名和参数。
3. 分类：输出固定 label。
4. 路由：输出某个 agent 或 workflow 名称。
5. SQL 生成：输出语法受限的查询片段。
6. 配置生成：输出 YAML 或 JSON。
7. 表单填写：输出字段值。
8. 评测打分：输出固定 schema。

如果输出格式不稳定，后续系统会出问题。

例如你希望模型输出：


```
{  
  "city": "Paris",  
  "country": "France",  
  "population": 2148000  
}
```

但模型可能输出：

Sure, here is the JSON: {"city": "Paris", "country": "France", "population": "about 2.1 million"}

这个结果对人类可读，但对程序不可靠。

问题包括：

1. 多了自然语言前缀。
2. 字段类型不符合 schema。
3. 可能缺字段。
4. 可能多字段。
5. 可能 JSON 语法不合法。
6. 可能把枚举值写成近义词。

结构化生成就是为了解决这类问题。

30.3 后处理和重试的问题

最朴素的方案是生成后 parse：

model output -> parse -> validate -> fail -> retry

这在小规模 demo 中常见，但生产问题很多。

第一，重试增加延迟。

一次失败可能让请求多跑一遍，TTFT 和 E2E latency 都变差。

第二，重试增加成本。

每次 retry 都要重新执行模型生成，除非 runtime 能很好地复用 cache。

第三，失败不是确定可修复。

模型可能连续输出不合法内容，尤其是 schema 复杂、字段多、上下文长时。

第四，后处理修复可能改错语义。

例如模型输出了字符串类型的 population，后处理强行转整数，可能把不确定表达错误地变成确定值。

第五，错误发生太晚。

生成完整答案后才发现不合法，已经浪费了全部 decode 成本。

Constrained decoding 的目标就是把格式约束提前到每一步 token 选择阶段。

30.4 Constrained decoding 的核心流程

普通 sampling 流程是：

```
logits  
-> penalty  
-> temperature  
-> top-k / top-p  
-> sample  
-> token
```

Constrained decoding 会插入 grammar mask：

```
logits
-> penalty
-> grammar mask
-> temperature
-> top-k / top-p
-> sample
-> token
-> update grammar state
```

Grammar mask 的作用是：

合法 token: 保留 logits

非法 token: logits = -inf

这样 softmax 后，非法 token 的概率就是 0。

伪代码：

```
def constrained_sample(logits, grammar_state, sampling_params):
    valid_token_ids = grammar_state.valid_next_tokens()
    mask = build_vocab_mask(valid_token_ids, vocab_size=logits.shape[-1])
    logits = logits.masked_fill(~mask, float("-inf"))
    token = sample_with_temperature_topk_topp(logits, sampling_params)
    grammar_state.accept(token)
    return token
```

这就是 structured output 属于 runtime 的原因：它直接参与 token selection。

30.5 Grammar state 是什么

Grammar state 表示当前已经生成的 token prefix 在约束规则中的位置。

例如约束输出 JSON object:

```
{"name": "Paris", "population": 2148000}
```

生成过程中的状态可能是：

```
step 0: 还没开始，只允许 "{" 或空白
step 1: 已生成 "{", 接下来允许字段名字符串
step 2: 已生成字段名，接下来允许 ":"
step 3: 已生成冒号，接下来允许字段值
step 4: 已生成字段值，接下来允许 ", " 或 "}"
```

Grammar state 每接收一个 token，就向前推进。

它要回答的问题是：

在当前状态下，下一个 token 哪些是合法的？

这个问题看起来简单，但实际很复杂，因为 tokenizer 的 token 不一定和字符边界对齐。

例如一个 token 可能是：

```
"Paris"
"\name"
"ation"
"}"
```

Grammar backend 必须在 tokenizer token 和语法状态之间建立映射。

30.6 Token-level 约束比字符级约束更难

很多 grammar 本来定义在字符层面。

例如 regex:

(Paris|London)

字符级别看，只允许输出 Paris 或 London。

但模型采样的是 token id，不是字符。

Tokenizer 可能把 Paris 切成一个 token，也可能切成多个 token；不同模型 tokenizer 不一样。

所以 constrained decoding 需要做的是：

字符级 grammar

-> 和 tokenizer vocabulary 对齐

-> 每个 grammar state 下计算合法 token ids

这就是 grammar backend 的价值。

它不只是 parse schema，而是要高效回答每一步合法 token 集合。

30.7 SGLang 支持哪些结构化约束

SGLang structured outputs 常见约束包括：

1. JSON schema。
2. Regular expression。
3. EBNF grammar。
4. Structural tag。
5. Choices 或固定候选。
6. Tool call schema。

SGLang 文档中提到的 grammar backend 包括：

1. XGrammar。
2. Outlines。
3. llguidance。

不同 backend 支持的约束类型和性能特点可能不同。

学习时先抓住抽象：

constraint spec -> grammar backend -> grammar state -> valid token mask

30.8 JSON schema

JSON schema 适合定义结构化对象。

例如：

```
{
  "type": "object",
  "properties": {
    "name": {"type": "string"},
    "population": {"type": "integer"}
  },
  "required": ["name", "population"]
}
```

它适合：

1. 信息抽取。

2. 表单填充。
3. 函数参数。
4. 评测结果。
5. Agent 中间状态。

JSON schema 的优点：

1. 和工程系统兼容好。
2. 可以表达字段类型。
3. 可以表达 required 字段。
4. 可以表达 enum、array、object。
5. 可以用 Pydantic 等工具生成。

局限：

1. 复杂 schema 可能增加 decoding 开销。
2. 只能保证语法和部分类型约束，不保证事实正确。
3. 约束越死，模型表达空间越小。
4. 对长文本字段，schema 只能约束外形，很难约束语义质量。

30.9 Regex

Regex 适合比较短、形式明确的输出。

例如：

`(positive|negative|neutral)`

或者：

`[A-D]`

适合场景：

1. 分类标签。
2. 多选题答案。
3. 简单 ID 格式。
4. 固定前缀后缀。
5. 小范围枚举。

Regex 的优点是简单直接。

缺点是：

1. 表达嵌套结构不方便。
2. 可读性容易变差。
3. 复杂 regex 可能难以维护。
4. 不适合复杂 JSON object。

如果只是要模型输出 A/B/C/D，regex 或 choices 比 JSON schema 更轻。

30.10 EBNF grammar

EBNF 适合自定义语法。

例如：

```
root ::= city | description
city  ::= "London" | "Paris" | "Berlin"
description ::= city " is capital of " country
country ::= "England" | "France" | "Germany"
```

EBNF 适合：

1. DSL 生成。
2. 简化 SQL 片段。
3. 命令语言。
4. 固定格式推理步骤。
5. 比 regex 更结构化、比 JSON schema 更自由的语法。

优点：

1. 表达力强。
2. 能定义递归或层级结构。
3. 适合领域特定语言。

缺点：

1. 编写门槛高。
2. 语法错误调试成本高。
3. 复杂 grammar 可能影响性能。
4. 不同 backend 支持的 grammar 格式可能有差异。

30.11 Structural tag

Structural tag 常用于工具调用或函数调用格式。

它的直觉是：当模型触发某个 begin tag 时，中间内容必须满足某个 schema，然后以 end tag 结束。

例如：

```
<function=get_weather>{"city":"Paris","unit":"celsius"}</function>
```

这种方式适合：

1. Tool calling。
2. 多工具选择。
3. XML-like 标记输出。
4. Agent 动作格式。
5. 需要触发式结构化片段的场景。

它比单纯 JSON schema 更适合混合输出：模型可能先自然语言解释，然后在特定触发标签后输出结构化工具调用。

但工程上要明确：

1. 哪些 tag 可以触发。
2. 每个 tag 对应什么 schema。
3. 是否允许多个 tool call。
4. 触发后是否必须停止。
5. 标签之间是否允许普通文本。

30.12 Choices

Choices 是最简单的 constrained generation。

例如：

```
choices = ["yes", "no"]
```

或者：

```
choices = ["A", "B", "C", "D"]
```

它本质上也是约束输出只能落在候选集合内。

Choices 适合：

1. 分类。

2. 多选题。
3. 路由选择。
4. 小范围枚举。
5. 判断题。

如果任务可以表达成 choices，不要上来就用复杂 JSON schema。越简单的约束通常越稳定、越快、越容易调试。

30.13 Structured output 在请求里怎么传

SGLang 支持 OpenAI-compatible API 和 native API 两类常见方式。

OpenAI-compatible API 中，JSON schema 常通过 response_format 表达：

```
response = client.chat.completions.create(
    model="model-name",
    messages=[{"role": "user", "content": "Extract city info."}],
    response_format={
        "type": "json_schema",
        "json_schema": {
            "name": "city_info",
            "schema": {
                "type": "object",
                "properties": {
                    "name": {"type": "string"},
                    "population": {"type": "integer"}
                },
                "required": ["name", "population"]
            }
        }
    },
)
```

Native /generate API 中，约束通常放在 sampling_params：

```
payload = {
    "text": "Give city information in JSON.",
    "sampling_params": {
        "temperature": 0,
        "max_new_tokens": 128,
        "json_schema": json_schema,
    },
}
```

Regex 和 EBNF 也类似：

```
sampling_params = {
    "temperature": 0,
    "max_new_tokens": 32,
    "regex": "(Paris|London)",
}

sampling_params = {
    "temperature": 0,
    "max_new_tokens": 32,
    "ebnf": 'root ::= "Hello" | "Hi"',
}
```

关键是：这些参数最终都会进入 runtime 的 grammar backend，而不是只作为 prompt 文本。

30.14 Runtime 内部执行链路

一个 structured output 请求进入 SGLang Runtime 后, 可以这样理解:

```
request
-> parse sampling params
-> detect json_schema / regex / ebnf / structural_tag
-> compile or initialize grammar object
-> create grammar state for request
-> scheduler schedules prefill/decode
-> model runner returns logits
-> grammar backend computes valid token mask
-> sampler masks logits and samples token
-> grammar state accepts token
-> output layer detokenizes and checks finish
```

这里有两类状态:

1. 模型生成状态: input ids、KV cache、output tokens。
2. 语法约束状态: 当前 grammar position、合法 token 集合、是否完成。

这两类状态必须同步推进。

如果 token 生成了, 但 grammar state 没更新, 下一步 mask 就会错。

如果请求 abort 或 finish, grammar state 也要清理。

30.15 Grammar mask 和 sampler 的顺序

一个实际 sampler 可能包含很多处理:

1. repetition penalty。
2. frequency penalty。
3. presence penalty。
4. custom logit processor。
5. grammar mask。
6. temperature。
7. top-k。
8. top-p。
9. min-p。
10. random sampling 或 greedy。

常见直觉是: grammar mask 应该在最终采样前生效, 保证非法 token 不会被采到。

例如:

```
raw logits
-> penalties
-> custom logit processors
-> grammar mask invalid tokens to -inf
-> sampling filters
-> sample next token
```

顺序细节不同框架可能不同, 但必须满足一个原则:

最终候选集合不能包含 grammar state 下非法的 token。

30.16 如果合法 token 集合为空怎么办

理论上, grammar backend 应该避免进入无合法 token 状态。

但工程上仍可能出现:

1. schema 不合法。
2. grammar 写错。
3. tokenizer 和 grammar 编译不兼容。
4. prompt 强烈诱导模型进入冲突状态。
5. backend bug。
6. custom logit processor 把合法 token 又全部屏蔽了。

如果合法集合为空，runtime 不能继续正常采样。

常见处理方式：

1. 返回错误。
2. 终止请求。
3. fallback 到 unconstrained decoding。
4. 放宽约束。
5. 记录指标和日志。

生产系统中一般不建议静默 fallback，因为用户以为拿到的是严格结构化输出，实际上不是。

30.17 Constrained decoding 不保证事实正确

Constrained decoding 只能保证形式合法，不保证内容正确。

例如 JSON schema 要求：

```
{"city": "string", "population": "integer"}
```

模型输出：

```
{"city": "Paris", "population": 999999999}
```

这在语法和类型上可能合法，但事实错误。

所以 structured output 解决的是：

格式可解析、字段可验证、工具调用可执行

它不自动解决：

事实正确、推理正确、业务规则正确

业务侧仍然需要 validation、retrieval grounding、规则校验或人工审核。

30.18 Prompt 仍然重要

即使用了 constrained decoding，也应该在 prompt 中明确说明输出格式。

原因是 grammar mask 只限制合法 token，不负责告诉模型“应该填什么内容”。

如果 prompt 不清楚，模型可能生成形式合法但语义差的结果。

例如 schema 允许：

```
{"name": "string", "population": "integer"}
```

但 prompt 没说要抽取哪个城市，模型可能随便填。

正确做法是：

1. Prompt 说明任务语义。
2. Schema 约束输出形式。
3. 后置 validator 检查业务规则。

三者配合，才是生产可用的结构化输出。

30.19 Structured output 对性能的影响

Structured output 不是免费功能。

它可能带来这些开销：

1. 请求开始时编译 schema 或 grammar。
2. 每个请求维护 grammar state。
3. 每个 decode step 计算 valid token mask。
4. mask logits 需要额外操作。
5. 复杂约束可能减少可选 token，影响模型自然生成路径。
6. batch 中不同请求 grammar 不同，增加调度和采样复杂度。

性能表现可能是：

1. TTFT 轻微上升，因为要初始化 grammar。
2. TPOT 上升，因为每步 decode 多了 mask 计算。
3. CPU 利用率上升，如果 grammar backend 主要在 CPU 侧工作。
4. p99 上升，如果复杂 schema 集中出现。

因此线上要监控 structured output 请求占比、grammar compile latency、mask latency 和失败率。

30.20 对 scheduler 的影响

第 29 章讲过，structured output 会影响 scheduler。

原因是每个 structured request 可能有额外 per-step 成本。

Scheduler 如果完全把它当普通 decode 请求，可能低估 decode step latency。

例如：

batch A: 64 个普通 decode 请求

batch B: 64 个复杂 JSON schema decode 请求

二者 decode token 数一样，但 batch B 的采样和 mask 可能更贵。

Scheduler 可以考虑：

1. 限制单 batch 中复杂 grammar 请求数量。
2. 统计 grammar mask latency。
3. 对超复杂 schema 做 admission control。
4. 把 structured workload 和普通 chat workload 分队列。
5. 在成本模型中加入 grammar complexity。

是否需要这些策略取决于业务规模和 p99 要求。

30.21 Streaming 下的结构化输出

Structured output 和 streaming 可以一起使用，但要注意用户看到的是逐步形成的结构。

例如 JSON streaming：

```
{
{"name"
{"name":"Paris"
{"name":"Paris","population"
...
```

中间片段可能不是完整 JSON。

这对前端和调用方有影响：

1. 不能每个 chunk 都当完整 JSON parse。

2. 需要等 finish 后再做完整 validation。
3. 如果要增量解析，需要 streaming parser。
4. stop condition 要和 grammar completion 协同。

因此结构化 streaming 的最佳实践通常是：

流式展示可以逐 chunk 显示

程序消费最好等完整对象结束后再 parse

除非你专门设计了增量协议。

30.22 Tool call 和 agent 的关系

Agent 场景中，structured output 很关键。

模型需要输出：

1. 调用哪个工具。
2. 工具参数是什么。
3. 是否继续思考。
4. 是否返回最终答案。

如果工具调用格式不稳定，agent runtime 就无法可靠执行。

Structural tag 或 JSON schema 可以让工具调用更可控：

```
<function=get_weather>{"city":"Paris","unit":"celsius"}</function>
```

这类输出被 constrained decoding 约束后，parser 更容易稳定识别工具名和参数。

但仍要校验：

1. 工具是否存在。
2. 参数是否在允许范围内。
3. 权限是否允许调用。
4. 参数值是否安全。
5. 是否需要用户确认。

Constrained decoding 保证格式，不替代工具安全策略。

30.23 和 RadixAttention 的关系

Structured output 和 RadixAttention 是不同层面的优化。

RadixAttention 解决：

如何复用共享 prefix 的 KV cache，减少重复 prefill？

Structured output 解决：

如何让 decode 输出满足格式约束？

它们可以同时工作。

例如一个固定 schema 的信息抽取任务：

shared prompt: instruction + schema explanation + document template

request-specific: document content

output: JSON schema constrained

RadixAttention 可以复用固定 instruction 和 schema 说明的 prefix。

Constrained decoding 可以保证输出 JSON 格式。

Scheduler 则要同时考虑 cache hit 和 grammar mask 成本。

30.24 常见错误

错误一：只在 prompt 里说“输出 JSON”，没有 constrained decoding。

结果是 demo 可能成功，线上各种边界情况失败。

错误二：schema 过度复杂。

过度嵌套、字段太多、枚举太长，会增加生成难度和 runtime 开销。

错误三：以为 constrained decoding 保证事实正确。

它只保证格式，不保证内容真实。

错误四：streaming 时每个 chunk 都 parse JSON。

中间 chunk 往往不是完整对象。

错误五：custom logit processor 和 grammar mask 冲突。

如果自定义处理器把所有合法 token 屏蔽掉，请求会失败。

错误六：忽略 tokenizer 差异。

Grammar backend 必须和当前模型 tokenizer 对齐，否则 valid token mask 可能不正确。

30.25 如何选择约束方式

可以按复杂度从低到高选择：

1. 固定分类：choices。
2. 简单格式：regex。
3. 标准对象：JSON schema。
4. 自定义语法：EBNF。
5. 工具调用混合文本：structural tag。

选择原则：

能用 choices 就不用 regex

能用 regex 就不用复杂 grammar

标准结构优先用 JSON schema

自定义语言再考虑 EBNF

工具调用考虑 structural tag 或 JSON schema

越简单的约束越容易调试，性能也通常更好。

30.26 调试 structured output

排查结构化输出问题，可以按下面顺序：

1. 确认请求是否真的传入 json_schema、regex、ebnf 或 structural_tag。
2. 确认 grammar backend 是否支持该约束类型。
3. 确认 schema 或 grammar 本身合法。
4. 确认 tokenizer 与模型一致。
5. 降低 temperature，先用 deterministic decoding 测试。
6. 简化 schema，逐步增加字段。
7. 查看 grammar compile latency 和 mask latency。
8. 检查是否 custom logit processor 冲突。
9. 检查 finish reason 和 max_new_tokens 是否过小。
10. 最后再考虑模型能力问题。

常见现象：

1. 一直不结束：可能 schema 允许很长输出，或 max_new_tokens 太大。

2. 过早截断：可能 max_new_tokens 太小。
3. 内容合法但语义差：prompt 不清楚或模型能力不足。
4. 延迟上升：grammar mask 或 compile 开销高。
5. 请求失败：schema/grammar/tokenizer/backend 不兼容。

30.27 面试官会怎么问

问题一：Constrained decoding 是什么？

回答要点：它是在每一步 decode 时根据 grammar state 计算合法 token 集合，把非法 token logits mask 成负无穷，让 sampler 只能选择满足 JSON schema、regex、EBNF 等约束的 token。

问题二：为什么不直接生成后 parse JSON？

回答要点：后处理失败发生太晚，会增加重试、延迟和成本；constrained decoding 把格式约束前移到 token selection 阶段，减少非法输出。

问题三：Structured output 能保证答案正确吗？

回答要点：不能。它主要保证格式和部分类型约束，不能保证事实正确、业务规则正确或工具调用安全。

问题四：JSON schema、regex、EBNF 怎么选？

回答要点：分类和枚举用 choices 或 regex；标准对象用 JSON schema；自定义 DSL 或复杂语法用 EBNF；工具调用可用 structural tag 或 JSON schema。

问题五：Structured output 为什么会影响 scheduler？

回答要点：因为每个请求需要 grammar state，每步 decode 要计算 valid token mask，复杂 schema 会增加 per-step 开销，影响 batch latency 和 p99。

30.28 标准回答模板

如果面试官问“SGLang structured generation 怎么实现”，可以这样回答：

SGLang 的 structured generation 不是简单靠 prompt 要求模型输出 JSON，而是把结构化约束放进 runtime 的 constrained decoding。

在每一步 decode 时，model runner 先输出 logits。Sampler 在采样前会查询当前 grammar state 下哪些 token 合法，然后构造 valid p、top-k 等采样处理后，只会从合法 token 中选择下一个 token。选出 token 后，grammar state 会接受这个 token 并推进到下一个 token。

这样做可以减少 JSON parse 失败、格式错误和重试，把输出合法性从生成后检查前移到生成过程中。但它只保证格式和部分类型约束，request grammar state 等开销，所以 scheduler 和性能监控也要把 structured output 请求单独看。

30.29 Structured Generation 公式、mask 门禁和可运行 demo

设词表为 $\mathcal{V}$ ，第 t 步模型输出 logits 为：

$$z_t \in \mathbb{R}^{|\mathcal{V}|}$$

Grammar state 记为 s_t ，当前状态下合法 token 集合为：

$$\mathcal{A}_t = g(s_t) \subseteq \mathcal{V}$$

Grammar mask 后的 logits 可以写成：

$$\tilde{z}_{t,j} = \begin{cases} z_{t,j}, & j \in \mathcal{A}_t \\ -\infty, & j \notin \mathcal{A}_t \end{cases}$$

采样分布为:

$$p_{t,j} = \frac{\exp(\tilde{z}_{t,j}/\tau)}{\sum_{k \in \mathcal{A}_t} \exp(\tilde{z}_{t,k}/\tau)}$$

如果 $\mathcal{A}_t$ 为空, runtime 不能继续正常采样, 应返回错误、终止请求或显式 fallback, 不能静默输出“看似结构化”的结果。

Grammar state 接收采样 token 后更新:

$$s_{t+1} = u(s_t, y_t)$$

教学版 structured generation 门禁可以写成:

$$G_{\text{struct}} = G_{\text{schema}} G_{\text{mask}} G_{\text{regex}} G_{\text{ebnf}} G_{\text{tag}} G_{\text{empty}} G_{\text{fact}} G_{\text{stream}} G_{\text{metric}}$$

其中:

1. G_{schema} : JSON schema-like grammar 能约束字段、冒号、逗号、类型和结束符。
2. G_{mask} : sampler 最终候选集合不包含非法 token。
3. G_{regex} : 简单分类或枚举可用 regex / choices 约束。
4. G_{ebnf} : 自定义 DSL-like sequence 可用 EBNF-like grammar 约束。
5. G_{tag} : structural tag 能约束 tool call 的 begin tag、内容和 end tag。
6. G_{empty} : 空合法 token 集合能被显式检测。
7. G_{fact} : 能说明格式合法不等于事实正确。
8. G_{stream} : streaming 中间 chunk 不被误当成完整对象。
9. G_{metric} : 能输出 mask calls、blocked invalid tokens、parseability 和 gate。

下面这个 0 依赖 demo 用 toy logits 和 toy grammar state 模拟 constrained decoding。它故意让非法 token 的 logit 更高, 证明 grammar mask 会阻止 sampler 选它; 同时让 JSON 的 `population` 字段生成一个格式合法但事实错误的整数, 证明 constrained decoding 不保证事实正确。

```
import json
```

```
class TinyJSONGrammar:
    def __init__(self):
        self.stage = 0
        self.history = []
        self.valid_history = []

    def valid_next_tokens(self):
        table = [
            {"{}"},
            {"name"},
            {":"},
            {"Paris", "London"},
            {","},
            {"population"},
            {":"},
            {"2148000", "999999999"},
            {""}]
        return table[self.stage] if self.stage < len(table) else set()
```

```

def accept(self, token):
    valid = self.valid_next_tokens()
    if token not in valid:
        raise ValueError(f"invalid token {token!r} at stage {self.stage}")
    self.valid_history.append(sorted(valid))
    self.history.append(token)
    self.stage += 1

def done(self):
    return self.stage == 9

def text(self):
    return "".join(self.history)

class ChoiceGrammar:
    def __init__(self, choices):
        self.choices = set(choices)
        self.token = None
        self.valid_history = []

    def valid_next_tokens(self):
        return set() if self.token is not None else set(self.choices)

    def accept(self, token):
        valid = self.valid_next_tokens()
        if token not in valid:
            raise ValueError(f"invalid choice {token!r}")
        self.valid_history.append(sorted(valid))
        self.token = token

    def done(self):
        return self.token is not None

    def text(self):
        return self.token or ""

class SequenceGrammar:
    def __init__(self, steps):
        self.steps = [set(step) for step in steps]
        self.stage = 0
        self.history = []
        self.valid_history = []

    def valid_next_tokens(self):
        return self.steps[self.stage] if self.stage < len(self.steps) else set()

    def accept(self, token):
        valid = self.valid_next_tokens()
        if token not in valid:
            raise ValueError(f"invalid token {token!r} at stage {self.stage}")
        self.valid_history.append(sorted(valid))
        self.history.append(token)
        self.stage += 1

```

```

def done(self):
    return self.stage == len(self.steps)

def text(self):
    return "".join(self.history)

class BrokenGrammar:
    def valid_next_tokens(self):
        return set()

def constrained_argmax(logits, valid_tokens):
    if not valid_tokens:
        raise ValueError("empty valid token set")
    chosen = max(valid_tokens, key=lambda tok: logits.get(tok, -10**9))
    chosen_score = logits.get(chosen, -10**9)
    blocked = sorted(
        tok for tok, score in logits.items()
        if tok not in valid_tokens and score > chosen_score
    )
    return chosen, blocked

def run_constrained(grammar, logits_plan):
    tokens = []
    blocked_count = 0
    mask_calls = 0
    while not grammar.done():
        logits = logits_plan[min(mask_calls, len(logits_plan) - 1)]
        valid = grammar.valid_next_tokens()
        token, blocked = constrained_argmax(logits, valid)
        blocked_count += len(blocked)
        mask_calls += 1
        grammar.accept(token)
        tokens.append(token)
    return {
        "text": grammar.text(),
        "tokens": tokens,
        "mask_calls": mask_calls,
        "blocked_high_score_invalid": blocked_count,
        "valid_history": getattr(grammar, "valid_history", []),
    }

json_logits = [
    {"Sure": 9, "{": 1},
    {"population": 9, "name": 2},
    {"": 1, "=": 8},
    {"Sure": 8, "Paris": 3, "London": 2},
    {"", ": 1, "}": 9},
    {"name": 8, "population": 2},
    {"": 1, "=": 9},
    {"99999999": 8, "2148000": 5, "many": 9},

```

```

    {"}": 1, ",": 7},
]
json_result = run_constrained(TinyJSONGrammar(), json_logits)
parsed = json.loads(json_result["text"])
fact_check = parsed == {"name": "Paris", "population": 2148000}

choice_result = run_constrained(
    ChoiceGrammar(["positive", "negative", "neutral"]),
    [{"very_positive": 9, "positive": 3, "negative": 1}],
)
ebnf_result = run_constrained(
    SequenceGrammar([
        {"Paris", "London"},
        {" is "},
        {"capital"},
        {" of "},
        {"France", "England"},
    ]),
    [
        {"Paris": 3, "Berlin": 9},
        {" is ": 1, " was ": 8},
        {"capital": 1, "city": 8},
        {" of ": 1, " in ": 7},
        {"France": 3, "Mars": 9},
    ],
)
tag_result = run_constrained(
    SequenceGrammar([
        {"<function=get_weather>"},
        {"{"},
        {""city"},
        {":"},
        {""Paris"},
        {"}"},
        {"</function>"},
    ]),
    [
        {"hello": 9, "<function=get_weather>": 1},
        {"[" : 8, "{" : 1},
        {""city": 1, "unit": 9},
        {"": 1, "=" : 7},
        {""Paris": 1, "London": 0},
        {"}": 1, ",": 8},
        {"</function>": 1, "</tool>": 7},
    ],
)
try:
    constrained_argmax({"anything": 1}, BrokenGrammar().valid_next_tokens())
    empty_valid_detected = False
except ValueError:
    empty_valid_detected = True

stream_chunks = [json_result["text"][:end] for end in (1, 7, 16, len(json_result["text"]))]
partial_parseable = []
for chunk in stream_chunks:

```



```

try:
    json.loads(chunk)
    partial_parseable.append(True)
except json.JSONDecodeError:
    partial_parseable.append(False)

summary = {
    "json_text": json_result["text"],
    "json_parseable": isinstance(parsed, dict),
    "json_fact_check": fact_check,
    "choice_text": choice_result["text"],
    "ebnf_text": ebnf_result["text"],
    "tag_text": tag_result["text"],
    "mask_calls": (
        json_result["mask_calls"]
        + choice_result["mask_calls"]
        + ebnf_result["mask_calls"]
        + tag_result["mask_calls"]
    ),
    "blocked_high_score_invalid": (
        json_result["blocked_high_score_invalid"]
        + choice_result["blocked_high_score_invalid"]
        + ebnf_result["blocked_high_score_invalid"]
        + tag_result["blocked_high_score_invalid"]
    ),
    "stream_partial_parseable": partial_parseable,
    "empty_valid_detected": empty_valid_detected,
}
gates = {
    "json_schema_mask_blocks_prefix": json_result["tokens"][0] == "{",
    "sampler_only_uses_valid_tokens": all(
        token in valid
        for result in [json_result, choice_result, ebnf_result, tag_result]
        for token, valid in zip(result["tokens"], result["valid_history"])
    ),
    "regex_choice_constrained": choice_result["text"] == "positive",
    "ebnf_sequence_constrained": ebnf_result["text"] == "Paris is capital of France",
    "structural_tag_closed": tag_result["text"].startswith("<function=")
    and tag_result["text"].endswith("</function>"),
    "empty_valid_detected": empty_valid_detected,
    "format_not_fact": summary["json_parseable"] and not summary["json_fact_check"],
    "streaming_partial_not_json": partial_parseable[-1] == [False, False, False]
    and partial_parseable[-1] is True,
    "metrics_ready": summary["blocked_high_score_invalid"] >= 8,
}
gates["structured_generation_gate"] = all(gates.values())

print("structured_generation_summary=", summary)
print("structured_generation_gates=", gates)

```

这段 demo 应该输出：

```

structured_generation_summary= {'json_text': '{"name":"Paris","population":999999999}', 'json_parseable': True, 'json_
structured_generation_gates= {'json_schema_mask_blocks_prefix': True, 'sampler_only_uses_valid_tokens': True, 'regex_cl

```

读这段代码时要注意：

1. 第一步 logits 里 Sure 比 { 分数高，但 JSON grammar 只允许 {，所以输出不会带自然语言前缀。
2. `choice_text='positive'` 说明简单分类最好用 `choices / regex` 这类轻约束，不必上复杂 JSON schema。
3. `tag_text` 展示 structural tag 能把工具名、参数 JSON 和结束标签绑在同一个约束里。
4. `empty_valid_detected=True` 说明合法 token 集合为空时必须显式失败，不能继续采样。
5. `json_fact_check=False` 说明 constrained decoding 只保证格式和类型边界，业务事实仍要单独校验。
6. `stream_partial_parseable` 的前三项为 False，说明 streaming 中间 chunk 通常不是完整 JSON，程序消费要等 finish 或使用专门的增量 parser。

30.30 小练习

1. 写一个 JSON schema，要求输出 `name`、`age`、`is_student` 三个字段。
2. 用 regex 约束模型只能输出 `positive`、`negative`、`neutral`。
3. 写一个简单 EBNF，让模型只能输出 `Hello` 或 `Hi`。
4. 画出 constrained decoding 从 logits 到 token 的流程图。
5. 解释为什么 grammar state 要随每个请求单独维护。
6. 举例说明 constrained decoding 不能保证事实正确。
7. 设计一个 structured output dashboard，包含 compile latency、mask latency、失败率、schema 类型分布和 p99。

30.31 本章总结

Structured generation 是复杂 LLM 应用走向生产的关键能力。它解决的不是“模型会不会说人话”，而是“模型输出能不能被程序可靠消费”。

Constrained decoding 的核心是在每一步采样前，根据 JSON schema、regex、EBNF 或 structural tag 维护的 grammar state 计算合法 token mask，屏蔽非法 token，让输出天然满足格式约束。

它比生成后 parse/retry 更靠前、更稳定，但也不是万能：它不能保证事实正确，复杂约束会带来性能开销，streaming 下也要注意中间 chunk 不一定是完整对象。

下一章会进入 Speculative Decoding 在 SGLang 中的角色，讨论如何用草稿模型、并行验证或其他机制减少主模型 decode 步数，从而进一步优化 TPOT 和吞吐。

第 31 章 Speculative Decoding 在 SGLang 中的角色

上一章讲了 Structured Generation 与 constrained decoding：runtime 可以在每一步采样前用 grammar mask 限制合法 token，让输出满足 JSON schema、regex、EBNF 或 structural tag。

本章看另一个直接影响 decode 性能的技术：Speculative Decoding。

前面的章节反复讲过，LLM 自回归生成的 decode 阶段通常每轮只生成一个 token。即使 prefill 很快，长答案仍然要经过很多轮主模型 forward。Speculative decoding 的核心目标就是减少主模型逐 token decode 的等待，把“一个 token 一轮”尽量变成“一轮验证多个候选 token”。

一句话概括：

Speculative decoding 用较便宜的 draft 来源先猜多个候选 token，再用 target model 并行验证这些候选；如果接受率足够高，就能用较少的 target model decode 轮数生成更多 token，从而改善 TPOT 和输出吞吐。

31.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，主要参考四类公开资料：

1. SGLang 官方 Speculative Decoding 文档，对 EAGLE、EAGLE3、MTP、DFLASH、Standalone draft model、NGRAM、Speculative Decoding V2 / overlap scheduler 等入口和参数给出了公开说明。
2. SGLang 官方 Adaptive Speculative Decoding 文档，对用近期 accept length 动态调整 speculative steps 的口径给出了说明。

3. SGLang server arguments / sampling / structured output 相关文档, 对 speculative 参数、grammar 约束、streaming、metrics 和 runtime 组合边界提供工程背景。
4. Speculative Decoding / Speculative Sampling 论文, 对 draft propose、target verify、接受-拒绝采样、修正分布和“保持 target 分布”的理论目标提供基础口径; EAGLE、EAGLE3、MTP 等论文用于理解不同 draft 来源的差异。

本章的写作边界也要说清:

1. 本章讲 SGLang-like serving runtime 中 speculative decoding 的角色, 不绑定某个 SGLang 版本的内部类名、源码文件、CUDA kernel、attention backend 细节或 benchmark 数值。
2. 本章把 EAGLE / EAGLE3 / MTP / Standalone / NGRAM 作为 draft 来源和工程路径来解释, 不展开它们完整训练算法。
3. 本章 demo 是 greedy 教学版, 用来证明 draft / verify / accept / fallback / metrics / KV cleanup 的执行链路; 严格保持 target sampling 分布需要论文中的 modified rejection sampling, 不能把教学 demo 当成完整论文算法。
4. 本章对 structured output 的讨论只覆盖 grammar state 如何影响 draft 候选和 accept 过程, 不实现真实 XGrammar / Outlines / llguidance。
5. 本章所有性能公式都是容量规划和面试解释用的简化模型, 真实收益必须用业务 workload replay、GPU profiler、SGLang metrics 和灰度实验验证。

31.1 本章目标

读完本章, 你应该能讲清:

1. Speculative decoding 主要解决 decode 阶段什么问题。
2. Draft model、target model、verify、accept length 分别是什么。
3. 为什么 speculative decoding 通常优化 TPOT, 而不是 prefill。
4. SGLang 中 EAGLE、EAGLE3、MTP、Standalone、NGRAM 等方案的大致定位。
5. 接受率、draft 成本、verify 成本如何共同决定收益。
6. Speculative decoding 和 scheduler、KV cache、structured output 的交互。
7. 面试中如何回答“speculative decoding 为什么能加速”。

31.2 为什么 decode 阶段慢

自回归生成的基本流程是:

prefill prompt

```
-> decode token 1
-> decode token 2
-> decode token 3
-> ...
```

每个 decode step 都需要主模型做一次 forward。

如果生成 1000 个 token, 朴素做法至少需要 1000 轮 decode forward。

Decode 的问题是:

1. 每轮只推进很少 token。
2. 轮数多。
3. 每轮都要读历史 KV cache。
4. 调度、采样、streaming 都要重复发生。
5. 用户会直接感受到 TPOT。

Prefill 可以通过 batching、prefix cache、RadixAttention、chunked prefill 等优化减少重复计算。

Decode 则更难, 因为下一个 token 依赖上一个 token, 天然串行。

Speculative decoding 要打破的是这个串行瓶颈的一部分。

31.3 核心思想：先猜，再验

Speculative decoding 的核心流程是：

1. draft source 快速生成多个候选 token
2. target model 一次性验证这些候选 token
3. 接受其中与 target model 一致或可接受的前缀
4. 如果某个 token 被拒绝，从 target model 分布采样修正 token
5. 重复下一轮

用更直观的例子：

当前上下文：The capital of France is

draft 猜：Paris . It is

target 验证：Paris . 被接受，It 被拒绝

本轮实际前进：Paris . + target 修正 token

如果 draft 猜得准，一轮 target verify 可以接受多个 token。

原来：

target forward 1 -> token A

target forward 2 -> token B

target forward 3 -> token C

现在：

draft guesses A B C

target verify once -> accepts A B C

主模型轮数减少，TPOT 就可能下降。

31.4 Draft 和 target

Speculative decoding 中通常有两个角色。

Target model：

1. 真正要服务的主模型。
2. 输出质量以它为准。
3. 参数量大、forward 贵。
4. 负责验证 draft token。

Draft source：

1. 用来提出候选 token。
2. 必须比 target 更便宜或更快。
3. 可以是小模型、特殊 draft head、MTP head、EAGLE draft model、ngram cache 等。
4. 猜得越准，接受率越高。

最经典版本是 small draft model + large target model：

small draft model：快速猜 4 个 token

large target model：一次 verify 4 个 token

但现代 serving engine 中，draft source 不一定是普通小模型，也可能是 EAGLE、MTP、NGRAM 等不同机制。

31.5 Verify 和 accept length

Verify 是 target model 对 draft tokens 的验证过程。

假设 draft 提出：

[d1, d2, d3, d4]

Target model 会计算这些位置上的分布，并判断 draft token 是否可以接受。

接受前缀长度叫 accept length：

draft: [d1, d2, d3, d4]

accept: [d1, d2]

reject: d3

accept length = 2

如果 accept length 越大，本轮 target verify 的收益越高。

如果 accept length 经常是 0 或 1，speculative decoding 可能不但没收益，还会增加 draft 开销。

所以接受率是 speculative decoding 最重要的指标之一。

31.6 为什么它不改变输出分布

经典 speculative decoding 的一个重要目标是：加速的同时保持 target model 的输出分布不变或尽量等价。

直觉上，draft 只是提出候选，最终是否接受由 target model 决定。

如果 draft 猜错，target 会拒绝并从正确分布中采样修正 token。

因此 draft 不应该决定最终质量，target 才是最终裁判。

工程实现中，不同算法对“严格保持分布”和“近似加速”有不同取舍。

面试回答时可以说：

Speculative decoding 的基本思想是 draft propose、target verify；在标准算法中，通过接受/拒绝规则可以保持 target 分布，工

不要简单说“小模型直接生成，所以质量会变差”。更准确是：小模型只是 draft，target model 负责验证。

31.7 最小伪代码

一个简化 speculative decoding loop：

```
while not finished:
    draft_tokens = draft_model.generate(context, k=4)

    target_logits = target_model.verify(context, draft_tokens)
    accepted, correction = accept_or_reject(target_logits, draft_tokens)

    output.extend(accepted)
    if correction is not None:
        output.append(correction)

    context.extend(accepted)
    if correction is not None:
        context.append(correction)
```

这个伪代码省略了很多工程细节：

1. KV cache 如何写入。
2. Draft model 的 KV cache 如何维护。
3. Target verify 的 attention metadata 如何构造。
4. 多请求 batch 如何一起 speculative。
5. 被拒绝 token 之后的 draft KV 如何丢弃或修正。
6. Streaming 如何返回多个 accepted tokens。

7. Structured output 如何约束候选 token。

但它已经表达了主干：draft 提案，target 验证，接受前缀。

31.8 主要优化什么指标

Speculative decoding 主要优化 decode 阶段。

相关指标包括：

1. TPOT。
2. output tokens/s。
3. decode step 有效产出 token 数。
4. accepted tokens/s。
5. target forward per output token。

它通常不直接优化 prefill。

原因是 prefill 本来就是一次处理 prompt 多 token，而 speculative decoding 解决的是 decode 每轮只出一个 token 的问题。

所以面试中要避免说：

Speculative decoding 加速 prompt prefill。

更准确：

它主要减少主模型 decode 轮数，提高每轮 verify 产出的 token 数，因此改善 TPOT 和输出吞吐。

31.9 收益取决于什么

Speculative decoding 的收益不是必然的。

它取决于：

1. Draft source 是否足够快。
2. Draft token 接受率是否足够高。
3. Target verify 是否能高效并行验证多个 token。
4. 额外 KV cache 和 memory 开销是否可接受。
5. Batch size 和调度是否适配 speculative 路径。
6. 采样参数是否让 draft 更难预测。
7. 结构化约束是否降低或提高候选可接受性。
8. 硬件是否能吃满 verify 的并行度。

可以用一个直觉公式：

收益 $\approx$ accepted tokens per target verify / extra draft overhead

如果 draft 很慢，或者 target 每次只接受 1 个 token，收益就会很差。

31.10 接受率为什么会变化

接受率不是固定常数。

它会随 workload 变化。

接受率较高的场景：

1. 确定性任务。
2. 低 temperature。
3. 格式固定输出。
4. 代码或模板化文本。
5. Draft model 和 target model 分布接近。

6. 上下文模式重复。

接受率较低的场景：

- 1. 高 temperature。
- 2. 创意写作。
- 3. 多样化开放问答。
- 4. Draft model 太弱。
- 5. 目标模型分布很尖锐但 draft 猜偏。
- 6. 强约束和 sampling 规则冲突。

这就是为什么 SGLang 文档中会有 adaptive speculative decoding：真实流量的 accept length 会随时间变化，固定 speculative steps 不一定总是最优。

31.11 SGLang 支持的 speculative decoding 类型

SGLang 文档中列出多种 speculative decoding 方案，包括：

- 1. EAGLE-2。
- 2. EAGLE-3。
- 3. MTP。
- 4. DFLASH。
- 5. Standalone draft model。
- 6. NGRAM。
- 7. Speculative Decoding V2 / overlap scheduler。
- 8. Adaptive speculative decoding。

学习时不需要一开始记住所有参数。

先把它们按 draft 来源分类：

方法	Draft 来源	直觉
Standalone	更小的 draft LLM	小模型先猜，大模型验证
EAGLE / EAGLE3	专门训练或适配的 draft 模块	更高效地产生候选 token/tree
MTP	模型内置多 token prediction 能力	一个模型结构中直接预测后续多个 token
DFLASH	专门 draft checkpoint	线性 draft block 验证
NGRAM	之前 token 的 ngram 匹配	不用额外模型，从重复片段中猜

核心都是：

产生候选 token -> target verify -> 接受一段 -> 减少 target decode 轮数

31.12 EAGLE / EAGLE3 的定位

EAGLE 是 SGLang 中推荐的 speculative decoding 路径之一。

它的直觉是：用更适合 speculative 的 draft 机制生成候选 token，target model 再验证。

文档中给出的启用方式通常包含：

```
python3 -m sglang.launch_server \
  --model <target-model> \
  --speculative-algorithm EAGLE \
  --speculative-draft-model-path <eagle-draft-model> \
  --speculative-num-steps 3 \
  --speculative-eagle-topk 4 \
  --speculative-num-draft-tokens 16
```

EAGLE3 是更新的方案，文档中也把它作为高性能推荐选项之一。

对学习者来说，关键不是背具体命令，而是理解这些参数含义：

1. `speculative_algorithm`: 使用哪种 speculative 方法。
2. `speculative_draft_model_path`: draft 模型或模块来源。
3. `speculative_num_steps`: draft 往前猜多少步。
4. `speculative_eagle_topk`: 每步分支候选数。
5. `speculative_num_draft_tokens`: 一次 verify 最多处理多少 draft tokens。

这些参数共同决定候选树大小、接受率、显存占用和计算开销。

31.13 Standalone draft model

Standalone 是最容易理解的 speculative decoding。

结构是：

target model: Qwen2.5-7B

draft model: Qwen2.5-1.5B

Draft model 更小，所以生成候选更便宜。

Target model 更大，所以负责最终验证。

优点：

1. 概念简单。
2. 容易解释。
3. 适合有同系列小模型的场景。

缺点：

1. 需要额外加载 draft model，占显存。
2. Draft 和 target tokenizer、模型族要匹配。
3. 接受率依赖两个模型分布接近程度。
4. 多模型调度更复杂。

如果 draft model 太弱，target 会频繁拒绝，收益会下降。

31.14 NGRAM speculative decoding

NGRAM 方法不依赖额外 draft model。

它从已有上下文或历史生成中找重复片段，作为 draft tokens。

直觉：

如果模型刚生成过类似 `ngram`，后面可能再次出现类似片段。

适合场景：

1. 代码生成中的重复模式。
2. 长文本中重复短语。
3. 模板化输出。
4. 不想额外加载 draft model 的场景。

优点：

1. 不需要额外模型权重。
2. 显存压力可能低于 standalone draft model。
3. 对重复性强的输出有用。

缺点：

1. 依赖重复模式。

2. 对开放式新内容帮助有限。
3. 命中和接受率不稳定。
4. 适用硬件和配置可能有限。

31.15 MTP: Multi-Token Prediction

MTP 指模型自身具备预测多个未来 token 的能力。

如果模型结构或权重支持 MTP，就可以把多 token prediction 接入 speculative decoding 流程。

直觉是：

普通模型 head：预测下一个 token

MTP heads：预测后面多个 token

然后 target 路径验证这些多 token 候选。

优点：

1. 不一定需要单独小模型。
2. Draft 和 target 更紧密。
3. 可能减少额外模型加载成本。

限制：

1. 需要模型本身支持。
2. 不同模型启用方式不同。
3. 参数调优仍然重要。

31.16 Adaptive speculative decoding

Adaptive speculative decoding 解决的是固定 speculative steps 不适合动态 workload 的问题。

如果 speculative_num_steps 太小：

draft 本来还能猜更多会被接受的 token，但提前停了

如果 speculative_num_steps 太大：

draft 猜了很多 token，但 target 拒绝，浪费 draft 计算和显存

Adaptive 策略会根据近期 accept length 调整 steps。

SGLang 文档中提到，它会用 EMA 追踪平均接受长度，并在候选 tier 中切换，例如：

candidate_steps = [1, 3, 7]

如果近期接受长度高，就倾向更大 steps。

如果近期接受长度低，就降到更小 steps。

这个思路本质是：

让 speculative depth 跟随当前 workload，而不是固定写死。

31.17 Speculative decoding 和 scheduler

Speculative decoding 会让 scheduler 更复杂。

普通 decode：

每个 running request 本轮生成 1 token

Speculative decode：

每个 running request 本轮可能 draft 多个 token
target verify 多个 token
最终接受 0 到 N 个 token

Scheduler 要考虑：

1. Draft 阶段的计算成本。
2. Verify 阶段的 target 计算成本。
3. 每个请求的 speculative depth。
4. Draft tokens 对 KV cache 的临时占用。
5. 接受 tokens 后如何更新 request state。
6. 拒绝 tokens 后如何丢弃临时状态。
7. Streaming 一次返回多个 accepted tokens。
8. 不同请求 accept length 不同导致 batch 动态变化。

这就是为什么 speculative decoding 不是简单在 sampler 前加一个小模型，而是 runtime 调度路径的一部分。

31.18 和 KV cache 的关系

Speculative decoding 对 KV cache 有额外要求。

Draft 阶段可能会产生候选 token 的 KV。

Target verify 也需要对候选 token 计算或读取 KV。

如果 token 被接受，对应 KV 可以成为正式上下文的一部分。

如果 token 被拒绝，对应临时 KV 不能继续作为正式历史。

因此需要区分：

committed KV: 已接受 token 的 KV

speculative KV: 候选 token 的临时 KV

discarded KV: 被拒绝 token 相关 KV

KV cache 管理要保证：

1. 接受的 token KV 被保留。
2. 拒绝的 token KV 被释放或覆盖。
3. Draft model 和 target model 的 KV 不混淆。
4. RadixAttention prefix cache 不错误缓存未接受 token。
5. Memory pool 能承受额外临时 KV 峰值。

这一点很重要：speculative decoding 可能提升 decode 吞吐，但也可能增加显存压力。

31.19 和 RadixAttention 的关系

RadixAttention 主要优化 prefill 重复计算。

Speculative decoding 主要优化 decode 主模型轮数。

二者互补：

RadixAttention: 让请求更快完成 prefill，降低 TTFT

Speculative decoding: 让请求 decode 更快，降低 TPOT

一个复杂 LLM program 可以同时使用：

1. RadixAttention 复用共享 prompt 前缀。
2. Scheduler 接纳未命中 suffix prefill。
3. Speculative decoding 加速后续长输出。
4. Structured output 约束 token 合法性。

但要注意：speculative 生成的未接受 token 不应该插入 RadixAttention 作为可复用正式 prefix。
只有 committed tokens 才能安全进入长期 cache。

31.20 和 structured output 的关系

Structured output 会影响 speculative decoding。

原因是 draft token 也必须满足 grammar 约束，否则 target verify 很可能拒绝或 runtime 无法接受。

两种可能设计：

1. Draft 阶段也使用 grammar mask，只产生合法候选。
2. Draft 阶段先猜，verify/accept 阶段再结合 grammar 过滤。

无论哪种，最终 committed tokens 必须满足 grammar state。

困难点包括：

1. Grammar state 要随 speculative token 前进。
2. 一次接受多个 token 时，要连续更新 grammar state。
3. 拒绝后，要回滚 speculative grammar state。
4. 合法 token 集合可能很小，影响 draft 接受率。
5. Structural tag 和 tool call 的结束条件要和 accept 逻辑一致。

所以 structured output + speculative decoding 是一个更复杂的 runtime 组合，不只是两个开关同时打开。

31.21 和 streaming 的关系

普通 decode 每轮通常返回一个 token 或一个文本增量。

Speculative decoding 一轮可能接受多个 token。

这对 streaming 有两个影响。

第一，用户可能一次收到更大的 chunk。

普通 decode: token token token

spec decode: [token token token] [token token] [token]

第二，TTFT 不一定明显改善，但后续输出可能更快。

如果 speculative round 本身有额外 draft/verify 开销，first token 甚至可能不变或略受影响。

因此评估时要分开看：

1. TTFT。
2. TPOT。
3. inter-token latency。
4. chunk size。
5. accepted tokens per round。

不要只看主观“流式更快”。

31.22 参数调优直觉

Speculative decoding 常见参数包括：

1. speculative algorithm。
2. draft model path。
3. speculative num steps。
4. draft tokens 数量。
5. top-k 或分支数。
6. accept threshold。

7. draft model quantization。
8. draft attention backend。

调参目标是：

提高 accepted tokens per target verify，同时控制 draft overhead 和显存占用

num_steps 太小：接受率可能高，但每轮前进少。

num_steps 太大：候选多，但后面很多被拒绝，浪费计算。

topk 太小：候选多样性不足。

topk 太大：候选树和 verify 成本上升。

Draft model 太大：draft overhead 高。

Draft model 太小：接受率低。

所以 speculative decoding 必须压测，不适合只凭经验开最大参数。

31.23 关键监控指标

上线 speculative decoding 后，至少要监控：

1. accept length 平均值。
2. acceptance rate。
3. accepted tokens/s。
4. drafted tokens/s。
5. rejected tokens/s。
6. target verify latency。
7. draft latency。
8. TPOT。
9. output tokens/s。
10. GPU memory usage。
11. OOM 或 allocation failure。
12. p95/p99 latency。

如果只看 output tokens/s，可能忽略 p99 变差或显存风险。

如果只看 acceptance rate，也可能忽略 draft 模型太慢。

要同时看收益和成本。

31.24 常见性能现象

现象一：开启 speculative 后吞吐没有提升。

可能原因：

1. 接受率低。
2. Draft model 太慢。
3. Verify 路径没有并行收益。
4. batch 太小，硬件没吃满。
5. 额外显存导致 batch size 下降。
6. workload 输出短，收益不明显。

现象二：平均吞吐提升，但 p99 变差。

可能原因：

1. 某些请求接受率极低。
2. Speculative depth 太大。
3. Draft/verify 临时 KV 造成显存压力。

4. Scheduler 对 speculative workload 成本估计不足。

现象三：低 temperature 收益明显，高 temperature 收益下降。

这是常见现象。高 temperature 下目标分布更随机，draft 更难猜中。

现象四：代码生成收益明显，开放写作收益一般。

代码和模板化文本重复模式强，draft 更容易预测；开放写作多样性更强，接受率可能更低。

31.25 OOM 和显存问题

Speculative decoding 可能增加显存占用。

来源包括：

1. Draft model 权重。
2. Draft model KV cache。
3. Target verify 临时 KV。
4. Candidate tree 或 draft token metadata。
5. CUDA graph 和 workspace。
6. 更大的 batch 或 verify token 数。

如果 OOM，可以考虑：

1. 降低 speculative_num_steps。
2. 降低 speculative_num_draft_tokens。
3. 降低 top-k 或分支数。
4. 使用更小或量化 draft model。
5. 降低 batch size。
6. 降低 KV cache 预算。
7. 关闭某些不兼容优化。
8. 分离 speculative 和非 speculative 流量。

Speculative decoding 的本质是用额外计算和显存换更少 target decode 轮数。显存不足时，收益可能被反噬。

31.26 什么时候适合开启

适合场景：

1. 输出较长。
2. Decode 是主要瓶颈。
3. Draft 接受率高。
4. 有合适 draft 模型或 EAGLE/MTP 支持。
5. 显存有余量。
6. 低 temperature 或确定性任务较多。
7. 代码、模板、结构化输出等可预测任务。

不适合或要谨慎：

1. 输出很短。
2. 瓶颈在 prefill、RAG、tokenizer 或网络。
3. 接受率低。
4. 显存紧张。
5. 高 temperature 创意生成。
6. p99 要求极严但 speculative 路径波动大。
7. 没有合适 draft source。

31.27 和 PD 分离的关系

后面章节会讲 PD 分离。

这里先给一个直觉：speculative decoding 主要优化 decode，所以它更直接影响 decode worker 或 decode 资源池。

在 PD 分离架构中：

prefill pool: 处理长 prompt prefill

decode pool: 处理逐 token generation

Speculative decoding 通常更适合放在 decode pool 的优化路径中。

但它也会影响系统整体资源：

1. Decode worker 显存需求变化。
2. Output tokens/s 上升后，下游 streaming 压力上升。
3. 更快 decode 可能改变 prefill/decode 队列平衡。
4. Draft model 也需要部署和调度。

所以它不是单机小优化，而会影响 serving 架构容量规划。

31.28 常见误解

误解一：Speculative decoding 是用小模型替代大模型。

不对。Draft source 只是提出候选，target model 负责验证，最终质量以 target 为准。

误解二：Speculative decoding 一定提升性能。

不一定。接受率低、draft 太慢、显存紧张或输出太短时，可能没有收益甚至变慢。

误解三：Speculative decoding 主要优化 TTFT。

通常不对。它主要优化 decode 阶段 TPOT 和输出吞吐；TTFT 更多受 queue、tokenize、prefill、prefix cache 影响。

误解四：参数越大越好。

不对。更多 draft steps 和更大 top-k 会增加候选和 verify 成本，接受率不够时会浪费。

误解五：和 structured output 没关系。

不对。Grammar 约束会影响 draft token 合法性、接受率和状态回滚。

31.29 面试官会怎么问

问题一：Speculative decoding 为什么能加速？

回答要点：它用便宜 draft source 先生成多个候选 token，再用 target model 并行验证，若接受多个 token，就相当于一次 target verify 前进多个 token，减少主模型 decode 轮数。

问题二：它主要优化 TTFT 还是 TPOT？

回答要点：主要优化 TPOT 和 output tokens/s，因为它作用在 decode 阶段；TTFT 主要由 queue、tokenization、prefill 和 prefix cache 决定。

问题三：收益取决于什么？

回答要点：取决于 draft 速度、接受率、target verify 并行效率、额外显存开销、batch 调度和 workload 采样特征。

问题四：SGLang 支持哪些 speculative 方案？

回答要点：包括 EAGLE/EAGLE3、MTP、DFLASH、Standalone draft model、NGRAM、adaptive speculative decoding 和实验性 overlap scheduler 等。

问题五：为什么 speculative decoding 可能变慢？

回答要点：如果 draft token 被大量拒绝，或者 draft 模型太慢、显存压力降低 batch、verify 开销高，额外成本会抵消减少 target 轮数的收益。

31.30 标准回答模板

如果面试官问“SGLang 中 speculative decoding 的角色是什么”，可以这样回答：

Speculative decoding 在 SGLang 中主要是 decode 阶段的加速手段。普通自回归生成每轮 target model 只能生成一个 token，长输出

SGLang 支持多种 draft 来源和算法，比如 EAGLE/EAGLE3、MTP、DFLASH、Standalone 小 draft model、NGRAM，以及 adaptive specul

这个优化是否有效取决于接受率和额外开销。接受率高、draft 便宜、输出较长、decode 是瓶颈时收益明显；如果接受率低、draft 开

31.31 Speculative Decoding 公式、runtime 门禁和可运行 demo

为了把上面的概念落到可验证链路，可以把一个请求抽象成：

$r_i = (X_i, Y_i, K_i, g_i)$

其中 X_i 是已提交上下文， Y_i 是还要生成的目标 token 序列， K_i 是每轮最多 draft token 数， g_i 表示是否带 structured output grammar state。

第 i 个请求在某轮得到 draft 候选：

$D_i = (d_{i,1}, d_{i,2}, \dots, d_{i,K_i})$

在 greedy 教学版中，target verify 计算同一段位置上的目标 token：

$Y_i^* = (y_{i,1}^*, y_{i,2}^*, \dots, y_{i,K_i}^*)$

接受前缀长度可以写成：

$A_i = \max\{a \in \{0, \dots, K_i\} \mid d_{i,j} = y_{i,j}^*, \forall 1 \leq j \leq a\}$

如果第 A_i+1 个 draft token 被拒绝，就提交前 A_i 个 accepted tokens，并从 target 分布产生一个 fallback / correction token。对采样版 speculative sampling，常见接受概率写成：

$\alpha_j = \min\left(1, \frac{p_{\text{target}}(d_j \mid c_j)}{\max(\epsilon, p_{\text{draft}}(d_j \mid c_j))}\right)$

拒绝后需要从修正分布采样：

$r_j(x) = \frac{\max(0, p_{\text{target}}(x \mid c_j) - p_{\text{draft}}(x \mid c_j))}{Z_j}$

这里 c_j 是当前位置上下文， ϵ 是避免除零的小常数， Z_j 是归一化常数。直觉是：draft 只是提案，target 的概率分布仍然是最终裁判。

工程上至少要看这些指标：

$R_{\text{acc}} = \frac{N_{\text{accepted}}}{\max(1, N_{\text{drafted}})}$

$T_{\text{verify}} = \frac{N_{\text{output}}}{\max(1, C_{\text{target}})}$

$G_{\text{call}} = \frac{C_{\text{baseline}}}{\max(1, C_{\text{spec}})}$

其中 R_{acc} 是接受率， T_{verify} 是每次 target verify 平均提交多少 output tokens， G_{call} 是 target 调用次数下降倍数。粗略延迟收益可以写成：

$S_{\text{latency}} = \frac{C_{\text{baseline}} L_{\text{target}}}{C_{\text{spec}} L_{\text{verify}} + N_{\text{drafted}}}$

如果 S_{latency} 大于 1，说明简化模型里端到端延迟下降；如果小于或接近 1，draft 开销、低接受率或调度开销可能抵消收益。

Speculative decoding 的上线门禁可以写成：

$G_{\text{spec}} = G_{\text{draft}} G_{\text{verify}} G_{\text{accept}} G_{\text{fallback}} G_{\text{kv}} G_{\text{mat}}$

这些因子分别要求 draft 来源可解释、target verify 可复盘、accept length 可统计、fallback 正确、临时 KV 可清理、grammar state 不越界、指标能支持灰度和回滚。

下面是一个 0 依赖教学 demo。它同时模拟三类请求：

1. `code_fast`: draft 和 target 很接近，接受率高，target 调用明显下降。
2. `creative_low_accept`: draft 经常猜错，收益很差，说明 speculative decoding 不是必然加速。
3. `json_tool`: draft 试图产生不满足 grammar 的 token，runtime 先阻断，再由 target fallback 修正。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class ToyRequest:
```

```
    request_id: str
    target_tokens: list
    draft_tokens: list
    grammar: dict | None = None
```

```
class ToySpeculativeDecoder:
```

```
    def __init__(self, requests, num_steps=4, target_ms=6.0, draft_ms=0.8):
        self.requests = requests
        self.num_steps = num_steps
        self.target_ms = target_ms
        self.draft_ms = draft_ms
```

```
    def _valid(self, request, pos, token):
        if request.grammar is None:
            return True
        return token in request.grammar.get(pos, set())
```

```
    def _draft_window(self, request, pos, metrics):
        drafted = []
        for offset in range(self.num_steps):
            draft_pos = pos + offset
            if draft_pos >= len(request.target_tokens):
                break
            token = request.draft_tokens[draft_pos]
            if not self._valid(request, draft_pos, token):
                metrics["grammar_blocked_draft"] += 1
                break
            drafted.append(token)
        metrics["drafted_tokens"] += len(drafted)
        return drafted
```

```
    def baseline(self, request):
        return list(request.target_tokens), len(request.target_tokens)
```

```
    def speculative(self, request):
        pos = 0
        output = []
        trace = []
        metrics = {
            "target_verify_calls": 0,
            "drafted_tokens": 0,
            "accepted_tokens": 0,
```



```

    "rejected_tokens": 0,
    "fallback_tokens": 0,
    "grammar_blocked_draft": 0,
    "temp_kv_peak": 0,
    "discarded_temp_tokens": 0,
}
while pos < len(request.target_tokens):
    drafted = self._draft_window(request, pos, metrics)
    metrics["target_verify_calls"] += 1
    metrics["temp_kv_peak"] = max(metrics["temp_kv_peak"], len(drafted))
    if not drafted:
        fallback = request.target_tokens[pos]
        output.append(fallback)
        metrics["fallback_tokens"] += 1
        trace.append({"pos": pos, "draft": [], "accepted": [], "fallback": fallback})
        pos += 1
        continue

    target_window = request.target_tokens[pos : pos + len(drafted)]
    accepted = []
    for draft_token, target_token in zip(drafted, target_window):
        if draft_token == target_token:
            accepted.append(draft_token)
        else:
            break

    accepted_len = len(accepted)
    output.extend(accepted)
    metrics["accepted_tokens"] += accepted_len
    fallback = None
    if accepted_len < len(drafted):
        fallback = target_window[accepted_len]
        output.append(fallback)
        rejected = len(drafted) - accepted_len
        metrics["rejected_tokens"] += rejected
        metrics["discarded_temp_tokens"] += rejected
        metrics["fallback_tokens"] += 1
        pos += accepted_len + 1
    else:
        pos += accepted_len
    trace.append({"pos": pos, "draft": drafted, "accepted": accepted, "fallback": fallback})
    metrics["committed_tokens"] = len(output)
    return output, metrics, trace

def run(self):
    rows = []
    summary = {
        "baseline_target_calls": 0,
        "spec_target_calls": 0,
        "drafted_tokens": 0,
        "accepted_tokens": 0,
        "rejected_tokens": 0,
        "fallback_tokens": 0,
        "grammar_blocked_draft": 0,
        "discarded_temp_tokens": 0,

```

```

        "temp_kv_peak": 0,
    }
    for request in self.requests:
        baseline_tokens, baseline_calls = self.baseline(request)
        spec_tokens, metrics, trace = self.speculative(request)
        summary["baseline_target_calls"] += baseline_calls
        summary["spec_target_calls"] += metrics["target_verify_calls"]
        for key in [
            "drafted_tokens",
            "accepted_tokens",
            "rejected_tokens",
            "fallback_tokens",
            "grammar_blocked_draft",
            "discarded_temp_tokens",
        ]:
            summary[key] += metrics[key]
        summary["temp_kv_peak"] = max(summary["temp_kv_peak"], metrics["temp_kv_peak"])
        rows.append(
            {
                "id": request.request_id,
                "match_target": spec_tokens == baseline_tokens,
                "baseline_calls": baseline_calls,
                "spec_calls": metrics["target_verify_calls"],
                "accepted": metrics["accepted_tokens"],
                "drafted": metrics["drafted_tokens"],
                "fallback": metrics["fallback_tokens"],
                "grammar_blocked": metrics["grammar_blocked_draft"],
                "trace_tail": trace[-2:],
            }
        )

    summary["output_tokens"] = sum(len(req.target_tokens) for req in self.requests)
    summary["acceptance_rate"] = round(summary["accepted_tokens"] / max(1, summary["drafted_tokens"]), 3)
    summary["tokens_per_target_call"] = round(summary["output_tokens"] / max(1, summary["spec_target_calls"]), 3)
    summary["target_call_reduction"] = round(summary["baseline_target_calls"] / max(1, summary["spec_target_calls"]), 3)
    baseline_latency = summary["baseline_target_calls"] * self.target_ms
    spec_latency = (
        summary["spec_target_calls"] * self.target_ms
        + summary["drafted_tokens"] * self.draft_ms
        + summary["grammar_blocked_draft"] * 0.2
        + summary["spec_target_calls"] * 0.3
    )
    summary["latency_speedup_est"] = round(baseline_latency / spec_latency, 3)

    gates = {
        "target_output_preserved": all(row["match_target"] for row in rows),
        "acceptance_rate_measured": 0.0 <= summary["acceptance_rate"] <= 1.0,
        "target_calls_reduced": summary["target_call_reduction"] > 1.0,
        "low_accept_case_visible": any(row["accepted"] <= 1 and row["spec_calls"] >= row["baseline_calls"] for row in rows),
        "grammar_blocks_invalid_draft": summary["grammar_blocked_draft"] > 0,
        "temp_kv_cleanup_visible": summary["discarded_temp_tokens"] == summary["rejected_tokens"]
        and summary["discarded_temp_tokens"] > 0,
        "metrics_ready": summary["tokens_per_target_call"] > 1.0,
    }
    gates["speculative_decoding_gate"] = all(gates.values())

```

```
return rows, summary, gates
```

```
requests = [  
    ToyRequest(  
        request_id="code_fast",  
        target_tokens=["def", "add", "(", "a", ",", "b", ")"], ":", "return", "a", "+", "b", "<eos>"],  
        draft_tokens=["def", "add", "(", "a", ",", "b", ")"], ":", "return", "a", "+", "b", "<eos>"],  
    ),  
    ToyRequest(  
        request_id="creative_low_accept",  
        target_tokens=["The", "answer", "is", "not", "obvious", "<eos>"],  
        draft_tokens=["A", "very", "different", "path", "appears", "<eos>"],  
    ),  
    ToyRequest(  
        request_id="json_tool",  
        target_tokens=["{", '"city"', ":", '"Paris"', "}"],  
        draft_tokens=["{", '"city"', ":", '"Paris"', "}"],  
        grammar={  
            0: {"{", "}"},  
            1: {"city"},  
            2: {":"},  
            3: {"Paris", "London"},  
            4: {"", ""},  
        },  
    ),  
]
```

```
rows, summary, gates = ToySpeculativeDecoder(requests).run()  
print("speculative_rows=", rows)  
print("speculative_summary=", summary)  
print("speculative_gates=", gates)
```

运行后可以看到类似输出：

```
speculative_summary= {'baseline_target_calls': 24, 'spec_target_calls': 13, 'drafted_tokens': 35, 'accepted_tokens': 18  
speculative_gates= {'target_output_preserved': True, 'acceptance_rate_measured': True, 'target_calls_reduced': True, 'L
```

这个 demo 的关键不是把真实 SGLang 内部实现复刻出来，而是让你能复盘 speculative decoding 的工程证据链：

1. target 输出没有被 draft 替代。
2. 接受率和每次 target verify 产出 token 数可观测。
3. 低接受率请求会暴露出来，不能只看总体平均收益。
4. structured output 的 grammar state 会影响 draft 候选。
5. rejected speculative token 的临时 KV 必须丢弃，不能进入正式上下文或 prefix cache。
6. 是否值得开启，要同时看 target call reduction、draft overhead、scheduler overhead、TPOT、显存和 p99。

31.32 小练习

1. 用伪代码写一个 draft/verify/accept 的 speculative decoding loop。
2. 解释为什么 speculative decoding 主要优化 TPOT。
3. 给出 3 个接受率高的 workload 和 3 个接受率低的 workload。
4. 对比 Standalone draft model 和 NGRAM draft source。
5. 设计一个 speculative decoding dashboard, 包含 accept length、draft latency、verify latency、TPOT、显存和 p99。

6. 解释为什么 rejected token 不能进入 RadixAttention prefix cache。
7. 说明 structured output 如何影响 speculative decoding 的 grammar state。

31.33 本章总结

Speculative decoding 的核心是“先猜，再验”。它通过便宜的 draft source 生成候选 token，再用 target model 并行验证，从而在接受率足够高时减少主模型 decode 轮数。

在 SGLang 中，它是和 RadixAttention、scheduler、KV cache、structured output 并列的重要 runtime 优化。RadixAttention 更偏 prefill 和 TTFT，speculative decoding 更偏 decode 和 TPOT。

但 speculative decoding 不是无脑开启的银弹。它需要合适的 draft source、较高接受率、足够显存和正确调度。真正上线时必须用真实 workload 压测，看 accepted tokens、draft overhead、verify latency、TPOT、显存和 p99。

下一章会进入 Multi-turn、tool use 和 agent serving 支持，讨论 SGLang 如何服务多轮状态、工具调用、agent 轨迹和复杂程序执行。

第 32 章 Multi-turn、Tool Use 和 Agent Serving 支持

上一章讲了 Speculative Decoding：它通过 draft/verify/accept 减少主模型 decode 轮数，主要优化 TPOT 和 output tokens/s。

本章回到 SGLang 的问题意识：复杂 LLM 应用往往不是一次 prompt -> answer，而是多轮对话、工具调用、外部环境交互、agent 轨迹和多步程序执行。

这些场景从应用层看是 agent framework，从 runtime 层看则是：多次 generation、共享上下文、结构化工具参数、工具结果回灌、动态分支、状态管理、cache 复用和调度公平性。

一句话概括：

Multi-turn、tool use 和 agent serving 的核心不是“模型会调用工具”这么简单，而是 runtime 能否高效、可靠地执行多步有状态 LLM 程序，并在每一轮复用上下文、约束工具格式、管理请求状态和控制尾延迟。

32.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，主要参考六类公开资料：

1. SGLang 论文对 structured language model programs、frontend language、fork / branch、RadixAttention 和 runtime 协同的系统说明。
2. SGLang Frontend Language 文档对 gen、multi-turn、Python control flow、fork、choices / regex constrained decoding、batching、streaming 和 multi-modal prompt 的公开口径。
3. SGLang OpenAI-compatible API、Tool Parser、Structured Outputs 和 server arguments 文档，对 tools schema、tool parser、tool choice、EBNF / JSON schema / structural tag、chat template、streaming 和 metrics 的说明。
4. ReAct、Toolformer、MRKL 等公开论文，对 agent loop、工具调用、observation 回灌和 reasoning / acting 交替的基本问题背景提供依据。
5. Agent serving / LLM serving 相关公开论文和文档，对多次 generation、prefix sharing、tool latency、session-aware routing、KV cache locality 和 task-level metrics 的工程边界提供参考。
6. 前面第 28 到 31 章关于 RadixAttention、scheduler、structured generation 和 speculative decoding 的本书内部口径。

本章边界也要先说清：

1. 本章讲 SGLang-like runtime 如何支持 multi-turn、tool use 和 agent serving，不绑定某个 SGLang 版本的真实 tool parser 类名、OpenAI-compatible 字段全集、server 参数全集或内部源码路径。
2. 本章把真实工具执行放在应用层或 agent framework，不把数据库、HTTP、代码解释器、浏览器、MCP server 等外部系统塞进模型 runtime。

3. 本章 demo 是教学版 agent serving audit, 只模拟 session routing、prefix cache、tool parser、validator、工具结果回灌、GPU slot 释放和指标门禁；不实现真实网络工具、真实权限系统或生产 trace 后端。
4. Tool parser、structured output 和 validator 不是同一个东西：parser 负责把模型文本转成对象，structured output 负责约束生成格式，validator 负责业务参数和权限检查。
5. Agent serving 的性能结论必须看 task-level E2E latency、工具耗时、model calls、prefix reuse、KV pressure、scheduler queue 和失败恢复，不能只看单次 chat completion 的 QPS。

32.1 本章目标

读完本章，你应该能讲清：

1. Multi-turn chat 在 runtime 中为什么不是普通单请求。
2. Tool use 的请求、生成、解析、执行、回灌流程是什么。
3. SGLang tool parser、structured output、tool_choice 分别解决什么问题。
4. Agent serving 为什么会产生多次 generation 和树状 prefix sharing。
5. RadixAttention 如何帮助多轮和 agent trajectory 复用 KV cache。
6. Scheduler 在 agent workload 中要面对哪些新问题。
7. 面试中如何从 serving runtime 视角解释 agent 支持。

32.2 从单轮 chat 到多轮状态

单轮 chat 很简单：

user prompt -> model answer

多轮 chat 则是：

```
system message
user turn 1
assistant turn 1
user turn 2
assistant turn 2
user turn 3
assistant turn 3
...
```

每一轮请求都要包含历史上下文，模型才能理解当前问题。

从 OpenAI-compatible API 看，请求是 messages 列表。

从 runtime 看，关键是这些 messages 会被 chat template 渲染成 token 序列：

```
messages
-> chat template
-> prompt text
-> token ids
-> prefill / prefix cache / decode
```

多轮的性能机会在于：下一轮通常共享上一轮完整历史。

例如：

```
turn 2 prompt: [S U1 A1 U2]
turn 3 prompt: [S U1 A1 U2 A2 U3]
```

turn 3 可以复用 turn 2 已经计算过的大段 KV cache。

这就是 RadixAttention 在 multi-turn chat 中的价值。

32.3 多轮 chat 的 runtime 状态

一个多轮会话至少有两类状态。

应用层状态：

1. session id。
2. message history。
3. 用户身份。
4. 工具配置。
5. 业务上下文。
6. 安全策略。

Runtime 层状态：

1. tokenized history。
2. prefix cache 命中路径。
3. KV cache 引用。
4. 当前 request state。
5. sampling params。
6. stop 条件。
7. streaming offset。
8. grammar state。

很多系统会把应用层状态放在上层服务或 agent framework 中，而 SGLang Runtime 负责高效执行每次 generation。

但 runtime 仍然需要看到足够稳定的 token prefix，才能复用 cache。

32.4 Chat template 为什么影响 cache

多轮对话的 prefix cache 命中依赖 token prefix 完全一致。

如果 chat template 变化，token 序列就变化。

例如：

```
<user>Hello</user><assistant>Hi</assistant>
```

和：

User: Hello

Assistant: Hi

语义相同，但 token 序列不同，cache 无法共享。

工程建议：

1. 同一模型使用稳定 chat template。
2. 工具说明顺序稳定。
3. system prompt 不要插入随机字段。
4. 历史裁剪策略稳定。
5. 上层不要每轮改变无关格式。

Multi-turn serving 的 cache 命中率，很多时候不是 runtime 算法问题，而是 prompt 拼接稳定性问题。

32.5 Tool use 的基本流程

Tool use 指模型在回答过程中选择调用外部工具。

典型流程：

1. 用户提问
2. 请求中包含可用 tools schema
3. 模型生成 tool call
4. runtime 或上层解析 tool name 和 arguments
5. 应用层执行工具
6. 工具结果作为 role=tool message 回灌
7. 模型基于工具结果继续生成最终回答

展开成 messages:

```
user: What's the weather in Boston?
assistant: tool_call get_weather({"city": "Boston"})
tool: {"temperature": "85F", "condition": "cloudy"}
assistant: It is 85F and cloudy in Boston.
```

注意：工具执行通常不在 LLM runtime 内部完成。

Runtime 负责生成和解析工具调用，上层应用负责真正执行工具、处理权限、超时和结果回灌。

32.6 Tool schema 是什么

工具通常用 schema 描述。

例如：

```
{
  "type": "function",
  "function": {
    "name": "get_current_weather",
    "description": "Get the current weather in a given location",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {"type": "string"},
        "unit": {"type": "string", "enum": ["celsius", "fahrenheit"]}
      },
      "required": ["city", "unit"]
    }
  }
}
```

这个 schema 有三层作用：

1. 告诉模型有哪些工具。
2. 约束工具参数格式。
3. 给 parser 或 validator 提供校验依据。

在 SGLang 中，tool use 可以结合 tool parser、chat template、structured output 和 tool choice 来实现更可靠的函数调用。

32.7 Tool parser 的作用

不同模型的工具调用格式不同。

有的输出 JSON。有的输出特殊 tag。有的输出 Python-like function call。有的用模型专属 token。

Tool parser 的作用是把模型原始输出解析成统一的 tool call 对象：

```
raw generated text
-> tool parser
-> normal_text
```

-> tool_calls: [{name, arguments}]

SGLang 文档中列出多种 parser, 例如 DeepSeek、GLM、GPT-OSS、Kimi、Llama、Mistral、Qwen、pythonic 等。

这说明一个工程事实: tool calling 不是完全统一的模型能力, 它强依赖模型训练格式、chat template 和 parser。如果 parser 和模型格式不匹配, 工具调用可能解析失败。

32.8 Tool choice

Tool choice 用来控制模型是否必须调用工具, 或必须调用哪个工具。

常见模式:

1. auto: 模型自行决定是否调用。
2. required: 必须至少调用一个工具。
3. specific function: 必须调用指定函数。
4. none: 不允许调用工具。

从 runtime 视角看, tool choice 可以被转化为更强的输出约束。

例如 required 模式要求模型输出 tool call, 而不是普通文本。

SGLang 文档提到 tool_choice 可通过 EBNF grammar 等方式实现更可靠的工具调用行为。

这 and 第 30 章 constrained decoding 是同一条线: 用 grammar 约束模型输出, 而不是只靠 prompt 希望模型听话。

32.9 Tool call 的结构化约束

工具调用最怕格式不稳定。

例如你希望:

```
{"city": "Boston", "unit": "fahrenheit"}
```

模型却输出:

city is Boston, use Fahrenheit please

人能看懂, 程序不好执行。

因此 tool call 通常需要 structured output 支持:

1. JSON schema 约束 arguments。
2. Structural tag 约束 begin/end 标签。
3. EBNF 约束 tool_choice。
4. Parser 解析模型专属格式。
5. Validator 做最终参数校验。

推荐理解为分层:

prompt / chat template: 告诉模型工具语义

constrained decoding: 限制工具调用格式

tool parser: 把文本解析成对象

validator: 校验业务参数

application: 执行工具

不要把所有责任都放在模型输出上。

32.10 工具执行不等于模型推理

Tool use 往往跨越 LLM runtime 和应用系统边界。

LLM Runtime 负责：

1. 接收 tools schema。
2. 渲染 chat template。
3. 生成 tool call。
4. 支持 constrained decoding。
5. 解析或返回 tool call。
6. 继续处理工具结果后的 generation。

应用层负责：

1. 判断工具是否允许调用。
2. 执行 HTTP、数据库、搜索、代码解释器等工具。
3. 处理工具超时和错误。
4. 清洗工具结果。
5. 把结果作为 tool message 回灌。
6. 做审计和安全控制。

这条边界很重要。

如果把真实工具执行塞进推理 runtime，runtime 会变得难以隔离、难以扩缩容，也更难保证安全。

32.11 Agent loop 是什么

Agent loop 是多轮 tool use 的泛化。

典型 ReAct-like loop：

```
while not done:
    model generates thought/action
    if action is tool call:
        execute tool
        append observation
    else if action is final answer:
        return answer
```

从 runtime 看，它会产生多次 generation：

```
gen action 1
tool result 1
gen action 2
tool result 2
gen final answer
```

每次 generation 都共享之前的 trajectory。

这正是 SGLang 强调复杂 LLM programs 的原因。

32.12 Agent trajectory 的 prefix sharing

Agent trajectory 通常长这样：

```
system prompt
tool descriptions
user task
thought 1
action 1
observation 1
```

thought 2
action 2
observation 2
final answer

后续每一步都共享前面的所有历史。

如果每次工具返回后都从头 prefill，成本很高。

RadixAttention 可以复用：

[system prompt][tool descriptions][user task][thought/action/observation history]

只对新增 observation 和下一轮 user/tool message 后的 suffix 做计算。

对于多分支 agent search，也会出现树状共享：

```
root task
-> plan A
    -> tool result A1
    -> tool result A2
-> plan B
    -> tool result B1
```

Radix tree 可以自然表达这些共享路径。

32.13 Agent serving 和普通 chat serving 的差异

普通 chat serving：

一个用户请求 -> 一个模型回答

Agent serving：

一个用户任务 -> 多次模型调用 + 多次工具调用 + 动态停止

差异包括：

1. 请求生命周期更长。
2. 生成次数不固定。
3. 工具耗时不可控。
4. 上下文不断增长。
5. 中间状态需要保存。
6. 可能有并行分支。
7. 失败点更多。
8. cache reuse 机会更多。

因此 agent serving 不能只按单次 chat completion 的 QPS 来评估。

要看的是整个 task 的端到端成功率、工具轮数、总 tokens、总延迟和成本。

32.14 Scheduler 面临的新问题

Agent workload 会给 scheduler 带来新问题。

第一，请求会分阶段到达。

模型生成 tool call 后，请求会等待工具结果；工具结果回来后，又产生下一次 generation。

第二，工具耗时不稳定。

一个工具可能 50ms 返回，也可能 5s 超时。

第三，agent 轨迹长短差异大。

有的任务 1 轮完成，有的任务 10 轮还没完成。

第四, cache locality 很重要。

同一个 agent session 的后续 generation 如果打到同一个 runtime, 更容易命中 prefix cache。

第五, 公平性更复杂。

一个长 agent 任务不能无限占用调度资源, 短 chat 请求也不能被完全饿死。

所以 agent serving 常需要:

1. session-aware routing。
2. max tool rounds。
3. max context length。
4. max total tokens。
5. per-step timeout。
6. priority queue。
7. cache-aware scheduling。

32.15 Session-aware routing

在多副本 serving 中, 同一个 session 如果每轮都打到不同 runtime, prefix cache 命中会变差。

例如:

turn 1 -> replica A

turn 2 -> replica B

turn 3 -> replica C

每个副本都有自己的 GPU KV cache, 后续轮次可能无法复用前面轮次的 cache。

Session-aware routing 的思路是:

同一个 session 尽量路由到同一个 runtime replica

好处:

1. 提高 RadixAttention 命中率。
2. 降低多轮 TTFT。
3. 减少重复 prefill。

代价:

1. 负载均衡更难。
2. 热门 session 可能压垮单副本。
3. 副本故障时 cache 丢失。
4. 扩缩容时迁移复杂。

这是平台层和 runtime 层的协同问题。

32.16 Context growth 和裁剪

多轮和 agent 的上下文会不断增长。

问题包括:

1. prompt 越来越长。
2. prefill 和 cache 成本上升。
3. KV cache 占用上升。
4. 超过模型 max context。
5. 工具结果可能很长。
6. 无关历史影响模型质量。

常见处理策略:

1. 滑动窗口保留最近历史。

2. 总结旧历史。
3. 只保留关键 tool results。
4. 对工具结果做压缩。
5. RAG 化历史。
6. 设置 max tool rounds 和 max tokens。

但裁剪会影响 prefix cache。

如果每轮裁剪策略不稳定, token prefix 会变化, cache 命中下降。

所以裁剪策略要尽量确定、可复现。

32.17 Tool result 回灌

工具结果通常作为 role=tool message 回到模型上下文。

例如:

```
assistant tool_call: get_weather({"city":"Boston"})
tool result: {"temperature":"85F"}
assistant final: Boston is 85F.
```

回灌时要注意:

1. 工具结果长度限制。
2. 工具结果格式稳定。
3. 错误信息如何表达。
4. 是否暴露敏感字段。
5. 是否需要引用来源。
6. 多工具结果顺序。

工具结果是 prompt 的一部分, 会进入 tokenizer 和 KV cache。

如果工具结果含随机字段、时间戳、trace id, prefix cache 复用会变差。

32.18 Tool errors 和恢复

工具调用可能失败。

失败类型:

1. 工具不存在。
2. 参数校验失败。
3. 权限不足。
4. 网络超时。
5. 下游返回 500。
6. 结果为空。
7. 结果过长。

处理方式:

1. 把错误作为 tool message 回灌, 让模型决定下一步。
2. 直接终止 agent。
3. 重试工具。
4. 切换备用工具。
5. 请求用户补充信息。

Serving runtime 需要支持 abort、timeout 和资源清理。

如果 agent 等工具时不释放或暂停相应资源, 很容易造成长尾和泄漏。

32.19 Parallel tool calls

有些模型或 agent 会一次提出多个工具调用。

例如：

```
call get_weather(city="Paris")
call get_hotels(city="Paris")
call get_flights(destination="Paris")
```

应用层可以并行执行这些工具。

Runtime 视角的影响：

1. 模型需要输出多个 tool calls。
2. Parser 要能处理多个 calls。
3. 工具结果回灌顺序要稳定。
4. 后续 generation 共享相同前缀。
5. 工具等待阶段不应占用 GPU decode slot。

Parallel tool calls 能降低 agent wall-clock latency，但会增加外部系统压力和错误处理复杂度。

32.20 Agent search 和分支

有些 agent 不只线性执行，还会搜索多个候选计划。

例如：

```
task
-> plan A
    -> execute A
    -> score A
-> plan B
    -> execute B
    -> score B
-> choose best
```

这会产生多分支 generation。

SGLang 的优势在这里更明显：

1. 多个分支共享 root prompt。
2. 每个分支可能继续扩展。
3. RadixAttention 可以复用共享前缀。
4. Scheduler 可以批量执行多个 branch 的 prefill/decode。
5. Structured output 可以约束 plan/score 格式。

这类 workload 比普通 chat 更接近 SGLang 论文里的复杂 LLM programs。

32.21 Agent serving 的指标

Agent serving 不能只看单次请求 QPS。

建议看：

任务级指标：

1. task success rate。
2. task E2E latency。
3. tool rounds per task。
4. model calls per task。
5. total input tokens per task。
6. total output tokens per task。

7. cost per task。

Runtime 指标:

1. TTFT per generation。
2. TPOT。
3. prefix cache hit length。
4. RadixAttention saved tokens。
5. waiting queue length。
6. running requests。
7. KV cache usage。
8. grammar mask latency。

Tool 指标:

1. tool call count。
2. tool latency。
3. tool error rate。
4. timeout rate。
5. retry count。

这些指标要分层看，否则很难判断慢在模型、runtime、工具还是 agent 逻辑。

32.22 安全边界

Tool use 和 agent serving 必须考虑安全。

常见风险:

1. Prompt injection 诱导调用危险工具。
2. 工具参数包含恶意输入。
3. 模型泄露工具结果中的敏感数据。
4. 工具调用越权。
5. 无限循环调用工具。
6. 代码执行工具造成破坏。
7. SSRF 或外部请求滥用。

Runtime 层能做一部分格式约束，但不能替代安全策略。

应用层必须做:

1. 工具 allowlist。
2. 参数 validation。
3. 权限校验。
4. 速率限制。
5. 沙箱执行。
6. 审计日志。
7. 用户确认机制。

不要因为有 structured output，就默认工具调用是安全的。

32.23 和 speculative decoding 的关系

Agent 场景中 speculative decoding 可能有帮助，也可能收益有限。

有帮助的情况:

1. 每轮输出较长。
2. 最终答案较长。
3. 工具结果总结较长。
4. 低 temperature、格式稳定。

收益有限的情况：

1. 每轮只输出短 tool call。
2. 工具等待时间主导 E2E latency。
3. 高不确定性规划，draft 接受率低。
4. structured output 和 tool constraints 使 draft 更复杂。

所以 agent serving 中要分开看：

模型 decode 时间

工具执行时间

调度排队时间

prefix cache 命中

如果主要慢在工具 API，speculative decoding 解决不了根因。

32.24 和 structured output 的关系

Tool use 几乎天然需要 structured output。

原因是工具参数必须被程序解析。

常见组合：

1. JSON schema 约束 function arguments。
2. Structural tag 包住 tool call。
3. Tool parser 解析模型原生格式。
4. Tool choice 约束是否必须调用。
5. Validator 做最终业务校验。

Structured output 让工具调用格式更稳定，但不能保证工具选择一定正确。

例如模型可能合法地调用 `get_weather`，但用户其实问的是酒店价格。

所以还需要工具选择策略、系统 prompt、业务规则和评测。

32.25 和 RadixAttention 的关系

Multi-turn、tool use 和 agent serving 都是 RadixAttention 的典型收益场景。

原因：

1. 多轮历史共享前缀。
2. 工具说明通常固定。
3. Agent trajectory 逐步增长。
4. 多分支计划共享 root。
5. Self-consistency 和 agent search 共享问题描述。

可以把三者关系记成：

Multi-turn 提供长共享历史

Tool use 提供固定工具说明和结构化调用

Agent serving 提供多步/分支/回灌轨迹

RadixAttention 负责复用这些共享 prefix 的 KV cache

如果 prompt template 稳定，收益会更明显。

32.26 常见工程坑

坑一：工具说明每轮顺序变化。

后果：token prefix 变化，cache 命中率下降。

坑二：工具结果太长。

后果：上下文快速膨胀，KV cache 占用上升。

坑三：tool parser 和模型格式不匹配。

后果：模型生成了工具调用，但系统解析不出来。

坑四：streaming tool call 参数碎片处理错误。

后果：增量 arguments 拼接失败，JSON parse 错误。

坑五：工具等待期间占用 GPU slot。

后果：running queue 被阻塞，吞吐下降。

坑六：agent 无限循环。

后果：成本失控，用户等待时间过长。

坑七：跨副本路由打散 session。

后果：多轮 prefix cache 命中率低。

32.27 面试官会怎么问

问题一：SGLang 为什么适合 agent serving?

回答要点：Agent workload 包含多次 generation、工具调用、结构化输出、多轮历史和分支控制流。SGLang 的 frontend/runtime 协同、RadixAttention、structured output、scheduler 能更好地表达和执行这些复杂 LLM programs。

问题二：Tool use 的完整链路是什么？

回答要点：请求带 tools schema，模型生成 tool call，tool parser 解析 name/arguments，应用层执行工具，把结果作为 tool message 回灌，模型继续生成最终答案。

问题三：Tool parser 解决什么问题？

回答要点：不同模型工具调用格式不同，parser 把模型原始输出转换成统一的 tool call 对象，便于应用层执行工具。

问题四：Multi-turn chat 为什么能受益于 RadixAttention？

回答要点：后续轮次共享之前完整历史，RadixAttention 可以复用历史 token 对应 KV cache，只计算新增 turn 的 suffix。

问题五：Agent serving 的性能瓶颈怎么拆？

回答要点：拆成模型 TTFT/TPOT、prefix cache 命中、scheduler queue、KV cache、工具执行延迟、工具错误率、agent 轮数和总 tokens。

32.28 标准回答模板

如果面试官问“SGLang 如何支持 multi-turn、tool use 和 agent serving”，可以这样回答：

从 runtime 视角看，multi-turn、tool use 和 agent serving 都是复杂 LLM program。它们不是一次简单 chat completion，而是多次

Multi-turn chat 中，后续轮次会共享前面完整历史。只要 chat template 和消息拼接稳定，RadixAttention 可以复用历史 token 的

Tool use 中，请求会携带 tools schema，模型生成 tool call，SGLang 可以通过 tool parser 把模型原始输出解析成统一的 tool nam

Agent serving 则会把这个过程循环起来：模型生成 action，外部工具返回 observation，再继续生成下一步。这个轨迹会不断增长，

32.29 Agent Serving 公式、状态门禁和可运行 demo

从 serving 视角看，一个 agent task 可以抽象为：

$$\tau_i = (s_i, u_i, g_{i,1}, c_{i,1}, o_{i,1}, \dots, g_{i,K_i}, a_i)$$

其中 s_i 是 session， u_i 是用户任务， $g_{i,k}$ 是第 k 次模型 generation， $c_{i,k}$ 是可选 tool call， $o_{i,k}$ 是 tool observation， a_i 是最终回答。 K_i 不是固定常数，这也是 agent serving 比单次 chat serving 更难调度的原因。

一次 task 的端到端延迟可以拆成：

$$L_i = \sum_{k=1}^{K_i} \left(L^{\{\mathrm{queue}\}}_{i,k} + L^{\{\mathrm{prefill}\}}_{i,k} + L^{\{\mathrm{decode}\}}_{i,k} + L^{\{\mathrm{streaming}\}}_{i,k} \right)$$

这个公式的直觉是：agent 慢不一定慢在模型，也可能慢在工具、排队、prefix cache miss、streaming 或 agent loop 轮数。

Multi-turn / agent prefix reuse 可以用：

$$R_{\{\mathrm{reuse}\}} = \frac{\sum_{i,k} H_{i,k}}{\max(1, \sum_{i,k} P_{i,k})}$$

其中 $P_{i,k}$ 是第 i 个 task 第 k 次 generation 的 prompt token 数， $H_{i,k}$ 是该 generation 命中的 prefix token 数。Session-aware routing 追求的是让后续轮次尽量落到已有 cache 的 replica 上：

$$R_{\{\mathrm{affinity}\}} = \frac{N_{\{\mathrm{same_replica}\}}}{\max(1, N_{\{\mathrm{followup}\}})}$$

Tool use 还要看 parser、validator 和 executor：

$$C_{\{\mathrm{parse}\}} = \frac{N_{\{\mathrm{parsed}\}}}{\max(1, N_{\{\mathrm{tool_request}\}})}$$
$$C_{\{\mathrm{block}\}} = \frac{N_{\{\mathrm{blocked}\}}}{\max(1, N_{\{\mathrm{tool_request}\}})}$$

C_{parse} 低说明模型格式、chat template 或 parser 不匹配； C_{block} 不是越低越好，高风险工具被权限系统正确阻断时，block 是安全证据。

Agent serving 的 runtime 门禁可以写成：

$$G_{\{\mathrm{agent}\}} = G_{\{\mathrm{session}\}} G_{\{\mathrm{cache}\}} G_{\{\mathrm{parser}\}} G_{\{\mathrm{tool}\}} G_{\{\mathrm{slot}\}} G_{\{\mathrm{round}\}}$$

这些因子分别要求 session affinity、prefix cache、tool parser、工具权限、等待工具时释放 GPU slot、agent round 上限和 task-level metrics 都能被观测。

下面是一个 0 依赖 toy demo。它模拟三个 task：

1. 第一个 task 生成 `get_weather` tool call，应用层执行工具，再回灌 tool result 生成最终回答。
2. 第二个 task 是同一 session 的后续轮次，用 session-aware routing 命中同一个 replica，并复用前一轮历史。
3. 第三个 task 试图调用需要确认的 `send_email`，validator 阻断后生成安全回复。

```
from dataclasses import dataclass
import json
```

```
@dataclass
```

```
class ToolSpec:
```

```
    name: str
```

```
    required: tuple
```

```
    needs_confirmation: bool = False
```

```
@dataclass
```

```
class AgentTask:
```

```
    task_id: str
```

```
    session_id: str
```

```
    user_tokens: list
```

steps: list

```
class ToyReplica:
    def __init__(self, name):
        self.name = name
        self.cached_prefixes = []
        self.model_calls = 0

    def lookup(self, tokens):
        best = 0
        for cached in self.cached_prefixes:
            n = 0
            for a, b in zip(tokens, cached):
                if a != b:
                    break
            n += 1
            best = max(best, n)
        return best

    def insert(self, tokens):
        key = tuple(tokens)
        if key not in self.cached_prefixes:
            self.cached_prefixes.append(key)

class ToyAgentServingAudit:
    def __init__(self, tasks, tools):
        self.tasks = tasks
        self.tools = {tool.name: tool for tool in tools}
        self.replicas = [ToyReplica("replica_a"), ToyReplica("replica_b")]
        self.affinity = {}
        self.base_tokens = ["SYS", "TOOLS", "get_weather", "send_email"]
        self.histories = {}

    def _route(self, session_id):
        if session_id in self.affinity:
            return self.affinity[session_id]
        replica = min(self.replicas, key=lambda item: item.model_calls)
        self.affinity[session_id] = replica
        return replica

    def _parse_call(self, fragments):
        raw = "".join(fragments)
        if not raw.startswith("CALL "):
            return None, {}, "bad_prefix"
        name, payload = raw[5:].split(" ", 1)
        try:
            args = json.loads(payload)
        except json.JSONDecodeError:
            return name, {}, "bad_json"
        return name, args, None

    def _validate_call(self, name, args, confirmed):
        tool = self.tools.get(name)
```

```

    if tool is None:
        return False, "unknown_tool"
    missing = [field for field in tool.required if field not in args]
    if missing:
        return False, "schema_missing_" + ",".join(missing)
    if tool.needs_confirmation and not confirmed:
        return False, "needs_confirmation"
    return True, "ok"

def _execute_tool(self, name, args):
    if name == "get_weather":
        return 120, ["weather", args["city"], "85F", "cloudy"]
    if name == "send_email":
        return 250, ["email", "sent"]
    return 0, ["unknown"]

def run(self):
    rows = []
    metrics = {
        "tasks": len(self.tasks),
        "model_calls": 0,
        "tool_calls_requested": 0,
        "tool_calls_executed": 0,
        "tool_calls_blocked": 0,
        "parse_success": 0,
        "validation_failures": 0,
        "tool_latency_ms": 0,
        "tool_wait_gpu_slots_held": 0,
        "naive_prefill_tokens": 0,
        "run_prefill_tokens": 0,
        "saved_prefill_tokens": 0,
        "output_tokens": 0,
        "stream_events": 0,
        "max_rounds_seen": 0,
    }
    for task in self.tasks:
        history = self.histories.setdefault(task.session_id, [])
        history.extend(task.user_tokens)
        task_rows = []
        for round_id, step in enumerate(task.steps, start=1):
            replica = self._route(task.session_id)
            prompt = self.base_tokens + history
            hit_len = replica.lookup(prompt)
            run_prefill = len(prompt) - hit_len
            metrics["naive_prefill_tokens"] += len(prompt)
            metrics["run_prefill_tokens"] += run_prefill
            metrics["saved_prefill_tokens"] += hit_len
            metrics["model_calls"] += 1
            replica.model_calls += 1
            metrics["max_rounds_seen"] = max(metrics["max_rounds_seen"], round_id)
            replica.insert(prompt)

            if step["kind"] == "tool_call":
                metrics["tool_calls_requested"] += 1
                name, args, parse_error = self._parse_call(step["fragments"])

```

```

generated_tokens = ["CALL", name or "parse_error"] + list(args.keys())
metrics["output_tokens"] += len(generated_tokens)
metrics["stream_events"] += len(step["fragments"])
history.extend(["ASSISTANT_CALL", name or "parse_error"] + list(args.keys()))
replica.insert(self.base_tokens + history)
if parse_error is None:
    metrics["parse_success"] += 1
valid, reason = self._validate_call(name, args, step.get("confirmed", False))
if valid:
    latency, result_tokens = self._execute_tool(name, args)
    metrics["tool_calls_executed"] += 1
    metrics["tool_latency_ms"] += latency
    history.extend(["TOOL_RESULT", name] + result_tokens)
else:
    metrics["tool_calls_blocked"] += 1
    metrics["validation_failures"] += 1
    history.extend(["TOOL_ERROR", name or "parse_error", reason])
task_rows.append(
    {
        "round": round_id,
        "replica": replica.name,
        "prompt_len": len(prompt),
        "hit_len": hit_len,
        "run_prefill": run_prefill,
        "tool": name,
        "valid": valid,
        "reason": reason,
    }
)
else:
    answer_tokens = step["output_tokens"]
    metrics["output_tokens"] += len(answer_tokens)
    metrics["stream_events"] += 1
    history.extend(["ASSISTANT_FINAL"] + answer_tokens)
    replica.insert(self.base_tokens + history)
    task_rows.append(
        {
            "round": round_id,
            "replica": replica.name,
            "prompt_len": len(prompt),
            "hit_len": hit_len,
            "run_prefill": run_prefill,
            "tool": None,
            "valid": True,
            "reason": "final",
        }
    )
rows.append({"task": task.task_id, "session": task.session_id, "rows": task_rows})

metrics["reuse_ratio"] = round(metrics["saved_prefill_tokens"] / max(1, metrics["naive_prefill_tokens"]), 3)
metrics["avg_tool_latency_ms"] = round(metrics["tool_latency_ms"] / max(1, metrics["tool_calls_executed"]), 1)
metrics["model_calls_per_task"] = round(metrics["model_calls"] / max(1, metrics["tasks"]), 3)
metrics["blocked_tool_rate"] = round(metrics["tool_calls_blocked"] / max(1, metrics["tool_calls_requested"]), 1)
metrics["session_affinity"] = {key: value.name for key, value in sorted(self.affinity.items())}
metrics["replica_model_calls"] = {replica.name: replica.model_calls for replica in self.replicas}

```

```

gates = {
    "session_affinity_kept": len(set(metrics["session_affinity"].values())) == 2
    and metrics["session_affinity"]["weather_session"] == "replica_a",
    "prefix_reuse_visible": metrics["saved_prefill_tokens"] > 0 and metrics["reuse_ratio"] > 0.35,
    "tool_parser_ok": metrics["parse_success"] == metrics["tool_calls_requested"],
    "permission_block_visible": metrics["tool_calls_blocked"] == 1,
    "gpu_slot_released_while_waiting": metrics["tool_wait_gpu_slots_held"] == 0,
    "agent_rounds_bounded": metrics["max_rounds_seen"] <= 3,
    "metrics_ready": metrics["model_calls"] == 5 and metrics["stream_events"] >= 5,
}
gates["agent_serving_gate"] = all(gates.values())
return rows, metrics, gates

tools = [
    ToolSpec("get_weather", ("city", "unit")),
    ToolSpec("send_email", ("to", "body"), needs_confirmation=True),
]

tasks = [
    AgentTask(
        task_id="weather_then_answer",
        session_id="weather_session",
        user_tokens=["USER", "weather", "Boston"],
        steps=[
            {"kind": "tool_call", "fragments": ["CALL get_weather ", '{"city":"Boston",', '"unit":"fahrenheit"}']},
            {"kind": "final", "output_tokens": ["Boston", "is", "85F", "and", "cloudy"]},
        ],
    ),
    AgentTask(
        task_id="followup_same_session",
        session_id="weather_session",
        user_tokens=["USER", "what", "to", "wear"],
        steps=[{"kind": "final", "output_tokens": ["Wear", "light", "clothes", "and", "carry", "water"]}],
    ),
    AgentTask(
        task_id="unsafe_email_blocked",
        session_id="ops_session",
        user_tokens=["USER", "email", "customer"],
        steps=[
            {"kind": "tool_call", "fragments": ["CALL send_email ", '{"to":"customer",', '"body":"discount"}']},
            {"kind": "final", "output_tokens": ["I", "need", "confirmation", "before", "sending"]},
        ],
    ),
]

rows, summary, gates = ToyAgentServingAudit(tasks, tools).run()
print("agent_serving_rows=", rows)
print("agent_serving_summary=", summary)
print("agent_serving_gates=", gates)

```

运行后可以看到类似输出：

```

agent_serving_summary= {'tasks': 3, 'model_calls': 5, 'tool_calls_requested': 2, 'tool_calls_executed': 1, 'tool_calls_t
agent_serving_gates= {'session_affinity_kept': True, 'prefix_reuse_visible': True, 'tool_parser_ok': True, 'permission_

```

这个 demo 要表达的工程重点是：

1. Multi-turn 和 agent serving 的单位是 task / session，不只是单次 completion。
2. 同一个 session 的后续 generation 应尽量命中同一个 runtime replica，提升 prefix cache 复用。
3. Tool parser 只负责解析格式，权限和确认仍要靠 validator / application。
4. 工具等待期间不应该占用 GPU decode slot。
5. 高风险工具被阻断不是失败，而是安全门禁生效。
6. Agent serving 的 dashboard 必须同时看 model calls、tool latency、prefix saved tokens、stream events、blocked tool rate 和 task-level latency。

32.30 小练习

1. 画一次 tool use 的完整链路：user、model、tool parser、tool execution、tool result、final answer。
2. 解释为什么 tool schema 顺序变化会影响 prefix cache 命中。
3. 设计一个 multi-turn chat 的 session-aware routing 策略。
4. 给出 5 个 agent serving 的关键指标。
5. 解释 tool parser 和 constrained decoding 的区别。
6. 设计一个防止 agent 无限循环的策略。
7. 画一个 agent search 的 prefix sharing tree，并标出 RadixAttention 可以复用的部分。

32.31 本章总结

Multi-turn、tool use 和 agent serving 是 SGLang 问题意识的集中体现：LLM 应用正在从单次生成变成多步、有状态、有工具、有分支的程序。

在这些场景中，runtime 需要处理的不只是模型 forward，还包括对话历史、工具 schema、tool parser、structured output、工具结果回灌、agent trajectory、cache locality、scheduler fairness 和安全边界。

SGLang 的 RadixAttention 可以复用多轮和 agent 轨迹中的共享 prefix，structured output 可以让工具调用更可靠，scheduler 可以管理多次 generation 的资源竞争。下一章会对比 SGLang 和 vLLM 的架构差异，把前面几章的 runtime、cache、scheduler、structured output 和 agent workload 放到同一张对比表里。

第 33 章 SGLang 和 vLLM 的架构对比

上一章讲了 Multi-turn、tool use 和 agent serving：SGLang 的重点不是只服务一次 chat completion，而是高效执行多轮、有状态、有工具、有分支的复杂 LLM programs。

到这里，本书第三部分和第四部分已经分别讲完了 vLLM 和 SGLang 的核心主线。

vLLM 主线是：PagedAttention、KV Cache Block Manager、continuous batching、scheduler、worker/executor、prefix cache、并行和调优。

SGLang 主线是：frontend/runtime 协同、RadixAttention、SGLang Runtime、scheduler、structured generation、speculative decoding、tool use 和 agent serving。

本章做一次系统对比。

一句话概括：

vLLM 更像面向通用高并发 LLM serving 的高性能执行引擎，核心是 KV cache 分页管理和 continuous batching；SGLang 更像面向复杂 LLM programs 的高性能 runtime，核心是前后端协同、RadixAttention、structured output 和多步程序执行。

33.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，主要参考五类公开资料：

1. vLLM / PagedAttention 论文，对 KV cache 动态增长、显存碎片、logical block 到 physical block 映射、block sharing 和 high-throughput serving 的问题背景给出基础口径。

2. vLLM 官方文档和本书第 16 到 25 章，对 OpenAI-compatible serving、PagedAttention、KV block manager、continuous batching、scheduler、worker / executor、prefix caching、并行和性能调优的工程口径做内部对齐。
3. SGLang 论文，对 frontend language、runtime、RadixAttention、structured output decoding、复杂 language model programs 和多步 / 分支 workload 的系统分层给出基础口径。
4. SGLang 官方文档和本书第 26 到 32 章，对 OpenAI-compatible API、native / offline engine、RadixAttention、scheduler、structured generation、speculative decoding、tool use 和 agent serving 做内部对齐。
5. 近年 serving 系统相关论文和文档，对 PagedAttention 后续 trade-off、prefix cache locality、agent serving、多 GPU routing、structured output 和 workload-sensitive routing 的边界做校准。

本章的边界也要说清：

1. 本章不是 benchmark 排名，也不写“谁全面替代谁”。不同版本、硬件、模型、attention backend、并行配置和 workload 会显著改变性能结果。
2. 本章只讲架构重心和选型方法：vLLM-like runtime 更偏通用高并发 serving 和 KV block 管理，SGLang-like runtime 更偏复杂 LLM program 的表达与执行。
3. PagedAttention 和 RadixAttention 不在同一抽象层：前者主要回答 KV cache 物理块如何管理，后者主要回答 token prefix / program trajectory 如何匹配和复用。
4. vLLM 可以作为 agent framework 的模型后端；SGLang 的优势是让部分复杂程序结构对 runtime 更可见。不能把“runtime 能不能服务 agent”误解成二选一。
5. 本章 demo 是教学版 workload router / architecture comparator，用 toy metrics 说明独立 chat、共享 RAG、结构化抽取和 agent tree 的不同适配性，不代表真实性能数字。

33.1 本章目标

读完本章，你应该能讲清：

1. vLLM 和 SGLang 的共同底层问题是什么。
2. 二者的问题出发点有什么不同。
3. PagedAttention 和 RadixAttention 的直觉差异。
4. 两者 scheduler 都要解决什么，SGLang 额外考虑什么。
5. structured output、tool use、agent workload 为什么更像 SGLang 的主战场。
6. 工程选型时如何判断更适合 vLLM、SGLang 或组合方案。
7. 面试中如何避免把二者说成简单替代关系。

33.2 共同底层问题

先不要急着区分。

vLLM 和 SGLang 都在解决同一个大问题：

如何在 GPU 上高效服务大语言模型自回归生成？

它们共同面对：

1. Prefill 和 decode 分离。
2. KV cache 显存增长。
3. 长短请求混合。
4. batch 动态变化。
5. TTFT、TPOT、吞吐之间的权衡。
6. Streaming 输出。
7. Sampling 参数。
8. 多 GPU 并行。
9. 线上稳定性和监控。

所以两者不是完全不同物种。

从 serving engine 基础能力看，它们都有类似模块：

API server

- > request parsing
- > tokenizer
- > scheduler
- > KV cache manager
- > model runner
- > sampler
- > output streamer

差别在于：它们对“最重要的问题”排序不同。

33.3 问题出发点对比

vLLM 的核心问题意识：

大量请求来了，如何高吞吐、低显存浪费、稳定地执行 prefill/decode？

SGLang 的核心问题意识：

复杂 LLM 程序来了，如何表达多步控制流，并高效复用其中的共享状态？

因此 vLLM 更强调：

- 1. PagedAttention。
- 2. KV block 管理。
- 3. Continuous batching。
- 4. OpenAI-compatible serving。
- 5. 通用在线推理吞吐。

SGLang 更强调：

- 1. Frontend language。
- 2. Runtime 执行复杂 LLM program。
- 3. RadixAttention。
- 4. Structured generation。
- 5. Multi-turn、tool use、agent serving。

这不是谁更先进的问题，而是设计重心不同。

33.4 一张总表

先用表建立整体印象。

维度	vLLM	SGLang
核心定位	通用高性能 LLM serving engine	面向复杂 LLM programs 的高性能 runtime
代表机制	PagedAttention	RadixAttention
核心痛点	KV cache 显存碎片、动态 batching	多次 generation 之间的 prefix sharing 和控制流
API 生态	OpenAI-compatible API 强	OpenAI-compatible、native API、offline engine、frontend language
Cache 重点	block-based KV cache 管理	radix tree 管理复杂 prefix sharing
Scheduler 重点	prefill/decode、token/KV budget、continuous batching	以上 + cache hit length、复杂程序分支、grammar/tool workload
Structured output	支持能力逐步增强	是核心问题意识之一

维度	vLLM	SGLang
Agent workload	可以服务，但不是最初核心表达层	更强调 multi-turn、tool use、agent serving
学习价值	理解现代 serving engine 基础	理解复杂 LLM program runtime
典型场景	通用 chat、completion、RAG、高并发 serving	agent、self-consistency、ToT、structured extraction、多轮复杂程序

这张表是直觉，不是绝对边界。两个项目都在快速演进，很多能力会互相靠近。

33.5 PagedAttention vs RadixAttention

这两个名字很容易混在一起。

PagedAttention 解决的核心问题是：

KV cache 如何像分页内存一样管理，减少显存碎片和连续分配要求？

它关注的是物理内存布局：

logical token blocks -> physical KV blocks

RadixAttention 解决的核心问题是：

不同请求或 generation call 的 token prefix 如何自动匹配和复用？

它关注的是 prefix 共享结构：

token prefix tree -> cached KV references

可以这样记：

PagedAttention: KV cache 怎么放

RadixAttention: 哪些 token prefix 的 KV cache 可以复用

当然, 真实系统里两者都需要和底层 KV layout 配合。RadixAttention 找到了可复用 prefix, 也要依赖 memory pool 或 paged layout 来引用实际 KV。

33.6 Prefix cache 对比

vLLM 的 prefix caching 常见直觉是基于 block hash：

block_hash = hash(parent_hash, block_tokens, extra_hashes)

新请求从左到右匹配 full blocks，命中就复用对应 KV blocks。

这种方式适合大量请求共享固定前缀，例如：

[system prompt][RAG document][question A]
 [system prompt][RAG document][question B]

SGLang 的 RadixAttention 更强调树状共享：

```

root prompt
-> branch A
    -> branch A1
    -> branch A2
-> branch B
    -> branch B1
    -> branch B2
  
```

它适合 self-consistency、Tree-of-Thought、agent search、多轮 chat 等复杂结构。
对比：

维度	vLLM prefix cache	SGLang RadixAttention
基本结构	block hash	radix tree
复用单位	full KV blocks	token prefix path 对应 KV
典型共享	线性前缀	树状前缀
代表场景	固定 system/RAG 前缀	多分支、多轮、多 generation
关注点	block 命中和生命周期	longest prefix match、split、insert、evict

33.7 Scheduler 对比

两者 scheduler 都要处理：

- 1. Waiting queue。
- 2. Running requests。
- 3. Prefill 和 decode 混合。
- 4. Token budget。
- 5. Sequence budget。
- 6. KV cache budget。
- 7. Streaming。
- 8. Abort 和 cleanup。

vLLM scheduler 的典型问题是：

本轮哪些 waiting 请求 prefill?
哪些 running 请求 decode?
KV blocks 是否够?
如何控制 batch tokens 和 sequence 数?

SGLang scheduler 也要回答这些问题，但还要额外考虑：

- 1. RadixAttention cache hit length。
- 2. Uncached suffix cost。
- 3. Cache-aware admission。
- 4. Radix tree node pin/evict。
- 5. Structured output grammar state。
- 6. Tool call 和 frontend program 分支。
- 7. Agent 中动态派生的 generation。
- 8. Speculative decoding 的 draft/verify 成本。

所以可以说：

SGLang scheduler 继承了 continuous batching 的问题，又叠加了复杂程序执行的问题。

33.8 Request 模型对比

vLLM 中，一个请求通常可以理解为一个或一组 sequence：

prompt tokens + generated tokens + block table + sampling params

SGLang 中，一个 generation 请求也有类似状态，但它可能来自更大的 LLM program：

frontend program
-> gen node 1
-> fork branch
-> gen node 2A / 2B
-> tool call
-> gen final

也就是说，vLLM 更常从“独立请求流”视角组织系统。

SGLang 更常从“程序执行图或多步调用”视角组织系统。

这会影响：

1. Cache 复用方式。
2. Scheduler 成本估计。
3. Request 生命周期。
4. Output 结构。
5. 工具和外部环境交互。

33.9 API 和入口对比

vLLM 的强项之一是 OpenAI-compatible API 和易部署的 serving 体验。

典型用法：

```
launch server -> /v1/chat/completions -> streaming response
```

SGLang 也支持 OpenAI-compatible API，但入口更多：

1. OpenAI-compatible API。
2. Native /generate API。
3. Offline Engine API。
4. SGLang frontend language。

这些入口反映了 SGLang 的定位：不仅是通用 chat server，也希望作为复杂 LLM program 的执行 runtime。

如果你的应用完全是普通 chat completion，OpenAI-compatible API 就够。

如果你需要在 Python 中组织多步 generation、fork、choices、structured output 和工具交互，SGLang 的 frontend/runtime 组合更有表达力。

33.10 Structured output 对比

结构化输出两边都可以做，但 SGLang 把它放在更核心的位置讲。

原因是 SGLang 面向复杂程序，而复杂程序经常需要：

1. JSON 抽取结果。
2. choices。
3. tool arguments。
4. EBNF grammar。
5. structural tag。
6. agent action schema。

SGLang structured generation 的 runtime 视角是：

```
constraint spec -> grammar backend -> grammar state -> valid token mask -> sampler
```

vLLM 也可以支持 structured outputs，但从本书的讲解主线看，vLLM 部分更集中在 KV memory 和 scheduling，SGLang 部分更集中在 structured generation 和复杂程序控制。

面试时可以说：

Structured output 不是 SGLang 独有，但它和 SGLang 的复杂 LLM program 定位更强相关。

33.11 Tool use 和 agent 对比

vLLM 可以作为 agent 后端模型服务。

很多 agent framework 完全可以调用 vLLM 的 OpenAI-compatible API。

但这种模式下，agent 逻辑通常在外部框架：

agent framework
-> vLLM chat completion
-> parse tool call
-> execute tool
-> vLLM chat completion

SGLang 更强调 runtime 与 LLM program 结构结合：

SGLang frontend / native runtime
-> multi-generation
-> fork / choices
-> structured output
-> RadixAttention reuse
-> tool / agent serving support

区别不是 “vLLM 不能做 agent”，而是：

vLLM 更常作为 agent 的模型服务后端

SGLang 更强调把复杂 agent-like LLM program 的执行效率纳入 runtime 设计

33.12 Speculative decoding 对比

Speculative decoding 不是 SGLang 独有，很多高性能 serving engine 都在支持。

它解决的是 decode 阶段主模型逐 token 串行的问题。

SGLang 文档中提供了多种 speculative 路径：

1. EAGLE。
2. EAGLE3。
3. MTP。
4. DFLASH。
5. Standalone draft model。
6. NGRAM。
7. Adaptive speculative decoding。

vLLM 生态中也有 speculative decoding 相关能力。

对比时不要说 “speculative decoding 是 SGLang 和 vLLM 的根本区别”。

更准确的说法是：

Speculative decoding 是现代 serving runtime 共同追求的 decode 优化方向；SGLang 在其 runtime 中提供了多种实现路径，并需要

33.13 并行和多 GPU 对比

两者都需要支持多 GPU serving。

常见并行包括：

1. Tensor Parallel。
2. Pipeline Parallel。
3. Data Parallel。
4. Expert Parallel。
5. 多副本部署。

vLLM 的多 GPU serving 更常围绕通用模型服务吞吐、KV block 管理、worker/executor 和并行配置展开。

SGLang 也支持多 GPU 和多硬件后端，并且在复杂 workload 下还要考虑：

1. Prefix cache locality。
2. Session-aware routing。
3. Agent 多轮请求打到同一 replica。

4. Tool 等待阶段不占 GPU。
5. Structured output 和 speculative decoding 的额外资源。

多 GPU 层面没有简单的“谁更好”。要看模型、硬件、并行方式、请求长度分布和 workload 类型。

33.14 性能调优思路对比

vLLM 调优常围绕：

1. max_num_batched_tokens。
2. max_num_seqs。
3. gpu_memory_utilization。
4. max_model_len。
5. chunked prefill。
6. prefix cache。
7. preemption。
8. TP/PP/DP。
9. TTFT、TPOT、KV blocks、GPU 利用率。

SGLang 调优除了这些通用指标，还要看：

1. RadixAttention hit length。
2. Saved prefill tokens。
3. Radix tree eviction。
4. Grammar mask latency。
5. Tool parser 失败率。
6. Agent rounds。
7. Speculative accept length。
8. Draft/verify latency。
9. Session cache locality。

也就是说：

vLLM 调优更偏通用 serving engine 指标

SGLang 调优更要结合复杂程序结构和 runtime 特性

33.15 选型：什么时候优先 vLLM

优先考虑 vLLM 的场景：

1. 主要是标准 chat/completion API。
2. 需要快速上线 OpenAI-compatible 服务。
3. 高并发通用文本生成。
4. workload 以独立请求为主。
5. 团队更熟悉 vLLM 生态。
6. 重点瓶颈是 KV cache 显存和 continuous batching。
7. 希望使用成熟的通用 serving 路线。

例如：

企业内部通用 Chat API

RAG 问答服务

批量摘要生成

多租户文本生成平台

这些场景 vLLM 很自然。

33.16 选型：什么时候优先 SGLang

优先考虑 SGLang 的场景：

1. 多步 LLM program。
2. Self-consistency 或多候选采样。
3. Tree-of-Thought 或搜索式推理。
4. Agent serving。
5. 多轮工具调用。
6. 复杂 structured output。
7. 大量共享 prefix 的分支 workload。
8. 希望用 frontend language 表达 LLM 控制流。
9. 需要 offline engine 批处理和自定义 server。

例如：

法律文档分析 agent

代码修复 agent

多候选推理评测

结构化抽取流水线

复杂工具调用系统

这些场景更能体现 SGLang 的设计重心。

33.17 也可以组合使用

实际工程里，不一定只能选一个。

可能的组合：

1. 普通 chat 流量走 vLLM。
2. 复杂 agent 流量走 SGLang。
3. 离线评测用 SGLang offline engine。
4. 平台层统一路由，根据 workload 选择 backend。
5. 不同模型或业务线使用不同 runtime。

关键是平台层要定义清楚边界：

请求类型 -> 路由策略 -> runtime backend -> 监控指标 -> 回滚方案

不要为了统一技术栈，把所有 workload 强行塞进同一个 runtime。

33.18 面试中容易犯的错误

错误一：说 SGLang 完全替代 vLLM。

不准确。两者设计重心不同，都在演进。很多场景 vLLM 仍然是非常自然的选择。

错误二：说 vLLM 不能做 agent。

不准确。vLLM 可以作为 agent 的模型 serving 后端，只是 agent 控制流通常在外部框架。

错误三：把 PagedAttention 和 RadixAttention 混为一谈。

PagedAttention 关注 KV cache 分页布局，RadixAttention 关注 prefix sharing 的树状复用。

错误四：只比较 benchmark 数字。

Benchmark 依赖模型、硬件、prompt 长度、输出长度、batch、并行、采样和 workload。架构对比要先看问题场景。

错误五：认为 structured output 是后处理。

在 SGLang 的 structured generation 中，约束可以进入 decoding，每步用 grammar mask 限制合法 token。

33.19 面试官会怎么问

问题一：SGLang 和 vLLM 最大区别是什么？

回答要点：vLLM 更强调高并发通用 LLM serving，核心是 PagedAttention、KV block 管理和 continuous batching；SGLang 更强调复杂 LLM programs，核心是 frontend/runtime 协同、RadixAttention、structured output 和 agent/multi-turn 支持。

问题二：PagedAttention 和 RadixAttention 有什么区别？

回答要点：PagedAttention 解决 KV cache 物理内存分页和碎片问题；RadixAttention 用 radix tree 管理 token prefix 到 KV cache 的映射，解决复杂程序中共享前缀自动复用问题。

问题三：vLLM 能不能做 agent serving？

回答要点：可以作为 agent 的模型服务后端。但 agent 控制流、工具调用解析、状态管理通常在外部框架；SGLang 更强调把复杂 LLM program 的表达和执行效率纳入 runtime 设计。

问题四：什么时候选 SGLang？

回答要点：多步 generation、分支搜索、self-consistency、Tree-of-Thought、multi-turn、tool use、agent、复杂 structured output、共享 prefix 明显的 workload。

问题五：什么时候选 vLLM？

回答要点：标准 OpenAI-compatible chat/completion、高并发通用 serving、独立请求流、主要瓶颈在 KV cache 管理和 continuous batching 的场景。

33.20 标准回答模板

如果面试官问“对比一下 SGLang 和 vLLM”，可以这样回答：

vLLM 和 SGLang 都是高性能 LLM serving/runtime 系统，都要处理 prefill、decode、KV cache、batching、scheduler、sampling 和

vLLM 更像通用高并发 LLM serving engine。它的核心创新是 PagedAttention，用类似分页内存的方式管理 KV cache，配合 continuous level scheduling，提高显存利用率和吞吐，适合大量独立 chat/completion/RAG 请求。

SGLang 更强调复杂 LLM programs 的表达和执行效率。很多应用不是一次 generate，而是包含多次 generation、fork、self-consistency、Tree-of-Thought、multi-turn、tool use、agent 和 structured output。SGLang 通过 frontend language 表达这些控

PagedAttention 和 RadixAttention 也不是一回事。PagedAttention 关注 KV cache 物理布局和 block 管理，RadixAttention 关注 t

所以我不会说 SGLang 简单替代 vLLM。标准高并发 OpenAI-compatible serving 很适合 vLLM；复杂 agent、结构化输出、多分支推理

33.21 架构对比公式、workload router 和可运行 demo

做架构对比时，不要只说“功能多”或“benchmark 高”。更好的方式是把 workload 抽象出来：

$$w_i = (c_i, P_i, O_i, H_i, B_i, S_i, T_i)$$

其中 c_i 是同类 generation calls 数量， P_i 是每次 prompt tokens， O_i 是 output tokens， H_i 是共享 prefix tokens， B_i 是分支因子， S_i 表示是否需要 structured output， T_i 表示是否包含 tool / agent 轨迹。

如果没有任何 cache，prefill token 数是：

$$P_{\{\mathrm{naive}\}} = \sum_i c_i P_i$$

vLLM-like prefix cache 常以完整 KV block 命中为基础。给定 block size b ，共享 prefix 中能稳定复用的完整 block token 数可以写成：

$$H_i^{\{\mathrm{block}\}} = b \left\lfloor \left\lfloor \frac{H_i}{b} \right\rfloor \right\rfloor$$

对应的简化 prefill 节省是：

$$S_{\{\mathrm{vllm}\}} = \sum_i (c_i - 1) H_i^{\{\mathrm{block}\}}$$

SGLang-like runtime 除了线性共享 prefix，还要看多步程序和分支轨迹。一个简化写法是：

$$S_{\{\mathrm{sglang}\}} = \sum_i (c_i - 1) H_i + \sum_i \max(0, B_i - 1) H_i^{\{\mathrm{branch}\}}$$

其中 H_{branch} 表示多分支 program 中额外共享的 root / trajectory token。这个式子只是教学抽象，真实系统还会受 page size、KV layout、eviction、scheduler、routing 和安全隔离影响。

运行时选择不能只看 cache，还要看 workload fit：

$$F(r, w_i) = F_{\{\mathrm{serving}\}} + F_{\{\mathrm{cache}\}} + F_{\{\mathrm{program}\}} + F_{\{\mathrm{grammar}\}} + F_{\{\mathrm{agent}\}}$$

r 是 runtime 类型。独立高并发 chat 通常让 F_{serving} 权重大；structured output、tool use、multi-turn 和分支搜索会提高 F_{program} 、 F_{grammar} 和 F_{agent} 的权重。

架构对比门禁可以写成：

$$G_{\{\mathrm{arch}\}} = G_{\{\mathrm{core}\}} G_{\{\mathrm{paged}\}} G_{\{\mathrm{radix}\}} G_{\{\mathrm{vllm}\}} G_{\{\mathrm{sglang}\}} G_{\{\mathrm{mtrn}\}}$$

这些因子分别要求共同 serving core、PagedAttention / block cache、RadixAttention / program reuse、vLLM 不被错误替代、SGLang 的复杂程序优势可见、平台层 workload router 可解释、指标可复盘。

下面是一个 0 依赖 toy demo。它把四类 workload 放进同一个对比器：

1. independent_chat: 大量独立短 chat，更偏通用 serving。
2. rag_shared_doc: 共享 RAG 文档，vLLM-like full block prefix cache 和 SGLang-like prefix reuse 都能受益。
3. json_extraction: 结构化抽取，多次 generation 和 structured output 更偏 SGLang-like program runtime。
4. agent_tree: agent / branch trajectory，多分支 prefix sharing 更能体现 RadixAttention 的价值。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Workload:
```

```
    name: str
    calls: int
    prompt_tokens: int
    output_tokens: int
    shared_prefix: int
    branch_factor: int = 1
    structured: bool = False
    tool_or_agent: bool = False
    independent: bool = True
```

```
class ToyArchitectureComparator:
```

```
    def __init__(self, workloads, block_size=4):
        self.workloads = workloads
        self.block_size = block_size

    def _full_block_hit(self, tokens):
        return self.block_size * (tokens // self.block_size)

    def _vllm_reuse(self, workload):
        if workload.calls <= 1:
            return 0
        return self._full_block_hit(workload.shared_prefix) * (workload.calls - 1)
```



```

def _sglang_reuse(self, workload):
    if workload.calls <= 1:
        return 0
    base = workload.shared_prefix * (workload.calls - 1)
    branch_bonus = 0
    if workload.branch_factor > 1:
        branch_bonus = (workload.branch_factor - 1) * self._full_block_hit(workload.shared_prefix // 2)
    return base + branch_bonus

def _fit_scores(self, workload):
    vllm = 0
    sglang = 0
    if workload.independent:
        vllm += 4
        sglang += 2
    if workload.shared_prefix > 0:
        vllm += 1
        sglang += 1
    if workload.calls > 1 and not workload.independent:
        sglang += 1
    if workload.branch_factor > 1:
        sglang += 3
    if workload.structured:
        vllm += 1
        sglang += 2
    if workload.tool_or_agent:
        vllm += 1
        sglang += 3
    return vllm, sglang

def run(self):
    rows = []
    totals = {
        "prompt_tokens_if_naive": 0,
        "vllm_run_prefill": 0,
        "sglang_run_prefill": 0,
        "vllm_saved_prefill": 0,
        "sglang_saved_prefill": 0,
        "vllm_wins": 0,
        "sglang_wins": 0,
        "mixed_route": 0,
    }
    for workload in self.workloads:
        naive = workload.calls * workload.prompt_tokens
        v_saved = self._vllm_reuse(workload)
        s_saved = self._sglang_reuse(workload)
        v_run = max(0, naive - v_saved)
        s_run = max(0, naive - s_saved)
        v_fit, s_fit = self._fit_scores(workload)
        if v_fit > s_fit:
            route = "vllm"
            totals["vllm_wins"] += 1
        elif s_fit > v_fit:
            route = "sglang"
            totals["sglang_wins"] += 1

```

```

else:
    route = "either"
    totals["mixed_route"] += 1
rows.append(
    {
        "workload": workload.name,
        "calls": workload.calls,
        "naive_prefill": naive,
        "vllm_saved": v_saved,
        "sglang_saved": s_saved,
        "vllm_run": v_run,
        "sglang_run": s_run,
        "vllm_fit": v_fit,
        "sglang_fit": s_fit,
        "recommended": route,
    }
)
totals["prompt_tokens_if_naive"] += naive
totals["vllm_run_prefill"] += v_run
totals["sglang_run_prefill"] += s_run
totals["vllm_saved_prefill"] += v_saved
totals["sglang_saved_prefill"] += s_saved

totals["vllm_saved_ratio"] = round(totals["vllm_saved_prefill"] / max(1, totals["prompt_tokens_if_naive"]), 3)
totals["sglang_saved_ratio"] = round(totals["sglang_saved_prefill"] / max(1, totals["prompt_tokens_if_naive"]), 3)
totals["sglang_extra_saved"] = totals["sglang_saved_prefill"] - totals["vllm_saved_prefill"]
gates = {
    "common_serving_core_visible": all(row["naive_prefill"] > 0 for row in rows),
    "paged_prefix_cache_visible": totals["vllm_saved_prefill"] > 0,
    "radix_program_reuse_visible": totals["sglang_saved_prefill"] > totals["vllm_saved_prefill"],
    "vllm_not_replaced": totals["vllm_wins"] >= 1,
    "sglang_program_fit_visible": totals["sglang_wins"] >= 2,
    "workload_router_needed": len({row["recommended"] for row in rows}) >= 2,
    "metrics_ready": totals["sglang_saved_ratio"] > totals["vllm_saved_ratio"],
}
gates["architecture_comparison_gate"] = all(gates.values())
return rows, totals, gates

workloads = [
    Workload("independent_chat", calls=8, prompt_tokens=20, output_tokens=16, shared_prefix=0, independent=True),
    Workload("rag_shared_doc", calls=4, prompt_tokens=80, output_tokens=24, shared_prefix=56, independent=True),
    Workload("json_extraction", calls=6, prompt_tokens=44, output_tokens=12, shared_prefix=32, structured=True, independent=True),
    Workload(
        "agent_tree",
        calls=5,
        prompt_tokens=72,
        output_tokens=18,
        shared_prefix=48,
        branch_factor=3,
        structured=True,
        tool_or_agent=True,
        independent=False,
    ),
]

```

```
rows, summary, gates = ToyArchitectureComparator(workloads).run()
print("architecture_rows=", rows)
print("architecture_summary=", summary)
print("architecture_gates=", gates)
```

运行后可以看到类似输出：

```
architecture_summary= {'prompt_tokens_if_naive': 1104, 'vllm_run_prefill': 584, 'sglang_run_prefill': 536, 'vllm_saved_
architecture_gates= {'common_serving_core_visible': True, 'paged_prefix_cache_visible': True, 'radix_program_reuse_vis
```

这个 demo 的重点是：

1. 两者都有共同 serving core，不能把一个说成“会 serving”，另一个说成“不会 serving”。
2. vLLM-like block prefix cache 在共享 RAG 文档上能明显节省 prefill。
3. SGLang-like program runtime 在 structured extraction 和 agent tree 里能看到更多 program structure。
4. 架构选型要看 workload，不是把所有请求都塞进一个结论。
5. 平台层可以用 workload router，把独立 chat / RAG / agent / structured extraction 分流到更合适的 backend。

33.22 小练习

1. 用一张表对比 PagedAttention 和 RadixAttention。
2. 给出 3 个更适合 vLLM 的 workload，说明原因。
3. 给出 3 个更适合 SGLang 的 workload，说明原因。
4. 设计一个平台层路由策略，把普通 chat 和 agent 任务路由到不同 runtime。
5. 解释为什么 benchmark 不能脱离 workload 讨论。
6. 画出 vLLM 和 SGLang 都共有的 serving engine 模块图。
7. 面试中用 2 分钟讲清 SGLang 和 vLLM 的区别。

33.23 本章总结

vLLM 和 SGLang 都是现代 LLM 推理系统的重要代表。

vLLM 的主线是高并发通用 serving: PagedAttention、KV block manager、continuous batching、scheduler、worker/executor 和调优。

SGLang 的主线是复杂 LLM program runtime: frontend/runtime 协同、RadixAttention、cache-aware scheduler、structured generation、speculative decoding、tool use 和 agent serving。

理解二者的最好方式不是问“谁更强”，而是问“我的 workload 是什么，瓶颈在哪里，runtime 能看到哪些结构，哪些优化真正能生效”。

下一章会进入 mini-sglang 源码学习路径，把 SGLang 这部分的概念落到一个更可读的源码路线中。

第 34 章 mini-sglang 源码学习路径

上一章对比了 SGLang 和 vLLM 的架构: vLLM 更强调通用高并发 serving, SGLang 更强调复杂 LLM programs 的表达和高效执行。

本章把 SGLang 这部分收束到源码学习路径。

这里说的 mini-sglang 指的是 SGLang 官方教学项目，目标不是覆盖完整生产系统，而是用更小、更可读的代码帮助你理解 SGLang 的核心设计。它适合在读真实 SGLang 源码之前作为过渡。

项目链接：

<https://github.com/sgl-project/mini-sglang>

一句话概括：

读 mini-sglang 不要从文件名开始, 而要带着 runtime 主链路去读: 入口 API、请求状态、scheduler、KV cache、RadixAttention、model runner、sampler、structured output 和 streaming, 每读一个模块都问它在请求生命周期中解决什么问题。

34.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时, 主要参考三类资料:

1. sgl-project/mini-sglang 官方仓库和 README。README 将 mini-sglang 定位为约 5000 行 Python 的紧凑 SGLang 实现, 并列出了 Radix Cache、Chunked Prefill、Overlap Scheduling、Tensor Parallelism、FlashAttention / FlashInfer backend 等核心学习点。
2. mini-sglang 仓库中的 system architecture / structure 文档。它把教学实现拆成 API server、request / tokenizer、scheduler、engine、KV cache、Radix cache、attention backend、message / protocol 等模块, 适合映射到本书前面讲过的 runtime 生命周期。
3. SGLang 论文和官方文档, 以及本书第 26 到 33 章对 SGLang runtime、RadixAttention、scheduler、structured output、speculative decoding、agent serving 和 vLLM 对比的内部口径。

本章的边界也要说清:

1. 本章是源码学习方法, 不是完整源码逐行讲解; 文件名、类名和实现细节可能随 mini-sglang 版本变化。
2. 本章不把 mini-sglang 当生产级 SGLang。教学实现的价值是帮助建立 runtime 骨架, 真实 SGLang 还包含更多模型、硬件后端、分布式、grammar backend、tool parser、speculative decoding 和生产监控细节。
3. 本章新增 demo 是源码路径审计器, 用来检查阅读笔记是否覆盖模块、生命周期、资源和实验, 而不是执行真实 mini-sglang。
4. 读源码的验收标准不是“看过多少文件”, 而是能把模块输入输出、状态变化、KV 生命周期、scheduler 输出和实验观察映射到 TTFT、TPOT、prefix hit、KV pressure、grammar mask 和 streaming 指标。

34.1 本章目标

读完本章, 你应该能讲清:

1. 为什么要先读 mini-sglang, 而不是直接啃完整 SGLang。
2. 阅读 mini-sglang 前应该具备哪些概念地图。
3. 如何按请求生命周期读源码。
4. 如何把代码模块映射到 SGLang Runtime 总览。
5. 如何重点观察 RadixAttention、scheduler、structured output。
6. 如何做源码笔记和调试实验。
7. 面试中如何用 mini-sglang 证明你理解推理框架。

34.2 为什么需要教学项目

真实 SGLang 是生产级项目。

它会包含大量工程细节:

1. 多模型支持。
2. 多硬件后端。
3. 多 GPU 并行。
4. CUDA graph。
5. attention backend。
6. grammar backend。
7. speculative decoding。
8. tool parser。
9. metrics 和 tracing。
10. 分布式部署。

这些都重要, 但初学源码时很容易迷路。

教学项目的价值是把主线缩小：

先看清请求如何跑通，再看生产系统如何扩展。

mini-sglang 的学习目标不是 “记住所有类名”，而是建立一套源码阅读坐标系。

34.3 阅读前的概念地图

读 mini-sglang 前，建议先把前面几章的概念串起来。

核心链路：

```
request
-> tokenize
-> scheduler
-> prefix cache / RadixAttention
-> KV cache allocation
-> prefill
-> decode loop
-> sampling
-> output / streaming
```

核心状态：

```
WAITING
-> PREFILLING
-> DECODING
-> FINISHED / ABORTED
```

核心资源：

```
GPU compute
KV cache memory
token budget
sequence budget
prefix cache entries
grammar state
```

核心优化：

```
continuous batching
RadixAttention
structured generation
speculative decoding
```

读源码时，每看到一个函数，就把它放回这几张图里。

34.4 不建议的阅读方式

不建议这样读：

打开 repo -> 从第一个文件开始 -> 每个类都看 -> 试图一次看懂全部

这样很容易陷入：

1. 文件名很多，不知道主线。
2. 配置细节太多，忘了请求生命周期。
3. 看到 scheduler 不知道它的输入输出。
4. 看到 cache 不知道它和 model runner 的关系。
5. 看到 frontend 语法不清楚 runtime 如何执行。

更好的方式是：

先跑通最小例子

- > 找入口
- > 跟一个请求
- > 画状态机
- > 画模块关系
- > 再读每个优化点

34.5 推荐阅读顺序

建议按 8 步读。

第一步：启动和最小请求。

第二步：API 或 Engine 入口。

第三步：请求对象和状态机。

第四步：scheduler 主循环。

第五步：KV cache 和 memory pool。

第六步：RadixAttention / prefix sharing。

第七步：model runner、prefill、decode、sampler。

第八步：structured output、tool use、streaming 等扩展。

每一步都要回答三个问题：

1. 这个模块输入是什么？
2. 输出是什么？
3. 它改变了哪些状态或资源？

34.6 第一步：跑通最小请求

如果你拿到 mini-sglang 源码，第一步不是读代码，而是跑通最小请求。

目标是看到：

prompt -> output text

你要记录：

1. 启动命令。
2. 模型路径。
3. 请求格式。
4. 输出格式。
5. 是否支持 streaming。
6. 日志里出现了哪些模块名。

如果项目提供 HTTP server，就发一个最小 /generate 或 chat 请求。

如果项目提供 offline engine，就直接在 Python 里调用 generate。

这一阶段不要优化性能，也不要改代码。

目标只是确定主链路可运行。

34.7 第二步：找到入口层

入口层通常负责：

1. 接收请求。
2. 解析 prompt 或 messages。

3. 解析 sampling params。
4. 调 tokenizer。
5. 构造内部 request。
6. 把请求交给 runtime。

你要找的问题：

外部 JSON 请求在哪里变成内部 Request 对象？

建议做一张表：

外部字段	内部字段	作用
prompt / text	input_ids	模型输入 token
max_new_tokens	output budget	生成上限
temperature	sampling params	控制采样
stream	output mode	是否流式返回
json_schema	grammar constraint	结构化输出

入口层看懂后，后面的 scheduler 才有上下文。

34.8 第三步：请求对象和状态机

接下来找 Request 类或类似结构。

重点字段通常包括：

1. request id。
2. input ids。
3. output ids。
4. sampling params。
5. status。
6. arrival time。
7. prompt length。
8. num computed tokens。
9. KV cache references。
10. prefix cache hit info。
11. finish reason。

你要画出状态变化：

WAITING -> PREFILLING -> DECODING -> FINISHED

并回答：

1. 请求什么时候进入 waiting？
2. 什么时候算完成 prefill？
3. 什么时候进入 decode？
4. stop 条件在哪里判断？
5. 资源在哪里释放？

如果只看模型 forward，不看 request state，就读不懂 serving engine。

34.9 第四步：scheduler 主循环

Scheduler 是最重要的模块之一。

你要找到类似下面的主循环：

```
while True:
    add_new_requests()
    schedule = scheduler.schedule()
    outputs = model_runner.forward(schedule)
    update_states(outputs)
```

重点看 scheduler 的输入：

1. waiting queue。
2. running requests。
3. token budget。
4. sequence budget。
5. KV cache 状态。
6. prefix cache 命中。

重点看 scheduler 的输出：

1. scheduled prefill requests。
2. scheduled decode requests。
3. 每个请求本轮处理 token 数。
4. KV allocation plan。
5. attention metadata。

读 scheduler 时不要只问 “它用了什么算法”，要问：

它如何在 TTFT、TPOT、吞吐和显存之间取舍？

34.10 第五步：KV cache 和 memory pool

KV cache 是 runtime 的核心资源。

你要找：

1. KV cache 初始化在哪里。
2. 每个 token 的 KV 写到哪里。
3. request 如何记录自己的 KV 位置。
4. decode 如何读取历史 KV。
5. 请求结束后如何释放。
6. prefix cache 是否会保留部分 KV。
7. ref count 或 pin 机制在哪里。

可以画一张映射图：

```
request token positions
-> logical positions
-> KV slots / pages
-> actual K/V tensors
```

如果 mini-sglang 为了教学简化了 paged layout，也没关系。

你要理解的是生命周期：

allocate -> write -> read -> append -> release / cache

34.11 第六步：RadixAttention

RadixAttention 是 mini-sglang 最值得重点看的部分之一。

你要找这些操作：

1. match prefix。
2. longest prefix length。
3. insert new path。

4. split node。
5. evict node。
6. pin / unpin cached KV。
7. LRU 或访问时间更新。

建议手工构造三个请求：

R1: A B C D

R2: A B E F

R3: A B E G

然后观察 radix tree 如何变化：

```
root
-> [A B]
    -> [C D]
    -> [E]
        -> [F]
        -> [G]
```

如果源码里没有可视化工具，可以自己加 debug print。

重点不是打印漂亮，而是确认你理解 split 和 insert。

34.12 第七步：model runner

Model runner 把 scheduler 的计划转成模型 forward。

你要看：

1. Prefill 输入怎么构造。
2. Decode 输入怎么构造。
3. Position ids 怎么处理。
4. KV slot mapping 怎么传给 attention。
5. Logits 从哪里取。
6. Sampler 在哪里调用。
7. 输出 token 如何回写 request。

Prefill 和 decode 要分开看。

Prefill:

多个 prompt tokens -> 写一批 KV -> 产生 first token logits

Decode:

每个 request 一个新 token -> 读历史 KV -> 写新 token KV -> 产生 next logits

如果这个边界看清楚，后面读完整 SGLang 的 attention backend 会容易很多。

34.13 第八步：sampler

Sampler 负责 logits 到 token。

你要看：

1. Greedy 在哪里实现。
2. Temperature 在哪里处理。
3. Top-k / top-p 在哪里处理。
4. Stop token 在哪里判断。
5. 多请求不同 sampling params 怎么处理。
6. Structured output mask 是否进入 sampler。

普通 sampler 流程：

logits
-> penalty
-> temperature
-> top-k/top-p
-> sample

Structured sampler 流程:

logits
-> grammar mask
-> sampling filters
-> sample
-> update grammar state

如果 mini-sglang 没有完整 structured output, 也可以记录它缺失在哪里, 以及完整 SGLang 会如何补上。

34.14 Structured output 源码怎么读

如果 mini-sglang 包含 structured output, 建议这样读:

1. JSON schema 或 regex 在入口哪里解析。
2. Grammar object 在哪里创建。
3. Grammar state 是否按 request 保存。
4. 每步 valid token mask 在哪里计算。
5. Mask 如何作用到 logits。
6. Grammar state 如何 accept token。
7. 请求结束时如何清理 grammar state。

你要把代码映射到第 30 章的流程:

constraint spec
-> grammar backend
-> grammar state
-> valid token mask
-> sampler

如果源码里用的是简化 grammar, 也要抓住这个接口边界。

34.15 Tool use 源码怎么读

如果 mini-sglang 包含 tool use 或 tool parser, 重点看:

1. Tools schema 如何进入请求。
2. Chat template 如何渲染 tools。
3. 模型输出在哪里解析成 tool calls。
4. Streaming tool call arguments 如何拼接。
5. Tool result 如何作为 message 回灌。
6. Tool parser 和 structured output 是否分离。

注意工具执行通常不属于 runtime 核心。

源码里可能只做 parser, 不真的调用外部 API。

这是合理的, 因为真实工具执行应该在应用层处理权限、超时和安全。

34.16 Streaming 源码怎么读

Streaming 是很多教学项目容易低估的部分。

你要看:

1. Token id 如何 detokenize 成文本。
2. 增量文本如何避免重复。
3. Unicode 边界如何处理。
4. stop string 是否跨 token 检查。
5. 客户端断开如何 abort。
6. finish event 如何发送。

Streaming 和 scheduler 是相互影响的。

如果 scheduler 长时间不推进 decode, streaming 就会卡顿。

所以读 streaming 时要回到 TPOT 和 decode step latency。

34.17 推荐做的 8 个实验

只读源码不够, 建议做小实验。

实验一: 最小 prompt 生成。

确认请求生命周期。

实验二: 两个共享 system prompt 的请求。

观察 prefix cache 命中。

实验三: 三个分支 prompt。

观察 radix tree split。

实验四: 长 prompt + 短 prompt 混合。

观察 scheduler 如何安排 prefill/decode。

实验五: max_new_tokens 很小。

观察请求何时 finished 和释放资源。

实验六: streaming 请求。

观察 token 输出节奏。

实验七: 结构化输出。

观察 grammar mask 或等价约束在哪里生效。

实验八: 模拟 abort。

观察 KV cache 和 request state 是否清理。

每个实验都要记录:

1. 输入。
2. 关键日志。
3. 状态变化。
4. KV cache 变化。
5. 输出。
6. 你学到的模块边界。

34.18 源码笔记模板

建议每个模块用同一个模板记笔记。

模块名: Scheduler

位置: ...

解决的问题: 每个 engine step 选择 prefill/decode 请求

输入: waiting queue、running requests、token budget、KV state

输出: scheduled batch、KV allocation、metadata

核心状态: request.status、num_computed_tokens、KV refs

关键函数: ...

我画的流程图: ...

还没看懂的问题: ...

这种笔记比摘抄源码有用。

面试时你能用它讲清楚模块职责。

34.19 如何从 mini-sglang 过渡到真实 SGLang

读完 mini-sglang 后, 再读真实 SGLang。

过渡方式:

- 1. 用同一条请求生命周期找入口。
- 2. 找真实 SRT 的 request 数据结构。
- 3. 找 scheduler 主循环。
- 4. 找 memory pool 和 KV cache 管理。
- 5. 找 RadixAttention 实现。
- 6. 找 grammar backend 接入点。
- 7. 找 speculative decoding worker。
- 8. 找 OpenAI API 和 native API 映射。

你会发现真实项目复杂很多, 但主线不变。

复杂性通常来自:

- 1. 更多模型。
- 2. 更多后端。
- 3. 更多并行方式。
- 4. 更多配置。
- 5. 更多异常处理。
- 6. 更多性能优化。

不要被复杂性吓到, 先把它挂到 mini-sglang 的骨架上。

34.20 和本书前面章节的对应关系

可以把 mini-sglang 模块映射到本书章节。

源码模块	对应章节	重点问题
API / Engine	第 14、27 章	请求如何进入 runtime
Request state	第 12、20、27 章	状态机和生命周期
Scheduler	第 19、20、29 章	prefill/decode 调度
KV cache	第 9、18、21 章	KV 分配、追加、释放
RadixAttention	第 28 章	prefix sharing
Sampler	第 8、30 章	logits 到 token
Structured output	第 30 章	grammar mask

源码模块	对应章节	重点问题
Tool use	第 32 章	tool parser 和回灌
Speculative decoding	第 31 章	draft/verify/accept
Streaming	第 13、20 章	增量输出和 abort

这张表可以作为源码阅读索引。

34.21 面试中怎么讲 mini-sglang

面试官不一定关心你背了多少文件名。

更关心你是否能从源码中抽象系统设计。

你可以这样讲：

1. 我先跑通最小请求。
2. 然后沿请求生命周期看入口、request state、scheduler、KV cache、model runner、sampler、output。
3. 我重点看了 RadixAttention 如何做 longest prefix match、insert、split 和 eviction。
4. 我用几个共享前缀请求验证了 radix tree 如何变化。
5. 我对比了 mini-sglang 和完整 SGLang，理解教学版省略了哪些生产能力。
6. 我能把源码模块映射到 TTFT、TPOT、KV cache、prefix hit 和 structured output 指标。

这比说“我看过 SGLang 源码”更可信。

34.22 常见误区

误区一：一上来读完整 SGLang 全部源码。

容易迷路。先用 mini-sglang 建立骨架。

误区二：只看 model forward。

Serving engine 的难点在 scheduler、KV cache、streaming 和状态管理，不只是 forward。

误区三：只记类名。

类名会变，模块职责和请求生命周期更稳定。

误区四：不跑实验。

不跑共享前缀、abort、streaming 等实验，很难真正理解 runtime 行为。

误区五：把教学实现当生产实现。

mini-sglang 是学习路线，不是完整生产系统。要知道它省略了什么。

34.23 面试官会怎么问

问题一：你会怎么读 mini-sglang 源码？

回答要点：先跑通最小请求，再沿 request lifecycle 阅读入口、request state、scheduler、KV cache、RadixAttention、model runner、sampler、output，不从文件顺序乱读。

问题二：读 scheduler 时看什么？

回答要点：看 waiting/running 输入、token budget、sequence budget、KV budget、cache hit length、输出的 prefill/decode batch 和 execution metadata。

问题三：读 RadixAttention 时看什么？

回答要点：看 longest prefix match、node split、insert、eviction、KV ref、pin/unpin，以及它如何影响 prefill suffix length。

问题四：mini-sglang 和完整 SGLang 的关系？

回答要点：mini-sglang 是教学项目，用更小代码解释核心 runtime；完整 SGLang 有更多模型、后端、并行、grammar、speculative decoding、tool parser 和生产工程细节。

问题五：怎么证明你真的看懂了？

回答要点：能画请求生命周期图，能解释状态变化和 KV cache 生命周期，能用共享前缀实验展示 radix tree 变化，能把源码模块和 TTFT/TPOT/cache hit 指标对应起来。

34.24 标准回答模板

如果面试官问 “你如何学习 mini-sglang 源码”，可以这样回答：

我不会从文件列表顺序读，而是先跑通一个最小 generate 请求，然后沿请求生命周期读源码。第一步看外部请求在哪里被解析、token

我会用几个实验验证理解，比如构造三个共享前缀请求观察 radix tree split，构造长短 prompt 混合观察 scheduler，构造 abort 请求。sglang 的目标不是背类名，而是建立 SGLang Runtime 的骨架，再用这个骨架去读完整 SGLang 源码。

34.25 源码阅读覆盖率、实验门禁和可运行 demo

为了避免 “我看过源码” 变成空话，可以把源码阅读笔记当成一个审计对象。设模块集合为：

$\mathcal{M} = \{m_1, m_2, \dots, m_N\}$

必读模块集合为 M_{req} ，已完成笔记的模块集合为 M_{seen} ，模块覆盖率是：

$C_{\{\text{module}\}} = \frac{|M_{\{\text{seen}\}} \cap M_{\{\text{req}\}}|}{\max(1, |M_{\{\text{req}\}}|)}$

资源生命周期也要覆盖。设必查资源集合为 R_{req} ，已在笔记中解释清楚的资源集合为 R_{seen} ：

$C_{\{\text{resource}\}} = \frac{|R_{\{\text{seen}\}} \cap R_{\{\text{req}\}}|}{\max(1, |R_{\{\text{req}\}}|)}$

只读不跑实验不够。设必做实验集合为 E_{req} ，已完成实验集合为 E_{done} ：

$C_{\{\text{experiment}\}} = \frac{|E_{\{\text{done}\}} \cap E_{\{\text{req}\}}|}{\max(1, |E_{\{\text{req}\}}|)}$

源码学习门禁可以写成：

$G_{\{\text{source}\}} = G_{\{\text{module}\}} G_{\{\text{order}\}} G_{\{\text{resource}\}} G_{\{\text{experiment}\}} G_{\{\text{signal}\}}$

这些因子分别要求模块地图完整、请求生命周期顺序可见、资源 allocate / read / write / release / cache 可复盘、实验覆盖核心路径、观察信号足够、笔记能回答模块输入输出和状态变化。

下面是一个 0 依赖源码路径审计 demo。它不执行真实 mini-sglang，而是检查你的阅读笔记是否覆盖了必读模块、资源生命周期和实验路径。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class ModuleNote:
    name: str
    lifecycle_stage: str
    inputs: tuple
    outputs: tuple
    states: tuple
    resources: tuple
```

```
@dataclass
```

```
class ExperimentTrace:
    name: str
```

```
modules_touched: tuple
observed: tuple
```

```
class ToySourcePathAuditor:
```

```
    REQUIRED_MODULES = {
        "api_server",
        "tokenizer",
        "request_state",
        "scheduler",
        "kv_cache",
        "radix_cache",
        "engine",
        "attention_backend",
        "sampler",
        "detokenizer",
        "streaming",
        "cleanup",
    }
```

```
    REQUIRED_RESOURCES = {
        "token_budget",
        "sequence_budget",
        "kv_allocate",
        "kv_write",
        "kv_read",
        "kv_release",
        "prefix_cache",
    }
```

```
    REQUIRED_EXPERIMENTS = {
        "minimal_generate",
        "shared_prefix",
        "radix_split",
        "mixed_prefill_decode",
        "max_tokens_finish",
        "streaming",
        "structured_output",
        "abort_cleanup",
    }
```

```
    EXPECTED_ORDER = ["api_server", "tokenizer", "request_state", "scheduler", "kv_cache", "engine", "sampler", "detokenizer"]
```

```
    def __init__(self, modules, experiments):
        self.modules = modules
        self.experiments = experiments
```

```
    def _order_score(self):
        positions = {note.name: index for index, note in enumerate(self.modules)}
        seen = [positions[name] for name in self.EXPECTED_ORDER if name in positions]
        return seen == sorted(seen) and len(seen) == len(self.EXPECTED_ORDER)
```

```
    def run(self):
        module_names = {note.name for note in self.modules}
        resource_names = {resource for note in self.modules for resource in note.resources}
        experiment_names = {exp.name for exp in self.experiments}
        touched_modules = {module for exp in self.experiments for module in exp.modules_touched}
        observed_signals = {signal for exp in self.experiments for signal in exp.observed}
```

```

summary = {
    "module_coverage": round(len(module_names & self.REQUIRED_MODULES) / len(self.REQUIRED_MODULES), 3),
    "resource_coverage": round(len(resource_names & self.REQUIRED_RESOURCES) / len(self.REQUIRED_RESOURCES),
    "experiment_coverage": round(len(experiment_names & self.REQUIRED_EXPERIMENTS) / len(self.REQUIRED_EXPERIMENTS),
    "lifecycle_order_ok": self._order_score(),
    "modules_touched_by_experiments": round(len(touched_modules & self.REQUIRED_MODULES) / len(self.REQUIRED_MODULES),
    "observed_signal_count": len(observed_signals),
    "missing_modules": sorted(self.REQUIRED_MODULES - module_names),
    "missing_resources": sorted(self.REQUIRED_RESOURCES - resource_names),
    "missing_experiments": sorted(self.REQUIRED_EXPERIMENTS - experiment_names),
}
gates = {
    "module_map_complete": summary["module_coverage"] == 1.0,
    "lifecycle_order_visible": summary["lifecycle_order_ok"],
    "resource_lifecycle_visible": summary["resource_coverage"] == 1.0,
    "experiments_cover_core_paths": summary["experiment_coverage"] == 1.0,
    "experiments_touch_runtime": summary["modules_touched_by_experiments"] >= 0.8,
    "signals_are_observable": summary["observed_signal_count"] >= 10,
    "source_path_notes_ready": not summary["missing_modules"] and not summary["missing_resources"],
}
gates["source_path_gate"] = all(gates.values())
return summary, gates

```

```

modules = [
    ModuleNote("api_server", "entry", ("json_request",), ("raw_prompt",), ("request_id",), ()),
    ModuleNote("tokenizer", "entry", ("raw_prompt",), ("input_ids",), ("prompt_len",), ()),
    ModuleNote("request_state", "runtime", ("input_ids",), ("waiting_request",), ("WAITING", "PREFILLING", "DECODING",)),
    ModuleNote("scheduler", "runtime", ("waiting_queue", "running_set"), ("schedule_plan",), ("phase",), ("token_budget",)),
    ModuleNote("kv_cache", "memory", ("schedule_plan",), ("slot_mapping",), ("kv_refs",), ("kv_allocate", "kv_write",)),
    ModuleNote("radix_cache", "memory", ("input_ids",), ("prefix_hit",), ("ref_count",), ("prefix_cache",)),
    ModuleNote("engine", "compute", ("schedule_plan", "slot_mapping"), ("logits",), ("prefill", "decode",), ()),
    ModuleNote("attention_backend", "compute", ("qkv", "slot_mapping"), ("context",), ("metadata",), ()),
    ModuleNote("sampler", "decode", ("logits",), ("next_token",), ("sampling_params",), ()),
    ModuleNote("detokenizer", "output", ("next_token",), ("text_delta",), ("offset",), ()),
    ModuleNote("streaming", "output", ("text_delta",), ("event",), ("finish_reason",), ()),
    ModuleNote("cleanup", "output", ("finish_reason",), ("released",), ("FINISHED", "ABORTED",), ("kv_release",)),
]

```

```

experiments = [
    ExperimentTrace("minimal_generate", ("api_server", "tokenizer", "scheduler", "engine", "sampler", "detokenizer"),),
    ExperimentTrace("shared_prefix", ("radix_cache", "kv_cache", "scheduler"), ("prefix_hit", "saved_prefill")),
    ExperimentTrace("radix_split", ("radix_cache",), ("node_split", "ref_count")),
    ExperimentTrace("mixed_prefill_decode", ("scheduler", "engine", "kv_cache"), ("prefill_rows", "decode_rows")),
    ExperimentTrace("max_tokens_finish", ("request_state", "cleanup"), ("FINISHED", "kv_release")),
    ExperimentTrace("streaming", ("detokenizer", "streaming", "scheduler"), ("text_delta", "stream_event")),
    ExperimentTrace("structured_output", ("sampler", "request_state"), ("grammar_mask", "valid_token")),
    ExperimentTrace("abort_cleanup", ("request_state", "cleanup", "kv_cache"), ("ABORTED", "kv_release")),
]

```

```

summary, gates = ToySourcePathAuditor(modules, experiments).run()
print("source_path_summary=", summary)
print("source_path_gates=", gates)

```

运行后可以看到类似输出：

节点 KV 迁移协议、connector API、调度器源码、并行策略搜索或生产 benchmark 排名。后续章节会继续拆 PD 分离系统架构、KV cache 迁移共享和 chunked prefill 与 disaggregated prefill 的取舍。

本讲新增 demo 是教学版资源画像审计器：用 token 数、prefix hit、简化 compute / KV 读写量、TTFT / TPOT 估算和干扰门禁，帮助面试时把“prefill 偏计算、decode 偏带宽和调度”讲成可验收的指标链，而不是只背结论。

35.1 本章目标

读完本章，你应该能讲清：

1. Prefill 和 Decode 在输入、输出和执行频率上的差异。
2. 为什么 Prefill 更偏大计算，Decode 更偏 KV cache 读取和调度。
3. TTFT、TPOT 分别由哪些阶段主导。
4. 为什么长 Prefill 会干扰 Decode streaming。
5. 为什么 Decode-first 会导致 Prefill 饥饿。
6. PD 分离为什么要先从资源画像差异说起。
7. 面试中如何解释 Prefill/Decode disaggregation 的动机。

35.2 再看一次生成生命周期

一个 LLM 请求不是一次 forward 完成。

完整生命周期是：

```
prompt tokens
-> prefill
-> first token
-> decode token 2
-> decode token 3
-> decode token 4
-> ...
-> finish
```

Prefill 处理输入 prompt。

Decode 处理输出生成。

如果用户输入 4000 tokens，输出 800 tokens，那么：

prefill：处理 4000 个输入 tokens，通常一次或分 chunk 完成

decode：生成 800 个输出 tokens，通常需要 800 轮左右

这两段的形态完全不同。

Prefill 是“宽”的：一次处理很多 token。

Decode 是“长”的：每次处理少量 token，但重复很多轮。

35.3 Prefill 的执行画像

Prefill 阶段输入是整个 prompt 或 prompt chunk。

例如：

input_ids: [t1, t2, ..., t4096]

模型要计算这些 token 的 hidden states，并为每层 attention 写入 KV cache。

Prefill 的特点：

1. 单次 token 数多。
2. 矩阵乘法规模较大。

- 3. GPU 算力利用率通常更容易做高。
- 4. 计算量随 prompt 长度上升。
- 5. 会产生大量 KV cache 写入。
- 6. 对 TTFT 影响直接。

从资源倾向看，Prefill 常被认为更偏 compute-bound。
这不是说它没有显存和带宽压力，而是说相对 Decode，它更容易把 GPU 计算单元喂饱。

35.4 Decode 的执行画像

Decode 阶段每轮通常只输入上一步生成的 token。
例如 running batch 有 128 个请求：
decode input: 每个请求 1 个 token, 总计 128 tokens
模型要为每个请求读取历史 KV cache, 计算下一个 token 的 logits, 再采样。
Decode 的特点：

- 1. 每个请求每轮 token 数很少。
- 2. 需要重复很多轮。
- 3. 每轮都要读取历史 KV cache。
- 4. 上下文越长，KV 读取越重。
- 5. 对调度开销敏感。
- 6. 对 streaming 平滑度影响直接。
- 7. 主要影响 TPOT。

Decode 常被认为更偏 memory-bound 或 bandwidth-sensitive。
因为它不断读取历史 KV，单轮计算量相对小，GPU 计算单元不一定容易吃满。

35.5 一张表总结差异

维度	Prefill	Decode
处理内容	输入 prompt	输出 token
每请求每轮 token 数	多个 token	通常 1 个 token
执行频率	每个请求通常一次或少数 chunks	每个输出 token 一轮
主要指标	TTFT	TPOT、ITL、输出吞吐
资源倾向	更偏 compute-bound	更偏 memory-bandwidth / scheduling-sensitive
KV cache 行为	大量写入 prompt KV	读取历史 KV，并追加新 KV
对 batch 的需求	大 token batch 提高算力利用	大 sequence batch 提高 decode 吞吐
风险	长 prompt 阻塞 decode	长输出占用 KV 和调度轮次
常见优化	prefix cache、chunked prefill、PD prefill pool	continuous batching、KV layout、speculative decoding

这张表是 PD 分离的基础。

35.6 TTFT 为什么主要看 Prefill

TTFT 是 time to first token。
可以拆成：

$TTFT = \text{queue time} + \text{preprocess time} + \text{prefill time} + \text{first token sampling} + \text{network flush}$

其中 prefill time 经常是大头，尤其是长 prompt 场景。

影响 prefill time 的因素：

1. 输入 token 数。
2. 是否命中 prefix cache。
3. 是否使用 RadixAttention。
4. prefill batch 大小。
5. 是否 chunked prefill。
6. GPU compute 能力。
7. attention backend。
8. 是否被 decode workload 干扰。

如果一个请求排队很久，TTFT 也会高。

如果 prompt 很长但 prefix cache 命中大段，TTFT 可能下降。

所以优化 TTFT 时，要看 queue 和 prefill，而不是只看 decode kernel。

35.7 TPOT 为什么主要看 Decode

TPOT 是 time per output token。

可以理解为：

$TPOT = \text{decode 阶段总耗时} / \text{输出 token 数}$

影响 TPOT 的因素：

1. Decode step latency。
2. Running batch size。
3. KV cache 读取效率。
4. 上下文长度。
5. Attention kernel。
6. Sampling 开销。
7. Structured output grammar mask。
8. Streaming 输出阻塞。
9. Speculative decoding 接受率。

TPOT 直接决定流式输出是否顺滑。

如果 decode step 被长 prefill 打断，用户会看到输出卡顿。

这就是 Prefill 和 Decode 混跑时最常见的冲突。

35.8 长 Prefill 如何干扰 Decode

假设系统中有 100 个请求正在流式 decode。

每轮 decode 都应该稳定输出 token。

突然来了一个 64000-token prompt。

如果 scheduler 把这个长 prefill 一次性放进 GPU 执行，那么 decode 请求可能长时间得不到调度。

用户看到的现象是：

前面输出很顺滑
突然卡住几秒
然后继续输出

这不是网络问题，而是 decode 被长 prefill 阻塞。

缓解办法包括：

1. Chunked prefill。
2. Decode-first scheduling。
3. 限制单 step prefill tokens。
4. 隔离长 prompt。
5. PD 分离，把 prefill 放到独立资源池。

PD 分离是更架构化的解决方式。

35.9 Decode-first 为什么也有问题

为了保护 streaming，很多 scheduler 会偏向 decode。

策略大概是：

先调度 running decode

再用剩余 token budget 调度 waiting prefill

这能让已有请求输出更平滑。

但如果系统 decode 压力长期很高，新请求可能一直等不到 prefill。

用户看到的现象是：

请求发出去后很久没有首 token

但一旦开始输出，后续速度还可以

这就是 prefill 饥饿。

所以 Prefill/Decode 混合调度天然矛盾的：

偏 prefill -> TTFT 好，TPOT 可能差

偏 decode -> TPOT 好，TTFT 可能差

PD 分离试图通过资源隔离减少这个矛盾。

35.10 GPU 利用率视角

Prefill 和 Decode 对 GPU 利用率的表现不同。

Prefill：

1. 大矩阵乘法更容易吃满 Tensor Core。
2. 单次 batch token 多时吞吐高。
3. 长 prompt 可能让 GPU 忙于计算。

Decode：

1. 每请求每轮 token 少。
2. 小 batch decode 可能 GPU 利用率低。
3. 大 running batch 能提高利用率，但可能增加延迟。
4. KV cache 读取和调度开销更明显。

所以一个 GPU 同时跑 prefill 和 decode 时，资源使用会不断切换形态。

这对 kernel、batch shape、CUDA graph、attention backend 都不友好。

35.11 KV cache 视角

Prefill 和 Decode 都和 KV cache 有关，但行为不同。

Prefill：

写入 prompt 的 KV cache

Decode:

读取已有 KV cache

追加新 token KV cache

Prefill 的 KV 写入可能一次很多。

Decode 的 KV 读取频繁且持续。

长上下文 decode 时，每轮都要关注大量历史 KV。

这导致 Decode 对 KV layout、memory bandwidth 和 cache locality 更敏感。

PD 分离后，还会引入新问题：Prefill 生成的 KV cache 要如何交给 Decode worker？

这就是后面章节要讲的 KV cache 迁移、共享和路由。

35.12 Batch shape 视角

Prefill batch 和 Decode batch 的 shape 也不同。

Prefill batch 更像：

total prefill tokens = sum(prompt_or_chunk_lengths)

Decode batch 更像：

decode tokens = number_of_running_sequences

举例：

Prefill batch:

request A: 4000 tokens

request B: 2000 tokens

request C: 1000 tokens

total = 7000 tokens

Decode batch:

128 running requests

total = 128 tokens this step

这两个 batch 对 kernel 和调度的要求完全不同。

混在一起会造成 batch shape 不稳定。

35.13 Latency SLO 视角

在线系统通常同时有 TTFT SLO 和 TPOT SLO。

例如：

p95 TTFT < 2s

p95 TPOT < 80ms/token

Prefill 主要威胁 TTFT。

Decode 主要威胁 TPOT。

混部时，一个策略很难同时让两者最优。

如果为了降低 TTFT，积极接纳长 prefill，TPOT 可能抖。

如果为了稳定 TPOT，decode-first，TTFT 可能升。

PD 分离的目标不是魔法消除成本，而是让两个 SLO 可以分别用不同资源池和调度策略治理。

35.14 成本视角

成本不是只看 GPU 数。

要看每类 GPU 在做什么。

如果 decode workload 在一张适合大矩阵计算的 GPU 上小 batch 跑，GPU 可能利用率不高。

如果 prefill workload 被 decode-heavy 调度打断，prefill 吞吐也上不去。

资源画像不同意味着可以考虑：

1. Prefill pool 使用更适合大计算的配置。
2. Decode pool 使用更适合低延迟和 KV 读取的配置。
3. 不同 pool 设置不同 batch/token budget。
4. 不同 pool 独立扩缩容。
5. 根据流量输入输出比例调整 P/D 比例。

这就是 PD 分离走向资源池化的原因。

35.15 输入输出比例决定压力

不同业务的 prefill/decode 压力不同。

短输入长输出：

prompt 100 tokens, output 2000 tokens

Decode 压力大。

长输入短输出：

prompt 16000 tokens, output 100 tokens

Prefill 压力大。

长输入长输出：

prompt 32000 tokens, output 4000 tokens

Prefill 和 Decode 压力都大。

因此 PD 分离资源配比不能固定照抄。

要看真实流量的输入输出 token 分布。

35.16 三类典型 workload

RAG 问答：

长文档上下文 + 相对短答案

更偏 prefill 压力，prefix cache 和 prefill pool 重要。

代码生成：

中等 prompt + 长输出

更偏 decode 压力，TPOT、speculative decoding 和 decode pool 重要。

Agent：

多轮短生成 + 工具等待 + 上下文逐步增长

既要看多轮 TTFT，也要看工具延迟和 cache locality。

不同 workload 的 PD 分离收益不同。

35.17 为什么 chunked prefill 还不够

Chunked prefill 可以缓解长 prefill 阻塞 decode。

它把长 prompt 拆成多个 chunk，让 decode 在 chunk 之间穿插执行。

但它仍然是在同一资源池里调度：

同一组 GPU 同时承担 prefill 和 decode

这意味着：

1. P/D 仍然争抢同一 GPU。
2. Scheduler 仍然要做复杂折中。
3. Batch shape 仍然混合。
4. 资源扩缩容不能按 P/D 独立做。
5. 长尾流量仍可能互相影响。

PD 分离是更进一步：

不同资源池分别处理 prefill 和 decode

当然，它也引入 KV 迁移和系统复杂度。

35.18 PD 分离的基本动机

现在可以概括 PD 分离的动机。

因为 Prefill 和 Decode：

1. 资源倾向不同。
2. 延迟指标不同。
3. batch shape 不同。
4. 调度目标不同。
5. KV cache 行为不同。
6. 扩缩容压力不同。

所以可以考虑把它们拆开：

Prefill workers：

负责处理 prompt，生成 prompt KV cache 和 first token

Decode workers：

负责接收 KV cache，持续生成后续 tokens

这样做的目标是：

1. Prefill 不阻塞 Decode。
2. Decode 不让 Prefill 饥饿。
3. P/D 资源可以独立扩缩容。
4. P/D scheduler 可以各自优化。
5. 长尾影响更容易隔离。

35.19 PD 分离不是免费午餐

PD 分离也有代价。

最大问题是 KV cache 如何从 Prefill worker 到 Decode worker。

需要考虑：

1. KV cache 传输带宽。
2. 网络延迟。
3. GPU-GPU、GPU-CPU、跨节点通信。

4. KV cache layout 兼容。
5. 请求路由和状态同步。
6. 失败恢复。
7. 多租户隔离。
8. 调试复杂度。

如果 KV 迁移成本太高，PD 分离收益可能被抵消。

所以 PD 分离适合在 P/D 干扰明显、资源池化收益明显、网络和工程能力能支撑时使用。

35.20 什么时候不需要 PD 分离

并不是所有系统都需要 PD 分离。

不一定需要的情况：

1. 流量规模小。
2. 请求长度短。
3. P/D 干扰不明显。
4. 单机 continuous batching 已经满足 SLO。
5. 团队还没有足够监控和调度能力。
6. KV 迁移成本高于收益。
7. 系统复杂度优先级低。

过早引入 PD 分离可能让系统更难稳定。

正确顺序通常是：

先做好单机 serving 和指标

- > 观察 P/D 干扰
- > 优化 scheduler 和 chunked prefill
- > 再评估 PD 分离

35.21 观测指标

要判断是否需要 PD 分离，先看指标。

Prefill 指标：

1. prefill latency。
2. input tokens/s。
3. scheduled prefill tokens。
4. prefill queue time。
5. prefix cache hit length。
6. chunked prefill 次数。

Decode 指标：

1. decode step latency。
2. TPOT / ITL。
3. output tokens/s。
4. running sequences。
5. KV cache read pressure。
6. speculative accepted tokens。

干扰指标：

1. 长 prefill 到来时 TPOT 是否抖动。
2. decode-heavy 时 waiting queue 是否增长。
3. p99 TTFT 和 p99 TPOT 是否交替恶化。
4. GPU utilization 是否随 batch shape 大幅波动。
5. KV cache allocation failure 是否集中发生。

没有指标就谈 PD 分离，很容易拍脑袋。

35.22 面试官会怎么问

问题一：为什么 Prefill 和 Decode 的资源画像不同？

回答要点：Prefill 一次处理大量 prompt tokens，更偏大矩阵计算和 KV 写入，影响 TTFT；Decode 每轮每请求通常一个 token，反复读取历史 KV，调度频繁，更影响 TPOT 和 streaming。

问题二：为什么长 Prefill 会影响 Decode？

回答要点：如果在同一 GPU worker 上一次性执行长 prompt prefill，会占用计算资源，running decode 请求得不到及时调度，导致 TPOT/ITL 抖动，用户看到 streaming 卡顿。

问题三：Decode-first 有什么问题？

回答要点：它保护 running 请求输出平滑，但在 decode 压力高时 waiting prefill 可能长期得不到调度，导致新请求 TTFT 变差。

问题四：PD 分离为什么能缓解这个矛盾？

回答要点：把 prefill 和 decode 放到不同资源池，各自用适合的 batch、调度和扩缩容策略，减少 P/D 互相干扰。

问题五：PD 分离的主要代价是什么？

回答要点：KV cache 需要从 prefill worker 迁移或共享给 decode worker，会带来网络带宽、延迟、layout、路由、状态同步和故障恢复复杂度。

35.23 标准回答模板

如果面试官问“为什么要做 PD 分离”，可以这样回答：

PD 分离的根本原因是 Prefill 和 Decode 的资源画像不同。Prefill 处理输入 prompt，通常一次处理很多 tokens，矩阵乘法规模较大 bound，并且会大量写入 KV cache，主要影响 TTFT。Decode 则是逐 token 生成，每个请求每轮通常只处理一个 token，要频繁读取历史。

如果二者在同一个 GPU worker 上混跑，长 prefill 可能阻塞 running decode，导致输出卡顿；反过来，如果 scheduler 长期 decode first，新请求 prefill 会饥饿，TTFT 变差。Chunked prefill 能缓解，但仍然是同一资源池里折中。

PD 分离把 prefill 和 decode 拆到不同资源池，让 prefill pool 专注处理输入和生成 prompt KV，让 decode pool 专注持续生成输出。

35.24 资源画像公式、干扰门禁和可运行 demo

面试里讲 Prefill / Decode 资源画像，最好不要只说“一个 compute-bound、一个 memory-bound”。更稳的讲法是先定义每个请求的 token 画像，再把 compute、KV 读写和延迟指标拆开。

设第 i 个请求的 prompt token 数为 P_i ，输出 token 数为 O_i ，prefix cache 命中 token 数为 H_i 。本章用一个简化模型：

$$P_i^{\mathrm{run}} = \max(0, P_i - H_i)$$

P_i^{run} 是真正需要执行 prefill 的 token 数。若共享前缀或 prefix cache 命中较多，长 prompt 不一定等于长 prefill。

Prefill 的简化 compute 画像可以写成：

$$C_{\mathrm{prefill}, i} = \alpha P_i^{\mathrm{run}} + \beta (P_i^{\mathrm{run}})^2$$

这里线性项代表 MLP、projection 等随 token 数增长的计算，二次项代表 full prefill attention 中 prompt 内部两两注意力带来的额外成本。真实框架会受 FlashAttention、chunk size、padding、batch shape 和硬件影响，本式只用于解释趋势。

Prefill 的 KV 写入量可以估成：

$$W_{\mathrm{prefill}, i} = P_i^{\mathrm{run}} m_{\mathrm{kv}}$$

m_{kv} 是每个 token 的 KV cache 占用。它由层数、KV heads、head dim 和 dtype bytes 决定。

Decode 每步只追加一个 token，但会反复读取历史上下文。若输出逐步增长，平均读取上下文长度可以粗略写成：

$$\bar{L}_i = P_i + \frac{0_{i-1}}{2}$$

于是 decode 的 KV 读取量近似为：

$$R_{\mathrm{decode}, i} = 0_i \bar{L}_i m_{\mathrm{kv}}$$

这解释了为什么短 prompt 长输出、长上下文续写、多轮 agent 的 Decode 会越来越偏 KV 读取和带宽敏感。

延迟上，TTFT 更靠近 queue 和 prefill：

$$T_{\mathrm{ttft}, i} \approx T_{\mathrm{queue}, i} + \frac{P_i^{\mathrm{run}}}{X_{\mathrm{prefill}}} + T_{\mathrm{first}, i}$$

TPOT 更靠近 decode step：

$$T_{\mathrm{tpot}, i} \approx \frac{T_{\mathrm{decode}, i}}{\max(1, 0_i)}$$

如果把长 prefill 和 running decode 混跑，最大干扰可以用一个很粗的 stall 估计：

$$S_{\mathrm{co}, i} = \frac{P_i^{\mathrm{run}}}{X_{\mathrm{prefill}}}$$

chunked prefill 把单次 stall 上限压到：

$$S_{\mathrm{chunk}, i} = \frac{\min(P_i^{\mathrm{run}}, C_{\mathrm{chunk}})}{X_{\mathrm{prefill}}}$$

PD 分离进一步把 P/D 放进不同资源池，但要支付 KV transfer：

$$K_{\mathrm{transfer}} = \sum_i P_i^{\mathrm{run}} m_{\mathrm{kv}}$$

所以下面这个 demo 不模拟真实 GPU，而是做一个资源画像审计：看每类请求是否能解释 prefill compute、decode KV read、TTFT / TPOT、长 prefill 干扰、decode-first 饥饿和 PD 分离的 KV 传输代价。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class RequestProfile:
```

```
    name: str
```

```
    prompt_tokens: int
```

```
    output_tokens: int
```

```
    prefix_hit_tokens: int = 0
```

```
class ToyPrefillDecodeProfiler:
```

```
    def __init__(
```

```
        self,
```

```
        requests,
```

```
        kv_mib_per_token=0.015625,
```

```
        prefill_tokens_per_ms=900.0,
```

```
        decode_tokens_per_ms=90.0,
```

```
    ):
```

```
        self.requests = requests
```

```
        self.kv_mib_per_token = kv_mib_per_token
```

```
        self.prefill_tokens_per_ms = prefill_tokens_per_ms
```

```
        self.decode_tokens_per_ms = decode_tokens_per_ms
```

```
    def _profile(self, request):
```

```
        run_prefill = max(0, request.prompt_tokens - request.prefix_hit_tokens)
```

```
        prefill_compute_units = run_prefill * 48 + (run_prefill * run_prefill) / 2048
```

```
        prefill_kv_write_mib = run_prefill * self.kv_mib_per_token
```

```

decode_steps = request.output_tokens
avg_ctx = request.prompt_tokens + max(0, request.output_tokens - 1) / 2
decode_kv_read_mib = decode_steps * avg_ctx * self.kv_mib_per_token
decode_compute_units = decode_steps * 12
ttft_ms = round(run_prefill / self.prefill_tokens_per_ms + 8.0, 3)
tpot_ms = round((1.0 / self.decode_tokens_per_ms) * (1 + avg_ctx / 8192), 4)
return {
    "name": request.name,
    "run_prefill_tokens": run_prefill,
    "decode_steps": decode_steps,
    "prefill_compute_units": round(prefill_compute_units, 1),
    "decode_compute_units": round(decode_compute_units, 1),
    "prefill_kv_write_mib": round(prefill_kv_write_mib, 1),
    "decode_kv_read_mib": round(decode_kv_read_mib, 1),
    "prefill_compute_per_mib": round(
        prefill_compute_units / max(1.0, prefill_kv_write_mib), 2
    ),
    "decode_compute_per_mib": round(
        decode_compute_units / max(1.0, decode_kv_read_mib), 4
    ),
    "ttft_ms": ttft_ms,
    "tpot_ms": tpot_ms,
}

def run(self):
    rows = [self._profile(req) for req in self.requests]
    total_prefill = sum(row["run_prefill_tokens"] for row in rows)
    total_decode = sum(row["decode_steps"] for row in rows)
    total_prefill_compute = sum(row["prefill_compute_units"] for row in rows)
    total_decode_kv_read = sum(row["decode_kv_read_mib"] for row in rows)
    long_prefill = max(row["run_prefill_tokens"] for row in rows)
    running_decode_rows = 96
    colocated_decode_stall_ms = round(long_prefill / self.prefill_tokens_per_ms, 3)
    chunked_prefill_stall_ms = round(
        min(long_prefill, 2048) / self.prefill_tokens_per_ms, 3
    )
    decode_first_wait_ms = round((running_decode_rows * 18) / self.decode_tokens_per_ms, 3)
    kv_transfer_mib = round(total_prefill * self.kv_mib_per_token, 1)
    pd_benefit_ms = round(colocated_decode_stall_ms - chunked_prefill_stall_ms, 3)
    summary = {
        "requests": len(rows),
        "total_run_prefill_tokens": total_prefill,
        "total_decode_steps": total_decode,
        "prefill_decode_token_ratio": round(total_prefill / max(1, total_decode), 3),
        "prefill_compute_units": round(total_prefill_compute, 1),
        "decode_kv_read_mib": round(total_decode_kv_read, 1),
        "max_prefill_tpot_stall_ms": colocated_decode_stall_ms,
        "chunked_prefill_stall_ms": chunked_prefill_stall_ms,
        "decode_first_prefill_wait_ms": decode_first_wait_ms,
        "kv_transfer_mib_for_pd": kv_transfer_mib,
        "pd_interference_saving_ms": pd_benefit_ms,
    }
    gates = {
        "prefill_compute_heavy": sum(
            row["prefill_compute_per_mib"] for row in rows

```

```

) / len(rows) > 1000,
"decode_kv_read_heavy": total_decode_kv_read
> sum(row["prefill_kv_write_mib"] for row in rows) * 20,
"ttft_tpot_separated": all(
    "ttft_ms" in row and "tpot_ms" in row for row in rows
),
"long_prefill_stalls_decode": colocated_decode_stall_ms > 10.0,
"decode_first_can_starve_prefill": decode_first_wait_ms
> chunked_prefill_stall_ms,
"pd_has_kv_transfer_cost": kv_transfer_mib > 0,
"pd_motivation_visible": pd_benefit_ms > 0,
"metrics_ready": summary["prefill_decode_token_ratio"] > 1.0,
}
gates["prefill_decode_profile_gate"] = all(gates.values())
return rows, summary, gates

```

```

requests = [
    RequestProfile("rag_long_prompt", prompt_tokens=16000, output_tokens=120, prefix_hit_tokens=4000),
    RequestProfile("short_chat", prompt_tokens=512, output_tokens=256),
    RequestProfile("code_long_output", prompt_tokens=800, output_tokens=2400),
    RequestProfile("agent_context_growth", prompt_tokens=3200, output_tokens=420, prefix_hit_tokens=1200),
]

```

```

rows, summary, gates = ToyPrefillDecodeProfiler(requests).run()
print("prefill_decode_rows=", rows)
print("prefill_decode_summary=", summary)
print("prefill_decode_gates=", gates)

```

一次运行的核心输出类似：

```

prefill_decode_summary= {'requests': 4, 'total_run_prefill_tokens': 15312, 'total_decode_steps': 3196, 'prefill_decode_
prefill_decode_gates= {'prefill_compute_heavy': True, 'decode_kv_read_heavy': True, 'ttft_tpot_separated': True, 'long_

```

这个输出里有三层含义：

1. rag_long_prompt 和 agent_context_growth 说明长 prompt 是否可怕，要看 run_prefill_tokens，而不是只看原始 prompt。
2. code_long_output 说明短 prompt 长输出可能让 decode KV 读取成为主压力，TPOT 不能只看输入长度。
3. max_prefill_tpot_stall_ms、chunked_prefill_stall_ms 和 decode_first_prefill_wait_ms 同时为真，说明 chunked prefill 能缓解长 prefill 卡 decode，但 decode-first 又可能让 waiting prefill 饥饿；PD 分离有动机，但 kv_transfer_mib_for_pd 说明它有明确传输代价。

所以本讲的最终门禁是 prefill_decode_profile_gate：能把 token 分布、prefix hit、prefill compute、decode KV read、TTFT、TPOT、长 prefill 干扰、decode-first 饥饿和 PD KV transfer 都说清，才算真正理解 Prefill 和 Decode 的资源画像。

35.25 小练习

1. 给定 prompt 8000 tokens、输出 200 tokens，判断主要压力偏 prefill 还是 decode。
2. 给定 prompt 200 tokens、输出 4000 tokens，判断主要压力偏 prefill 还是 decode。
3. 画出长 prefill 阻塞 decode 的时间线。
4. 解释 decode-first 为什么会造成 prefill 饥饿。
5. 设计一个 dashboard，用于判断是否需要 PD 分离。
6. 列出 PD 分离后 KV cache 迁移需要解决的 5 个问题。
7. 对比 chunked prefill 和 PD 分离分别缓解什么。

35.26 本章总结

Prefill 和 Decode 的差异，是理解高级 serving 架构的起点。

Prefill 更像大块输入计算，影响 TTFT；Decode 更像长时间逐 token 服务，影响 TPOT 和 streaming。二者混合调度会天然产生冲突：prefill 过多会卡 decode，decode-first 又会让 prefill 饥饿。

PD 分离的动机就是把这两类资源画像不同的 workload 拆开，用不同资源池、不同调度策略和不同扩缩容方式分别治理。

下一章会正式讲 PD 分离解决什么问题，并进一步展开它的收益、代价、适用场景和反模式。

第 36 章 PD 分离解决什么问题

上一章讲了 Prefill 和 Decode 的资源画像为什么不同：Prefill 更偏大块输入计算和 KV 写入，主要影响 TTFT；Decode 更偏逐 token 生成、KV 读取和重复调度，主要影响 TPOT。

本章正式讨论 PD 分离解决什么问题。

PD 分离也叫 Prefill-Decode Disaggregation，或者 Disaggregated Prefill。它的核心思想是把 Prefill 和 Decode 从同一个统一执行引擎里拆出来，让不同资源池分别处理不同阶段。

一句话概括：

PD 分离解决的是统一引擎中 Prefill 和 Decode 互相干扰、资源画像不匹配、扩缩容耦合和尾延迟难治理的问题；代价是必须解决 KV cache 迁移、路由、状态同步和系统复杂度。

36.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 `WRITING_PLAN.md` 对公开资料做校准：参考 DistServe 论文对 prefill / decode 分离、goodput、TTFT / TPOT 和资源池化的系统动机说明；参考 Splitwise 论文对 phase splitting、prompt computation 与 token generation 资源画像差异、KV / 状态传输代价的说明；参考 SGLang PD Disaggregation 文档对 prefill interruption、DP attention imbalance、KV transfer、Mooncake / NIXL backend 和 PD multiplexing 的公开口径；参考 vLLM disaggregated prefilling 文档对 P/D 分离示例、KV connector 和实验性边界的说明；并参考 Mooncake 论文对 KV cache-centric disaggregated architecture 的背景说明。

本讲只回答“PD 分离解决什么问题”：它从统一 engine 的 P/D 干扰、资源画像不匹配、独立扩缩容和尾延迟治理出发，解释为什么需要把 prefill pool 和 decode pool 拆开。不把某个框架的命令行参数、transfer backend、connector API、真实 KV layout、跨节点网络配置或 benchmark 数字写成通用结论。下一章会进一步拆系统架构，后续章节再展开 KV cache 迁移、共享、路由和 chunked prefill 的组合。

本讲新增 demo 是教学版 PD 动机审计器：用同一批 toy workloads 对比 unified engine 和 PD 分离后的 p99 decode jitter、prefill wait、独立扩缩容、KV transfer 成本和慢传输反例，帮助把“PD 分离有用但不是免费午餐”讲成可验证的门禁。

36.1 本章目标

读完本章，你应该能讲清：

1. 什么是 PD 分离。
2. 统一引擎混合调度 Prefill/Decode 有哪些问题。
3. PD 分离主要改善哪些指标。
4. 为什么它可以降低 P/D 互相干扰。
5. 为什么它能让资源池独立扩缩容。
6. PD 分离带来哪些新问题。
7. 面试中如何回答“PD 分离解决什么问题”。

36.2 什么是 PD 分离

传统 unified serving engine 中, 一个 GPU worker 同时处理 prefill 和 decode:

```
request -> same engine
        -> prefill prompt
        -> decode tokens
```

PD 分离把它拆成:

```
request
  -> prefill worker / prefill pool
      -> compute prompt KV cache
      -> transfer or share KV cache
  -> decode worker / decode pool
      -> generate output tokens
```

也就是:

Prefill workers 负责输入阶段

Decode workers 负责输出阶段

注意: PD 分离不是把模型拆成前半层和后半层。

它拆的是请求生命周期阶段, 不是 Transformer 层。

Prefill worker 和 Decode worker 通常都需要能运行同一个模型, 只是执行阶段和调度策略不同。

36.3 统一引擎的问题一: Prefill interruption

SGLang 文档中提到 unified scheduling 的一个核心问题是 prefill interruption。

意思是: 新来的 prefill batch 会打断正在进行的 decode batch。

时间线:

```
t0: decode requests 正在稳定输出
t1: 一个长 prompt 到达
t2: scheduler 调度长 prefill
t3: decode 暂停或延迟
t4: 用户看到 streaming 卡顿
```

这种问题本质是 Prefill 和 Decode 争抢同一组 GPU compute。

长 prompt prefill 可能让 decode step latency 突然升高。

表现为:

1. TPOT 抖动。
2. ITL 变差。
3. streaming 一卡一卡。
4. p95/p99 输出延迟上升。

PD 分离后, prefill 进入 prefill pool, 不再直接占用 decode pool 的执行时间。

36.4 统一引擎的问题二: Decode 让 Prefill 饥饿

反过来, 如果 scheduler 为了保护 streaming, 长期 decode-first, 也会导致 waiting prefill 进不来。

时间线:

```
t0: running decode 很多
t1: new request 进入 waiting queue
t2: scheduler 每轮优先 decode
```

t3: prefill 一直没有足够 budget

t4: 新请求 TTFT 很高

表现为:

1. waiting queue 增长。
2. p95/p99 TTFT 上升。
3. 新用户长时间看不到首 token。
4. GPU 似乎很忙, 但新请求体验差。

PD 分离后, Prefill 有自己的资源池和调度器, Decode 压力不会直接吞掉 Prefill 的全部机会。

36.5 统一引擎的问题三: 资源画像不匹配

Prefill 和 Decode 适合的执行策略不同。

Prefill 更希望:

1. 大 token batch。
2. 高 input tokens/s。
3. 更强计算吞吐。
4. 大 prompt chunk 处理能力。
5. 高效 KV 写入。

Decode 更希望:

1. 稳定低 step latency。
2. 高 output tokens/s。
3. 高效 KV 读取。
4. 低调度开销。
5. 平滑 streaming。

放在同一个 engine 里, 配置只能折中。

例如:

max_num_batched_tokens 调大 -> prefill 吞吐好, 但 decode 抖动风险上升

max_num_batched_tokens 调小 -> decode 稳定, 但 prefill 吞吐下降

PD 分离让 P/D 可以使用不同参数和策略。

36.6 统一引擎的问题四: 扩缩容耦合

不同业务的输入输出比例不同。

长输入短输出: Prefill 压力大。

短输入长输出: Decode 压力大。

统一 engine 中, 每加一个 replica, Prefill 和 Decode 能力一起增加。

但真实需求可能是:

Prefill 已经够了, Decode 不够

或者:

Decode 够了, Prefill 不够

统一扩容会浪费资源。

PD 分离后可以:

prefill pool 独立扩容

decode pool 独立扩容

资源配比可以根据流量动态调整。

36.7 统一引擎的问题五：尾延迟难治理

统一 engine 中，p99 往往来自 P/D 互相影响。

常见长尾：

1. 超长 prompt 到来，decode 抖动。
2. decode-heavy 流量高峰，新请求 TTFT 飙升。
3. KV cache 接近满，调度和 eviction 抖动。
4. 长输出请求占用 running slots 很久。
5. batch shape 在 prefill/decode 间剧烈变化。

PD 分离的价值之一是隔离长尾来源。

Prefill pool 的 p99 TTFT 问题，可以针对 input tokens、prefix cache、chunked prefill 处理。

Decode pool 的 p99 TPOT 问题，可以针对 running batch、KV layout、speculative decoding 处理。

治理对象更清晰。

36.8 统一引擎的问题六：DP attention imbalance

SGLang 文档还提到 DP attention imbalance。

直觉是：在 data-parallel attention 或类似多 worker 并行场景中，不同 DP worker 同时处理的阶段可能不同。

例如：

DP worker 0: 正在处理 prefill batch

DP worker 1: 正在处理 decode batch

由于 prefill 和 decode 的执行时间、batch shape、资源压力不同，某些 worker 会拖慢整体同步或导致 decode latency 上升。

这不是简单的“GPU 不够”，而是不同阶段混在同一并行组织里导致负载不均。

PD 分离后，prefill 和 decode 的 worker 角色更明确，可以分别组织并行策略。

36.9 PD 分离改善哪些指标

PD 分离可能改善：

1. p95/p99 TPOT。
2. streaming 平滑度。
3. 长 prompt 到来时的 decode 抖动。
4. p95/p99 TTFT。
5. Prefill input tokens/s。
6. Decode output tokens/s。
7. GPU 利用率稳定性。
8. 不同 workload 的资源利用率。
9. P/D 独立扩缩容效率。

但它不保证所有指标都自动变好。

特别是平均 TTFT 可能受 KV transfer 影响。

如果 KV 迁移慢，首 token 后进入 decode 的等待时间可能增加。

所以 PD 分离要看端到端收益。

36.10 PD 分离如何降低 Decode 抖动

统一 engine 中, decode 和 prefill 混跑。

PD 分离后, decode pool 主要处理:

已完成 prefill 的请求

-> 持续 decode

-> streaming output

Decode pool 不再频繁插入大块 prefill。

因此 decode step latency 更稳定。

这对流式体验很重要。

用户不只关心平均 TPOT, 也关心 inter-token latency 是否稳定。

PD 分离可以减少这种抖动来源。

36.11 PD 分离如何改善 Prefill 吞吐

Prefill pool 可以专门优化 input token 处理。

例如:

1. 更大的 prefill batch token budget。
2. 更适合长 prompt 的 chunk 策略。
3. 更积极的 prefix cache 复用。
4. 更适合 compute-bound 的并行配置。
5. 不被 running decode 频繁打断。

统一 engine 中, 为了保护 decode, prefill 往往被限制。

独立 prefill pool 可以更大胆地做 batched prefill。

这对长上下文 RAG、文档分析、代码仓库问答等很重要。

36.12 PD 分离如何支持异构资源

由于 P/D 资源画像不同, 可以用不同资源配置。

例如:

Prefill pool:

更强计算吞吐

更大 prefill token budget

更适合长 prompt

Decode pool:

更关注低延迟

更关注 KV cache 读取

更稳定 running batch

在更复杂系统中, 还可能有:

1. 不同 GPU 型号。
2. 不同 TP/DP 配置。
3. 不同 batch 策略。
4. 不同扩缩容策略。
5. 不同 QoS 等级。

但异构也带来复杂性, 尤其是 KV cache layout 兼容和传输效率。

36.13 PD 分离的核心代价：KV cache 迁移

Prefill 结束后，Decode 需要 prompt 的 KV cache。

如果 P 和 D 在不同 worker 上，就必须解决：

Prefill worker 上生成的 KV cache 如何给 Decode worker 使用？

常见方式：

1. 通过高速网络传输 KV。
2. 使用 RDMA、NVLink、专用 transfer engine。
3. GPU staging buffer。
4. CPU 或远端缓存中转。
5. 共享存储或统一内存抽象。

SGLang 文档中提到 Mooncake、NIXL 等 transfer backend，就是为了解决这一类 KV transfer 问题。

KV 迁移是 PD 分离的核心难点。

如果迁移太慢，收益会被抵消。

36.14 KV 迁移成本有多重要

KV cache 大小和这些因素有关：

1. prompt token 数。
2. 模型层数。
3. head 数和 head dimension。
4. KV dtype。
5. TP/DP 切分方式。
6. 是否 MLA、GQA、MHA。

长 prompt 的 KV 可能非常大。

如果跨节点传输，网络带宽和延迟会成为关键瓶颈。

因此 PD 分离设计必须回答：

省下的 P/D 干扰成本，是否大于 KV transfer 成本？

这是评估 PD 分离是否值得的核心。

36.15 路由问题

PD 分离后，请求不再只路由到一个 worker。

它需要经历：

router

- > choose prefill worker
- > prefill
- > choose decode worker
- > transfer KV
- > decode

Router 要考虑：

1. Prefill pool 负载。
2. Decode pool 负载。
3. KV transfer locality。
4. 模型版本一致性。
5. TP/DP 配置兼容。
6. 故障和重试。

7. 多租户隔离。

路由不当会导致：

1. Prefill 排队。
2. Decode 排队。
3. KV transfer 跨慢链路。
4. cache locality 丢失。
5. p99 变差。

所以 PD 分离一定伴随更复杂的 router。

36.16 状态同步问题

统一 engine 中，一个 request 的状态通常在一个进程或一个 engine 内管理。

PD 分离后，请求状态跨多个组件：

1. Router。
2. Prefill worker。
3. Transfer backend。
4. Decode worker。
5. API server / stream connection。

需要同步：

1. request id。
2. prompt tokens。
3. KV cache metadata。
4. first token 或 prefill result。
5. decode worker 分配的 slots。
6. finish reason。
7. abort signal。
8. error state。

任何一环失败，都要清理资源。

例如 decode worker 断开，prefill worker 上暂存的 KV metadata 不能永远占着。

36.17 故障恢复问题

PD 分离引入更多失败点。

可能失败：

1. Prefill worker OOM。
2. Decode worker OOM。
3. KV transfer 超时。
4. Router 选错或目标不可达。
5. 网络抖动。
6. Decode worker 中途宕机。
7. 客户端断开。

需要处理：

1. 是否重试 prefill。
2. 是否换 decode worker。
3. 已传输 KV 如何清理。
4. 已输出 token 如何处理。
5. Streaming 如何返回错误。
6. Metrics 如何归因。

这就是为什么 PD 分离不适合作为初学者第一版 serving 架构。

36.18 什么时候 PD 分离收益明显

收益明显的场景通常有：

1. 流量规模大。
2. Prefill 和 Decode 干扰明显。
3. p99 TPOT 被长 prompt 打爆。
4. p99 TTFT 被 decode-heavy 流量拖高。
5. 输入输出比例波动大。
6. 长上下文请求多。
7. 有高性能 KV transfer 能力。
8. 需要 P/D 独立扩缩容。
9. 有成熟监控和路由系统。

例如：

长上下文 RAG + 大量流式聊天混合
代码生成输出 + 文档分析长输入混合
多租户模型服务平台

这些场景里，统一 engine 的折中可能已经不够。

36.19 什么时候不适合 PD 分离

不适合或要谨慎：

1. 流量小。
2. 请求短。
3. 单机 serving 已满足 SLO。
4. KV transfer 网络条件差。
5. 团队缺少分布式调度经验。
6. 监控不完善。
7. 模型或 KV layout 频繁变化。
8. 调试稳定性比极致性能更重要。

如果没有明确 P/D 干扰证据，先不要上 PD 分离。

先做：

1. continuous batching。
2. chunked prefill。
3. prefix cache。
4. scheduler 调优。
5. 限流和隔离。
6. 指标观测。

再判断 PD 分离是否值得。

36.20 和 chunked prefill 的关系

Chunked prefill 和 PD 分离都能缓解长 prefill 阻塞 decode。

但层次不同。

Chunked prefill：

同一 engine 内，把长 prefill 切小，穿插 decode

PD 分离：

不同资源池分别处理 prefill 和 decode
对比：

维度	Chunked prefill	PD 分离
改动范围	scheduler 内策略	系统架构
是否分资源池	否	是
是否需要 KV 迁移	否	是
复杂度	中等	高
主要收益	降低长 prefill 阻塞	隔离 P/D 干扰，独立扩缩容
适合阶段	单机优化优先	大规模架构优化

工程上通常先做 chunked prefill，再评估 PD 分离。

36.21 和 prefix cache 的关系

Prefix cache 和 RadixAttention 可以减少 prefill 成本。

如果 prefix cache 命中率很高，prefill 压力会下降。

这可能降低 PD 分离的必要性。

但在以下情况，PD 分离仍然有价值：

- 1. Cache miss 仍然很多。
- 2. 新文档长 prompt 多。
- 3. Decode 长输出压力大。
- 4. P/D 流量比例波动大。
- 5. 需要独立扩缩容。

也就是说，prefix cache 是减少 prefill 工作量，PD 分离是隔离 prefill/decode 执行资源。
二者不是替代关系。

36.22 和 speculative decoding 的关系

Speculative decoding 主要优化 decode。

如果 decode 是瓶颈，可以先尝试 speculative decoding。

但它不能解决所有问题。

例如：

- 1. 长 prefill 仍可能打断 decode。
- 2. Prefill queue 仍可能很长。
- 3. P/D 扩缩容仍耦合。
- 4. KV cache 和 batch shape 仍混在同一 engine。

PD 分离和 speculative decoding 可以组合：

Prefill pool 负责高吞吐 prompt 处理
Decode pool 使用 speculative decoding 加速输出
但组合后系统更复杂，需要更强监控和调度。

36.23 PD 分离的价值边界

可以用一句话判断：

如果你的主要问题是 P/D 在同一资源池里互相干扰，并且 KV transfer 成本可控，PD 分离有价值。

如果主要问题是：

1. 模型太慢。
2. 单请求太短。
3. Tokenizer 慢。
4. RAG 检索慢。
5. 网络 streaming 慢。
6. GPU 数太少。

PD 分离不一定解决根因。

它是高级 serving 架构，不是通用性能按钮。

36.24 面试官会怎么问

问题一：PD 分离解决什么问题？

回答要点：解决统一 engine 中 prefill 和 decode 资源画像不同却混合调度导致的互相干扰、尾延迟、资源扩缩容耦合和调度折中问题。

问题二：Unified scheduling 有什么问题？

回答要点：长 prefill 会打断 decode，导致 TPOT/streaming 抖动；decode-first 又会让 prefill 饥饿，导致 TTFT 上升；不同 DP worker 混合 P/D 还可能负载不均。

问题三：PD 分离的核心代价是什么？

回答要点：KV cache 要从 prefill worker 传给 decode worker，引入网络带宽、延迟、layout 兼容、路由、状态同步、故障恢复复杂度。

问题四：PD 分离和 chunked prefill 的区别？

回答要点：chunked prefill 是同一 engine 内把长 prefill 切小，与 decode 穿插；PD 分离是系统架构上拆成 prefill/decode 资源池，需要 KV 迁移但能独立扩缩容和隔离干扰。

问题五：什么时候不该做 PD 分离？

回答要点：流量小、请求短、单机已满足 SLO、KV transfer 条件差、监控和调度能力不足时，不应过早引入。

36.25 标准回答模板

如果面试官问“PD 分离解决什么问题”，可以这样回答：

PD 分离解决的是 Prefill 和 Decode 在统一 serving engine 中混合调度带来的资源干扰和扩缩容耦合问题。Prefill 一次处理大量 token，影响 TTFT；Decode 每轮每请求通常只生成一个 token，要频繁读取 KV cache，更偏 memory bandwidth 和调度敏感，影响 TPO。

在统一 engine 中，长 prompt prefill 可能打断正在进行的 decode，导致输出卡顿；如果 scheduler 长期 decode-first，又会让 waiting prefill 饥饿，新请求 TTFT 变高。同时 P/D 资源画像不同，却只能用同一套 batch、token budget、并行和调度策略。

PD 分离把 prefill 和 decode 拆到不同资源池，让 prefill pool 专注 input tokens/s 和 prompt KV 生成，让 decode pool 专注 output tokens/s。

它的代价是 KV cache 必须从 prefill worker 迁移或共享给 decode worker，因此会引入 transfer backend、网络带宽、KV layout、状态同步等复杂度。

36.26 PD 分离收益公式、代价门禁和可运行 demo

上一章已经把单个请求的 Prefill / Decode 资源画像拆开。本章要进一步回答：在一个混合 workload 里，PD 分离到底解决什么问题，怎么判断它不是把系统复杂度白白提高？

先定义每类 workload j ：

$W_j = (n_j, P_j, O_j, H_j)$

其中 n_j 是请求数, P_j 是 prompt token 数, O_j 是输出 token 数, H_j 是 prefix 命中 token 数。真正执行的 prefill token 数为:

$$P_j^{\mathrm{run}} = \max(0, P_j - H_j)$$

整批流量的 prefill 与 decode token 量分别是:

$$N_{\mathrm{prefill}} = \sum_j n_j P_j^{\mathrm{run}}$$

$$N_{\mathrm{decode}} = \sum_j n_j O_j$$

统一 engine 中, 长 prefill 对 decode 的单次干扰可以粗略估成:

$$S_{\mathrm{unified}} = \frac{\max_j P_j^{\mathrm{run}}}{X_{\mathrm{prefill}}}$$

其中 X_{prefill} 是统一 engine 中单 worker prefill tokens/ms 的教学版吞吐。这个量不是精确 kernel 时间, 而是用来解释 “一个超长 prefill 会在 decode 时间线上制造多大 stall”。

PD 分离后, decode pool 不再被大块 prefill 直接打断, 但每个请求要支付 KV transfer:

$$K_{\mathrm{transfer}} = \sum_j n_j P_j^{\mathrm{run}} m_{\mathrm{kv}}$$

如果最大单请求 KV transfer 时间写成:

$$T_{\mathrm{transfer, max}} = \frac{\max_j P_j^{\mathrm{run}} m_{\mathrm{kv}}}{B_{\mathrm{transfer}}}$$

那么一个教学版收益判断可以写成:

$$G_{\mathrm{pd}} = G_{\mathrm{interfere}} G_{\mathrm{jitter}} G_{\mathrm{starve}} G_{\mathrm{scale}} G_{\mathrm{transfer}} G_{\mathrm{wait}}$$

含义是: 必须先看到 unified engine 里确实有 P/D 干扰; PD 后 decode jitter 和 prefill 饥饿要下降; P/D 独立扩缩容要有意义; KV transfer 成本必须被显式计入; 如果 transfer 带宽太差, 系统要能判断 PD 分离收益会被吞掉。

下面的 demo 不模拟真实 GPU, 也不代表任何框架 benchmark。它只把 PD 分离的动机做成一个可运行审计: 同一组 workload 在 unified engine 和 PD 架构下分别估算 jitter、wait、transfer 和门禁。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Workload:
```

```
    name: str
    requests: int
    prompt_tokens: int
    output_tokens: int
    prefix_hit_tokens: int = 0
```

```
class ToyPDDisaggregationAuditor:
```

```
    def __init__(self, workloads, kv_mib_per_token=0.015625):
        self.workloads = workloads
        self.kv_mib_per_token = kv_mib_per_token
        self.unified_workers = 8
        self.prefill_workers = 4
        self.decode_workers = 6
        self.unified_prefill_tokens_per_ms = 900.0
        self.unified_decode_tokens_per_ms = 90.0
        self.prefill_pool_tokens_per_ms = 1400.0
        self.decode_pool_tokens_per_ms = 150.0
        self.transfer_mib_per_ms = 800.0
        self.slow_transfer_mib_per_ms = 8.0
```



```

def _row(self, workload):
    run_prefill_per_req = max(0, workload.prompt_tokens - workload.prefix_hit_tokens)
    total_run_prefill = run_prefill_per_req * workload.requests
    total_decode_tokens = workload.output_tokens * workload.requests
    kv_transfer_mib = total_run_prefill * self.kv_mib_per_token
    return {
        "name": workload.name,
        "requests": workload.requests,
        "run_prefill_per_req": run_prefill_per_req,
        "total_run_prefill": total_run_prefill,
        "total_decode_tokens": total_decode_tokens,
        "kv_transfer_mib": round(kv_transfer_mib, 1),
    }

def run(self):
    rows = [self._row(workload) for workload in self.workloads]
    total_prefill = sum(row["total_run_prefill"] for row in rows)
    total_decode = sum(row["total_decode_tokens"] for row in rows)
    total_transfer_mib = sum(row["kv_transfer_mib"] for row in rows)
    longest_prefill = max(row["run_prefill_per_req"] for row in rows)
    largest_transfer_mib = max(
        row["run_prefill_per_req"] * self.kv_mib_per_token for row in rows
    )
    running_decode_rows = 160

    unified_long_stall = longest_prefill / self.unified_prefill_tokens_per_ms
    unified_decode_jitter_p99 = 12.0 + unified_long_stall
    unified_prefill_wait_p99 = (
        running_decode_rows * 16
    ) / self.unified_decode_tokens_per_ms

    pd_largest_transfer = largest_transfer_mib / self.transfer_mib_per_ms
    pd_decode_jitter_p99 = 12.0 + pd_largest_transfer
    pd_prefill_wait_p99 = longest_prefill / (
        self.prefill_workers * self.prefill_pool_tokens_per_ms
    )
    slow_transfer_largest = largest_transfer_mib / self.slow_transfer_mib_per_ms
    interference_saving = unified_decode_jitter_p99 - pd_decode_jitter_p99

    unified = {
        "workers": self.unified_workers,
        "prefill_ms": round(
            total_prefill
            / (self.unified_workers * self.unified_prefill_tokens_per_ms),
            3,
        ),
        "decode_ms": round(
            total_decode / (self.unified_workers * self.unified_decode_tokens_per_ms),
            3,
        ),
        "decode_jitter_p99_ms": round(unified_decode_jitter_p99, 3),
        "prefill_wait_p99_ms": round(unified_prefill_wait_p99, 3),
    }
    pd = {
        "prefill_workers": self.prefill_workers,

```

```

        "decode_workers": self.decode_workers,
        "prefill_pool_ms": round(
            total_prefill / (self.prefill_workers * self.prefill_pool_tokens_per_ms),
            3,
        ),
        "decode_pool_ms": round(
            total_decode / (self.decode_workers * self.decode_pool_tokens_per_ms),
            3,
        ),
        "total_kv_transfer_mib": round(total_transfer_mib, 1),
        "largest_request_transfer_ms": round(pd_largest_transfer, 3),
        "decode_jitter_p99_ms": round(pd_decode_jitter_p99, 3),
        "prefill_wait_p99_ms": round(pd_prefill_wait_p99, 3),
        "interference_saving_ms": round(interference_saving, 3),
        "slow_transfer_largest_ms": round(slow_transfer_largest, 3),
    }
    summary = {
        "total_requests": sum(row["requests"] for row in rows),
        "total_run_prefill_tokens": total_prefill,
        "total_decode_tokens": total_decode,
        "prefill_decode_token_ratio": round(total_prefill / max(1, total_decode), 3),
        "unified_decode_jitter_p99_ms": unified["decode_jitter_p99_ms"],
        "pd_decode_jitter_p99_ms": pd["decode_jitter_p99_ms"],
        "unified_prefill_wait_p99_ms": unified["prefill_wait_p99_ms"],
        "pd_prefill_wait_p99_ms": pd["prefill_wait_p99_ms"],
        "total_kv_transfer_mib": pd["total_kv_transfer_mib"],
        "slow_transfer_would_erase_benefit": pd["slow_transfer_largest_ms"]
        > pd["interference_saving_ms"],
    }
    gates = {
        "unified_interference_visible": unified["decode_jitter_p99_ms"] > 20.0,
        "pd_reduces_decode_jitter": pd["decode_jitter_p99_ms"]
        < unified["decode_jitter_p99_ms"] * 0.65,
        "pd_reduces_prefill_starvation": pd["prefill_wait_p99_ms"]
        < unified["prefill_wait_p99_ms"] * 0.5,
        "independent_scaling_visible": self.prefill_workers != self.decode_workers,
        "kv_transfer_cost_counted": pd["total_kv_transfer_mib"] > 0,
        "not_free_lunch_visible": summary["slow_transfer_would_erase_benefit"],
        "p_d_metrics_ready": summary["prefill_decode_token_ratio"] > 1.0,
    }
    gates["pd_motivation_gate"] = all(gates.values())
    return rows, unified, pd, summary, gates

workloads = [
    Workload("rag_long_prompt", requests=80, prompt_tokens=16000, output_tokens=120, prefix_hit_tokens=4000),
    Workload("short_chat", requests=240, prompt_tokens=700, output_tokens=300),
    Workload("code_long_output", requests=60, prompt_tokens=900, output_tokens=2000),
    Workload("agent_context_growth", requests=100, prompt_tokens=3000, output_tokens=500, prefix_hit_tokens=800),
]

rows, unified, pd, summary, gates = ToyPDDisaggregationAuditor(workloads).run()
print("pd_rows=", rows)
print("unified_summary=", unified)
print("pd_summary=", pd)

```

一次运行的核心输出类似：

这个输出有几个关键信号：

- 所以本章的最终门禁不是“有没有把 P/D 拆开”，而是 `pd_motivation_gate`：必须同时证明 unified 干扰存在、PD 后 jitter / starvation 下降、P/D 独立扩缩容有意义、KV transfer 成本被计算，并且系统能识别慢传输吞掉收益的反例。

1. 画出 unified engine 中长 prefill 打断 decode 的时间线。
2. 画出 PD 分离后请求从 prefill pool 到 decode pool 的生命周期。
3. 列出 PD 分离可能改善的 5 个指标。
4. 列出 PD 分离新增的 5 个系统复杂度。
5. 对比 chunked prefill、prefix cache、speculative decoding 和 PD 分离分别优化什么。
6. 给定一个长输入短输出 RAG 场景，判断 PD 分离是否可能有收益。
7. 给定一个短输入短输出客服场景，判断是否需要 PD 分离。

下一章会进入 PD 分离的系统架构, 具体拆解 router、prefill worker、decode worker、KV transfer backend、状态同步和请求生命周期。

一句话概括:

PD 分离的系统架构核心是：用 router 串起 prefill pool、decode pool 和 KV transfer backend，把请求从“单 engine 内部状态机”升级成“跨组件分布式状态机”。

37.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 `WRITING_PLAN.md` 对公开资料做校准：参考 SGLang PD Disaggregation 文档对 Model Gateway、prefill / decode worker 注册、prefill policy、decode policy、bootstrap / transfer backend、Mooncake / NIXL backend、heterogeneous TP 和 backpressure 的公开口径；参考 vLLM disaggregated prefilling 示例对 KV connector、prefill worker、decode worker 和实验性边界的说明；参考 DistServe、Splitwise 和 Mooncake 论文对 P/D 分离、KV transfer、资源池化和状态传递的系统动机说明。

本讲只讲 PD 分离系统架构的稳定抽象：router、prefill worker、decode worker、KV transfer backend、bootstrap metadata、跨组件状态机、故障清理和指标观测。不把某个 SGLang / vLLM 版本的 CLI 参数、内部类名、真实连接器字段、网络配置、RDMA/NIXL/Mooncake 具体实现或 benchmark 数字写成唯一标准答案。第 38 章会专门展开 KV cache 迁移、共享和路由，本章先把“组件如何协作”讲清。

本讲新增 demo 是教学版 PD 架构审计器：用 0 依赖 Python 模拟 router 选择 prefill / decode worker、decode 预留目标 slot、prefill 生成 KV metadata、transfer 成功或失败、streaming 完成和 client abort 清理，帮助把“跨组件状态机”从架构图落到可验收证据。

37.1 本章目标

读完本章，你应该能讲清：

1. PD 分离架构里有哪些核心组件。
2. Router 在 PD 分离中负责什么。
3. Prefill worker 和 Decode worker 的职责边界是什么。
4. KV transfer backend 为什么是架构核心。
5. 一个请求在 PD 分离系统中的端到端生命周期。
6. PD 分离需要维护哪些 metadata 和状态。
7. 生产系统中如何做路由、健康检查、故障处理和观测。
8. 面试中如何画出 PD 分离系统架构图。

37.2 从 unified engine 到 PD 架构

Unified engine 的请求路径通常是：

```
client
-> api server
-> scheduler
-> prefill
-> decode loop
-> stream output
```

所有阶段通常在同一个 serving engine 或同一组 worker 内完成。

PD 分离后，请求路径变成：

```
client
-> router
-> choose prefill worker
-> run prefill
-> transfer KV cache
-> choose decode worker
-> run decode loop
-> stream output
```

架构上至少多了三类东西：

1. Prefill pool。

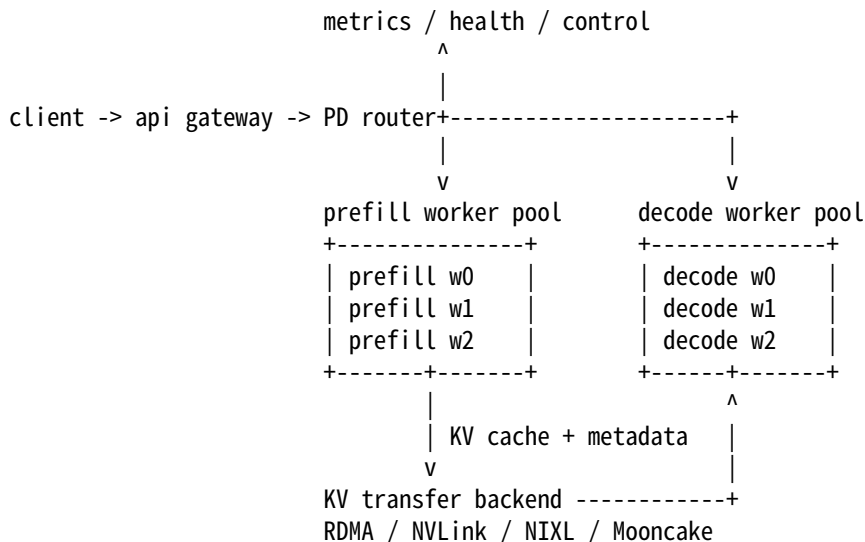
2. Decode pool。
3. KV transfer / sharing 机制。

同时 router 不再只是一个普通负载均衡器。

它需要理解请求阶段、worker 类型、KV locality、transfer 状态和故障清理。

37.3 总体架构图

一个典型 PD 分离架构可以画成：



这里有几条关键路径：

1. 控制路径：router 注册 worker、检查健康、更新负载、处理扩缩容。
2. 请求路径：client 请求经过 router，依次进入 prefill 和 decode。
3. 数据路径：prompt KV cache 从 prefill worker 转移到 decode worker。
4. 返回路径：decode worker 生成 token，router 或 API 层把 token stream 返回给 client。

不要把它理解成普通微服务拆分。

普通微服务传的是 JSON 或 protobuf。

PD 分离最重的东西是 KV cache，它在 GPU memory 里，体积大，layout 复杂，传输成本高。

37.4 核心组件一：PD Router

Router 是 PD 分离的中枢。

在普通 serving 中，router 可能只做：

1. 选择一个 worker。
2. 转发请求。
3. 做健康检查。
4. 做简单重试。

PD 分离中，router 至少要做：

1. 维护 prefill worker 列表。
2. 维护 decode worker 列表。
3. 分别跟踪 P/D worker 的负载。
4. 为每个请求选择 prefill worker。
5. 为每个请求选择 decode worker。
6. 协调 KV transfer。

7. 维护 request state。
8. 处理 streaming 返回。
9. 处理 abort、timeout、retry 和资源清理。

SGLang Model Gateway 的 PD 模式就是这一类 router 的例子。它支持 `--pd-disaggregation`, 并通过 `--prefill` 和 `--decode` 注册不同类型的 worker。

示意命令:

```
python -m sglang_router.launch_router \  
  --pd-disaggregation \  
  --prefill http://prefill1:30001 9001 \  
  --decode http://decode1:30011 \  
  --prefill-policy cache_aware \  
  --decode-policy power_of_two
```

这里 `--prefill` 后面的额外端口通常用于 prefill side 的 bootstrap 或 transfer 协调。

37.5 Router 为什么不能只随机选 worker

PD 分离里, 如果 router 随机选择 prefill 和 decode worker, 系统很容易变差。

原因是它要同时考虑:

1. Prefill worker 当前排队 token 数。
2. Decode worker 当前 running sequences。
3. Decode worker 剩余 KV cache capacity。
4. Prefill 到 decode 的网络距离。
5. KV transfer backend 是否可用。
6. 模型版本是否一致。
7. TP/DP 配置是否兼容。
8. Prefix cache 或 KV cache locality。
9. Worker 是否处于健康状态。

一个错误选择可能导致:

prefill 很快完成

- > 但 decode worker 没有 slot
- > KV 等待或 transfer 超时
- > TTFT / p99 变差

或者:

选中的 decode worker 很空

- > 但它和 prefill worker 跨慢链路
- > KV 传输时间很长
- > 端到端收益被抵消

所以 PD router 本质上是一个阶段感知、负载感知、数据位置感知的调度器。

37.6 核心组件二: Prefill Worker

Prefill worker 负责处理输入 prompt。

它的主要职责是:

1. 接收 prompt tokens。
2. 执行 prefill forward。
3. 生成每层 KV cache。
4. 可能生成 first token logits。
5. 暴露 KV cache metadata。
6. 和 transfer backend 协作发送 KV。

7. 在请求失败或超时时清理临时状态。

Prefill worker 更关注：

1. input tokens/s。
2. 大 batch token budget。
3. 长 prompt 处理能力。
4. prefix cache 命中。
5. KV 写入效率。
6. transfer 准备时间。

它通常不负责长时间 decode。

也就是说，prefill worker 的请求生命周期应该尽量短：完成 prompt 计算、交付 KV、释放临时资源。

37.7 Prefill Worker 的内部流程

Prefill worker 收到请求后，大致流程是：

receive prefill request

- > validate model / tokenizer / request id
- > allocate KV cache blocks
- > run prefill forward
- > produce KV cache
- > prepare transfer metadata
- > wait for decode destination info
- > transfer or expose KV
- > return prefill result

其中最重要的是 metadata。

Prefill 不能只告诉 decode：“我算完了”。

它还要告诉 decode：

1. KV cache 对应哪个 request。
2. prompt 长度是多少。
3. 每层 KV 的地址或 block 映射是什么。
4. dtype 和 layout 是什么。
5. tensor parallel 切分方式是什么。
6. decode worker 应该把 KV 放到哪些目标 slot。
7. transfer 是否完成。

这些 metadata 一旦错了，轻则输出错误，重则进程崩溃或读写非法显存。

37.8 核心组件三：Decode Worker

Decode worker 负责持续生成输出 token。

它的主要职责是：

1. 为请求分配 decode slot。
2. 接收或拉取 prefill 阶段生成的 KV cache。
3. 把 KV cache 放入本地 KV cache manager。
4. 从 first token 或 next token 开始 decode。
5. 参与 continuous batching。
6. 执行 sampling。
7. 持续 streaming token。
8. 请求完成后释放 KV cache。

Decode worker 更关注：

1. output tokens/s。
2. TPOT。
3. inter-token latency。
4. running batch 稳定性。
5. KV cache 读取效率。
6. decode slot 和显存容量。

Decode worker 的核心目标是让输出阶段稳定。

它应该尽量避免被大块 prefill 打断。

37.9 Decode Worker 的内部流程

Decode worker 的流程可以理解成：

```
receive decode assignment
-> reserve decode slots / KV blocks
-> provide destination metadata to prefill side
-> receive KV cache
-> mark request ready for decode
-> join decode scheduler
-> generate tokens iteratively
-> stream output
-> release resources
```

注意一个关键点：decode worker 不能等 KV 到了才思考放哪里。

它通常要先预留目标 KV block 或 slot，然后把目标信息告诉 prefill side 或 transfer backend。

否则 prefill side 不知道 KV 应该写到 decode side 的哪个位置。

这也是为什么很多 PD 系统会有 bootstrap 机制。

37.10 核心组件四：KV Transfer Backend

KV transfer backend 是 PD 分离最核心、也最难的组件。

没有高效 KV transfer，PD 分离就只是把问题从调度干扰变成网络瓶颈。

KV transfer backend 要解决：

1. KV 从哪个 GPU 地址读。
2. KV 写到哪个目标 GPU 地址。
3. 跨节点时用什么网络协议。
4. 是否使用 RDMA。
5. 是否需要 CPU staging。
6. 是否需要 GPU staging buffer。
7. 如何处理不同 TP layout。
8. 如何报告完成或失败。
9. 如何处理 timeout 和资源清理。

SGLang 文档中提到的 Mooncake、NIXL、Ascend backend，都是这一层的实现选择。

可以把 transfer backend 理解成：

```
prefill GPU memory
-> transfer engine
-> decode GPU memory
```

它不是普通 HTTP body 传输。

它处理的是高吞吐、低延迟、大体积、layout 敏感的张量数据。

37.11 KV Transfer 的几种形态

常见形态包括：

1. 同机多 GPU：通过 NVLink、PCIe 或 CUDA IPC 传输。
2. 跨节点 GPU：通过 RDMA、InfiniBand、RoCE、UCX 等能力传输。
3. CPU staging：先从 GPU 拷到 CPU，再从 CPU 发到目标侧。
4. GPU staging buffer：先在 GPU 上整理成连续 buffer，再批量传输。
5. 远端 KV cache：把 KV 写入独立缓存层，再由 decode 拉取。
6. 共享内存或统一内存抽象：让 P/D 通过统一地址或对象访问。

不同形态的权衡不同：

方式	优点	代价
同机 NVLink	延迟低、带宽高	受单机 GPU 拓扑限制
RDMA 跨节点	可扩展到多机	部署和调试复杂
CPU staging	实现相对简单	多一次拷贝，延迟高
GPU staging buffer	对异构 TP 更友好	需要额外显存和 layout 转换
远端 KV cache	可做复用和分层	一致性和延迟更难控制

第 38 章会专门展开 KV Cache 迁移、共享和路由。

本章先建立系统架构视角。

37.12 Bootstrap 机制是什么

很多 PD 系统中会出现 bootstrap server 或 bootstrap port。

它解决的是：prefill side 和 decode side 如何交换 transfer 前的控制信息。

例如：

decode worker：

我已经为 request-123 分配了目标 KV blocks
目标地址 / block ids / rank 信息如下

prefill worker：

我已经生成 request-123 的源 KV blocks
准备按你的目标信息传过去

Bootstrap 通常不传大块 KV 本体。

它传的是控制信息和 metadata。

包括：

1. request id。
2. source worker id。
3. destination worker id。
4. source KV block 信息。
5. destination KV block 信息。
6. sequence length。
7. layer/rank mapping。
8. transfer session id。
9. timeout 和状态。

如果 bootstrap 失败，KV transfer 就无法安全开始。

37.13 请求生命周期：从接入到首 token

一个 PD 请求的前半段可以拆成：

1. client 发送请求
2. router 接收请求并生成 request id
3. router 选择 prefill worker
4. router 选择或预选择 decode worker
5. decode worker 预留 KV blocks / decode slot
6. prefill worker 执行 prompt prefill
7. prefill worker 生成 KV cache
8. transfer backend 把 KV 传到 decode worker
9. decode worker 标记 request ready
10. decode worker 生成第一个或下一个 token
11. router / API server 返回首 token

在这个链路里，TTFT 变成：

TTFT = 接入排队
+ prefill 排队
+ prefill 执行
+ decode slot 等待
+ KV transfer
+ 首轮 decode / sampling
+ streaming 返回

这说明 PD 分离虽然降低了 P/D 干扰，但也新增了 KV transfer 和跨组件协调成本。

所以它不是免费优化。

37.14 请求生命周期：持续 decode

首 token 之后，请求进入 decode 阶段。

流程是：

decode worker scheduler
-> select running requests
-> run one decode step
-> sample next token
-> append KV for new token
-> stream token
-> check finish condition
-> repeat

这部分和 unified engine 的 decode loop 类似。

区别在于：

1. Decode worker 的 running set 主要由已完成 prefill 的请求组成。
2. 它不需要频繁插入长 prompt prefill。
3. 它可以使用更适合 decode 的 batch 和调度策略。
4. 它的资源瓶颈更清晰，主要是 KV cache capacity、memory bandwidth、decode batch 和 sampling。

这就是 PD 分离改善 TPOT 和 streaming 稳定性的主要来源。

37.15 跨组件状态机

Unified engine 中，请求状态可以很简单：

WAITING -> PREFILLING -> DECODING -> FINISHED

PD 分离后，状态会更细：

RECEIVED

- > PREFILL_QUEUED
- > PREFILL_RUNNING
- > PREFILL_DONE
- > DECODE_ASSIGNED
- > KV_TRANSFER_PREPARING
- > KV_TRANSFERRING
- > DECODE_READY
- > DECODING
- > FINISHED

还需要错误状态：

PREFILL_FAILED
DECODE_ASSIGN_FAILED
KV_TRANSFER_FAILED
DECODE_FAILED
CLIENT_ABORTED
TIMEOUT

每个状态都要回答三个问题：

1. 当前资源在哪里。
2. 如果失败，谁负责清理。
3. 是否可以重试。

这是 PD 分离比单 engine 难很多的根本原因。

37.16 必须维护的 Metadata

PD 请求至少需要维护这些 metadata：

1. request id。
2. client stream id。
3. model id 和 model version。
4. tokenizer / chat template 信息。
5. prompt token ids。
6. prompt length。
7. sampling parameters。
8. prefill worker id。
9. decode worker id。
10. prefill queue time。
11. decode assignment time。
12. KV source block mapping。
13. KV destination block mapping。
14. TP/DP rank mapping。
15. transfer session id。
16. transfer start/end time。
17. output token count。
18. finish reason。
19. error code。
20. abort signal。

这些 metadata 不只是为了日志。

它们直接参与路由、transfer、重试、清理和指标归因。

37.17 Prefill Pool 和 Decode Pool 的扩缩容

PD 分离的一个核心收益是 P/D 可以独立扩缩容。

例如长输入短输出场景：

input tokens/s 压力大

output tokens/s 压力小

可以增加 prefill worker：

prefill replicas: 8 -> 16

decode replicas: 8

短输入长输出场景：

input tokens/s 压力小

output tokens/s 压力大

可以增加 decode worker：

prefill replicas: 4

decode replicas: 8 -> 16

但扩缩容不能只看请求数。

应该看：

1. prefill waiting tokens。
2. prefill input tokens/s。
3. decode running sequences。
4. decode output tokens/s。
5. decode KV cache utilization。
6. KV transfer bandwidth utilization。
7. TTFT 和 TPOT 的 p95/p99。

如果只扩 prefill，不扩 transfer 网络，可能只是把瓶颈推到 KV transfer。

如果只扩 decode，但 prefill queue 已经爆了，用户仍然等不到首 token。

37.18 路由策略

PD router 通常需要两类策略。

第一类是 prefill routing：

1. round robin。
2. least queued tokens。
3. cache aware。
4. locality aware。
5. tenant aware。

第二类是 decode routing：

1. least running sequences。
2. least KV cache usage。
3. power of two choices。
4. locality aware。
5. SLO aware。

SGLang Model Gateway 中也区分了 --prefill-policy 和 --decode-policy。

这是合理的，因为 prefill 和 decode 的负载指标不同。

Prefill 看的是输入 token 排队和 prompt 计算。

Decode 看的是 running set、KV cache 和 step latency。

37.19 Cache-Aware Routing 在 PD 中的含义

Cache-aware routing 在普通多 replica serving 中, 通常是为了把相似前缀路由到有 prefix cache 的 worker。

在 PD 分离中, 它更复杂。

Router 可能要考虑:

1. Prefill worker 是否有可复用 prefix cache。
2. Decode worker 是否接近相关 KV locality。
3. P/D 是否在同机或近网络拓扑。
4. 是否值得为了 cache 命中选择一个更忙的 worker。
5. Cache 命中节省的 prefill 时间是否大于排队和 transfer 成本。

一个简单判断公式是:

如果 cache 命中节省时间 > 额外排队时间 + 额外传输时间

则值得 cache-aware routing

否则选择更空或更近的 worker

真实系统不会这么简单, 但这个公式能帮你理解权衡。

37.20 健康检查和故障隔离

PD 架构中, 健康检查要分层。

Router 至少要知道:

1. Prefill worker 是否可接新请求。
2. Decode worker 是否可接新请求。
3. Transfer backend 是否可用。
4. Bootstrap 通道是否可用。
5. Worker 的模型版本是否匹配。
6. Worker 的 load 是否异常。
7. KV cache 使用率是否过高。

健康状态也不只是 alive/dead。

更实用的状态包括:

READY

DRAINING

OVERLOADED

TRANSFER_DEGRADED

UNHEALTHY

例如某个 decode worker 还活着, 但 KV cache 已经接近满, router 就不应该继续给它分配新 decode 请求。

37.21 故障处理: Prefill 失败

Prefill 失败可能发生在:

1. prefill worker OOM。
2. prompt 太长。
3. model forward 出错。
4. prefix cache 或 KV allocation 失败。
5. worker 重启。

处理方式通常是:

1. 如果还没有输出 token，可以换一个 prefill worker 重试。
2. 如果是请求本身非法，直接返回错误。
3. 如果 decode worker 已经预留 slot，要通知它释放资源。
4. 记录 prefill failure metrics。

Prefill 失败相对容易处理，因为通常还没有向用户输出 token。

37.22 故障处理：KV Transfer 失败

KV transfer 失败更麻烦。

可能原因：

1. 网络超时。
2. RDMA 连接失败。
3. decode worker 目标 slot 失效。
4. prefill worker 清理过早。
5. metadata 不一致。
6. transfer backend 内部错误。

处理方式包括：

1. 中止当前请求。
2. 清理 prefill side 临时 KV。
3. 清理 decode side 预留 slot。
4. 必要时重新 prefill 并选择新 decode worker。
5. 对异常 worker 降权或熔断。

如果首 token 还没返回，重试空间较大。

如果已经开始 streaming，再切换 decode worker 就很困难，因为新 worker 需要完整上下文和已生成 token 的一致状态。

37.23 故障处理：Decode 失败

Decode 失败可能发生在生成中途。

这时最难处理，因为客户端可能已经收到了部分 token。

选择通常有：

1. 直接结束 stream 并返回错误。
2. 尝试从已生成 token 重新构造 prompt，重新 prefill，再继续生成。
3. 对非流式请求隐藏失败并重试。

方案 2 很复杂，而且可能导致输出不一致。

生产系统通常更倾向于：

streaming 已输出后，decode worker 失败 -> 返回错误并清理资源

除非业务明确接受重试带来的语义变化。

37.24 Client Abort 和资源清理

用户取消请求时，PD 系统要清理多个地方。

可能存在的资源包括：

1. Router 中的 request state。
2. Prefill worker waiting queue 条目。
3. Prefill worker 已分配 KV blocks。
4. Transfer session。

5. Decode worker 预留 slot。
6. Decode worker running request。
7. Streaming connection。

如果只清理 router，不通知 worker，就会泄漏显存或让后台继续生成。

所以 abort 必须广播到相关组件。

可以理解为：

client abort

- > router marks request aborted
- > cancel prefill if running
- > cancel transfer if running
- > cancel decode if running
- > release KV blocks

37.25 观测指标

PD 分离必须有更细的 metrics。

只看总 latency 不够。

建议至少拆分：

1. end-to-end TTFT。
2. prefill queue time。
3. prefill execution time。
4. decode assignment time。
5. KV transfer wait time。
6. KV transfer duration。
7. first decode step time。
8. TPOT。
9. inter-token latency p95/p99。
10. prefill input tokens/s。
11. decode output tokens/s。
12. transfer bandwidth。
13. prefill failure count。
14. decode failure count。
15. transfer timeout count。
16. KV cache utilization。

最好把一次请求的时间线打出来：

request-123:

```
router_queue: 5ms
prefill_queue: 20ms
prefill_compute: 180ms
decode_assign: 3ms
kv_transfer: 40ms
first_decode: 12ms
ttfb/ttft: 260ms
```

没有这种拆分，就很难判断 PD 分离到底是提升还是劣化。

37.26 部署形态一：单机多 GPU PD

最简单的 PD 形态是单机多 GPU。

例如：

GPU 0-3: prefill pool

GPU 4-7: decode pool

优点:

1. 网络拓扑简单。
2. KV transfer 延迟较低。
3. 调试比跨节点容易。
4. 适合验证收益。

缺点:

1. 扩展能力有限。
2. P/D 比例受单机 GPU 数限制。
3. 故障隔离粒度有限。

如果团队第一次做 PD 分离, 通常应该先从单机或小规模集群验证。

37.27 部署形态二: 跨节点 PD

跨节点 PD 更接近大规模生产场景。

示意:

prefill nodes:

node p0: GPU 0-7

node p1: GPU 0-7

decode nodes:

node d0: GPU 0-7

node d1: GPU 0-7

node d2: GPU 0-7

优点:

1. P/D 可以大规模独立扩容。
2. 可以按业务流量调整资源比例。
3. 可以做多租户隔离。
4. 可以按硬件类型组织资源池。

缺点:

1. KV transfer 依赖网络能力。
2. RDMA / IB / RoCE 调试复杂。
3. 拓扑感知路由更重要。
4. 故障面更大。
5. 观测和资源清理更难。

跨节点 PD 的核心问题是:

网络是否足够快、足够稳定, 能让 KV transfer 成本小于 P/D 隔离收益。

37.28 部署形态三: 异构 P/D 资源池

更高级的部署会让 prefill 和 decode 使用不同硬件或不同并行配置。

例如:

Prefill pool:

更适合大 batch compute

更大的 input token budget

可能使用更高 TP

Decode pool:

- 更关注稳定 latency
- 更大 KV cache 容量
- 可能使用 DP attention

异构资源池的价值是把硬件用在更适合的阶段。

但它会带来 KV layout 问题。

例如 prefill TP=4, decode attention TP=1, 则 KV head 的切分方式不同。

这时需要 layout 转换或 staging buffer。

SGLang 文档中提到的 heterogeneous TP with GPU staging buffer, 就是为了解决这类问题。

37.29 和普通 Load Balancer 的区别

面试里容易把 PD router 说成 Nginx 或普通 LB。

这是不准确的。

普通 LB 只需要回答:

这个请求发给哪个后端?

PD router 要回答:

这个请求的 prefill 给谁?

decode 给谁?

KV 从哪里到哪里?

目标 decode 是否有 slot?

transfer 是否完成?

失败后谁清理?

streaming 谁负责?

所以 PD router 更接近一个分布式 serving coordinator。

它既在数据面上转发请求, 也在控制面上维护 worker、状态和资源。

37.30 和 Pipeline Parallel 的区别

PD 分离也容易和 pipeline parallel 混淆。

Pipeline parallel 是按模型层切分:

GPU 0: layer 0-9

GPU 1: layer 10-19

GPU 2: layer 20-29

PD 分离是按请求生命周期阶段切分:

prefill worker: 跑完整模型的 prefill 阶段

decode worker: 跑完整模型的 decode 阶段

两者不是一回事。

Prefill worker 和 decode worker 通常都需要完整模型能力。

它们只是执行不同阶段。

37.31 和 Expert Parallel / MoE 并行的区别

MoE expert parallel 是模型内部专家路由。

例如 DeepSeek、Mixtral 这类 MoE 模型中, 不同 token 会路由到不同 expert。

PD 分离不是 expert routing。

它不决定 token 去哪个 expert。

它决定一个请求的 prefill 阶段和 decode 阶段分别在哪些 serving worker 上执行。

当然，在大模型生产系统中，PD 分离可以和 TP、DP、EP、MoE all-to-all 同时存在。

这也是高级 serving 架构复杂的地方。

37.32 一个最小 PD Router 伪代码

下面用伪代码描述 PD router 的主流程：

```
def handle_request(req):
    request_id = new_request_id()
    state = RequestState(request_id, req)

    prefill = choose_prefill_worker(req)
    decode = choose_decode_worker(req)

    try:
        decode_reservation = decode.reserve_slots(request_id, req.max_tokens)

        prefill_result = prefill.run_prefill(
            request_id=request_id,
            tokens=req.prompt_tokens,
            decode_target=decode_reservation,
        )

        transfer_result = transfer_kv(
            source=prefill_result.kv_source,
            target=decode_reservation.kv_target,
        )

        decode.start_decode(
            request_id=request_id,
            kv_ready=transfer_result,
            sampling_params=req.sampling_params,
        )

        for token in decode.stream(request_id):
            yield token

    except ClientAbort:
        cancel_everywhere(state)
    except Exception as err:
        cleanup_everywhere(state)
    raise err
```

真实系统会复杂得多。

但主线就是：选 P、选 D、预留 D、跑 P、传 KV、启动 D、stream、清理。

37.33 架构设计中的关键取舍

设计 PD 架构时，最重要的取舍包括：

1. 是否预先选择 decode worker。

2. KV transfer 是 push 还是 pull。
3. Router 是否在数据面中转 token stream。
4. Prefill 是否生成 first token。
5. Decode worker 是否支持从 prefill result 直接接管。
6. Transfer metadata 放在 router、worker 还是独立 coordinator。
7. 失败后是否自动重试。
8. 是否允许异构 TP/DP。
9. 是否做 cache-aware routing。
10. 是否引入远端 KV cache。

没有唯一正确答案。

要看业务目标：低延迟、低成本、高吞吐、可扩展性、调试复杂度，哪个更重要。

37.34 Prefill 是否应该生成第一个 token

一个细节问题是：first token 到底由 prefill worker 生成，还是由 decode worker 生成？

可能方案一：prefill worker 计算 prompt 后直接采样 first token。

优点：

1. 首 token 可能更快。
2. prefill logits 已经在手边。

缺点：

1. Decode worker 接管时要同步 first token 状态。
2. KV cache 需要包含 first token 之后的状态。
3. 状态机更复杂。

可能方案二：prefill worker 只生成 prompt KV，decode worker 负责 first token 和后续 token。

优点：

1. 职责更清晰。
2. Decode 状态更一致。
3. Streaming 更容易归一。

缺点：

1. TTFT 可能多一次 decode step。

很多系统会根据实现复杂度和延迟目标选择不同方案。

面试中可以说明这个 trade-off，而不是武断说只有一种实现。

37.35 Push KV 还是 Pull KV

KV transfer 有两种思路。

Push：

prefill worker 主动把 KV 推到 decode worker

Pull：

decode worker 从 prefill worker 或远端缓存拉取 KV

Push 的优点：

1. Prefill 完成后可以立即发送。
2. 流程直观。

Push 的缺点：

1. Decode 目标必须提前确定。
2. 如果 decode worker 忙或失败，处理复杂。

Pull 的优点：

1. Decode worker 控制接收节奏。
2. 更适合远端 KV cache 或多级 cache。

Pull 的缺点：

1. 可能增加等待。
2. 需要更复杂的 source discovery。

实际系统可能是混合模式：metadata 由 router 协调，数据传输由 transfer backend 执行。

37.36 PD 分离的反压机制

PD 系统必须有 backpressure。

否则 prefill 太快，decode 接不住，会导致大量 KV 暂存和内存压力。

常见反压信号包括：

1. Decode worker 可用 slot 数。
2. Decode KV cache 剩余容量。
3. Transfer queue 长度。
4. Prefill queue 长度。
5. Router pending request 数。
6. Per-tenant token budget。

一种简单策略是：

如果没有 decode capacity
不启动新的 prefill

这样可以避免 prefill 算完后无处可去。

但代价是 prefill 资源可能空转。

另一种策略是允许少量 ahead-of-time prefill，但限制 pending transfer 数量。

这就是吞吐和稳定性的权衡。

37.37 多租户和 QoS

生产环境中，PD 分离通常服务多个业务或租户。

QoS 要同时作用在 P 和 D 两侧：

1. 某租户最多占用多少 prefill token budget。
2. 某租户最多占用多少 decode running slots。
3. 长 prompt 是否需要降优先级。
4. 长输出是否需要限制 max tokens。
5. 高优先级请求是否可以抢占队列。
6. 不同租户是否共享 prefix cache。
7. 不同租户是否允许共享 KV cache 基础设施。

如果只在入口限流，不在 P/D 阶段分别限流，仍可能出现一个租户把 decode pool 占满，导致其他租户 TTFT 正常但 TPOT 爆炸。

37.38 安全和版本一致性

PD 架构必须保证 prefill 和 decode 使用兼容的模型。

至少要校验：

1. model id。
2. model weights version。
3. tokenizer version。
4. rope scaling 配置。
5. max context length。
6. KV dtype。
7. attention layout。
8. TP/DP 配置。
9. quantization 配置。

如果 prefill 和 decode 模型版本不一致，KV cache 即使能传过去，也不代表语义正确。

最危险的是 silent wrong output。

所以生产系统中，router 注册 worker 时必须记录能力和版本信息。

37.39 什么时候 Router 应该拒绝请求

PD router 不应该无限接收请求。

应该拒绝或排队受控的情况包括：

1. 没有健康 prefill worker。
2. 没有健康 decode worker。
3. Transfer backend 不可用。
4. Decode KV cache 全局接近满。
5. 请求 prompt 超过限制。
6. 请求 max_tokens 超过租户配额。
7. 当前 pending transfer 过多。
8. 模型版本没有匹配的 P/D worker。

拒绝请求比把请求塞进系统然后超时更好。

因为后者会浪费 prefill 计算和 KV 资源。

37.40 面试官会怎么问

问题一：请画一下 PD 分离的系统架构。

回答要点：client 到 router，router 分别管理 prefill pool 和 decode pool；prefill worker 生成 prompt KV；KV transfer backend 把 KV 交给 decode worker；decode worker 做持续生成和 streaming；router 负责路由、状态、健康检查和失败清理。

问题二：PD router 和普通 load balancer 有什么区别？

回答要点：普通 LB 只选一个后端；PD router 要分别选择 prefill/decode worker，协调 KV transfer，维护 request state，感知 P/D 负载、KV locality、decode slot、模型版本和故障清理。

问题三：为什么 KV transfer backend 是核心？

回答要点：因为 prefill 产生的 KV cache 必须被 decode 使用，KV 体积大、在 GPU memory 中、layout 和 TP/DP 切分敏感。传输成本决定 PD 分离收益是否能覆盖系统复杂度。

问题四：PD 分离中最难的故障是什么？

回答要点：KV transfer 失败和 decode 中途失败最麻烦。前者涉及两侧资源清理和可能重新 prefill；后者可能已经向客户端输出 token，很难无感迁移。

问题五：如何判断 PD 架构是否设计合理？

回答要点：要看分阶段指标，包括 prefill queue、prefill compute、decode assignment、KV transfer、first decode、TPOT、transfer timeout、KV cache utilization，而不是只看总 QPS。

37.41 标准回答模板

如果面试官问 “PD 分离的系统架构是什么样”，可以这样回答：

PD 分离架构通常由 router、prefill worker pool、decode worker pool 和 KV transfer backend 组成。Router 接收请求后，不是简单

Prefill worker 负责处理 prompt，执行 prefill forward，生成每层 KV cache，并准备 KV metadata。Decode worker 负责预留 decode

中间最关键的是 KV transfer backend。因为 prefill 和 decode 不在同一个 worker 上，prompt KV 必须从 prefill side 迁移或共享。

所以 PD 分离本质上是把单 engine 内部的 WAITING -> PREFILLING -> DECODING 状态机，升级成跨 router、prefill worker、transfer

37.42 架构状态机、清理门禁和可运行 demo

PD 架构图容易画，难的是证明它的状态机和资源清理真的闭环。一个请求跨 router、prefill worker、decode worker 和 transfer backend 后，任何一步失败都可能留下 KV blocks、decode slot、transfer session 或 stream 状态。

可以把请求抽象成：

$R_i = (r_i, P_i, O_i, m_i, s_i)$

其中 r_i 是 request id， P_i 是 prompt token 数， O_i 是最大输出 token 数， m_i 是模型版本， s_i 是客户端 stream 状态。

Router 的 worker 选择至少要满足：

$G_{\{\mathrm{route}\},i} = H_{\{\mathrm{prefill}\}} H_{\{\mathrm{decode}\}} M_{\{\mathrm{prefill}\}} M_{\{\mathrm{decode}\}} C_{\{\mathrm{decode}\}}$

含义是：prefill / decode worker 都健康，模型版本匹配，decode worker 有足够 KV / slot 容量，transfer backend 可用。任何一个条件不满足，router 都不应该盲目启动 prefill。

KV transfer metadata 可以抽象成：

$K_i = (r_i, w_i^{\{\mathrm{src}\}}, w_i^{\{\mathrm{dst}\}}, B_i^{\{\mathrm{src}\}}, B_i^{\{\mathrm{dst}\}}, L_i, d_i)$

其中 source / destination worker、source / destination block、序列长度、dtype / layout 都必须一致。metadata 缺失比 transfer 慢更危险，因为它可能导致 silent wrong output。

一个 PD 请求的 TTFT 可以拆成：

$T_{\{\mathrm{ttft}\}} = T_{\{\mathrm{router}\}} + T_{\{\mathrm{pq}\}} + T_{\{\mathrm{prefill}\}} + T_{\{\mathrm{reserve}\}} + T_{\{\mathrm{transfer}\}}$

最终架构门禁可以写成：

$G_{\{\mathrm{pdarch}\}} = G_{\{\mathrm{route}\}} G_{\{\mathrm{reserve}\}} G_{\{\mathrm{meta}\}} G_{\{\mathrm{transfer}\}} G_{\{\mathrm{abort}\}} G_{\{\mathrm{stream}\}}$

下面的 demo 用纯 Python 模拟三条路径：

1. 正常请求：router 选择 P/D worker，decode 先预留，prefill 生成 KV metadata，transfer 成功，decode streaming 完成。
2. transfer 失败请求：transfer 失败后清理 prefill side 和 decode side。
3. client abort 请求：decode 已预留但 prefill 尚未开始时取消，必须释放预留 slot。

```
from dataclasses import dataclass
```

```
@dataclass
class Worker:
```

```

name: str
role: str
model_version: str
zone: str
healthy: bool = True
queued_tokens: int = 0
running_sequences: int = 0
kv_free_tokens: int = 0

```

@dataclass

```

class PDRequest:
    request_id: str
    prompt_tokens: int
    output_tokens: int
    model_version: str
    fail_transfer: bool = False
    abort_after_reserve: bool = False

```

```

class ToyPDArchitectureAuditor:
    def __init__(self, prefill_workers, decode_workers, kv_mib_per_token=0.015625):
        self.prefill_workers = prefill_workers
        self.decode_workers = decode_workers
        self.kv_mib_per_token = kv_mib_per_token
        self.cleanup_events = []
        self.transfer_sessions = []

    def _choose_prefill(self, request):
        candidates = [
            worker for worker in self.prefill_workers
            if worker.healthy and worker.model_version == request.model_version
        ]
        if not candidates:
            return None
        return min(candidates, key=lambda worker: worker.queued_tokens)

    def _choose_decode(self, request):
        need_tokens = request.prompt_tokens + request.output_tokens
        candidates = [
            worker for worker in self.decode_workers
            if worker.healthy
            and worker.model_version == request.model_version
            and worker.kv_free_tokens >= need_tokens
        ]
        if not candidates:
            return None
        return min(candidates, key=lambda worker: (worker.running_sequences, -worker.kv_free_tokens))

    def _transfer_rate(self, prefill, decode):
        return 1200.0 if prefill.zone == decode.zone else 300.0

    def _cleanup(self, request, prefill=None, decode=None, reason="unknown"):
        if prefill:
            prefill.queued_tokens = max(0, prefill.queued_tokens - request.prompt_tokens)

```

```

if decode:
    decode.running_sequences = max(0, decode.running_sequences - 1)
    decode.kv_free_tokens += request.prompt_tokens + request.output_tokens
self.cleanup_events.append({"request_id": request.request_id, "reason": reason})

def process(self, requests):
    rows = []
    max_decode_running = 0
    stream_events = 0
    total_transfer_mib = 0.0
    for request in requests:
        states = ["RECEIVED"]
        prefill = self._choose_prefill(request)
        decode = self._choose_decode(request)
        if prefill is None or decode is None:
            rows.append({
                "request_id": request.request_id,
                "status": "ROUTE_FAILED",
                "states": states,
            })
            continue

        states.extend(["PREFILL_ASSIGNED", "DECODE_RESERVED"])
        decode.running_sequences += 1
        decode.kv_free_tokens -= request.prompt_tokens + request.output_tokens
        max_decode_running = max(max_decode_running, decode.running_sequences)

        if request.abort_after_reserve:
            states.append("CLIENT_ABORTED")
            self._cleanup(request, decode=decode, reason="client_abort_after_reserve")
            rows.append({
                "request_id": request.request_id,
                "status": "CLIENT_ABORTED",
                "prefill": prefill.name,
                "decode": decode.name,
                "states": states,
                "cleanup": True,
            })
            continue

        prefill.queued_tokens += request.prompt_tokens
        states.extend(["PREFILL_RUNNING", "PREFILL_DONE", "KV_TRANSFERRING"])
        kv_mib = request.prompt_tokens * self.kv_mib_per_token
        transfer_ms = kv_mib / self._transfer_rate(prefill, decode)
        session = {
            "request_id": request.request_id,
            "source": prefill.name,
            "target": decode.name,
            "source_blocks": (request.prompt_tokens + 127) // 128,
            "target_reserved_tokens": request.prompt_tokens + request.output_tokens,
            "kv_mib": round(kv_mib, 1),
            "transfer_ms": round(transfer_ms, 3),
        }
        total_transfer_mib += kv_mib
        self.transfer_sessions.append(session)

```



```

if request.fail_transfer:
    states.append("KV_TRANSFER_FAILED")
    self._cleanup(request, prefill=prefill, decode=decode, reason="kv_transfer_failed")
    rows.append({
        "request_id": request.request_id,
        "status": "KV_TRANSFER_FAILED",
        "prefill": prefill.name,
        "decode": decode.name,
        "states": states,
        "kv_metadata": session,
        "cleanup": True,
    })
    continue

states.extend(["DECODE_READY", "DECODING", "FINISHED"])
stream_count = min(4, max(1, request.output_tokens // 100))
stream_events += stream_count
self._cleanup(request, prefill=prefill, decode=decode, reason="finished")
rows.append({
    "request_id": request.request_id,
    "status": "FINISHED",
    "prefill": prefill.name,
    "decode": decode.name,
    "states": states,
    "kv_metadata": session,
    "stream_events": stream_count,
    "cleanup": True,
})

summary = {
    "requests": len(rows),
    "finished": sum(row["status"] == "FINISHED" for row in rows),
    "client_aborted": sum(row["status"] == "CLIENT_ABORTED" for row in rows),
    "transfer_failed": sum(row["status"] == "KV_TRANSFER_FAILED" for row in rows),
    "transfer_sessions": len(self.transfer_sessions),
    "total_transfer_mib": round(total_transfer_mib, 1),
    "cleanup_events": len(self.cleanup_events),
    "stream_events": stream_events,
    "max_decode_running": max_decode_running,
}

gates = {
    "router_selects_phase_workers": all(
        row.get("prefill") and row.get("decode")
        for row in rows if row["status"] != "ROUTE_FAILED"
    ),
    "decode_reserved_before_transfer": all(
        "DECODE_RESERVED" in row["states"]
        and row["states"].index("DECODE_RESERVED") < row["states"].index("KV_TRANSFERRING")
        for row in rows if "KV_TRANSFERRING" in row["states"]
    ),
    "kv_metadata_complete": all(
        {
            "request_id",
            "source",

```

```

        "target",
        "source_blocks",
        "target_reserved_tokens",
        "kv_mib",
    }.issubset(row["kv_metadata"])
    for row in rows if "kv_metadata" in row
),
"transfer_failure_cleanup": any(
    row["status"] == "KV_TRANSFER_FAILED" and row.get("cleanup")
    for row in rows
),
"client_abort_cleanup": any(
    row["status"] == "CLIENT_ABORTED" and row.get("cleanup")
    for row in rows
),
"model_version_checked": all(
    row.get("status") == "ROUTE_FAILED" or row.get("prefill") != "prefill_v2"
    for row in rows
),
"metrics_ready": summary["transfer_sessions"] == 2
and summary["cleanup_events"] == 3,
}
gates["pd_architecture_gate"] = all(gates.values())
return rows, summary, gates

```

```

prefill_workers = [
    Worker("prefill_a", "prefill", "v1", "rack_a", queued_tokens=5000),
    Worker("prefill_b", "prefill", "v1", "rack_b", queued_tokens=1000),
    Worker("prefill_v2", "prefill", "v2", "rack_a", queued_tokens=0),
]
decode_workers = [
    Worker("decode_a", "decode", "v1", "rack_a", running_sequences=22, kv_free_tokens=40000),
    Worker("decode_b", "decode", "v1", "rack_b", running_sequences=6, kv_free_tokens=18000),
    Worker("decode_bad", "decode", "v1", "rack_b", healthy=False, kv_free_tokens=90000),
]
requests = [
    PDRequest("req_ok", prompt_tokens=8000, output_tokens=300, model_version="v1"),
    PDRequest("req_transfer_fail", prompt_tokens=16000, output_tokens=120, model_version="v1", fail_transfer=True),
    PDRequest("req_abort", prompt_tokens=1200, output_tokens=400, model_version="v1", abort_after_reserve=True),
]

rows, summary, gates = ToyPDArchitectureAuditor(prefill_workers, decode_workers).process(requests)
print("pd_architecture_rows=", rows)
print("pd_architecture_summary=", summary)
print("pd_architecture_gates=", gates)

```

一次运行的核心输出类似：

```

pd_architecture_summary= {'requests': 3, 'finished': 1, 'client_aborted': 1, 'transfer_failed': 1, 'transfer_sessions': 2, 'cleanup_events': 3}
pd_architecture_gates= {'router_selects_phase_workers': True, 'decode_reserved_before_transfer': True, 'kv_metadata_complete': True}

```

这个 demo 的关键不是数值大小，而是架构证据：

1. decode_reserved_before_transfer=True 说明 decode worker 先预留目标 KV / slot，再让 P 侧开始安全 transfer。
2. kv_metadata_complete=True 说明 transfer 不只是 “传一坨 tensor”，而是带 request id、source、target、

source blocks、target reserved tokens 和 transfer metrics。

3. `transfer_failure_cleanup=True` 说明 transfer 失败会清理 P/D 两侧资源，不让预留 slot 或临时 KV 泄漏。
4. `client_abort_cleanup=True` 说明 client 取消不能只清理 router，还要释放 decode reserve。
5. `model_version_checked=True` 说明 router 没有把 v1 请求误发给 `prefill_v2` 这类不兼容 worker。

所以本章最终门禁是 `pd_architecture_gate`：只有 router、reserve、metadata、transfer failure、client abort、version check 和 metrics 都能闭环，PD 分离才不是一张架构图，而是可运行、可清理、可观测的分布式状态机。

37.43 小练习

1. 画出一个包含 router、prefill pool、decode pool 和 KV transfer backend 的 PD 架构图。
2. 写出一个请求从 client 到首 token 返回的完整时间线。
3. 列出 PD router 必须维护的 10 个 metadata 字段。
4. 解释为什么 decode worker 通常要先预留 KV blocks。
5. 对比 push KV 和 pull KV 的优缺点。
6. 设计 5 个用于观测 PD 分离系统的关键指标。
7. 说明 prefill worker 失败、KV transfer 失败、decode worker 失败分别如何处理。
8. 解释 PD 分离和 pipeline parallel 的区别。

37.44 本章总结

PD 分离的系统架构不是把一个 engine 拆成两个函数，而是把请求生命周期拆成跨组件协作。

它的核心组件是 router、prefill worker、decode worker 和 KV transfer backend。

Router 负责阶段感知路由和状态协调；prefill worker 负责 prompt 计算和 KV 生成；decode worker 负责持续生成和 streaming；KV transfer backend 负责把最重、最敏感的 KV cache 从 P 侧交给 D 侧。

PD 分离的主要收益来自 P/D 资源隔离、独立扩缩容和更稳定的 decode latency。

它的主要复杂度来自 KV transfer、metadata、跨组件状态机、故障恢复和观测。

下一章会继续深入最关键的问题：KV Cache 迁移、共享和路由。

第 38 章 KV Cache 迁移、共享和路由

上一章讲了 PD 分离的系统架构：router、prefill worker、decode worker 和 KV transfer backend 共同组成一个跨组件状态机。

本章深入最关键的问题：KV Cache 如何从 Prefill 侧交给 Decode 侧。

PD 分离真正难的地方，不是把请求分给两个 worker，而是把 Prefill 阶段生成的大量 KV cache，以正确、快速、可恢复的方式交给 Decode 阶段使用。

一句话概括：

KV Cache 迁移、共享和路由是 PD 分离能否落地的核心。它决定了 TTFT 是否真的下降，TPOT 是否稳定，以及系统复杂度是否可控。

38.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 `WRITING_PLAN.md` 对公开资料做校准：参考 SGLang PD Disaggregation 文档对 KV transfer backend、Mooncake / NIXL、bootstrap port、heterogeneous TP staging buffer、PD multiplexing 和 routing policy 的公开口径；参考 vLLM disaggregated prefilling 文档对 KV connector、prefill / decode 示例和实验性边界的说明；参考 Mooncake 论文对 KV cache-centric disaggregated architecture、KV cache 管理和传输瓶颈的系统动机说明；并参考 PagedAttention / vLLM 资料对 paged KV blocks、block table 和 KV cache metadata 的稳定抽象。

本讲只讲 KV cache 迁移、共享和路由的通用系统问题：KV 大小估算、paged block metadata、layout / dtype / TP 兼容、push / pull / remote read、decode reservation、KV-aware routing、prefix-aware routing、失败清理和多租户隔离。不把某个框架版本的真实 connector 字段、RDMA/NIXL/Mooncake API、CUDA IPC 细节、网络拓扑参数或远端 KV cache 产品实现写成通用标准。

本讲新增 demo 是教学版 KV transfer router：用 0 依赖 Python 模拟 KV metadata 兼容性检查、decode capacity 预留、路由 cost score、prefix cache 命中、多租户缓存阻断、短 prompt recompute 路径和 transfer 失败清理计划，帮助把 “KV-aware routing” 从口号落到可运行证据。

38.1 本章目标

读完本章，你应该能讲清：

1. 为什么 KV Cache 是 PD 分离里的核心数据。
2. KV Cache 的大小、layout 和 metadata 为什么重要。
3. KV Cache 迁移有哪些基本模式。
4. push、pull、共享内存、远端 KV cache 的区别。
5. Decode worker 为什么通常要先预留 KV blocks。
6. KV-aware routing 和 prefix-aware routing 怎么工作。
7. KV transfer 失败时应该如何清理和重试。
8. 面试中如何回答 “PD 分离的 KV Cache 怎么传”。

38.2 先回忆 KV Cache 是什么

Transformer 自回归生成时，每一层 attention 都会产生 Key 和 Value。

对于已经处理过的 token，后续 decode 不需要重新计算它们的 K/V，而是直接读取缓存。

这就是 KV Cache。

简化理解：

```
prompt tokens
-> prefill forward
-> per-layer K/V tensors
-> KV cache
-> decode reads KV cache repeatedly
```

在 unified engine 中，KV cache 通常留在同一个 worker 的 GPU memory 里。

```
same worker:
  prefill writes KV
  decode reads KV
```

PD 分离后，Prefill 和 Decode 不在同一个 worker 上。

```
prefill worker:
  prefill writes KV
```

```
decode worker:
  decode needs KV
```

中间就出现了问题：

How to move or share KV?

这就是本章主题。

38.3 为什么 KV Cache 不是普通数据

KV Cache 和普通 RPC payload 不一样。

普通 RPC 传的可能有：

1. prompt 文本。
2. token ids。
3. request metadata。
4. 采样参数。
5. JSON 或 protobuf。

KV Cache 的特点是：

1. 体积大。
2. 在 GPU memory 中。
3. 每一层都有。
4. 和模型结构强相关。
5. 和 dtype 强相关。
6. 和 tensor parallel 切分方式强相关。
7. 和 paged KV block layout 强相关。
8. 需要被 Decode 高频读取。

所以不能把它简单理解成 “把一个数组传过去”。

它更像是一个带有复杂物理布局和所有权语义的分布式内存对象。

38.4 KV Cache 大小估算

理解 KV 迁移，必须先理解它为什么贵。

对于一个 Transformer 模型，KV cache 大小大致和下面因素成正比：

1. 层数 `num_layers`。
2. KV head 数 `num_kv_heads`。
3. head dimension `head_dim`。
4. token 数 `seq_len`。
5. dtype 字节数。
6. batch 中请求数量。

单个 token 的 KV 大小可以粗略写成：

```
per_token_kv_bytes
= num_layers * 2 * num_kv_heads * head_dim * dtype_bytes
```

这里的 2 表示 K 和 V。

如果是 32 层、32 个 KV heads、head_dim 128、FP16：

```
32 * 2 * 32 * 128 * 2 bytes
= 524288 bytes
≈ 512 KB / token
```

如果 prompt 有 4000 tokens：

```
512 KB * 4000
≈ 2 GB
```

实际模型可能使用 GQA/MQA，`num_kv_heads` 会小一些，KV cache 也会小一些。

但结论不变：长 prompt 的 KV cache 可能非常大。

这也是为什么 PD 分离不能只看计算时间，还必须看 KV transfer 成本。

38.5 KV Cache 迁移的基本问题

Prefill worker 生成 KV 后，Decode worker 要使用它。

这件事至少包含 8 个问题：

1. 源 KV 在哪里。
2. 目标 worker 是谁。
3. 目标 worker 是否有足够 KV capacity。
4. 源和目标的 KV layout 是否一致。
5. 源和目标的 TP rank mapping 是否一致。
6. 传输走什么 backend。
7. 传输失败如何清理。
8. 传完后谁拥有 KV 的生命周期。

因此，KV transfer 不是一个单纯的数据复制动作。

它需要 router、prefill worker、decode worker 和 transfer backend 共同协作。

38.6 一个最小 KV Transfer 流程

最小流程可以画成：

```
client
-> router
-> prefill worker
    run prefill
    produce KV
-> router chooses decode worker
-> decode worker reserves KV blocks
-> transfer KV from prefill to decode
-> decode worker starts decode
-> stream tokens
```

注意一个关键点：

Decode worker 通常要先预留目标 KV blocks。

原因是 transfer 不能把 KV 传到一个 “不知道放哪” 的地方。

目标侧需要提前告诉 transfer backend：

1. request id。
2. 目标 block ids。
3. 每层目标地址。
4. 每个 TP rank 的目标地址。
5. 可接收的 dtype 和 layout。

然后源侧才能把 KV 写过去或让目标侧拉取。

38.7 Push 模式

Push 模式是指：Prefill worker 主动把 KV 推给 Decode worker。

流程：

```
router -> choose prefill worker
router -> choose decode worker
decode worker -> reserve destination blocks
prefill worker -> run prefill
prefill worker -> push KV to decode worker
decode worker -> mark KV ready
decode worker -> start decode
```

优点：

- 1. 流程直观。
- 2. Prefill 结束后可以立即发起传输。
- 3. Router 容易理解 transfer 进度。
- 4. 适合源侧更主动的实现。

缺点：

- 1. Prefill worker 需要知道目标地址或目标 block mapping。
- 2. 源侧逻辑更复杂。
- 3. 如果 decode worker 已经变化，源侧需要处理目标失效。
- 4. 多租户和权限隔离更麻烦。

Push 模式适合系统控制面比较集中、P/D worker 通信关系明确的场景。

38.8 Pull 模式

Pull 模式是指：Decode worker 主动从 Prefill worker 拉取 KV。

流程：

```
router -> choose prefill worker
prefill worker -> run prefill
prefill worker -> expose source KV metadata
router -> choose decode worker
decode worker -> reserve destination blocks
decode worker -> pull KV from prefill worker
decode worker -> mark KV ready
decode worker -> start decode
```

优点：

- 1. Decode worker 掌握目标内存分配。
- 2. 更符合“消费方拉取”的语义。
- 3. Decode worker 可以根据本地状态决定何时拉取。
- 4. 对目标侧资源控制更自然。

缺点：

- 1. Decode worker 需要访问源侧 KV metadata。
- 2. 源侧要在一段时间内保留 KV。
- 3. 如果 pull 延迟，prefill worker 的 KV memory 会被占住。
- 4. Router 要管理 source KV 的过期和清理。

Pull 模式适合 decode 侧资源控制更复杂的场景。

38.9 Push 和 Pull 的本质区别

Push 和 Pull 的区别不只是方向不同。

它们背后是所有权和状态机不同。

维度	Push	Pull
谁发起数据移动	Prefill worker	Decode worker
谁更复杂	Prefill 侧	Decode 侧
目标 block 何时确定	传输前必须确定	拉取前必须确定
源 KV 保留时间	通常较短	可能更长
失败清理重点	目标写入失败	源 KV 过期和拉取失败
适合场景	控制面集中	消费方资源控制强

生产系统中也可能混合使用：控制面看起来是 push，底层传输实现却是由目标侧 RDMA read 完成。

所以面试时不要死记 “push 一定是源写，pull 一定是目标读”。

更准确的说法是：

Push/Pull 描述的是控制语义；底层数据面可能由 RDMA write、RDMA read、GPU IPC、NIXL 或其他 backend 完成。

38.10 共享模式

除了迁移，还有一种思路是共享。

共享模式下，Prefill 产生的 KV 不一定复制到 Decode worker 的本地 KV cache，而是让 Decode worker 以某种方式直接访问同一份 KV。

可能的共享方式包括：

1. 同机多进程 GPU IPC。
2. NVLink 互联下的 peer access。
3. RDMA 暴露远端 GPU memory。
4. 独立 KV cache server。
5. 分层 KV cache 系统。

共享的优点：

1. 可以减少复制。
2. 可以提升 prefix cache 复用。
3. 对长 prompt 可能更省带宽。
4. 可以把 KV cache 做成系统级资源。

共享的缺点：

1. 远端读取延迟可能影响 decode TPOT。
2. 一致性和生命周期更复杂。
3. 故障边界变大。
4. 调试和观测更困难。
5. 对网络和硬件拓扑更敏感。

因此，共享不是天然优于迁移。

如果 decode 每一步都要频繁读取远端 KV，而远端读取延迟不可控，TPOT 可能变差。

38.11 迁移模式和共享模式的区别

迁移模式：

Prefill worker GPU memory
-> copy KV
-> Decode worker GPU memory

Decode 后续主要读本地 KV。

共享模式：

Prefill side / remote cache keeps KV
-> Decode worker reads shared KV

Decode 后续可能读远端 KV，或者部分读本地、部分读远端。

对比：

维度	迁移	共享
数据复制	有	少或无
Decode 读取	本地为主	可能远端
TPOT 稳定性	通常更好	取决于远端访问
TTFT	受复制时间影响	受远端访问准备影响
Prefix 复用	需要额外机制	更自然
生命周期	相对清晰	更复杂

一个简单判断:

如果请求生成很长, decode 会反复读 KV, 那么把 KV 放到 decode 本地通常更稳。

如果大量请求共享长 prefix, 共享或远端 KV cache 的收益可能更大。

38.12 Paged KV Cache 对迁移的影响

现代 serving engine 通常不会为每个请求分配连续大块 KV memory。

它们会使用 paged KV cache。

也就是把 KV cache 切成固定大小的 block。

request A tokens:

block 7 -> block 9 -> block 21

request B tokens:

block 3 -> block 4

这样可以减少内存碎片, 并支持更灵活的调度和复用。

但它也让 KV transfer 更复杂。

因为要传的不只是一个连续 tensor, 而是一组 block mapping:

1. 第几个 token 在哪个 block。
2. block 内 offset 是多少。
3. 每层 K/V 的物理地址。
4. 每个 TP rank 的 block 对应关系。
5. source block 和 destination block 的映射。

所以 KV transfer metadata 至少要包含:

```
request_id
model_id
model_version
num_layers
kv_dtype
block_size
seq_len
source_worker_id
source_rank_ids
source_block_ids
destination_worker_id
destination_rank_ids
destination_block_ids
layout_version
```

没有这些 metadata, decode worker 即使拿到了 bytes, 也不知道怎么解释。

38.13 Layout 兼容性

KV layout 描述的是 K/V 在内存中的排列方式。

不同 engine 或不同 kernel 可能使用不同 layout。

例如：

[layer][block][head][token][dim]

或者：

[block][layer][kv][head][token][dim]

或者为了 kernel 访问效率做了更复杂的打包。

如果 prefill side 和 decode side layout 不一致，就会出现两个问题：

1. 传过去不能直接用。
2. 需要 layout conversion。

layout conversion 可能发生在：

1. Prefill worker 传输前。
2. Transfer backend 中。
3. Decode worker 接收后。

但无论在哪做，它都会引入额外成本。

所以生产系统里通常希望 P/D worker 使用一致的：

1. 模型版本。
2. dtype。
3. TP/DP 配置。
4. KV block size。
5. KV layout version。
6. attention kernel 期望的内存格式。

否则 KV transfer 会从“搬运”变成“搬运加转换”。

38.14 Tensor Parallel 下的 KV 迁移

Tensor Parallel 会把模型参数和计算切到多个 GPU rank 上。

KV cache 也会跟着切分。

例如一个 TP=4 的 worker：

prefill worker P：

- rank 0 owns part of KV
- rank 1 owns part of KV
- rank 2 owns part of KV
- rank 3 owns part of KV

Decode worker 也是 TP=4：

decode worker D：

- rank 0 needs corresponding KV
- rank 1 needs corresponding KV
- rank 2 needs corresponding KV
- rank 3 needs corresponding KV

如果 P 和 D 的 TP 配置一致，mapping 相对简单：

P rank 0 -> D rank 0
P rank 1 -> D rank 1
P rank 2 -> D rank 2
P rank 3 -> D rank 3

如果 TP 配置不一致，问题会复杂很多。

例如 P 是 TP=8，D 是 TP=4。

这时 KV shard 可能需要重分片。

P rank 0 + P rank 1 -> D rank 0
P rank 2 + P rank 3 -> D rank 1
...

重分片意味着：

1. 更多网络传输。
2. 更多内存拷贝。
3. 更复杂的 metadata。
4. 更高的失败概率。
5. 更难保证性能稳定。

所以实际系统中，P/D pool 通常会尽量保持兼容的并行配置。

异构不是不能做，但必须把 KV reshaping 的成本算进去。

38.15 Decode 为什么要先预留 KV Blocks

这是 PD 分离面试里很常见的问题。

Decode worker 在接收 KV 前通常要先预留 KV blocks，原因有四个。

第一，确认容量。

如果 decode worker 没有足够 KV capacity，传输到一半才发现放不下，会浪费 prefill 计算和网络带宽。

第二，确定目标地址。

KV transfer 需要明确写入位置。

不管是 RDMA write、GPU copy，还是目标侧 pull，都需要目标 buffer。

第三，建立 block mapping。

Decode 的 paged attention 需要知道每个 request 的 block table。

传输前就要知道：

source block 10 -> destination block 81
source block 11 -> destination block 82
source block 20 -> destination block 83

第四，避免 race condition。

如果多个请求同时进入 decode worker，没有 reservation，就可能发生资源竞争。

所以正确流程通常是：

decode worker checks capacity
decode worker reserves blocks
decode worker returns destination metadata
KV transfer starts
decode worker marks request ready

38.16 KV Transfer Backend 做什么

KV transfer backend 是数据面能力。

它不一定是一个独立服务，也可能是 worker 内部的一组库和通信线程。

它负责：

1. 注册 GPU memory。
2. 建立连接。
3. 暴露 source buffer。
4. 暴露 destination buffer。
5. 执行 copy、read 或 write。
6. 处理 completion event。
7. 上报 timeout 和 error。
8. 做必要的的数据校验。
9. 控制并发传输。
10. 清理传输资源。

常见底层能力包括：

1. CUDA IPC。
2. NCCL。
3. RDMA。
4. NVLink / NVSwitch。
5. GPUDirect RDMA。
6. NIXL。
7. Mooncake 等 KV transfer / KV cache 系统。

面试中不需要把每种 backend 的 API 细节背下来。

但要讲清楚：

Transfer backend 的价值是把 KV cache 从 “worker 内部内存” 变成 “跨 worker 可移动或可访问的数据对象”。

38.17 Router 在 KV 迁移中的职责

Router 不直接搬 KV，但它必须理解 KV。

Router 至少要维护：

1. request id。
2. tenant id。
3. model id。
4. model version。
5. tokenizer version。
6. prompt length。
7. expected output length。
8. prefill worker id。
9. decode worker id。
10. source KV metadata。
11. destination KV metadata。
12. transfer state。
13. timeout deadline。
14. retry count。
15. cleanup owner。

Router 的关键职责不是 “转发请求”，而是协调状态：

WAITING

-> PREFILL_ASSIGNED

-> PREFILL_RUNNING
-> PREFILL_DONE
-> DECODE_RESERVED
-> KV_TRANSFERRING
-> KV_READY
-> DECODING
-> FINISHED

如果任何一步失败，router 要知道该释放哪些资源。

例如：

PREFILL_DONE but KV_TRANSFERRING failed

需要清理：

1. Prefill side source KV。
2. Decode side reserved blocks。
3. Transfer backend connection / handle。
4. Request state。
5. Client response。

38.18 KV-aware Routing

普通路由可能只看 worker 负载。

例如：

choose least loaded worker

KV-aware routing 要看更多信息。

对于 decode worker，router 需要关注：

1. 剩余 KV capacity。
2. running request 数。
3. decode batch size。
4. 当前 TPOT。
5. pending transfer 数。
6. 网络拓扑距离。
7. 是否已有相关 prefix KV。
8. 模型版本是否匹配。

一个简单的 score 可以写成：

```
score(decode_worker)
= a * available_kv_blocks
- b * running_requests
- c * pending_transfers
- d * network_cost(prefill_worker, decode_worker)
- e * current_tpot
```

真实系统不会这么简单，但思路类似。

Router 不能只问 “哪个 worker 空”。

它还要问：

哪个 worker 能低成本接住这个请求的 KV，并稳定完成 decode？

38.19 Prefix-aware Routing

很多请求会共享相同前缀。

例如：

1. 同一个 system prompt。
2. 同一个 RAG 模板。
3. 同一段长文档上下文。
4. 同一个 agent 工具说明。
5. 同一个多轮对话历史前缀。

如果某个 worker 已经缓存了这段 prefix 的 KV，新请求路由到它，可能可以复用 KV，减少 prefill。

这叫 prefix-aware routing 或 cache-aware routing。

示例：

request A prompt:

[system prompt][doc][question 1]

request B prompt:

[system prompt][doc][question 2]

二者共享：

[system prompt][doc]

如果共享 prefix 很长，复用收益很大。

Router 可以维护 prefix cache index：

prefix_hash -> worker ids that hold KV

路由时优先选择已经有 prefix KV 的 worker。

但这也有代价：

1. cache index 需要更新。
2. prefix hash 要考虑 tokenizer 和 model version。
3. 热点 prefix 可能导致 worker 倾斜。
4. prefix KV 生命周期要管理。
5. 多租户之间不能随意共享。

所以 prefix-aware routing 不是简单地“命中缓存就去那个 worker”。

还要平衡 load、KV capacity 和 tail latency。

38.20 Prefix Cache 和 PD 分离的关系

Prefix cache 在 unified engine 中已经有价值。

PD 分离后，它更重要，也更复杂。

原因是：

1. Prefill 成本集中在 prefill pool。
2. Decode worker 需要 prompt KV 才能生成。
3. Prefix KV 可能在 P 侧、D 侧或远端 cache 中。
4. Router 需要知道 KV locality。

几种情况：

case 1: prefix KV already on selected decode worker
-> skip or reduce transfer

case 2: prefix KV on another decode worker
-> decide migrate, remote read, or recompute

case 3: prefix KV on prefill worker
-> maybe reuse prefill side KV

case 4: prefix KV in remote KV cache
-> fetch or attach

case 5: no prefix KV
-> run full prefill

这说明 PD router 需要理解 locality。
没有 locality 信息的 router，很难发挥 KV cache 复用的价值。

38.21 Recompute、Migrate、Remote Read 的取舍

当 KV 不在目标 decode worker 上时，系统有三种选择。

- 第一，重新计算。
run prefill again on selected worker
 - 第二，迁移 KV。
copy KV to selected decode worker
 - 第三，远端读取。
decode reads KV from remote cache or remote worker
- 取舍可以这样看：

方案	优点	缺点
Recompute	实现简单，不依赖远端 KV	浪费计算，TTFT 高
Migrate	Decode 本地读，TPOT 稳	需要传输，TTFT 受影响
Remote Read	减少复制，可复用	Decode 受远端延迟影响

- 一个经验判断：
- 1. prompt 短，recompute 可能最简单。
 - 2. prompt 长、输出也长，migrate 通常更稳。
 - 3. prefix 很长且被大量请求共享，remote cache 可能值得。
 - 4. 网络很差时，远端读取会显著拖累 decode。

38.22 KV Cache 的生命周期

KV cache 生命周期必须清楚，否则系统很容易泄漏 GPU memory。
一个请求的 KV 生命周期大致是：

- allocate source blocks
 - > write prompt KV
 - > expose source metadata
 - > transfer or share
 - > release source blocks
- > decode appends new KV
- > finish / abort
- > release decode blocks

在 PD 分离中，source 和 destination 的生命周期可能不同。
Prefill side source KV：

1. Prefill 期间必须存在。
2. Transfer 完成前必须存在。
3. Transfer 完成后通常可以释放。
4. 如果做 prefix cache, 可能继续保留。

Decode side KV:

1. Transfer 前预留。
2. Transfer 完成后进入 ready。
3. Decode 期间持续追加。
4. 请求结束后释放。
5. 如果做 cache reuse, 部分 prefix blocks 可能保留。

生命周期管理的核心问题是:

谁负责释放? 什么时候释放? 失败时释放哪些?

38.23 Reference Count

如果 KV cache 支持共享, 就需要引用计数或类似机制。

例如多个请求共享同一个 prefix block:

```
prefix block X
  used by request A
  used by request B
  used by request C
```

这时不能在 request A 完成后就释放 block X。

需要:

```
ref_count(prefix block X) = 3
request A done -> ref_count = 2
request B done -> ref_count = 1
request C done -> ref_count = 0 -> can evict
```

生产系统中还要考虑:

1. request abort。
2. timeout。
3. worker crash。
4. router state 丢失。
5. prefix cache eviction。
6. 多租户隔离。

引用计数不一定显式叫 ref_count, 但系统必须有等价的所有权管理。

38.24 KV Eviction 策略

KV cache 不是无限的。

当 GPU memory 不够时, 系统要决定驱逐哪些 KV。

常见策略:

1. LRU: 驱逐最近最少使用。
2. LFU: 驱逐使用频率低的。
3. TTL: 超过时间就驱逐。
4. Size-aware: 优先驱逐大对象。
5. Cost-aware: 保留重算成本高的 prefix。
6. Tenant-aware: 按租户配额驱逐。

对 LLM serving 来说，cost-aware 很重要。
因为一个很长的 system prompt 或文档 prefix，虽然最近用得不多，但重算成本很高。
而一个短 prefix 即使命中率高，收益也可能不大。
因此更合理的 eviction score 可能同时考虑：

```
eviction_score
= recompute_cost
  * expected_reuse_probability
  / memory_size
```

实际系统会更复杂，但核心思想是：
KV cache eviction 不只是内存问题，也是调度和成本问题。

38.25 多级 KV Cache

KV cache 可以分层。
例如：

```
GPU KV cache
-> CPU memory
-> local SSD
-> remote KV cache service
```

不同层级的特点：

层级	延迟	容量	适合存什么
GPU	最低	最小	活跃 decode KV
CPU	中等	较大	暂时不活跃 prefix
SSD	较高	更大	低频长 prefix
Remote cache	取决于网络	可扩展	跨 worker 复用 prefix

多级 KV cache 可以提升复用率，但会引入新的问题：

- 1. Promote / demote 策略。
- 2. Prefetch 策略。
- 3. Eviction 策略。
- 4. 一致性管理。
- 5. 远端访问对 TPOT 的影响。

第 40 章会专门讨论多级 KV Cache。

本章只需要记住：

PD 分离让 KV 从单 worker 内部对象变成系统级资源，多级 KV cache 是这个方向的自然延伸。

38.26 KV Transfer 的性能指标

观测 KV transfer，不能只看总 QPS。
需要分阶段指标。

关键指标包括：

- 1. kv_transfer_bytes_total。
- 2. kv_transfer_latency_ms。
- 3. kv_transfer_bandwidth_gbps。
- 4. kv_transfer_pending_count。

5. kv_transfer_timeout_count。
6. kv_transfer_failure_count。
7. source_kv_retention_ms。
8. decode_reserved_blocks。
9. decode_kv_utilization。
10. prefix_cache_hit_rate。
11. prefix_cache_bytes。
12. remote_kv_read_latency_ms。
13. time_to_first_token_ms。
14. time_per_output_token_ms。

这些指标要结合看。

例如：

TTFT increases
and kv_transfer_latency increases
and pending_transfer_count increases

说明瓶颈可能在 KV transfer。

再例如：

TPOT increases
and remote_kv_read_latency increases

说明远端 KV 读取可能拖慢 decode。

38.27 KV Transfer 的常见瓶颈

常见瓶颈有：

1. 网络带宽不足。
2. RDMA 连接数过多。
3. GPU memory copy 和 compute 争抢。
4. KV layout conversion 太慢。
5. 目标 decode worker KV capacity 不足。
6. Pending transfer 队列过长。
7. 源 KV 保留太久导致 prefill worker 内存压力。
8. 小块 transfer 太多导致调度开销高。
9. 跨机架或跨可用区传输。
10. Prefix cache 命中导致热点 worker 过载。

一个典型问题是：

Prefill 很快，Decode 也不忙，但 TTFT 仍然高。

这时要检查中间是否卡在：

1. Decode block reservation。
2. KV metadata exchange。
3. KV transfer queue。
4. Actual data movement。
5. Transfer completion notification。

38.28 故障一：Prefill 成功但 Transfer 失败

这是最典型的故障。

状态：

prefill done
source KV exists
decode blocks reserved
transfer failed

可能原因:

1. 网络 timeout。
2. Decode worker crash。
3. Source worker crash。
4. Destination buffer invalid。
5. Layout metadata mismatch。
6. RDMA connection reset。

处理策略:

1. 标记 request transfer failed。
2. 释放 decode reserved blocks。
3. 释放或保留 source KV。
4. 判断是否可以重试 transfer。
5. 如果不能重试, 重新 prefill 或返回错误。

能否重试取决于 source KV 是否还存在。

如果 source KV 还在, 可以选择新的 decode worker 再传一次。

如果 source KV 已经没了, 就只能重新 prefill。

38.29 故障二: Decode Reserved 后 Worker 崩溃

状态:

decode worker reserved blocks
before or during transfer
decode worker crashed

处理策略:

1. Router 标记 decode worker unhealthy。
2. 放弃该 worker 上的 reservation。
3. 如果 source KV 仍在, 选择新的 decode worker。
4. 重新 reserve blocks。
5. 重新 transfer。
6. 如果 source KV 不在, 重新 prefill。

关键点是:

reserved blocks on crashed worker are not reliable resources

不能假设它们之后还能恢复。

38.30 故障三: Decode 中途失败

Decode 已经开始输出 token 后再失败, 最麻烦。

状态:

client already received partial tokens
decode worker crashed

这时很难做到完全无感恢复。

原因是:

1. 已输出 token 需要保持一致。
2. 采样可能有随机性。
3. Decode 期间新生成 token 的 KV 也在 worker 上。
4. 其他 worker 不一定有完整 KV。
5. Streaming 协议可能已经把内容发给客户端。

可选策略：

1. 返回错误，要求客户端重试。
2. 保存 generated tokens，重新 prefill prompt + generated tokens。
3. 如果有 checkpointed KV，迁移到新 worker。
4. 对确定性采样场景尝试恢复。

生产系统通常会把 decode 中途失败视为难以无感恢复的故障。

面试中可以明确说：

Prefill 失败和 transfer 失败通常可以重试；decode 中途失败因为已经对外输出 token，恢复语义更复杂。

38.31 KV Metadata 不一致

KV metadata 不一致会导致严重问题。

例如：

1. 模型版本不一致。
2. dtype 不一致。
3. block size 不一致。
4. layout version 不一致。
5. TP rank mapping 不一致。
6. prompt tokenization 不一致。
7. RoPE scaling 配置不一致。

这些问题不一定会立即报错。

更糟糕的是，它们可能导致 silent correctness bug。

也就是模型还能输出，但输出语义是错的。

所以 transfer 前必须校验：

```
model_id matches
model_version matches
tokenizer_version matches
kv_dtype matches
layout_version matches
parallel_config matches
rope_config matches
```

宁可拒绝请求，也不要和不兼容的 KV 交给 decode。

38.32 安全和多租户隔离

KV cache 不是普通缓存。

它包含用户输入的语义信息。

在多租户系统中，KV cache 共享必须非常谨慎。

风险包括：

1. 租户 A 复用租户 B 的 prefix KV。
2. Prefix hash 碰撞导致错误共享。

3. Debug dump 泄露 KV metadata。
4. Remote KV cache 权限控制不严。
5. Worker 复用后没有清理旧 KV。

基本原则：

1. Prefix cache key 必须包含 tenant boundary。
2. 不同租户默认不共享 KV。
3. KV metadata 不能暴露给无关租户。
4. Worker 下线或复用时清理敏感 KV。
5. 远端 KV cache 要有访问控制。

面试中提到多租户隔离，会显得你对生产系统更敏感。

38.33 KV 路由策略示例

一个简化的 decode worker 选择策略：

```
for each decode worker:
    if model_version mismatch:
        skip
    if available_kv_blocks < required_blocks:
        skip
    if pending_transfers too high:
        penalize
    if prefix_cache_hit:
        reward
    if same_node_as_prefill:
        reward
    if current_tpot high:
        penalize
```

choose best score

对应伪代码：

```
def choose_decode_worker(request, prefill_worker, candidates):
    best_worker = None
    best_score = None

    for worker in candidates:
        if worker.model_version != request.model_version:
            continue
        if worker.available_kv_blocks < request.required_kv_blocks:
            continue

        score = 0
        score += worker.available_kv_blocks * 0.1
        score -= worker.running_requests * 1.0
        score -= worker.pending_transfers * 2.0
        score -= worker.current_tpot_ms * 0.5
        score -= network_cost(prefill_worker, worker)

        if worker.has_prefix(request.prefix_hash):
            score += request.prefix_reuse_score

        if best_score is None or score > best_score:
            best_score = score
```

```
best_worker = worker
```

```
return best_worker
```

这只是教学伪代码。

真实系统里 score 的权重要来自压测和线上观测。

38.34 什么时候不应该迁移 KV

不是所有请求都适合 KV 迁移。

不适合的情况包括：

1. Prompt 很短，重新计算比传输更便宜。
2. 网络拥塞严重，迁移会拖慢 TTFT。
3. Decode worker 本地 KV capacity 紧张。
4. P/D layout 不兼容，需要昂贵转换。
5. 请求预计只生成很短输出。
6. 源 KV 已经快过期或不可靠。
7. 跨机房传输成本过高。

这时可以选择：

1. 在同一个 worker 上完成请求。
2. 重新 prefill。
3. 路由到更近的 decode worker。
4. 降级为 unified scheduling。
5. 拒绝或排队。

优秀的 PD 系统应该能判断：

$\text{transfer benefit} > \text{transfer cost}$

而不是无脑迁移所有 KV。

38.35 和 Chunked Prefill 的关系

Chunked Prefill 是把长 prompt prefill 切成多个 chunk 执行。

它和 KV 迁移有关，但不是同一个问题。

Chunked Prefill 解决的是：

1. 长 prompt prefill 太大。
2. Prefill 阶段阻塞 decode。
3. 大 batch 形态不稳定。
4. TTFT 和 TPOT 之间需要折中。

KV 迁移解决的是：

1. Prefill 生成的 KV 如何给 Decode。
2. P/D worker 间如何共享状态。
3. Decode worker 如何接管请求。

二者可以结合。

例如：

```
long prompt
-> chunked prefill on prefill pool
-> progressively produce KV
-> transfer KV chunks
-> decode starts after enough KV ready
```

下一章会继续讲 Chunked Prefill 与 Disaggregated Prefill。

38.36 面试官会怎么问

问题一：PD 分离中 KV Cache 怎么从 Prefill 传到 Decode？

回答要点：Prefill worker 生成 prompt KV 和 metadata；router 选择 decode worker；decode worker 先检查 capacity 并预留目标 KV blocks；然后通过 transfer backend 以 push 或 pull 的方式把 KV 从 source blocks 迁移到 destination blocks；完成后 decode worker 标记 KV ready，进入 decode loop。

问题二：为什么 Decode worker 要先预留 KV blocks？

回答要点：为了确认容量、确定目标地址、建立 source 到 destination 的 block mapping，并避免并发请求抢占资源。否则传输到一半发现放不下，会浪费 prefill 计算和网络带宽。

问题三：Push 和 Pull 有什么区别？

回答要点：Push 是 Prefill 侧主动把 KV 交给 Decode 侧，Pull 是 Decode 侧根据 source metadata 主动拉取。区别主要是控制语义和状态所有权；底层实现可能仍然是 RDMA read/write 或其他 backend。

问题四：KV-aware routing 看哪些信息？

回答要点：看 decode worker 的 KV 剩余容量、running requests、pending transfers、TPOT、模型版本、网络拓扑、prefix cache 命中、P/D worker 距离等，不只是看 worker 是否空闲。

问题五：KV transfer 失败怎么处理？

回答要点：释放 decode reserved blocks，清理 transfer handle；如果 source KV 还在，可以换 decode worker 重试 transfer；如果 source KV 不在，可能需要重新 prefill；如果已经进入 decode 并对外输出 token，则无感恢复更困难。

问题六：为什么模型版本不一致不能复用 KV？

回答要点：KV cache 是模型内部状态，和权重、RoPE、tokenizer、dtype、layout、TP 配置都相关。版本不一致即使 shape 能对上，也可能产生 silent correctness bug。

38.37 标准回答模板

如果面试官问“PD 分离里的 KV Cache 迁移怎么设计”，可以这样回答：

PD 分离中，Prefill worker 处理 prompt 后会生成每层 KV cache。因为 Decode worker 不在同一个进程或同一个 GPU 上，所以必须把

一个典型流程是：router 先选择 prefill worker 执行 prefill；prefill 完成后生成 source KV metadata，包括 request id、seq len

这里最关键的是 metadata 和状态机。KV cache 不是普通 bytes，它和模型版本、KV layout、paged block table、dtype、tensor parallelism aware routing，考虑 decode worker 的剩余 KV blocks、pending transfers、TPOT、网络拓扑和 prefix cache locality。

故障处理也很重要。如果 transfer 失败，要释放 decode 侧预留 block，并根据 source KV 是否还存在决定重试 transfer 还是重新 prefill

38.38 KV Transfer 路由公式、隔离门禁和可运行 demo

KV transfer 和 routing 的核心不是“选择一个空闲 decode worker”，而是在兼容性、容量、拓扑、缓存命中、多租户隔离和 transfer 成本之间做取舍。

设一次 KV transfer 请求为：

$Q_i = (r_i, u_i, m_i, t_i, P_i, 0_i, h_i, L_i, d_i, p_i, z_i)$

其中 r_i 是 request id, u_i 是 tenant, m_i 是 model version, t_i 是 tokenizer version, P_i 是 prompt token 数, 0_i 是预计输出 token 数, h_i 是 prefix hash, L_i 是 layout version, d_i 是 KV dtype, p_i 是 TP size, z_i 是 source zone。

目标 decode worker w 必须先过兼容门：

$$G_{\{\mathrm{compat}\}}(i,w)=G_{\mathrm{m}}G_{\mathrm{t}}G_{\mathrm{d}}G_{\mathrm{LG}_p}$$

分别表示 model、tokenizer、dtype、layout 和 parallel config 兼容。任何一项不兼容，都不应该把 KV 交给这个 worker。

目标容量门可以写成：

$$B_{\{\mathrm{need}\},i}=\left\lceil\frac{P_{i+0_i}}{S_{\{\mathrm{block}\}}}\right\rceil$$

$$G_{\{\mathrm{cap}\}}(i,w)=\mathbf{1}[B_{\{\mathrm{free}\},w}\geq B_{\{\mathrm{need}\},i}]$$

KV transfer 成本粗估为：

$$T_{\{\mathrm{transfer}\}}(i,w)=\frac{P_{i\mathrm{m}_{\{\mathrm{kv}\}}}}{B_{\{\mathrm{net}\}}(z_i,z_w)}$$

一个教学版 decode worker score 可以写成：

$$S(i,w)=aB_{\{\mathrm{free}\},w}-bR_w-cQ_w-dT_{\{\mathrm{tpot}\},w}-eT_{\{\mathrm{transfer}\}}(i,w)+fC_{\{\mathrm{prefix}\}}(i,w)$$

其中 R_w 是 running sequences, Q_w 是 pending transfers, $T_{\{\mathrm{tpot}\},w}$ 是当前 TPOT, $C_{\{\mathrm{prefix}\}}$ 表示同租户可复用 prefix cache 的奖励。注意“同租户”很重要，跨租户 prefix 命中必须阻断，不能因为 hash 命中就复用。

下面的 demo 覆盖三类路径：

1. long_rag: 长 prompt 且同租户 prefix cache 命中，选择 cache 更有价值的 decode worker。
2. short_chat: 短 prompt，重算比迁移更简单，走 recompute。
3. cross_tenant: 看到了他租户 prefix cache，但不能复用，只能按无缓存路径路由。

```
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class KVRequest:
```

```
    request_id: str
    tenant: str
    model_version: str
    tokenizer_version: str
    prompt_tokens: int
    output_tokens: int
    dtype: str
    layout_version: str
    tp_size: int
    source_zone: str
    prefix_hash: str
```

```
@dataclass
```

```
class DecodeWorkerCandidate:
```

```
    name: str
    zone: str
    model_version: str
    tokenizer_version: str
    dtype: str
    layout_version: str
    tp_size: int
    available_blocks: int
    running_sequences: int
    pending_transfers: int
    current_tpot_ms: float
    prefix_cache: dict = field(default_factory=dict)
```



```

class ToyKVTransferRouter:
    def __init__(self, block_size=128, kv_mib_per_token=0.015625):
        self.block_size = block_size
        self.kv_mib_per_token = kv_mib_per_token
        self.skipped = []

    def _required_blocks(self, request):
        return (
            request.prompt_tokens + request.output_tokens + self.block_size - 1
        ) // self.block_size

    def _network_rate(self, source_zone, target_zone):
        return 1200.0 if source_zone == target_zone else 300.0

    def _compatible(self, request, worker):
        checks = {
            "model": request.model_version == worker.model_version,
            "tokenizer": request.tokenizer_version == worker.tokenizer_version,
            "dtype": request.dtype == worker.dtype,
            "layout": request.layout_version == worker.layout_version,
            "tp": request.tp_size == worker.tp_size,
        }
        return all(checks.values()), checks

    def route(self, request, workers):
        required_blocks = self._required_blocks(request)
        recompute_ms = round(request.prompt_tokens / 900.0, 3)
        kv_transfer_mib = request.prompt_tokens * self.kv_mib_per_token
        if request.prompt_tokens < 512:
            return {
                "request_id": request.request_id,
                "decision": "recompute",
                "required_blocks": required_blocks,
                "recompute_ms": recompute_ms,
                "kv_transfer_mib": round(kv_transfer_mib, 1),
                "reason": "short_prompt_recompute_cheaper",
            }

        scored = []
        tenant_cache_blocked = False
        for worker in workers:
            compatible, checks = self._compatible(request, worker)
            if not compatible:
                self.skipped.append({
                    "request_id": request.request_id,
                    "worker": worker.name,
                    "checks": checks,
                })
                continue
            if worker.available_blocks < required_blocks:
                self.skipped.append({
                    "request_id": request.request_id,
                    "worker": worker.name,

```

```

        "checks": {"capacity": False},
    })
    continue
    rate = self.network_rate(request.source_zone, worker.zone)
    transfer_ms = kv_transfer_mib / rate
    cache_tenant = worker.prefix_cache.get(request.prefix_hash)
    prefix_reuse = cache_tenant == request.tenant
    if cache_tenant is not None and cache_tenant != request.tenant:
        tenant_cache_blocked = True
    prefix_bonus = 25.0 if prefix_reuse else 0.0
    score = (
        worker.available_blocks * 0.02
        - worker.running_sequences * 1.0
        - worker.pending_transfers * 3.0
        - worker.current_tpot_ms * 0.4
        - transfer_ms
        + prefix_bonus
    )
    scored.append({
        "worker": worker,
        "score": round(score, 3),
        "transfer_ms": round(transfer_ms, 3),
        "prefix_reuse": prefix_reuse,
        "tenant_cache_blocked": tenant_cache_blocked,
    })

if not scored:
    return {
        "request_id": request.request_id,
        "decision": "route_failed",
        "required_blocks": required_blocks,
    }

best = max(scored, key=lambda row: row["score"])
worker = best["worker"]
worker.available_blocks -= required_blocks
worker.pending_transfers += 1
return {
    "request_id": request.request_id,
    "decision": "migrate",
    "decode_worker": worker.name,
    "required_blocks": required_blocks,
    "reserved_blocks": required_blocks,
    "kv_transfer_mib": round(kv_transfer_mib, 1),
    "transfer_ms": best["transfer_ms"],
    "score": best["score"],
    "prefix_reuse": best["prefix_reuse"],
    "tenant_cache_blocked": best["tenant_cache_blocked"],
    "failure_plan": "release_destination_blocks_and_retry_if_source_kv_exists",
}

def audit(self, requests, workers):
    rows = [self.route(request, workers) for request in requests]
    summary = {
        "requests": len(rows),
    }

```

```

    "migrate": sum(row["decision"] == "migrate" for row in rows),
    "recompute": sum(row["decision"] == "recompute" for row in rows),
    "route_failed": sum(row["decision"] == "route_failed" for row in rows),
    "total_transfer_mib": round(
        sum(row.get("kv_transfer_mib", 0) for row in rows if row["decision"] == "migrate"),
        1,
    ),
    "reserved_blocks": sum(row.get("reserved_blocks", 0) for row in rows),
    "skipped_incompatible": len([
        skipped for skipped in self.skipped
        if skipped.get("checks", {}).get("layout") is False
        or skipped.get("checks", {}).get("tp") is False
    ]),
    "tenant_cache_blocked": any(row.get("tenant_cache_blocked") for row in rows),
}
gates = {
    "metadata_compatibility_checked": summary["skipped_incompatible"] > 0,
    "decode_capacity_reserved": summary["reserved_blocks"] > 0,
    "routing_uses_cost_score": all(
        "score" in row for row in rows if row["decision"] == "migrate"
    ),
    "tenant_isolation_enforced": summary["tenant_cache_blocked"],
    "recompute_path_visible": summary["recompute"] == 1,
    "failure_plan_visible": all(
        "failure_plan" in row for row in rows if row["decision"] == "migrate"
    ),
    "metrics_ready": summary["total_transfer_mib"] > 0
    and summary["requests"] == 3,
}
gates["kv_transfer_routing_gate"] = all(gates.values())
return rows, summary, gates

```

```

workers = [
    DecodeWorkerCandidate("decode_fast", "rack_a", "v1", "tok1", "fp16", "paged_v1", 4, 220, 18, 1, 12.0),
    DecodeWorkerCandidate("decode_cache", "rack_b", "v1", "tok1", "fp16", "paged_v1", 4, 180, 5, 0, 11.0, {"doc_a": " "}),
    DecodeWorkerCandidate("decode_bad_layout", "rack_a", "v1", "tok1", "fp16", "paged_v2", 4, 500, 1, 0, 9.0, {"doc_a": " "}),
]
requests = [
    KVRequest("long_rag", "tenant_a", "v1", "tok1", 8192, 256, "fp16", "paged_v1", 4, "rack_a", "doc_a"),
    KVRequest("short_chat", "tenant_a", "v1", "tok1", 256, 128, "fp16", "paged_v1", 4, "rack_a", "none"),
    KVRequest("cross_tenant", "tenant_b", "v1", "tok1", 4096, 256, "fp16", "paged_v1", 4, "rack_a", "doc_a"),
]

```

```

rows, summary, gates = ToyKVTransferRouter().audit(requests, workers)
print("kv_transfer_rows=", rows)
print("kv_transfer_summary=", summary)
print("kv_transfer_gates=", gates)

```

一次运行的核心输出类似：

```

kv_transfer_summary= {'requests': 3, 'migrate': 2, 'recompute': 1, 'route_failed': 0, 'total_transfer_mib': 192.0, 'reserved_blocks': 0, 'skipped_incompatible': 1, 'tenant_cache_blocked': False, 'tenant_isolation_enforced': False, 'recompute_path_visible': True, 'failure_plan_visible': True, 'metrics_ready': True}
kv_transfer_gates= {'metadata_compatibility_checked': True, 'decode_capacity_reserved': True, 'routing_uses_cost_score': True, 'tenant_isolation_enforced': False, 'recompute_path_visible': True, 'failure_plan_visible': True, 'metrics_ready': True}

```

这个 demo 展示了几个关键点：

1. metadata_compatibility_checked=True: layout / TP 不兼容的 worker 被跳过，避免 silent wrong output。

2. `decode_capacity_reserved=True`: 迁移路径先 reserve blocks, 再开始 transfer, 不把 KV 传到未知位置。
3. `routing_uses_cost_score=True`: 路由不是只看空闲, 而是综合 KV capacity、running sequences、pending transfers、TPOT、网络和 prefix reuse。
4. `tenant_isolation_enforced=True`: `cross_tenant` 看到了 `doc_a` 的 prefix cache, 但因为属于其他 tenant, 不能复用。
5. `recompute_path_visible=True`: 短 prompt 直接 recompute, 比迁移更简单。
6. `failure_plan_visible=True`: 每个迁移请求都带有 transfer 失败后的清理 / 重试计划。

所以本章最终门禁是 `kv_transfer_routing_gate`: 只有 metadata、capacity、route score、tenant isolation、recompute fallback、failure plan 和 metrics 都能闭环, KV transfer 才不是“复制 bytes”, 而是可治理的系统能力。

38.39 小练习

1. 估算一个 32 层、GQA、FP16 模型在 8000 token prompt 下的 KV cache 大小。
2. 画出 push KV transfer 的状态机。
3. 画出 pull KV transfer 的状态机。
4. 解释为什么 paged KV cache 会让 transfer metadata 更复杂。
5. 写出 KV transfer metadata 至少应该包含的 12 个字段。
6. 设计一个 decode worker routing score, 至少考虑 5 个因素。
7. 说明 prefix-aware routing 为什么可能导致热点 worker。
8. 讨论什么时候 recompute 比 migrate 更合理。
9. 解释 decode 中途失败为什么比 prefill 失败更难恢复。
10. 设计 8 个观测 KV transfer 的关键指标。

38.40 本章总结

PD 分离真正困难的地方, 是让 Decode worker 正确、高效地使用 Prefill worker 生成的 KV Cache。

KV Cache 不是普通数据。它体积大、在 GPU memory 中、layout 复杂, 并且和模型版本、dtype、paged block、TP rank mapping、attention kernel 强相关。

KV 迁移常见模式包括 push、pull 和共享。Push/Pull 描述的是控制语义, 底层可能由 RDMA、GPU IPC、NIXL、Mooncake 或其他 backend 实现。

Decode worker 通常要先预留 KV blocks, 确认容量、确定目标地址、建立 block mapping, 然后才能开始 transfer。

Router 需要做 KV-aware routing 和 prefix-aware routing, 不仅看 worker 负载, 还要看 KV capacity、pending transfers、网络拓扑、prefix locality 和 TPOT。

故障处理的核心是资源清理和语义恢复。Prefill 或 transfer 失败通常可以重试; decode 中途失败因为已经对外输出 token, 恢复更困难。

下一章会继续讨论 Chunked Prefill 与 Disaggregated Prefill, 看看长 prompt prefill 如何进一步切分和调度。

第 39 章 Chunked Prefill 与 Disaggregated Prefill

上一章讲了 KV Cache 迁移、共享和路由: PD 分离真正难的地方, 是让 Decode worker 正确、高效地使用 Prefill worker 生成的 KV Cache。

本章继续讨论 Prefill 侧的另一个关键问题: 长 prompt 怎么处理。

在真实业务中, prompt 可能很长。RAG、Agent、多轮对话、代码生成、长文档问答都会把输入上下文拉得很长。如果一次性把长 prompt 全部 prefill 完, 就可能阻塞 decode、拉高 TTFT、造成显存峰值和调度抖动。

Chunked Prefill 和 Disaggregated Prefill 都是在解决这个问题, 但它们不是同一个概念。

一句话概括:

Chunked Prefill 是把长 prompt 的 prefill 切成多个 chunk 调度；Disaggregated Prefill 是把 prefill 作为独立资源池或独立阶段来服务。前者是执行粒度优化，后者是系统架构优化，二者可以结合。

39.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 `WRITING_PLAN.md` 对公开资料做校准：参考 vLLM chunked prefill 相关文档对 decode-first scheduling、`max_num_batched_tokens`、ITL / TTFT 折中和 chunked prefill 默认行为的说明；参考 vLLM disaggregated prefilling 文档对 prefill 实例、decode 实例、KV connector 和 experimental 边界的说明；参考 SGLang PD Disaggregation 文档对 prefill / decode server、Mooncake / NIXL、bootstrap、timeout、heterogeneous TP staging buffer、PD multiplexing 和 routing policy 的公开口径；并参考 SARATHI / SARATHI-Serve 论文对 chunked-prefills、decode-maximal batching、stall-free serving 和 prefill / decode 利用率差异的系统动机说明。

本讲只讲 Chunked Prefill 与 Disaggregated Prefill 的通用关系：长 prompt prefill stall、chunk size、token budget、decode-first interleaving、position / mask / KV block table 连续性、PD prefill pool、按 chunk transfer、backpressure 和失败清理。不把某个框架版本的真实 CLI 参数默认值、connector 字段、Mooncake / NIXL API、scheduler 类名、benchmark 数字或生产配置写成通用标准。

本讲新增 demo 是教学版 Chunked / Disaggregated Prefill 审计器：用 0 依赖 Python 模拟 full prefill stall、chunked scheduling、decode step 插入、token budget、position 连续性、短请求不饥饿、PD transfer 流水化、backpressure 和 abort cleanup，帮助把“chunked prefill 能缓解长 prompt 干扰”落到可运行证据。

39.1 本章目标

读完本章，你应该能讲清：

1. 为什么长 prompt prefill 会成为 serving 瓶颈。
2. Chunked Prefill 解决什么问题。
3. Chunked Prefill 和普通 prefill 的区别。
4. Disaggregated Prefill 和 PD 分离是什么关系。
5. Chunked Prefill 如何影响 TTFT、TPOT 和吞吐。
6. Chunked Prefill 如何和 continuous batching 结合。
7. Chunked Prefill 如何和 KV transfer 结合。
8. 面试中如何区分 Chunked Prefill、Disaggregated Prefill 和 PD 分离。

39.2 长 Prompt Prefill 的问题

先看一个普通请求：

```
prompt length = 128 tokens
max output = 512 tokens
```

Prefill 很短，主要成本在后续 decode。

再看长上下文请求：

```
prompt length = 32000 tokens
max output = 512 tokens
```

这时 prefill 本身就是一个很大的计算任务。

它会带来：

1. 长时间占用 GPU compute。
2. 一次性写入大量 KV cache。
3. 形成很大的 activation 和 attention kernel 压力。
4. 让短请求排队。
5. 打断正在稳定 decode 的 batch。
6. 拉高整体 TTFT。

7. 放大 tail latency。

在 unified engine 中，长 prompt prefill 的典型问题是：

t0: decode batch 正在稳定输出
t1: 长 prompt 请求到达
t2: scheduler 调度长 prefill
t3: decode batch 被挤占或延迟
t4: 所有 streaming 用户都感觉卡顿

这就是前面章节说过的 prefill interruption。

39.3 为什么不能总是一次性 Prefill

一次性 prefill 的流程很简单：

```
take full prompt
-> run one large prefill forward
-> produce all prompt KV
-> enter decode
```

它的优点是：

1. 实现简单。
2. KV 生成一次完成。
3. 状态机简单。
4. 没有 chunk 边界管理。

但对长 prompt，它的问题很明显：

1. 单次计算时间长。
2. 不能灵活插入 decode step。
3. 容易造成调度饥饿。
4. batch shape 可能很极端。
5. 显存峰值更高。
6. TTFT 更容易被长 prompt 主导。

所以需要一种折中：

不要一次吃完整个 prompt，而是分块吃。

这就是 Chunked Prefill。

39.4 什么是 Chunked Prefill

Chunked Prefill 指的是把一个长 prompt 拆成多个 token chunk，分多次 prefill 执行。

例如：

```
prompt length = 32000
chunk size = 4096
```

```
chunk 0: tokens [0, 4096)
chunk 1: tokens [4096, 8192)
chunk 2: tokens [8192, 12288)
...
chunk 7: tokens [28672, 32000)
```

执行时不是一次性跑 32000 tokens，而是：

```
run prefill chunk 0
write KV for chunk 0
run prefill chunk 1
```

```
write KV for chunk 1
...
run prefill final chunk
write KV for final chunk
then enter decode
```

注意：Chunked Prefill 不是把模型层切开。

它切的是输入 token 序列。

每个 chunk 仍然要经过完整 Transformer 层。

39.5 Chunked Prefill 的直觉

可以把长 prompt prefill 想成一个很大的任务。

一次性 prefill 是：

```
[===== long prefill =====][decode][decode][decode]
```

Chunked Prefill 是：

```
[prefill chunk][decode][prefill chunk][decode][prefill chunk][decode]...
```

或者在 prefill pool 中：

```
[prefill chunk A][prefill chunk B][prefill chunk C]...
```

它的核心价值是把一个大任务拆成多个可调度的小任务。

这样 scheduler 可以在 chunk 边界做决策：

1. 是否插入 decode step。
2. 是否调度短 prompt。
3. 是否切换 batch 组合。
4. 是否暂停某个长请求。
5. 是否进行资源让渡。

所以 Chunked Prefill 的本质是调度粒度优化。

39.6 Chunked Prefill 解决什么问题

Chunked Prefill 主要解决四类问题。

第一，降低 prefill 对 decode 的长时间阻塞。

长 prefill 被切成多个 chunk 后，decode step 可以在 chunk 间插入。

第二，改善调度公平性。

短请求不必一直等一个超长 prompt prefill 完。

第三，控制显存和 batch 形态。

每次只处理一段 prompt，可以降低单次峰值压力。

第四，给 PD 分离和 KV transfer 提供更细粒度的流水。

Prefill 侧可以逐块产生 KV，Decode 侧或远端 KV cache 可以逐块接收。

39.7 Chunked Prefill 不解决什么问题

Chunked Prefill 不是万能的。

它不直接解决：

1. P/D 资源池是否分离。
2. KV cache 如何跨 worker 迁移。
3. Decode worker 如何选。
4. 远端 KV cache 如何管理。
5. 多租户资源隔离。
6. 网络带宽瓶颈。

它只是把 prefill 执行切成 chunk。

如果系统仍然是 unified engine, P/D 仍然会共享同一组 GPU, 只是 prefill interruption 被缓解。

如果系统是 PD 分离, Chunked Prefill 可以让 prefill pool 内部调度更细, 也可以让 KV transfer 更流水化。

39.8 Chunked Prefill 和 Continuous Batching

Continuous batching 的核心是: 每个 step 动态组合请求, 而不是固定一个 batch 从头跑到尾。

在 decode 阶段, 它通常表现为:

step 0: req A, B, C
step 1: req A, B, C, D
step 2: req A, C, D

Chunked Prefill 加入后, scheduler 面对的不只是 decode step, 还有 prefill chunk。

它要在下面几类工作之间做选择:

1. 新请求的 prefill chunk。
2. 长请求的后续 prefill chunk。
3. 已进入生成阶段的 decode step。
4. KV transfer completion 后等待 decode 的请求。

调度器可以形成这样的时间线:

t0: decode step for running requests
t1: prefill chunk for long request L
t2: decode step for running requests
t3: prefill chunk for short request S
t4: decode step for running requests
t5: prefill chunk for long request L

这就是把 prefill chunk 当成可调度单元。

39.9 Chunk Size 的影响

Chunk size 是 Chunked Prefill 的核心参数。

如果 chunk size 太大:

1. 单个 chunk 仍然会阻塞 decode。
2. 调度粒度不够细。
3. 长请求仍然容易造成尾延迟。

如果 chunk size 太小:

1. kernel launch 和调度开销增加。
2. attention 计算效率可能下降。
3. KV block 管理更频繁。
4. transfer metadata 更多。
5. 总吞吐可能下降。

所以 chunk size 是折中:

large chunk:
better compute efficiency
worse latency fairness

small chunk:
better scheduling fairness
more overhead

生产系统通常会根据：

1. 模型大小。
2. GPU 类型。
3. prompt 长度分布。
4. decode latency SLO。
5. KV transfer 带宽。
6. batch scheduler 策略。

来选择 chunk size。

39.10 Chunked Prefill 对 TTFT 的影响

TTFT 是 time to first token。

对于一个长 prompt 请求来说，first token 必须等完整 prompt 的 KV 都准备好，才能开始生成。

所以 Chunked Prefill 不一定降低这个请求自己的理论最短 TTFT。

因为它仍然要处理完整 prompt。

但它会改善系统级 TTFT：

1. 短请求不用被长请求一直挡住。
2. Decode step 可以在 chunk 间执行。
3. Scheduler 可以更公平地服务不同请求。
4. 尾延迟会更稳定。

换句话说：

Chunked Prefill 不会让 32000 token 的计算凭空消失，
但会让这 32000 token 的计算不再一口气堵住整个系统。

39.11 Chunked Prefill 对 TPOT 的影响

TPOT 是 time per output token。

在 unified engine 中，长 prefill 会打断 decode，导致 TPOT 抖动。

Chunked Prefill 可以让 decode step 更频繁地插入，从而降低 TPOT tail latency。

例如没有 chunking：

```
decode step  
decode step  
[long prefill 800ms]  
decode step delayed
```

有 chunking：

```
decode step  
[prefill chunk 100ms]  
decode step  
[prefill chunk 100ms]  
decode step
```

TPOT 的峰值会更低。

但 chunk 太小也可能因为调度开销过大而让整体 TPOT 变差。

所以仍然是折中。

39.12 Chunked Prefill 对吞吐的影响

从吞吐角度看，Chunked Prefill 有两面性。

正向影响：

1. batch 组合更灵活。
2. GPU 空洞可能减少。
3. 短请求可以更快进入系统。
4. Decode 不容易被长 prefill 长时间饿死。

负向影响：

1. 更多 scheduler 决策。
2. 更多 kernel launch。
3. 更多 KV metadata 更新。
4. 更复杂的 attention mask / position 管理。
5. 可能降低大矩阵计算效率。

所以 Chunked Prefill 的目标不是 “无条件提高吞吐”。

更准确地说：

Chunked Prefill 用一定 overhead 换更好的延迟公平性和更可控的调度。

39.13 Chunked Prefill 的状态机

一个支持 Chunked Prefill 的请求，不再只有 WAITING -> PREFILLING -> DECODING。

它可能是：

WAITING

```
-> PREFILL_CHUNK_RUNNING
-> PREFILL_CHUNK_DONE
-> PREFILL_PAUSED
-> PREFILL_CHUNK_RUNNING
-> PREFILL_DONE
-> DECODING
-> FINISHED
```

需要维护的字段包括：

1. request_id。
2. prompt_len。
3. chunk_size。
4. next_token_offset。
5. prefilled_token_count。
6. kv_block_table。
7. position_offset。
8. attention_state。
9. priority。
10. deadline。

其中 next_token_offset 很关键。

它表示下一次 prefill chunk 从 prompt 的哪个 token 开始。

39.14 Position 和 Attention Mask

Chunked Prefill 不能破坏模型语义。

虽然 prompt 被切成多个 chunk，但对模型来说，它必须等价于一次性处理完整 prompt。

这意味着：

1. position id 必须连续。
2. RoPE 位置必须正确。
3. chunk 后面的 token 必须能 attend 到前面 chunk 的 KV。
4. causal mask 必须保持正确。
5. KV cache 写入位置必须对应全局 token offset。

例如：

chunk 0: tokens [0, 4096), positions [0, 4096)

chunk 1: tokens [4096, 8192), positions [4096, 8192)

chunk 1 不能把 position 从 0 重新开始。

否则模型语义就错了。

39.15 KV Block 写入

Chunked Prefill 会逐步写入 KV block。

例如 block size 是 16 tokens：

chunk 0 writes blocks 0..255

chunk 1 writes blocks 256..511

chunk 2 writes blocks 512..767

Scheduler 需要确保：

1. 已写入的 KV block 不被错误释放。
2. 后续 chunk 能找到前面的 KV。
3. block table 持续更新。
4. chunk 边界不破坏 block 边界。
5. 最后一个 chunk 可能不是完整 block。

在 paged KV cache 中，chunked prefill 的本质是：

incrementally extend the request's KV block table

39.16 Chunk 边界和 Block 边界

chunk size 和 KV block size 不一定相同。

例如：

chunk size = 4096 tokens

block size = 16 tokens

这时一个 chunk 对应 256 个 blocks。

如果：

chunk size = 3000

block size = 16

那么 chunk 边界可能落在 block 中间。

这会带来额外复杂度：

1. partial block 写入。

- 2. block offset 记录。
- 3. transfer 时的非整块处理。
- 4. cache reuse 时的边界匹配。

为了简化实现，系统可能倾向于让 chunk size 是 block size 的整数倍。
但这不是必须，只是工程上更方便。

39.17 Disaggregated Prefill 是什么

Disaggregated Prefill 字面意思是 “分离式 prefill” 。

它通常指把 prefill 阶段从统一 serving engine 中拆出来，使用独立的 prefill workers 或 prefill pool 来处理输入阶段。

典型结构：

```
client
-> router
-> prefill pool
    run prefill
    produce KV
-> decode pool
    receive KV
    generate tokens
```

这基本就是 PD 分离中的 P 侧。

有时文档里会把 Disaggregated Prefill 和 PD Disaggregation 混用。

但为了解清楚，可以这样区分：

- 1. Disaggregated Prefill 强调 prefill 从主 decode engine 中拆出来。
- 2. PD 分离强调 prefill pool 和 decode pool 都作为独立资源池协作。
- 3. Chunked Prefill 强调 prefill 内部按 token chunk 切分执行。

39.18 Chunked Prefill vs Disaggregated Prefill

二者不是同一层面的概念。

维度	Chunked Prefill	Disaggregated Prefill
关注点	执行粒度	系统架构
切分对象	prompt tokens	prefill 阶段和资源池
是否必须跨 worker	不必须	通常是
主要收益	减少长 prefill 阻塞	P/D 资源隔离和独立扩缩容
主要代价	调度和状态更复杂	KV transfer 和分布式状态更复杂

简单说：

Chunked Prefill:
how to run a long prefill

Disaggregated Prefill:
where to run prefill

二者可以组合：

run chunked prefill on disaggregated prefill pool

39.19 三种架构对比

第一种: Unified engine + normal prefill。

same worker:
full prefill
decode

特点: 实现简单, 但长 prefill 容易打断 decode。

第二种: Unified engine + chunked prefill。

same worker:
prefill chunk
decode step
prefill chunk
decode step

特点: 缓解 prefill interruption, 但 P/D 仍然共享资源。

第三种: PD disaggregation + chunked prefill。

prefill pool:
prefill chunk
prefill chunk
prefill chunk

decode pool:
receive KV
decode step
decode step

特点: P/D 资源隔离, 同时 prefill 内部也有更细粒度调度, 但 KV transfer 和状态最复杂。

39.20 Chunked Prefill 和 KV Transfer

在 PD 分离中, Chunked Prefill 会影响 KV transfer。

两种策略:

第一, 全部 prefill 完再 transfer。

prefill chunk 0
prefill chunk 1
...
prefill chunk N
transfer all KV
decode

优点:

1. 状态简单。
2. transfer 只做一次或少数几次。
3. decode 接收完整 prompt KV。

缺点:

1. 不能流水化。
2. TTFT 包含完整 prefill 和完整 transfer。
3. source KV 在 prefill worker 保留时间长。

第二, 边 prefill 边 transfer。

```
prefill chunk 0 -> transfer chunk 0
prefill chunk 1 -> transfer chunk 1
...
prefill chunk N -> transfer chunk N
decode
```

优点:

1. prefill 和 transfer 可以重叠。
2. 降低最后一次集中 transfer 的等待。
3. 更适合长 prompt。

缺点:

1. transfer metadata 更复杂。
2. decode side 要接收 partial KV。
3. 失败清理更复杂。
4. chunk 完整性和顺序要保证。

39.21 流水化 Prefill 和 Transfer

边 prefill 边 transfer 可以形成流水线:

time --->

```
prefill worker: [chunk0][chunk1][chunk2][chunk3]
transfer:      [kv0] [kv1] [kv2] [kv3]
decode worker:      receive receive receive ready
```

这样 final chunk 完成后, 前面的 KV 可能已经传完。

TTFT 从:

$\text{full_prefill_time} + \text{full_transfer_time}$

变成近似:

$\max(\text{prefill_pipeline_time}, \text{transfer_pipeline_time}) + \text{final_sync}$

但流水化也要求系统支持:

1. Partial KV metadata。
2. Chunk completion event。
3. Destination block incremental reservation。
4. Transfer ordering。
5. Abort cleanup。
6. Backpressure。

如果实现不成熟, 流水化可能带来更多 bug。

39.22 Decode 能不能提前开始

一个常见问题是: Prompt 还没全部 prefill 完, decode 能不能开始?

一般自回归生成不能在完整 prompt KV 准备好之前生成第一个新 token。

因为第一个输出 token 需要 attend 到完整 prompt。

所以通常:

```
all prompt chunks ready
-> first decode token
```

但有一些特殊场景可以做 overlap 或 speculative-like 优化, 例如:

1. Prompt 后半部分不影响某些中间处理。
2. 模型或任务支持分段编码和后续融合。
3. 使用特定近似算法。
4. 多阶段检索或工具调用把上下文逐步注入。

这些不是标准 LLM serving 的默认语义。

所以面试回答要稳：

对普通 causal LM，请求进入 decode 前需要完整 prompt 的 KV。Chunked Prefill 可以让 prefill 和调度/transfer 流水化，但不能随意在 prompt 未完整处理时生成首 token。

39.23 Prefill Chunk 的调度优先级

Scheduler 要决定下一个执行什么。

可选工作包括：

1. Decode step。
2. 新短请求的 full prefill。
3. 长请求的下一个 prefill chunk。
4. 已等待很久请求的 prefill chunk。
5. 高优先级租户请求。

一种简单策略：

```
if decode step deadline close:
    run decode
elif high priority short prefill exists:
    run short prefill
elif long prefill chunk has waited too long:
    run next chunk
else:
    maximize batch efficiency
```

真实系统会更复杂。

关键是不要让长 prompt 饿死短请求，也不要让长 prompt 自己永远跑不完。

这需要公平性机制。

39.24 Fairness：短请求和长请求

没有 Chunked Prefill 时，长请求一旦开始 prefill，短请求只能等。

有 Chunked Prefill 后，scheduler 可以在 chunk 间插入短请求。

但如果过度偏向短请求，长请求可能一直被切碎和延后。

这叫 starvation。

常见公平性策略包括：

1. Waiting time boost。
2. Per-tenant quota。
3. Max prefill chunks per scheduling window。
4. Decode deadline protection。
5. Long request progress guarantee。

例如：

$\text{score} = \text{base_priority} + \text{waiting_time_bonus} + \text{progress_bonus}$

对长请求来说，progress_bonus 可以避免它一直只完成前几个 chunk。

39.25 Backpressure

Chunked Prefill 和 PD 分离结合时, backpressure 很重要。

如果 prefill pool 很快, decode pool 或 transfer backend 很慢, 就会出现:

`prefill chunks produced faster than transfer/decode can consume`

后果:

1. Prefill worker 上 source KV 堆积。
2. Transfer queue 变长。
3. Decode worker reserved blocks 增多。
4. GPU memory 被未消费 KV 占满。
5. TTFT 反而变差。

所以 prefill scheduler 不能只看自己空不空。

它还要看:

1. Pending transfer 数。
2. Decode pool KV capacity。
3. Router 端 request backlog。
4. Source KV retention time。
5. Transfer backend 带宽。

如果下游拥塞, 上游 prefill 要限速。

39.26 Chunked Prefill 的 Metadata

支持 chunking 后, metadata 会增加。

一个请求需要维护:

```
request_id
prompt_token_ids
prompt_len
chunk_size
num_chunks
current_chunk_index
next_token_offset
prefilled_token_count
kv_block_table
chunk_to_block_mapping
position_offset
transfer_state_per_chunk
deadline
priority
```

如果结合 PD transfer, 还需要:

```
source_worker_id
destination_worker_id
source_blocks_per_chunk
destination_blocks_per_chunk
chunk_transfer_status
chunk_checksum_or_version
```

这说明 Chunked Prefill 不是只改一个 for loop。

它会影响 scheduler、KV manager、transfer backend 和 request state machine。

39.27 Chunked Prefill 的失败处理

失败可能发生在多个阶段。

第一，某个 prefill chunk 失败。

处理方式：

1. 释放该请求已分配但不再使用的 KV。
2. 判断是否能从上一个成功 chunk 重试。
3. 如果状态不可靠，重新从头 prefill。
4. 返回错误。

第二，chunk 已 prefill，但 transfer 失败。

处理方式：

1. 保留 source chunk KV。
2. 重试该 chunk transfer。
3. 或切换 decode worker 后重传所有已完成 chunk。

第三，部分 chunk 已到达 decode worker，但后续 chunk 失败。

处理方式：

1. 释放 decode side partial KV。
2. 清理 block table。
3. 清理 transfer state。
4. 必要时重新 prefill。

这里最重要的是不要留下 partial KV 泄漏。

39.28 Chunked Prefill 和 Prefix Cache

Chunked Prefill 可以和 prefix cache 结合。

例如：

```
prompt = [cached prefix][new suffix]
```

如果 cached prefix 已经有 KV，就只需要 prefill new suffix。

如果 suffix 很长，还可以 chunked prefill：

```
reuse prefix KV
prefill suffix chunk 0
prefill suffix chunk 1
...
```

这会带来新的边界问题：

1. prefix 长度可能不是 chunk size 整数倍。
2. prefix block 可能是共享 block。
3. suffix block 是私有 block。
4. position offset 要从 prefix length 开始。
5. eviction 不能释放仍被 suffix 请求引用的 prefix。

所以 prefix cache 命中后，不是简单跳过前几个 token。

还要正确接上 KV block table 和 position。

39.29 Chunked Prefill 和 Speculative Decoding

Chunked Prefill 和 speculative decoding 是两类不同优化。

Chunked Prefill 优化输入阶段：

long prompt -> chunks

Speculative decoding 优化输出阶段:

draft multiple tokens -> verify

二者可以同时存在。

例如:

chunked prefill prepares prompt KV

then speculative decoding accelerates generation

不要把它们混淆。

一个是 prefill scheduling, 一个是 decode acceleration。

39.30 Chunked Prefill 和 Pipeline Parallel

Chunked Prefill 也不是 Pipeline Parallel。

Pipeline Parallel 切的是模型层。

stage 0: layers 0-9

stage 1: layers 10-19

stage 2: layers 20-31

Chunked Prefill 切的是输入 token。

chunk 0: tokens 0-4095

chunk 1: tokens 4096-8191

二者关注点不同。

当然, 它们可以组合。

一个长 prompt chunk 仍然可以通过 pipeline parallel 的多个 stage 执行。

39.31 实现 Chunked Prefill 的简化伪代码

一个极简调度器可能这样写:

```
def schedule_step(waiting_prefills, running_decodes):
    if should_run_decode(running_decodes):
        return make_decode_batch(running_decodes)

    prefill_batch = []
    token_budget = MAX_PREFILL_TOKENS_PER_STEP

    for req in waiting_prefills:
        if token_budget <= 0:
            break

        remaining = req.prompt_len - req.prefilled_token_count
        chunk_len = min(req.chunk_size, remaining, token_budget)

        if chunk_len <= 0:
            continue

        prefill_batch.append((req, req.prefilled_token_count, chunk_len))
        token_budget -= chunk_len

    return PrefillChunkBatch(prefill_batch)
```

执行 chunk 后：

```
def on_prefill_chunk_done(req, chunk_len):
    req.prefilled_token_count += chunk_len

    if req.prefilled_token_count == req.prompt_len:
        req.state = "PREFILL_DONE"
    else:
        req.state = "PREFILL_PAUSED"
```

这只是教学伪代码。

真实系统还要处理 KV block、position、attention metadata、prefix cache、优先级、abort 和 transfer。

39.32 常见实现坑

Chunked Prefill 常见坑包括：

1. position id 从每个 chunk 重新开始。
2. chunk 之间 attention mask 不正确。
3. 前面 chunk 的 KV 没有加入后续 chunk 的 context。
4. block table 更新不完整。
5. partial block 处理错误。
6. chunk transfer 完成顺序和 chunk index 不一致。
7. abort 后 partial KV 没有释放。
8. chunk 太小导致吞吐下降。
9. chunk 太大导致 decode 仍然抖动。
10. prefix cache 命中后 position offset 错误。

这些坑本质上都来自一个事实：

对模型语义来说，chunked prefill 必须等价于 full prefill。

39.33 什么时候应该使用 Chunked Prefill

适合使用 Chunked Prefill 的场景：

1. prompt 长度分布长尾明显。
2. 系统中短请求和长请求混合。
3. decode streaming 平滑度要求高。
4. unified engine 中 prefill interruption 明显。
5. PD prefill pool 中长 prompt 造成队列抖动。
6. 希望 prefill 和 KV transfer 流水化。

不一定需要的场景：

1. prompt 普遍很短。
2. 离线批处理，不关心 streaming latency。
3. 只有少量长请求，且资源充足。
4. 实现复杂度无法接受。
5. chunk overhead 大于收益。

39.34 如何调参

调 Chunked Prefill，重点看这些参数：

1. chunk_size。
2. max_prefill_tokens_per_batch。
3. decode_priority。
4. max_prefill_chunks_per_window。

5. prefill_queue_timeout。
6. transfer_pipeline_depth。
7. max_pending_transfer_chunks。
8. long_request_priority_boost。

观测这些指标：

1. TTFT p50/p95/p99。
2. TPOT p50/p95/p99。
3. prefill chunk latency。
4. decode step latency。
5. prefill queue length。
6. number of paused prefills。
7. chunk transfer latency。
8. GPU utilization。
9. KV cache utilization。
10. request starvation count。

调参目标不是单点最优，而是在吞吐、TTFT、TPOT 和公平性之间找到平衡。

39.35 面试官会怎么问

问题一：什么是 Chunked Prefill？

回答要点：把长 prompt 的 prefill 按 token chunk 切成多次执行，每个 chunk 仍经过完整模型层。它把一个大 prefill 任务拆成多个可调度单元，缓解长 prefill 阻塞 decode 和短请求的问题。

问题二：Chunked Prefill 和 PD 分离是什么关系？

回答要点：Chunked Prefill 是 prefill 执行粒度优化，PD 分离是系统架构优化。Chunked Prefill 可以在 unified engine 中使用，也可以在 PD 分离的 prefill pool 中使用。

问题三：Chunked Prefill 会不会降低长 prompt 自己的 TTFT？

回答要点：不一定。普通 causal LM 生成首 token 前仍需要完整 prompt KV，所以长请求自己的理论计算量不变。但它能改善系统级 TTFT 和 TPOT 尾延迟，因为长 prefill 不会一次性堵住整个系统。

问题四：Chunk size 太大或太小有什么问题？

回答要点：太大则调度粒度粗，仍会阻塞 decode；太小则 kernel launch、scheduler、KV metadata 和 transfer overhead 增加，可能降低吞吐。

问题五：Chunked Prefill 如何和 KV transfer 结合？

回答要点：可以全部 prefill 完再 transfer，也可以边 prefill 边 transfer。后者能让 prefill 和 transfer 流水化，但需要 partial KV metadata、chunk completion、目标 block 增量预留、顺序保证和失败清理。

问题六：Disaggregated Prefill 和 Chunked Prefill 有什么区别？

回答要点：Disaggregated Prefill 关注 prefill 在独立资源池执行，回答 where to run prefill；Chunked Prefill 关注长 prompt 如何分块执行，回答 how to run a long prefill。

39.36 标准回答模板

如果面试官问“Chunked Prefill 和 Disaggregated Prefill 有什么区别”，可以这样回答：

Chunked Prefill 和 Disaggregated Prefill 是两个层面的概念。

Chunked Prefill 是执行粒度优化。它把一个长 prompt 的 prefill 按 token chunk 拆成多次执行，每个 chunk 仍然经过完整 Transf

Disaggregated Prefill 是系统架构优化。它把 prefill 从统一 engine 中拆出来，用独立的 prefill worker 或 prefill pool 处理输

二者可以结合：在 disaggregated prefill pool 中使用 chunked prefill。这样长 prompt 会被切成多个 prefill chunk，KV 可以按 c

39.37 Chunked / Disaggregated Prefill 公式、门禁和可运行 demo

先把请求抽象成：

$C_i = (r_i, P_i, O_i, H_i, S_i, D_i, A_i)$

其中 r_i 是 request id, P_i 是 prompt token 数, O_i 是预计输出 token 数, H_i 是 prefix cache 命中 token 数, S_i 是租户或优先级信号, D_i 表示是否走 disaggregated prefill, A_i 表示 abort / timeout 条件。

真正需要执行的 prefill token 数是：

$P_i^{\mathrm{run}} = \max(0, P_i - H_i)$

如果 chunk size 是 S_{chunk} , chunk 数为：

$N_i^{\mathrm{chunk}} = \left\lceil \frac{P_i^{\mathrm{run}}}{S_{\mathrm{chunk}}} \right\rceil$

第 k 个 chunk 的 token 数可以写成：

$q_{i,k} = \min(S_{\mathrm{chunk}}, P_i^{\mathrm{run}} - kS_{\mathrm{chunk}})$

在 mixed prefill / decode 调度里，一轮 token budget 不能只看 prefill chunk，还要把 decode rows 算进去：

$B_t^{\mathrm{used}} = D_t + \sum_{(i,k) \in A_t} q_{i,k} \leq B_{\mathrm{tok}}$

full prefill 的最大 stall 可以粗略写成：

$T_i^{\mathrm{full}} = \frac{P_i^{\mathrm{run}}}{X_{\mathrm{prefill}}}$

chunked prefill 的单次最大 stall 变为：

$T_i^{\mathrm{chunk,max}} = \max_k \left(\frac{q_{i,k}}{X_{\mathrm{prefill}}} + \delta_{\mathrm{sched}} \right)$

如果结合 PD transfer，全部 prefill 完再 transfer 的时间近似是 prefill 和 transfer 相加；按 chunk 流水化后可以近似看成：

$T_i^{\mathrm{pipe}} \approx \max(T_i^{\mathrm{prefill}}, T_i^{\mathrm{transfer}}) + T_{\mathrm{sync}}$

最终门禁可以写成：

$G_{\mathrm{chunkpd}} = G_{\mathrm{chunk}} G_{\mathrm{decode}} G_{\mathrm{budget}} G_{\mathrm{position}} G_{\mathrm{fair}} G_{\mathrm{...}}$

下面的 demo 覆盖三类请求：

1. long_rag: 长 prompt，命中一段 prefix，走 disaggregated prefill 和按 chunk transfer。
2. short_chat: 短 prompt，在 chunked scheduling 下优先完成，证明短请求不被长请求长期挡住。
3. agent_long: 长 prompt，但执行到两个 chunk 后 abort，用来验证 partial KV 和 pending transfer cleanup。

```
from dataclasses import dataclass
from math import ceil
```

```
@dataclass
```

```
class PrefillJob:
```

```
    request_id: str
    prompt_tokens: int
    output_tokens: int
    prefix_hit_tokens: int
    priority: int
    disaggregated: bool
    abort_after_chunks: int = 0
```

```
class ToyChunkedDisaggPrefillAuditor:
```

```
    def __init__(self, chunk_size=1024, block_size=128, token_budget=1280):
```

```

self.chunk_size = chunk_size
self.block_size = block_size
self.token_budget = token_budget
self.decode_rows = 192
self.prefill_tokens_per_ms = 512.0
self.kv_mib_per_token = 0.015625
self.transfer_mib_per_ms = 40.0
self.transfer_depth = 2

def _run_prefill_tokens(self, job):
    return max(0, job.prompt_tokens - job.prefix_hit_tokens)

def _chunk_tokens(self, remaining):
    return min(self.chunk_size, remaining)

def _chunk_latency_ms(self, tokens):
    return round(tokens / self.prefill_tokens_per_ms + 0.05, 3)

def _new_blocks(self, start_pos, end_pos):
    return ceil(end_pos / self.block_size) - (start_pos // self.block_size)

def full_prefill_summary(self, jobs):
    rows = []
    elapsed = 0.0
    short_wait_ms = 0.0
    max_stall = 0.0
    for job in jobs:
        run_tokens = self._run_prefill_tokens(job)
        prefill_ms = round(run_tokens / self.prefill_tokens_per_ms, 3)
        if job.request_id == "short_chat":
            short_wait_ms = round(elapsed, 3)
        elapsed += prefill_ms
        max_stall = max(max_stall, prefill_ms)
        rows.append({
            "request_id": job.request_id,
            "run_prefill_tokens": run_tokens,
            "full_prefill_ms": prefill_ms,
        })
    return rows, {
        "requests": len(jobs),
        "total_run_prefill_tokens": sum(row["run_prefill_tokens"] for row in rows),
        "max_full_prefill_stall_ms": round(max_stall, 3),
        "short_chat_full_wait_ms": short_wait_ms,
    }

def chunked_schedule(self, jobs):
    state = {
        job.request_id: {
            "job": job,
            "remaining": self._run_prefill_tokens(job),
            "next_offset": 0,
            "chunk_index": 0,
            "done": False,
            "aborted": False,
            "completion_round": None,

```

```

    }
    for job in jobs
}
rows = []
round_id = 0
active_decode_steps = 6
pending_transfer_chunks = 0
max_pending_transfer_chunks = 0
backpressure_events = 0
cleanup_events = 0
order = [
    "short_chat",
    "long_rag",
    "agent_long",
    "long_rag",
    "agent_long",
    "long_rag",
    "long_rag",
    "long_rag",
]

for request_id in order:
    item = state[request_id]
    if item["done"] or item["aborted"]:
        continue
    round_id += 1
    decode_first = active_decode_steps > 0
    if decode_first:
        active_decode_steps -= 1
    if round_id % 2 == 0 and pending_transfer_chunks > 0:
        pending_transfer_chunks -= 1

    job = item["job"]
    tokens = self._chunk_tokens(item["remaining"])
    position_start = job.prefix_hit_tokens + item["next_offset"]
    position_end = position_start + tokens
    transfer_mib = round(tokens * self.kv_mib_per_token, 1) if job.disaggregated else 0.0
    if job.disaggregated:
        pending_transfer_chunks += 1
        if pending_transfer_chunks > self.transfer_depth:
            backpressure_events += 1
    max_pending_transfer_chunks = max(max_pending_transfer_chunks, pending_transfer_chunks)

    item["remaining"] -= tokens
    item["next_offset"] += tokens
    item["chunk_index"] += 1
    done = item["remaining"] == 0
    aborted = bool(job.abort_after_chunks and item["chunk_index"] >= job.abort_after_chunks)
    if aborted:
        item["aborted"] = True
        cleanup_events += 1
        pending_transfer_chunks = max(0, pending_transfer_chunks - 1)
    elif done:
        item["done"] = True
        item["completion_round"] = round_id

```

```

rows.append({
    "round": round_id,
    "request_id": request_id,
    "chunk_index": item["chunk_index"] - 1,
    "tokens": tokens,
    "decode_first": decode_first,
    "budget_used": tokens + (self.decode_rows if decode_first else 0),
    "position_start": position_start,
    "position_end": position_end,
    "new_kv_blocks": self._new_blocks(position_start, position_end),
    "transfer_mib": transfer_mib,
    "state": "ABORTED" if aborted else ("PREFILL_DONE" if done else "PREFILL_PAUSED"),
})

chunked_summary = {
    "chunk_size": self.chunk_size,
    "scheduled_rounds": round_id,
    "chunks_run": len(rows),
    "decode_steps_interleaved": sum(row["decode_first"] for row in rows),
    "max_chunk_prefill_ms": max(self._chunk_latency_ms(row["tokens"]) for row in rows),
    "short_chat_completion_round": state["short_chat"]["completion_round"],
    "long_rag_completion_round": state["long_rag"]["completion_round"],
    "aborted_requests": sum(item["aborted"] for item in state.values()),
    "cleanup_events": cleanup_events,
    "max_pending_transfer_chunks": max_pending_transfer_chunks,
    "backpressure_events": backpressure_events,
}
return rows, chunked_summary

def pd_transfer_summary(self, rows):
    by_request = {}
    for row in rows:
        if row["transfer_mib"] <= 0:
            continue
        stats = by_request.setdefault(
            row["request_id"], {"chunks": 0, "transfer_mib": 0.0, "prefill_ms": 0.0}
        )
        stats["chunks"] += 1
        stats["transfer_mib"] += row["transfer_mib"]
        stats["prefill_ms"] += self._chunk_latency_ms(row["tokens"])
    pipeline_saving = 0.0
    for stats in by_request.values():
        transfer_ms = stats["transfer_mib"] / self.transfer_mib_per_ms
        all_after_ms = stats["prefill_ms"] + transfer_ms
        pipelined_ms = max(stats["prefill_ms"], transfer_ms) + 0.2
        pipeline_saving += max(0.0, all_after_ms - pipelined_ms)
    return {
        "disaggregated_requests": len(by_request),
        "disaggregated_chunks": sum(stats["chunks"] for stats in by_request.values()),
        "total_transfer_mib": round(sum(stats["transfer_mib"] for stats in by_request.values()), 1),
        "pipeline_transfer_saving_ms": round(pipeline_saving, 3),
    }

def position_continuity_ok(self, jobs, rows):

```



```

jobs_by_id = {job.request_id: job for job in jobs}
last_end = {}
for row in rows:
    job = jobs_by_id[row["request_id"]]
    expected_start = last_end.get(row["request_id"], job.prefix_hit_tokens)
    if row["position_start"] != expected_start:
        return False
    last_end[row["request_id"]] = row["position_end"]
return True

def audit(self, jobs):
    full_rows, full_summary = self.full_prefill_summary(jobs)
    chunk_rows, chunked_summary = self.chunked_schedule(jobs)
    pd_summary = self.pd_transfer_summary(chunk_rows)
    gates = {
        "long_prefill_chunked": any(
            self._run_prefill_tokens(job) > self.chunk_size for job in jobs
        ),
        "decode_interleaved": chunked_summary["decode_steps_interleaved"] > 0,
        "token_budget_respected": all(row["budget_used"] <= self.token_budget for row in chunk_rows),
        "position_continuity_checked": self.position_continuity_ok(jobs, chunk_rows),
        "short_request_not_starved": (
            chunked_summary["short_chat_completion_round"]
            < chunked_summary["long_rag_completion_round"]
        ),
        "pd_transfer_pipeline_visible": pd_summary["pipeline_transfer_saving_ms"] > 0,
        "backpressure_cleanup_visible": (
            chunked_summary["backpressure_events"] > 0
            and chunked_summary["cleanup_events"] > 0
        ),
        "metrics_ready": (
            full_summary["total_run_prefill_tokens"] > 0
            and chunked_summary["chunks_run"] == len(chunk_rows)
        ),
    }
    gates["chunked_disagg_prefill_gate"] = all(gates.values())
    return full_rows, full_summary, chunk_rows, chunked_summary, pd_summary, gates

```

```

jobs = [
    PrefillJob("long_rag", 6144, 256, 1024, 1, True),
    PrefillJob("short_chat", 384, 128, 0, 3, False),
    PrefillJob("agent_long", 3584, 512, 512, 1, True, abort_after_chunks=2),
]

```

```

full_rows, full_summary, chunk_rows, chunked_summary, pd_summary, gates = (
    ToyChunkedDisaggPrefillAuditor().audit(jobs)
)
print("chunked_prefill_full_summary=", full_summary)
print("chunked_prefill_rows=", chunk_rows)
print("chunked_prefill_chunked_summary=", chunked_summary)
print("chunked_prefill_pd_summary=", pd_summary)
print("chunked_prefill_gates=", gates)

```

一次运行的核心输出类似：

```
chunked_prefill_full_summary= {'requests': 3, 'total_run_prefill_tokens': 8576, 'max_full_prefill_stall_ms': 10.0, 'sho
chunked_prefill_chunked_summary= {'chunk_size': 1024, 'scheduled_rounds': 8, 'chunks_run': 8, 'decode_steps_interleaved
chunked_prefill_pd_summary= {'disaggregated_requests': 2, 'disaggregated_chunks': 7, 'total_transfer_mib': 112.0, 'pipe
chunked_prefill_gates= {'long_prefill_chunked': True, 'decode_interleaved': True, 'token_budget_respected': True, 'posi
```

这个 demo 展示了几个关键点：

1. max_full_prefill_stall_ms=10.0 到 max_chunk_prefill_ms=2.05：长 prefill 的单次阻塞被切小。
2. decode_steps_interleaved=6：decode-first interleaving 被显式计数，而不是只说“可以插入 decode”。
3. token_budget_respected=True：每轮把 decode rows 和 prefill chunk 一起算进 token budget。
4. position_continuity_checked=True：chunk 边界没有让 position 从 0 重启。
5. short_request_not_starved=True：短请求第 1 轮完成，长请求第 8 轮完成，说明 chunked scheduling 能让短请求绕过长 prefill。
6. pd_transfer_pipeline_visible=True：disaggregated prefill 下，按 chunk transfer 可以把部分 transfer 和 prefill 重叠。
7. backpressure_cleanup_visible=True：transfer backlog 和 abort cleanup 都进入门禁，避免 partial KV 泄漏。

所以本章最终门禁是 chunked_disagg_prefill_gate：只有 chunking、decode interleaving、token budget、position continuity、公平性、PD transfer、backpressure / cleanup 和 metrics 都能闭环，Chunked Prefill 才是可治理的 serving 能力。

39.38 小练习

1. 画出 full prefill 和 chunked prefill 的时间线对比。
2. 解释为什么 chunked prefill 必须保持 position id 连续。
3. 设计一个调度策略，在 decode step 和 prefill chunk 之间做选择。
4. 分析 chunk size 太大和太小的影响。
5. 说明 chunked prefill 如何缓解 prefill interruption。
6. 画出 chunked prefill 与 KV transfer 流水化的时序图。
7. 列出支持 chunked prefill 需要新增的 10 个 metadata 字段。
8. 解释为什么普通 causal LM 通常不能在 prompt 未完整 prefill 前开始 decode。
9. 对比 Chunked Prefill、Disaggregated Prefill、Pipeline Parallel。
10. 设计一组指标评估 chunked prefill 是否有效。

39.39 本章总结

Chunked Prefill 是把长 prompt prefill 切成多个 token chunk 执行，让 scheduler 可以在 chunk 边界做调度决策。

它的核心价值不是减少总计算量，而是把一个长时间阻塞的大任务拆成多个可调度的小任务，从而改善系统级 TTFT、TPOT 和公平性。

Disaggregated Prefill 是把 prefill 阶段放到独立资源池中执行，是系统架构层面的优化。Chunked Prefill 是 prefill 内部执行粒度的优化。

二者可以结合：在独立 prefill pool 中对长 prompt 做 chunked prefill，并把 KV 按 chunk 生成、传输或缓存。

实现 Chunked Prefill 时，必须保证它在模型语义上等价于 full prefill：position、attention mask、KV block table、prefix cache 和 partial block 都要正确处理。

下一章会继续讨论多级 KV Cache：GPU、CPU 和远端缓存，看看 KV 如何从单个 worker 的显存对象进一步变成跨层级、跨节点的系统级缓存资源。

第 40 章 多级 KV Cache：GPU、CPU 和远端缓存

前面几章已经把 KV Cache 从单个请求里的中间状态，逐步讲成了 serving 系统里的核心资源。

第 38 章讲了 KV Cache 迁移、共享和路由。第 39 章讲了长 prompt prefill 如何通过 chunked prefill 分块执行。本章继续往后走：如果 KV Cache 太大，或者希望跨 worker、跨节点复用，系统应该怎么管理它？

答案通常不是“全部放 GPU 显存”。

真实系统会把 KV Cache 做成多级缓存：

GPU KV Cache

- > CPU KV Cache
- > local SSD / NVMe KV Cache
- > remote KV Cache service
- > recompute

一句话概括：

多级 KV Cache 是把 KV 从单 worker 的 GPU 显存对象，升级成跨内存层级、跨节点、可迁移、可淘汰、可预取的系统级缓存资源。

40.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 WRITING_PLAN.md 对公开资料做校准：参考 LMCache 文档对 KV cache 复用、offload、CPU / local storage / remote storage connector 和 vLLM connector 的公开口径；参考 SGLang HiCache 文档对 L1 GPU KV cache、L2 host cache、L3 distributed storage、cache controller、eviction 和 writing policy 的说明；参考 Mooncake 论文对 KVCache-centric disaggregated architecture、KV cache 管理和远端 KV 传输瓶颈的系统动机说明；并参考 vLLM prefix caching / PagedAttention 资料对 full block、block hash、block table、reference count 和 KV block metadata 的稳定抽象。

本讲只讲多级 KV Cache 的通用系统问题：GPU / CPU / SSD / remote / recompute 的层级画像、residency-aware scheduling、promote / demote / prefetch / eviction、成本模型、正确性 key、多租户隔离、partial block 风险和观测指标。不把某个框架版本的真实 connector API、存储 backend 名称、cache controller 字段、默认阈值、网络拓扑参数或 benchmark 数字写成通用标准。

本讲新增 demo 是教学版多级 KV Cache 审计器：用 0 依赖 Python 模拟 GPU hot block 保护、CPU promote、remote fetch、SSD 不划算时 recompute、GPU 空间不足时 demote、跨租户命中阻断、residency metrics 和最终 gate，帮助把“多级缓存不是只看 hit rate”落到可运行证据。

40.1 本章目标

读完本章，你应该能讲清：

1. 为什么只靠 GPU KV Cache 不够。
2. GPU、CPU、SSD、远端 KV Cache 分别适合存什么。
3. KV offload、swap、remote KV store 和 recompute 的区别。
4. 多级 KV Cache 的 promote、demote、prefetch 和 eviction 策略。
5. 多级 KV Cache 如何影响 TTFT、TPOT、吞吐和显存利用率。
6. 多级 KV Cache 和 prefix cache、PD 分离、chunked prefill 的关系。
7. 实现多级 KV Cache 时需要哪些 metadata。
8. 面试中如何设计一个多级 KV Cache 系统。

40.2 为什么只靠 GPU KV Cache 不够

KV Cache 的特点是：

1. 体积大。
2. 增长快。
3. 与序列长度线性相关。
4. decode 每步都要访问历史 KV。
5. 活跃请求必须低延迟访问。
6. 非活跃 prefix 又可能有复用价值。

以一个大模型为例，单 token 的 KV 可能是几十 KB 到几百 KB。prompt 长度一旦达到几万 tokens，单个请求的 KV Cache 就可能达到 GB 级。

如果所有 KV 都常驻 GPU，会遇到几个问题：

- 1. GPU 显存被 KV 占满，batch size 上不去。
- 2. 长上下文请求挤压短请求。
- 3. prefix cache 命中后也未必有足够显存保留所有 prefix。
- 4. PD 分离中，prefill 生成的 KV 不一定马上被 decode 使用。
- 5. 多轮对话或 agent 任务中，请求可能短时间 idle，但之后又继续。
- 6. 跨 worker 复用 prefix 时，KV 不能只存在本地 GPU。

所以系统需要回答一个问题：

- 哪些 KV 必须在 GPU？
- 哪些 KV 可以暂时放 CPU？
- 哪些 KV 值得放到远端供其他 worker 复用？
- 哪些 KV 干脆丢掉，后面需要时重算？

这就是多级 KV Cache 的核心。

40.3 KV Cache 的访问冷热

不是所有 KV 都一样重要。

对一个正在 decode 的请求来说，历史 KV 每一步都要参与 attention。

例如：

```
step t:
  query = current token hidden state
  keys/values = all previous tokens' KV
```

这部分 KV 是热数据，必须尽量放在 GPU。

但还有很多 KV 并不总是热的：

- 1. 刚 prefill 完但还没被 decode 接管的 KV。
- 2. 多轮对话中暂时 idle 的 session KV。
- 3. prefix cache 中命中率高但当前没有请求使用的公共前缀。
- 4. RAG 模板、system prompt、工具描述这类重复 prefix。
- 5. 长请求中未来可能被恢复的 KV。
- 6. 被 preempt 的请求 KV。

这些 KV 不一定要一直占用 GPU。

多级缓存的基本思想是：

```
hot KV      -> GPU
warm KV     -> CPU
cold KV     -> SSD / remote store
too cold   -> evict and recompute later
```

40.4 四个层级的基本画像

可以先用一张表建立直觉。

层级	延迟	容量	带宽	典型用途
GPU HBM	最低	最小	最高	活跃 decode、当前 prefill
CPU DRAM	中等	较大	受 PCIe/NVLink 限制	idle session、暂存 prefix、swap
Local SSD/NVMe	高	更大	受本地 IO 限制	低频长 prefix、冷 KV

层级	延迟	容量	带宽	典型用途
Remote KV Store	取决于网络	可扩展	受网络和服务端限制	跨 worker 共享、PD 分离、全局 prefix cache

这张表背后的核心取舍是：

越靠近 GPU，访问越快，但容量越贵。

越远离 GPU，容量越大，但访问越慢，系统复杂度越高。

多级 KV Cache 不是越多层越好。每加一层，都要付出 metadata、调度、故障恢复和性能抖动的代价。

40.5 GPU KV Cache

GPU KV Cache 是最核心的一层。

它存放：

1. 正在 decode 的请求 KV。
2. 本轮 prefill 正在写入的 KV。
3. 马上要被 decode 使用的 prefix KV。
4. 高复用、高命中、低延迟要求的 prefix blocks。

GPU KV 的特点是：

1. 访问延迟最低。
2. attention kernel 可以直接读。
3. 不需要 host-device 传输。
4. 容量最紧张。
5. 碎片和 block 管理最敏感。

因此 GPU KV Cache 的管理目标通常是：

优先保证活跃 decode 的连续推进，避免因为显存不足导致 TPOT 抖动。

如果为了缓存更多历史 prefix，把活跃 decode 的 KV 挤出去，通常得不偿失。

40.6 CPU KV Cache

CPU KV Cache 是最常见的第二层。

它可以存放：

1. 被 preempt 请求的 KV。
2. 暂时 idle 的 session KV。
3. 短期内可能恢复的多轮对话 KV。
4. 不适合常驻 GPU 但重算成本较高的长 prefix。
5. PD 分离中等待 transfer 或等待 decode 接管的 KV。

CPU KV Cache 的优势是容量大，成本低。

它的问题是传输。

GPU <-> CPU

via PCIe or NVLink

如果 decode 每一步都要从 CPU 拉历史 KV，性能会非常差。因为 decode 是高频小步执行，TPOT 对延迟非常敏感。

所以 CPU KV 更适合做：

1. 请求暂停期间的保存层。
2. 恢复前的一次性 promote 来源。
3. 显存压力下的 swap 目标。

4. GPU prefix cache 的后备层。

它不适合作为每个 decode step 的在线 KV 读取路径。

40.7 Local SSD / NVMe KV Cache

SSD 或 NVMe 层更冷。

它适合：

1. 很长但低频复用的 prefix。
2. 大量 session 的冷状态。
3. 离线预热生成的公共 prompt KV。
4. 成本敏感场景下的长上下文缓存。

但 SSD 的问题也明显：

1. 延迟比 CPU 高得多。
2. IO 调度会带来 tail latency。
3. KV block 数量多时，小 IO 放大严重。
4. 序列化、压缩、校验和 metadata 管理复杂。
5. 随机读可能拖垮恢复路径。

因此 SSD KV Cache 更适合“恢复前批量加载”，而不是 decode 过程中频繁访问。

例如：

request resumes

- > load KV blocks from SSD to CPU
- > promote needed blocks to GPU
- > enter decode

而不是：

each decode step

- > read old KV from SSD

后者通常不可接受。

40.8 Remote KV Cache

Remote KV Cache 是跨节点的 KV 存储或缓存服务。

它可能以几种形态出现：

1. 独立 KV cache service。
2. 分布式内存缓存。
3. 对象存储或本地盘加索引服务。
4. prefill worker 暴露的 KV serving 接口。
5. decode worker 之间的 peer-to-peer KV sharing。

Remote KV Cache 的价值在于：

1. 跨 worker 复用 prefix。
2. 支持 PD 分离中的 KV 交接。
3. 避免每个 worker 重复 prefill 相同 system prompt。
4. 提升全局缓存容量。
5. 支持 session 迁移和故障恢复。

但它引入了网络瓶颈。

远端 KV 读取路径大致是：

decode worker

- > query metadata
- > locate remote KV blocks
- > fetch over network
- > place into local GPU/CPU blocks
- > update local block table

这比本地 GPU 命中复杂得多。

如果远端拉取发生在用户等待首 token 的关键路径上，TTFT 可能明显变差。如果发生在 decode 中间，TPOT 会抖动。

40.9 Recompute 也是一个层级

很多人讲多级缓存时只讲 GPU、CPU、SSD、remote store，但在推理系统里，recompute 也应该被看成最后一层。

如果一个 KV block 被淘汰了，系统并不一定彻底失败。

只要还保留原始 tokens、position、模型版本和必要上下文，就可以重新 prefill 得到 KV。

所以层级可以写成：

GPU -> CPU -> SSD -> Remote -> Recompute

Recompute 的优势是：

- 1. 不占缓存容量。
- 2. 不需要存储旧 KV。
- 3. 避免跨节点传输大对象。

Recompute 的代价是：

- 1. 消耗 GPU compute。
- 2. 增加 TTFT 或恢复延迟。
- 3. 可能打断当前 decode batch。
- 4. 对长 prompt 很贵。

因此 eviction 策略本质上是在比较：

保留 KV 的存储成本

vs

未来命中时传输 KV 的成本

vs

未来 miss 时重新计算的成本

40.10 Offload、Swap、Remote Cache 的区别

这几个词经常混用，但面试时最好区分清楚。

概念	核心含义	典型方向	主要目的
Offload	把 GPU 上的数据卸到较慢层级	GPU -> CPU/SSD	降低 GPU 显存压力
Swap	在显存不足时换出/换入运行状态	GPU <-> CPU	保护系统不 OOM
Remote Cache	把 KV 放到远端服务或节点	local <-> remote	跨 worker 复用或迁移
Recompute	丢弃 KV，之后重算	cache -> compute	用算力换存储

简单说：

offload 关注放到哪里;
swap 关注运行时换入换出;
remote cache 关注跨节点共享;
recompute 关注不存, 后面重算。

40.11 多级 KV Cache 的基本状态机

一个 KV block 可以处于多个状态。

例如:

GPU_RESIDENT
CPU_RESIDENT
SSD_RESIDENT
REMOTE_RESIDENT
IN_TRANSFER
EVICTED
RECOMPUTE_REQUIRED

更细一点, 还需要区分:

1. 是否被活跃请求引用。
2. 是否属于完整 block。
3. 是否属于 partial block。
4. 是否已经可被 prefix cache 命中。
5. 是否正在被写入。
6. 是否正在被 transfer。
7. 是否已经校验完成。
8. 是否可以被淘汰。

一个典型状态流转是:

prefill writes KV to GPU
-> request finishes or becomes idle
-> demote GPU KV to CPU
-> memory pressure increases
-> demote cold CPU KV to SSD or remote
-> request resumes
-> prefetch SSD/remote KV to CPU
-> promote CPU KV to GPU
-> decode continues

注意: 状态机必须和 scheduler、block manager、prefix cache 一起工作。否则 scheduler 以为 KV 在 GPU, 实际已经被换出, 就会在 attention kernel 读错地址。

40.12 Promote 和 Demote

Promote 是把 KV 从慢层级搬到快层级。

例如:

CPU -> GPU
SSD -> CPU
Remote -> CPU/GPU

Demote 是把 KV 从快层级搬到慢层级。

例如:

GPU -> CPU
CPU -> SSD

GPU -> remote

Promote 通常发生在：

1. 请求即将进入 decode。
2. prefix cache 命中，需要将命中 blocks 放回 GPU。
3. scheduler 预测某个 session 很快恢复。
4. PD 分离中 decode worker 即将接管 prefill KV。

Demote 通常发生在：

1. 请求完成但 prefix 有复用价值。
2. session 进入 idle。
3. GPU block 不够。
4. 系统希望保护活跃 decode。
5. 某些长 prefix 短期内不会访问。

Promote 和 demote 的难点不是拷贝本身，而是何时拷贝、拷贝多少、是否阻塞用户请求。

40.13 Prefetch

Prefetch 是提前把 KV 搬到更快层级。

例如：

```
request waiting in queue
-> scheduler predicts it will run soon
-> prefetch KV from CPU to GPU
-> when scheduled, decode can start immediately
```

Prefetch 的收益是降低关键路径延迟。

Prefetch 的风险是抢占资源。

如果预测错了，就会出现：

1. GPU 显存被无用 KV 占用。
2. PCIe 或网络带宽被无用 transfer 占用。
3. 活跃请求受到干扰。
4. 刚 prefetch 的 KV 又被淘汰。

一个简单策略是只 prefetch 高置信度请求：

```
if request is near head of queue
    and needed_kv_bytes <= available_gpu_budget
    and estimated_start_time < threshold:
        prefetch_to_gpu(request.kv_blocks)
```

更复杂的系统会把 prefetch 当成调度问题：

```
prefetch_priority
= resume_probability
* recompute_cost
/ transfer_cost
```

40.14 Eviction

Eviction 是决定丢掉哪些 KV。

在普通缓存里，LRU 是常见策略。但 KV Cache 不能只看最近访问时间。

原因是：

1. 不同 KV block 大小和重算成本不同。

2. 长 prefix 的后续 block 依赖前面的 prefix 语义。
3. 有些 prefix 虽然最近没用，但未来复用概率很高。
4. 有些 KV 属于活跃 decode，不能淘汰。
5. 有些 partial block 不适合单独缓存。
6. 多租户场景要考虑配额和隔离。

一个更合理的打分可以是：

```
eviction_score
= reuse_probability
* recompute_cost
* latency_sensitivity
/ memory_bytes
```

分数高表示更值得保留，分数低表示更适合淘汰。

也可以反过来定义 victim score：

```
victim_score
= memory_bytes
/ (reuse_probability * recompute_cost + epsilon)
```

实际系统不会只靠一个公式，而是会加硬规则：

1. 活跃 decode KV 不淘汰。
2. 正在 transfer 的 KV 不淘汰。
3. 被多个请求引用的 prefix block 优先保留。
4. 超过租户配额的 KV 优先淘汰。
5. 校验失败或版本不匹配的 KV 直接丢弃。

40.15 多级 KV Cache 和 Prefix Cache

Prefix cache 关注的是“相同前缀能不能复用”。

多级 KV Cache 关注的是“可复用 KV 放在哪里”。

两者通常要结合。

例如一个 system prompt 被大量请求复用：

system prompt tokens -> KV blocks

这些 blocks 可能同时有多个位置：

remote store: full copy for global reuse

CPU cache: warm copy for local worker

GPU cache: hot copy for active batch

prefix cache 的索引可能告诉你：

hash H hits blocks [b1, b2, b3]

但 block manager 还要知道：

b1 is on GPU

b2 is on CPU

b3 is remote only

如果只知道命中，不知道位置，scheduler 仍然无法估算成本。

所以 prefix cache 命中应该返回的不只是 hit length，还应该包含：

1. block ids。
2. residency level。
3. transfer cost estimate。

4. GPU promote requirement。
5. 是否需要 recompute。

40.16 多级 KV Cache 和 PD 分离

PD 分离中, Prefill worker 生成 KV, Decode worker 使用 KV。

这天然需要跨组件管理 KV。

最简单的路径是:

```
prefill worker GPU
-> transfer KV
-> decode worker GPU
```

但多级缓存会让路径变多:

```
prefill GPU -> prefill CPU -> remote KV -> decode CPU -> decode GPU
```

或者:

```
prefill GPU -> remote KV
decode worker cache hit remote KV
decode worker promotes to GPU
```

这样做的好处是:

1. prefill worker 不必等待 decode worker 立即接收。
2. decode worker 可以异步拉取 KV。
3. 多个 decode worker 可以复用同一份远端 prefix。
4. session 可以跨 worker 迁移。

代价是:

1. TTFT 中多了一段 remote fetch。
2. metadata 一致性更复杂。
3. 失败清理更复杂。
4. 网络带宽可能成为瓶颈。
5. KV 生命周期不再绑定单个 worker。

PD 分离下的关键原则是:

Decode worker 在开始 decode 前, 必须确认需要的 KV 已经在本地可访问层级, 最好已经在 GPU。

否则用户会在 streaming 过程中看到明显卡顿。

40.17 多级 KV Cache 和 Chunked Prefill

Chunked Prefill 会分批生成 KV。

这让多级缓存有了更细粒度的操作空间。

例如:

```
chunk 0 generated -> store to CPU or remote
chunk 1 generated -> store to CPU or remote
chunk 2 generated -> keep on GPU
```

好处是:

1. prefill 和 KV transfer 可以流水化。
2. 长 prompt 不必等全部完成后才开始写入远端。
3. 部分 prefix 可以提前进入缓存索引。
4. GPU 上只保留短期需要的 chunk。

但要小心：普通 causal LM 在生成首 token 前仍需要完整 prompt 的 KV。

所以 chunked prefill 与多级缓存结合时，常见目标不是“没 prefill 完就开始 decode”，而是：

1. 降低 prefill 单次调度粒度。
2. 分摊 KV transfer。
3. 降低 GPU 峰值占用。
4. 提前把冷 chunk demote 到 CPU 或 remote。

例如长 prompt：

```
prompt = 32000 tokens
chunk size = 4096
```

系统可以：

```
chunk 0 prefill -> KV to CPU
chunk 1 prefill -> KV to CPU
```

```
...
```

```
last chunk prefill -> all required KV promote to GPU for decode
```

这在实现上很复杂，因为 decode 开始前必须确保完整 KV 可被 attention kernel 正确访问。

40.18 调度器需要知道什么

多级 KV Cache 不能只由 cache manager 自己决定。scheduler 必须感知 KV 的位置和移动成本。

否则它会做出错误决策。

例如 scheduler 看到一个请求 prompt cache 命中 16000 tokens，以为很便宜：

```
hit length = 16000
uncached length = 128
```

但如果命中的 KV 全在远端，实际关键路径可能是：

```
fetch 16000 tokens KV from remote
-> copy to GPU
-> run suffix prefill
-> decode
```

这未必比重新 prefill 便宜。

调度器至少需要这些信息：

1. 命中 KV 当前在哪一层。
2. promote 到 GPU 需要多少 bytes。
3. transfer 预计耗时。
4. GPU 是否有足够 block。
5. CPU/SSD/remote 当前队列是否拥塞。
6. 如果选择 recompute，需要多少 token compute。
7. 请求的 TTFT 和 TPOT 敏感度。
8. 当前 decode batch 是否能承受一次大 promote。

因此，多级 KV Cache 会把 scheduling 从“算 token budget”升级为：

同时算 compute budget、memory budget、transfer budget 和 latency budget。

40.19 一个成本模型

可以用一个简单成本模型帮助决策。

对某段 KV，有几种选择：

1. GPU hit。

2. CPU promote。
3. Remote fetch。
4. Recompute。

分别估算：

`cost_gpu_hit = 0`

`cost_cpu_promote`
`= kv_bytes / pcie_bandwidth + copy_overhead`

`cost_remote_fetch`
`= network_rtt + kv_bytes / network_bandwidth + deserialize_overhead`

`cost_recompute`
`= prefill_tokens / prefill_throughput + scheduling_delay`

然后比较：

`choose min(cost_cpu_promote, cost_remote_fetch, cost_recompute)`

但真实系统还要加限制：

1. GPU block 是否够。
2. 当前 PCIe 是否拥塞。
3. 当前网络是否拥塞。
4. prefill worker 是否忙。
5. 用户请求是否快超时。
6. 这个 KV 是否允许跨租户复用。

所以面试里不要把成本模型说成万能公式。它只是帮助解释系统取舍。

40.20 正确性：KV 不是普通字节缓存

KV Cache 看起来像一段 tensor，但它不是普通 byte cache。

复用一段 KV 必须满足严格条件：

1. 模型权重版本一致。
2. tokenizer 版本一致。
3. token ids 一致。
4. position ids 一致。
5. RoPE scaling、sliding window、attention mask 一致。
6. dtype 和量化格式一致。
7. tensor parallel / pipeline parallel 切分方式一致。
8. adapter、LoRA、prompt adapter 一致。
9. 多模态输入的 image/audio embedding 一致。
10. 租户权限允许复用。

如果这些条件不满足，KV 命中就是错误命中。

错误命中很危险，因为它不一定报错，而是生成质量异常、事实错乱或跨用户信息泄露。

所以多级 KV Cache 的 key 不能只是 prompt hash。

它至少应该包含：

`cache_key = hash(`
`model_id,`
`model_revision,`
`tokenizer_revision,`
`token_ids,`

```
position_range,  
attention_config,  
kv_dtype,  
parallel_config,  
adapter_id,  
tenant_scope,  
multimodal_hashes  
)
```

40.21 Metadata 设计

一个 KV block 的 metadata 可能包括:

```
KVBlockMeta:  
  block_id  
  cache_key  
  token_start  
  token_end  
  num_tokens  
  layer_range  
  kv_shape  
  dtype  
  device_level  
  gpu_block_id  
  cpu_ptr  
  ssd_offset  
  remote_uri  
  ref_count  
  last_access_time  
  reuse_count  
  recompute_cost  
  transfer_cost_estimate  
  tenant_id  
  model_revision  
  checksum  
  state
```

这些字段不是都必须一开始实现, 但方向是清楚的:

1. 要知道这段 KV 是谁的。
2. 要知道它对应哪些 tokens 和 positions。
3. 要知道它现在在哪里。
4. 要知道谁正在引用它。
5. 要知道它能不能被复用、转移、淘汰或重算。

KV metadata 的一致性比 KV tensor 本身更容易出 bug。

40.22 引用计数和生命周期

Prefix KV 可能被多个请求共享。

例如:

R1, R2, R3 all use same system prompt blocks

这些 blocks 不能因为 R1 结束就释放。

需要引用计数:

ref_count = number of active users

但 ref_count 也不等于是否保留。

当 ref_count 变成 0 后, KV 可以进入 cache 状态:

ACTIVE

- > CACHED
- > DEMOTED
- > EVICTED

生命周期大致是:

ALLOCATED

- > WRITING
- > READY
- > REFERENCED
- > CACHED
- > DEMOTING / PROMOTING
- > EVICTED

每个状态都要定义清楚能否:

1. 被 attention 读取。
2. 被 prefix cache 命中。
3. 被 transfer。
4. 被 eviction。
5. 被另一个请求引用。

40.23 Partial Block 问题

Paged KV Cache 通常按 block 管理。

例如 block size = 16 tokens。

完整 block 比较容易缓存:

tokens [0, 16)
tokens [16, 32)
tokens [32, 48)

partial block 更麻烦:

tokens [48, 53)

问题包括:

1. partial block 可能还会继续写。
2. hash 边界不稳定。
3. 多个请求共享 partial block 容易出错。
4. offload 后恢复要处理未填满部分。
5. chunked prefill 可能在 block 中间结束。

很多系统会选择:

only full blocks are eligible for prefix cache sharing
partial blocks stay request-private

这样会损失一点复用率, 但能显著降低正确性风险。

40.24 多租户和安全隔离

远端 KV Cache 最大的风险之一是跨租户泄露。

KV 虽然不是原文 token, 但它包含 prompt 信息, 不能随意共享。

多租户场景下必须考虑：

1. tenant scope。
2. user scope。
3. org scope。
4. model scope。
5. adapter scope。
6. data retention policy。
7. cache encryption。
8. access control。
9. audit log。

一个保守策略是：

默认只在同 tenant、同 model revision、同 adapter、同权限域内复用 KV。

公共 system prompt 可以单独标记为 global cache，但必须由平台明确声明，而不是自动跨租户共享。

40.25 网络瓶颈

Remote KV Cache 的瓶颈通常不是 “能不能传”，而是 “传输是否值得”。

如果 KV 很大，网络会成为 TTFT 的一部分。

例如：

KV size = 2 GB
effective network bandwidth = 25 GB/s
transfer time $\approx$ 80 ms

这还没算：

1. RPC 开销。
2. 排队延迟。
3. 序列化开销。
4. GPU staging copy。
5. metadata 查询。
6. tail latency。

如果重新 prefill 只要 60 ms，那么 remote fetch 反而不划算。

所以 remote KV Cache 需要回答：

fetch KV 是否比 recompute 更便宜？

这取决于模型大小、prompt 长度、prefill 吞吐、网络带宽、GPU 空闲程度和当前队列状态。

40.26 压缩和量化

为了降低存储和传输成本，可以压缩或量化 KV。

例如：

1. FP16/BF16 KV 存储。
2. FP8 KV cache。
3. INT8 KV cache。
4. 按层压缩。
5. 冷层级压缩，promote 时解压。

这样可以降低：

1. GPU 显存占用。
2. CPU 内存占用。
3. SSD 空间。

4. 网络传输 bytes。

但代价是：

1. 可能影响模型质量。
2. 需要额外 encode/decode kernel。
3. 解压可能进入关键路径。
4. 不同层级 dtype 不一致，metadata 更复杂。
5. prefix cache key 必须包含 KV dtype 和压缩格式。

如果面试官问 “能不能把远端 KV 压缩存”，回答可以，但要强调正确性和延迟权衡。

40.27 和 Attention Kernel 的关系

Attention kernel 通常希望 KV 已经在 GPU 上，并且布局符合预期。

例如 PagedAttention 需要 block table 指向 GPU blocks。

如果 KV 在 CPU 或远端，kernel 不能直接读。

所以多级 KV Cache 不能让 attention kernel 变成：

```
if block on GPU: read GPU
if block on CPU: copy then read
if block remote: RPC then read
```

这会把 kernel 路径搞得非常复杂，也会严重拖慢 decode。

更合理的做法是：

```
scheduler/cache manager ensures required KV is GPU-resident
then attention kernel runs on normal GPU block table
```

也就是说，多级缓存主要发生在调度和准备阶段，不应该污染高频 attention kernel 的主路径。

40.28 一个简化实现

可以把系统拆成几层：

```
Scheduler
-> KVCacheManager
    -> GPUBlockManager
    -> CPUBlockStore
    -> RemoteKVClient
    -> EvictionPolicy
    -> Prefetcher
```

伪代码如下：

```
def prepare_request(req):
    hits = prefix_cache.lookup(req.cache_key)
    plan = kv_manager.plan(req, hits)

    if not plan.can_run:
        return WAIT

    for block in plan.blocks_to_promote:
        kv_manager.promote_to_gpu(block)

    for block in plan.blocks_to_recompute:
        scheduler.enqueue_prefill(block.token_range)
```

```

if kv_manager.gpu_free_blocks() < plan.required_blocks:
    victims = eviction_policy.pick_victims(plan.required_blocks)
    kv_manager.demote_or_evict(victims)

req.block_table = kv_manager.build_gpu_block_table(req)
return READY

```

这个伪代码省略了大量细节，但表达了重点：

1. 先查 prefix。
2. 再看 KV 在哪里。
3. 再决定 promote、recompute 或等待。
4. 最后构建 GPU block table。

40.29 常见策略组合

一个务实的起步版本可以这样做：

1. GPU 只保留活跃 decode 和 高频 full prefix blocks。
2. CPU 保存 idle session KV 和 刚被 preempt 的请求 KV。
3. 远端只保存明确可跨 worker 复用的公共 prefix。
4. partial block 不进入全局 cache。
5. remote fetch 只在预计比 recompute 便宜时使用。
6. prefetch 只对队首请求启用。
7. eviction 先保护活跃 decode，再保护高复用 prefix。

不要一开始就做全功能多级缓存。

更好的路线是：

```

GPU block manager
-> CPU offload for preemption
-> local prefix cache
-> remote prefix cache
-> cost-based prefetch and eviction

```

40.30 常见坑

坑一：把 CPU KV 当成 decode 在线读取层。

decode 每步都跨 PCIe 读历史 KV，TPOT 会非常差。CPU 更适合恢复前 promote，而不是每步在线读取。

坑二：只看 cache hit，不看 residency。

远端命中不等于低成本命中。scheduler 必须知道 KV 在 GPU、CPU 还是 remote。

坑三：错误复用 KV。

模型版本、tokenizer、position、adapter、tenant scope 任一不一致，都可能导致错误结果或信息泄露。

坑四：过度 prefetch。

prefetch 错了会浪费 GPU blocks、PCIe 带宽和网络带宽，还可能挤掉真正活跃的 KV。

坑五：淘汰 partial block。

partial block 边界不稳定，跨请求复用容易出错。初版系统应尽量只缓存 full blocks。

坑六：remote KV 进入 TPOT 关键路径。

如果 streaming 过程中突然从远端拉 KV，用户会看到明显卡顿。

坑七：metadata 与 tensor 生命周期不一致。

metadata 说 block 在 GPU，但实际已经释放，这是非常危险的 bug。

坑八：忽略失败清理。

transfer 中断、worker crash、remote 写入失败，都要能清理半成品 KV。

40.31 观测指标

多级 KV Cache 必须可观测。

关键指标包括：

1. GPU KV utilization。
2. CPU KV utilization。
3. SSD KV bytes。
4. remote KV bytes。
5. GPU KV hit rate。
6. CPU KV hit rate。
7. remote KV hit rate。
8. recompute fallback count。
9. promote latency。
10. demote latency。
11. prefetch success rate。
12. prefetch waste rate。
13. eviction count by reason。
14. transfer bandwidth。
15. transfer queue length。
16. KV metadata lookup latency。
17. TTFT with cache hit vs miss。
18. TPOT impact during promote/demote。
19. remote fetch p50/p95/p99。
20. cache correctness failure count。

不要只看命中率。

一个高命中率但远端 fetch 很慢的系统，用户体验可能更差。

更重要的是：

cache hit 是否真的降低了端到端延迟和 GPU compute？

40.32 调参思路

常见参数包括：

1. gpu_kv_cache_budget_bytes。
2. cpu_kv_cache_budget_bytes。
3. remote_cache_enabled。
4. max_promote_bytes_per_step。
5. max_demote_bytes_per_step。
6. prefetch_queue_depth。
7. remote_fetch_timeout_ms。
8. min_recompute_saving_threshold。
9. full_block_cache_only。
10. tenant_cache_quota_bytes。

调参目标不是让所有层级都尽量满，而是让关键路径稳定。

优先级通常是：

保护 TPOT 稳定性

-> 降低 TTFT

-> 提高吞吐

- > 提高缓存复用率
- > 降低重算成本

如果为了提高 cache hit rate 导致 TPOT 抖动，通常是错误优化。

40.33 面试官会怎么问

问题一：为什么需要多级 KV Cache？

回答要点：GPU 显存容量有限，而 KV Cache 随序列长度和并发增长很快。活跃 decode KV 需要低延迟留在 GPU，但 idle session、可复用 prefix、PD 分离中等待消费的 KV 可以放到 CPU、SSD 或远端。多级 KV Cache 用容量换延迟，用传输或重算换显存。

问题二：CPU offload 适合什么场景？

回答要点：适合被 preempt 的请求、暂时 idle 的 session、短期可能恢复且重算成本较高的 KV。不适合 decode 每步在线从 CPU 读取，因为 PCIe/NVLink 传输会严重影响 TPOT。

问题三：Remote KV Cache 和 prefix cache 是什么关系？

回答要点：prefix cache 解决“是否可以复用前缀 KV”，remote KV cache 解决“可复用 KV 放在哪里以及如何跨 worker 获取”。prefix cache 命中后，还要看 KV 是 GPU-resident、CPU-resident 还是 remote-only。

问题四：Remote fetch 一定比 recompute 好吗？

回答要点：不一定。要比较 KV bytes、网络带宽、RPC 延迟、队列拥塞、prefill 吞吐和 GPU 空闲程度。如果 remote fetch 比重新 prefill 更慢，或者会影响 TPOT，就不应该 fetch。

问题五：多级 KV Cache 的正确性风险是什么？

回答要点：KV 复用必须保证模型版本、tokenizer、token ids、position、attention 配置、dtype、parallel config、adapter、多模态输入和租户权限一致。错误命中可能不会报错，但会导致生成异常或信息泄露。

问题六：实现多级 KV Cache 最容易踩什么坑？

回答要点：只看命中率不看位置和传输成本；让远端 fetch 进入 decode 关键路径；partial block 复用错误；metadata 和实际 tensor 生命周期不一致；prefetch 过度；缺少失败清理和租户隔离。

40.34 标准回答模板

如果面试官问“设计一个多级 KV Cache”，可以这样回答：

我会先把 KV 分成热、温、冷几类。活跃 decode 的 KV 必须留在 GPU，因为 attention kernel 每步都要读，不能让 CPU 或远端访问进

系统上我会让 scheduler、KV cache manager 和 prefix cache 协同工作。prefix cache 命中时，不只返回 hit length，还要返回 blo

正确性上，KV cache key 必须包含 model revision、tokenizer、token ids、position、attention config、dtype、parallel config、private。

策略上，先保证活跃 decode 的 TPOT 稳定，再优化 TTFT 和命中率。remote fetch 和 prefetch 都不能无脑做，必须比较 transfer co

40.35 多级 KV Cache 成本、驻留门禁和可运行 demo

先把一个 KV block 抽象成：

$B_j = (b_j, u_j, m_j, L_j, N_j, M_j, R_j, F_j)$

其中 b_j 是 block id， u_j 是 tenant， m_j 是 model revision， L_j 是 residency level， N_j 是 token 数， M_j 是 KV 大小， R_j 是 ref count， F_j 是 reuse / frequency 信号。

GPU 容量约束可以写成：

$\sum_{j: L_j = \text{GPU}} M_j \leq M_{\text{gpu}}$

CPU promote 成本:

$$C_{\{\mathrm{cpu}\},j}=\frac{M_j}{B_{\{\mathrm{cpu}\}}}+\delta_{\{\mathrm{cpu}\}}$$

Remote fetch 成本:

$$C_{\{\mathrm{remote}\},j}=R_{\{\mathrm{net}\}}+\frac{M_j}{B_{\{\mathrm{net}\}}}+\delta_{\{\mathrm{remote}\}}$$

重算成本:

$$C_{\{\mathrm{recompute}\},j}=\frac{N_j}{X_{\{\mathrm{prefill}\}}}+\delta_{\{\mathrm{sched}\}}$$

一个教学版决策可以写成:

$$A_j=\min(C_{\{\mathrm{cpu}\},j},C_{\{\mathrm{remote}\},j},C_{\{\mathrm{recompute}\},j})$$

但这个 min 只能在正确性门通过后使用。tenant、model revision、token ids、position、dtype、parallel config、adapter 和 multimodal hash 任一不兼容，都不能复用。

最终门禁可以写成:

$$G_{\{\mathrm{mlkv}\}}=G_{\{\mathrm{hot}\}}G_{\{\mathrm{cpu}\}}G_{\{\mathrm{remote}\}}G_{\{\mathrm{recompute}\}}G_{\{\mathrm{tenant}\}}G_{\{\mathrm{mat}\}}$$

下面的 demo 覆盖三类请求:

1. resume_rag: 需要一个 CPU-resident block、一个 remote block 和一个 missing tail。
2. cross_tenant_probe: 看到了 remote block, 但属于其他 tenant, 必须阻断。
3. cold_doc: SSD 上有冷 block, 但 fetch 比 recompute 更慢, 选择 recompute。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class KVBlock:
    block_id: str
    tenant: str
    model_revision: str
    level: str
    tokens: int
    ref_count: int
    reuse_count: int
```

```
@dataclass
```

```
class KVRequest:
    request_id: str
    tenant: str
    model_revision: str
    needed_blocks: list
```

```
class ToyMultiLevelKVCacheAuditor:
```

```
    def __init__(self, gpu_capacity_blocks=4, block_mib=16.0):
        self.gpu_capacity_blocks = gpu_capacity_blocks
        self.block_mib = block_mib
        self.cpu_mib_per_ms = 8.0
        self.remote_mib_per_ms = 8.0
        self.ssd_mib_per_ms = 2.0
        self.prefill_tokens_per_ms = 256.0
        self.blocks = {
            "active_a": KVBlock("active_a", "tenant_a", "v1", "GPU", 128, 1, 20),
            "active_b": KVBlock("active_b", "tenant_a", "v1", "GPU", 128, 1, 18),
```

```

        "prefix_hot": KVBlock("prefix_hot", "tenant_a", "v1", "GPU", 128, 0, 8),
        "idle_cpu": KVBlock("idle_cpu", "tenant_a", "v1", "CPU", 128, 0, 5),
        "doc_remote": KVBlock("doc_remote", "tenant_a", "v1", "REMOTE", 128, 0, 12),
        "shared_doc_b": KVBlock("shared_doc_b", "tenant_b", "v1", "REMOTE", 128, 0, 12),
        "cold_ssd": KVBlock("cold_ssd", "tenant_a", "v1", "SSD", 512, 0, 1),
    }
    self.demotions = []
    self.evictions = []

def _gpu_used(self):
    return sum(block.level == "GPU" for block in self.blocks.values())

def _ensure_gpu_space(self):
    if self._gpu_used() < self.gpu_capacity_blocks:
        return []
    candidates = [
        block for block in self.blocks.values()
        if block.level == "GPU" and block.ref_count == 0
    ]
    if not candidates:
        return []
    victim = min(candidates, key=lambda block: block.reuse_count)
    victim.level = "CPU"
    self.demotions.append(victim.block_id)
    return [victim.block_id]

def _compatible(self, request, block):
    return request.tenant == block.tenant and request.model_revision == block.model_revision

def _cpu_cost(self):
    return round(self.block_mib / self.cpu_mib_per_ms + 0.2, 3)

def _remote_cost(self):
    return round(1.0 + self.block_mib / self.remote_mib_per_ms + 0.3, 3)

def _ssd_cost(self):
    return round(1.0 + self.block_mib / self.ssd_mib_per_ms + 0.5, 3)

def _recompute_cost(self, tokens):
    return round(tokens / self.prefill_tokens_per_ms + 0.5, 3)

def handle_block(self, request, block_id):
    if block_id not in self.blocks:
        return {
            "request_id": request.request_id,
            "block_id": block_id,
            "action": "recompute_missing",
            "cost_ms": self._recompute_cost(512),
            "transfer_mib": 0.0,
        }
    block = self.blocks[block_id]
    if not self._compatible(request, block):
        return {
            "request_id": request.request_id,
            "block_id": block_id,

```

```

        "action": "blocked_incompatible_or_tenant",
        "cost_ms": 0.0,
        "transfer_mib": 0.0,
    }
    if block.level == "GPU":
        block.ref_count += 1
        return {
            "request_id": request.request_id,
            "block_id": block_id,
            "action": "gpu_hit",
            "cost_ms": 0.0,
            "transfer_mib": 0.0,
        }
    if block.level == "CPU":
        demoted = self._ensure_gpu_space()
        block.level = "GPU"
        block.ref_count += 1
        return {
            "request_id": request.request_id,
            "block_id": block_id,
            "action": "promote_cpu_to_gpu",
            "cost_ms": self._cpu_cost(),
            "transfer_mib": self.block_mib,
            "demoted": demoted,
        }
    if block.level == "REMOTE":
        remote_cost = self._remote_cost()
        recompute_cost = self._recompute_cost(block.tokens * 16)
        if remote_cost <= recompute_cost:
            demoted = self._ensure_gpu_space()
            block.level = "GPU"
            block.ref_count += 1
            return {
                "request_id": request.request_id,
                "block_id": block_id,
                "action": "fetch_remote_to_gpu",
                "cost_ms": remote_cost,
                "transfer_mib": self.block_mib,
                "demoted": demoted,
            }
        return {
            "request_id": request.request_id,
            "block_id": block_id,
            "action": "recompute_instead_of_remote",
            "cost_ms": recompute_cost,
            "transfer_mib": 0.0,
        }
    if block.level == "SSD":
        ssd_cost = self._ssd_cost()
        recompute_cost = self._recompute_cost(block.tokens)
        action = "fetch_ssd_to_gpu" if ssd_cost <= recompute_cost else "recompute_instead_of_ssd"
        return {
            "request_id": request.request_id,
            "block_id": block_id,
            "action": action,

```

```

        "cost_ms": min(ssd_cost, recompute_cost),
        "transfer_mib": self.block_mib if action == "fetch_ssd_to_gpu" else 0.0,
    }
    raise ValueError(block.level)

def audit(self, requests):
    rows = []
    for request in requests:
        for block_id in request.needed_blocks:
            rows.append(self.handle_block(request, block_id))
    summary = {
        "requests": len(requests),
        "gpu_hits": sum(row["action"] == "gpu_hit" for row in rows),
        "cpu_promotes": sum(row["action"] == "promote_cpu_to_gpu" for row in rows),
        "remote_fetches": sum(row["action"] == "fetch_remote_to_gpu" for row in rows),
        "ssd_fetches": sum(row["action"] == "fetch_ssd_to_gpu" for row in rows),
        "recomputes": sum(row["action"].startswith("recompute") for row in rows),
        "tenant_blocks": sum(row["action"] == "blocked_incompatible_or_tenant" for row in rows),
        "demotions": len(self.demotions),
        "evictions": len(self.evictions),
        "gpu_used_blocks": self._gpu_used(),
        "cpu_used_blocks": sum(block.level == "CPU" for block in self.blocks.values()),
        "remote_used_blocks": sum(block.level == "REMOTE" for block in self.blocks.values()),
        "total_transfer_mib": round(sum(row["transfer_mib"] for row in rows), 1),
        "estimated_latency_ms": round(sum(row["cost_ms"] for row in rows), 3),
    }
    gates = {
        "active_gpu_blocks_protected": all(
            self.blocks[block_id].level == "GPU" for block_id in ["active_a", "active_b"]
        ),
        "cpu_promote_visible": summary["cpu_promotes"] == 1,
        "remote_fetch_visible": summary["remote_fetches"] == 1,
        "recompute_fallback_visible": summary["recomputes"] == 2,
        "tenant_isolation_enforced": summary["tenant_blocks"] == 1,
        "demotion_makes_gpu_space": summary["demotions"] == 1,
        "residency_metrics_ready": (
            summary["total_transfer_mib"] == 32.0
            and summary["gpu_used_blocks"] == 4
        ),
    }
    gates["multi_level_kv_gate"] = all(gates.values())
    return rows, summary, gates

```

```

requests = [
    KVRequest("resume_rag", "tenant_a", "v1", ["idle_cpu", "doc_remote", "missing_tail"]),
    KVRequest("cross_tenant_probe", "tenant_a", "v1", ["shared_doc_b"]),
    KVRequest("cold_doc", "tenant_a", "v1", ["cold_ssd"]),
]

```

```

rows, summary, gates = ToyMultiLevelKVCacheAuditor().audit(requests)
print("multi_level_kv_rows=", rows)
print("multi_level_kv_summary=", summary)
print("multi_level_kv_gates=", gates)

```

一次运行的核心输出类似：


```
multi_level_kv_summary= {'requests': 3, 'gpu_hits': 0, 'cpu_promotes': 1, 'remote_fetches': 1, 'ssd_fetches': 0, 'recompute_fallbacks': 1, 'tenant_isolation_enforced': True, 'demotion_makes_gpu_space': True, 'residency_metrics_ready': True}
multi_level_kv_gates= {'active_gpu_blocks_protected': True, 'cpu_promote_visible': True, 'remote_fetch_visible': True, 'recompute_fallback_visible': True, 'tenant_isolation_enforced': True, 'demotion_makes_gpu_space': True, 'residency_metrics_ready': True}
```

这个 demo 展示了几个关键点：

1. `active_gpu_blocks_protected=True`：有 ref count 的 active decode KV 没有被 demote。
2. `cpu_promote_visible=True`：CPU-resident KV 可以 promote 回 GPU，但要计 transfer 成本。
3. `remote_fetch_visible=True`：远端 KV 在比重算更便宜时进入 GPU。
4. `recompute_fallback_visible=True`：missing block 和 SSD 冷 block 都走 recompute，说明 cache hit 不等于一定 fetch。
5. `tenant_isolation_enforced=True`：其他 tenant 的 remote block 被阻断。
6. `demotion_makes_gpu_space=True`：GPU 满时只 demote ref count 为 0 的 block，为 remote fetch 腾空间。
7. `residency_metrics_ready=True`：summary 同时给出 transfer MiB、GPU / CPU / remote residency 和 latency。

所以本章最终门禁是 `multi_level_kv_gate`：只有 hot KV 保护、CPU promote、remote fetch、recompute fallback、tenant isolation、demotion 和 residency metrics 都能闭环，多级 KV Cache 才不是“多加几层存储”，而是可治理的运行时缓存系统。

40.36 小练习

1. 画出 GPU、CPU、SSD、Remote、Recompute 五层 KV Cache 的访问路径。
2. 解释为什么 CPU KV 不适合作为 decode 每步在线读取层。
3. 设计一个判断 remote fetch 还是 recompute 的成本模型。
4. 给一个 32000-token prompt，分析哪些 KV 应该留 GPU，哪些可以 demote。
5. 说明 prefix cache 命中为什么还要返回 residency level。
6. 设计 `KVBlockMeta` 的字段，并解释每个字段用途。
7. 分析 partial block 为什么不适合跨请求共享。
8. 设计一个多租户 KV Cache 的隔离策略。
9. 列出 10 个观测多级 KV Cache 的指标。
10. 说明多级 KV Cache 如何服务 PD 分离。

40.37 本章总结

多级 KV Cache 的核心是把 KV 从 GPU 显存中的临时对象，升级成跨层级、跨节点、可迁移、可淘汰、可预取的系统级资源。

GPU 层负责活跃 decode 和低延迟访问；CPU 层适合 idle session、preemption 和短期恢复；SSD 和远端层适合更冷、更大、跨 worker 复用的 KV；recompute 则是最后的兜底层。

多级缓存不是单纯提高命中率。真正重要的是在显存、计算、传输、延迟和正确性之间做权衡。

实现时，scheduler 必须知道 KV 的 residency 和移动成本；prefix cache 必须和 block manager 结合；attention kernel 的主路径应尽量只面对 GPU-resident KV；cache key 必须包含模型、token、position、adapter、并行配置和租户权限等正确性信息。

下一章会继续讨论跨节点 serving 和网络瓶颈。多级 KV Cache 一旦进入远端访问，就不可避免地受到网络拓扑、带宽、延迟、拥塞和通信模式的限制。

第 41 章 跨节点 Serving 和网络瓶颈

上一章讲了多级 KV Cache：当 KV 不能只放在单个 GPU 上时，需要 GPU、CPU、SSD、远端缓存和 recompute 共同组成一个系统级缓存层级。

一旦 KV、模型分片、请求调度或 prefix cache 走出单机，网络就会进入关键路径。

跨节点 serving 的难点不只是“多几台机器”。真正的问题是：

模型计算、KV Cache、调度状态、请求流和 streaming 输出，哪些必须跨节点通信？
这些通信是否会进入 TTFT 或 TPOT 的关键路径？

一句话概括：

跨节点 serving 的核心不是把 GPU 堆起来，而是在模型并行、数据并行、PD 分离、KV transfer 和网络拓扑之间做取舍，避免网络把吞吐收益吃掉。

41.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 WRITING_PLAN.md 对公开资料做校准：参考 vLLM parallelism / scaling 文档对 tensor parallel、pipeline parallel、data parallel、单节点 / 多节点部署和 Ray backend 的公开口径；参考 SGLang server arguments、PD disaggregation 和多节点部署相关文档对 TP / DP、PD worker、KV transfer、routing policy 和多节点运行边界的说明；参考 NCCL collective communication 文档对 all-reduce、all-gather、reduce-scatter 等 collective 通信的稳定抽象；并参考 DistServe、Splitwise、Mooncake 等论文对 P/D 分离、KV transfer、goodput、跨节点资源池和网络代价的系统动机说明。

本讲只讲跨节点 serving 的通用网络瓶颈：DP 请求分流、TP 高频 collective、PP activation 传输、PD KV transfer、remote KV fetch、topology-aware routing、data plane / control plane 隔离、admission control、backpressure 和 p99 观测。不把某个框架版本的 CLI 参数、Ray / Kubernetes 部署命令、NCCL 环境变量、网卡型号、带宽实测值或 benchmark 排名写成通用标准。

本讲新增 demo 是教学版跨节点网络瓶颈审计器：用 0 依赖 Python 模拟 same-node / same-rack / cross-rack 链路，比较跨节点 TP 的 per-token collective 代价、PP activation 代价、PD KV transfer 成本、remote fetch vs recompute、拓扑感知 decode worker 路由、pending transfer backpressure 和 control / data plane 分离，帮助把“网络会吃掉跨节点扩展收益”落到可运行证据。

41.1 本章目标

读完本章，你应该能讲清：

1. 跨节点 serving 为什么比单机多卡复杂得多。
2. TP、PP、DP、PD 分离在跨节点时分别产生什么网络通信。
3. 为什么 TP 通常不适合跨低速网络。
4. KV transfer、remote KV cache 和 prefix sharing 如何受网络影响。
5. 带宽、延迟、拥塞、拓扑距离如何影响 TTFT、TPOT 和吞吐。
6. 如何判断网络瓶颈来自 all-reduce、pipeline activation、KV transfer 还是 router。
7. 跨节点 serving 的部署策略和调优方法。
8. 面试中如何设计一个跨节点推理服务。

41.2 跨节点 Serving 到底跨了什么

跨节点不一定只有一种形态。

常见有四类：

1. 多个独立 replica，外部负载均衡。
2. 一个模型 replica 内部跨节点做 TP/PP。
3. Prefill worker 和 Decode worker 跨节点，也就是跨节点 PD 分离。
4. KV Cache、prefix cache 或 session state 跨节点共享。

它们的通信模式完全不同。

DP replica:

request -> one replica
replicas mostly independent

TP across nodes:

every layer may communicate across nodes

PP across nodes:
stage boundary transfers hidden states

PD across nodes:
prefill side transfers KV to decode side

remote KV cache:
worker fetches or pushes KV blocks over network

所以不能笼统说“跨节点慢”。要问清楚：跨节点的是请求、激活、KV、权重分片通信，还是控制面 metadata。

41.3 网络为什么会进入关键路径

Serving 的延迟指标主要有两个：

1. TTFT：从请求进入系统到首 token 返回。
2. TPOT：生成过程中每个 token 的平均或尾部间隔。

网络可能进入 TTFT：

request arrives
-> router selects remote worker
-> remote prefix lookup
-> fetch KV from remote cache
-> prefill or decode
-> first token returns

网络也可能进入 TPOT：

decode step
-> TP all-reduce across nodes
-> next layer
-> TP all-reduce across nodes
-> ...
-> output token

或者：

decode step waits for missing KV from remote

只影响 TTFT 的网络开销，有时还能接受。影响每个 decode step 的网络开销，通常更危险。

因为 decode 是高频循环：

one network hiccup per request -> occasional TTFT spike
one network hiccup per token -> continuous TPOT jitter

41.4 带宽和延迟不是一回事

网络瓶颈通常分成两个维度：

1. Bandwidth：单位时间能传多少数据。
2. Latency：一次通信来回或单向需要多久。

大对象传输更看带宽。

例如 KV transfer：

KV bytes = 2 GB
effective bandwidth = 25 GB/s
transfer time $\approx$ 80 ms

小而频繁的通信更看延迟。

例如 TP all-reduce:

每层一次或多次 `collective`
每个 `decode token` 走几十层

即使每次传的数据不大，如果通信频率极高，延迟也会累积成 TPOT。

面试中可以这样说：

KV transfer 更偏大块带宽问题，跨节点 TP 更偏高频 `latency + collective bandwidth` 问题。

41.5 常见互联层级

从快到慢，常见互联大致是：

GPU HBM
-> NVLink / NVSwitch
-> PCIe
-> InfiniBand / RoCE / RDMA network
-> ordinary Ethernet

不同层级适合的通信不同。

互联	典型范围	特点	适合通信
NVLink/NVSwitch	单机多 GPU	低延迟高带宽	TP、GPU-GPU KV copy
PCIe	单机 CPU-GPU/GPU-GPU	带宽和拓扑受限	CPU offload、一般 GPU copy
InfiniBand/RDMA	跨节点	高性能网络，但部署复杂	PP、PD KV transfer、跨节点通信
RoCE	跨节点以太网 RDMA	依赖网络配置	RDMA transfer、分布式 serving
普通以太网	跨节点	延迟高、抖动大	控制面、低频请求转发

跨节点 serving 通常要尽量把高频同步通信限制在高速互联范围内。

41.6 Data Parallel 跨节点

Serving 中的 Data Parallel 通常是多个独立 replica。

例如：

router
-> replica 0: model copy + KV cache
-> replica 1: model copy + KV cache
-> replica 2: model copy + KV cache
-> replica 3: model copy + KV cache

DP 的跨节点通信最少。

每个请求只进入一个 replica，decode 过程中不需要和其他 replica 同步。

它的优点是：

1. 横向扩吞吐简单。
2. 故障隔离好。
3. TPOT 不依赖跨 replica 通信。
4. 调试相对容易。

它的代价是：

1. 每个 replica 都要一份权重。
2. 每个 replica 有独立 KV Cache。

3. prefix cache 容易重复。
4. 负载均衡要考虑 KV locality。
5. 热点 session 可能集中到某个 replica。

DP 适合模型单个 replica 能放下，主要目标是提升总 QPS 的场景。

41.7 Tensor Parallel 跨节点

Tensor Parallel 把每层矩阵或 attention heads 切到多个 rank。

它的问题是通信频率高。

decode 每生成一个 token，都要经过所有层。每层可能需要：

1. all-reduce。
2. all-gather。
3. reduce-scatter。
4. rank 间同步。

如果 TP group 跨节点，通信路径变成：

```
GPU rank on node A
-> network
-> GPU rank on node B
```

这会直接影响 TPOT。

因此常见原则是：

TP 尽量限制在单节点 NVLink/NVSwitch 内。
跨节点扩展优先考虑 PP 或 DP。

例如 2 台 8 卡机器：

推荐：TP=8, PP=2
谨慎：TP=16, PP=1

前者把高频 TP 通信留在节点内，只让 pipeline stage 跨节点传 hidden states。后者每层都可能跨节点 collective，TPOT 更容易变差。

当然，如果网络极强、collective 优化很好、模型必须这样切，也不是绝对不能跨节点 TP。但它应该是经过压测验证的选择，而不是默认选择。

41.8 Pipeline Parallel 跨节点

Pipeline Parallel 按层切分模型。

跨节点时，stage 之间传递 hidden states。

例如：

```
node 0: layers 0-19
node 1: layers 20-39
```

请求执行路径是：

```
input hidden states
-> node 0 forward
-> transfer activations to node 1
-> node 1 forward
-> logits
```

PP 的跨节点通信频率通常低于 TP，因为它不是每层都 collective，而是在 stage 边界传 activation。

但 PP 有自己的问题：

1. 单请求延迟增加。
2. pipeline bubble。
3. stage 负载不均。
4. continuous batching 更复杂。
5. stage 之间的通信也会影响 TPOT。

Serving 中 decode step 很小，如果 batch 不够或调度不好，pipeline 很容易填不满。

所以 PP 适合：

1. 模型单节点放不下。
2. 跨节点 TP 通信太贵。
3. 层切分比较均衡。
4. 有足够并发填 pipeline。

不适合：

1. 极低延迟、小并发场景。
2. stage 间网络很差。
3. 模型层耗时严重不均且无法均衡切分。

41.9 TP + PP 组合

大模型 serving 常见组合是节点内 TP、节点间 PP。

例如 4 台机器，每台 8 张 GPU：

TP = 8

PP = 4

total GPUs = 32

含义是：

每个 pipeline stage 用一整台机器的 8 张 GPU 做 TP

4 台机器按层组成 PP

这样做的好处是：

1. 每层内部高频 TP 通信走节点内 NVLink/NVSwitch。
2. 跨节点只传 stage activation。
3. 可以支持单节点放不下的大模型。
4. 网络通信模式更可控。

代价是：

1. 单个 replica 占用大量 GPU。
2. pipeline bubble 需要并发填充。
3. 任一节点慢都会拖慢整个 replica。
4. KV Cache 也按 stage/rank 分布，管理更复杂。

面试中这个组合非常常见，应该熟练表达。

41.10 PD 分离跨节点

PD 分离把 prefill 和 decode 放到不同 worker 或资源池。

跨节点 PD 的通信核心是 KV transfer。

流程大致是：

request -> prefill worker

prefill worker computes prompt KV

prefill worker transfers KV to decode worker

decode worker starts streaming decode

网络开销主要体现在：

1. prefill 完成后 KV 传输时间。
2. decode worker 等待 KV 的时间。
3. 多个 transfer 并发导致的网络拥塞。
4. 传输失败或超时后的恢复。

PD 分离是否值得，取决于：

P/D 资源隔离收益

> KV transfer 成本 + 调度复杂度 + 网络抖动成本

如果网络很差，跨节点 PD 可能把原来的 compute interference 变成 network interference。

这也是为什么 PD 分离常常需要高性能 transfer backend，例如 RDMA、GPUDirect RDMA、UCX、NIXL、Mooncake 这类能力或系统。

41.11 KV Transfer 的网络成本

KV transfer 的数据量与 prompt length、层数、KV heads、head dim、dtype 和 TP 切分有关。

可以粗略理解为：

```
kv_bytes
~= num_tokens
   * num_layers
   * kv_heads
   * head_dim
   * 2 # K and V
   * bytes_per_element
```

如果模型使用 GQA/MQA，KV heads 更少，KV transfer 压力会降低。

如果 prompt 很长，KV transfer 会非常大。

例如：

```
short prompt: 512 tokens
long prompt: 32000 tokens
```

后者 KV bytes 可能是前者的 62.5 倍。

因此跨节点 PD 对长 prompt 更敏感。Chunked prefill 可以把 transfer 流水化，但不能消除总 bytes。

41.12 Remote KV Cache 的网络成本

Remote KV Cache 和 PD KV transfer 很像，但访问模式可能不同。

PD transfer 通常是一次请求内从 prefill 到 decode 的交接。

Remote KV Cache 可能是多请求、多 worker 共享的缓存。

访问路径可能是：

```
worker -> metadata service -> remote KV store -> worker
```

它的成本包括：

1. metadata lookup。
2. remote read/write。
3. 网络传输。
4. 反序列化或解压。
5. CPU staging。
6. GPU copy。
7. consistency check。

所以 remote cache hit 不等于快。

更准确的判断是：

$\text{remote_hit_latency} + \text{promote_latency} < \text{recompute_latency}$

并且不能明显影响其他请求的 TPOT。

41.13 Router 也会制造网络瓶颈

很多人只关注 GPU 间通信，忽略 router。

跨节点 serving 中，router 负责：

1. 选择 replica。
2. 选择 prefill worker。
3. 选择 decode worker。
4. 根据 KV locality 路由。
5. 做 admission control。
6. 管理 streaming 连接。
7. 处理重试和故障转移。

如果 router 不理解网络拓扑，就可能做出坏决策。

例如：

prefill worker on rack A
decode worker on rack B
remote KV cache on rack C

这个请求可能经历多次跨 rack 传输。

更好的路由要考虑：

1. worker 当前负载。
2. GPU KV capacity。
3. prefix cache locality。
4. P/D worker 网络距离。
5. pending KV transfers。
6. 当前网络拥塞。
7. tenant 或 session affinity。

这就是 topology-aware routing 和 KV-aware routing。

41.14 拓扑距离

网络拓扑不是平的。

常见层级包括：

same GPU island
same node
same rack
same cluster fabric
cross rack
cross zone

同样是“远端”，距离可能完全不同。

例如：

same node NVLink transfer: very fast
same rack RDMA: acceptable
cross rack under congestion: unstable

cross zone: usually not for TPOT path

跨节点 serving 的原则是：

高频、低延迟通信放近；低频、大块、可异步通信可以放远。

对应到不同通信：

1. TP collective: 尽量同机。
2. PP activation: 尽量同 rack 或高性能 fabric。
3. PD KV transfer: 最好 topology-aware 配对。
4. remote KV cache: 尽量按 locality 分片或复制。
5. 控制面 metadata: 可以更远，但要有缓存和超时。

41.15 拥塞和尾延迟

网络平均带宽够，不代表 serving 稳定。

LLM serving 很怕 tail latency。

例如：

p50 transfer latency = 20 ms

p99 transfer latency = 300 ms

如果 p99 transfer 在 TTFT 关键路径上，用户会感到首 token 很慢。

如果 p99 communication 在 decode step 中，streaming 会卡顿。

造成 tail latency 的原因包括：

1. 多个 worker 同时传大 KV。
2. collective 与 KV transfer 争用网络。
3. remote cache 热点 shard。
4. rack 间 oversubscription。
5. RDMA 配置或网卡队列问题。
6. TCP retransmission 或 congestion control。
7. metadata service 抖动。

因此观测 p50 不够，必须看 p95/p99。

41.16 Control Plane 和 Data Plane

跨节点 serving 可以分成控制面和数据面。

控制面包括：

1. worker 注册。
2. heartbeat。
3. 负载信息上报。
4. KV metadata 查询。
5. 路由决策。
6. transfer plan。
7. 失败恢复。

数据面包括：

1. 请求输入。
2. streaming token 输出。
3. KV transfer。
4. activation transfer。
5. collective communication。
6. remote cache read/write。

控制面通常数据量小，但对一致性和超时敏感。

数据面数据量大，直接影响带宽和延迟。

设计上要避免控制面阻塞数据面，也要避免数据面拥塞导致控制面心跳误判。

41.17 Streaming 输出路径

Serving 不是只算完返回一个结果，很多场景需要 token streaming。

跨节点时，streaming 路径可能是：

decode worker -> router/gateway -> client

也可能是：

decode worker -> frontend connection owner -> client

需要注意：

1. token 输出很小但频繁。
2. 不应被大 KV transfer 阻塞。
3. gateway 不能成为单点瓶颈。
4. backpressure 要能传回 decode scheduler。
5. worker fail 后 streaming 如何结束要明确。

如果 streaming channel 和大对象 transfer 共用同一拥塞路径，也可能出现用户看到 token 断断续续的问题。

41.18 跨节点 Batch 和负载均衡

Continuous batching 在单机内已经复杂，跨节点后更复杂。

对于 DP replicas，router 要把请求分配给不同 replica。

简单 least-loaded 策略可能不够。

因为一个 worker 负载低，不代表它适合这个请求。

还要看：

1. 该请求 prefix 是否在这个 worker。
2. 这个 worker 的 KV blocks 是否足够。
3. 当前 decode batch 是否稳定。
4. 是否有 pending transfer。
5. 网络距离是否近。
6. 租户配额是否允许。

例如：

worker A: load 70%, prefix hit on GPU

worker B: load 20%, no prefix hit, remote fetch required

worker A 可能反而更快。

所以跨节点路由目标不是最小当前负载，而是最小预计完成成本。

41.19 Admission Control

跨节点系统更需要 admission control。

如果没有限制，可能出现：

大量长 prompt 同时进入

-> prefill worker 忙

-> KV transfer 排队

- > decode worker 等 KV
- > network saturated
- > TTFT/TPOT 全部恶化

Admission control 可以限制:

1. 同时进行的 KV transfer 数。
2. 每个 worker 的 pending transfer bytes。
3. 每个租户的长 prompt 并发。
4. remote cache read/write QPS。
5. 跨 rack traffic。
6. prefill queue length。
7. decode queue length。

一个简单规则是:

```
if pending_kv_transfer_bytes > threshold:
    stop admitting long-prefill requests
```

这不是降低吞吐, 而是避免系统进入拥塞崩溃。

41.20 Backpressure

Backpressure 是下游压力向上传递。

跨节点 PD 中尤其重要。

例如 decode worker 消费不过来:

```
decode worker KV capacity low
-> pending transfers increase
-> prefill workers should slow down
-> router should stop assigning more long prompts
```

如果没有 backpressure, prefill 侧会继续生成大量 KV, 最后造成:

1. 网络排队。
2. CPU staging buffer 堆积。
3. GPU blocks 预留失败。
4. 请求超时。
5. 大量无用 prefill 计算。

Backpressure 信号可以包括:

1. decode queue length。
2. available KV blocks。
3. pending transfer bytes。
4. network utilization。
5. remote cache latency。
6. TPOT p95/p99。

41.21 故障模式

跨节点 serving 的故障模式比单机多。

常见包括:

1. worker crash。
2. network timeout。
3. partial KV transfer。
4. remote cache write succeeded but metadata update failed。
5. metadata says KV exists but data missing。

6. TP/PP group 中一个 rank 掉线。
7. router 重试导致重复请求。
8. streaming 中途断开。
9. prefill 成功但 decode worker 已不可用。
10. remote cache shard 热点或不可用。

系统要明确每种失败后的策略：

1. retry。
2. reroute。
3. recompute。
4. cancel request。
5. clean partial KV。
6. degrade to local only。

不要让半完成的 KV 长期占用 cache，也不要让 metadata 指向不存在的数据。

41.22 观测指标

跨节点 serving 的指标要按通信类型拆开。

总体指标：

1. TTFT p50/p95/p99。
2. TPOT p50/p95/p99。
3. end-to-end latency。
4. throughput tokens/s。
5. request timeout rate。

网络数据面指标：

1. KV transfer bytes/s。
2. KV transfer latency p50/p95/p99。
3. remote KV read latency。
4. remote KV write latency。
5. activation transfer latency。
6. collective communication time。
7. RDMA/TCP error count。
8. network retransmission。
9. NIC utilization。
10. cross-rack traffic bytes。

调度指标：

1. pending transfer count。
2. pending transfer bytes。
3. prefill queue length。
4. decode queue length。
5. router decision latency。
6. topology-aware routing hit rate。
7. prefix locality hit rate。
8. admission rejection count。
9. backpressure activation count。

GPU 侧指标：

1. GPU utilization。
2. GPU memory utilization。
3. KV block utilization。
4. preemption count。
5. decode step latency。

6. prefill latency。

关键是把延迟拆到阶段，否则只看总 latency 不知道瓶颈在哪。

41.23 如何定位网络瓶颈

可以按路径定位。

第一步，看是 TTFT 还是 TPOT 变差。

TTFT 变差，TPOT 正常：

优先看 prefill queue、remote prefix lookup、KV transfer、admission、router。

TPOT 变差：

优先看 decode step、TP collective、PP activation、remote KV 是否进入 decode 路径、网络拥塞。

第二步，看是否和请求长度相关。

长 prompt 更差：可能是 prefill 或 KV transfer。

长 output 更差：可能是 decode step 或 TP/PP 通信。

第三步，看是否和拓扑相关。

same-node 正常，cross-node 差：网络或跨节点通信。

same-rack 正常，cross-rack 差：拓扑/oversubscription。

某些 worker 差：网卡、链路、配置或热点。

第四步，看 p99 是否远高于 p50。

p50 正常，p99 很差：拥塞、排队、热点、重试或 metadata 抖动。

41.24 部署策略

一个务实的部署策略是：

1. 能用 DP 扩吞吐时，优先用 DP。
2. 单节点能放下模型时，尽量避免跨节点模型并行。
3. 需要 TP 时，优先把 TP 限制在单节点高速互联内。
4. 模型单节点放不下时，用节点内 TP + 节点间 PP。
5. PD 分离跨节点时，让 P/D worker topology-aware 配对。
6. remote KV cache 不要无脑进入关键路径。
7. 大 KV transfer 要做 admission control 和 backpressure。
8. prefix cache 要考虑 locality，而不是全局命中率至上。

可以用一个决策树：

模型单节点能放下？

yes -> 用单节点 TP + 多 replica DP

no -> 节点内 TP + 节点间 PP

长 prompt 严重干扰 decode？

yes -> 考虑 PD 分离

-> 网络能承受 KV transfer？

yes -> 跨节点 PD

no -> 同机 PD 或 chunked prefill

prefix 复用率很高？

yes -> local prefix cache first

-> remote cache only if fetch < recompute

41.25 调优思路

调优跨节点 serving，不要一上来改复杂架构。

先做基线：

1. 单机单卡。
2. 单机多卡 TP。
3. 多 replica DP。
4. 节点内 TP + 节点间 PP。
5. PD 分离。
6. remote KV cache。

每一步都记录：

1. TTFT。
2. TPOT。
3. tokens/s。
4. GPU utilization。
5. network utilization。
6. p95/p99。

常见调参方向：

1. 减小跨节点 TP group。
2. 增加 DP replicas。
3. 调整 PP stage 切分。
4. 限制并发 KV transfer bytes。
5. 提高 prefix locality routing 权重。
6. 限制 remote cache 进入关键路径。
7. 为 streaming 与大对象 transfer 隔离队列或连接。
8. 增加 backpressure 阈值。

41.26 常见坑

坑一：把 TP 横跨普通网络。

每层高频 collective 进入网络，TPOT 可能严重恶化。

坑二：只看总带宽，不看 p99。

Serving 对尾延迟敏感，p99 transfer 或 p99 collective 会直接影响用户体验。

坑三：remote cache hit 率很高但延迟更差。

远端命中如果比 recompute 还慢，就是错误优化。

坑四：router 只看 worker load。

跨节点路由还要看 KV locality、网络距离、pending transfer 和 KV capacity。

坑五：KV transfer 没有 backpressure。

prefill 侧持续产出 KV，decode 侧消费不过来，会造成网络拥塞和大量请求超时。

坑六：控制面和数据面互相影响。

大对象传输拥塞导致 heartbeat 或 metadata 请求超时，系统可能误判 worker 故障。

坑七：streaming 输出和 KV transfer 争用路径。

用户看到 token 卡顿，但 GPU 可能并不慢，真正问题是网络队列被大对象堵住。

坑八：跨节点故障清理不完整。

partial KV、预留 blocks、metadata 和请求状态没有一致清理，会造成内存泄漏和错误命中。

41.27 面试官会怎么问

问题一：为什么 TP 不建议跨节点？

回答要点：TP 每层都可能所有 all-reduce/all-gather/reduce-scatter，decode 每个 token 都要经过所有层。如果跨节点，通信频率高、延迟累积，会直接影响 TPOT。通常建议 TP 放在单节点 NVLink/NVSwitch 内，跨节点用 PP 或 DP。

问题二：跨节点 PD 分离的最大瓶颈是什么？

回答要点：Prefill 生成的 KV 必须传给 decode worker，KV transfer 的 bytes 与 prompt length、层数、KV heads、dtype 等相关。网络带宽、延迟、拥塞和 transfer backend 会决定 PD 分离收益是否大于传输成本。

问题三：Remote KV Cache 命中一定能降低延迟吗？

回答要点：不一定。要比较 remote lookup、read、network transfer、deserialize、promote 到 GPU 的总成本与 recompute 成本。如果 remote hit 在关键路径上更慢，或者影响 TPOT，就不应该用。

问题四：跨节点 router 应该考虑哪些信息？

回答要点：除了 worker load，还要考虑 KV capacity、prefix locality、pending transfers、P/D worker 网络距离、当前网络拥塞、decode TPOT、租户配额和 session affinity。

问题五：如何定位跨节点 serving 的网络瓶颈？

回答要点：先看 TTFT 还是 TPOT 变差；再按 prefill、decode、KV transfer、remote cache、collective、PP activation、router 分阶段打点；比较 same-node/same-rack/cross-rack；看 p50/p95/p99 和 pending transfer bytes。

41.28 标准回答模板

如果面试官问“如何设计跨节点 LLM serving”，可以这样回答：

我会先区分跨节点通信类型。Serving 里跨节点可能是 DP replica 之间的请求分流，也可能是一个 replica 内部的 TP/PP 模型并行，

部署上，能用 DP 横向扩吞吐时优先用 DP，因为每个 replica 独立，decode 过程不需要跨节点同步。如果模型单节点放不下，我会优

如果长 prompt prefill 干扰 decode，再考虑 PD 分离。但跨节点 PD 的关键是 KV transfer，必须比较 P/D 隔离收益和 KV 传输成本。aware 和 KV-aware routing，考虑 decode worker 的 KV capacity、pending transfer、prefix locality 和 P/D 网络距离，而不是只

对 remote KV cache，我不会只看命中率，而会比较 remote fetch + promote 的成本和 recompute 成本，并避免 remote fetch 进入 TP

观测上，我会按阶段拆指标：TTFT、TPOT、KV transfer latency、collective time、PP activation transfer、remote cache latency

41.29 跨节点通信成本、路由门禁和可运行 demo

先把一条网络链路抽象成：

$$E_L = (B_L, \alpha_L, \rho_L)$$

其中 B_L 是有效带宽， α_L 是基础延迟， ρ_L 是 p99 拥塞放大系数。

传输 M MiB 的 p99 时间可以粗略写成：

$$T_{\{\mathrm{net}\}}(M, L) = \rho_L \left(\alpha_L + \frac{M}{B_L} \right)$$

跨节点 TP 的 decode token 通信代价近似为：

$$T_{\{\mathrm{tp}, \mathrm{tok}\}} = L \cdot n_{\{\mathrm{coll}\}} T_{\{\mathrm{net}\}}(M_{\{\mathrm{coll}\}}, L)$$

其中 L 是层数， $n_{\{\mathrm{coll}\}}$ 是每层 collective 次数， $M_{\{\mathrm{coll}\}}$ 是一次 collective 的通信量。它危险的地方在于这个成本进入每个 output token。

PP activation 跨节点代价可以写成：

$$T_{\{\mathrm{pp}\}} = n_{\{\mathrm{boundary}\}} T_{\{\mathrm{net}\}} (M_{\{\mathrm{act}\}}, l)$$

PD KV transfer 成本:

$$M_{\{\mathrm{kv}\}, i} = P_i m_{\{\mathrm{kv}\}}$$

$$T_{\{\mathrm{pd}\}, i} = T_{\{\mathrm{net}\}} (M_{\{\mathrm{kv}\}, i}, l)$$

Remote KV 是否值得取, 要和 recompute 比:

$$A_{\{\mathrm{remote}\}} = \mathbf{1} [T_{\{\mathrm{remote}\}} < T_{\{\mathrm{recompute}\}}]$$

一个拓扑感知 decode worker score 可以写成:

$$S(w) = T_{\{\mathrm{load}\}, w} + T_{\{\mathrm{kv}\}}(w) + \lambda Q_{\{\mathrm{transfer}\}, w}$$

最终门禁可以写成:

$$G_{\{\mathrm{xnode}\}} = G_{\{\mathrm{tp}\}} G_{\{\mathrm{pp}\}} G_{\{\mathrm{pd}\}} G_{\{\mathrm{remote}\}} G_{\{\mathrm{router}\}} G_{\{\mathrm{back}\}}$$

下面的 demo 覆盖这些路径:

1. same-node TP vs cross-rack TP: 证明高频 collective 进入慢网络会打爆 TPOT。
2. PP activation: 证明跨节点 PP 和跨节点 TP 是不同通信模式。
3. PD KV transfer: 把长 prompt KV transfer MiB 和 p99 transfer time 显式算出来。
4. remote hot / cold KV: 一个选择 remote fetch, 一个选择 recompute。
5. topology-aware router: 不只看 decode worker load, 还看 P/D 网络距离和 pending transfer。
6. backpressure: pending transfer 超阈值时产生背压事件。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class LinkProfile:
```

```
    name: str
    bandwidth_mib_per_ms: float
    latency_ms: float
    p99_multiplier: float
```

```
@dataclass
```

```
class DecodeWorker:
```

```
    name: str
    link_name: str
    load_ms: float
    pending_transfer_mib: float
```

```
class ToyCrossNodeNetworkAuditor:
```

```
    def __init__(self):
```

```
        self.links = {
            "same_node": LinkProfile("same_node", 307.2, 0.02, 1.1),
            "same_rack_rdma": LinkProfile("same_rack_rdma", 51.2, 0.08, 1.3),
            "cross_rack": LinkProfile("cross_rack", 12.8, 0.25, 2.0),
            "tcp_fallback": LinkProfile("tcp_fallback", 3.2, 0.60, 3.0),
        }
        self.layers = 40
        self.collectives_per_layer = 2
        self.collective_mib = 2.0
        self.kv_mib_per_token = 0.015625
        self.prefill_tokens_per_ms = 512.0
```



```

self.pending_kv_limit_mib = 256.0

def transfer_ms(self, mib, link_name, p99=True):
    link = self.links[link_name]
    base = link.latency_ms + mib / link.bandwidth_mib_per_ms
    if p99:
        base *= link.p99_multiplier
    return round(base, 3)

def tensor_parallel_decode_ms(self, link_name):
    one_collective = self.transfer_ms(self.collective_mib, link_name, p99=True)
    return round(self.layers * self.collectives_per_layer * one_collective, 3)

def pipeline_activation_ms(self, activation_mib, boundaries, link_name):
    return round(boundaries * self.transfer_ms(activation_mib, link_name, p99=True), 3)

def pd_kv_transfer_ms(self, prompt_tokens, link_name):
    kv_mib = prompt_tokens * self.kv_mib_per_token
    return kv_mib, self.transfer_ms(kv_mib, link_name, p99=True)

def remote_or_recompute(self, remote_mib, recompute_tokens, link_name):
    fetch_ms = self.transfer_ms(remote_mib, link_name, p99=True)
    recompute_ms = round(recompute_tokens / self.prefill_tokens_per_ms + 0.5, 3)
    decision = "fetch_remote" if fetch_ms < recompute_ms else "recompute"
    return {"fetch_ms": fetch_ms, "recompute_ms": recompute_ms, "decision": decision}

def route_decode_worker(self, prompt_tokens, workers):
    kv_mib = prompt_tokens * self.kv_mib_per_token
    scored = []
    for worker in workers:
        transfer = self.transfer_ms(kv_mib, worker.link_name, p99=True)
        queue_penalty = worker.pending_transfer_mib / 128.0
        score = round(worker.load_ms + transfer + queue_penalty, 3)
        scored.append({
            "worker": worker.name,
            "link": worker.link_name,
            "score": score,
            "transfer_ms": transfer,
            "pending_transfer_mib": worker.pending_transfer_mib,
        })
    return min(scored, key=lambda row: row["score"]), scored

def audit(self):
    tp_same_node = self.tensor_parallel_decode_ms("same_node")
    tp_cross_rack = self.tensor_parallel_decode_ms("cross_rack")
    pp_cross_node = self.pipeline_activation_ms(
        activation_mib=16.0, boundaries=1, link_name="same_rack_rdma"
    )
    kv_mib, pd_transfer = self.pd_kv_transfer_ms(
        prompt_tokens=8192, link_name="same_rack_rdma"
    )
    remote_hot = self.remote_or_recompute(64.0, 8192, "cross_rack")
    remote_cold = self.remote_or_recompute(64.0, 512, "cross_rack")
    workers = [
        DecodeWorker("decode_same_rack", "same_rack_rdma", 6.0, 64.0),

```

```

        DecodeWorker("decode_cross_rack", "cross_rack", 3.0, 320.0),
    ]
    best, scored = self.route_decode_worker(prompt_tokens=8192, workers=workers)
    total_pending = sum(worker.pending_transfer_mib for worker in workers)
    summary = {
        "tp_same_node_tpot_overhead_ms": tp_same_node,
        "tp_cross_rack_tpot_overhead_ms": tp_cross_rack,
        "pp_cross_node_activation_ms": pp_cross_node,
        "pd_kv_transfer_mib": round(kv_mib, 1),
        "pd_kv_transfer_ms": pd_transfer,
        "remote_hot_decision": remote_hot["decision"],
        "remote_hot_fetch_ms": remote_hot["fetch_ms"],
        "remote_hot_recompute_ms": remote_hot["recompute_ms"],
        "remote_cold_decision": remote_cold["decision"],
        "router_choice": best["worker"],
        "pending_transfer_mib": total_pending,
        "backpressure_events": 1 if total_pending > self.pending_kv_limit_mib else 0,
        "data_plane_mib": round(kv_mib + 64.0 + 64.0, 1),
        "control_plane_mib": 0.2,
    }
    gates = {
        "tp_cross_node_risk_visible": (
            summary["tp_cross_rack_tpot_overhead_ms"]
            > summary["tp_same_node_tpot_overhead_ms"] * 10
        ),
        "pipeline_cost_separate": (
            summary["pp_cross_node_activation_ms"]
            < summary["tp_cross_rack_tpot_overhead_ms"]
        ),
        "pd_kv_transfer_counted": summary["pd_kv_transfer_mib"] == 128.0,
        "remote_vs_recompute_checked": (
            summary["remote_hot_decision"] == "fetch_remote"
            and summary["remote_cold_decision"] == "recompute"
        ),
        "topology_aware_router": summary["router_choice"] == "decode_same_rack",
        "backpressure_visible": summary["backpressure_events"] == 1,
        "control_data_plane_separated": (
            summary["data_plane_mib"] > summary["control_plane_mib"] * 100
        ),
        "metrics_ready": len(scored) == 2 and summary["pd_kv_transfer_ms"] > 0,
    }
    gates["cross_node_network_gate"] = all(gates.values())
    return scored, summary, gates

```

```

scored, summary, gates = ToyCrossNodeNetworkAuditor().audit()
print("cross_node_worker_scores=", scored)
print("cross_node_network_summary=", summary)
print("cross_node_network_gates=", gates)

```

一次运行的核心输出类似：

```

cross_node_network_summary= {'tp_same_node_tpot_overhead_ms': 2.32, 'tp_cross_rack_tpot_overhead_ms': 64.96, 'pp_cross_
cross_node_network_gates= {'tp_cross_node_risk_visible': True, 'pipeline_cost_separate': True, 'pd_kv_transfer_counted

```

这个 demo 展示了几个关键点：

1. `tp_cross_node_tpot_overhead_ms=64.96` 远高于 `same-node TP`: 跨节点 TP 是高频 per-token 风险。
2. `pp_cross_node_activation_ms=0.51` 单独计量: PP activation 不是 TP all-reduce。
3. `pd_kv_transfer_mib=128.0`: PD 分离跨节点时, 长 prompt KV transfer 必须进入 TTFT 成本。
4. `remote_hot_decision=fetch_remote` 且 `remote_cold_decision=recompute`: remote hit 不等于一定 fetch。
5. `router_choice=decode_same_rack`: 拓扑感知路由可以选择负载稍高但网络更近、pending transfer 更少的 worker。
6. `backpressure_events=1`: pending transfer 超阈值时要限流, 不能让 prefill 侧无限制造 KV。
7. `control_data_plane_separated=True`: 大 KV transfer 和 control plane metadata 不能混在同一个无保护路径里。

所以本章最终门禁是 `cross_node_network_gate`: 只有 TP / PP / PD / remote cache 的通信代价、topology-aware routing、backpressure、control/data plane 隔离和 metrics 都可观测, 跨节点 serving 才不是“多堆机器”, 而是可治理的网络系统。

41.30 小练习

1. 画出 DP、TP、PP、PD 分离四种跨节点 serving 的通信路径。
2. 解释为什么 TP 跨节点会影响 TPOT。
3. 对比 KV transfer 和 TP all-reduce 的网络瓶颈差异。
4. 给定 2 台 8 卡机器, 说明为什么常见选择是 TP=8、PP=2。
5. 设计一个 topology-aware router 的打分函数。
6. 设计一个 admission control 策略限制大 KV transfer。
7. 分析 remote KV hit 什么时候不如 recompute。
8. 说明 streaming 输出为什么可能被大对象 transfer 间接影响。
9. 列出跨节点 serving 的 15 个关键观测指标。
10. 设计一个排查 TPOT p99 突然升高的流程。

41.31 本章总结

跨节点 serving 的核心问题是通信模式。DP 主要跨节点分流请求, 通信最少; TP 是高频 collective, 通常应限制在单节点高速互联内; PP 跨节点传 activation, 适合模型单节点放不下的场景; PD 分离跨节点的关键瓶颈是 KV transfer; remote KV cache 则比较 fetch 成本和 recompute 成本。

网络瓶颈不只是带宽问题, 也包括延迟、拥塞、拓扑距离、tail latency、metadata 抖动和失败恢复。

设计跨节点 serving 时, 要优先保护 TPOT 稳定性, 避免高频通信进入慢网络; 用 topology-aware 和 KV-aware routing 提高 locality; 用 admission control 和 backpressure 限制大 KV transfer; 用分阶段指标定位瓶颈。

下一章会回到架构取舍, 系统讨论 PD 分离的优缺点、适用场景和反模式。不是所有系统都应该做 PD 分离, 也不是所有长 prompt 问题都应该通过跨节点架构解决。

第 42 章 PD 分离的优缺点、适用场景和反模式

前面几章已经从多个角度拆过 PD 分离:

1. 第 35 章讲 Prefill 和 Decode 的资源画像为什么不同。
2. 第 36 章讲 PD 分离解决什么问题。
3. 第 37 章讲 PD 分离的系统架构。
4. 第 38 章讲 KV Cache 迁移、共享和路由。
5. 第 39 章讲 Chunked Prefill 和 Disaggregated Prefill。
6. 第 40 章讲多级 KV Cache。
7. 第 41 章讲跨节点 serving 和网络瓶颈。

到这里, 应该回到一个工程决策问题:

到底什么时候应该做 PD 分离?

什么时候不该做?

哪些做法看起来先进, 实际是反模式?

一句话概括：

PD 分离不是推理系统的默认终态，而是当 P/D 干扰、资源扩缩容耦合和尾延迟问题足够严重，并且 KV transfer、路由、监控、故障恢复能力足够成熟时，才值得引入的高级架构。

42.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 WRITING_PLAN.md 对公开资料做校准：参考 DistServe 论文对 disaggregated prefill / decoding、goodput、TTFT / TPOT SLO、prefill 与 decode 资源干扰和 placement 的说明；参考 Splitwise 论文对 prompt computation 与 token generation 资源画像差异、phase split 和状态传输代价的说明；参考 SGLang PD Disaggregation 文档对 PD worker、transfer backend、routing policy、prefill interruption、DP attention imbalance 和 PD multiplexing 的公开口径；参考 vLLM disaggregated prefilling 文档对 prefill / decode 实例、KV connector 和 experimental 边界的说明；并结合前几章对 KV transfer、chunked prefill、多级 KV cache 和跨节点网络瓶颈的门禁。

本讲只讲“是否应该引入 PD 分离”的工程决策：收益、成本、适用 workload、不适用 workload、替代方案、运维前置条件和反模式识别。不把任何论文 benchmark、框架默认配置、connector 实现、网络条件、硬件型号或某个生产团队的落地路径写成通用结论。

本讲新增 demo 是教学版 PD tradeoff auditor：用 0 依赖 Python 模拟企业 RAG、短问答、代码生成、跨节点坏网络和缺少运维能力五类 case，计算 P/D 隔离收益、KV transfer 成本、复杂度惩罚和 ops readiness，输出 adopt / reject 决策、反模式原因、替代方案和最终门禁。

42.1 本章目标

读完本章，你应该能讲清：

1. PD 分离的核心收益是什么。
2. PD 分离的真实成本是什么。
3. 哪些 workload 适合 PD 分离。
4. 哪些 workload 不适合 PD 分离。
5. 如何判断 PD 分离收益是否大于成本。
6. 常见反模式有哪些。
7. 如何分阶段落地 PD 分离，而不是一步到位过度设计。
8. 面试中如何回答“要不要做 PD 分离”。

42.2 先给结论

PD 分离适合解决的问题是：

Prefill 和 Decode 在同一个资源池中互相干扰，导致 TTFT、TPOT、尾延迟和扩缩容策略都难以同时优化。

它不适合解决的问题是：

模型本身太慢、GPU 太少、网络太差、请求太短、监控不足、工程团队还没有能力维护分布式状态机。

判断标准可以先记一句：

$P/D \text{ 隔离收益} > KV \text{ transfer 成本} + \text{系统复杂度成本} + \text{故障恢复成本}$

如果这个不等式不成立，PD 分离就是负优化。

42.3 PD 分离的主要优点

PD 分离的优点可以分成六类。

第一，降低 Prefill 对 Decode 的干扰。

长 prompt prefill 不再直接插入 decode worker 的关键循环，decode TPOT 更稳定。

第二，降低 Decode 对 Prefill 的饥饿。

如果 unified scheduler 长期 decode-first, 新请求可能一直等不到 prefill。PD 分离后 prefill pool 有自己的调度机会。

第三, 资源池独立扩缩容。

输入压力大时扩 prefill pool, 输出压力大时扩 decode pool, 而不是所有 worker 都按同一种比例扩容。

第四, 调度策略可以分化。

Prefill 可以追求 input tokens/s、chunked prefill、prefix cache 命中; Decode 可以追求稳定 TPOT、continuous batching、streaming 平滑度。

第五, 隔离不同 batch shape。

Prefill 的 batch 往往 token 数大、shape 变化大; Decode 的 batch 每步 token 少但频率高。拆开后两个阶段不用互相迁就。

第六, 为高级系统能力打基础。

例如:

1. 独立 prefill pool。
2. decode-optimized pool。
3. KV-aware routing。
4. remote KV cache。
5. long-context serving。
6. 多租户资源隔离。
7. topology-aware scheduling。

42.4 PD 分离的主要缺点

PD 分离不是免费午餐。

它的缺点也很明确。

第一, KV transfer 成本。

Prefill worker 生成的 KV 必须被 Decode worker 使用。KV 体积大, 传输慢了会直接拉高 TTFT。

第二, 分布式状态机复杂。

一个请求不再只是一个 engine 内部状态, 而是跨 router、prefill worker、transfer backend、decode worker。

第三, 故障恢复更难。

Prefill 成功但 transfer 失败怎么办? Decode worker 接收一半失败怎么办? Streaming 已经输出部分 token 后 worker 崩溃怎么办? 这些都比单机复杂。

第四, 观测要求更高。

只看总 QPS 或总 latency 不够, 必须分阶段看 prefill、transfer、decode、routing、KV capacity、网络。

第五, 调参空间变大。

Prefill pool 大小、decode pool 大小、chunk size、transfer 并发、router 策略、KV 预留策略、backpressure 阈值都会影响结果。

第六, 资源可能空转。

如果流量形态变化大, prefill pool 可能闲着, decode pool 忙; 或者反过来。资源池拆开后, 利用率治理更复杂。

第七, 系统尾延迟来源更多。

以前尾延迟主要来自排队、prefill、decode。现在还会来自 KV transfer、metadata lookup、remote cache、网络拥塞、worker 协调。

42.5 收益不是看平均值

PD 分离的收益通常体现在尾延迟和稳定性，而不是平均吞吐一定大幅提升。

例如 unified engine:

TTFT p50 = 300 ms

TTFT p99 = 8 s

TPOT p50 = 35 ms

TPOT p99 = 800 ms

PD 分离后:

TTFT p50 = 350 ms

TTFT p99 = 2 s

TPOT p50 = 38 ms

TPOT p99 = 120 ms

平均值可能没有明显变好，甚至 p50 稍微变差，因为多了 transfer。但 p99 明显改善，streaming 更稳定。

所以评估 PD 分离时要看:

1. TTFT p95/p99。
2. TPOT p95/p99。
3. inter-token latency tail。
4. 长 prompt 到达时 decode 是否卡顿。
5. prefill queue 是否被 decode 压制。
6. 多租户是否互相干扰。
7. 输入输出比例变化时是否能独立扩缩容。

只看平均 QPS，可能误判 PD 分离价值。

42.6 成本模型

可以用一个简单模型判断是否值得做。

PD 分离收益:

benefit

= reduced_decode_tail_latency
+ reduced_prefill_queue_delay
+ independent_scaling_gain
+ better_isolation_gain

PD 分离成本:

cost

= kv_transfer_latency
+ transfer_queue_delay
+ extra_routing_latency
+ resource_fragmentation_cost
+ failure_recovery_cost
+ engineering_complexity_cost

值得做的条件是:

benefit > cost

这不是数学上精确的公式，而是工程上防止拍脑袋的检查框架。

更具体一点:

PD is useful if:

prefill_interruption is frequent

and decode_tail_latency is SLO problem
and kv_transfer_p95 is acceptable
and traffic has enough scale
and ops can debug distributed failures

42.7 适用场景一：长 Prompt 和长上下文

PD 分离最典型的适用场景是长 prompt。

例如：

1. RAG 长文档问答。
2. 代码库问答。
3. 合同、论文、病历分析。
4. 多轮对话上下文很长。
5. Agent 带大量工具描述和历史轨迹。

这些请求的特点是：

prefill 很重，decode 需要稳定。

在 unified engine 中，一个 64000-token prompt 可能明显打断正在 streaming 的 decode 请求。

PD 分离后，长 prompt 进入 prefill pool，decode pool 可以继续稳定输出。

但要注意：长 prompt 也意味着 KV transfer 更大。只有当 transfer backend 足够强，或者可以通过 chunked prefill、KV locality、同机 PD、多级 KV cache 降低成本时，PD 分离才更容易成立。

42.8 适用场景二：输入输出比例波动大

有些业务流量不是稳定的。

例如白天大量短问答，晚上大量长文档总结；或者某个租户突然提交大批长 prompt。

统一 engine 的问题是：

所有 worker 都同时承担 prefill 和 decode。

当输入压力上升时，decode 体验也被影响。

PD 分离后可以：

increase prefill pool
keep decode pool stable

或者输出压力上升时：

increase decode pool
keep prefill pool smaller

这种独立扩缩容在大规模多租户平台里很有价值。

42.9 适用场景三：Streaming SLO 很严格

如果业务强依赖 streaming 体验，TPOT 和 inter-token latency 的 p95/p99 很重要。

例如：

1. 对话助手。
2. 语音实时助手。
3. IDE coding assistant。
4. 在线客服。
5. 交互式 agent。

用户对首 token 慢可能还能接受一点，但输出过程中一卡一卡，体验非常差。

PD 分离的价值是让 decode pool 尽量不被长 prefill 打断。

如果观察到：

```
long prefill arrives
-> decode step latency spike
-> streaming stalls
```

PD 分离就值得评估。

42.10 适用场景四：多租户隔离

多租户平台里，不同租户的 workload 可能完全不同。

一个租户提交大量长文档请求，可能拖慢其他租户的 streaming。

PD 分离可以配合资源隔离：

1. 给高优先级租户保留 decode capacity。
2. 限制低优先级租户 prefill 并发。
3. 为长 prompt 租户使用独立 prefill queue。
4. 对 decode pool 做更严格 SLO 保护。

这里 PD 分离的价值不只是性能，还有资源治理。

42.11 适用场景五：Prefill 和 Decode 适合不同硬件

有时 prefill 和 decode 对硬件偏好不同。

Prefill 更偏大 batch、大矩阵吞吐。

Decode 更偏低延迟、KV 读取、调度频率和显存带宽。

如果集群有异构资源，例如不同 GPU 型号、不同互联拓扑，可以考虑：

prefill pool: 更强 compute 或更适合大 batch 的资源

decode pool: 更稳定低延迟和充足 KV cache 的资源

但异构资源会增加调度和调参复杂度，不能只因为“听起来合理”就上。

42.12 不适合场景一：请求都很短

如果请求大多是：

```
prompt = 50 tokens
output = 50 tokens
```

prefill 本身不重，P/D 干扰不明显。

这时引入 PD 分离可能只会增加：

1. router 开销。
2. KV transfer 开销。
3. 状态同步开销。
4. 故障恢复复杂度。

结果是 p50/p95 都变差。

短请求场景更应该先优化：

1. batching。
2. scheduler。
3. tokenizer。

4. HTTP/gRPC overhead。
5. prefix cache。
6. replica 数量。

42.13 不适合场景二：流量规模不够

PD 分离需要足够规模才能摊薄复杂度。

如果一天只有少量请求，或者单机 vLLM/SGLang 已经满足 SLO，强行做 PD 分离没有意义。

因为你会付出：

1. 更多服务组件。
2. 更多监控指标。
3. 更多故障模式。
4. 更复杂部署。
5. 更高维护成本。

小规模系统更适合：

single engine + continuous batching + prefix cache + chunked prefill

先把这些做好，再考虑 PD 分离。

42.14 不适合场景三：KV Transfer 条件差

如果网络或同机互联无法支撑 KV transfer，PD 分离会很难。

典型情况：

1. 跨节点只有普通以太网。
2. RDMA 不稳定。
3. GPU-GPU direct copy 能力不足。
4. KV transfer p99 很高。
5. transfer 经常和其他流量抢带宽。
6. decode worker 经常等 KV。

这时 PD 分离可能把问题从：

prefill 打断 decode

变成：

decode 等待 KV transfer

用户看到的仍然是慢，只是瓶颈换了位置。

42.15 不适合场景四：主要瓶颈不在 P/D 干扰

如果系统慢是因为：

1. 模型太大，单 token compute 慢。
2. GPU 利用率很低，batching 没做好。
3. tokenizer 或预处理很慢。
4. RAG 检索很慢。
5. 后处理或工具调用慢。
6. 客户端网络慢。
7. speculative decoding 接受率低。
8. KV cache block manager 有 bug。

PD 分离不一定解决根因。

工程上最忌讳的是没有定位瓶颈，就用复杂架构覆盖症状。

42.16 不适合场景五：缺少可观测性和运维能力

PD 分离需要更强的可观测性。

至少要能看到：

1. prefill queue latency。
2. prefill execution latency。
3. decode assignment latency。
4. KV transfer latency。
5. transfer queue length。
6. decode step latency。
7. TTFT breakdown。
8. TPOT breakdown。
9. worker failure rate。
10. cleanup failure count。

如果这些都没有，PD 分离上线后出问题会很难排查。

没有观测的分布式系统，不是高级架构，而是黑盒事故源。

42.17 反模式一：为了架构先进而 PD 分离

最常见反模式是：

因为大厂论文或开源系统在做 PD 分离，所以我们也做。

这是错误的。

PD 分离应该由指标驱动：

1. 是否有 prefill interruption。
2. 是否有 decode p99 抖动。
3. 是否有 prefill 饥饿。
4. 是否有 P/D 扩缩容耦合。
5. KV transfer 是否可控。
6. 团队是否能维护复杂系统。

没有这些证据，不该把 PD 分离作为默认架构。

42.18 反模式二：只拆服务，不拆资源

有些系统把 API 上拆成 prefill service 和 decode service，但底层仍然抢同一批 GPU 或同一套调度资源。

例如：

```
prefill service and decode service
-> same worker pool
-> same GPU queue
```

这只是服务名拆开了，资源干扰还在。

真正的 PD 分离至少要做到：

1. prefill 和 decode 有可区分的资源池或调度策略。
2. decode 的关键循环能被保护。
3. P/D 负载能分别观测。
4. 扩缩容或配额能分别控制。

否则收益很有限。

42.19 反模式三：只看 Prefill 加速，不看 Decode 等待

PD 分离后，prefill pool 可能很快完成 prompt 计算。

但如果 decode worker 没有 slot，或者 KV transfer 排队，首 token 仍然慢。

错误指标是：

prefill latency decreased

正确指标是：

TTFT breakdown improved

decode TPOT tail improved

一个请求的路径是：

queue -> prefill -> transfer -> first decode -> streaming

只优化中间一段，不代表端到端体验变好。

42.20 反模式四：无脑跨节点 PD

跨节点 PD 看起来扩展性好，但 KV transfer 可能成为瓶颈。

如果同机资源足够，或者可以先做同机 PD，不一定要一开始跨节点。

演进路径可以是：

unified engine

- > chunked prefill

- > same-node P/D separation

- > topology-aware cross-node PD

不要直接跳到：

global prefill pool + global decode pool + remote KV cache everywhere

这会把调试难度拉满。

42.21 反模式五：Remote KV Cache 当万能药

有些设计认为：只要把 KV 放进远端缓存，decode worker 随时拉，就解决了迁移问题。

问题是 remote KV fetch 仍然有成本。

如果每次首 token 都要：

metadata lookup

- > remote read

- > network transfer

- > CPU staging

- > GPU promote

TTFT 可能比直接 prefill 还差。

Remote KV Cache 应该是 cost-based 的：

use remote KV only if fetch_cost < recompute_cost and latency SLO allows it

42.22 反模式六：没有 Backpressure

PD 分离后，如果 prefill pool 只管生产 KV，decode pool 消费不过来，就会堆积。

表现为：

1. pending transfer bytes 上升。
2. decode worker KV blocks 不够。
3. transfer queue 变长。
4. prefill 计算浪费。
5. 大量请求超时。

必须有 backpressure:

decode pressure -> transfer pressure -> prefill admission -> router admission

否则 PD 分离会在高峰期自我放大拥塞。

42.23 反模式七：忽略失败清理

PD 分离会产生很多中间状态:

1. prefill 已完成但 decode 未接管。
2. KV transfer 进行中。
3. decode worker 已预留 blocks。
4. router 状态已更新但 worker 失败。
5. remote KV 写入一半。

如果失败清理不完整, 会造成:

1. GPU blocks 泄漏。
2. CPU staging buffer 泄漏。
3. metadata 指向不存在的 KV。
4. 请求永远卡在中间状态。
5. retry 产生重复计算。

生产系统里, 清理逻辑和 happy path 一样重要。

42.24 反模式八：忽略租户和优先级

PD 分离后, 资源池更多, 队列更多。

如果没有租户隔离, 一个租户的大量长 prompt 可能占满 prefill pool、transfer backend 或 remote KV cache。

需要考虑:

1. tenant quota。
2. priority queue。
3. max long-prefill concurrency。
4. per-tenant KV cache budget。
5. decode slot reservation。
6. admission control。

否则 PD 分离只是把原来的 GPU 争抢, 变成多个组件上的资源争抢。

42.25 替代方案：先做哪些优化

在上 PD 分离之前, 通常应该先评估更简单的方案。

方案一: Chunked Prefill。

把长 prompt prefill 切小, 在 unified engine 中和 decode 更好地穿插。

方案二: Prefix Cache / RadixAttention。

减少重复 prefill, 降低 TTFT 和 prefill 负载。

方案三: Scheduler 调参。

调整 max_num_batched_tokens、decode-first 策略、waiting/running 比例、preemption 策略。

方案四：增加 DP replicas。

如果模型单副本能放下，横向扩 replica 可能比 PD 分离简单。

方案五：Speculative Decoding。

如果瓶颈在长输出 decode，speculative decoding 可能比 PD 分离更直接。

方案六：多级 KV Cache。

如果问题是 idle session 或 prefix 复用，可以先做 GPU/CPU KV 管理和 local prefix cache。

PD 分离应该是这些方案仍不足以解决 P/D 干扰和扩缩容问题后的选择。

42.26 分阶段落地路线

不要一步到位做最复杂形态。

一个更稳妥的路线是：

阶段 0: unified engine baseline

阶段 1: 加强指标，拆 TTFT/TPOT 阶段耗时

阶段 2: chunked prefill + prefix cache

阶段 3: 同机内 prefill/decode 资源隔离实验

阶段 4: 明确 KV transfer backend 和 block metadata

阶段 5: 小规模 PD pool，上线灰度流量

阶段 6: topology-aware routing 和 backpressure

阶段 7: 跨节点 PD 和 remote KV cache

每个阶段都要验证：

1. 哪个指标变好。
2. 哪个指标变差。
3. 是否引入新故障。
4. 是否值得进入下一阶段。

如果阶段 2 已经满足 SLO，就不一定要继续做 PD 分离。

42.27 判断清单

上线 PD 分离前，可以问这些问题。

Workload:

1. 长 prompt 占比是多少？
2. 长输出占比是多少？
3. P/D 比例是否波动？
4. 是否存在明显 prefill interruption？
5. 是否存在 decode-first 导致的 prefill 饥饿？

性能：

1. 当前 TTFT p95/p99 根因是什么？
2. 当前 TPOT p95/p99 根因是什么？
3. KV transfer p95/p99 预计是多少？
4. transfer 是否比 recompute 更便宜？
5. decode worker 是否会频繁等 KV？

资源：

1. Prefill 和 Decode 是否需要不同扩缩容比例？
2. GPU 资源是否足够拆池？

3. 网络是否能承受 KV transfer?
4. KV cache capacity 是否足够?
5. 是否需要多租户隔离?

工程:

1. 是否有分阶段指标?
2. 是否有失败清理?
3. 是否有 backpressure?
4. 是否有 topology-aware routing?
5. 团队是否能排查分布式故障?

如果这些问题大多答不上来, 先不要做完整 PD 分离。

42.28 一个具体决策例子

场景 A: 企业 RAG 平台。

```
prompt p50 = 2K tokens
prompt p95 = 64K tokens
output p50 = 512 tokens
streaming SLO strict
long prompt arrives -> TPOT p99 spikes
```

这里 PD 分离值得评估, 因为长 prefill 明显影响 decode。

但还要继续看:

1. KV transfer 是否可控。
2. 是否可以先 chunked prefill。
3. prefix cache 命中率如何。
4. P/D pool 比例怎么定。
5. 网络 p99 是否稳定。

场景 B: 短问答客服。

```
prompt p95 = 512 tokens
output p95 = 128 tokens
single engine TTFT/TPOT already meets SLO
```

这里不应该优先做 PD 分离。

更适合做:

1. 多 replica DP。
2. scheduler 调参。
3. prefix cache。
4. HTTP/gRPC 优化。
5. tokenizer 并行。

场景 C: 代码生成。

```
prompt medium
output very long
decode is bottleneck
```

PD 分离可能不是第一选择。应先评估 speculative decoding、decode batch、KV cache capacity、TPOT 优化。

42.29 和其他优化的选择关系

可以按瓶颈选择优化。

主要瓶颈	优先考虑
长 prefill 打断 decode	Chunked prefill, PD 分离
重复 prompt 多	Prefix cache, RadixAttention
长输出 decode 慢	Speculative decoding, decode batching
模型单卡放不下	TP/PP
总 QPS 不够	DP replicas, 负载均衡
KV 显存不够	PagedAttention, KV block manager, 多级 KV
多租户互相干扰	Admission control, quota, PD 分离
跨节点通信慢	拓扑优化, 减少跨节点 TP, KV locality

PD 分离通常不是第一个按钮，而是在长 prefill、decode 稳定性、扩缩容和资源隔离问题同时出现时的系统级方案。

42.30 评估实验设计

评估 PD 分离时，要设计对照实验。

Baseline:

unified engine + current scheduler

候选方案:

unified + chunked prefill

unified + prefix cache

same-node PD

cross-node PD

PD + remote KV cache

流量要覆盖:

1. 短输入短输出。
2. 长输入短输出。
3. 短输入长输出。
4. 长输入长输出。
5. 多租户混合流量。
6. burst 流量。
7. cache hit / cache miss 混合。

指标要覆盖:

1. TTFT p50/p95/p99。
2. TPOT p50/p95/p99。
3. input tokens/s。
4. output tokens/s。
5. GPU utilization。
6. KV transfer latency。
7. pending transfer bytes。
8. prefill queue latency。
9. decode queue latency。
10. error / timeout rate。

只有当 PD 分离在目标 workload 和目标 SLO 上稳定胜出，才值得继续推进。

42.31 面试官会怎么问

问题一：PD 分离的优点是什么？

回答要点：隔离 Prefill 和 Decode 的资源干扰，降低长 prefill 对 decode TPOT 的影响，缓解 decode-first 导致的 prefill 饥饿，支持 P/D 独立扩缩容，并让两个阶段使用不同调度策略和资源配置。

问题二：PD 分离的缺点是什么？

回答要点：核心代价是 KV transfer 和分布式状态管理。还包括路由复杂、故障恢复复杂、观测和调参复杂、资源池可能空转、网络和 transfer p99 可能抵消收益。

问题三：什么时候适合做 PD 分离？

回答要点：长 prompt 或长上下文比例高，prefill interruption 明显，decode TPOT p99 受影响，P/D 输入输出比例波动大，需要独立扩缩容或多租户隔离，并且 KV transfer 成本可控、监控和故障恢复能力足够。

问题四：什么时候不适合做 PD 分离？

回答要点：请求短、流量小、单机 unified engine 已满足 SLO、主要瓶颈不在 P/D 干扰、KV transfer 条件差、缺少可观测性和运维能力时，不适合过早引入。

问题五：PD 分离有哪些反模式？

回答要点：为了架构先进而做；只拆服务不拆资源；只看 prefill 变快不看端到端；无脑跨节点；remote KV cache 当万能药；没有 backpressure；失败清理不完整；忽略租户隔离和优先级。

42.32 标准回答模板

如果面试官问“你会在什么情况下引入 PD 分离”，可以这样回答：

我不会把 PD 分离当成默认架构，而会先看 workload 和指标。如果系统里长 prompt 或长上下文请求较多，unified engine 中经常出 first 策略又让 prefill queue 饥饿，说明 P/D 混合调度已经成为问题。

这时 PD 分离的收益是把 prefill 和 decode 放到不同资源池，让 prefill pool 专注 input tokens/s 和 prompt KV 生成，让 decode

但我会同时评估成本。核心成本是 KV transfer：prefill 生成的 KV 必须迁移或共享给 decode worker，如果 transfer p95/p99 太高

所以我的判断标准是：P/D 隔离收益是否大于 KV transfer 成本和系统复杂度成本。如果请求都很短、流量不大、单机 continuous ba

42.33 PD 分离收益/代价门禁、反模式识别和可运行 demo

先把一个 workload case 抽象成：

$$W_i = (P_i, O_i, T_i^{\mathrm{ttft}}, T_i^{\mathrm{tpot}}, K_i, C_i, R_i)$$

其中 P_i 是 prompt p95， O_i 是 output p95， T_i^{ttft} 和 T_i^{tpot} 是 unified engine 的尾延迟， K_i 是 KV transfer p99， C_i 是系统复杂度惩罚， R_i 是运维 readiness。

PD 分离的教学版收益可以写成：

$$B_i = a \max(0, T_i^{\mathrm{ttft}} - S_{\mathrm{ttft}}) + b \max(0, T_i^{\mathrm{tpot}} - S_{\mathrm{tpot}}) + D_i$$

其中 S_{ttft} 和 S_{tpot} 是目标 SLO， D_i 是独立扩缩容或资源隔离收益。

代价可以写成：

$$C_i^{\mathrm{pd}} = K_i + C_i^{\mathrm{state}} + C_i^{\mathrm{ops}}$$

一个最小决策分数：

$$S_i^{\mathrm{pd}} = B_i - C_i^{\mathrm{pd}}$$

但 $S_i^{\mathrm{pd}} > 0$ 仍不够。还要过 readiness 门：

$$G_{\mathrm{ready}} = G_{\mathrm{metric}} G_{\mathrm{cleanup}} G_{\mathrm{backpressure}}$$

最终门禁：

$$G_{\mathrm{pdtrade}} = G_{\mathrm{positive}} G_{\mathrm{short}} G_{\mathrm{decode}} G_{\mathrm{transfer}} G_{\mathrm{ops}}$$

下面的 demo 覆盖五类 case:

1. enterprise_rag: 长 prompt、TPOT p99 抖动、transfer 可控、运维能力齐备, 推荐先做 same-node PD。
2. short_qa: 短请求, unified 已经满足, 拒绝 PD, 建议 DP replicas。
3. code_generation: 主要瓶颈是长输出 decode, 不优先 PD, 建议 speculative decoding。
4. cross_node_bad_network: 有长 prompt 干扰, 但跨节点 KV transfer 太慢, 拒绝跨节点 PD。
5. long_prompt_no_ops: 分数为正, 但缺 metrics / cleanup / backpressure, 先补观测和治理。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class PDCase:
```

```
    name: str
    prompt_p95: int
    output_p95: int
    unified_ttft_p99_ms: float
    unified_tpot_p99_ms: float
    target_ttft_ms: float
    target_tpot_ms: float
    kv_transfer_p99_ms: float
    scaling_saving_ms: float
    complexity_penalty_ms: float
    prefill_interference: bool
    decode_bottleneck: bool
    metrics_ready: bool
    cleanup_ready: bool
    backpressure_ready: bool
```

```
class ToyPDTradeoffAuditor:
```

```
    def benefit_score(self, case):
```

```
        ttft_tail_saving = max(0.0, case.unified_ttft_p99_ms - case.target_ttft_ms) * 0.05
```

```
        tpot_tail_saving = max(0.0, case.unified_tpot_p99_ms - case.target_tpot_ms) * 0.2
```

```
        if not case.prefill_interference:
```

```
            ttft_tail_saving *= 0.25
```

```
            tpot_tail_saving *= 0.25
```

```
        cost = case.kv_transfer_p99_ms + case.complexity_penalty_ms
```

```
        return round(ttft_tail_saving + tpot_tail_saving + case.scaling_saving_ms - cost, 3)
```

```
    def classify(self, case):
```

```
        score = self.benefit_score(case)
```

```
        reasons = []
```

```
        alternatives = []
```

```
        if case.prompt_p95 < 1024 and case.output_p95 < 512:
```

```
            reasons.append("short_requests")
```

```
            alternatives.append("dp_replicas")
```

```
        if case.decode_bottleneck and not case.prefill_interference:
```

```
            reasons.append("decode_bottleneck_not_pd")
```

```
            alternatives.append("speculative_decoding")
```

```
        if case.kv_transfer_p99_ms > 250:
```

```
            reasons.append("kv_transfer_too_slow")
```

```
            alternatives.append("same_node_pd_or_chunked_prefill")
```

```
        if not (case.metrics_ready and case.cleanup_ready and case.backpressure_ready):
```

```
            reasons.append("ops_readiness_missing")
```

```
            alternatives.append("instrument_first")
```

```

    if not case.prefill_interference:
        reasons.append("no_prefill_interference_evidence")
    decision = "adopt_pd" if score > 0 and not reasons else "do_not_adopt_pd_yet"
    if decision == "adopt_pd":
        reasons.append("benefit_exceeds_transfer_and_complexity")
        alternatives.append("same_node_pd_first")
    return {
        "case": case.name,
        "score": score,
        "decision": decision,
        "reasons": reasons,
        "alternatives": alternatives,
    }

def audit(self, cases):
    rows = [self.classify(case) for case in cases]
    summary = {
        "cases": len(rows),
        "adopt_pd": sum(row["decision"] == "adopt_pd" for row in rows),
        "rejected": sum(row["decision"] != "adopt_pd" for row in rows),
        "anti_patterns": sum(
            any(reason in row["reasons"] for reason in [
                "short_requests",
                "decode_bottleneck_not_pd",
                "kv_transfer_too_slow",
                "ops_readiness_missing",
                "no_prefill_interference_evidence",
            ])
            for row in rows
        ),
        "positive_cases": [row["case"] for row in rows if row["decision"] == "adopt_pd"],
        "blocked_cases": [row["case"] for row in rows if row["decision"] != "adopt_pd"],
    }
    gates = {
        "positive_case_visible": summary["positive_cases"] == ["enterprise_rag"],
        "short_request_rejected": any("short_requests" in row["reasons"] for row in rows),
        "decode_bottleneck_not_pd": any("decode_bottleneck_not_pd" in row["reasons"] for row in rows),
        "slow_transfer_rejected": any("kv_transfer_too_slow" in row["reasons"] for row in rows),
        "ops_readiness_required": any("ops_readiness_missing" in row["reasons"] for row in rows),
        "cost_model_used": all("score" in row for row in rows),
        "alternatives_visible": all(row["alternatives"] for row in rows),
        "anti_patterns_visible": summary["anti_patterns"] >= 4,
    }
    gates["pd_tradeoff_gate"] = all(gates.values())
    return rows, summary, gates

```

```

cases = [
    PDCase("enterprise_rag", 64000, 512, 8000, 800, 2000, 120, 80, 120, 150, True, False, True, True, True),
    PDCase("short_qa", 512, 128, 600, 70, 1000, 100, 3, 5, 120, False, False, True, True, True),
    PDCase("code_generation", 4096, 4096, 1600, 650, 1800, 200, 20, 20, 140, False, True, True, True, True),
    PDCase("cross_node_bad_network", 64000, 512, 8000, 800, 2000, 120, 700, 120, 160, True, False, True, True, True),
    PDCase("long_prompt_no_ops", 32000, 512, 6000, 500, 2000, 120, 60, 80, 150, True, False, False, False, False),
]
rows, summary, gates = ToyPDTradeoffAuditor().audit(cases)

```

```
print("pd_tradeoff_rows=", rows)
print("pd_tradeoff_summary=", summary)
print("pd_tradeoff_gates=", gates)
```

一次运行的核心输出类似：

```
pd_tradeoff_summary= {'cases': 5, 'adopt_pd': 1, 'rejected': 4, 'anti_patterns': 4, 'positive_cases': ['enterprise_rag']}
pd_tradeoff_gates= {'positive_case_visible': True, 'short_request_rejected': True, 'decode_bottleneck_not_pd': True, 's
```

这个 demo 展示了几个关键点：

1. enterprise_rag 是正例：长 prompt 干扰明显、transfer 可控、运维能力具备，才进入 adopt_pd。
2. short_qa 被拒绝：短请求不应为了架构先进而 PD 分离。
3. code_generation 被拒绝：瓶颈在长输出 decode，优先 speculative decoding 或 decode batching。
4. cross_node_bad_network 被拒绝：KV transfer p99 太高，跨节点 PD 会把收益吃掉。
5. long_prompt_no_ops 被拒绝：即使分数为正，metrics、cleanup 和 backpressure 不齐也不能上线。
6. alternatives_visible=True：每个 rejected case 都给出替代路径，避免把 PD 当成唯一按钮。

所以本章最终门禁是 pd_tradeoff_gate：只有正例、短请求反例、decode 瓶颈反例、慢 transfer 反例、运维 readiness、成本模型、替代方案和反模式识别都能闭环，PD 分离决策才是工程判断，而不是架构偏好。

42.34 小练习

1. 给出一个适合 PD 分离的 workload，并说明原因。
2. 给出一个不适合 PD 分离的 workload，并说明原因。
3. 设计一个实验比较 unified、chunked prefill、same-node PD 和 cross-node PD。
4. 画出 PD 分离的收益成本模型。
5. 分析为什么 remote KV cache hit 率高也可能让 TTFT 变差。
6. 说明“只拆服务不拆资源”为什么不是有效 PD 分离。
7. 设计一个 backpressure 信号链路。
8. 列出上线 PD 分离前必须具备的 10 个观测指标。
9. 分析代码生成场景中 PD 分离和 speculative decoding 谁更优先。
10. 写一个面试回答：什么时候不该做 PD 分离？

42.35 本章总结

PD 分离的核心收益是隔离 Prefill 和 Decode 的资源干扰，让两个阶段可以使用不同资源池、调度策略和扩缩容策略，从而改善 TTFT/TPOt 尾延迟和多租户资源治理。

它的核心成本是 KV transfer 和分布式状态管理，并且会引入路由、metadata、故障恢复、backpressure、观测和调参复杂度。

PD 分离适合长 prompt、长上下文、streaming SLO 严格、P/D 比例波动大、多租户隔离要求强，并且 KV transfer 条件可控的场景。

它不适合短请求、小流量、单机已满足 SLO、瓶颈不在 P/D 干扰、网络条件差或缺少可观测性与运维能力的场景。

下一章会进入更落地的问题：如何把一个单机 serving engine 逐步升级成 PD 分离架构。重点不是重新写一个系统，而是识别哪些模块要抽象、哪些状态要外置、哪些接口要先稳定下来。

第 43 章 如何把单机 Serving Engine 升级成 PD 分离架构

上一章讲了 PD 分离的优缺点、适用场景和反模式。

这一章进入更落地的问题：

如果我已经有一个单机 serving engine，应该如何一步步把它改造成 PD 分离架构？

这里的重点不是一上来重写一个分布式系统，而是识别：

1. 哪些模块必须先抽象出来。

2. 哪些状态必须从单进程内存里剥离出来。
3. 哪些接口必须先稳定。
4. 哪些能力可以分阶段上线。
5. 哪些改造如果顺序错了，会让系统变得很难维护。

一句话概括：

从单机 serving engine 升级到 PD 分离，本质是把 “一个进程内部的 generate 状态机” 逐步拆成 “router、prefill worker、decode worker、KV transfer 和分布式请求状态” 共同维护的系统。

43.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 `WRITING_PLAN.md` 对公开资料做校准：参考 DistServe 论文对 disaggregated prefill / decoding、TTFT / TPOT SLO、P/D placement 和 goodput 的说明；参考 Splitwise 论文对 prompt computation 与 token generation 资源画像差异、phase split 和状态传输代价的说明；参考 SGLang PD Disaggregation 文档对 prefill / decode worker、transfer backend、routing policy、bootstrap server、PD multiplexing、prefill interruption 和 DP attention imbalance 的公开口径；参考 vLLM disaggregated prefilling 文档对 prefill / decode 实例、KV connector 和 experimental 边界的说明；并结合前面章节对 KV transfer、chunked prefill、多级 KV cache、跨节点网络瓶颈和 PD 取舍门禁的结论。

本讲只讲 “如何从已有单机 serving engine 迁移到 PD 分离” 的工程路线：模块边界、接口契约、状态拆分、迁移阶段、same-node 原型、cross-node 前置门禁、观测、backpressure、灰度、回滚和失败清理。不把任何论文 benchmark、框架默认 CLI 参数、connector 字段、worker 启动命令、真实网络带宽、某个版本的默认策略或某个团队的生产路径写成通用结论。

本讲新增 demo 是教学版 PD migration auditor：用 0 依赖 Python 模拟单机重构、逻辑 PD、same-node PD 和 cross-node readiness 四个阶段，检查 request stage、model runner 接口、KV metadata、双 scheduler、router 状态机、same-node 原型、cleanup / cancel、metrics / backpressure / fallback 是否按顺序就绪，输出 migration rows、summary 和最终迁移门禁。

43.1 本章目标

读完本章，你应该能讲清：

1. 单机 serving engine 中哪些模块会阻碍 PD 分离。
2. 为什么不能直接把 `prefill()` 和 `decode()` 拆成两个服务。
3. 升级前需要先抽象哪些接口。
4. Request state、KV metadata、scheduler state 应该如何拆分。
5. Prefill worker 和 Decode worker 的职责如何重新定义。
6. KV transfer 如何接入现有 KV cache manager。
7. 如何分阶段从 unified engine 演进到 PD 分离。
8. 上线 PD 分离时如何灰度、回滚和观测。
9. 面试中如何回答 “如何把单机推理框架升级成 PD 分离”。

43.2 先看单机 Engine 长什么样

一个简化的单机 serving engine 通常是这样：

```
client
-> HTTP API
-> request queue
-> scheduler
-> model runner
    -> prefill
    -> decode loop
-> streamer
```

代码里可能长这样：

```

class Engine:
    def add_request(self, request):
        self.waiting_queue.append(request)

    def step(self):
        batch = self.scheduler.schedule(self.waiting_queue, self.running)

        if batch.has_prefill:
            self.model_runner.prefill(batch.prefill_requests)

        if batch.has_decode:
            tokens = self.model_runner.decode(batch.decode_requests)
            self.streamer.send(tokens)

```

这个结构适合教学项目和单机推理，但它有一个隐含假设：

一个请求的所有状态都在同一个 engine 进程里。

包括：

1. 请求输入和采样参数。
2. 当前生成到第几个 token。
3. KV cache block 分配结果。
4. prefill 是否完成。
5. decode 是否正在运行。
6. streaming channel。
7. 取消、超时、失败状态。

PD 分离会打破这个假设。

43.3 为什么不能直接拆两个服务

很多人会把 PD 分离理解成：

```

prefill_service.compute_prompt()
decode_service.generate_tokens()

```

这只是函数拆分，不是系统架构。

真正的问题是：

Prefill 生成的 KV 在哪里？

Decode 怎么知道 KV 的 shape、block id、layer 数、dtype、layout？

请求失败时谁清理 KV？

Decode worker 崩溃后 router 怎么处理已经 prefill 完的请求？

客户端取消请求时 prefill、transfer、decode 三边如何同步取消？

如果这些问题没有答案，拆服务只会得到一个更慢、更脆弱的 unified engine。

所以升级顺序应该是：

先抽象状态和接口，再拆 worker；

先做同进程逻辑分层，再做跨进程部署；

先做 same-node transfer，再做 cross-node transfer；

先能观测和回滚，再扩大流量。

43.4 升级前要先做的模块边界

单机 engine 要升级 PD 分离，至少要把五个边界切清楚。

第一，API 接入层和 engine 核心分离。

HTTP/gRPC server 不应该直接操作 KV cache、scheduler 内部队列和 model runner。

它应该只负责：

1. 接收请求。
2. 鉴权和限流。
3. 创建 request id。
4. 建立 streaming channel。
5. 把请求交给 engine/router。

第二，请求状态和执行状态分离。

请求状态是：

request_id、prompt、sampling_params、tenant、deadline、priority、stream handle

执行状态是：

current_stage、prefill_worker、decode_worker、kv_location、generated_tokens、finish_reason

第三，scheduler 和 model runner 分离。

Scheduler 负责决定谁运行。

Model runner 负责执行模型计算。

不要让 scheduler 直接依赖模型 forward 的内部 tensor。

第四，KV cache manager 和 model runner 分离。

Model runner 可以读写 KV，但 KV block 的生命周期、分配、释放、metadata 应该由 KV manager 管。

第五，streaming 输出和 decode loop 分离。

Decode worker 可以产生 token，但最好不要让它直接持有所有客户端连接状态。否则 decode worker 崩溃时很难恢复和回收。

43.5 单机 Engine 的状态拆分

升级前最重要的是状态盘点。

单机状态可以分成四类。

第一类，请求元数据。

request_id
tenant_id
prompt_token_ids
sampling_params
max_new_tokens
priority
deadline
trace_id

这类状态应该能被 router、prefill worker、decode worker 共同理解。

第二类，执行进度。

stage = WAITING_PREFILL / PREFILLING / WAITING_TRANSFER / DECODING / FINISHED / FAILED
prefill_started_at
prefill_finished_at
decode_started_at
generated_token_count
last_token_id
finish_reason

这类状态最好由 router 或外部 request state store 维护。

第三类，KV metadata。

```
kv_id
model_id
layer_count
num_heads
head_dim
dtype
block_size
block_ids
sequence_length
device_location
transfer_status
```

这类状态不能只存在于某个 worker 的 Python 对象里，因为 decode worker 必须能根据它接管请求。

第四类，运行时临时状态。

```
CUDA graph handle
temporary tensor
attention workspace
sampling buffer
local queue node
```

这类状态通常不能跨 worker 迁移，也不应该外置。

工程上最容易犯的错，是把第三类 KV metadata 和第四类 runtime temporary state 混在一起。

43.6 第一步：把 Prefill 和 Decode 变成显式阶段

很多教学 engine 里，prefill 和 decode 虽然概念上不同，但代码上混在 step() 里。

升级的第一步，是让请求阶段显式化。

例如：

```
class RequestStage:
    WAITING_PREFILL = "waiting_prefill"
    PREFILLING = "prefilling"
    WAITING_DECODE = "waiting_decode"
    DECODING = "decoding"
    FINISHED = "finished"
    FAILED = "failed"
```

请求对象里不要只有一个 is_finished，而应该有清晰阶段：

```
class RequestState:
    def __init__(self, request_id, prompt_token_ids, sampling_params):
        self.request_id = request_id
        self.prompt_token_ids = prompt_token_ids
        self.sampling_params = sampling_params
        self.stage = RequestStage.WAITING_PREFILL
        self.kv_metadata = None
        self.generated_token_ids = []
```

这样即使仍然是单进程，也能模拟 PD 生命周期：

```
WAITING_PREFILL
-> PREFILLING
-> WAITING_DECODE
```

-> DECODING
-> FINISHED

这一步的价值是：

1. 后续 router 可以接管 stage transition。
2. prefill worker 和 decode worker 可以只处理自己关心的阶段。
3. 失败恢复可以按照阶段设计。
4. 指标可以按阶段统计。

43.7 第二步：抽象 Model Runner 接口

单机 model runner 通常暴露一个粗粒度接口：

```
tokens = model.generate(input_ids, sampling_params)
```

这不适合 serving engine。

推理框架内部至少要拆成：

```
class ModelRunner:
    def prefill(self, batch):
        """Run prompt tokens and write KV cache."""

    def decode(self, batch):
        """Run one or more decode steps using existing KV cache."""
```

进一步，为 PD 分离做准备，prefill 的返回值不能只是 logits。

它应该返回：

```
class PrefillResult:
    request_id: str
    next_token_id: int
    kv_metadata: KVMetadata
    prompt_length: int
```

Decode 的输入也不能只是一串 token：

```
class DecodeInput:
    request_id: str
    last_token_id: int
    kv_metadata: KVMetadata
    sampling_params: SamplingParams
```

也就是说，decode 应该通过 kv_metadata 找到已经生成的 KV，而不是依赖同一个 Python 对象引用。

43.8 第三步：抽象 KV Cache Manager

PD 分离里最关键的不是 worker 拆分，而是 KV 生命周期。

一个单机 KV cache manager 至少要支持：

```
class KVCacheManager:
    def allocate(self, request_id, num_tokens):
        pass

    def get_metadata(self, request_id):
        pass

    def pin(self, kv_id):
        pass
```



```
def unpin(self, kv_id):  
    pass
```

```
def free(self, kv_id):  
    pass
```

为了升级 PD，还需要增加 transfer 相关抽象：

```
class KVTransferManager:  
    def export(self, kv_metadata):  
        pass  
  
    def import_(self, transfer_handle):  
        pass  
  
    def release(self, transfer_handle):  
        pass
```

这里不要过早绑定某一种后端。

后端可能是：

1. 同 GPU 内部引用。
2. 同节点 GPU-to-GPU 拷贝。
3. CPU staging buffer。
4. RDMA transfer。
5. NIXL。
6. Mooncake。
7. 远端 KV cache。

接口要表达的是：

这段 KV 可以被另一个执行单元接管。

而不是一开始就把实现写死成某个 `cudaMemcpyPeer`。

43.9 第四步：把 Scheduler 拆成两类

Unified engine 里通常只有一个 scheduler：

waiting queue + running queue -> next batch

PD 分离后至少有两个 scheduler：

1. Prefill scheduler。
2. Decode scheduler。

Prefill scheduler 关注：

1. prompt token 数。
2. chunk size。
3. prefix cache 命中。
4. input tokens/s。
5. prefill queue waiting time。

Decode scheduler 关注：

1. active sequence 数。
2. 每轮 decode batch 大小。
3. TPOT。
4. max running requests。
5. KV capacity。

6. streaming deadline。

在单机阶段，可以先做逻辑拆分：

```
one process
  prefill_scheduler
  decode_scheduler
  same model_runner
  same kv_manager
```

这时还没有真正 PD 分离，但代码已经具备演进条件。

43.10 第五步：引入 Router 状态机

Router 在 PD 分离里不是普通负载均衡。

它维护的是请求生命周期。

简化状态机如下：

```
RECEIVED
-> PREFILL_ASSIGNED
-> PREFILL_RUNNING
-> PREFILL_DONE
-> KV_TRANSFERRING
-> DECODE_ASSIGNED
-> DECODING
-> FINISHED
```

失败路径包括：

```
PREFILL_FAILED
TRANSFER_FAILED
DECODE_FAILED
CANCELLED
TIMEOUT
```

Router 至少要维护：

```
request_id -> request state
request_id -> prefill worker
request_id -> decode worker
request_id -> kv transfer handle
request_id -> stream handle
```

单机改造时，可以先把 router 做成进程内组件：

```
API server -> local router -> local engine
```

等状态机稳定后，再把它升级为独立服务。

43.11 第六步：把 Worker 角色显式化

原来的 engine worker 是全能型：

```
prefill + decode + kv allocation + streaming
```

PD 分离后，worker 要分角色。

Prefill worker 负责：

1. 接收 prompt token。
2. 运行 prefill。
3. 写入本地 KV。

4. 生成首 token 或 prefill logits。
5. 导出 KV metadata。
6. 触发或等待 KV transfer。

Decode worker 负责：

1. 接收 decode assignment。
2. 导入或挂载 KV。
3. 持续 decode。
4. 执行 sampling。
5. 输出 token stream。
6. 请求结束后释放 KV。

注意职责边界：

Prefill worker 不应该继续持有请求直到生成结束。

Decode worker 不应该重新计算完整 prompt。

Router 不应该直接操作 CUDA tensor。

43.12 第七步：先做 Same-Node PD

不要一上来跨节点。

推荐演进路径是：

阶段 1：同进程逻辑 PD

阶段 2：同节点多进程 PD

阶段 3：同节点多 GPU PD

阶段 4：跨节点 PD

Same-node PD 的好处是：

1. 网络变量少。
2. 故障域小。
3. debug 简单。
4. transfer 可以先用较简单的机制。
5. 可以验证状态机和调度策略。

如果 same-node PD 都无法带来收益，直接 cross-node PD 通常更危险。

43.13 第八步：接入 KV Transfer

KV transfer 是 PD 分离的核心数据路径。

一个 transfer 流程可以抽象成：

prefill worker

- > freeze/pin KV blocks
- > export metadata
- > transfer data
- > notify router
- > decode worker import
- > decode starts

关键点有四个。

第一，KV layout 必须一致。

Decode worker 需要知道：

1. layer 数。
2. 每层 K/V 的 shape。
3. dtype。

4. block size。
5. head layout。
6. sequence length。
7. block table。

第二，transfer 状态必须可观测。

至少要有：

```
transfer_pending
transfer_running
transfer_done
transfer_failed
transfer_cancelled
```

第三，失败要能清理。

例如：

```
prefill done, transfer failed -> free prefill KV
transfer done, decode assignment failed -> free decode-side KV
client cancelled during transfer -> cancel transfer and free both sides
```

第四，decode 不能在 KV 未 ready 时盲目开始。

否则会出现难 debug 的错误：

1. 读到未初始化 KV。
2. block table 不完整。
3. sequence length 错位。
4. attention mask 错误。
5. token 输出异常但不报错。

43.14 第九步：重新设计 Streaming 路径

单机 engine 里，streaming 通常很简单：

decode loop -> HTTP response stream

PD 分离后，decode worker 可能不在 API server 所在进程，甚至不在同一节点。

有三种常见做法。

第一，Decode worker 直接 stream 给 client。

优点是路径短。

缺点是连接管理复杂，worker 扩缩容和故障恢复困难。

第二，Decode worker stream 给 router/API server，再由 API server 转发给 client。

优点是客户端连接集中管理。

缺点是 router/API server 可能成为数据转发瓶颈。

第三，Decode worker 写入 message channel，API server 消费并返回。

优点是解耦强。

缺点是额外延迟和系统复杂度更高。

教学项目或早期系统可以先选择第二种：

decode worker -> router/API -> client

因为它更容易做鉴权、取消、超时、trace 和回放。

43.15 第十步：引入 Backpressure

PD 分离后，系统不再是一个队列，而是多个队列：

```
router queue
prefill queue
transfer queue
decode queue
streaming queue
```

如果没有 backpressure，常见事故是：

1. prefill 太快，decode 接不住，KV 堆积。
2. decode 太满，prefill 做了很多无效工作。
3. transfer backlog 增长，TTFT p99 爆炸。
4. router 继续接请求，最终所有队列一起雪崩。

Backpressure 信号至少包括：

```
prefill_queue_depth
decode_queue_depth
transfer_queue_depth
decode_kv_free_blocks
gpu_memory_pressure
ttft_p95/p99
tpot_p95/p99
```

Router 应该根据这些信号决定：

1. 是否接新请求。
2. 是否延迟 prefill。
3. 是否选择另一个 decode worker。
4. 是否降级回 unified engine。
5. 是否拒绝低优先级租户。

43.16 第十一步：加完整观测

没有观测，不要上线 PD 分离。

至少要看五段延迟：

```
request_queue_wait
prefill_time
kv_transfer_time
decode_wait_time
decode_time / tpot
```

还要看资源指标：

1. prefill GPU utilization。
2. decode GPU utilization。
3. prefill input tokens/s。
4. decode output tokens/s。
5. KV cache used/free blocks。
6. transfer bandwidth。
7. transfer p95/p99。
8. worker error rate。
9. cancel rate。
10. fallback rate。

Trace 里至少要包含：

request_id
router_id
prefill_worker_id
decode_worker_id
kv_id
transfer_id
stage timestamps
failure reason

否则 PD 分离出问题时，很容易只看到：

用户说慢了。

GPU 看起来没满。

日志里没有明显错误。

这是最难排查的状态。

43.17 第十二步：灰度和回滚

PD 分离必须能灰度。

常见灰度维度包括：

1. 按租户灰度。
2. 按模型灰度。
3. 按 prompt length 灰度。
4. 按请求比例灰度。
5. 按 region 或集群灰度。

推荐先灰度长 prompt 请求：

```
if prompt_len > threshold:
    use_pd_path
else:
    use_unified_path
```

因为长 prompt 更可能从 PD 分离里获益。

同时要保留回滚路径：

```
PD path unhealthy -> route new requests to unified engine
running PD requests -> allow finish or controlled cancel
failed transfer -> retry or fallback recompute
```

注意：已经进入 decode 的 streaming 请求，不一定能无损回滚。

所以回滚策略要区分：

1. 新请求回滚。
2. prefill 中请求回滚。
3. transfer 中请求回滚。
4. decode 中请求回滚。

43.18 一个推荐演进路线

如果从一个单机 engine 开始，推荐路线如下。

第一阶段：整理单机代码。

目标：让 engine 内部有清晰的 request state、prefill stage、decode stage、KV manager。

API -> Engine -> Prefill/Decode logical stages

第二阶段：双 scheduler。

目标：让 prefill 和 decode 在逻辑上有不同队列和调度策略。

prefill_queue
decode_queue

第三阶段：进程内 router。

目标：让 router 接管 request lifecycle。

API -> local router -> local engine

第四阶段：worker role 拆分。

目标：让同一套 worker 可以以 prefill-only 或 decode-only 模式运行。

worker --role=prefill
worker --role=decode

第五阶段：same-node PD。

目标：验证状态机、KV transfer、观测和故障清理。

router -> prefill worker -> local transfer -> decode worker

第六阶段：cross-node PD。

目标：引入网络拓扑、RDMA/NIXL/Mooncake、跨节点故障恢复和容量调度。

router -> prefill node -> network KV transfer -> decode node

第七阶段：高级优化。

目标：KV-aware routing、prefix-aware routing、多级 KV、租户隔离、自动扩缩容。

43.19 最小可行 PD 原型

一个教学级最小 PD 原型可以这样设计：

PDRouter

- request table
- choose_prefill_worker()
- choose_decode_worker()
- update_stage()

PrefillWorker

- prefill(batch)
- export_kv()

DecodeWorker

- import_kv()
- decode_loop()

KVTransferManager

- transfer(kv_metadata, src, dst)

Metrics

- stage latency
- queue depth
- transfer latency

请求路径：

1. client sends request
2. router creates request state
3. router assigns prefill worker
4. prefill worker runs prompt
5. prefill worker returns kv metadata and first token candidate
6. router assigns decode worker
7. transfer manager moves or exposes KV
8. decode worker imports KV
9. decode worker streams tokens
10. router marks request finished and releases resources

这个原型不需要一开始支持所有能力。

可以先不做：

1. 多租户。
2. prefix-aware routing。
3. remote KV cache。
4. 自动扩缩容。
5. 复杂重试。

但必须做：

1. request id。
2. stage state。
3. KV metadata。
4. transfer result。
5. cancel。
6. timeout。
7. resource cleanup。

43.20 关键接口设计示例

一个简化接口可以这样写。

```
class PDRouter:
    def submit(self, request):
        state = self.request_store.create(request)
        prefill_worker = self.choose_prefill_worker(request)
        self.assign_prefill(state, prefill_worker)

    def on_prefill_done(self, request_id, prefill_result):
        state = self.request_store.get(request_id)
        state.kv_metadata = prefill_result.kv_metadata
        state.last_token_id = prefill_result.next_token_id

        decode_worker = self.choose_decode_worker(state)
        transfer = self.transfer_manager.start(
            state.kv_metadata,
            src=prefill_result.worker_id,
            dst=decode_worker.worker_id,
        )
        state.transfer_id = transfer.transfer_id

    def on_transfer_done(self, request_id):
        state = self.request_store.get(request_id)
        self.assign_decode(state)
```

Prefill worker:


```

class PrefillWorker:
    def run_prefill(self, request):
        allocation = self.kv_manager.allocate(request.request_id, len(request.prompt_token_ids))
        logits = self.model_runner.prefill(request.prompt_token_ids, allocation)
        next_token = self.sampler.sample(logits, request.sampling_params)
        kv_metadata = self.kv_manager.get_metadata(request.request_id)
        return PrefillResult(request.request_id, next_token, kv_metadata)

```

Decode worker:

```

class DecodeWorker:
    def start_decode(self, decode_input):
        self.kv_manager.import_(decode_input.kv_metadata)
        state = self.create_local_state(decode_input)

        while not state.finished:
            logits = self.model_runner.decode(state.last_token_id, state.kv_metadata)
            token = self.sampler.sample(logits, state.sampling_params)
            self.streamer.send(state.request_id, token)
            state.update(token)

```

这些代码只是示意，真实系统会更复杂。

但它体现了一个核心原则：

跨组件传 metadata 和 handle，不传 Python 对象引用。

43.21 容量规划怎么变

Unified engine 里，容量规划通常看整体 QPS、GPU 利用率、显存和延迟。

PD 分离后，要分别看 P 和 D。

Prefill pool 容量主要由输入 token 决定：

$$\text{required_prefill_capacity} \sim \text{peak_input_tokens_per_second} / \text{prefill_tokens_per_second_per_gpu}$$

Decode pool 容量主要由输出 token 和并发序列决定：

$$\text{required_decode_capacity} \sim \text{peak_output_tokens_per_second} / \text{decode_tokens_per_second_per_gpu}$$

但这还不够，还要考虑 KV 驻留：

$$\text{decode_kv_capacity} \geq \text{active_sequences} * \text{average_context_length} * \text{kv_bytes_per_token}$$

以及 transfer：

$$\text{transfer_bandwidth} \geq \text{prefill_completed_kv_bytes_per_second}$$

如果 prefill 扩得太多而 decode 不扩，系统会变成：

prefill 很快完成，KV 大量等待 decode，TTFT 仍然很差，显存还被占满。

所以 PD 分离不是简单地多加 prefill worker 或 decode worker，而是三个容量一起配：

1. Prefill compute。
2. Decode compute 和 KV capacity。
3. KV transfer bandwidth。

43.22 故障处理设计

PD 分离的故障处理要按阶段设计。

Prefill 前失败：

请求还没有消耗大量资源，可以重试或换 prefill worker。

Prefill 中失败：

释放已分配 KV，重新排队或返回错误。

Prefill 后 transfer 前失败：

如果 prefill worker 还活着，可以重试 transfer；否则需要重新 prefill。

Transfer 中失败：

取消 transfer，清理 src/dst 临时 KV，必要时重新 transfer 或重新 prefill。

Decode 中失败：

如果已经 stream 出 token，通常无法透明重试；可以终止请求、返回错误，或在支持 deterministic replay 的条件下尝试恢复。

客户端取消：

router 负责广播 cancel 到 prefill、transfer、decode，并最终确认 KV 释放。

这里最重要的是 cleanup。

很多线上问题不是请求失败本身，而是失败后 KV 没释放，最终把显存耗尽。

43.23 数据一致性问题

PD 分离里有几个容易忽略的一致性问题。

第一，token 位置一致。

Prefill 结束时，decode 的起始 position 必须和 prompt length 对齐。

第二，首 token 归属一致。

有些系统在 prefill 阶段顺便采样首 token，有些系统让 decode 阶段采样首 token。

两种都可以，但必须统一约定：

`prefill returns next_token -> decode starts from that token`

`prefill returns logits only -> decode samples first token`

第三，sampling 状态一致。

如果使用随机采样，需要考虑 RNG seed、temperature、top-p、repetition penalty 等状态是否在 decode worker 上完整恢复。

第四，KV block table 一致。

Prefill 侧导出的 block table 和 decode 侧导入后的 block table 必须语义一致，即使物理 block id 不同。

第五，模型版本一致。

Prefill worker 和 decode worker 必须使用相同模型权重、tokenizer、rope scaling、quantization config。

否则会出现非常隐蔽的质量问题。

43.24 与 Prefix Cache 的关系

如果系统已经有 prefix cache，升级 PD 时要重新考虑 cache ownership。

有三种做法。

第一，prefix cache 只在 prefill pool 内部使用。

`router -> prefix-aware prefill worker -> generate KV -> transfer to decode`

这种最简单。

第二，decode worker 也有本地 prefix cache。

这会让路由变复杂，因为 prefill 和 decode 都有 locality。

第三，引入远端 prefix/KV cache。

这适合大规模长上下文系统，但会增加网络和一致性成本。

早期升级建议采用第一种。

也就是：

先让 prefill pool 负责 prefix cache 命中，再把最终 KV 交给 decode。

不要一开始同时做 PD 分离、prefix-aware decode routing 和 remote KV cache。

43.25 与 Chunked Prefill 的关系

Chunked prefill 是升级 PD 前常见的中间态。

如果系统已有 chunked prefill，可以这样演进：

unified engine + chunked prefill

-> logical P/D scheduler

-> prefill chunks on prefill worker

-> final KV transfer to decode worker

一个关键问题是：

KV 是每个 chunk transfer，还是 prefill 完整结束后 transfer？

两种方式取舍不同。

完整结束后 transfer：

1. 实现简单。
2. 状态少。
3. decode 启动更晚。

Chunk 级 transfer：

1. 可以隐藏部分 transfer 时间。
2. pipeline 更充分。
3. 状态机更复杂。
4. 更容易出现 partial KV 清理问题。

早期建议先做完整 prefill 后 transfer。

等稳定后再考虑 chunk-level pipeline。

43.26 常见改造反模式

第一，只拆服务，不拆状态。

表现是 prefill service 和 decode service 都依赖同一个巨大 engine 对象，最终状态同步一团乱。

第二，只拆 API，不拆资源池。

两个服务还是抢同一批 GPU，P/D 干扰没有消失。

第三，KV metadata 不标准化。

每个 worker 用自己的隐式结构，transfer 后 decode 侧只能靠特殊逻辑适配。

第四，没有 cancel 和 cleanup。

压测时看起来能跑，真实流量下一堆取消请求导致 KV 泄漏。

第五，没有 fallback。

PD path 一出问题，新请求无路可走，只能全站故障。

第六，跨节点一步到位。

状态机、观测、transfer、网络问题同时出现，debug 成本极高。

第七，只看平均延迟。

PD 分离主要看尾延迟、队列等待、transfer p99 和 TPOT jitter。平均值改善不代表系统可靠。

43.27 面试官会怎么问

问题一：如何把一个单机 LLM serving engine 升级成 PD 分离？

回答要点：先不要直接拆服务，而是先显式化 request stage，抽象 model runner 的 prefill/decode 接口，抽象 KV cache manager 和 KV metadata，再引入 router 状态机、双 scheduler、prefill/decode worker role 和 KV transfer。落地上先做同进程逻辑 PD，再 same-node PD，最后 cross-node PD，并配套观测、backpressure、灰度和回滚。

问题二：升级过程中最关键的接口是什么？

回答要点：最关键的是 KV metadata 和 request state。Decode worker 不能依赖 prefill worker 的 Python 对象引用，而要通过标准化 KV metadata 接管 KV。同时 router 要维护请求阶段、worker assignment、transfer handle、失败状态和 stream handle。

问题三：为什么不能直接把 prefill 和 decode 拆成两个服务？

回答要点：因为真正困难的是 KV transfer、分布式状态、故障清理、取消、回滚和观测。只拆服务不拆状态和资源池，无法解决 P/D 干扰，反而会增加网络 and 状态同步成本。

问题四：升级 PD 分离时如何降低风险？

回答要点：先做逻辑拆分和 same-node PD，按长 prompt 或特定租户灰度；保留 unified fallback；对 prefill、transfer、decode 分阶段打点；按阶段设计失败清理；上线前压测 transfer p99、decode TPOT、KV 泄漏和 cancel 场景。

问题五：PD 分离后容量规划有什么变化？

回答要点：要分别规划 prefill compute、decode compute/KV capacity 和 KV transfer bandwidth。输入 token 压力决定 prefill pool，输出 token 和 active sequences 决定 decode pool，prefill 完成速率决定 transfer 带宽需求。

43.28 标准回答模板

如果面试官问“你如何把单机 serving engine 改造成 PD 分离架构”，可以这样回答：

我不会一开始就把 prefill 和 decode 拆成两个远程服务，因为 PD 分离的难点不只是函数拆分，而是 request state、KV metadata、

我会先在单机 engine 内部做逻辑拆分：把请求状态显式化成 waiting_prefill、prefilling、waiting_decode、decoding、finished

第二步，我会拆 scheduler。Prefill scheduler 关注 prompt tokens、chunk size、prefix cache 和 input tokens/s；Decode sched

第三步，引入 router 状态机，由 router 维护 request_id 到 prefill worker、decode worker、kv transfer handle 和 stream hand

落地顺序上，我会先做 same-node PD，验证 KV transfer、取消、失败清理、metrics 和 fallback；稳定后再做 cross-node PD，引入 RDMA/NIXL/Mooncake 这类传输后端和 topology-aware routing。

上线时我会按长 prompt、租户或流量比例灰度，保留 unified engine fallback，并重点观察 prefill latency、transfer p95/p99、

43.29 单机到 PD 升级路线、接口门禁和可运行 demo

先把一次迁移拆成有依赖关系的 step:

$$M_j = (p_j, C_j, D_j, r_j)$$

其中 p_j 是迁移阶段, C_j 是该 step 交付的 capability 集合, D_j 是前置依赖, r_j 是风险等级。

接口 readiness 可以用一个最小覆盖率表达:

$$R_{\{\mathrm{iface}\}} = \frac{I_{\{\mathrm{stage}\}} + I_{\{\mathrm{runner}\}} + I_{\{\mathrm{kv}\}}}{3}$$

其中 $I_{\{\mathrm{stage}\}}$ 表示 request stage 已显式化, $I_{\{\mathrm{runner}\}}$ 表示 model runner 已拆出 prefill / decode 接口, $I_{\{\mathrm{kv}\}}$ 表示 KV metadata contract 已稳定。

PD path 的队列压力不能只看一个 waiting queue, 而要看多段 backlog:

$$Q_{\{\mathrm{pd}\}} = Q_{\{\mathrm{router}\}} + Q_{\{\mathrm{prefill}\}} + Q_{\{\mathrm{transfer}\}} + Q_{\{\mathrm{decode}\}}$$

迁移门禁可以写成:

$$G_{\{\mathrm{pdmig}\}} = G_{\{\mathrm{stage}\}} G_{\{\mathrm{runner}\}} G_{\{\mathrm{kv}\}} G_{\{\mathrm{sched}\}} G_{\{\mathrm{router}\}} G_{\{\mathrm{same_node_pd}\}} G_{\{\mathrm{cleanup}\}} G_{\{\mathrm{observability}\}} G_{\{\mathrm{fallback}\}}$$

这个门禁的含义是: 如果 request stage、runner split、KV contract、双 scheduler、router 状态机、same-node 原型、cleanup / cancel、observability / fallback 任何一项不成立, 就不要扩大到 cross-node PD。

下面的 demo 覆盖 9 个迁移 step:

1. explicit_request_stage: 单进程内先把 request stage 显式化。
2. model_runner_interface_split: 把 model runner 拆出 prefill / decode 接口。
3. kv_metadata_contract: 稳定 KV metadata 和 KV manager 边界。
4. scheduler_split: 先在逻辑上拆 prefill scheduler 和 decode scheduler。
5. router_state_machine: 让 router 接管 request lifecycle。
6. same_node_pd_prototype: 先做 same-node PD, 验证 KV transfer 和 decode reservation。
7. cleanup_cancel_paths: 补 transfer failure cleanup 和 client cancel。
8. metrics_backpressure_fallback: 补 metrics、backpressure 和 unified fallback。
9. cross_node_pd_readiness: 最后才进入跨节点门禁。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class MigrationStep:
```

```
    name: str
    phase: str
    capabilities: tuple
    dependencies: tuple
    risk: int
```

```
class ToyPDMigrationAuditor:
```

```
    def __init__(self):
        self.required_capabilities = [
            "explicit_stage",
            "runner_prefill_decode",
            "kv_metadata",
            "prefill_scheduler",
            "decode_scheduler",
            "router_state_machine",
            "same_node_pd",
            "transfer_cleanup",
            "client_cancel",
```

```

        "metrics",
        "backpressure",
        "fallback",
    ]

def audit(self, steps):
    rows = []
    completed = set()
    positions = {}
    for index, step in enumerate(steps):
        deps_ready = all(dep in completed for dep in step.dependencies)
        ready = deps_ready and step.risk <= 2
        row = {
            "step": step.name,
            "phase": step.phase,
            "capabilities": list(step.capabilities),
            "deps_ready": deps_ready,
            "risk": step.risk,
            "ready": ready,
        }
        rows.append(row)
        positions[step.name] = index
        if ready:
            completed.add(step.name)

    capability_ready = {
        capability: any(
            row["ready"] and capability in row["capabilities"]
            for row in rows
        )
        for capability in self.required_capabilities
    }
    same_node_before_cross_node = (
        positions["same_node_pd_prototype"] < positions["cross_node_pd_readiness"]
    )
    summary = {
        "steps": len(rows),
        "ready_steps": sum(row["ready"] for row in rows),
        "interfaces_ready": sum(
            capability_ready[name]
            for name in ["explicit_stage", "runner_prefill_decode", "kv_metadata"]
        ),
        "state_machine_ready": capability_ready["router_state_machine"],
        "cleanup_paths": sum(
            capability_ready[name]
            for name in ["transfer_cleanup", "client_cancel"]
        ),
        "fallback_ready": capability_ready["fallback"],
        "same_node_before_cross_node": same_node_before_cross_node,
    }
    gates = {
        "request_stage_explicit": capability_ready["explicit_stage"],
        "runner_interface_split": capability_ready["runner_prefill_decode"],
        "kv_metadata_contract": capability_ready["kv_metadata"],
        "scheduler_split_ready": (

```

```

        capability_ready["prefill_scheduler"] and capability_ready["decode_scheduler"]
    ),
    "router_state_machine_ready": capability_ready["router_state_machine"],
    "same_node_pd_first": capability_ready["same_node_pd"] and same_node_before_cross_node,
    "cleanup_and_cancel_ready": (
        capability_ready["transfer_cleanup"] and capability_ready["client_cancel"]
    ),
    "observability_fallback_ready": (
        capability_ready["metrics"]
        and capability_ready["backpressure"]
        and capability_ready["fallback"]
    ),
}
gates["pd_migration_gate"] = all(gates.values()) and summary["ready_steps"] == len(rows)
return rows, summary, gates

```

```

steps = [
    MigrationStep("explicit_request_stage", "single_process_refactor", ("explicit_stage",), (), 1),
    MigrationStep(
        "model_runner_interface_split",
        "single_process_refactor",
        ("runner_prefill_decode",),
        ("explicit_request_stage",),
        1,
    ),
    MigrationStep(
        "kv_metadata_contract",
        "single_process_refactor",
        ("kv_metadata", "kv_manager_split"),
        ("model_runner_interface_split",),
        1,
    ),
    MigrationStep(
        "scheduler_split",
        "logical_pd",
        ("prefill_scheduler", "decode_scheduler"),
        ("explicit_request_stage",),
        1,
    ),
    MigrationStep(
        "router_state_machine",
        "logical_pd",
        ("router_state_machine", "request_store"),
        ("kv_metadata_contract", "scheduler_split"),
        1,
    ),
    MigrationStep(
        "same_node_pd_prototype",
        "same_node_pd",
        ("same_node_pd", "decode_reservation", "kv_transfer"),
        ("router_state_machine",),
        2,
    ),
    MigrationStep(

```

```

        "cleanup_cancel_paths",
        "same_node_pd",
        ("transfer_cleanup", "client_cancel"),
        ("same_node_pd_prototype",),
        1,
    ),
    MigrationStep(
        "metrics_backpressure_fallback",
        "same_node_pd",
        ("metrics", "backpressure", "fallback"),
        ("cleanup_cancel_paths",),
        1,
    ),
    MigrationStep(
        "cross_node_pd_readiness",
        "cross_node_pd",
        ("topology_gate", "cross_node_pd"),
        ("metrics_backpressure_fallback",),
        2,
    ),
]
rows, summary, gates = ToyPDMigrationAuditor().audit(steps)
print("pd_migration_rows=", rows)
print("pd_migration_summary=", summary)
print("pd_migration_gates=", gates)

```

一次运行的核心输出类似：

```

pd_migration_summary= {'steps': 9, 'ready_steps': 9, 'interfaces_ready': 3, 'state_machine_ready': True, 'cleanup_paths': True, 'scheduler_ready': True, 'router_ready': True, 'kv_metadata_contract': True, 'same_node_pd_prototype': True, 'cross_node_pd_readiness': True}
pd_migration_gates= {'request_stage_explicit': True, 'runner_interface_split': True, 'kv_metadata_contract': True, 'scheduler_ready': True, 'router_ready': True, 'same_node_pd_prototype': True, 'cross_node_pd_readiness': True}

```

这个 demo 要表达的不是 “9 个 step 全打勾就能上线”，而是一个顺序约束：

1. 先把单机 engine 内部的 request stage、runner interface 和 KV metadata 稳住。
2. 再把 scheduler 和 router 逻辑拆开。
3. 再用 same-node PD 验证 transfer、reservation、cleanup、cancel、metrics 和 fallback。
4. 最后才进入 cross-node PD readiness。

所以本章最终门禁是 `pd_migration_gate`：只有接口、状态机、scheduler、KV contract、same-node 原型、失败清理、观测、backpressure 和 fallback 全部可验证，单机 engine 升级成 PD 分离才是可控迁移，而不是一次高风险重写。

43.30 小练习

1. 画出一个单机 serving engine 的请求生命周期，并标出哪些状态需要为 PD 分离外置。
2. 设计一个 KVMetadata 结构，包含 decode worker 接管 KV 所需的字段。
3. 写一个简化的 PD router 状态机。
4. 说明为什么 request state 和 runtime temporary state 不能混在一起。
5. 设计 same-node PD 的最小原型。
6. 设计 cross-node PD 的灰度策略。
7. 给出三种 KV transfer 失败场景和对应 cleanup 方案。
8. 比较 “prefill 完整结束后 transfer” 和 “chunk 级 transfer” 的优缺点。
9. 设计一组压测指标，验证 PD 分离是否真的改善长 prompt 请求。
10. 写一个面试回答：为什么不能直接把 prefill 和 decode 拆成两个服务？

43.31 本章总结

从单机 serving engine 升级到 PD 分离，不是简单拆函数，也不是普通微服务改造。

真正要做的是把单进程内的 generate 状态机拆成跨 router、prefill worker、decode worker、KV transfer backend 和 request state store 的分布式状态机。

推荐升级顺序是：先显式化 request stage，再抽象 model runner、KV cache manager 和 KV metadata；然后拆 prefill/decode scheduler，引入 router 状态机和 worker role；最后从 same-node PD 逐步走向 cross-node PD。

上线 PD 分离前，必须具备完整观测、backpressure、cancel、cleanup、灰度和 fallback。否则 PD 分离很容易从性能优化变成稳定性风险。

到这里，本书第五部分“PD 分离与高级 Serving 架构”就完成了闭环。下一章开始进入第六部分：如何通过 nano-vLLM、tiny-llm、mini-sglang 这类教学项目学习推理框架源码，并把前面讲过的 engine 模块落到代码阅读和实战改造中。

第 44 章 nano-vLLM 学习路线

前面五部分已经讲完了推理框架的核心概念：generate loop、KV Cache、batching、scheduler、PagedAttention、vLLM、SGLang、prefix cache、PD 分离和跨节点 serving。

从这一章开始进入第六部分：教学项目源码路线与升级实战。

这一部分的目标不是把生产级 vLLM 或 SGLang 的每一行代码都读完，而是借助更小的项目建立源码阅读骨架。

nano-vLLM 是一个很适合入门的项目，因为它有几个特点：

1. 代码量小，核心逻辑集中。
2. 接口风格接近 vLLM。
3. 包含 engine、scheduler、sequence、block manager、model runner、attention、sampler 等关键模块。
4. 支持 prefix caching、tensor parallel、torch compilation、CUDA graph 等优化方向。
5. 适合拿来做二次改造练习。

一句话概括：

学 nano-vLLM 的重点不是记住每个类名，而是把一个轻量实现和生产级 serving engine 的模块边界对应起来，知道请求如何进入 engine、如何被 scheduler 组织、KV block 如何分配、model runner 如何执行 prefill/decode，以及它距离生产系统还缺什么。

44.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 WRITING_PLAN.md 对公开资料做校准：参考 GeeeekExplorer/nano-vllm 官方仓库和 README 对 nano-vLLM 作为轻量 vLLM 实现、约 1200 行 Python、包含 Prefix Caching、Tensor Parallelism、Torch compilation 和 CUDA graph 等学习点的说明；参考仓库中 nanovllm/engine 目录对 llm_engine.py、scheduler.py、sequence.py、block_manager.py、model_runner.py 等模块边界的呈现；并结合本书前面关于 generate loop、KV cache、block manager、continuous batching、vLLM scheduler、prefix cache、TP 和 PD 分离的章节。

本讲只讲“如何用 nano-vLLM 建立 vLLM-like serving engine 的源码阅读骨架”：模块地图、阅读顺序、请求生命周期、KV block lifecycle、model runner batch contract、sampler、可观测实验和改造练习。不把某个 GitHub 提交的源码行号、函数签名、内部字段名、benchmark 数字、真实 CUDA graph 行为、生产级 vLLM 的完整 worker/executor 设计或某个版本的默认实现写成通用结论。

本讲新增 demo 是教学版 nano-vLLM source auditor：用 0 依赖 Python 模拟源码阅读笔记和实验 trace，检查 example、LLM、sampling params、sequence、engine、scheduler、block manager、model runner、attention、sampler 和模型结构是否覆盖；同时检查 request state、waiting/running queue、KV block table、free blocks、slot mapping、logits、sampled token 和 finished cleanup 是否能被实验观察到。

44.1 本章目标

读完本章，你应该能讲清：

1. 为什么 nano-vLLM 适合作为 vLLM-like 源码入门。
2. 阅读 nano-vLLM 应该按什么顺序。
3. LLM、LLMEngine、Scheduler、Sequence、BlockManager、ModelRunner 分别对应推理框架里的什么模块。
4. 如何从一次 generate() 调用追踪到 prefill、decode、KV cache 和 sampling。
5. 如何通过 nano-vLLM 理解 continuous batching 和 block-based KV 管理。
6. nano-vLLM 和生产级 vLLM 的差距在哪里。
7. 如何基于 nano-vLLM 设计改造练习和面试项目。

44.2 先明确学习目标

读教学项目源码，最忌讳两件事。

第一，像读论文一样从头到尾逐行看。

这样很容易陷入工具函数和实现细节，忘记主线。

第二，直接拿生产级框架的复杂度要求教学项目。

教学项目不会完整覆盖：

1. 多租户。
2. 分布式控制面。
3. 复杂异步 streaming。
4. 完整 OpenAI-compatible API。
5. Prefix cache 的所有边界情况。
6. 生产级 metrics、logs、traces。
7. Worker 崩溃恢复。
8. 跨节点 PD 分离。

所以读 nano-vLLM 的目标应该是：

用最小代码看清 vLLM-like engine 的骨架。

具体要回答五个问题：

1. 请求对象怎么表示？
2. 请求队列怎么调度？
3. KV cache 怎么按 block 管理？
4. prefill 和 decode 怎么共用 model runner？
5. 一轮 engine step 里到底发生了什么？

44.3 推荐先看项目结构

当前 nano-vLLM 的核心目录大致是：

```
nanovllm/  
  __init__.py  
  config.py  
  llm.py  
  sampling_params.py  
  engine/  
    llm_engine.py  
    scheduler.py  
    sequence.py  
    block_manager.py  
    model_runner.py  
  layers/  
    attention.py  
    sampler.py  
    linear.py
```

```
layernorm.py
rotary_embedding.py
embed_head.py
models/
  qwen3.py
utils/
```

这正好对应一个推理框架的分层：

```
user API
-> LLM wrapper
-> engine
-> scheduler
-> sequence state
-> block manager
-> model runner
-> model/layers
```

可以先建立映射表：

nano-vLLM 模块	推理框架概念	重点问题
llm.py	用户入口	generate() 如何把 prompt 变成请求
sampling_params.py	采样参数	temperature、max tokens 等如何传递
llm_engine.py	engine 主循环	请求如何进入队列，何时 step
scheduler.py	调度器	waiting/running 如何组成 batch
sequence.py	请求状态	prompt、output、block table、完成状态怎么保存
block_manager.py	KV block 管理	block 如何分配、复用和释放
model_runner.py	模型执行器	prefill/decode batch 如何执行
attention.py	attention kernel 封装	如何使用 KV cache 做注意力
sampler.py	token 采样	logits 如何变成 next token
qwen3.py	模型结构	权重、layer、forward 如何组织

这张表比记住某个函数名更重要。
因为项目可能迭代，但模块职责不会轻易变。

44.4 推荐阅读顺序

不要从 models/qwen3.py 开始。
模型结构代码很容易把你带回 Transformer 细节，但这本书关注的是 serving engine。
推荐顺序是：

- 1. example.py：看用户如何调用。
- 2. llm.py：看外部 API 如何封装 engine。
- 3. sampling_params.py：看采样参数如何表达。
- 4. engine/sequence.py：看一个请求在 engine 内部如何表示。
- 5. engine/llm_engine.py：看 engine 主流程。
- 6. engine/scheduler.py：看请求如何被选入 batch。
- 7. engine/block_manager.py：看 KV block 如何管理。
- 8. engine/model_runner.py：看 prefill/decode 如何执行。
- 9. layers/attention.py：看 KV cache 如何参与 attention。
- 10. layers/sampler.py：看 logits 如何采样成 token。
- 11. models/qwen3.py：最后再看模型 forward 细节。

这个顺序是从请求生命周期往下读。
也就是：

API -> request -> engine -> scheduler -> KV -> model runner -> attention -> sampler

这和线上 debug 的顺序一致。

线上排查推理问题时，你通常也不会先看模型层，而是先问：请求有没有进来？排队多久？是否被 schedule？KV 是否分配？prefill/decode 哪个慢？

44.5 第一遍：只追踪一次 generate

第一遍阅读只做一件事：追踪一次 generate()。

你应该画出这样的路径：

```
prompts
-> LLM.generate()
-> tokenize
-> create Sequence
-> LLMEngine.add_request()
-> engine loop
-> Scheduler.schedule()
-> ModelRunner.execute()
-> Sampler
-> append token
-> finish or continue
-> detokenize outputs
```

这一遍先不要纠结：

1. CUDA graph 怎么做。
2. tensor parallel 怎么切。
3. attention kernel 具体怎么优化。
4. prefix cache 细节。
5. 编译优化参数。

只需要搞清楚：

一个 prompt 如何变成一个持续生成 token 的 Sequence。

建议边读边记录四个对象：

1. 用户看到的 request。
2. engine 内部的 sequence。
3. scheduler 看到的 waiting/running item。
4. model runner 看到的 batch。

很多推理框架面试题，本质都是在问这四个视角之间如何转换。

44.6 第二遍：读 Sequence

Sequence 是理解 serving engine 的关键。

一个 sequence 通常要保存：

1. prompt token ids。
2. generated token ids。
3. 当前长度。
4. 采样参数。
5. 是否完成。
6. finish reason。
7. KV block 映射。
8. prefix cache 相关状态。

在 naive generate 里，请求状态可能只是一个 `input_ids` tensor。

但在 serving engine 里，请求状态必须长期存在，因为 decode 是多轮执行：

```
step 1 -> append token
step 2 -> append token
step 3 -> append token
...
```

每一轮都要知道：

这条序列当前在哪里？

它的 KV 在哪些 block？

它是否已经达到 `max_tokens`？

它是否遇到 `eos`？

它下一轮还能不能进入 batch？

读 `sequence.py` 时，重点看：

1. prompt 和 output 是否分开保存。
2. token append 在哪里发生。
3. sequence length 如何计算。
4. finish 条件如何判断。
5. block table 或 block id 如何挂在 sequence 上。

面试中如果让你“设计一个推理请求对象”，你的答案就应该从这里抽象出来。

44.7 第三遍：读 Scheduler

Scheduler 是 serving engine 的大脑。

教学项目里的 scheduler 通常比生产级简单，但仍然能体现核心思想：

从 waiting 队列和 running 队列里选择一批 sequence，让 model runner 执行下一步。

读 scheduler 时，重点看三件事。

第一，waiting 和 running 如何区分。

waiting: 还没完成 prefill，或还没被加入 active set

running: 已经有 KV，正在 decode

第二，prefill 和 decode 如何调度。

有些实现会把 prefill 和 decode 放在同一个 schedule 结果里，有些会分开。

关键是识别：

哪些 sequence 要跑 prefill？

哪些 sequence 要跑 decode？

第三，batch 限制来自哪里。

常见限制包括：

1. 最大 batch size。
2. 最大 token 数。
3. KV block 可用量。
4. 最大 running sequence 数。
5. prompt 长度。

Scheduler 不只是“凑 batch”。

它要在吞吐和延迟之间做取舍：

prefill 太多 -> decode TPOT 抖动
decode 优先 -> 新请求 TTFT 变差
batch 太大 -> 单步慢
batch 太小 -> GPU 利用率低

这正好对应前面 continuous batching 和 PD 分离的内容。

44.8 第四遍：读 Block Manager

BlockManager 对应 vLLM 最核心的思想之一：用 block 管理 KV cache。

不要把它理解成普通显存分配器。

它解决的是：

大量请求的 KV cache 长度不同、生命周期不同，如何避免整段连续显存分配带来的浪费和碎片？

读 `block_manager.py` 时，重点看：

1. block size 是多少。
2. 总 block 数如何计算。
3. free block 如何维护。
4. sequence 如何申请 block。
5. decode 生成新 token 时是否需要追加 block。
6. 请求结束时如何释放 block。
7. prefix cache 命中时 block 如何复用。

可以把它和第 18 章对应起来：

Sequence -> logical token positions

BlockManager -> physical KV blocks

block table -> logical-to-physical mapping

这也是面试最常问的部分。

如果你能从 nano-vLLM 的 block manager 讲到生产级 vLLM 的 PagedAttention，就说明已经抓住了主线。

44.9 第五遍：读 Model Runner

ModelRunner 是 engine 和模型之间的桥。

它通常负责：

1. 加载模型。
2. 准备输入 tensor。
3. 准备 position ids。
4. 准备 slot mapping 或 block table。
5. 调用模型 forward。
6. 接收 logits。
7. 调用 sampler。
8. 返回 next token。

读 `model_runner.py` 时，不要只看 forward。

重点是看 batch 如何从高层对象变成 tensor：

Sequence list

-> input token ids

-> positions

-> block tables / slot mapping

-> model forward

-> logits

-> sampled token ids

同时要区分 prefill 和 decode 的输入形态：

prefill: one request may contribute many prompt tokens

decode: one running request usually contributes one new token per step

如果一个实现没有非常显式地区分 prefill/decode，也要在阅读时主动标注出来。

因为生产级优化几乎都围绕这个差异展开。

44.10 第六遍：读 Attention

layers/attention.py 是连接模型计算和 KV cache 的地方。

读 attention 时，重点不是重新学习 attention 公式，而是搞清楚 KV cache 如何被访问。

关键问题包括：

1. 当前 token 的 Q 从哪里来。
2. K/V 如何写入 cache。
3. 历史 K/V 如何从 block 中读出。
4. prefill 和 decode 是否走不同路径。
5. block table 如何参与 attention。
6. attention 输出如何回到模型后续层。

可以用一句话理解：

ModelRunner 负责把 batch 描述清楚，Attention 负责按这些描述读写 KV 并完成计算。

如果你只看模型层代码，很容易以为 KV cache 是普通 tensor。

但 serving engine 里的 KV cache 是被 block manager、scheduler 和 attention kernel 共同管理的系统资源。

44.11 第七遍：读 Sampler

Sampler 代码通常很短，但不要忽略它。

因为推理服务的输出行为由 sampler 决定。

读 sampler 时，重点看：

1. temperature 如何使用。
2. greedy 和 sampling 如何切换。
3. top-k、top-p 是否支持。
4. logits shape 是什么。
5. 每个 sequence 是否可以有不同 sampling params。
6. 采样结果如何写回 sequence。

生产级系统里，sampler 还会涉及：

1. repetition penalty。
2. presence/frequency penalty。
3. logprobs。
4. stop words。
5. structured output 约束。
6. speculative decoding 验证。

nano-vLLM 的 sampler 可以作为最小入口。

后续如果你要扩展功能，可以从这里开始。

44.12 第八遍：读模型结构

最后再看 `models/qwen3.py`。

这一部分用于回答：

serving engine 准备好的 `input`、`position`、`KV cache metadata`，最终如何进入具体模型 `forward`？

读模型结构时，重点看：

- 1. `embedding`。
- 2. `decoder layers`。
- 3. `attention` 层如何接收 `position` 和 `KV cache`。
- 4. `MLP`。
- 5. `norm`。
- 6. `lm head`。
- 7. 权重加载。

这部分和普通 Transformer 实现相似，但 serving 场景多了几个关键点：

- 1. 不再每次传完整上下文。
- 2. `KV cache` 是外部管理的。
- 3. `decode` 时通常只输入新 `token`。
- 4. `position` 必须和已有 `KV` 对齐。
- 5. `tensor parallel` 会改变 `linear` 层和权重加载方式。

所以读模型结构时，要一直带着 serving 问题看。

44.13 把 nano-vLLM 映射回本书前面章节

可以把 nano-vLLM 作为前面章节的实物索引。

本书章节	nano-vLLM 观察点
第 7 章最小 generate loop	<code>LLM.generate()</code> 和 engine loop
第 8 章采样	<code>sampling_params.py</code> 、 <code>sampler.py</code>
第 9 章 KV Cache	<code>block_manager.py</code> 、 <code>attention.py</code>
第 10 章 batched prefill	model runner 如何组织 prompt batch
第 11 章 batched decode	running sequence 如何逐步 decode
第 12 章 scheduler	<code>scheduler.py</code>
第 17 章 PagedAttention	block table 和 attention 访问 KV
第 18 章 Block Manager	<code>block_manager.py</code>
第 19 章 Continuous Batching	engine step + waiting/running 调度
第 20 章 vLLM 请求调度流程	从 generate 到 schedule 到 execute
第 23 章 Prefix Cache	prefix block 复用逻辑
第 24 章 Serving 并行	tensor parallel 相关代码

这样读源码就不是孤立阅读，而是把抽象概念落回代码。

44.14 一轮 Engine Step 应该怎么理解

读完核心模块后，要能讲清一轮 engine step。

一个简化版本是：

- 1. `scheduler` 查看 `waiting/running sequences`
- 2. 判断哪些请求可以 `prefill`，哪些请求可以 `decode`
- 3. 为新请求分配 `KV blocks`
- 4. 为 batch 准备 `input ids`、`positions`、`block tables`

5. model runner 执行 forward
6. sampler 产生 next token
7. sequence 追加 token
8. 判断是否 finished
9. finished sequence 释放 KV blocks
10. 未完成 sequence 留在 running 队列等待下一轮

这就是 continuous batching 的核心。

它和 naive generate 的差别是：

naive generate: 一个请求完整生成完再处理下一个

serving engine: 每轮从多个请求中动态组成 batch, 只推进一步或一段

如果你能用 nano-vLLM 的代码解释这十步, 就说明源码主线已经打通。

44.15 读 Prefix Cache 时看什么

nano-vLLM 支持 prefix caching, 这是连接 vLLM 和 SGLang 思想的重要入口。

读 prefix cache 时, 重点看：

1. prefix 如何被 hash。
2. cache 命中后复用的是 token、hidden state 还是 KV block。
3. 命中的 block 如何挂到当前 sequence。
4. 未命中的后缀如何继续 prefill。
5. 请求结束后 block 是否还能保留。
6. cache eviction 如何做。

要特别注意 ownership：

一个 block 是当前 sequence 独占, 还是 prefix cache 共享？

释放 sequence 时是否真的释放物理 block？

这能帮助你理解为什么生产级 prefix cache 需要引用计数、pin/unpin、eviction policy 和内存压力控制。

44.16 读 Tensor Parallel 时看什么

nano-vLLM 也可以作为 tensor parallel 入门。

但这里不要先陷入分布式通信细节。

先看三个问题：

1. 哪些 linear 层被切分。
2. 权重加载时如何按 rank 取 shard。
3. forward 中哪里需要通信或聚合。

常见切分包括：

qkv projection: column parallel

output projection: row parallel

mlp up/gate: column parallel

mlp down: row parallel

lm head: vocab parallel or replicated

把它和第 24 章联系起来：

TP 提升单 replica 可承载模型规模, 但会引入每层通信; serving 中 decode 高频, 跨节点 TP 尤其危险。

教学项目通常不会覆盖生产级所有通信优化, 但足够理解 TP 如何进入模型层。

44.17 nano-vLLM 和生产级 vLLM 的差距

nano-vLLM 的价值在于小，不在于完整。

它和生产级 vLLM 至少有这些差距：

1. API server 能力不足。
2. 异步 streaming 能力不足。
3. 多模型、多 LoRA、多租户能力不足。
4. 调度策略相对简单。
5. Prefix cache 边界情况更少。
6. Metrics、logs、traces 不完整。
7. Worker/executor 架构不完整。
8. 分布式部署和故障恢复不完整。
9. 生产级内存治理不足。
10. 插件化 kernel 和硬件后端适配不足。

但它保留了最核心的骨架：

sequence state + scheduler + block manager + model runner + attention + sampler

这正是入门应该先掌握的部分。

44.18 适合做的改造练习

读完 nano-vLLM 后，最好的学习方式是改代码。

推荐练习一：增加 TTFT/TPOT 统计。

记录每个 sequence 的：

1. request arrived time。
2. prefill start/end。
3. first token time。
4. 每个 decode token 时间。
5. finish time。

推荐练习二：打印 scheduler 决策。

每轮 step 输出：

1. waiting 数量。
2. running 数量。
3. prefill batch。
4. decode batch。
5. KV free blocks。

推荐练习三：实现简单 max token budget。

限制每轮 prefill token 数，观察 TTFT 和 TPOT 变化。

推荐练习四：实现 decode-first 策略。

优先保证 running sequence 的 decode，再插入 prefill，观察新请求饥饿问题。

推荐练习五：实现简单 chunked prefill。

把长 prompt 拆成多个 chunk，避免一次 prefill 长时间阻塞 decode。

推荐练习六：扩展 sampler。

加入 top-k、top-p、repetition penalty 或 stop token 逻辑。

推荐练习七：增加 block 使用可视化。

输出每个 sequence 占用哪些 block，结束后哪些 block 被释放。

推荐练习八：模拟 KV 泄漏。

故意不释放 finished sequence 的 block，观察可用 block 如何下降。

推荐练习九：实现最小 HTTP API。

把离线 generate() 包一层服务接口，理解 offline inference 和 online serving 的差异。

推荐练习十：抽象 prefill/decode 接口。

为后续 PD 分离做准备，让 model runner 的 prefill 和 decode 更显式。

44.19 适合写进简历的项目描述

如果你基于 nano-vLLM 做过改造，可以这样写项目描述：

基于 nano-vLLM 实现了一个轻量级 LLM serving engine 学习项目，阅读并改造 engine、scheduler、block manager、model runner 和 first 调度策略，用压测对比不同策略下的首 token 延迟、decode 抖动和 KV 利用率。

更偏系统方向可以写：

在 nano-vLLM 基础上抽象 request state、KV metadata 和 prefill/decode 执行接口，模拟 vLLM-like continuous batching 流程，并

面试时不要只说“我读过 nano-vLLM”。

要说清：

1. 你读了哪些模块。
2. 你画出了什么请求路径。
3. 你改了什么策略。
4. 你压测了什么指标。
5. 你发现了什么 tradeoff。

44.20 面试官会怎么问

问题一：nano-vLLM 适合用来学什么？

回答要点：适合学习 vLLM-like serving engine 的核心骨架，包括 LLM API、engine loop、sequence state、scheduler、KV block manager、model runner、attention 和 sampler。它代码量小，便于从一次 generate 调用追踪到 prefill、decode、KV cache 和 sampling。

问题二：你会按什么顺序读 nano-vLLM？

回答要点：先看 example 和 LLM.generate，再看 sampling params 和 sequence 表示；然后看 llm_engine 主循环、scheduler、block_manager、model_runner；最后看 attention、sampler 和具体模型结构。顺序是沿请求生命周期从上到下读。

问题三：nano-vLLM 里的 block manager 解决什么问题？

回答要点：它用固定大小 block 管理不同长度、不同生命周期请求的 KV cache，避免为每个请求分配连续大段 KV，降低显存碎片和浪费。Sequence 记录逻辑 token，block manager 维护物理 block，attention 通过 block table 读写 KV。

问题四：nano-vLLM 和生产级 vLLM 差在哪里？

回答要点：nano-vLLM 保留核心骨架，但缺少生产级 API server、异步 streaming、多租户、多模型、复杂调度、完整观测、worker/executor 架构、分布式部署、故障恢复和更复杂的内存治理。

问题五：如何基于 nano-vLLM 做一个有价值的项目？

回答要点：可以增加 TTFT/TPOT 指标、scheduler 日志、KV block 可视化，实现 token budget、decode-first、chunked prefill、sampler 扩展或最小 HTTP API，并通过压测比较不同调度策略对吞吐、TTFT、TPOT 和 KV 利用率的影响。

44.21 标准回答模板

如果面试官问 “你怎么通过 nano-vLLM 学 vLLM”，可以这样回答：

我会把 nano-vLLM 当成 vLLM-like serving engine 的最小骨架来看，而不是只看模型 forward。阅读顺序是沿请求生命周期走：先看

读完后我会重点总结一轮 engine step: scheduler 选择 prefill/decode 请求, block manager 分配或追加 KV block, model runner

nano-vLLM 和生产级 vLLM 的差距主要在生产能力上，比如 API server、异步 streaming、多租户、多模型、worker/executor、复杂

如果要做项目，我会在 nano-vLLM 上增加 TTFT/TPOT 统计、scheduler 决策日志和 KV block 使用可视化，再实现简单 token budget、first 或 chunked prefill，用压测比较不同策略对首 token 延迟、decode 抖动、吞吐和显存利用率的影响。

44.22 nano-vLLM 源码覆盖率、实验门禁和可运行 demo

先把源码学习对象抽象成模块集合：

$$\mathcal{M} = \{m_1, m_2, \dots, m_N\}$$

阅读覆盖率可以写成：

$$C_{\{\mathrm{module}\}} = \frac{|\mathcal{M}_{\{\mathrm{seen}\}} \cap \mathcal{M}_{\{\mathrm{req}\}}|}{|\mathcal{M}_{\{\mathrm{req}\}}|} \times \max(1, |\mathcal{M}_{\{\mathrm{req}\}}|)$$

资源生命周期覆盖率可以写成：

$$C_{\{\mathrm{resource}\}} = \frac{|\mathcal{R}_{\{\mathrm{seen}\}} \cap \mathcal{R}_{\{\mathrm{req}\}}|}{|\mathcal{R}_{\{\mathrm{req}\}}|} \times \max(1, |\mathcal{R}_{\{\mathrm{req}\}}|)$$

实验覆盖率可以写成：

$$C_{\{\mathrm{experiment}\}} = \frac{|\mathcal{E}_{\{\mathrm{done}\}} \cap \mathcal{E}_{\{\mathrm{req}\}}|}{|\mathcal{E}_{\{\mathrm{req}\}}|} \times \max(1, |\mathcal{E}_{\{\mathrm{req}\}}|)$$

最终门禁：

$$G_{\{\mathrm{nano}\}} = G_{\{\mathrm{module}\}} G_{\{\mathrm{path}\}} G_{\{\mathrm{step}\}} G_{\{\mathrm{state}\}} G_{\{\mathrm{kv}\}} G_{\{\mathrm{ru}\}}$$

这组公式的作用，是避免 “读过 nano-vLLM” 停在文件名层面。你需要证明自己知道：

1. 哪些模块构成 request lifecycle。
2. 哪些状态和资源在模块之间传递。
3. 哪些实验能观察到这些状态变化。
4. 哪些点属于教学项目边界，不应该夸大成生产级能力。

下面的 demo 模拟两类输入：

1. ModuleNote：每个源码模块的 layer、涉及资源和可观察信号。
2. ExperimentTrace：每个实验触达哪些模块、观察到哪些信号。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class ModuleNote:
```

```
    name: str
```

```
    layer: str
```

```
    resources: tuple
```

```
    signals: tuple
```

```
@dataclass
```

```
class ExperimentTrace:
```

```
    name: str
```

```
    touched_modules: tuple
```

```
    observed_signals: tuple
```

```

class ToyNanoVLLMSourceAuditor:
    def __init__(self):
        self.required_modules = [
            "example",
            "llm",
            "sampling_params",
            "sequence",
            "llm_engine",
            "scheduler",
            "block_manager",
            "model_runner",
            "attention",
            "sampler",
            "qwen3_model",
        ]
        self.required_resources = [
            "request_state",
            "waiting_queue",
            "running_queue",
            "kv_block_table",
            "free_blocks",
            "input_ids",
            "positions",
            "slot_mapping",
            "logits",
            "sampled_token",
            "finished_cleanup",
        ]
        self.required_experiments = [
            "trace_generate",
            "prefill_decode_step",
            "scheduler_budget",
            "kv_block_reuse",
            "prefix_cache_hit",
            "decode_finish_cleanup",
            "sampler_params",
        ]

    def coverage(self, seen, required):
        return round(len(set(seen) & set(required)) / max(1, len(required)), 3)

    def audit(self, module_notes, experiments):
        module_names = [note.name for note in module_notes]
        resources_seen = sorted({resource for note in module_notes for resource in note.resources})
        experiment_names = [experiment.name for experiment in experiments]
        observed_signals = sorted(
            {signal for note in module_notes for signal in note.signals}
            | {signal for experiment in experiments for signal in experiment.observed_signals}
        )
        module_positions = {name: index for index, name in enumerate(module_names)}
        lifecycle_order_ok = all(
            module_positions[self.required_modules[i]] < module_positions[self.required_modules[i + 1]]
            for i in range(len(self.required_modules) - 1)
        )

```

```

)
touched_runtime_modules = sorted(
    {module for experiment in experiments for module in experiment.touched_modules}
)
summary = {
    "module_coverage": self.coverage(module_names, self.required_modules),
    "resource_coverage": self.coverage(resources_seen, self.required_resources),
    "experiment_coverage": self.coverage(experiment_names, self.required_experiments),
    "lifecycle_order_ok": lifecycle_order_ok,
    "runtime_module_touch": self.coverage(touched_runtime_modules, self.required_modules),
    "observed_signal_count": len(observed_signals),
    "missing_modules": sorted(set(self.required_modules) - set(module_names)),
    "missing_resources": sorted(set(self.required_resources) - set(resources_seen)),
    "missing_experiments": sorted(set(self.required_experiments) - set(experiment_names)),
}
gates = {
    "module_map_complete": summary["module_coverage"] == 1.0,
    "generate_path_visible": "trace_generate" in experiment_names,
    "engine_step_visible": "prefill_decode_step" in experiment_names,
    "sequence_state_visible": "request_state" in resources_seen,
    "scheduler_decision_visible": "scheduler_budget" in experiment_names,
    "kv_lifecycle_visible": (
        "kv_block_reuse" in experiment_names and "finished_cleanup" in resources_seen
    ),
    "runner_batch_contract_visible": all(
        resource in resources_seen for resource in ["input_ids", "positions", "slot_mapping"]
    ),
    "experiments_cover_core_paths": summary["experiment_coverage"] == 1.0,
    "signals_observable": summary["observed_signal_count"] >= 12,
}
gates["nano_vllm_source_gate"] = all(gates.values())
return summary, gates

```

```

module_notes = [
    ModuleNote("example", "entry", ("request_state",), ("prompt_list",)),
    ModuleNote("llm", "api", ("request_state",), ("generate_call",)),
    ModuleNote("sampling_params", "api", ("sampled_token",), ("temperature", "max_tokens")),
    ModuleNote(
        "sequence",
        "state",
        ("request_state", "kv_block_table", "sampled_token"),
        ("token_append", "finish_reason"),
    ),
    ModuleNote(
        "llm_engine",
        "engine",
        ("waiting_queue", "running_queue"),
        ("engine_step", "prefill_decode_flag"),
    ),
    ModuleNote(
        "scheduler",
        "scheduler",
        ("waiting_queue", "running_queue", "free_blocks"),
        ("scheduled_tokens", "preempted_seq"),
    ),

```


读成了可复盘的推理框架源码路线。

44.23 小练习

1. 画出 nano-vLLM 从 LLM.generate() 到返回文本的完整调用路径。
2. 总结 Sequence 中哪些字段属于请求输入，哪些字段属于执行状态，哪些字段属于 KV 状态。
3. 画出 scheduler 中 waiting/running 队列的状态变化。
4. 解释 block manager 如何为一个新请求分配 KV block。
5. 解释 decode 生成新 token 时，sequence、block manager 和 attention 分别发生什么变化。
6. 给 nano-vLLM 增加 TTFT、TPOT 和总 latency 统计。
7. 给 scheduler 增加一轮 step 的 debug 日志。
8. 实现一个简单的 prefill token budget，并压测不同 budget 的效果。
9. 实现一个简单 chunked prefill，并观察 decode TPOT 是否更稳定。
10. 写一个面试回答：nano-vLLM 和生产级 vLLM 的核心差距是什么？

44.24 本章总结

nano-vLLM 是学习 vLLM-like 推理框架的好入口。

它的价值不是完整复刻生产级 vLLM，而是用较小代码展示 serving engine 的核心骨架：LLM API、engine loop、sequence state、scheduler、block manager、model runner、attention 和 sampler。

阅读 nano-vLLM 时，应该沿请求生命周期从上到下读，而不是从模型层开始逐行看。第一遍追踪一次 generate()，第二遍看 Sequence，第三遍看 Scheduler，第四遍看 BlockManager，第五遍看 ModelRunner，最后再看 attention、sampler 和模型结构。

真正掌握 nano-vLLM 的标志，是你能讲清一轮 engine step 中 scheduler、KV block、model runner、attention、sampler 和 sequence state 如何协作。

下一章会继续看另一个教学项目：tiny-llm。它的侧重点会更偏从极简 LLM 实现理解模型、推理和工程结构之间的关系。

第 45 章 tiny-llm 学习路线

上一章讲了 nano-vLLM 学习路线。nano-vLLM 更像一个浓缩版 vLLM-like serving engine：重点是 Sequence、Scheduler、BlockManager、ModelRunner、attention 和 sampler 如何组成一个推理引擎。

这一章看另一个教学项目：tiny-llm。

tiny-llm 的定位有所不同。它是一个面向系统工程师的 LLM serving 课程项目，目标是在 Apple Silicon 和 MLX 环境中，从底层数组和矩阵 API 开始，一步步实现 Qwen 类模型推理和 serving 系统。

它的路线大致是：

1. 第 1 周：attention、RoPE、GQA、RMSNorm、MLP、模型加载、generate、sampling。
2. 第 2 周：KV cache、量化 matmul、FlashAttention、continuous batching、chunked prefill。
3. 第 3 周：paged attention、MoE、speculative decoding、RAG、Agent、long context 等高级主题。

一句话概括：

学 tiny-llm 的重点是从“模型算子和单请求 generate”一路走到“KV cache、batching、chunked prefill、paged attention 这些 serving 优化”，理解推理系统不是凭空出现的，而是在模型 forward 的每个瓶颈上逐步长出来的。

45.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 WRITING_PLAN.md 对公开资料做校准：参考 skyzh/tiny-llm 官方仓库和课程首页对 tiny-llm 作为面向系统工程师的 LLM serving 课程项目、基于 Apple Silicon / MLX / 矩阵 API 从零实现 Qwen3 推理和 serving 系统的定位说明；参考课程路线对 attention、RoPE、GQA、RMSNorm、MLP、模型加载、

generate、sampling、KV cache、quantized matmul、FlashAttention、continuous batching、chunked prefill、paged attention、MoE、speculative decoding、RAG、Agent 和 long context 的分周安排；并结合本书前面关于最小 generate、KV cache、sampling、batched prefill/decode、continuous batching、chunked prefill、PagedAttention 和 serving 性能指标的章节。

本讲只讲 “如何用 tiny-llm 从模型实现一路读到 serving 优化”：算子 shape、position 对齐、GQA 与 KV cache、generate loop、sampling、KV cache 等价性、TTFT / TPOT、batching trace、chunk position continuity 和 block table trace。不把 MLX API 细节、某个课程版本的目录顺序、真实性能数字、硬件特性、生产级 vLLM/SGLang worker 架构或完整分布式控制面写成通用结论。

本讲新增 demo 是教学版 tiny-llm learning auditor：用 0 依赖 Python 模拟 13 个 learning unit，从 attention、RoPE、GQA、RMSNorm/MLP、模型加载、generate、sampling，一路到 KV cache、quantized matmul、FlashAttention、continuous batching、chunked prefill 和 paged attention；检查概念覆盖、实验覆盖、依赖顺序、serving 优化覆盖和最终学习门禁。

45.1 本章目标

读完本章，你应该能讲清：

- 1. tiny-llm 和 nano-vLLM 的学习重点有什么不同。
- 2. 为什么 tiny-llm 适合系统工程师学习 LLM inference serving。
- 3. 应该按什么顺序读 tiny-llm 的课程和源码。
- 4. 如何从 attention、RoPE、GQA、RMSNorm、MLP 过渡到完整模型推理。
- 5. 如何从单请求 generate 过渡到 KV cache 和 continuous batching。
- 6. 如何理解 FlashAttention、chunked prefill、paged attention 在 serving 中的位置。
- 7. 如何基于 tiny-llm 设计改造练习和面试项目。

45.2 tiny-llm 和 nano-vLLM 的区别

两个项目都适合学习推理框架，但侧重点不同。

项目	更适合学习	阅读主线
nano-vLLM	vLLM-like engine 骨架	API、engine、scheduler、sequence、block manager、model runner
tiny-llm	从模型实现到 serving 优化	attention、RoPE、GQA、模型加载、generate、KV cache、batching、paged attention

nano-vLLM 更像：

给你一个已经成型的轻量 serving engine，学习它的模块边界。

tiny-llm 更像：

从 attention 和模型 forward 开始，逐步把它改造成 serving engine。

所以两者互补。

推荐学习顺序是：

- 1. 先用 tiny-llm 理解模型推理和底层算子。
- 2. 再用 nano-vLLM 理解 vLLM-like engine 的整体骨架。
- 3. 最后用 mini-sglang 理解 SGLang 的 runtime、prefix sharing 和 structured generation。

如果你已经读过 nano-vLLM，再读 tiny-llm，就要反过来问：

nano-vLLM 里的 ModelRunner、Attention、BlockManager、Scheduler，分别是从小 tiny-llm 哪些基础步骤演进来的？

45.3 tiny-llm 的课程式结构

tiny-llm 的一个优点是路线非常课程化。

它不是一上来给你一个完整框架，而是分周推进。

第 1 周主要回答：

如何不用高级神经网络封装，从基础矩阵 API 实现一个能生成文本的 Qwen-like decoder-only 模型？

包括：

1. Attention。
2. RoPE positional encoding。
3. Grouped Query Attention。
4. RMSNorm 和 MLP。
5. 模型权重加载。
6. Generate response。
7. Sampling。

第 2 周主要回答：

如何把能跑的单请求模型，变成更高效的 inference serving 系统？

包括：

1. KV cache。
2. Quantized matmul and linear。
3. FlashAttention。
4. Continuous batching。
5. Chunked prefill。

第 3 周主要回答：

如何继续向生产级推理框架靠近？

包括：

1. Paged attention。
2. MoE。
3. Speculative decoding。
4. RAG pipeline。
5. Agent / tool calling。
6. Long context。

这条路线和本书前面章节高度对应。

45.4 推荐阅读顺序

tiny-llm 最好按课程顺序读，而不是按目录随便跳。

推荐顺序是：

1. 先读 README 和 book overview，确认课程目标和环境。
2. Week 1 overview：建立“从算子到模型”的全局图。
3. Attention：理解最小 attention 计算。
4. RoPE：理解 position 如何进入 Q/K。
5. GQA：理解 query heads 和 KV heads 为什么可以不同。
6. RMSNorm and MLP：补齐 decoder layer。
7. Qwen3 model：把模块组装成完整模型。
8. Generate response：实现自回归生成。
9. Sampling：从 logits 到 token。
10. Week 2 overview：切换到 serving 优化视角。
11. KV cache：理解为什么 decode 不能每步重算全上下文。

12. Quantized matmul: 理解权重量化和推理算子。
13. FlashAttention: 理解 memory-efficient attention。
14. Continuous batching: 理解多请求调度。
15. Chunked prefill: 理解长 prompt 和 decode 抖动。
16. Paged attention: 理解 block-based KV 管理。

这个顺序有一个清晰主线:

模型能算 -> 模型能生成 -> 生成能复用 KV -> 多请求能 batch -> 长 prompt 能切 chunk -> KV 能分页管理
这也是一个 naive inference 程序演进成 serving engine 的过程。

45.5 第一阶段：从 Attention 开始

tiny-llm 的第一站是 attention。

不要觉得 attention 公式已经学过就跳过。

在 serving 语境下, attention 需要关注的不仅是:

$\text{softmax}(QK^T / \sqrt{d})V$

还要关注:

1. Q/K/V 的 shape。
2. batch 维度如何组织。
3. causal mask 如何生效。
4. prefill 和 decode 的 attention shape 有什么不同。
5. 中间 attention score 是否占用大量内存。
6. 后续 KV cache 会缓存 K/V 的哪一部分。

建议你在读 attention 时写下两个 shape。

Prefill:

Q: [batch, prompt_len, num_heads, head_dim]
K: [batch, prompt_len, num_kv_heads, head_dim]
V: [batch, prompt_len, num_kv_heads, head_dim]

Decode:

Q: [batch, 1, num_heads, head_dim]
K/V cache: [batch, past_len + 1, num_kv_heads, head_dim]

这个差异是后面 KV cache、FlashAttention、continuous batching 和 PD 分离的基础。

45.6 第二阶段：RoPE 和位置

RoPE 是很多 decoder-only LLM 的关键组件。

读 tiny-llm 的 RoPE 时, 重点不是背公式, 而是理解:

为什么 decode 时 position 必须和 KV cache 长度严格对齐?

在自回归生成中, 第 t 个 token 的 position 不能错。

如果 position 错了, 表面上模型仍然能 forward, 但输出质量会明显异常。

Serving 系统里 position 相关 bug 很常见:

1. prompt length 计算错。
2. padding 后 position 没处理好。
3. chunked prefill 后 position 没连续。
4. KV cache 复用后 position 偏移错。
5. sliding window 或 long context 扩展后 position 策略不一致。

所以读 RoPE 时，要把它和后面的 KV cache 连接起来：

KV cache 记录历史 token 的 K/V，RoPE 决定这些 K/V 对应的位置信息。

45.7 第三阶段：Grouped Query Attention

GQA 是现代 LLM serving 非常重要的结构。

它的核心是：

$\text{num_query_heads} > \text{num_kv_heads}$

也就是说，多个 query heads 共享更少的 K/V heads。

这对 serving 很重要，因为 KV cache 大小和 num_kv_heads 成正比。

KV bytes 近似为：

$\text{KV bytes} \sim 2 * \text{num_layers} * \text{seq_len} * \text{num_kv_heads} * \text{head_dim} * \text{bytes_per_elem}$

所以 GQA 可以显著降低 KV cache 内存。

读 tiny-llm 的 GQA 时，重点看：

1. Q heads 和 KV heads 的 shape 差异。
2. K/V 如何 repeat 或 broadcast 给 Q heads。
3. GQA 对 attention 计算的影响。
4. GQA 对 KV cache 容量的影响。

这也是面试常问点：

为什么 GQA/MQA 对推理更友好？

答案不是“参数少”这么简单，而是 KV cache 更小，decode 阶段读 KV 的内存压力更低。

45.8 第四阶段：RMSNorm 和 MLP

RMSNorm 和 MLP 看起来更偏模型结构，但也影响推理性能。

读这一部分时，重点看：

1. RMSNorm 是否在 attention 和 MLP 前使用。
2. Norm 是否是逐 token 计算。
3. MLP 是普通 FFN 还是 gated MLP。
4. 中间维度 expansion 有多大。
5. 激活函数是什么。
6. 权重 shape 如何加载。

Serving 中，MLP 往往贡献大量矩阵乘计算。

Attention 在长上下文时很显眼，但每层 MLP 的 GEMM 也非常重要。

后面读量化 matmul 时，要记住：

量化优化的主要对象之一就是 Linear/MLP 权重矩阵。

45.9 第五阶段：组装 Qwen-like 模型

读到 Qwen3 model 时，要从组件视角切换到完整 forward 视角。

一个 decoder-only 模型大致是：

```
token ids
-> token embedding
-> N x decoder layer
```

- > RMSNorm
- > attention with RoPE/GQA/KV
- > residual
- > RMSNorm
- > MLP
- > residual
- > final norm
- > lm head
- > logits

读模型组装代码时，要回答：

1. config 如何定义层数、hidden size、head 数、kv head 数。
2. 权重文件如何映射到模块。
3. 每层 forward 的输入输出 shape 是什么。
4. attention 如何接收 position 和 KV cache。
5. logits 只取最后一个 token 还是所有 token。

推理服务通常只关心最后位置的 logits：

prefill：需要 prompt 最后一个位置的 logits 来采样首 token

decode：每步输入一个 token，输出这个位置的 logits

理解这一点，才能理解为什么 prefill 和 decode 可以分阶段优化。

45.10 第六阶段：Generate Response

有了模型 forward，还不等于有推理服务。

下一步是自回归生成：

```
input prompt
-> model forward
-> logits
-> sample next token
-> append token
-> repeat until eos or max_tokens
```

这就是最小 generate loop。

读 tiny-llm 的 generate 时，要重点看：

1. prompt 如何 tokenize。
2. 第一次 forward 输入的是完整 prompt 还是最后 token。
3. 新 token 如何 append。
4. eos 如何判断。
5. max tokens 如何限制。
6. 是否每一步都重算完整上下文。

如果没有 KV cache，naive generate 的复杂度会很差：

```
step 1: compute prompt_len
step 2: compute prompt_len + 1
step 3: compute prompt_len + 2
...
```

这就是第 2 周 KV cache 出现的动机。

45.11 第七阶段：Sampling

Sampling 把 logits 变成 next token。

读 tiny-llm 的 sampling 时，重点看：

1. greedy 如何实现。
2. temperature 如何缩放 logits。
3. top-k 如何截断候选集合。
4. top-p 如何按累计概率截断。
5. 随机数如何采样。
6. 采样结果如何写回 generate loop。

在 serving 系统里，sampling 还有一个重要工程问题：

同一个 batch 里的不同请求，可能有不同 sampling params。

例如：

1. 请求 A 使用 greedy。
2. 请求 B 使用 temperature 0.8。
3. 请求 C 使用 top-p 0.9。

所以生产级 sampler 通常要支持 per-request sampling params。

如果 tiny-llm 的实现更简单，也可以把它作为改造练习。

45.12 第八阶段：KV Cache

KV cache 是从“模型推理”进入“推理系统”的分水岭。

没有 KV cache，decode 每一步都要重算历史 token。

有了 KV cache，流程变成：

prefill:

```
input full prompt
compute K/V for prompt
store K/V in cache
output next-token logits
```

decode:

```
input only last token
compute K/V for new token
append to cache
attend over cached K/V
output next-token logits
```

读 tiny-llm 的 KV cache 时，要重点看：

1. cache 的 shape。
2. 每层是否有独立 cache。
3. K 和 V 是否分开存。
4. prefill 如何写入 cache。
5. decode 如何 append cache。
6. current position 如何维护。
7. cache 最大长度如何限制。

这部分要和第 9 章、第 18 章、第 21 章联系起来。

KV cache 不是一个模型小优化，而是 serving engine 的核心资源。

45.13 第九阶段：量化 Matmul

tiny-llm 包含 quantized matmul and linear，这是系统工程师很值得看的部分。

LLM 推理里，大量时间花在矩阵乘：

1. Q/K/V projection。
2. Attention output projection。
3. MLP up/gate/down projection。
4. LM head。

量化的目标是降低：

1. 权重显存。
2. 内存带宽压力。
3. 部分计算成本。

读量化 matmul 时，重点看：

1. 权重如何存储。
2. scale 和 zero point 如何表示。
3. 反量化是在 matmul 前做，还是融合进 matmul。
4. CPU 和 GPU 实现差异。
5. 精度和速度如何测试。

要记住一个工程判断：

量化不是只看模型文件变小，还要看端到端 TTFT、TPOT、吞吐和质量是否可接受。

45.14 第十阶段：FlashAttention

FlashAttention 的核心价值是减少 attention 中间矩阵的内存读写。

普通 attention 可能显式形成：

attention scores: [batch, heads, query_len, key_len]

长上下文下，这个矩阵非常大。

FlashAttention 通过分块和在线 softmax，避免把完整 attention score materialize 到显存。

读 tiny-llm 的 FlashAttention 时，重点看：

1. 为什么要分 block。
2. softmax 的 max 和 sum 如何在线维护。
3. causal mask 如何处理。
4. CPU 和 GPU 实现差异。
5. prefill 和 decode 哪个更受益。

和 serving 的关系是：

prefill 阶段 query_len 大，FlashAttention 通常更关键；decode 阶段 query_len 通常为 1，瓶颈更多在读 KV 和小 batch 效率。

45.15 第十一阶段：Continuous Batching

Continuous batching 是从单请求 generate 到 serving engine 的关键一步。

Naive batching 是：

一批请求一起开始，一起生成到结束。

问题是不同请求输出长度不同，短请求会等待长请求，或者 batch 很快变空。

Continuous batching 是：

每一轮 decode 都重新组织 active requests，新请求可以在合适时机加入，完成请求可以立刻退出。

读 tiny-llm 的 continuous batching 时，重点看：

1. 请求状态如何表示。
2. waiting 和 running 如何维护。
3. 每轮 batch 如何构造。

4. prefill 和 decode 是否混合。
5. 新请求何时加入。
6. 完成请求何时移除。
7. KV cache 如何跟随请求状态变化。

这部分可以和第 19 章、第 20 章、第 44 章 nano-vLLM 对照。

45.16 第十二阶段：Chunked Prefill

Chunked prefill 是处理长 prompt 和 decode 抖动的重要技术。

长 prompt prefill 会占用较长计算时间。

如果它和 decode 混在同一个 engine 中，可能导致：

decode token 间隔变大，streaming 卡顿，TPOT p99 变差。

Chunked prefill 的思路是：

不要一次处理完整长 prompt，而是拆成多个 chunk，让 scheduler 可以在 chunk 之间插入 decode。

读 tiny-llm 的 chunked prefill 时，重点看：

1. prompt 如何切 chunk。
2. chunk 之间 position 如何连续。
3. KV cache 如何逐 chunk append。
4. scheduler 如何在 prefill chunk 和 decode 之间取舍。
5. chunk size 如何影响 TTFT、TPOT 和吞吐。

这部分可以直接连接第 39 章。

也是后续 PD 分离之前最值得尝试的优化之一。

45.17 第十三阶段：Paged Attention

Paged attention 是 vLLM 的核心思想之一。

tiny-llm 的第 3 周进入 paged attention，非常适合作为从 fixed KV cache 到 block-based KV cache 的过渡。

普通 KV cache 通常按 request 分配连续空间：

```
request A: [0 ... max_len]
request B: [0 ... max_len]
```

这会浪费显存，因为很多请求不会生成到 max_len。

Paged attention 把 KV cache 拆成固定大小 block：

logical tokens -> block table -> physical KV blocks

读 paged attention 时，重点看：

1. block size 如何选择。
2. logical token position 如何映射到 block id 和 offset。
3. block table 如何传给 attention。
4. decode 追加 token 时何时分配新 block。
5. 请求结束时如何释放 block。
6. attention kernel 如何按 block table 读取不连续 KV。

这部分和第 17、18、21 章完全对应。

45.18 tiny-llm 和本书章节映射

可以把 tiny-llm 当成本书前半部分的代码练习索引。

tiny-llm 主题	对应本书章节	学习重点
Attention	第 4、9、17 章	Q/K/V、causal mask、KV cache 入口
RoPE	第 4、9 章	position 和 KV cache 对齐
GQA	第 4、15、21 章	KV cache 显存和 decode 读带宽
RMSNorm/MLP	第 1、15 章	模型层计算成本
Generate	第 7 章	最小自回归循环
Sampling	第 8 章	logits 到 token
KV Cache	第 9、18、21 章	prefill/decode 分离基础
Quantized Matmul	第 15、25 章	显存、带宽、精度取舍
FlashAttention	第 15、17 章	attention IO 优化
Continuous Batching	第 19、20 章	iteration-level scheduling
Chunked Prefill	第 39 章	长 prompt 和 TPOT 抖动
Paged Attention	第 17、18、21 章	block-based KV 管理

这样读 tiny-llm 时，每个课程章节都能对应到本书的系统概念。

45.19 适合做的改造练习

tiny-llm 很适合做从底层到系统的改造练习。

练习一：画 shape trace。

对 attention、RoPE、GQA、MLP、lm head 画出每一步 tensor shape。

练习二：实现并对比 naive generate 和 KV cache generate。

记录每生成一个 token 的耗时，观察上下文变长后的差异。

练习三：给 generate loop 增加 TTFT 和 TPOT 统计。

区分 prompt prefill 时间和 decode token 时间。

练习四：实现 per-request sampling params。

让同一批请求支持不同 temperature、top-k、top-p。

练习五：对比不同量化 matmul 的速度和误差。

记录输出误差、perplexity 近似指标或生成质量差异。

练习六：给 continuous batching 增加 scheduler 日志。

每轮记录 waiting、running、batch size、prefill token 数和 decode request 数。

练习七：实现 chunked prefill 的不同 chunk size。

比较 TTFT、TPOT p95/p99 和吞吐。

练习八：实现简单 paged KV block manager。

支持 allocate、append、free 和 block table 查询。

练习九：模拟 KV cache OOM。

观察 scheduler 如何拒绝新请求或等待 finished 请求释放 KV。

练习十：写一份从 tiny-llm 到 vLLM 的差距分析。

列出缺失的 API server、streaming、block manager、worker/executor、metrics、分布式和故障恢复能力。

45.20 适合写进简历的项目描述

如果你基于 tiny-llm 做过实战，可以这样写：

基于 tiny-llm 从底层矩阵 API 实现 Qwen-like LLM 推理流程，完成 attention、RoPE、GQA、RMSNorm、MLP、模型加载、generate 和更偏系统优化可以写：

在 tiny-llm 基础上实现推理 serving 优化实验，包括量化 matmul、FlashAttention、KV cache、continuous batching 和 paged KV。面试时要强调：

1. 你不是只跑了 demo。
2. 你理解每个模型组件的 shape 和推理作用。
3. 你知道 KV cache 为什么改变复杂度。
4. 你做过 batching、chunked prefill 或 paged attention 相关实验。
5. 你能把 tiny-llm 的实现映射到 vLLM-like serving engine。

45.21 常见误区

误区一：只把 tiny-llm 当模型实现教程。

它确实从模型组件开始，但后半部分重点是 inference serving。

误区二：跳过 attention/RoPE/GQA，直接看 batching。

这样很容易不理解 KV cache 的 shape、position 和内存成本。

误区三：只看能不能生成文本。

生成文本只是第一步，更重要的是 TTFT、TPOT、吞吐、显存和 batch 行为。

误区四：把 MLX 细节当成唯一重点。

MLX 是实现环境，真正可迁移的是推理系统思想：KV cache、batching、attention 优化、block 管理。

误区五：认为 tiny-llm 可以替代生产框架。

它更适合学习和实验，不是完整生产 serving 系统。

45.22 面试官会怎么问

问题一：tiny-llm 适合学什么？

回答要点：适合从底层实现理解 LLM inference serving 的演进路径：先实现 attention、RoPE、GQA、RMSNorm、MLP、模型加载、generate 和 sampling，再加入 KV cache、量化 matmul、FlashAttention、continuous batching、chunked prefill 和 paged attention。

问题二：tiny-llm 和 nano-vLLM 有什么区别？

回答要点：nano-vLLM 更像一个已经成型的轻量 vLLM-like engine，重点看 engine、scheduler、sequence、block manager、model runner；tiny-llm 更课程化，从模型算子和单请求 generate 开始，逐步引入 KV cache、batching、chunked prefill、paged attention 等 serving 优化。

问题三：为什么 GQA 对推理重要？

回答要点：GQA 让多个 query heads 共享更少的 KV heads，降低 KV cache 大小和 decode 阶段读取 KV 的带宽压力。KV cache 大小与 num_kv_heads 成正比，所以 GQA 对长上下文和高并发 serving 很重要。

问题四：KV cache 如何改变 generate 的复杂度？

回答要点：没有 KV cache 时，每生成一个 token 都要重算完整上下文；有 KV cache 后，prefill 计算 prompt 并缓存 K/V，decode 每步只输入新 token，追加新 K/V，并 attend over cached K/V，从而避免重复计算历史 token。

问题五：chunked prefill 解决什么问题？

回答要点：长 prompt prefill 会阻塞 decode，导致 streaming TPOT 抖动。Chunked prefill 把长 prompt 拆成多个 chunk，让 scheduler 能在 chunk 之间插入 decode，改善 decode 尾延迟，但会引入 chunk size、position 连续性和 KV 逐步写入的复杂度。

45.23 标准回答模板

如果面试官问“你怎么通过 tiny-llm 学推理系统”，可以这样回答：

我会把 tiny-llm 当成从模型 forward 演进到 serving engine 的课程来读。第一阶段先实现 attention、RoPE、GQA、RMSNorm 和 MLP

第二阶段把这些组件组装成 Qwen-like decoder-only 模型，完成模型加载、generate loop 和 sampling。这里我会重点区分 prefill

第三阶段进入 serving 优化，引入 KV cache，让 prefill 写入 prompt K/V，decode 每步只计算新 token 的 K/V 并追加到 cache。然

相比 nano-vLLM，tiny-llm 更适合理解这些能力是怎么从底层一步步长出来的；nano-vLLM 更适合看一个 vLLM-like engine 的模块边界。两个结合起来，一个解释底层计算和优化动机，一个解释 engine 架构。

如果做项目，我会基于 tiny-llm 增加 TTFT/TPOT 统计、scheduler 日志、KV cache 使用统计，比较 naive generate、KV cache deco

45.24 tiny-llm 学习路线覆盖率、实验门禁和可运行 demo

先把 tiny-llm 的学习单元抽象成：

$U_j = (p_j, C_j, E_j, D_j)$

其中 p_j 是阶段， C_j 是概念集合， E_j 是实验集合， D_j 是依赖的前置单元。

概念覆盖率：

$C_{\mathrm{concept}} = \frac{|C_{\mathrm{seen}} \cap C_{\mathrm{req}}|}{\max(1, |C_{\mathrm{req}}|)}$

实验覆盖率：

$C_{\mathrm{experiment}} = \frac{|E_{\mathrm{done}} \cap E_{\mathrm{req}}|}{\max(1, |E_{\mathrm{req}}|)}$

最终门禁：

$G_{\mathrm{tiny}} = G_{\mathrm{op}} G_{\mathrm{gen}} G_{\mathrm{kv}} G_{\mathrm{serve}} G_{\mathrm{metric}} G_{\mathrm{order}}$

这组公式强调：tiny-llm 不是只跑通一个文本生成 demo，而是要按顺序证明：

1. 算子到模型 forward 的 shape 路径正确。
2. generate 和 sampling 能形成最小自回归闭环。
3. KV cache 与 naive generate 的等价性可验证。
4. continuous batching、chunked prefill、paged attention 等 serving 优化能被实验观察。
5. TTFT、TPOT、batching trace、chunk position 和 block table 都能输出证据。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class LearningUnit:
    name: str
    phase: str
    concepts: tuple
    experiments: tuple
    dependencies: tuple
```

```
class ToyTinyLLMLearningAuditor:
    def __init__(self):
```

```

self.required_concepts = [
    "attention_shape",
    "rope_position",
    "gqa_kv_heads",
    "rmsnorm_mlp",
    "model_load",
    "generate_loop",
    "sampling_policy",
    "kv_cache",
    "quantized_matmul",
    "flash_attention",
    "continuous_batching",
    "chunked_prefill",
    "paged_attention",
]
self.required_experiments = [
    "shape_trace",
    "position_alignment",
    "generate_trace",
    "sampling_params",
    "kv_cache_equivalence",
    "ttft_tpot_metrics",
    "batching_trace",
    "chunk_position_continuity",
    "block_table_trace",
]

def coverage(self, seen, required):
    return round(len(set(seen) & set(required)) / max(1, len(required)), 3)

def audit(self, units):
    completed = set()
    rows = []
    for unit in units:
        deps_ready = all(dep in completed for dep in unit.dependencies)
        row = {
            "unit": unit.name,
            "phase": unit.phase,
            "concepts": list(unit.concepts),
            "experiments": list(unit.experiments),
            "deps_ready": deps_ready,
        }
        rows.append(row)
        if deps_ready:
            completed.add(unit.name)
    concepts_seen = sorted({concept for unit in units for concept in unit.concepts})
    experiments_done = sorted({experiment for unit in units for experiment in unit.experiments})
    summary = {
        "units": len(rows),
        "ready_units": sum(row["deps_ready"] for row in rows),
        "concept_coverage": self.coverage(concepts_seen, self.required_concepts),
        "experiment_coverage": self.coverage(experiments_done, self.required_experiments),
        "dependency_order_ok": all(row["deps_ready"] for row in rows),
        "serving_units": sum(unit.phase == "serving_optimization" for unit in units),
        "missing_concepts": sorted(set(self.required_concepts) - set(concepts_seen)),
    }

```

```

    "missing_experiments": sorted(set(self.required_experiments) - set(experiments_done)),
}
gates = {
    "operator_to_model_ready": all(
        concept in concepts_seen
        for concept in [
            "attention_shape",
            "rope_position",
            "gqa_kv_heads",
            "rmsnorm_mlp",
            "model_load",
        ]
    ),
    "generate_and_sampling_ready": all(
        concept in concepts_seen for concept in ["generate_loop", "sampling_policy"]
    ),
    "kv_cache_equivalence_ready": (
        "kv_cache" in concepts_seen and "kv_cache_equivalence" in experiments_done
    ),
    "serving_optimizations_visible": all(
        concept in concepts_seen
        for concept in [
            "flash_attention",
            "continuous_batching",
            "chunked_prefill",
            "paged_attention",
        ]
    ),
    "metrics_experiments_ready": all(
        experiment in experiments_done
        for experiment in ["tftft_tpot_metrics", "batching_trace", "block_table_trace"]
    ),
    "dependency_order_ok": summary["dependency_order_ok"],
}
gates["tiny_llm_learning_gate"] = all(gates.values())
return rows, summary, gates

```

```

units = [
    LearningUnit("attention", "model_kernel", ("attention_shape",), ("shape_trace",), ()),
    LearningUnit("rope", "model_kernel", ("rope_position",), ("position_alignment",), ("attention",)),
    LearningUnit("gqa", "model_kernel", ("gqa_kv_heads",), ("shape_trace",), ("rope",)),
    LearningUnit("rmsnorm_mlp", "model_kernel", ("rmsnorm_mlp",), ("shape_trace",), ("gqa",)),
    LearningUnit("load_qwen", "model_forward", ("model_load",), ("shape_trace",), ("rmsnorm_mlp",)),
    LearningUnit(
        "generate",
        "single_request",
        ("generate_loop",),
        ("generate_trace", "tftft_tpot_metrics"),
        ("load_qwen",),
    ),
    LearningUnit("sampling", "single_request", ("sampling_policy",), ("sampling_params",), ("generate",)),
    LearningUnit(
        "kv_cache",
        "serving_optimization",
    ),
]

```

```

        ("kv_cache",),
        ("kv_cache_equivalence", "ttft_tpot_metrics"),
        ("sampling",),
    ),
    LearningUnit(
        "quantized_matmul",
        "serving_optimization",
        ("quantized_matmul",),
        ("ttft_tpot_metrics",),
        ("load_qwen",),
    ),
    LearningUnit(
        "flash_attention",
        "serving_optimization",
        ("flash_attention",),
        ("ttft_tpot_metrics",),
        ("attention",),
    ),
    LearningUnit(
        "continuous_batching",
        "serving_optimization",
        ("continuous_batching",),
        ("batching_trace",),
        ("kv_cache",),
    ),
    LearningUnit(
        "chunked_prefill",
        "serving_optimization",
        ("chunked_prefill",),
        ("chunk_position_continuity", "ttft_tpot_metrics"),
        ("continuous_batching",),
    ),
    LearningUnit(
        "paged_attention",
        "serving_optimization",
        ("paged_attention",),
        ("block_table_trace",),
        ("chunked_prefill",),
    ),
]
rows, summary, gates = ToyTinyLLMLearningAuditor().audit(units)
print("tiny_llm_learning_summary=", summary)
print("tiny_llm_learning_gates=", gates)

```

一次运行的核心输出类似：

```

tiny_llm_learning_summary= {'units': 13, 'ready_units': 13, 'concept_coverage': 1.0, 'experiment_coverage': 1.0, 'depend
tiny_llm_learning_gates= {'operator_to_model_ready': True, 'generate_and_sampling_ready': True, 'kv_cache_equivalence_

```

这个 demo 的重点是学习顺序：

1. attention -> rope -> gqa -> rmsnorm_mlp -> load_qwen 证明模型 forward 的 shape 和 position 基础。
2. generate -> sampling 证明最小自回归生成闭环。
3. kv_cache 证明 serving 优化从避免重复计算开始。
4. quantized_matmul 和 flash_attention 证明算子层优化。
5. continuous_batching -> chunked_prefill -> paged_attention 证明多请求调度和 KV 显存治理。

所以本章最终门禁是 tiny_llm_learning_gate：只有算子、模型、generate、sampling、KV cache、serving 优

化、指标实验和依赖顺序都能闭环，tiny-llm 才不是“跑了课程 demo”，而是成为理解推理系统演进路线的证据。

45.25 小练习

1. 画出 tiny-llm 中 attention、RoPE、GQA 的 tensor shape。
2. 推导 GQA 对 KV cache 大小的影响。
3. 对比 naive generate 和 KV cache generate 的计算流程。
4. 给 generate loop 增加 TTFT 和 TPOT 统计。
5. 解释 prefill 和 decode 在输入 shape、计算量和 KV 写入上的差异。
6. 实现 per-request sampling params，并说明 batch sampler 如何处理不同参数。
7. 对比量化 matmul 前后的显存、速度和生成质量。
8. 解释 FlashAttention 为什么更适合长 prefill。
9. 设计一个 continuous batching 的状态机。
10. 实验不同 chunk size 对 TTFT、TPOT 和吞吐的影响。
11. 设计一个最小 paged KV block manager。
12. 写一个面试回答：tiny-llm 和 nano-vLLM 应该如何结合学习？

45.26 本章总结

tiny-llm 是一个从底层模型实现走向 LLM serving 的课程型项目。

它的价值在于把推理系统拆成一条清晰演进线：先实现 attention、RoPE、GQA、RMSNorm、MLP、模型加载、generate 和 sampling，再引入 KV cache、量化 matmul、FlashAttention、continuous batching、chunked prefill 和 paged attention。

读 tiny-llm 时，不要只关注“能不能生成文本”，而要关注每一步为什么会成为 serving 优化的前置条件：GQA 影响 KV cache 大小，RoPE 影响 position 对齐，KV cache 区分 prefill/decode，continuous batching 改变请求调度，chunked prefill 缓解长 prompt 阻塞，paged attention 解决 KV 显存管理。

nano-vLLM 和 tiny-llm 应该结合起来学：tiny-llm 帮你理解底层计算和优化动机，nano-vLLM 帮你理解 vLLM-like engine 的模块边界。

下一章会进入 mini-sglang 学习路线，重点会从 vLLM-like engine 转向 SGLang 的 runtime、prefix sharing、structured generation 和 agent/tool serving 支持。

第 46 章 mini-sglang 学习路线

前两章分别讲了 nano-vLLM 和 tiny-llm。

nano-vLLM 更适合学习 vLLM-like engine 的骨架：Sequence、Scheduler、BlockManager、ModelRunner、attention、sampler。

tiny-llm 更适合从 attention、RoPE、GQA、generate、KV cache、continuous batching、chunked prefill、paged attention 这条线理解推理系统如何从模型 forward 演进出来。

这一章看 mini-sglang。

本书第 34 章已经讲过 mini-sglang 的源码学习路径，重点是如何沿请求生命周期读源码。本章不重复那条源码阅读清单，而是把 mini-sglang 放在第六部分的教学项目序列里，回答一个更系统的问题：

读完 nano-vLLM 和 tiny-llm 后，为什么还需要读 mini-sglang？它能补上哪一块能力？

一句话概括：

mini-sglang 的学习价值在于把你从“通用 vLLM-like serving engine”带到“SGLang-style runtime”：重点理解 Radix Cache、prefix sharing、overlap scheduling、online serving、structured generation、tool/agent serving，以及这些能力如何围绕复杂 LLM program 的执行效率展开。

46.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 `WRITING_PLAN.md` 对公开资料做校准：参考 `sgl-project/mini-sglang` 官方仓库和 `README` 对 `mini-sglang` 作为约 5000 行 Python 紧凑 SGLang 实现、支持 Radix Cache、Chunked Prefill、Overlap Scheduling、Tensor Parallelism、FlashAttention / FlashInfer backend、OpenAI-compatible API、interactive shell、offline / online benchmark 的说明；参考仓库文档对 API server、request / tokenizer、scheduler、engine、KV cache、Radix cache、attention backend、message / protocol 等模块结构的说明；并结合第 26 到 34 章对 SGLang runtime、RadixAttention、scheduler、structured generation、speculative decoding、agent serving、SGLang vs vLLM 和 `mini-sglang` 源码路径的口径。

本讲只讲“为什么在 `tiny-llm` 和 `nano-vLLM` 之后继续读 `mini-sglang`，以及读出哪些 SGLang runtime 特色”：Radix Cache、prefix sharing、chunked prefill、overlap scheduling、online serving、interactive shell、TP、kernel backend、structured generation 边界、tool / agent serving 边界和完整 SGLang 差距。不把某个 `mini-sglang` commit 的源码行号、真实启动参数、benchmark 数字、FlashInfer / FlashAttention 具体 kernel 行为、完整 SGLang 的 grammar / speculative / distributed 实现或生产部署能力写成通用结论。

本讲新增 demo 是教学版 `mini-sglang learning auditor`：用 0 依赖 Python 模拟 11 个 runtime capability，从 request state、Radix Cache、chunked prefill、overlap scheduling、online serving、interactive shell、TP、kernel backend，一路到 structured generation / tool-agent 边界和 production gap map；检查 capability coverage、experiment coverage、dependency order、runtime-specific evidence 和最终学习门禁。

46.1 本章目标

读完本章，你应该能讲清：

1. `mini-sglang` 和 `nano-vLLM`、`tiny-llm` 的学习重点有什么不同。
2. 为什么 `mini-sglang` 是理解 SGLang 的过渡项目。
3. 应该重点看 `mini-sglang` 的哪些能力，而不是重复读 `vLLM-like` 基础模块。
4. Radix Cache 和普通 prefix cache 的区别。
5. Overlap scheduling 解决什么问题。
6. OpenAI-compatible online serving、shell、offline benchmark 分别适合验证什么。
7. `mini-sglang` 和完整 SGLang 的差距在哪里。
8. 如何基于 `mini-sglang` 做一个有辨识度的推理系统项目。

46.2 三个教学项目的分工

可以把 `nano-vLLM`、`tiny-llm`、`mini-sglang` 看成三块拼图。

项目	重点	最适合回答的问题
<code>tiny-llm</code>	从算子到 serving 优化	KV cache、FlashAttention、continuous batching 为什么出现
<code>nano-vLLM</code>	<code>vLLM-like</code> engine 骨架	一个轻量 serving engine 如何组织请求和 KV block
<code>mini-sglang</code>	SGLang-style runtime	复杂 LLM program、prefix sharing、结构化输出和在线服务如何高效执行

如果只读 `tiny-llm`，可能会偏底层算子和单机推理。

如果只读 `nano-vLLM`，可能会理解 engine 骨架，但对 SGLang 的 prefix/program runtime 特色不够敏感。

如果只读 `mini-sglang`，可能会看到很多 runtime 能力，但缺少前面两个项目提供的基础坐标系。

推荐顺序是：

`tiny-llm` -> `nano-vLLM` -> `mini-sglang`

对应认知升级：

模型如何算

- > engine 如何调度
- > runtime 如何执行复杂 LLM program

46.3 mini-sglang 的核心特征

mini-sglang 是 SGLang 的紧凑实现，目标是用更小代码解释现代 LLM serving 系统的复杂性。

从公开介绍看，它的核心能力包括：

1. Radix Cache。
2. Chunked Prefill。
3. Overlap Scheduling。
4. Tensor Parallelism。
5. FlashAttention / FlashInfer 等优化 kernel。
6. OpenAI-compatible online serving。
7. Interactive shell。
8. Offline / online benchmark。

这些能力说明它已经不只是教学级 generate loop，而是一个更接近真实 serving runtime 的项目。

学习时应该抓住一个主线：

mini-sglang 如何在复杂请求、共享前缀、长上下文和在线服务场景下，尽量复用已有计算并稳定推进 decode？

46.4 和第 34 章的关系

第 34 章已经给过源码阅读路径：

入口 API

- > 请求状态
- > scheduler
- > KV cache
- > RadixAttention / prefix sharing
- > model runner
- > sampler
- > structured output / streaming

本章的角度不同。

本章更关注：

1. 读 mini-sglang 前你应该已经从 tiny-llm、nano-vLLM 得到什么基础。
2. 读 mini-sglang 时应该特别强化哪些 SGLang 特有概念。
3. 如何把它变成一个可展示的系统项目。
4. 如何从 mini-sglang 过渡到完整 SGLang。

也就是说，第 34 章是“怎么读源码”，本章是“为什么读、读出什么、怎么用它做项目”。

46.5 先带着 SGLang 问题读

读 mini-sglang 前，先回忆 SGLang 要解决的问题。

vLLM-like engine 主要关注：

大量独立请求如何高吞吐、低延迟地生成。

SGLang 进一步关注：

复杂 LLM programs 如何高效执行。

复杂 LLM programs 可能包括：

1. 多轮对话。
2. few-shot prompt。
3. 多分支生成。
4. structured output。
5. tool calling。
6. agent loop。
7. RAG prompt assembly。
8. 多请求共享长前缀。

这些场景有一个共同点：

很多请求之间存在可复用前缀，很多请求内部存在结构化执行状态。

所以读 mini-sglang 时，不要只问它是不是有 scheduler 和 KV cache。

要问：

它如何发现和复用 prefix？

它如何把复杂请求映射成 runtime 可调度的状态？

它如何让 decode 稳定推进，同时隐藏 CPU 调度开销？

46.6 第一重点：Radix Cache

Radix Cache 是 mini-sglang 最值得重点看的能力。

普通 prefix cache 可以理解成：

hash(prefix tokens) -> cached KV

如果完整 prefix 命中，就复用。

Radix Cache 更像一棵前缀树：

root

-> shared prefix A

-> branch B

-> branch C

它适合处理部分共享前缀。

例如三个请求：

R1: system prompt + tool spec + question A

R2: system prompt + tool spec + question B

R3: system prompt + tool spec + question C

它们共享：

system prompt + tool spec

Radix Cache 的目标是让这段共享前缀的 KV 不要重复 prefill。

读 Radix Cache 时，重点看七个操作：

1. prefix match。
2. longest prefix lookup。
3. node split。
4. insert。
5. pin / unpin。
6. eviction。
7. cached KV 和 request KV 的 ownership。

最重要的问题是 ownership：

命中的 cached KV 是被当前 request 独占，还是多个 request 共享？
当前 request 结束时，哪些 KV 可以释放，哪些要留在 radix cache？
如果这个问题没想清楚，很容易写出 KV 泄漏或错误释放。

46.7 Radix Cache 和 PagedAttention 的关系

很多人会混淆 Radix Cache 和 PagedAttention。

它们解决的问题不同。

PagedAttention 解决：

一个请求的 KV 如何用不连续 block 管理，减少显存浪费和碎片。

Radix Cache 解决：

多个请求之间的共享前缀 KV 如何复用，减少重复 prefill。

可以这样理解：

PagedAttention: logical positions -> physical blocks

Radix Cache: prefix token path -> reusable KV blocks

两者可以组合：

Radix tree node 持有一段 prefix 对应的 KV blocks；request 命中 prefix 后，把这些 blocks 接到自己的 block table 或 KV reference。

面试中常见错误是说：

有了 PagedAttention 就不需要 prefix cache。

这是错的。

PagedAttention 主要是内存管理方式，prefix/Radix Cache 是计算复用策略。

46.8 第二重点：Chunked Prefill

mini-sglang 支持 chunked prefill，这和前面第 39 章、tiny-llm 第 2 周内容形成呼应。

在 SGLang 场景中，chunked prefill 更重要，因为 prompt 可能非常长：

1. system prompt。
2. few-shot examples。
3. tool definitions。
4. RAG retrieved chunks。
5. conversation history。
6. schema / grammar spec。

如果一次性 prefill，decode 会被长时间阻塞。

Chunked prefill 的关键问题是：

1. chunk size 如何定。
2. chunk 之间是否允许 decode 插队。
3. prefix cache 命中后剩余 suffix 如何 chunk。
4. position 如何连续。
5. KV 如何逐 chunk 写入。
6. 如果请求取消，已经写入的 partial KV 如何清理。

读 mini-sglang 时，要把 chunked prefill 和 Radix Cache 一起看：

先命中共享 prefix，再对未命中的 suffix 做 chunked prefill。

这才是长 prompt serving 的真实路径。

46.9 第三重点：Overlap Scheduling

mini-sglang 的一个重要能力是 overlap scheduling。

它要解决的是：

CPU 侧调度、请求处理、metadata 构造，不要阻塞 GPU 计算。

一个 naive engine 可能是：

CPU schedule

- > prepare batch metadata
- > GPU forward
- > CPU update states
- > next schedule

这会导致 GPU 在 CPU 调度期间空等。

Overlap scheduling 的思路是：

GPU 正在执行 batch N 时，CPU 准备 batch N+1 的调度和 metadata。

简化表示：

time --->

CPU:	schedule N	schedule N+1	schedule N+2
GPU:	forward N	forward N+1	forward N+2

读 overlap scheduling 时，重点看：

1. 哪些工作在 CPU 侧。
2. 哪些工作在 GPU 侧。
3. 两者之间如何同步。
4. 请求状态何时可见。
5. batch metadata 是否可以提前构造。
6. 取消、abort、完成请求如何处理。
7. 关闭 overlap 后性能如何变化。

这个能力很适合做实验：

开启 overlap scheduling vs 关闭 overlap scheduling
比较吞吐、TPOT 和 GPU utilization

46.10 第四重点：Online Serving

nano-vLLM 和 tiny-llm 更偏 offline 或教学执行路径。

mini-sglang 更值得看的一个点是 online serving。

Online serving 和 offline inference 的差异是：

offline：一批请求已经给定，系统尽快处理完

online：请求持续到达，要处理排队、streaming、取消、超时、负载和错误

读 online serving 时，重点看：

1. OpenAI-compatible API 如何接入。
2. chat messages 如何转 prompt。
3. sampling params 如何解析。
4. streaming response 如何返回。
5. 请求如何进入 runtime 队列。
6. 客户端断开如何 abort。
7. 错误如何返回。

这部分能补上教学项目常缺的服务化视角。

如果你想做简历项目，online serving 比单纯 offline generate 更有说服力。

46.11 第五重点：Interactive Shell

mini-sglang 支持 interactive shell。

Shell 看起来只是一个 demo，但它能帮助你观察多轮对话状态。

重点看：

1. 多轮 history 如何保存。
2. `/reset` 如何清理状态。
3. 每轮 prompt 如何拼接。
4. 多轮共享前缀是否能命中 Radix Cache。
5. streaming token 如何打印。
6. 用户中断如何处理。

多轮对话是 prefix sharing 的天然场景。

例如：

```
turn 1: system + user1
turn 2: system + user1 + assistant1 + user2
turn 3: system + user1 + assistant1 + user2 + assistant2 + user3
```

后续轮次包含前面轮次的大量 prefix。

这正是 Radix Cache 可以发挥作用的地方。

46.12 第六重点：Tensor Parallelism

mini-sglang 支持 tensor parallel。

这让它比很多教学项目更接近生产 serving。

读 TP 时，不需要一开始看所有通信细节。

先看：

1. `--tp` 参数如何进入配置。
2. worker 或 model runner 如何知道自己的 rank。
3. 权重如何 shard。
4. linear 层如何切分。
5. attention heads 如何分配。
6. forward 中哪些地方需要通信。

然后再连接第 24 章和第 41 章：

TP 可以承载更大模型，但 decode 阶段每层通信会进入 TPOT 关键路径；跨节点 TP 尤其要谨慎。

mini-sglang 的 TP 适合用来理解生产系统里的模型并行入口。

46.13 第七重点：Kernel Backend

mini-sglang 集成 FlashAttention、FlashInfer 等优化 kernel。

读 kernel backend 时，不要陷入每个 CUDA kernel 的实现细节。

先看接口边界：

1. runtime 如何选择 attention backend。
2. prefill 和 decode 是否使用不同 kernel。

3. attention metadata 如何传入 kernel。
4. KV layout 对 kernel 有什么要求。
5. fallback 路径是什么。

Serving engine 的关键不是 “会写所有 kernel”，而是知道：

scheduler 和 KV manager 产出的 metadata 必须满足 kernel backend 的输入约束。

如果 metadata 错了，kernel 再快也没用。

46.14 第八重点：Structured Generation

完整 SGLang 很强调 structured generation 和 constrained decoding。

mini-sglang 是否完整覆盖所有 grammar 能力，会随版本变化。

学习时应该关注接口边界，而不是强行要求教学项目覆盖全部能力。

结构化生成的核心链路是：

```
schema / regex / grammar
-> grammar state
-> valid token mask
-> sampler
-> update grammar state
```

读 mini-sglang 时，重点问：

1. 请求中是否能携带结构化约束。
2. 约束在哪里解析。
3. 每个 request 的 grammar state 存在哪里。
4. valid token mask 是否进入 sampler。
5. streaming 时如何保证结构合法。
6. 约束失败如何处理。

如果 mini-sglang 没有完整实现，也可以把它作为设计练习：

如何给 mini-sglang 增加一个最小 regex constrained decoding？

46.15 第九重点：Tool / Agent Serving

SGLang 的另一个价值是更适合复杂 LLM program。

Tool / Agent serving 不是简单多一个 HTTP API。

它会影响 runtime：

1. Prompt 里有 tool schema。
2. 输出可能是 tool call JSON。
3. Streaming 时 tool arguments 需要增量拼接。
4. Tool result 会回灌成下一轮上下文。
5. 多轮 agent loop 会产生大量共享 prefix。
6. structured generation 可能约束 tool call 格式。

读 mini-sglang 时，可以关注：

1. chat template 是否支持 tools。
2. tool schema 如何拼进 prompt。
3. 输出 parser 是否和 runtime 解耦。
4. tool call 是否只是解析，不负责真实执行。
5. agent loop 是否留给应用层。

真实工具执行通常不应该放进推理引擎核心，因为它涉及权限、网络、超时、安全和审计。
推理 runtime 更适合负责：
生成和解析 tool call，并高效维护上下文。

46.16 mini-sglang 和完整 SGLang 的差距

mini-sglang 是教学项目，不是完整生产系统。
它可能省略或简化：

- 1. 更多模型架构。
- 2. 更多硬件后端。
- 3. 更完整的分布式部署。
- 4. 更复杂的 grammar backend。
- 5. 更完整的 speculative decoding。
- 6. 更复杂的 tool parser。
- 7. 多租户隔离。
- 8. 完整 metrics、tracing、dashboard。
- 9. 故障恢复和自动扩缩容。
- 10. 更复杂的 PD 分离和跨节点 serving。

但它保留了学习 SGLang 的关键骨架：
runtime + scheduler + KV cache + Radix Cache + model runner + sampler + online serving
这就是它的价值。

46.17 和生产 SGLang 源码怎么衔接

读完 mini-sglang 后，不要直接在完整 SGLang 里乱搜。
应该带着映射去读。
映射方式：

mini-sglang 认知	完整 SGLang 中要找的能力
request state	更完整的请求对象、会话状态、abort 状态
scheduler	更复杂的调度策略、token budget、prefill/decode 策略
Radix Cache	更完整的 prefix cache、eviction、ref count
model runner	多模型、多后端、CUDA graph、attention backend
sampler	penalties、logprobs、grammar mask、speculative verify
online serving	OpenAI API、streaming、metrics、错误处理
TP	分布式 worker、通信、rank 管理

这样读完整 SGLang 时，每个复杂模块都有锚点。

46.18 适合做的改造练习

mini-sglang 的练习应该围绕 SGLang 特色展开。
练习一：Radix Cache 可视化。
构造多个共享 system prompt 的请求，打印 radix tree split、insert、match、evict。
练习二：prefix 命中率统计。
统计请求的 prefix hit length、hit ratio、节省 prefill token 数。

练习三：chunked prefill ablation。

对比不同 chunk size 下 TTFT、TPOT p99、吞吐和 GPU utilization。

练习四：overlap scheduling ablation。

关闭 overlap scheduling，对比吞吐和 GPU 空闲时间。

练习五：online serving 压测。

用持续到达的请求测 TTFT、TPOT、QPS、错误率和 streaming 平滑度。

练习六：多轮对话 prefix reuse。

用 shell 或 chat API 构造多轮对话，观察后续轮次是否命中历史 prefix。

练习七：最小 structured decoding。

实现一个简单 digit-only 或 JSON-like token mask，接入 sampler。

练习八：tool call parser。

让模型输出一个简化 tool call JSON，并实现 streaming parser。

练习九：KV cache 泄漏测试。

构造 abort、timeout、client disconnect，检查 KV blocks 和 radix cache pin 是否释放。

练习十：mini-sglang 到完整 SGLang 对照报告。

列出 mini-sglang 简化了哪些生产能力，以及完整 SGLang 如何补齐。

46.19 适合写进简历的项目描述

如果你基于 mini-sglang 做过改造，可以这样写：

基于 mini-sglang 阅读和改造 SGLang-style LLM serving runtime，重点分析 scheduler、KV cache、Radix Cache、chunked prefill

如果更偏 structured generation / agent：

在 mini-sglang 基础上实现简化 structured decoding 和 tool call parser，将 grammar/token mask 接入 sampler，并支持流式解析

这比单纯写“读过 SGLang 源码”更有说服力。

因为它说明你理解：

1. SGLang 的特色在哪里。
2. 你做过可观测指标。
3. 你做过 ablation。
4. 你理解 prefix、scheduler、KV、sampler 之间的关系。

46.20 常见误区

误区一：把 mini-sglang 当成另一个 vLLM clone。

它确实有通用 serving engine 能力，但学习重点应该放在 SGLang runtime 特色上。

误区二：只看 benchmark，不看请求生命周期。

性能数字没有请求状态、cache 命中、调度策略支撑，很难解释。

误区三：把 Radix Cache 理解成普通 hash cache。

Radix Cache 的关键是部分前缀匹配、tree split、节点复用和 eviction。

误区四：忽略 overlap scheduling。

很多 serving 性能问题不只在 GPU kernel，也在 CPU 调度和 metadata 准备是否阻塞 GPU。

误区五：把 tool execution 放进 engine 核心。

推理 runtime 可以解析和生成 tool call, 但真实工具执行通常属于应用层或 agent runtime。

误区六: 忘记和完整 SGLang 对照。

mini-sglang 是学习桥梁, 不是终点。

46.21 面试官会怎么问

问题一: mini-sglang 相比 nano-vLLM 和 tiny-llm, 最值得学什么?

回答要点: tiny-llm 适理解底层算子和 serving 优化动机, nano-vLLM 适理解 vLLM-like engine 骨架, mini-sglang 适理解 SGLang-style runtime, 重点是 Radix Cache、prefix sharing、chunked prefill、overlap scheduling、online serving、structured generation 和 tool/agent serving。

问题二: Radix Cache 和普通 prefix cache 有什么不同?

回答要点: 普通 prefix cache 更像完整 prefix hash 命中; Radix Cache 用前缀树支持最长前缀匹配、部分共享、节点 split/insert/evict。它适合 system prompt、tool schema、few-shot、多轮对话等大量共享前缀场景。

问题三: Overlap scheduling 解决什么问题?

回答要点: 它让 CPU 侧调度和 batch metadata 准备与 GPU forward 重叠, 避免 GPU 等 CPU。GPU 执行 batch N 时, CPU 可以准备 batch N+1, 从而提升吞吐并降低调度开销对 TPOT 的影响。

问题四: mini-sglang 为什么适合学习在线 serving?

回答要点: 它支持 OpenAI-compatible server、interactive shell、offline/online benchmark, 可以观察持续请求到达、streaming、取消、错误、排队、TTFT/TPOT、prefix cache 命中和 scheduler 行为, 比只跑 offline generate 更接近真实服务。

问题五: mini-sglang 和完整 SGLang 的关系是什么?

回答要点: mini-sglang 是紧凑教学实现, 保留 runtime、scheduler、KV cache、Radix Cache、model runner、sampler、online serving 等骨架; 完整 SGLang 有更多模型、后端、分布式、grammar、speculative decoding、tool parser、metrics、故障恢复和生产工程能力。

46.22 标准回答模板

如果面试官问 “你怎么通过 mini-sglang 学 SGLang”, 可以这样回答:

我会把 mini-sglang 放在 tiny-llm 和 nano-vLLM 之后读。tiny-llm 帮我理解 attention、RoPE、GQA、KV cache、FlashAttention、vLLM 帮我理解一个 vLLM-like engine 的 sequence、scheduler、block manager、model runner 骨架; mini-sglang 则用来理解 SGLang-style runtime 的特色。

读 mini-sglang 时, 我不会只看它有没有普通 scheduler 和 KV cache, 而会重点看 Radix Cache、chunked prefill、overlap scheduling、多轮对话这类共享前缀场景; chunked prefill 处理长 prompt, 避免 prefill 长时间阻塞 decode; overlap scheduling 让 CPU

我会做几个实验验证理解: 构造共享 system prompt 的请求观察 radix tree split 和 prefix hit length; 关闭 overlap scheduling

mini-sglang 不是完整生产 SGLang, 它会简化更多模型、后端、分布式、grammar、speculative decoding、metrics 和故障恢复能力

46.23 mini-sglang runtime 学习覆盖率、实验门禁和可运行 demo

先把 mini-sglang 的 runtime capability 抽象成:

$R_j = (c_j, E_j, X_j, D_j)$

其中 c_j 是能力类别, E_j 是证据集合, X_j 是实验集合, D_j 是依赖的前置能力。

能力覆盖率:

$$C_{\{\mathrm{cap}\}} = \frac{|R_{\{\mathrm{seen}\}} \cap R_{\{\mathrm{req}\}}|}{|\max(1, |R_{\{\mathrm{req}\}}|)|}$$

实验覆盖率:

$$C_{\{\mathrm{exp}\}} = \frac{|X_{\{\mathrm{done}\}} \cap X_{\{\mathrm{req}\}}|}{|\max(1, |X_{\{\mathrm{req}\}}|)|}$$

最终门禁:

$$G_{\{\mathrm{minisg}\}} = G_{\{\mathrm{radix}\}} G_{\{\mathrm{chunk}\}} G_{\{\mathrm{overlap}\}} G_{\{\mathrm{online}\}} G_{\{\mathrm{struct}\}} G_{\{\mathrm{shell}\}}$$

这组公式强调: 读 mini-sglang 不是重复 “有 scheduler、有 KV cache” 的通用 serving engine 学习, 而是要证明 SGLang-style runtime 的特色能被实验观察。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class RuntimeCapability:
```

```
    name: str
```

```
    category: str
```

```
    evidence: tuple
```

```
    experiments: tuple
```

```
    depends_on: tuple
```

```
class ToyMiniSGLangLearningAuditor:
```

```
    def __init__(self):
```

```
        self.required_capabilities = [
```

```
            "runtime_request_state",
```

```
            "radix_cache",
```

```
            "chunked_prefill",
```

```
            "overlap_scheduling",
```

```
            "online_serving",
```

```
            "interactive_shell",
```

```
            "tensor_parallel",
```

```
            "kernel_backend",
```

```
            "structured_generation_boundary",
```

```
            "tool_agent_boundary",
```

```
            "production_gap_map",
```

```
        ]
```

```
        self.required_experiments = [
```

```
            "shared_prefix_radix_split",
```

```
            "prefix_hit_metrics",
```

```
            "chunk_size_ablation",
```

```
            "overlap_ablation",
```

```
            "online_streaming_trace",
```

```
            "multi_turn_shell_reuse",
```

```
            "structured_mask_design",
```

```
            "tool_call_parser_trace",
```

```
            "abort_cleanup_check",
```

```
            "sglang_gap_report",
```

```
        ]
```

```
    def coverage(self, seen, required):
```

```
        return round(len(set(seen) & set(required)) / max(1, len(required)), 3)
```

```
    def audit(self, capabilities):
```

```
        completed = set()
```

```
        rows = []
```

```
        for capability in capabilities:
```

```

deps_ready = all(dep in completed for dep in capability.depends_on)
row = {
    "capability": capability.name,
    "category": capability.category,
    "evidence": list(capability.evidence),
    "experiments": list(capability.experiments),
    "deps_ready": deps_ready,
}
rows.append(row)
if deps_ready:
    completed.add(capability.name)
names = [capability.name for capability in capabilities]
evidence = sorted({item for capability in capabilities for item in capability.evidence})
experiments = sorted({item for capability in capabilities for item in capability.experiments})
summary = {
    "capabilities": len(rows),
    "ready_capabilities": sum(row["deps_ready"] for row in rows),
    "capability_coverage": self.coverage(names, self.required_capabilities),
    "experiment_coverage": self.coverage(experiments, self.required_experiments),
    "evidence_count": len(evidence),
    "runtime_specific": sum(
        capability.category == "sglang_runtime" for capability in capabilities
    ),
    "dependency_order_ok": all(row["deps_ready"] for row in rows),
    "missing_capabilities": sorted(set(self.required_capabilities) - set(names)),
    "missing_experiments": sorted(set(self.required_experiments) - set(experiments)),
}
gates = {
    "radix_cache_visible": (
        "radix_cache" in names and "shared_prefix_radix_split" in experiments
    ),
    "chunked_prefill_visible": (
        "chunked_prefill" in names and "chunk_size_ablation" in experiments
    ),
    "overlap_scheduling_visible": (
        "overlap_scheduling" in names and "overlap_ablation" in experiments
    ),
    "online_serving_visible": (
        "online_serving" in names and "online_streaming_trace" in experiments
    ),
    "structured_tool_boundaries_visible": all(
        name in names for name in ["structured_generation_boundary", "tool_agent_boundary"]
    ),
    "abort_cleanup_visible": "abort_cleanup_check" in experiments,
    "production_gap_visible": (
        "production_gap_map" in names and "sglang_gap_report" in experiments
    ),
    "dependency_order_ok": summary["dependency_order_ok"],
}
gates["mini_sglang_learning_gate"] = all(gates.values())
return rows, summary, gates

```

```

capabilities = [
    RuntimeCapability(

```

```

        "runtime_request_state",
        "sglang_runtime",
        ("request_lifecycle", "queue_state"),
        ("online_streaming_trace",),
        (),
    ),
    RuntimeCapability(
        "radix_cache",
        "sglang_runtime",
        ("longest_prefix_match", "tree_split", "kv_ownership"),
        ("shared_prefix_radix_split", "prefix_hit_metrics"),
        ("runtime_request_state",),
    ),
    RuntimeCapability(
        "chunked_prefill",
        "scheduler",
        ("suffix_chunk", "position_continuity", "partial_kv_cleanup"),
        ("chunk_size_ablation",),
        ("radix_cache",),
    ),
    RuntimeCapability(
        "overlap_scheduling",
        "scheduler",
        ("cpu_schedule_next", "gpu_forward_current", "metadata_ready"),
        ("overlap_ablation",),
        ("chunked_prefill",),
    ),
    RuntimeCapability(
        "online_serving",
        "serving_api",
        ("openai_api", "streaming", "client_abort"),
        ("online_streaming_trace", "abort_cleanup_check"),
        ("runtime_request_state",),
    ),
    RuntimeCapability(
        "interactive_shell",
        "serving_api",
        ("multi_turn_history", "reset_state", "stream_print"),
        ("multi_turn_shell_reuse",),
        ("radix_cache",),
    ),
    RuntimeCapability(
        "tensor_parallel",
        "parallelism",
        ("tp_rank", "weight_shard", "head_partition"),
        ("sglang_gap_report",),
        ("runtime_request_state",),
    ),
    RuntimeCapability(
        "kernel_backend",
        "kernel",
        ("attention_backend", "kv_layout_contract", "fallback_path"),
        ("sglang_gap_report",),
        ("chunked_prefill",),
    ),

```

```

RuntimeCapability(
    "structured_generation_boundary",
    "sglang_runtime",
    ("grammar_state", "valid_token_mask", "sampler_boundary"),
    ("structured_mask_design",),
    ("runtime_request_state",),
),
RuntimeCapability(
    "tool_agent_boundary",
    "sglang_runtime",
    ("tool_schema_prompt", "streaming_parser", "app_layer_tool_exec"),
    ("tool_call_parser_trace",),
    ("structured_generation_boundary",),
),
RuntimeCapability(
    "production_gap_map",
    "readiness",
    ("distributed_gap", "metrics_gap", "fault_recovery_gap"),
    ("sglang_gap_report",),
    ("online_serving", "tool_agent_boundary"),
),
]
rows, summary, gates = ToyMiniSGLangLearningAuditor().audit(capabilities)
print("mini_sglang_learning_summary=", summary)
print("mini_sglang_learning_gates=", gates)

```

一次运行的核心输出类似：

```

mini_sglang_learning_summary={'capabilities': 11, 'ready_capabilities': 11, 'capability_coverage': 1.0, 'experiment_co
mini_sglang_learning_gates= {'radix_cache_visible': True, 'chunked_prefill_visible': True, 'overlap_scheduling_visible

```

这个 demo 的重点是把 mini-sglang 的学习对象和前两个项目区分开：

1. radix_cache_visible=True: 必须能观察最长前缀匹配、tree split 和 KV ownership。
2. chunked_prefill_visible=True: 必须能做 chunk size ablation, 并检查 position / partial KV。
3. overlap_scheduling_visible=True: 必须能对比 CPU 调度和 GPU forward 是否重叠。
4. online_serving_visible=True: 必须能观察 streaming、client abort 和请求排队。
5. structured_tool_boundaries_visible=True: 要知道 grammar mask、tool parser 和真实工具执行的边界。
6. production_gap_visible=True: 要能说清 mini-sglang 与完整 SGLang 的差距。

所以本章最终门禁是 mini_sglang_learning_gate: 只有 Radix Cache、chunked prefill、overlap scheduling、online serving、structured/tool 边界、abort cleanup、生产差距和依赖顺序都可验证，mini-sglang 才真正承担起“SGLang runtime 过渡项目”的学习价值。

46.24 小练习

1. 对比 tiny-llm、nano-vLLM、mini-sglang 三者的学习重点。
2. 画出 mini-sglang 中一个 online request 的生命周期。
3. 构造三个共享前缀请求，画出 Radix Cache 的 tree split 和 insert。
4. 统计 prefix hit length 和节省的 prefill token 数。
5. 解释 Radix Cache 和 PagedAttention 的区别。
6. 对比 chunked prefill 开关或不同 chunk size 下的 TTFT、TPOT 和吞吐。
7. 关闭 overlap scheduling，观察吞吐和 GPU utilization 的变化。
8. 用 interactive shell 分析多轮对话中的 prefix reuse。
9. 设计一个最小 structured decoding token mask，并说明如何接入 sampler。
10. 设计一个 tool call streaming parser。
11. 构造 abort 请求，检查 KV cache 和 Radix Cache pin 是否释放。
12. 写一页 mini-sglang 到完整 SGLang 的差距分析。

46.25 本章总结

mini-sglang 是学习 SGLang-style runtime 的关键过渡项目。

它和 tiny-llm、nano-vLLM 的分工不同：tiny-llm 帮你理解底层计算和 serving 优化动机，nano-vLLM 帮你理解 vLLM-like engine 骨架，mini-sglang 帮你理解复杂 LLM program runtime、prefix sharing、Radix Cache、overlap scheduling、online serving、structured generation 和 tool/agent serving。

读 mini-sglang 时，重点不是重复学习普通 scheduler 或 KV cache，而是看 SGLang 的特色如何落到工程实现：共享前缀如何命中和复用，长 prompt 如何 chunked prefill，CPU 调度如何与 GPU forward 重叠，在线请求如何进入 runtime，结构化输出和工具调用如何影响 sampler 和 streaming。

读完这三个教学项目后，你应该能把“模型计算、engine 调度、runtime 语义”三层打通。

下一章会进入第 47 章：如何从这些教学项目中抽象出推理框架的核心模块，形成自己的 mini serving engine 设计图。

第 47 章 从教学项目抽象推理框架核心模块

前面三章分别看了三个教学项目：

1. nano-vLLM：理解 vLLM-like serving engine 的骨架。
2. tiny-llm：理解模型 forward 如何一步步演进 KV cache、continuous batching、chunked prefill 和 paged attention。
3. mini-sglang：理解 SGLang-style runtime、Radix Cache、prefix sharing、overlap scheduling、structured generation 和 online serving。

这三章不是为了让你背三个项目的目录结构。

真正的目标是：

从多个教学项目中抽象出推理框架的稳定模块边界，形成自己的 mini serving engine 设计图。

从这一章开始，第六部分会从“读源码”进入“改框架”。

后面几章会分别做升级：

1. 从 naive scheduler 升级到 continuous batching。
2. 从 list KV cache 升级到 paged KV cache。
3. 从同步 generate 升级到异步 streaming engine。
4. 从单模型服务升级到多模型服务。
5. 从单机服务升级到分布式 serving。

在升级之前，先要有一张清楚的框架设计图。

47.0 本讲资料边界与第二轮精修口径

本讲第二轮精修前，先按 WRITING_PLAN.md 对公开资料做校准：继续沿用 skyzh/tiny-llm 对 attention、RoPE、GQA、generate、KV cache、continuous batching、chunked prefill 和 paged attention 的课程式演进口径；沿用 GeeeekExplorer/nano-vllm 对 Sequence、Scheduler、BlockManager、ModelRunner、Attention、Sampler 和 vLLM-like engine 骨架的公开口径；沿用 sgl-project/mini-sglang 对 SGLang runtime、Radix Cache、chunked prefill、overlap scheduling、online serving、structured/tool 边界和 mini-sglang 模块结构的公开口径；并结合本书第 2、7 到 24、27 到 34、44 到 46 章的教学版 demo 和门禁。

本讲只讲“从教学项目抽象出自己的 mini serving engine 模块边界”：Request、Scheduler、KVCacheManager、BatchBuilder、ModelRunner、Sampler、OutputProcessor、EngineLoop、Metrics 的职责、输入输出、状态 ownership、依赖顺序和后续可替换路径。不把任何一个教学项目的源码目录、类名、函数签名、真实性能数字、生产级分布式控制面、完整 HTTP server、真实 GPU kernel 或某个框架的内部实现写成唯一设计。

本讲新增 demo 是教学版 core module auditor：用 0 依赖 Python 模拟 9 个核心模块，检查 module coverage、state coverage、boundary coverage、dependency order、replaceable modules 和最终 core module gate，

重点证明 Scheduler 不调用模型、BatchBuilder 拥有 batch metadata、ModelRunner 不修改 request 状态、KV manager 拥有 KV blocks、OutputProcessor 负责 finish / cleanup、Metrics 贯穿全链路。

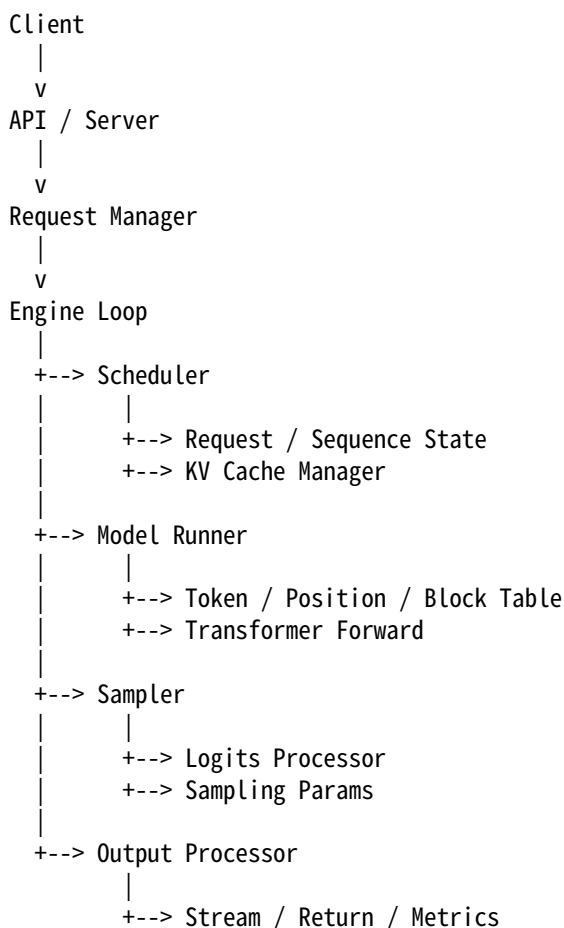
47.1 本章目标

读完本章，你应该能讲清：

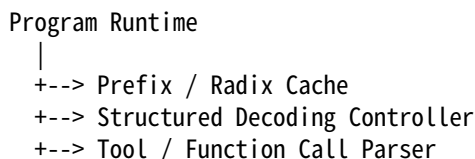
1. 一个 LLM 推理框架最小需要哪些核心模块。
2. API 层、engine 层、scheduler、KV manager、model runner、sampler 的职责边界。
3. request、sequence、batch、block、token 这些概念分别表示什么。
4. prefill 和 decode 在调度、模型执行、KV 写入上的区别。
5. 为什么推理框架的主循环通常围绕 iteration step 组织。
6. 如何从 nano-vLLM、tiny-llm、mini-sglang 中抽象出共同架构。
7. 如何设计一个可逐步升级的 mini serving engine。
8. 哪些模块应该先做简单版本，哪些模块可以后续替换。

47.2 先画总架构

一个最小 LLM serving engine 可以抽象成下面这张图：



如果加入 SGLang-style runtime，还会多出：



但不要一开始就把系统设计得很大。

最小版本只需要回答一个问题：

一批请求进入系统后，如何被调度成 batch，如何执行 prefill/decode，如何维护 KV cache，如何采样并返回 token？这就是推理框架的主骨架。

47.3 从三个项目抽象共同模块

可以把三个教学项目中的内容映射到同一张模块表里。

抽象模块	tiny-llm 提供的视角	nano-vLLM 提供的视角	mini-sglang 提供的视角
模型层	attention、RoPE、GQA、MLP、RMSNorm	model runner 调用模型	backend、kernel、TP、runtime 执行
请求状态	generate loop 中的 prompt/output	Sequence 状态机	request/runtime state、program state
调度器	从单请求到 batching	waiting/running 队列	prefix-aware、overlap、online scheduling
KV cache	list cache、paged attention 动机	block manager、block table	Radix Cache + KV pool 协作
prefill/decode	从计算差异理解	engine step 中分开处理	chunked prefill、prefix reuse
sampler	temperature/top-p 等采样	batch sampler	structured generation、tool call 约束
服务层	课程实验入口	offline API	OpenAI-compatible server、shell、benchmark
性能观测	统计 latency 和显存	TTFT/TPOT、scheduler log	prefix hit、overlap、online p99

这张表说明：

不同项目讲的是不同切面，但最后都会落到同一组工程模块。

真正学会推理框架，不是能复述某个项目的代码路径，而是能把这些模块拆出来、重新组合，并解释每个模块的职责。

47.4 最小推理框架的模块清单

一个可以逐步升级的 mini serving engine，至少需要这些模块：

- 1. Tokenizer：文本和 token 之间的转换。
- 2. Request：保存用户输入、采样参数、状态、输出 token。
- 3. Scheduler：决定每一轮执行哪些请求。
- 4. KVCacheManager：分配、释放、查询 KV cache。
- 5. BatchBuilder：把多个请求整理成模型 forward 需要的输入。
- 6. ModelRunner：执行模型 forward，返回 logits。
- 7. Sampler：根据 logits 和采样参数选出下一个 token。
- 8. OutputProcessor：更新请求状态、detokenize、streaming 输出。
- 9. Engine：串起整个 step loop。
- 10. Metrics：统计 TTFT、TPOT、latency、吞吐、显存使用。

如果要支持在线服务，还需要：

- 1. HTTP Server：接收请求。
- 2. Request Queue：跨线程或跨协程传递请求。
- 3. Streaming Channel：把 token 持续返回给客户端。
- 4. Abort Manager：处理取消请求。
- 5. Error Handler：处理超时、OOM、模型错误和客户端断连。

如果要支持 SGLang-style 能力，还需要：

- 1. PrefixCache 或 RadixCache。
- 2. ProgramState。
- 3. GrammarController 或 StructuredDecodingController。
- 4. ToolCallParser。
- 5. RuntimeScheduler。

但是第一版不要全做。

第一版建议只做：

Request + Scheduler + KVCacheManager + ModelRunner + Sampler + Engine Loop + Metrics

这个组合足以支撑后面几章的升级。

47.5 Request：请求不是一段文本

新手最容易把请求理解成一个字符串：

prompt = "请介绍一下 transformer"

但在推理框架里，请求是一个持续变化的执行状态。

一个 Request 至少包含：

字段	含义
request_id	请求唯一 ID
prompt	原始输入文本
prompt_tokens	tokenized prompt
output_tokens	已生成 token
sampling_params	temperature、top_p、max_tokens 等
status	waiting、running、finished、aborted
prefill_done	是否完成 prefill
kv_blocks	该请求占用的 KV block
arrival_time	到达时间
first_token_time	首 token 时间
finish_time	完成时间

如果支持 streaming，还需要：

字段	含义
stream_channel	输出 token 的通道
last_streamed_index	上次已经输出到哪个 token
client_disconnected	客户端是否断开

如果支持 prefix cache，还需要：

字段	含义
prefix_cache_hit_len	命中的前缀 token 数
computed_tokens	已经有 KV 的 token 数
remaining_prefill_tokens	还需要 prefill 的 token 数

所以请求状态不是静态输入，而是 engine 每一轮 step 都会更新的对象。

47.6 Sequence 和 Request 的关系

很多框架里会出现 Sequence、SequenceGroup、Request、Session 这些名字。

不要被名字吓到。

可以这样理解：

概念	常见含义
Request	用户提交的一次生成任务
Sequence	一条具体生成路径
SequenceGroup	同一个 request 下的一组候选序列
Session	多轮对话或长期状态

如果只支持普通单条生成：

一个 Request 约等于一个 Sequence

如果支持 beam search：

一个 Request 可能包含多个 Sequence

如果支持多轮对话和 prefix reuse：

一个 Session 下面可能有多个 Request，共享历史前缀

第一版 mini engine 不需要实现复杂层级。

建议先用一个 Request 表示一条生成路径，后面需要 beam search 或 session 再拆。

47.7 Scheduler：推理框架的决策中心

Scheduler 负责回答：

这一轮 engine step，哪些请求可以进入模型 forward？每个请求执行多少 token？

最简单的 scheduler 可以只有两个队列：

waiting：还没开始执行的请求

running：已经进入系统、正在生成的请求

每一轮调度时：

1. 从 waiting 中取若干请求进入 running。
2. 对未 prefill 的请求执行 prefill。
3. 对已 prefill 的请求执行 decode。
4. 如果请求完成，把它从 running 移除。
5. 如果请求 abort，释放资源并移除。

伪代码如下：

```
def schedule(waiting, running, kv_manager, token_budget):
    batch = []

    while waiting and token_budget > 0:
        req = waiting[0]
        needed = len(req.prompt_tokens)
        if needed > token_budget:
            break
        if not kv_manager.can_allocate(req, needed):
            break
        waiting.pop(0)
        running.append(req)
        batch.append((req, "prefill", needed))
        token_budget -= needed

    for req in running:
        if token_budget <= 0:
            break
        if req.prefill_done and not req.finished:
            batch.append((req, "decode", 1))
            token_budget -= 1

    return batch
```

这段逻辑很粗糙，但它已经包含 scheduler 的本质：

1. 管理 waiting/running 状态。
2. 受 token budget 限制。
3. 受 KV cache 容量限制。
4. 区分 prefill 和 decode。
5. 输出一个可执行 batch。

后面第 48 章会在这个基础上升级 continuous batching。

47.8 为什么要有 token budget

推理框架不是简单地“有多少请求就全塞进模型”。

原因是 prefill 和 decode 的 token 数差异很大。

例如：

请求 A: prompt 4096 tokens, 正在 prefill

请求 B: 已经 prefill 完, 每步 decode 1 token

请求 C: 已经 prefill 完, 每步 decode 1 token

如果 scheduler 每轮都无脑处理最长 prefill, 请求 B 和 C 的 decode 可能被长 prompt 阻塞, TPOT 抖动会很大。

所以 scheduler 通常要控制每一轮处理的 token 总数：

本轮最多处理 2048 个 token

这就是 token budget。

token budget 的作用是：

1. 防止超大 batch 导致 OOM。
2. 控制每轮 step 的时间。
3. 给 decode 请求保留执行机会。
4. 为 chunked prefill 提供基础。

5. 让吞吐和延迟之间可以调参。

没有 token budget，后面很难实现 continuous batching 和 chunked prefill。

47.9 KVCacheManager：显存不是 append list

KV cache 是推理框架中最重要的状态之一。

最简单的 KV cache 可以想象成：

request_id -> list of K/V tensors

但这种设计很快会遇到问题：

- 1. 不同请求长度不同，显存碎片严重。
- 2. 请求结束后释放不方便复用。
- 3. batch attention 难以统一访问。
- 4. 无法高效支持 prefix sharing。
- 5. 难以扩展到 paged attention。

更好的抽象是 block-based KV cache：

Physical KV Blocks

block 0
block 1
block 2
...

Request Block Table

req A -> [3, 7, 8]
req B -> [1, 2]
req C -> [9]

KVCacheManager 至少要支持：

接口	作用
can_allocate(num_tokens)	判断是否有足够 KV 空间
allocate(req, num_tokens)	给请求分配 block
append(req, num_tokens)	decode 时追加 token 空间
free(req)	请求结束后释放 block
get_block_table(req)	返回模型 forward 需要的 block table
num_free_blocks()	查询剩余 block 数

第一版可以先实现 list KV cache。

但接口要按 block manager 的方向设计，这样第 49 章升级 paged KV cache 时不需要重写 scheduler 和 engine。

47.10 ModelRunner：不要把调度逻辑塞进模型

ModelRunner 的职责是执行模型 forward。

它不应该决定哪些请求进 batch。

也不应该负责请求何时完成。

它只关心：

给我 input_ids、positions、kv cache metadata、attention metadata，我返回 logits。

典型输入包括：

输入	含义
input_ids	本轮要计算的 token
positions	每个 token 的 position
slot_mapping	token 写入 KV cache 的位置
block_tables	每个请求的 KV block 表
seq_lens	每个请求当前上下文长度
is_prefill	prefill 或 decode 标记

典型输出是：

logits: [num_sequences, vocab_size]

或者在 prefill 场景下：

logits: 只取每个请求最后一个 token 的 logits

这里有一个重要原则：

Scheduler 负责选 batch, BatchBuilder 负责组 metadata, ModelRunner 只负责跑模型。

如果把这些职责混在一起，后面想换 scheduler、换 KV manager、换 kernel backend 都会很痛苦。

47.11 BatchBuilder：容易被忽略的胶水层

很多人设计推理框架时只写：

```
batch = scheduler.schedule()  
logits = model(batch)
```

但真实系统里，scheduler 输出的通常不是模型能直接吃的 tensor。

Scheduler 输出的是逻辑计划：

```
req A: prefill token [0:512]  
req B: decode 1 token  
req C: decode 1 token
```

ModelRunner 需要的是物理输入：

```
input_ids  
positions  
seq_lens  
block_tables  
slot_mapping  
attention_metadata
```

把逻辑计划转换成物理输入的模块，就是 BatchBuilder。

它的职责包括：

1. 拼接 input token。
2. 计算 position。
3. 查询每个请求的 block table。
4. 计算 KV 写入位置。
5. 生成 attention metadata。
6. 标记每个请求对应的 logits 位置。

为什么要单独抽出来？

因为后续升级都会影响它：

升级	BatchBuilder 变化
continuous batching	batch 中混合不同阶段请求
chunked prefill	一个 prompt 被切成多个片段
paged attention	需要 block table 和 slot mapping
prefix cache	部分 token 已有 KV，只计算未命中部分
speculative decoding	一次可能处理多个 draft token
structured generation	可能需要额外 logits mask metadata

所以 BatchBuilder 是推理框架里最值得重视的胶水层之一。

47.12 Sampler: 采样不是 argmax

最简单的 sampler 是：

```
next_token = logits.argmax(dim=-1)
```

但真实框架里 sampler 需要处理很多采样参数：

1. temperature。
2. top-k。
3. top-p。
4. repetition penalty。
5. frequency penalty。
6. presence penalty。
7. stop token。
8. max tokens。
9. random seed。

如果支持 OpenAI-compatible API，还会遇到：

1. 多个 stop strings。
2. logprobs。
3. best_of 或 n。
4. response format。
5. tool call 结构。

如果支持 structured generation，sampler 还要接收 token mask：

当前状态下，哪些 token 是合法的？

因此 sampler 的输入不应该只是 logits。

建议设计成：

```
def sample(logits, requests, sampling_metadata):
    ...
```

其中 sampling_metadata 可以包含：

1. 每个请求的采样参数。
2. 每个请求的已生成 token。
3. structured decoding mask。
4. stop token 和 stop string 状态。
5. logprob 需求。

第一版可以只做 greedy 或 temperature sampling。

但接口要保留按 request 处理不同 sampling params 的能力。

47.13 OutputProcessor: 生成 token 后还没结束

模型 forward 和 sampler 之后，会得到每个请求的新 token。

但这不代表请求已经完成。

OutputProcessor 需要做：

1. 把 token append 到 request.output_tokens。
2. 更新请求长度。
3. 检查 EOS。
4. 检查 stop token。
5. 检查 stop string。
6. 检查 max_tokens。
7. detokenize 新增文本。
8. streaming 输出。
9. 更新 TTFT、TPOT、finish_time。
10. 对完成请求触发资源释放。

如果支持 tool call，还要做：

1. 增量解析 JSON。
2. 判断 tool call 是否完整。
3. 决定是否继续生成自然语言。
4. 输出兼容 OpenAI API 的 chunk。

如果支持 structured generation，还要做：

1. 更新 grammar state。
2. 给下一轮 sampler 准备 token mask。
3. 处理非法状态。

所以生成 token 只是中间结果。

请求生命周期真正结束，需要 OutputProcessor 做状态收尾。

47.14 Engine Loop: 推理框架的心跳

推理框架的核心通常是一个循环：

```
while engine_has_work():
    new_requests = request_queue.pop_all()
    scheduler.add_requests(new_requests)

    batch_plan = scheduler.schedule()
    if not batch_plan:
        wait_for_new_requests()
        continue

    model_inputs = batch_builder.build(batch_plan)
    logits = model_runner.forward(model_inputs)
    next_tokens = sampler.sample(logits, batch_plan.requests)
    output_processor.process(batch_plan, next_tokens)
    scheduler.remove_finished()
```

这个 loop 里有几个关键点：

1. 新请求可以在任意时刻到达。
2. 每一轮只执行 scheduler 选中的 batch。
3. batch 可能混合 prefill 和 decode。
4. model runner 不直接修改请求状态。
5. sampler 只决定 token，不负责释放资源。

6. output processor 更新请求状态。
7. scheduler 根据最新状态进入下一轮。

这个 loop 就是后续所有升级的骨架。

47.15 Prefill 和 Decode 的统一抽象

推理框架里最重要的区别之一是 prefill 和 decode。

阶段	输入	输出	计算特点	KV 行为
prefill	prompt tokens	prompt 最后位置 logits	token 多，矩阵大	写入整段 prompt 的 K/V
decode	上一个 token	下一个 token logits	每请求 1 token	追加新 token 的 K/V

在 scheduler 看来，它们可以统一成：

本轮为某个请求计算一段 token

区别只是：

prefill: token span 长度可能很大

decode: token span 长度通常是 1

如果引入 chunked prefill，prefill 也会被切成多个小 span：

prefill chunk 0: tokens [0:1024]

prefill chunk 1: tokens [1024:2048]

prefill chunk 2: tokens [2048:3072]

于是 prefill 和 decode 可以进入同一个 batch 计划：

req A: prefill tokens [1024:2048]

req B: decode token [17]

req C: decode token [56]

这就是 continuous batching 和 chunked prefill 的基础。

47.16 状态机视角

一个请求在 engine 内部可以用状态机表示：

```

WAITING
|
v
PREFILLING
|
v
DECODING
|
+--> FINISHED
+--> ABORTED
+--> ERROR

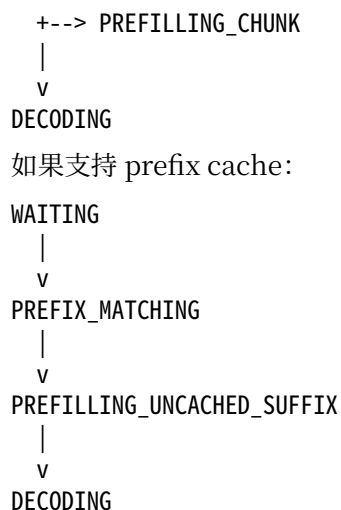
```

如果支持 chunked prefill：

```

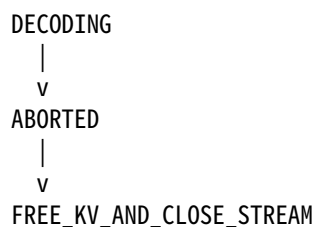
WAITING
|
v
PREFILLING_CHUNK
|

```

如果支持 prefix cache:

如果支持 streaming abort:



状态机的价值是让你明确:

1. 哪些状态会占用 KV cache。
2. 哪些状态可以被 scheduler 选择。
3. 哪些状态需要输出 token。
4. 哪些状态需要释放资源。
5. 哪些状态可以转移到 finish。

很多推理框架 bug，本质都是状态机没设计清楚。

47.17 Metrics：没有观测就没有优化

推理框架项目一定要内置 metrics。

否则你无法证明自己的改造有意义。

最小 metrics 包括:

指标	含义
QPS	每秒完成请求数
output TPS	每秒输出 token 数
TTFT	time to first token
TPOT	time per output token
latency	请求总延迟
running requests	当前运行请求数
waiting requests	当前排队请求数
KV usage	KV cache 使用率
prefill tokens/s	prefill 吞吐
decode tokens/s	decode 吞吐

更进一步可以统计:

1. p50/p90/p99 TTFT。
2. p50/p90/p99 TPOT。
3. scheduler 每轮 batch size。
4. 每轮 prefill/decode token 数。
5. OOM 次数。
6. abort 次数。
7. prefix cache hit length。
8. GPU utilization。

后面每做一次升级，都应该用 metrics 证明：

这个改造改善了什么，牺牲了什么？

例如：

1. continuous batching 通常提升吞吐，但可能增加单请求延迟波动。
2. chunked prefill 通常降低 decode 抖动，但可能增加长 prompt 的 TTFT。
3. paged KV cache 通常提升显存利用率，但会引入 block table 管理复杂度。
4. streaming engine 提升用户感知体验，但增加异步状态管理难度。

47.18 模块边界的反模式

写 mini engine 时，最常见的坏设计有几种。

反模式一：scheduler 直接调用模型。

问题：调度策略和模型执行耦合，后面无法替换 model runner 或做异步执行。

反模式二：model runner 修改 request 状态。

问题：模型层知道太多业务状态，streaming、abort、finish 逻辑会散落各处。

反模式三：KV cache 直接挂在 request 的 tensor list 上。

问题：后面升级 paged KV cache、prefix cache、eviction 都要大改。

反模式四：sampler 只支持全 batch 同一参数。

问题：在线 serving 中每个请求的 temperature、top_p、max_tokens 都可能不同。

反模式五：没有 request id 和状态机。

问题：debug 时无法追踪一个请求在 waiting、prefill、decode、finish 之间如何变化。

反模式六：没有 metrics。

问题：所有优化都只能凭感觉，无法比较 TTFT、TPOT、吞吐和显存。

反模式七：一开始就设计生产级大框架。

问题：还没跑通 prefill/decode，就引入多模型、分布式、grammar、spec decode，复杂度失控。

正确做法是：

先用清晰模块边界跑通最小闭环，再逐个替换内部实现。

47.19 推荐的第一版 mini engine

如果你要从 0 做一个项目，第一版可以这样定范围。

支持：

1. 单模型。
2. 单进程。
3. offline generate API。
4. 多请求 batch。

5. 区分 prefill 和 decode。
6. naive waiting/running scheduler。
7. 简单 KV cache。
8. greedy sampling。
9. TTFT、TPOT、latency 统计。

暂不支持：

1. HTTP server。
2. streaming。
3. paged attention。
4. prefix cache。
5. tensor parallel。
6. structured generation。
7. tool call。
8. speculative decoding。
9. 多模型。
10. 分布式。

第一版的目标不是性能最好，而是结构正确。

可以把项目目录设计成：

```
mini_engine/  
  engine.py  
  request.py  
  scheduler.py  
  kv_cache.py  
  batch.py  
  model_runner.py  
  sampler.py  
  output.py  
  metrics.py  
  benchmark.py
```

每个文件只做一件事。

47.20 第一版执行流程

一个 offline generate 可以这样执行：

1. 用户提交 prompts。
2. Tokenizer 把 prompt 转成 prompt_tokens。
3. 为每个 prompt 创建 Request。
4. Request 加入 scheduler waiting 队列。
5. Engine loop 开始运行。
6. Scheduler 选择 prefill 请求。
7. KVCacheManager 分配 KV 空间。
8. BatchBuilder 构造 prefill 输入。
9. ModelRunner 执行 prefill forward。
10. Sampler 采样第一个 token。
11. OutputProcessor 更新 output_tokens 和 TTFT。
12. Scheduler 下一轮选择 decode 请求。
13. ModelRunner 执行 decode forward。
14. Sampler 采样下一个 token。
15. OutputProcessor 判断 EOS 或 max_tokens。
16. 请求完成后释放 KV cache。
17. 所有请求完成后返回文本和 metrics。

注意这里有一个细节：

prefill 之后通常会直接采样第一个输出 token。

也就是说，prefill 不只是写 prompt KV，它还会产生 prompt 最后一个位置的 logits，用来采样第一个生成 token。

decode 则用上一次生成的 token 作为输入，采样下一 token。

47.21 从第一版到后续章节的升级路线

第 47 章得到的是骨架。

后面几章会在这套骨架上替换模块。

后续章节	替换或增强的模块	核心变化
第 48 章	Scheduler	naive batch -> continuous batching
第 49 章	KVCacheManager、BatchBuilder	list KV -> paged KV / block table
第 50 章	Engine、OutputProcessor、API	synchronous generate -> async streaming
第 51 章	RequestManager、ModelRunner	single model -> multi model routing
第 52 章	ModelRunner、KVCacheManager、Scheduler	single node -> distributed serving

这个顺序很重要。

不要一开始就做 streaming。

不要一开始就做分布式。

不要一开始就做 prefix cache。

原因是这些能力都会依赖前面的基础模块：

- 1. 没有清晰 request state，就做不好 streaming abort。
- 2. 没有 BatchBuilder，就做不好 paged attention metadata。
- 3. 没有 token budget，就做不好 chunked prefill。
- 4. 没有 KV manager，就做不好 prefix cache 和 eviction。
- 5. 没有 metrics，就不知道升级是否有效。

47.22 如何写成简历项目

如果你把这一章落实成项目，简历上不要写：

实现了一个大模型推理框架。

这句话太泛。

可以写得更具体：

从 0 实现单机 LLM serving engine，包含 Request 状态机、waiting/running scheduler、KV cache manager、batch builder、model

如果你后续完成第 48、49、50 章，可以继续升级：

在自研 mini serving engine 上实现 continuous batching、token budget、chunked prefill 和 paged KV cache，构造不同 prompt/o

如果你想突出工程能力，可以写：

为推理 engine 增加 per-request trace 和 scheduler decision log，记录每轮 selected requests、prefill/decode token 数、KV b

这些描述比“复现 vLLM”更可信。

47.23 面试高频问题

问题一：一个 LLM 推理框架最核心的模块有哪些？

回答要点：API/request manager、request state、scheduler、KV cache manager、batch builder、model runner、sampler、output processor、metrics。在线服务还需要 request queue、streaming channel、abort/error handling。

问题二：Scheduler 和 ModelRunner 的边界是什么？

回答要点：Scheduler 决定本轮执行哪些请求、每个请求执行多少 token，并受 token budget 和 KV 容量约束；ModelRunner 只负责根据 input ids、positions、block table、attention metadata 执行 forward，返回 logits。调度策略不应塞进模型层。

问题三：为什么需要 BatchBuilder？

回答要点：Scheduler 输出的是逻辑 batch plan，ModelRunner 需要物理 tensor 和 metadata。BatchBuilder 负责拼 input ids、positions、seq lens、slot mapping、block table、attention metadata，是 continuous batching、chunked prefill、paged attention、prefix cache 的胶水层。

问题四：prefill 和 decode 有什么区别？

回答要点：prefill 输入完整 prompt 或 prompt chunk，计算量大，写入一段 K/V，并用最后位置 logits 采样首 token；decode 每轮通常输入一个新 token，追加 K/V，并采样下一个 token。prefill 影响 TTFT，decode 影响 TPOT。

问题五：为什么第一版也要做 metrics？

回答要点：没有 metrics 就无法验证优化效果。至少要统计 TTFT、TPOT、latency、output TPS、waiting/running 请求数、KV 使用率。后续 continuous batching、chunked prefill、paged KV cache 都需要这些指标做对比。

问题六：第一版 mini engine 为什么不建议直接做分布式或 streaming？

回答要点：这些能力依赖稳定的 request state、scheduler、KV manager、output processor 和 metrics。先跑通单机 offline engine，可以把模块边界定清楚；后面再升级 streaming、paged KV、多模型、分布式，改动会更可控。

47.24 标准回答模板

如果面试官问“你会如何设计一个最小 LLM 推理框架”，可以这样回答：

我会先把框架拆成几个核心模块：Request 状态、Scheduler、KVCacheManager、BatchBuilder、ModelRunner、Sampler、OutputProc

第一版我会只支持单机单模型 offline generate，先跑通 prefill 和 decode。prefill 处理 prompt token，写入 prompt 的 KV，并用

这个设计的重点是模块边界，而不是第一版性能最好。后续可以逐步把 naive scheduler 升级为 continuous batching，把简单 KV ca

47.25 核心模块边界、状态 ownership 和可运行 demo

先把核心模块抽象成：

$M_j = (r_j, I_j, O_j, S_j, D_j)$

其中 r_j 是职责， I_j 是输入集合， O_j 是输出集合， S_j 是该模块拥有的状态， D_j 是依赖模块。

模块覆盖率：

$$C_{\{\mathrm{module}\}} = \frac{|M_{\{\mathrm{seen}\}} \cap M_{\{\mathrm{req}\}}|}{\max(1, |M_{\{\mathrm{req}\}}|)}$$

状态覆盖率：

$$C_{\{\mathrm{state}\}} = \frac{|S_{\{\mathrm{seen}\}} \cap S_{\{\mathrm{req}\}}|}{\max(1, |S_{\{\mathrm{req}\}}|)}$$

边界覆盖率:

$$C_{\{\mathrm{boundary}\}} = \frac{|B_{\{\mathrm{pass}\}}|}{|\{\max(1, |B_{\{\mathrm{req}\}}|)\}|}$$

最终门禁:

$$G_{\{\mathrm{core}\}} = G_{\{\mathrm{request}\}} G_{\{\mathrm{scheduler}\}} G_{\{\mathrm{kv}\}} G_{\{\mathrm{batch}\}} G_{\{\mathrm{runner}\}} G_{\{\mathrm{...}\}}$$

这组公式强调: 第一版 mini engine 的关键不是功能多, 而是模块边界稳。下面的 demo 不执行真实模型, 而是检查设计图本身是否可升级。

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class CoreModule:
```

```
    name: str
    responsibility: str
    inputs: tuple
    outputs: tuple
    owns_state: tuple
    depends_on: tuple
    replaceable: bool
```

```
class ToyCoreModuleAuditor:
```

```
    def __init__(self):
```

```
        self.required_modules = [
            "request",
            "scheduler",
            "kv_cache_manager",
            "batch_builder",
            "model_runner",
            "sampler",
            "output_processor",
            "engine_loop",
            "metrics",
        ]
```

```
        self.required_states = [
            "request_status",
            "waiting_queue",
            "running_set",
            "kv_blocks",
            "batch_metadata",
            "logits",
            "output_tokens",
            "ttft_tpot_latency",
        ]
```

```
        self.required_boundaries = [
            "scheduler_not_model_runner",
            "batch_builder_owns_metadata",
            "model_runner_no_request_mutation",
            "output_processor_finishes_requests",
            "kv_manager_owns_blocks",
            "metrics_observe_all_steps",
        ]
```

```
    def coverage(self, seen, required):
```

```

    return round(len(set(seen) & set(required)) / max(1, len(required)), 3)

def audit(self, modules):
    names = [module.name for module in modules]
    states = sorted({state for module in modules for state in module.owns_state})
    outputs = sorted({item for module in modules for item in module.outputs})
    completed = set()
    rows = []
    for module in modules:
        deps_ready = all(dep in completed for dep in module.depends_on)
        row = {
            "module": module.name,
            "responsibility": module.responsibility,
            "deps_ready": deps_ready,
            "replaceable": module.replaceable,
            "owns_state": list(module.owns_state),
        }
        rows.append(row)
        if deps_ready:
            completed.add(module.name)
    boundary_checks = {
        "scheduler_not_model_runner": "logits" not in next(
            module.owns_state for module in modules if module.name == "scheduler"
        ),
        "batch_builder_owns_metadata": "batch_metadata" in next(
            module.owns_state for module in modules if module.name == "batch_builder"
        ),
        "model_runner_no_request_mutation": "request_status" not in next(
            module.owns_state for module in modules if module.name == "model_runner"
        ),
        "output_processor_finishes_requests": "request_status" in next(
            module.owns_state for module in modules if module.name == "output_processor"
        ),
        "kv_manager_owns_blocks": "kv_blocks" in next(
            module.owns_state for module in modules if module.name == "kv_cache_manager"
        ),
        "metrics_observe_all_steps": "ttft_tpot_latency" in next(
            module.owns_state for module in modules if module.name == "metrics"
        ),
    }
    summary = {
        "modules": len(rows),
        "ready_modules": sum(row["deps_ready"] for row in rows),
        "module_coverage": self.coverage(names, self.required_modules),
        "state_coverage": self.coverage(states, self.required_states),
        "boundary_coverage": self.coverage(
            [name for name, ok in boundary_checks.items() if ok],
            self.required_boundaries,
        ),
        "replaceable_modules": sum(module.replaceable for module in modules),
        "engine_outputs": outputs,
        "missing_modules": sorted(set(self.required_modules) - set(names)),
        "missing_states": sorted(set(self.required_states) - set(states)),
    }
    gates = {

```

```

        "request_state_visible": "request_status" in states,
        "scheduler_boundary_clear": boundary_checks["scheduler_not_model_runner"],
        "kv_manager_boundary_clear": boundary_checks["kv_manager_owns_blocks"],
        "batch_builder_boundary_clear": boundary_checks["batch_builder_owns_metadata"],
        "model_runner_boundary_clear": boundary_checks["model_runner_no_request_mutation"],
        "output_processor_boundary_clear": boundary_checks["output_processor_finishes_requests"],
        "metrics_ready": boundary_checks["metrics_observe_all_steps"],
        "upgrade_path_ready": summary["replaceable_modules"] >= 6,
    }
    gates["core_module_gate"] = all(gates.values())
    return rows, summary, gates

```

```

modules = [
    CoreModule(
        "request",
        "hold per-request inputs and execution state",
        ("prompt", "sampling_params"),
        ("prompt_tokens",),
        ("request_status", "output_tokens"),
        (),
        False,
    ),
    CoreModule(
        "scheduler",
        "select prefill and decode work under token and KV budgets",
        ("waiting_queue", "running_set", "kv_free"),
        ("batch_plan",),
        ("waiting_queue", "running_set"),
        ("request",),
        True,
    ),
    CoreModule(
        "kv_cache_manager",
        "allocate append query and free KV blocks",
        ("batch_plan",),
        ("block_table", "kv_free"),
        ("kv_blocks",),
        ("request",),
        True,
    ),
    CoreModule(
        "batch_builder",
        "convert logical batch plan into model runner metadata",
        ("batch_plan", "block_table"),
        ("model_inputs",),
        ("batch_metadata",),
        ("scheduler", "kv_cache_manager"),
        True,
    ),
    CoreModule(
        "model_runner",
        "run model forward and return logits",
        ("model_inputs",),
        ("logits",),
    ),
]

```



```

        ("logits",),
        ("batch_builder",),
        True,
    ),
    CoreModule(
        "sampler",
        "choose next token per request",
        ("logits", "sampling_params"),
        ("next_tokens",),
        (),
        ("model_runner",),
        True,
    ),
    CoreModule(
        "output_processor",
        "append tokens finish requests and trigger cleanup",
        ("next_tokens", "batch_plan"),
        ("finished_ids", "stream_chunks"),
        ("request_status", "output_tokens"),
        ("sampler",),
        True,
    ),
    CoreModule(
        "metrics",
        "record queue wait TTFT TPOT latency throughput and KV usage",
        ("batch_plan", "finished_ids"),
        ("metric_report",),
        ("ttft_tpot_latency",),
        ("output_processor",),
        True,
    ),
    CoreModule(
        "engine_loop",
        "orchestrate schedule build forward sample output cleanup",
        ("requests",),
        ("responses", "metric_report"),
        (),
        ("metrics",),
        False,
    ),
]
rows, summary, gates = ToyCoreModuleAuditor().audit(modules)
print("core_module_summary=", summary)
print("core_module_gates=", gates)

```

一次运行的核心输出类似：

```

core_module_summary= {'modules': 9, 'ready_modules': 9, 'module_coverage': 1.0, 'state_coverage': 1.0, 'boundary_coverage': 1.0, 'kv_manager_boundary_clear': True, 'scheduler_boundary_clear': True, 'request_state_visible': True}
core_module_gates= {'request_state_visible': True, 'scheduler_boundary_clear': True, 'kv_manager_boundary_clear': True, 'engine_loop_boundary_clear': True}

```

这个 demo 展示了第一版 mini engine 的几个硬边界：

1. Scheduler 只输出 batch_plan，不拥有 logits，也不直接跑模型。
2. KVCacheManager 拥有 kv_blocks，Request 不直接挂一串 tensor list。
3. BatchBuilder 拥有 batch_metadata，负责把逻辑 plan 变成物理输入。
4. ModelRunner 只返回 logits，不修改 request_status。
5. OutputProcessor 负责 append token、finish request 和触发 cleanup。

6. Metrics 贯穿全链路，至少能记录 TTFT、TPOT 和 latency。
7. 至少 6 个模块可替换，说明后续 continuous batching、paged KV、streaming、multi-model 和 distributed serving 有升级空间。

所以本章最终门禁是 `core_module_gate`：只有模块、状态、边界、依赖和可替换路径都清楚，所谓“从教学项目抽象自己的推理框架”才不是画一张大图，而是可以落到后续章节逐步替换的工程骨架。

47.26 小练习

1. 画出一个最小 LLM serving engine 的模块图。
2. 为 Request 设计字段，区分输入字段、执行状态字段、性能统计字段。
3. 设计 waiting/running scheduler 的状态转移。
4. 写出 prefill 和 decode 的输入输出差异。
5. 设计 KVCacheManager 的最小接口。
6. 设计 BatchBuilder 如何从 batch plan 生成 input ids、positions 和 block table。
7. 给 engine loop 增加 TTFT、TPOT 和 latency 统计。
8. 解释为什么 sampler 要支持 per-request sampling params。
9. 写一个 debug log，打印每轮 scheduler 选择了哪些请求。
10. 设计一个 token budget 策略，并说明它如何影响 TTFT 和 TPOT。
11. 说明第一版 mini engine 暂不支持哪些能力，为什么。
12. 写一页从 tiny-llm、nano-vLLM、mini-sglang 抽象共同模块的总结。

47.27 本章总结

读完 tiny-llm、nano-vLLM 和 mini-sglang 后，下一步不是继续堆更多项目，而是抽象出推理框架的共同模块。

一个最小 LLM serving engine 至少需要 Request、Scheduler、KVCacheManager、BatchBuilder、ModelRunner、Sampler、OutputProcessor、Engine Loop 和 Metrics。

其中最关键的边界是：Scheduler 负责决策，BatchBuilder 负责把逻辑计划转成物理输入，ModelRunner 负责 forward，Sampler 负责选 token，OutputProcessor 负责更新请求状态和输出，KVCacheManager 负责显存状态。

第一版不要追求生产级能力，而要先跑通单机、单模型、offline、多请求、prefill/decode、KV cache 和 metrics。

有了这个骨架，后续才能有控制地升级 scheduler、KV cache、streaming、多模型和分布式 serving。

下一章会进入第 48 章：从 naive scheduler 升级到 continuous batching。

第 48 章 从 naive scheduler 升级到 continuous batching

上一章我们从 tiny-llm、nano-vLLM、mini-sglang 中抽象出一个最小推理框架骨架：Request、Scheduler、KVCacheManager、BatchBuilder、ModelRunner、Sampler、OutputProcessor 和 Metrics。

从这一章开始，我们不再只是画模块图，而是对这个 mini engine 做第一轮升级：

把 naive scheduler 升级成 continuous batching scheduler。

这是推理框架最重要的升级之一。

原因很简单：

LLM serving 的吞吐和延迟，很大程度取决于 scheduler 如何把不同阶段、不同长度、不同到达时间的请求组织进每一轮 model forward。

如果 scheduler 设计不好，即使模型 kernel 很快，系统也会出现 GPU 空转、decode 抖动、长 prompt 阻塞、batch 利用率低、TTFT 和 TPOT 很差等问题。

48.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径, 只讲把教学版 naive scheduler 升级成单机、单模型、同步 engine loop 内的 continuous batching scheduler。

公开资料校准主要参考三类口径:

1. Orca 论文对 iteration-level scheduling 和 selective batching 的系统动机: 生成式模型的 batch 不应该被 request lifetime 锁死, 而应该在 iteration 边界动态调度。
2. vLLM 文档对 max_num_batched_tokens、max_num_seqs、chunked prefill、decode 请求优先和 preemption 观测的公开说明: scheduler 必须同时看 token budget、running 数量、prefill / decode 平衡和 KV 容量。
3. vLLM / PagedAttention 论文对 block-based KV cache 的口径: continuous batching 不是单独的 batch 技巧, 而是和 KV block allocation、free、reuse、admission control 一起成立。

本章不实现真实 vLLM scheduler、PagedAttention kernel、prefix cache、抢占恢复、CPU swap、多租户优先级、分布式 worker、异步 HTTP server、CUDA graph 或生产级 fairness。我们只保留最小可运行骨架:

waiting queue -> running set -> BatchPlan -> BatchBuilder -> OutputProcessor -> cleanup / metrics

本章新增公式和 demo 只用于验证升级门禁:

naive request-level batch lock 是否可见;
iteration boundary 是否被使用;
请求能否动态加入和动态退出;
decode-first 与 bounded prefill 是否同时成立;
token budget、max running requests、KV capacity 是否共同约束调度;
finished / cancelled 请求是否能释放资源。

48.1 本章目标

读完本章, 你应该能讲清:

1. naive batching 和 continuous batching 的区别。
2. 为什么 request-level batch 不适合在线 LLM serving。
3. iteration-level batching 的核心思想。
4. prefill 和 decode 如何混排。
5. token budget、max running requests、KV capacity 如何共同约束调度。
6. continuous batching 如何影响 TTFT、TPOT、吞吐和显存使用。
7. 如何从一个简单 waiting/running scheduler 改造成 continuous batching scheduler。
8. 如何设计 scheduler 日志和压测指标验证改造效果。

48.2 naive scheduler 长什么样

最简单的 scheduler 通常是 request-level batching:

收集一批请求 -> 一起 prefill -> 一起 decode 到全部完成 -> 再处理下一批请求

伪代码类似:

```
def naive_generate(requests):
    batch = requests[:max_batch_size]

    prefill(batch)

    while not all_finished(batch):
        decode(batch)

    return outputs(batch)
```

这种设计适合离线批处理，但不适合在线服务。

在线服务中，请求是持续到达的：

t=0ms req A arrives
t=20ms req B arrives
t=35ms req C arrives
t=80ms req D arrives

如果 req A 已经进入 decode，naive scheduler 可能会让 B/C/D 等到 A 完成后才能进来。

这会导致：

1. 新请求 TTFT 很差。
2. GPU batch size 可能很小。
3. 已完成请求不能及时释放 batch slot。
4. 长输出请求拖住整批请求。
5. 请求长度差异越大，浪费越明显。

这就是 request-level batch 的根本问题：

batch 的生命周期被最长请求绑定。

48.3 一个直观例子

假设有三个请求：

请求	prompt 长度	output 长度
A	128	200
B	128	20
C	128	30

naive batching 会把 A/B/C 组成一批。

prefill 后开始 decode。

B 在第 20 步完成。

C 在第 30 步完成。

A 要到第 200 步才完成。

如果 batch slot 不能动态补新请求，那么第 31 到第 200 步，原本属于 B/C 的位置就浪费了。

更糟的是，如果 D/E/F 在这期间到达，它们可能需要等 A 完成后才能进入下一批。

于是系统会出现：

GPU 还在跑，但 batch 利用率越来越低；新请求还在排队，TTFT 越来越差。

continuous batching 解决的正是这个问题。

48.4 continuous batching 的核心思想

continuous batching 的核心不是“把 batch 做大”。

它的核心是：

每一轮 decode iteration 都重新调度，把新请求、未完成请求、刚释放资源的请求动态组合成当前最合适的 batch。

也可以叫 iteration-level batching。

对比一下：

方式	batch 形成时机	batch 生命周期	新请求何时进入
naive batching	一批请求开始时	直到整批完成	下一批开始时
continuous batching	每轮 iteration	只对当前 step 有效	下一轮或若干轮后即可进入

continuous batching 的执行节奏是：

step 0: 调度 A/B/C prefill
step 1: 调度 A/B/C decode
step 2: 调度 A/B/C decode, 同时接收 D/E
step 3: 调度 A/B/C decode + D prefill chunk
step 4: B 完成, 调度 A/C/D decode + E prefill chunk
step 5: C 完成, 调度 A/D/E decode + F prefill chunk

注意这里的 batch 不是固定的一组请求。

每一轮 scheduler 都会重新做选择。

48.5 continuous batching 不等于无限插队

continuous batching 容易被误解成：

新请求一来就马上插进当前 batch。

实际不是。

GPU forward 一旦开始, 当前 batch 就固定了。

新请求最多只能进入下一轮或后续某一轮。

所以 continuous batching 的粒度是：

iteration boundary

也就是每次 model forward 结束后, scheduler 有一次重新选择 batch 的机会。

在这个边界上, scheduler 会考虑：

1. 哪些 running 请求需要继续 decode。
2. 哪些 waiting 请求可以开始 prefill。
3. 是否有 KV 空间。
4. 是否还有 token budget。
5. 是否超过 max running requests。
6. 是否要优先 decode, 避免 TPOT 抖动。
7. 是否允许 prefill chunk 混入本轮。

因此 continuous batching 的本质是：

在 iteration boundary 做动态重组, 而不是在 GPU forward 中途修改 batch。

48.6 第一步：定义调度输入和输出

升级 scheduler 前, 先要明确接口。

Scheduler 的输入包括：

输入	含义
waiting	尚未开始或等待 prefill 的请求
running	已经占用 KV、正在 prefill/decode 的请求
finished	已完成、待清理或已清理的请求
kv_manager	查询和分配 KV 空间

输入	含义
token_budget	本轮最多计算多少 token
max_running_requests	最多同时运行多少请求
policy	decode-first、prefill-first、fairness 等策略

Scheduler 的输出不应该是 tensor。

它应该输出 batch plan:

```
BatchPlan(
  scheduled=[
    ScheduleItem(req=A, kind="decode", token_start=128, token_count=1),
    ScheduleItem(req=B, kind="decode", token_start=256, token_count=1),
    ScheduleItem(req=C, kind="prefill", token_start=0, token_count=512),
  ],
  num_prefill_tokens=512,
  num_decode_tokens=2,
)
```

为什么不是直接返回 input_ids?

因为 input_ids、positions、block_tables、slot_mapping 应该由 BatchBuilder 构造。

Scheduler 只负责逻辑决策。

48.7 Request 需要增加哪些字段

为了支持 continuous batching, Request 需要记录更多执行进度。

最小字段包括:

字段	含义
prompt_tokens	prompt token 列表
output_tokens	已生成 token 列表
computed_prompt_tokens	prompt 中已经完成 prefill 的 token 数
prefill_done	prompt 是否完成 prefill
num_generated_tokens	已生成 token 数
status	waiting/running/finished/aborted
kv_blocks	已分配 KV block
last_token	decode 本轮要输入的 token
arrival_time	请求到达时间
first_scheduled_time	第一次被调度时间
first_token_time	首 token 时间
last_decode_time	上次 decode 完成时间

computed_prompt_tokens 很关键。

没有 chunked prefill 时, 它要么是 0, 要么是完整 prompt 长度。

有 chunked prefill 后, 它可以是:

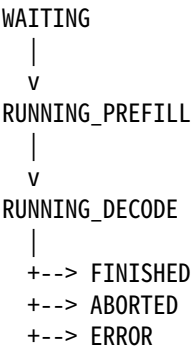
0 -> 512 -> 1024 -> 1536 -> prompt_len

它让 scheduler 知道:

这个请求还剩多少 prompt token 没有计算?

48.8 最小 continuous batching 状态机

请求状态可以这样设计：



如果支持 chunked prefill，RUNNING_PREFILL 会持续多轮。

状态转移如下：

当前状态	条件	下一状态
WAITING	被 scheduler 接纳，KV 分配成功	RUNNING_PREFILL
RUNNING_PREFILL	prompt 未算完	RUNNING_PREFILL
RUNNING_PREFILL	prompt 算完并采样首 token	RUNNING_DECODE
RUNNING_DECODE	未达到停止条件	RUNNING_DECODE
RUNNING_DECODE	EOS/max_tokens/stop	FINISHED
任意运行状态	用户取消	ABORTED
任意运行状态	OOM/异常	ERROR

Scheduler 只应该选择：

1. WAITING 中可以进入 prefill 的请求。
2. RUNNING_PREFILL 中还没算完 prompt 的请求。
3. RUNNING_DECODE 中还没完成的请求。

已经 FINISHED、ABORTED、ERROR 的请求不应该再被调度。

48.9 token budget 的作用

continuous batching 的关键约束是 token budget。

假设本轮 token budget 是 2048。

那么 scheduler 可以选择：

- 方案 A：2048 个 prefill token
- 方案 B：1024 个 prefill token + 128 个 decode token
- 方案 C：512 个 prefill token + 256 个 decode token
- 方案 D：只做 512 个 decode token

注意 decode token 和 prefill token 的计算成本不完全等价。

但是在教学版 engine 中，可以先用 token 数作为近似预算。

token budget 至少解决三个问题：

1. 防止一轮 batch 太大导致 OOM。
2. 防止长 prompt prefill 独占 GPU 太久。
3. 控制一轮 step 时间，让 decode TPOT 更稳定。

如果没有 token budget, 一个 32K prompt 可能会让所有 decode 请求等很久。

这会造成:

平均吞吐看起来还行, 但 TPOT p99 非常差。

面试里讲 continuous batching, 一定要同时讲 token budget。

否则回答会停留在“动态组 batch”的表面。

48.10 decode-first 策略

很多推理框架会优先保证 decode 请求。

原因是 decode 阶段直接影响用户感知的流式输出速度。

如果用户已经看到模型开始输出, 但后续 token 一卡一卡的, 体验会很差。

decode-first 的思想是:

每一轮先给 running decode 请求分配 1 token, 再用剩余 token budget 处理 prefill。

伪代码:

```
def schedule_decode_first(waiting, running, token_budget):
    plan = []

    for req in running:
        if token_budget <= 0:
            break
        if req.status == "RUNNING_DECODE" and not req.finished:
            plan.append(ScheduleItem(req, "decode", req.next_token_pos, 1))
            token_budget -= 1

    while waiting and token_budget > 0:
        req = waiting[0]
        chunk = min(req.remaining_prompt_tokens, token_budget)
        plan.append(ScheduleItem(req, "prefill", req.computed_prompt_tokens, chunk))
        token_budget -= chunk
        waiting.pop(0)
        running.append(req)

    return plan
```

decode-first 的优点:

1. TPOT 更稳定。
2. 流式输出更平滑。
3. 已经开始输出的请求不容易被长 prompt 饿死。

缺点:

1. waiting 请求的 TTFT 可能变差。
2. 如果 running decode 很多, prefill 可能长期进不来。
3. 需要 admission control 防止 running 请求过多。

所以 decode-first 不能单独使用, 还要配合 max running requests、prefill quota 或 fairness 策略。

48.11 prefill-first 策略的问题

prefill-first 看起来也有道理:

尽快让新请求完成 prefill, 尽快产生首 token。

这有利于 TTFT。

但是如果 prefill 长度很大，它会拖慢所有 running decode 请求。

例如：

running: 128 个 decode 请求，每个需要 1 token

waiting: 1 个 8192-token prompt

token_budget: 8192

如果本轮全部拿去做 prefill，128 个 decode 请求会等完整个长 prompt forward。

用户感知就是：

已经在输出的请求突然卡住。

所以在线服务一般不会简单 prefill-first。

更常见的是：

decode-first + bounded prefill

或者：

decode 优先，但每轮保留一部分 budget 给 prefill，避免新请求饿死。

48.12 bounded prefill

bounded prefill 的意思是限制每轮 prefill token 数。

例如：

total_token_budget = 4096

max_prefill_tokens_per_step = 1024

那么每轮最多只处理 1024 个 prefill token，剩余 budget 留给 decode 或其他请求。

如果一个 prompt 有 8192 token，就会被拆成 8 轮：

step 1: prefill [0:1024]

step 2: prefill [1024:2048]

...

step 8: prefill [7168:8192]

这就是 chunked prefill 的基础。

bounded prefill 解决的是：

1. 长 prompt 阻塞 decode。
2. 单轮 forward 时间过长。
3. TPOT p99 抖动。
4. GPU memory 峰值不可控。

代价是：

1. 长 prompt 的 TTFT 可能增加。
2. scheduler 和 BatchBuilder 更复杂。
3. 需要维护 computed_prompt_tokens。

所以 bounded prefill 是吞吐、TTFT、TPOT 之间的权衡。

48.13 admission control: 不是所有 waiting 都能进来

continuous batching 还有一个容易忽略的问题：

如果每轮都不断接纳新请求，running 队列会越来越大。

running 请求越多, KV cache 占用越高。

decode 每轮要处理的 token 也越多。

如果 running 太大, 会出现:

1. KV cache 被占满。
2. decode batch 太大, 单步延迟上升。
3. 新请求虽然进来了, 但整体 TPOT 变差。
4. OOM 风险增加。

所以 scheduler 需要 admission control。

最简单的规则:

`len(running) < max_running_requests`

更真实的规则:

KV cache 是否足够容纳 `prompt + max_new_tokens` 的上界?

或者:

当前 running decode token 数是否超过上限?

在教学版 engine 中, 可以先用三个约束:

1. `max_running_requests`。
2. `token_budget`。
3. `kv_manager.can_allocate(req)`。

这三个约束已经足够解释 continuous batching 的核心行为。

48.14 一个更完整的调度伪代码

下面是一个教学版 continuous batching scheduler。

它采用:

1. decode-first。
2. bounded prefill。
3. max running requests。
4. KV capacity check。

```
def schedule(
    waiting,
    running,
    kv_manager,
    token_budget,
    max_prefill_tokens,
    max_running_requests,
):
    plan = []
    remaining_budget = token_budget
    remaining_prefill_budget = max_prefill_tokens

    # 1. Prioritize one decode token for each running decode request.
    for req in list(running):
        if remaining_budget <= 0:
            break
        if req.status != "RUNNING_DECODE":
            continue
        if req.finished or req.aborted:
```

```

        continue

    plan.append(ScheduleItem(req=req, kind="decode", token_start=req.next_pos, token_count=1))
    remaining_budget -= 1

# 2. Continue chunked prefill for already admitted requests.
for req in list(running):
    if remaining_budget <= 0 or remaining_prefill_budget <= 0:
        break
    if req.status != "RUNNING_PREFILL":
        continue

    chunk = min(req.remaining_prompt_tokens, remaining_budget, remaining_prefill_budget)
    if chunk <= 0:
        continue

    plan.append(ScheduleItem(req=req, kind="prefill", token_start=req.computed_prompt_tokens, token_count=chunk))
    remaining_budget -= chunk
    remaining_prefill_budget -= chunk

# 3. Admit new waiting requests if there is room.
while waiting and remaining_budget > 0 and remaining_prefill_budget > 0:
    if len(running) >= max_running_requests:
        break

    req = waiting[0]
    if not kv_manager.can_allocate(req):
        break

    waiting.pop(0)
    running.append(req)
    req.status = "RUNNING_PREFILL"
    kv_manager.allocate(req)

    chunk = min(req.remaining_prompt_tokens, remaining_budget, remaining_prefill_budget)
    if chunk <= 0:
        break

    plan.append(ScheduleItem(req=req, kind="prefill", token_start=0, token_count=chunk))
    remaining_budget -= chunk
    remaining_prefill_budget -= chunk

return BatchPlan(plan)

```

这段代码不是生产实现，但结构是对的。

它体现了几个关键原则：

1. decode 请求每轮最多调度 1 token。
2. prefill 可以被切 chunk。
3. running prefill 比新 waiting 请求优先，避免已经接纳的请求长期卡在 prefill 中间。
4. 新请求进入 running 前先检查 KV capacity。
5. batch plan 只描述逻辑 token span，不构造 tensor。

48.15 调度顺序的权衡

上面的顺序是：

decode -> existing prefill -> new prefill
这是一个稳妥的教学版策略。
但不同系统可能选择不同顺序。

顺序	优点	风险
decode -> prefill	TPOT 稳定	TTFT 可能变差
prefill -> decode	TTFT 可能更好	decode 抖动大
existing prefill -> new prefill -> decode	已接纳请求更快首 token	streaming 卡顿
decode + fixed prefill quota	折中	参数需要调优
priority-based	可支持 SLA	实现复杂

面试时不要说某一种策略永远最好。
正确表达是：
调度策略是在 TTFT、TPOT、吞吐、显存和公平性之间做权衡。

48.16 BatchBuilder 需要怎么变

continuous batching 升级后，BatchBuilder 的输入会更复杂。
原来可能是：
一批请求全部 prefill
或者：
一批请求全部 decode
现在可能是混合 batch：
req A: decode 1 token
req B: decode 1 token
req C: prefill tokens [0:512]
req D: prefill tokens [1024:1536]

BatchBuilder 要为每个 ScheduleItem 生成：

metadata	说明
input_ids	本轮要计算的 token
positions	每个 token 的 position
seq_lens	当前序列长度
slot_mapping	每个 token 写入 KV 的位置
block_tables	每个请求的 KV block 表
query_lens	每个请求本轮 query token 数
logits_indices	哪些 token 位置需要取 logits

对于 decode：
input_ids = [last_generated_token]
positions = [current_seq_len - 1]
query_len = 1
对于 prefill chunk：
input_ids = prompt_tokens[start:start+chunk]
positions = [start, start+1, ...]
query_len = chunk

如果 prefill chunk 不是最后一段，通常不需要采样输出 token。

只有当 prompt prefill 完成后，才需要取最后一个 prompt token 的 logits 采样首 token。

所以 BatchBuilder 还要告诉 OutputProcessor：

哪些 request 在本轮 forward 后需要采样？

这就是 logits_indices 的作用。

48.17 OutputProcessor 需要怎么变

continuous batching 后，OutputProcessor 不能再假设每个请求每轮都有输出 token。

对于 prefill chunk，有两种情况：

1. 不是最后一个 chunk：只更新 computed_prompt_tokens，不采样。
2. 是最后一个 chunk：更新 computed_prompt_tokens，采样首 token，进入 decode。

对于 decode：

1. append 新 token。
2. 更新 num_generated_tokens。
3. 检查 EOS/stop/max_tokens。
4. 更新 last_decode_time。
5. 如果完成，标记 FINISHED。

伪代码：

```
def process_outputs(batch_plan, sampled_tokens):
    for item in batch_plan.items:
        req = item.req

        if item.kind == "prefill":
            req.computed_prompt_tokens += item.token_count
            if req.computed_prompt_tokens < len(req.prompt_tokens):
                continue

            token = sampled_tokens[req.request_id]
            req.output_tokens.append(token)
            req.first_token_time = now()
            req.status = "RUNNING_DECODE"
            continue

        if item.kind == "decode":
            token = sampled_tokens[req.request_id]
            req.output_tokens.append(token)
            req.last_decode_time = now()
            if should_stop(req):
                req.status = "FINISHED"
```

这里的关键点是：

被调度不等于一定产生输出 token。

prefill 中间 chunk 只是在补 KV。

48.18 KV capacity 如何影响调度

continuous batching 会让多个请求长期同时处于 running 状态。

这意味着 KV cache 是 admission control 的核心约束。

一个请求的 KV 需求大致是：

`prompt_len + max_new_tokens`

如果只按 `prompt_len` 分配，decode 过程中可能不断追加 KV，最终 OOM。

教学版可以先简化成两种策略。

策略一：保守预分配。

进入 `running` 前，为 `prompt_len + max_new_tokens` 预留 KV。

优点：不会 decode 到一半没空间。

缺点：显存利用率低，因为很多请求不会生成到 `max_new_tokens`。

策略二：按需追加。

`prefill` 时分配 `prompt KV`，`decode` 每步追加 `1 token KV`。

优点：显存利用率高。

缺点：`decode` 中途可能 KV 不足，需要抢占、等待或终止。

教学版第一阶段建议用保守预分配，逻辑简单。

等第 49 章升级 `paged KV cache` 后，再讨论更细的 `block` 分配、回收和抢占。

48.19 抢占和暂停：第一版可以先不做

生产级 `scheduler` 可能会支持 `preemption`。

例如：

`KV cache` 不够时，把低优先级请求暂停，释放部分 KV，优先服务高优先级请求。

或者：

把某个请求的 KV `swap` 到 CPU，再恢复。

这些能力很复杂，第一版不建议实现。

第一版可以采用简单规则：

1. 如果 KV 不够，新请求继续留在 `waiting`。
2. 已经 `running` 的请求不抢占。
3. 如果 `decode` 追加 KV 失败，标记 `error` 或等待下一轮。
4. 通过 `max running requests` 降低 KV 打满概率。

这样能保持 `scheduler` 清晰。

等你理解 `continuous batching` 后，再考虑 `preemption`。

48.20 scheduler debug log

`continuous batching` 一定要加 `debug log`。

否则出了问题很难定位。

每轮至少打印：

字段	含义
<code>step_id</code>	第几轮 <code>engine step</code>
<code>waiting_count</code>	<code>waiting</code> 请求数
<code>running_count</code>	<code>running</code> 请求数
<code>scheduled_count</code>	本轮调度请求数

字段	含义
prefill_reqs	本轮 prefill 请求数
decode_reqs	本轮 decode 请求数
prefill_tokens	本轮 prefill token 数
decode_tokens	本轮 decode token 数
token_budget_used	使用了多少 token budget
kv_free_blocks	剩余 KV block
finished_count	本轮完成请求数

示例日志：

step=128 waiting=42 running=96 scheduled=96 prefill_reqs=4 decode_reqs=92 prefill_tokens=1024 decode_tokens=92 budget=1.

这行日志可以快速回答：

1. scheduler 是否在持续接纳新请求。
2. decode 是否被长 prefill 饿死。
3. token budget 是否利用充分。
4. KV 是否成为瓶颈。
5. running 是否接近 max_running_requests。

48.21 压测场景设计

验证 continuous batching，不能只跑几个 prompt。

至少要设计几类 workload。

场景一：短 prompt、短输出。

prompt_len: 64-128
output_len: 16-32
arrival: fixed QPS

观察重点：

1. QPS。
2. TTFT。
3. batch 利用率。

场景二：短 prompt、长输出。

prompt_len: 64-128
output_len: 256-512
arrival: fixed QPS

观察重点：

1. running 队列长度。
2. decode batch size。
3. TPOT p99。

场景三：长 prompt、短输出。

prompt_len: 4096-8192
output_len: 16-32
arrival: fixed QPS

观察重点：

1. TTFT。
2. decode 是否被 prefill 阻塞。
3. chunked prefill 的效果。

场景四：混合长度。

80% short prompt + 20% long prompt
output_len: mixed
arrival: poisson

观察重点：

- 1. p99 TTFT。
- 2. p99 TPOT。
- 3. 短请求是否被长请求拖慢。
- 4. token budget 参数是否合理。

场景五：突发流量。

前 10 秒低 QPS，中间 10 秒高 QPS，后 10 秒恢复低 QPS

观察重点：

- 1. waiting 队列积压。
- 2. 恢复时间。
- 3. KV cache 使用峰值。
- 4. abort/error 数量。

48.22 对比指标

升级前后要对比 naive scheduler 和 continuous batching。

至少记录：

指标	预期变化
output tokens/s	通常上升
QPS	通常上升
平均 TTFT	取决于策略
p99 TTFT	可能改善，也可能因 decode-first 变差
平均 TPOT	通常更稳定
p99 TPOT	通常改善
GPU utilization	通常上升
KV usage	通常上升
waiting queue length	高负载下更平滑
OOM 次数	取决于 admission control

不要只看平均值。

推理 serving 的问题经常藏在 p99。

尤其要看：

TTFT p99 和 TPOT p99

如果 continuous batching 提升了吞吐，但 TPOT p99 爆炸，用户体验可能更差。

48.23 常见 bug

bug 一：prefill 中间 chunk 也采样 token。

结果：prompt 还没算完就开始生成，position 和 KV 状态错乱。

bug 二：decode 请求一轮被调度多个 token。

结果：没有 speculative decoding 的情况下，下一 token 依赖上一 token，不能一次调度多个未知 token。

bug 三：请求 finished 后没有释放 KV。

结果：KV usage 持续上涨，最终新请求进不来。

bug 四：waiting 请求进入 running 后，KV 分配失败但状态已修改。

结果：请求既不在 waiting，也不能正常 running，变成悬挂状态。

bug 五：token budget 只统计 prefill，不统计 decode。

结果：running decode 很多时，实际 batch 远超预算。

bug 六：BatchBuilder 没区分 logits 位置。

结果：从 prefill chunk 中间位置采样，或者 decode logits 对错请求。

bug 七：没有处理 abort。

结果：客户端断开后请求还在占用 KV 和 decode slot。

bug 八：running prefill 被新 waiting 请求长期挤压。

结果：某些请求进入 running 后迟迟拿不到首 token。

48.24 面试高频问题

问题一：continuous batching 和 naive batching 的区别是什么？

回答要点：naive batching 在 request 级别形成固定 batch，一批请求通常要一起 decode 到完成，新请求要等下一批；continuous batching 在 iteration 级别动态重组 batch，每轮 forward 结束后可以加入新请求、移除完成请求，让 batch slot 持续被利用。

问题二：为什么 continuous batching 能提升吞吐？

回答要点：它避免 batch 生命周期被最长请求绑定，完成请求释放的 slot 可以被新请求填充；在线请求持续到达时，GPU 更容易保持较高 batch 利用率，减少小 batch 和空转。

问题三：continuous batching 如何影响 TTFT 和 TPOT？

回答要点：它通常提升吞吐并改善 TPOT 稳定性，但具体取决于策略。decode-first 可以降低 TPOT 抖动，但可能让 waiting 请求 TTFT 变差；prefill quota 和 chunked prefill 可以在 TTFT 和 TPOT 之间做权衡。

问题四：为什么需要 token budget？

回答要点：token budget 限制每轮处理的 token 数，防止长 prompt prefill 独占 GPU 或导致 OOM，也让每轮 step 时间更可控。没有 token budget，长 prefill 会严重影响 decode TPOT p99。

问题五：decode-first 有什么优缺点？

回答要点：优点是保障已经进入 decode 的请求每轮持续产生 token，TPOT 和 streaming 体验更稳定；缺点是如果 running decode 太多，waiting 请求可能迟迟无法 prefill，TTFT 变差，所以需要 max running requests、prefill quota 或 fairness 策略。

问题六：continuous batching 下 BatchBuilder 为什么更复杂？

回答要点：batch 中可能同时有 decode、完整 prefill、prefill chunk。BatchBuilder 需要为不同请求生成 input ids、positions、seq lens、slot mapping、block table、query lens 和 logits indices，并区分哪些请求本轮需要采样。

48.25 标准回答模板

如果面试官问“你如何把 naive scheduler 升级成 continuous batching”，可以这样回答：

我会先把 scheduler 的输出定义成 batch plan，而不是直接构造 tensor。每个 ScheduleItem 描述一个 request 本轮执行 prefill 通

naive scheduler 的问题是 request-level batch 生命周期被最长请求绑定，一批请求里短请求完成后 slot 不能及时复用，新请求要 level batching，每轮 model forward 结束后重新调度：移除完成请求，继续调度 running decode，请求允许在 iteration boundary

具体策略上，我会先实现 decode-first + bounded prefill。每轮先给 RUNNING_DECODE 请求调度 1 个 token，保证 TPOT 和 streaming。验证时我会对比 naive scheduler 和 continuous batching 在不同 workload 下的 TTFT、TPOT、output tokens/s、QPS、KV usage 和

48.26 Naive Scheduler 到 Continuous Batching 升级门禁和可运行 demo

把 naive scheduler 升级成 continuous batching，不能只写一句“每轮动态组 batch”。最小验收要能同时量化静态 batch 空洞、每轮 token budget、prefill chunk、running 上限、KV 准入和 cleanup。

先定义请求 i 的实际输出 token 数为 y_i 。如果 request-level batch 被最长输出锁住，静态 decode 行数近似为：

$$W_{\mathrm{static}} = N \max_i y_i$$

continuous batching 的 decode 行数等于每轮真正参与 decode 的请求数之和：

$$W_{\mathrm{cont}} = \sum_{t=1}^T |D_t|$$

本轮 token budget 约束：

$$|D_t| + \sum_{i \in P_t} c_{i,t} \leq B_{\mathrm{tok}}$$

其中 D_t 是本轮 decode 请求集合， P_t 是本轮 prefill 请求集合， $c_{i,t}$ 是请求 i 本轮 prefill chunk token 数。

bounded prefill 约束：

$$\sum_{i \in P_t} c_{i,t} \leq B_{\mathrm{prefill}}$$

running 上限和 KV 准入约束：

$$R_t \leq R_{\mathrm{max}}$$

$$K_t + K_i^{\mathrm{reserve}} \leq K_{\mathrm{max}}$$

最终升级门禁可以写成：

$$G_{\mathrm{upgrade}} = G_{\mathrm{lock}} G_{\mathrm{iter}} G_{\mathrm{join}} G_{\mathrm{exit}} G_{\mathrm{decode}} G_{\mathrm{cleanup}}$$

下面这个 0 依赖 demo 不跑真实模型，只模拟 scheduler 的核心状态变化：A/B/C 是第一批请求，B/C 比 A 短；X 会在 waiting 中被取消；D/E 在 A 还未结束时动态加入；E 的长 prompt 被 bounded prefill 切成多轮；X 先被 max_running_requests 和 kv_capacity 延迟，取消后必须释放队头阻塞。

```
from dataclasses import dataclass
import math
```

```
@dataclass
class Request:
    request_id: str
    arrival_step: int
    prompt_len: int
    max_new_tokens: int
    target_new_tokens: int
    cancel_step: int = None
    status: str = "NEW"
    computed_prompt_tokens: int = 0
    generated_tokens: int = 0
    first_scheduled_step: int = None
    first_token_step: int = None
    finish_step: int = None

    @property
```

```

def remaining_prompt_tokens(self):
    return self.prompt_len - self.computed_prompt_tokens

@dataclass
class ScheduleItem:
    request_id: str
    kind: str
    token_start: int
    token_count: int

class ToyKVManager:
    def __init__(self, total_blocks, block_size):
        self.total_blocks = total_blocks
        self.block_size = block_size
        self.allocations = {}
        self.max_used_blocks = 0

    def reserve_blocks(self, req):
        return math.ceil((req.prompt_len + req.max_new_tokens) / self.block_size)

    def used_blocks(self):
        return sum(self.allocations.values())

    def free_blocks(self):
        return self.total_blocks - self.used_blocks()

    def can_reserve(self, req):
        return self.reserve_blocks(req) <= self.free_blocks()

    def reserve(self, req):
        blocks = self.reserve_blocks(req)
        self.allocations[req.request_id] = blocks
        self.max_used_blocks = max(self.max_used_blocks, self.used_blocks())

    def release(self, req):
        self.allocations.pop(req.request_id, None)

class ToyNaiveToContinuousBatcher:
    def __init__(self, requests, token_budget, max_prefill_tokens, max_running_requests, kv_manager):
        self.requests = {req.request_id: req for req in requests}
        self.token_budget = token_budget
        self.max_prefill_tokens = max_prefill_tokens
        self.max_running_requests = max_running_requests
        self.kv_manager = kv_manager
        self.waiting = []
        self.running = []
        self.finished_order = []
        self.cancelled = []
        self.deferred_reasons = []
        self.trace = []

    def defer(self, req, reason):

```

```

item = f"{req.request_id}:{reason}"
if item not in self.deferred_reasons:
    self.deferred_reasons.append(item)

def arrive_and_cancel(self, step):
    for req in self.requests.values():
        if req.arrival_step == step:
            req.status = "WAITING"
            self.waiting.append(req)

    for req in list(self.waiting) + list(self.running):
        if req.cancel_step == step:
            req.status = "ABORTED"
            if req in self.waiting:
                self.waiting.remove(req)
            if req in self.running:
                self.running.remove(req)
                self.kv_manager.release(req)
            self.cancelled.append(req.request_id)

def schedule_step(self, step):
    remaining_budget = self.token_budget
    remaining_prefill = self.max_prefill_tokens
    plan = []

    for req in list(self.running):
        if remaining_budget <= 0:
            break
        if req.status == "RUNNING_DECODE":
            plan.append(ScheduleItem(req.request_id, "decode", req.prompt_len + req.generated_tokens - 1, 1))
            remaining_budget -= 1

    for req in list(self.running):
        if remaining_budget <= 0 or remaining_prefill <= 0:
            break
        if req.status != "RUNNING_PREFILL":
            continue
        chunk = min(req.remaining_prompt_tokens, remaining_budget, remaining_prefill)
        if chunk < req.remaining_prompt_tokens:
            self.defer(req, "prefill_budget")
        plan.append(ScheduleItem(req.request_id, "prefill", req.computed_prompt_tokens, chunk))
        remaining_budget -= chunk
        remaining_prefill -= chunk

    while self.waiting and remaining_budget > 0 and remaining_prefill > 0:
        if len(self.running) >= self.max_running_requests:
            self.defer(self.waiting[0], "max_running_requests")
            break
        req = self.waiting[0]
        if not self.kv_manager.can_reserve(req):
            self.defer(req, "kv_capacity")
            break

        self.waiting.pop(0)
        self.running.append(req)

```

```

req.status = "RUNNING_PREFILL"
req.first_scheduled_step = step
self.kv_manager.reserve(req)

chunk = min(req.remaining_prompt_tokens, remaining_budget, remaining_prefill)
if chunk < req.remaining_prompt_tokens:
    self.defer(req, "prefill_budget")
plan.append(ScheduleItem(req.request_id, "prefill", req.computed_prompt_tokens, chunk))
remaining_budget -= chunk
remaining_prefill -= chunk

return plan

def execute(self, step, plan):
    for item in plan:
        req = self.requests[item.request_id]
        if item.kind == "prefill":
            req.computed_prompt_tokens += item.token_count
            if req.computed_prompt_tokens < req.prompt_len:
                continue
            req.generated_tokens += 1
            req.status = "RUNNING_DECODE"
            if req.first_token_step is None:
                req.first_token_step = step + 1
        else:
            req.generated_tokens += 1

        if req.generated_tokens >= req.target_new_tokens:
            req.status = "FINISHED"
            req.finish_step = step + 1
            self.finished_order.append(req.request_id)

    for req in list(self.running):
        if req.status == "FINISHED":
            self.running.remove(req)
            self.kv_manager.release(req)

def run(self, max_steps=20):
    for step in range(max_steps):
        self.arrive_and_cancel(step)
        plan = self.schedule_step(step)
        self.trace.append(
            {
                "step": step,
                "plan_kinds": [item.kind for item in plan],
                "decode": [item.request_id for item in plan if item.kind == "decode"],
                "prefill": [(item.request_id, item.token_start, item.token_count) for item in plan if item.kind == "prefill"],
                "waiting": [req.request_id for req in self.waiting],
                "running": [req.request_id for req in self.running],
                "used_blocks": self.kv_manager.used_blocks(),
                "free_blocks": self.kv_manager.free_blocks(),
            }
        )
        self.execute(step, plan)
    live = self.waiting or self.running

```

```

        future = any(req.status == "NEW" for req in self.requests.values())
        if not live and not future:
            break

    completed = [req for req in self.requests.values() if req.status == "FINISHED"]
    static_rows = len(completed) * max(req.target_new_tokens for req in completed)
    continuous_rows = sum(req.target_new_tokens for req in completed)
    queue_wait = {
        req.request_id: req.first_scheduled_step - req.arrival_step
        for req in completed
    }
    ttft = {
        req.request_id: req.first_token_step - req.arrival_step
        for req in completed
    }
    gates = {
        "request_level_lock_visible": static_rows > continuous_rows,
        "iteration_boundary_used": len(self.trace) > 1,
        "dynamic_join_visible": self.requests["D"].first_scheduled_step < self.requests["A"].finish_step,
        "dynamic_exit_visible": self.requests["B"].finish_step < self.requests["A"].finish_step,
        "decode_first_ready": all(
            "prefill" not in row["plan_kinds"][: row["plan_kinds"].count("decode")]
            for row in self.trace
        ),
        "bounded_prefill_ready": any("prefill_budget" in item for item in self.deferred_reasons),
        "kv_admission_ready": any("kv_capacity" in item for item in self.deferred_reasons),
        "cleanup_ready": self.kv_manager.used_blocks() == 0,
    }
    gates["scheduler_upgrade_gate"] = all(gates.values())
    summary = {
        "finished_order": self.finished_order,
        "cancelled": self.cancelled,
        "queue_wait_steps": queue_wait,
        "ttft_steps": ttft,
        "static_decode_rows": static_rows,
        "continuous_output_rows": continuous_rows,
        "saved_rows": static_rows - continuous_rows,
        "max_running": max(len(row["running"]) for row in self.trace),
        "max_kv_blocks_used": self.kv_manager.max_used_blocks,
        "deferred_reasons": self.deferred_reasons,
        "trace_tail": self.trace[-4:],
    }
    return summary, gates

```

```

requests = [
    Request("A", arrival_step=0, prompt_len=4, max_new_tokens=6, target_new_tokens=6),
    Request("B", arrival_step=0, prompt_len=4, max_new_tokens=2, target_new_tokens=2),
    Request("C", arrival_step=0, prompt_len=4, max_new_tokens=3, target_new_tokens=3),
    Request("X", arrival_step=1, prompt_len=12, max_new_tokens=4, target_new_tokens=4, cancel_step=4),
    Request("D", arrival_step=2, prompt_len=5, max_new_tokens=2, target_new_tokens=2),
    Request("E", arrival_step=3, prompt_len=8, max_new_tokens=2, target_new_tokens=2),
]

```

```

batcher = ToyNaiveToContinuousBatcher(

```

```

    requests=requests,
    token_budget=8,
    max_prefill_tokens=8,
    max_running_requests=3,
    kv_manager=ToyKVManager(total_blocks=8, block_size=4),
)
summary, gates = batcher.run()
print("scheduler_upgrade_summary=", summary)
print("scheduler_upgrade_gates=", gates)

```

一次运行的核心输出类似：

```

scheduler_upgrade_summary= {'finished_order': ['B', 'C', 'A', 'D', 'E'], 'cancelled': ['X'], 'queue_wait_steps': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'waiting_queue_length': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'kv_usage': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'tpot_p99': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'output_tps': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'waiting_queue_length_p99': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'kv_usage_p99': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'tpot_p99_p99': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}, 'output_tps_p99': {'A': 0, 'B': 0, 'C': 0, 'D': 0, 'E': 0}},
scheduler_upgrade_gates= {'request_level_lock_visible': True, 'iteration_boundary_used': True, 'dynamic_join_visible': True}

```

这个 demo 的重点不是性能数值，而是升级验收证据：

1. static_decode_rows 大于 continuous_output_rows，说明 request-level batch lock 的空洞可见。
2. D/E 在 A 还没完成时进入 running，说明 dynamic join 生效。
3. B/C 在 A 之前完成并释放 KV，说明 dynamic exit 和 cleanup 生效。
4. X 因 running 上限和 KV 容量被延迟，随后取消，说明 waiting 队列不能只入不出。
5. E 的 prompt 被切成多轮 prefill，说明 bounded prefill 没有让长 prompt 独占 iteration。
6. 最后 scheduler_upgrade_gate=True，说明调度、预算、准入和清理路径同时闭环。

48.27 小练习

1. 画出 naive batching 和 continuous batching 的时序图。
2. 构造 A/B/C 三个不同 output 长度请求，说明 naive batching 如何浪费 batch slot。
3. 为 Request 增加 computed_prompt_tokens、status、first_token_time 字段。
4. 实现一个返回 BatchPlan 的 scheduler，而不是直接返回 tensor。
5. 实现 decode-first 调度策略。
6. 增加 max_prefill_tokens_per_step，支持 bounded prefill。
7. 设计 max_running_requests，观察它对 TTFT 和 TPOT 的影响。
8. 给 scheduler 增加每轮 debug log。
9. 设计短 prompt 长输出 workload，对比 naive 和 continuous batching 的 output TPS。
10. 设计长 prompt 短输出 workload，观察 chunked prefill 对 TPOT p99 的影响。
11. 统计 waiting queue length 和 KV usage，判断瓶颈来自哪里。
12. 写一个面试回答：continuous batching 为什么不是简单把 batch size 调大？

48.28 本章总结

continuous batching 是 LLM serving scheduler 的核心能力。

naive batching 在 request 级别形成固定 batch，batch 生命周期被最长请求绑定，不适合持续请求到达的在线服务。

continuous batching 把调度粒度降到 iteration 级别，每轮 forward 结束后重新组合 batch，让完成请求释放资源，让新请求在合适时机进入 running，从而提升 batch 利用率和吞吐。

实现 continuous batching 时，关键不是一句“动态 batching”，而是要处理好 token budget、decode-first、bounded prefill、admission control、KV capacity、BatchBuilder metadata 和 OutputProcessor 状态更新。

调度策略没有绝对最优。decode-first 更重视 TPOT 和 streaming 平滑度，prefill quota 会影响 TTFT，max running requests 和 KV capacity 决定系统能接纳多少并发请求。

真正能证明升级有效的，是 scheduler log 和压测指标，尤其是 TTFT p99、TPOT p99、output TPS、KV usage 和 waiting queue length。

下一章会进入第 49 章：从 list KV cache 升级到 paged KV cache。

第 49 章 从 list KV cache 升级到 paged KV cache

上一章我们把 naive scheduler 升级成了 continuous batching。

但只改 scheduler 还不够。

如果底层 KV cache 仍然是教学版 list 或连续张量，continuous batching 很快会撞到三个问题：

1. 长短请求混合时显存浪费严重。
2. 请求动态加入和退出时，KV 空间难以复用。
3. scheduler 很难在调度前准确判断还能接纳多少请求。

本章继续沿着 mini engine 的升级路线，把 KV cache 从最简单的 list 结构升级成 paged KV cache。

这一章不是重复 PagedAttention 原理。

前面的第 17、18、21 章已经讲过 block table、block manager 和 vLLM memory management。

本章关注一个更工程化的问题：

如果你已经有一个能跑的教学版推理框架，应该怎样一步步把 KV cache 改成 paged 版本？

49.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲把教学版 list KV cache 升级成单机、单模型、单进程内的 paged KV cache。

公开资料校准主要参考三类口径：

1. vLLM / PagedAttention 论文对固定大小 KV blocks、logical block 到 physical block 映射、非连续 KV 存储和高吞吐 serving 的公开定义。
2. vLLM 文档对 KV cache blocks、KV cache usage、preemption、chunked prefill 和 scheduler admission 的公开口径。
3. 本书第 17、18、21、48 章已经建立的 PagedAttention、KV block manager、vLLM memory management 和 continuous batching 边界。

本章不实现真实 attention kernel、GPU KV tensor layout、prefix cache、block hash、ref count sharing、LRU eviction、CPU swap、preemption recompute、多 worker KV 分片或多租户隔离。我们只验证一个 mini engine 改造中最小但关键的工程闭环：

global block pool -> request block table -> allocate_until -> slot_mapping -> model input metadata -> free / reuse / metrics

第二轮新增 demo 的验收重点是：

list KV 私有缓存是否被全局 block pool 替代；
request block table 是否随 prefill / decode 增长；
decode 跨 block 边界是否追加 block；
BatchBuilder 是否生成 slot mapping；
finished / aborted / failed 是否幂等释放；
释放后的 block 是否能被新请求复用；
scheduler 是否能在 admission 阶段拦住 KV block 不足。

49.1 本章目标

读完本章，你应该能讲清：

1. 教学版 list KV cache 的结构为什么不适合 online serving。
2. paged KV cache 要新增哪些核心数据结构。
3. request 的 block table 如何维护。
4. prefill 和 decode 阶段分别如何分配 KV blocks。
5. BatchBuilder 为什么需要生成 slot mapping 和 block table。
6. scheduler 如何基于 free blocks 做 admission control。

7. finished、abort、failed 请求如何释放 KV blocks。
8. 如何用日志和压测证明 paged KV cache 生效。

面试时，很多人能背 PagedAttention 的定义，但讲不清 “如果让我改一个 mini engine，我具体改哪里”。本章就是为了补上这块。

49.2 教学版 list KV cache 长什么样

在最小教学项目里，KV cache 往往写得很直接。

例如每个 request 保存一个 list:

```
class RequestState:
    def __init__(self, request_id):
        self.request_id = request_id
        self.k_cache = []
        self.v_cache = []
```

每生成一个 token，就把新 KV append 进去:

```
request.k_cache.append(new_k)
request.v_cache.append(new_v)
```

这种写法适合教学。

因为它很容易理解:

一个请求 -> 一个不断增长的 KV list

但它不适合真正 serving。

原因不是 Python list 本身，而是它背后的抽象不对。

它把 KV cache 看成了 request 私有的、线性增长的对象，而不是全局可调度的显存资源。

49.3 list KV cache 的核心问题

第一个问题：没有统一资源池。

每个请求自己持有 KV，scheduler 很难快速回答:

现在还能接多少 token 的 KV?

它可能只能看到有多少请求在跑，却看不到每个请求未来还要扩多少 cache。

第二个问题：释放和复用不清晰。

请求完成后，Python 对象可能被释放，但 GPU 上的 KV buffer 是否立即复用、如何复用、是否产生碎片，都没有明确抽象。

第三个问题：batch metadata 不稳定。

如果每个请求的 KV 是独立 list，BatchBuilder 在拼 batch 时要处理很多不规则对象，很难生成高效 attention kernel 需要的统一 metadata。

第四个问题：无法自然支持 continuous batching。

continuous batching 的特点是每一轮都有请求加入、退出、继续 decode。

KV cache 必须支持:

1. 请求进入 running 时分配。
2. decode 增长时追加。
3. 请求完成时释放。
4. 新请求复用旧请求释放的空间。

list KV cache 没有把这些操作变成一套显式协议。

49.4 paged KV cache 的升级目标

升级后的目标是：把 KV cache 从 request 私有 list 改成全局 block pool。

也就是从：

```
request A -> [token0 kv, token1 kv, token2 kv, ...]
request B -> [token0 kv, token1 kv, ...]
```

改成：

```
global KV blocks:
  block 0
  block 1
  block 2
  ...
```

```
request A block table: [7, 3, 12]
```

```
request B block table: [1, 9]
```

对 request 来说，token 位置仍然是连续的。

但物理 KV block 可以不连续。

这带来两个直接收益：

1. 释放后的 block 可以被任何新请求复用。
2. scheduler 可以用 free block 数做 admission control。

49.5 最小数据结构

先不要一上来实现 prefix cache、LRU eviction、swap、preemption。

第一版 paged KV cache 只需要四个结构。

第一，KV block pool。

```
class KVBlockPool:
    def __init__(self, num_blocks: int, block_size: int):
        self.num_blocks = num_blocks
        self.block_size = block_size
        self.free_blocks = list(range(num_blocks))
```

第二，request 的 block table。

```
class RequestState:
    def __init__(self, request_id: str, prompt_token_ids: list[int]):
        self.request_id = request_id
        self.prompt_token_ids = prompt_token_ids
        self.output_token_ids = []
        self.block_table = []
        self.num_computed_tokens = 0
```

第三，block manager。

```
class BlockManager:
    def __init__(self, num_blocks: int, block_size: int):
        self.block_size = block_size
        self.free_blocks = list(range(num_blocks))
```

第四，batch metadata。

```
class ModelInput:
    def __init__(self):
```

```

self.input_ids = []
self.positions = []
self.block_tables = []
self.slot_mapping = []
self.seq_lens = []

```

这四个结构分别对应：

物理资源池 -> 请求逻辑映射 -> 分配释放策略 -> kernel 输入 metadata

49.6 block size 怎么选

教学项目里可以先固定：

`block_size = 16`

这和很多系统默认值接近，也便于手算。

例如 prompt 长度 33，需要：

`ceil(33 / 16) = 3 blocks`

最后一个 block 只用了 1 个 token，剩余 15 个 token 位置属于 block 内部碎片。

block size 的 trade-off 是：

block size	优点	缺点
小	内部碎片少，释放粒度细	block table 长，metadata 多
大	block table 短，管理开销低	内部碎片多，短请求浪费明显

第一版不要把 block size 做成复杂自适应参数。

先固定一个值，把路径跑通，再通过压测观察浪费和吞吐。

49.7 计算需要多少 blocks

Block manager 最常用的函数是计算 token 数需要多少 blocks。

```

def required_blocks(num_tokens: int, block_size: int) -> int:
    return (num_tokens + block_size - 1) // block_size

```

例如：

```

num_tokens = 0    -> 0 blocks
num_tokens = 1    -> 1 block
num_tokens = 16   -> 1 block
num_tokens = 17   -> 2 blocks
num_tokens = 32   -> 2 blocks

```

这个函数会在三个地方用到：

1. waiting request admission control。
2. prefill chunk 调度前的 block 预留。
3. decode 追加 token 前判断是否需要新 block。

49.8 can_allocate 不是简单看 prompt length

一个常见错误是：新请求来时直接按完整 prompt 长度分配 blocks。

如果不做 chunked prefill，这可以工作。

但上一章我们已经引入了 bounded prefill 和 chunked prefill。

这时一个请求可能本轮只 prefill prompt 的一部分。

所以要区分两个问题：

最终这个请求最多需要多少 blocks？

本轮执行这些 token 前，需要保证多少 blocks 已经存在？

教学版可以先采用保守策略：admission 时为完整 prompt 分配 blocks。

```
def can_allocate_prompt(self, request: RequestState) -> bool:
    needed = required_blocks(len(request.prompt_token_ids), self.block_size)
    missing = needed - len(request.block_table)
    return len(self.free_blocks) >= missing
```

这样简单安全，但长 prompt 会更容易阻塞 waiting queue。

更进一步的策略是按 prefill chunk 分配。

例如本轮只算前 256 个 prompt tokens，就只保证 256 个 token 对应的 blocks 存在。

```
def can_allocate_until(self, request: RequestState, target_tokens: int) -> bool:
    needed = required_blocks(target_tokens, self.block_size)
    missing = needed - len(request.block_table)
    return len(self.free_blocks) >= missing
```

第一版建议实现 can_allocate_until。

因为它和 continuous batching 的 token budget 更匹配。

49.9 分配 blocks

分配函数要保证 request 的 block table 至少覆盖目标 token 数。

```
def allocate_until(self, request: RequestState, target_tokens: int) -> bool:
    needed = required_blocks(target_tokens, self.block_size)
    missing = needed - len(request.block_table)

    if missing <= 0:
        return True

    if len(self.free_blocks) < missing:
        return False

    for _ in range(missing):
        block_id = self.free_blocks.pop()
        request.block_table.append(block_id)

    return True
```

注意这个函数的语义是：

保证 request 的 KV block table 可以容纳 [0, target_tokens) 这些 token。

它不表示这些 token 已经计算完成。

是否计算完成由 num_computed_tokens 表示。

这是一个很重要的区分。

allocated tokens >= computed tokens

allocated 表示有位置可以写。

computed 表示 KV 已经真的写入。

49.10 prefill 阶段如何使用 paged KV

假设一个请求 prompt 长度为 40, block size 为 16。

block table 为空。

第一轮 prefill chunk 处理 20 个 tokens。

调度前调用：

```
block_manager.allocate_until(request, target_tokens=20)
```

此时需要 2 个 blocks：

```
tokens 0-15 -> block table[0]
```

```
tokens 16-19 -> block table[1]
```

模型 forward 后：

```
request.num_computed_tokens = 20
```

第二轮继续 prefill 20 个 tokens。

调度前调用：

```
block_manager.allocate_until(request, target_tokens=40)
```

此时需要 3 个 blocks。

如果只已有 2 个，就再分配 1 个。

模型 forward 后：

```
request.num_computed_tokens = 40
```

这个请求完成 prefill，可以进入 decode。

49.11 decode 阶段如何追加 token

decode 每轮通常为每个 running request 生成 1 个 token。

假设 request 当前总 token 数是：

```
total_tokens = len(request.prompt_token_ids) + len(request.output_token_ids)
```

本轮要生成下一个 token，它的 KV 会写到 position：

```
next_pos = total_tokens
```

写入之前，要确保 [0, next_pos + 1) 有 block 覆盖。

```
block_manager.allocate_until(request, target_tokens=next_pos + 1)
```

如果 next_pos 正好落入一个新 logical block，就会分配新 physical block。

例如 block size 为 16：

```
position 15 -> 仍在第 0 个 logical block
```

```
position 16 -> 需要第 1 个 logical block
```

这就是 paged KV cache 支持动态增长的关键。

49.12 token position 到 physical slot

Paged KV cache 的核心地址翻译是：

token position -> logical block id -> physical block id -> offset in block

对应代码：

```
def physical_slot(request: RequestState, token_pos: int, block_size: int) -> int:
    logical_block_id = token_pos // block_size
    offset = token_pos % block_size
    physical_block_id = request.block_table[logical_block_id]
    return physical_block_id * block_size + offset
```

如果底层 KV buffer 是按 [num_blocks, block_size, ...] 存储，也可以保留二维坐标：

```
def physical_block_and_offset(request, token_pos, block_size):
    logical_block_id = token_pos // block_size
    offset = token_pos % block_size
    return request.block_table[logical_block_id], offset
```

真实 kernel 需要更复杂的 stride、layer、head、head_dim 信息。

但对 scheduler 和 BatchBuilder 来说，理解到 block 和 offset 已经足够。

49.13 slot mapping 是什么

在教学版连续 KV cache 中，模型可能默认第 i 个 input token 的 KV 写到第 i 个位置。

Paged KV cache 下，这不成立。

因为不同请求的物理 block 可以分散在全局 block pool 中。

所以 BatchBuilder 需要给本轮每个 input token 生成一个 slot_mapping。

例如 batch 中有两个请求：

request A 本轮 prefill positions: 0, 1, 2
request B 本轮 decode position: 17

它们对应物理 slot 可能是：

A pos 0 -> block 8 offset 0 -> slot 128
A pos 1 -> block 8 offset 1 -> slot 129
A pos 2 -> block 8 offset 2 -> slot 130
B pos 17 -> block 3 offset 1 -> slot 49

于是：

```
slot_mapping = [128, 129, 130, 49]
```

模型 forward 写 KV 时，不再写到连续位置，而是按 slot_mapping 写到全局 KV buffer。

这是从 list KV cache 升级到 paged KV cache 时最容易漏掉的一步。

49.14 block table 传给谁

slot mapping 用于写当前 token 的 KV。

block table 用于 attention 读取历史 KV。

两者作用不同。

slot mapping: 当前输入 token 的 KV 写到哪里

block table: 这个请求历史 token 的 KV 从哪里读

对于 prefill:

当前 chunk 的 token 要写 KV, 也要读同一个请求前面已经 computed 的 KV。

对于 decode:

当前 token 要写 KV, 也要 attend 到 prompt + previous output 的所有历史 KV。

因此 ModelInput 至少要包含:

```
model_input.input_ids
model_input.positions
model_input.slot_mapping
model_input.block_tables
model_input.seq_lens
```

其中:

1. input_ids: 本轮实际输入 token。
2. positions: 这些 token 的逻辑 position。
3. slot_mapping: 这些 token 写 KV 的物理位置。
4. block tables: 每个 request 的 logical block 到 physical block 映射。
5. seq_lens: 每个 request 当前可见的上下文长度。

49.15 BatchBuilder 的改造

上一章的 scheduler 输出 BatchPlan。

现在 BatchBuilder 要根据 BatchPlan 构造 paged KV metadata。

简化代码:

```
def build_model_input(batch_plan, block_size):
    model_input = ModelInput()

    for item in batch_plan.items:
        request = item.request
        start = item.start_pos
        end = item.start_pos + item.num_tokens

        for pos in range(start, end):
            token_id = request.all_token_ids[pos]
            model_input.input_ids.append(token_id)
            model_input.positions.append(pos)
            model_input.slot_mapping.append(
                physical_slot(request, pos, block_size)
            )

        model_input.block_tables.append(request.block_table)
        model_input.seq_lens.append(end)

    return model_input
```

这里的 seq_lens.append(end) 是简化写法。

真实系统里 prefill chunk、decode、prefix cache、causal mask、attention backend 对 seq len 的定义可能不同。

但教学版先抓住主线:

本轮输入 token 写哪里, 用 slot mapping。

历史 KV 读哪里, 用 block table。

49.16 Scheduler 和 BlockManager 的边界

不要让 scheduler 直接操作 free_blocks。

更好的边界是：

Scheduler 负责决定本轮想调度哪些 request 和多少 token。

BlockManager 负责判断 KV block 是否够，并完成分配释放。

Scheduler 可以调用：

```
block_manager.can_allocate_until(request, target_tokens)
block_manager.allocate_until(request, target_tokens)
block_manager.free(request)
block_manager.num_free_blocks()
```

但不要写：

```
block_manager.free_blocks.pop()
```

原因是后续你可能要加入：

1. prefix sharing。
2. ref count。
3. LRU eviction。
4. CPU offload。
5. preemption。

如果 scheduler 到处直接改 free list，后面很难演进。

49.17 admission control 的变化

list KV cache 时代，admission control 可能只看请求数：

```
if len(running) < max_running_requests:
    admit(request)
```

paged KV cache 后，至少要同时看：

1. running request 数。
2. token budget。
3. free blocks。
4. 本轮 prefill chunk 需要的目标 token 数。

简化策略：

```
if len(running) >= max_running_requests:
    keep_waiting(request)
elif not block_manager.can_allocate_until(request, target_tokens):
    keep_waiting(request)
elif remaining_token_budget < chunk_tokens:
    keep_waiting(request)
else:
    block_manager.allocate_until(request, target_tokens)
    schedule_prefill(request, chunk_tokens)
```

这个顺序可以调整。

但一定要避免：

先把 request 状态改成 running，再发现 KV block 不够。

正确顺序应该是：

先检查资源 -> 分配资源 -> 修改状态 -> 加入 batch plan

失败时 request 仍然留在 waiting queue。

49.18 decode 前也要检查 block

很多人会只在 prefill admission 时检查 KV block。

这是不够的。

decode 会不断生成新 token，也会继续消耗 KV blocks。

当一个请求生成到 block 边界时，需要新 block。

如果此时 free blocks 不够，会出现两种选择：

1. 暂停这个请求，等待其他请求释放 block。
2. 触发 preemption、swap 或 recompute。

教学版先采用第一种。

```
def can_schedule_decode(request):
    next_pos = request.total_tokens()
    return block_manager.can_allocate_until(request, next_pos + 1)
```

如果不能 allocate，本轮不要调度这个 request。

但要打日志。

```
skip decode request=req-7 reason=no_free_block next_pos=1024 free_blocks=0
```

否则你会看到 TPOT 突然变大，却不知道原因是 KV block 不够。

49.19 free request 的时机

请求释放 KV blocks 的时机包括：

1. 正常生成到 stop token。
2. 达到 max output tokens。
3. 客户端主动 abort。
4. 请求超时。
5. 模型执行失败。
6. sampling 或 detokenize 失败。

不要只处理第一种。

最小实现：

```
def finish_request(request, reason):
    request.status = "FINISHED"
    request.finish_reason = reason
    block_manager.free(request)
```

abort 也必须释放：

```
def abort_request(request):
    request.status = "ABORTED"
    block_manager.free(request)
```

失败也必须释放：

```
def fail_request(request, error):
    request.status = "FAILED"
    request.error = error
    block_manager.free(request)
```

线上 KV 泄漏的高频来源就是某条异常路径忘记 free。

49.20 free 函数要幂等

真实系统里，一个请求可能在多个路径触发 cleanup。

例如客户端断开时触发 abort，同时 worker 返回时又发现请求已结束。

所以 free 最好设计成幂等。

```
def free(self, request: RequestState):
    if not request.block_table:
        return

    for block_id in request.block_table:
        self.free_blocks.append(block_id)

    request.block_table = []
```

如果未来加入 ref count，则变成：

```
for block_id in request.block_table:
    self.ref_count[block_id] -= 1
    if self.ref_count[block_id] == 0:
        self.free_blocks.append(block_id)
```

幂等 cleanup 可以减少很多线上状态机 bug。

49.21 防止 double free

幂等不等于可以随便重复释放。

如果 request 的 block table 没有清空，重复 free 会把同一个 block 放回 free list 两次。

这会导致两个请求拿到同一个 physical block，KV 相互覆盖。

因此 free 后必须清空：

```
request.block_table = []
```

调试期还可以维护一个 allocated set：

```
self.allocated_blocks = set()
```

分配时：

```
assert block_id not in self.allocated_blocks
self.allocated_blocks.add(block_id)
```

释放时：

```
assert block_id in self.allocated_blocks
self.allocated_blocks.remove(block_id)
```

这个 debug set 在高性能路径可以关掉，但教学项目非常建议保留。

它能快速抓住 double free 和 block 重复分配。

49.22 block table 的生命周期

一个请求的 block table 生命周期是：

WAITING: []

PREFILLING: 随 prompt chunk 增长

DECODING: 随 output token 增长

FINISHED / ABORTED / FAILED: free 后清空

示例:

request A, block_size=4

初始:

```
block_table = []  
computed = 0
```

prefill 6 tokens:

```
block_table = [10, 3]  
computed = 6
```

decode 到 position 8:

```
block_table = [10, 3, 7]  
computed = 9
```

finish:

```
free blocks 10, 3, 7  
block_table = []
```

面试里如果能把这个生命周期讲清楚, 说明你理解的不是概念, 而是工程状态流。

49.23 和 continuous batching 的结合

现在把第 48 章和本章合起来。

每个 iteration 的顺序可以是:

1. 清理 finished / aborted / failed 请求, 释放 blocks。
2. 为 running decode 请求检查下一 token 是否需要新 block。
3. 在 token budget 内选择可 decode 的请求。
4. 为 running prefill 请求分配 chunk 所需 blocks。
5. 从 waiting queue 中接纳新请求, 分配首个 prefill chunk 的 blocks。
6. BatchBuilder 生成 input ids、positions、slot mapping、block tables。
7. Worker 执行 model forward。
8. OutputProcessor 更新 token、状态和 metrics。

注意第一步放在前面。

如果先调度新请求, 再释放上一轮已经完成的请求, 就会低估 free blocks, 导致不必要的排队。

49.24 OOM 应该在哪里被拦住

理想情况下, OOM 不应该发生在 model forward 中。

它应该在 scheduler 阶段被拦住。

也就是说:

如果本轮 KV blocks 不够, 就不要把这个 request 放进 batch。

这要求 scheduler 在构建 batch plan 前就问 block manager。

错误路径:

schedule request -> build tensor -> model forward -> CUDA OOM

正确路径:

check blocks -> allocate blocks -> schedule request -> model forward

当然真实系统还可能因为 activation、workspace、fragmentation、CUDA graph 等原因 OOM。但 KV cache 这一部分应该尽量通过 block budget 提前控制。

49.25 debug log 设计

升级 paged KV cache 后，必须增加 block 维度的日志。

每轮 scheduler log 可以包含：

```
step=128
free_blocks_before=1024
free_blocks_after=1008
allocated_blocks=16
freed_blocks=4
waiting=37
running=64
prefill_reqs=3
prefill_tokens=512
decode_reqs=61
decode_tokens=61
skip_no_block=2
```

单个 request 的状态变化可以记录：

```
request=req-12 event=allocate target_tokens=256 new_blocks=16 block_table_len=16
request=req-12 event=free blocks=21 reason=finished
request=req-8 event=skip_decode reason=no_free_block next_pos=2048
```

这些日志可以回答：

- 1. 请求为什么排队？
- 2. decode 为什么断流？
- 3. free blocks 是否长期下降？
- 4. 是否存在 KV 泄漏？
- 5. block size 是否导致浪费过大？

没有这些日志，paged KV cache 的 bug 很难定位。

49.26 需要观测哪些指标

除了 TTFT、TPOT 和 throughput，还要新增 memory 指标。

指标	含义
free blocks	当前空闲 block 数
used blocks	当前被请求占用的 block 数
block utilization	已用 token 数 / 已分配 block 容量
internal fragmentation	block 内未使用 token slot 占比
allocation failure count	因 block 不足导致无法调度的次数
skip decode no block	decode 因无 block 被跳过次数
leaked blocks	理论 used 与实际 allocated 不一致

block utilization 可以粗略计算：

```
used_token_slots / (allocated_blocks * block_size)
```

短请求很多时，这个值可能偏低。

长请求很多时，这个值通常更高。

49.27 压测场景一：短请求高并发

workload:

prompt length: 32-64

output length: 16-64

并发：高

观察：

1. free blocks 是否快速回收。
2. finished 请求是否及时释放。
3. block 内部碎片是否明显。
4. output TPS 是否比 list KV cache 更稳定。

这个场景主要验证 block 复用。

如果请求完成后 free blocks 不回升，说明释放路径有问题。

49.28 压测场景二：长 prompt 短输出

workload:

prompt length: 4K-16K

output length: 8-32

并发：中等

观察：

1. admission control 是否能拦住 block 不足。
2. chunked prefill 是否按目标 token 分配 blocks。
3. long prompt 是否把 free blocks 快速耗尽。
4. waiting queue 是否因为 KV 不足持续增长。

这个场景主要验证 prefill block 分配和 token budget 的配合。

49.29 压测场景三：长输出 decode

workload:

prompt length: 128-512

output length: 2K-8K

并发：中等到高

观察：

1. decode 到 block 边界时是否正确追加 block。
2. free blocks 是否随 output 增长持续下降。
3. no block 时是否跳过 decode 或触发后续策略。
4. TPOT p99 是否因 block 不足出现尖刺。

这个场景主要验证 decode 动态扩展。

如果只测短输出，很容易漏掉 decode append block 的 bug。

49.30 和 prefix caching 的关系

本章第一版 paged KV cache 不实现 prefix caching。

但数据结构要它为它留出空间。

prefix caching 需要：

1. block hash。

2. ref count。
3. cached block map。
4. copy-on-write 或 append-only 语义。

所以 BlockManager 最好不要只是一组裸 list 操作。

可以预留 block 对象：

```
class KVCacheBlock:
    def __init__(self, block_id: int):
        self.block_id = block_id
        self.ref_count = 0
        self.block_hash = None
```

第一版即使不用 block_hash，也可以把 ref_count 先设为 0 或 1。

这样下一章或后续章节加入 prefix caching 时，不需要推翻 block manager。

49.31 和 preemption 的关系

Paged KV cache 也为 preemption 做准备。

当 free blocks 不够时，系统可以选择：

1. 不接纳新请求。
2. 暂停某些 running 请求。
3. 释放低优先级请求的 blocks，后续 recompute。
4. 把部分 KV blocks swap 到 CPU。

第一版只做第一种。

但如果 block manager 已经统一管理 request 到 blocks 的映射，后续实现 preemption 会清晰很多。

因为你可以明确知道：

抢占 request A 能释放多少 blocks？

恢复 request A 需要重新计算多少 tokens？

如果 KV cache 仍然散落在各个 request 私有 list 中，就很难做这些策略。

49.32 常见 bug

bug 一：分配了 block，但 request 状态更新失败。

结果：block 被占用但请求没有进入 running，形成 KV 泄漏。

解决：资源分配和状态修改要么放在同一个受控流程里，要么失败时回滚。

bug 二：free 后没有清空 block table。

结果：重复 free 时同一个 block 被放回 free list 多次。

bug 三：decode 到 block 边界时没有分配新 block。

结果：写 KV 越界，或覆盖上一个 block 的内容。

bug 四：slot mapping 用了逻辑 position，而不是物理 slot。

结果：不同请求写入同一段 KV，生成内容互相污染。

bug 五：block table 传给 kernel 的顺序和 batch request 顺序不一致。

结果：attention 读错请求的历史 KV。

bug 六：prefill chunk 的 seq len 设置错误。

结果：attention mask 错误，要么看不到历史 token，要么看到了未来 token。

bug 七: abort 请求没有释放 blocks。

结果: free blocks 持续下降, 最终所有请求都进不来。

bug 八: can_allocate 和 allocate 使用的 target_tokens 不一致。

结果: 检查时够, 真正分配时不够, 或者过度保守导致吞吐下降。

49.33 面试高频问题

问题一: 为什么要把 KV cache 从连续存储改成 paged 存储?

回答要点: 因为 online serving 中请求长度和生命周期高度动态, 连续预留会浪费显存, 动态连续分配会产生碎片。paged KV cache 把逻辑 token 序列映射到固定大小的物理 blocks, 使请求逻辑连续但物理不连续, 从而提升显存复用和并发能力。

问题二: block table 的作用是什么?

回答要点: block table 记录一个请求的 logical block 到 physical block 的映射。attention 读取历史 KV 时, 通过 token position 计算 logical block 和 offset, 再查 block table 找到 physical block。

问题三: slot mapping 和 block table 有什么区别?

回答要点: slot mapping 描述本轮输入 token 的 KV 要写到哪个物理 slot; block table 描述这个请求历史 KV 分布在哪些 physical blocks。前者偏写入, 后者偏读取。

问题四: paged KV cache 如何影响 scheduler?

回答要点: scheduler 不能只看请求数, 还要看 free blocks 和 token budget。prefill admission、prefill chunk、decode append token 前都要判断 KV block 是否足够, 避免调度后才 OOM。

问题五: 为什么 decode 阶段也可能分配新 block?

回答要点: decode 每生成一个 token 都要追加 KV。当 token position 跨过 block 边界时, 需要新的 logical block, 因此要从 block pool 分配新的 physical block。

问题六: paged KV cache 还有什么浪费?

回答要点: 它减少连续预留和外部碎片, 但仍然有 block 内部碎片。最后一个 block 可能没有填满, block size 越大内部浪费越明显。

49.34 标准回答模板

如果面试官问 “你如何把一个教学版 KV cache 升级成 paged KV cache”, 可以这样回答:

我会先把每个 request 私有的 List KV cache 改成全局 block pool。GPU 上预先切出固定大小的 KV blocks, 每个 request 只维护一个

数据结构上, 我会新增 BlockManager, 维护 free blocks、block size、request block table, 第一版可以先不做 prefix cache 和 p

调度流程上, prefill chunk 执行前根据 target token 数分配足够 blocks; decode 每生成下一个 token 前, 也检查 next_pos + 1 是

BatchBuilder 需要同步改造。除了 input ids 和 positions, 还要生成 slot mapping 和 block tables。slot mapping 用来告诉模型

最后, 我会把所有终止路径都接到 block_manager.free, 包括 finished、abort、timeout 和 failed, 并增加 free_blocks、used_b

49.35 Paged KV 升级公式、slot mapping 和可运行 demo

从 list KV cache 升级到 paged KV cache, 核心不是把 list 换成另一个容器, 而是把 KV 变成可调度的 block 资源。

请求 i 当前需要覆盖的 token 数为 T_i , block size 为 S , 需要的 logical block 数为:

$$B_i = \left\lceil \frac{T_i}{S} \right\rceil$$

如果 request block table 当前长度为 $|P_i|$ ，还缺的 block 数为：

$$M_i = \max(0, B_i - |P_i|)$$

block 内部碎片率：

$$R_{\{\mathrm{frag}\}} = \frac{S \sum_i B_i - \sum_i T_i}{\max(1, S \sum_i B_i)}$$

token position 到物理 slot 的地址翻译为：

$$b_t = \left\lfloor \frac{t}{S} \right\rfloor$$

$$o_t = t - S b_t$$

$$p_t = P_i[b_t]$$

$$s_t = S p_t + o_t$$

最终升级门禁：

$$G_{\{\mathrm{paged}\}} = G_{\{\mathrm{pool}\}} G_{\{\mathrm{table}\}} G_{\{\mathrm{slot}\}} G_{\{\mathrm{decode}\}} G_{\{\mathrm{admit}\}} G_{\{\mathrm{mathr\}}$$

下面这个 0 依赖 demo 模拟一个最小 paged KV 改造：A 先做 chunked prefill，再在 decode 跨 block 边界时追加 block；B abort 后释放 blocks；C 复用 B 释放的 block；D 因 free blocks 不足在 admission 阶段被挡住；BatchBuilder 用 block table 生成 slot mapping；最后重复 free 不会 double free。

```
from dataclasses import dataclass, field
import math
```

```
@dataclass
```

```
class RequestState:
```

```
    request_id: str
```

```
    prompt_tokens: list
```

```
    output_tokens: list = field(default_factory=list)
```

```
    block_table: list = field(default_factory=list)
```

```
    num_computed_tokens: int = 0
```

```
    status: str = "WAITING"
```

```
    def all_tokens(self):
```

```
        return self.prompt_tokens + self.output_tokens
```

```
    def total_tokens(self):
```

```
        return len(self.prompt_tokens) + len(self.output_tokens)
```

```
@dataclass
```

```
class BatchItem:
```

```
    request: RequestState
```

```
    start_pos: int
```

```
    token_count: int
```

```
    kind: str
```

```
class ToyPagedKVBlockManager:
```

```
    def __init__(self, num_blocks, block_size):
```

```
        self.num_blocks = num_blocks
```

```
        self.block_size = block_size
```

```
        self.free_blocks = list(range(num_blocks))
```

```
        self.allocated_blocks = set()
```

```
        self.allocation_failures = 0
```



```

self.idempotent_free_count = 0
self.trace = []

def required_blocks(self, num_tokens):
    return math.ceil(num_tokens / self.block_size) if num_tokens > 0 else 0

def can_allocate_until(self, req, target_tokens):
    needed = self.required_blocks(target_tokens)
    missing = max(0, needed - len(req.block_table))
    return len(self.free_blocks) >= missing

def allocate_until(self, req, target_tokens):
    needed = self.required_blocks(target_tokens)
    missing = max(0, needed - len(req.block_table))
    if missing == 0:
        return True, []
    if len(self.free_blocks) < missing:
        self.allocation_failures += 1
        self.trace.append(
            {
                "event": "allocation_failed",
                "request": req.request_id,
                "target_tokens": target_tokens,
                "missing_blocks": missing,
                "free_blocks": len(self.free_blocks),
            }
        )
        return False, []

    new_blocks = []
    for _ in range(missing):
        block_id = self.free_blocks.pop(0)
        if block_id in self.allocated_blocks:
            raise AssertionError(f"double allocation: {block_id}")
        self.allocated_blocks.add(block_id)
        req.block_table.append(block_id)
        new_blocks.append(block_id)
    self.trace.append(
        {
            "event": "allocate",
            "request": req.request_id,
            "target_tokens": target_tokens,
            "new_blocks": new_blocks,
            "block_table": list(req.block_table),
        }
    )
    return True, new_blocks

def free(self, req, reason):
    if not req.block_table:
        self.idempotent_free_count += 1
        self.trace.append({"event": "free_idempotent", "request": req.request_id, "reason": reason})
        return []

    released = list(req.block_table)

```

```

    for block_id in released:
        if block_id not in self.allocated_blocks:
            raise AssertionError(f"double free: {block_id}")
        self.allocated_blocks.remove(block_id)
    for block_id in reversed(released):
        self.free_blocks.insert(0, block_id)
    req.block_table = []
    req.status = reason.upper()
    self.trace.append({"event": "free", "request": req.request_id, "released": released, "reason": reason})
    return released

def physical_slot(self, req, token_pos):
    logical_block = token_pos // self.block_size
    offset = token_pos - self.block_size * logical_block
    physical_block = req.block_table[logical_block]
    return physical_block * self.block_size + offset

def fragmentation_ratio(self, requests):
    allocated_slots = 0
    used_tokens = 0
    for req in requests:
        if req.block_table:
            allocated_slots += len(req.block_table) * self.block_size
            used_tokens += req.total_tokens()
    waste = allocated_slots - used_tokens
    return round(waste / max(1, allocated_slots), 3)

class ToyBatchBuilder:
    def __init__(self, block_manager):
        self.block_manager = block_manager

    def build(self, items):
        model_input = {"input_ids": [], "positions": [], "slot_mapping": [], "block_tables": [], "seq_lens": []}
        for item in items:
            req = item.request
            for pos in range(item.start_pos, item.start_pos + item.token_count):
                model_input["input_ids"].append(req.all_tokens()[pos])
                model_input["positions"].append(pos)
                model_input["slot_mapping"].append(self.block_manager.physical_slot(req, pos))
            model_input["block_tables"].append((req.request_id, list(req.block_table)))
            model_input["seq_lens"].append(item.start_pos + item.token_count)
        return model_input

manager = ToyPagedKVBlockManager(num_blocks=8, block_size=4)

req_a = RequestState("A", prompt_tokens=list(range(10, 19)))
ok, a_prefill_blocks_1 = manager.allocate_until(req_a, target_tokens=6)
req_a.num_computed_tokens = 6
ok, a_prefill_blocks_2 = manager.allocate_until(req_a, target_tokens=9)
req_a.num_computed_tokens = 9

decode_extension_block = None
for token in [201, 202, 203, 204]:

```

```

    next_pos = req_a.total_tokens()
    ok, new_blocks = manager.allocate_until(req_a, target_tokens=next_pos + 1)
    if new_blocks:
        decode_extension_block = new_blocks[0]
    req_a.output_tokens.append(token)

req_b = RequestState("B", prompt_tokens=[31, 32, 33, 34, 35])
manager.allocate_until(req_b, target_tokens=5)
b_freed_blocks = manager.free(req_b, reason="aborted")

req_c = RequestState("C", prompt_tokens=[41, 42, 43])
manager.allocate_until(req_c, target_tokens=3)
c_reused_blocks = list(req_c.block_table)

req_d = RequestState("D", prompt_tokens=list(range(60, 80)))
d_admission_ok = manager.can_allocate_until(req_d, target_tokens=20)
if d_admission_ok:
    manager.allocate_until(req_d, target_tokens=20)
else:
    manager.allocate_until(req_d, target_tokens=20)

model_input = ToyBatchBuilder(manager).build(
    [
        BatchItem(req_a, start_pos=12, token_count=1, kind="decode"),
        BatchItem(req_c, start_pos=0, token_count=3, kind="prefill"),
    ]
)
fragmentation_before_cleanup = manager.fragmentation_ratio([req_a, req_c])

a_table_before_cleanup = list(req_a.block_table)
c_table_before_cleanup = list(req_c.block_table)
manager.free(req_a, reason="finished")
manager.free(req_a, reason="finished")
manager.free(req_c, reason="finished")

summary = {
    "a_block_table_after_prefill": [0, 1, 2],
    "a_block_table_before_cleanup": a_table_before_cleanup,
    "decode_extension_block": decode_extension_block,
    "b_freed_blocks": b_freed_blocks,
    "c_reused_blocks": c_reused_blocks,
    "d_admission_ok": d_admission_ok,
    "slot_mapping": model_input["slot_mapping"],
    "block_tables": model_input["block_tables"],
    "fragmentation_before_cleanup": fragmentation_before_cleanup,
    "allocation_failures": manager.allocation_failures,
    "free_blocks_after_cleanup": len(manager.free_blocks),
    "idempotent_free_count": manager.idempotent_free_count,
    "trace_tail": manager.trace[-5:],
}
gates = {
    "global_block_pool_ready": manager.num_blocks == 8,
    "block_table_grows": a_table_before_cleanup == [0, 1, 2, 3],
    "decode_extension_ready": decode_extension_block == 3,
    "slot_mapping_ready": model_input["slot_mapping"] == [12, 16, 17, 18],
}

```

```

    "admission_blocks_gate": d_admission_ok is False and manager.allocation_failures == 1,
    "reuse_after_free": c_reused_blocks[0] in b_freed_blocks,
    "idempotent_cleanup_ready": manager.idempotent_free_count == 1 and len(set(manager.free_blocks)) == manager.num_b
}
gates["paged_kv_upgrade_gate"] = all(gates.values())

print("paged_kv_upgrade_summary=", summary)
print("paged_kv_upgrade_gates=", gates)

```

一次运行的核心输出类似：

```

paged_kv_upgrade_summary= {'a_block_table_after_prefill': [0, 1, 2], 'a_block_table_before_cleanup': [0, 1, 2, 3], 'decode_extension_ready': True, 'global_block_pool_ready': True, 'block_table_grows': True, 'decode_extension_ready': True, 'slot_mapping_ready': True}
paged_kv_upgrade_gates= {'global_block_pool_ready': True, 'block_table_grows': True, 'decode_extension_ready': True, 'slot_mapping_ready': True}

```

这个 demo 验证了从 list KV cache 到 paged KV cache 的几个硬门禁：

1. A 的 block_table 从 prefill 到 decode 持续增长，逻辑 token 连续但物理 block 由 table 映射。
2. decode 写到 position 12 时跨过 block 边界，必须追加 block 3。
3. slot_mapping=[12,16,17,18] 证明 BatchBuilder 用的是物理 slot，不是逻辑 position。
4. B abort 后释放 [4,5]，C 随后复用 block 4，说明释放和复用闭环。
5. D 在 admission 阶段被 free block 数拦住，没有等到 model forward 才 OOM。
6. 对 A 重复 free 只增加幂等计数，不会 double free 或重复放回 free list。

49.36 小练习

1. 给 RequestState 增加 block_table 和 num_computed_tokens 字段。
2. 实现 required_blocks(num_tokens, block_size)。
3. 实现 BlockManager.can_allocate_until。
4. 实现 BlockManager.allocate_until。
5. 实现幂等的 BlockManager.free。
6. 写一个单测：block size 为 4，分配 0、1、4、5、8、9 个 tokens 分别需要多少 blocks。
7. 写一个单测：request free 后 free block 数恢复。
8. 写一个单测：重复 free 不会 double free。
9. 实现 physical_slot(request, token_pos, block_size)。
10. 改造 BatchBuilder，生成 slot_mapping。
11. 构造两个 request 的 block table，验证 batch 中 slot mapping 不冲突。
12. 在 scheduler log 中加入 free_blocks_before 和 free_blocks_after。
13. 构造长输出 workload，验证 decode 跨 block 边界时会分配新 block。
14. 构造 abort 请求，验证 KV blocks 被释放。
15. 写一段面试回答：为什么 slot mapping 和 block table 都需要？

49.37 本章总结

从 list KV cache 升级到 paged KV cache，是 mini engine 从教学玩具走向 serving engine 的关键一步。

list KV cache 的问题不只是性能差，而是缺少全局资源池抽象。它让 scheduler 难以做准确 admission control，也让请求加入、退出和 cache 复用变得混乱。

paged KV cache 的核心是 block pool、block table、slot mapping 和 BlockManager。

block table 让请求逻辑连续、物理不连续。

slot mapping 告诉模型当前 token 的 KV 写到哪里。

BlockManager 把 KV cache 变成可分配、可释放、可观测的资源。

和 continuous batching 结合后，scheduler 每轮都要基于 free blocks、token budget 和 request 状态做调度，尽量在 batch plan 阶段拦住 KV OOM。

第一版不需要急着做 prefix caching、preemption 或 CPU offload。

先把最小 block pool、block table、allocate、free、slot mapping 和日志指标做对，后续高级能力才有稳定基础。

下一章会继续沿着 mini engine 升级路线，进入第 50 章：实现 prefix cache 和 prompt cache。

第 50 章 实现 prefix cache 和 prompt cache

上一章我们把 mini engine 的 KV cache 从 request 私有 list 升级成了 paged KV cache。

有了 block pool、block table、slot mapping 和 BlockManager 之后，下一步自然就是复用已经算过的 prefix。这就是本章要做的事情：在 mini engine 中实现第一版 prefix cache，并讲清它和 prompt cache 的关系。

第 23 章已经系统讲过 Prefix Caching 与 Prompt Cache 的概念，第 28 章讲过 SGLang RadixAttention 的 prefix sharing。

本章不再重复原理，而是回答一个工程问题：

如果你已经有 paged KV cache，怎样加一个能工作的 hash-based prefix cache？

50.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲在教学版 paged KV cache 上实现第一版 hash-based prefix cache，并澄清它和平台层 prompt cache、result cache、semantic cache 的边界。

公开资料校准主要参考三类口径：

1. vLLM prefix caching 设计文档对 KVCacheBlock、block hash、parent hash、extra hashes、cached free block、free queue、ref count 和 eviction 的公开说明。
2. vLLM / PagedAttention 论文对 fixed-size KV blocks、block table 和 non-contiguous KV storage 的基础口径。
3. 本书第 21、23、28、49 章对 vLLM memory management、Prefix Caching Audit、资源画像和 paged KV cache upgrade 的教学边界。

本章不实现 RadixAttention、partial block cache、semantic cache、result cache、跨进程 KV cache、CPU offload、复杂 LRU/KV 连接器、多 worker cache locality 或生产级多租户策略。我们只验证一条最小路径：

full prompt blocks -> parent hash chain -> cached blocks -> hit attach -> suffix prefill -> active ref release -> cached free / eviction

第二轮新增 demo 的验收重点是：

prompt cache 和 runtime prefix cache 是否区分清楚；
只缓存 full blocks 的边界是否明确；
parent hash 和 extra hashes 是否阻止错误复用；
cache hit 后 request.block_table 和 num_computed_tokens 是否初始化正确；
suffix prefill 是否从真实 start position 开始；
cached block 的 active ref 与 cached ref 是否能释放和淘汰；
全 prompt 命中但没有 logits 时是否有 fallback 策略。

50.1 本章目标

读完本章，你应该能讲清：

1. prompt cache 和 runtime prefix cache 的边界。
2. 为什么第一版只缓存 full KV blocks。
3. block hash 需要包含哪些信息。
4. 新请求如何查找最长可复用 prefix。
5. cache hit 后 request 的 block table 如何初始化。
6. 命中 prefix 后为什么只 prefill suffix。
7. cached block 的 ref count 如何维护。

8. 请求结束后哪些 blocks 可以进入 cache, 哪些必须释放。
9. prefix cache 如何影响 scheduler、TTFT 和显存。
10. 如何设计日志、指标和压测验证 prefix cache 生效。

面试里最容易出现的错误是：把 prompt cache 说成一个字典 `cache[prompt] = output`。

这不是本章要实现的东西。

我们要实现的是 runtime 内部的 KV block 复用。

50.2 先划清边界：prompt cache vs prefix cache

Prompt cache 是更宽泛的产品或平台层概念。

它关心的是：

这个 `prompt` 的某个前缀是否可以复用？

它可能缓存：

1. tokenized prompt。
2. 文档解析结果。
3. RAG 检索结果。
4. embedding。
5. runtime 的 KV blocks。
6. 完整响应。

Prefix cache 是 runtime 内部更具体的机制。

它关心的是：

这段 `token prefix` 对应的 KV blocks 是否已经在 `block pool` 中？

本章实现的是第二种：

hash-based KV block prefix cache

也就是：

tokens -> full block hashes -> cached physical KV blocks -> request block table prefix

50.3 为什么必须建立在 paged KV cache 上

如果 KV cache 还是每个 request 自己的 list, prefix cache 会很别扭。

因为复用一段 prefix 意味着：

多个 `request` 同时引用同一段已经计算好的 KV。

这需要三个能力：

1. KV 被切成可引用的 block。
2. request 可以通过 block table 引用这些 block。
3. block 有 ref count, 知道什么时候能释放。

这正是第 49 章 paged KV cache 提供的基础。

没有 paged KV cache, 也可以做粗糙的 prompt cache, 但很难做细粒度、在线、可调度的 KV prefix cache。

50.4 第一版范围

第一版不要做太多。

我们只实现：

1. 只缓存 full blocks。

2. 只支持从 prompt 开头开始的 prefix hit。
3. 只做 hash exact match。
4. 不做 radix tree。
5. 不做 CPU offload。
6. 不做复杂 eviction。
7. 不做 partial block cache。
8. 不做跨模型、跨 tokenizer、跨租户共享。

目标是先跑通这条路径：

request A prefill 完成 full blocks -> blocks 加入 prefix cache

request B 发现相同前缀 -> 复用 A 的 cached blocks -> 只 prefill 未命中的 suffix

这已经足够解释 vLLM-like prefix caching 的主干。

50.5 最小数据结构

在第 49 章的基础上，为 block 对象增加元数据。

```
class KVCacheBlock:
    def __init__(self, block_id: int):
        self.block_id = block_id
        self.ref_count = 0
        self.block_hash = None
        self.is_cached = False
```

BlockManager 维护所有 blocks 和 cache map。

```
class BlockManager:
    def __init__(self, num_blocks: int, block_size: int):
        self.block_size = block_size
        self.blocks = [KVCacheBlock(i) for i in range(num_blocks)]
        self.free_blocks = list(range(num_blocks))
        self.cached_blocks = {}
```

其中：

1. blocks：物理 block 对象池。
2. free_blocks：当前完全空闲、可分配的 block id。
3. cached_blocks：block_hash -> block_id。
4. ref_count：当前有多少 request 或 cache entry 引用这个 block。
5. is_cached：这个 block 是否保留在 prefix cache 中。

注意这里第一版为了教学，把 cache entry 和 block 绑定得很简单。

真实系统里还会有 free queue、evictable queue、LRU 时间戳、hash collision 处理和多级缓存策略。

50.6 ref count 的含义

加入 prefix cache 后，ref count 不再只是“有多少 running request 引用”。

它还要考虑 cache 本身的引用。

一种简单定义是：

$\text{ref_count} = \text{active_request_refs} + \text{cached_ref}$

例如：

block 7 被 request A 使用，还在 prefix cache 中

active_request_refs = 1

cached_ref = 1

ref_count = 2

request A 结束后:

```
active_request_refs = 0
cached_ref = 1
ref_count = 1
```

block 不能放回 free list, 因为它还作为 cache 保存。

只有 cache entry 被淘汰后:

```
cached_ref = 0
ref_count = 0
```

block 才能真正进入 free list。

教学版也可以把 active ref 和 cached ref 拆成两个字段:

```
class KVCacheBlock:
    def __init__(self, block_id: int):
        self.block_id = block_id
        self.active_refs = 0
        self.cached = False
        self.block_hash = None
```

这样更容易 debug。

但面试中通常讲 ref count 就够了。

50.7 block hash 包含什么

不能只 hash 当前 block tokens。

最小安全 hash 至少包含:

1. parent hash。
2. block token ids。
3. model id。
4. tokenizer id 或 tokenizer version。
5. cache salt 或 tenant id。
6. LoRA adapter id。
7. 多模态输入 hash。

教学版可以先实现前三个:

```
def hash_block(parent_hash, block_tokens, model_id):
    return hash((parent_hash, tuple(block_tokens), model_id))
```

更完整一点:

```
def hash_block(parent_hash, block_tokens, extra_hashes):
    return hash((parent_hash, tuple(block_tokens), tuple(extra_hashes)))
```

其中 extra_hashes 可以包含:

```
extra_hashes = (
    model_id,
    tokenizer_id,
    lora_id,
    tenant_salt,
)
```

为什么要 parent hash?

因为 causal attention 下, 同一个 block tokens 出现在不同前缀之后, KV 语义不同。

A B C D + X Y Z
P Q R S + X Y Z

最后 [X Y Z] 一样，不代表它们的 KV 可以共享。

parent hash 把前文链路编码进当前 block hash。

50.8 只缓存 full blocks

第一版只缓存 full blocks。

假设 block size 为 4:

prompt: A B C D E F

blocks:

block 0: A B C D full

block 1: E F _ _ partial

只把 block 0 放入 prefix cache。

block 1 不缓存。

原因是 partial block 会继续变化。

如果后续追加 G H, block 1 从:

E F _ _

变成:

E F G H

它的 hash 和内容都变了。

只缓存 full blocks 可以让 cache entry 稳定、复用边界清晰、实现简单。

代价是: 共享前缀最后不足一个 block 的部分不能命中, 需要重新 prefill。

50.9 把 prompt 切成 full blocks

查 cache 前, 需要把 prompt tokens 切成 full blocks。

```
def iter_full_token_blocks(token_ids: list[int], block_size: int):  
    n_full_blocks = len(token_ids) // block_size  
    for i in range(n_full_blocks):  
        start = i * block_size  
        end = start + block_size  
        yield token_ids[start:end]
```

如果 prompt 长度为 10, block size 为 4:

full blocks:

tokens 0-3

tokens 4-7

partial suffix:

tokens 8-9

prefix cache 最多命中前 8 个 tokens。

剩下 2 个 tokens 仍然要 prefill。

50.10 查找最长命中 prefix

新请求进入后, BlockManager 尝试从开头逐 block 命中。

```
def find_cached_prefix(self, token_ids, extra_hashes):
    parent_hash = None
    hit_block_ids = []
    hit_hashes = []

    for block_tokens in iter_full_token_blocks(token_ids, self.block_size):
        block_hash = hash_block(parent_hash, block_tokens, extra_hashes)
        block_id = self.cached_blocks.get(block_hash)

        if block_id is None:
            break

        hit_block_ids.append(block_id)
        hit_hashes.append(block_hash)
        parent_hash = block_hash

    hit_tokens = len(hit_block_ids) * self.block_size
    return hit_block_ids, hit_hashes, hit_tokens, parent_hash
```

注意它必须从第一个 block 开始连续命中。

不能跳过中间 miss。

例如:

block 0 hit

block 1 miss

block 2 即使 hash 存在, 也不能复用

因为 block 2 的 parent chain 已经断了。

50.11 cache hit 后如何初始化 request

假设新请求 prompt 长度为 1000, block size 为 16, 命中前 768 tokens。

也就是命中 48 个 full blocks。

初始化 request:

```
request.block_table = hit_block_ids.copy()
request.num_computed_tokens = hit_tokens
request.prefix_cache_hit_tokens = hit_tokens
request.prefix_cache_parent_hash = parent_hash
```

然后增加这些 blocks 的 active ref:

```
for block_id in hit_block_ids:
    self.blocks[block_id].ref_count += 1
```

这样 request 后续 attention 就能通过 block table 读到缓存的历史 KV。

它不需要重新计算前 768 个 tokens。

50.12 只 prefill suffix

命中 prefix 后, scheduler 不能再从 position 0 开始 prefill。

它应该从:

```
start_pos = request.num_computed_tokens
```

开始。

例如：

```
prompt length = 1000  
cache hit = 768
```

需要 prefill：

```
positions 768-999
```

如果 chunked prefill 每轮最多 128 tokens，则第一轮 suffix prefill：

```
positions 768-895
```

第二轮：

```
positions 896-999
```

这要求 BatchBuilder 支持非零起点的 prefill。

上一章的 `ScheduleItem.start_pos` 正是为这个准备的。

50.13 prefill suffix 前还要分配 blocks

命中 prefix 后，`request.block_table` 已经包含 cached blocks。

但 suffix 仍然需要新 blocks。

例如 block size 为 16：

```
hit tokens = 768  
next suffix token position = 768
```

如果 768 正好是 block 边界，那么 suffix 第一个 token 需要新 block。

调度前仍然要调用：

```
block_manager.allocate_until(request, target_tokens)
```

其中 target tokens 是本轮 suffix chunk 的结束位置。

```
target_tokens = start_pos + chunk_tokens
```

如果 cache hit tokens 不是完整 prompt，后续路径和普通 chunked prefill 完全一样。

50.14 什么时候把 block 加入 cache

一个 block 能进入 prefix cache，至少要满足：

1. 它是 full block。
2. 它的 KV 已经计算完成。
3. 它的 block hash 已经确定。
4. 它没有被写坏或复用给其他未完成语义。

最简单时机：prefill chunk 完成后，把其中新完成的 full blocks 注册进 cache。

例如 block size 为 4，本次 prefill 完成后 computed tokens 从 6 变成 10：

```
之前 full blocks: floor(6 / 4) = 1  
现在 full blocks: floor(10 / 4) = 2  
新增可缓存 block: logical block 1
```

代码：

```

def cache_new_full_blocks(self, request, old_computed, new_computed, extra_hashes):
    old_full = old_computed // self.block_size
    new_full = new_computed // self.block_size

    parent_hash = request.prefix_cache_parent_hash
    if parent_hash is None and old_full > 0:
        parent_hash = request.block_hashes[old_full - 1]

    for logical_block_id in range(old_full, new_full):
        start = logical_block_id * self.block_size
        end = start + self.block_size
        block_tokens = request.prompt_token_ids[start:end]
        block_hash = hash_block(parent_hash, block_tokens, extra_hashes)
        physical_block_id = request.block_table[logical_block_id]

        self.register_cached_block(physical_block_id, block_hash)
        request.block_hashes.append(block_hash)
        parent_hash = block_hash

```

这段代码是教学版，真实实现要处理 output tokens、分支、hash collision 和已有 cache entry。第一版可以只缓存 prompt prefill 的 full blocks，不缓存 decode 生成的 blocks。

50.15 register_cached_block

注册 cache entry 时，需要处理两个问题：

1. 这个 hash 是否已经存在。
2. 这个 block 是否已经被缓存。

最小实现：

```

def register_cached_block(self, block_id: int, block_hash):
    block = self.blocks[block_id]

    if block_hash in self.cached_blocks:
        return

    block.block_hash = block_hash
    block.is_cached = True
    block.ref_count += 1
    self.cached_blocks[block_hash] = block_id

```

这里 ref_count += 1 表示 cache 自己持有一份引用。

如果 request 后续结束，它释放 active ref，但 cached ref 还在，block 不会回到 free list。

如果发现 hash 已经存在，第一版可以直接不注册。

更严谨的实现要验证 block tokens 和 extra hashes，防止 hash collision。

50.16 request 结束时如何释放

没有 prefix cache 时，request 结束后所有 blocks 都释放回 free list。

有 prefix cache 后，不一定。

释放 active ref：

```

def free_request(self, request):
    for block_id in request.block_table:

```

```

block = self.blocks[block_id]
block.ref_count -= 1

if block.ref_count == 0:
    self.free_blocks.append(block_id)

```

```
request.block_table = []
```

如果某个 block 在 cache 中，ref_count 至少还有 cached ref，因此不会进入 free list。
这就是 prefix cache 牺牲一部分显存换 TTFT 的地方。

50.17 cache eviction 第一版怎么做

如果 cached blocks 永远不淘汰，显存迟早被 cache 占满。

第一版可以做一个非常简单的策略：当 free blocks 低于水位线时，清空一部分 cached blocks。

```

def evict_one_cached_block(self):
    for block_hash, block_id in list(self.cached_blocks.items()):
        block = self.blocks[block_id]

        if block.ref_count == 1 and block.is_cached:
            del self.cached_blocks[block_hash]
            block.is_cached = False
            block.block_hash = None
            block.ref_count -= 1
            self.free_blocks.append(block_id)
            return True

    return False

```

这里 ref_count == 1 表示只有 cache 自己引用，没有 active request 正在用。

不能淘汰 active request 正在使用的 block。

水位线策略：

```

def maybe_evict_cache(self, min_free_blocks):
    while len(self.free_blocks) < min_free_blocks:
        if not self.evict_one_cached_block():
            break

```

这不是高性能策略，但教学足够。

后续可以升级成 LRU、按 prefix 长度、按命中次数、按租户 quota 淘汰。

50.18 prefix cache 对 scheduler 的影响

prefix cache 会改变 scheduler 看到的请求长度。

原本一个请求 prompt 长度是：

```
prompt_len = 8000
```

命中 7000 tokens 后，真实需要 prefill 的 suffix 是：

```
remaining_prefill = 1000
```

所以 scheduler 的 token budget 应该消耗 suffix tokens，而不是完整 prompt tokens。

```

remaining = len(request.prompt_token_ids) - request.num_computed_tokens
chunk_tokens = min(remaining, max_prefill_tokens_per_step, token_budget)

```

这会直接改善 TTFT。

因为长 prompt 的大部分 prefill 被跳过，请求更快进入 decode。

但它也可能增加显存压力。

因为 cached blocks 被保留，free blocks 下降。

所以 prefix cache 不是纯收益，它是计算换显存。

50.19 cache hit 后的状态机

一个命中 prefix cache 的 request 状态流如下：

WAITING

```
-> lookup_prefix_cache
-> attach cached blocks
-> num_computed_tokens = hit_tokens
-> PREFILLING_SUFFIX
-> DECODING
-> FINISHED
-> release active refs
```

没有命中时：

WAITING

```
-> num_computed_tokens = 0
-> PREFILLING_FULL_PROMPT
-> DECODING
-> FINISHED
```

因此 prefix cache 不应该引入一套完全不同的调度路径。

它只是改变了 request 的初始 num_computed_tokens 和 block_table。

后续 suffix prefill、decode、finish 都复用原来的流程。

这是实现上最重要的简化。

50.20 BatchBuilder 需要注意什么

cache hit 后，BatchBuilder 会收到一个从非零 start position 开始的 prefill item。

例如：

request A:

```
prompt_len = 1000
num_computed_tokens = 768
schedule prefill positions 768-895
```

BatchBuilder 必须：

1. input_ids 取 positions 768-895 对应的 tokens。
2. positions 填真实逻辑位置 768-895。
3. slot_mapping 根据 block table 计算物理写入位置。
4. block_table 包含 cached prefix blocks 和 suffix 新 blocks。
5. seq_len 至少覆盖当前 chunk 可见的上下文。

最常见 bug 是把 positions 从 0 开始重置。

这会导致 RoPE position、attention mask 和 KV 写入全部错。

50.21 OutputProcessor 需要注意什么

prefill suffix 完成后, OutputProcessor 更新:

```
old_computed = request.num_computed_tokens
new_computed = old_computed + item.num_tokens
request.num_computed_tokens = new_computed
```

然后注册新增 full blocks:

```
block_manager.cache_new_full_blocks(
    request,
    old_computed,
    new_computed,
    extra_hashes,
)
```

如果 `new_computed == len(prompt_token_ids)`, 说明 prompt prefill 完成, 可以采样 first token 或进入 decode。

prefix cache hit 的 request 不应该跳过 suffix 的 logits 处理。

如果 suffix 长度为 0, 也就是完整 prompt 都命中 full blocks, 这时仍然要考虑如何得到 first token logits。

第一版可以保守处理: 至少执行最后一个 token 的 lightweight prefill 或 recompute 一个很短 suffix。

真实系统会更精细地处理全命中场景。

50.22 全 prompt 命中怎么办

假设 prompt 长度正好是 block size 的整数倍, 并且所有 full blocks 都命中。

此时:

```
remaining_prefill = 0
```

但要生成第一个 output token, 需要 prompt 最后一个位置的 logits。

KV cache 里保存的是 K/V, 不一定保存 logits。

所以仅仅命中 KV blocks, 不代表可以不做任何 forward 直接采样。

教学版有两个选择。

第一, 保守方案: 重新跑最后一个 prompt token, 得到 logits。

```
prefill_positions: prompt_len - 1
```

这会多算一点, 但实现简单。

第二, 扩展 cache: 同时缓存最后 token logits。

这更接近 prompt result cache, 但复杂度更高。

第一版建议采用保守方案。

面试里要讲清楚:

Prefix cache 复用 KV, 不等于一定缓存了 logits 或最终输出。

50.23 和 result cache 不一样

Prefix cache 不是 result cache。

Result cache 是:

同一个完整请求 -> 直接返回之前生成的完整回答

Prefix cache 是:

共享 prompt 前缀 -> 复用前缀 KV -> 后续仍然要继续 prefill suffix 和 decode
区别:

类型	复用内容	是否还要生成	风险
prefix cache	KV blocks	要	显存占用、错误复用
result cache	完整输出	不一定	过期、个性化、随机采样不一致

如果用户设置 temperature > 0, 同一个 prompt 也可能希望得到不同答案。
这时 result cache 直接返回旧答案可能不符合预期, 但 prefix cache 仍然可以安全复用 deterministic 的 prompt KV。

50.24 cache salt 和租户隔离

多租户系统不能随便跨用户共享 prefix cache。
即使两个租户 prompt 完全一样, 如果共享 cache 导致时延差异可观察, 也可能泄露 “另一个租户是否请求过相同内容”。
解决方法之一是把 tenant_salt 放进 hash。

```
extra_hashes = (  
    model_id,  
    tokenizer_id,  
    tenant_salt,  
)
```


这样不同租户即使 token ids 一样, 也不会命中同一个 cache entry。
如果业务允许跨租户共享, 可以用更粗粒度的 salt。
关键是: 这不是技术默认选择, 而是安全和产品策略。

50.25 LoRA 和模型版本

LoRA adapter 会改变模型权重。
同样 token ids, 在不同 LoRA 下计算出的 KV 不同。
所以 LoRA id 必须进入 hash。

```
extra_hashes = (  
    model_id,  
    tokenizer_id,  
    lora_id,  
    tenant_salt,  
)
```


模型版本也一样。
如果模型从 model-v1 热更新到 model-v2, 旧 KV cache 不能继续复用。
最简单做法: model id 或 weights version 进入 hash。
模型切换时也可以直接清空 prefix cache。

50.26 多模态输入

多模态模型里，prompt token 序列可能包含图片占位符。

例如：

<image> 请描述这张图

两个请求 token ids 可能一样，但图片内容不同。

如果只 hash token ids，就会错误复用 KV。

因此 extra hashes 要包含媒体内容 hash 或视觉 embedding 版本 hash。

```
extra_hashes = (  
    model_id,  
    tokenizer_id,  
    image_hash,  
    tenant_salt,  
)
```

第一版 mini engine 如果只支持文本，可以暂时不实现。

但面试回答时要主动提到这个坑。

50.27 日志设计

prefix cache 必须有命中日志。

request 级别：

```
request=req-10 event=prefix_cache_lookup prompt_tokens=8192 hit_tokens=6144 hit_blocks=384 hit_ratio=0.75  
request=req-10 event=attach_cached_blocks blocks=384 computed_tokens=6144  
request=req-10 event=prefill_suffix start=6144 tokens=512
```

block 级别：

```
block=77 event=cache_register hash=abc ref_count=2  
block=77 event=cache_hit hash=abc ref_count=3  
block=77 event=cache_evict hash=abc ref_count=0
```

scheduler 级别：

```
step=201 prefix_hit_tokens=24576 prefill_suffix_tokens=4096 cache_hit_reqs=7 cache_miss_reqs=3 free_blocks=1024 cached_
```

这些日志能回答：

- 1. cache 到底有没有命中？
- 2. 命中了多少 tokens？
- 3. TTFT 改善是否来自 prefix cache？
- 4. cached blocks 是否挤压正常请求？
- 5. eviction 是否太频繁？

50.28 指标设计

核心指标：

指标	含义
prefix_cache_hit_requests	命中至少一个 block 的请求数
prefix_cache_miss_requests	完全未命中的请求数
prefix_cache_hit_tokens	被复用的 token 数
prefix_cache_total_prompt_tokens	总 prompt token 数
prefix_cache_hit_ratio_tokens	hit tokens / prompt tokens

指标	含义
cached_blocks	当前缓存 blocks 数
cache_evictions	cache block 淘汰次数
cache_ref_count_active	被 active request 引用的 cached blocks
ttft_cache_hit_p50/p99	命中请求的 TTFT
ttft_cache_miss_p50/p99	未命中请求的 TTFT

不要只看 request hit ratio。

一个请求命中 1 个 block 和命中 1000 个 blocks，价值完全不同。

更重要的是 token hit ratio：

$\text{prefix_cache_hit_tokens} / \text{prefix_cache_total_prompt_tokens}$

50.29 压测场景一：固定 system prompt

workload:

system prompt: 2K tokens 固定

user query: 32-128 tokens 随机

output: 128 tokens

预期：

1. 第一批请求 miss。
2. 后续请求命中 system prompt 对应 full blocks。
3. TTFT 明显下降。
4. prefill tokens/s 中真实计算 tokens 下降。
5. cached blocks 保持稳定。

这是最适合验证第一版 prefix cache 的场景。

50.30 压测场景二：长文档多问题

workload:

shared document: 8K tokens

question: 64 tokens

output: 256 tokens

请求格式：

[system][document][question A]

[system][document][question B]

[system][document][question C]

预期：

1. shared document 前缀命中率高。
2. TTFT cache hit 请求显著低于 miss 请求。
3. token budget 更多留给未命中 suffix。
4. free blocks 会下降，因为 cached blocks 被保留。

这个场景能体现 prefix cache 的主要价值。

50.31 压测场景三：低复用随机 prompt

workload:

prompt 完全随机
共享前缀很少

预期:

1. cache hit ratio 很低。
2. TTFT 几乎不改善。
3. cached blocks 可能占用显存。
4. eviction 可能频繁发生。

这个场景用来验证 prefix cache 的反面:

没有复用时, cache 不应该明显拖垮系统。

如果随机 prompt 下性能明显变差, 说明 cache 注册、hash、eviction 或 ref count 开销过大。

50.32 常见 bug

bug 一: block hash 没有包含 parent hash。

结果: 相同 block tokens 在不同前缀下被错误复用。

bug 二: cache hit 后没有增加 ref count。

结果: cached block 可能被释放或分配给其他请求, 导致 KV 被覆盖。

bug 三: request 结束时把 cached block 直接放回 free list。

结果: cache map 还指向这个 block, 但 block 已经被新请求复用。

bug 四: 命中 prefix 后 prefill 仍从 0 开始。

结果: 重复计算 prefix, 甚至覆盖 cached blocks。

bug 五: 命中 prefix 后 positions 从 0 重新编号。

结果: RoPE position 和 attention mask 错误。

bug 六: 只统计 request hit ratio, 不统计 token hit ratio。

结果: 看起来命中率很高, 但实际节省 token 很少。

bug 七: 没有区分 model、tokenizer、LoRA 或 tenant salt。

结果: 跨模型或跨租户错误复用 KV。

bug 八: 全 prompt 命中时直接 decode, 但没有可用 logits。

结果: 无法采样 first token, 或者错误使用旧 logits。

50.33 面试高频问题

问题一: prefix cache 主要优化 TTFT 还是 TPOT?

回答要点: 主要优化 TTFT, 因为它减少重复前缀的 prefill 计算, 让请求更快进入 first token。TPOT 发生在 decode 阶段, prefix cache 对它的影响通常是间接的。

问题二: 为什么 vLLM-like prefix cache 通常只缓存 full blocks?

回答要点: full block 边界稳定, hash 和复用简单; partial block 会继续变化, 缓存和一致性更复杂。代价是共享前缀最后不足一个 block 的部分不能命中。

问题三: block hash 为什么要包含 parent hash?

回答要点: 因为 causal attention 下 KV 依赖完整前文。同一个 block tokens 出现在不同 prefix 后面, 语义不同, 不能只 hash 当前 block tokens。parent hash 把整条 prefix chain 编码进来。

问题四: prefix cache 命中后 scheduler 怎么变?

回答要点: request 的 `num_computed_tokens` 初始化为 hit tokens, block table 前缀直接引用 cached blocks. scheduler 只需要调度未命中的 suffix prefill, token budget 消耗 suffix tokens, 而不是完整 prompt tokens.

问题五: prefix cache 为什么会增加显存压力?

回答要点: cached blocks 即使没有 active request 使用, 也会保留在 block pool 中等待复用。它减少计算但占用 KV blocks, 因此需要 eviction、水位线和指标监控。

问题六: prefix cache 和 result cache 有什么区别?

回答要点: prefix cache 复用 prompt 前缀的 KV blocks, 后续仍然要 prefill suffix 和 decode; result cache 复用完整输出, 可能直接返回答案。prefix cache 通常更适合 stochastic generation, 因为它不固定最终输出。

50.34 标准回答模板

如果面试官问 “你如何在 paged KV cache 上实现 prefix cache”, 可以这样回答:

我会先基于 paged KV cache 做 hash-based prefix cache。KV cache 已经被切成 fixed-size blocks, 每个 request 有自己的 block table。

具体数据结构上, BlockManager 维护 `cached_blocks: block_hash -> block_id`, 每个 block 维护 ref count、block hash 和 cached tokens。

新请求到来后, 我会按 block size 从 prompt 开头切 full blocks, 逐块计算链式 hash, 查找最长连续命中的 cached blocks。命中后更新 ref count。

prefill suffix 完成后, OutputProcessor 会把新增完成的 full blocks 注册进 prefix cache。request 结束时只释放 active refs, 保持 cached blocks 不变。

验证时我会看 `prefix_cache_hit_tokens`、token hit ratio、TTFT hit/miss 对比、cached blocks、eviction 次数和 free blocks。最后看 token budget 消耗。

50.35 Prefix / Prompt Cache 公式、生命周期和可运行 demo

第一版 prefix cache 的核心是 full block 复用。对请求 i , prompt token 数为 T_i , block size 为 S , 最多可查找的 full block 数为:

$$B_i^{\mathrm{full}} = \left\lfloor \frac{T_i}{S} \right\rfloor$$

若连续命中 H_i 个 full blocks, 则命中 token 数为:

$$T_i^{\mathrm{hit}} = S H_i$$

本轮还需要真实 prefill 的 suffix token 数为:

$$T_i^{\mathrm{run}} = T_i - T_i^{\mathrm{hit}}$$

token 级命中率比 request 级命中率更有意义:

$$R_{\mathrm{hit, tok}} = \frac{\sum_i T_i^{\mathrm{hit}}}{\max(1, \sum_i T_i)}$$

block hash 使用 parent hash 串起上下文:

$$h_j = H(h_{j-1}, X_j, E)$$

其中 X_j 是第 j 个 full block 的 tokens, E 是 model、tokenizer、LoRA、tenant salt、多模态 hash 等 extra hashes。

最终升级门禁:

$$G_{\mathrm{prefix}} = G_{\mathrm{boundary}} G_{\mathrm{full}} G_{\mathrm{parent}} G_{\mathrm{extra}} G_{\mathrm{suffix}} G_{\mathrm{suffix}} G_{\mathrm{suffix}}$$

下面这个 0 依赖 demo 模拟第一版 hash-based prefix cache:

1. A 先完成 prefill, 把两个 full blocks 注册进 cache。
2. B 使用相同 system prompt, 命中 A 的两个 full blocks, 只 prefill suffix。
3. C 使用不同 tenant salt, 同样 tokens 也不能命中。
4. D 使用不同 LoRA, 同样 tokens 也不能命中。
5. E 完整命中 prompt, 但由于没有缓存 logits, 走最后 token fallback。
6. 请求结束后 active ref 释放, cached free block 可以留在 cache, 也可以被 eviction 淘汰。

```
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class KVCacheBlock:
```

```
    block_id: int
```

```
    ref_count: int = 0
```

```
    block_hash: tuple = None
```

```
    is_cached: bool = False
```

```
@dataclass
```

```
class RequestState:
```

```
    request_id: str
```

```
    prompt_tokens: list
```

```
    extra_hashes: tuple
```

```
    block_table: list = field(default_factory=list)
```

```
    block_hashes: list = field(default_factory=list)
```

```
    num_computed_tokens: int = 0
```

```
    prefix_cache_hit_tokens: int = 0
```

```
class ToyPrefixPromptCacheManager:
```

```
    def __init__(self, num_blocks, block_size):
```

```
        self.block_size = block_size
```

```
        self.blocks = [KVCacheBlock(i) for i in range(num_blocks)]
```

```
        self.free_blocks = list(range(num_blocks))
```

```
        self.cached_blocks = {}
```

```
        self.cache_order = []
```

```
        self.metrics = {
```

```
            "lookups": 0,
```

```
            "hit_blocks": 0,
```

```
            "hit_tokens": 0,
```

```
            "total_prompt_tokens": 0,
```

```
            "evictions": 0,
```

```
        }
```

```
        self.trace = []
```

```
    def full_blocks(self, tokens):
```

```
        count = len(tokens) // self.block_size
```

```
        for index in range(count):
```

```
            start = index * self.block_size
```

```
            yield index, tokens[start : start + self.block_size]
```

```
    def hash_block(self, parent_hash, block_tokens, extra_hashes):
```

```
        return (parent_hash, tuple(block_tokens), tuple(extra_hashes))
```

```
    def allocate_until(self, req, target_tokens):
```

```
        needed = (target_tokens + self.block_size - 1) // self.block_size
```

```
        while len(req.block_table) < needed:
```

```
            if not self.free_blocks:
```

```
                return False
```

```
            block_id = self.free_blocks.pop(0)
```

```
            block = self.blocks[block_id]
```

```
            if block.is_cached and block.ref_count == 1:
```

```

        self.evict_block(block_id)
        block.ref_count += 1
        req.block_table.append(block_id)
    return True

def register_full_blocks(self, req, computed_tokens):
    parent_hash = None
    for logical_id, block_tokens in self.full_blocks(req.prompt_tokens[:computed_tokens]):
        block_hash = self.hash_block(parent_hash, block_tokens, req.extra_hashes)
        if logical_id >= len(req.block_hashes):
            req.block_hashes.append(block_hash)
        block_id = req.block_table[logical_id]
        block = self.blocks[block_id]
        if block_hash not in self.cached_blocks:
            block.block_hash = block_hash
            block.is_cached = True
            block.ref_count += 1
            self.cached_blocks[block_hash] = block_id
            self.cache_order.append(block_hash)
            self.trace.append({"event": "cache_register", "block": block_id})
        parent_hash = block_hash

def lookup_and_attach(self, req):
    self.metrics["lookups"] += 1
    self.metrics["total_prompt_tokens"] += len(req.prompt_tokens)
    parent_hash = None
    hit_blocks = []
    for _, block_tokens in self.full_blocks(req.prompt_tokens):
        block_hash = self.hash_block(parent_hash, block_tokens, req.extra_hashes)
        block_id = self.cached_blocks.get(block_hash)
        if block_id is None:
            break
        if block_id in self.free_blocks:
            self.free_blocks.remove(block_id)
        self.blocks[block_id].ref_count += 1
        req.block_table.append(block_id)
        req.block_hashes.append(block_hash)
        hit_blocks.append(block_id)
        parent_hash = block_hash
    hit_tokens = len(hit_blocks) * self.block_size
    req.num_computed_tokens = hit_tokens
    req.prefix_cache_hit_tokens = hit_tokens
    self.metrics["hit_blocks"] += len(hit_blocks)
    self.metrics["hit_tokens"] += hit_tokens
    self.trace.append({"event": "lookup", "request": req.request_id, "hit_blocks": hit_blocks, "hit_tokens": hit_tokens})
    return hit_blocks, hit_tokens

def free_request(self, req):
    for block_id in req.block_table:
        block = self.blocks[block_id]
        block.ref_count -= 1
        if block.ref_count == 0 or (block.is_cached and block.ref_count == 1):
            if block_id not in self.free_blocks:
                self.free_blocks.append(block_id)
    req.block_table = []

```

```

def evict_block(self, block_id):
    block = self.blocks[block_id]
    if not block.is_cached or block.ref_count != 1:
        return False
    self.cached_blocks.pop(block.block_hash, None)
    if block.block_hash in self.cache_order:
        self.cache_order.remove(block.block_hash)
    block.block_hash = None
    block.is_cached = False
    block.ref_count -= 1
    self.metrics["evictions"] += 1
    self.trace.append({"event": "cache_evict", "block": block_id})
    return True

def evict_one_cached_free(self):
    for block_hash in list(self.cache_order):
        block_id = self.cached_blocks[block_hash]
        block = self.blocks[block_id]
        if block_id in self.free_blocks and block.ref_count == 1:
            return self.evict_block(block_id), block_id
    return False, None

def cached_free_blocks(self):
    return [block.block_id for block in self.blocks if block.is_cached and block.ref_count == 1 and block.block_id in self.free_blocks]

def token_hit_ratio(self):
    return round(self.metrics["hit_tokens"] / max(1, self.metrics["total_prompt_tokens"]), 3)

```

```

extra_a = ("model-v1", "tok-v1", "lora-none", "tenant-a", "text-only")
extra_salt_b = ("model-v1", "tok-v1", "lora-none", "tenant-b", "text-only")
extra_lora = ("model-v1", "tok-v1", "lora-finance", "tenant-a", "text-only")
shared_prefix = [11, 12, 13, 14, 21, 22, 23, 24]

```

```

manager = ToyPrefixPromptCacheManager(num_blocks=6, block_size=4)

```

```

req_a = RequestState("A", shared_prefix + [101, 102], extra_a)
manager.allocate_until(req_a, target_tokens=len(req_a.prompt_tokens))
req_a.num_computed_tokens = len(req_a.prompt_tokens)
manager.register_full_blocks(req_a, computed_tokens=req_a.num_computed_tokens)
a_cached_tables = list(req_a.block_table)
a_parent_chain_ready = req_a.block_hashes[1][0] == req_a.block_hashes[0]
manager.free_request(req_a)

```

```

req_b = RequestState("B", shared_prefix + [201, 202], extra_a)
b_hit_blocks, b_hit_tokens = manager.lookup_and_attach(req_b)
b_free_queue_after_touch = list(manager.free_blocks)
b_suffix_start = req_b.num_computed_tokens
manager.allocate_until(req_b, target_tokens=len(req_b.prompt_tokens))
b_suffix_positions = list(range(b_suffix_start, len(req_b.prompt_tokens)))
b_suffix_slot_mapping = [
    req_b.block_table[pos // manager.block_size] * manager.block_size + pos % manager.block_size
    for pos in b_suffix_positions
]

```

```

b_ref_counts_after_attach = {block_id: manager.blocks[block_id].ref_count for block_id in b_hit_blocks}
manager.free_request(req_b)
b_ref_counts_after_free = {block_id: manager.blocks[block_id].ref_count for block_id in b_hit_blocks}

req_c = RequestState("C", shared_prefix, extra_salt_b)
c_hit_blocks, _ = manager.lookup_and_attach(req_c)
req_d = RequestState("D", shared_prefix, extra_lora)
d_hit_blocks, _ = manager.lookup_and_attach(req_d)

req_e = RequestState("E", shared_prefix, extra_a)
e_hit_blocks, e_hit_tokens = manager.lookup_and_attach(req_e)
e_full_prompt_hit = e_hit_tokens == len(req_e.prompt_tokens)
e_fallback_position = len(req_e.prompt_tokens) - 1 if e_full_prompt_hit else None
manager.free_request(req_e)

cached_free_before_evict = manager.cached_free_blocks()
evicted, evicted_block = manager.evict_one_cached_free()
cached_free_after_evict = manager.cached_free_blocks()

summary = {
    "a_cached_tables": a_cached_tables,
    "b_hit_blocks": b_hit_blocks,
    "b_saved_prefill_tokens": b_hit_tokens,
    "b_run_prefill_tokens": len(req_b.prompt_tokens) - b_hit_tokens,
    "b_suffix_start": b_suffix_start,
    "b_suffix_positions": b_suffix_positions,
    "b_suffix_slot_mapping": b_suffix_slot_mapping,
    "b_free_queue_after_touch": b_free_queue_after_touch,
    "b_ref_counts_after_attach": b_ref_counts_after_attach,
    "b_ref_counts_after_free": b_ref_counts_after_free,
    "salt_miss_blocks": c_hit_blocks,
    "lora_miss_blocks": d_hit_blocks,
    "e_full_prompt_hit": e_full_prompt_hit,
    "e_fallback_position": e_fallback_position,
    "cached_free_before_evict": cached_free_before_evict,
    "evicted_block": evicted_block,
    "cached_free_after_evict": cached_free_after_evict,
    "token_hit_ratio": manager.token_hit_ratio(),
    "metrics": manager.metrics,
    "trace_tail": manager.trace[-5:],
}
}
gates = {
    "prompt_prefix_boundary_ready": True,
    "full_block_reuse_ready": b_hit_blocks == [0, 1],
    "parent_hash_chain_ready": a_parent_chain_ready,
    "extra_hash_isolation_ready": c_hit_blocks == [] and d_hit_blocks == [],
    "suffix_prefill_ready": b_suffix_start == 8 and b_suffix_positions == [8, 9],
    "cached_ref_count_ready": b_ref_counts_after_attach == {0: 2, 1: 2} and b_ref_counts_after_free == {0: 1, 1: 1},
    "eviction_ready": evicted is True and evicted_block == 0,
    "full_prompt_hit_fallback_ready": e_full_prompt_hit is True and e_fallback_position == 7,
}
}
gates["prefix_prompt_cache_upgrade_gate"] = all(gates.values())

print("prefix_prompt_cache_summary=", summary)
print("prefix_prompt_cache_gates=", gates)

```


一次运行的核心输出类似：

```
prefix_prompt_cache_summary= {'a_cached_tables': [0, 1, 2], 'b_hit_blocks': [0, 1], 'b_saved_prefill_tokens': 8, 'b_run_
prefix_prompt_cache_gates= {'prompt_prefix_boundary_ready': True, 'full_block_reuse_ready': True, 'parent_hash_chain_r
```

这个 demo 验证了几个容易出错的边界：

1. B 只复用 full blocks [0,1]，最后 2 个 suffix tokens 仍然要从 position 8 开始 prefill。
2. b_suffix_slot_mapping=[12,13] 证明 suffix KV 写到新分配的 physical block，而不是覆盖 cached prefix。
3. C 和 D tokens 相同但 extra hashes 不同，因此不会错误复用。
4. 命中 cached-free blocks 后会从 free queue touch 出来，并增加 active ref。
5. 请求结束后 active ref 释放，cached ref 保留；显存紧张时只淘汰没有 active request 的 cached-free block。
6. E 完整命中 prompt，但没有缓存 logits，所以必须有最后 token fallback 或等价策略。

50.36 小练习

1. 给 KVCacheBlock 增加 ref_count、block_hash、is_cached 字段。
2. 给 BlockManager 增加 cached_blocks 字典。
3. 实现 iter_full_token_blocks(token_ids, block_size)。
4. 实现包含 parent hash 的 hash_block。
5. 实现 find_cached_prefix，返回 hit block ids 和 hit tokens。
6. cache hit 后初始化 request.block_table 和 request.num_computed_tokens。
7. 改造 scheduler，让 prefill 从 num_computed_tokens 开始。
8. 改造 BatchBuilder，验证非零 start position 的 positions 不重置。
9. 实现 register_cached_block。
10. 实现 request 结束时只释放 active refs。
11. 实现 evict_one_cached_block，只淘汰没有 active request 的 cached block。
12. 增加 prefix cache hit tokens 指标。
13. 构造固定 system prompt workload，验证第二个请求 TTFT 下降。
14. 构造随机 prompt workload，验证 hit ratio 接近 0。
15. 写一段面试回答：为什么 prefix cache 需要 parent hash 和 extra hashes？

50.37 本章总结

prefix cache 是 paged KV cache 之后最自然的升级。

它的核心不是缓存字符串，也不是缓存最终答案，而是复用已经计算完成的 KV blocks。

第一版 hash-based prefix cache 可以很简单：只缓存 full blocks，用 parent hash 串起 prefix chain，用 extra hashes 保证模型、tokenizer、LoRA、多租户和多模态条件一致。

新请求命中 cache 后，本质上只是把 request 的 block table 前缀接到 cached blocks，并把 num_computed_tokens 设置为 hit tokens。后续 scheduler、BatchBuilder、OutputProcessor 仍然走原来的 suffix prefill 和 decode 流程。

prefix cache 的收益主要体现在 TTFT 和 prefill tokens/s，但代价是占用更多 KV blocks。因此必须有 ref count、eviction、水位线、hit token ratio、cached blocks 和 free blocks 等观测能力。

第一版不要急着实现 RadixAttention、partial block cache 或复杂 eviction。

先把 full-block hash cache、ref count、suffix prefill 和指标做正确，再继续升级。

下一章会进入第 51 章：实现 preemption、recompute 和 swap 的最小版本。

第 51 章 实现 preemption、recompute 和 swap

上一章我们在 paged KV cache 之上实现了 prefix cache。

prefix cache 带来的收益很直接：复用已经算过的前缀 KV，降低重复 prefill 成本，改善 TTFT。

但它也带来了一个更现实的问题：

KV blocks 更容易不够用。

即使不开 prefix cache，只要在线流量里存在长 prompt、长输出、高并发和突发请求，paged KV cache 也会遇到资源紧张。

这时 scheduler 不能只有两个选择：

要么接请求，要么 OOM。

它需要第三种能力：

在 KV 空间不足时，临时让出一部分 running 请求占用的资源，等资源恢复后再继续执行。

这就是本章要实现的 preemption。

51.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 paged KV cache 在 KV block 压力下的 preemption、recompute 和 swap 状态机。

公开资料校准主要参考三类口径：

1. vLLM optimization 文档对 KV cache 空间不足、preemption、recompute / swap 策略、max_num_seqs、max_num_batched_tokens 和 KV cache usage 的公开说明。
2. vLLM / PagedAttention 论文对 block-based KV cache 和动态 serving request 的资源压力背景。
3. 本书第 49、50 章对 paged KV cache、prefix cache、cached-free block、ref count 和 eviction 的教学抽象。

本章不实现真实异步 CPU/GPU KV copy、pinned memory、PCIe/NVLink 带宽模型、多 GPU / TP rank 一致 swap、KV connector、远程 KV cache、生产级优先级队列或完整 vLLM preemption policy。我们只验证一个最小闭环：

KV budget 不足 -> 先 cache eviction -> 再 preempt running request -> recompute 或 swap 恢复 -> 记录指标
第二轮新增 demo 的验收重点是：

preemption 和 cache eviction、abort、backpressure 是否区分清楚；
资源紧张时是否先淘汰 cached-free blocks，再 preempt active request；
recompute preemption 是否保留 prompt tokens 和 output tokens；
恢复时是否用 prompt + output 重建 KV；
swap out / swap in 是否保持逻辑 block 顺序；
swap in 失败时 CPU KV 是否保留；
preemption 指标是否能解释尾延迟风险。

51.1 本章目标

读完本章，你应该能讲清：

1. preemption 在推理框架里解决什么问题。
2. preemption、eviction、abort、backpressure 的区别。
3. recompute 和 swap 两种策略的核心差异。
4. 为什么 vLLM-like engine 通常更偏向 recompute。
5. scheduler 在什么时机触发 preemption。
6. 被 preempt 的 request 状态如何保存。
7. recompute 恢复时如何重新分配 KV blocks。
8. swap 恢复时如何从 CPU KV 拷回 GPU。
9. preemption 会怎样影响 TTFT、TPOT、吞吐和显存。
10. 如何用日志、指标和压测判断 preemption 是否过于频繁。

面试中要先说结论：

preemption 是鲁棒性机制，不是性能优化手段。
它的目标是避免系统在 KV cache 压力下直接 OOM，同时让服务继续前进。
如果系统频繁 preempt，说明容量、调度参数、请求形态或缓存策略有问题。

51.2 preemption 解决的不是算力问题

LLM serving 里最容易耗尽的资源有两类：

- 1. GPU compute。
- 2. GPU memory，尤其是 KV cache blocks。

preemption 主要解决第二类。
假设 block pool 里一共有 1000 个 blocks。
当前 running 请求已经占用了 980 个。
下一轮 scheduler 想做两件事：

- 1. 给 running 请求继续 decode，可能每个请求都需要追加 KV。
- 2. 接纳 waiting 请求做 prefill，可能一次要分配很多 KV blocks。

如果 free blocks 不够，简单实现通常会失败：
allocate blocks -> failed -> OOM / request failed / engine crash
更合理的实现是：
allocate blocks -> failed -> preempt some running requests -> free their GPU KV -> retry scheduling
注意这里不是 GPU 算不过来。
而是 GPU KV 空间不够。

51.3 四个概念不要混

preemption 容易和 eviction、abort、backpressure 混在一起。
它们解决的问题不同。

概念	对象	结果	用户请求是否还继续
preemption	running request	暂停并释放或迁移 KV	继续
eviction	cached block	淘汰无 active request 的缓存	不影响 active 请求
abort	request	主动终止请求并释放资源	不继续
backpressure	waiting request / upstream	限流、排队或拒绝新请求	可能继续，也可能拒绝

prefix cache 的 eviction 只应该动没有 active request 引用的 cached blocks。
preemption 动的是正在执行的 request。
abort 是直接结束请求。
backpressure 是在入口处减少进入 engine 的压力。
这四个机制可以同时存在，但不能互相替代。

51.4 preemption 的两种策略

被 preempt 的 request 要让出 GPU KV。

让出之后，后面怎么恢复？

常见有两种策略。

第一种是 recompute。

preempt request -> 释放它的 GPU KV blocks -> request 回到 waiting
resume 时重新 prefill prompt + 已生成 tokens，重建 KV

第二种是 swap。

preempt request -> 把 GPU KV blocks 拷贝到 CPU memory -> 释放 GPU KV blocks
resume 时把 CPU KV 拷回 GPU，继续 decode

对比：

策略	释放 GPU KV	恢复成本	额外资源	适合场景
recompute	是	重新计算 prefill	GPU compute	prompt 不太长、CPU 带宽紧张、
swap	是	CPU/GPU 拷贝	CPU memory + PCIe/NVLink 带宽	上下文很长、重算很贵、带宽充足

教学版建议先实现 recompute。

原因是它的数据结构更简单，bug 更少，也更容易解释 scheduler 和 KV block 生命周期。

51.5 为什么通常优先讲 recompute

recompute 的最大优点是：不需要保存整段 KV。

被 preempt 后，只要保存 request 的逻辑状态：

1. prompt token ids。
2. 已生成 token ids。
3. sampling params。
4. stop criteria。
5. request id、arrival time、priority 等调度信息。

恢复时，把完整上下文拼出来：

context_tokens = prompt_token_ids + output_token_ids

然后重新 prefill 到当前位置。

这会浪费计算，但避免了 CPU KV 存储、异步拷贝、pin memory、带宽竞争和 swap 状态一致性问题。

很多线上场景中，preemption 本来就应该是低频兜底机制。

既然低频，优先选择实现简单、稳定性高的 recompute 是合理的。

51.6 swap 为什么更复杂

swap 看起来更节省计算。

但工程复杂度明显更高。

你需要考虑：

1. CPU KV buffer 如何分配。
2. GPU block 到 CPU block 的映射如何维护。
3. swap out 和 swap in 是否异步。

4. 拷贝时 request 是否还能被 scheduler 选中。
5. CPU 内存不够怎么办。
6. PCIe/NVLink 带宽被 KV 拷贝打满怎么办。
7. swap in 失败后 request 状态如何回滚。
8. block table 里的物理 block id 何时更新。
9. prefix cache block 是否允许 swap。
10. 多 GPU / TP 场景下每个 rank 的 KV 是否一致迁移。

所以 swap 不是 “把 tensor .cpu() 一下” 这么简单。

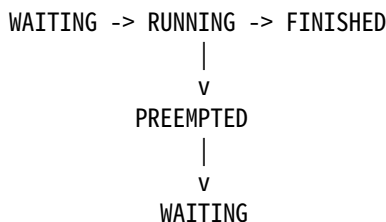
它是一套内存层级管理机制。

教学版可以先把 swap 作为接口和状态机讲清楚，不急实现高性能异步拷贝。

51.7 request 状态机

第 48 章的 continuous batching 已经有 waiting 和 running。

加入 preemption 后，建议显式增加 preempted 状态。



如果实现 swap，可以再细分：

RUNNING -> SWAPPING_OUT -> SWAPPED -> SWAPPING_IN -> RUNNING

教学版 recompute 可以简单一点：

```

class RequestStatus:
    WAITING = "waiting"
    RUNNING = "running"
    PREEMPTED = "preempted"
    FINISHED = "finished"
    ABORTED = "aborted"
  
```

request 里需要增加几个字段：

```

class RequestState:
    def __init__(self, request_id, prompt_token_ids):
        self.request_id = request_id
        self.prompt_token_ids = prompt_token_ids
        self.output_token_ids = []
        self.block_table = []
        self.num_computed_tokens = 0
        self.status = RequestStatus.WAITING
        self.preempted_count = 0
        self.last_preempted_at = None
  
```

关键点是：

preemption 释放的是 GPU KV，不应该丢失 request 的逻辑 token 序列。

51.8 recompute preemption 的最小实现

recompute 版本的 preempt 操作很短。

伪代码:

```
def preempt_by_recompute(request, block_manager):
    block_manager.free_request_blocks(request)

    request.block_table = []
    request.num_computed_tokens = 0
    request.status = RequestStatus.PREEMPTED
    request.preempted_count += 1
```

如果这个 request 命中过 prefix cache, 释放时要注意 ref count。

```
def free_request_blocks(request):
    for block_id in request.block_table:
        block = self.blocks[block_id]
        block.active_refs -= 1

        if block.active_refs == 0 and not block.cached:
            self.free_blocks.append(block_id)

    request.block_table = []
```

cached block 不应该因为 request 被 preempt 就直接回到 free list。

它只释放 active ref。

如果还有 cache ref, 它仍然留在 prefix cache 中。

51.9 recompute 恢复流程

被 preempt 的 request 不应该回到队尾当成全新请求, 也不应该立刻无条件插队。

一种简单做法是把它放回 waiting queue, 但保留优先级信息。

恢复时:

```
def resume_recompute(request):
    request.status = RequestStatus.WAITING
    request.block_table = []
    request.num_computed_tokens = 0
```

scheduler 下次选中它时, 按照普通 prefill 处理。

但 prefill 的输入不只是 prompt。

它应该是:

```
context_tokens = request.prompt_token_ids + request.output_token_ids
```

也就是说, 已经生成过的 tokens 也要参与 recompute。

否则恢复后的 KV 只覆盖 prompt, 不包含已生成输出的历史上下文。

这是 recompute 最常见的 bug。

51.10 为什么 output tokens 也要重算

假设一个请求:

prompt: P0 P1 P2

already generated: Y0 Y1 Y2

preempt 发生在准备生成 Y3 之前。

此时 KV cache 里保存的是:

P0 P1 P2 Y0 Y1 Y2

如果恢复时只 prefill prompt:

P0 P1 P2

然后直接 decode Y3, 模型看不到 Y0/Y1/Y2 的 KV。

结果会错。

所以 recompute 的上下文必须包含 prompt 和已经生成的输出。

recompute context = prompt + generated outputs

这也是为什么 preemption 不是免费的。

请求已经生成得越长, recompute 成本越高。

51.11 scheduler 什么时候触发 preemption

最小实现里有两个触发点。

第一个触发点: 接纳 waiting 请求前, 发现 KV blocks 不够。

```
if not block_manager.can_allocate(num_required_blocks):  
    victim = select_victim(running_requests)  
    preempt_by_recompute(victim, block_manager)
```

第二个触发点: running 请求 decode 追加 block 时, 发现没有 free block。

```
if request_needs_new_block(req) and not block_manager.has_free_block():  
    victim = select_victim(running_requests)  
    preempt_by_recompute(victim, block_manager)
```

更实际的 scheduler 会在每轮调度前做预算:

required_blocks = blocks needed by selected prefill chunks + blocks needed by decode growth

如果预算不够, 再决定是否:

1. 减少本轮 prefill chunk。
2. 暂缓接纳 waiting 请求。
3. 淘汰 prefix cache blocks。
4. preempt running requests。
5. 触发 backpressure。

preemption 应该排在比较靠后的位置。

因为它会伤害延迟。

51.12 victim 选择策略

victim 就是被 preempt 的请求。

最简单策略是选最后进入 running 的请求。

```
def select_victim(running_requests):  
    return running_requests[-1]
```

但真实系统里要更谨慎。

常见策略包括:

1. 优先 preempt 已经 preempt 次数少的请求。
2. 优先 preempt 低优先级请求。
3. 优先 preempt recompute 成本低的请求。
4. 避免 preempt 已经接近完成的请求。

5. 避免反复 preempt 同一个请求。
6. 避免 preempt latency-sensitive 请求。

一个教学版评分函数可以这样写：

```
def victim_score(req):
    context_len = len(req.prompt_token_ids) + len(req.output_token_ids)
    return (
        req.priority * 1000000
        + req.preempted_count * 10000
        + context_len
    )

def select_victim(running_requests):
    return min(running_requests, key=victim_score)
```

这里分数越小越适合被 preempt。

低优先级、preempt 次数少、上下文短的请求更容易被选中。

实际实现中不要迷信某个公式，要根据业务 SLO 调整。

51.13 preempt 多少个请求

不要一遇到空间不足就只 preempt 一个请求然后盲目重试。

更好的做法是计算缺口。

```
def preempt_until_enough(required_blocks, running_requests, block_manager):
    victims = []

    while block_manager.num_free_blocks() < required_blocks:
        victim = select_victim(running_requests)
        if victim is None:
            break

        running_requests.remove(victim)
        preempt_by_recompute(victim, block_manager)
        victims.append(victim)

    return victims
```

但这里有一个细节：

preempt victim 释放的 blocks 未必等于它 block_table 的长度。

如果 victim 的某些 blocks 是 cached blocks，释放 active ref 后它们可能仍然被 prefix cache 持有。

这些 blocks 不会回到 free list。

所以判断要以实际 free block 数为准，而不是估算。

51.14 preemption 和 prefix cache 的关系

prefix cache 会影响 preemption。

第一，它占用 KV blocks。

cached blocks 没有 active request 使用时，仍然可能留在 block pool 中。

这会降低 free blocks，增加 preemption 概率。

第二，它也能降低 recompute 成本。

如果被 preempt 的 request 恢复时, prompt prefix 命中 prefix cache, 就不需要从 0 重算全部上下文。恢复路径可以变成:

find cached prefix -> 引用 cached blocks -> recompute suffix context

第三, 资源紧张时通常应该先考虑 cache eviction, 再考虑 preempt active request。

推荐顺序是:

1. 降低本轮接纳的新 prefill tokens。
2. 淘汰没有 active request 引用的 cached blocks。
3. 仍然不够时, 再 preempt running requests。

因为 eviction 影响未来命中率, preemption 直接影响当前请求延迟。

两者都不是免费操作。

51.15 preemption 和 chunked prefill 的关系

chunked prefill 可以降低 preemption 频率。

如果一个 32K prompt 一次性 prefill, 需要一次性分配大量 KV blocks。

在高并发下, 这很容易挤爆 block pool。

chunked prefill 把它拆成多轮:

32K prompt -> 2K + 2K + ... + 2K

scheduler 每轮只需要为当前 chunk 做预算。

这让系统有机会在 iteration boundary 上重新观察 free blocks、finished requests 和 waiting queue。

但 chunked prefill 不是万能的。

完整 prompt 的 KV 最终还是要占空间。

它只是降低单轮峰值压力, 不会改变总 KV 需求。

51.16 swap 的最小数据结构

如果要实现 swap, 需要在 request 上记录 CPU KV 位置。

教学版可以抽象成:

```
class RequestState:
    def __init__(self, request_id, prompt_token_ids):
        self.request_id = request_id
        self.prompt_token_ids = prompt_token_ids
        self.output_token_ids = []
        self.block_table = []
        self.cpu_block_table = []
        self.status = RequestStatus.WAITING
```

再增加一个 CPU block manager:

```
class CPUBlockManager:
    def __init__(self, num_blocks):
        self.free_blocks = list(range(num_blocks))

    def allocate(self, n):
        if len(self.free_blocks) < n:
            return None
        blocks = self.free_blocks[:n]
```

```
self.free_blocks = self.free_blocks[n:]  
return blocks
```

GPU block table 和 CPU block table 都是逻辑 block index 到物理 block id 的映射。
区别只是物理位置不同。

51.17 swap out 流程

swap out 的伪代码：

```
def swap_out(request, gpu_mgr, cpu_mgr, kv_copy):  
    cpu_blocks = cpu_mgr.allocate(len(request.block_table))  
    if cpu_blocks is None:  
        return False  
  
    for gpu_block, cpu_block in zip(request.block_table, cpu_blocks):  
        kv_copy.gpu_to_cpu(gpu_block, cpu_block)  
  
    gpu_mgr.free_request_blocks(request)  
    request.cpu_block_table = cpu_blocks  
    request.block_table = []  
    request.status = RequestStatus.SWAPPED  
    return True
```

真实系统通常不会同步逐个拷贝。

它会考虑异步 copy、stream、pinned memory 和批量拷贝。

但教学版先把状态变化讲清楚即可。

51.18 swap in 流程

swap in 的伪代码：

```
def swap_in(request, gpu_mgr, cpu_mgr, kv_copy):  
    gpu_blocks = gpu_mgr.allocate(len(request.cpu_block_table))  
    if gpu_blocks is None:  
        return False  
  
    for cpu_block, gpu_block in zip(request.cpu_block_table, gpu_blocks):  
        kv_copy.cpu_to_gpu(cpu_block, gpu_block)  
  
    cpu_mgr.free(request.cpu_block_table)  
    request.cpu_block_table = []  
    request.block_table = gpu_blocks  
    request.status = RequestStatus.RUNNING  
    return True
```

swap in 成功后，request 可以继续 decode。

如果 swap in 失败，不应该丢掉 CPU KV。

request 仍然保持 SWAPPED 状态，等待下一轮资源可用。

51.19 swap 的一致性问题

swap 必须保证逻辑 token 位置和 KV block 顺序一致。

不能因为 GPU block id 变化就改变 block table 的逻辑顺序。

例如 swap out 前:

```
logical block 0 -> GPU block 7  
logical block 1 -> GPU block 3  
logical block 2 -> GPU block 12
```

swap in 后可以变成:

```
logical block 0 -> GPU block 20  
logical block 1 -> GPU block 5  
logical block 2 -> GPU block 9
```

物理 id 可以变。

逻辑顺序不能变。

BatchBuilder 后续生成 attention metadata 时, 必须用新的 request.block_table。

51.20 recompute 和 swap 如何选择

一个实用判断:

如果 preemption 低频, 优先 recompute。

如果上下文极长且重算成本高, 可以考虑 swap。

更具体一点:

适合 recompute:

1. 中短上下文聊天。
2. preemption 只是偶发兜底。
3. CPU 内存或 PCIe 带宽紧张。
4. 系统更看重实现稳定性。
5. prefix cache 命中率较高, 重算 suffix 不大。

适合 swap:

1. 长上下文请求多。
2. preempt 后重算成本非常高。
3. CPU 内存充足。
4. GPU/CPU 传输带宽充足。
5. engine 已经有成熟的多级 KV cache 管理。

不要把 swap 当成无成本优化。

swap 可能把瓶颈从 GPU memory 转移到 CPU memory 和总线带宽。

51.21 对延迟指标的影响

preemption 会影响多个指标。

对被 preempt 的请求:

1. E2E latency 变差。
2. 如果已经开始 streaming, 后续 token 间隔可能出现大 gap。
3. recompute 会增加恢复后的 TTFT-like delay。
4. swap 会增加 copy delay。

对其他请求:

1. 可能避免 OOM。
2. 可能释放 KV, 让高优先级请求继续。
3. 也可能因为 recompute 或 swap 占资源, 造成整体抖动。

所以 preemption 指标必须和 TTFT、TPOT、E2E latency 一起看。
只看吞吐可能会误判。

51.22 需要记录哪些日志

每次 preemption 都应该打结构化日志。

建议字段：

字段	含义
request_id	被 preempt 的请求
reason	触发原因，例如 no_free_blocks
strategy	recompute 或 swap
num_blocks_freed	实际释放 GPU blocks 数
num_context_tokens	prompt + output token 数
preempted_count	该请求累计 preempt 次数
free_blocks_before	preempt 前 free blocks
free_blocks_after	preempt 后 free blocks
running_queue_size	running 请求数
waiting_queue_size	waiting 请求数

日志示例：

```
preempt request=req-42 strategy=recompute reason=no_free_blocks \  
context_tokens=4096 blocks_freed=256 free_blocks_before=3 free_blocks_after=259 preempted_count=1
```

这类日志对排查线上延迟尖刺非常有用。

51.23 需要暴露哪些指标

建议指标：

指标	含义
preemption_total	preemption 总次数
preemption_recompute_total	recompute 次数
preemption_swap_total	swap 次数
preempted_requests_total	被 preempt 的请求数
preemption_repeated_requests	被多次 preempt 的请求数
preemption_freed_blocks	preemption 实际释放 blocks 数
recompute_tokens_total	recompute 重算 tokens 数
swap_in_bytes_total	swap in 字节数
swap_out_bytes_total	swap out 字节数
swap_in_latency_ms	swap in 延迟
swap_out_latency_ms	swap out 延迟
free_kv_blocks	当前 free blocks
preemption_rate	单位时间 preemption 次数

最关键的是：

`preemption_rate` 不应该长期处于高位。

如果它持续升高，说明系统在靠牺牲尾延迟维持运行。

51.24 压测场景一：突发长 prompt

workload:

短请求: prompt 128 tokens, output 128 tokens, 持续到达

突发请求: 每隔一段时间注入 prompt 16K tokens, output 256 tokens

观察:

1. free blocks 是否快速下降。
2. 长 prompt 是否触发 preemption。
3. 短请求 TPOT 是否出现尖刺。
4. preempted request 的 E2E latency 是否明显变差。
5. chunked prefill 后 preemption 是否减少。

这个场景用来验证 scheduler 在长 prompt 冲击下是否能保持服务不崩。

51.25 压测场景二：长输出 decode 压力

workload:

prompt 512 tokens

output 4K tokens

高并发持续到达

这个场景里 KV cache 压力不是一次性来自 prefill, 而是 decode 逐步增长。

观察:

1. running requests 的 block_table 是否持续变长。
2. decode 追加新 block 时是否触发 preemption。
3. victim 是否总是同一批请求。
4. repeated preemption 是否过高。
5. max_num_seqs 降低后 preemption 是否缓解。

它可以暴露 victim 选择策略是否公平。

51.26 压测场景三：prefix cache 占满 blocks

workload:

大量共享 system prompt 请求, 让 prefix cache 留住很多 blocks

随后注入随机长 prompt 请求

观察:

1. cached blocks 是否占据大量 block pool。
2. free blocks 不足时是否先执行 cache eviction。
3. eviction 后是否减少 preemption。
4. cached block ref count 是否正确。
5. cache map 是否出现悬挂 block id。

这个场景用来验证 prefix cache eviction 和 preemption 的优先级关系。

51.27 常见 bug

bug 一: preempt 后丢失 output tokens。

结果: recompute 恢复时上下文不完整, 后续生成错误。

bug 二: preempt 后只重算 prompt, 不重算已生成 tokens。

结果: 模型继续 decode 时看不到历史输出。

bug 三：释放 cached blocks 时直接放回 free list。

结果：prefix cache map 指向的 block 被新请求覆盖。

bug 四：victim 选择总是同一个请求。

结果：某些请求被反复 preempt，E2E latency 极差甚至饥饿。

bug 五：swap in 失败后清空 CPU block table。

结果：request 无法恢复，KV 丢失。

bug 六：swap in 后没有更新 GPU block table。

结果：attention kernel 读旧 block id 或空 block。

bug 七：preemption 计数没有暴露指标。

结果：线上只看到 latency 抖动，不知道根因是 KV 不足。

bug 八：把 preemption 当成常规调度优化。

结果：系统长期高频重算或 swap，尾延迟越来越差。

51.28 面试高频问题

问题一：vLLM 里的 preemption 是什么？

回答要点：当 KV cache 空间不足以继续服务当前调度集合时，engine 可以 preempt 一部分 running 请求，释放它们占用的 GPU KV，后续再通过 recompute 或 swap 恢复。它是避免 OOM 的鲁棒性机制，不是性能优化。

问题二：recompute 和 swap 有什么区别？

回答要点：recompute 会释放 GPU KV，恢复时根据 prompt 和已生成 tokens 重新 prefill 重建 KV；swap 会把 GPU KV 拷到 CPU，恢复时再拷回 GPU。recompute 消耗计算但实现简单，swap 节省重算但消耗 CPU 内存和传输带宽，状态管理更复杂。

问题三：为什么 recompute 恢复时要包含 output tokens？

回答要点：因为被 preempt 时 KV cache 表示的是 prompt 加已生成输出的完整上下文。如果恢复时只重算 prompt，后续 decode 看不到已经生成 tokens 的历史 KV，生成语义会错。

问题四：频繁 preemption 说明什么？

回答要点：通常说明 KV cache budget 不够、并发过高、请求上下文过长、max_num_seqs 或 max_num_batched_tokens 太激进、prefix cache 占用过多，或者缺少合理的 chunked prefill 和 admission control。

问题五：preemption 和 prefix cache eviction 谁优先？

回答要点：一般应先减少本轮调度压力，再淘汰没有 active request 引用的 cached blocks，最后才 preempt running requests。eviction 影响未来复用，preemption 直接影响当前请求延迟。

51.29 标准回答模板

如果面试官问“KV cache 不够时 serving engine 怎么办”，可以这样回答：

我会先把 KV cache 作为 scheduler 的核心资源预算。每轮调度前，scheduler 根据 selected prefill chunks、decode 增长和 max ru

如果仍然不够，就需要 preemption。preemption 是把部分 running request 暂停，释放它们占用的 GPU KV，后续再恢复。恢复策略主

victim 选择不能随便做。一般要考虑请求优先级、上下文长度、已 preempt 次数、是否接近完成、是否 latency-sensitive，避免反复 preempt 同一个请求。preemption 后要记录结构化日志和指标，比如 preemption_total、recompute_tokens_t

最后要强调，preemption 是防 OOM 的鲁棒性机制，不是性能优化。如果线上频繁 preemption，应该回头调 max_num_seqs、max_num_l

51.30 Preemption / Recompute / Swap 公式、状态机和可运行 demo

preemption 的触发条件可以先抽象成 KV block 缺口：

$$F_t < B_{\{\mathrm{need}\},t}$$

其中 F_t 是当前 free GPU KV blocks, $B_{\{\mathrm{need}\},t}$ 是本轮想调度的 prefill / decode 额外 block 数。

对 recompute 策略, 被 preempt 请求 i 的恢复重算 token 数是：

$$C_{\{\mathrm{recompute}\},i} = |X_i| + |Y_i|$$

其中 X_i 是 prompt tokens, Y_i 是已经生成的 output tokens。

对 swap 策略, 如果被迁移的 block 数是 $B_{\{\mathrm{swap}\}}$, 单 block page bytes 是 $P_{\{\mathrm{page}\}}$, swap 数据量是：

$$M_{\{\mathrm{swap}\}} = B_{\{\mathrm{swap}\}} P_{\{\mathrm{page}\}}$$

单位窗口内 preemption rate:

$$R_{\{\mathrm{preempt}\}} = \frac{N_{\{\mathrm{preempt}\}}}{\max(1, \Delta t)}$$

最终门禁：

$$G_{\{\mathrm{preempt}\}} = G_{\{\mathrm{pressure}\}} G_{\{\mathrm{evict}\}} G_{\{\mathrm{victim}\}} G_{\{\mathrm{recompute}\}} G_{\{\mathrm{swap}\}}$$

下面这个 0 依赖 demo 模拟一个最小调度回合：

1. GPU block pool 剩余空间不足, waiting 请求需要 4 个 blocks。
2. scheduler 先淘汰 cached-free block, 仍然不够。
3. scheduler 选择低优先级 running request 做 recompute preemption, 释放 GPU KV。
4. 被 preempt 请求保留 prompt + output tokens, 后续恢复时重建完整上下文。
5. 另一个请求演示 swap out / swap in; swap in 失败时 CPU block table 不丢失。

from dataclasses import dataclass, field

@dataclass

class RequestState:

```
    request_id: str
    prompt_tokens: list
    output_tokens: list
    priority: int
    block_table: list = field(default_factory=list)
    cpu_block_table: list = field(default_factory=list)
    status: str = "WAITING"
    num_computed_tokens: int = 0
    preempted_count: int = 0
    last_preempted_at: int = None
```

```
    def context_tokens(self):
        return self.prompt_tokens + self.output_tokens
```

class GPUBlockManager:

```
    def __init__(self, num_blocks, block_size):
        self.block_size = block_size
        self.free_blocks = list(range(num_blocks))
        self.cached_free_blocks = []

    def required_blocks(self, token_count):
        return (token_count + self.block_size - 1) // self.block_size
```

```

def allocate_for_tokens(self, req, token_count):
    needed = self.required_blocks(token_count)
    if len(self.free_blocks) < needed:
        return False
    req.block_table = [self.free_blocks.pop(0) for _ in range(needed)]
    req.num_computed_tokens = token_count
    req.status = "RUNNING"
    return True

def hold_cached_free_block(self):
    block_id = self.free_blocks.pop(0)
    self.cached_free_blocks.append(block_id)
    return block_id

def evict_cached_free(self):
    if not self.cached_free_blocks:
        return []
    block_id = self.cached_free_blocks.pop(0)
    self.free_blocks.append(block_id)
    return [block_id]

def free_request(self, req):
    released = list(req.block_table)
    self.free_blocks.extend(released)
    req.block_table = []
    req.num_computed_tokens = 0
    return released

class CPUBlockManager:
    def __init__(self, num_blocks):
        self.free_blocks = list(range(num_blocks))

    def allocate(self, count):
        if len(self.free_blocks) < count:
            return None
        blocks = self.free_blocks[:count]
        self.free_blocks = self.free_blocks[count:]
        return blocks

    def free(self, blocks):
        self.free_blocks.extend(blocks)

def select_victim(running):
    def score(req):
        return (req.priority, req.preempted_count, len(req.context_tokens()))

    return min(running, key=score)

def preempt_by_recompute(req, gpu_mgr, step):
    released = gpu_mgr.free_request(req)
    req.status = "PREEMPTED"

```



```

req.preempted_count += 1
req.last_preempted_at = step
return released, len(req.context_tokens())

def resume_by_recompute(req, gpu_mgr):
    context = req.context_tokens()
    ok = gpu_mgr.allocate_for_tokens(req, token_count=len(context))
    return ok, context

def swap_out(req, gpu_mgr, cpu_mgr):
    cpu_blocks = cpu_mgr.allocate(len(req.block_table))
    if cpu_blocks is None:
        return False, []
    copy_plan = list(zip(req.block_table, cpu_blocks))
    gpu_mgr.free_request(req)
    req.cpu_block_table = cpu_blocks
    req.status = "SWAPPED"
    return True, copy_plan

def swap_in(req, gpu_mgr, cpu_mgr):
    if len(gpu_mgr.free_blocks) < len(req.cpu_block_table):
        return False, []
    gpu_blocks = [gpu_mgr.free_blocks.pop(0) for _ in req.cpu_block_table]
    copy_plan = list(zip(req.cpu_block_table, gpu_blocks))
    cpu_mgr.free(req.cpu_block_table)
    req.cpu_block_table = []
    req.block_table = gpu_blocks
    req.status = "RUNNING"
    return True, copy_plan

gpu = GPUBlockManager(num_blocks=8, block_size=2)
low = RequestState("low", [101, 102, 103], [201, 202], priority=0)
high = RequestState("high", [301, 302], [401], priority=10)
burst = RequestState("burst", list(range(700, 708)), [], priority=5)

gpu.allocate_for_tokens(low, len(low.context_tokens()))
gpu.allocate_for_tokens(high, len(high.context_tokens()))
cached_block = gpu.hold_cached_free_block()
free_before = len(gpu.free_blocks)
required_blocks = gpu.required_blocks(len(burst.prompt_tokens))

evicted_cache_blocks = []
victims = []
freed_by_preemption = 0
recompute_tokens_total = 0
if free_before < required_blocks:
    evicted_cache_blocks = gpu.evict_cached_free()
if len(gpu.free_blocks) < required_blocks:
    victim = select_victim([low, high])
    released, recompute_tokens = preempt_by_recompute(victim, gpu, step=7)
    victims.append(victim.request_id)

```

```

    freed_by_preemption += len(released)
    recompute_tokens_total += recompute_tokens

gpu.allocate_for_tokens(burst, len(burst.prompt_tokens))
burst_blocks = list(burst.block_table)
gpu.free_request(burst)
resume_ok, low_recompute_context = resume_by_recompute(low, gpu)

swap_gpu = GPUBlockManager(num_blocks=3, block_size=2)
swap_cpu = CPUBlockManager(num_blocks=3)
swap_req = RequestState("swap", [501, 502, 503], [601], priority=1)
swap_gpu.allocate_for_tokens(swap_req, len(swap_req.context_tokens()))
swap_out_ok, swap_out_plan = swap_out(swap_req, swap_gpu, swap_cpu)
failed_gpu = GPUBlockManager(num_blocks=0, block_size=2)
swap_in_fail_ok, _ = swap_in(swap_req, failed_gpu, swap_cpu)
cpu_table_after_failed_swap_in = list(swap_req.cpu_block_table)
swap_in_ok, swap_in_plan = swap_in(swap_req, swap_gpu, swap_cpu)

metrics = {
    "preemption_total": len(victims),
    "preemption_recompute_total": len(victims),
    "preemption_swap_total": int(swap_out_ok and swap_in_ok),
    "preemption_freed_blocks": freed_by_preemption,
    "recompute_tokens_total": recompute_tokens_total,
    "swap_out_blocks": len(swap_out_plan),
    "swap_in_blocks": len(swap_in_plan),
    "cache_evictions_before_preempt": len(evicted_cache_blocks),
}
summary = {
    "free_before": free_before,
    "required_blocks": required_blocks,
    "cached_block": cached_block,
    "evicted_cache_blocks": evicted_cache_blocks,
    "free_after_eviction": free_before + len(evicted_cache_blocks),
    "victims": victims,
    "freed_by_preemption": freed_by_preemption,
    "burst_blocks": burst_blocks,
    "low_recompute_context": low_recompute_context,
    "low_resume_ok": resume_ok,
    "low_resume_blocks": list(low.block_table),
    "low_preempted_count": low.preempted_count,
    "swap_out_plan": swap_out_plan,
    "swap_in_fail_ok": swap_in_fail_ok,
    "cpu_table_after_failed_swap_in": cpu_table_after_failed_swap_in,
    "swap_in_plan": swap_in_plan,
    "swap_status_after_in": swap_req.status,
    "metrics": metrics,
}
gates = {
    "kv_pressure_detected": free_before < required_blocks,
    "eviction_before_preemption": evicted_cache_blocks == [5] and victims == ["low"],
    "victim_policy_ready": victims == ["low"],
    "recompute_context_preserved": low_recompute_context == [101, 102, 103, 201, 202],
    "recompute_resume_ready": resume_ok and len(low.block_table) == 3 and low.status == "RUNNING",
    "swap_failure_preserves_cpu": swap_in_fail_ok is False and cpu_table_after_failed_swap_in == [0, 1],

```

```

    "swap_in_ready": swap_in_ok and swap_req.status == "RUNNING" and swap_req.cpu_block_table == [],
    "metrics_ready": metrics["preemption_total"] == 1 and metrics["recompute_tokens_total"] == 5,
}
gates["preemption_upgrade_gate"] = all(gates.values())

print("preemption_upgrade_summary=", summary)
print("preemption_upgrade_gates=", gates)

```

一次运行的核心输出类似：

```

preemption_upgrade_summary= {'free_before': 2, 'required_blocks': 4, 'cached_block': 5, 'evicted_cache_blocks': [5], 'fr
preemption_upgrade_gates= {'kv_pressure_detected': True, 'eviction_before_preemption': True, 'victim_policy_ready': Tru

```

这个 demo 证明了几个关键点：

1. KV 不足时先 eviction，再 preempt active request。
2. victim 选择没有抢占高优先级请求。
3. recompute preemption 释放 GPU KV，但保留 prompt + output tokens。
4. 恢复时重算上下文是 [prompt, output]，不是只重算 prompt。
5. swap in 失败后 CPU block table 仍然保留，request 不会丢状态。
6. swap in 成功后 GPU block id 可以变化，但逻辑 block 顺序仍由新的 block table 表示。
7. 指标里能看到 preemption 次数、释放 blocks、重算 tokens 和 swap blocks。

51.31 小练习

1. 给 RequestState 增加 status、preempted_count、last_preempted_at 字段。
2. 实现 preempt_by_recompute(request, block_manager)。
3. 确保 preempt 后只释放 GPU KV，不丢失 prompt tokens 和 output tokens。
4. 恢复时用 prompt_token_ids + output_token_ids 作为 recompute context。
5. 修改 scheduler，在 free blocks 不足时触发 preemption。
6. 实现一个最简单的 select_victim。
7. 给 victim 选择增加 preempted_count 约束，避免反复 preempt 同一请求。
8. 增加 preemption_total 和 recompute_tokens_total 指标。
9. 增加 preemption 结构化日志。
10. 构造长 prompt 突发压测，观察 preemption 是否发生。
11. 开启 chunked prefill，对比 preemption 次数变化。
12. 构造 prefix cache 占用 block pool 的场景，验证先 eviction 再 preemption。
13. 设计 CPUBlockManager，实现同步版 swap out。
14. 实现同步版 swap in，并确保失败时不清空 CPU block table。
15. 写一段面试回答：为什么 preemption 不是性能优化？

51.32 本章总结

preemption 是 serving engine 在 KV cache 压力下保护系统稳定性的机制。

它的核心不是让系统更快，而是在资源不足时避免直接 OOM，并让请求后续仍有机会继续执行。

最小教学版建议先实现 recompute：preempt 时释放 request 的 GPU KV blocks，保留 prompt tokens、output tokens 和采样状态；恢复时重新 prefill prompt + output 重建 KV。

swap 可以减少重算，但需要 CPU KV buffer、GPU/CPU block 映射、异步拷贝、带宽控制和失败回滚，工程复杂度更高。

preemption 必须和 scheduler、paged KV cache、prefix cache eviction、chunked prefill、admission control 和指标体系一起设计。

频繁 preemption 是危险信号。

真正的优化方向通常是降低并发压力、缩短上下文、调整 max_num_seqs 和 max_num_batched_tokens、改善 chunked prefill、控制 prefix cache 占用，或者扩充 KV cache budget。

下一章可以继续升级 mini engine：把这些机制串成一个完整的调度循环，并讨论如何用系统化 benchmark 验证每次改造真的有效。

第 52 章 把调度、KV cache、prefix cache 和 preemption 串起来

前面几章我们按模块升级了 mini engine。

第 48 章把 naive scheduler 升级成 continuous batching。

第 49 章把 request 私有 list KV cache 升级成 paged KV cache。

第 50 章在 paged KV cache 上实现了 prefix cache。

第 51 章补上了 KV 空间不足时的 preemption、recompute 和 swap。

这些机制单独看都不难。

真正容易出问题的是：

它们在同一个 engine step 里怎么配合？

本章就把这些机制串成一个完整调度循环。

52.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 vLLM-like engine step 如何把 continuous batching、paged KV cache、prefix cache、chunked prefill 和 preemption 串成一个统一调度循环。

公开资料校准主要参考四类口径：

1. vLLM scheduler / optimization 文档对 decode 优先、chunked prefill、max_num_batched_tokens、max_num_seqs、KV cache usage 和 preemption 的公开说明。
2. vLLM prefix caching 设计文档对 KVCacheBlock、block hash、parent hash、ref count、cached-free block、free queue 和 eviction 的公开说明。
3. vLLM / PagedAttention 论文对 block-based KV cache、logical block 到 physical block 映射、动态 serving request 和高吞吐调度的基础抽象。
4. 本书第 48 到 51 章对 continuous batching、paged KV cache、prefix cache、preemption、recompute 和 swap 的教学版实现边界。

本章不复刻某个框架版本的源码字段、线程模型、CUDA kernel、multi-worker 通信、KV connector、真实 swap 后端、分布式 prefix cache 或完整生产优先级策略。我们只验证一个最小闭环：

waiting/running 队列 -> prefix lookup once -> decode-first -> suffix prefill -> token/KV budget -> eviction/preemption -> commit plan -> BatchBuilder metadata -> OutputProcessor 状态更新 -> invariants

第二轮新增 demo 的验收重点是：

prefix cache lookup 是否只对 waiting 请求做一次；
decode 是否先拿到预算，避免 streaming TPOT 抖动；
prefix hit 后是否只调度 suffix prefill；
token budget 和 KV block budget 是否同时生效；
KV 压力下是否先 evict cached-free block，再 preempt active request；
commit plan 是否先分配 block，再让 BatchBuilder 读 metadata；
positions、slot mapping、block table 是否按真实上下文位置构造；
OutputProcessor 是否正确更新 output token 和 num_computed_tokens；
队列、block、computed token、metadata 和 cleanup 不变量是否能被检查。

52.1 本章目标

读完本章，你应该能讲清：

1. 一个 vLLM-like engine step 里 scheduler 大致做哪些事。

2. waiting、running、preempted、finished 请求如何流转。
3. prefix cache lookup 应该发生在什么时候。
4. KV block 预算应该在调度前还是调度后计算。
5. decode、prefill、chunked prefill 的优先级如何取舍。
6. free blocks 不够时，cache eviction 和 preemption 的顺序如何安排。
7. BatchBuilder 如何把 scheduler 决策变成模型可执行的 batch metadata。
8. ModelRunner、Sampler、OutputProcessor 如何更新 request 状态。
9. 哪些日志和断言能帮助定位调度 bug。
10. 面试里如何描述一个完整 serving engine 的主循环。

这一章不是引入新概念。

它的目标是把前面所有零件装成一台能转起来的机器。

52.2 先看完整循环

一个最小 engine loop 可以抽象成：

```
while engine_has_work():
    new_requests = api_server.pop_new_requests()
    scheduler.add_requests(new_requests)

    schedule = scheduler.schedule()
    if schedule.is_empty():
        wait_for_new_work()
        continue

    batch = batch_builder.build(schedule)
    model_outputs = model_runner.forward(batch)
    sampled_tokens = sampler.sample(model_outputs, batch)
    output_processor.process(schedule, sampled_tokens)
```

这段代码看起来简单。

但复杂度都藏在 scheduler.schedule() 和 output_processor.process() 里。

其中 scheduler 负责回答：

这一轮哪些请求可以执行？每个请求执行 prefill 还是 decode？需要多少 token budget 和 KV blocks？

OutputProcessor 负责回答：

模型执行完之后，请求状态、KV blocks、prefix cache、streaming 输出和 metrics 怎么更新？

52.3 request 队列

先定义几个队列。

```
class Scheduler:
    def __init__(self):
        self.waiting = deque()
        self.running = []
        self.swapped = deque()
        self.finished = []
```

这里 waiting 包含三类请求：

1. 新进入 engine、还没 prefill 的请求。
2. recompute preemption 后等待恢复的请求。
3. prefix cache lookup 完成但还没被调度的请求。

running 是当前持有 GPU KV、可以继续 prefill 或 decode 的请求。

swapped 是 KV 在 CPU 上、等待 swap in 的请求。

finished 通常不会长期留在 scheduler 里，只是为了本轮统一清理和统计。

教学版如果不实现 swap，可以先没有 swapped。

52.4 request 的最小状态

request 需要保存逻辑状态和执行状态。

```
class RequestState:
    def __init__(self, request_id, prompt_token_ids, sampling_params):
        self.request_id = request_id
        self.prompt_token_ids = prompt_token_ids
        self.output_token_ids = []
        self.sampling_params = sampling_params

        self.status = RequestStatus.WAITING
        self.block_table = []
        self.num_computed_tokens = 0

        self.prefix_hit_tokens = 0
        self.preempted_count = 0
        self.arrival_time = now_ms()
        self.last_scheduled_time = None
```

这里最关键的字段是 num_computed_tokens。

它表示：

当前 request 已经有多少上下文 tokens 的 KV 可用。

不同机制都会修改它：

1. 新请求初始为 0。
2. prefix cache 命中后变成 hit tokens。
3. prefill chunk 执行后增加 chunk size。
4. decode 生成一个 token 后增加 1。
5. recompute preemption 后重置为 0，或者重新经过 prefix cache lookup 后变成 hit tokens。

只要 num_computed_tokens 维护错，后面 positions、slot mapping、attention metadata 都会错。

52.5 scheduler 每轮要做的事

一个完整的 schedule() 可以分成八步：

1. 清理已经 finished / aborted 的请求。
2. 尝试 swap in 或恢复 preempted 请求。
3. 对新 waiting 请求做 prefix cache lookup。
4. 预留 running decode 的 KV 增长预算。
5. 在 token budget 内选择 prefill chunks。
6. 如果 KV blocks 不够，先 cache eviction，再 preemption。
7. 生成本轮 schedule plan。
8. 更新被选中请求的临时调度状态。

不要把这些步骤写成一团。

工程上最容易 debug 的方式是让 schedule() 返回一个明确的 plan。

```

class SchedulePlan:
    def __init__(self):
        self.prefill_items = []
        self.decode_items = []
        self.preempted = []
        self.evicted_blocks = []

```

plan 是 scheduler 和 BatchBuilder 之间的契约。

52.6 decode 优先还是 prefill 优先

在线 serving 通常会偏向 decode 优先。

原因是 decode 决定 TPOT。

如果已经在 streaming 的请求长时间拿不到 decode step，用户会看到输出卡顿。

所以常见策略是：

先为 running decode 请求预留预算，再用剩余 budget 安排 waiting prefill。

伪代码：

```

decode_items = select_running_decode_requests(running)
remaining_token_budget = max_num_batched_tokens - len(decode_items)
prefill_items = select_prefill_chunks(waiting, remaining_token_budget)

```

但这不是绝对规则。

如果完全偏向 decode，新请求 TTFT 会变差。

如果过度偏向 prefill，已有 streaming 请求 TPOT 会抖。

调度的本质就是在 TTFT、TPOT、吞吐和公平性之间做权衡。

52.7 token budget 和 block budget

scheduler 至少要同时管两个 budget。

第一个是 token budget。

本轮最多处理多少 input tokens。

它主要由 max_num_batched_tokens 控制。

第二个是 block budget。

本轮最多能分配多少新的 KV blocks。

它由 block pool 的 free blocks 决定。

两者不能互相替代。

例如 decode 一轮只处理 1 个 token，但如果这个 token 正好跨过 block 边界，就需要新分配一个 block。

又比如 prefix cache 命中 8K tokens，token budget 可能只消耗 suffix 128 tokens，但 block table 已经引用了大量 cached blocks。

所以调度时要同时检查：

```

if used_tokens + new_tokens > max_num_batched_tokens:
    reject_this_chunk()

if required_new_blocks > free_blocks:
    handle_memory_pressure()

```

52.8 prefix cache lookup 的时机

prefix cache lookup 应该在 waiting 请求第一次参与调度前完成。

流程是：

new request -> tokenize -> waiting queue -> prefix cache lookup -> update block_table / num_computed_tokens -> schedule suffix prefill

伪代码：

```
def prepare_waiting_request(req):
    if req.prefix_lookup_done:
        return

    hit_blocks, hit_tokens = block_manager.find_cached_prefix(req.prompt_token_ids)
    if hit_blocks:
        block_manager.add_active_refs(hit_blocks)
        req.block_table.extend(hit_blocks)
        req.num_computed_tokens = hit_tokens
        req.prefix_hit_tokens = hit_tokens

    req.prefix_lookup_done = True
```

prefix cache lookup 不应该每轮都重复做。

否则会引入额外开销，也可能重复增加 ref count。

如果 request 被 recompute preempt，释放了 active refs 后，可以把 prefix_lookup_done 重置，让它恢复时重新查一次 prefix cache。

因为这期间 cache 状态可能已经变化。

52.9 prefill chunk 怎么选

对 waiting 请求，scheduler 不一定一次 prefill 完。

它可以选择一个 chunk。

```
def next_prefill_chunk(req, remaining_token_budget):
    context_tokens = req.prompt_token_ids + req.output_token_ids
    start = req.num_computed_tokens
    remaining = len(context_tokens) - start
    chunk_size = min(remaining, remaining_token_budget, max_prefill_chunk_size)
    return start, chunk_size
```

注意这里用的是 context_tokens，不是只有 prompt。

对普通新请求，output_token_ids 为空。

对 recompute 恢复请求，output_token_ids 可能不为空。

因此这一个函数同时覆盖新请求和 recompute 请求。

52.10 running 请求何时 decode

一个 request 可以 decode 的前提是：

num_computed_tokens == len(prompt_token_ids) + len(output_token_ids)

也就是完整上下文 KV 已经可用。

如果还没 prefill 完，它仍然处于 prefill 阶段。

伪代码：


```
def is_ready_to_decode(req):
    return req.num_computed_tokens == len(req.prompt_token_ids) + len(req.output_token_ids)
```

decode 后会采样出一个新 token。

这个 token 在下一轮成为上下文的一部分。

所以 OutputProcessor 要做：

```
req.output_token_ids.append(next_token)
req.num_computed_tokens += 1
```

这里有一个容易混淆的点。

decode step 的输入是上一 token 或最后一个 prompt token，但执行后生成的是新 token 的 logits。

工程实现里 num_computed_tokens 如何增加，取决于 KV 写入的是哪个 token 的 KV。

教学版可以采用简化模型：每轮 decode 会为新加入上下文的 token 补齐 KV，并让 num_computed_tokens 增加 1。

真实框架里要更严格地区分 input token、sampled token 和下一轮 decode token。

面试时重点讲状态一致性，不必纠缠某个教学伪代码的 off-by-one。

52.11 KV block 分配的统一入口

不要让 BatchBuilder、ModelRunner、OutputProcessor 到处直接分配 block。

建议所有 KV 分配都经过 BlockManager。

```
class BlockManager:
    def ensure_blocks_for_tokens(self, req, target_num_tokens):
        required_blocks = ceil_div(target_num_tokens, self.block_size)

        while len(req.block_table) < required_blocks:
            block_id = self.allocate_one()
            req.block_table.append(block_id)
```

scheduler 在生成 plan 前先做预算。

真正分配可以发生在 schedule commit 阶段。

这样 BatchBuilder 拿到的 request 已经有足够 block table。

```
def commit_plan(plan):
    for item in plan.prefill_items:
        target = item.start + item.num_tokens
        block_manager.ensure_blocks_for_tokens(item.request, target)

    for item in plan.decode_items:
        target = len(item.request.prompt_token_ids) + len(item.request.output_token_ids) + 1
        block_manager.ensure_blocks_for_tokens(item.request, target)
```

不要等 kernel 执行到一半才发现 block 不够。

52.12 memory pressure 处理顺序

当 block budget 不够时，推荐顺序是：

1. 缩小本轮 prefill chunk。
2. 暂缓接纳部分 waiting 请求。
3. 淘汰无 active refs 的 prefix cache blocks。
4. preempt 低优先级 running 请求。
5. 触发入口 backpressure 或拒绝新请求。

伪代码：

```
def handle_memory_pressure(required_blocks):
    shrink_prefill_plan()

    if enough_blocks(required_blocks):
        return

    block_manager.evict_cached_blocks_until(required_blocks)

    if enough_blocks(required_blocks):
        return

    preempt_until_enough(required_blocks)

    if not enough_blocks(required_blocks):
        enable_backpressure()
```

这不是唯一顺序。

但它体现了一个原则：

先减少新增压力，再释放可牺牲缓存，最后才动 active request。

52.13 schedule plan 的数据结构

plan item 要明确告诉 BatchBuilder 做什么。

```
class PrefillItem:
    def __init__(self, request, start, num_tokens):
        self.request = request
        self.start = start
        self.num_tokens = num_tokens

class DecodeItem:
    def __init__(self, request):
        self.request = request

class SchedulePlan:
    def __init__(self):
        self.prefill_items = []
        self.decode_items = []
```

PrefillItem.start 很重要。

它可能不是 0。

常见原因：

1. prefix cache 命中了前缀。
2. chunked prefill 已经完成前几个 chunk。
3. recompute 恢复时部分上下文再次命中 prefix cache。

BatchBuilder 必须从 start 开始构造 input ids、positions 和 slot mapping。

52.14 BatchBuilder 做什么

BatchBuilder 把 plan 转成模型执行需要的 batch。

它通常构造：

1. input token ids。
2. positions。
3. request 到 batch row 的映射。
4. block tables。
5. slot mapping。
6. attention metadata。

对 prefill item:

```
tokens = context_tokens[item.start : item.start + item.num_tokens]
positions = range(item.start, item.start + item.num_tokens)
```

对 decode item:

```
tokens = [last_context_token(req)]
positions = [len(req.prompt_token_ids) + len(req.output_token_ids) - 1]
```

不同框架对 decode token 的定义略有不同。

但无论如何，positions 不能因为 batch 拼接而从 0 重置。

这是 BatchBuilder 最容易犯的错误之一。

52.15 OutputProcessor 做什么

模型执行后，OutputProcessor 要更新系统状态。

对 prefill item:

```
req.num_computed_tokens += item.num_tokens
```

```
if req.num_computed_tokens == len(req.prompt_token_ids) + len(req.output_token_ids):
    req.status = RequestStatus.RUNNING
```

如果 prefill 完成的是 full block，还可以注册 prefix cache。

```
block_manager.register_new_full_blocks(req)
```

对 decode item:

```
token = sampled_tokens[req.request_id]
req.output_token_ids.append(token)
req.num_computed_tokens += 1
stream(token)
```

```
if should_stop(req):
    finish_request(req)
```

finish 时要释放 active KV refs。

```
def finish_request(req):
    block_manager.free_request_blocks(req)
    req.status = RequestStatus.FINISHED
```

如果某些 blocks 已经注册到 prefix cache，它们不会立即回到 free list。

52.16 一个完整 schedule 伪代码

把前面拼起来，可以得到一个教学版 scheduler。

```
def schedule(self):
    self.cleanup_finished()
    self.restore_preempted_requests()
```

```

for req in self.waiting:
    self.prepare_prefix_cache(req)

plan = SchedulePlan()
token_budget = self.max_num_batched_tokens

for req in self.running:
    if self.is_ready_to_decode(req):
        plan.decode_items.append(DecodeItem(req))
        token_budget -= 1

        if token_budget == 0:
            break

for req in list(self.waiting):
    if token_budget <= 0:
        break

    start, n = self.next_prefill_chunk(req, token_budget)
    if n == 0:
        continue

    plan.prefill_items.append(PrefillItem(req, start, n))
    token_budget -= n

required_blocks = self.estimate_new_blocks(plan)
self.handle_memory_pressure(required_blocks, plan)
self.commit_plan(plan)

return plan

```

真实系统会复杂很多。

但主干就是这几步。

52.17 commit plan 为什么重要

scheduler 不能只“想好”要执行哪些请求。

它还要把状态推进到一个可执行的中间状态。

commit plan 至少要做：

1. 为 plan 中的请求分配需要的 KV blocks。
2. 把 waiting 请求移动到 running。
3. 标记本轮被调度的 token 范围。
4. 记录 last scheduled time。
5. 更新统计指标。

伪代码：

```

def commit_plan(plan):
    for item in plan.prefill_items:
        req = item.request
        target = item.start + item.num_tokens
        block_manager.ensure_blocks_for_tokens(req, target)
        move_to_running_if_needed(req)
        req.last_scheduled_time = now_ms()

```

```

for item in plan.decode_items:
    req = item.request
    target = len(req.prompt_token_ids) + len(req.output_token_ids) + 1
    block_manager.ensure_blocks_for_tokens(req, target)
    req.last_scheduled_time = now_ms()

```

commit 成功后, BatchBuilder 才能安全读取 block table。

52.18 失败要能回滚

如果 commit plan 过程中分配 block 失败怎么办?

最差的实现是分配了一半, 然后直接抛异常。

这会留下脏状态。

更好的方式是让分配具备事务感:

```

allocated = []
try:
    for req in requests:
        new_blocks = block_manager.allocate_for(req)
        allocated.extend((req, new_blocks))
except AllocationError:
    for req, blocks in allocated:
        block_manager.free_blocks_from_request(req, blocks)
    raise

```

教学版可以简单一点:

先 estimate, 确认足够后再 allocate。

但要知道 estimate 和真实 allocate 之间可能有差异。

比如 cached blocks 的 ref count、并发修改、异步 abort 都可能改变实际 free blocks。

单线程教学引擎里问题不大, 多线程真实引擎里必须小心。

52.19 状态不变量

复杂 scheduler 必须有 invariants。

建议每轮结束后检查:

1. 一个 request 只能出现在一个队列里。
2. running request 必须有合法 block table。
3. num_computed_tokens <= len(prompt_token_ids) + len(output_token_ids)。
4. block ref count 不能为负。
5. free list 里不能包含 active request 正在引用的 block。
6. cached block 如果在 cache map 中, 不能同时作为普通 free block。
7. prefix cache hit tokens 必须是 block size 的整数倍, 除非明确支持 partial block。
8. preempted request 不能持有 GPU-only active refs。
9. swapped request 不能同时持有 GPU block table 和 CPU block table, 除非处于迁移中间态。
10. finished request 不能留在 running 队列。

这些断言比很多日志都有用。

它们可以在 bug 刚出现时立刻失败, 而不是等生成结果错乱才发现。

52.20 可观测性：每轮调度日志

建议每轮 scheduler 输出一条 debug 级别结构化日志。

字段包括：

字段	含义
step_id	engine step 编号
num_waiting	waiting 请求数
num_running	running 请求数
num_swapped	swapped 请求数
prefill_reqs	本轮 prefill 请求数
prefill_tokens	本轮 prefill tokens
decode_reqs	本轮 decode 请求数
free_blocks_before	调度前 free blocks
free_blocks_after	commit 后 free blocks
cached_blocks	prefix cache blocks
evicted_blocks	本轮 eviction blocks
preempted_reqs	本轮 preempt 请求数

示例：

```
schedule step=1024 waiting=8 running=64 prefill_reqs=2 prefill_tokens=512 \
decode_reqs=62 free_blocks_before=120 free_blocks_after=88 cached_blocks=300 \
evicted_blocks=0 preempted_reqs=0
```

当 TTFT 或 TPOT 抖动时，这条日志能快速告诉你：

1. 是 waiting 太多。
2. 还是 decode 太少。
3. 还是 free blocks 快没了。
4. 还是 preemption/eviction 在频繁发生。

52.21 指标分层

不要只暴露一个总吞吐。

至少分四层指标。

第一层，请求层：

1. QPS。
2. TTFT p50/p99。
3. TPOT p50/p99。
4. E2E latency p50/p99。
5. active requests。
6. waiting queue length。

第二层，调度层：

1. scheduled_prefill_tokens。
2. scheduled_decode_tokens。
3. max_num_batched_tokens utilization。
4. running requests。
5. preemptions。
6. chunked prefill count。

第三层，KV 层：

1. free blocks。

2. used blocks。
3. cached blocks。
4. allocation failures。
5. block reuse rate。
6. eviction count。

第四层，模型执行层：

1. forward latency。
2. GPU utilization。
3. input tokens/s。
4. output tokens/s。
5. kernel time。
6. sampling time。

分层之后，问题定位会清楚很多。

52.22 压测场景一：稳定短请求

workload:

prompt 128 tokens

output 128 tokens

固定并发

无共享 prefix

预期：

1. waiting queue 稳定。
2. preemption 为 0。
3. prefix cache hit ratio 接近 0。
4. free blocks 在稳定区间波动。
5. decode batch size 接近 running 请求数。

这个场景是 baseline。

如果 baseline 都不稳定，不要急着测复杂场景。

52.23 压测场景二：共享 system prompt

workload:

system prompt 2K tokens 固定

user prompt 64 tokens 随机

output 128 tokens

预期：

1. prefix cache hit tokens 上升。
2. TTFT 下降。
3. scheduled_prefill_tokens 中真实计算部分下降。
4. cached blocks 上升后趋于稳定。
5. preemption 不应该明显增加。

如果 TTFT 没下降，检查 prefix cache lookup、ref count、suffix prefill start position。

如果 preemption 增加，检查 cached block eviction 水位线。

52.24 压测场景三：长上下文混合短请求

workload:

80% 请求: prompt 128 tokens, output 128 tokens

20% 请求: prompt 16K tokens, output 512 tokens

预期:

1. chunked prefill 生效。
2. 短请求 TTFT 不被长请求完全拖垮。
3. prefill tokens 和 decode tokens 在每轮都有合理比例。
4. free blocks 下降但不频繁归零。
5. preemption 只在压力峰值偶发。

这个场景最能验证完整 scheduler 的工程质量。

52.25 压测场景四：故意打爆 KV

workload:

逐步增加并发和 max output tokens

直到 KV block pool 接近耗尽

预期不是 “永远不 preempt”。

预期是:

1. allocation failures 不导致 engine crash。
2. 先出现 chunk 缩小或 waiting 增长。
3. 然后出现 cache eviction。
4. 最后才出现 preemption。
5. preempted request 能恢复并完成, 或被明确 abort。

这个场景验证鲁棒性。

不要只测性能好看的区间。

52.26 常见 bug

bug 一: prefix cache lookup 每轮重复执行。

结果: ref count 重复增加, cached block 永远无法释放。

bug 二: chunked prefill 的 positions 从 0 开始。

结果: RoPE position 错误, 长 prompt 输出异常。

bug 三: recompute 请求只 prefill prompt。

结果: 已生成 output tokens 没有进入 KV, 上下文断裂。

bug 四: commit plan 后 waiting/running 队列重复持有同一请求。

结果: 同一个 request 被重复调度。

bug 五: 估算 block 足够, 但实际分配失败后没有回滚。

结果: block table 半更新, free list 和 ref count 不一致。

bug 六: finish request 时把 cached block 放回 free list。

结果: prefix cache 悬挂引用。

bug 七: 过度 decode 优先。

结果: TPOT 稳定但新请求 TTFT 很差。

bug 八: 过度 prefill 优先。

结果: TTFT 看似不错, 但 streaming 输出卡顿。

52.27 面试高频问题

问题一：一个 serving engine 的 scheduler 每轮大概做什么？

回答要点：每轮在 iteration boundary 上重新选择 batch。它会处理 waiting/running 请求，做 prefix cache lookup, 优先安排 running decode, 再用剩余 token budget 安排 prefill chunk, 同时检查 KV block budget。如果 free blocks 不够，先缩小 prefill 或推迟请求，再做 cache eviction, 最后 preempt running 请求。最后生成 schedule plan 给 BatchBuilder。

问题二：为什么 scheduler 要同时看 token budget 和 KV block budget？

回答要点：token budget 控制本轮计算量，KV block budget 控制显存资源。decode 可能只加一个 token 但跨 block 边界需要新 block；prefix cache 命中会减少计算 tokens，但仍引用 KV blocks。两者约束不同，必须同时管理。

问题三：prefix cache lookup 放在哪里？

回答要点：通常在 waiting 请求第一次参与调度前做。命中后更新 request 的 block table、active refs、num_computed_tokens 和 hit tokens，然后 scheduler 只调度 suffix prefill。不能每轮重复 lookup，否则 ref count 和开销都会出问题。

问题四：BatchBuilder 最容易出什么错？

回答要点：最容易把 positions 从 0 重置，或者忽略 prefill chunk 的 start offset。prefix cache、chunked prefill、recompute 都可能导致 prefill 从非零位置开始，因此 input ids、positions、slot mapping 和 block table 必须按真实上下文位置构造。

问题五：如何判断调度策略有问题？

回答要点：要看 TTFT、TPOT、E2E latency、waiting queue、running queue、scheduled prefill/decode tokens、free blocks、cached blocks、evictions、preemptions 和 allocation failures。如果 TTFT 高，可能新请求进不来或 prefill budget 太小；如果 TPOT 抖，可能 decode 被 prefill 挤压；如果 preemption 高，说明 KV budget 或并发参数有问题。

52.28 标准回答模板

如果面试官问“你怎么设计一个 vLLM-like scheduler 主循环”，可以这样回答：

我会把 engine 看成 iteration-level 调度系统。每个 engine step 在 GPU forward 结束后重新调度一次，scheduler 维护 waiting、

每轮调度时，我会优先考虑 running 请求的 decode，保证 streaming TPOT 不抖；然后用剩余 max_num_batched_tokens 选择 waiting

如果 KV blocks 不够，我不会直接 OOM。处理顺序一般是缩小本轮 prefill chunk、推迟部分 waiting 请求、淘汰无 active refs 的 p

BatchBuilder 根据 schedule plan 构造 input ids、positions、slot mapping、block tables 和 attention metadata。这里最关键的

最后我会用 invariants 和分层指标保证系统可 debug，比如一个 request 只能在一个队列里，ref count 不能为负，free list 不能

52.29 统一调度循环公式、状态不变量和可运行 demo

把本章机制合起来后，可以先把每轮调度的 token 用量写成：

$$U_t = |D_t| + \sum_{i \in P_t} c_{i,t}$$

其中 D_t 是本轮 decode 请求集合， P_t 是本轮 prefill 请求集合， $c_{i,t}$ 是请求 i 本轮 prefill chunk tokens。

token budget 约束是：

$$U_t \leq B_{\mathrm{tok}}$$

新增 KV block 需求是：

$$K_t^{\mathrm{need}} = \sum_{i \in D_t \cup P_t} \max(0, \left\lceil \frac{T_{i,t}^{\mathrm{target}}}{S} \right\rceil - |P_i|)$$

其中 S 是 block size, $T_{i,t}^{\mathrm{target}}$ 是本轮执行后请求 i 至少需要覆盖到的上下文长度, $|P_i|$ 是当前 block table 长度。

KV 压力门可以写成:

$$G_{\mathrm{pressure}} = \mathbf{1}[K_t^{\mathrm{need}} > F_t]$$

其中 F_t 是调度前可直接分配的 free blocks。

prefix 命中后的剩余 prefill token 数是:

$$R_i^{\mathrm{prefill}} = T_i - T_i^{\mathrm{hit}}$$

每轮不变量门禁可以拆成:

$$G_{\mathrm{invariant}} = G_{\mathrm{queue}} G_{\mathrm{block}} G_{\mathrm{computed}} G_{\mathrm{metadata}} G_{\mathrm{clear}}$$

最终统一调度循环门禁:

$$G_{\mathrm{loop}} = G_{\mathrm{prefix}} G_{\mathrm{decode}} G_{\mathrm{prefill}} G_{\mathrm{budget}} G_{\mathrm{pressure}}$$

下面这个 0 依赖 demo 模拟一个完整 engine step:

1. cache_hit 只做一次 prefix lookup, 命中 4 个 tokens。
2. stream 作为 running streaming 请求优先 decode。
3. cache_hit 只 prefill suffix, cache_miss 从 0 开始 prefill。
4. 本轮需要 3 个新 KV blocks, 但 free list 只有 1 个。
5. scheduler 先 evict cached-free block, 再 preempt 低优先级 running 请求。
6. commit plan 统一分配 blocks, BatchBuilder 再构造 positions、slot mapping 和 block tables。
7. OutputProcessor 追加 decode token, 推进 prefill computed tokens, 并检查状态不变量。

```
from dataclasses import dataclass, field
```

```
def ceil_div(a, b):
    return (a + b - 1) // b

@dataclass
class RequestState:
    request_id: str
    prompt_tokens: list
    output_tokens: list = field(default_factory=list)
    priority: int = 0
    status: str = "WAITING"
    block_table: list = field(default_factory=list)
    num_computed_tokens: int = 0
    prefix_lookup_done: bool = False
    prefix_hit_tokens: int = 0
    want_decode: bool = False
    preempted_count: int = 0

    def context_tokens(self):
        return self.prompt_tokens + self.output_tokens
```

```
@dataclass
class PrefillItem:
    request: RequestState
    start: int
    num_tokens: int
```

```
@dataclass
```

```
class DecodeItem:  
    request: RequestState
```

```
@dataclass
```

```
class SchedulePlan:  
    prefill_items: list = field(default_factory=list)  
    decode_items: list = field(default_factory=list)  
    evicted_blocks: list = field(default_factory=list)  
    preempted: list = field(default_factory=list)  
    commit_allocations: dict = field(default_factory=dict)
```

```
class ToyUnifiedBlockManager:
```

```
    def __init__(self, block_size):  
        self.block_size = block_size  
        self.free_blocks = [7]  
        self.cached_free_blocks = [6]  
        self.prefix_cache = {"shared": [5]}  
        self.active_ref = {0: 1, 1: 1, 2: 1, 3: 1, 4: 1, 5: 0}
```

```
    def required_blocks(self, token_count):  
        return ceil_div(token_count, self.block_size)
```

```
    def physical_slot(self, req, position):  
        block_index = position // self.block_size  
        offset = position % self.block_size  
        return req.block_table[block_index] * self.block_size + offset
```

```
    def lookup_prefix_once(self, req):  
        if req.prefix_lookup_done:  
            return req.prefix_hit_tokens  
        if req.request_id == "cache_hit":  
            hit_blocks = list(self.prefix_cache["shared"])  
            req.block_table.extend(hit_blocks)  
            req.num_computed_tokens = self.block_size  
            req.prefix_hit_tokens = self.block_size  
            for block_id in hit_blocks:  
                self.active_ref[block_id] = self.active_ref.get(block_id, 0) + 1  
        req.prefix_lookup_done = True  
        return req.prefix_hit_tokens
```

```
    def estimate_new_blocks(self, plan):  
        total = 0  
        for item in plan.decode_items:  
            req = item.request  
            target = len(req.context_tokens()) + 1  
            total += max(0, self.required_blocks(target) - len(req.block_table))  
        for item in plan.prefill_items:  
            req = item.request  
            target = item.start + item.num_tokens  
            total += max(0, self.required_blocks(target) - len(req.block_table))
```

```

    return total

def evict_cached_free_until_visible(self):
    if not self.cached_free_blocks:
        return []
    block_id = self.cached_free_blocks.pop(0)
    self.free_blocks.append(block_id)
    return [block_id]

def free_request_blocks(self, req):
    released = list(req.block_table)
    for block_id in released:
        self.active_ref[block_id] = max(0, self.active_ref.get(block_id, 1) - 1)
    self.free_blocks.extend(released)
    req.block_table = []
    req.num_computed_tokens = 0
    return released

def allocate_until(self, req, target_tokens):
    required = self.required_blocks(target_tokens)
    allocated = []
    while len(req.block_table) < required:
        block_id = self.free_blocks.pop(0)
        req.block_table.append(block_id)
        self.active_ref[block_id] = self.active_ref.get(block_id, 0) + 1
        allocated.append(block_id)
    return allocated

class ToyUnifiedScheduler:
    def __init__(self, block_mgr, waiting, running, max_tokens):
        self.block_mgr = block_mgr
        self.waiting = list(waiting)
        self.running = list(running)
        self.preempted = []
        self.max_tokens = max_tokens
        self.prefix_hits = {}
        self.free_before = 0
        self.required_new_blocks = 0

    def ready_to_decode(self, req):
        return req.want_decode and req.num_computed_tokens == len(req.context_tokens())

    def next_prefill_chunk(self, req, token_budget):
        start = req.num_computed_tokens
        remaining = len(req.context_tokens()) - start
        return start, min(remaining, token_budget)

    def preempt_low_priority(self):
        candidates = [req for req in self.running if not req.want_decode]
        victim = min(candidates, key=lambda req: (req.priority, req.preempted_count))
        context = victim.context_tokens()
        released = self.block_mgr.free_request_blocks(victim)
        victim.status = "PREEMPTED"
        victim.preempted_count += 1

```

```

self.running.remove(victim)
self.preempted.append(victim)
return victim.request_id, released, context

def schedule(self):
    for req in self.waiting:
        self.prefix_hits[req.request_id] = self.block_mgr.lookup_prefix_once(req)

    plan = SchedulePlan()
    used_tokens = 0
    for req in self.running:
        if self.ready_to_decode(req):
            plan.decode_items.append(DecodeItem(req))
            used_tokens += 1

    for req in list(self.waiting):
        if used_tokens >= self.max_tokens:
            break
        start, chunk = self.next_prefill_chunk(req, self.max_tokens - used_tokens)
        if chunk:
            plan.prefill_items.append(PrefillItem(req, start, chunk))
            used_tokens += chunk

    self.free_before = len(self.block_mgr.free_blocks)
    self.required_new_blocks = self.block_mgr.estimate_new_blocks(plan)
    if self.required_new_blocks > len(self.block_mgr.free_blocks):
        plan.evicted_blocks = self.block_mgr.evict_cached_free_until_visible()
    if self.required_new_blocks > len(self.block_mgr.free_blocks):
        plan.preempted.append(self.preempt_low_priority())

    for item in plan.decode_items:
        req = item.request
        target = len(req.context_tokens()) + 1
        plan.commit_allocations[req.request_id] = self.block_mgr.allocate_until(req, target)
    for item in plan.prefill_items:
        req = item.request
        target = item.start + item.num_tokens
        plan.commit_allocations[req.request_id] = self.block_mgr.allocate_until(req, target)
        req.status = "RUNNING"
        if req in self.waiting:
            self.waiting.remove(req)
            self.running.append(req)
    return plan

class ToyBatchBuilder:
    def __init__(self, block_mgr):
        self.block_mgr = block_mgr

    def build(self, plan):
        positions = []
        slot_mapping = []
        block_tables = []
        for item in plan.decode_items:
            req = item.request

```

```

        position = len(req.context_tokens())
        positions.append(position)
        slot_mapping.append(self.block_mgr.physical_slot(req, position))
        block_tables.append((req.request_id, list(req.block_table)))
    for item in plan.prefill_items:
        req = item.request
        for position in range(item.start, item.start + item.num_tokens):
            positions.append(position)
            slot_mapping.append(self.block_mgr.physical_slot(req, position))
            block_tables.append((req.request_id, list(req.block_table)))
    return {
        "positions": positions,
        "slot_mapping": slot_mapping,
        "block_tables": block_tables,
    }

```

class ToyOutputProcessor:

```

    def process(self, plan, sampled_tokens):
        for item in plan.decode_items:
            req = item.request
            req.output_tokens.append(sampled_tokens[req.request_id])
            req.num_computed_tokens += 1
        for item in plan.prefill_items:
            req = item.request
            req.num_computed_tokens = item.start + item.num_tokens

```

def check_invariants(scheduler, metadata, block_mgr):

```

    queues = {
        "waiting": scheduler.waiting,
        "running": scheduler.running,
        "preempted": scheduler.preempted,
    }
    memberships = []
    for queue_name, queue in queues.items():
        memberships.extend((req.request_id, queue_name) for req in queue)
    request_ids = [item[0] for item in memberships]
    active_blocks = []
    for req in scheduler.running:
        active_blocks.extend(req.block_table)
    all_requests = scheduler.waiting + scheduler.running + scheduler.preempted
    return {
        "queue_ownership": len(request_ids) == len(set(request_ids)),
        "no_duplicate_active_blocks": len(active_blocks) == len(set(active_blocks)),
        "computed_not_beyond_context": all(
            req.num_computed_tokens <= len(req.context_tokens()) for req in all_requests
        ),
        "free_blocks_not_active": set(block_mgr.free_blocks).isdisjoint(active_blocks),
        "prefix_hit_block_aligned": all(
            req.prefix_hit_tokens % block_mgr.block_size == 0
            for req in all_requests
            if req.prefix_hit_tokens
        ),
        "metadata_rows_consistent": (

```

```

        len(metadata["positions"]) == len(metadata["slot_mapping"])
        and len(metadata["block_tables"]) == 3
    ),
    "preempted_cleanup_done": all(not req.block_table for req in scheduler.preempted),
}

stream = RequestState(
    "stream",
    prompt_tokens=[1, 2, 3, 4],
    output_tokens=[11, 12],
    status="RUNNING",
    block_table=[0, 1],
    num_computed_tokens=6,
    priority=10,
    want_decode=True,
)
low = RequestState(
    "low",
    prompt_tokens=[5, 6, 7, 8, 9, 10, 11, 12],
    output_tokens=[21, 22],
    status="RUNNING",
    block_table=[2, 3, 4],
    num_computed_tokens=10,
    priority=0,
)
cache_hit = RequestState("cache_hit", prompt_tokens=[101, 102, 103, 104, 105, 106])
cache_miss = RequestState("cache_miss", prompt_tokens=[201, 202, 203, 204, 205, 206, 207, 208])

block_mgr = ToyUnifiedBlockManager(block_size=4)
scheduler = ToyUnifiedScheduler(
    block_mgr=block_mgr,
    waiting=[cache_hit, cache_miss],
    running=[stream, low],
    max_tokens=12,
)
plan = scheduler.schedule()
metadata = ToyBatchBuilder(block_mgr).build(plan)
ToyOutputProcessor().process(plan, {"stream": 99})
invariants = check_invariants(scheduler, metadata, block_mgr)

summary = {
    "prefix_hits": scheduler.prefix_hits,
    "final_decode_items": [item.request.request_id for item in plan.decode_items],
    "final_prefill_items": [
        (item.request.request_id, item.start, item.num_tokens) for item in plan.prefill_items
    ],
    "free_before": scheduler.free_before,
    "required_new_blocks": scheduler.required_new_blocks,
    "evicted_blocks": plan.evicted_blocks,
    "preempted": plan.preempted,
    "commit_allocations": plan.commit_allocations,
    "metadata_positions": metadata["positions"],
    "metadata_slot_mapping": metadata["slot_mapping"],
    "metadata_block_tables": metadata["block_tables"],
}

```

```

    "stream_output_tokens": stream.output_tokens,
    "cache_hit_computed": cache_hit.num_computed_tokens,
    "cache_miss_computed": cache_miss.num_computed_tokens,
    "free_after": len(block_mgr.free_blocks),
    "invariants": invariants,
}
gates = {
    "prefix_lookup_once_ready": scheduler.prefix_hits == {"cache_hit": 4, "cache_miss": 0},
    "decode_first_ready": summary["final_decode_items"] == ["stream"],
    "suffix_prefill_ready": summary["final_prefill_items"][0] == ("cache_hit", 4, 2),
    "kv_budget_ready": scheduler.required_new_blocks == 3 and scheduler.free_before == 1,
    "memory_pressure_order_ready": plan.evicted_blocks == [6] and plan.preempted[0][0] == "low",
    "commit_plan_ready": plan.commit_allocations == {
        "stream": [],
        "cache_hit": [7],
        "cache_miss": [6, 2],
    },
    "batch_metadata_ready": metadata["slot_mapping"] == [6, 28, 29, 24, 25, 26, 27, 8, 9, 10, 11],
    "output_processor_ready": stream.output_tokens == [11, 12, 99]
    and cache_hit.num_computed_tokens == 6
    and cache_miss.num_computed_tokens == 8,
    "invariants_ready": all(invariants.values()),
}
gates["unified_scheduler_loop_gate"] = all(gates.values())

print("unified_scheduler_loop_summary=", summary)
print("unified_scheduler_loop_gates=", gates)

```

一次运行的核心输出类似：

```

unified_scheduler_loop_summary= {'prefix_hits': {'cache_hit': 4, 'cache_miss': 0}, 'final_decode_items': ['stream'], 'f
unified_scheduler_loop_gates= {'prefix_lookup_once_ready': True, 'decode_first_ready': True, 'suffix_prefill_ready': Tr

```

这个 demo 证明了几个关键点：

1. prefix cache lookup 是 waiting 请求进入调度前的状态推进，不是每轮重复查。
2. decode-first、suffix prefill、token budget 和 KV block budget 必须在同一个 plan 里收敛。
3. KV 压力处理顺序影响线上延迟：先动 cached-free block，再动 active request。
4. BatchBuilder 依赖 commit 后的 block table，不能自己临时分配 block。
5. OutputProcessor 负责把模型输出写回 request 状态，并让下一轮 scheduler 看到一致状态。
6. 统一 loop 的验收不是只看输出 token，而是同时看队列、blocks、positions、slot mapping、computed tokens、cleanup 和 metrics。

52.30 小练习

1. 定义 SchedulePlan、PrefillItem 和 DecodeItem。
2. 给 scheduler 增加 waiting、running、swapped 队列。
3. 实现 prepare_prefix_cache(req)，确保只执行一次 lookup。
4. 实现 next_prefill_chunk(req, token_budget)。
5. 实现 is_ready_to_decode(req)。
6. 实现 decode 优先的 schedule()。
7. 给 schedule 增加 token budget 检查。
8. 给 schedule 增加 block budget 估算。
9. 实现 commit_plan(plan)，统一分配 KV blocks。
10. 实现 memory pressure 处理顺序：缩小 chunk、evict cache、preempt request。
11. 实现 BatchBuilder 的 prefill positions，验证 start 不为 0。
12. 实现 OutputProcessor 更新 num_computed_tokens。

13. finish request 时释放 active refs, 并保留 cached blocks。
14. 增加每轮 schedule debug 日志。
15. 增加状态不变量检查。
16. 构造短请求 baseline 压测。
17. 构造共享 system prompt 压测。
18. 构造长短混合压测。
19. 构造 KV block pool 打爆压测。
20. 写一段面试回答: scheduler 如何同时管理 token budget 和 KV budget?

52.31 本章总结

到这一章, mini engine 的主干已经成型。

continuous batching 负责在 iteration boundary 上动态组织请求。

paged KV cache 让 KV 显存变成可分配、可释放、可统计的 block pool。

prefix cache 让新请求可以复用已有 full blocks, 只计算 suffix。

preemption 让系统在 KV 压力下不会轻易 OOM。

这些机制必须通过 scheduler 主循环串起来。

真正的难点不是某个单点算法, 而是状态一致性: 队列状态、num_computed_tokens、block table、ref count、positions、slot mapping、cache entry 和 metrics 必须一起正确。

工程上要用明确的 schedule plan、统一的 BlockManager 入口、commit 阶段、状态不变量、结构化日志和分层指标来控制复杂度。

下一章可以继续讨论如何把这个 mini engine 变成一个可压测、可调参、可回归的实验框架, 让每次优化都有数据支撑。

第 53 章 构建 serving engine 的 benchmark、指标和调参实验框架

第 52 章我们把 scheduler、paged KV cache、prefix cache 和 preemption 串成了一个完整 engine loop。

到这一步, mini engine 已经不再只是能跑 demo。

它开始具备真实 serving engine 的核心复杂度。

但工程上还有一个更重要的问题:

每次改 scheduler、KV cache、prefix cache 或 preemption 之后, 怎么知道系统真的变好了?

只看单条请求的输出是否正确远远不够。

serving engine 的优化目标通常不是“某一次 forward 更快”, 而是:

1. 首 token 更快。
2. 流式输出更稳定。
3. 总吞吐更高。
4. 显存使用更可控。
5. 高并发和长上下文下不 OOM。
6. prefix cache 命中时确实减少 prefill 成本。
7. KV 压力下 preemption 不会把尾延迟打爆。

这些都需要 benchmark、指标和可重复实验来回答。

本章就把 mini engine 升级成一个可压测、可观测、可调参、可回归的实验框架。

53.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 serving engine 的 benchmark 指标、workload、trace、参数扫描和回归门禁。

公开资料校准主要参考四类口径：

1. vLLM benchmarking / optimization 文档对 serving benchmark、chunked prefill、decode 优先、max_num_batched_tokens、max_num_seqs、KV cache usage 和 preemption 指标的公开说明。
2. vLLM prefix caching 设计文档对 prefix cache hit tokens、cached blocks、ref count、free queue 和 eviction 观测点的公开说明。
3. SGLang benchmark / profiling 文档对在线 serving、吞吐、延迟、profiling 和调参实验的公开口径。
4. NVIDIA GenAI-Perf / TensorRT-LLM benchmark 资料对 TTFT、inter-token latency、request latency、throughput、concurrency 和 streaming 指标的公开口径。

本章不追求复现任何公开 benchmark 数字，不绑定真实 GPU 型号、模型权重、框架版本、压测工具 CLI、Prometheus / Grafana 部署、CUDA profiling 细节或生产容量结论。我们只验证一个最小闭环：

```
workload spec -> benchmark config -> request trace -> engine step trace -> summary metrics -> regression comparison -> tuning decision
```

第二轮新增 demo 的验收重点是：

是否覆盖 short / long / shared prefix / KV pressure workload;
TTFT、TPOT、E2E、throughput 是否从请求 trace 推导;
queue、active requests、KV peak、cleanup 是否从 step trace 推导;
baseline 和 candidate 是否使用可复现实验指纹;
prefix cache 收益是否同时看 hit tokens 和 TTFT;
preemption 是否作为风险指标而不是收益指标;
调参结论是否同时检查用户延迟、吞吐、KV 压力和失败请求。

53.1 本章目标

读完本章，你应该能讲清：

1. serving engine benchmark 要测哪些指标。
2. TTFT、TPOT、E2E latency、throughput 分别代表什么。
3. 为什么平均值经常误导 serving 系统优化。
4. 如何设计短请求、长请求、共享 prefix、长短混合和 KV 压力场景。
5. 如何给 scheduler、KV cache、prefix cache 和 preemption 增加指标。
6. 如何记录每轮 engine step 的结构化 trace。
7. 如何做参数扫描和对比实验。
8. 如何判断一次优化是有效优化还是局部作弊。
9. 面试里如何讲 serving benchmark 和性能调优方法论。

这一章的重点不是追求某个漂亮数字。

重点是建立一套能长期使用的实验纪律。

53.2 为什么 benchmark 是 serving engine 的核心模块

很多人写推理框架时，会先关注 kernel、attention、KV cache 和 scheduler。

这些当然重要。

但如果没有 benchmark，优化很快会变成猜谜。

比如你改了 scheduler，让 prefill 更积极。

你可能看到整体吞吐上涨了。

但同时也可能发生：

1. decode 被 prefill 挤压。
2. TPOT 抖动变大。
3. streaming 体验变差。
4. 长请求占满 KV blocks。
5. 新请求 TTFT 变高。
6. preemption 次数增加。
7. p99 E2E latency 恶化。

如果只看总 tokens/s，你会以为系统变好了。

但用户可能觉得它变慢了。

再比如你启用了 prefix cache。

你可能看到共享 prompt 场景的 TTFT 下降。

但如果 cache eviction 策略不好，可能又会导致：

1. cached blocks 长期占用显存。
2. active 请求可用 KV blocks 变少。
3. preemption 变多。
4. 无共享 prefix 的 workload 反而变差。

所以 benchmark 不应该是项目最后补的脚本。

它应该是 serving engine 的核心模块之一。

53.3 四个最基础的用户侧指标

先从用户真正感受到的指标开始。

53.3.1 TTFT

TTFT 是 time to first token。

它表示：

请求提交到第一个输出 token 返回之间的时间。

TTFT 主要受这些因素影响：

1. waiting queue 排队时间。
2. tokenization 时间。
3. prefix cache lookup 时间。
4. prefill 计算时间。
5. chunked prefill 是否被拆成多轮。
6. scheduler 是否让新请求及时进入 batch。
7. GPU 是否被已有 decode 或长 prefill 占满。

对于聊天机器人，TTFT 非常关键。

用户通常可以接受模型慢慢输出，但很难接受很久没有任何响应。

53.3.2 TPOT

TPOT 是 time per output token。

它表示：

从第一个 token 之后，后续每个输出 token 的平均间隔。

更严格地说，我们通常会记录每两个相邻输出 token 之间的 inter-token latency。

然后统计平均值、p50、p90、p99。

TPOT 主要受这些因素影响：

1. decode batch 大小。
2. attention kernel 性能。
3. KV cache 读写效率。
4. prefill 是否挤占 decode。
5. scheduler 是否优先保障 running 请求。
6. GPU 上是否有过多小 batch 或碎片化 batch。

如果 TPOT 抖动很大，用户会感到输出一卡一卡的。

53.3.3 E2E latency

E2E latency 是 end-to-end latency。

它表示：

请求提交到请求完成之间的总时间。

它包括 TTFT、所有 decode step、排队、调度、网络、后处理等时间。

E2E latency 对离线任务、agent 长链路和批处理任务很重要。

但它不是唯一指标。

两个请求的 E2E latency 可能相同，但用户体验完全不同：

请求 A：很快出首 token，然后稳定流式输出。

请求 B：等很久才出首 token，然后一次性吐完。

对交互式应用来说，请求 A 通常更好。

53.3.4 Throughput

throughput 可以有多种定义：

1. requests/s。
2. input tokens/s。
3. output tokens/s。
4. total tokens/s。
5. successful requests/s。

serving engine 里最常见的是 output tokens/s 和 total tokens/s。

但吞吐必须和延迟一起看。

一个系统可以通过无限排队把 GPU 打满，让 tokens/s 很高。

但用户 TTFT 和 E2E latency 会非常差。

所以 benchmark 报告里不能只放吞吐。

至少要同时放：

1. TTFT p50/p90/p99。
2. TPOT p50/p90/p99。
3. E2E p50/p90/p99。
4. output tokens/s。
5. request success rate。

53.4 平均值为什么危险

很多初学 benchmark 会写：

`avg_latency = sum(latencies) / len(latencies)`

这个数字不是没用。

但在 serving 系统里，它经常掩盖问题。

举个例子：

99 个请求延迟 1s，1 个请求延迟 60s。

平均延迟是 1.59s。

平均值看起来还行。

但那个 60s 的请求可能已经超时、取消或导致用户流失。

serving engine 必须关注尾延迟。

常见统计包括：

1. p50：一半请求低于这个值。
2. p90：90% 请求低于这个值。
3. p95：95% 请求低于这个值。
4. p99：99% 请求低于这个值。
5. max：最坏请求。

调 scheduler 时尤其要看 p99。

因为很多策略的副作用只会体现在少数 unlucky request 上。

比如：

1. 长请求反复被 preempt。
2. 某些请求一直排在 waiting queue。
3. prefix cache eviction 导致少数请求完全 miss。
4. decode 被大 prefill chunk 间歇性阻塞。

平均值会把这些问題稀释掉。

尾延迟会把它们暴露出来。

53.5 engine 内部指标

用户侧指标告诉我们“系统表现怎么样”。

内部指标告诉我们“为什么会这样”。

一个教学版 mini engine 至少应该记录以下内部指标。

53.5.1 scheduler 指标

scheduler_step_total
scheduler_step_duration_ms
waiting_queue_size
running_queue_size
swapped_queue_size
scheduled_prefill_requests
scheduled_decode_requests
scheduled_prefill_tokens
scheduled_decode_tokens
num_batched_tokens
num_batched_requests

这些指标用来回答：

1. 每轮调度花了多久。
2. waiting queue 是否持续积压。

3. running queue 是否被维持在合理大小。
4. prefill 和 decode 的比例是否合理。
5. batch 是否太小导致 GPU 利用率低。
6. batch 是否太大导致延迟恶化。

53.5.2 KV cache 指标

kv_total_blocks
kv_free_blocks
kv_used_blocks
kv_cached_blocks
kv_active_blocks
kv_allocation_success_total
kv_allocation_failure_total
kv_eviction_total
kv_fragmentation_estimate

这些指标用来回答：

1. KV block pool 是否经常被打满。
2. allocation failure 是否发生在高并发或长上下文下。
3. cached blocks 是否挤压 active blocks。
4. eviction 是否过于频繁。
5. block size 是否选择得不合理。

53.5.3 prefix cache 指标

prefix_cache_lookup_total
prefix_cache_hit_total
prefix_cache_miss_total
prefix_cache_hit_tokens_total
prefix_cache_reused_blocks_total
prefix_cache_evicted_blocks_total
prefix_cache_active_ref_blocks

这些指标用来回答：

1. prefix cache 是否真的命中。
2. 命中后节省了多少 prefill tokens。
3. cache blocks 是否被 active request 引用。
4. eviction 是否误删了活跃 block。
5. cache 命中收益是否抵消了 cache 管理开销。

53.5.4 preemption 指标

preemption_total
preemption_recompute_total
preemption_swap_total
preempted_requests_current
recompute_tokens_total
swap_out_bytes_total
swap_in_bytes_total
repeated_preemption_total

这些指标用来回答：

1. KV 压力是否已经过高。
2. 哪些 workload 触发 preemption。

3. 同一个请求是否被反复 preempt。
4. recompute 带来了多少额外计算。
5. swap 是否被 CPU/GPU 带宽限制。

要记住：

preemption 是鲁棒性机制，不是性能优化指标。

preemption_total 高通常不是好事。

它说明系统正在资源压力下艰难维持。

53.6 请求级 trace

聚合指标适合看趋势。

但定位具体 bug 时，还需要请求级 trace。

一个 request trace 可以记录：

```
@dataclass
class RequestTrace:
    request_id: str
    arrival_time: float
    first_scheduled_time: float | None = None
    first_token_time: float | None = None
    finish_time: float | None = None

    prompt_len: int = 0
    output_len: int = 0
    num_prefill_steps: int = 0
    num_decode_steps: int = 0
    num_preemptions: int = 0
    num_recompute_tokens: int = 0

    prefix_cache_hit_tokens: int = 0
    prefix_cache_reused_blocks: int = 0
```

完成请求后可以计算：

```
def summarize_trace(trace: RequestTrace) -> dict:
    ttft = None
    if trace.first_token_time is not None:
        ttft = trace.first_token_time - trace.arrival_time

    e2e = None
    if trace.finish_time is not None:
        e2e = trace.finish_time - trace.arrival_time

    return {
        "request_id": trace.request_id,
        "prompt_len": trace.prompt_len,
        "output_len": trace.output_len,
        "ttft_ms": None if ttft is None else ttft * 1000,
        "e2e_ms": None if e2e is None else e2e * 1000,
        "num_prefill_steps": trace.num_prefill_steps,
        "num_decode_steps": trace.num_decode_steps,
        "num_preemptions": trace.num_preemptions,
        "prefix_cache_hit_tokens": trace.prefix_cache_hit_tokens,
    }
```

请求级 trace 的价值在于可以解释尾延迟。

比如某个请求 p99 E2E latency 很高。

你可以检查：

1. 它是不是排队太久。
2. 它是不是 prompt 太长。
3. 它是不是被 chunked prefill 拆了太多轮。
4. 它是不是 prefix cache miss。
5. 它是不是被 preempt 过。
6. 它是不是在 decode 阶段被大 prefill 挤压。

如果没有 trace，只看最终 latency 很难定位根因。

53.7 engine step 级 trace

除了请求级 trace，还应该记录每轮 engine step 的调度结果。

例如：

```
@dataclass
class EngineStepTrace:
    step_id: int
    timestamp: float
    waiting_queue_size: int
    running_queue_size: int
    free_blocks_before: int
    free_blocks_after: int
    scheduled_prefill_tokens: int
    scheduled_decode_tokens: int
    scheduled_requests: int
    prefix_cache_hits: int
    evicted_blocks: int
    preempted_requests: int
    step_duration_ms: float
```

每轮输出一行 JSONL：

```
{"step": 128, "waiting": 14, "running": 32, "free_blocks_before": 9, "prefill_tokens": 512, "decode_tokens": 31, "preempted_requests": 0, "prefix_cache_hits": 1, "evicted_blocks": 0, "step_duration_ms": 1.2}
```

JSONL 的好处是简单。

你可以直接用脚本做后处理，也可以导入可视化工具。

engine step trace 适合回答：

1. 哪一轮开始 waiting queue 积压。
2. 哪一轮 free blocks 快用完。
3. 哪一轮发生 eviction。
4. 哪一轮发生 preemption。
5. 哪一轮 prefill tokens 太多导致 decode 抖动。

这类 trace 对调 scheduler 特别有用。

53.8 最小指标收集器

教学项目里不一定一开始就接 Prometheus。

可以先写一个内存版 metrics collector。


```
from collections import defaultdict
from dataclasses import dataclass, field
```

```
@dataclass
class Metrics:
    counters: dict[str, int] = field(default_factory=lambda: defaultdict(int))
    gauges: dict[str, float] = field(default_factory=dict)
    histograms: dict[str, list[float]] = field(default_factory=lambda: defaultdict(list))

    def inc(self, name: str, value: int = 1) -> None:
        self.counters[name] += value

    def set_gauge(self, name: str, value: float) -> None:
        self.gauges[name] = value

    def observe(self, name: str, value: float) -> None:
        self.histograms[name].append(value)
```

再写一个 percentile 函数:

```
def percentile(values: list[float], p: float) -> float | None:
    if not values:
        return None
    values = sorted(values)
    index = int((len(values) - 1) * p)
    return values[index]
```

输出 summary:

```
def summarize_histogram(values: list[float]) -> dict:
    if not values:
        return {}
    return {
        "count": len(values),
        "avg": sum(values) / len(values),
        "p50": percentile(values, 0.50),
        "p90": percentile(values, 0.90),
        "p99": percentile(values, 0.99),
        "max": max(values),
    }
```

这个实现很粗糙，但足够教学使用。

真实线上系统会用更专业的 histogram 实现，避免保存所有样本。

53.9 benchmark workload 设计

benchmark 最大的坑是 workload 不真实。

只测一种 prompt 长度、一个并发数、一个输出长度，很容易得到误导结论。

至少要设计五类 workload。

53.9.1 短请求 baseline

短请求用于验证基础吞吐和调度开销。

典型配置:

prompt_len: 32-128
output_len: 32-128
concurrency: 1, 4, 8, 16, 32
shared_prefix: false

它主要观察：

1. 小 batch 下 engine overhead。
2. batch 变大后的吞吐增长。
3. TTFT 是否随并发线性恶化。
4. decode TPOT 是否稳定。

如果短请求 baseline 都不稳定，先不要急着测复杂场景。

53.9.2 长 prompt 场景

长 prompt 用于测试 prefill、chunked prefill 和 KV block 分配。

典型配置：

prompt_len: 2048-8192
output_len: 64-256
concurrency: 1, 4, 8, 16
shared_prefix: false

它主要观察：

1. prefill 是否导致 TTFT 很高。
2. chunked prefill 是否改善 decode 抖动。
3. KV blocks 是否快速耗尽。
4. long prompt 是否挤压短请求。

53.9.3 共享 prefix 场景

共享 prefix 用于测试 prefix cache。

典型配置：

system_prompt_len: 1024-4096
unique_suffix_len: 32-256
output_len: 64-256
shared_prefix_ratio: 0.5-1.0

它主要观察：

1. prefix_cache_hit_total 是否上升。
2. prefix_cache_hit_tokens_total 是否符合预期。
3. TTFT 是否下降。
4. cached blocks 是否挤压 active requests。
5. eviction 是否过早发生。

一个好的 prefix cache benchmark 必须同时包含命中和不命中的请求。

否则你只能证明 “理想情况下它有用”。

53.9.4 长短混合场景

长短混合用于测试 scheduler 公平性。

典型配置：

```
short_prompt_len: 64
short_output_len: 64
long_prompt_len: 4096
long_output_len: 256
long_request_ratio: 0.1-0.3
concurrency: 16-64
```

它主要观察：

1. 短请求 TTFT 是否被长请求拖垮。
2. 长请求是否长期饥饿。
3. chunked prefill 是否有效。
4. decode 优先策略是否保护 streaming。
5. p99 是否明显恶化。

这个场景非常接近真实线上服务。

因为实际请求长度往往是重尾分布。

53.9.5 KV 压力场景

KV 压力场景用于测试 eviction 和 preemption。

典型配置：

```
prompt_len: 2048-8192
output_len: 512-2048
concurrency: high
kv_blocks: deliberately limited
prefix_cache: optional
```

它主要观察：

1. free_blocks 是否接近 0。
2. allocation failure 是否被正确处理。
3. eviction 是否先于 preemption。
4. preemption_total 是否上升。
5. repeated_preemption_total 是否可控。
6. 系统是否避免 OOM。

这个场景不是为了追求好看结果。

它是为了故意打爆系统，验证鲁棒性。

53.10 workload 生成器

可以定义一个简单请求规格：

```
@dataclass
class RequestSpec:
    request_id: str
    arrival_time: float
    prompt_len: int
    output_len: int
    shared_prefix_id: str | None = None
```

再写 workload 生成器：

```
import random
```

```
def generate_short_requests(num_requests: int, qps: float) -> List[RequestSpec]:
```

```

requests = []
now = 0.0
for i in range(num_requests):
    now += random.expovariate(qps)
    requests.append(RequestSpec(
        request_id=f"req-{i}",
        arrival_time=now,
        prompt_len=random.randint(32, 128),
        output_len=random.randint(32, 128),
    ))
return requests

```

这里用 `expovariate(qps)` 模拟泊松到达。

真实线上请求通常不是完全泊松分布，但它比固定间隔更接近实际突发情况。

共享 prefix 可以这样生成：

```

def generate_shared_prefix_requests(
    num_requests: int,
    qps: float,
    shared_prefix_ratio: float,
) -> List[RequestSpec]:
    requests = []
    now = 0.0
    for i in range(num_requests):
        now += random.expovariate(qps)
        use_shared = random.random() < shared_prefix_ratio
        requests.append(RequestSpec(
            request_id=f"req-{i}",
            arrival_time=now,
            prompt_len=4096 if use_shared else random.randint(256, 1024),
            output_len=random.randint(64, 256),
            shared_prefix_id="system-prompt-a" if use_shared else None,
        ))
    return requests

```

教学项目里 prompt 可以先用虚拟 token 表示。

也就是不一定真的构造自然语言文本，而是直接构造 token ids。

这样 benchmark 更稳定，也更容易控制长度。

53.11 open-loop 和 closed-loop 压测

serving benchmark 有两种常见模式。

53.11.1 closed-loop

closed-loop 是：

固定并发数。一个请求完成后，再发下一个请求。

它像这样：

```

while benchmark_not_done:
    while inflight < concurrency:
        send_request()
    wait_one_request_finish()

```

closed-loop 简单，适合测系统在固定并发下的稳定吞吐。

但它有一个问题：

系统变慢时，发送请求的速度也会自动变慢。

因此它不容易暴露排队崩溃。

53.11.2 open-loop

open-loop 是：

按预先设定的到达时间发送请求，不管前面的请求是否完成。

它更接近真实线上流量。

如果系统处理不过来，waiting queue 会堆积，TTFT 和 E2E 会上升。

open-loop 适合回答：

1. 某个 QPS 下系统是否稳定。
2. 排队是否持续增长。
3. 系统饱和点在哪里。
4. admission control 是否生效。

缺点是实现稍复杂，并且更容易把系统打崩。

教学项目建议两种都做：

1. closed-loop 用于快速本地回归。
2. open-loop 用于容量评估和压力测试。

53.12 参数扫描

serving engine 有很多关键参数：

```
max_num_seqs
max_num_batched_tokens
max_model_len
block_size
gpu_memory_utilization
enable_chunked_prefill
max_prefill_chunk_tokens
enable_prefix_cache
prefix_cache_max_blocks
preemption_mode
```

不要手工改一个参数跑一次。

应该写参数扫描脚本。

例如：

```
def run_sweep():
    for max_num_seqs in [8, 16, 32, 64]:
        for max_tokens in [512, 1024, 2048, 4096]:
            config = EngineConfig(
                max_num_seqs=max_num_seqs,
                max_num_batched_tokens=max_tokens,
            )
            result = run_benchmark(config, workload="mixed")
            save_result(config, result)
```

每次实验至少记录：

1. git commit。
2. 模型名称。
3. GPU 型号。
4. batch 参数。
5. workload 配置。
6. 随机种子。
7. benchmark 开始和结束时间。
8. 指标 summary。
9. 原始 trace 文件路径。

没有这些元信息，实验结果很难复现。

53.13 如何比较两次实验

比较实验时，不要只问：

吞吐有没有涨？

应该按层次比较。

第一层：正确性。

请求是否全部成功？输出长度是否符合预期？是否 OOM？是否有异常？

第二层：用户体验。

TTFT、TPOT、E2E 的 p50/p90/p99 是否改善？

第三层：资源效率。

tokens/s 是否提升？GPU 利用率是否更高？KV blocks 是否使用更充分？

第四层：副作用。

preemption 是否增加？eviction 是否增加？allocation failure 是否增加？尾延迟是否恶化？

一次优化可以接受某些 trade-off。

但必须把 trade-off 说清楚。

例如：

max_num_batched_tokens 从 1024 增加到 4096 后，output tokens/s 提升 18%，但 TTFT p99 从 1.2s 上升到 2.8s。这个配置适合离线，这才是工程上有用的结论。

53.14 常见调参方向

53.14.1 TTFT 太高

可能原因：

1. waiting queue 积压。
2. prefill chunk 太大。
3. max_num_seqs 太小，新请求进不来。
4. 长 prompt 阻塞短 prompt。
5. prefix cache 没命中。
6. token budget 被 decode 或其他 prefill 用完。

常见调整：

1. 启用 chunked prefill。
2. 降低 max_prefill_chunk_tokens。
3. 增加 max_num_seqs。
4. 对短请求做优先级保护。

5. 优化 prefix cache lookup 和命中策略。

53.14.2 TPOT 抖动大

可能原因：

1. decode 没有被优先调度。
2. 大 prefill chunk 抢占 GPU。
3. batch size 波动太大。
4. KV cache 访问不稳定。
5. preemption 或 swap 干扰 decode。

常见调整：

1. decode 优先。
2. 限制每轮 prefill token budget。
3. 控制 running queue 大小。
4. 减少 swap。
5. 优化 attention metadata 构造。

53.14.3 吞吐低

可能原因：

1. batch 太小。
2. max_num_batched_tokens 太低。
3. max_num_seqs 太低。
4. CPU 侧调度或 tokenization 成瓶颈。
5. GPU 等待数据。
6. prefix cache 没有减少实际计算。

常见调整：

1. 增加 max_num_batched_tokens。
2. 增加 max_num_seqs。
3. 合并小请求。
4. 减少 Python 调度开销。
5. 异步化 tokenization 和输出处理。

53.14.4 preemption 过多

可能原因：

1. KV blocks 不够。
2. max_num_seqs 太高。
3. max_num_batched_tokens 太激进。
4. 长上下文太多。
5. prefix cache 占用过多 blocks。
6. admission control 缺失。

常见调整：

1. 降低并发。
2. 限制 max_model_len。
3. 降低 max_num_seqs。
4. 改善 prefix cache eviction。
5. 启用更保守的 admission control。
6. 增加 KV cache budget。

53.15 benchmark 报告模板

一次完整 benchmark 报告可以长这样：

Experiment: mixed-long-short-v1
Model: tiny-llm-1b
GPU: A100-80G
Commit: abc1234
Seed: 42

Config:

max_num_seqs: 32
max_num_batched_tokens: 2048
block_size: 16
chunked_prefill: true
max_prefill_chunk_tokens: 512
prefix_cache: true
preemption_mode: recompute

Workload:

requests: 1000
arrival: open-loop qps=8
short_ratio: 0.8
long_ratio: 0.2
shared_prefix_ratio: 0.5

Results:

success_rate: 100%
output_tokens_per_sec: 18420
total_tokens_per_sec: 32780

ttft_ms: avg=420 p50=310 p90=870 p99=2100
tpot_ms: avg=42 p50=38 p90=61 p99=120
e2e_ms: avg=4300 p50=2800 p90=9200 p99=18000

Internal:

prefix_cache_hit_rate: 62%
prefix_cache_hit_tokens_total: 1.8M
kv_eviction_total: 120
preemption_total: 3
allocation_failure_total: 0

Conclusion:

Prefix cache significantly improves shared-prefix TTFT.
p99 E2E still high for long requests under qps=8.
Next experiment should reduce max_prefill_chunk_tokens to protect decode latency.

报告里一定要有 conclusion。

没有结论的 benchmark 只是数字堆砌。

53.16 最容易犯的 benchmark 错误

错误一：只测单请求。

结果：看不到 continuous batching、排队、KV 压力和 scheduler trade-off。

错误二：只看平均延迟。

结果: p99 问题被掩盖。

错误三: 只看 tokens/s。

结果: 可能牺牲 TTFT 和 TPOT 换吞吐。

错误四: 没有固定随机种子。

结果: 两次实验 workload 不一样, 结论不可信。

错误五: 没有记录配置和 commit。

结果: 数字无法复现。

错误六: 只测理想共享 prefix。

结果: prefix cache 看起来收益巨大, 但真实混合流量下收益不稳定。

错误七: 没有 warmup。

结果: 首次 kernel 初始化、内存分配、cache cold start 污染数据。

错误八: 把 preemption 当成成功指标。

结果: 系统虽然没 OOM, 但尾延迟可能已经不可接受。

错误九: 没有区分输入 tokens/s 和输出 tokens/s。

结果: 长 prompt workload 下吞吐数字容易被误读。

错误十: 没有失败请求统计。

结果: 只统计成功请求会让系统看起来比真实情况更好。

53.17 面试高频问题

问题一: LLM serving benchmark 你会看哪些指标?

回答要点: 用户侧看 TTFT、TPOT、E2E latency、request success rate、output tokens/s 和 total tokens/s, 并且要看 p50/p90/p99。系统侧看 waiting/running queue、scheduled tokens、KV free/used/cached blocks、prefix cache hit tokens、evictions、preemptions 和 allocation failures。

问题二: 为什么不能只看吞吐?

回答要点: 吞吐可以通过扩大 batch 或增加排队来提高, 但这可能显著恶化 TTFT、TPOT 和 p99 E2E latency。交互式 serving 需要同时优化吞吐和延迟, 尤其要保护首 token 和流式输出稳定性。

问题三: open-loop 和 closed-loop benchmark 有什么区别?

回答要点: closed-loop 固定并发, 请求完成后再发新请求, 适合测固定并发下的稳定吞吐; open-loop 按固定到达率发请求, 不管前面是否完成, 更接近真实线上流量, 能暴露排队积压和系统饱和点。

问题四: 如何验证 prefix cache 真的有效?

回答要点: 要构造共享 prefix 和非共享 prefix 的混合 workload, 观察 hit rate、hit tokens、reused blocks、TTFT 改善和 prefill tokens 减少。同时要看 cached blocks 是否挤压 active KV, eviction 和 preemption 是否增加。

问题五: preemption 变多说明系统更鲁棒了吗?

回答要点: 不一定。preemption 可以避免 OOM, 是鲁棒性兜底机制, 但频繁 preemption 通常说明 KV budget、并发参数、上下文长度或 prefix cache 占用有问题。评估时要看 repeated preemption、recompute tokens、尾延迟和成功率。

53.18 标准回答模板

如果面试官问 “你会怎么做 LLM serving benchmark 和调优”，可以这样回答：

我会先把指标分成用户侧和系统侧。用户侧重点看 TTFT、TPOT、E2E latency、success rate、output tokens/s 和 total tokens/s, workload 不能只测单请求。我会至少设计短请求 baseline、长 prompt、共享 prefix、长短混合和 KV 压力场景。短请求看基础开销和压测模式上我会同时做 closed-loop 和 open-loop。closed-loop 用固定并发做快速回归，open-loop 用固定到达率观察排队、饱和点。调优时不会只追求 tokens/s。比如增大 max_num_batched_tokens 可能提高吞吐，但也可能恶化 TTFT 和 TPOT；启用 prefix cache 可

53.19 Benchmark Framework 公式、回归门禁和可运行 demo

一次 benchmark 样本可以抽象成：

$r_i = (a_i, f_i, z_i, e_i, p_i, o_i, q_i, k_i, h_i, s_i)$

其中 a_i 是到达时间, f_i 是首 token 时间, z_i 是输出 token 时间序列, e_i 是完成时间, p_i 是 prompt tokens, o_i 是 output tokens, q_i 是排队时间, k_i 是 KV 峰值, h_i 是 prefix hit tokens, s_i 是成功标记。

首 token 延迟：

$L_i^{\mathrm{first}} = f_i - a_i$

平均 inter-token latency：

$L_i^{\mathrm{tok}} = \frac{e_i - f_i}{\max(1, o_i - 1)}$

输出吞吐：

$\Theta_{\mathrm{out}} = \frac{\sum_i o_i}{\max_i e_i - \min_i a_i}$

prefix token 命中率：

$R_{\mathrm{hit}} = \frac{\sum_i h_i}{\max(1, \sum_i p_i)}$

candidate 相对 baseline 的吞吐变化：

$\Delta_{\mathrm{out}} = \frac{\Theta_{\mathrm{out}}^{\mathrm{cand}} - \Theta_{\mathrm{out}}^{\mathrm{base}}}{\max(1, \Theta_{\mathrm{out}}^{\mathrm{base}})}$

最终 benchmark framework 门禁：

$G_{\mathrm{benchfw}} = G_{\mathrm{workload}} \cdot G_{\mathrm{trace}} \cdot G_{\mathrm{slo}} \cdot G_{\mathrm{throughput}} \cdot G_{\mathrm{kv}} \cdot G_{\mathrm{hit}}$

下面这个 0 依赖 demo 不调用真实模型，只审计两组 toy benchmark trace：

1. baseline 没有 prefix cache，长 prompt 和 KV pressure 场景触发多次 preemption。
2. candidate 开启 prefix cache 和更合理的 chunked prefill 后，TTFT / TPOT / E2E p95 改善，output tokens/s 提升。
3. 框架同时检查 workload 覆盖、实验指纹、SLO、吞吐、KV peak、cleanup、prefix 收益、preemption 风险和最终调参结论。

```
from dataclasses import dataclass
```

```
def percentile(values, p):
    values = sorted(values)
    if not values:
        return None
    index = int((len(values) - 1) * p)
    return values[index]
```

```
def hist(values):
```

```

return {
    "avg": round(sum(values) / len(values), 2),
    "p50": round(percentile(values, 0.50), 2),
    "p90": round(percentile(values, 0.90), 2),
    "p95": round(percentile(values, 0.95), 2),
    "max": round(max(values), 2),
}

```

```
@dataclass(frozen=True)
```

```
class BenchmarkConfig:
```

```

    name: str
    max_num_seqs: int
    max_num_batched_tokens: int
    chunked_prefill: bool
    prefix_cache: bool
    preemption_mode: str
    seed: int

```

```
def fingerprint(self):
```

```

    return (
        self.name,
        self.max_num_seqs,
        self.max_num_batched_tokens,
        self.chunked_prefill,
        self.prefix_cache,
        self.preemption_mode,
        self.seed,
    )

```

```
@dataclass
```

```
class RequestTrace:
```

```

    request_id: str
    profile: str
    arrival_ms: float
    first_token_ms: float
    finish_ms: float
    prompt_tokens: int
    output_tokens: int
    queue_wait_ms: float
    kv_peak_blocks: int
    prefix_hit_tokens: int = 0
    preemptions: int = 0
    success: bool = True

```

```
def ttft_ms(self):
```

```
    return self.first_token_ms - self.arrival_ms
```

```
def tpot_ms(self):
```

```
    return (self.finish_ms - self.first_token_ms) / max(1, self.output_tokens - 1)
```

```
def e2e_ms(self):
```

```
    return self.finish_ms - self.arrival_ms
```

```

class ToyServingBenchmarkFramework:
    def __init__(self, config, traces, step_trace):
        self.config = config
        self.traces = traces
        self.step_trace = step_trace

    def classify_bottleneck(self, summary):
        if summary["failure_count"] > 0:
            return "failure"
        if summary["preemption_total"] > 1 or summary["kv_peak_blocks"] > 20:
            return "kv_capacity"
        if summary["ttft_ms"]["p95"] > 700:
            return "queue_prefill"
        if summary["tpot_ms"]["p95"] > 90:
            return "decode_streaming"
        return "balanced"

    def summarize(self):
        ok = [trace for trace in self.traces if trace.success]
        duration_s = (max(t.finish_ms for t in ok) - min(t.arrival_ms for t in ok)) / 1000.0
        prompt_tokens = sum(t.prompt_tokens for t in ok)
        output_tokens = sum(t.output_tokens for t in ok)
        summary = {
            "fingerprint": self.config.fingerprint(),
            "profiles": sorted({t.profile for t in ok}),
            "request_count": len(self.traces),
            "success_rate": round(len(ok) / len(self.traces), 3),
            "ttft_ms": hist([t.ttft_ms() for t in ok]),
            "tpot_ms": hist([t.tpot_ms() for t in ok]),
            "e2e_ms": hist([t.e2e_ms() for t in ok]),
            "output_tokens_per_s": round(output_tokens / duration_s, 2),
            "total_tokens_per_s": round((prompt_tokens + output_tokens) / duration_s, 2),
            "prefix_hit_rate": round(sum(t.prefix_hit_tokens for t in ok) / max(1, prompt_tokens), 3),
            "preemption_total": sum(t.preemptions for t in ok),
            "queue_p95_ms": round(percentile([s["waiting"] for s in self.step_trace], 0.95), 2),
            "active_p95": round(percentile([s["active"] for s in self.step_trace], 0.95), 2),
            "kv_peak_blocks": max(s["kv_blocks"] for s in self.step_trace),
            "kv_after_cleanup": self.step_trace[-1]["kv_blocks"],
            "failure_count": len(self.traces) - len(ok),
        }
        summary["bottleneck"] = self.classify_bottleneck(summary)
        return summary

    def request_rows(self):
        return [
            (
                t.request_id,
                round(t.ttft_ms(), 2),
                round(t.tpot_ms(), 2),
                round(t.e2e_ms(), 2),
                t.prefix_hit_tokens,
                t.preemptions,
            )
            for t in self.traces
        ]

```

```
]
```

```
baseline_config = BenchmarkConfig("baseline", 16, 1024, False, False, "recompute", 42)
candidate_config = BenchmarkConfig("candidate", 32, 2048, True, True, "recompute", 42)
```

```
baseline_traces = [
    RequestTrace("short_a", "short", 0, 120, 360, 64, 8, 40, 6),
    RequestTrace("short_b", "short", 50, 210, 470, 96, 8, 70, 8),
    RequestTrace("long_a", "long", 100, 900, 1500, 2048, 8, 180, 18, preemptions=1),
    RequestTrace("shared_a", "shared_prefix", 150, 760, 1100, 1024, 8, 100, 12),
    RequestTrace("shared_b", "shared_prefix", 200, 820, 1160, 1088, 8, 110, 13),
    RequestTrace("kv_hot", "kv_pressure", 250, 1300, 2100, 1536, 8, 260, 22, preemptions=2),
]
candidate_traces = [
    RequestTrace("short_a", "short", 0, 90, 310, 64, 8, 20, 5),
    RequestTrace("short_b", "short", 50, 160, 380, 96, 8, 30, 7),
    RequestTrace("long_a", "long", 100, 620, 1180, 2048, 8, 90, 16),
    RequestTrace("shared_a", "shared_prefix", 150, 420, 760, 1024, 8, 60, 10, prefix_hit_tokens=768),
    RequestTrace("shared_b", "shared_prefix", 200, 390, 700, 1088, 8, 40, 10, prefix_hit_tokens=1024),
    RequestTrace("kv_hot", "kv_pressure", 250, 820, 1500, 1536, 8, 140, 18, preemptions=1),
]
baseline_steps = [
    {"waiting": 3, "active": 2, "kv_blocks": 12},
    {"waiting": 4, "active": 4, "kv_blocks": 22},
    {"waiting": 2, "active": 3, "kv_blocks": 20},
    {"waiting": 0, "active": 1, "kv_blocks": 0},
]
candidate_steps = [
    {"waiting": 2, "active": 2, "kv_blocks": 10},
    {"waiting": 3, "active": 4, "kv_blocks": 18},
    {"waiting": 1, "active": 4, "kv_blocks": 17},
    {"waiting": 0, "active": 0, "kv_blocks": 0},
]
```

```
baseline = ToyServingBenchmarkFramework(baseline_config, baseline_traces, baseline_steps)
candidate = ToyServingBenchmarkFramework(candidate_config, candidate_traces, candidate_steps)
base_summary = baseline.summarize()
cand_summary = candidate.summarize()
```

```
comparison = {
    "ttft_p95_delta_ms": round(cand_summary["ttft_ms"]["p95"] - base_summary["ttft_ms"]["p95"], 2),
    "tpot_p95_delta_ms": round(cand_summary["tpot_ms"]["p95"] - base_summary["tpot_ms"]["p95"], 2),
    "e2e_p95_delta_ms": round(cand_summary["e2e_ms"]["p95"] - base_summary["e2e_ms"]["p95"], 2),
    "output_tps_delta_ratio": round(
        (cand_summary["output_tokens_per_s"] - base_summary["output_tokens_per_s"])
        / max(1, base_summary["output_tokens_per_s"]),
        3,
    ),
    "preemption_delta": cand_summary["preemption_total"] - base_summary["preemption_total"],
    "bottleneck_change": (base_summary["bottleneck"], cand_summary["bottleneck"]),
}
```

```
required_profiles = {"short", "long", "shared_prefix", "kv_pressure"}
gates = {
```

```

"workload_coverage_ready": set(cand_summary["profiles"]) == required_profiles,
"trace_metrics_ready": cand_summary["ttft_ms"]["p95"] == 520
and cand_summary["tpot_ms"]["p95"] == 80,
"slo_ready": cand_summary["ttft_ms"]["p95"] <= 700
and cand_summary["tpot_ms"]["p95"] <= 90
and cand_summary["success_rate"] == 1.0,
"throughput_regression_ready": comparison["output_tps_delta_ratio"] >= 0.25,
"kv_cleanup_ready": cand_summary["kv_peak_blocks"] <= 20
and cand_summary["kv_after_cleanup"] == 0,
"prefix_effect_ready": cand_summary["prefix_hit_rate"] > base_summary["prefix_hit_rate"],
"preemption_risk_ready": cand_summary["preemption_total"] < base_summary["preemption_total"],
"reproducibility_ready": baseline_config.seed == candidate_config.seed
and base_summary["fingerprint"][0] == "baseline"
and cand_summary["fingerprint"][0] == "candidate",
"decision_ready": comparison["bottleneck_change"] == ("kv_capacity", "balanced"),
}
gates["serving_benchmark_framework_gate"] = all(gates.values())

summary = {
    "baseline": base_summary,
    "candidate": cand_summary,
    "candidate_request_rows": candidate.request_rows(),
    "comparison": comparison,
    "decision": "accept_candidate_for_interactive_serving",
}

print("serving_benchmark_framework_summary=", summary)
print("serving_benchmark_framework_gates=", gates)

```

一次运行的核心输出类似：

```

serving_benchmark_framework_summary= {'baseline': {'fingerprint': ('baseline', 16, 1024, False, False, 'recompute', 42),
280, 'tpot_p95_delta_ms': -5.71, 'e2e_p95_delta_ms': -320, 'output_tps_delta_ratio': 0.4, 'preemption_delta': -
2, 'bottleneck_change': ('kv_capacity', 'balanced')}, 'decision': 'accept_candidate_for_interactive_serving'}
serving_benchmark_framework_gates= {'workload_coverage_ready': True, 'trace_metrics_ready': True, 'slo_ready': True, 't

```

这个 demo 证明了几个关键点：

1. benchmark framework 不是只算 QPS，而是把 workload、config、request trace、step trace 和调参结论连起来。
2. candidate 的吞吐提升只有在 TTFT、TPOT、E2E、KV peak、cleanup、preemption 和失败请求都不过线时才算有效。
3. prefix cache 的收益不能只看 hit rate，要看 shared-prefix 请求 TTFT 是否改善，以及 KV pressure 是否可控。
4. preemption_total 下降是风险收敛信号；preemption_total 上升不能被包装成性能收益。
5. 实验指纹必须包含关键参数和 seed，否则两次 benchmark 没有可比较性。

53.20 小练习

1. 给 mini engine 增加 TTFT、TPOT、E2E latency 统计。
2. 实现一个内存版 Metrics collector。
3. 给 scheduler 增加 waiting/running queue size 指标。
4. 给 BlockManager 增加 free/used/cached blocks 指标。
5. 给 prefix cache 增加 hit tokens 和 reused blocks 指标。
6. 给 preemption 增加 repeated preemption 指标。
7. 实现请求级 RequestTrace。
8. 实现 engine step 级 JSONL trace。

9. 写一个 short request closed-loop benchmark。
10. 写一个 open-loop QPS benchmark。
11. 构造共享 prefix workload, 验证 TTFT 是否下降。
12. 构造长短混合 workload, 观察短请求 p99 TTFT。
13. 构造 KV block pool 打爆 workload, 验证 preemption 是否触发。
14. 写一个参数扫描脚本, 对比不同 max_num_batched_tokens。
15. 写一个 benchmark report markdown 模板。

53.21 本章总结

serving engine 的优化必须靠 benchmark 闭环。

只看单请求、平均延迟或 tokens/s 都不够。

一个可靠的 benchmark 框架至少要包含用户侧指标、系统侧指标、请求级 trace、engine step trace、可重复 workload、参数扫描和实验报告。

TTFT 反映首 token 体验。

TPOT 反映流式输出稳定性。

E2E latency 反映请求总完成时间。

throughput 反映系统处理能力。

它们必须一起看。

workload 也必须覆盖短请求、长 prompt、共享 prefix、长短混合和 KV 压力场景。

否则很容易优化了一个理想场景, 却伤害真实线上流量。

从工程习惯上, 每次改 scheduler、KV cache、prefix cache、preemption 或调参, 都应该跑 benchmark, 记录配置和 trace, 再用数据判断 trade-off。

下一章可以继续讨论: 当 benchmark 发现 CPU 调度、tokenization、输出处理或网络成为瓶颈时, 如何把 mini engine 从同步 loop 升级成异步 serving 架构。

第 54 章 把同步 engine loop 升级成异步 serving 架构

第 53 章我们给 mini engine 建立了 benchmark、指标和 trace。

有了这些工具之后, 你很快会发现一个问题:

GPU forward 不一定是唯一瓶颈。

在教学版 engine 里, 我们经常写一个同步主循环:

```
while engine_has_work():
    new_requests = api_server.pop_new_requests()
    scheduler.add_requests(new_requests)

    schedule = scheduler.schedule()
    batch = batch_builder.build(schedule)
    outputs = model_runner.forward(batch)
    tokens = sampler.sample(outputs)
    output_processor.process(tokens)
```

这个结构适合学习。

它把请求进入、调度、GPU 执行、采样、状态更新、输出返回都放在一条路径里。

但真实 serving 系统不能长期停留在这个结构。

当并发上来后, 同步 loop 会暴露很多问题:

1. HTTP 接收请求会阻塞 engine step。
2. tokenization 会抢 CPU。
3. detokenization 会拖慢 streaming 输出。
4. 输出队列慢会反压 scheduler。
5. metrics 和日志写入会干扰主循环。
6. client 断开连接后 request 状态很难及时清理。
7. 多个 worker 或多张卡之间缺少清晰的异步边界。

本章就把 mini engine 从一个同步 loop 升级成异步 serving 架构。

54.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版单机 serving engine 如何从同步 loop 升级成有异步边界的 runtime。

公开资料校准主要参考四类口径：

1. vLLM serving / engine / optimization 文档对异步请求处理、scheduler、streaming、KV cache、pre-emption 和指标的公开描述。
2. SGLang server / runtime 文档对在线 serving、tokenizer / scheduler / detokenizer、streaming、abort 和 benchmark 的公开口径。
3. FastAPI / ASGI streaming response 的通用工程口径，用来说明 HTTP handler、连接生命周期和后台任务不要直接占住 GPU loop。
4. 本书第 13、14、52、53 章对 token streaming、最小 HTTP API、统一调度循环和 benchmark framework 的教学抽象。

本章不实现真实生产 HTTP server、真实多进程 tokenizer pool、epoll / ASGI runtime、跨 GPU worker、Ray / RPC 控制面、OpenAI-compatible 完整协议、真实 tokenizer 增量解码优化或生产级观测系统。我们只验证一个最小闭环：

API admission -> tokenizer queue -> engine input queue -> engine core loop -> output queue -> per-client stream queue -> cancel / backpressure cleanup

第二轮新增 demo 的验收重点是：API 不直接改 engine 状态；tokenizer 与 engine 解耦；engine 每轮 drain 有上限；OutputProcessor 不直接写网络；慢 client 只取消自己；client disconnect 进入 cancel queue；cancel 和 cleanup 幂等；所有关键队列有上限并可观测。

54.1 本章目标

读完本章，你应该能讲清：

1. 同步 engine loop 为什么适合教学但不适合高并发 serving。
2. serving engine 里哪些工作应该异步化。
3. API server、tokenizer、engine、output worker 之间如何通过队列连接。
4. request 生命周期如何跨异步组件流转。
5. streaming 输出如何避免阻塞 GPU 调度。
6. backpressure、timeout、cancel 如何设计。
7. async 架构下如何保证状态一致性。
8. 哪些指标可以判断异步拆分是否真的有效。
9. 面试里如何描述一个生产级 LLM serving runtime。

本章重点不是某个 Python async 语法。

重点是 serving 系统的异步边界设计。

54.2 同步 loop 的隐藏问题

同步 loop 最大优点是简单。

每轮发生什么都在一段代码里。

但它的问题也来自这里：

任何一个慢步骤都会拖慢整个 engine。

比如一次 engine step 里：

pop request:	0.2 ms
tokenization:	5.0 ms
schedule:	0.5 ms
build batch:	0.7 ms
GPU forward:	12.0 ms
sample:	0.4 ms
detokenization:	3.0 ms
send response:	10.0 ms
metrics/logging:	1.0 ms

如果这些都串行执行，本轮总耗时就是：

32.8 ms

但真正需要卡住 GPU 调度的，也许只有 schedule、build batch、GPU forward、sample 和核心状态更新。

tokenization、detokenization、网络发送、日志写入都可以放到旁路。

同步 loop 会把所有延迟加在一起。

异步架构的目标是：

让 GPU 主循环只等待必须等待的事情。

54.3 哪些工作应该从主循环拆出去

一个 LLM serving 请求大致经历这些阶段：

1. HTTP/gRPC 接入。
2. 鉴权、参数校验、限流。
3. prompt tokenization。
4. admission control。
5. scheduler 排队。
6. prefill/decode GPU 执行。
7. sampling。
8. detokenization。
9. streaming response。
10. metrics、trace、日志。
11. finish、cancel、timeout 清理。

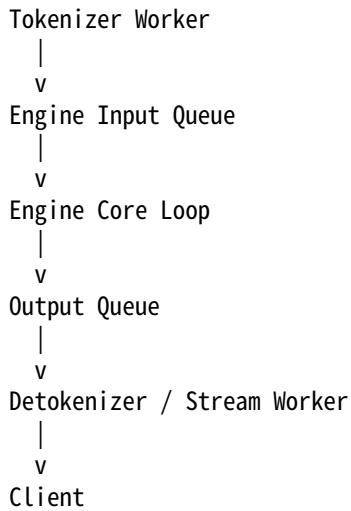
其中真正必须和 GPU engine loop 强绑定的是：

1. scheduler 排队。
2. batch 构造。
3. model forward。
4. sampling。
5. request 核心状态更新。
6. KV cache 分配和释放。

其他部分都应该尽量异步化。

可以拆成下面几个组件：

```
Client
|
v
API Server
|
v
```



这样做的核心收益是：

1. API server 不直接阻塞 GPU loop。
2. tokenization 可以并行。
3. detokenization 和网络发送可以并行。
4. engine loop 可以稳定按 iteration step 推进。
5. slow client 不会直接拖慢所有请求。

54.4 最小异步架构

先定义几个队列：

```
from dataclasses import dataclass
from queue import Queue
```

```
engine_input_queue: Queue["TokenizedRequest"] = Queue(maxsize=1024)
engine_output_queue: Queue["EngineOutput"] = Queue(maxsize=4096)
cancel_queue: Queue[str] = Queue(maxsize=1024)
```

再定义请求对象：

```
@dataclass
class RawRequest:
    request_id: str
    prompt: str
    max_tokens: int
    temperature: float
    arrival_time: float

    @dataclass
    class TokenizedRequest:
        request_id: str
        prompt_token_ids: list[int]
        max_tokens: int
        temperature: float
        arrival_time: float
```

```
@dataclass
```

```
class EngineOutput:
    request_id: str
    token_id: int | None
    finished: bool
    error: str | None = None
```

异步组件之间只传必要信息。

不要把复杂的 engine 内部对象直接暴露给 API server。

这能减少跨线程状态污染。

54.5 API server 的职责

API server 不应该直接做 GPU 调度。

它的职责是：

1. 接收请求。
2. 生成 request_id。
3. 校验参数。
4. 做基础限流。
5. 把 RawRequest 交给 tokenizer。
6. 维护 client connection 和输出通道。
7. 处理 client disconnect。

伪代码：

```
async def handle_generate(http_request):
    raw = RawRequest(
        request_id=new_request_id(),
        prompt=http_request.prompt,
        max_tokens=http_request.max_tokens,
        temperature=http_request.temperature,
        arrival_time=time.time(),
    )

    output_stream = output_registry.register(raw.request_id)

    try:
        await tokenizer_queue.put(raw)
        async for text in output_stream:
            yield text
    finally:
        output_registry.unregister(raw.request_id)
        cancel_queue.put(raw.request_id)
```

这里有一个关键点：

client 断开连接时，要把 cancel 信号传给 engine。

否则 engine 会继续为已经没人接收的请求生成 token。

这会浪费 GPU 和 KV cache。

54.6 Tokenizer worker

tokenization 通常在 CPU 上完成。

它可能比你想象中更容易成为瓶颈。

尤其是：

1. prompt 很长。
2. QPS 很高。
3. chat template 很复杂。
4. tokenizer 是 Python 实现或锁竞争严重。
5. API server 和 tokenizer 在同一个 event loop 里互相影响。

tokenizer worker 可以这样写：

```
def tokenizer_worker(tokenizer, tokenizer_queue, engine_input_queue):
    while True:
        raw: RawRequest = tokenizer_queue.get()
        try:
            token_ids = tokenizer.encode(raw.prompt)
            tokenized = TokenizedRequest(
                request_id=raw.request_id,
                prompt_token_ids=token_ids,
                max_tokens=raw.max_tokens,
                temperature=raw.temperature,
                arrival_time=raw.arrival_time,
            )
            engine_input_queue.put(tokenized)
        except Exception as exc:
            engine_output_queue.put(EngineOutput(
                request_id=raw.request_id,
                token_id=None,
                finished=True,
                error=str(exc),
            ))
```

真实系统里 tokenizer worker 可以是线程池，也可以是独立进程。

选择取决于 tokenizer 实现是否释放 GIL、CPU 核数、内存开销和部署复杂度。

54.7 Engine core loop

engine core loop 仍然保持 iteration-level scheduling。

区别是它不再直接处理 HTTP 请求和字符串输出。

它只从 input queue 拉取已经 tokenized 的请求。

```
def engine_loop(engine):
    while True:
        drain_cancel_queue(engine)
        drain_new_requests(engine)

        schedule = engine.scheduler.schedule()
        if schedule.is_empty():
            wait_for_work_briefly()
            continue

        batch = engine.batch_builder.build(schedule)
        outputs = engine.model_runner.forward(batch)
        sampled = engine.sampler.sample(outputs, batch)
        engine.output_processor.process(schedule, sampled)
```

drain_new_requests 不应该无限拉取。

否则可能导致本轮一直处理新请求，不进入 GPU forward。

应该设置上限:

```
def drain_new_requests(engine, max_drain: int = 128) -> None:
    for _ in range(max_drain):
        try:
            req = engine_input_queue.get_nowait()
        except Empty:
            break
        engine.add_request(req)
```

这个小细节很重要。

异步系统里, 任何 drain loop 都要避免饿死主循环。

54.8 OutputProcessor 不应该直接写网络

教学版里可能会在 output_processor 里直接:

```
send_token_to_client(request_id, token)
```

这在高并发下很危险。

网络发送可能变慢。

client 可能断开。

HTTP stream 可能被 backpressure 卡住。

如果 OutputProcessor 直接写网络, 就会把这些慢操作带进 engine loop。

更好的做法是:

```
class OutputProcessor:
    def process(self, schedule, sampled_tokens):
        for req, token_id in sampled_tokens:
            req.output_token_ids.append(token_id)
            finished = self.check_finished(req)

            engine_output_queue.put(EngineOutput(
                request_id=req.request_id,
                token_id=token_id,
                finished=finished,
            ))

            if finished:
                self.finish_request(req)
```

OutputProcessor 只负责把 token id 放进 output queue。

真正的 detokenization 和网络发送交给 stream worker。

54.9 Detokenizer 和 stream worker

detokenizer worker 从 output queue 读取 token id, 转成 text, 再写给对应 client。

```
def detokenizer_worker(tokenizer, engine_output_queue, output_registry):
    while True:
        out: EngineOutput = engine_output_queue.get()

        if out.error is not None:
            output_registry.send_error(out.request_id, out.error)
            output_registry.close(out.request_id)
```

```

        continue

    if out.token_id is not None:
        text = tokenizer.decode([out.token_id])
        output_registry.send(out.request_id, text)

    if out.finished:
        output_registry.close(out.request_id)

```

这里也有坑。

很多 tokenizer 的 decode 不是简单地逐 token decode 再拼接。

有些 BPE token 需要和上下文一起 decode 才能得到正确字符串边界。

更稳妥的方式是为每个请求维护增量 detokenization 状态：

```

class DetokenizationState:
    def __init__(self):
        self.token_ids: list[int] = []
        self.emitted_text = ""

    def append(self, tokenizer, token_id: int) -> str:
        self.token_ids.append(token_id)
        full_text = tokenizer.decode(self.token_ids)
        delta = full_text[len(self.emitted_text):]
        self.emitted_text = full_text
        return delta

```

这不是最高效实现。

但它能解释核心问题：

streaming detokenization 需要维护请求级状态。

真实系统会用更高效的增量 detokenizer。

54.10 backpressure

异步架构不是把所有东西丢进无限队列。

无限队列只会把延迟和内存问题藏起来。

每个队列都应该有容量上限。

当队列满了，要明确处理策略。

常见队列包括：

1. tokenizer_queue。
2. engine_input_queue。
3. engine_output_queue。
4. 每个 client 的 stream queue。

不同队列满了，策略不同。

54.10.1 tokenizer_queue 满

说明入口请求超过 CPU tokenization 能力。

可以：

1. 返回 429。
2. 排队但设置最大等待时间。

3. 扩容 tokenizer worker。
4. 降低上游 QPS。

54.10.2 engine_input_queue 满

说明 tokenizer 已经生成了很多请求，但 engine 接不动。

可以：

1. 暂停 tokenizer 输出。
2. 对 API server 反压。
3. 触发 admission control。
4. 返回 overload 错误。

54.10.3 engine_output_queue 满

这是非常危险的信号。

说明 engine 生成 token 的速度超过输出侧消费能力。

如果 engine loop 阻塞在 put 上，GPU 调度会被拖慢。

可以选择：

1. 给 output queue 足够容量。
2. 使用非阻塞 put，失败时 cancel 慢请求。
3. 按 request 维护小队列，慢 client 只影响自己。
4. 对 streaming 输出做合并，减少发送次数。

54.10.4 client stream queue 满

说明某个 client 消费太慢。

不要让它拖累全局 output queue。

常见策略是：

1. 超过阈值后取消该请求。
2. 合并多个 token 后再发送。
3. 设置最大 buffered bytes。
4. 关闭慢连接。

backpressure 的原则是：

慢组件必须被隔离，不能拖垮 GPU 主循环。

54.11 timeout 和 cancel

异步系统必须认真处理 cancel。

取消可能来自：

1. client 断开连接。
2. 请求超过最大排队时间。
3. 请求超过最大执行时间。
4. 上游主动取消。
5. output queue 反压。
6. engine 内部错误。

cancel 不是简单地从某个 dict 里删掉 request。

如果请求已经进入 engine，它可能占有：

1. waiting queue 位置。
2. running queue 位置。
3. KV blocks。
4. prefix cache active refs。
5. output registry 状态。
6. trace 和 metrics 状态。

所以 cancel 应该统一由 engine core 处理。

API server 只发送 cancel signal。

```
def drain_cancel_queue(engine, max_drain: int = 1024) -> None:
    for _ in range(max_drain):
        try:
            request_id = cancel_queue.get_nowait()
        except Empty:
            break
        engine.cancel_request(request_id)
```

cancel_request 要保证幂等：

```
def cancel_request(self, request_id: str) -> None:
    req = self.requests.get(request_id)
    if req is None:
        return
    if req.status in (RequestStatus.FINISHED, RequestStatus.CANCELLED):
        return

    self.scheduler.remove(req)
    self.block_manager.release_request_blocks(req)
    self.prefix_cache.release_active_refs(req)
    req.status = RequestStatus.CANCELLED
    self.metrics.inc("request_cancel_total")
    self.requests.pop(request_id, None)
```

幂等很重要。

同一个请求可能同时因为 client disconnect 和 timeout 被取消。

54.12 admission control

异步架构需要 admission control。

否则入口队列会把系统压垮。

admission control 要回答：

这个请求现在能不能进入 engine？

它可以检查：

1. tokenizer_queue 长度。
2. engine_input_queue 长度。
3. waiting queue 长度。
4. running queue 长度。
5. KV free blocks。
6. 估算 prompt blocks。
7. max_model_len。
8. 当前 preemption rate。
9. 当前 p99 TTFT。

最小策略可以很简单：


```
def should_admit(req, metrics, config) -> bool:
    if metrics.gauges["waiting_queue_size"] > config.max_waiting_requests:
        return False
    if len(req.prompt_token_ids) > config.max_model_len:
        return False
    if metrics.gauges["kv_free_blocks"] < config.min_free_blocks_to_admit:
        return False
    return True
```

这比把请求全部塞进 waiting queue 更好。

拒绝请求虽然不好，但比让所有请求一起超时更可控。

54.13 状态所有权

异步架构最容易出 bug 的地方是状态所有权不清晰。

一个重要原则是：

engine 内部状态只允许 engine core 修改。

engine 内部状态包括：

1. RequestState。
2. waiting/running/swapped 队列。
3. KV block table。
4. prefix cache active refs。
5. scheduler metadata。
6. request status。

API server 不应该直接改 request status。

detokenizer 不应该释放 KV blocks。

tokenizer 不应该把请求插入 running queue。

它们只能通过消息告诉 engine：

1. 新请求来了。
2. 某个请求取消了。
3. 输出侧失败了。

engine core 决定如何更新内部状态。

这样可以避免大量锁和竞态。

54.14 线程、进程还是 asyncio

异步 serving 可以用不同实现方式。

54.14.1 asyncio

适合：

1. HTTP/gRPC 网络层。
2. streaming response。
3. 大量连接管理。
4. 非阻塞 IO。

不适合直接跑重 CPU tokenization。

CPU 重任务会阻塞 event loop。

54.14.2 线程

适合：

1. tokenizer 释放 GIL 的场景。
2. detokenization 和轻量 CPU 工作。
3. 简单共享内存队列。

缺点是要注意线程安全。

54.14.3 进程

适合：

1. CPU-heavy tokenization。
2. 隔离不同 worker。
3. 避免 GIL。
4. 多卡多 engine 部署。

缺点是序列化和跨进程通信成本更高。

54.14.4 实用组合

常见组合是：

```
asyncio API server
+ tokenizer thread/process pool
+ dedicated engine thread/process
+ output async tasks
```

不要为了“全 async”把 GPU engine loop 写得过于复杂。

很多生产系统会让 engine core 独占一个线程或进程，保持调度状态简单。

54.15 异步架构下的指标

第 53 章的指标仍然需要保留。

但异步架构还要增加队列和组件指标。

54.15.1 队列指标

```
tokenizer_queue_size
engine_input_queue_size
engine_output_queue_size
client_stream_queue_size
queue_put_blocked_total
queue_put_timeout_total
queue_drop_total
```

这些指标用来判断 backpressure 是否发生。

54.15.2 组件耗时

```
api_request_validation_ms
tokenization_ms
engine_waiting_ms
scheduler_step_ms
gpu_forward_ms
detokenization_ms
```

stream_send_ms

这些指标用来定位瓶颈到底在 CPU、GPU 还是网络。

54.15.3 cancel 和 timeout

request_cancel_total
client_disconnect_total
request_timeout_total
admission_reject_total
slow_client_cancel_total

这些指标用来判断系统是否在过载状态下健康降级。

如果 admission_reject_total 上升，但 p99 延迟稳定，说明限流可能在保护系统。

如果没有 reject，但 timeout 大量增加，说明系统在假装接收请求，实际无法完成。

54.16 异步拆分后的 benchmark 对比

异步化不是天然优化。

它也可能引入额外队列延迟和上下文切换。

所以需要 benchmark 验证。

对比同步版和异步版时，至少看：

1. TTFT p50/p90/p99。
2. TPOT p50/p90/p99。
3. output tokens/s。
4. GPU busy time。
5. tokenization queue wait。
6. output queue wait。
7. client disconnect 后释放 KV 的延迟。
8. engine step duration 抖动。

一个理想结果可能是：

同步版：GPU forward 12ms，但 engine step 平均 31ms。

异步版：GPU forward 12ms，engine step 平均 15ms。

这说明 CPU 和网络工作被移出了主循环。

但如果结果是：

异步版 TTFT p99 更高，engine_input_queue 长期积压。

说明异步队列或 tokenizer worker 成了新瓶颈。

54.17 常见 bug

bug 一：output queue 满导致 engine loop 阻塞。

结果：GPU 空转，所有请求 TPOT 抖动。

bug 二：client 断开后没有 cancel engine request。

结果：僵尸请求继续占用 KV cache 和 decode slots。

bug 三：cancel 直接由 API server 修改 engine 状态。

结果：线程竞态，block table 或 queue 状态不一致。

bug 四：drain input queue 没有上限。

结果：高 QPS 下 engine 一直接新请求，GPU forward 被饿死。

bug 五：tokenizer 在 asyncio event loop 里同步执行。

结果：网络连接处理和 streaming 输出都被 CPU tokenization 阻塞。

bug 六：逐 token decode 不维护 detokenization 状态。

结果：streaming 文本出现乱码、重复或空字符串。

bug 七：队列无限大。

结果：系统不拒绝请求，但延迟无限增长，最终内存爆掉。

bug 八：timeout 后没有释放 prefix cache active refs。

结果：cached blocks ref count 泄漏，后续 eviction 失效。

bug 九：metrics/logging 同步写磁盘。

结果：磁盘抖动影响 engine step latency。

bug 十：没有请求状态机。

结果：同一个请求可能同时 finished、cancelled、streaming，清理逻辑混乱。

54.18 面试高频问题

问题一：为什么 LLM serving engine 不能只写一个同步 loop？

回答要点：同步 loop 适合教学，但高并发下 tokenization、detokenization、网络发送、日志和 metrics 都可能阻塞 GPU 主循环。生产系统需要把 API server、tokenizer、engine core、output worker 拆开，让 GPU loop 只等待必须等待的调度、batch 构造、forward、sampling 和状态更新。

问题二：异步架构里 request 状态应该由谁修改？

回答要点：engine 内部状态应该只由 engine core 修改，包括 request status、waiting/running 队列、KV block table、prefix cache refs。API server、tokenizer、detokenizer 只能通过消息传递新请求、cancel 或输出失败，避免跨线程竞态。

问题三：如何处理 client disconnect？

回答要点：API server 检测到连接断开后发送 cancel signal，engine core 在自己的循环里消费 cancel queue，幂等地从 scheduler 移除请求，释放 KV blocks 和 prefix cache active refs，更新状态和指标。不能只关闭 HTTP 连接而不通知 engine。

问题四：backpressure 怎么做？

回答要点：所有关键队列都要有上限，包括 tokenizer queue、engine input queue、engine output queue 和 client stream queue。队列满时要选择拒绝、限流、cancel 慢请求或 admission control，不能用无限队列隐藏过载。慢 client 也要隔离，不能阻塞全局 output queue。

问题五：asyncio、线程和进程怎么选？

回答要点：asyncio 适合网络 IO 和 streaming 连接管理；线程适合轻量 CPU 工作或释放 GIL 的 tokenizer；进程适合 CPU-heavy tokenization、多卡隔离和避免 GIL。常见架构是 async API server 加 tokenizer worker pool，加独立 engine thread/process，加异步 output tasks。

54.19 标准回答模板

如果面试官问“生产级 LLM serving runtime 为什么要异步化”，可以这样回答：

教学版 engine 可以写成同步 loop：接请求、tokenize、schedule、build batch、GPU forward、sample、detokenize、send response。

我会把系统拆成几个异步组件：API server 负责接入、参数校验、限流和连接管理；tokenizer worker 负责 prompt tokenization；engine core 负责 level scheduling、batch 构造、GPU forward、sampling、KV cache 和 request 状态更新；output queue 把 token id 交给 detokenizer。

这里最重要的是状态所有权和 backpressure。engine 内部状态只能由 engine core 修改，其他组件通过消息传递新请求和 cancel 信号。验证异步化是否有效不能只看吞吐，要看 TTFT、TPOT、E2E、engine step duration、GPU busy time、tokenizer queue wait、output

54.20 异步 serving 架构公式、背压门禁和可运行 demo

把同步 loop 拆成异步组件后，可以先把一次请求的端到端时间拆成：

$$L_i^{\mathrm{e2e}} = L_i^{\mathrm{api}} + L_i^{\mathrm{tok}} + L_i^{\mathrm{queue}} + L_i^{\mathrm{engine}} + L_i^{\mathrm{stream}}$$

其中 L_i^{engine} 才是 GPU 主循环真正必须承担的部分。

每个队列都要有容量约束：

$$Q_j(t) \leq C_j$$

慢 client 的背压门可以写成：

$$G_{\mathrm{slow}, i} = \mathbf{1}[Q_i^{\mathrm{stream}}(t) > C_i^{\mathrm{stream}}]$$

engine 每轮拉取新请求的上限：

$$N_t^{\mathrm{drain}} \leq D_{\max}$$

取消清理延迟：

$$L_i^{\mathrm{cancel}} = t_i^{\mathrm{cleanup}} - t_i^{\mathrm{signal}}$$

状态所有权门禁：

$$G_{\mathrm{owner}} = G_{\mathrm{api}, \mathrm{msg}} G_{\mathrm{tok}, \mathrm{msg}} G_{\mathrm{out}, \mathrm{msg}} G_{\mathrm{engine}, \mathrm{mut}}$$

最终异步 serving 架构门禁：

$$G_{\mathrm{async}} = G_{\mathrm{admit}} G_{\mathrm{tok}} G_{\mathrm{drain}} G_{\mathrm{outq}} G_{\mathrm{stream}} G_{\mathrm{cancel}}$$

下面这个 0 依赖 demo 不启动 HTTP server，也不使用 asyncio。它用有界队列模拟异步 serving 架构里的关键边界：API 层只做 admission，tokenizer 与 engine 解耦，engine 每轮 drain 有上限，OutputProcessor 不直接写网络，慢 client 只取消自己，client disconnect 通过 cancel queue 回到 engine。

```
from collections import deque
from dataclasses import dataclass, field
```

```
def ceil_div(a, b):
    return (a + b - 1) // b
```

```
class BoundedQueue:
    def __init__(self, name, capacity):
        self.name = name
        self.capacity = capacity
        self.items = deque()
        self.put_failures = 0
        self.max_size = 0

    def put(self, item):
        if len(self.items) >= self.capacity:
            self.put_failures += 1
            return False
        self.items.append(item)
        self.max_size = max(self.max_size, len(self.items))
```

```

        return True

    def get_nowait(self):
        if not self.items:
            return None
        return self.items.popleft()

    def __len__(self):
        return len(self.items)

@dataclass
class RawRequest:
    request_id: str
    prompt: str
    max_tokens: int
    client_kind: str

@dataclass
class TokenizedRequest:
    request_id: str
    prompt_tokens: list
    max_tokens: int
    client_kind: str

@dataclass
class EngineOutput:
    request_id: str
    token_id: int | None
    finished: bool
    error: str | None = None

@dataclass
class RequestState:
    request_id: str
    prompt_tokens: list
    max_tokens: int
    client_kind: str
    output_tokens: list = field(default_factory=list)
    status: str = "RUNNING"
    kv_blocks: int = 0

class ClientStream:
    def __init__(self, request_id, capacity, drain_per_put):
        self.request_id = request_id
        self.capacity = capacity
        self.drain_per_put = drain_per_put
        self.buffer = deque()
        self.emitted = []
        self.closed = False
        self.backpressure_events = 0

```

```

def put(self, token_id):
    if self.closed:
        return False
    if len(self.buffer) >= self.capacity:
        self.backpressure_events += 1
        return False
    self.buffer.append(token_id)
    for _ in range(self.drain_per_put):
        if self.buffer:
            self.emitted.append(self.buffer.popleft())
    return True

def close(self):
    while self.buffer:
        self.emitted.append(self.buffer.popleft())
    self.closed = True

class ToyAsyncServingRuntime:
    def __init__(self):
        self.tokenizer_queue = BoundedQueue("tokenizer", 3)
        self.engine_input_queue = BoundedQueue("engine_input", 3)
        self.engine_output_queue = BoundedQueue("engine_output", 4)
        self.cancel_queue = BoundedQueue("cancel", 8)
        self.streams = {}
        self.requests = {}
        self.api_admitted = []
        self.api_rejected = []
        self.tokenized = []
        self.first_drain_added = []
        self.engine_input_after_first_drain = None
        self.finished = []
        self.cancelled = []
        self.cleanup_events = []
        self.metrics = {
            "api_admission_reject_total": 0,
            "tokenizer_requests_total": 0,
            "engine_output_queue_blocked_total": 0,
            "client_stream_backpressure_total": 0,
            "slow_client_cancel_total": 0,
            "client_disconnect_total": 0,
            "request_cancel_total": 0,
            "duplicate_cancel_total": 0,
            "kv_released_blocks": 0,
        }

    def receive_api_request(self, raw):
        self.streams[raw.request_id] = ClientStream(
            raw.request_id,
            capacity=1 if raw.client_kind == "slow" else 4,
            drain_per_put=0 if raw.client_kind == "slow" else 1,
        )
        if not self.tokenizer_queue.put(raw):
            self.api_rejected.append(raw.request_id)

```

```

        self.metrics["api_admission_reject_total"] += 1
        self.streams[raw.request_id].close()
        return False
    self.api_admitted.append(raw.request_id)
    return True

def tokenizer_worker_drain(self):
    while len(self.tokenizer_queue):
        raw = self.tokenizer_queue.get_nowait()
        tokenized = TokenizedRequest(
            request_id=raw.request_id,
            prompt_tokens=list(range(len(raw.prompt.split()))),
            max_tokens=raw.max_tokens,
            client_kind=raw.client_kind,
        )
        self.engine_input_queue.put(tokenized)
        self.tokenized.append(raw.request_id)
        self.metrics["tokenizer_requests_total"] += 1

def drain_new_requests(self, max_drain):
    added = []
    for _ in range(max_drain):
        tokenized = self.engine_input_queue.get_nowait()
        if tokenized is None:
            break
        state = RequestState(
            request_id=tokenized.request_id,
            prompt_tokens=tokenized.prompt_tokens,
            max_tokens=tokenized.max_tokens,
            client_kind=tokenized.client_kind,
            kv_blocks=ceil_div(len(tokenized.prompt_tokens), 4),
        )
        self.requests[state.request_id] = state
        added.append(state.request_id)
    if not self.first_drain_added:
        self.first_drain_added = list(added)
        self.engine_input_after_first_drain = len(self.engine_input_queue)
    return added

def send_cancel(self, request_id, reason):
    if reason == "client_disconnect":
        self.metrics["client_disconnect_total"] += 1
        if request_id in self.streams:
            self.streams[request_id].close()
    self.cancel_queue.put((request_id, reason))

def drain_cancel_queue(self):
    while len(self.cancel_queue):
        request_id, reason = self.cancel_queue.get_nowait()
        self.cancel_request(request_id, reason)

def cancel_request(self, request_id, reason):
    req = self.requests.pop(request_id, None)
    if req is None:
        self.metrics["duplicate_cancel_total"] += 1

```



```

        return
    self.metrics["request_cancel_total"] += 1
    self.metrics["kv_released_blocks"] += req.kv_blocks
    req.status = "CANCELLED"
    self.cancelled.append(request_id)
    self.cleanup_events.append((request_id, reason, "engine"))
    if request_id in self.streams:
        self.streams[request_id].close()

def engine_step(self):
    for req in list(self.requests.values()):
        token_id = 100 + len(req.output_tokens) + 1
        req.output_tokens.append(token_id)
        finished = len(req.output_tokens) >= req.max_tokens
        ok = self.engine_output_queue.put(EngineOutput(req.request_id, token_id, finished))
        if not ok:
            self.metrics["engine_output_queue_blocked_total"] += 1
            self.send_cancel(req.request_id, "output_queue_backpressure")
        if finished and req.request_id in self.requests:
            self.metrics["kv_released_blocks"] += req.kv_blocks
            self.requests.pop(req.request_id)
            req.status = "FINISHED"
            self.finished.append(req.request_id)
            self.cleanup_events.append((req.request_id, "finished", "engine"))

def stream_worker_flush(self):
    while len(self.engine_output_queue):
        out = self.engine_output_queue.get_nowait()
        stream = self.streams[out.request_id]
        if out.token_id is not None:
            ok = stream.put(out.token_id)
            if not ok:
                self.metrics["client_stream_backpressure_total"] += 1
                self.metrics["slow_client_cancel_total"] += 1
                stream.close()
                self.send_cancel(out.request_id, "slow_client")
                continue
        if out.finished:
            stream.close()

runtime = ToyAsyncServingRuntime()
for raw in [
    RawRequest("r1", "a b c d", 3, "fast"),
    RawRequest("r2", "a b c d e f g h", 3, "slow"),
    RawRequest("r3", "a b c d", 3, "disconnect"),
    RawRequest("r4", "a b", 1, "fast"),
]:
    runtime.receive_api_request(raw)

runtime.tokenizer_worker_drain()
runtime.drain_new_requests(max_drain=2)
runtime.engine_step()
runtime.stream_worker_flush()

```

```

runtime.drain_new_requests(max_drain=2)
runtime.send_cancel("r3", "client_disconnect")
runtime.drain_cancel_queue()
runtime.send_cancel("r3", "client_disconnect")
runtime.engine_step()
runtime.stream_worker_flush()
runtime.drain_cancel_queue()

runtime.engine_step()
runtime.stream_worker_flush()
runtime.drain_cancel_queue()

stream_summary = {
    request_id: {"emitted": stream.emitted, "closed": stream.closed}
    for request_id, stream in sorted(runtime.streams.items())
}
queue_summary = {
    "tokenizer": len(runtime.tokenizer_queue),
    "engine_input": len(runtime.engine_input_queue),
    "engine_output": len(runtime.engine_output_queue),
    "cancel": len(runtime.cancel_queue),
}
summary = {
    "api_admitted": runtime.api_admitted,
    "api_rejected": runtime.api_rejected,
    "tokenized": runtime.tokenized,
    "first_drain_added": runtime.first_drain_added,
    "engine_input_after_first_drain": runtime.engine_input_after_first_drain,
    "finished": runtime.finished,
    "cancelled": runtime.cancelled,
    "client_streams": stream_summary,
    "queues_empty": queue_summary,
    "cleanup_events": runtime.cleanup_events,
    "metrics": runtime.metrics,
}
gates = {
    "api_admission_ready": runtime.api_rejected == ["r4"],
    "tokenizer_decoupled_ready": runtime.tokenized == ["r1", "r2", "r3"],
    "bounded_drain_ready": runtime.first_drain_added == ["r1", "r2"]
    and runtime.engine_input_after_first_drain == 1,
    "engine_output_queue_ready": runtime.metrics["engine_output_queue_blocked_total"] == 0,
    "stream_queue_backpressure_ready": runtime.metrics["client_stream_backpressure_total"] == 1
    and runtime.metrics["slow_client_cancel_total"] == 1,
    "cancel_signal_ready": runtime.metrics["client_disconnect_total"] == 2
    and "r3" in runtime.cancelled,
    "engine_state_ownership_ready": all(event[2] == "engine" for event in runtime.cleanup_events),
    "idempotent_cleanup_ready": runtime.metrics["duplicate_cancel_total"] >= 1,
    "slow_client_isolated_ready": runtime.finished == ["r1"] and "r2" in runtime.cancelled,
    "queues_drained_ready": all(value == 0 for value in queue_summary.values()),
}
gates["async_serving_architecture_gate"] = all(gates.values())

print("async_serving_architecture_summary=", summary)
print("async_serving_architecture_gates=", gates)

```

一次运行的核心输出类似：

```
async_serving_architecture_summary= {'api_admitted': ['r1', 'r2', 'r3'], 'api_rejected': ['r4'], 'tokenized': ['r1', 'r2'], 'detokenized': ['r3'], 'api_admission_ready': True, 'tokenizer_decoupled_ready': True, 'bounded_drain_ready': True}
```

这个 demo 证明了几个关键点:

1. API 层只做 admission 和消息投递, 不直接修改 engine 内部状态。
2. tokenizer 和 engine input queue 解耦, engine 每轮 drain 有上限。
3. OutputProcessor 只写 output queue, 不直接写网络。
4. 慢 client 触发 per-client stream backpressure, 只取消自己的请求。
5. client disconnect 和重复 cancel 都回到 engine core, cleanup 是幂等的。

54.21 小练习

1. 给 mini engine 增加 engine_input_queue 和 engine_output_queue。
2. 把 API request 转成 RawRequest, 不要直接进入 scheduler。
3. 实现一个 tokenizer worker, 把 RawRequest 转成 TokenizedRequest。
4. 修改 engine loop, 只从 input queue 拉取 tokenized requests。
5. 给 drain_new_requests 增加每轮最大拉取数量。
6. 修改 OutputProcessor, 让它只写 EngineOutput, 不直接写网络。
7. 实现 detokenizer worker, 把 token id 转成 streaming text。
8. 给每个 request 增加 detokenization state。
9. 实现 client disconnect 到 cancel queue 的路径。
10. 实现幂等的 cancel_request。
11. 给所有队列设置 maxsize。
12. 增加 queue size、queue wait、cancel total 指标。
13. 构造慢 client 压测, 验证不会阻塞 engine loop。
14. 构造 tokenizer CPU 瓶颈压测, 观察 tokenizer queue。
15. 对比同步版和异步版的 TTFT、TPOT、engine step duration。

54.22 本章总结

同步 engine loop 适合教学, 但不适合长期支撑高并发 serving。

真实系统里, GPU forward 不是唯一瓶颈。

tokenization、detokenization、网络 streaming、日志、metrics、慢 client、cancel 和 timeout 都会影响整体性能和稳定性。

异步 serving 架构的核心是把 API server、tokenizer、engine core、output worker 拆开, 用有界队列连接。

engine core 应该独占 request 状态、scheduler 队列、KV block table 和 prefix cache refs。

其他组件通过消息和队列与 engine 通信, 不直接修改 engine 内部状态。

backpressure 是异步架构的关键。

无限队列不是解决方案, 只是把过载变成更大的延迟和内存风险。

client disconnect、timeout 和 cancel 必须最终回到 engine core, 由 engine 幂等清理 request、KV blocks 和 refs。

异步化之后仍然要通过 benchmark 验证效果。

如果 engine step 更稳定、GPU idle 更少、慢 client 不拖累全局 TPOT、队列不长期积压, 说明架构拆分是有效的。

下一章可以继续讨论: 当单个 engine worker 不够时, 如何扩展到多 worker、多 GPU 和 request router。

第 55 章 多 worker、多 GPU 和 request router：从单机 engine 到分布式 serving

第 54 章我们把同步 engine loop 拆成了异步 serving 架构。

现在系统大概长这样：

API Server

- > Tokenizer Worker
- > Engine Input Queue
- > Engine Core
- > Output Queue
- > Stream Worker

这个结构可以把 CPU、网络 and GPU 主循环解耦。

但它仍然是单 engine worker。

当流量继续上涨，单个 engine worker 会遇到新瓶颈：

1. 单张 GPU 显存不够。
2. 单张 GPU 算力不够。
3. 单个 engine 的 KV cache block pool 不够。
4. 单个 tokenizer/output worker 不够。
5. 单个进程故障会影响全部请求。
6. 多模型、多租户、多 LoRA adapter 需要隔离。
7. 不同 GPU 的负载不均会导致尾延迟恶化。

所以生产级 serving 不能只考虑一个 engine。

它需要多个 worker、多张 GPU，以及一个负责路由请求的 request router。

本章讨论从单机 engine 走向分布式 serving 的第一步。

55.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 serving engine 如何从单 worker 扩展到多个 worker、多张 GPU 和 request router。

公开资料校准主要参考四类口径：

1. vLLM parallelism / scaling 文档对单卡、多卡 tensor parallel、pipeline parallel、多节点 serving、Ray / multiprocessing runtime 和网络通信边界的公开说明。
2. vLLM data parallel deployment 文档对 data parallel rank、internal / hybrid / external load balancing、每个 DP rank 的队列、API server 瓶颈和 KV cache aware routing 方向的公开口径。
3. SGLang server / runtime 文档对 TP / DP、router、worker、online serving、abort 和 metrics 的公开工程抽象。
4. 本书第 24、33、41、52、53、54 章对并行方式、vLLM / SGLang 架构对比、跨节点网络、统一调度循环、benchmark framework 和异步 serving 边界的教学抽象。

本章不实现真实分布式 RPC、真实 GPU 通信、Ray actor、Kubernetes service、NCCL collective、跨 worker KV cache 迁移、真实 DP rank 同步、MoE expert routing、生产级一致性协议或完整外部负载均衡器。我们只验证一个最小闭环：

Router snapshot -> capability filter -> global admission -> sticky candidate -> load-aware fallback -> request-to-worker map -> heartbeat failure -> retry / fail-stream decision

第二轮新增 demo 的验收重点是：router 只做 worker 级路由，不直接修改 worker 内部 scheduler / KV 状态；round-robin 只能作为 baseline；路由要同时看能力、健康、KV 空间、队列、延迟、preemption 和 prefix locality；sticky 过载时必须 fallback；worker failure 后没有输出的请求可以重试，已经 streaming 的请求必须显式失败。

55.1 本章目标

读完本章，你应该能讲清：

1. 多 worker serving 和单 worker serving 的核心差异。
2. request router 应该根据哪些信息分配请求。
3. 为什么不能只用 round-robin。
4. worker load metrics 应该包含哪些指标。
5. KV cache、prefix cache 和 sticky routing 的关系。
6. 多 GPU 场景下 tensor parallel 和 replica parallel 的区别。
7. request 失败、worker 崩溃和重试应该怎么处理。
8. router、worker、API server 的状态边界。
9. 分布式 serving 的 benchmark 应该额外关注什么。

本章不是讲大规模集群调度系统。

我们先建立一个最小但正确的多 worker serving 心智模型。

55.2 为什么需要多个 worker

一个 engine worker 通常绑定一个模型实例。

最简单的情况下，一个 worker 占用一张 GPU。

Worker 0 -> GPU 0 -> Model Replica 0

当一张 GPU 不够时，可以增加 replica：

Worker 0 -> GPU 0 -> Model Replica 0

Worker 1 -> GPU 1 -> Model Replica 1

Worker 2 -> GPU 2 -> Model Replica 2

Worker 3 -> GPU 3 -> Model Replica 3

每个 worker 都有自己的：

1. scheduler。
2. waiting/running queues。
3. KV block manager。
4. prefix cache。
5. engine loop。
6. metrics。
7. output stream 状态。

这叫 replica parallel。

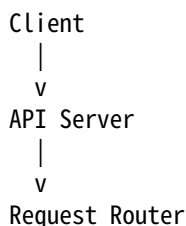
每个 replica 都能独立处理完整请求。

router 的任务是决定：

这个新请求应该发给哪个 worker？

55.3 最小多 worker 架构

一个最小多 worker serving 架构可以长这样：



```

|
+--> Worker 0 -> GPU 0
+--> Worker 1 -> GPU 1
+--> Worker 2 -> GPU 2
+--> Worker 3 -> GPU 3

```

如果沿用第 54 章的异步拆分，每个 worker 内部仍然有自己的 input/output queue:

Request Router

```

|
+--> Worker 0 Input Queue -> Engine 0 -> Worker 0 Output Queue
+--> Worker 1 Input Queue -> Engine 1 -> Worker 1 Output Queue
+--> Worker 2 Input Queue -> Engine 2 -> Worker 2 Output Queue
+--> Worker 3 Input Queue -> Engine 3 -> Worker 3 Output Queue

```

router 不应该直接修改 worker 内部 scheduler。

router 只负责选择 worker，并把 tokenized request 发送给该 worker。

worker 内部怎么排队、怎么分配 KV、怎么 preempt，仍然由 worker 自己决定。

55.4 WorkerHandle

router 需要维护 worker 的基本信息。

可以定义一个 WorkerHandle:

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class WorkerLoad:
    waiting_queue_size: int
    running_queue_size: int
    kv_free_blocks: int
    kv_used_blocks: int
    engine_input_queue_size: int
    recent_ttft_p90_ms: float
    recent_tpot_p90_ms: float
    preemption_rate: float
    healthy: bool
```

```
@dataclass
```

```
class WorkerHandle:
    worker_id: str
    model_name: str
    gpu_id: int
    input_queue: "Queue[TokenizedRequest]"
    load: WorkerLoad
```

router 不需要知道 worker 的所有内部细节。

但它至少要知道:

1. worker 是否健康。
2. worker 当前是否过载。
3. worker 是否有足够 KV 空间接收请求。
4. worker 最近的延迟是否恶化。
5. worker 是否匹配请求的模型或 adapter。

55.5 为什么 round-robin 不够

最简单的路由是 round-robin:

```
class RoundRobinRouter:
    def __init__(self, workers):
        self.workers = workers
        self.next_idx = 0

    def route(self, req):
        worker = self.workers[self.next_idx]
        self.next_idx = (self.next_idx + 1) % len(self.workers)
        return worker
```

它的问题是:

它假设每个请求成本一样, 每个 worker 状态一样。

但 LLM serving 里这个假设通常不成立。

两个请求可能差别很大:

请求 A: prompt 32 tokens, output 64 tokens

请求 B: prompt 16000 tokens, output 2048 tokens

如果 round-robin 把多个长请求连续打到同一个 worker, 这个 worker 的 KV blocks 和 prefill 时间会被打爆。

另一个 worker 可能还很空。

round-robin 还忽略:

1. worker 当前 waiting queue。
2. worker 当前 running queue。
3. KV free blocks。
4. prefix cache 命中可能性。
5. 最近 preemption rate。
6. GPU 异构性能。
7. worker 是否刚经历 OOM 或重启。

所以 round-robin 只能作为 baseline。

生产系统至少需要 load-aware routing。

55.6 load-aware routing

load-aware routing 的目标是把请求发给更可能快速完成的 worker。

最小策略可以是打分:

```
def estimate_request_blocks(req, block_size: int) -> int:
    total_tokens = len(req.prompt_token_ids) + req.max_tokens
    return (total_tokens + block_size - 1) // block_size

def score_worker(worker: WorkerHandle, req, block_size: int) -> float:
    load = worker.load
    if not load.healthy:
        return float("inf")

    needed_blocks = estimate_request_blocks(req, block_size)
    if load.kv_free_blocks < needed_blocks:
        return float("inf")
```

```

score = 0.0
score += load.waiting_queue_size * 10.0
score += load.running_queue_size * 5.0
score += load.engine_input_queue_size * 2.0
score += load.preemption_rate * 100.0
score += load.recent_ttft_p90_ms * 0.01
score += load.recent_tpot_p90_ms * 0.05
return score

```

```

def route(req, workers, block_size: int):
    candidates = [w for w in workers if w.model_name == req.model_name]
    scored = [(score_worker(w, req, block_size), w) for w in candidates]
    score, worker = min(scored, key=lambda x: x[0])
    if score == float("inf"):
        raise OverloadedError("no worker can admit request")
    return worker

```

这个策略很粗糙。

但它体现了几个关键点：

1. 先过滤健康状态和模型匹配。
2. 估算请求会消耗多少 KV blocks。
3. 避免把请求发给明显没有 KV 空间的 worker。
4. 使用 queue size、延迟和 preemption 作为负载信号。
5. 过载时明确拒绝，而不是盲目排队。

55.7 请求成本估算

LLM request 的成本不是一个数字。

它至少包括：

1. prefill 成本：主要由 prompt tokens 决定。
2. decode 成本：主要由 output tokens 和并发数决定。
3. KV cache 成本：由 prompt tokens + generated tokens 决定。
4. streaming 成本：由输出 token 数和客户端消费速度决定。

可以用简单结构记录估算：

```

@dataclass
class RequestCost:
    prompt_tokens: int
    max_output_tokens: int
    estimated_total_tokens: int
    estimated_kv_blocks: int

```

构造方式：

```

def estimate_cost(req, block_size: int) -> RequestCost:
    prompt_tokens = len(req.prompt_token_ids)
    max_output_tokens = req.max_tokens
    total = prompt_tokens + max_output_tokens
    blocks = (total + block_size - 1) // block_size
    return RequestCost(
        prompt_tokens=prompt_tokens,
        max_output_tokens=max_output_tokens,
        estimated_total_tokens=total,

```



```
        estimated_kv_blocks=blocks,  
    )
```

这个估算偏保守，因为请求可能提前遇到 EOS。

但对于 admission control 和 routing，保守估算通常比乐观估算更安全。

55.8 least-loaded 也不总是最优

直觉上，把请求发给当前最空的 worker 很合理。

但在 LLM serving 里，least-loaded 也可能犯错。

原因是 prefix cache。

假设很多请求共享相同 system prompt：

你是一个专业代码助手。请遵循以下规则：...

如果前面请求都打到 Worker 0，Worker 0 的 prefix cache 里已经有这段 prompt 的 KV blocks。

新请求继续发给 Worker 0，可能直接复用 prefix。

如果发给最空的 Worker 2，就要重新 prefill。

所以 router 有一个 trade-off：

负载均衡 vs prefix cache locality

完全按负载路由，可能损失 prefix cache 命中。

完全按 prefix 路由，可能让某个 worker 过热。

55.9 sticky routing

sticky routing 指的是相似请求尽量打到同一个 worker。

常见 sticky key 包括：

1. tenant_id。
2. user_id。
3. conversation_id。
4. system_prompt_hash。
5. prefix_hash。
6. lora_adapter_id。

例如按 prefix hash 选择 worker：

```
def choose_by_prefix_hash(req, workers):  
    idx = stable_hash(req.prefix_hash) % len(workers)  
    return workers[idx]
```

这样做有利于 prefix cache。

但它也可能造成热点。

比如某个热门 tenant 或热门 prompt 占大部分流量。

所以实际 router 通常会把 sticky 和 load-aware 结合起来。

55.10 sticky + load-aware

一种实用策略是：

1. 先根据 sticky key 得到首选 worker。
2. 如果首选 worker 健康且不过载，就发给它。

3. 如果首选 worker 过载，就在候选 worker 里选负载最低的。

伪代码：

```
def route_with_sticky(req, workers, block_size: int):
    candidates = [w for w in workers if w.model_name == req.model_name]
    preferred = candidates[stable_hash(req.prefix_hash) % len(candidates)]

    preferred_score = score_worker(preferred, req, block_size)
    if preferred_score < 100.0:
        return preferred

    return route(req, candidates, block_size)
```

这里的 100.0 只是示意。

真实系统会基于 queue size、KV 空间、TTFT、preemption 和错误率做阈值。

sticky routing 的目标不是绝对粘住。

而是在不伤害尾延迟的前提下提高 cache locality。

55.11 多模型和多 adapter 路由

真实 serving 往往不止一个模型。

可能有：

1. base model A。
2. base model B。
3. chat model。
4. code model。
5. embedding model。
6. 多个 LoRA adapter。

router 必须先做能力匹配：

```
def can_serve(worker, req) -> bool:
    if worker.model_name != req.model_name:
        return False
    if req.adapter_id is not None and req.adapter_id not in worker.loaded_adapters:
        return False
    return True
```

如果 adapter 可以动态加载，还要考虑加载成本。

这时 router 可能优先选择已经加载 adapter 的 worker。

已加载 adapter 的 worker：低延迟，高 locality。

未加载 adapter 的 worker：需要加载，可能阻塞或排队。

这和 prefix cache 是类似问题。

都是 locality 与 load balance 的权衡。

55.12 Replica parallel 和 tensor parallel

多 GPU serving 有两类常见扩展方式。

55.12.1 Replica parallel

每张 GPU 或每组 GPU 放一份完整模型副本。

Worker 0 -> GPU 0 -> Full Model
Worker 1 -> GPU 1 -> Full Model
Worker 2 -> GPU 2 -> Full Model
Worker 3 -> GPU 3 -> Full Model

优点：

1. 架构简单。
2. worker 之间请求独立。
3. 容错容易。
4. router 只需要做请求级负载均衡。

缺点：

1. 单个请求只能用一张 GPU 的显存和算力。
2. 模型太大时放不下一张 GPU。
3. 每张 GPU 都要加载完整权重，显存重复。

55.12.2 Tensor parallel

一个模型被切到多张 GPU 上，一个请求需要多张 GPU 协同完成。

Worker 0 -> GPU 0 + GPU 1 + GPU 2 + GPU 3 -> Sharded Model

优点：

1. 可以服务单卡放不下的大模型。
2. 单请求可以使用多卡算力。
3. 对超大模型是必要手段。

缺点：

1. GPU 之间通信开销高。
2. 一个 worker 故障可能影响整组 GPU。
3. 调度和部署更复杂。
4. router 看到的是一个多 GPU worker，而不是多个独立 GPU。

从 router 视角看，tensor parallel worker 仍然是一个可路由目标：

Request Router -> TP Worker 0(GPU 0,1,2,3)
Request Router -> TP Worker 1(GPU 4,5,6,7)

也就是说：

router 做 worker 级路由，worker 内部处理模型并行。

55.13 Pipeline parallel 和 expert parallel

更复杂的模型并行还有 pipeline parallel 和 expert parallel。

55.13.1 Pipeline parallel

模型层被分到不同 GPU。

GPU 0: layers 0-15
GPU 1: layers 16-31
GPU 2: layers 32-47
GPU 3: layers 48-63

它能降低单卡显存压力。

但在线 decode 场景下，pipeline bubble 和小 batch 会影响效率。

55.13.2 Expert parallel

MoE 模型里，不同 expert 分布在不同 GPU。

每个 token 会路由到部分 expert。

这会引入 token 级别的跨卡通信和负载均衡问题。

这些并行方式通常由模型执行框架内部处理。

本章重点仍然是 request router。

router 不需要知道每一层或每个 expert 怎么切。

它只需要知道 worker 的能力、健康状态和负载。

55.14 worker health check

多 worker 系统必须有健康检查。

worker 可能出现：

1. 进程崩溃。
2. GPU OOM。
3. CUDA error。
4. NCCL hang。
5. engine loop 卡死。
6. 延迟异常升高。
7. KV block 泄漏。
8. output queue 堵塞。

router 需要定期获取 worker heartbeat：

```
@dataclass
class WorkerHeartbeat:
    worker_id: str
    timestamp: float
    healthy: bool
    error: str | None
    load: WorkerLoad
```

如果 heartbeat 超时，router 应该把 worker 标记为 unhealthy：

```
def update_health(worker, now, heartbeat_timeout_s: float):
    if now - worker.last_heartbeat_time > heartbeat_timeout_s:
        worker.load.healthy = False
```

unhealthy worker 不应该接收新请求。

已经在 unhealthy worker 上执行的请求，要根据语义决定是否重试。

55.15 请求重试

请求重试不是总是安全的。

对于 LLM generation，请求可能已经向 client stream 了一部分 token。

如果这时 worker 崩溃，再换 worker 重试，会产生几个问题：

1. 新 worker 生成的文本不一定和前面一致。
2. sampling 有随机性。
3. client 已经收到部分输出，不能无缝替换。
4. prefix cache 和 KV 状态不能跨 worker 自动迁移。

所以重试策略要分阶段。

55.15.1 还没开始输出

如果请求还在 router、input queue 或 prefill 前，没有任何 token 发给 client，可以安全重试到其他 worker。
safe to retry

55.15.2 已经输出 token

如果已经向 client 发送过 token，通常不能透明重试。

可以选择：

1. 返回错误并结束 stream。
2. 告诉上游请求失败。
3. 如果业务允许，重新开始一次新请求。

但不要假装是同一个连续 stream。

55.15.3 deterministic decode

如果 temperature=0，并且模型、版本、随机种子、kernel 行为都完全确定，理论上可以更接近可重试。
但生产上仍然要谨慎。

跨 worker 重放通常没有 KV 状态，只能重新 prefill。

55.16 worker crash 后如何清理

worker 崩溃时，router 和 API server 要处理三类状态：

1. 未发送到 worker 的请求。
2. 已发送但未输出的请求。
3. 已经 streaming 的请求。

可以维护 request location：

```
@dataclass
class RoutedRequestState:
    request_id: str
    worker_id: str | None
    stream_started: bool
    status: str
```

worker crash 后：

```
def handle_worker_failure(worker_id):
    for state in routed_requests_by_worker(worker_id):
        if not state.stream_started:
            retry_on_another_worker(state.request_id)
        else:
            fail_stream(state.request_id, "worker failed")
```

这里还是同一个原则：

没有输出过的请求可以重试，已经输出过的请求应该显式失败。

55.17 分布式状态边界

多 worker 系统更需要明确状态所有权。

可以这样划分：

55.17.1 API server owns connection state

API server 负责:

1. client 连接。
2. HTTP/gRPC stream。
3. request_id。
4. auth 和 quota。
5. client disconnect。

55.17.2 Router owns routing state

router 负责:

1. worker 列表。
2. worker health。
3. worker load snapshot。
4. request 到 worker 的映射。
5. retry 决策。
6. admission reject。

55.17.3 Worker owns engine state

worker 负责:

1. scheduler queue。
2. RequestState。
3. KV block table。
4. prefix cache。
5. model runner。
6. sampling。
7. worker 内部 cancel 和 cleanup。

router 不应该直接释放 worker 的 KV blocks。

worker 不应该直接决定全局流量分配。

API server 不应该越过 router 随机选择 worker。

状态边界清楚, 多 worker 系统才不会变成竞态地狱。

55.18 输出路径怎么走

多 worker 的输出路径有两种常见设计。

55.18.1 worker 直接回 API server

API Server -> Router -> Worker

API Server <- Worker

优点:

1. 输出路径短。
2. router 不需要处理大量 token stream。
3. router 只做控制面。

缺点:

1. API server 要维护 request 到 worker 的连接。
2. worker 和 API server 之间需要 stream 通道。
3. worker failure 处理稍复杂。

55.18.2 worker 输出经过 router

API Server -> Router -> Worker

API Server <- Router <- Worker

优点:

1. router 可以统一记录输出状态。
2. failure 和 retry 状态集中。
3. API server 只和 router 交互。

缺点:

1. router 可能成为 token stream 瓶颈。
2. router 数据面压力很大。
3. 大量输出 token 会增加网络和内存压力。

很多系统会让 router 偏控制面，避免所有 token 都经过 router。

但教学实现可以先让输出经过 router，方便理解 request 状态。

55.19 全局 admission control

第 54 章讨论的是单 worker admission control。

多 worker 系统需要全局 admission control。

它要回答:

整个 serving 集群现在还能不能接这个请求?

可以检查:

1. 健康 worker 数量。
2. 可服务该模型的 worker 数量。
3. 总 waiting queue size。
4. 总 KV free blocks。
5. 最大 worker preemption rate。
6. 全局 p99 TTFT。
7. router queue length。
8. tokenizer pool queue length。

最小策略:

```
def global_admit(req, workers, config) -> bool:
    candidates = [w for w in workers if can_serve(w, req) and w.load.healthy]
    if not candidates:
        return False

    if sum(w.load.waiting_queue_size for w in candidates) > config.max_total_waiting:
        return False

    if max(w.load.preemption_rate for w in candidates) > config.max_preemption_rate:
        return False

    return True
```

全局 admission control 和 routing 应该协同。

不要先全局接收请求，再发现没有 worker 能承载。

55.20 多 worker benchmark

多 worker benchmark 不能只看总 tokens/s。

还要看负载是否均衡。

新增指标包括：

```
router_qps
router_route_latency_ms
router_reject_total
worker_selected_total{worker_id}
worker_inflight_requests{worker_id}
worker_waiting_queue_size{worker_id}
worker_running_queue_size{worker_id}
worker_kv_free_blocks{worker_id}
worker_ttft_p99_ms{worker_id}
worker_tpot_p99_ms{worker_id}
worker_preemption_total{worker_id}
worker_failure_total{worker_id}
request_retry_total
sticky_route_hit_total
sticky_route_fallback_total
```

重点观察：

1. 请求是否集中到少数 worker。
2. 某些 worker 是否长期 queue 高。
3. prefix cache 命中是否因为负载均衡下降。
4. sticky fallback 是否过多。
5. worker failure 后是否正确停止路由。
6. retry 是否造成重复输出。
7. router 本身是否成为瓶颈。

多 worker benchmark 的报告应该按 worker 展示指标。

只给一个全局平均值会掩盖热点。

55.21 常见错误

错误一：只用 round-robin。

结果：长请求和短请求成本差异被忽略，某些 worker 容易被长 prompt 打爆。

错误二：router 不看 KV free blocks。

结果：请求被发到没有 KV 空间的 worker，导致 preemption 或 reject 增加。

错误三：为了 prefix cache 完全 sticky。

结果：热门 tenant 或热门 prompt 把单个 worker 打成热点。

错误四：worker 崩溃后仍然继续路由。

结果：请求不断失败，排队延迟恶化。

错误五：已输出 token 的请求被透明重试。

结果：client 收到不连续或重复的文本。

错误六：router 直接修改 worker 内部状态。

结果：跨进程状态竞态，scheduler 和 KV block table 不一致。

错误七：只看全局平均吞吐。

结果：少数 worker 的 p99 延迟和热点被掩盖。

错误八：router 同时承担所有 token 数据面。

结果：router 成为 streaming 瓶颈，扩容 worker 也无效。

错误九：不区分 replica parallel 和 tensor parallel。

结果：错误地把一个 tensor parallel worker 当成多个独立 worker 路由。

错误十：没有全局 admission control。

结果：所有 worker 都已经过载，router 仍然继续接收请求，最终全局 timeout。

55.22 面试高频问题

问题一：多 GPU LLM serving 里，router 的职责是什么？

回答要点：router 负责根据模型能力、worker 健康、负载、KV 空间、延迟、preemption、prefix cache locality 等信息选择 worker。它不应该直接修改 worker 内部 scheduler 或 KV 状态。router 还负责全局 admission、worker failure 处理和必要的 retry 决策。

问题二：为什么 round-robin 不适合 LLM serving？

回答要点：LLM 请求成本差异很大，短 prompt 和长 prompt 对 prefill、decode 和 KV cache 的压力完全不同。worker 之间的 queue、KV free blocks、preemption rate、prefix cache 状态也不同。round-robin 忽略这些信号，容易造成热点和尾延迟恶化。

问题三：prefix cache 会如何影响路由？

回答要点：如果相似请求打到同一个 worker，就可能复用 prefix cache，降低 prefill 成本和 TTFT。但过度 sticky 会造成热点。所以路由需要在 cache locality 和 load balance 之间权衡，常见做法是 sticky 优先，过载时 fallback 到 load-aware routing。

问题四：replica parallel 和 tensor parallel 有什么区别？

回答要点：replica parallel 是每个 worker 有完整模型副本，请求级别分配到不同 worker，架构简单但单请求受单卡或单副本限制。tensor parallel 是一个模型切到多张 GPU，一个请求需要多卡协同，能服务更大模型但通信和部署复杂。从 router 视角看，一个 TP group 通常是一个 worker。

问题五：worker 崩溃后请求能不能重试？

回答要点：如果请求还没向 client 输出 token，通常可以重试到其他 worker。如果已经 streaming 了一部分 token，透明重试通常不安全，因为 sampling 和 KV 状态无法无缝恢复，可能产生重复或不连续文本。此时应该显式失败 stream，或者由业务层重新发起请求。

55.23 标准回答模板

如果面试官问“你会怎么设计多 GPU LLM serving 的 request router”，可以这样回答：

我会把多 GPU serving 先抽象成多个 worker，每个 worker 可能是一张 GPU 上的完整模型副本，也可能是一个 tensor parallel group。

路由策略不会只用 round-robin，因为 LLM 请求成本差异很大。我会先做能力过滤，比如 model_name、adapter_id、worker health、aware score 选择 worker，避免把请求发给 KV 空间不足、queue 很长或 preemption 很高的 worker。

同时我会考虑 prefix cache locality。对于共享 system prompt、tenant、conversation 或 adapter 的请求，可以用 sticky routing。

故障处理上，worker heartbeat 超时或报错后，router 立即停止给它发新请求。对于已经路由过去的请求，如果还没有输出 token，

55.24 Multi-Worker Router 公式、故障门禁和可运行 demo

多 worker 路由先把请求成本估算成 KV block 数：

$$C_i^{\mathrm{block}} = \left\lceil \frac{P_i + 0_i^{\max}}{B} \right\rceil$$

其中 P_i 是 prompt tokens, $0_i^{\max}$ 是最大输出 token 数, B 是 block size。

worker 的可路由条件可以写成:

$$H_w M_{w,i} A_{w,i} \mathbf{1}[F_w \geq C_i^{\mathrm{block}}] = 1$$

其中 H_w 表示健康, $M_{w,i}$ 表示模型匹配, $A_{w,i}$ 表示 adapter 可用, F_w 表示 free KV blocks。

一个教学版 load score 可以写成:

$$S(w,i) = \alpha_q Q_w + \alpha_r R_w + \alpha_e E_w + \alpha_p P_w^{\mathrm{preempt}} + \alpha_f L_w^{\mathrm{first}} + \alpha_t L_w^{\mathrm{total}}$$

sticky routing 不是绝对粘住, 而是满足阈值才使用首选 worker:

$$w_i = \begin{cases} w_i^{\mathrm{sticky}}, & S(w_i^{\mathrm{sticky}}, i) \leq \tau \\ \arg\min_{w \in \mathcal{C}_i} S(w,i), & S(w_i^{\mathrm{sticky}}, i) > \tau \end{cases}$$

全局 admission control 至少要保证候选 worker 总容量足够:

$$\sum_{w \in \mathcal{C}_i} F_w \geq C_i^{\mathrm{block}}$$

worker failure 后的 retry 门禁可以写成:

$$G_{\mathrm{retry},i} = \mathbf{1}[B_i^{\mathrm{stream}} = 0]$$

也就是没有向 client 输出过 token 的请求才允许透明重试。

最终用一个组合门禁收束:

$$G_{\mathrm{router}} = G_{\mathrm{cap}} G_{\mathrm{admit}} G_{\mathrm{load}} G_{\mathrm{sticky}} G_{\mathrm{health}} G_{\mathrm{retry}}$$

下面这个 0 依赖 demo 把这些约束串起来:

```
from dataclasses import dataclass, field
from math import ceil, inf
```

```
def stable_hash(text: str) -> int:
    total = 0
    for index, char in enumerate(text):
        total += (index + 1) * ord(char)
    return total
```

```
@dataclass
class WorkerLoad:
    waiting_queue_size: int
    running_queue_size: int
    kv_free_blocks: int
    kv_used_blocks: int
    engine_input_queue_size: int
    recent_ttft_p90_ms: float
    recent_tpot_p90_ms: float
    preemption_rate: float
    healthy: bool
    last_heartbeat_s: float
    error: str | None = None
```

```
@dataclass
```

```

class WorkerHandle:
    worker_id: str
    model_name: str
    gpu_group: tuple[int, ...]
    loaded_adapters: set[str]
    hot_prefixes: set[str]
    load: WorkerLoad
    selected_total: int = 0

```

@dataclass

```

class TokenizedRequest:
    request_id: str
    model_name: str
    prompt_tokens: int
    max_output_tokens: int
    prefix_hash: str
    adapter_id: str | None = None

```

@dataclass

```

class RoutedRequestState:
    request_id: str
    worker_id: str | None
    stream_started: bool
    status: str
    events: list[str] = field(default_factory=list)

```

@dataclass

```

class RouterConfig:
    block_size: int
    sticky_score_threshold: float
    heartbeat_timeout_s: float
    max_total_waiting: int
    max_preemption_rate: float

```

```

class ToyMultiWorkerRouter:
    def __init__(self, workers: list[WorkerHandle], config: RouterConfig):
        self.workers = workers
        self.config = config
        self.rr_index = 0
        self.states: dict[str, RoutedRequestState] = {}
        self.metrics = {
            "router_reject_total": 0,
            "sticky_route_hit_total": 0,
            "sticky_route_fallback_total": 0,
            "request_retry_total": 0,
            "stream_fail_total": 0,
            "worker_failure_total": 0,
            "capability_filter_total": 0,
        }

    def estimate_blocks(self, req: TokenizedRequest) -> int:

```

```

    return ceil((req.prompt_tokens + req.max_output_tokens) / self.config.block_size)

def can_serve(self, worker: WorkerHandle, req: TokenizedRequest) -> bool:
    if worker.model_name != req.model_name:
        return False
    if req.adapter_id is not None and req.adapter_id not in worker.loaded_adapters:
        return False
    return True

def candidates(self, req: TokenizedRequest) -> List[WorkerHandle]:
    result = [worker for worker in self.workers if self.can_serve(worker, req)]
    self.metrics["capability_filter_total"] += 1
    return result

def score_worker(self, worker: WorkerHandle, req: TokenizedRequest) -> float:
    load = worker.load
    needed_blocks = self.estimate_blocks(req)
    if not load.healthy or load.kv_free_blocks < needed_blocks:
        return inf

    locality_penalty = 0.0 if req.prefix_hash in worker.hot_prefixes else 10.0
    score = 0.0
    score += load.waiting_queue_size * 10.0
    score += load.running_queue_size * 6.0
    score += load.engine_input_queue_size * 3.0
    score += load.preemption_rate * 100.0
    score += load.recent_ttft_p90_ms * 0.01
    score += load.recent_tpot_p90_ms * 0.05
    score += needed_blocks * 0.5
    score += locality_penalty
    return round(score, 3)

def global_admit(self, req: TokenizedRequest) -> bool:
    candidates = [w for w in self.candidates(req) if w.load.healthy]
    needed_blocks = self.estimate_blocks(req)
    if not candidates:
        return False
    if sum(w.load.waiting_queue_size for w in candidates) > self.config.max_total_waiting:
        return False
    if max(w.load.preemption_rate for w in candidates) > self.config.max_preemption_rate:
        return False
    if sum(w.load.kv_free_blocks for w in candidates) < needed_blocks:
        return False
    return True

def round_robin_route(self, req: TokenizedRequest) -> WorkerHandle | None:
    candidates = self.candidates(req)
    if not candidates:
        return None
    worker = candidates[self.rr_index % len(candidates)]
    self.rr_index += 1
    return worker

def sticky_preferred(self, req: TokenizedRequest, candidates: List[WorkerHandle]) -> WorkerHandle:
    for worker in candidates:

```

```

        if req.prefix_hash in worker.hot_prefixes:
            return worker
    return candidates[stable_hash(req.prefix_hash) % len(candidates)]

def load_aware_route(
    self, req: TokenizedRequest, exclude_worker_id: str | None = None
) -> WorkerHandle | None:
    candidates = [w for w in self.candidates(req) if w.worker_id != exclude_worker_id]
    scored = [(self.score_worker(worker, req), worker) for worker in candidates]
    viable = [(score, worker) for score, worker in scored if score < inf]
    if not viable:
        self.metrics["router_reject_total"] += 1
        return None
    return min(viable, key=lambda item: (item[0], item[1].worker_id))[1]

def route_with_sticky(self, req: TokenizedRequest) -> WorkerHandle | None:
    if not self.global_admit(req):
        self.metrics["router_reject_total"] += 1
        return None

    candidates = self.candidates(req)
    preferred = self.sticky_preferred(req, candidates)
    preferred_score = self.score_worker(preferred, req)
    if preferred_score <= self.config.sticky_score_threshold:
        self.metrics["sticky_route_hit_total"] += 1
        return preferred

    self.metrics["sticky_route_fallback_total"] += 1
    return self.load_aware_route(req)

def commit_route(
    self, req: TokenizedRequest, worker: WorkerHandle, stream_started: bool
) -> RoutedRequestState:
    worker.selected_total += 1
    worker.load.waiting_queue_size += 1
    worker.load.kv_free_blocks -= self.estimate_blocks(req)
    state = RoutedRequestState(
        request_id=req.request_id,
        worker_id=worker.worker_id,
        stream_started=stream_started,
        status="routed",
        events=[f"route:{worker.worker_id}"],
    )
    self.states[req.request_id] = state
    return state

def update_health(self, now_s: float) -> list[str]:
    unhealthy = []
    for worker in self.workers:
        stale = now_s - worker.load.last_heartbeat_s > self.config.heartbeat_timeout_s
        if stale and worker.load.healthy:
            worker.load.healthy = False
            worker.load.error = "heartbeat_timeout"
            unhealthy.append(worker.worker_id)
    return unhealthy

```

```

def handle_worker_failure(self, worker_id: str, retry_requests: dict[str, TokenizedRequest]):
    self.metrics["worker_failure_total"] += 1
    for state in list(self.states.values()):
        if state.worker_id != worker_id or state.status != "routed":
            continue
        if state.stream_started:
            state.status = "failed_stream"
            state.events.append("fail_stream:worker_failed")
            self.metrics["stream_fail_total"] += 1
            continue

        retry_req = retry_requests[state.request_id]
        retry_worker = self.load_aware_route(retry_req, exclude_worker_id=worker_id)
        if retry_worker is None:
            state.status = "failed_before_stream"
            state.events.append("retry_failed:no_worker")
            continue
        state.worker_id = retry_worker.worker_id
        state.status = "retried"
        state.events.append(f"retry:{retry_worker.worker_id}")
        retry_worker.selected_total += 1
        self.metrics["request_retry_total"] += 1

def worker_selection_counts(self) -> dict[str, int]:
    return {worker.worker_id: worker.selected_total for worker in self.workers}

```

```

workers = [
    WorkerHandle(
        worker_id="w0",
        model_name="chat",
        gpu_group=(0,),
        loaded_adapters={"lora-a"},
        hot_prefixes={"sys-a"},
        load=WorkerLoad(5, 4, 24, 40, 2, 900.0, 90.0, 0.20, True, 12.0),
    ),
    WorkerHandle(
        worker_id="w1",
        model_name="chat",
        gpu_group=(1,),
        loaded_adapters={"lora-a"},
        hot_prefixes=set(),
        load=WorkerLoad(1, 1, 32, 20, 1, 320.0, 35.0, 0.02, True, 0.0),
    ),
    WorkerHandle(
        worker_id="w2",
        model_name="chat",
        gpu_group=(2, 3),
        loaded_adapters={"lora-a"},
        hot_prefixes={"sys-c"},
        load=WorkerLoad(0, 0, 64, 0, 0, 0.0, 0.0, 0.00, False, 12.0, "starting"),
    ),
    WorkerHandle(
        worker_id="w3",

```

```

        model_name="code",
        gpu_group=(4,),
        loaded_adapters=set(),
        hot_prefixes=set(),
        load=WorkerLoad(0, 1, 40, 12, 0, 260.0, 28.0, 0.01, True, 12.0),
    ),
]

config = RouterConfig(
    block_size=128,
    sticky_score_threshold=80.0,
    heartbeat_timeout_s=10.0,
    max_total_waiting=12,
    max_preemption_rate=0.50,
)
router = ToyMultiWorkerRouter(workers, config)

req_a = TokenizedRequest("r1", "chat", 800, 256, "sys-a", "lora-a")
req_b = TokenizedRequest("r2", "chat", 64, 64, "sys-a", "lora-a")
req_big = TokenizedRequest("r3", "chat", 9000, 4096, "sys-big", "lora-a")
req_code = TokenizedRequest("r4", "code", 128, 128, "code-a", None)

round_robin_choice = router.round_robin_route(req_a).worker_id
load_aware_choice = router.load_aware_route(req_a).worker_id

worker_for_a = router.route_with_sticky(req_a)
state_a = router.commit_route(req_a, worker_for_a, stream_started=False)

worker_for_b = router.route_with_sticky(req_b)
state_b = router.commit_route(req_b, worker_for_b, stream_started=True)

big_admitted = router.global_admit(req_big)
if not big_admitted:
    router.metrics["router_reject_total"] += 1

code_worker = router.route_with_sticky(req_code)
if code_worker is not None:
    router.commit_route(req_code, code_worker, stream_started=False)

unhealthy_after_timeout = router.update_health(now_s=15.0)
router.handle_worker_failure(
    "w1", {"r1": req_a, "r2": req_b, "r3": req_big, "r4": req_code}
)

state_rows = {
    request_id: {
        "worker_id": state.worker_id,
        "stream_started": state.stream_started,
        "status": state.status,
        "events": state.events,
    }
    for request_id, state in sorted(router.states.items())
}
selection_counts = router.worker_selection_counts()
active_counts = [count for count in selection_counts.values() if count > 0]

```

```

selection_imbalance = max(active_counts) - min(active_counts) if active_counts else 0

summary = {
    "round_robin_choice_for_long": round_robin_choice,
    "load_aware_choice_for_long": load_aware_choice,
    "sticky_preferred_for_sys_a": "w0",
    "actual_worker_for_r1": worker_for_a.worker_id,
    "actual_worker_for_r2": worker_for_b.worker_id,
    "big_request_admitted": big_admitted,
    "code_worker": code_worker.worker_id if code_worker else None,
    "unhealthy_after_timeout": unhealthy_after_timeout,
    "state_rows": state_rows,
    "selection_counts": selection_counts,
    "selection_imbalance": selection_imbalance,
    "metrics": router.metrics,
}

gates = {
    "capability_filter_ready": code_worker.worker_id == "w3",
    "global_admission_ready": big_admitted is False,
    "load_aware_ready": round_robin_choice == "w0" and load_aware_choice == "w1",
    "sticky_fallback_ready": worker_for_a.worker_id == "w1"
    and router.metrics["sticky_route_fallback_total"] >= 1,
    "health_check_ready": unhealthy_after_timeout == ["w1"],
    "retry_gate_ready": state_rows["r1"]["status"] == "retried"
    and state_rows["r2"]["status"] == "failed_stream",
    "state_boundary_ready": all("route:" in row["events"][0] for row in state_rows.values()),
    "metrics_ready": router.metrics["request_retry_total"] == 1
    and router.metrics["stream_fail_total"] == 1,
}
gates["multi_worker_router_gate"] = all(gates.values())

print("multi_worker_router_summary=", summary)
print("multi_worker_router_gates=", gates)

```

一次运行的核心输出类似：

```

multi_worker_router_summary= {'round_robin_choice_for_long': 'w0', 'load_aware_choice_for_long': 'w1', 'sticky_preferred_for_sys_a': 'w0', 'actual_worker_for_r1': 'w0', 'actual_worker_for_r2': 'w1', 'big_request_admitted': False, 'code_worker': None, 'unhealthy_after_timeout': [], 'state_rows': {'r1': {'status': 'retried'}, 'r2': {'status': 'failed_stream'}}, 'selection_counts': {'r1': 1, 'r2': 1}, 'selection_imbalance': 1, 'metrics': {'request_retry_total': 1, 'stream_fail_total': 1, 'sticky_route_fallback_total': 1}}
multi_worker_router_gates= {'capability_filter_ready': True, 'global_admission_ready': True, 'load_aware_ready': True, 'sticky_fallback_ready': True, 'health_check_ready': True, 'retry_gate_ready': True, 'state_boundary_ready': True, 'metrics_ready': True, 'multi_worker_router_gate': True}

```

这个 demo 证明了几个关键点：

1. round-robin 会把长请求打到高负载 sticky worker，load-aware routing 会避开它。
2. router 先做模型和 adapter 能力过滤，code 请求不会被发给 chat worker。
3. sticky 只是首选，首选 worker 过载时必须 fallback。
4. 全局 admission control 会拒绝超出候选 worker 总 KV 容量的大请求。
5. heartbeat 超时后 worker 停止接收新请求。
6. worker failure 后，未输出请求可以重试，已经 streaming 的请求显式失败。
7. router 维护 request-to-worker map，但不直接修改 worker 内部 scheduler、KV block table 或 prefix cache。

55.25 小练习

1. 实现 WorkerHandle 和 WorkerLoad。
2. 给每个 worker 增加 heartbeat。
3. 实现最简单的 round-robin router。
4. 给 router 增加 model_name 过滤。

5. 实现 `estimate_request_blocks`。
6. 实现基于 `queue size` 和 `KV free blocks` 的 `load-aware routing`。
7. 增加 `preemption_rate` 和 `recent_ttft_p90_ms` 到 `worker score`。
8. 实现 `prefix_hash sticky routing`。
9. 实现 `sticky` 过载 `fallback`。
10. 增加 `worker unhealthy` 后停止路由。
11. 维护 `request_id` 到 `worker_id` 的映射。
12. 模拟 `worker crash`，测试未输出请求重试。
13. 模拟 `worker crash`，测试已输出请求失败 `stream`。
14. 给 `router` 增加 `global admission control`。
15. 写一个多 `worker benchmark`，输出每个 `worker` 的 `TTFT`、`TPOT`、`KV free blocks` 和 `selected_total`。

55.26 本章总结

从单 `worker` 到多 `worker`，核心变化不是多起几个进程这么简单。

系统多了一个 `request router`。

`router` 要根据 `worker` 能力、健康状态、负载、`KV` 空间、延迟、`preemption` 和 `cache locality` 做决策。

`round-robin` 可以作为 `baseline`，但不能作为生产级策略。

`LLM` 请求成本差异很大，长 `prompt`、长输出和短请求对 `worker` 的压力完全不同。

`prefix cache` 又让路由变得更复杂。

为了提高 `cache` 命中，相似请求应该尽量 `sticky` 到同一个 `worker`。

但过度 `sticky` 会造成热点。

所以实际策略通常是 `sticky + load-aware`。

多 `GPU serving` 还要区分 `replica parallel` 和 `tensor parallel`。

`replica parallel` 是多个完整模型副本，`router` 做请求级负载均衡。

`tensor parallel` 是一个模型跨多张 `GPU`，一个 `TP group` 从 `router` 视角看通常是一个 `worker`。

故障处理也必须谨慎。

没有输出过的请求可以重试，已经输出过 `token` 的请求通常不能透明重试。

最后，多 `worker benchmark` 必须按 `worker` 维度展示指标。

只看全局平均吞吐会掩盖热点、尾延迟和负载不均。

下一章可以继续讨论：多副本 `serving` 之外，如果模型太大或吞吐要求更高，如何理解 `tensor parallel`、`pipeline parallel` 和分布式 `KV cache` 的执行细节。

第 56 章 Tensor Parallel、Pipeline Parallel 和分布式 KV Cache

第 55 章我们从 `request router` 的角度看了多 `worker`、多 `GPU serving`。

`router` 眼里的 `worker` 可能是一张 `GPU` 上的完整模型副本，也可能是一组 `GPU` 共同服务一个模型。

本章进入 `worker` 内部。

当模型大到单卡放不下，或者单卡吞吐不够时，就需要模型并行。

常见方式包括：

1. `tensor parallel`。
2. `pipeline parallel`。
3. `expert parallel`。
4. `data/replica parallel`。

5. 它们的组合。

对 LLM inference 来说，模型并行不仅影响权重怎么切，还会影响：

1. 每层 forward 的通信。
2. prefill 和 decode 的延迟。
3. KV cache 放在哪里。
4. scheduler 看到的 worker 边界。
5. failure domain。
6. benchmark 指标怎么解释。

本章重点不是推导并行训练算法。

重点是理解 serving 里模型并行和 KV cache 的工程含义。

56.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 serving engine 在单个 worker 内部如何理解 tensor parallel、pipeline parallel 和分布式 KV cache。

公开资料校准主要参考四类口径：

1. vLLM parallelism / scaling 文档对单卡、多卡 tensor parallel、多节点 tensor + pipeline parallel、Ray / multiprocessing runtime、GPU KV cache size 和跨节点通信的公开说明。
2. vLLM data parallel deployment 文档对 DP rank、独立 KV cache、running / waiting queue 负载均衡和 KV cache aware routing 方向的公开说明。
3. NVIDIA NCCL collective 文档对 AllReduce、AllGather、ReduceScatter、Broadcast 和 AlltoAll 的稳定通信语义说明。
4. Megatron-LM 论文和工程口径对 tensor parallel、pipeline parallel 与 data parallel 组合的经典抽象说明，并结合本书第 24、41、52、55 章对并行方式、跨节点网络、统一调度循环和 request router 的教学边界。

本章不实现真实 GPU kernel、NCCL collective、Ray actor、pipeline schedule、MoE expert parallel、context parallel、生产级 TP/PP placement、跨节点 RDMA 调优、分布式 prefix cache 协议或真实 KV connector。我们只验证一个最小闭环：

Parallel group plan -> TP rank shard -> PP stage layer ownership -> distributed KV block table -> collective cost -> pipeline bubble -> cross-rank cleanup -> migration vs recompute decision

第二轮新增 demo 的验收重点是：router 看到的是 parallel group，不是单张 GPU；TP rank 的逻辑 block table 必须一致；TP 下 KV 按 head shard 分布；PP 下 KV 按 layer stage 分布；cancel / finish / preemption 必须跨 rank / stage 一致释放；TP size、PP size 和 KV 迁移决策必须用通信、bubble、显存和 workload 共同判断。

56.1 本章目标

读完本章，你应该能讲清：

1. replica parallel、tensor parallel、pipeline parallel 的区别。
2. tensor parallel 为什么需要 all-reduce 或 all-gather。
3. pipeline parallel 为什么在线 decode 容易有 bubble。
4. 为什么模型并行会改变单请求延迟。
5. KV cache 在 tensor parallel 下如何切分。
6. KV cache 在 pipeline parallel 下如何按层分布。
7. 为什么跨 worker 迁移 KV cache 很难。
8. scheduler 和 parallel group 的边界是什么。
9. 面试中如何解释多 GPU inference 的 trade-off。

56.2 先区分三个概念

很多人会把多 GPU 的几个概念混在一起。

先把它们分开。

56.2.1 Replica parallel

每个 worker 都有完整模型副本。

Worker 0: full model on GPU 0

Worker 1: full model on GPU 1

Worker 2: full model on GPU 2

Worker 3: full model on GPU 3

一个请求只进入其中一个 worker。

优点是简单、隔离好、router 容易做负载均衡。

缺点是单请求不能使用多卡显存。

如果模型单卡放不下，replica parallel 不解决问题。

56.2.2 Tensor parallel

模型的一些张量在多张 GPU 上切分。

TP Worker 0:

GPU 0: shard 0

GPU 1: shard 1

GPU 2: shard 2

GPU 3: shard 3

一个请求会同时使用这个 TP group 里的所有 GPU。

优点是能服务单卡放不下的大模型。

缺点是每层 forward 都可能需要跨卡通信。

56.2.3 Pipeline parallel

模型层被切到不同 GPU。

GPU 0: layers 0-15

GPU 1: layers 16-31

GPU 2: layers 32-47

GPU 3: layers 48-63

一个请求从 GPU 0 依次流到 GPU 3。

优点是按层降低显存压力。

缺点是流水线 bubble 和 stage imbalance 很难处理。

尤其在 decode batch 较小时，pipeline 利用率可能很差。

56.3 Router 看到的并行组

第 55 章说过：router 不应该把一个 tensor parallel group 里的 GPU 当成独立 worker。

如果一个模型需要 4 张 GPU 组成 TP group:

TP Group 0: GPU 0,1,2,3

TP Group 1: GPU 4,5,6,7

router 的视角应该是：

Request Router

+--> Worker 0: TP Group 0

+--> Worker 1: TP Group 1

不是：

Request Router

+--> GPU 0

+--> GPU 1

+--> GPU 2

+--> GPU 3

因为一个请求不能只发给 GPU 0。

它需要整个 TP group 协同完成 forward。

所以 worker 的粒度应该是：

能独立完成一个请求 forward 的最小执行单元。

这个执行单元可能是一张 GPU，也可能是一组 GPU。

56.4 Tensor parallel 的基本直觉

LLM 里最重的计算通常是矩阵乘法。

比如线性层：

$$Y = XW$$

如果 W 很大，可以把它切到多张 GPU。

有两种常见切法：

1. column parallel。
2. row parallel。

56.4.1 Column parallel

把权重按输出维度切分：

$$W = [W_0, W_1, W_2, W_3]$$

每张 GPU 计算一部分输出：

$$Y_0 = XW_0$$

$$Y_1 = XW_1$$

$$Y_2 = XW_2$$

$$Y_3 = XW_3$$

最后可以得到分片输出：

$$Y = [Y_0, Y_1, Y_2, Y_3]$$

如果下一层也能接受分片输入，就可以暂时不 all-gather。

如果后续操作需要完整 hidden states，就需要 all-gather。

56.4.2 Row parallel

把权重按输入维度切分：

$$W = [W_0; W_1; W_2; W_3]$$

输入 X 也被切分:

$$X = [X_0, X_1, X_2, X_3]$$

每张 GPU 计算部分结果:

$$Z_0 = X_0 W_0$$
$$Z_1 = X_1 W_1$$
$$Z_2 = X_2 W_2$$
$$Z_3 = X_3 W_3$$

完整输出需要求和:

$$Y = Z_0 + Z_1 + Z_2 + Z_3$$

这通常需要 all-reduce。

56.5 为什么 tensor parallel 有通信开销

tensor parallel 的核心 trade-off 是:

用更多 GPU 分摊计算和显存, 但引入跨 GPU 通信。

每层 Transformer 里可能出现多次通信:

1. attention projection 后需要同步。
2. MLP projection 后需要同步。
3. logits 计算可能需要 gather 或 reduce。
4. residual、layer norm 的布局也会影响通信。

通信开销取决于:

1. TP size。
2. hidden size。
3. batch tokens 数。
4. GPU 间互联, NVLink、PCIe、InfiniBand。
5. collective 实现, NCCL 等。
6. 是否能和计算重叠。

所以 TP size 不是越大越好。

TP size 变大时, 每张 GPU 的计算减少, 但通信次数和同步成本可能增加。

当 batch tokens 很小时, decode 阶段尤其容易被通信延迟主导。

56.6 Prefill 和 decode 对 TP 的压力不同

prefill 阶段一次处理 prompt 的多个 token。

$$\text{batch tokens} = \text{sum}(\text{prompt lengths})$$

矩阵乘法比较大, GPU 利用率更容易打满。

TP 的通信开销可以被较大的计算量摊薄。

decode 阶段每个 request 通常每轮只生成一个 token。

$$\text{batch tokens} = \text{running requests count}$$

如果并发不高, 每轮 decode 的矩阵很小。

这时通信 latency 占比会变高。

所以同一个 TP 配置下:

prefill 可能很高效, decode 可能被通信和同步拖慢。

这也是 serving scheduler 要尽量把 decode batch 做大的原因之一。

56.7 Tensor parallel 下的 KV cache

KV cache 存的是每层 attention 的 key/value。

在 TP 下，attention heads 通常会被分到不同 GPU。

例如模型有 32 个 heads，TP size = 4：

GPU 0: heads 0-7 的 K/V

GPU 1: heads 8-15 的 K/V

GPU 2: heads 16-23 的 K/V

GPU 3: heads 24-31 的 K/V

每张 GPU 只保存自己负责 heads 的 KV cache。

这意味着：

1. KV cache 本身也被 sharded。
2. 每张 GPU 的 KV block manager 管理本地 shard。
3. 一个 request 的 KV blocks 在 TP group 内是一组对应分片。
4. 分配和释放要在所有 TP ranks 上保持一致。

可以把一个逻辑 block 看成：

Logical KV Block 17

rank 0 shard: heads 0-7

rank 1 shard: heads 8-15

rank 2 shard: heads 16-23

rank 3 shard: heads 24-31

从 scheduler 角度看，它分配的是逻辑 block。

从每个 GPU 角度看，它分配的是本地物理 block shard。

56.8 TP 下 block allocation 的一致性

TP group 里的每个 rank 都要对同一个 request 使用一致的 block table。

如果 rank 0 认为 request A 使用 blocks [1, 2, 3]，rank 1 却认为它使用 [1, 2, 4]，attention 会直接错乱。

所以 block allocation 通常由一个统一的调度逻辑决定，然后广播给各 rank。

伪代码：

```
class TPBlockManager:
    def allocate(self, request_id: str, num_blocks: int) -> List[int]:
        block_ids = self.logical_allocator.allocate(num_blocks)
        self.broadcast_block_table(request_id, block_ids)
        return block_ids

    def free(self, request_id: str) -> None:
        block_ids = self.request_blocks.pop(request_id)
        self.broadcast_free(request_id, block_ids)
        self.logical_allocator.free(block_ids)
```

真实系统里不一定真的每次用 Python broadcast。

但原则是一样的：

TP ranks 必须对 request 的 block table 达成一致。

56.9 Pipeline parallel 的基本直觉

pipeline parallel 按层切模型。

假设 4 个 stage:

Stage 0: embedding + layers 0-15

Stage 1: layers 16-31

Stage 2: layers 32-47

Stage 3: layers 48-63 + lm_head

一次 forward 要依次经过所有 stage:

hidden -> Stage 0 -> Stage 1 -> Stage 2 -> Stage 3 -> logits

为了提高利用率, 可以把 batch 切成 micro-batches, 让不同 stage 同时处理不同 micro-batch。

训练里这很常见。

但 serving decode 有特殊问题。

56.10 Pipeline bubble

pipeline 需要填充和排空。

假设 4 个 stage, 只有一个 micro-batch:

time 1: MB0 -> S0

time 2: MB0 -> S1

time 3: MB0 -> S2

time 4: MB0 -> S3

每个时刻只有一个 stage 在工作, 其他 stage 空闲。

这就是 bubble。

如果有多个 micro-batch:

time 1: MB0 -> S0

time 2: MB1 -> S0, MB0 -> S1

time 3: MB2 -> S0, MB1 -> S1, MB0 -> S2

time 4: MB3 -> S0, MB2 -> S1, MB1 -> S2, MB0 -> S3

利用率会提高。

但在线 LLM serving 的 decode 阶段, 每轮 token 生成有强依赖:

第 $t+1$ 个 token 依赖第 t 个 token 的采样结果。

所以不能随便把同一个请求的未来 token 提前放进 pipeline。

只能依赖不同请求和不同 micro-batch 来填充 pipeline。

当并发不够时, pipeline bubble 会很明显。

56.11 Pipeline parallel 下的 KV cache

pipeline parallel 按层切分。

所以 KV cache 也自然按层分布。

Stage 0: layers 0-15 的 KV cache

Stage 1: layers 16-31 的 KV cache

Stage 2: layers 32-47 的 KV cache

Stage 3: layers 48-63 的 KV cache

一个 request 的完整 KV cache 分布在多个 stage 上。

每个 stage 只保存自己负责层的 K/V。

这带来几个工程要求：

1. 每个 stage 都要知道 request 的位置和 block table。
2. request finish/cancel 时，所有 stage 都要释放对应 KV。
3. preemption 时，所有 stage 要一致处理。
4. 某个 stage OOM 会影响整个 pipeline worker。
5. stage 间要传递 hidden states，而不是完整 KV。

从 scheduler 视角看，仍然应该分配逻辑 request 资源。

但底层释放和清理必须覆盖所有 stage。

56.12 TP + PP 组合

大模型常常同时使用 tensor parallel 和 pipeline parallel。

例如 8 张 GPU：

Pipeline Stage 0: GPU 0,1,2,3 组成 TP group

Pipeline Stage 1: GPU 4,5,6,7 组成 TP group

这叫 TP=4, PP=2。

模型层分成两个 pipeline stage。

每个 stage 内部再用 4 张 GPU 做 tensor parallel。

router 看到的仍然是一个 worker：

Worker 0: TP=4, PP=2, GPUs 0-7

这个 worker 才能独立完成一个请求。

如果有 16 张 GPU，可以有两个这样的 worker：

Worker 0: TP=4, PP=2, GPUs 0-7

Worker 1: TP=4, PP=2, GPUs 8-15

router 在 Worker 0 和 Worker 1 之间做请求级路由。

worker 内部做 TP/PP 通信和调度。

56.13 分布式 KV cache 为什么难迁移

第 55 章说过，worker crash 后，已经输出 token 的请求通常不能透明重试。

一个重要原因就是 KV cache 很难跨 worker 迁移。

KV cache 不是一个小对象。

对于长上下文和大模型，它可能非常大。

一个粗略估算：

$$\text{KV bytes} = \text{layers} * \text{tokens} * \text{kv_heads} * \text{head_dim} * 2(K,V) * \text{dtype_bytes}$$

如果：

layers = 80

tokens = 32000

kv_heads = 8

head_dim = 128

dtype_bytes = 2

那么：

$$\begin{aligned}\text{KV bytes} &= 80 * 32000 * 8 * 128 * 2 * 2 \\ &\approx 10 \text{ GB}\end{aligned}$$

这只是一个 request 的 KV cache 量级。

跨 worker 迁移意味着要把这些 GPU memory 从一个 worker 搬到另一个 worker，并且还要保持：

1. layer 顺序一致。
2. head shard 一致。
3. block table 一致。
4. dtype 和 layout 一致。
5. position encoding 状态一致。
6. sampling 状态一致。

这非常昂贵。

所以多数 serving 系统不会把正在 decode 的请求随意迁移到另一个 worker。

56.14 KV cache 的 locality

因为 KV cache 难迁移，所以 request 一旦进入某个 worker，通常会在这个 worker 上完成。

这叫 request locality。

它影响很多设计：

1. cancel 必须发回原 worker。
2. output stream 必须关联原 worker。
3. worker 过载时，已经 running 的请求不能简单搬走。
4. router 只能影响新请求，不能轻易重平衡正在运行的请求。
5. prefix cache 也只在 worker 本地有效。

这传统无状态 HTTP 服务很不一样。

LLM serving worker 是有大量 GPU resident state 的。

56.15 分布式 prefix cache

prefix cache 在单 worker 内已经不简单。

到了多 worker，更复杂。

有三种思路。

56.15.1 worker-local prefix cache

每个 worker 只缓存自己处理过的 prefix。

优点：

1. 实现简单。
2. 不需要跨 worker KV 传输。
3. cache lookup 快。

缺点：

1. 相同 prefix 打到不同 worker 会重复 prefill。
2. cache 命中依赖 sticky routing。
3. 热点 prefix 可能造成 worker 热点。

这是最常见的起点。

56.15.2 centralized prefix directory

维护一个全局目录，记录哪个 worker 有某个 prefix。

prefix_hash -> worker_ids

router 可以优先把请求发给已有 prefix 的 worker。

但目录只记录位置，不复制 KV。

真正的 KV 仍然在 worker 本地。

优点是能提高 locality。

缺点是目录有一致性和过期问题。

56.15.3 remote KV reuse

理论上，可以从远端 worker 拉取 KV cache。

但这很难：

1. KV 很大。
2. GPU 间跨节点传输慢。
3. layout 和 block table 要兼容。
4. 传输期间 prefix 可能被 eviction。
5. 拉取成本可能超过重新 prefill。

所以 remote KV reuse 需要非常谨慎。

只有当 prefix 很长、网络很快、复用很多时才可能划算。

56.16 Scheduler 和 parallel group 的关系

在模型并行 worker 里，scheduler 仍然做 request-level 和 token-level 调度。

但它调度的 batch 会被底层执行器分发到多个 GPU rank。

可以分成两层：

Scheduler layer:

decide which requests/tokens run this step

Distributed executor layer:

run the batch across TP/PP ranks

scheduler 不应该关心每个 matmul 怎么切。

executor 不应该决定全局 waiting queue 谁先跑。

边界可以这样理解：

1. scheduler 输出 logical batch。
2. block manager 输出 logical block table。
3. distributed executor 把 batch 和 block table 转成 rank-local tensors。
4. 每个 rank 运行自己的 shard。
5. executor 汇总 logits 或 sampled token。
6. scheduler/output processor 更新 request 状态。

这个边界清晰，系统才可维护。

56.17 rank-local 数据

TP/PP 下，每个 rank 都有自己的本地数据。

常见 rank-local 数据包括：

1. 权重 shard。
2. KV cache shard。
3. CUDA graph 或 kernel workspace。
4. local block table。
5. local attention metadata。
6. local hidden states。
7. communication buffers。

不能假设 rank 0 拿到了全部数据。

很多 debug 难点都来自这里。

例如 logits 可能只有最后一个 pipeline stage 才有。

例如某些 TP 切分下，每个 rank 只有 vocab 的一部分 logits。

例如 KV block OOM 可能只发生在某个 rank。

所以 metrics 也要区分 global 和 rank-local。

56.18 分布式执行的指标

除了前面章节的指标，模型并行 worker 还需要：

```
tp_size
pp_size
rank_forward_ms{rank}
rank_kv_used_blocks{rank}
rank_kv_free_blocks{rank}
rank_oom_total{rank}
collective_all_reduce_ms
collective_all_gather_ms
collective_send_recv_ms
pipeline_stage_idle_ms{stage}
pipeline_bubble_ratio
executor_sync_ms
```

这些指标能回答：

1. 是计算慢，还是通信慢。
2. 哪个 rank 或 stage 是 straggler。
3. pipeline bubble 是否严重。
4. KV cache 是否在某个 rank 上不均衡。
5. TP size 是否过大。
6. PP 切层是否不均衡。

没有这些指标，多 GPU 性能问题很难定位。

56.19 TP size 怎么选

选择 TP size 时要看四件事：

1. 模型权重是否能放下。
2. KV cache 是否能放下。
3. 单请求延迟要求。
4. GPU 互联带宽。

如果模型单卡放不下，TP 是必要的。

如果模型单卡能放下，但想提高吞吐，TP 不一定总是好。

因为 TP 会增加通信和同步。

粗略经验：

1. NVLink 机器上可以更积极使用 TP。
2. PCIe 机器上 TP 通信成本更明显。
3. decode batch 小时，TP size 过大容易拖慢 TPOT。
4. prefill-heavy workload 更容易从 TP 获益。
5. latency-sensitive 场景要谨慎扩大 TP size。

最终还是要用第 53 章的 benchmark 方法验证。

56.20 PP size 怎么选

选择 PP size 时要看：

1. 每个 stage 的权重显存。
2. 每个 stage 的 KV cache 显存。
3. stage 计算是否均衡。
4. stage 间通信带宽。
5. decode 并发是否足够填满 pipeline。

PP size 太小，单 stage 放不下。

PP size 太大，pipeline bubble 和通信开销变大。

在线 serving 里，PP 往往比 TP 更难调。

因为 decode 是逐 token 依赖的，无法像训练那样轻松堆大量 micro-batch。

如果必须使用 PP，要特别关注：

1. stage idle time。
2. pipeline bubble ratio。
3. per-stage latency。
4. micro-batch scheduling。
5. prefill/decode 混合时的 pipeline 调度。

56.21 常见错误

错误一：把 TP group 里的每张 GPU 当独立 worker 路由。

结果：请求无法完整 forward，调度边界错误。

错误二：只看多卡总算力，不看通信。

结果：TP size 增大后 decode 反而更慢。

错误三：认为 prefill 快就代表 decode 也快。

结果：长 prompt benchmark 表现很好，但在线聊天 TPOT 很差。

错误四：PP 切层只按层数平均。

结果：不同 stage 计算不均衡，最慢 stage 决定整体速度。

错误五：cancel 只释放某个 rank 的 KV。

结果：其他 rank KV 泄漏，后续请求 OOM。

错误六：假设 KV cache 可以随便跨 worker 迁移。

结果：迁移成本超过重新 prefill，或者 layout/block table 不一致。

错误七: prefix cache 做了全局目录, 但没有处理 eviction 过期。

结果: router 以为 worker 有 prefix, 实际已经被淘汰。

错误八: 只采集 rank 0 指标。

结果: 某个非 0 rank OOM、慢 kernel 或通信等待被掩盖。

错误九: 忽略 pipeline bubble。

结果: 多卡显存够了, 但 GPU 利用率很低。

错误十: 把模型并行当成免费扩展。

结果: 显存问题解决了, 但延迟、通信、容错和调度复杂度显著上升。

56.22 面试高频问题

问题一: tensor parallel 和 replica parallel 有什么区别?

回答要点: replica parallel 是多个完整模型副本, 每个请求只进入一个 worker, router 做请求级负载均衡。tensor parallel 是一个模型的权重和计算切到多张 GPU, 一个请求需要整个 TP group 协同完成。TP 可以解决单卡放不下的问题, 但会引入跨卡通信和同步。

问题二: 为什么 TP size 不是越大越好?

回答要点: TP size 增大能降低单卡权重和计算压力, 但会增加 all-reduce、all-gather 等通信开销。decode 阶段 batch tokens 小, 通信 latency 占比更高, TP size 过大可能让 TPOT 变差。选择 TP size 要结合显存、互联带宽、prefill/decode workload 和 benchmark。

问题三: pipeline parallel 为什么在线 inference 难?

回答要点: PP 按层切模型, 需要多个 micro-batch 填满 pipeline。训练可以用大量 micro-batch, 但在线 decode 每个请求的下一 token 依赖上一 token 采样结果, 并发不足时 pipeline bubble 很大。还要处理 stage imbalance、stage 间通信和分布式 KV 释放。

问题四: TP/PP 下 KV cache 怎么放?

回答要点: TP 下 attention heads 通常被切到不同 rank, 每个 rank 保存自己 head shard 的 KV, 逻辑 block 在所有 TP ranks 上要保持一致。PP 下模型按层切分, 每个 stage 保存自己负责层的 KV。request finish、cancel、preemption 时, 所有 rank/stage 都要一致清理。

问题五: 为什么正在 decode 的请求不容易跨 worker 迁移?

回答要点: 因为请求的 KV cache 很大, 且分布在 GPU memory、TP ranks 或 PP stages 上。迁移要保持 layer、head shard、block table、position、dtype、layout 和 sampling 状态一致, 传输成本很高。多数 serving 系统会让请求在原 worker 上完成, 而不是运行中迁移。

56.23 标准回答模板

如果面试官问“多 GPU LLM inference 里 tensor parallel、pipeline parallel 和 KV cache 有什么工程难点”, 可以这样回答:

我会先区分并行粒度。replica parallel 是多份完整模型副本, router 在 worker 之间做请求级负载均衡; tensor parallel 是一个模

TP 的核心 trade-off 是显存和计算分摊换通信。比如 column parallel、row parallel 会带来 all-gather 或 all-reduce。prefill 阶段 batch tokens 多, 计算量大, 通信相对容易摊薄; decode 阶段每轮 token 少, 如果并发不够, 通信 latency 会

PP 的难点是 pipeline bubble 和 stage imbalance。在线 decode 有逐 token 依赖, 不能像训练一样随便堆未来 token 填 pipeline。

KV cache 是 serving 的关键状态。TP 下 KV 通常按 heads 分片, 每个 rank 保存本地 shard, 但逻辑 block table 必须一致; PP 下每 local prefix cache, 而不是频繁迁移 KV。

56.24 Distributed Parallel KV 公式、通信门禁和可运行 demo

对于一个请求，KV cache 的总字节数可以估算为：

$$M_i^{\{\mathrm{kv}\}} = 2LH N_i D_h b$$

其中 2 来自 K/V，L 是层数，H 是 KV heads， N_i 是已缓存 token 数， D_h 是 head dim，b 是每个元素字节数。

TP size 为 T 时，每个 rank 通常只保存自己的 head shard：

$$M_{\{i,r\}}^{\{\mathrm{tp}\}} = \frac{M_i^{\{\mathrm{kv}\}}}{T}$$

PP size 为 P 时，每个 stage 只保存自己负责层的 KV：

$$M_{\{i,s\}}^{\{\mathrm{pp}\}} = 2L_{\mathrm{sh}} N_i D_h b$$

一次 collective 的简化通信代价可以写成：

$$T_{\{\mathrm{comm}\}}(m) = \alpha + \frac{m}{B_{\{\mathrm{link}\}}}\rho$$

pipeline bubble 的教学版比例：

$$R_{\{\mathrm{bubble}\}} = \frac{P-1}{M_{\{\mathrm{micro}\}} + P - 1}$$

远程拉取 KV 与重算的决策可以写成：

$$D_i^{\{\mathrm{move}\}} = \mathbf{1} \left[T_{\{\mathrm{transfer}\}}(M_i^{\{\mathrm{kv}\}}) < T_i^{\{\mathrm{recompute}\}} \right]$$

最终用一个组合门禁收束：

$$G_{\{\mathrm{dpkv}\}} = G_{\{\mathrm{group}\}} G_{\{\mathrm{tp}\}} G_{\{\mathrm{pp}\}} G_{\{\mathrm{table}\}} G_{\{\mathrm{comm}\}} G_{\{\mathrm{bubble}\}}$$

下面这个 0 依赖 demo 用 toy 数字验证这些边界：

```
from dataclasses import dataclass, field
from math import ceil
```

```
@dataclass
```

```
class ParallelConfig:
```

```
    num_layers: int
    num_kv_heads: int
    head_dim: int
    dtype_bytes: int
    tp_size: int
    pp_size: int
    block_size: int
    link_bandwidth_mib_s: float
    link_latency_ms: float
```

```
@dataclass
```

```
class RequestProfile:
```

```
    request_id: str
    prompt_tokens: int
    generated_tokens: int
    output_budget: int
    micro_batches: int
```

```
@dataclass
```

```
class TPRankKV:
```

```
    rank_id: int
    head_range: tuple[int, int]
```

```

block_table: list[int] = field(default_factory=list)
local_blocks: dict[int, str] = field(default_factory=dict)

```

@dataclass

class PPStageKV:

```

    stage_id: int
    layer_range: tuple[int, int]
    block_table: list[int] = field(default_factory=list)
    local_layers: dict[int, list[int]] = field(default_factory=dict)

```

class ToyDistributedParallelKVAuditor:

def __init__(self, config: ParallelConfig):

```

    self.config = config
    self.tp_ranks = self._build_tp_ranks()
    self.pp_stages = self._build_pp_stages()
    self.cleanup_events = []

```

def _build_tp_ranks(self) -> list[TPRankKV]:

```

    heads_per_rank = self.config.num_kv_heads // self.config.tp_size
    ranks = []
    for rank in range(self.config.tp_size):
        start = rank * heads_per_rank
        end = start + heads_per_rank
        ranks.append(TPRankKV(rank_id=rank, head_range=(start, end)))
    return ranks

```

def _build_pp_stages(self) -> list[PPStageKV]:

```

    layers_per_stage = self.config.num_layers // self.config.pp_size
    stages = []
    for stage in range(self.config.pp_size):
        start = stage * layers_per_stage
        end = start + layers_per_stage
        stages.append(PPStageKV(stage_id=stage, layer_range=(start, end)))
    return stages

```

def token_count(self, req: RequestProfile) -> int:

```

    return req.prompt_tokens + req.generated_tokens

```

def required_blocks(self, req: RequestProfile) -> int:

```

    return ceil(self.token_count(req) / self.config.block_size)

```

def kv_bytes_total(self, req: RequestProfile) -> int:

```

    return (
        2
        * self.config.num_layers
        * self.config.num_kv_heads
        * self.token_count(req)
        * self.config.head_dim
        * self.config.dtype_bytes
    )

```

def kv_bytes_per_tp_rank(self, req: RequestProfile) -> int:

```

    return self.kv_bytes_total(req) // self.config.tp_size

```

```

def kv_bytes_per_pp_stage(self, req: RequestProfile) -> List[int]:
    result = []
    for stage in self.pp_stages:
        layer_count = stage.layer_range[1] - stage.layer_range[0]
        result.append(
            2
            * layer_count
            * self.config.num_kv_heads
            * self.token_count(req)
            * self.config.head_dim
            * self.config.dtype_bytes
        )
    return result

def allocate_distributed_kv(self, req: RequestProfile) -> List[int]:
    logical_blocks = list(range(self.required_blocks(req)))
    for rank in self.tp_ranks:
        rank.block_table = list(logical_blocks)
        rank.local_blocks = {
            block_id: f"rank{rank.rank_id}_block{block_id}" for block_id in logical_blocks
        }
    for stage in self.pp_stages:
        stage.block_table = list(logical_blocks)
        stage.local_layers = {
            layer: list(logical_blocks)
            for layer in range(stage.layer_range[0], stage.layer_range[1])
        }
    return logical_blocks

def tp_head_coverage(self) -> List[int]:
    heads = []
    for rank in self.tp_ranks:
        heads.extend(range(rank.head_range[0], rank.head_range[1]))
    return heads

def tp_block_tables_consistent(self) -> bool:
    tables = [rank.block_table for rank in self.tp_ranks]
    return all(table == tables[0] for table in tables)

def pp_layer_coverage(self) -> List[int]:
    layers = []
    for stage in self.pp_stages:
        layers.extend(range(stage.layer_range[0], stage.layer_range[1]))
    return layers

def collective_ms(self, message_mib: float, factor: float) -> float:
    transfer = message_mib / self.config.link_bandwidth_mib_s * 1000.0
    return round(self.config.link_latency_ms + transfer * factor, 3)

def communication_report(self, req: RequestProfile) -> Dict[str, float]:
    hidden_mib = req.prompt_tokens * self.config.num_kv_heads * self.config.head_dim
    hidden_mib *= self.config.dtype_bytes / (1024 * 1024)
    decode_mib = self.config.num_kv_heads * self.config.head_dim
    decode_mib *= self.config.dtype_bytes / (1024 * 1024)

```



```

factor = (self.config.tp_size - 1) / self.config.tp_size
prefill_ms = self.collective_ms(hidden_mib, factor)
decode_ms = self.collective_ms(decode_mib, factor)
return {
    "prefill_all_reduce_ms": prefill_ms,
    "decode_all_reduce_ms": decode_ms,
    "decode_latency_share": round(decode_ms / max(prefill_ms, 0.001), 3),
}

def pipeline_bubble_ratio(self, req: RequestProfile) -> float:
    return round(
        (self.config.pp_size - 1) / (req.micro_batches + self.config.pp_size - 1), 3
    )

def migration_decision(self, req: RequestProfile) -> dict[str, object]:
    kv_mib = self.kv_bytes_total(req) / (1024 * 1024)
    transfer_ms = self.collective_ms(kv_mib, 1.0)
    recompute_ms = round(req.prompt_tokens * 0.035, 3)
    decision = "fetch_remote" if transfer_ms < recompute_ms else "recompute"
    return {
        "kv_mib": round(kv_mib, 3),
        "transfer_ms": transfer_ms,
        "recompute_ms": recompute_ms,
        "decision": decision,
    }

def cleanup_request(self, req: RequestProfile) -> bool:
    for rank in self.tp_ranks:
        rank.block_table = []
        rank.local_blocks = {}
        self.cleanup_events.append((req.request_id, "tp_rank", rank.rank_id))
    for stage in self.pp_stages:
        stage.block_table = []
        stage.local_layers = {}
        self.cleanup_events.append((req.request_id, "pp_stage", stage.stage_id))
    return all(not rank.block_table for rank in self.tp_ranks) and all(
        not stage.block_table for stage in self.pp_stages
    )

def audit(self, req: RequestProfile) -> tuple[dict[str, object], dict[str, bool]]:
    logical_blocks = self.allocate_distributed_kv(req)
    kv_total = self.kv_bytes_total(req)
    kv_tp = self.kv_bytes_per_tp_rank(req)
    kv_pp = self.kv_bytes_per_pp_stage(req)
    communication = self.communication_report(req)
    migration = self.migration_decision(req)
    table_ok = self.tp_block_tables_consistent()
    cleanup_ok = self.cleanup_request(req)

    summary = {
        "worker_unit": "tp_pp_group",
        "tp_head_ranges": [rank.head_range for rank in self.tp_ranks],
        "tp_head_coverage": self.tp_head_coverage(),
        "logical_blocks": logical_blocks,
        "tp_block_tables_consistent_before_cleanup": table_ok,
    }

```

```

        "pp_layer_ranges": [stage.layer_range for stage in self.pp_stages],
        "pp_layer_coverage": self.pp_layer_coverage(),
        "kv_total_mib": round(kv_total / (1024 * 1024), 3),
        "kv_per_tp_rank_mib": round(kv_tp / (1024 * 1024), 3),
        "kv_per_pp_stage_mib": [round(value / (1024 * 1024), 3) for value in kv_pp],
        "communication": communication,
        "pipeline_bubble_ratio": self.pipeline_bubble_ratio(req),
        "migration": migration,
        "cleanup_events": self.cleanup_events,
        "cleanup_ok": cleanup_ok,
    }
    gates = {
        "parallel_group_boundary_ready": summary["worker_unit"] == "tp_pp_group",
        "tp_head_shard_ready": summary["tp_head_coverage"]
        == list(range(self.config.num_kv_heads)),
        "tp_block_table_ready": summary["tp_block_tables_consistent_before_cleanup"],
        "pp_layer_ownership_ready": summary["pp_layer_coverage"]
        == list(range(self.config.num_layers)),
        "kv_accounting_ready": abs(
            summary["kv_total_mib"]
            - summary["kv_per_tp_rank_mib"] * self.config.tp_size
        )
        <= 0.004,
        "collective_cost_ready": communication["prefill_all_reduce_ms"] > 0
        and communication["decode_all_reduce_ms"] > 0,
        "pipeline_bubble_ready": summary["pipeline_bubble_ratio"] > 0,
        "migration_decision_ready": migration["decision"] in {"fetch_remote", "recompute"},
        "cross_rank_cleanup_ready": cleanup_ok,
    }
    gates["distributed_parallel_kv_gate"] = all(gates.values())
    return summary, gates

```

```

config = ParallelConfig(
    num_layers=8,
    num_kv_heads=8,
    head_dim=16,
    dtype_bytes=2,
    tp_size=4,
    pp_size=2,
    block_size=16,
    link_bandwidth_mib_s=12000.0,
    link_latency_ms=0.08,
)
request = RequestProfile(
    request_id="req-long",
    prompt_tokens=80,
    generated_tokens=16,
    output_budget=32,
    micro_batches=2,
)

```

```

auditor = ToyDistributedParallelKVAuditor(config)
summary, gates = auditor.audit(request)

```

```
print("distributed_parallel_kv_summary=", summary)
print("distributed_parallel_kv_gates=", gates)
```

一次运行的核心输出类似：

```
distributed_parallel_kv_summary= {'worker_unit': 'tp_pp_group', 'tp_head_ranges': [(0, 2), (2, 4), (4, 6), (6, 8)], 'tp_head_ranges_long', 'tp_rank', 0), ('req-long', 'tp_rank', 1), ('req-long', 'tp_rank', 2), ('req-long', 'tp_rank', 3), ('req-long', 'pp_stage', 0), ('req-long', 'pp_stage', 1)], 'cleanup_ok': True}
distributed_parallel_kv_gates= {'parallel_group_boundary_ready': True, 'tp_head_shard_ready': True, 'tp_block_table_ready': True}
```

这个 demo 证明了几个关键点：

1. router 的 worker 单元是 `tp_pp_group`，不是单张 GPU。
2. TP rank 按 KV heads 分片，但逻辑 block table 必须一致。
3. PP stage 按 layer range 拥有本地 KV。
4. KV bytes 可以从 total、TP rank shard 和 PP stage shard 三个视角对齐。
5. collective 成本和 pipeline bubble 都会进入 TPOT / E2E 解释。
6. 远程 KV 迁移是否值得做，必须比较 transfer 与 recompute，而不是默认搬迁。
7. finish / cancel / preemption 的清理要覆盖所有 TP ranks 和 PP stages。

56.25 小练习

1. 画出 replica parallel、TP=2、PP=2 的 worker 拓扑图。
2. 给一个 32 heads、TP=4 的模型写出每个 rank 负责哪些 heads。
3. 估算一个请求的 KV cache 大小。
4. 写一个函数，根据 `layers/tokens/kv_heads/head_dim/dtype_bytes` 计算 KV bytes。
5. 解释 column parallel 为什么可能需要 all-gather。
6. 解释 row parallel 为什么可能需要 all-reduce。
7. 给 TP worker 增加 `rank_kv_free_blocks` 指标。
8. 给 TP worker 增加 `collective_all_reduce_ms` 指标。
9. 给 PP worker 增加 `pipeline_stage_idle_ms` 指标。
10. 构造一个 decode batch 很小的 benchmark，观察 TPOT。
11. 构造一个长 prompt prefill benchmark，观察 TP 的收益。
12. 模拟 cancel 时只释放 rank 0 KV，说明会发生什么。
13. 设计一个 worker-local prefix cache 的 sticky routing 策略。
14. 设计一个 `prefix_hash -> worker_ids` 的 prefix directory。
15. 比较重新 prefill 和远程 KV 拉取在不同 prefix 长度下的成本。

56.26 本章总结

多 GPU inference 不是简单把 GPU 数量乘上去。

replica parallel、tensor parallel 和 pipeline parallel 的边界不同。

replica parallel 让多个完整模型副本独立处理请求，router 做请求级负载均衡。

tensor parallel 把一个模型的张量和计算切到多张 GPU，一个请求需要整个 TP group 协同完成。

pipeline parallel 按层切分模型，一个请求依次经过多个 stage。

从 router 视角看，能独立完成 forward 的并行组才是 worker。

TP 的关键成本是通信。

TP size 越大，单卡显存和计算压力越小，但 all-reduce、all-gather 和同步开销越明显。

prefill 和 decode 对 TP 的表现不同。

prefill 计算量大，更容易摊薄通信；decode batch 小时，TPOT 容易受通信影响。

PP 的关键问题是 pipeline bubble 和 stage imbalance。

在线 decode 有逐 token 依赖，并发不足时 pipeline 利用率会下降。

KV cache 在模型并行下也会分布式化。

TP 下 KV 通常按 head shard 分布在不同 ranks。

PP 下 KV 按层分布在不同 stages。

request finish、cancel、preemption 必须在所有 ranks/stages 上一致清理。

KV cache 很大，并且和 block table、position、layout、rank shard 强绑定。

所以正在 decode 的请求很难跨 worker 迁移。

实际系统通常依赖 request locality、worker-local prefix cache 和 sticky routing，而不是频繁搬迁 KV。

下一章可以继续讨论：如果要把这些能力产品化，需要怎样设计 OpenAI-compatible API、流式协议、错误码和限流鉴权。

第 57 章 OpenAI-compatible API、流式协议、错误码和限流鉴权

第 56 章我们讨论了多 GPU worker 内部的 tensor parallel、pipeline parallel 和分布式 KV cache。

到这里，一个 LLM serving engine 的内部能力已经比较完整：

1. continuous batching。
2. PagedAttention 和 KV block manager。
3. prefix cache。
4. preemption。
5. benchmark 和 trace。
6. 异步 engine loop。
7. 多 worker router。
8. 多 GPU parallel group。

但这些还只是内部系统。

如果要真正对外提供服务，还需要一个稳定的 API 层。

用户不会直接调用 scheduler，也不会直接操作 KV cache。

用户看到的是 HTTP endpoint、JSON request、streaming response、错误码、鉴权和限流。

这一层做不好，engine 再强也很难被可靠使用。

本章讨论如何把 inference engine 包装成 OpenAI-compatible serving API。

57.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 serving engine 如何暴露 OpenAI-compatible API 的稳定工程边界。

公开资料校准主要参考四类口径：

1. OpenAI Chat Completions API reference 对 messages、choices、finish_reason、stream chunk、stream_options.include_usage 等字段的公开说明。
2. OpenAI streaming guide 对 stream=true、HTTP streaming over server-sent events、Chat Completions / legacy Completions 增量 chunk 的公开说明。
3. OpenAI error codes guide 对 401、403、429、500、503 等错误含义，以及认证、额度、rate limit 和 overload 的公开说明。
4. OpenAI rate limits guide 对 RPM、RPD、TPM、TPD、IPM、audio minutes 等多维速率限制，以及 synchronous request 的 RPM / TPM 区分的公开说明。

本章不复刻 OpenAI 官方 API 的全部字段、模型行为、SDK 实现、Responses API 完整事件流、tool calls、vision / audio content parts、project / organization 权限体系、真实 billing quota、真实风控策略或官方错误消息。我们只实现一个教学版兼容子集：

Bearer auth -> JSON validation -> model permission -> chat template -> token budget -> RPM / TPM / concurrency limit -> EngineRequest -> non-stream response or SSE stream -> error response with request_id

第二轮新增 demo 的验收重点是：兼容层要诚实声明支持字段；不支持字段不能静默吞掉；streaming chunk 要可解析且以 [DONE] 结束；鉴权、权限和限流要在昂贵 tokenization / engine admission 前完成；rate limit 与系统 admission control 要区分；错误响应必须有稳定的 type、code、param、message 和 request_id。

57.1 本章目标

读完本章，你应该能讲清：

1. 为什么很多 serving 系统要兼容 OpenAI API。
2. /v1/chat/completions 和 /v1/completions 的核心差异。
3. chat messages 如何转成 prompt 和 token ids。
4. SSE streaming 协议如何组织增量输出。
5. 非流式和流式接口的状态管理差异。
6. 错误码应该如何分层。
7. 鉴权、配额和限流应该放在哪些位置。
8. request id、trace id、idempotency key 有什么作用。
9. API 层如何把请求安全地交给 router 和 engine。

本章重点不是复刻某个厂商的所有字段。

重点是理解一个生产级 LLM API 层应该具备哪些工程边界。

57.2 为什么要兼容 OpenAI API

OpenAI-compatible API 已经成为事实标准之一。

很多 SDK、agent framework、评测工具、IDE 插件和业务系统默认支持它。

如果你的 serving 系统兼容这些接口，用户接入成本会低很多。

兼容的收益包括：

1. 可以直接使用 OpenAI SDK。
2. 可以接入 LangChain、LlamaIndex、AutoGen 等框架。
3. 可以复用已有评测脚本。
4. 可以让业务迁移更容易。
5. 可以降低 API 文档解释成本。

但兼容不等于盲目复制。

你需要明确：

1. 哪些字段支持。
2. 哪些字段忽略。
3. 哪些字段不支持并返回错误。
4. streaming 格式是否严格兼容。
5. 错误响应格式是否稳定。

不要让 API 看起来兼容，实际行为却不兼容。

这会让用户 debug 非常痛苦。

57.3 常见 endpoint

最常见的 endpoint 包括：

```
GET /v1/models
POST /v1/completions
POST /v1/chat/completions
```

POST /v1/embeddings

对生成式 LLM serving 来说, 最重要的是:

POST /v1/chat/completions

它接收 messages, 返回 assistant message。

一个最小请求:

```
{
  "model": "tiny-llm-chat",
  "messages": [
    {"role": "system", "content": " 你是一个有帮助的助手。"},
    {"role": "user", "content": " 解释一下 KV cache。"}
  ],
  "max_tokens": 256,
  "temperature": 0.7,
  "stream": true
}
```

一个最小非流式响应:

```
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1710000000,
  "model": "tiny-llm-chat",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "KV cache 是推理时缓存历史 token 的 key/value..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 18,
    "completion_tokens": 64,
    "total_tokens": 82
  }
}
```

57.4 Chat Completions 和 Completions

/v1/completions 接收的是 prompt:

```
{
  "model": "tiny-llm",
  "prompt": "The capital of France is",
  "max_tokens": 16
}
```

/v1/chat/completions 接收的是 messages:

```
{
  "model": "tiny-llm-chat",
  "messages": [
```

```

    {"role": "system", "content": "You are helpful."},
    {"role": "user", "content": "What is the capital of France?"}
  ]
}

```

对 engine 来说，两者最终都会变成 token ids。

区别在 API 层：

1. completions 直接把 prompt tokenize。
2. chat completions 需要先套 chat template。
3. chat messages 有 role 和多轮上下文。
4. chat 可能包含 tool calls、multi-modal content 等复杂结构。

可以把转换路径理解为：

HTTP JSON

```

-> validate request
-> normalize fields
-> render prompt text by template
-> tokenize
-> route to worker
-> stream or collect output

```

57.5 Chat template

chat model 训练时通常使用固定格式。

例如：

```

<|system|>
你是一个有帮助的助手。
<|user|>
解释一下 KV cache。
<|assistant|>

```

API 层要把 messages 渲染成模型期望的 prompt。

伪代码：

```

def render_chat_prompt(messages: list[dict]) -> str:
    parts = []
    for msg in messages:
        role = msg["role"]
        content = msg["content"]
        if role == "system":
            parts.append(f"<|system|>\n{content}\n")
        elif role == "user":
            parts.append(f"<|user|>\n{content}\n")
        elif role == "assistant":
            parts.append(f"<|assistant|>\n{content}\n")
        else:
            raise BadRequestError(f"unsupported role: {role}")
    parts.append("<|assistant|>\n")
    return "".join(parts)

```

真实系统不要手写错模板。

应该使用 tokenizer 或模型配置里定义的 chat template。

因为模板细节会影响模型输出质量。

一个多余空格、缺少 assistant prompt、错误 role token，都可能让模型行为异常。

57.6 请求字段校验

API 层必须在请求进入 engine 前做严格校验。

常见字段包括：

```
model
messages / prompt
max_tokens
temperature
top_p
top_k
stop
stream
n
seed
presence_penalty
frequency_penalty
logprobs
user
```

校验要回答：

1. 字段类型是否正确。
2. 数值范围是否合理。
3. model 是否存在。
4. prompt/messages 是否为空。
5. prompt tokens 是否超过 max_model_len。
6. max_tokens 是否超过服务限制。
7. 不支持的字段如何处理。
8. stop sequence 数量和长度是否受限。

例如：

```
def validate_sampling_params(req):
    if req.max_tokens <= 0:
        raise BadRequestError("max_tokens must be positive")
    if req.max_tokens > 4096:
        raise BadRequestError("max_tokens exceeds limit")
    if not (0.0 <= req.temperature <= 2.0):
        raise BadRequestError("temperature must be between 0 and 2")
    if not (0.0 < req.top_p <= 1.0):
        raise BadRequestError("top_p must be in (0, 1]")
```

坏请求应该尽早失败。

不要让明显非法请求进入 tokenizer、router 或 GPU engine。

57.7 从 API request 到 EngineRequest

API request 不应该直接作为 engine 内部对象。

更好的做法是转换成稳定的内部结构：

```
from dataclasses import dataclass
```

```
@dataclass
```



```

class SamplingParams:
    max_tokens: int
    temperature: float
    top_p: float
    stop: list[str]
    seed: int | None

@dataclass
class EngineRequest:
    request_id: str
    model: str
    prompt: str
    prompt_token_ids: list[int]
    sampling: SamplingParams
    stream: bool
    tenant_id: str
    user_id: str | None
    trace_id: str
    created_at: float

```

这样 API 层和 engine 层解耦。

以后 API 新增字段，不一定要污染 scheduler。

engine 只关心真正影响生成的参数和调度需要的元信息。

57.8 非流式响应

非流式接口比较简单。

API server 把请求交给 engine，然后等待完整输出。

request -> engine -> collect all tokens -> detokenize -> JSON response

伪代码：

```

async def chat_completions(req):
    engine_req = await build_engine_request(req)
    result = await engine.generate(engine_req)
    return build_chat_completion_response(engine_req, result)

```

非流式响应适合：

1. 短文本。
2. 后台任务。
3. eval。
4. 不需要边生成边展示的场景。

缺点是用户必须等全部 token 生成完才收到响应。

长输出场景下，TTFT 体验很差。

57.9 流式响应

流式接口会在生成过程中逐步返回 token delta。

OpenAI-compatible streaming 通常使用 SSE：

Content-Type: text/event-stream

每个事件形如：

data: {json}\n\n

最后发送:

data: [DONE]\n\n

一个 chat completion chunk 示例:

```
data: {"id":"chatcmpl-abc123","object":"chat.completion.chunk","created":1710000000,"model":"tiny-llm-chat","choices":[{"index":0,"delta":{"role":"assistant"},"finish_reason":null}]}
```

```
data: {"id":"chatcmpl-abc123","object":"chat.completion.chunk","created":1710000000,"model":"tiny-llm-chat","choices":[{"index":0,"delta":{"content":"KV"},"finish_reason":null}]}
```

```
data: {"id":"chatcmpl-abc123","object":"chat.completion.chunk","created":1710000000,"model":"tiny-llm-chat","choices":[{"index":0,"delta":{"content":" cache"},"finish_reason":null}]}
```

```
data: {"id":"chatcmpl-abc123","object":"chat.completion.chunk","created":1710000000,"model":"tiny-llm-chat","choices":[{"index":0,"delta":{},"finish_reason":"stop"}]}
```

data: [DONE]

注意第一个 chunk 通常会发送 role。

后续 chunk 发送 content delta。

结束 chunk 发送 finish_reason。

57.10 SSE 的几个坑

SSE 看起来简单, 但有很多细节。

57.10.1 每个事件要以空行结束

SSE 事件之间用空行分隔。

也就是:

data: {...}\n\n

少一个换行, 客户端可能一直等不到事件。

57.10.2 不要缓存响应

应该设置类似 header:

Cache-Control: no-cache

Connection: keep-alive

Content-Type: text/event-stream

如果经过 nginx 或网关, 还要避免 proxy buffering。

否则 token 已经生成了, 但客户端迟迟收不到。

57.10.3 处理 client disconnect

流式连接可能随时断开。

断开后必须 cancel engine request。

否则会产生僵尸请求。

这是第 54 章强调过的问题。

57.10.4 delta 不是完整文本

stream chunk 里的 `delta.content` 是增量文本。

客户端需要自己拼接。

服务端不要每次都发送完整历史文本。

否则带宽会随输出长度二次增长。

57.11 finish_reason

`finish_reason` 告诉用户生成为什么结束。

常见值包括：

```
stop
length
content_filter
tool_calls
error
```

最常见的是：

1. `stop`: 遇到 EOS 或 stop sequence。
2. `length`: 达到 `max_tokens`。
3. `error`: 生成中发生错误。

engine 内部可以有更细的结束原因：

```
class FinishReason:
    EOS = "eos"
    STOP_SEQUENCE = "stop_sequence"
    MAX_TOKENS = "max_tokens"
    CANCELLED = "cancelled"
    ERROR = "error"
```

API 层再映射到对外协议：

```
def to_api_finish_reason(reason):
    if reason in ("eos", "stop_sequence"):
        return "stop"
    if reason == "max_tokens":
        return "length"
    if reason == "error":
        return "error"
    return "stop"
```

不要把内部 enum 原样暴露给用户。

内部实现可以变，对外协议应该稳定。

57.12 Usage 统计

非流式响应通常包含 `usage`：

```
"usage": {
    "prompt_tokens": 18,
    "completion_tokens": 64,
    "total_tokens": 82
}
```

这些数字看似简单，但很重要。

它们用于：

1. 计费。
2. 配额。
3. 成本分析。
4. debug prompt 长度。
5. benchmark。

prompt_tokens 应该来自 tokenizer 后的长度。

completion_tokens 应该来自实际生成 token 数。

如果 stream 模式也要返回 usage，可以在最终 chunk 附带，或者提供额外配置。

但要文档化行为。

57.13 错误响应格式

错误响应必须稳定。

一个 OpenAI-compatible 风格错误可以长这样：

```
{
  "error": {
    "message": "max_tokens exceeds limit",
    "type": "invalid_request_error",
    "param": "max_tokens",
    "code": "max_tokens_exceeded"
  }
}
```

建议至少包含：

1. message：给人看的错误说明。
2. type：错误大类。
3. param：相关字段，可为空。
4. code：稳定机器可读错误码。
5. request_id：方便排查。

不要只返回：

```
{"detail":"error"}
```

这对调用方和排障都不够。

57.14 错误码分层

常见 HTTP 状态码和错误类型：

```
400 invalid_request_error
401 authentication_error
403 permission_denied
404 model_not_found
408 request_timeout
409 conflict_error
413 context_length_exceeded
429 rate_limit_error
499 client_closed_request
500 internal_server_error
503 service_unavailable
```

57.14.1 400 Bad Request

请求字段非法。

例如：

1. temperature 超范围。
2. messages 为空。
3. role 不合法。
4. stop 类型错误。

57.14.2 401 Unauthorized

缺少 API key 或 API key 无效。

57.14.3 403 Forbidden

认证成功，但没有权限访问该模型或 tenant。

57.14.4 413 Context Length Exceeded

prompt tokens + max_tokens 超过模型或服务限制。

这个错误很常见，应该清楚告诉用户限制是多少。

57.14.5 429 Rate Limit

租户或用户超过 QPS、并发、token rate 或配额限制。

57.14.6 503 Service Unavailable

服务过载、没有健康 worker、GPU 不可用、模型未加载。

429 和 503 要区分。

429 通常是用户额度或速率问题。

503 通常是服务端当前不可用或过载。

57.15 流式错误怎么处理

流式请求的错误分两类。

57.15.1 stream 开始前错误

如果还没发送任何 SSE event，可以直接返回普通 JSON 错误和对应 HTTP status。

例如参数校验失败：

```
HTTP 400
{"error": ...}
```

57.15.2 stream 开始后错误

如果已经发送了部分 token，HTTP status 已经是 200。

这时不能再改成 500。

只能在 stream 里发送错误事件，或发送结束 chunk。

例如：

```
data: {"error":{"message":"worker failed","type":"server_error","code":"worker_failed"}}
```

```
data: [DONE]
```

这也是为什么 worker failure 后，已经输出过 token 的请求不能透明重试。

API 协议层也已经进入 streaming 状态。

57.16 鉴权

API 层通常用 bearer token:

Authorization: Bearer sk-xxx

鉴权要在任何昂贵操作之前完成。

顺序应该是:

```
parse headers
-> authenticate api key
-> identify tenant/user
-> check permission
-> parse body
-> validate request
-> tokenize
-> route
```

鉴权结果可以转换成上下文:

```
@dataclass
class AuthContext:
    tenant_id: str
    user_id: str | None
    api_key_id: str
    allowed_models: set[str]
    qps_limit: int
    token_per_minute_limit: int
```

后续限流、审计和计费都依赖这个上下文。

不要把原始 API key 写进日志。

日志里最多记录 api_key_id 或 hash 后的标识。

57.17 权限模型

最小权限模型可以按 model 控制:

```
def check_model_permission(auth: AuthContext, model: str) -> None:
    if model not in auth.allowed_models:
        raise PermissionDeniedError(f"no access to model: {model}")
```

更复杂的系统还会控制:

1. 最大上下文长度。
2. 最大 max_tokens。
3. 是否允许 streaming。
4. 是否允许 tool calls。
5. 是否允许某些 LoRA adapter。
6. 每分钟 token 配额。
7. 每日或每月预算。

权限检查应该在 tokenization 前尽量完成。

但 context length 这种检查必须等 tokenization 后才能准确判断。

57.18 限流维度

LLM API 的限流不能只看 QPS。

因为请求成本差异巨大。

常见限流维度包括：

1. requests per minute。
2. tokens per minute。
3. concurrent requests。
4. concurrent tokens。
5. prompt tokens per minute。
6. completion tokens per minute。
7. daily budget。
8. per-model quota。

一个请求可能 QPS 很低，但 tokens 很高。

例如用户每分钟只发 2 个请求，但每个 prompt 100k tokens。

如果只按 QPS 限流，系统仍然会被打爆。

57.19 token-based rate limit

可以用 token bucket 做 tokens per minute 限制。

请求进入前先估算成本：

`estimated_tokens = prompt_tokens + max_tokens`

如果 token bucket 不够，就返回 429。

伪代码：

```
def check_token_limit(auth, prompt_tokens: int, max_tokens: int):
    estimated = prompt_tokens + max_tokens
    bucket = token_buckets[auth.tenant_id]
    if not bucket.try_consume(estimated):
        raise RateLimitError("token rate limit exceeded")
```

请求完成后，可以根据实际 completion_tokens 做修正。

但 admission 时必须用估算值。

否则用户可以用很大的 max_tokens 占住资源。

57.20 concurrency limit

并发限制用于保护 engine 的运行资源。

例如：

```
def acquire_concurrency_slot(auth):
    limiter = concurrency_limiters[auth.tenant_id]
    if not limiter.try_acquire():
        raise RateLimitError("too many concurrent requests")
```

```
def release_concurrency_slot(auth):
    concurrency_limiters[auth.tenant_id].release()
```

必须确保任何结束路径都会 release:

1. 正常完成。
2. 生成错误。
3. client disconnect。
4. timeout。
5. validation 后失败。

通常用 try/finally 包住请求生命周期。

并发 slot 泄漏会导致租户永久无法发请求。

57.21 admission control 和 rate limit 的区别

rate limit 是用户或租户维度的限制。

admission control 是系统容量维度的限制。

二者都可能拒绝请求，但含义不同。

rate limit: 你这个 tenant 超额度了。

admission control: 系统现在接不下这个请求。

前者通常返回 429。

后者可以返回 503 或 429，取决于产品语义。

建议明确区分错误码:

```
rate_limit_exceeded
engine_overloaded
no_healthy_worker
kv_capacity_exceeded
```

这样用户和运维都能知道该扩容、降速还是改请求参数。

57.22 request_id、trace_id 和 idempotency_key

57.22.1 request_id

每个请求都应该有服务端生成的 request_id。

它用于:

1. 日志串联。
2. metrics label。
3. 返回给用户报障。
4. router 映射。
5. cancel。

例如:

```
chatcmpl-9f2a...
```

57.22.2 trace_id

trace_id 用于跨服务链路追踪。

一个用户请求可能经过:

```
gateway -> api server -> tokenizer -> router -> worker -> output stream
```


所有日志都应该带 trace_id。
如果上游传了 trace header，可以继承。
否则服务端生成。

57.22.3 idempotency_key

idempotency key 用于处理客户端重试。
例如网络超时后，客户端不知道请求是否已经被服务端接收。
如果它带着同一个 idempotency key 再发一次，服务端可以避免重复执行。
但生成式请求的幂等并不简单。
如果第一次请求已经开始 streaming，第二次请求应该如何返回？
最小策略可以是：

1. 非流式请求：缓存最终结果一段时间。
2. 流式请求：只在 stream 未开始前允许幂等复用。
3. 已开始 stream 的请求：返回 conflict 或让客户端重新请求。

不要承诺做不到的强幂等。

57.23 日志和隐私

LLM API 日志很敏感。
prompt 里可能有用户隐私、代码、密钥或商业数据。
日志策略要明确：

1. 默认不要记录完整 prompt。
2. 可以记录 token counts、model、tenant、latency、finish_reason。
3. 如果需要 debug prompt，必须受权限控制。
4. API key 必须脱敏。
5. 错误日志不要包含过长 content。
6. trace 里避免记录原始用户文本。

一个安全的 access log 可以是：

request_id=chatcmpl-abc tenant=t1 model=m1 prompt_tokens=128 completion_tokens=64 ttft_ms=320 e2e_ms=2800 status=200 fi
这足够用于大部分运维分析。

57.24 API 层指标

除了 engine 指标，API 层还要有自己的指标：

```
http_requests_total{endpoint,status}
http_request_duration_ms{endpoint}
auth_failure_total
permission_denied_total
rate_limit_reject_total{reason}
request_validation_error_total{param}
context_length_exceeded_total
stream_started_total
stream_client_disconnect_total
stream_error_after_start_total
tokens_prompt_total{model,tenant}
tokens_completion_total{model,tenant}
```

这些指标能区分：

1. 是 API 层拒绝多。
2. 还是 router/engine 过载。
3. 是用户请求非法。
4. 还是模型生成失败。
5. 是客户端主动断开。
6. 还是服务端 streaming 错误。

不要只看 engine tokens/s。

产品化服务必须看 API 层成功率和错误分布。

57.25 OpenAI-compatible 不等于完全相同

兼容 API 时，要诚实声明差异。

例如：

Supported:

- model
- messages
- max_tokens
- temperature
- top_p
- stop
- stream
- seed

Ignored:

- user

Unsupported:

- logprobs
- response_format
- tools
- parallel_tool_calls

对 unsupported 字段，建议返回 400，而不是默默忽略。

对 ignored 字段，也要文档化。

默默忽略重要字段会制造错误预期。

比如用户传了 `response_format={"type":"json_object"}`，以为服务一定输出 JSON。

如果服务端实际忽略，就可能造成生产事故。

57.26 最小请求处理流程

把本章串起来，一个请求处理流程可以是：

1. 生成 request_id / trace_id
2. 解析 Authorization header
3. 鉴权，得到 AuthContext
4. 解析 JSON body
5. 校验字段类型和范围
6. 检查 model 权限
7. 渲染 chat template
8. tokenization
9. 检查 context length

10. 检查 token rate limit
11. 获取 concurrency slot
12. 构造 EngineRequest
13. 全局 admission control
14. router 选择 worker
15. 非流式：等待完整结果并返回 JSON
16. 流式：建立 SSE, 逐 chunk 返回 delta
17. 结束时记录 usage、metrics、日志
18. 任何路径都释放 concurrency slot

这个顺序不是唯一的。

但几个原则不变：

1. 便宜检查放前面。
2. 昂贵操作放后面。
3. 鉴权先于 tokenization。
4. context length 必须基于真实 token 数。
5. concurrency slot 必须可靠释放。
6. client disconnect 必须 cancel engine request。

57.27 常见错误

错误一：没有严格校验请求字段。

结果：非法参数进入 engine，错误变成内部异常。

错误二：chat template 写错。

结果：模型输出质量异常，但很难从 engine 指标看出来。

错误三：SSE chunk 少了空行。

结果：客户端一直收不到增量事件。

错误四：stream 断开后没有 cancel。

结果：僵尸请求继续占用 KV cache 和 decode slots。

错误五：只按 QPS 限流。

结果：少量超长 prompt 请求仍然可以打爆系统。

错误六：并发 slot 没有 finally release。

结果：租户并发额度泄漏，后续请求全部被拒绝。

错误七：把 API key 写进日志。

结果：严重安全事故。

错误八：unsupported 字段被默默忽略。

结果：用户以为功能生效，实际没有，业务语义出错。

错误九：stream 开始后还试图修改 HTTP status。

结果：协议行为混乱，客户端无法可靠处理错误。

错误十：错误码只有 500。

结果：用户、运维和自动重试逻辑都无法判断问题类型。

57.28 面试高频问题

问题一：为什么 LLM serving 要兼容 OpenAI API?

回答要点：OpenAI API 是事实标准之一，兼容后可以复用 SDK、agent framework、评测工具和业务接入代码。但兼容要明确支持、忽略和不支持的字段，不能看起来兼容实际语义不一致。

问题二：chat completions 请求进入 engine 前要做哪些步骤?

回答要点：先鉴权和权限检查，再解析和校验 JSON 字段，把 messages 用模型 chat template 渲染成 prompt, tokenize, 检查 context length 和 rate limit, 构造内部 EngineRequest, 经过 admission control 和 router 选择 worker, 最后进入 engine。

问题三：SSE streaming 有哪些坑?

回答要点：每个 event 要用 data: ...
结束，要设置 text/event-stream 和禁用缓存，要处理 proxy buffering, 要检测 client disconnect 并 cancel engine request, delta 只发送增量文本，stream 开始后不能再修改 HTTP status。

问题四：LLM API 限流为什么不能只看 QPS?

回答要点：LLM 请求成本差异很大，prompt tokens、max_tokens、并发和模型类型都会影响 GPU 和 KV cache 压力。低 QPS 的超长 prompt 也能打爆系统，所以需要 requests per minute、tokens per minute、concurrent requests、context length、per-model quota 等多维限流。

问题五：rate limit 和 admission control 有什么区别?

回答要点：rate limit 是租户或用户维度，表示用户超过额度或速率，通常返回 429。admission control 是系统容量维度，表示当前 worker、KV cache 或队列无法承载，可能返回 503 或特定 overload 错误。二者都拒绝请求，但含义和处理方式不同。

57.29 标准回答模板

如果面试官问“你会怎么把 LLM engine 包装成 OpenAI-compatible API”，可以这样回答：

我会把 API 层和 engine 层明确解耦。API 层提供 /v1/chat/completions、/v1/completions、/v1/models 等 endpoint，兼容常见字

请求进入 engine 前会经过一条清晰流水线：生成 request_id 和 trace_id，鉴权得到 tenant/user，上下文里检查模型权限，解析和 based rate limit 和 concurrency limit，然后构造内部 EngineRequest，交给 admission control 和 router。engine 内部不直接依

流式响应我会用 SSE，Content-Type 是 text/event-stream，每个 chunk 用 data: JSON 加空行，最后发送 data: [DONE]。第一个 ch

限流上不会只看 QPS，因为 LLM 请求成本差异很大。我会同时做 requests per minute、tokens per minute、concurrent requests、model quota，并区分 rate limit 和系统 admission control。错误响应要有稳定的 type、code、param、message 和 request_id，方

57.30 API Compatibility 公式、错误门禁和可运行 demo

OpenAI-compatible API 层可以先把一次请求抽象成：

$a_i = (k_i, m_i, p_i, o_i, s_i, u_i, c_i, r_i)$

其中 k_i 是 API key, m_i 是 model, p_i 是 prompt tokens, o_i 是 max output tokens, s_i 表示是否 stream, u_i 是 user / tenant, c_i 是 concurrency slot, r_i 是 request id。

上下文长度门禁：

$G_{\{\mathrm{ctx}\}, i} = \mathbf{1}[p_i + o_i \leq L_{m_i}^{\{\max\}}]$

RPM 和 TPM 限流可以写成：

$R_u^{\{\mathrm{req}\}}(t) \leq R_u^{\{\max\}}$

$R_u^{\{\mathrm{tok}\}}(t) + p_i + o_i \leq T_u^{\{\max\}}$

并发门禁:

$$C_u(t) < C_u^{\{\max\}}$$

Usage 统计:

$$U_i = p_i + y_i$$

其中 y_i 是实际输出 token 数。

最终 API 层门禁:

$$G_{\{\mathrm{api}\}} = G_{\{\mathrm{auth}\}} G_{\{\mathrm{schema}\}} G_{\{\mathrm{model}\}} G_{\{\mathrm{ctx}\}} G_{\{\mathrm{rate}\}} G_{\{\mathrm{st}\}}$$

下面这个 0 依赖 demo 模拟一个最小 OpenAI-compatible Chat Completions API 子集:

```
from dataclasses import dataclass
import json
```

```
@dataclass
```

```
class APITenant:
    tenant_id: str
    api_key: str
    allowed_models: set[str]
    rpm_limit: int
    tpm_limit: int
    concurrency_limit: int
    used_requests: int = 0
    used_tokens: int = 0
    active_requests: int = 0
```

```
@dataclass
```

```
class EngineRequest:
    request_id: str
    tenant_id: str
    model: str
    prompt: str
    prompt_tokens: int
    max_tokens: int
    temperature: float
    stream: bool
```

```
class ToyOpenAICompatibleAPI:
```

```
    def __init__(self):
        self.models = {"tiny-chat": {"max_context_tokens": 128}}
        self.tenants = {
            "sk-ok": APITenant("tenant-a", "sk-ok", {"tiny-chat"}, 10, 220, 2),
            "sk-rate": APITenant("tenant-rate", "sk-rate", {"tiny-chat"}, 1, 220, 2, 1, 0),
            "sk-busy": APITenant("tenant-busy", "sk-busy", {"tiny-chat"}, 10, 220, 1, 0, 0, 1),
            "sk-nomodel": APITenant("tenant-no-model", "sk-nomodel", set(), 10, 220, 2),
        }
        self.supported_fields = {
            "model",
            "messages",
            "max_tokens",
            "temperature",
```

```

        "top_p",
        "stop",
        "stream",
        "stream_options",
        "user",
    }
    self.metrics = {
        "auth_fail_total": 0,
        "bad_request_total": 0,
        "rate_limit_total": 0,
        "admission_reject_total": 0,
        "stream_response_total": 0,
        "non_stream_response_total": 0,
    }
    self.next_id = 1

def new_request_id(self) -> str:
    request_id = f"req-{self.next_id:04d}"
    self.next_id += 1
    return request_id

def error(self, status: int, request_id: str, message: str, err_type: str, code: str, param=None):
    if status == 401:
        self.metrics["auth_fail_total"] += 1
    elif status == 429:
        self.metrics["rate_limit_total"] += 1
    elif status == 503:
        self.metrics["admission_reject_total"] += 1
    else:
        self.metrics["bad_request_total"] += 1
    return status, {
        "error": {
            "message": message,
            "type": err_type,
            "param": param,
            "code": code,
        },
        "request_id": request_id,
    }

def authenticate(self, headers: dict[str, str], request_id: str):
    auth = headers.get("Authorization", "")
    if not auth.startswith("Bearer "):
        return None, self.error(401, request_id, "missing bearer token", "authentication_error", "missing_api_key")
    key = auth.removeprefix("Bearer ").strip()
    tenant = self.tenants.get(key)
    if tenant is None:
        return None, self.error(401, request_id, "invalid API key", "authentication_error", "invalid_api_key")
    return tenant, None

def validate_schema(self, body: dict, request_id: str):
    unsupported = sorted(set(body) - self.supported_fields)
    if unsupported:
        return self.error(400, request_id, "unsupported request field", "invalid_request_error", "unsupported_fields")
    if body.get("model") not in self.models:

```

```

        return self.error(400, request_id, "unknown model", "invalid_request_error", "model_not_found", "model")
    messages = body.get("messages")
    if not isinstance(messages, list) or not messages:
        return self.error(400, request_id, "messages must be a non-empty list", "invalid_request_error", "invalid_messages")
    for index, message in enumerate(messages):
        if message.get("role") not in {"system", "user", "assistant"}:
            return self.error(400, request_id, "unsupported role", "invalid_request_error", "unsupported_role", "role")
        if not isinstance(message.get("content"), str) or not message["content"]:
            return self.error(400, request_id, "content must be a non-empty string", "invalid_request_error", "invalid_content")
    max_tokens = body.get("max_tokens", 16)
    if not isinstance(max_tokens, int) or not 1 <= max_tokens <= 64:
        return self.error(400, request_id, "max_tokens out of supported range", "invalid_request_error", "invalid_max_tokens")
    temperature = body.get("temperature", 1.0)
    if not isinstance(temperature, (int, float)) or not 0 <= temperature <= 2:
        return self.error(400, request_id, "temperature out of range", "invalid_request_error", "invalid_temperature")
    return None

def check_model_permission(self, tenant: APITenant, model: str, request_id: str):
    if model not in tenant.allowed_models:
        return self.error(403, request_id, "model not allowed for tenant", "permission_error", "model_not_allowed")
    return None

def render_chat_prompt(self, messages: list[dict]) -> str:
    parts = []
    for message in messages:
        parts.append(f"<|{message['role']}|>\n{message['content']}\n")
    parts.append("<|assistant|>\n")
    return "".join(parts)

def count_tokens(self, text: str) -> int:
    return max(1, (len(text) + 3) // 4)

def check_rate_limits(self, tenant: APITenant, needed_tokens: int, request_id: str):
    if tenant.used_requests + 1 > tenant.rpm_limit:
        return self.error(429, request_id, "rate limit exceeded", "rate_limit_error", "requests_per_minute")
    if tenant.used_tokens + needed_tokens > tenant.tpm_limit:
        return self.error(429, request_id, "token rate limit exceeded", "rate_limit_error", "tokens_per_minute")
    if tenant.active_requests >= tenant.concurrency_limit:
        return self.error(429, request_id, "concurrency limit exceeded", "rate_limit_error", "concurrent_requests")
    return None

def build_engine_request(self, tenant: APITenant, body: dict, request_id: str):
    prompt = self.render_chat_prompt(body["messages"])
    prompt_tokens = self.count_tokens(prompt)
    max_tokens = body.get("max_tokens", 16)
    model_config = self.models[body["model"]]
    if prompt_tokens + max_tokens > model_config["max_context_tokens"]:
        return None, self.error(400, request_id, "context length exceeded", "invalid_request_error", "context_length_exceeded")
    limit_error = self.check_rate_limits(tenant, prompt_tokens + max_tokens, request_id)
    if limit_error is not None:
        return None, limit_error
    engine_request = EngineRequest(
        request_id=request_id,
        tenant_id=tenant.tenant_id,
        model=body["model"],

```

```

        prompt=prompt,
        prompt_tokens=prompt_tokens,
        max_tokens=max_tokens,
        temperature=float(body.get("temperature", 1.0)),
        stream=bool(body.get("stream", False)),
    )
    return engine_request, None

def fake_engine_tokens(self, max_tokens: int) -> List[str]:
    return ["KV", " cache", " stores", " keys"][0:max_tokens]

def usage(self, engine_request: EngineRequest, completion_tokens: int) -> dict:
    return {
        "prompt_tokens": engine_request.prompt_tokens,
        "completion_tokens": completion_tokens,
        "total_tokens": engine_request.prompt_tokens + completion_tokens,
    }

def non_stream_response(self, engine_request: EngineRequest, tokens: List[str]) -> dict:
    self.metrics["non_stream_response_total"] += 1
    return {
        "id": "chatcmpl-" + engine_request.request_id,
        "object": "chat.completion",
        "created": 1710000000,
        "model": engine_request.model,
        "choices": [
            {
                "index": 0,
                "message": {"role": "assistant", "content": "".join(tokens)},
                "finish_reason": "stop",
            }
        ],
        "usage": self.usage(engine_request, len(tokens)),
        "request_id": engine_request.request_id,
    }

def stream_response(self, engine_request: EngineRequest, tokens: List[str], include_usage: bool) -> List[str]:
    self.metrics["stream_response_total"] += 1
    chunks = []
    base = {
        "id": "chatcmpl-" + engine_request.request_id,
        "object": "chat.completion.chunk",
        "created": 1710000000,
        "model": engine_request.model,
    }
    role_chunk = dict(base)
    role_chunk["choices"] = [{"index": 0, "delta": {"role": "assistant"}, "finish_reason": None}]
    chunks.append("data: " + json.dumps(role_chunk, ensure_ascii=False) + "\n\n")
    for token in tokens:
        token_chunk = dict(base)
        token_chunk["choices"] = [{"index": 0, "delta": {"content": token}, "finish_reason": None}]
        chunks.append("data: " + json.dumps(token_chunk, ensure_ascii=False) + "\n\n")
    finish_chunk = dict(base)
    finish_chunk["choices"] = [{"index": 0, "delta": {}, "finish_reason": "stop"}]
    if include_usage:

```



```

        finish_chunk["usage"] = self.usage(engine_request, len(tokens))
        chunks.append("data: " + json.dumps(finish_chunk, ensure_ascii=False) + "\n\n")
        chunks.append("data: [DONE]\n\n")
    return chunks

```

```

def handle_chat_completions(self, headers: dict[str, str], body: dict):
    request_id = self.new_request_id()
    tenant, auth_error = self.authenticate(headers, request_id)
    if auth_error is not None:
        return auth_error
    schema_error = self.validate_schema(body, request_id)
    if schema_error is not None:
        return schema_error
    permission_error = self.check_model_permission(tenant, body["model"], request_id)
    if permission_error is not None:
        return permission_error
    engine_request, build_error = self.build_engine_request(tenant, body, request_id)
    if build_error is not None:
        return build_error

    tenant.used_requests += 1
    tenant.used_tokens += engine_request.prompt_tokens + engine_request.max_tokens
    tenant.active_requests += 1
    try:
        tokens = self.fake_engine_tokens(engine_request.max_tokens)
        if engine_request.stream:
            include_usage = bool(body.get("stream_options", {}).get("include_usage", False))
            return 200, self.stream_response(engine_request, tokens, include_usage)
        return 200, self.non_stream_response(engine_request, tokens)
    finally:
        tenant.active_requests -= 1

```

```

api = ToyOpenAICompatibleAPI()
headers_ok = {"Authorization": "Bearer sk-ok"}
base_body = {
    "model": "tiny-chat",
    "messages": [
        {"role": "system", "content": "Be concise."},
        {"role": "user", "content": "Explain KV cache."},
    ],
    "max_tokens": 4,
    "temperature": 0.7,
}

```

```

status_ok, non_stream = api.handle_chat_completions(headers_ok, dict(base_body))
stream_body = dict(base_body)
stream_body["stream"] = True
stream_body["stream_options"] = {"include_usage": True}
status_stream, stream_chunks = api.handle_chat_completions(headers_ok, stream_body)
status_auth, auth_error = api.handle_chat_completions({}, dict(base_body))
bad_field = dict(base_body)
bad_field["logprobs"] = True
status_bad_field, bad_field_error = api.handle_chat_completions(headers_ok, bad_field)
status_permission, permission_error = api.handle_chat_completions({"Authorization": "Bearer sk-nomodel"}, dict(base_body))

```

```

long_body = dict(base_body)
long_body["messages"] = [{"role": "user", "content": "x" * 700}]
status_context, context_error = api.handle_chat_completions(headers_ok, long_body)
status_rate, rate_error = api.handle_chat_completions({"Authorization": "Bearer sk-rate"}, dict(base_body))
status_busy, busy_error = api.handle_chat_completions({"Authorization": "Bearer sk-busy"}, dict(base_body))

parsed_stream_objects = []
for chunk in stream_chunks:
    payload = chunk.removeprefix("data: ").strip()
    if payload != "[DONE]":
        parsed_stream_objects.append(json.loads(payload))

summary = {
    "non_stream_status": status_ok,
    "non_stream_content": non_stream["choices"][0]["message"]["content"],
    "non_stream_usage": non_stream["usage"],
    "stream_status": status_stream,
    "stream_chunk_count": len(stream_chunks),
    "stream_first_delta": parsed_stream_objects[0]["choices"][0]["delta"],
    "stream_last_payload": stream_chunks[-1].strip(),
    "stream_usage": parsed_stream_objects[-1].get("usage"),
    "auth_error": (status_auth, auth_error["error"]["code"]),
    "bad_field_error": (status_bad_field, bad_field_error["error"]["code"], bad_field_error["error"]["param"]),
    "permission_error": (status_permission, permission_error["error"]["code"]),
    "context_error": (status_context, context_error["error"]["code"]),
    "rate_error": (status_rate, rate_error["error"]["code"]),
    "busy_error": (status_busy, busy_error["error"]["code"]),
    "metrics": api.metrics,
}

}

gates = {
    "auth_ready": summary["auth_error"] == (401, "missing_api_key"),
    "schema_ready": summary["bad_field_error"] == (400, "unsupported_field", "logprobs"),
    "model_permission_ready": summary["permission_error"] == (403, "model_not_allowed"),
    "context_ready": summary["context_error"] == (400, "context_length_exceeded"),
    "rate_limit_ready": summary["rate_error"] == (429, "requests_per_minute")
    and summary["busy_error"] == (429, "concurrent_requests"),
    "non_stream_ready": summary["non_stream_status"] == 200
    and summary["non_stream_content"] == "KV cache stores keys",
    "stream_ready": summary["stream_status"] == 200
    and summary["stream_first_delta"] == {"role": "assistant"}
    and summary["stream_last_payload"] == "data: [DONE]",
    "usage_ready": summary["stream_usage"] is not None
    and summary["non_stream_usage"]["completion_tokens"] == 4,
    "error_shape_ready": all(
        key in auth_error["error"] for key in ["message", "type", "param", "code"]
    )
    and "request_id" in auth_error,
    "privacy_ready": "sk-" not in json.dumps(summary, ensure_ascii=False),
}

gates["openai_compatible_api_gate"] = all(gates.values())

print("openai_compatible_api_summary=", summary)
print("openai_compatible_api_gates=", gates)

```

一次运行的核心输出类似：

```
openai_compatible_api_summary= {'non_stream_status': 200, 'non_stream_content': 'KV cache stores keys', 'non_stream_usage': 0}
openai_compatible_api_gates= {'auth_ready': True, 'schema_ready': True, 'model_permission_ready': True, 'context_ready': True}
```

这个 demo 证明了几个关键点:

1. API key、schema、model permission、context length 和 rate limit 都在进入 engine 前完成。
2. 不支持字段 logprobs 被明确拒绝, 不会静默忽略。
3. 非流式响应包含 choices、assistant message、finish_reason、usage 和 request id。
4. 流式响应是可解析的 SSE chunk, 第一个 chunk 发送 assistant role, 最后发送 data: [DONE]。
5. 错误响应结构稳定, 包含 message、type、param、code 和 request_id。
6. RPM、并发和 context 约束的错误语义不同, 不要混成一个 overload。
7. 摘要和日志里不能泄露 API key。

57.31 小练习

1. 实现 /v1/models, 返回当前可用模型列表。
2. 定义 ChatCompletionRequest 和 ChatCompletionResponse。
3. 实现 chat messages 字段校验。
4. 使用模型 chat template 渲染 prompt。
5. 把 API request 转成内部 EngineRequest。
6. 实现非流式 chat completion 响应。
7. 实现 SSE streaming chunk 格式。
8. 在第一个 streaming chunk 里发送 assistant role。
9. 在最后发送 finish_reason 和 [DONE]。
10. 实现 client disconnect 后 cancel engine request。
11. 实现 OpenAI-style error response。
12. 增加 bearer token 鉴权。
13. 增加 per-tenant QPS limit。
14. 增加 token-based rate limit。
15. 增加 concurrency slot, 并用 finally 保证释放。
16. 增加 request_id 和 trace_id 到日志。
17. 确保日志不打印 API key 和完整 prompt。
18. 写一个 stream benchmark, 统计 TTFT 和 client disconnect。

57.32 本章总结

serving engine 的内部能力最终要通过 API 层交付给用户。

OpenAI-compatible API 能显著降低接入成本, 因为大量 SDK、框架和工具已经支持它。

但兼容必须诚实。

支持、忽略和不支持的字段都要明确。

chat completions 的核心是把 messages 按模型 chat template 渲染成 prompt, 再 tokenize 后交给 engine。

chat template 的细节会直接影响模型输出质量。

流式响应通常使用 SSE。

每个 chunk 都是 data: JSON 加空行, 最后发送 [DONE]。

streaming 最大的工程点是 client disconnect、proxy buffering、增量 detokenization 和 stream 开始后的错误处理。

错误码、鉴权和限流是产品化 serving 的基础。

错误响应要稳定可解析。

鉴权要在昂贵操作前完成。

限流不能只看 QPS, 还要看 tokens、并发、context length 和模型维度。

rate limit 和 admission control 要区分。

一个成熟 API 层还应该要有 request_id、trace_id、usage、审计日志和隐私保护。

下一章可以继续讨论：如何把 serving 系统部署到生产环境，包括容器镜像、模型加载、健康检查、滚动升级和灰度发布。

第 58 章 生产部署：容器镜像、模型加载、健康检查、滚动升级和灰度发布

第 57 章我们把 inference engine 包装成了 OpenAI-compatible API。

到这里，一个 serving 系统已经具备了对外服务能力。

但能在本地跑通，不等于能在生产环境稳定运行。

生产部署会带来一组新的问题：

1. 镜像怎么构建。
2. CUDA、driver、PyTorch、NCCL 版本怎么匹配。
3. 模型权重怎么分发到节点。
4. 服务启动时什么时候算 ready。
5. 健康检查应该检查什么。
6. GPU 故障、OOM、NCCL hang 怎么处理。
7. 如何滚动升级。
8. 如何灰度发布新模型或新 engine。
9. 如何回滚。
10. 如何避免升级过程打爆线上流量。

很多推理系统的线上事故，不是 scheduler 算法错了，而是部署边界没设计好。

本章讨论如何把一个 LLM serving 系统放进生产环境。

58.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 LLM serving 生产部署的工程边界。

公开资料校准主要参考四类口径：

1. Kubernetes probes 文档对 startup、liveness、readiness 三类 probe 的语义说明，以及 Deployment rolling update 中 maxSurge、maxUnavailable 的公开行为。
2. NVIDIA Container Toolkit、CUDA compatibility 和 NCCL 文档对 GPU runtime、driver、CUDA、容器运行时与多卡通信健康检查的公开边界说明。
3. vLLM production serving、OpenAI-compatible server、distributed serving 和 health endpoint 相关公开文档对推理服务启动、健康检查、模型加载和多 worker 部署的工程抽象说明。
4. 本书第 55 到 57 章对 request router、parallel group、分布式 KV、API compatibility、鉴权和限流边界的教学抽象。

本章不写成 Kubernetes YAML 教程，也不绑定某个云厂商、镜像仓库、GPU 型号、driver 版本、NCCL 环境变量、真实 Helm chart、真实 service mesh、真实 autoscaler、真实灰度平台或真实生产 SLO。我们只保留一个可复盘的部署闭环：

image/runtime compatibility -> model artifact manifest -> staged startup -> startup/live/ready probes -> worker register -> rolling capacity -> drain -> canary routing -> regression gate -> rollback path -> observability

第二轮新增 demo 的验收重点是：镜像、runtime 和模型 manifest 要可校验；worker 只有在模型、tokenizer、parallel group、KV cache 和 warmup 都完成后才 ready；滚动升级期间不能让有效 ready 容量跌破下限；drain 不能接新请求但要处理已有请求；canary 必须有稳定分流、回滚门禁和 revision 级观测。

58.1 本章目标

读完本章，你应该能讲清：

1. LLM serving 镜像和普通 Web 服务镜像有什么不同。
2. 为什么 CUDA、driver、PyTorch、NCCL 版本要一起管理。
3. 模型权重分发有哪些常见方式。
4. cold start 为什么是 LLM serving 的核心部署问题。
5. readiness probe 和 liveness probe 的区别。
6. 为什么服务启动成功不等于模型可服务。
7. 滚动升级时如何 drain 老 worker。
8. 灰度发布时 router 应该如何分流。
9. 如何设计回滚路径。
10. 面试中如何描述一个生产级 LLM serving 部署方案。

本章重点不是 Kubernetes 教程。

重点是理解 LLM inference 在生产部署中的特殊性。

58.2 为什么 LLM serving 部署更难

普通 HTTP 服务部署通常关心：

1. 镜像大小。
2. 配置注入。
3. 端口监听。
4. 健康检查。
5. 日志和 metrics。
6. 扩缩容。

LLM serving 还额外关心：

1. GPU 是否可见。
2. GPU driver 是否兼容。
3. CUDA runtime 是否匹配。
4. PyTorch 是否能加载 CUDA kernel。
5. NCCL 是否能建立通信。
6. 模型权重是否完整。
7. tokenizer、config、chat template 是否匹配。
8. 权重加载需要多久。
9. KV cache 预分配是否成功。
10. warmup 是否完成。
11. 多卡 parallel group 是否全部 ready。
12. 节点之间网络是否满足要求。

也就是说，LLM serving 的 ready 不是进程启动。

真正的 ready 是：

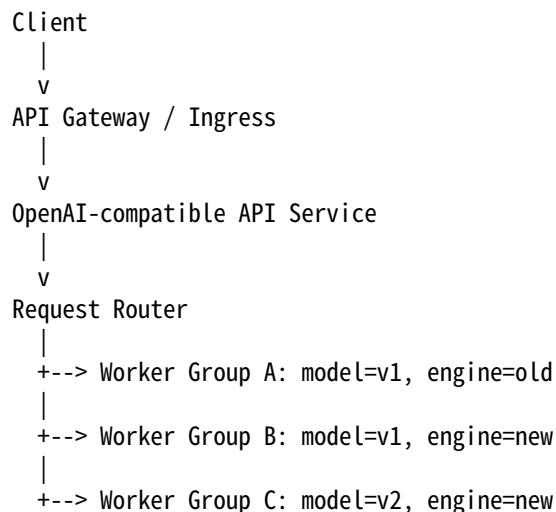
进程已启动

- + GPU 可用
- + runtime 兼容
- + 模型权重加载完成
- + tokenizer/config 加载完成
- + parallel group 初始化完成
- + KV cache 分配完成
- + warmup 通过
- + API 层可以接收请求

少任何一步，都可能导致流量进入后失败。

58.3 生产部署的基本形态

一个典型部署形态是：



在简单系统里，API service、router、worker 可能在同一个进程。

在复杂系统里，它们会拆成多个组件。

生产部署要回答几个问题：

1. 每个组件如何打包。
2. 每个组件如何配置。
3. 每个组件如何发现其他组件。
4. worker 如何注册到 router。
5. router 如何判断 worker 可用。
6. worker 下线前如何处理已有请求。
7. 升级时如何保证新老版本共存。

这些问题比“能不能 forward 一次模型”更接近真实工程。

58.4 镜像构建的核心原则

LLM serving 镜像通常很大。

原因包括：

1. CUDA runtime。
2. cuDNN、NCCL、TensorRT 等 GPU 库。
3. PyTorch 或其他深度学习框架。
4. 自定义 CUDA kernel。
5. tokenizer 相关依赖。
6. Python 包。
7. 可能还包含模型权重。

镜像构建的核心原则是：

1. runtime 版本固定。
2. 依赖版本固定。
3. 构建过程可复现。
4. 镜像职责清晰。
5. 权重是否进镜像要明确取舍。

一个常见分层是：

```
base image:
  cuda runtime
  system libraries

framework image:
  pytorch
  transformers
  tokenizers
  nccl-related dependencies

engine image:
  serving engine code
  custom kernels
  api server

model artifact:
  weights
  config
  tokenizer
  chat template
```

不要把所有东西都混在一个不可解释的镜像里。

否则一旦线上出问题，很难判断是代码变了、依赖变了，还是模型变了。

58.5 CUDA、Driver、PyTorch 和 NCCL

GPU serving 最容易踩的坑之一是版本矩阵。

需要同时考虑：

1. 宿主机 NVIDIA driver。
2. 容器内 CUDA runtime。
3. PyTorch 编译使用的 CUDA 版本。
4. 自定义 kernel 编译使用的 CUDA 版本。
5. NCCL 版本。
6. GPU 架构，比如 A100、H100、L40S。

一个典型错误是：

本地 A100 + CUDA 11.8 跑通
线上 H100 + CUDA 12.x 失败

或者：

单卡跑通
多卡 NCCL 初始化 hang

生产环境应该维护一张明确的兼容矩阵：

GPU: A100
Driver: 535.xx
Container CUDA: 12.1
PyTorch: 2.x cu121
NCCL: 2.x
Engine: commit abc123
Kernel build: sm80

GPU: H100
Driver: 550.xx

Container CUDA: 12.4
PyTorch: 2.x cu124
NCCL: 2.x
Engine: commit def456
Kernel build: sm90

不要依赖 “应该兼容”。

要用启动自检和 smoke test 证明兼容。

58.6 镜像里要不要放模型权重

这是一个重要设计选择。

方案一：权重放进镜像。

优点：

1. 镜像即完整交付物。
2. 部署时不需要额外下载模型。
3. 版本对应关系清晰。
4. 启动路径更少依赖外部系统。

缺点：

1. 镜像非常大。
2. 构建和推送很慢。
3. 每次模型更新都要重建镜像。
4. 镜像仓库存储压力大。
5. 多模型场景不灵活。

方案二：权重不放进镜像，启动时从对象存储或共享文件系统加载。

优点：

1. 镜像较小。
2. 代码和模型可以独立发布。
3. 多模型管理更灵活。
4. 节点本地缓存可以复用。

缺点：

1. 首次启动慢。
2. 依赖对象存储或文件系统可用性。
3. 权重完整性校验更重要。
4. 需要处理并发下载和缓存淘汰。

方案三：节点预热，把权重提前同步到本地盘。

优点：

1. worker 启动快。
2. 不需要每个 pod 重复下载。
3. 适合大模型和固定模型池。

缺点：

1. 节点运维复杂。
2. 本地盘容量管理复杂。
3. 模型版本清理需要额外机制。

生产中经常采用方案二或三。

模型权重太大时，直接放进镜像通常会拖慢整个发布链路。

58.7 模型目录必须版本化

模型不是只有 .safetensors 或 .bin。

一个可服务模型至少包括：

1. 权重文件。
2. config.json。
3. tokenizer 文件。
4. tokenizer config。
5. generation config。
6. chat template。
7. special tokens 配置。
8. quantization config。
9. tensor parallel 切分配置。

生产部署时必须把这些东西作为一个整体版本管理。

错误做法是：

权重来自 model-v2

tokenizer 来自 model-v1

chat template 是手写旧版本

这种错误不一定会导致服务启动失败。

更危险的是它可能只导致输出质量下降。

所以模型目录应该有 manifest：

```
{
  "model_id": "chat-model-v2",
  "revision": "2026-05-31-001",
  "base_model": "example/base-model",
  "weight_format": "safetensors",
  "dtype": "bfloat16",
  "tensor_parallel_size": 4,
  "tokenizer_hash": "...",
  "config_hash": "...",
  "chat_template_hash": "...",
  "weights_hash": "..."
}
```

启动时至少要校验：

1. manifest 存在。
2. 必要文件存在。
3. hash 匹配。
4. dtype 和 engine 配置匹配。
5. tensor parallel size 和部署配置匹配。
6. tokenizer 能正常 encode/decode。
7. chat template 能渲染一个最小 messages。

58.8 启动流程应该分阶段

不要让 worker 一启动就对外 ready。

一个合理启动流程是：

1. 读取配置
2. 检查 GPU 可见性
3. 检查 CUDA / torch / NCCL 基本可用性

4. 拉取或定位模型权重
5. 校验模型 manifest 和文件 hash
6. 加载 tokenizer / config / chat template
7. 加载模型权重到 CPU 或 GPU
8. 初始化 tensor parallel / pipeline parallel group
9. 初始化 KV cache block manager
10. 执行 warmup prefill
11. 执行 warmup decode
12. 注册到 router
13. readiness probe 返回成功

其中任何一步失败，都不应该让流量进入。

这和普通 Web 服务非常不同。

普通服务可能监听端口后就 ready。

LLM worker 必须等模型真的可用。

58.9 Cold Start 为什么重要

LLM worker cold start 可能非常慢。

原因包括：

1. 下载几十 GB 到几百 GB 的权重。
2. 从磁盘读取权重。
3. 权重搬到 GPU。
4. 初始化 CUDA context。
5. 编译或加载 custom kernel。
6. 初始化 NCCL communicator。
7. 预分配 KV cache。
8. warmup kernel。

普通服务扩容可能几十秒。

LLM serving 扩容可能几分钟甚至更久。

这会影响：

1. 自动扩缩容策略。
2. 滚动升级速度。
3. 故障恢复时间。
4. 峰值流量承载能力。

所以 LLM serving 不能只依赖 “流量来了再扩容”。

需要提前预热容量。

58.10 Warmup 要做什么

warmup 不是形式主义。

它可以提前暴露很多问题。

最小 warmup 包括：

1. tokenizer encode 一段最小 prompt。
2. chat template 渲染一段 messages。
3. 执行一次短 prefill。
4. 执行几步 decode。
5. 验证输出 token 合法。
6. 验证 streaming 路径可用。

7. 验证 usage 统计不崩。

多 GPU worker 还要验证：

1. all-reduce 可用。
2. all-gather 可用。
3. NCCL communicator 没有 hang。
4. rank 配置正确。
5. parallel group 中所有 rank 都完成 warmup。

一个 warmup 失败的 worker 不应该注册到 router。

如果它已经注册，则必须立刻摘除。

58.11 Health Check 分层

健康检查通常至少分三类：

1. liveness。
2. readiness。
3. startup。

58.11.1 Liveness

liveness 表示进程是否还活着。

例如：

GET /health/live

它可以检查：

1. event loop 是否响应。
2. 主进程是否未进入死锁。
3. HTTP server 是否还能处理请求。

liveness 不应该做很重的模型 forward。

否则健康检查本身会消耗 GPU。

如果 liveness 失败，平台通常会重启进程。

58.11.2 Readiness

readiness 表示是否可以接收业务流量。

例如：

GET /health/ready

readiness 应该检查：

1. 模型已加载。
2. tokenizer 已加载。
3. KV cache 已初始化。
4. warmup 已完成。
5. worker 已连接 router。
6. 当前不在 drain 状态。
7. GPU 没有进入不可恢复错误状态。

如果 readiness 失败，平台应该停止给它转发新流量。

但不一定要重启。

58.11.3 Startup Probe

startup probe 用来给慢启动服务更多时间。

LLM worker 加载模型可能需要很久。

如果只用 liveness，平台可能误以为它卡死，然后不断重启。

这会形成循环：

加载模型 3 分钟

平台 1 分钟未通过健康检查

重启

再次加载模型

再次重启

startup probe 的作用是告诉平台：

启动期间请耐心等待，不要过早杀掉进程。

58.12 Readiness 不是简单返回 200

readiness 的核心是状态机。

可以定义 worker 状态：

STARTING

DOWNLOADING_MODEL

LOADING_MODEL

INITIALIZING_PARALLEL_GROUP

WARMING_UP

READY

DRAINING

UNHEALTHY

STOPPING

只有 READY 可以接收新请求。

DRAINING 表示：

不接收新请求，但继续处理已有请求。

UNHEALTHY 表示：

不接收新请求，并且可能需要重启或人工介入。

router 不应该只看 HTTP 200。

更好的方式是 worker 周期性上报：

1. state。
2. model id。
3. model revision。
4. engine version。
5. parallel group id。
6. available KV blocks。
7. running requests。
8. queue length。
9. recent error rate。
10. last heartbeat time。

58.13 Worker 注册和摘除

worker 启动完成后，需要注册到 router。

注册信息可以包括：

```
{
  "worker_id": "worker-a-001",
  "model_id": "chat-model-v1",
  "model_revision": "2026-05-31-001",
  "engine_version": "abc123",
  "endpoint": "10.0.1.23:9000",
  "tensor_parallel_size": 4,
  "max_context_length": 32768,
  "supported_features": ["stream", "json_mode"],
  "state": "READY"
}
```

router 收到注册后，不应该立刻永久相信它。

还要持续依赖 heartbeat。

如果 heartbeat 超时，router 应该把 worker 标记为不可用。

摘除 worker 有两种：

1. 非优雅摘除。
2. 优雅 drain。

非优雅摘除用于 crash、GPU error、节点失联。

优雅 drain 用于发布、扩容、维护。

58.14 Drain 的正确语义

drain 是生产部署里非常重要的概念。

它的语义是：

停止接收新请求，等待已有请求完成或超时，然后退出。

LLM serving 的 drain 比普通服务更难。

原因是生成请求可能持续很久。

特别是：

1. stream 请求。
2. 长上下文请求。
3. max_tokens 很大的请求。
4. agent 多轮调用。

一个 worker 进入 drain 后：

1. readiness 应该变为失败。
2. router 不再分配新请求。
3. 已有请求继续运行。
4. 新到达该 worker 的请求返回 retryable error 或被 router 重试。
5. 超过 drain timeout 后，剩余请求被 cancel。
6. 释放 KV cache 和并发 slot。
7. worker 安全退出。

如果没有 drain，滚动升级会直接中断线上 stream。

用户体验会很差。

58.15 滚动升级的基本流程

滚动升级目标是：

逐步替换老版本 worker，同时保持服务可用。

一个基本流程：

1. 构建新镜像
2. 部署少量新 worker
3. 新 worker 启动、加载模型、warmup
4. 新 worker 通过 readiness
5. router 开始给新 worker 少量流量
6. 观察 metrics 和 error rate
7. 老 worker 进入 drain
8. 老 worker 处理完已有请求后退出
9. 继续替换下一批 worker
10. 全部替换完成

关键点是：

1. 新 worker ready 后才减少老 worker。
2. 老 worker drain 完成前不要强杀。
3. 任何阶段发现异常都可以暂停。
4. 保留足够冗余容量。

LLM serving 升级尤其要注意容量。

如果同时启动新 worker 和保留老 worker，GPU 资源可能不够。

如果先杀老 worker 再启动新 worker，容量会下降。

所以滚动升级需要和资源调度一起设计。

58.16 Max Surge 和 Max Unavailable

在 Kubernetes 里，滚动升级常见参数是：

1. max surge。
2. max unavailable。

含义可以简化理解为：

max surge：升级过程中允许额外创建多少新实例

max unavailable：升级过程中允许多少老实例不可用

对普通服务，可以设置较大的 max surge。

但对 GPU worker，资源通常很紧。

你可能没有额外 GPU 创建新 worker。

这时升级策略要更谨慎。

常见选择：

1. 预留一部分 GPU 作为升级缓冲。
2. 在低峰期升级。
3. 先缩小部分低优先级模型副本。
4. 使用独立灰度集群。
5. 按节点逐批替换。

不要假设 GPU 集群像 CPU Web 服务一样可以随时 surge。

58.17 灰度发布是什么

灰度发布是让一小部分流量先使用新版本。

新版本可能是：

1. 新模型权重。
2. 新 tokenizer。
3. 新 chat template。
4. 新推理 engine。
5. 新 scheduler 参数。
6. 新 CUDA kernel。
7. 新 API 行为。

灰度发布不是简单 “部署一个新 pod”。

关键是 router 要能按策略分流。

例如：

```
model=chat-model-v1:
  old revision: 95%
  new revision: 5%
```

或者：

```
tenant=A: 100% new
tenant=B: 0% new
internal users: 100% new
public users: 1% new
```

灰度的本质是把风险控制在可观测、可回滚的范围内。

58.18 灰度维度

常见灰度维度包括：

1. 按请求比例。
2. 按租户。
3. 按 API key。
4. 按用户 id hash。
5. 按模型名 alias。
6. 按 region。
7. 按 endpoint。
8. 按请求类型。

按比例灰度简单，但可能不稳定。

同一个用户的请求有时走新版本，有时走旧版本，会导致体验不一致。

按 user hash 灰度更稳定：

```
hash(user_id) % 100 < 5 -> new version
else -> old version
```

按租户灰度适合企业客户。

按模型 alias 灰度适合模型发布：

```
chat-latest -> chat-v2 for 5% traffic
chat-latest -> chat-v1 for 95% traffic
```

58.19 灰度时要观察什么

灰度不是只看服务有没有 500。

LLM serving 要观察：

1. HTTP error rate。
2. engine error rate。
3. timeout rate。
4. TTFT。
5. TPOT。
6. output tokens per second。
7. GPU utilization。
8. GPU memory usage。
9. KV cache block usage。
10. preemption 次数。
11. OOM 次数。
12. NCCL error。
13. client disconnect rate。
14. 平均输出长度。
15. finish_reason 分布。
16. 内容安全拦截率。
17. 质量评测分数。
18. 用户反馈。

只看技术指标不够。

新模型可能没有报错，但回答质量下降。

所以模型灰度还应该接入离线评测和线上采样评测。

58.20 回滚必须提前设计

回滚不是事故发生后临时想办法。

上线前就要回答：

1. 如何把 router 流量切回旧版本。
2. 旧版本 worker 是否还保留。
3. 旧模型权重是否还在节点本地缓存。
4. 旧镜像是否还可拉取。
5. 配置是否可以回滚。
6. 数据结构是否兼容。
7. API 行为是否兼容。

对 serving 系统来说，最常见的回滚是流量回滚：

```
new version weight: 5% -> 0%
old version weight: 95% -> 100%
```

这比重新部署旧版本更快。

所以灰度期间不要太早删除旧 worker。

如果旧 worker 已经全部下线，回滚就变成重新冷启动。

冷启动可能需要几分钟，事故窗口会被拉长。

58.21 配置发布也要灰度

很多事故不是代码导致的，而是配置导致的。

LLM serving 常见配置包括：

1. max batch size。
2. max num batched tokens。
3. max context length。
4. GPU memory utilization。
5. block size。
6. scheduler policy。
7. preemption mode。
8. prefix cache 开关。
9. stream flush interval。
10. timeout。
11. rate limit。
12. tenant quota。

这些配置改错，会直接影响容量和稳定性。

例如：

`max_num_batched_tokens` 从 8192 改到 65536

可能导致吞吐提升，也可能导致 TTFT 飙升或 OOM。

配置发布应该具备：

1. 版本号。
2. 审计记录。
3. 灰度范围。
4. 自动校验。
5. 一键回滚。
6. 与 metrics 联动的告警。

不要把配置当成无风险修改。

58.22 模型加载失败怎么处理

模型加载失败很常见。

原因包括：

1. 权重文件缺失。
2. 文件 hash 不匹配。
3. tokenizer 文件缺失。
4. dtype 不支持。
5. GPU 显存不足。
6. tensor parallel size 不匹配。
7. quantization kernel 不支持当前 GPU。
8. safetensors 文件损坏。
9. 对象存储超时。
10. 本地磁盘空间不足。

处理原则：

1. 启动失败时不要 ready。
2. 错误日志要明确具体文件和原因。
3. 不要反复高速重启打爆对象存储。
4. 对可重试错误做 backoff。
5. 对不可重试错误快速失败。
6. 暴露启动阶段 metrics。

例如对象存储超时可以重试。

但 manifest hash 不匹配不应该无限重试。

58.23 GPU OOM 的生产处理

GPU OOM 分两类：

1. 启动期 OOM。
2. 运行期 OOM。

启动期 OOM 通常说明：

1. 模型太大。
2. dtype 配错。
3. tensor parallel size 太小。
4. GPU memory utilization 配太高。
5. 预分配 KV cache 过大。

运行期 OOM 通常说明：

1. admission control 不严。
2. max context length 太大。
3. max batch tokens 太大。
4. KV block manager 有泄漏。
5. preemption 或 eviction 失效。
6. 某些请求绕过了限流。

生产处理原则：

1. 尽量在 admission 阶段拒绝超容量请求。
2. OOM 后把 worker 标记 unhealthy。
3. 记录触发 OOM 前的 batch 和请求摘要。
4. 不要记录完整 prompt。
5. 必要时重启 worker。
6. 调低相关配置后再恢复流量。

GPU OOM 后进程不一定还能处于可靠状态。

不要假设 catch 到异常就万事大吉。

58.24 NCCL Hang 怎么办

多 GPU serving 中，NCCL hang 是很麻烦的问题。

它可能表现为：

1. 请求一直不返回。
2. GPU utilization 异常。
3. 某个 rank 卡住。
4. 健康检查还能返回，但模型 forward 卡住。

常见原因：

1. rank 配置错误。
2. 某个 rank 进程崩溃。
3. 网络异常。
4. NCCL 版本和 driver 不匹配。
5. collective 调用顺序不一致。
6. 某个请求路径只在部分 rank 执行。

生产上需要：

1. forward 超时。
2. rank 级 heartbeat。
3. NCCL error 日志。
4. hang detector。

5. worker group 整体摘除。
6. 必要时重启整个 parallel group。

对于 tensor parallel group，不要只重启一个 rank。

通常要把整个 group 当成一个 failure domain。

58.25 日志、Metrics 和 Trace

生产部署必须让系统可观测。

日志要回答：

1. 哪个请求失败。
2. 失败在哪个阶段。
3. 进入了哪个 worker。
4. 使用哪个模型版本。
5. 是否 stream。
6. 是否被 cancel。
7. 是否触发 rate limit 或 admission control。

metrics 要回答：

1. 系统现在是否健康。
2. 容量是否足够。
3. 延迟是否变差。
4. GPU 是否成为瓶颈。
5. KV cache 是否紧张。
6. 新版本是否比旧版本差。

trace 要回答：

1. 请求在 API 层花了多久。
2. router 排队多久。
3. prefill 花了多久。
4. decode 每步花了多久。
5. stream flush 是否及时。
6. 哪个 worker 或 rank 慢。

没有这些信息，灰度和回滚只能靠猜。

58.26 生产告警

告警不要只配 GPU utilization。

LLM serving 常见告警包括：

1. 5xx error rate 高。
2. 429 突增。
3. 503 overload 突增。
4. TTFT P95/P99 升高。
5. TPOT P95/P99 升高。
6. queue waiting time 升高。
7. running requests 接近上限。
8. KV cache free blocks 过低。
9. GPU memory 接近上限。
10. worker heartbeat 超时。
11. readiness 失败数量过多。
12. model load failure。
13. NCCL error。
14. OOM。

15. client disconnect rate 突增。

告警应该带上维度：

1. model id。
2. model revision。
3. engine version。
4. tenant。
5. region。
6. worker group。

否则你只知道 “系统坏了”，不知道 “哪个版本坏了”。

58.27 自动扩缩容的难点

普通服务可以按 CPU 或 QPS 扩缩容。

LLM serving 更复杂。

因为请求成本差异很大。

扩容指标可以包括：

1. queue waiting time。
2. TTFT。
3. tokens per second。
4. running requests。
5. pending requests。
6. KV cache pressure。
7. GPU utilization。
8. admission rejection rate。

但即使触发扩容，新 worker 也需要 cold start。

所以自动扩容适合处理缓慢变化的流量。

对于突发峰值，更依赖：

1. 预留容量。
2. 限流。
3. 降级。
4. 请求排队。
5. 按租户优先级调度。

自动缩容也要谨慎。

缩容前必须 drain。

而且要考虑模型权重缓存是否会被清理。

58.28 多模型部署

生产环境通常不只一个模型。

可能有：

1. chat 模型。
2. code 模型。
3. embedding 模型。
4. rerank 模型。
5. small 模型。
6. large 模型。
7. 多个 revision。

多模型部署有几种方式：

1. 每个 worker 固定一个模型。
2. 一个 worker 支持多个小模型。
3. 动态加载和卸载模型。
4. 按模型池分集群。

大模型 serving 通常采用固定模型 worker。

原因是模型加载太慢，动态切换成本太高。

小模型或 embedding 模型可以考虑多模型共享 worker。

router 必须知道每个 worker 支持哪些模型和 revision。

不要把请求发给不支持该模型的 worker。

58.29 Blue-Green 部署

Blue-Green 是另一种发布方式。

假设当前生产环境是 Blue。

新版本部署到 Green。

流程是：

1. Blue 承接 100% 流量
2. Green 完整部署并 warmup
3. 对 Green 做内部验证
4. router 或 gateway 把流量切到 Green
5. 观察一段时间
6. 如果有问题，切回 Blue

优点：

1. 回滚很快。
2. 新环境可以完整验证。
3. 新老版本隔离清晰。

缺点：

1. 需要接近双倍资源。
2. GPU 成本高。
3. 大模型权重需要两套容量。

对于关键模型或重大版本升级，Blue-Green 很有价值。

对于资源紧张的常规升级，滚动升级和灰度更常见。

58.30 Canary 部署

Canary 更强调小流量试探。

流程是：

1. 部署少量 canary worker
2. 路由 1% 流量到 canary
3. 对比 canary 和 stable 指标
4. 指标正常后扩大到 5%、10%、25%、50%
5. 最终全量

Canary 需要强依赖指标对比。

对 LLM serving 来说，最好同时比较：

1. 技术指标。
2. 生成长度分布。
3. 错误分布。
4. 安全策略触发率。
5. 抽样质量评测。

如果只比较 HTTP 500, canary 的价值会大幅下降。

58.31 模型 Alias 和版本路由

生产 API 通常不希望用户直接绑定内部 revision。

可以提供 alias:

```
chat-latest -> chat-v2-20260531
chat-stable -> chat-v1-20260401
chat-fast -> chat-small-v3
```

router 根据 alias 解析到具体 revision。

灰度时 alias 可以映射到多个 revision:

```
chat-latest:
  chat-v1-20260401: 90%
  chat-v2-20260531: 10%
```

这样用户请求仍然是:

```
{
  "model": "chat-latest",
  "messages": [...]}
}
```

内部可以逐步迁移。

注意:

1. 日志必须记录 alias 和 resolved revision。
2. usage 计费要基于实际 revision 或产品策略。
3. 回放和 debug 时必须知道当时解析到哪个版本。

58.32 部署前检查清单

每次发布前可以检查:

1. 镜像 digest 是否固定。
2. 代码 commit 是否明确。
3. CUDA/PyTorch/NCCL 版本是否在兼容矩阵内。
4. 模型 manifest 是否完整。
5. 权重 hash 是否校验通过。
6. tokenizer 和 chat template 是否匹配。
7. warmup case 是否通过。
8. benchmark 是否没有明显退化。
9. 配置 diff 是否审查。
10. 灰度策略是否设置。
11. 回滚策略是否确认。
12. 告警是否启用。
13. dashboard 是否能区分新老版本。
14. drain timeout 是否合理。
15. 资源容量是否足够。

这类清单很朴素, 但能避免大量事故。

58.33 一个最小生产部署配置

可以想象一个简化配置：

```
model:
  id: chat-model
  revision: 2026-05-31-001
  path: s3://models/chat-model/2026-05-31-001
  tensor_parallel_size: 4
  dtype: bfloat16

engine:
  max_context_length: 32768
  max_num_batched_tokens: 8192
  gpu_memory_utilization: 0.9
  enable_prefix_cache: true

deployment:
  replicas: 4
  startup_timeout_seconds: 1200
  drain_timeout_seconds: 300
  readiness:
    require_warmup: true
    require_router_registered: true

traffic:
  alias: chat-latest
  canary_weight: 0
```

真实系统会更复杂。

但核心维度基本都在这里：

1. 模型版本。
2. engine 参数。
3. 部署生命周期。
4. 流量策略。

58.34 最小 Worker 状态机

一个 worker 可以用这样的状态机管理生命周期：

```
STARTING
-> DOWNLOADING_MODEL
-> LOADING_MODEL
-> INITIALIZING
-> WARMING_UP
-> READY
-> DRAINING
-> STOPPING
```

失败路径：

```
any state -> UNHEALTHY
```

readiness 逻辑：

```
if state == READY:
    return 200
else:
```

```
return 503
```

router 分流逻辑:

```
eligible_workers = workers.filter(  
    state == READY  
    and model_id matches  
    and revision allowed by traffic policy  
    and capacity available  
)
```

这个状态机比简单的布尔变量更可靠。

58.35 常见错误

错误一：进程启动就 readiness 返回成功。

结果：流量进入后发现模型还没加载完。

错误二：没有 startup probe。

结果：大模型加载时被平台反复重启。

错误三：没有 drain。

结果：滚动升级直接中断线上 stream 请求。

错误四：新 worker 没 ready 就杀老 worker。

结果：升级过程中容量骤降，TTFT 和 503 飙升。

错误五：模型权重、tokenizer、chat template 没有整体版本化。

结果：服务能跑，但输出质量异常。

错误六：灰度只看 500。

结果：质量退化和延迟退化没有及时发现。

错误七：旧 worker 过早删除。

结果：发现问题后只能重新 cold start 回滚。

错误八：多卡 worker 只重启单个 rank。

结果：parallel group 状态不一致，继续 hang 或错误。

错误九：配置直接全量发布。

结果：一个 batch 参数改错导致全站 OOM。

错误十：日志没有记录模型 revision。

结果：线上问题无法判断是哪个模型版本造成的。

58.36 面试高频问题

问题一：LLM serving 的 readiness 应该检查什么？

回答要点：不能只检查进程和端口。应该检查模型权重加载完成、tokenizer/config/chat template 加载完成、GPU 和 CUDA 可用、parallel group 初始化完成、KV cache 初始化完成、warmup 通过、worker 已注册到 router，并且当前不处于 drain 或 unhealthy 状态。

问题二：为什么 LLM worker 需要 startup probe？

回答要点：模型加载和 warmup 可能需要几分钟。如果只有 liveness probe，平台可能在启动期间误判服务卡死并反复重启，导致永远无法完成加载。startup probe 用来给慢启动过程足够时间。

问题三：滚动升级 LLM serving 有哪些风险？

回答要点：新 worker cold start 慢，GPU 资源无法随意 surge；如果新 worker 没 ready 就下线老 worker，会造成容量下降；如果没有 drain，会中断已有 stream；如果灰度和回滚没有设计好，新版本异常会影响大量请求。

问题四：模型灰度要观察哪些指标？

回答要点：除了 HTTP error rate，还要看 TTFT、TPOT、timeout、GPU memory、KV cache pressure、preemption、OOM、client disconnect、finish_reason 分布、输出长度分布、安全拦截率，以及离线或线上抽样质量评测。

问题五：模型权重应该放进镜像吗？

回答要点：小模型或强版本绑定场景可以放进镜像，优点是交付物完整；大模型通常不适合，因为镜像过大、构建和分发慢。更常见的是权重对象存储或节点本地缓存，启动时校验 manifest 和 hash，但要处理 cold start、缓存和外部依赖可用性。

问题六：为什么回滚要保留旧 worker？

回答要点：LLM worker cold start 很慢。如果旧 worker 已经全部删除，回滚需要重新拉权重、加载模型、warmup，事故恢复时间会很长。灰度期间保留旧 worker，可以通过 router 快速把流量切回旧版本。

58.37 标准回答模板

如果面试官问“你会如何把一个 LLM serving engine 部署到生产环境”，可以这样回答：

我会先把交付物拆清楚：镜像里固定 CUDA runtime、PyTorch、NCCL、自定义 kernel 和 serving 代码；模型权重、tokenizer、confi

worker 的启动会分阶段：读取配置、检查 GPU、加载或下载模型、校验 manifest、加载 tokenizer 和权重、初始化 TP/PP group、初

发布上我会支持滚动升级和灰度。新 worker 必须先 ready，再逐步接流量；老 worker 下线前进入 drain，停止接收新请求但继续处理

回滚会提前设计。灰度期间保留旧 worker 和旧模型缓存，如果新版本异常，router 直接把流量权重切回旧 revision，而不是重新 c

58.38 Production Deployment 公式、发布门禁和可运行 demo

把生产部署压缩成公式，可以先看 readiness 门禁：

$G_{\{\mathrm{ready}\}} = G_{\{\mathrm{proc}\}} G_{\{\mathrm{gpu}\}} G_{\{\mathrm{runtime}\}} G_{\{\mathrm{model}\}} G_{\{\mathrm{tok}\}} G_{\{\mathrm{warmup}\}}$
cold start 时间可以粗略拆成：

$T_{\{\mathrm{cold}\}} = T_{\{\mathrm{image}\}} + T_{\{\mathrm{weights}\}} + T_{\{\mathrm{load}\}} + T_{\{\mathrm{init}\}} + T_{\{\mathrm{warm}\}}$

滚动升级时刻 t 的有效容量：

$C_{\{\mathrm{roll}\}}(t) = C_{\{\mathrm{ready,new}\}}(t) + C_{\{\mathrm{ready,old}\}}(t) - C_{\{\mathrm{drain}\}}(t)$

容量门禁：

$C_{\{\mathrm{roll}\}}(t) \geq C_{\{\min\}}$

canary 流量比例：

$R_{\{\mathrm{canary}\}} = \frac{N_{\{\mathrm{canary}\}}}{\max(1, N_{\{\mathrm{total}\}})}$

canary 不触发回滚的门禁：

$G_{\{\mathrm{rollback}\}} = \mathbf{1}[E_{\{\mathrm{canary}\}} \leq E_{\{\max\}}] \mathbf{1}[L_{\{\mathrm{canary}\}} \leq L_{\{\max\}}]$

最终生产部署门禁：

$G_{\{\mathrm{deploy}\}} = G_{\{\mathrm{artifact}\}} G_{\{\mathrm{ready}\}} G_{\{\mathrm{roll}\}} G_{\{\mathrm{canary}\}} G_{\{\mathrm{rollback}\}}$

下面这个 0 依赖 demo 模拟一次最小生产发布审计：

```
from dataclasses import dataclass, field
```

```
@dataclass
```

```
class ImageArtifact:
    name: str
    cuda_runtime: str
    pytorch: str
    nccl: str
    min_driver: int
    gpu_arches: set[str]
    health_endpoints: set[str]
```

```
@dataclass
```

```
class ModelArtifact:
    revision: str
    model_hash: str
    tokenizer_hash: str
    config_hash: str
    chat_template_hash: str
    expected_cuda: str
    manifest: dict[str, str]
```

```
@dataclass
```

```
class WorkerState:
    worker_id: str
    revision: str
    capacity: int
    status: str = "STARTING"
    active_requests: int = 0
    accepts_new: bool = False
    registered: bool = False
    phases: dict[str, bool] = field(default_factory=dict)
```

```
@dataclass
```

```
class DeploymentConfig:
    old_revision: str
    new_revision: str
    max_surge: int
    max_unavailable: int
    min_ready_capacity: int
    canary_percent: int
    max_error_rate: float
    max_latency_p95_ms: int
```

```
@dataclass
```

```
class CanaryMetrics:
    total_requests: int
    canary_requests: int
    canary_errors: int
    canary_latency_p95_ms: int
```

```

class ToyProductionDeploymentAuditor:
    def __init__(self, image: ImageArtifact, model: ModelArtifact, config: DeploymentConfig):
        self.image = image
        self.model = model
        self.config = config
        self.router: dict[str, WorkerState] = {}
        self.metrics = {
            "startup_failure_total": 0,
            "readiness_fail_total": 0,
            "drain_total": 0,
            "canary_requests_total": 0,
            "rollback_total": 0,
        }

    def runtime_compatible(self, node_driver: int, gpu_arch: str) -> bool:
        return (
            node_driver >= self.image.min_driver
            and gpu_arch in self.image.gpu_arches
            and self.image.cuda_runtime == self.model.expected_cuda
            and {"/health/live", "/health/ready", "/health/startup"} <= self.image.health_endpoints
        )

    def artifact_ready(self) -> bool:
        required = {
            "model_hash": self.model.model_hash,
            "tokenizer_hash": self.model.tokenizer_hash,
            "config_hash": self.model.config_hash,
            "chat_template_hash": self.model.chat_template_hash,
        }
        return all(value and self.model.manifest.get(key) == value for key, value in required.items())

    def start_worker(self, worker: WorkerState, runtime_ok: bool, artifact_ok: bool) -> WorkerState:
        checks = {
            "process": True,
            "gpu": runtime_ok,
            "runtime": runtime_ok,
            "model": artifact_ok,
            "tokenizer": artifact_ok,
            "parallel_group": runtime_ok,
            "kv_cache": artifact_ok,
            "warmup": runtime_ok and artifact_ok,
        }
        worker.phases = checks
        if all(checks.values()):
            worker.status = "READY"
            worker.accepts_new = True
        else:
            worker.status = "UNHEALTHY"
            self.metrics["startup_failure_total"] += 1
        return worker

    def probes(self, worker: WorkerState) -> dict[str, bool]:
        live = worker.status in {"STARTING", "READY", "DRAINING"}

```

```

startup = bool(worker.phases) and all(worker.phases.values())
ready = worker.status == "READY" and worker.accepts_new and worker.registered
if not ready:
    self.metrics["readiness_fail_total"] += 1
return {"live": live, "startup": startup, "ready": ready}

def register_worker(self, worker: WorkerState):
    if worker.status == "READY":
        worker.registered = True
        self.router[worker.worker_id] = worker

def drain_worker(self, worker: WorkerState) -> List[str]:
    events = [f"{worker.worker_id}:stop_accepting"]
    worker.status = "DRAINING"
    worker.accepts_new = False
    while worker.active_requests > 0:
        worker.active_requests -= 1
        events.append(f"{worker.worker_id}:finished_existing")
    worker.status = "DRAINED"
    self.metrics["drain_total"] += 1
    events.append(f"{worker.worker_id}:drained")
    return events

def rolling_capacity(self, workers: List[WorkerState]) -> Dict[str, int | bool]:
    ready_new = sum(w.capacity for w in workers if w.revision == self.config.new_revision and w.status == "READY")
    ready_old = sum(w.capacity for w in workers if w.revision == self.config.old_revision and w.status in {"READY", "DRAINING"})
    draining = sum(w.capacity for w in workers if w.status == "DRAINING")
    effective = ready_new + ready_old - draining
    return {
        "ready_new": ready_new,
        "ready_old": ready_old,
        "draining": draining,
        "effective": effective,
        "min": self.config.min_ready_capacity,
        "capacity_ok": effective >= self.config.min_ready_capacity,
    }

def route_canary(self, users: List[str]) -> Dict[str, int]:
    counts = {self.config.old_revision: 0, self.config.new_revision: 0}
    for user in users:
        bucket = sum(ord(ch) for ch in user) % 100
        revision = self.config.new_revision if bucket < self.config.canary_percent else self.config.old_revision
        counts[revision] += 1
    self.metrics["canary_requests_total"] = counts[self.config.new_revision]
    return counts

def canary_regression_gate(self, metrics: CanaryMetrics) -> Dict[str, float | int | bool]:
    error_rate = metrics.canary_errors / max(1, metrics.canary_requests)
    canary_ratio = metrics.canary_requests / max(1, metrics.total_requests)
    passed = error_rate <= self.config.max_error_rate and metrics.canary_latency_p95_ms <= self.config.max_latency_p95_ms
    if not passed:
        self.metrics["rollback_total"] += 1
    return {
        "error_rate": round(error_rate, 3),
        "latency_p95_ms": metrics.canary_latency_p95_ms,
    }

```

```

        "canary_ratio": round(canary_ratio, 3),
        "pass": passed,
    }

image = ImageArtifact(
    name="serving:rev-2026-07",
    cuda_runtime="12.4",
    pytorch="2.4",
    nccl="2.20",
    min_driver=550,
    gpu_arches={"sm80", "sm90"},
    health_endpoints={"/health/live", "/health/ready", "/health/startup"},
)
model = ModelArtifact(
    revision="rev-new",
    model_hash="sha256:model",
    tokenizer_hash="sha256:tok",
    config_hash="sha256:cfg",
    chat_template_hash="sha256:tpl",
    expected_cuda="12.4",
    manifest={
        "model_hash": "sha256:model",
        "tokenizer_hash": "sha256:tok",
        "config_hash": "sha256:cfg",
        "chat_template_hash": "sha256:tpl",
    },
)
config = DeploymentConfig(
    old_revision="rev-old",
    new_revision="rev-new",
    max_surge=1,
    max_unavailable=0,
    min_ready_capacity=16,
    canary_percent=20,
    max_error_rate=0.02,
    max_latency_p95_ms=180,
)
auditor = ToyProductionDeploymentAuditor(image, model, config)

runtime_ok = auditor.runtime_compatible(node_driver=555, gpu_arch="sm90")
artifact_ok = auditor.artifact_ready()
new_worker = auditor.start_worker(WorkerState("new-0", "rev-new", capacity=8), runtime_ok, artifact_ok)
auditor.register_worker(new_worker)
probe_report = auditor.probes(new_worker)

old_workers = [
    WorkerState("old-0", "rev-old", capacity=8, status="READY", active_requests=2, accepts_new=True, registered=True),
    WorkerState("old-1", "rev-old", capacity=8, status="READY", active_requests=0, accepts_new=True, registered=True),
]
capacity_before_drain = auditor.rolling_capacity(old_workers + [new_worker])
drain_events = auditor.drain_worker(old_workers[0])
capacity_after_drain = auditor.rolling_capacity(old_workers + [new_worker])

users = [f"tenant-{i:02d}" for i in range(9)] + [f"tenant-{i:02d}" for i in range(10, 17)]

```

```

users += ["tenant-18", "tenant-19", "tenant-27", "tenant-28"]
canary_routes = auditor.route_canary(users)
canary_metrics = CanaryMetrics(total_requests=1000, canary_requests=200, canary_errors=1, canary_latency_p95_ms=142)
canary_report = auditor.canary_regression_gate(canary_metrics)
observability_logs = [
    {"worker": new_worker.worker_id, "revision": new_worker.revision, "event": "ready"},
    {"worker": "old-0", "revision": "rev-old", "event": "drained"},
]

summary = {
    "runtime_compatible": runtime_ok,
    "artifact_ready": artifact_ok,
    "startup_phases": new_worker.phases,
    "probe_report": probe_report,
    "registered_workers": sorted(auditor.router),
    "capacity_before_drain": capacity_before_drain,
    "drain_events": drain_events,
    "capacity_after_drain": capacity_after_drain,
    "canary_routes": canary_routes,
    "canary_report": canary_report,
    "rollback_ready": old_workers[1].status == "READY" and canary_report["pass"],
    "observability_logs": observability_logs,
    "metrics": auditor.metrics,
}

gates = {
    "artifact_gate": summary["artifact_ready"] and summary["runtime_compatible"],
    "readiness_gate": all(summary["startup_phases"].values()) and summary["probe_report"]["ready"],
    "rolling_capacity_gate": summary["capacity_before_drain"]["capacity_ok"]
    and summary["capacity_after_drain"]["capacity_ok"],
    "drain_gate": summary["drain_events"][-1] == "old-0:drained" and old_workers[0].active_requests == 0,
    "canary_gate": summary["canary_routes"]["rev-new"] == 4 and summary["canary_report"]["canary_ratio"] == 0.2,
    "rollback_gate": summary["rollback_ready"],
    "observability_gate": all("revision" in row for row in summary["observability_logs"]),
}
gates["production_deployment_gate"] = all(gates.values())

print("production_deployment_summary=", summary)
print("production_deployment_gates=", gates)

```

一次运行的核心输出类似：

```

production_deployment_summary= {'runtime_compatible': True, 'artifact_ready': True, 'startup_phases': {'process': True,
0'], 'capacity_before_drain': {'ready_new': 8, 'ready_old': 16, 'draining': 0, 'effective': 24, 'min': 16, 'capacity_ok':
0:stop_accepting', 'old-0:finished_existing', 'old-0:finished_existing', 'old-0:drained'], 'capacity_after_drain': {'r
old': 16, 'rev-new': 4}, 'canary_report': {'error_rate': 0.005, 'latency_p95_ms': 142, 'canary_ratio': 0.2, 'pass': True]
0', 'revision': 'rev-new', 'event': 'ready'}, {'worker': 'old-0', 'revision': 'rev-old', 'event': 'drained'}}, 'metrics'
production_deployment_gates= {'artifact_gate': True, 'readiness_gate': True, 'rolling_capacity_gate': True, 'drain_gate

```

这个 demo 证明了几个关键点：

1. 镜像 runtime、GPU driver、GPU arch、health endpoint 和模型 manifest 在启动前就能被审计。
2. readiness 必须晚于模型、tokenizer、parallel group、KV cache 和 warmup。
3. rolling update 的有效容量要显式计算，不能只看新 Pod 是否创建成功。
4. drain 语义是停止接新请求，同时让已有请求自然结束。
5. canary 不是随机上线，而是有稳定分流、指标门禁、旧 revision 保留和可回滚路径。
6. 日志和 metrics 必须带 revision，事故复盘才知道是镜像、模型还是配置造成的问题。

58.39 小练习

1. 为 worker 增加 STARTING、READY、DRAINING、UNHEALTHY 状态。
2. 实现 /health/live 和 /health/ready。
3. 让 readiness 只有在模型加载和 warmup 完成后返回成功。
4. 增加一个最小 warmup 请求，包含 prefill 和 decode。
5. 为模型目录增加 manifest 文件。
6. 启动时校验权重、tokenizer、config 的 hash。
7. 实现 worker 注册到 router。
8. 实现 worker heartbeat。
9. router 在 heartbeat 超时后摘除 worker。
10. 实现 drain：停止接收新请求，等待已有请求完成。
11. 为 stream 请求增加 drain timeout 后 cancel。
12. 实现按 revision 的路由。
13. 实现按 user hash 的 5% canary。
14. 在日志中记录 model alias 和 resolved revision。
15. 增加 TTFT、TPOT、KV cache free blocks 的 metrics。
16. 写一个脚本模拟滚动升级。
17. 写一个脚本模拟新版本 5xx 升高后自动回滚流量。
18. 对比“权重放镜像”和“启动时下载权重”的 cold start 时间。

58.40 本章总结

生产部署是 LLM serving 从实验系统走向可用系统的关键一步。

LLM worker 的部署复杂度远高于普通 Web 服务。

它不仅启动进程，还要保证 GPU runtime、模型权重、tokenizer、parallel group、KV cache 和 warmup 都正确。

镜像构建要固定 CUDA、PyTorch、NCCL 和自定义 kernel 版本。

模型权重、config、tokenizer 和 chat template 必须整体版本化。

readiness 表示能否接收业务流量，不等于进程是否存活。

liveness、readiness 和 startup probe 要分清。

滚动升级必须先让新 worker ready，再 drain 老 worker。

灰度发布要通过 router 控制流量比例，并观察技术指标和质量指标。

回滚路径必须提前设计，最好能通过流量切换快速恢复。

配置发布也要灰度和可回滚，因为 serving 参数会直接影响容量、延迟和稳定性。

到这里，我们已经从单机推理、KV cache、调度、分布式 serving、API 层，一直走到了生产部署。

下一章可以继续讨论：如何做线上容量规划、成本估算、SLO 设计和故障演练。

第 59 章 线上容量规划、成本估算、SLO 设计和故障演练

第 58 章我们讨论了如何把 LLM serving engine 部署到生产环境。

部署成功只是第一步。

真正上线后，团队马上会遇到更现实的问题：

1. 需要多少张 GPU。
2. 高峰期能不能扛住。
3. TTFT 和 TPOT 应该承诺到什么水平。
4. 成本为什么这么高。

5. 要不要为大客户预留容量。
6. 长上下文请求会不会拖垮系统。
7. 某个 worker 掉了以后容量还够不够。
8. 对象存储、镜像仓库、调度系统出问题服务怎么降级。
9. 灰度版本有性能退化时如何发现。
10. 如何向业务、产品和面试官解释容量规划方法。

这些问题已经不只是 engine 内部实现。

它们连接了 serving engine、推理平台、业务流量、成本预算和稳定性工程。

本章讨论 LLM serving 线上容量规划、成本估算、SLO 设计和故障演练。

59.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径，只讲教学版 LLM serving 的容量规划、成本估算、SLO 设计和故障演练框架。

公开资料校准主要参考四类口径：

1. Google SRE Workbook 对 SLO、SLI、error budget、burn rate 和告警策略的公开方法论。
2. vLLM benchmark、metrics、optimization / tuning 文档对 TTFT、TPOT、E2E latency、KV cache usage、preemption、queue time、running / waiting requests 和压测指标的公开口径。
3. Kubernetes resource management 和 deployment 文档对 resource request / limit、副本数、滚动发布和可用容量的通用工程语义。
4. 本书第 53 到 58 章对 benchmark framework、异步 serving、多 worker router、分布式 KV、API 层和生产部署门禁的教学抽象。

本章不提供某个 GPU、某个模型、某个云厂商或某个框架版本的通用容量答案，也不替代真实压测、真实账单、真实 SLO 审批、真实值班制度或真实故障演练平台。我们只保留一个能解释、能复算、能演练的闭环：

workload token profile -> benchmark stable capacity -> GPU count -> KV concurrency -> cost attribution -> SLO and error budget -> admission policy -> fault drill -> rollback and runbook

第二轮新增 demo 的验收重点是：容量不能只按 QPS 算；要同时用 input token/s、output token/s、request rate 和 KV active tokens 估算 worker 数；规划容量要包含利用率、安全余量、N+1 和发布容量；成本要能按 GPU 小时和 token 粗分摊；SLO 要能计算 error budget；故障演练要证明 worker 丢失、依赖不可用、长上下文突增和新版本退化都有可执行动作。

59.1 本章目标

读完本章，你应该能讲清：

1. LLM serving 容量为什么不能只看 QPS。
2. request QPS、input token/s、output token/s 的区别。
3. TTFT、TPOT、E2E latency 和吞吐之间的关系。
4. 如何用 benchmark 结果估算线上 GPU 数量。
5. 为什么上下文长度分布会显著影响容量。
6. KV cache 容量如何限制并发请求数。
7. 成本估算应该按 GPU 小时、token 和租户拆分。
8. SLO 应该如何定义、监控和触发告警。
9. 故障演练应该覆盖哪些场景。
10. 面试中如何回答“如何规划一个 LLM serving 集群容量”。

本章重点不是给出一个万能公式。

重点是建立一套能落地的工程估算框架。

59.2 为什么不能只看 QPS

普通 Web 服务经常用 QPS 做容量规划。

例如：

单实例 1000 QPS

高峰 10000 QPS

需要 10 个实例

LLM serving 不能这样简单估算。

原因是两个请求的成本可能差几个数量级：

请求 A：

input tokens: 32

output tokens: 16

请求 B：

input tokens: 32000

output tokens: 2048

它们都是 1 个请求。

但对 GPU、KV cache、scheduler、网络和队列的压力完全不同。

LLM serving 至少要同时看：

1. request QPS。
2. input token/s。
3. output token/s。
4. prefill token/s。
5. decode token/s。
6. 平均输入长度。
7. P95/P99 输入长度。
8. 平均输出长度。
9. P95/P99 输出长度。
10. 并发请求数。
11. KV cache 占用。
12. TTFT 和 TPOT。

如果只看 QPS，很容易低估长上下文和长输出的成本。

59.3 容量规划的核心单位

LLM serving 的核心资源消耗来自两部分：

1. prefill。
2. decode。

prefill 处理输入 prompt。

decode 逐 token 生成输出。

因此容量规划的基本单位不是 request，而是 token。

可以先建立几个指标：

RPS = 每秒请求数

InputTokensPerRequest = 平均输入 token 数

OutputTokensPerRequest = 平均输出 token 数

InputTokenRate = RPS * InputTokensPerRequest

OutputTokenRate = RPS * OutputTokensPerRequest

例如：

RPS = 20

平均输入 = 1000 tokens

平均输出 = 200 tokens

InputTokenRate = 20 * 1000 = 20000 tokens/s

OutputTokenRate = 20 * 200 = 4000 tokens/s

这比 “20 QPS” 有意义得多。

因为 engine 真正要处理的是：

每秒 20000 个 prompt tokens

每秒 4000 个 generated tokens

59.4 Prefill 和 Decode 的容量不同

prefill 和 decode 的资源画像不同。

prefill 通常更像大矩阵计算。

特点是：

1. 并行度高。
2. 对算力敏感。
3. 输入越长，单次计算越重。
4. 影响 TTFT。

decode 通常逐 token 迭代。

特点是：

1. 每步处理一个或少量 token。
2. 对 batch size、KV cache 读取和调度敏感。
3. 持续占用请求槽位。
4. 影响 TPOT 和整体吞吐。

所以容量规划要分开看：

prefill capacity >= input token rate

decode capacity >= output token rate

KV cache capacity >= active sequence tokens

其中任何一个不满足，服务都会出问题。

prefill 不够时，典型现象是：

1. TTFT 升高。
2. 请求排队时间变长。
3. 新请求迟迟拿不到第一个 token。

decode 不够时，典型现象是：

1. TPOT 升高。
2. streaming 变慢。
3. 已经开始的请求长时间占用 worker。

KV cache 不够时，典型现象是：

1. 可接收并发下降。
2. scheduler 频繁 preempt。
3. OOM 或请求被拒绝。

59.5 从 Benchmark 到容量估算

容量估算必须建立在 benchmark 上。

不能只看模型参数量或 GPU 理论 FLOPS。

一个最小 benchmark 应该固定：

1. 模型。
2. GPU 型号。
3. tensor parallel size。
4. dtype。
5. max context length。
6. scheduler 参数。
7. max_num_batched_tokens。
8. gpu_memory_utilization。
9. prefix cache 是否开启。
10. 输入长度分布。
11. 输出长度分布。
12. 并发水平。

benchmark 输出至少包括：

1. request throughput。
2. input token throughput。
3. output token throughput。
4. TTFT P50/P95/P99。
5. TPOT P50/P95/P99。
6. E2E latency P50/P95/P99。
7. GPU utilization。
8. GPU memory usage。
9. KV cache usage。
10. preemption 次数。
11. OOM 次数。
12. 失败率。

假设某个 4 卡 worker benchmark 结果是：

```
model: chat-32b
gpu: 4 * H100
input length: avg 1000, p95 4000
output length: avg 200, p95 800
```

stable capacity:

```
RPS: 18
input tokens/s: 18000
output tokens/s: 3600
TTFT P95: 900 ms
TPOT P95: 35 ms
```

如果线上高峰预估是：

```
RPS: 90
input tokens/s: 90000
output tokens/s: 18000
```

只按平均吞吐估算：

```
workers = ceil(90 / 18) = 5
```

但这只是理论下限。

生产环境还要加冗余。

59.6 安全余量

线上不能按 100% benchmark 容量运行。

原因包括：

1. 流量有突刺。
2. 输入长度分布会变化。
3. 输出长度不可完全预测。
4. 某些请求会触发长上下文。
5. GPU 性能存在波动。
6. worker 可能重启或下线。
7. 灰度升级会临时占用容量。
8. 故障恢复需要预留空间。
9. 监控和日志也有开销。
10. benchmark 通常比真实业务更干净。

常见做法是只把单 worker 稳定容量的 60% 到 80% 作为规划容量。

例如：

```
benchmark stable RPS = 18
planning utilization = 70%
planning RPS per worker = 18 * 0.7 = 12.6
```

```
peak RPS = 90
workers = ceil(90 / 12.6) = 8
```

如果一个 worker 需要 4 张 GPU：

```
GPU count = 8 * 4 = 32
```

这比只按 benchmark 极限算出来的 20 张 GPU 更接近生产现实。

59.7 N+1 和故障容量

容量规划还要考虑 worker 故障。

如果集群有 8 个 worker，每个 worker 承载 12.5% 流量。

当一个 worker 掉线，剩余 7 个 worker 要承载全部流量。

剩余 worker 的负载会变成：

```
new load per worker = 1 / 7 = 14.3%
relative increase = 14.3 / 12.5 - 1 = 14.4%
```

如果原来已经跑到 85% 利用率，一个 worker 掉线后可能直接过载。

因此要定义故障容量目标。

例如：

```
正常状态：P95 流量下利用率 <= 70%
N-1 状态：损失 1 个 worker 后利用率 <= 85%
N-2 状态：允许降级，但错误率不能失控
```

大规模集群还要考虑机架、可用区或节点池故障。

容量不是只看总 GPU 数。

还要看故障域分布。

59.8 上下文长度分布

LLM serving 的容量经常被长上下文请求打穿。

平均输入长度可能很低：

avg input tokens = 800

但 P99 可能很高：

p99 input tokens = 28000

长 prompt 会带来几类问题：

1. prefill 时间长。
2. TTFT 高。
3. KV cache 占用大。
4. batch packing 更困难。
5. 对其他短请求造成排队影响。

容量规划要按分布而不是平均值。

至少要统计：

1. P50 输入长度。
2. P90 输入长度。
3. P95 输入长度。
4. P99 输入长度。
5. 最大输入长度。
6. P50 输出长度。
7. P90 输出长度。
8. P95 输出长度。
9. P99 输出长度。
10. 最大输出长度。

如果业务同时有短问答和长文档总结，不要把它们混在一个平均数里。

应该分 workload 建模。

59.9 Workload 分层

一个线上模型可能服务多类业务：

workload A: 客服短问答

input avg: 300
output avg: 120
latency sensitive: high

workload B: 代码生成

input avg: 2000
output avg: 800
latency sensitive: medium

workload C: 长文档总结

input avg: 24000
output avg: 1000
latency sensitive: low

如果都进入同一个 worker pool，长文档总结可能拖慢客服短问答。

常见隔离方式包括：

1. 按模型拆 worker pool。

2. 按租户拆 worker pool。
3. 按上下文长度拆 worker pool。
4. 按优先级拆队列。
5. 按 SLA 拆流量入口。
6. 长任务进入异步 batch 或低优先级队列。

容量规划时要分别估算每类 workload。

不要让低优先级长请求吃掉高优先级短请求的 SLO。

59.10 KV Cache 容量估算

KV cache 是 LLM serving 的关键容量约束。

对每个 active sequence，需要保存历史 token 的 key/value。

简化估算可以写成：

KV bytes per token
= $2 * \text{num_layers} * \text{num_kv_heads} * \text{head_dim} * \text{bytes_per_element}$

其中：

1. 2 表示 key 和 value。
2. num_layers 是层数。
3. num_kv_heads 是 KV head 数。
4. head_dim 是每个 head 的维度。
5. bytes_per_element 取决于 fp16、bf16、fp8 等。

假设：

num_layers = 40
num_kv_heads = 8
head_dim = 128
bytes_per_element = 2

则：

KV bytes per token = $2 * 40 * 8 * 128 * 2 = 163840$ bytes
 ≈ 160 KB/token

如果一个请求总上下文长度是：

input 4000 + generated 1000 = 5000 tokens

则这个请求的 KV cache 约为：

$5000 * 160 \text{ KB} \approx 800 \text{ MB}$

这只是示意，真实模型还要考虑 tensor parallel、GQA/MQA、block size、对齐、碎片和实现细节。

但它能帮助你理解：长上下文并发非常贵。

59.11 并发上限不是 QPS 上限

decode 阶段请求会持续停留在系统里。

因此并发上限同样重要。

可以用 Little's Law 做粗略估计：

$\text{Concurrency} = \text{ArrivalRate} * \text{AverageLatency}$

例如：

RPS = 20

平均端到端延迟 = 10s

$\text{active requests} \approx 20 * 10 = 200$

如果每个 active request 平均占用 3000 tokens KV cache:

$\text{active tokens} = 200 * 3000 = 600000 \text{ tokens}$

这会直接决定 KV cache 需要多少显存。

当输出变长时，即使 RPS 不变，并发也会上升。

因此 capacity dashboard 必须同时展示：

1. waiting requests。
2. running requests。
3. active sequences。
4. active tokens。
5. KV cache allocated blocks。
6. KV cache free blocks。
7. preempted sequences。

59.12 Admission Control

容量规划不只是买更多 GPU。

还要决定什么时候拒绝请求。

如果系统已经接近极限，继续接收新请求可能导致：

1. 所有请求延迟变差。
2. KV cache 被打满。
3. preemption 频繁发生。
4. worker OOM。
5. 大面积 5xx。

更好的做法是 admission control。

也就是在入口处判断请求是否能进入系统。

常见判断条件包括：

1. 当前队列长度。
2. 预计输入 token 数。
3. max_tokens。
4. 当前 KV cache 空闲量。
5. 当前 running request 数。
6. 租户配额。
7. 请求优先级。
8. deadline 或 timeout。

拒绝请求时应该返回明确错误：

```
{
  "error": {
    "type": "capacity_exceeded",
    "message": "The model is temporarily overloaded. Please retry later.",
    "code": "overloaded"
  }
}
```

有控制地拒绝一部分请求，比让所有请求一起超时更好。

59.13 成本估算的基本模型

LLM serving 成本主要来自 GPU。

一个基础成本模型是：

$$\text{MonthlyCost} = \text{GPUCount} * \text{GPUHourlyPrice} * 24 * 30$$

例如：

$$\text{GPUCount} = 32$$

$$\text{GPUHourlyPrice} = \$4$$

$$\text{MonthlyCost} = 32 * 4 * 24 * 30 = \$92160$$

这还只是 GPU 租用成本。

真实成本还包括：

1. CPU 和内存。
2. 本地 NVMe。
3. 对象存储。
4. 镜像仓库。
5. 跨可用区网络。
6. 日志和 metrics。
7. 负载均衡。
8. 空闲冗余容量。
9. 工程维护成本。
10. 失败重试带来的额外 token。

容量规划时不要只算满载 token 成本。

线上系统必须为峰值、故障和发布留空间，这些都会转化为空闲成本。

59.14 Token 成本

业务通常更关心每 1000 token 或每 100 万 token 的成本。

可以从集群视角估算：

$$\text{CostPerOutputToken} = \text{ClusterCostPerSecond} / \text{OutputTokensPerSecond}$$

例如：

$$32 \text{ GPUs}$$

$$\$4 / \text{GPU-hour}$$

$$\text{ClusterCostPerHour} = 32 * 4 = \$128/\text{hour}$$

$$\text{ClusterCostPerSecond} = 128 / 3600 = \$0.0356/\text{s}$$

$$\text{OutputTokensPerSecond} = 18000$$

$$\text{CostPerOutputToken} = 0.0356 / 18000 = \$0.00000198$$

$$\text{CostPerMillionOutputTokens} \approx \$1.98$$

但这个数字容易误导。

因为它假设一直跑在该吞吐下。

如果实际平均利用率只有 35%，成本会变成：

$$\text{EffectiveCost} = \text{IdealCost} / \text{Utilization}$$

$$= 1.98 / 0.35$$

$$\approx \$5.66 \text{ per million output tokens}$$

所以成本优化的核心之一是提升有效利用率。

但利用率不能无限提高。

过高利用率会牺牲 SLO 和故障余量。

59.15 成本归因

多租户 serving 系统需要成本归因。

否则你只知道总账单很高，不知道谁在消耗。

日志中至少要记录：

1. tenant id。
2. user id 或 service id。
3. model id。
4. model revision。
5. input tokens。
6. output tokens。
7. cached input tokens。
8. TTFT。
9. TPOT。
10. E2E latency。
11. worker id。
12. GPU type。
13. finish reason。
14. error code。

成本归因可以按不同口径计算：

按 token 分摊：简单，适合产品计费

按 GPU time 分摊：更接近真实成本

按 reserved capacity 分摊：适合大客户专属容量

按 peak capacity 分摊：适合需要保障峰值的业务

面试中要强调：token 计费简单，但不一定等于资源成本。

长上下文请求的 input token 和 KV cache 占用可能比 output token 更贵。

59.16 SLO、SLA 和 SLI

先区分三个概念：

SLI: Service Level Indicator, 实际观测指标

SLO: Service Level Objective, 内部目标

SLA: Service Level Agreement, 对外承诺

例如：

SLI: /v1/chat/completions 的 TTFT P95

SLO: TTFT P95 < 1.5s, 按 5 分钟窗口统计

SLA: 月度可用性 99.9%, 违反后赔付

工程上通常先建立 SLI, 再设 SLO, 最后才谨慎承诺 SLA。

不要一开始就承诺过细的外部 SLA。

LLM serving 的延迟强依赖：

1. 模型大小。
2. 输入长度。
3. 输出长度。

4. 并发水平。
5. 是否命中 prefix cache。
6. 租户优先级。
7. GPU 型号。
8. 当前故障状态。

因此 SLO 应该带条件。

59.17 LLM Serving 的关键 SLI

常见 SLI 包括：

1. availability。
2. HTTP 5xx rate。
3. model error rate。
4. timeout rate。
5. throttle rate。
6. TTFT P50/P95/P99。
7. TPOT P50/P95/P99。
8. E2E latency P50/P95/P99。
9. request queue time。
10. input token throughput。
11. output token throughput。
12. running requests。
13. waiting requests。
14. KV cache usage。
15. preemption rate。
16. GPU utilization。
17. GPU memory usage。
18. worker restart count。
19. router no-capacity count。
20. client disconnect rate。

对 streaming 接口来说，TTFT 和 TPOT 比单纯 E2E latency 更重要。

因为用户体感通常是：

1. 多久看到第一个 token。
2. 后续 token 是否流畅。

59.18 SLO 示例

一个生产系统可以这样定义 SLO：

Availability:

99.9% requests receive non-5xx response

For short-chat workload:

condition: input_tokens <= 1024 and max_tokens <= 512

TTFT P95 <= 1.5s

TPOT P95 <= 50ms

timeout rate <= 0.5%

For long-context workload:

condition: input_tokens <= 32000 and max_tokens <= 1024

TTFT P95 <= 8s

TPOT P95 <= 80ms

timeout rate <= 1%

Capacity control:

overload rejection returns 429 or 503 with stable error body
no silent timeout due to unlimited queueing

注意这里按 workload 分开定义。

短请求和长请求不应该共用一个延迟目标。

否则 SLO 要么不现实，要么没有约束力。

59.19 Error Budget

SLO 可以转化为 error budget。

例如月可用性 SLO 是 99.9%。

一个 30 天月总时间是：

$30 * 24 * 60 = 43200 \text{ minutes}$

允许不可用时间是：

$43200 * 0.1\% = 43.2 \text{ minutes}$

这就是 error budget。

如果一个月内已经消耗了 30 分钟，发布策略就应该变保守。

例如：

1. 暂停非紧急发布。
2. 降低灰度速度。
3. 增加人工审批。
4. 优先修稳定性问题。
5. 扩容高风险模型池。

error budget 的价值是把“稳定性”和“迭代速度”连接起来。

59.20 告警设计

告警不能只看 HTTP 500。

LLM serving 退化经常先表现为延迟和容量压力。

建议分层告警：

用户可见层：

availability drop
5xx rate increase
timeout rate increase
TTFT P95/P99 increase
TP0T P95/P99 increase

容量层：

queue length high
no available worker
KV cache free blocks low
preemption rate high
GPU memory near limit

执行层：

worker restart

CUDA OOM
NCCL error
model load failure
tokenizer error

依赖层:

object storage error
registry pull failure
metrics pipeline delay
router registry unavailable

告警要避免两个极端:

1. 指标一抖就报警, 导致告警疲劳。
2. 只在完全不可用时报警, 已经太晚。

比较实用的方式是多窗口告警。

例如:

fast burn:

- 5 分钟错误预算消耗过快, 立即报警

slow burn:

- 1 小时错误预算持续消耗, 创建工单或低优告警

59.21 Dashboard 应该展示什么

一个 LLM serving dashboard 至少应该有这些面板:

1. RPS。
2. input tokens/s。
3. output tokens/s。
4. TTFT P50/P95/P99。
5. TPOT P50/P95/P99。
6. E2E latency P50/P95/P99。
7. queue time。
8. running/waiting requests。
9. input/output length distribution。
10. KV cache usage/free blocks。
11. preemption count。
12. GPU utilization。
13. GPU memory usage。
14. error rate by error code。
15. timeout and cancel rate。
16. worker state distribution。
17. model revision distribution。
18. tenant traffic distribution。
19. prefix cache hit rate。
20. cost per model or tenant。

关键是 dashboard 必须能按维度切分:

1. model。
2. revision。
3. tenant。
4. route。
5. worker pool。
6. GPU type。

7. region or zone。

否则总量指标会掩盖局部故障。

59.22 压测策略

上线前压测不能只发固定 prompt。

至少要覆盖：

1. 短输入短输出。
2. 短输入长输出。
3. 长输入短输出。
4. 长输入长输出。
5. 混合输入长度分布。
6. 混合输出长度分布。
7. streaming 和 non-streaming。
8. prefix cache 命中和不命中。
9. 高并发突刺。
10. 慢客户端消费。
11. client cancel。
12. 超过上下文限制的非法请求。
13. 超过 max_tokens 的请求。
14. 多租户混合流量。

压测要回答几个问题：

1. 稳定容量是多少。
2. 拐点在哪里。
3. 过载时系统如何表现。
4. 是否会优雅拒绝。
5. 是否会 OOM。
6. 恢复后是否能回到正常状态。

最重要的不是峰值数字，而是过载行为是否可控。

59.23 线上容量预测流程

一个可落地的容量预测流程是：

1. 收集历史流量。
2. 按模型、租户和 workload 切分。
3. 统计 RPS、input tokens/s、output tokens/s。
4. 统计输入和输出长度分布。
5. 估算高峰流量和增长率。
6. 用真实分布做 benchmark。
7. 得到单 worker 稳定容量。
8. 应用利用率目标和安全余量。
9. 应用 N+1 或故障域冗余。
10. 得到 worker 和 GPU 数。
11. 估算成本。
12. 和预算、SLO、发布策略一起评审。

可以写成一个简单表格：

```
model: chat-32b
workload: short-chat
peak rps: 90
peak input tokens/s: 90000
peak output tokens/s: 18000
```

single worker planning rps: 12.6
worker count: 8
tp size: 4
gpu count: 32
monthly gpu cost: \$92160
normal utilization target: 70%
N-1 utilization target: <= 85%

这个表比一句“需要 32 张卡”更有解释力。

59.24 自动扩缩容的难点

普通服务可以根据 CPU 或 QPS 自动扩缩容。

LLM serving 的自动扩缩容更难。

原因包括：

1. GPU 资源稀缺。
2. worker cold start 很慢。
3. 模型权重下载慢。
4. warmup 需要时间。
5. 扩容后短时间内不一定能接流量。
6. 缩容需要 drain，不能直接杀 stream。
7. 负载由 token 分布决定，不只是 QPS。
8. 突发流量可能比扩容速度快得多。

因此自动扩缩容通常要结合：

1. 预测式扩容。
2. 定时扩容。
3. 按 token throughput 扩容。
4. 按 queue time 扩容。
5. 按 KV cache pressure 扩容。
6. 保留 warm pool。
7. 限流和降级。

不要指望冷启动扩容解决秒级突刺。

秒级突刺主要靠预留容量、队列、限流和降级。

59.25 降级策略

当容量不足时，可以考虑降级。

常见降级方式包括：

1. 降低低优先级租户配额。
2. 拒绝超长上下文请求。
3. 降低 max_tokens 上限。
4. 关闭部分非关键模型。
5. 把低优先级任务切到异步队列。
6. 使用更小模型兜底。
7. 关闭 expensive decoding 选项。
8. 降低并发上限。
9. 对重复请求利用缓存。
10. 对非关键场景返回稍后重试。

降级必须是产品和工程共同定义的。

不能线上出事时临时决定。

每个降级策略都要明确：

1. 触发条件。
2. 影响范围。
3. 用户可见行为。
4. 恢复条件。
5. 负责人。

59.26 故障演练为什么重要

没有演练的故障预案通常不可用。

LLM serving 的故障演练尤其重要，因为很多问题只在 GPU 和长连接场景下暴露。

演练目标不是制造事故。

演练目标是验证：

1. 故障能否被监控发现。
2. 告警是否发给正确的人。
3. runbook 是否可执行。
4. 系统是否能自动隔离故障。
5. 降级是否生效。
6. 回滚是否足够快。
7. 用户影响是否符合预期。
8. 事后是否能够通过日志定位。

如果只写文档不演练，真正故障时通常会发现关键步骤缺失。

59.27 常见故障演练场景

建议覆盖这些场景：

1. 单 worker 进程退出。
2. 单 GPU OOM。
3. 多卡 worker 某个 rank hang。
4. NCCL 初始化失败。
5. 模型权重下载失败。
6. tokenizer 或 config 缺失。
7. readiness 一直不成功。
8. router 和 worker 心跳中断。
9. KV cache 接近耗尽。
10. prefix cache 异常膨胀。
11. 长上下文流量突增。
12. 某租户请求量突增。
13. client 大量取消 stream。
14. 下游网络抖动导致慢客户端。
15. 新模型 revision TTFT 退化。
16. 新 engine 版本 OOM 升高。
17. 对象存储不可用。
18. 镜像仓库不可用。
19. metrics pipeline 延迟。
20. 可用区部分故障。

每个演练都应该有明确预期。

例如：

场景：单 worker 进程退出

预期:

1. liveness 失败。
2. worker 从 router 摘除。
3. 新请求不再路由到该 worker。
4. running stream 失败率受限。
5. 剩余 worker 利用率上升但不超过 N-1 目标。
6. 告警触发。
7. worker 自动重启并完成 warmup 后重新注册。

59.28 Runbook

runbook 是故障处理手册。

一个好的 runbook 不应该只写 “查看日志”。

它应该包含:

1. 故障现象。
2. 可能原因。
3. 影响判断。
4. 查询 dashboard 链接。
5. 关键日志字段。
6. 快速止血步骤。
7. 回滚步骤。
8. 降级步骤。
9. 验证恢复的方法。
10. 升级联系人。

例如 KV cache 压力过高的 runbook 可以写:

现象:

KV cache free blocks 持续低于 10%
preemption rate 升高
TTFT/TPOT 同时退化

可能原因:

长上下文流量增加
输出长度增加
某租户突增
max_num_batched_tokens 或 memory utilization 配置变化

止血:

降低低优先级租户 max_tokens
对超长输入返回 overload 或引导异步处理
扩容 worker pool
如由新版本导致, 切回旧 revision

验证:

KV cache free blocks 回升
preemption rate 下降
TTFT/TPOT 恢复到 SLO 内

runbook 越具体, 故障时越有用。

59.29 发布和容量的关系

发布会消耗容量。

第 58 章讲滚动升级时提到, 新 worker 要先 ready, 老 worker 要 drain。

这意味着发布期间可能同时存在：

1. 旧 worker。
2. 新 worker。
3. draining worker。
4. warmup 中的 worker。

如果 GPU 资源没有 surge 空间，发布过程会挤占线上容量。

容量规划要明确：

normal capacity: 正常服务容量

failure capacity: worker 或节点故障后的容量

release capacity: 滚动升级和灰度期间需要的额外容量

如果没有额外 GPU，可以采用更保守发布：

1. 低峰发布。
2. 小批次替换。
3. 先扩容再发布。
4. 降低灰度速度。
5. 发布期间限制低优先级流量。

发布策略和容量策略必须一起设计。

59.30 常见错误

错误一：只用 QPS 做容量规划。

结果：长上下文和长输出请求打爆 KV cache 和 TTFT。

错误二：用 benchmark 极限吞吐直接算 GPU 数。

结果：线上没有安全余量，轻微波动就过载。

错误三：只看平均输入长度。

结果：P99 长 prompt 导致少量请求拖垮整体延迟。

错误四：忽略 KV cache 容量。

结果：GPU utilization 看起来不高，但系统无法接更多并发。

错误五：SLO 不区分 workload。

结果：短问答和长文档使用同一延迟目标，指标不可解释。

错误六：告警只看 5xx。

结果：TTFT、TPOT 和队列已经严重退化时才发现。

错误七：没有 admission control。

结果：过载时无限排队，最后所有请求一起超时。

错误八：成本只按满载 token 算。

结果：忽略峰值冗余和低利用率，实际账单远高于预期。

错误九：不做故障演练。

结果：预案在真实故障时不可执行。

错误十：发布不考虑容量。

结果：升级过程中 worker drain 和 cold start 导致线上容量骤降。

59.31 面试高频问题

问题一：为什么 LLM serving 容量规划不能只看 QPS？

回答要点：因为不同请求的输入长度、输出长度和上下文长度差异很大，同样是 1 个请求，prefill、decode 和 KV cache 成本可能差几个数量级。应该同时看 request QPS、input token/s、output token/s、输入输出长度分布、并发数、KV cache 占用、TTFT 和 TPOT。

问题二：如何从 benchmark 估算需要多少张 GPU？

回答要点：先用接近真实业务的输入输出长度分布做 benchmark，得到单 worker 在 SLO 约束下的稳定吞吐，而不是极限吞吐。然后按线上峰值流量计算 worker 数，应用利用率目标和安全余量，再考虑 N+1 故障冗余、发布 surge、可用区分布，最后乘以每个 worker 的 tensor parallel GPU 数得到总 GPU 数。

问题三：KV cache 如何影响容量？

回答要点：每个 active sequence 都需要为历史 token 保存 key/value，KV cache 占用与层数、KV head 数、head dim、dtype 和 active token 数相关。长上下文和长输出会显著增加 KV cache 占用，限制并发请求数。即使 GPU 计算利用率不高，KV cache 不足也会导致 preemption、拒绝请求或 OOM。

问题四：LLM serving 应该定义哪些 SLO？

回答要点：至少包括 availability、5xx rate、timeout rate、TTFT、TPOT、E2E latency，并按 workload 和输入输出长度条件区分。streaming 场景要特别关注 TTFT 和 TPOT。还应该监控 queue time、KV cache pressure、preemption、worker health 等内部指标作为 SLO 退化的前置信号。

问题五：过载时应该怎么处理？

回答要点：不要无限排队。应该通过 admission control 基于队列长度、token 预算、KV cache 空闲量、租户配额和优先级判断是否接收请求。容量不足时可以限流、拒绝低优先级请求、降低 max_tokens、限制超长上下文、切到小模型或异步处理。可控拒绝比全局超时更好。

问题六：如何做故障演练？

回答要点：选择典型故障场景，例如 worker 退出、GPU OOM、NCCL hang、模型下载失败、KV cache 耗尽、长上下文突增、对象存储不可用等。每个演练要定义预期行为、监控告警、止血步骤和恢复验证。演练后更新 runbook，确保真实故障时可执行。

59.32 标准回答模板

如果面试官问“你会如何做一个 LLM serving 集群的容量规划和稳定性设计”，可以这样回答：

我不会只用 QPS 来规划容量，因为 LLM 请求的成本主要由 input tokens、output tokens、上下文长度和并发决定。首先我会按模型、

然后我会用接近真实分布的压测数据得到单 worker 在 SLO 约束下的稳定容量，包括 request throughput、prefill token/s、decode

对 KV cache 我会单独建模，因为 active sequence 的历史 token 会持续占用显存。长上下文和长输出会限制并发，即使 GPU compute

SLO 上我会区分 workload 定义，例如短 chat 的 TTFT P95、TPOT P95、timeout rate，长上下文任务使用另一套目标。告警不能只看 capacity。过载时通过 admission control、租户限流、max_tokens 限制、长上下文降级和小模型兜底来保护系统。

最后我会定期做故障演练，包括 worker 退出、GPU OOM、NCCL hang、模型加载失败、对象存储不可用、长上下文突增和新版本性能退

59.33 Capacity SLO Fault Drill 公式、容量门禁和可运行 demo

先把 workload 的峰值 token 压力写成：

$$Q_{\{\mathrm{in}\}} = \sum_k \lambda_k X_k$$

$$Q_{\{\mathrm{out}\}} = \sum_k \lambda_k Y_k$$

其中 λ_k 是第 k 类请求的峰值请求率， X_k 和 Y_k 分别是规划输入、输出 token 数。

单 worker 稳定容量要乘以规划利用率：

$$P_{\mathrm{worker}}^{\mathrm{plan}} = \rho P_{\mathrm{worker}}^{\mathrm{bench}}$$

worker 数可以从请求率、prefill、decode 和 KV 四个方向取最大值：

$$N_{\mathrm{worker}} = \max(N_{\mathrm{req}}, N_{\mathrm{prefill}}, N_{\mathrm{decode}}, N_{\mathrm{kv}})$$

KV active token 估算：

$$A_{\mathrm{tok}} = \sum_k C_k(X_k + Y_k)$$

GPU 数量：

$$N_{\mathrm{gpu}} = N_{\mathrm{worker}} T_{\mathrm{gpu}}$$

月度 error budget：

$$B_{\mathrm{err}} = (1 - A_{\mathrm{slo}}) T_{\mathrm{month}}$$

小时成本粗估：

$$C_{\mathrm{hour}} = N_{\mathrm{gpu}} C_{\mathrm{gpu}} + C_{\mathrm{store}} + C_{\mathrm{net}} + C_{\mathrm{obs}}$$

最终容量与演练门禁：

$$G_{\mathrm{capslo}} = G_{\mathrm{profile}} G_{\mathrm{bench}} G_{\mathrm{kv}} G_{\mathrm{cost}} G_{\mathrm{slo}} G_{\mathrm{month}}$$

下面这个 0 依赖 demo 模拟一次最小容量、成本、SLO 和故障演练审计：

```
from dataclasses import dataclass
from math import ceil
```

```
@dataclass
```

```
class WorkloadProfile:
```

```
    name: str
    peak_rps: float
    input_tokens_p95: int
    output_tokens_p95: int
    active_requests_p95: int
    priority: int
```

```
@dataclass
```

```
class WorkerBenchmark:
```

```
    stable_request_rps: float
    prefill_tokens_per_s: int
    decode_tokens_per_s: int
    kv_active_tokens: int
    gpus_per_worker: int
    utilization_target: float
```

```
@dataclass
```

```
class CostConfig:
```

```
    gpu_hour_price: float
    storage_hour: float
    network_hour: float
    observability_hour: float
```

```
@dataclass
```

```
class SLOTarget:
```

```

availability: float
ttft_p95_ms: int
tpot_p95_ms: int
timeout_rate_max: float
window_hours: int

```

`@dataclass`

`class FaultScenario:`

```

    name: str
    lost_workers: int
    dependency_available: bool
    long_context_multiplier: float
    candidate_tpot_p95_ms: int
    runbook_steps: list[str]

```

`class ToyCapacitySLOFaultDrillAuditor:`

`def __init__(self, workloads, benchmark, cost, slo):`

```

    self.workloads = workloads
    self.benchmark = benchmark
    self.cost = cost
    self.slo = slo

```

`def traffic_profile(self) -> dict:`

```

    peak_rps = sum(w.peak_rps for w in self.workloads)
    input_tps = sum(w.peak_rps * w.input_tokens_p95 for w in self.workloads)
    output_tps = sum(w.peak_rps * w.output_tokens_p95 for w in self.workloads)
    active_tokens = sum(w.active_requests_p95 * (w.input_tokens_p95 + w.output_tokens_p95) for w in self.workloads)
    return {
        "peak_rps": round(peak_rps, 2),
        "input_tps": int(input_tps),
        "output_tps": int(output_tps),
        "active_tokens": int(active_tokens),
    }

```

`def capacity_plan(self) -> dict:`

```

    traffic = self.traffic_profile()
    rho = self.benchmark.utilization_target
    req_workers = ceil(traffic["peak_rps"] / (self.benchmark.stable_request_rps * rho))
    prefill_workers = ceil(traffic["input_tps"] / (self.benchmark.prefill_tokens_per_s * rho))
    decode_workers = ceil(traffic["output_tps"] / (self.benchmark.decode_tokens_per_s * rho))
    kv_workers = ceil(traffic["active_tokens"] / (self.benchmark.kv_active_tokens * rho))
    base_workers = max(req_workers, prefill_workers, decode_workers, kv_workers)
    safe_workers = ceil(base_workers * 1.25)
    n_plus_one_workers = safe_workers + 1
    release_workers = n_plus_one_workers + 1
    return {
        "traffic": traffic,
        "required_by_axis": {
            "request": req_workers,
            "prefill": prefill_workers,
            "decode": decode_workers,
            "kv": kv_workers,
        },
    },

```

```

        "base_workers": base_workers,
        "safe_workers": safe_workers,
        "n_plus_one_workers": n_plus_one_workers,
        "release_workers": release_workers,
        "planned_gpus": release_workers * self.benchmark.gpus_per_worker,
    }

def cost_report(self, plan: dict) -> dict:
    gpu_hour = plan["planned_gpus"] * self.cost.gpu_hour_price
    total_hour = gpu_hour + self.cost.storage_hour + self.cost.network_hour + self.cost.observability_hour
    tokens_per_hour = (plan["traffic"]["input_tps"] + plan["traffic"]["output_tps"]) * 3600
    cost_per_million_tokens = total_hour / max(1, tokens_per_hour / 1_000_000)
    return {
        "gpu_hour_cost": round(gpu_hour, 2),
        "total_hour_cost": round(total_hour, 2),
        "cost_per_million_tokens": round(cost_per_million_tokens, 3),
    }

def slo_report(self, observed: dict) -> dict:
    budget_minutes = (1 - self.slo.availability) * self.slo.window_hours * 60
    burned_minutes = observed["incident_minutes"] + observed["timeout_rate"] * self.slo.window_hours * 60
    return {
        "availability_ok": observed["availability"] >= self.slo.availability,
        "ttft_ok": observed["ttft_p95_ms"] <= self.slo.ttft_p95_ms,
        "tpot_ok": observed["tpot_p95_ms"] <= self.slo.tpot_p95_ms,
        "timeout_ok": observed["timeout_rate"] <= self.slo.timeout_rate_max,
        "error_budget_minutes": round(budget_minutes, 2),
        "burned_minutes": round(burned_minutes, 2),
        "burn_ratio": round(burned_minutes / max(1, budget_minutes), 3),
    }

def admission_decision(self, plan: dict, incoming: WorkloadProfile) -> dict:
    projected_tokens = plan["traffic"]["active_tokens"] + incoming.active_requests_p95 * (
        incoming.input_tokens_p95 + incoming.output_tokens_p95
    )
    kv_limit = plan["release_workers"] * self.benchmark.kv_active_tokens * self.benchmark.utilization_target
    admitted = projected_tokens <= kv_limit or incoming.priority <= 1
    action = "admit" if admitted else "reject_or_degrade"
    return {"projected_active_tokens": int(projected_tokens), "kv_limit": int(kv_limit), "action": action}

def fault_drill(self, plan: dict, scenarios: list[FaultScenario]) -> dict:
    rows = {}
    for scenario in scenarios:
        remaining_workers = plan["release_workers"] - scenario.lost_workers
        capacity_ok = remaining_workers >= plan["base_workers"]
        dependency_ok = scenario.dependency_available or "use_cached_model_artifacts" in scenario.runbook_steps
        long_context_ok = scenario.long_context_multiplier <= 1.5 or "cap_context_length" in scenario.runbook_steps
        release_ok = scenario.candidate_tpot_p95_ms <= self.slo.tpot_p95_ms or "rollback_revision" in scenario.runbook_steps
        runbook_ok = {"detect", "mitigate", "verify"}.issubset(set(scenario.runbook_steps))
        rows[scenario.name] = {
            "remaining_workers": remaining_workers,
            "capacity_ok": capacity_ok,
            "dependency_ok": dependency_ok,
            "long_context_ok": long_context_ok,
            "release_ok": release_ok,

```

```

        "runbook_ok": runbook_ok,
        "pass": capacity_ok and dependency_ok and long_context_ok and release_ok and runbook_ok,
    }
    return rows

workloads = [
    WorkloadProfile("short_chat", 80.0, 256, 96, 120, 1),
    WorkloadProfile("rag_summary", 12.0, 4096, 512, 36, 2),
    WorkloadProfile("code_gen", 18.0, 1024, 512, 54, 2),
]
benchmark = WorkerBenchmark(
    stable_request_rps=50.0,
    prefill_tokens_per_s=180000,
    decode_tokens_per_s=12000,
    kv_active_tokens=120000,
    gpus_per_worker=4,
    utilization_target=0.7,
)
cost = CostConfig(gpu_hour_price=2.8, storage_hour=18.0, network_hour=12.0, observability_hour=5.0)
slo = SLOTarget(availability=0.995, ttft_p95_ms=700, tpot_p95_ms=80, timeout_rate_max=0.01, window_hours=720)
auditor = ToyCapacitySLOFaultDrillAuditor(workloads, benchmark, cost, slo)

plan = auditor.capacity_plan()
cost_report = auditor.cost_report(plan)
slo_report = auditor.slo_report(
    {"availability": 0.997, "ttft_p95_ms": 640, "tpot_p95_ms": 72, "timeout_rate": 0.00005, "incident_minutes": 18}
)
incoming_long_context = WorkloadProfile("tenant_burst", 6.0, 12000, 1024, 48, 3)
admission = auditor.admission_decision(plan, incoming_long_context)
fault_rows = auditor.fault_drill(
    plan,
    [
        FaultScenario("worker_lost", 1, True, 1.0, 72, ["detect", "mitigate", "verify"]),
        FaultScenario("object_store_down", 0, False, 1.0, 72, ["detect", "use_cached_model_artifacts", "mitigate", "verify"]),
        FaultScenario("long_context_burst", 0, True, 2.0, 74, ["detect", "cap_context_length", "mitigate", "verify"]),
        FaultScenario("candidate_tpot_regression", 0, True, 1.0, 110, ["detect", "rollback_revision", "mitigate", "verify"]),
    ],
)

summary = {
    "traffic": plan["traffic"],
    "required_by_axis": plan["required_by_axis"],
    "base_workers": plan["base_workers"],
    "safe_workers": plan["safe_workers"],
    "n_plus_one_workers": plan["n_plus_one_workers"],
    "release_workers": plan["release_workers"],
    "planned_gpus": plan["planned_gpus"],
    "cost_report": cost_report,
    "slo_report": slo_report,
    "admission": admission,
    "fault_rows": fault_rows,
}
gates = {
    "profile_gate": summary["traffic"]["input_tps"] == 88064 and summary["traffic"]["output_tps"] == 23040,

```

```

"benchmark_capacity_gate": summary["required_by_axis"] == {"request": 4, "prefill": 1, "decode": 3, "kv": 4},
"kv_capacity_gate": summary["base_workers"] == 4 and summary["release_workers"] == 7,
"cost_gate": summary["cost_report"]["total_hour_cost"] == 113.4,
"slo_gate": all(summary["slo_report"][key] for key in ["availability_ok", "ttft_ok", "tpot_ok", "timeout_ok"])
and summary["slo_report"]["burn_ratio"] < 0.1,
"admission_gate": summary["admission"]["action"] == "reject_or_degrade",
"fault_drill_gate": all(row["pass"] for row in summary["fault_rows"].values()),
"runbook_gate": all(row["runbook_ok"] for row in summary["fault_rows"].values()),
}
gates["capacity_slo_fault_drill_gate"] = all(gates.values())

print("capacity_slo_fault_drill_summary=", summary)
print("capacity_slo_fault_drill_gates=", gates)

```

一次运行的核心输出类似：

```

capacity_slo_fault_drill_summary= {'traffic': {'peak_rps': 110.0, 'input_tps': 88064, 'output_tps': 23040, 'active_token': 110000},
capacity_slo_fault_drill_gates= {'profile_gate': True, 'benchmark_capacity_gate': True, 'kv_capacity_gate': True, 'cost_gate': True, 'slo_gate': True, 'admission_gate': True, 'fault_drill_gate': True, 'runbook_gate': True}

```

这个 demo 证明了几个关键点：

1. 容量规划要同时看 request、prefill、decode 和 KV active tokens，不能只看 QPS。
2. benchmark 极限值要乘以规划利用率，再叠加安全余量、N+1 和发布容量。
3. KV cache 可能和 decode 一样成为主约束，长上下文 burst 要走 admission 或降级。
4. 成本要从 GPU 小时扩展到存储、网络 and 观测，并能按 token 粗略归因。
5. SLO 不是一句口号，必须能计算 error budget 和 burn ratio。
6. 故障演练要覆盖 worker、依赖系统、长上下文突增和新版本性能退化，并且每个场景都有 detect、mitigate、verify。

59.34 小练习

1. 为 benchmark 脚本增加 input token/s 和 output token/s 统计。
2. 为请求日志增加 input_tokens、output_tokens、tenant_id 和 model_revision。
3. 统计一组请求的输入长度 P50/P95/P99。
4. 统计一组请求的输出长度 P50/P95/P99。
5. 基于 benchmark 结果写一个 GPU 数量估算表。
6. 把规划利用率从 100% 改为 70%，比较 GPU 数变化。
7. 增加一个 KV cache bytes per token 的估算函数。
8. 在 dashboard 中展示 running requests 和 waiting requests。
9. 在 dashboard 中展示 KV cache free blocks。
10. 实现一个简单 admission control：队列过长时返回 overload。
11. 对超长输入请求返回明确错误，而不是进入 engine 后失败。
12. 为不同租户设置不同 QPS 和 token 配额。
13. 为短请求和长请求分别定义 TTFT SLO。
14. 写一个压测场景：短输入短输出。
15. 写一个压测场景：长输入长输出。
16. 写一个混合流量压测场景。
17. 模拟一个 worker 退出，观察 router 是否摘除。
18. 模拟 KV cache 压力升高，观察 preemption 和 TTFT。
19. 写一个 worker OOM 的故障演练 runbook。
20. 写一个新版本 TPOT 退化后的回滚 runbook。

59.35 本章总结

LLM serving 的容量规划不能只看 QPS。

真正重要的是 request QPS、input token/s、output token/s、输入输出长度分布、并发请求数和 KV cache 占用。

prefill 主要影响 TTFT, decode 主要影响 TPOT, KV cache 决定长上下文和高并发下能同时容纳多少 active sequence。

容量估算应该基于真实 workload 的 benchmark, 并用稳定容量而不是极限吞吐。

生产规划要加入安全余量、N+1 故障冗余、发布容量和可用区分布。

成本估算不仅要看 GPU 小时, 也要考虑利用率、空闲冗余、存储、网络、日志和多租户归因。

SLO 应该按 workload 和输入输出长度条件定义, streaming 场景要重点关注 TTFT 和 TPOT。

告警不能只看 5xx, 还要覆盖 queue time、KV cache pressure、preemption、OOM、NCCL error、worker health 和依赖系统。

过载时应该通过 admission control、限流、降级和明确错误保护系统, 而不是无限排队。

故障演练要覆盖 worker、GPU、模型加载、router、KV cache、依赖系统、长上下文突增和新版本性能退化。

到这里, 一个 LLM serving engine 已经不仅能跑起来, 也能被容量化、成本化、SLO 化和演练化。

下一章可以继续讨论: 如何把这些工程能力沉淀成面试项目、作品集和系统设计表达。

第 60 章 推理框架面试高频问题与标准回答

第 59 章我们讨论了线上容量规划、成本估算、SLO 设计和故障演练。

到这里, 第二十四册已经从最小 generate loop 一直讲到 vLLM、SGLang、PagedAttention、continuous batching、prefix cache、PD 分离、分布式 serving、生产部署和容量治理。

本章的目标不是继续引入新模块, 而是把前面所有内容压缩成面试可用的回答。

推理框架面试通常不是考你背一个框架的参数名, 而是考你能不能从请求生命周期、调度、KV cache、显存、延迟、吞吐、分布式和生产稳定性这些维度建立完整判断。

如果你只回答 “vLLM 用了 PagedAttention, 所以更快”, 面试官很容易继续追问:

1. PagedAttention 到底解决什么问题。
2. 为什么 continuous batching 比静态 batching 更适合 LLM。
3. KV cache block 怎么分配和释放。
4. prefill 和 decode 为什么要区别调度。
5. prefix cache 命中后节省了什么。
6. PD 分离带来了什么新瓶颈。
7. TTFT 高和 TPOT 高分别怎么排查。
8. 长上下文请求为什么会拖垮系统。
9. 如何设计一个 vLLM-like serving engine。
10. 如何判断调优是否真的有效。

本章按面试主题组织高频问题和标准回答。

60.0 本讲资料边界与第二轮精修口径

本章按第二轮精修口径, 只讲 LLM 推理框架面试表达的知识组织和可验证准备度。

公开资料校准主要参考四类口径:

1. vLLM architecture、PagedAttention、automatic prefix caching、metrics、optimization / tuning 和 OpenAI-compatible serving 文档对 engine、worker、scheduler、KV cache、prefix cache、指标和 API 的公开边界说明。
2. SGLang 论文和公开文档对 RadixAttention、structured generation、runtime / frontend 分层和复杂语言模型程序执行的公开说明。
3. TensorRT-LLM 公开文档对 in-flight batching、paged KV cache、quantization、多 GPU / 多节点 serving 和 NVIDIA GPU 推理优化生态的公开说明。

4. 本书第 1 到 59 章对最小 generate loop、KV cache、continuous batching、PagedAttention、prefix cache、speculative decoding、PD 分离、分布式 serving、生产部署和容量治理的教学抽象。

本章不押某个厂商、某个版本或某道面试题的标准答案，也不要求背诵真实框架源码类名、参数全集、benchmark 排名或云厂商部署细节。我们只保留一个面试可复盘闭环：

question -> system boundary -> lifecycle -> data structure -> scheduling trade-off -> metrics -> debug path -> production governance -> project evidence

第二轮新增 demo 的验收重点是：回答不是只报框架名，而要覆盖生命周期、KV/PagedAttention、调度、prefill/decode、prefix cache、分布式、API、生产治理、容量 SLO、debug 路径和项目证据；同时能指出薄弱题目并给出下一轮复习计划。

60.1 本章目标

读完本章，你应该能做到：

1. 用 2 分钟讲清 LLM serving engine 的核心架构。
2. 用 3 分钟讲清一个请求从 HTTP API 到 token streaming 的完整生命周期。
3. 准确解释 PagedAttention、continuous batching、chunked prefill、prefix cache 和 PD 分离。
4. 区分 TTFT、TPOT、吞吐、并发、显存和成本之间的 trade-off。
5. 面对系统设计题时，能从模块、数据结构、调度策略、指标和故障处理展开。
6. 面对 debug 题时，能给出有顺序的排查路径。
7. 面对开放题时，能讨论取舍，而不是只报框架名。

60.2 面试回答的基本框架

推理框架类问题可以用同一套回答骨架。

不要一上来就讲单个优化点。

先把系统边界讲清楚。

一个比较稳的回答顺序是：

1. 目标：低 TTFT、低 TPOT、高吞吐、低显存、高稳定性、低成本。
2. 请求生命周期：API -> tokenizer -> scheduler -> model runner -> KV cache -> sampler -> streamer。
3. 核心瓶颈：prefill、decode、KV cache、batching、网络、调度。
4. 核心机制：continuous batching、PagedAttention、prefix cache、chunked prefill、speculative decoding、parallelism。
5. 生产能力：限流、监控、灰度、故障隔离、容量规划。
6. trade-off：延迟、吞吐、显存、质量、复杂度和成本。

这套框架可以覆盖大部分问题。

面试官如果问“如何设计一个推理框架”，你可以按这套展开。

面试官如果问“为什么线上 TPOT 变高”，你也可以从这套框架中抽出 decode、KV cache、batching、worker、网络和客户端消费速度来排查。

60.3 问题一：LLM Serving Engine 到底是什么

标准回答：

LLM serving engine 是把模型 checkpoint 变成可在线服务的运行时系统。它不只是调用一次 model.generate，而是负责请求接入、t

它的核心目标是在有限 GPU 显存和算力下，同时优化 TTFT、TPOT、吞吐、并发、稳定性和成本。相比普通深度学习推理服务，LLM se
追问要点：

1. 普通模型服务通常一次 forward 就结束，LLM 需要逐 token decode。
2. LLM 请求成本主要由 input token、output token、上下文长度和并发决定。
3. serving engine 的核心不是模型结构本身，而是调度和运行时资源管理。

60.4 问题二：一个请求的生命周期是什么

标准回答：

一个 chat completion 请求进入系统后，通常先经过 API gateway 做鉴权、限流和参数校验。然后 tokenizer 把 messages 或 prompt

请求第一次进入模型执行 prefill，模型并行处理 prompt，生成每层 KV cache，并得到最后位置 logits。之后进入 decode 阶段，每 k、top-p、stop token 等规则选择下一个 token。streamer 把 token 逐步返回客户端。请求完成、取消或超时后，engine 释放 KV ca

容易漏掉的点：

1. tokenizer 和 detokenizer 可能成为 CPU 侧瓶颈。
2. scheduler 不是只凑 batch，还要控制 token budget 和 KV cache budget。
3. streaming 请求完成前会持续占用运行时状态。
4. client cancel 后必须释放 KV cache，否则会造成泄漏。

60.5 问题三：Prefill 和 Decode 有什么区别

标准回答：

Prefill 是处理输入 prompt 的阶段。它把完整 prompt token 送入模型，并为每层生成对应的 KV cache。prefill 通常并行度更高，转

Decode 是生成输出 token 的阶段。每一步通常只输入最新 token，复用历史 KV cache，生成下一个 token。decode 是自回归串行过程

因此 serving engine 不能把 prefill 和 decode 当成同一种任务调度。长 prefill 可能阻塞已有请求的流式输出，而 decode 过多也

面试延伸：

1. TTFT 高优先看 queue time、prefill 负载、prompt length、prefix cache 命中率。
2. TPOT 高优先看 decode batch、KV cache 带宽、GPU kernel、sampling、网络 and 客户端 backpressure。
3. chunked prefill 的目的就是把长 prefill 切小，避免独占 GPU。

60.6 问题四：KV Cache 解决什么问题

标准回答：

KV cache 缓存每一层历史 token 的 key 和 value。自回归生成时，新 token 需要关注历史上下文。如果没有 KV cache，每生成一个 t

有了 KV cache 后，decode 阶段只需要计算新 token 的 Q/K/V，并把新 K/V append 到 cache 中。历史 K/V 可以复用，从而显著降低重

但 KV cache 不是免费优化。它把计算压力转成显存容量和显存带宽压力。长上下文、高并发和长输出都会让 KV cache 成为 serving

公式表达可以简化为：

$$\text{KV bytes per token} = 2 * \text{layers} * \text{kv_heads} * \text{head_dim} * \text{bytes_per_element}$$

其中 2 分别表示 key 和 value。

追问要点：

1. GQA/MQA 可以减少 KV head 数，从而降低 KV cache。
2. KV cache 是每个活跃请求独有的运行时状态。
3. KV cache 释放、复用、迁移和量化都会影响 serving 性能。

60.7 问题五：PagedAttention 解决什么问题

标准回答：

PagedAttention 的核心思想是借鉴操作系统分页，把逻辑上连续的 KV cache 切成固定大小 block，并通过 block table 映射到物理显

它主要解决 LLM serving 中变长请求导致的 KV cache 显存碎片和浪费问题。请求可以按需追加 block，请求结束后释放 block，调度

PagedAttention 不改变 attention 的数学定义，它是 KV cache 管理和 attention kernel 访问方式上的系统优化。

面试中一定要强调：

1. PagedAttention 不是 sparse attention。
2. 它不是近似 attention。
3. 它解决的是 serving 中 KV cache 分配、碎片和动态增长问题。
4. block size 有 trade-off：小 block 碎片少但管理开销高，大 block 管理简单但尾部浪费更大。

60.8 问题六：Continuous Batching 为什么重要

标准回答：

LLM 请求的输出长度不同，decode 是逐 token 迭代。如果使用静态 batch，一个 batch 里的短请求完成后，可能要等长请求结束，GPU

Continuous batching 允许系统在每个 decode iteration 动态更新 batch：完成的请求退出，新请求加入，运行中的请求继续生成。

它的关键不是简单把请求凑成 batch，而是在每一轮迭代里根据 token budget、KV cache budget、优先级和调度策略选择哪些请求执行。容易混淆：

1. dynamic batching 通常是在请求进入模型前凑批。
2. continuous batching 是生成过程中持续更新 batch。
3. continuous batching 需要 KV cache manager 支持动态追加和释放。
4. 如果调度策略不好，continuous batching 也可能造成长请求拖慢短请求。

60.9 问题七：Chunked Prefill 解决什么问题

标准回答：

Chunked prefill 把长 prompt 的 prefill 拆成多个较小 chunk 执行，而不是让一个超长 prefill 一次占用 GPU 很久。

它解决的问题是长 prompt 对已有 decode 请求的干扰。没有 chunked prefill 时，一个很长的输入可能让正在 streaming 的请求 token

chunked prefill 的代价是调度更复杂，可能引入额外 kernel launch 或状态管理开销。调得太碎会降低吞吐，调得太大又无法缓解 context

面试表达重点：

1. 它主要平衡新请求 TTFT 和老请求 TPOT。
2. 它通常和 token budget、continuous batching 一起出现。
3. 它不是提升模型质量的算法，而是 serving 调度优化。

60.10 问题八：Prefix Cache 和 Prompt Cache 有什么价值

标准回答：

Prefix cache 或 prompt cache 用来复用相同前缀的 prefill 结果。很多业务请求有重复的 system prompt、工具说明、RAG 模板、few-shot 示例或多轮对话历史。如果每次都重新 prefill，会浪费大量计算。

命中 prefix cache 后，系统可以复用已计算的 KV cache，只对新增 token 做 prefill 或 decode，从而降低 TTFT、减少 prefill cost。

但 prefix cache 也有代价：需要缓存管理、哈希匹配、生命周期控制、显存或 CPU 内存占用，以及多租户隔离。缓存命中率低时，性能下降。追问要点：

1. cache key 通常要和 token 序列、模型版本、LoRA adapter、chat template 等绑定。
2. 多租户场景要避免跨租户泄露上下文。
3. prefix cache 命中率应该进入 dashboard。
4. SGLang 的 RadixAttention 更强调对共享 prefix 的树状复用。

60.11 问题九：vLLM 的核心贡献是什么

标准回答：

vLLM 的核心贡献可以概括为面向 LLM serving 的高吞吐和高显存效率运行时。它最经典的机制是 PagedAttention，用 block table 管理 memory，支持 compatible API、量化、多种并行方式、speculative decoding 和生产部署相关能力。

面试中我不会把 vLLM 简化成一个 attention kernel。更准确地说，它是一个 serving engine：scheduler、KV cache manager、model executor。

回答边界：

1. 不要说 vLLM 只靠 PagedAttention。
2. 不要把 PagedAttention 说成改变模型输出质量。
3. 不要忽略 scheduler 和 memory management。
4. 不要忽略版本差异，具体参数和实现细节会随 vLLM 版本演化。

60.12 问题十：SGLang 和 vLLM 有什么区别

标准回答：

vLLM 更常被理解为高吞吐、显存高效的通用 LLM serving engine，代表性机制包括 PagedAttention、continuous batching、prefix caching。

SGLang 也是高性能 serving 框架，但它更强调 runtime 和程序化生成场景，例如 RadixAttention 复用共享 prefix、structured generation、turn 和 agent/tool-use serving。它也支持 continuous batching、paged attention、PD 分离、speculative decoding、并行和量化。

二者不是简单谁替代谁，而是关注点和生态侧重不同。面试中应该从 workload 出发比较：如果大量请求共享长 prefix、结构化输出和 agent/tool-use，SGLang 更合适；如果是高吞吐、低延迟的通用 serving，vLLM 的生态和成熟度也很有吸引力。

注意事项：

1. 不要把 SGLang 说成只有前端 DSL。
2. 不要把 vLLM 说成不能做结构化输出。
3. 两者能力都在快速变化，回答时要强调版本和 workload 边界。

60.13 问题十一：PD 分离解决什么问题

标准回答：

PD 分离指把 prefill 和 decode 放到不同资源池或不同节点上执行。它出现的原因是 prefill 和 decode 的资源画像不同：prefill 更密集，decode 更稀疏。

把二者拆开后，可以为 prefill 和 decode 分别配置硬件、batching 策略和调度策略，减少长 prefill 对 decode streaming 的干扰，提高 decode 的吞吐。

但 PD 分离不是免费午餐。它会引入 KV cache 迁移或共享问题、网络开销、路由复杂度、跨节点一致性、故障恢复和调试复杂度。只有当 prefill 和 decode 的资源需求差异显著时，PD 分离才有价值。

面试追问：

1. KV cache 如何从 prefill 节点交给 decode 节点。
2. 网络带宽是否会成为瓶颈。
3. 短请求是否适合 PD 分离。
4. 如何做 prefill/decode 容量配比。
5. PD 分离失败时如何降级回单体 serving。

60.14 问题十二：TTFT 高怎么排查

标准回答：

TTFT 表示用户从发出请求到收到第一个 token 的时间。TTFT 高通常和排队、tokenization、prefill、prefix cache、router 负载和调度有关。

我会先拆分 TTFT：client 到 gateway、gateway 排队、router 选择、tokenization、scheduler queue time、prefill execution、decode execution。

如果 TTFT 只在长 prompt 上升，重点看 prefill 和 chunked prefill。如果短请求也上升，重点看队列、路由、worker capacity、CPU 排查顺序：

1. 是否是全局问题还是单模型、单租户、单 worker pool 问题。
2. queue time 是否升高。
3. input length P95/P99 是否升高。
4. prefill tokens/s 是否下降。
5. prefix cache hit rate 是否下降。
6. worker 是否重启、OOM 或处于 warmup。
7. tokenizer CPU 是否打满。
8. router 是否把流量打到热点 worker。

60.15 问题十三：TPOT 高怎么排查

标准回答：

TPOT 表示每个输出 token 的平均或分位延迟，主要影响 streaming 是否流畅。TPOT 高通常发生在 decode 阶段，和 running batch size 有关。

我会先确认 TPOT 是所有请求变高，还是长上下文、长输出、特定模型、特定 GPU 或特定租户变高。然后看 decode batch size、acti

如果 GPU utilization 不高但 TPOT 高，可能是 memory bandwidth、KV cache 访问、调度碎片或 CPU/网络瓶颈。如果 KV cache press

常见原因：

1. running batch 过大导致单 token iteration 变慢。
2. 长上下文请求增多，KV cache 读取成本上升。
3. preemption 或 swap 增加。
4. structured generation 或复杂 sampling 带来 CPU/GPU 额外开销。
5. 慢客户端导致 streaming backpressure。
6. 新 kernel、新模型 revision 或量化配置引入性能回退。

60.16 问题十四：吞吐上去了但用户觉得更卡，为什么

标准回答：

吞吐和用户体验不是同一个指标。系统总 output tokens/s 上升，可能是因为 batch 更大或长请求更多，但这可能同时提高 TTFT 或

例如 scheduler 为了吞吐把 batch 塞得很满，GPU 利用率上升，但短请求排队更久，streaming token 间隔更不稳定。或者系统处理了

所以调优时必须同时看吞吐、TTFT、TPOT、queue time、输入输出长度分布和 workload 分桶，不能只看单一吞吐指标。

一句话总结：

吞吐优化如果没有 SLO 约束，可能只是把用户延迟换成了更好看的 tokens/s。

60.17 问题十五：如何设计一个 vLLM-like 推理框架

标准回答：

我会把系统拆成 API 层、router、scheduler、KV cache manager、model runner、sampler、streamer、metrics 和 worker 管理几个

API 层负责 OpenAI-compatible 接口、鉴权、限流和参数校验。router 按模型、租户、负载和 KV capacity 选择 worker。scheduler

第一版可以先实现单机单模型、greedy sampling、简单 KV cache 和 continuous batching。第二版再加入 paged KV cache、prefix c

架构图可以口述为：

Client

-> API Gateway

- > Router
- > Engine Scheduler
- > Model Runner
- > KV Cache Manager
- > Sampler
- > Streamer
- > Client

60.18 问题十六：Scheduler 应该怎么设计

标准回答：

LLM scheduler 的核心职责是在每个 iteration 决定哪些请求进入 batch，同时控制 token budget、KV cache budget、优先级和 SLO。

一个简单设计是维护 waiting queue 和 running set。每轮先处理 running decode 请求，再根据剩余 token budget 和 KV cache 空闲

调度策略要根据目标取舍。如果目标是最低 TTFT，可能更积极调度新 prefill；如果目标是稳定 TPOT，要保护 decode iteration；如

关键输入：

1. waiting queue 长度。
2. running requests 数。
3. 每个请求已生成长度和剩余预算。
4. input token 和 chunk size。
5. KV cache free blocks。
6. tenant priority。
7. deadline 或 timeout。

60.19 问题十七：KV Cache Manager 应该怎么设计

标准回答：

KV cache manager 负责为每个 active sequence 分配、追加、查找和释放 KV cache。生产级设计通常不会给每个请求预留连续大块显

请求 prefill 时按 token 数申请 blocks，decode 每生成新 token 可能追加 block。请求结束、取消或超时后释放 blocks。如果启用

需要说明的 trade-off：

1. block size 影响碎片和管理开销。
2. prefix cache 需要引用计数和隔离。
3. preemption 可以缓解压力，但会影响延迟。
4. swap 到 CPU 或远端缓存能扩容量，但会引入网络和恢复成本。

60.20 问题十八：如何做推理框架压测

标准回答：

推理框架压测不能只用固定 prompt 和平均 latency。必须构造接近真实业务的输入输出长度分布，并同时测 request throughput、i

压测要覆盖短输入短输出、短输入长输出、长输入短输出、长输入长输出、混合流量、streaming、client cancel、prefix cache 命

最终要找到稳定容量和拐点，而不是只追求峰值吞吐。稳定容量应该是在 SLO 满足、错误率可控、没有频繁 OOM 或 preemption 的情

面试加分点：

1. benchmark 要记录模型、GPU、dtype、parallel size、scheduler 参数和版本。
2. 结果要按 workload 分桶，而不是只给全局平均。
3. 压测要看过载行为是否优雅拒绝。
4. 新版本上线前要和 baseline 做回归对比。

60.21 问题十九：如何做容量规划

标准回答：

我会先按模型、租户和 workload 收集流量，统计 request QPS、input token/s、output token/s、输入长度 P50/P95/P99、输出长度

容量估算时不能用 benchmark 极限值，而要乘以规划利用率，比如 60% 到 80%，再考虑峰值流量、N+1 故障冗余、发布 surge、可用区

一句话总结：

LLM serving 容量规划的单位不是 QPS，而是 token、active sequence、KV cache 和 SL0。

60.22 问题二十：量化一定能降低延迟吗

标准回答：

不一定。量化通常能降低权重显存和显存带宽压力，尤其是 weight-only INT8/INT4 在 decode 阶段可能有收益。但实际延迟是否下降

如果系统瓶颈在 KV cache、scheduler、CPU tokenizer、网络或客户端 backpressure，权重量化不一定明显降低 TPOT。低 bit 量化通

面试常见误区：

1. INT4 不等于一定比 FP16 快。
2. 显存下降不等于延迟下降。
3. perplexity 小幅变化不代表业务格式稳定。
4. 量化收益强依赖 kernel 和硬件。

60.23 问题二十一：Speculative Decoding 为什么能加速

标准回答：

Speculative decoding 用一个更小或更快的 draft model 先生成多个候选 token，再由 target model 一次 forward 并行验证这些候选

它的加速比取决于 draft model 速度、候选长度、接受率、target model 验证效率和任务分布。draft 太弱会导致接受率低，draft 太

追问要点：

1. 它主要优化 decode，不解决长 prompt prefill 本身。
2. 高温采样、代码、数学或长上下文场景接受率可能不同。
3. 接受率高不一定总收益高，还要看 proposal 成本和系统调度开销。

60.24 问题二十二：Structured Generation 为什么影响 serving

标准回答：

Structured generation 要求模型输出满足 JSON schema、正则、grammar 或工具调用格式。它通常通过 constrained decoding 在每一

这会影响 serving，因为每个 decode step 除了模型 forward，还要维护状态机、计算合法 token mask，并和 tokenizer vocabulary

但它的价值也很大：可以提升工具调用、JSON 输出和自动化系统的格式可靠性，减少重试和后处理成本。

回答 trade-off：

1. 结构化输出提升格式稳定性。
2. 约束解码可能增加每步开销。
3. schema 太复杂会降低吞吐。
4. 评估时要同时看格式合法率、任务正确率、TTFT、TPOT 和重试率。

60.25 问题二十三：多 GPU serving 有哪些并行方式

标准回答：

常见并行方式包括 tensor parallel、pipeline parallel、data parallel、expert parallel 和 context parallel。

Tensor parallel 把单层矩阵计算切到多张 GPU 上，适合单卡放不下或需要更高单请求吞吐的大模型，但通信频繁。Pipeline parallel

serving 中选择并行方式要看模型大小、单请求延迟、吞吐目标、GPU 拓扑、通信带宽、KV cache 分布和故障域。

面试加分点：

1. TP 增大可能降低单卡显存，但增加通信。
2. DP 扩展吞吐简单，但每份副本都要加载权重。
3. PP 对在线低延迟 serving 不一定总合适。
4. MoE serving 还要考虑 expert load balancing。

60.26 问题二十四：线上 OOM 怎么排查

标准回答：

我会先区分 OOM 发生在模型加载、prefill、decode、KV cache 分配、prefix cache、CUDA graph capture 还是临时 workspace。然后

如果是请求流量导致，重点看输入长度 P99、输出长度、running requests、active tokens、KV cache free blocks 和 preemption。

止血上可以降低 max_tokens、限制长上下文、降低 batch token budget、关闭或限制 prefix cache、扩容 worker、回滚版本或提高排

排查关键词：

1. weight memory。
2. KV cache memory。
3. activation 或 workspace。
4. fragmentation。
5. prefix cache growth。
6. config regression。

60.27 问题二十五：如何设计监控 Dashboard

标准回答：

我会把 dashboard 分成用户体验、吞吐容量、KV cache、GPU worker、错误和成本几个层次。

用户体验层看 TTFT、TPOT、E2E latency、timeout 和 stream cancel。吞吐层看 RPS、input tokens/s、output tokens/s、queue time

所有指标都要支持按 model、tenant、worker pool、GPU type、region 和 revision 切分，否则总量指标会掩盖局部故障。

60.28 问题二十六：如何回答开放系统设计题

如果面试官问：设计一个支持 OpenAI-compatible API 的 LLM serving 平台。

可以按下面结构回答：

第一，我会先明确需求：支持哪些模型、上下文长度、streaming、并发、SLO、多租户、工具调用、结构化输出和成本目标。

第二，架构上分 API gateway、auth/rate limit、router、engine worker、scheduler、KV cache manager、model runner、streamer

第三，请求路径是 gateway 接入，tokenizer 处理，router 根据模型和负载选 worker，scheduler 做 continuous batching，model runner

第四，性能优化包括 PagedAttention、continuous batching、chunked prefill、prefix cache、量化、speculative decoding、合理

第五，稳定性包括 health check、滚动升级、灰度、drain、超时、client cancel、OOM 保护、故障演练和容量规划。

第六，监控包括 TTFT、TPOT、tokens/s、queue time、KV cache、GPU、error rate、timeout、cost 和租户维度。

最后补一句 trade-off:

系统设计的关键不是堆优化点，而是在目标 SLO 下平衡延迟、吞吐、显存、成本和复杂度。

60.29 高频追问清单

这些问题适合用来模拟面试：

1. 为什么 LLM serving 不能用普通 Web 服务的 QPS 思路规划容量。
2. 为什么 output token 通常比 input token 更贵。
3. 为什么 GPU utilization 高不一定代表用户体验好。
4. 为什么 GPU utilization 不高也可能接不了更多请求。
5. 为什么长上下文会同时影响 TTFT、TPOT 和 KV cache。
6. 为什么 continuous batching 需要 iteration-level scheduler。
7. 为什么 PagedAttention 不改变模型输出分布。
8. 为什么 block size 不是越小越好。
9. 为什么 prefix cache 需要考虑模型版本和 chat template。
10. 为什么 structured generation 可能降低吞吐。
11. 为什么 speculative decoding 的接受率不能单独代表收益。
12. 为什么 PD 分离可能被网络瓶颈抵消收益。
13. 为什么 rolling upgrade 会消耗 serving 容量。
14. 为什么 benchmark 要用真实输入输出长度分布。
15. 为什么平均 latency 不能代表线上体验。
16. 为什么 P99 比 P50 更重要。
17. 为什么 tokenizer 也可能成为瓶颈。
18. 为什么 client cancel 要显式释放 KV cache。
19. 为什么多租户场景不能只做全局限流。
20. 为什么成本归因不能只按 request 数量。

60.30 面试中容易犯的错误

错误一：把推理框架等同于 model.generate。

问题：忽略 scheduler、KV cache、streaming、monitoring 和生产治理。

错误二：把 vLLM 等同于 PagedAttention。

问题：忽略 continuous batching、prefix cache、worker/executor、API 和部署生态。

错误三：把 PagedAttention 说成近似 attention。

问题：PagedAttention 是 KV cache 管理和访存方式优化，不是改变 attention 数学定义。

错误四：只看吞吐，不看 TTFT 和 TPOT。

问题：吞吐变高可能牺牲用户体感和尾延迟。

错误五：用平均输入长度规划容量。

问题：P95/P99 长上下文才是打穿系统的常见原因。

错误六：认为量化一定更快。

问题：收益取决于 kernel、硬件、反量化开销和真实瓶颈。

错误七：只会讲框架名，不会讲数据结构。

问题：面试官通常会追问 block table、waiting queue、running batch、prefix tree、KV blocks。

错误八：忽略生产问题。

问题：真实 serving 要处理限流、鉴权、灰度、drain、OOM、故障演练、容量和成本。

60.31 最终 3 分钟回答模板

如果面试官让你整体讲一下 “你如何理解 LLM 推理框架”，可以这样回答：

我理解 LLM 推理框架不是简单调用 generate，而是一个面向自回归生成的 serving engine。它要把请求接入、tokenization、调度、

一个请求进入系统后，会先经过 API gateway 做鉴权、限流和参数校验，然后 tokenizer 转成 token，router 选择 worker，scheduler

推理框架的核心难点有三个。第一是调度，因为请求长度和输出长度差异很大，静态 batching 会浪费资源，所以需要 continuous batching level scheduling。第二是 KV cache，因为长上下文和高并发会让 KV cache 成为显存瓶颈，所以需要类似 PagedAttention 的 block

以 vLLM 为例，它通过 PagedAttention、continuous batching、chunked prefill、prefix caching、多种 attention kernel、量化和 use 场景。二者都不是单一优化点，而是完整 serving runtime。

线上落地时，我会重点监控 TTFT、TPOT、queue time、input/output tokens/s、running/waiting requests、KV cache free blocks、

60.32 Inference Engine Interview Readiness 公式、表达门禁和可运行 demo

面试准备度可以先用覆盖率表达。

对一个主题 j ：

$$C_j = \frac{|A_j \cap R_j|}{\max(1, |R_j|)}$$

其中 A_j 是候选回答覆盖的概念集合， R_j 是该题要求覆盖的概念集合。

项目证据覆盖率：

$$C_{\mathrm{proj}} = \frac{N_{\mathrm{demo}} + N_{\mathrm{metric}} + N_{\mathrm{incident}}}{N_{\mathrm{need}}}$$

debug 路径覆盖率：

$$C_{\mathrm{debug}} = \frac{N_{\mathrm{ordered}}}{N_{\mathrm{debug}}}$$

trade-off 覆盖率：

$$C_{\mathrm{trade}} = \frac{N_{\mathrm{trade}}}{N_{\mathrm{topic}}}$$

最终面试准备门禁：

$$G_{\mathrm{interview}} = G_{\mathrm{life}} G_{\mathrm{kv}} G_{\mathrm{sched}} G_{\mathrm{metric}} G_{\mathrm{debug}} G_{\mathrm{trade}}$$

下面这个 0 依赖 demo 模拟一次推理框架面试准备度审计：

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class QuestionRubric:
```

```
    question_id: str
```

```
    topic: str
```

```
    required_concepts: set[str]
```

```
    min_coverage: float
```

```
@dataclass
```

```
class CandidateAnswer:
```

```
    question_id: str
```

```
    concepts: set[str]
```

```
    metrics: set[str]
```

```

debug_steps: List[str]
tradeoffs: Set[str]

```

```
@dataclass
```

```
class ProjectEvidence:
```

```

    demos: Set[str]
    metrics: Set[str]
    incidents: Set[str]
    artifacts: Set[str]

```

```
class ToyInferenceEngineInterviewAuditor:
```

```

    def __init__(self, rubrics: List[QuestionRubric], answers: List[CandidateAnswer], evidence: ProjectEvidence):
        self.rubrics = {rubric.question_id: rubric for rubric in rubrics}
        self.answers = {answer.question_id: answer for answer in answers}
        self.evidence = evidence

```

```
    def concept_coverage(self) -> Dict[str, Dict]:
```

```

        rows = {}
        for question_id, rubric in self.rubrics.items():
            answer = self.answers[question_id]
            covered = sorted(answer.concepts & rubric.required_concepts)
            missing = sorted(rubric.required_concepts - answer.concepts)
            score = len(covered) / max(1, len(rubric.required_concepts))
            rows[question_id] = {
                "topic": rubric.topic,
                "covered": covered,
                "missing": missing,
                "score": round(score, 3),
                "pass": score >= rubric.min_coverage,
            }
        return rows

```

```
    def metric_gate(self) -> Dict:
```

```

        required = {"ttft", "tpot", "queue_time", "kv_pressure", "preemption", "tokens_per_s", "error_rate"}
        observed = set().union(*(answer.metrics for answer in self.answers.values()))
        covered = sorted(required & observed)
        return {"covered": covered, "missing": sorted(required - observed), "pass": required <= observed}

```

```
    def debug_gate(self) -> Dict:
```

```

        expected_prefix = ["scope", "reproduce", "segment", "measure", "hypothesize", "fix", "verify"]
        best = max((answer.debug_steps for answer in self.answers.values()), key=len)
        ordered = best[: len(expected_prefix)] == expected_prefix
        return {"best_path": best, "ordered": ordered, "pass": ordered and len(best) >= len(expected_prefix)}

```

```
    def tradeoff_gate(self) -> Dict:
```

```

        required = {"latency_vs_throughput", "memory_vs_batch", "quality_vs_speed", "cost_vs_reliability"}
        observed = set().union(*(answer.tradeoffs for answer in self.answers.values()))
        return {"covered": sorted(required & observed), "missing": sorted(required - observed), "pass": required <= observed}

```

```
    def project_gate(self) -> Dict:
```

```

        required_demos = {"paged_kv", "continuous_batching", "prefix_cache", "api_streaming", "capacity_slo"}
        required_metrics = {"ttft", "tpot", "kv_pressure", "preemption", "cost_per_token"}
        required_incidents = {"oom", "slow_tpot", "worker_lost"}

```

```

demo_ok = required_demos <= self.evidence.demos
metric_ok = required_metrics <= self.evidence.metrics
incident_ok = required_incidents <= self.evidence.incidents
return {
    "demo_ok": demo_ok,
    "metric_ok": metric_ok,
    "incident_ok": incident_ok,
    "artifact_count": len(self.evidence.artifacts),
    "pass": demo_ok and metric_ok and incident_ok and len(self.evidence.artifacts) >= 3,
}

def audit(self) -> tuple[dict, dict]:
    concept_rows = self.concept_coverage()
    metric = self.metric_gate()
    debug = self.debug_gate()
    tradeoff = self.tradeoff_gate()
    project = self.project_gate()
    weak_questions = [qid for qid, row in concept_rows.items() if not row["pass"]]
    revision_plan = []
    for qid in weak_questions:
        revision_plan.append(f"review:{concept_rows[qid]['topic']}")
    if not metric["pass"]:
        revision_plan.append("add:metric_dashboard_story")
    if not debug["pass"]:
        revision_plan.append("practice:ordered_debug_path")
    if not project["pass"]:
        revision_plan.append("collect:project_evidence")
    summary = {
        "concept_rows": concept_rows,
        "metric_gate": metric,
        "debug_gate": debug,
        "tradeoff_gate": tradeoff,
        "project_gate": project,
        "weak_questions": weak_questions,
        "revision_plan": revision_plan,
    }
    gates = {
        "lifecycle_ready": concept_rows["q_lifecycle"]["pass"],
        "kv_ready": concept_rows["q_kv"]["pass"],
        "scheduler_ready": concept_rows["q_scheduler"]["pass"],
        "metrics_ready": metric["pass"],
        "debug_ready": debug["pass"],
        "production_ready": concept_rows["q_production"]["pass"],
        "project_evidence_ready": project["pass"],
        "tradeoff_ready": tradeoff["pass"],
    }
    gates["inference_engine_interview_gate"] = all(gates.values())
    return summary, gates

```

```

rubrics = [
    QuestionRubric(
        "q_lifecycle",
        "request lifecycle",
        {"api", "tokenizer", "router", "scheduler", "prefill", "decode", "sampler", "streamer", "cleanup"},
    )
]

```

```

    0.85,
),
QuestionRubric(
    "q_kv",
    "KV cache and PagedAttention",
    {"kv_cache", "block_table", "physical_block", "ref_count", "fragmentation", "prefix_cache", "copy_on_write"},
    0.80,
),
QuestionRubric(
    "q_scheduler",
    "continuous batching",
    {"waiting_queue", "running_batch", "iteration_level", "token_budget", "kv_budget", "chunked_prefill", "preemption"},
    0.80,
),
QuestionRubric(
    "q_production",
    "production serving",
    {"auth", "rate_limit", "health_check", "rolling_update", "canary", "capacity_plan", "slo", "fault_drill"},
    0.80,
),
],
answers = [
    CandidateAnswer(
        "q_lifecycle",
        {"api", "tokenizer", "router", "scheduler", "prefill", "decode", "sampler", "streamer", "cleanup"},
        {"ttft", "tpot", "queue_time", "tokens_per_s"},
        ["scope", "reproduce", "segment", "measure", "hypothesize", "fix", "verify"],
        {"latency_vs_throughput", "memory_vs_batch"},
    ),
    CandidateAnswer(
        "q_kv",
        {"kv_cache", "block_table", "physical_block", "ref_count", "fragmentation", "prefix_cache", "copy_on_write"},
        {"kv_pressure", "preemption"},
        ["scope", "reproduce", "segment", "measure", "hypothesize", "fix", "verify"],
        {"memory_vs_batch", "quality_vs_speed"},
    ),
    CandidateAnswer(
        "q_scheduler",
        {"waiting_queue", "running_batch", "iteration_level", "token_budget", "kv_budget", "chunked_prefill", "preemption"},
        {"ttft", "tpot", "queue_time", "preemption"},
        ["scope", "reproduce", "segment", "measure", "hypothesize", "fix", "verify"],
        {"latency_vs_throughput", "cost_vs_reliability"},
    ),
    CandidateAnswer(
        "q_production",
        {"auth", "rate_limit", "health_check", "rolling_update", "canary", "capacity_plan", "slo", "fault_drill"},
        {"error_rate", "cost_per_token", "kv_pressure"},
        ["scope", "reproduce", "segment", "measure", "hypothesize", "fix", "verify"],
        {"cost_vs_reliability", "quality_vs_speed"},
    ),
],
evidence = ProjectEvidence(
    demos={"paged_kv", "continuous_batching", "prefix_cache", "api_streaming", "capacity_slo"},
    metrics={"ttft", "tpot", "kv_pressure", "preemption", "cost_per_token"},
    incidents={"oom", "slow_tpot", "worker_lost"},

```

```

    artifacts={"benchmark_report", "architecture_diagram", "runbook", "demo_outputs"},
)
auditor = ToyInferenceEngineInterviewAuditor(rubrics, answers, evidence)
summary, gates = auditor.audit()

print("inference_engine_interview_summary=", summary)
print("inference_engine_interview_gates=", gates)

```

一次运行的核心输出类似：

```

inference_engine_interview_summary= {'concept_rows': {'q_lifecycle': {'topic': 'request lifecycle', 'covered': ['api',
inference_engine_interview_gates= {'lifecycle_ready': True, 'kv_ready': True, 'scheduler_ready': True, 'metrics_ready':

```

这个 demo 证明了几个关键点：

1. 面试准备不是背框架名，而是把问题映射到概念覆盖率。
2. 生命周期题要覆盖 API、tokenizer、router、scheduler、prefill、decode、sampler、streamer 和 cleanup。
3. KV 题要能讲 block table、physical block、ref count、fragmentation、prefix cache 和 copy-on-write。
4. 调度题要能讲 waiting queue、running batch、iteration-level、token budget、KV budget、chunked prefill 和 preemption。
5. 生产题要能讲 auth、rate limit、health check、rolling update、canary、capacity、SLO 和 fault drill。
6. 项目证据要有 demo、metrics、事故复盘和可展示 artifact，否则系统设计表达会空。

60.33 小练习

1. 用 2 分钟回答 “LLM serving engine 是什么”。
2. 用 3 分钟画出一个请求从 API 到 streaming token 的生命周期。
3. 用 2 分钟解释 prefill 和 decode 的区别。
4. 用 2 分钟解释 KV cache 的作用和代价。
5. 用 2 分钟解释 PagedAttention 为什么不是近似 attention。
6. 用 2 分钟解释 continuous batching 和 dynamic batching 的区别。
7. 用 2 分钟解释 chunked prefill 解决什么问题。
8. 用 2 分钟解释 prefix cache 的收益和风险。
9. 用 3 分钟比较 vLLM 和 SGLang。
10. 用 3 分钟解释 PD 分离的收益和代价。
11. 写一份 TTFT 高的排查清单。
12. 写一份 TPOT 高的排查清单。
13. 写一份线上 OOM 的排查清单。
14. 设计一个 KV cache manager 的数据结构。
15. 设计一个 continuous batching scheduler 的伪代码。
16. 设计一个推理框架 benchmark，包括输入输出长度分布。
17. 设计一个 OpenAI-compatible serving engine 的系统架构。
18. 设计一个 prefix cache dashboard。
19. 设计一个多租户限流和 admission control 策略。
20. 把本章 3 分钟模板录音一次，检查是否能自然讲完。

60.34 本章总结

推理框架面试考的不是单点名词，而是完整系统理解。

你需要能从请求生命周期讲到 scheduler，从 scheduler 讲到 KV cache，从 KV cache 讲到显存和长上下文，从 prefill/decode 讲到 TTFT/TPOT，从单机 engine 讲到分布式 serving 和生产稳定性。

vLLM、SGLang、PagedAttention、continuous batching、prefix cache、chunked prefill、speculative decoding 和 PD 分离都应该放在同一张系统图里理解。

一个好的面试回答要同时覆盖：

1. 它解决什么问题。

2. 它的核心机制是什么。
3. 它带来什么收益。
4. 它有什么代价。
5. 它适合什么 workload。
6. 它上线后看什么指标。
7. 它出问题时怎么排查。

到这里，第二十四册第一版已经完成了从最小推理框架到生产级 serving engine 的完整闭环。

后续第二轮完善时，可以继续补充源码级细节、更多代码实现、真实 benchmark 表格、不同版本 vLLM/SGLang 的实现差异，以及更贴近生产系统的调度器和 KV cache manager 代码。